Notes

Trevor Vaughn

November 10, 2026

HORATIO

"O day and night, but this is wondrous strange!"

HAMLET

"And therefore as a stranger give it welcome.
There are more things in heaven and earth, Horatio,
Than are dreamt of in your philosophy."

—William Shakespeare, *Hamlet*, Act I, Scene V

Contents

1 Foreword **13**

2 Highlights **13**

3 Analysis and Asymptotics **14**
3.1 Stolz–Cesàro . . . 14
3.2 Wallis Product . . . 15
3.3 Stirling's Formula . . . 16
3.4 Karamata's Inequality . . . 18
3.5 Polynomial Root Bounds . . . 20
3.5.1 Real Roots from Their Mean and Variance . . . 20
3.5.2 Positive Coefficients and Consecutive Ratios . . . 21
3.5.3 Gauss–Lucas Theorem . . . 21
3.6 Banach Fixed Point Theorem . . . 22
3.7 Inverse Function Theorem . . . 23
3.8 Implicit Function Theorem . . . 25
3.9 Lagrange Multipliers . . . 26
3.10 The Isoperimetric Inequality . . . 27

4 Linear Algebra **35**
4.1 Determinant and Inverse Identities . . . 35
4.1.1 Woodbury Identity . . . 37
4.1.2 Matrix Determinant Lemma . . . 38
4.2 Characteristic Polynomial . . . 39
4.2.1 Determinant of Affine Matrix Polynomial . . . 39
4.3 Frobenius Form . . . 40
4.3.1 Algorithm Description . . . 40
4.3.2 Correctness . . . 41
4.3.3 Failure Probability . . . 41
4.3.4 Polynomial Matrix Composition and Powering . . . 42
4.4 Dynamic Frobenius Form and Exact Distances . . . 43
4.5 Black Box Linear Algebra . . . 49
4.5.1 Minimal Polynomial . . . 49
4.5.2 Determinant . . . 50
4.5.3 Wiedemann's Algorithm . . . 52
4.6 Selected-Entry Matrix Multiplication . . . 53
4.6.1 Canceling Structured Interference . . . 53
4.6.2 Why Matrix Multiplication Produces These Errors . . . 54
4.6.3 Amplifying Reuse with a Coordinate Grid . . . 55
4.6.4 Recursion and Running Time . . . 56
4.7 Lights Out . . . 57
4.8 Perron-Frobenius Theorem . . . 58
4.9 Continuity of Eigenvalues . . . 59
4.10 Gershgorin's Circle Theorem . . . 60

5 Complex Analysis **63**
5.1 Cauchy's Theorem . . . 63
5.2 Cauchy's Integral Formula . . . 66
5.3 Laurent Series . . . 67
5.4 Residue Theorem . . . 68

5.5 Cotangent Partial Fractions . . . 69
5.6 Euler's Product for Sine . . . 71

6 Number Theory and Arithmetic **72**
6.1 Bertrand's Postulate and Products of Small Primes . . . 72
6.2 Lucas's Theorem . . . 74
6.3 Quadratic Reciprocity . . . 75
6.4 Values of the Zeta Function at Even Integers . . . 78
6.5 Hensel's Lemma . . . 80
6.6 Arithmetic Algorithms . . . 81
6.6.1 Linear Sieve . . . 81
6.6.2 Extended GCD with Preprocessing . . . 84
6.6.3 Rational Reconstruction . . . 87
6.6.4 Modular Square Roots . . . 89
6.6.5 Generalized Floor Sum . . . 91
6.6.6 Multiplicative Function Prefix Sums . . . 94
6.6.7 LCM and GCD Convolution . . . 96
6.6.8 Erdős–Ginzburg–Ziv . . . 97
6.6.9 Modular Subset Sum . . . 100
6.6.10 Modular Subset Sum using Runs . . . 101
6.6.11 Integer 3SUM . . . 104
6.7 Fast Factorials and Deterministic Factorization . . . 107
6.7.1 Fast Factorial . . . 107
6.7.2 Deterministic Integer Factorization . . . 109
6.8 Miller–Rabin Primality Test . . . 112

7 Combinatorics **118**
7.1 Entropy Method . . . 118
7.1.1 Shearer's Lemma . . . 118
7.1.2 Entropy Bound for Binomial Sums . . . 119
7.1.3 Irregular Moore Bound . . . 120
7.2 Matchings . . . 122
7.2.1 Hall's Marriage Theorem . . . 122
7.2.2 Gale–Ryser Theorem . . . 123
7.2.3 Cantor–Schröder–Bernstein . . . 123
7.2.4 Brégman's Theorem . . . 124
7.2.5 Kahn–Lovász Theorem . . . 125
7.2.6 The van der Waerden Permanent Theorem . . . 127
7.3 Concentration Inequalities . . . 129
7.3.1 Hoeffding . . . 129
7.3.2 Chernoff . . . 131
7.3.3 Bernstein . . . 131
7.4 Martingales . . . 132
7.4.1 Azuma and McDiarmid . . . 133
7.4.2 Freedman's Inequality . . . 135
7.4.3 Generalized Hölder and Read-k Concentration . . . 137
7.5 Probabilistic Method . . . 138
7.5.1 Isolation Lemma . . . 138
7.5.2 Random-Order Independent Sets . . . 139
7.5.3 Counting Independent Sets in Regular Graphs . . . 142
7.5.4 Triangle-Free Independent Sets and Ramsey Numbers . . . 143
7.5.5 Moser–Tardos Resampling . . . 146

7.6 Polynomial Method . . . 148
7.6.1 Triangular Independence . . . 148
7.6.2 Combinatorial Nullstellensatz . . . 149
7.6.3 Multiplicity Schwartz–Zippel . . . 154
7.6.4 Zero Patterns of Polynomial Maps . . . 156
7.6.5 Regular Subgraphs from Chevalley–Warning . . . 157
7.6.6 Alon–Tarsi Orientations and List Coloring . . . 158
7.6.7 Restricted Intersections . . . 159
7.6.8 Finite-Field Kakeya Sets . . . 161
7.6.9 Slice Rank and Low-Degree Tensor Methods . . . 162
7.7 Extremal Set Theory . . . 175
7.7.1 Bollobás Two Families Theorem . . . 175
7.7.2 Exterior Algebra and the Skew Two Families Theorem . . . 176
7.7.3 Sunflower Lemma . . . 177
7.7.4 Kruskal–Katona and Intersecting Families . . . 184
7.8 Ramsey Theory . . . 187
7.8.1 Hales–Jewett Theorem . . . 187
7.8.2 The Van der Waerden Theorem . . . 190
7.8.3 Schur's Theorem . . . 190
7.8.4 Szemerédi's Cube Lemma . . . 191
7.8.5 Infinite Ramsey Theorem . . . 192
7.8.6 Kőnig's Infinity Lemma . . . 193
7.8.7 Compactness for Colorings and Matchings . . . 194
7.9 Additive Combinatorics . . . 195
7.9.1 Kneser's Addition Theorem . . . 195
7.9.2 The Erdős–Heilbronn Conjecture . . . 196
7.9.3 Iterated Modular Sumsets . . . 197
7.9.4 Ruzsa's Triangle Inequality and Covering Lemma . . . 199
7.9.5 Plünnecke–Ruzsa Inequalities . . . 200
7.9.6 Multiway Sumset Inequalities . . . 201
7.9.7 Short Sums and Long Arithmetic Progressions . . . 202
7.9.8 A Subset-Sum Intersection Bound . . . 203
7.9.9 Bounded Knapsack . . . 205
7.9.10 Witnesses for Long Progressions in Repeated Sumsets . . . 209
7.9.11 Witnesses for Long Progressions in Subset Sums . . . 213
7.9.12 Additive Energy . . . 217
7.9.13 Solymosi's Sum–Product Bound . . . 218
7.9.14 Balog–Szemerédi–Gowers . . . 219
7.9.15 Algorithmic BSG . . . 220
7.9.16 Snevily's Conjecture . . . 224
7.10 Enumerative and Algebraic Combinatorics . . . 226
7.10.1 Lindström–Gessel–Viennot Lemma . . . 226
7.10.2 Burnside's Lemma and Pólya Enumeration . . . 228
7.10.3 Q-analogues . . . 233
7.10.4 Jacobi Triple Product . . . 238
7.10.5 Pentagonal Number Theorem . . . 240
7.10.6 Hook-Length Formula . . . 241
7.10.7 The Fundamental Theorem of Symmetric Polynomials . . . 245
7.10.8 Robinson–Schensted Correspondence . . . 247
7.10.9 MacMahon's Master Theorem . . . 249
7.10.10 Ehrhart Polynomials . . . 250

7.10.11 The Stanley–Wilf Conjecture . . . 253
7.11 Möbius Inversion on Posets . . . 255
7.11.1 Incidence Algebra and the Möbius Function . . . 255
7.11.2 The Inversion Formula . . . 256
7.11.3 Products of Posets . . . 257
7.11.4 Differences, Inclusion–Exclusion, and Divisor Sums . . . 257
7.11.5 Subspaces over a Finite Field . . . 258
7.11.6 Set Partitions . . . 259
7.11.7 Weighted Inversion . . . 259
7.12 Geometry . . . 260
7.12.1 Sperner's Lemma . . . 260
7.12.2 Brouwer's Fixed-Point Theorem . . . 261
7.12.3 Tucker's Lemma . . . 262
7.12.4 The Borsuk–Ulam Theorem . . . 264
7.13 Discrepancy Theory . . . 264
7.13.1 Komlós's Theorem . . . 264
7.13.2 Rank-One Matrix Discrepancy . . . 271

8 Graph Theory **276**
8.1 Max Flow–Min Cut and Menger's Theorem . . . 276
8.2 Matchings . . . 277
8.2.1 The Reduced Gallai–Edmonds Decomposition . . . 277
8.2.2 Kőnig's Theorem . . . 278
8.3 Mader's Density and Connectivity Theorem . . . 280
8.4 Long Directed Paths . . . 280
8.5 Cycles with Many Chords . . . 281
8.6 Vertex Partitions under Degree Constraints . . . 283
8.7 Path Covers and Independent Representatives . . . 284
8.8 Tree Packing and Arboricity . . . 285
8.9 Directed Branching Packing . . . 287
8.10 Packing and Covering Cycles . . . 288
8.11 Vertex and Edge Coloring . . . 290
8.11.1 Brooks's Theorem . . . 290
8.11.2 Kőnig's Edge Coloring Theorem . . . 291
8.11.3 Vizing's Theorem . . . 291
8.11.4 Galvin's List Edge-Coloring Theorem . . . 292
8.12 Planarity and Euler's Formula . . . 293
8.12.1 Sylvester–Gallai . . . 294
8.12.2 The Colorful Sylvester–Gallai Theorem . . . 295
8.12.3 Crossings and Large Planar Subgraphs . . . 295
8.12.4 The Crossing Lemma . . . 296
8.12.5 Szemerédi–Trotter . . . 298
8.13 The Erdős–Sós Theorem . . . 299
8.14 Huang's Hypercube Theorem . . . 304

9 Convexity and Duality **305**
9.1 Sion's Minimax Theorem . . . 305
9.2 Linear Programming Duality . . . 309
9.3 Lagrangian Relaxation . . . 311
9.4 Seidel's Algorithm . . . 314
9.5 Majorization . . . 316
9.5.1 Muirhead's Inequality . . . 317

9.6 SMAWK and Totally Monotone Matrices . . . 318
9.7 Concave Min-Plus Convolution . . . 321
9.7.1 Description . . . 321
9.7.2 Crossings . . . 321
9.7.3 Boundary Conditions . . . 321
9.8 Rectangular Monotone Min-Plus Product . . . 322
9.8.1 Column-Monotone Right Matrices . . . 325
9.9 Radon's Theorem . . . 325
9.10 Helly's Theorem . . . 326
9.11 Carathéodory's Theorem . . . 326
9.12 Colorful Carathéodory . . . 327
9.13 Tverberg's Theorem . . . 328
9.14 Matroid Intersection . . . 330
9.15 Symmetric Submodular Minimization . . . 334

10 Data Structures **337**
10.1 RMQ . . . 337
10.2 DSU . . . 338
10.2.1 Rank Facts . . . 339
10.2.2 Why the Ackermann Hierarchy Appears . . . 339
10.2.3 Levels and Progress . . . 339
10.2.4 Charging a Find . . . 339
10.3 DSU with Potential . . . 340
10.4 Persistent Queue . . . 340
10.5 Online Suffix Max . . . 342
10.6 Mo's Algorithm . . . 342
10.7 A Fast Succinct Hash Table . . . 344
10.7.1 Small Buckets . . . 345
10.7.2 Storing the Full Keys Succinctly . . . 345
10.7.3 Local Fingerprints . . . 346
10.7.4 A Constant-Time Dictionary Inside a Bucket . . . 346
10.7.5 Cost of Fingerprint Rebuilding . . . 347
10.7.6 Deamortizing Rebuilds . . . 348
10.7.7 The Update Buffers Are Succinct . . . 349
10.7.8 Dynamic Resizing . . . 349
10.7.9 Summary . . . 350
10.8 Simulating Fully Random Hashing . . . 350
10.8.1 Private Random Cells . . . 350
10.8.2 Splitting a Large Set into Small Fixed Sets . . . 351
10.8.3 Randomize After Splitting . . . 353
10.9 Soft Heaps . . . 353
10.9.1 A Forest of Heaps . . . 354
10.9.2 Combine Two Representatives, Keep the Larger Key . . . 354
10.9.3 Root Lists and Heap Operations . . . 355
10.9.4 Why Only Few Items Are Corrupted . . . 356
10.9.5 Amortized Cost . . . 356
10.9.6 Selecting from a Binary Min-Heap . . . 357
10.9.7 Selecting from Sorted Lists . . . 358
10.9.8 Selecting the Smallest Pairwise Sums . . . 359
10.10 Sampling . . . 359
10.10.1 Alias Tables . . . 359
10.10.2 Exponential Races . . . 360

10.10.3 Priority Sampling . . . 361

11 Trees **362**
11.1 LCA . . . 362
11.2 $O(n)$ Binary Lifting . . . 363
11.3 Virtual Tree . . . 365
11.4 XOR-Linked Traversal . . . 367
11.5 DFS . . . 367
11.6 Diameter . . . 368
11.7 Shallowest Decomposition Tree . . . 369
11.8 Static Top Tree . . . 371
11.8.1 The Four Operations . . . 372
11.8.2 Heavy Paths . . . 373
11.8.3 The Expression Tree . . . 373
11.8.4 Weight-Balanced Merging . . . 374
11.8.5 The DP Interface . . . 376
11.8.6 Point Updates . . . 376
11.8.7 Rooted-Subtree Queries . . . 377
11.8.8 Path Queries . . . 377
11.9 Splay Tree . . . 380
11.9.1 Rotation . . . 380
11.9.2 Splaying . . . 380
11.9.3 Access Lemma . . . 381
11.9.4 Augmentation . . . 383
11.10 Link-Cut Tree . . . 383
11.10.1 Preferred Paths . . . 383
11.10.2 How a Preferred Path Is Stored . . . 384
11.10.3 Connecting the Auxiliary Trees . . . 385
11.10.4 Access . . . 385
11.10.5 Why Access Is Fast . . . 386
11.10.6 The Analysis Tree . . . 387
11.10.7 How Many Auxiliary Trees Does Access Cross? . . . 388
11.10.8 Rerooting . . . 389
11.10.9 Dynamic-Forest Operations . . . 389
11.10.10 Path Queries and Updates . . . 389
11.11 Top Tree . . . 390
11.11.1 Real and Fake Nodes . . . 391
11.11.2 The Same Path/Point Interface Reappears . . . 391
11.11.3 Access with Virtual-Child Trees . . . 392
11.11.4 Aggregates . . . 392
11.11.5 Two Kinds of Lazy Tags . . . 394
11.11.6 Push Before Changing the Decomposition . . . 394
11.11.7 Path Operations . . . 395
11.11.8 Subtree Operations . . . 395
11.11.9 Link, Cut, and Reroot . . . 396
11.11.10 Amortized Analysis . . . 396
11.12 Lazy Anti-Monopoly Tree . . . 397

12 Graph Algorithms **400**
12.1 Shortest Paths . . . 400
12.1.1 Bellman–Ford–Dijkstra . . . 400
12.1.2 Eppstein's Algorithm . . . 402

12.1.3 Deterministic Negative Weight Shortest Paths . . . 403
12.2 Low-Diameter Decompositions . . . 410
12.2.1 Undirected Low-Diameter Decompositions . . . 410
12.2.2 Directed Low-Diameter Decompositions and Negative Weight Shortest Paths . . . 411
12.3 Minimum Cut . . . 417
12.3.1 Directed Minimum Cut in Almost-Linear Time . . . 417
12.4 Low-Stretch Spanning Trees . . . 421
12.5 Low-Stretch Spanning Trees from Perturbed Dijkstra . . . 425
12.6 Dynamic Distances . . . 430
12.6.1 Local Information That Survives Changes . . . 431
12.6.2 What the Terminal Graph Must Preserve . . . 432
12.6.3 Weighted Routing Forests . . . 433
12.6.4 From Terminal Words to the Physical Child Graph . . . 434
12.6.5 Physical Routes and Search Graphs . . . 436
12.6.6 Route Repair and Sparse Selection . . . 438
12.6.7 Implicit Paths, Reporting, and Record Lifetimes . . . 440
12.6.8 Closing the Recursion . . . 440
12.7 Minimum-Ratio Cycles . . . 442
12.7.1 Route Corrections . . . 443
12.7.2 Local Corrections . . . 443
12.7.3 Why These Candidates Suffice . . . 444
12.7.4 Reusing the Distance Recursion . . . 445
12.8 Flows . . . 446
12.8.1 Highest Label Push-Relabel . . . 446
12.8.2 Height-Bounded Push–Relabel . . . 447
12.8.3 Flow Rounding . . . 452
12.8.4 Almost-Linear Maximum Flow . . . 455
12.8.5 Cut-Matching Game and Sparse Cuts . . . 461
12.9 Laplacian Solving . . . 463
12.10 Spectral Sparsification . . . 467
12.11 Connectivity . . . 473
12.11.1 Connectivity Augmentation and Splitting Off . . . 473
12.11.2 All-Pairs Connectivity . . . 479
12.12 Hamiltonicity . . . 487
12.12.1 Undirected Hamiltonicity . . . 487
12.12.2 Directed Hamiltonicity . . . 491
12.13 Directed Minimum Spanning Tree . . . 495
12.13.1 The Two Reductions . . . 495
12.13.2 Meldable Incoming-Edge Heaps . . . 495
12.13.3 Detecting Cycles with DSU . . . 496
12.13.4 Maintaining Contractions . . . 496
12.13.5 Fast Directed MST Algorithm . . . 497
12.13.6 Correctness . . . 497
12.13.7 Rollback Reconstruction . . . 498
12.13.8 Running Time . . . 498
12.14 Matching and Coloring . . . 499
12.14.1 Bipartite Regular Matching . . . 499
12.14.2 Kasteleyn Signing and Planar Perfect Matchings . . . 499
12.14.3 Bipartite Edge Coloring . . . 503
12.14.4 Chromatic Number . . . 508
12.14.5 The Tutte Polynomial . . . 510

12.15 Node-Weighted Triangle . . . 514
12.15.1 One Fixed Weight . . . 515
12.15.2 The Square Case . . . 515
12.15.3 Rectangular Version . . . 517
12.15.4 Minimum Node-Weighted Triangle . . . 520
12.16 Incremental SCCs . . . 521
12.17 Arboricity . . . 524
12.17.1 Assigning Edges to Endpoints . . . 525
12.17.2 The Minimum Uniform Capacity . . . 526
12.17.3 Testing the Missing Unit of Capacity . . . 526
12.18 An Algebraic Criterion for Bipartite Perfect Matching . . . 527

13 Algebraic and Spectral Graph Theory **529**
13.1 The Matrix–Tree Theorem . . . 529
13.1.1 Counting Hamiltonian Paths . . . 530
13.1.2 Counting k-Matchings in a Bipartite Graph . . . 531
13.1.3 The BEST Theorem . . . 533
13.2 Cheeger Inequality . . . 535
13.2.1 An Explicit Constant-Degree Expander . . . 538
13.3 Expander Products . . . 542
13.3.1 Lifting a Walk to Edge Labels . . . 542
13.3.2 Derandomized Squaring . . . 543
13.3.3 The Zig-Zag Product . . . 544
13.4 Alon–Boppana . . . 545
13.4.1 The Infinite Binary Tree . . . 545
13.4.2 From a Tree to a Graph Ball . . . 546
13.5 Lower Bound on Spectral Sparsification . . . 547

14 Electrical Flows **548**
14.1 Electrical Flows . . . 549
14.1.1 Potentials and Ohm's Law . . . 550
14.1.2 Effective Resistance . . . 550
14.1.3 Maximum Principle . . . 550
14.1.4 Dirichlet's Principle . . . 551
14.1.5 Series and Parallel Laws . . . 553
14.1.6 Rayleigh Monotonicity . . . 553
14.1.7 Effective Resistance is a Metric . . . 554
14.2 Random Walks and Electrical Networks . . . 554
14.2.1 Voltages are Hitting Probabilities . . . 554
14.2.2 Currents are Expected Net Traversals . . . 555
14.2.3 Hitting a Vertex Through a Given Edge . . . 556
14.2.4 Hitting Times . . . 556
14.2.5 Commute Time . . . 557
14.2.6 Tetali's Hitting-Time Formula . . . 557
14.2.7 Cover-Time Bound . . . 558
14.3 Kron Reduction . . . 558
14.4 Infinite Graphs, Recurrence, and Transience . . . 559
14.4.1 Infinite Thomson Principle . . . 560
14.4.2 Escape Probability . . . 561
14.4.3 Nash–Williams Recurrence Criterion . . . 561
14.4.4 Trees . . . 562
14.5 Random Spanning Trees . . . 562

14.5.1 Weighted Random Spanning Trees . . . 563
14.6 Localization of Electrical Flows . . . 565
14.6.1 Controlling One Trajectory . . . 565
14.6.2 Choosing the Trajectories . . . 567

15 Matrix Analysis **567**
15.1 Sampling Matrices without Replacement . . . 567
15.2 Schur and Kronecker Product . . . 568
15.3 Differentiating Matrix Functions . . . 569
15.3.1 Product, Inverse, and Power Rules . . . 569
15.3.2 Divided Differences . . . 570
15.3.3 Traces Recover the Scalar Derivative Rule . . . 572
15.3.4 The Matrix Exponential . . . 573
15.4 Polynomial Approximation of Matrix Functions . . . 574
15.5 Integral Representations and Fractional Powers . . . 580
15.5.1 The Logarithm . . . 580
15.5.2 Fractional Powers . . . 581
15.5.3 Operator Convexity of Inversion . . . 582
15.5.4 Löwner–Heinz and Operator Concavity of Powers . . . 583
15.6 Golden–Thompson Inequality . . . 583
15.7 Trace Rearrangement and Schatten Hölder Inequalities . . . 585
15.8 Generalized Klein Inequality . . . 588
15.9 Wielandt Minimax Formula . . . 589
15.10 The Operator Jensen Inequality . . . 591
15.11 Lieb's Concavity Theorem . . . 592
15.11.1 Joint Convexity of Matrix Relative Entropy . . . 594
15.11.2 A Variational Formula for the Trace . . . 595
15.11.3 Lieb's Trace-Exponential Theorem . . . 595
15.12 Lieb–Thirring Inequalities . . . 596
15.13 Matrix Concentration . . . 599

16 Convex Optimization **605**
16.1 Linear Systems and Preconditioned Richardson Iteration . . . 606
16.2 Smooth Gradient Descent . . . 607
16.3 Bregman Proximal Steps and Relative Curvature . . . 608
16.3.1 Relative Descent . . . 608
16.3.2 Where Norms Enter . . . 609
16.4 Mirror Descent, Entropy, and Multiplicative Weights . . . 609
16.4.1 Entropy on the Simplex . . . 610
16.4.2 Soft Maximum . . . 611
16.5 Matrix Multiplicative Weights . . . 611
16.6 Sparsification and Sensitivity . . . 613
16.6.1 Leverage Scores . . . 613
16.6.2 The Sampling Algorithm . . . 614
16.6.3 Why Higher Powers Can Require More Rows . . . 615
16.7 Saddle Points and Mirror-Prox . . . 616
16.7.1 From an Implicit Step to Prediction and Correction . . . 616
16.7.2 Lipschitz Operators . . . 617
16.8 Solving Box–Simplex Games . . . 618
16.8.1 The Interaction Determines the Curvature . . . 618
16.8.2 From Local Curvature to a Global Bound . . . 619
16.8.3 Choosing One Reference with Adaptive Curvature . . . 620

16.8.4 One Sweep Instead of a Joint Minimization . . . 621
16.8.5 Prediction and Correction . . . 623
16.8.6 Algorithm and Convergence . . . 624
16.9 Positive Semidefinite Packing . . . 626
16.10 Optimal Online Discrepancy Minimization . . . 630
16.10.1 A Gaussian-Preserving Step . . . 630
16.10.2 Coupling Three Moves into One Sign . . . 631
16.10.3 Algorithm and Concentration . . . 632

17 Polynomial Algorithms **634**
17.1 Formal Power Series Operations . . . 634
17.1.1 Inverse . . . 634
17.1.2 Logarithm . . . 634
17.1.3 Square Root . . . 635
17.1.4 Inverse Square Root . . . 635
17.1.5 Exponential . . . 635
17.1.6 Powers . . . 636
17.2 Polynomial Arithmetic . . . 636
17.2.1 Division . . . 636
17.2.2 Taylor Shift . . . 637
17.2.3 Evaluation Shift . . . 637
17.2.4 Half-GCD and the Euclidean Quotient Sequence . . . 638
17.2.5 Modular Inversion . . . 640
17.2.6 Resultants and Discriminants . . . 640
17.2.7 Root Finding over a Prime Field . . . 641
17.2.8 Minimum Linear Recurrence . . . 642
17.3 Rational Functions and Transposition . . . 643
17.3.1 Bostan–Mori . . . 643
17.3.2 The Transposition Principle . . . 644
17.3.3 Transposed Multiplication and Reversal Pairings . . . 645
17.3.4 Transposed Bostan–Mori and Modular Powers . . . 645
17.4 Composition and Compositional Inversion . . . 646
17.4.1 Power Projection . . . 646
17.4.2 Composition by Transposition . . . 647
17.4.3 Lagrange Inversion . . . 648
17.4.4 Compositional Inversion by One Power Projection . . . 649
17.5 Evaluation and Interpolation . . . 650
17.5.1 Multipoint Evaluation by Transposition . . . 650
17.5.2 Interpolation . . . 651
17.5.3 Chirp-Z: Geometric Evaluation . . . 652
17.5.4 Inverse Chirp-Z . . . 652
17.5.5 Hermite Interpolation . . . 653
17.6 Multivariate Evaluation by Integer Lifting . . . 654
17.6.1 Product Grids and Nilpotent Taylor Expansion . . . 655
17.6.2 Recursive Prime-Power Reduction . . . 655
17.6.3 Extension Fields . . . 657
17.7 Multilinear Polynomials . . . 659
17.7.1 Multilinear-Term Detection . . . 659
17.8 Algebraic Series, D-Finiteness, and Recurrence Algorithms . . . 663
17.8.1 Differential Equations and Coefficient Recurrences . . . 663
17.8.2 Algebraic Series and Closure Properties . . . 665
17.8.3 Root Power Sums and Products of Rational Sequences . . . 666

17.8.4 Sparse Interpolation through Rational Recurrences . . . 667
17.8.5 Distant Polynomial and Geometric Recurrences . . . 669
17.8.6 Batched Recurrences with a Varying Parameter . . . 671
17.8.7 Exponential and Logarithmic Substitution . . . 672
17.8.8 Linear-Time Translation and Single-Coefficient Queries . . . 674
17.8.9 Diagonals, Constant Terms, and Residues . . . 677
17.8.10 Coefficients in Finite Fields by Reading Index Digits . . . 678
17.9 Fast Formal-Series Equations and Transformations . . . 681
17.9.1 Newton Iteration for Scalar Differential Equations . . . 681
17.9.2 CDQ Convolution and Causal Power-Series Equations . . . 682
17.10 Fractional-Linear and Cubic Substitution . . . 684
17.10.1 Homogeneous Fractional-Linear Substitution . . . 685
17.10.2 A Reversible Symmetric Laurent Substitution . . . 686
17.10.3 Chebyshev and Cubic Substitution . . . 687
17.11 Dirichlet Series . . . 688
17.11.1 The Dirichlet Derivative and Triangular Recurrences . . . 689
17.11.2 Compressed Prefix Tables . . . 691
17.11.3 Compressed Multiplication . . . 692
17.11.4 Compressed Logarithms and Exponentials . . . 693
17.11.5 Prime Counting . . . 695
17.11.6 Logarithmic Binning and Boundary Correction . . . 696

18 Set Power Series **698**
18.1 Subset Convolution . . . 698
18.2 Transposed Subset Convolution . . . 700
18.3 Inversion and Quotient . . . 700
18.4 Exponential and Logarithm . . . 702
18.5 Composition . . . 703
18.6 Power Projection . . . 705
18.7 Power . . . 706

1 Foreword

The goal of these notes is primarily to explain interesting things and record things I've come up with which do not merit a publication. I have only come up with a small portion of the content contained herein although much I have simplified, or improved the explanation thereof. Some of the content is unfinished. Some of the algorithms contained herein have been implemented in my library.

2 Highlights

The following parts are simplifications, improved explanations, less well-known content, or material that I consider to be of unusual quality or interest.

Section	Title
3.3	Stirling's Formula
4.3	Frobenius Form
4.6	Selected-Entry Matrix Multiplication
6.6.10	Modular Subset Sum using Runs
6.6.11	Integer 3SUM
6.7	Fast Factorials and Deterministic Factorization
7.1.3	Irregular Moore Bound
7.2.6	The van der Waerden Permanent Theorem
7.6	Polynomial Method
7.7.3	Sunflower Lemma
7.9	Additive Combinatorics
7.12	Geometry
7.13.1	Komlós's Theorem
8.13	The Erdős–Sós Theorem
9.1	Sion's Minimax Theorem
9.7	Concave Min-Plus Convolution
9.8	Rectangular Monotone Min-Plus Product
10.7	A Fast Succinct Hash Table
10.9	Soft Heaps
11.7	Shallowest Decomposition Tree
11.8	Static Top Tree
11.11	Top Tree
12.1.3	Deterministic Negative Weight Shortest Paths
12.2.2	Directed Low-Diameter Decompositions and Negative Weight Shortest Paths
12.3.1	Directed Minimum Cut in Almost-Linear Time
12.8.4	Almost-Linear Maximum Flow
12.10	Spectral Sparsification
12.11	Connectivity
12.12	Hamiltonicity
12.14.5	The Tutte Polynomial
12.15	Node-Weighted Triangle
13.4	Alon–Boppana
14.4	Infinite Graphs, Recurrence, and Transience
14.5	Random Spanning Trees
14.6	Localization of Electrical Flows
15.6	Golden–Thompson Inequality
15.11	Lieb's Concavity Theorem

Section	Title
15.12	Lieb–Thirring Inequalities
16.8	Solving Box–Simplex Games
16.9	Positive Semidefinite Packing
16.10	Optimal Online Discrepancy Minimization
17.3	Rational Functions and Transposition
17.4	Composition and Compositional Inversion
17.6	Multivariate Evaluation by Integer Lifting
17.7	Multilinear Polynomials
17.8	Algebraic Series, D-Finiteness, and Recurrence Algorithms
17.11	Dirichlet Series
18.6	Power Projection

3 Analysis and Asymptotics

3.1 Stolz–Cesàro

Stolz–Cesàro is the discrete analogue of l'Hôpital's rule. The useful idea is that a ratio of two growing sequences is controlled by the ratios of their increments.

Theorem 1 (Stolz–Cesàro)**.** *Let (a_n) and (b_n) be real sequences.*

(i) *Suppose b_n is strictly increasing and*

$$b_n \to \infty.$$

Then

$$\liminf_{n\to\infty} \frac{a_{n+1}-a_n}{b_{n+1}-b_n} \le \liminf_{n\to\infty} \frac{a_n}{b_n} \le \limsup_{n\to\infty} \frac{a_n}{b_n} \le \limsup_{n\to\infty} \frac{a_{n+1}-a_n}{b_{n+1}-b_n}.$$

(ii) *Suppose*

$$a_n \to 0, \qquad b_n \to 0,$$

and b_n is strictly decreasing. Then the same inequalities hold:

$$\liminf_{n\to\infty} \frac{a_{n+1}-a_n}{b_{n+1}-b_n} \le \liminf_{n\to\infty} \frac{a_n}{b_n} \le \limsup_{n\to\infty} \frac{a_n}{b_n} \le \limsup_{n\to\infty} \frac{a_{n+1}-a_n}{b_{n+1}-b_n}.$$

In either case, if

$$\frac{a_{n+1}-a_n}{b_{n+1}-b_n} \to L,$$

then

$$\frac{a_n}{b_n} \to L.$$

Proof. Write

$$\ell = \liminf_{n\to\infty} \frac{a_{n+1}-a_n}{b_{n+1}-b_n}, \qquad u = \limsup_{n\to\infty} \frac{a_{n+1}-a_n}{b_{n+1}-b_n}.$$

Fix $\varepsilon > 0$.

For the ∞ case, for all sufficiently large k,

$$(\ell-\varepsilon)(b_{k+1}-b_k) \le a_{k+1}-a_k \le (u+\varepsilon)(b_{k+1}-b_k).$$

Summing from a fixed sufficiently large N to $n-1$ gives

$$(\ell-\varepsilon)(b_n-b_N) \le a_n-a_N \le (u+\varepsilon)(b_n-b_N).$$

Divide by b_n and let $n \to \infty$. Since $b_n \to \infty$, the fixed terms disappear, yielding

$$\ell - \varepsilon \le \liminf_{n\to\infty} \frac{a_n}{b_n} \le \limsup_{n\to\infty} \frac{a_n}{b_n} \le u + \varepsilon.$$

For the $0/0$ case, it is more convenient to reverse both differences. Since b_n is strictly decreasing,

$$b_k - b_{k+1} > 0,$$

and

$$\frac{a_k - a_{k+1}}{b_k - b_{k+1}} = \frac{a_{k+1} - a_k}{b_{k+1} - b_k}.$$

Thus, for all sufficiently large k,

$$(\ell - \varepsilon)(b_k - b_{k+1}) \le a_k - a_{k+1} \le (u + \varepsilon)(b_k - b_{k+1}).$$

Sum from $k = n$ to $m - 1$:

$$(\ell - \varepsilon)(b_n - b_m) \le a_n - a_m \le (u + \varepsilon)(b_n - b_m).$$

Now let $m \to \infty$. Since

$$a_m \to 0, \qquad b_m \to 0,$$

we obtain

$$(\ell - \varepsilon)b_n \le a_n \le (u + \varepsilon)b_n.$$

A strictly decreasing sequence converging to 0 is positive, so dividing by b_n gives

$$\ell - \varepsilon \le \frac{a_n}{b_n} \le u + \varepsilon$$

for all sufficiently large n. Hence again

$$\ell - \varepsilon \le \liminf_{n\to\infty} \frac{a_n}{b_n} \le \limsup_{n\to\infty} \frac{a_n}{b_n} \le u + \varepsilon.$$

Letting $\varepsilon \to 0$ proves both statements. □

3.2 Wallis Product

Theorem 2 (Wallis Product)**.**

$$\frac{\pi}{2} = \prod_{k=1}^{\infty} \frac{(2k)^2}{(2k-1)(2k+1)}.$$

Proof. Let

$$I_m = \int_0^{\pi/2} \sin^m x \, dx.$$

Integration by parts gives, for $m \ge 2$,

$$I_m = \frac{m-1}{m} I_{m-2}.$$

Since $I_0 = \pi/2$ and $I_1 = 1$, this gives

$$I_{2n} = \frac{\pi}{2} \prod_{k=1}^{n} \frac{2k-1}{2k}, \qquad I_{2n+1} = \prod_{k=1}^{n} \frac{2k}{2k+1}.$$

Because $0 \le \sin x \le 1$,

$$I_{2n+1} \le I_{2n} \le I_{2n-1}.$$

The recurrence also gives

$$\frac{I_{2n-1}}{I_{2n+1}} = \frac{2n+1}{2n} \to 1.$$

Therefore, by squeezing,

$$\frac{I_{2n}}{I_{2n+1}} \to 1.$$

On the other hand,

$$\frac{I_{2n}}{I_{2n+1}} = \frac{\pi}{2}\frac{(2n-1)!!(2n+1)!!}{((2n)!!)^2} = \frac{\pi/2}{\displaystyle\prod_{k=1}^{n}\frac{(2k)^2}{(2k-1)(2k+1)}}.$$

Thus the product tends to $\pi/2$. □

The same argument immediately gives the asymptotic form of the central binomial coefficient, which will determine the constant in Stirling's formula.

Corollary 3 (Central Binomial Coefficient)**.**

$$\binom{2n}{n} \sim \frac{4^n}{\sqrt{\pi n}}.$$

Proof. Set

$$c_n = \frac{\binom{2n}{n}}{4^n} = \frac{(2n-1)!!}{(2n)!!}.$$

Then

$$I_{2n} = \frac{\pi}{2}c_n, \qquad I_{2n+1} = \frac{1}{(2n+1)c_n}.$$

Since $I_{2n}/I_{2n+1} \to 1$,

$$\frac{\pi}{2}(2n+1)c_n^2 \to 1.$$

Therefore

$$c_n \sim \frac{1}{\sqrt{\pi n}},$$

which is the claim. □

3.3 Stirling's Formula

Theorem 4 (Stirling's Formula)**.**

$$n! \sim \sqrt{2\pi n}\left(\frac{n}{e}\right)^n.$$

Proof. We proceed in three steps. The integral approximation

$$\log(n!) = \sum_{k=1}^{n}\log k \approx \int_1^n \log x\,dx = n\log n - n + 1$$

suggests first removing the terms $n\log n - n$. Define

$$s_n = \log(n!) - n\log n + n.$$

We determine the size of the remaining error using Stolz–Cesàro with $b_n = \log n$. We have

$$s_{n+1} - s_n = 1 - n\log\left(1 + \frac{1}{n}\right),$$

and

$$\log(n+1) - \log n = \log\left(1 + \frac{1}{n}\right).$$

Thus

$$\frac{s_{n+1} - s_n}{\log(n+1) - \log n} = \frac{1}{\log(1+1/n)} - n.$$

and hence

$$\frac{s_{n+1} - s_n}{\log(n+1) - \log n} \to \frac{1}{2}.$$

Stolz–Cesàro therefore gives

$$\frac{s_n}{\log n} \to \frac{1}{2}.$$

So the next-order term is forced to be $\frac{1}{2}\log n$. This motivates

$$c_n = \log(n!) - \left(n + \frac{1}{2}\right)\log n + n.$$

We have

$$c_{n+1} - c_n = 1 - \left(n + \frac{1}{2}\right)\log\left(1 + \frac{1}{n}\right).$$

Set

$$m = n + \frac{1}{2}.$$

Then

$$\log\left(1 + \frac{1}{n}\right) = \log\frac{m + 1/2}{m - 1/2} = \int_{-1/2}^{1/2} \frac{dt}{m+t}.$$

Therefore

$$\begin{aligned} m\log\left(1 + \frac{1}{n}\right) - 1 &= \int_{-1/2}^{1/2} \left(\frac{m}{m+t} - 1\right) dt \\ &= \int_0^{1/2} \left(\frac{m}{m+t} + \frac{m}{m-t} - 2\right) dt \\ &= \int_0^{1/2} \frac{2t^2}{m^2 - t^2}\, dt. \end{aligned}$$

In particular,

$$0 \le m\log\left(1 + \frac{1}{n}\right) - 1 = O(n^{-2}).$$

Hence

$$|c_{n+1} - c_n| = O(n^{-2}).$$

Since $\sum_n n^{-2} < \infty$, the sequence (c_n) converges. Write

$$c_n \to \log C.$$

Exponentiating gives

$$n! \sim C\sqrt{n}\left(\frac{n}{e}\right)^n$$

for some constant $C > 0$.

Apply the preceding asymptotic to

$$\binom{2n}{n} = \frac{(2n)!}{(n!)^2}.$$

We obtain

$$\begin{aligned}\binom{2n}{n} &\sim \frac{C\sqrt{2n}(2n/e)^{2n}}{C^2 n(n/e)^{2n}} \\ &= \frac{\sqrt{2}}{C}\frac{4^n}{\sqrt{n}}.\end{aligned}$$

But by Corollary 3,

$$\binom{2n}{n} \sim \frac{4^n}{\sqrt{\pi n}}.$$

Comparing constants gives

$$\frac{\sqrt{2}}{C} = \frac{1}{\sqrt{\pi}},$$

so

$$C = \sqrt{2\pi}.$$

Therefore

$$n! \sim \sqrt{2\pi n}\left(\frac{n}{e}\right)^n.$$

□

3.4 Karamata's Inequality

Karamata's inequality is a consequence of two facts:

- majorization says that the partial-sum differences have one sign;
- convexity says that secant slopes increase as their endpoints move to the right.

Summation by parts then combines these two.

Theorem 5 (Karamata's Inequality)**.** *Let*

$$x, y \in \mathbb{R}^n, \qquad x \prec y,$$

and let f be convex on an interval containing all coordinates of x and y. Then

$$\sum_{i=1}^{n} f(x_i) \le \sum_{i=1}^{n} f(y_i).$$

More generally, if

$$x \prec_w y$$

and f is convex and nondecreasing, then the same inequality holds.

Proof. Since both majorization and the desired inequality are invariant under permuting coordinates, assume

$$x_1 \ge \cdots \ge x_n, \qquad y_1 \ge \cdots \ge y_n.$$

Define the partial sums

$$X_k = \sum_{i=1}^{k} x_i, \qquad Y_k = \sum_{i=1}^{k} y_i,$$

and their difference

$$D_k = Y_k - X_k, \qquad D_0 = 0.$$

Since $x \prec y$,

$$D_k \geq 0 \qquad (1 \leq k < n),$$

and

$$D_n = 0.$$

If $x_i = y_i$ for some i, we may delete both coordinates. The two remaining sequences are still decreasing, and their partial-sum inequalities are unchanged after reindexing. Thus we may assume

$$x_i \neq y_i \qquad \text{for every } i.$$

For each i, define the secant slope

$$c_i = \frac{f(y_i) - f(x_i)}{y_i - x_i}.$$

The basic one-dimensional fact about a convex function is that its secant slope is nondecreasing in either endpoint: if

$$a \leq a', \qquad b \leq b',$$

then

$$\frac{f(b) - f(a)}{b - a} \leq \frac{f(b') - f(a')}{b' - a'}$$

whenever the quotients are defined.

Since

$$x_i \geq x_{i+1}, \qquad y_i \geq y_{i+1},$$

both endpoints of the i-th secant lie no farther left than the corresponding endpoints of the $(i+1)$-st secant. Therefore

$$c_1 \geq c_2 \geq \cdots \geq c_n.$$

Now

$$y_i - x_i = D_i - D_{i-1},$$

so

$$\begin{aligned} \sum_{i=1}^{n} \big(f(y_i) - f(x_i)\big) &= \sum_{i=1}^{n} c_i (y_i - x_i) \\ &= \sum_{i=1}^{n} c_i (D_i - D_{i-1}). \end{aligned}$$

Summation by parts gives

$$\sum_{i=1}^{n} c_i (D_i - D_{i-1}) = c_n D_n + \sum_{i=1}^{n-1} (c_i - c_{i+1}) D_i.$$

For ordinary majorization,

$$D_n = 0,$$

while

$$c_i - c_{i+1} \geq 0, \qquad D_i \geq 0.$$

Hence

$$\sum_{i=1}^{n} \big(f(y_i) - f(x_i)\big) \geq 0,$$

which proves

$$\sum_i f(x_i) \le \sum_i f(y_i).$$

For weak majorization $x \prec_w y$, we instead have

$$D_i \ge 0 \qquad (1 \le i \le n).$$

If f is also nondecreasing, every secant slope satisfies

$$c_i \ge 0.$$

Thus the additional endpoint term

$$c_n D_n$$

is also nonnegative, and the same argument proves the weak form. □

3.5 Polynomial Root Bounds

3.5.1 Real Roots from Their Mean and Variance

The first two coefficients determine the sum and sum of squares of the roots. When the roots are real, this controls how far any one root can lie from their mean: the other roots must compensate for its deviation, and doing so consumes part of the same sum-of-squares budget.

Theorem 6 (Real-rooted polynomial bound)**.** *Let*

$$p(x) = x^n + a_{n-1}x^{n-1} + \cdots + a_0, \qquad n \ge 2,$$

have only real roots. Then every root lies in the interval with endpoints

$$-\frac{a_{n-1}}{n} \pm \frac{n-1}{n}\sqrt{a_{n-1}^2 - \frac{2n}{n-1}a_{n-2}}.$$

Proof. Write the roots, counted with multiplicity, as $r_1, \ldots, r_n$. Let $\mu = n^{-1}\sum_i r_i$ and $\sigma^2 = n^{-1}\sum_i (r_i - \mu)^2$. Vieta's formulas give

$$\sum_i r_i = -a_{n-1}, \qquad \sum_i r_i^2 = \left(\sum_i r_i\right)^2 - 2\sum_{i<j} r_i r_j = a_{n-1}^2 - 2a_{n-2}.$$

Consequently,

$$\mu = -\frac{a_{n-1}}{n}, \qquad \sigma^2 = \frac{a_{n-1}^2 - 2a_{n-2}}{n} - \frac{a_{n-1}^2}{n^2} = \frac{n-1}{n^2}a_{n-1}^2 - \frac{2}{n}a_{n-2}.$$

Fix i. Since the deviations sum to zero, Cauchy–Schwarz gives

$$\begin{aligned}(r_i - \mu)^2 &= \left(\sum_{j \ne i}(r_j - \mu)\right)^2 \\ &\le (n-1)\sum_{j \ne i}(r_j - \mu)^2 \\ &= (n-1)\big(n\sigma^2 - (r_i - \mu)^2\big).\end{aligned}$$

Rearranging yields $(r_i - \mu)^2 \le (n-1)\sigma^2$, or

$$|r_i - \mu| \le \sqrt{n-1}\,\sigma = \frac{n-1}{n}\sqrt{a_{n-1}^2 - \frac{2n}{n-1}a_{n-2}}.$$

Substitute the value of μ. □

The bound is sharp: equality for a particular root holds exactly when the other $n-1$ roots are equal. More generally, the proof shows that any n real numbers of mean μ and variance σ^2 all lie within $\sqrt{n-1}\,\sigma$ of μ.

3.5.2 Positive Coefficients and Consecutive Ratios

Positive coefficients give bounds on root magnitudes without requiring the roots to be real.

Theorem 7 (Eneström–Kakeya)**.** *Let $p(z) = \sum_{k=0}^{n} a_k z^k$, where $n \geq 1$ and every $a_k > 0$. Put*

$$m = \min_{0 \leq k < n} \frac{a_k}{a_{k+1}}, \qquad M = \max_{0 \leq k < n} \frac{a_k}{a_{k+1}}.$$

Then every complex root z of p satisfies

$$m \leq |z| \leq M.$$

In particular, every real root belongs to $[-M, -m]$.

Proof. First suppose $q(z) = \sum_{k=0}^{n} b_k z^k$ has $0 < b_0 \leq b_1 \leq \cdots \leq b_n$. Then

$$(1-z)q(z) = b_0 + \sum_{k=1}^{n} (b_k - b_{k-1}) z^k - b_n z^{n+1}.$$

If $q(w) = 0$ and $t = |w| > 1$, the triangle inequality gives

$$\begin{aligned} b_n t^{n+1} &\leq b_0 + \sum_{k=1}^{n} (b_k - b_{k-1}) t^k \\ &\leq t^n \left(b_0 + \sum_{k=1}^{n} (b_k - b_{k-1}) \right) = b_n t^n, \end{aligned}$$

a contradiction. Thus every root of q lies in the closed unit disk.

For the original polynomial, take $q(w) = p(Mw)$. Its coefficients $b_k = a_k M^k$ are nondecreasing because

$$\frac{b_{k+1}}{b_k} = M \frac{a_{k+1}}{a_k} \geq 1.$$

If $p(z) = 0$, then $q(z/M) = 0$, so $|z| \leq M$.

For the lower bound, apply the upper bound to the reversed polynomial

$$p^*(w) = w^n p(1/w) = a_n + a_{n-1} w + \cdots + a_0 w^n.$$

Since $a_0 > 0$, every root z of p is nonzero, and $1/z$ is a root of p^*. Therefore

$$\frac{1}{|z|} \leq \max_{0 \leq k < n} \frac{a_{k+1}}{a_k} = \frac{1}{m},$$

which gives $|z| \geq m$.

Finally, $p(x) > 0$ for $x \geq 0$, so every real root is negative. □

3.5.3 Gauss–Lucas Theorem

Differentiation cannot move roots outside their convex hull.

Theorem 8 (Gauss–Lucas)**.** *Let $p \in \mathbb{C}[z]$ be nonconstant. Every zero of p' lies in the convex hull of the zeros of p.*

Proof. Write $p(z) = c\prod_{j=1}^{n}(z - r_j)$, listing the roots with multiplicity. For z not a root of p, the product rule gives

$$\frac{p'(z)}{p(z)} = \sum_{j=1}^{n} \frac{1}{z - r_j}.$$

Let $p'(w) = 0$. If $p(w) = 0$, there is nothing to prove. Otherwise,

$$0 = \sum_{j=1}^{n} \frac{1}{w - r_j} = \sum_{j=1}^{n} \frac{\overline{w} - \overline{r_j}}{|w - r_j|^2}.$$

Take complex conjugates and collect the terms containing w:

$$w \sum_{j=1}^{n} \frac{1}{|w - r_j|^2} = \sum_{j=1}^{n} \frac{r_j}{|w - r_j|^2}.$$

Consequently,

$$w = \sum_{j=1}^{n} \alpha_j r_j, \qquad \alpha_j = \frac{|w - r_j|^{-2}}{\sum_{i=1}^{n} |w - r_i|^{-2}} > 0, \qquad \sum_{j=1}^{n} \alpha_j = 1.$$

Thus w is a convex combination of the roots. □

3.6 Banach Fixed Point Theorem

A map $T : X \to X$ on a metric space is a *contraction* if there is some $q < 1$ such that

$$d(Tx, Ty) \le q\, d(x, y)$$

for every $x, y \in X$.

The point is that repeatedly applying a contraction makes successive errors shrink geometrically.

Theorem 9 (Banach Fixed Point Theorem)**.** *Let X be a complete metric space and let $T : X \to X$ be a contraction with constant $q < 1$. Then T has a unique fixed point x^*.*

Moreover, starting from any $x_0 \in X$ and defining $x_{n+1} = T(x_n)$, we have

$$d(x_n, x^*) \le \frac{q^n}{1 - q}\, d(x_1, x_0).$$

Proof. Since T contracts distances,

$$d(x_{n+1}, x_n) \le q^n d(x_1, x_0).$$

Thus for $m > n$, the triangle inequality gives

$$d(x_m, x_n) \le \sum_{j=n}^{m-1} d(x_{j+1}, x_j) \le d(x_1, x_0) \sum_{j=n}^{\infty} q^j = \frac{q^n}{1 - q} d(x_1, x_0).$$

Hence (x_n) is Cauchy. Completeness gives $x_n \to x^*$ for some $x^* \in X$.

A contraction is continuous, so

$$T(x^*) = T\left(\lim_n x_n\right) = \lim_n T(x_n) = \lim_n x_{n+1} = x^*.$$

Thus x^* is a fixed point. Letting $m \to \infty$ in the preceding estimate gives the claimed error bound.

Finally, if y^* is another fixed point, then

$$d(x^*, y^*) = d(Tx^*, Ty^*) \le q\, d(x^*, y^*).$$

Since $q < 1$, this forces $x^* = y^*$. □

The theorem is especially useful when an equation can be rewritten as $x = T(x)$. If T is a contraction on a closed region and maps that region into itself, the equation has exactly one solution there.

3.7 Inverse Function Theorem

Suppose $F : \mathbb{R}^n \to \mathbb{R}^n$ is differentiable near a, and let $A = DF(a)$. If A is invertible, then near a the map F should behave like the invertible linear map

$$F(a) + A(x - a).$$

The clean way to make this precise is to solve $F(x) = y$ by fixed-point iteration. Rearranging the equation gives

$$x = x - A^{-1}(F(x) - y).$$

At $x = a$, the derivative of the right-hand side with respect to x is

$$I - A^{-1}DF(a) = 0.$$

Thus, sufficiently close to a, this map is a strong contraction.

Theorem 10 (Inverse Function Theorem)**.** *Let $U \subseteq \mathbb{R}^n$ be open and let $F : U \to \mathbb{R}^n$ be continuously differentiable. Suppose $a \in U$ and $DF(a)$ is invertible.*

Then there are open neighborhoods V of a and W of $F(a)$ such that

$$F : V \to W$$

is a bijection whose inverse $G : W \to V$ is continuously differentiable.

Moreover,

$$DG(y) = DF(G(y))^{-1}.$$

In particular,

$$DG(F(a)) = DF(a)^{-1}.$$

Proof. Set $b = F(a)$ and $A = DF(a)$. For each y near b, define

$$T_y(x) = x - A^{-1}(F(x) - y).$$

A point x is fixed by T_y exactly when $F(x) = y$.

The derivative with respect to x is

$$DT_y(x) = I - A^{-1}DF(x).$$

At $x = a$ this is 0. Since DF is continuous, we may choose a closed ball $\overline{B}(a, r) \subseteq U$ and some $q < 1$ such that

$$\|I - A^{-1}DF(x)\| \leq q \qquad (x \in \overline{B}(a, r)).$$

By the mean value inequality,

$$\|T_y(x) - T_y(x')\| \leq q\|x - x'\|$$

throughout this ball.

We next ensure that T_y maps the ball into itself. Since

$$T_y(a) = a + A^{-1}(y - b),$$

for $x \in \overline{B}(a, r)$,

$$|T_y(x) - a| \le |T_y(x) - T_y(a)| + |T_y(a) - a| \le qr + |A^{-1}(y - b)|.$$

Thus if y is close enough to b that

$$\|A^{-1}(y - b)\| < (1 - q)r,$$

then T_y maps $\overline{B}(a, r)$ into itself.

Let

$$W = \{y : \|A^{-1}(y - b)\| < (1 - q)r\}.$$

For every $y \in W$, Banach's theorem gives a unique $x \in \overline{B}(a, r)$ satisfying $F(x) = y$. In fact the strict inequality above shows $x \in B(a, r)$. Define this unique solution to be $G(y)$.

Let

$$V = B(a, r) \cap F^{-1}(W).$$

Then V and W are open, and $F : V \to W$ is bijective with inverse G.

It remains to prove that G is continuously differentiable.

First observe that G is locally Lipschitz. Define $\Psi(x) = x - A^{-1}F(x)$. The derivative bound above says that Ψ is q-Lipschitz on the ball. If $x = G(y)$ and $x' = G(y')$, then

$$x = \Psi(x) + A^{-1}y, \qquad x' = \Psi(x') + A^{-1}y'.$$

Therefore

$$\|x - x'\| \le q\|x - x'\| + \|A^{-1}\|\,\|y - y'\|,$$

so

$$\|G(y) - G(y')\| \le \frac{\|A^{-1}\|}{1 - q}\|y - y'\|.$$

Now fix $y \in W$, put $x = G(y)$, and let

$$G(y + h) = x + k.$$

The Lipschitz estimate gives $k = O(\|h\|)$. Since $F(x + k) = y + h$ and $F(x) = y$,

$$h = F(x + k) - F(x) = DF(x)k + o(\|k\|) = DF(x)k + o(\|h\|).$$

The matrix $DF(x)$ is invertible: indeed $\|I - A^{-1}DF(x)\| \le q < 1$, so $A^{-1}DF(x)$ is invertible by the geometric-series argument. Hence

$$k = DF(x)^{-1}h + o(\|h\|).$$

Thus G is differentiable and

$$DG(y) = DF(G(y))^{-1}.$$

Continuity follows from the continuity of DF and of matrix inversion on the invertible matrices. $\square$

3.8 Implicit Function Theorem

The implicit function theorem says that if a system of equations can be uniquely solved for some variables at the linearized level, then it can also be solved for those variables locally in the nonlinear system.

Theorem 11 (Implicit Function Theorem)**.** *Let*

$$F : U \subseteq \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}^m$$

be continuously differentiable, and suppose

$$F(a, b) = 0.$$

If the $m \times m$ matrix

$$D_y F(a, b)$$

is invertible, then there are neighborhoods X of a and Y of b, and a unique continuously differentiable function

$$g : X \to Y$$

such that

$$F(x, g(x)) = 0$$

for every $x \in X$.
Moreover,

$$Dg(x) = -D_y F(x, g(x))^{-1} D_x F(x, g(x)).$$

Proof. The theorem is an immediate application of the inverse function theorem. Define

$$H(x, y) = (x, F(x, y)).$$

Its derivative at (a, b) is the block matrix

$$DH(a, b) = \begin{pmatrix} I & 0 \\ D_x F(a, b) & D_y F(a, b) \end{pmatrix}.$$

This is invertible because $D_y F(a, b)$ is invertible.
By the inverse function theorem, H has a continuously differentiable local inverse near

$$H(a, b) = (a, 0).$$

Since the first component of $H(x, y)$ is exactly x, its inverse must have the form

$$H^{-1}(x, z) = (x, \psi(x, z)).$$

Set

$$g(x) = \psi(x, 0).$$

Then

$$H(x, g(x)) = (x, 0),$$

which means exactly

$$F(x, g(x)) = 0.$$

Local uniqueness follows from the local injectivity of H.
To compute the derivative, differentiate

$$F(x, g(x)) = 0.$$

The chain rule gives

$$D_x F(x, g(x)) + D_y F(x, g(x)) Dg(x) = 0.$$

Solving for $Dg(x)$ yields

$$Dg(x) = -D_y F(x, g(x))^{-1} D_x F(x, g(x)).$$

□

3.9 Lagrange Multipliers

Suppose we want to optimize $f : \mathbb{R}^n \to \mathbb{R}$ subject to m continuously differentiable equality constraints

$$g_1(x) = \cdots = g_m(x) = 0.$$

Write

$$g = (g_1, \ldots, g_m) : \mathbb{R}^n \to \mathbb{R}^m.$$

If the constraint gradients are linearly independent, the implicit function theorem says that the feasible set is locally an $(n - m)$-dimensional surface. At a constrained extremum, f cannot change to first order in any tangent direction to this surface. Its gradient must therefore point entirely in the normal directions, which are exactly the directions spanned by the constraint gradients.

Theorem 12 (Lagrange Multipliers)**.** *Let $f, g_1, \ldots, g_m$ be continuously differentiable near $a \in \mathbb{R}^n$, and suppose*

$$g_1(a) = \cdots = g_m(a) = 0.$$

Assume that

$$\nabla g_1(a), \ldots, \nabla g_m(a)$$

are linearly independent.

If a is a local maximum or minimum of f subject to the constraints $g_1 = \cdots = g_m = 0$, then there are unique scalars $\lambda_1, \ldots, \lambda_m$ such that

$$\nabla f(a) = \sum_{i=1}^{m} \lambda_i \nabla g_i(a).$$

Proof. Since the constraint gradients are linearly independent, the Jacobian $Dg(a)$ has rank m. After permuting the coordinates, write

$$x = (u, v), \qquad u \in \mathbb{R}^{n-m}, \qquad v \in \mathbb{R}^m,$$

so that the square matrix $D_v g(a)$ is invertible.
Write $a = (u_0, v_0)$. By the implicit function theorem, all nearby feasible points are uniquely of the form

$$(u, \phi(u))$$

for a continuously differentiable function ϕ, with

$$g(u, \phi(u)) = 0.$$

Differentiating this identity at u_0 gives

$$D_u g(a) + D_v g(a) D\phi(u_0) = 0,$$

and hence

$$D\phi(u_0) = -D_v g(a)^{-1} D_u g(a).$$

Now define the restriction of f to the feasible surface by

$$h(u) = f(u, \phi(u)).$$

Since a is a constrained local extremum, u_0 is an ordinary local extremum of h. Therefore $Dh(u_0) = 0$. By the chain rule,

$$D_u f(a) + D_v f(a) D\phi(u_0) = 0.$$

Substituting the formula for $D\phi(u_0)$,

$$D_u f(a) = D_v f(a) D_v g(a)^{-1} D_u g(a).$$

Define the row vector

$$\lambda^T = D_v f(a) D_v g(a)^{-1}.$$

Then

$$D_u f(a) = \lambda^T D_u g(a), \qquad D_v f(a) = \lambda^T D_v g(a).$$

Together these say

$$Df(a) = \lambda^T Dg(a).$$

Writing this in terms of gradients gives

$$\nabla f(a) = \sum_{i=1}^{m} \lambda_i \nabla g_i(a).$$

The multipliers are unique because the constraint gradients are linearly independent. □

3.10 The Isoperimetric Inequality

The isoperimetric inequality says that among all simple closed curves of a given length, the circle encloses the largest area:

$$4\pi A \leq L^2.$$

There are two ingredients.

First, the area enclosed by an oriented curve can be written as

$$2A = \int (x\, dy - y\, dx).$$

For polygons, this is just the shoelace formula, and the shoelace formula itself follows immediately by writing the area of each triangle as a determinant and observing that internal edges cancel.

Second, if a periodic function has mean zero, then it cannot be large without changing substantially:

$$\int_0^{2\pi} f^2 \le \int_0^{2\pi} (f')^2.$$

This is the periodic Wirtinger inequality.

Lemma 13 (Shoelace Formula)**.** *Let*

$$p_i = (x_i, y_i), \qquad i = 1, \dots, n,$$

be the vertices of a simple polygon P, listed counterclockwise, with $p_{n+1} = p_1$. Then

$$\operatorname{area}(P) = \frac{1}{2} \sum_{i=1}^{n} \det(p_i, p_{i+1}) = \frac{1}{2} \sum_{i=1}^{n} (x_i y_{i+1} - y_i x_{i+1}).$$

Proof. For three points a, b, c, the signed area of the oriented triangle abc is

$$[\triangle abc] = \frac{1}{2} \det(b - a, c - a).$$

Expanding the determinant gives

$$\begin{aligned} \det(b - a, c - a) &= \det(b, c) - \det(b, a) - \det(a, c) \\ &= \det(a, b) + \det(b, c) + \det(c, a). \end{aligned}$$

Thus

$$2[\triangle abc] = \det(a, b) + \det(b, c) + \det(c, a).$$

In other words, twice the signed area of a triangle is the sum of $\det(u, v)$ over its three oriented boundary edges $u \to v$.

Now triangulate P, and orient every triangle counterclockwise. Since the triangles have disjoint interiors,

$$\operatorname{area}(P) = \sum_{\triangle \in T} \operatorname{area}(\triangle).$$

Hence

$$2 \operatorname{area}(P) = \sum_{\triangle \in T} \sum_{(u,v) \in \partial \triangle} \det(u, v).$$

Every internal edge uv occurs in exactly two triangles. One triangle traverses it as $u \to v$, while the other traverses it as $v \to u$, so their contributions cancel:

$$\det(u, v) + \det(v, u) = 0.$$

Thus only the boundary edges remain. These are

$$p_1 \to p_2, \quad p_2 \to p_3, \quad \dots, \quad p_n \to p_1,$$

and therefore

$$2 \operatorname{area}(P) = \sum_{i=1}^{n} \det(p_i, p_{i+1}).$$

□

Lemma 14 (Signed Area Formula)**.** *Let*

$$\gamma(t) = (x(t), y(t)), \qquad a \le t \le b,$$

be a counterclockwise simple closed piecewise C^1 curve enclosing a region of area A. Then

$$2A = \int_a^b \big(x(t)y'(t) - y(t)x'(t)\big)\, dt.$$

Proof. First suppose that γ is polygonal, with consecutive vertices $p_1, \dots, p_n$. On an edge from p to q, parameterized by

$$\gamma(t) = (1-t)p + tq, \qquad 0 \le t \le 1,$$

we have

$$\gamma'(t) = q - p,$$

and hence

$$\det(\gamma(t), \gamma'(t)) = \det(p, q).$$

Therefore

$$\int_{\text{edge}} (x\, dy - y\, dx) = \det(p, q).$$

Summing over all edges and applying Lemma 13 gives

$$\int_\gamma (x\, dy - y\, dx) = 2A.$$

For a general piecewise C^1 simple closed curve, choose increasingly fine inscribed polygonal approximations P_k. Since γ is piecewise C^1, the polygonal parameterizations converge uniformly to γ, while their derivatives converge to γ' in L^1 on each C^1 piece. Consequently,

$$\int_{\partial P_k} (x\, dy - y\, dx) \longrightarrow \int_\gamma (x\, dy - y\, dx).$$

The enclosed polygonal regions also converge to the region enclosed by γ, so

$$\text{area}(P_k) \longrightarrow A.$$

Applying the polygonal formula and taking limits gives

$$2A = \int_\gamma (x\, dy - y\, dx).$$

□

We next prove the analytic ingredient. We begin with the version on an interval.
Minimizing

$$\lambda \int_0^\pi (u')^2 - \int_0^\pi u^2$$

subject to

$$u(0) = u(\pi) = 0,$$

we guess that $u'u'' \propto uu'$, so $u = C\sin t$.
We then factor the extremizer out:

$$u(t) = v(t)\sin t.$$

The gap in the inequality will measure exactly how far v is from being constant.

Lemma 15 (Dirichlet Wirtinger Inequality)**.** *Let*

$$u : [0, \pi] \to \mathbb{R}$$

be continuously differentiable with

$$u(0) = u(\pi) = 0.$$

Then

$$\int_0^\pi u(t)^2\,dt \le \int_0^\pi u'(t)^2\,dt.$$

Equality holds exactly for

$$u(t) = C\sin t.$$

Proof. For $0 < t < \pi$, write

$$u(t) = v(t)\sin t.$$

Then

$$u' = v'\sin t + v\cos t.$$

Consequently,

$$(u')^2 - u^2 = (v')^2\sin^2 t + 2vv'\sin t\cos t + v^2(\cos^2 t - \sin^2 t).$$

The last two terms form a derivative:

$$\frac{d}{dt}\big(v^2\sin t\cos t\big) = 2vv'\sin t\cos t + v^2(\cos^2 t - \sin^2 t).$$

Thus

$$(u')^2 - u^2 = (v')^2\sin^2 t + \frac{d}{dt}\big(v^2\sin t\cos t\big).$$

Integrating first over

$$[\varepsilon, \pi - \varepsilon]$$

gives

$$\int_\varepsilon^{\pi-\varepsilon}\big((u')^2 - u^2\big)\,dt = \int_\varepsilon^{\pi-\varepsilon}(v')^2\sin^2 t\,dt + \big[v^2\sin t\cos t\big]_\varepsilon^{\pi-\varepsilon}.$$

Since u is continuously differentiable and vanishes at the endpoints,

$$u(t) = O(t) \qquad (t \to 0)$$

and

$$u(t) = O(\pi - t) \qquad (t \to \pi).$$

Since $\sin t$ has the same behavior, $v = u/\sin t$ remains bounded near both endpoints. Hence

$$v(t)^2 \sin t \cos t \to 0$$

at both endpoints.
Letting $\varepsilon \to 0$ therefore gives

$$\int_0^\pi \left((u')^2 - u^2\right) dt = \int_0^\pi \sin^2 t \left(\frac{d}{dt}\frac{u(t)}{\sin t}\right)^2 dt \geq 0.$$

This proves the inequality.
Equality holds precisely when

$$v' = 0,$$

so v is constant and

$$u(t) = C \sin t.$$

□

Lemma 16 (Mean-Zero Wirtinger Inequality)**.** *Let*

$$f : [0, \pi] \to \mathbb{R}$$

be continuously differentiable and suppose

$$\int_0^\pi f(t)\, dt = 0.$$

Then

$$\int_0^\pi f(t)^2\, dt \leq \int_0^\pi f'(t)^2\, dt.$$

Proof. Define

$$F(x) = \int_0^x f(t)\, dt.$$

Then

$$F(0) = 0,$$

while the mean-zero assumption gives

$$F(\pi) = 0.$$

Thus Lemma 15 gives

$$\int_0^\pi F^2 \leq \int_0^\pi (F')^2 = \int_0^\pi f^2.$$

Since $F' = f$, integration by parts gives

$$\begin{aligned}\int_0^\pi f^2 &= \int_0^\pi F' f \\ &= [Ff]_0^\pi - \int_0^\pi F f' \\ &= -\int_0^\pi F f'.\end{aligned}$$

Hence Cauchy–Schwarz gives

$$\int_0^\pi f^2 \le \left(\int_0^\pi F^2\right)^{1/2} \left(\int_0^\pi (f')^2\right)^{1/2}.$$

Using

$$\int_0^\pi F^2 \le \int_0^\pi f^2,$$

we obtain

$$\int_0^\pi f^2 \le \left(\int_0^\pi f^2\right)^{1/2} \left(\int_0^\pi (f')^2\right)^{1/2}.$$

Canceling the first factor proves

$$\int_0^\pi f^2 \le \int_0^\pi (f')^2.$$

□

We now pass from an interval to a circle.

The only issue is that the preceding lemma requires an interval of length π on which the integral is zero. If a function has mean zero on the whole circle, such a semicircle always exists.

Indeed, slide a semicircle around the circle and measure the integral of f over it. Rotating the semicircle by π replaces it by its complement, so its integral changes sign. Somewhere in between, it must be zero.

Lemma 17 (Periodic Wirtinger Inequality)**.** *Let*

$$f : \mathbb{R} \to \mathbb{R}$$

be continuously differentiable and 2π-periodic. If

$$\int_0^{2\pi} f(t)\,dt = 0,$$

then

$$\int_0^{2\pi} f(t)^2\,dt \le \int_0^{2\pi} f'(t)^2\,dt.$$

Proof. For $a \in \mathbb{R}$, define

$$H(a) = \int_a^{a+\pi} f(t)\,dt.$$

The function H is continuous.

The intervals

$$[a, a+\pi] \qquad \text{and} \qquad [a+\pi, a+2\pi]$$

together form one full period, so

$$H(a) + H(a+\pi) = \int_a^{a+2\pi} f(t)\,dt = 0.$$

Therefore

$$H(a+\pi) = -H(a).$$

If $H(a) = 0$ for some a, we already have the desired cut. Otherwise, $H(a)$ and $H(a+\pi)$ have opposite signs. By continuity, there is some $b \in [a, a+\pi]$ such that

$$H(b) = 0.$$

Thus

$$\int_b^{b+\pi} f(t)\,dt = 0.$$

Since the integral over the entire period is zero,

$$\int_{b+\pi}^{b+2\pi} f(t)\,dt = 0$$

as well.
Apply Lemma 16 to the first semicircle:

$$\int_b^{b+\pi} f^2 \le \int_b^{b+\pi} (f')^2.$$

Apply it again to the second:

$$\int_{b+\pi}^{b+2\pi} f^2 \le \int_{b+\pi}^{b+2\pi} (f')^2.$$

Adding gives

$$\int_b^{b+2\pi} f^2 \le \int_b^{b+2\pi} (f')^2.$$

By periodicity,

$$\int_0^{2\pi} f^2 \le \int_0^{2\pi} (f')^2.$$

□

Applying Lemma 17 separately to the two coordinates gives the vector form: if

$$\gamma : [0, 2\pi] \to \mathbb{R}^2$$

is periodic and

$$\int_0^{2\pi} \gamma(t)\,dt = 0,$$

then

$$\int_0^{2\pi} |\gamma(t)|^2\,dt \le \int_0^{2\pi} |\gamma'(t)|^2\,dt.$$

Theorem 18 (Isoperimetric Inequality)**.** *Let Γ be a simple closed piecewise C^1 curve of length L, enclosing area A. Then*

$$4\pi A \le L^2.$$

Equality holds exactly for circles.

Proof. Parameterize Γ counterclockwise at constant speed on $[0, 2\pi]$:

$$\gamma(t) = (x(t), y(t)), \qquad |\gamma'(t)| = \frac{L}{2\pi}$$

wherever γ' exists.
Translation does not change length or area, so translate the curve so that

$$\int_0^{2\pi} \gamma(t)\,dt = 0.$$

By Lemma 14,

$$2A = \int_0^{2\pi} \det(\gamma(t), \gamma'(t))\,dt.$$

Using

$$|\det(u, v)| \le |u||v|$$

and then Cauchy–Schwarz,

$$\begin{aligned} 2A &\le \int_0^{2\pi} |\gamma(t)|\,|\gamma'(t)|\,dt \\ &\le \left(\int_0^{2\pi} |\gamma(t)|^2\,dt\right)^{1/2} \left(\int_0^{2\pi} |\gamma'(t)|^2\,dt\right)^{1/2}. \end{aligned}$$

Periodic Wirtinger gives

$$\int_0^{2\pi} |\gamma(t)|^2\,dt \le \int_0^{2\pi} |\gamma'(t)|^2\,dt,$$

so

$$2A \le \int_0^{2\pi} |\gamma'(t)|^2\,dt.$$

Since the speed is constant,

$$\int_0^{2\pi} |\gamma'(t)|^2\,dt = 2\pi \left(\frac{L}{2\pi}\right)^2 = \frac{L^2}{2\pi}.$$

Therefore

$$2A \le \frac{L^2}{2\pi},$$

or

$$4\pi A \le L^2.$$

Suppose equality holds. Equality in periodic Wirtinger implies

$$\gamma(t) = p\cos t + q\sin t$$

after the translation, for some fixed vectors $p, q \in \mathbb{R}^2$.
Since γ has constant speed,

$$|\gamma'(t)|^2 = |p|^2\sin^2 t + |q|^2\cos^2 t - 2\langle p, q\rangle \sin t\cos t$$

is independent of t. Hence

$$|p| = |q|, \qquad \langle p, q\rangle = 0.$$

Thus p and q are perpendicular vectors of equal length, and

$$\gamma(t) = p\cos t + q\sin t$$

parameterizes a circle.
Conversely, a circle of radius r satisfies

$$L = 2\pi r, \qquad A = \pi r^2,$$

so equality holds. □

4 Linear Algebra

4.1 Determinant and Inverse Identities

Lemma 19 (Cauchy–Binet Formula)**.** *Let A be an $m \times n$ matrix and B an $n \times m$ matrix over a field, with $m \le n$. Then*

$$\det(AB) = \sum_{S \in \binom{[n]}{m}} \det(A_{[m],S})\det(B_{S,[m]}).$$

Proof. Set $P(x) = \det(A\,\mathrm{diag}(x_1, \ldots, x_n)B)$, which is homogeneous of degree m. For each i, row additions make column i of A supported in at most one row, without changing P. Then x_i occurs in at most one row of the product, so P is affine in x_i.

Thus every monomial of P uses exactly m distinct variables. Writing $\mathbf{1}_S$ for the indicator vector of S, we obtain

$$\det(AB) = P(\mathbf{1}) = \sum_{S \in \binom{[n]}{m}} P(\mathbf{1}_S) = \sum_{S \in \binom{[n]}{m}} \det(A_{[m],S}B_{S,[m]}) = \sum_{S \in \binom{[n]}{m}} \det(A_{[m],S})\det(B_{S,[m]}).$$

□

Lemma 20.

$$\det{}'(I) = \mathrm{Tr}$$

Proof. We have:

$$\det{}'(I)(T) = \nabla_T \det(I) = \lim_{\varepsilon \to 0} \frac{\det(I + \varepsilon T) - \det(I)}{\varepsilon}$$

$\det(I + \varepsilon T)$ is a polynomial in ε of order n. The constant term is 1, while the linear term is $\mathrm{Tr}(T)$. □

Lemma 21. *For an invertible matrix A, we have:*

$$\det{}'(A)(T) = \det(A)\,\mathrm{Tr}(A^{-1}T)$$

Proof. Consider the following function of X:

$$\det(X) = \det(AA^{-1}X) = \det(A)\det(A^{-1}X)$$

Using Lemma 20:

$$\det{}'(A)(T) = \det(A)\det{}'(I)(A^{-1}T) = \det(A)\,\mathrm{Tr}(A^{-1}T)$$

□

Lemma 22 (Jacobi's Formula)**.**

$$\frac{d}{dt}\det A = \mathrm{Tr}\left(\mathrm{adj}(A)\frac{dA}{dt}\right)$$

Proof. Let $A(t)$ be a differentiable matrix function. We introduce a variable x and consider $A_x(t) = A(t) - xI$, $A_x(t)$ is invertible for infinitely many values of x.

For all x where $A_x(t)$ is invertible, we can apply Lemma 21 with $T = \frac{dA_x}{dt} = \frac{dA}{dt}$:

$$\frac{d}{dt}\det(A_x) = \det(A_x)\,\mathrm{Tr}\left(A_x^{-1}\frac{dA}{dt}\right)$$

Using $\det(A_x)A_x^{-1} = \mathrm{adj}(A_x)$, this becomes:

$$\frac{d}{dt}\det(A(t) - xI) = \mathrm{Tr}\left(\mathrm{adj}(A(t) - xI)\frac{dA}{dt}\right)$$

Since they agree for infinitely many x, they agree as polynomials, substituting $x = 0$ we have:

$$\frac{d}{dt}\det A = \mathrm{Tr}\left(\mathrm{adj}(A)\frac{dA}{dt}\right)$$

□

Lemma 23. *Let A be an $n \times n$ matrix. For any subset of indices $S \subseteq \{1, 2, \ldots, n\}$, let A_S denote the $(n - |S|) \times (n - |S|)$ principal submatrix. Then:*

$$\frac{p_A^{(k)}(x)}{k!} = \sum_{|S|=k} p_{A_S}(x)$$

Proof. We evaluate $p_A(x + y)$ in two different ways.

We can consider the Taylor expansion of $p_A(x + y)$ around x:

$$p_A(x + y) = \sum_{k=0}^{n} \frac{p_A^{(k)}(x)}{k!} y^k$$

Second:

$$p_A(x + y) = \det\big((x + y)I - A\big) = \det\big(yI + (xI - A)\big)$$

The coefficient y here is the sum of the determinant of all $(n - k) \times (n - k)$ sized minors $\det(xI_{n-k} - A_S) = p_{A_S}(x)$.

Equating the coefficients of y^k yields:

$$\frac{p_A^{(k)}(x)}{k!} = \sum_{|S|=k} p_{A_S}(x)$$

□

4.1.1 Woodbury Identity

The Woodbury identity is most naturally viewed as an identity between the two Schur complements of the same block matrix.

Recall that for

$$M = \begin{pmatrix} A & B \\ C & D \end{pmatrix},$$

if A is invertible, the Schur complement of A is

$$M/A = D - CA^{-1}B.$$

The block factorization

$$M = \begin{pmatrix} I & 0 \\ CA^{-1} & I \end{pmatrix} \begin{pmatrix} A & 0 \\ 0 & M/A \end{pmatrix} \begin{pmatrix} I & A^{-1}B \\ 0 & I \end{pmatrix}$$

immediately gives

$$\det M = \det(A) \det(M/A).$$

If M/A is also invertible, inverting the factorization shows that the upper-left block of M^{-1} is

$$(M^{-1})_{11} = A^{-1} + A^{-1}B(M/A)^{-1}CA^{-1}.$$

Similarly, if D is invertible, then

$$M/D = A - BD^{-1}C,$$

and eliminating the D-block first shows that

$$(M^{-1})_{11} = (M/D)^{-1}.$$

These two descriptions of the same block give Woodbury immediately.

Theorem 24 (Woodbury Identity)**.** *Let A and C be invertible, and suppose*

$$C^{-1} + VA^{-1}U$$

is invertible. Then

$$(A + UCV)^{-1} = A^{-1} - A^{-1}U \left(C^{-1} + VA^{-1}U\right)^{-1} VA^{-1}.$$

Proof. Consider

$$M = \begin{pmatrix} A & U \\ -V & C^{-1} \end{pmatrix}.$$

Taking the Schur complement of A gives

$$M/A = C^{-1} + VA^{-1}U.$$

Hence the upper-left block of M^{-1} is

$$(M^{-1})_{11} = A^{-1} - A^{-1}U \left(C^{-1} + VA^{-1}U\right)^{-1} VA^{-1}.$$

On the other hand, taking the Schur complement of C^{-1} gives

$$M/C^{-1} = A - UC(-V) = A + UCV.$$

Therefore

$$(M^{-1})_{11} = (M/C^{-1})^{-1} = (A + UCV)^{-1}.$$

Equating the two expressions for the same block of M^{-1} gives the identity. □

Thus the identity can be remembered from the block matrix

$$\begin{pmatrix} A & U \\ -V & C^{-1} \end{pmatrix},$$

whose two Schur complements are

$$M/A = C^{-1} + VA^{-1}U, \qquad M/C^{-1} = A + UCV.$$

4.1.2 Matrix Determinant Lemma

The determinant analogue of Woodbury follows from exactly the same block matrix.

Theorem 25 (Matrix Determinant Lemma)**.** *Let*

$$A \in \mathbb{F}^{n\times n}, \qquad C \in \mathbb{F}^{k\times k}$$

be invertible, and let

$$U \in \mathbb{F}^{n\times k}, \qquad V \in \mathbb{F}^{k\times n}.$$

Then

$$\det(A + UCV) = \det(A)\det(C)\det\left(C^{-1} + VA^{-1}U\right).$$

Proof. Again take

$$M = \begin{pmatrix} A & U \\ -V & C^{-1} \end{pmatrix}.$$

Computing its determinant using the Schur complement of A,

$$\det M = \det(A)\det(M/A) = \det(A)\det\left(C^{-1} + VA^{-1}U\right).$$

Computing it instead using the Schur complement of C^{-1},

$$\det M = \det(C^{-1})\det(M/C^{-1}) = \det(C)^{-1}\det(A + UCV).$$

Equating the two expressions and multiplying by $\det(C)$ gives

$$\det(A + UCV) = \det(A)\det(C)\det\left(C^{-1} + VA^{-1}U\right).$$

□

For $C = I$, this becomes

$$\det(A + UV) = \det(A)\det(I + VA^{-1}U).$$

Thus Woodbury and the matrix determinant lemma are the inverse and determinant consequences of the same observation:

$$M/A = C^{-1} + VA^{-1}U, \qquad M/C^{-1} = A + UCV.$$

The large $n \times n$ update $A+UCV$ and the small $k \times k$ matrix $C^{-1}+VA^{-1}U$ are the two Schur complements of one block matrix.

The following is the positive-semidefiniteness version of the determinant lemma.

Lemma 26 (Schur Complement Positivity)**.** *For $B \succ 0$, $\begin{pmatrix} A & X \\ X^* & B \end{pmatrix} \succeq 0 \iff A \succeq XB^{-1}X^*$.*

Proof. Conjugating a block matrix by an invertible matrix preserves positive semidefiniteness. We calculate:

$$\begin{pmatrix} I & -XB^{-1} \\ 0 & I \end{pmatrix} \begin{pmatrix} A & X \\ X^* & B \end{pmatrix} \begin{pmatrix} I & 0 \\ -B^{-1}X^* & I \end{pmatrix} = \begin{pmatrix} A - XB^{-1}X^* & 0 \\ 0 & B \end{pmatrix}$$

Because $B \succ 0$, the block diagonal matrix on the right is positive semidefinite if and only if $A - XB^{-1}X^* \succeq 0$. □

4.2 Characteristic Polynomial

To compute the characteristic polynomial $p(x) = \det(xI - A)$ of an $n \times n$ matrix A, the algorithm first reduces A to an upper Hessenberg matrix H (where $H_{i,j} = 0$ for all $i > j + 1$) in $O(n^3)$ time. It then evaluates the characteristic polynomial using a dynamic programming approach.

Lemma 27 (Hessenberg Characteristic Polynomial Recurrence)**.** *Let H_k be the $k \times k$ leading principal submatrix of an upper Hessenberg matrix H, and let $p_k(x) = \det(xI_k - H_k)$ be its characteristic polynomial. With the base case $p_0(x) = 1$, the polynomials satisfy the recurrence:*

$$p_{k+1}(x) = (x - H_{k,k})p_k(x) - \sum_{i=0}^{k-1} \left(H_{i,k} \prod_{j=i}^{k-1} H_{j+1,j} \right) p_i(x)$$

Proof. We evaluate $p_{k+1}(x) = \det(xI_{k+1} - H_{k+1})$ by performing a Laplace expansion along its last column. The entries in the last column of $xI_{k+1} - H_{k+1}$ are $-H_{0,k}, -H_{1,k}, \ldots, -H_{k-1,k}$, with the diagonal entry being $(x - H_{k,k})$.

The cofactor for the diagonal entry $(x - H_{k,k})$ is the determinant of the top-left $k \times k$ block, which is $p_k(x)$.

For any other entry $-H_{i,k}$ (where $i < k$) in the last column, removing its row and column leaves a block upper-triangular matrix. The top-left block is $(xI_i - H_i)$, which has determinant $p_i(x)$. Because H is upper Hessenberg, the bottom-right block is upper triangular, with its main diagonal consisting entirely of the subdiagonal elements of H: $-H_{i+1,i}, -H_{i+2,i+1}, \ldots, -H_{k,k-1}$. The alternating signs from the cofactor expansion cancels with the negative signs of these subdiagonal elements. □

4.2.1 Determinant of Affine Matrix Polynomial

The algorithm computes the polynomial $P(x) = \det(A + xB)$ in $O(n^3)$ time.

The algorithm samples a random scalar $a \in \mathbb{F}$ and evaluates the matrix at this point, defining $A' = A + aB$. Assuming A' is invertible, we can factor it out of the original polynomial:

$$\begin{aligned} P(x) &= \det(A + aB + (x-a)B) \\ &= \det(A' + (x-a)B) \\ &= \det(A') \det(I + (x-a)A'^{-1}B) \end{aligned}$$

Let $M = -A'^{-1}B$. The polynomial becomes $P(x) = \det(A') \det(I - (x-a)M)$.

Notice that $\det(I - tM) = t^n \det(t^{-1}I - M) = t^n p_M(t^{-1})$, the reversed characteristic polynomial of M. Let $R(t)$ denote this reversed polynomial. The algorithm computes $p_M(t)$ using the Hessenberg DP, reverses its coefficients to obtain $R(t)$, multiplies the polynomial by the scalar $\det(A')$, and performs a Taylor shift $x \mapsto x - a$ to recover $P(x) = \det(A')R(x-a)$.

Lemma 28 (Failure Probability of the Affine Determinant)**.** *If $P(x) = \det(A + xB)$ is not identically the zero polynomial, the probability that the randomized reduction algorithm incorrectly fails (by returning 0) is bounded by $\frac{n}{|\mathbb{F}|}$.*

Proof. The algorithm only fails if the shifted matrix $A' = A + aB$ is not invertible, if and only if $P(a) = 0$.

The algorithm samples a uniformly and out of all possible random choices for a, at most n of them can cause A' to be singular. If $P(x)$ is identically the zero polynomial, the algorithm will correctly find $\det(A') = 0$ for any a. □

4.3 Frobenius Form

4.3.1 Algorithm Description

To compute the Frobenius form of a matrix $A \in \mathbb{F}^{n \times n}$ in $O(n^3)$ time, the algorithm employs a randomized Krylov subspace approach. The Frobenius form decomposes the vector space V into a direct sum of A-cyclic subspaces $V = \bigoplus_{i=1}^{m} W_i$. This yields a block-diagonal matrix where each block is the companion matrix of an invariant factor $P_i(z)$ of A. By the structure theorem of finitely generated modules over a PID, these invariant factors divide each other: $P_m(z) \mid P_{m-1}(z) \mid \cdots \mid P_1(z)$.

1. **State:** Maintain a set of generator vectors $\{w_1, \ldots, w_k\}$ for the resolved cyclic subspaces $W_1, \ldots, W_k$, and a row-reduced basis $\mathcal{B}$ representing their combined span $W = \bigoplus_{i=1}^{k} W_i$.

2. **Random Sampling:** Draw a uniformly random vector $x \in \mathbb{F}^n$ to act as the candidate generator for the next cyclic block, W_{k+1}.

3. **Krylov Sequence:** Generate the Krylov sequence $x, Ax, A^2x, \ldots$ and incrementally reduce each new vector against the existing basis $\mathcal{B}$ using Gaussian elimination. This process halts when a linear dependence is found, yielding the minimal annihilating polynomial $C(z)$ of x modulo the previously resolved space W.

4. **Orthogonalization:** The linear dependence indicates that $C(A)x \in W$. To ensure the newly found cyclic subspace forms a direct summand, x must be updated to $\tilde{x} = x - u$ (for some $u \in W$) such that $C(A)\tilde{x} = 0$. The algorithm extracts the necessary shift by performing a polynomial division over the concatenated dependency arrays, and backtracks to apply it to x. It then recomputes the Krylov sequence.

5. **Iteration:** The vectors $\tilde{x}, A\tilde{x}, \ldots, A^{\deg(C)-1}\tilde{x}$ are added to $\mathcal{B}$. This repeats until $\dim(W) = n$.

4.3.2 Correctness

Theorem 29 (Divisibility of the Residuals)**.** *Let $W = \bigoplus_{i=1}^{k} W_i$ be the resolved space, where each $W_i = \langle w_i \rangle_A$ is a cyclic submodule with minimal polynomial $P_i(z)$. Let x be the random candidate vector, and $C(z)$ be its minimal polynomial modulo W. The Krylov reduction yields a dependency of the form:*

$$C(A)x = \sum_{i=1}^{k} R^{(i)}(A)w_i$$

Then the polynomial $C(z)$ divides $R^{(i)}(z)$ in $\mathbb{F}[z]$ for all $1 \leq i \leq k$.

Proof. As shown in the next section, with good probability, the minimal polynomial of the newly sampled block must divide the minimal polynomials of all previously extracted blocks: $C(z) \mid P_i(z)$ for all $i \leq k$.

Because the true vector space decomposes into a direct sum of cyclic submodules, we can uniquely decompose our random candidate as $x = \tilde{x} + \sum_{i=1}^{k} v_i$, where $\tilde{x}$ belongs to the target complementary subspace (meaning it is exactly annihilated by $C(A)$) and $v_i \in W_i$.

Applying $C(A)$ to this decomposition yields $C(A)x = C(A)\tilde{x} + \sum_{i=1}^{k} C(A)v_i = \sum_{i=1}^{k} C(A)v_i$. Since each $v_i \in \langle w_i \rangle_A$, there exist polynomials $S^{(i)}(z)$ such that $v_i = S^{(i)}(A)w_i$. Thus:

$$C(A)x = \sum_{i=1}^{k} C(A)S^{(i)}(A)w_i$$

Equating this with the relationship obtained from our Krylov reduction, we have:

$$\sum_{i=1}^{k} \big(C(A)S^{(i)}(A) - R^{(i)}(A)\big)w_i = 0$$

Because W is a direct sum, the components must independently evaluate to zero: $\big(C(A)S^{(i)}(A) - R^{(i)}(A)\big)w_i = 0$. This implies that the minimal polynomial of w_i, which is $P_i(z)$, must divide $C(z)S^{(i)}(z) - R^{(i)}(z)$.

Since we previously established that $C(z) \mid P_i(z)$, it follows by transitivity that $C(z)$ must divide $C(z)S^{(i)}(z) - R^{(i)}(z)$. Consequently, $C(z)$ must divide the residual polynomial $R^{(i)}(z)$ in $\mathbb{F}[z]$. □

4.3.3 Failure Probability

Theorem 30. *The probability that the randomized Krylov algorithm fails to extract the Frobenius form of $A \in \mathbb{F}^{n \times n}$ is bounded by $\frac{n}{|\mathbb{F}|}$.*

Proof. The algorithm fails if and only if, at some iteration, the randomly drawn vector x fails to generate the correct cyclic subspace in the quotient V/W. Let $P(z)$ be the true invariant factor of the remaining space. The vector x fails if its minimal polynomial modulo W is a strict divisor of $P(z)$.

We view V/W as a finitely generated torsion module over the principal ideal domain $\mathbb{F}[z]$, where z acts as A. The module decomposes into primary cyclic submodules:

$$V/W \cong \bigoplus_{j=1}^{\ell} \mathbb{F}[z]/\langle Q_j(z)^{e_j} \rangle$$

where $Q_j(z)$ are the distinct monic irreducible factors of $P(z)$, and e_j are their multiplicities.

A random vector $x \in V/W$ fails to have $P(z)$ as its minimal polynomial if and only if it is annihilated by $P(z)/Q_j(z)$ for at least one irreducible factor Q_j. This occurs exactly when the projection of x onto the primary component $\mathbb{F}[z]/\langle Q_j(z)^{e_j} \rangle$ is a multiple of $Q_j(z)$.

Because x is drawn uniformly at random, its projection onto any component is uniformly distributed. The submodule consisting of multiples of $Q_j(z)$ has a vector space codimension equal to $\deg(Q_j)$. Therefore, the probability that x falls into this "bad" submodule is $|\mathbb{F}|^{-\deg(Q_j)}$.

Applying the union bound over all distinct irreducible factors $Q_j \mid P(z)$, the probability that x fails in a given step is bounded by:

$$\mathbb{P}(\text{Failure at step}) \leq \sum_{Q_j|P} \frac{1}{|\mathbb{F}|^{\deg(Q_j)}}$$

Because $\sum \deg(P_i) = n$, the worst-case upper bound occurs when the characteristic polynomial splits entirely into n distinct linear factors. In this scenario:

$$\mathbb{P}(\text{Total Failure}) \leq \frac{n}{|\mathbb{F}|}$$

□

4.3.4 Polynomial Matrix Composition and Powering

With the Frobenius form $A = T^{-1}FT$ explicitly constructed, we can evaluate any polynomial function $f(A)$, such as matrix powers A^k.

Module Isomorphism on Cyclic Blocks Let $W_i = \langle w_i \rangle_A$ be a cyclic subspace of dimension d_i with minimal polynomial $P_i(z)$. The subspace is spanned by the Krylov basis $\mathcal{B}_i = \{w_i, Aw_i, \ldots, A^{d_i-1}w_i\}$.

There is a natural $\mathbb{F}[z]$-module isomorphism between W_i and the polynomial quotient ring $\mathbb{F}[z]/\langle P_i(z) \rangle$, given by the mapping:

$$A^j w_i \longleftrightarrow z^j \pmod{P_i(z)}$$

Under this isomorphism, applying the matrix function $f(A)$ to a basis vector $A^j w_i$ corresponds simply to polynomial multiplication in the quotient ring:

$$f(A)\big(A^j w_i\big) = A^j f(A) w_i \longleftrightarrow z^j f(z) \pmod{P_i(z)}$$

Constructing the Blocks To construct the matrix block $S_i = f(C_i)$ representing the action of $f(A)$ on the subspace W_i, we need to find the coordinates of $f(A)(A^j w_i)$ in the basis $\mathcal{B}_i$. By the isomorphism above, these coordinates are the coefficients of the polynomial:

$$R_{i,j}(z) = z^j f(z) \pmod{P_i(z)}$$

1. First, computes the base polynomial $R_{i,0}(z) = f(z) \pmod{P_i(z)}$.
2. This polynomial forms the 0-th row (or column depending on preference) of the block S_i.
3. For each subsequent basis vector $j \in \{1, \ldots, d_i - 1\}$, the algorithm calculates the next polynomial by shifting the previous one and reducing modulo $P_i(z)$:

   $$R_{i,j}(z) = z \cdot R_{i,j-1}(z) \pmod{P_i(z)}$$

   This populates the $d_i \times d_i$ block S_i in $O(d_i^2)$ time.

Once all blocks S_i are constructed, they are assembled into the block-diagonal matrix $S = \text{diag}(S_1, \ldots, S_m)$, and the final matrix is recovered via the similarity transform $f(A) = T^{-1}ST$.

Theorem 31 (Complexity of Matrix Exponentiation)**.** *Using the Frobenius form, the matrix power A^k can be computed in $O(n^3 + n \log n \log k)$ time.*

Proof. We compute the Frobenius form $A = T^{-1}FT$ in $O(n^3)$ time. For each invariant factor $P_i(z)$, our target function is $f(z) = z^k$. We evaluate the base polynomial $R_{i,0}(z) = z^k \pmod{P_i(z)}$ using binary exponentiation in the polynomial ring $\mathbb{F}[z]/\langle P_i(z) \rangle$ or using specialized algorithms. Because $\deg(P_i) = d_i$, each polynomial multiplication takes $O(d_i \log d_i)$ with NTT. The total time to compute $z^k \pmod{P_i(z)}$ is $O(d_i^2 \log k)$.

Summing this cost over all blocks yields $\sum_i O(d_i \log d_i \log k) \leq O(n \log n \log k)$. Constructing the blocks via shifts takes $\sum_i O(d_i^2) = O(n^2)$ time. Finally, recovering the matrix via $T^{-1}ST$ requires two $O(n^3)$ matrix multiplications. □

4.4 Dynamic Frobenius Form and Exact Distances

A rank-one update can change every vector in a Frobenius basis. The corresponding resolvent numerators behave much better: every numerator is acted on by the same linear map. We therefore keep these maps lazily for k updates and apply their composition to all numerators only at the end of the phase.

Write $\mathrm{MM}(a, b, c)$ for the time to multiply an $a \times b$ matrix by a $b \times c$ matrix. Degree-$O(n)$ polynomial arithmetic over $\mathbb{F}$ costs $\widetilde{O}(n)$.

Theorem 32. *Let $A \in \mathbb{F}^{n\times n}$ undergo single-entry updates, and fix $v \in \mathbb{F}^n$ that remains cyclic for both A and $A^\top$. For every $1 \le k \le n$, after $\widetilde{O}(n^\omega)$ preprocessing one can maintain A in $O(n^2)$ space with worst-case update time*

$$\widetilde{O}\Big(nk + \frac{n}{k^2}\,\mathrm{MM}(n, k, k)\Big).$$

A query (s, t) returns $\mathrm{adj}(zI - A)_{st}$ in $\widetilde{O}(nk)$ time.

Resolvent coordinates. Set

$$p(z) = \det(zI - A), \qquad J(z) = \mathrm{adj}(zI - A),$$

and define

$$R_s = e_s^\top J v, \qquad \bar{R}_t = v^\top J e_t, \qquad N = v^\top J v.$$

Thus R is the numerator vector of $(zI - A)^{-1}v$, while $\bar{R}$ is the same object for $A^\top$.

Since $J/p = (zI - A)^{-1}$, expansion at infinity gives

$$\frac{J}{p} = \sum_{\ell\ge 0} A^\ell z^{-\ell-1}. \tag{1}$$

This identifies the numerator arrays with the two Krylov bases. If

$$K = [v, Av, \ldots, A^{n-1}v],$$

then row s of K is given by the first n Laurent coefficients of R_s/p. Since v is cyclic, K is nonsingular, and hence $R_1, \ldots, R_n$ form a basis of the degree-$< n$ polynomials. Applying Equation (1) to $A^\top$ shows in the same way that $\bar{R}_1, \ldots, \bar{R}_n$ form a basis.

For each t, let f_t be the unique degree-$< n$ polynomial with $e_t = f_t(A)v$. From the adjugate identity $J(zI - A) = pI$, every polynomial f satisfies

$$Jf(A) \equiv f(z)J \pmod p.$$

Taking $f = f_t$, multiplying by v, and then taking row s or multiplying by $v^\top$ gives

$$J_{st} = R_s f_t \bmod p, \qquad \bar{R}_t = N f_t \bmod p. \tag{2}$$

Both $\{f_t\}$ and $\{\bar{R}_t\}$ are bases, so multiplication by N is invertible modulo p. Therefore

$$J_{st} = R_s \bar{R}_t N^{-1} \bmod p. \tag{3}$$

This is the only identity needed to answer queries.

Preprocessing. Construct

$$K = [v, Av, \dots, A^{n-1}v], \qquad \bar{K} = [v, A^\top v, \dots, (A^\top)^{n-1}v]$$

by doubling. Solving $K\alpha = A^n v$ gives

$$p(z) = z^n - \sum_{\ell<n} \alpha_\ell z^\ell,$$

because the minimal polynomial of a cyclic vector has degree n.

The same Krylov data recover the numerator arrays. If

$$H_s(z) = \sum_{\ell<n} (A^\ell v)_s z^{-\ell-1},$$

then Equation (1) gives $R_s/p - H_s = O(z^{-n-1})$. Multiplying by p and taking the nonnegative powers of z yields

$$R_s = [pH_s]_{\geq 0}.$$

Applying this reconstruction to $\bar{K}$ gives $\bar{R}$, and finally $N = \sum_s v_s R_s$. The total preprocessing cost is $\widetilde{O}(n^\omega)$.

Transport through a block of updates. All dynamic behavior is captured by one identity. Let A^0 be a base state and suppose

$$A = A^0 + \Delta, \qquad \Delta = \sum_{a=1}^{r} \delta_a e_{i_a} e_{j_a}^\top.$$

Use superscript 0 for the base quantities and no superscript for the target quantities. For each update define

$$u_a = \delta_a \bar{R}^0_{i_a} (N^0)^{-1} \bmod p_0, \qquad P_a = R_{j_a},$$

where P_a is evaluated at the target state.

Lemma 33 (Transport)**.** *For every degree-$< n$ polynomial F,*

$$\mathcal{T}(F) = \sigma F - \sum_{a=1}^{r} P_a \operatorname{quo}(u_a F, p_0), \qquad \sigma = \frac{p + \sum_a P_a u_a}{p_0}. \tag{4}$$

The quotient defining σ is exact, and

$$R_s = \mathcal{T}(R^0_s) \qquad \textit{for every } s.$$

The same statement applied to $A^\top$ transports the barred numerators.

Proof. Since

$$zI - A = (zI - A^0) - \Delta,$$

the elementary inverse identity $(M - \Delta)^{-1} = M^{-1} + M^{-1}\Delta(M - \Delta)^{-1}$ with $M = zI - A^0$ gives

$$(zI - A)^{-1} = (zI - A^0)^{-1} + (zI - A^0)^{-1}\Delta(zI - A)^{-1}. \tag{5}$$

Substituting

$$(zI - A)^{-1} = \frac{J}{p}, \qquad (zI - A^0)^{-1} = \frac{J^0}{p_0}$$

into Equation (5) and multiplying by pp_0 gives

$$p_0 J = pJ^0 + J^0 \Delta J. \tag{6}$$

Multiply Equation (6) by v and take row s. Using the definition of Δ,

$$p_0 R_s = p R^0_s + \sum_a \delta_a J^0_{s i_a} P_a. \tag{7}$$

By Equation (3) in the base state,

$$\delta_a J^0_{s i_a} = \delta_a R^0_s \bar{R}^0_{i_a} (N^0)^{-1} \bmod p_0 = u_a R^0_s \bmod p_0.$$

For $F = R^0_s$, substitute

$$u_a F = p_0 \operatorname{quo}(u_a F, p_0) + (u_a F \bmod p_0)$$

into Equation (7). This gives

$$p_0 R_s = \left(p + \sum_a P_a u_a \right) F - p_0 \sum_a P_a \operatorname{quo}(u_a F, p_0). \tag{8}$$

Because the R^0_s's form a basis, some linear combination of them is 1. Applying that combination to Equation (8) shows

$$p_0 \mid p + \sum_a P_a u_a.$$

Thus σ in Equation (4) is a polynomial. Dividing Equation (8) by p_0 proves Equation (4) on the basis $\{R^0_s\}$, and hence on every degree-$< n$ polynomial by linearity. □

One update. Now specialize Lemma 33 to

$$A' = A + \delta e_i e_j^\top.$$

Let

$$R = R_j, \qquad \bar{R} = \bar{R}_i, \qquad \eta = \delta N^{-1} \bmod p, \qquad \rho = \eta R \bar{R}.$$

By Equation (3),

$$\rho \bmod p = \delta J_{ji}.$$

The matrix determinant identity

$$\det(M - x y^\top) = \det M - y^\top \operatorname{adj}(M) x$$

applied with $M = zI - A$, $x = \delta e_i$, and $y = e_j$ therefore gives

$$q := \det(zI - A') = p - \delta J_{ji} = p - (\rho \bmod p). \tag{9}$$

Consequently

$$\sigma := \frac{q + \rho}{p}$$

is a polynomial.

We also need the two numerators incident to the update. Specializing Equation (6) to the base state A, target state A', and $\Delta = \delta e_i e_j^\top$ gives

$$p J' = q J + \delta J e_i e_j^\top J'. \tag{10}$$

Multiply Equation (10) by v and take row j:

$$p R'_j = q R_j + \delta J_{ji} R'_j = q R_j + (p - q) R'_j,$$

where the last equality uses Equation (9). Hence $R'_j = R_j = R$. Applying the same argument to the transpose gives $\bar{R}'_i = \bar{R}_i = \bar{R}$.

The rank-one instance of Equation (4) uses

$$u = \delta \bar{R} N^{-1} \bmod p.$$

It is slightly cleaner to use the unreduced representative $\eta\bar{R}$. Indeed, replacing u by $u + ph$ changes σ by Rh, while

$$\operatorname{quo}((u + ph)F, p) = \operatorname{quo}(uF, p) + hF;$$

these two changes cancel in Equation (4). We may therefore write the right update as

$$E(F) = \sigma F - R \operatorname{quo}(\eta \bar{R} F, p). \tag{11}$$

Applying Equation (11) to $A^\top$, which exchanges R with $\bar{R}$, gives

$$\bar{E}(F) = \sigma F - \bar{R} \operatorname{quo}(\eta R F, p). \tag{12}$$

Each map costs $\widetilde{O}(n)$ to evaluate.

Phases. After $r \le k$ updates we store only

$$E_1, \dots, E_r, \qquad \bar{E}_1, \dots, \bar{E}_r,$$

rather than updating all n numerator polynomials. A current right numerator is obtained by applying $E_r \cdots E_1$ to its phase-start value, and a barred numerator by applying $\bar{E}_r \cdots \bar{E}_1$. Since $N = \sum_s v_s R_s$, linearity also gives

$$N = E_r \cdots E_1(N^0), \tag{13}$$

so N requires no separate state.

A query evaluates $R_s, \bar{R}_t, N$ through the corresponding chains and returns Equation (3). To process $A \leftarrow A + \delta e_i e_j^\top$, evaluate $R_j, \bar{R}_i, N$, compute q, σ from Equation (9), and append the maps Equations (11) and (12). Each chain has length at most k, so both queries and ordinary updates cost

$$\widetilde{O}(nk). \tag{14}$$

Closing a phase. At the end of the phase we materialize all numerator arrays. Applying the k stored maps separately to all n numerators would cost $\widetilde{O}(n^2 k)$, so instead we compile the phase into the single block operator $\mathcal{T}$ from Lemma 33.

For update a, its base coefficient

$$u_a = \delta_a \bar{R}^0_{i_a} (N^0)^{-1} \bmod p_0$$

depends only on the phase-start state and can be stored when that update arrives. Its target coefficient is

$$P_a = R^{\mathrm{end}}_{j_a}.$$

We obtain each P_a by applying the frozen right chain to $R^0_{j_a}$; all k values therefore cost $\widetilde{O}(nk^2)$. The barred coefficients are obtained from the barred chain in exactly the same way.

By Lemma 33, rebuilding the right array has now reduced to evaluating

$$\mathcal{T}(F) = \sigma F - \sum_{a=1}^{k} P_a \operatorname{quo}(u_a F, p_0) \tag{15}$$

simultaneously on the n phase-start numerators.

Lemma 34 (Batched transport)**.** *The n values in Equation (15) can be computed in*

$$\widetilde{O}\Big(\frac{n}{k}\,\mathrm{MM}(n,k,k)\Big)$$

operations and $O(n^2)$ space.

Proof. We first convert the quotient in Equation (15) into a truncated convolution. For a degree-$< n$ polynomial f, define its reversal by

$$f^*(x) = x^{n-1} f(x^{-1}), \qquad p_0^*(x) = x^n p_0(x^{-1}).$$

If

$$uF = p_0 Q + \rho, \qquad \deg \rho < n,$$

then reversing this division identity and discarding the reversed remainder gives

$$x^{n-2} Q(x^{-1}) \equiv u^*(x) F^*(x) (p_0^*(x))^{-1} \pmod{x^{n-1}}. \tag{16}$$

Because p_0 is monic, $p_0^*(0) = 1$, so the inverse in Equation (16) exists modulo x^{n-1}.

Consequently, for each u_a there is a polynomial h_a, computable in $\widetilde{O}(n)$, such that

$$\mathrm{coeff}(\mathrm{quo}(u_a F, p_0)) = L_{h_a}^\top \,\mathrm{coeff}(F),$$

where L_h is lower-triangular Toeplitz multiplication by h. Thus the coefficient matrix of $\mathcal{T}$ is

$$T = L_\sigma - \sum_{a=1}^{k} L_{P_a} L_{h_a}^\top. \tag{17}$$

The L_σ term can be applied to all n inputs in $\widetilde{O}(n^2)$ time. It remains to multiply $\sum_a L_{P_a} L_{h_a}^\top$ by the $n \times n$ matrix B of input coefficient vectors.

To see the polynomial product represented by this matrix multiplication, collect $P_1, \ldots, P_k$ and $h_1, \ldots, h_k$ into polynomial vectors $P(x), h(x)$, and the n columns of B into a polynomial vector $B(x)$. The definitions of lower- and upper-triangular Toeplitz multiplication show that, up to the coefficient reversals already used in Equation (16) and known monomial shifts, the desired coefficients are precisely those of

$$P(x)^\top \big(h(x) B(x)^\top \bmod x^n\big). \tag{18}$$

Without the reduction modulo x^n, we could reassociate the three factors immediately. We therefore only need to organize the coefficient pairs retained by that truncation:

$$\mathcal{D}_n = \{(i,j) : 0 \le i, j < n,\ i + j < n\}.$$

Assume, by zero-padding, that n is a power of two. We partition $\mathcal{D}_n$ into dyadic squares. For $(i,j) \in \mathcal{D}_n$, the inequality $i + j < n$ is equivalent to

$$i \le n - 1 - j.$$

If equality holds, assign the pair to scale 1. Otherwise let I be the smallest aligned dyadic interval containing the two positions i and $n - 1 - j$. By minimality, i lies in the left half of I and $n - 1 - j$ in the right half. Their two half-intervals define a unique square of coefficient pairs. Hence every pair in $\mathcal{D}_n$ lies in exactly one such square.

Fix squares of side length ℓ, and put $s = n/\ell$. There are $s/2$ squares at this scale. Splitting every polynomial into blocks of ℓ consecutive coefficients, collect the relevant blocks into

$$\mathcal{P} \in \mathbb{F}[x]^{s \times k}, \qquad \mathcal{H} \in \mathbb{F}[x]^{k \times (s/2)}, \qquad \mathcal{B} \in \mathbb{F}[x]^{(s/2) \times n}.$$

Every entry has degree $< \ell$. The definition of the dyadic squares ensures that their entire contribution to Equation (18) is obtained from

$$\mathcal{P}\mathcal{H}\mathcal{B}, \tag{19}$$

followed only by known monomial shifts: all squares at this scale have the same inner shift $x^{n-2\ell}$, while the row of $\mathcal{P}$ determines the outer block shift.

It remains to evaluate the degree-$O(\ell)$ product in Equation (19). Evaluate its factors at $O(\ell)$ field points.

If $s \geq k$, partition every s-dimension into blocks of size k. At each evaluation point both numerical matrix products in Equation (19) cost

$$O\Big(\frac{s}{k}\,\mathrm{MM}(n,k,k)\Big).$$

Since there are $O(\ell)$ evaluation points and $\ell s = n$, the whole scale costs

$$O\Big(\frac{n}{k}\,\mathrm{MM}(n,k,k)\Big). \tag{20}$$

If $s < k$, a single evaluation is too narrow to use a $k \times k$ product efficiently, so batch k/s evaluation points. Vertically stack their $s \times k$ matrices $\mathcal{P}$ and horizontally concatenate their $k \times (s/2)$ matrices $\mathcal{H}$. One $k \times k$-scale multiplication then contains, as diagonal blocks, the $\mathcal{P}\mathcal{H}$ products for all evaluations in the batch. Multiplying these diagonal blocks by the correspondingly stacked $\mathcal{B}$'s costs another $O(\mathrm{MM}(n,k,k))$. Thus one batch has this cost, and the $O(\ell)$ evaluation points form

$$O\left(\frac{\ell}{k/s}\right) = O(n/k)$$

batches. The whole scale therefore again satisfies Equation (20).

There are $O(\log n)$ scales. Multipoint evaluation, interpolation, coefficient reversals, and monomial shifts contribute only $\widetilde{O}(n^2)$, while processing the scales successively uses $O(n^2)$ space. □

Applying Lemma 34 to both numerator arrays and including the $\widetilde{O}(nk^2)$ compilation work, closing a phase costs

$$\widetilde{O}\Big(nk^2 + \frac{n}{k}\,\mathrm{MM}(n,k,k)\Big). \tag{21}$$

Worst-case updates. When a phase closes, freeze its two chains and spread the work in Equation (21) over the next k updates. The next phase starts immediately. Until the previous endpoint arrays are materialized, a numerator is evaluated by first traversing the frozen old chain and then the current new chain, hence at most $2k$ maps.

The rebuild finishes by the next phase boundary, when its materialized endpoint arrays replace the old chains. Dividing Equation (21) by k gives worst-case update time

$$\widetilde{O}\Big(nk + \frac{n}{k^2}\,\mathrm{MM}(n,k,k)\Big),$$

while queries cost $\widetilde{O}(nk)$. A constant number of dense arrays plus $O(nk)$ phase data uses $O(n^2)$ space. This proves Theorem 32.

Exact dynamic distances. For a directed unweighted graph, fix distinct $\lambda_1, \ldots, \lambda_n \in \mathbb{F}$, set $v = \mathbf{1}$, and choose an independent uniform $\xi_{ij} \in \mathbb{F}$ for every possible directed edge $i \to j$. For the current graph let

$$A_{ii} = \lambda_i, \qquad A_{ij} = \begin{cases} \xi_{ij}, & (i,j) \in E, \\ 0, & (i,j) \notin E, \end{cases} \quad i \neq j. \tag{22}$$

An edge insertion or deletion changes exactly one entry of A.

For any fixed graph state, consider the two Krylov determinants

$$\det[\mathbf{1}, A\mathbf{1}, \ldots, A^{n-1}\mathbf{1}], \qquad \det[\mathbf{1}, A^\top\mathbf{1}, \ldots, (A^\top)^{n-1}\mathbf{1}].$$

They are nonzero polynomials in the ξ_{ij}'s: setting every $\xi_{ij} = 0$ makes $A = \mathrm{diag}(\lambda_1, \ldots, \lambda_n)$, at which point both Krylov matrices are Vandermonde matrices. Each determinant has degree $O(n^2)$, so Schwartz–Zippel gives total cyclicity failure probability $O(n^2/|\mathbb{F}|)$ for a fixed state.

Now fix s, t, and let $d = \mathrm{dist}(s, t) < \infty$. For $\ell < d$, $(A^\ell)_{st} = 0$. At $\ell = d$, a contributing walk cannot use a diagonal entry: deleting such a self-loop would produce an s-to-t walk using fewer than d graph edges. Therefore

$$(A^d)_{st} = \sum_{\substack{P: s \rightsquigarrow t \\ |P| = d}} \prod_{e \in P} \xi_e. \tag{23}$$

Every shortest path is simple, and distinct directed simple paths have distinct edge sets. Thus the summands in Equation (23) are distinct monomials, so the polynomial is nonzero and vanishes with probability at most $d/|\mathbb{F}|$.

Finally, taking the (s, t) entry of Equation (1) gives

$$\frac{J_{st}}{p} = \sum_{\ell \geq 0} (A^\ell)_{st} z^{-\ell-1}. \tag{24}$$

Whenever $(A^d)_{st} \neq 0$, the first nonzero term of Equation (24) has exponent $-d-1$. Since p is monic of degree n, the leading exponent of J_{st}/p is also $\deg J_{st} - n$. Equating the two gives

$$\mathrm{dist}(s, t) = n - 1 - \deg J_{st}. \tag{25}$$

If t is unreachable from s, every coefficient in Equation (24) vanishes, so $J_{st} = 0$.

For L graph states and Q distance queries, a union bound gives failure probability

$$O\left(\frac{Ln^2 + Qn}{|\mathbb{F}|}\right).$$

The same bound holds against adaptive updates and queries: after fixing the adversary's coins, its transcript against an exact oracle determines the state and query sequence up to the first failure.

4.5 Black Box Linear Algebra

In the following algorithms, we consider a matrix $A \in \mathbb{F}^{n \times n}$ given strictly as a black-box function $f(v) = Av$.

4.5.1 Minimal Polynomial

Algorithm Description To compute the minimal polynomial of A, the algorithm generates two random vectors $u, v \in \mathbb{F}^n$. It then computes the scalar sequence $s_k = u^T A^k v$ for $k = 0, \ldots, 2n$. Finally, it applies an algorithm to find the minimal linear recurrence of the sequence s_k, which corresponds to the minimal polynomial of A.

Proof of Correctness Let $A \in \mathbb{F}^{n \times n}$. We define three polynomials:

1. $m_A(x)$: The minimal polynomial of the matrix A.
2. $m_{A,v}(x)$: The minimal polynomial of A with respect to the vector v, which is the monic polynomial of least degree such that $m_{A,v}(A)v = 0$.
3. $m_{u,v}(x)$: The minimal polynomial of the scalar sequence $s_k = u^T A^k v$, which is the monic polynomial of least degree such that $\sum_{i=0}^{d} c_i s_{k+i} = 0$ for all k.

Divisibility:
Because $m_A(A) = 0$, it follows that $m_A(A)v = 0$, meaning $m_{A,v}(x)$ must divide $m_A(x)$. Similarly, since $m_{A,v}(A)v = 0$, left-multiplying by $u^T A^k$ gives $u^T A^k m_{A,v}(A)v = 0$. This means the sequence s_k satisfies the recurrence defined by $m_{A,v}(x)$, so $m_{u,v}(x)$ must divide $m_{A,v}(x)$. This establishes the chain of divisibility:

$$m_{u,v}(x) \mid m_{A,v}(x) \mid m_A(x)$$

To prove $m_{u,v}(x) = m_A(x)$ with high probability, we bound the probability of a strict division occurring at either step.

Part 1: The Probability that $m_{A,v}(x) = m_A(x)$
Let $m_A(x)$ be factored into its distinct irreducible monic polynomials over $\mathbb{F}$:

$$m_A(x) = p_1(x)^{e_1} p_2(x)^{e_2} \cdots p_r(x)^{e_r}$$

where $r \leq n$. For $m_{A,v}(x)$ to be a proper divisor of $m_A(x)$, it must divide the polynomial $q_i(x) = m_A(x)/p_i(x)$ for at least one index $i \in \{1, \ldots, r\}$. This happens if and only if $q_i(A)v = 0$, which means v resides in the nullspace of the matrix $q_i(A)$.

Let $W_i = \ker(q_i(A))$. Because $m_A(x)$ is strictly the minimal polynomial of A, the matrix $q_i(A)$ cannot be the zero matrix. Therefore, W_i is a proper subspace of $\mathbb{F}^n$, meaning $\dim(W_i) \leq n - 1$. If v is chosen uniformly at random from $\mathbb{F}^n$, the probability that v falls into a specific proper subspace W_i is at most $1/|\mathbb{F}|$. By the union bound over all r irreducible factors, the probability that v falls into any of these kernels is:

$$P(m_{A,v}(x) \neq m_A(x)) = P\left(v \in \bigcup_{i=1}^{r} W_i\right) \leq \frac{r}{|\mathbb{F}|} \leq \frac{n}{|\mathbb{F}|}$$

Part 2: The Probability that $m_{u,v}(x) = m_{A,v}(x)$
Assume we have chosen a "good" vector v such that $m_{A,v}(x) = m_A(x)$. We now pick $u \in \mathbb{F}^n$ uniformly at random. By the same logic, for $m_{u,v}(x)$ to be a proper divisor of $m_A(x)$, it must divide $q_i(x) = m_A(x)/p_i(x)$ for some index i. This happens if and only if the sequence generated by $q_i(x)$ evaluates to zero for all k:

$$u^T A^k q_i(A)v = 0 \quad \text{for all } k \geq 0$$

Let $w_i = q_i(A)v$. Because we assumed $m_{A,v}(x) = m_A(x)$, we know that $w_i \neq 0$. The condition $u^T A^k w_i = 0$ for all k means that the vector u must be orthogonal to the Krylov subspace $\mathcal{K} = \text{span}\{w_i, Aw_i, A^2 w_i, \ldots\}$.

Since $w_i \neq 0$, the subspace $\mathcal{K}$ has a dimension of at least 1. Therefore, its orthogonal complement $\mathcal{K}^\perp$ is a proper subspace of $\mathbb{F}^n$ with dimension at most $n - 1$. The probability that a uniformly random u falls into $\mathcal{K}^\perp$ for a specific index i is at most $1/|\mathbb{F}|$. Applying the union bound again across the r factors:

$$P(m_{u,v}(x) \neq m_A(x) \mid v \text{ is good}) \leq \frac{r}{|\mathbb{F}|} \leq \frac{n}{|\mathbb{F}|}$$

Conclusion
Combining the two failure probabilities using the union bound, the total probability that the algorithm fails to find the true minimal polynomial is bounded by:

$$P(\text{failure}) \leq \frac{2n}{|\mathbb{F}|}$$

Therefore, for a sufficiently large field (where $|\mathbb{F}| \gg 2n$), the minimal recurrence of the scalar sequence s_k is exactly $m_A(x)$ with high probability.

4.5.2 Determinant

Description To evaluate $\det(A)$ for a matrix $A \in \mathbb{F}^{n \times n}$, the algorithm generates a random diagonal matrix D (represented by its diagonal entries $c_1, \ldots, c_n$) and a random vector $v \in \mathbb{F}^n$. It defines the linear map $A' = AD$ and computes its minimal polynomial $m_{AD}(x)$ using a minimal recurrence algorithm on the Krylov sequence $v, A'v, (A')^2 v, \ldots$. If the degree of $m_{AD}(x)$ is n, the determinant of A is extracted from the polynomial's constant term.

Proof of Correctness The minimal polynomial $m_{AD}(x)$ always divides the characteristic polynomial $p_{AD}(x) = \det(xI - AD)$. If A is singular, AD is singular, and $m_{AD}(x)$ will trivially have a constant term of 0, yielding a correct determinant of 0. Therefore, we assume A is non-singular. If we can prove that AD generically has distinct eigenvalues, then with high probability, its minimal polynomial will have degree n, allowing us to safely assume $m_{AD}(x) = p_{AD}(x)$.

Lemma 35 (Krylov Subspace Dimension)**.** *A matrix $M \in \mathbb{F}^{n\times n}$ is non-derogatory (i.e., its minimal polynomial has degree n) if and only if there exists a vector $y \in \mathbb{F}^n$ such that the Krylov matrix*

$$K_M(y) = \begin{bmatrix} y & My & M^2y & \dots & M^{n-1}y \end{bmatrix}$$

is non-singular.

Proof. By the rational canonical form, the vector space decomposes into cyclic subspaces governed by the invariant factors of M. M is non-derogatory if and only if its largest invariant factor (the minimal polynomial) has degree n. If so, summing the generators of the coprime cyclic subspaces yields a vector y that generates an n-dimensional cyclic subspace, making $K_M(y)$ full rank. Conversely, if $K_M(y)$ is full rank, y generates an n-dimensional cyclic subspace, forcing $\deg(m_M) = n$. □

Let $\mathbf{x} = (x_1, \dots, x_n)$ and $\mathbf{y} = (y_1, \dots, y_n)$ be formal variables, and let $D_{\mathbf{x}} = \operatorname{diag}(x_1, \dots, x_n)$. We construct the formal Krylov matrix $K_{AD_{\mathbf{x}}}(\mathbf{y})$, whose determinant $F(\mathbf{x}, \mathbf{y}) = \det(K_{AD_{\mathbf{x}}}(\mathbf{y}))$ is a multivariate polynomial.

Lemma 36 (Generic Distinct Eigenvalues)**.** *If A is non-singular, the formal polynomial $F(\mathbf{x}, \mathbf{y})$ is not identically zero.*

Proof. Let $B = A^{-1}$. Since $\det(A)$ is a non-zero constant, the roots of $\det(\lambda I - AD_{\mathbf{x}}) = 0$ are distinct if and only if the roots of $Q(\lambda, \mathbf{x}) = \det(\lambda B - D_{\mathbf{x}})$ are distinct. We proceed by induction on n.

Base Case ($n = 1$): $Q(\lambda, x_1) = \lambda B_{11} - x_1$. Since A is invertible, $B_{11} \neq 0$, yielding exactly one root. The discriminant is trivially non-zero.

Inductive Step: Assume the claim holds for $n - 1$. Because B is non-singular, we may permute the coordinates such that the upper-left $(n - 1) \times (n - 1)$ submatrix, B_{n-1}, is invertible. Expanding $Q(\lambda, \mathbf{x})$ along the last diagonal entry yields:

$$Q(\lambda, \mathbf{x}) = Q_0(\lambda, x_1, \dots, x_{n-1}) - x_n Q_1(\lambda, x_1, \dots, x_{n-1})$$

where $Q_1 = \det(\lambda B_{n-1} - D_{n-1})$. By the inductive hypothesis, there exists an assignment $\mathbf{c} \in \mathbb{F}^{n-1}$ such that $Q_1(\lambda, \mathbf{c})$ has $n - 1$ distinct, non-zero roots. Fixing these values, we study the two-variable polynomial $Q(\lambda, x_n) = Q_0(\lambda) - x_n Q_1(\lambda)$.

We construct the reverse polynomial. Let $w = 1/\lambda$, and define:

$$R(w, x_n) = w^n Q(1/w, x_n) = R_0(w) - x_n w R_1(w)$$

Substituting $z = 1/x_n$ and multiplying by z yields:

$$H(w, z) = zR(w, 1/z) = zR_0(w) - wR_1(w)$$

Evaluating at $z = 0$ leaves $H(w, 0) = -wR_1(w)$. Because $Q_1(\lambda)$ has $n - 1$ distinct non-zero roots, its reverse polynomial $R_1(w)$ has $n - 1$ distinct finite roots, none of which are zero. Therefore, $H(w, 0)$ has exactly n distinct roots.

Thus, there exists an assignment for $\mathbf{x}$ yielding n distinct roots, meaning the generic $AD_{\mathbf{x}}$ is non-derogatory and $F(\mathbf{x}, \mathbf{y}) \not\equiv 0$. □

Lemma 37 (Failure Probability)**.** *For random assignments from a finite field $\mathbb{F}$, the probability that the algorithm fails is bounded by $\frac{n(n+1)}{2|\mathbb{F}|}$.*

Proof. To bound the probability of failure, we calculate the total degree of $F(\mathbf{x}, \mathbf{y})$. The k-th column of the Krylov matrix (for $k = 0, \ldots, n-1$) is $(AD_{\mathbf{x}})^k \mathbf{y}$. The entries in this column are homogeneous polynomials of degree k in $\mathbf{x}$ and degree 1 in $\mathbf{y}$. Because the determinant is a sum of products containing exactly one entry from each column, the total degree of F is:

$$\deg(F) = \sum_{k=0}^{n-1} k + \sum_{k=0}^{n-1} 1 = \frac{n(n-1)}{2} + n = \frac{n(n+1)}{2}$$

By the Schwartz-Zippel Lemma, if we draw the entries for D and v uniformly at random from a finite field $\mathbb{F}$, the probability that they form a root of F is bounded by:

$$\mathbb{P}(F(\mathbf{c}, \mathbf{v}) = 0) \leq \frac{n(n+1)}{2|\mathbb{F}|}$$

Finally, the algorithm extracts the constant term c_0 of the minimal polynomial. Since $p_{AD}(0) = \det(-AD) = (-1)^n \det(A) \det(D)$, we correctly compute the determinant as:

$$\det(A) = \frac{(-1)^n c_0}{\prod_{i=1}^{n} c_i}$$

□

4.5.3 Wiedemann's Algorithm

Description Wiedemann's algorithm solves the linear system $Ax = b$ for x. It generates a random vector u and computes the scalar sequence $s_k = u^T A^k b$. It finds the minimal polynomial of this sequence, $p(x) = \sum_{i=0}^{d} p_i x^i$. Assuming A is non-singular, $p_0 \neq 0$. The algorithm then computes the solution vector using Horner's method: $x = -\frac{1}{p_0} \sum_{i=1}^{d} p_i A^{i-1} b$.

Proof of Correctness Let $m_{A,b}(x)$ be the minimal polynomial of A with respect to the vector b, meaning it is the polynomial of minimal degree such that $m_{A,b}(A)b = 0$. With high probability, the minimal polynomial $p(x)$ of the sequence $u^T A^k b$ (found via linear recurrence finding) is exactly $m_{A,b}(x)$.

By definition, $p(A)b = 0$. Expanding this polynomial expression yields:

$$\sum_{i=0}^{d} p_i A^i b = 0$$

Because A is non-singular, the constant term of its minimal polynomial is non-zero, so $p_0 \neq 0$. We can isolate the $i = 0$ term:

$$p_0 b + \sum_{i=1}^{d} p_i A^i b = 0$$

$$p_0 b + A \left(\sum_{i=1}^{d} p_i A^{i-1} b \right) = 0$$

Rearranging to solve for b, we get:

$$A \left[-\frac{1}{p_0} \sum_{i=1}^{d} p_i A^{i-1} b \right] = b$$

Since $Ax = b$, the unique solution x is the term inside the brackets. To compute x efficiently without storing $O(d)$ dense vectors, the algorithm uses Horner's method, evaluating the sum iteratively from $i = d$ down to 1 via $R \leftarrow A(R) + p_i b$, initializing with $R = p_d b$. After d iterations, R equals the summation, giving the correct solution x using only $O(d)$ matrix-vector multiplications and $O(n)$ space.

4.6 Selected-Entry Matrix Multiplication

Let $X \in \mathbb{Z}^{n\times d}$ and $Y \in \mathbb{Z}^{d\times m}$. We want w specified entries of XY. Computing their dot products separately costs $O(wd)$, while computing the full matrix product may calculate many unwanted entries. We will *share intermediate multiplications* between outputs, but recurse only on the entries actually requested.

Theorem 38 (Selected-entry matrix multiplication)**.** *For every constant $0 < \tau \le 1$ there is an integer $\kappa = O(\tau^{-2})$ such that, if $d^\kappa \le \min\{n, m\}$, any $w \ge 1$ distinct specified entries of XY can be computed deterministically in time*

$$\widetilde{O}\big((n+m)d^\kappa + d^\tau\sqrt{nmw}\big)\,. \tag{26}$$

The input entries are integers of magnitude polynomial in $n + m$.

The main obstacle to sharing is familiar: multiplying sums of left and right inputs produces cross terms. Our construction exploits a particular kind of cross term, one whose left factor depends on only one index and whose right factor depends on only the other. First we show how to cancel such terms; then we explain why matrix multiplication naturally produces them.

4.6.1 Canceling Structured Interference

One output, four terms. Suppose we want $D = p_1q_1 + p_2q_2 + p_3q_3 + p_4q_4$. The p's are left blocks and the q's right blocks, so products are always written in this order. Place the four pairs in a 2×2 square. Add a third row and column as follows:

$$P = \begin{pmatrix} p_1 & p_2 & 0 \\ p_3 & p_4 & 0 \\ -p_1 - p_3 & -p_2 - p_4 & 0 \end{pmatrix}, \qquad Q = \begin{pmatrix} q_1 & q_2 & -q_1 - q_2 \\ q_3 & q_4 & -q_3 - q_4 \\ 0 & 0 & 0 \end{pmatrix}. \tag{27}$$

The added entries do not change the answer: $D = \sum_{i,j=1}^{3} P_{ij}Q_{ij}$. Their purpose is to make

$$\sum_i P_{ij} = 0 \quad \text{for each column } j, \qquad \sum_j Q_{ij} = 0 \quad \text{for each row } i. \tag{28}$$

Notice also the blank last column of P and last row of Q.

What errors can these checksums remove? Imagine that instead of multiplying P_{ij} by Q_{ij}, we have access to nine products $A_{ij}B_{ij}$ with

$$A_{ij} = P_{ij} + x_i, \qquad B_{ij} = Q_{ij} + y_j. \tag{29}$$

Here the unwanted left term is constant across each row of the 3×3 array; the unwanted right term is constant down each column. The x_i and y_j may be arbitrary matrix blocks. There are four kinds of terms after expansion: intended products, left-only errors, right-only errors, and products of two errors. The checksum conditions kill the two mixed kinds; the last kind factors because its left part depends only on i and its right part only on j. Precisely, (28) gives

$$\begin{aligned}\sum_{i,j} A_{ij}B_{ij} &= \sum_{i,j} P_{ij}Q_{ij} + \sum_j \big(\sum_i P_{ij}\big)y_j + \sum_i x_i\big(\sum_j Q_{ij}\big) + \sum_{i,j} x_iy_j \\ &= D + \big(\sum_i x_i\big)\big(\sum_j y_j\big).\end{aligned}$$

Thus *all* the interference is one product, not nine independent errors. Moreover, the blank border reveals its factors: $A_{i3} = x_i$ and $B_{3j} = y_j$. We therefore recover

$$D = \sum_{i,j=1}^{3} A_{ij}B_{ij} - \left(\sum_{i=1}^{3} A_{i3}\right)\left(\sum_{j=1}^{3} B_{3j}\right). \tag{30}$$

Locally, ten products are worse than the original four. The point is that the nine positive products can be *shared*; only the last correction must belong to this output. We now arrange many outputs so that they can use the same positive products.

4.6.2 Why Matrix Multiplication Produces These Errors

Encode input rows and columns, not outputs. Consider a block product $C = UV$ with four blocks in its middle dimension. For each input row r, form a 3×3 table P_r by applying the left recipe in (27) to $U_{r,1}, \dots, U_{r,4}$. Similarly, for each input column c, form Q_c from $V_{1,c}, \dots, V_{4,c}$ using the right recipe. We write $P_r(i,j)$ and $Q_c(i,j)$ for individual table entries. The output in row r, column c satisfies

$$C_{r,c} = \sum_{i,j=1}^{3} P_r(i,j) Q_c(i,j). \tag{31}$$

The advantage of encoding input rows and columns separately is simple: *all outputs in matrix row r use the same left table P_r, and all outputs in matrix column c use the same right table Q_c.*

A small sharing example. Consider a product $C = UV$ with six output rows and three output columns. Divide its eighteen outputs into two groups of nine, labeled

$$C = \begin{pmatrix} L_{11} & L_{12} & L_{13} \\ L_{21} & L_{22} & L_{23} \\ L_{31} & L_{32} & L_{33} \\ H_{11} & H_{12} & H_{13} \\ H_{21} & H_{22} & H_{23} \\ H_{31} & H_{32} & H_{33} \end{pmatrix}.$$

We want each output to participate in nine shared multiplications, one for each position of its 3×3 encoder tables. To accomplish this, pair every L_{uv} with every H_{ij}, assigning one multiplication to each pair.

The natural choice is for each output to use the other output's indices as its position: in the multiplication shared by L_{uv} and H_{ij}, the former contributes position (i,j) of its encoders, while the latter contributes position (u,v). Thus the two outputs contribute

	left factor	right factor
L_{uv}	$P_u(i,j)$	$Q_v(i,j)$
H_{ij}	$P_{3+i}(u,v)$	$Q_j(u,v)$.

Adding their left contributions, adding their right contributions, and multiplying gives the shared product

$$W_{uv;ij} = \big(P_u(i,j) + P_{3+i}(u,v)\big)\big(Q_v(i,j) + Q_j(u,v)\big). \tag{32}$$

For example,

$$W_{12;21} = (U_{1,3} + U_{5,2})(V_{3,2} + V_{2,1}).$$

Other pairs may involve checksum or zero entries, but the rule is always the same.

Why is this assignment useful beyond ensuring that every output uses all nine positions? Fix L_{uv} and vary its partner H_{ij}. Every partner contributes from the same position (u,v) of its encoders. Since P_{3+i} depends only on the partner's row i, and Q_j only on its column j, the shared factors have the form

$$P_u(i,j) + x_i, \qquad Q_v(i,j) + y_j.$$

These are precisely the row-dependent and column-dependent errors that our checksum identity cancels. The same argument applies when fixing an output H_{ij} instead.

Fix one output and look at all its partners. Keep L_{uv} fixed and let H_{ij} range over the other nine outputs. In (32), the first summands form precisely the left and right tables $P_u(i,j)$, $Q_v(i,j)$ of L_{uv}. The foreign contributions are $P_{3+i}(u,v)$ and $Q_j(u,v)$. Here (u,v) *is fixed*. As j changes, H_{ij} stays in the same matrix row, so its left contribution does not change. As i changes, H_{ij} stays in the same matrix column, so its right contribution does not change. These are exactly the repeated errors of (29).

For example, fix L_{12}. Every partner H_{ij} reads position $(1,2)$ of its tables; this happens to be an original input position, not a checksum. Its unwanted factors are

$$\underbrace{\begin{pmatrix} U_{4,2} & U_{4,2} & U_{4,2} \\ U_{5,2} & U_{5,2} & U_{5,2} \\ U_{6,2} & U_{6,2} & U_{6,2} \end{pmatrix}}_{\text{foreign left factors}}, \qquad \underbrace{\begin{pmatrix} V_{2,1} & V_{2,2} & V_{2,3} \\ V_{2,1} & V_{2,2} & V_{2,3} \\ V_{2,1} & V_{2,2} & V_{2,3} \end{pmatrix}}_{\text{foreign right factors}}.$$

Other fixed outputs may read checksum entries, but the repetition persists: every $P_r(u,v)$ depends only on input row r, and every $Q_c(u,v)$ only on input column c. Thus the decoder (30) recovers L_{uv} from its nine W products and one private product. By symmetry it recovers H_{ij} from the nine products sharing that output.

This example *does not yet save work*: eighteen outputs use eighty-one shared products. Its role is to show that sharing and cancellation can be made compatible. We next arrange much more reuse without changing the local checksum trick.

4.6.3 Amplifying Reuse with a Coordinate Grid

The preceding experiment labels a shared product by two pairs: (i,j), the slot used by its L output, and (u,v), the slot used by its H output. Fixing either pair and varying the other gives the nine shared products needed by one output. The clean way to make one multiplication serve more than two outputs is to use *more pairs of coordinates*.

Let a grid point consist of k pairs

$$z = ((a_1,b_1),\dots,(a_k,b_k)), \qquad a_h, b_h \in [3].$$

For each direction h, fix every pair except (a_h,b_h). The nine points obtained by varying this one pair form a *line* (a 3×3 slice). Associate one output with every line and one shared multiplication with every point. Every point belongs to exactly k lines (one per direction), while every line has nine points. Thus 9^k shared products serve $k9^{k-1}$ outputs: the shared cost is only $9/k$ per output.

To make this an actual block multiplication, set

$$R = k3^{k-1}, \qquad S = 3^{k-1},$$

and let U have R block rows and four block columns, and V four block rows and S block columns. For a tuple a, write a_{-h} for the tuple with its hth coordinate removed. Label an output by a direction h, a row tuple a_{-h}, and a column tuple b_{-h}; its position in UV is

$$(\underbrace{(h,a_{-h})}_{\text{output row}}, \underbrace{b_{-h}}_{\text{output column}}).$$

For example, if $k=3$ and $z=((a_1,b_1),(a_2,b_2),(a_3,b_3))$, the three lines through z correspond to the outputs

varying pair	output row	output column
(a_1,b_1)	$(1,a_2,a_3)$	(b_2,b_3)
(a_2,b_2)	$(2,a_1,a_3)$	(b_1,b_3)
(a_3,b_3)	$(3,a_1,a_2)$	(b_1,b_2)

The same point is therefore a shared multiplication used by three different outputs. The three outputs use it in the slots (a_1,b_1), (a_2,b_2), and (a_3,b_3) of their respective 3×3 tables.

Each block row r again carries P_r, and each block column c carries Q_c. At any grid point z, the k lines through it supply k left contributions and k right contributions. We pool them into *one* multiplication:

$$A_z = \sum_{h=1}^{k} P_{(h,a_{-h})}(a_h, b_h), \qquad B_z = \sum_{h=1}^{k} Q_{b_{-h}}(a_h, b_h), \qquad W_z = A_z B_z. \tag{33}$$

For $k = 2$, this is exactly the crossed pairing (32).

Why the decoder still works. Fix the output corresponding to direction h and to the fixed tuples a_{-h}, b_{-h}. Its nine shared products are obtained by varying $(a_h, b_h) = (i, j)$. Its own terms in (33) are $P_{(h,a_{-h})}(i, j)$ and $Q_{b_{-h}}(i, j)$, exactly its checksum tables. For any different direction $\ell \neq h$, its *slot* (a_ℓ, b_ℓ) stays fixed while (i, j) varies. Moreover, its left table is indexed only by a-coordinates, so changing $j = b_h$ cannot change its left contribution; its right table is indexed only by b-coordinates, so changing $i = a_h$ cannot change its right contribution. Hence all foreign left terms depend only on i, and all foreign right terms only on j. The nine shared products have exactly the form (29). Our decoder recovers the output with nine shared products and one private correction.

There is one private child for each of the $RS = k9^{k-1}$ outputs, and 9^k shared children in all. Thus the number of smaller products is

$$T = RS + 9^k = (k + 9)9^{k-1}, \qquad \frac{T}{RS} = 1 + \frac{9}{k}, \tag{34}$$

compared with $4RS$ for four independent products per output. This is the saving obtained by the high-dimensional grid.

4.6.4 Recursion and Running Time

Apply the rule to blocks of an $N \times D$ by $D \times M$ product. In each step, split the outer dimensions into R, S parts and the middle dimension into four parts. All children have dimensions $(N/R) \times (D/4)$ by $(D/4) \times (M/S)$. An entry requested in a parent requires the corresponding entry of its nine shared children and one private child. Propagate only these requests, merging duplicates within each child. At middle dimension one, multiply only the requested scalar pairs, then reconstruct requested entries upwards.

Why shared children do not cause too many requests. Let a subproblem have z requested entries among its s possible output positions. Its private children receive z requests and have s positions in total. Its shared children receive at most $9z$ requests, but have only $(9/k)s$ positions: this is the reuse counted above. So counting requests alone is too pessimistic, while counting all positions ignores that most entries are not requested.

For each subproblem v, write z_v for its number of distinct requests and s_v for its number of output positions. Define

$$\Phi = \sum_v \sqrt{z_v s_v}.$$

This lies above the number of requests, since $z_v \leq s_v$. It is convenient because, for any group of subproblems, Cauchy–Schwarz gives $\sum_v \sqrt{z_v s_v} \leq \sqrt{(\sum_v z_v)(\sum_v s_v)}$. Consequently, one parent contributes at most $\sqrt{zs}$ to the potential of its private children and $(9/\sqrt{k})\sqrt{zs}$ to that of its shared children. Thus, if Φ_j is the potential at recursion depth j,

$$\Phi_{j+1} \leq \left(1 + \frac{9}{\sqrt{k}}\right) \Phi_j, \qquad \Phi_0 = \sqrt{NMw}. \tag{35}$$

After $t = \log_4 D$ steps, the total number of marked entries at any level is at most $\sqrt{NMw}(1 + 9/\sqrt{k})^t$. This bounds the leaf multiplications and the work to recover marked entries, up to a factor polynomial in t.

Preparing the child inputs. The factor matrices of each child are signed sums of at most $O(k)$ parent blocks. We may prepare *every* child, even those never requested. At depth j there are at most T^j children, each with left input size $ND/(4R)^j$ and right input size $MD/(4S)^j$. Preparing all child inputs at depths 1 through t therefore costs

$$O_k\left(\sum_{j=1}^{t}\left[ND\left(\frac{T}{4R}\right)^j + MD\left(\frac{T}{4S}\right)^j\right]\right) = O_k\left(N\left(\frac{T}{R}\right)^t + M\left(\frac{T}{S}\right)^t\right). \tag{36}$$

Here $T/R = (1+9/k)3^{k-1}$ and $T/S = (k+9)3^{k-1}$. Therefore, taking

$$\kappa = \left\lceil \log_4\big((k+9)3^{k-1}\big)\right\rceil \tag{37}$$

ensures preparation time $O_k((N+M)D^\kappa)$.

Finally pad the middle dimension d to a power of four and pad the outer dimensions to multiples of R^t, S^t. Since $R^t, S^t \le d^\kappa$ up to factors depending only on k, the hypothesis $d^\kappa \le \min\{n, m\}$ keeps the padded outer dimensions $O_k(n), O_k(m)$. All intermediate values have $O_k(\log(n+m))$ bits: each step uses only $O_k(1)$ additions, and there are $O(\log d)$ steps. Choose

$$k = \max\left\{4, \left\lceil\left(\frac{9}{4^\tau - 1}\right)^2\right\rceil\right\}.$$

Then $1 + 9/\sqrt{k} \le 4^\tau$ and $\kappa = O(k) = O(\tau^{-2})$. Combining the two costs proves the theorem; $d = 1$ is immediate.

Improved constants. For better constants, take $q = 4$ instead of $q = 3$. Each recursion step then reduces the middle dimension by a factor of 9 rather than 4, using 16 shared products per output instead of 9. The same proof applies unchanged. Optimizing the parameters in the regime $n = m = N$ and $w = N^2/\sqrt{d}$ improves the exponent saving from approximately 0.0001816 to 0.0002050. We use $q = 3$ above only because its smaller arrays make the construction easier to explain.

4.7 Lights Out

To prove that a vector lies in the image of a matrix, it is often easier to check the obstructions to solving the corresponding linear system. For any matrix B over a field,

$$\operatorname{im} B = (\ker B^{\mathsf{T}})^{\perp}.$$

Indeed, every column of B is orthogonal to $\ker B^{\mathsf{T}}$, and the two spaces have the same dimension by rank–nullity. Thus $Bx = b$ is solvable exactly when $z^{\mathsf{T}} b = 0$ for every $z \in \ker B^{\mathsf{T}}$.

For a symmetric matrix over $\mathbb{F}_2$, its diagonal automatically passes this test. The reason is that the off-diagonal terms in a quadratic form occur in equal pairs, while $z_i^2 = z_i$.

Theorem 39 (The Diagonal Lies in the Image)**.** *Let $A \in \mathbb{F}_2^{n\times n}$ be symmetric, and let $d = (a_{11}, \dots, a_{nn})^{\mathsf{T}}$. Then $Ax = d$ has a solution.*

Proof. For every $z \in \mathbb{F}_2^n$,

$$z^{\mathsf{T}} A z = \sum_i a_{ii} z_i^2 + \sum_{i<j} (a_{ij} + a_{ji}) z_i z_j = \sum_i a_{ii} z_i = d^{\mathsf{T}} z.$$

Consequently, $Az = 0$ implies $d^{\mathsf{T}} z = 0$. Since A is symmetric, $d \in (\ker A)^{\perp} = \operatorname{im} A$. □

Toggling on a graph. Let G be a finite simple undirected graph, with adjacency matrix B over $\mathbb{F}_2$. A configuration is a vector $b \in \mathbb{F}_2^{V(G)}$, with 1 meaning that a light is on. Pressing a vertex toggles its neighbors. Fix a set $T \subseteq V(G)$ of vertices whose presses also toggle the pressed vertex itself. The matrix of moves is

$$A = B + \operatorname{diag}(\mathbf{1}_T).$$

Presses commute, and pressing twice has no effect. Hence pressing the vertices indicated by $x \in \mathbb{F}_2^{V(G)}$ changes the configuration from b to $b + Ax$.

Corollary 40 (Lights Out)**.** *With these rules, the configuration whose lit vertices are exactly T can always be cleared. In particular, when every press toggles both the pressed vertex and all its neighbors, the all-on configuration can always be cleared.*

For the usual self-toggling rule, this equivalently says that every finite simple graph has a set $X \subseteq V(G)$ such that

$$|X \cap N[v]| \equiv 1 \pmod 2 \qquad \text{for every } v \in V(G),$$

where $N[v] = \{v\} \cup N(v)$ is the closed neighborhood of v.

Proof. The matrix A is symmetric and has diagonal $\mathbf{1}_T$, so Theorem 39 gives $Ax = \mathbf{1}_T$. For $T = V(G)$, the coordinate equation at v says that an odd number of vertices in $N[v]$ are pressed. □

Proposition 41 (Solvable Configurations)**.** *For a symmetric move matrix $A \in \mathbb{F}_2^{n\times n}$, a configuration b can be cleared if and only if*

$$z^{\mathsf{T}} b = 0 \qquad \text{for every } z \in \ker A.$$

If x_0 clears b, all clearing press vectors form the affine space $x_0 + \ker A$. Thus every solvable configuration has exactly $2^{n-\operatorname{rank} A}$ clearing press vectors, and exactly $2^{\operatorname{rank} A}$ configurations are solvable.

Proof. If $Az = 0$, then $z^{\mathsf{T}}(b+Ax) = z^{\mathsf{T}}b$ for every press vector x. Thus $z^{\mathsf{T}}b$ is an invariant of all moves and must vanish if b can be cleared. Conversely, vanishing of all these invariants says precisely that $b \in (\ker A)^{\perp} = \operatorname{im} A$. Finally, $Ax = Ax_0$ is equivalent to $x - x_0 \in \ker A$, and rank–nullity gives both counts. □

4.8 Perron-Frobenius Theorem

The Perron-Frobenius theorem establishes the spectral properties of non-negative matrices. A matrix $A \geq 0$ is irreducible if its directed graph is strongly connected; for any i, j, there exists an integer $k > 0$ such that $(A^k)_{ij} > 0$.

Theorem 42 (Perron-Frobenius for Irreducible Matrices)**.** *Let A be an $n \times n$ non-negative, irreducible matrix. Then:*

1. *A has a real, strictly positive eigenvalue $r > 0$ (the Perron root).*
2. *There exists a strictly positive eigenvector $v > 0$ corresponding to r.*
3. *For any other eigenvalue $\lambda \in \mathbb{C}$ of A, $|\lambda| \leq r$.*
4. *The eigenvalue r is algebraically (and geometrically) simple.*

Proof. Let $\Delta = \{x \in \mathbb{R}^n : x \geq 0, \sum_{i=1}^n x_i = 1\}$ be the simplex. For any non-zero $x \geq 0$, define the function:

$$r(x) = \max\{\rho \geq 0 \mid Ax \geq \rho x\} = \min_{x_i > 0} \frac{(Ax)_i}{x_i}$$

Let $r = \sup_{x\in\Delta} r(x)$.

Because A is irreducible, the matrix $B = (I + A)^{n-1}$ is positive ($B > 0$). Furthermore, B commutes with A. For any $x \in \Delta$, if $Ax \geq \rho x$, then $A(Bx) = B(Ax) \geq \rho(Bx)$, which implies $r(Bx) \geq r(x)$. The

map $x \mapsto Bx$ sends Δ into $\mathbb{R}^n_{>0}$. In this interior, $r(x)$ is continuous. Since the image $B(\Delta)$ is compact and contained in the interior, the supremum r is finite and achieved at some positive vector $v = Bx > 0$. Thus, $Av \geq rv$.

$Av = rv$**.** Suppose for contradiction that $Av \geq rv$ but $Av \neq rv$. This means the vector $z = Av - rv$ is non-negative and non-zero ($z \gneq 0$). Because $B > 0$, multiplying by B yields a positive vector: $Bz > 0$. Thus, $B(Av - rv) > 0 \implies A(Bv) > r(Bv)$. Since strict inequality holds for all coordinates of Bv, we can find some small $\epsilon > 0$ such that $A(Bv) \geq (r + \epsilon)(Bv)$. This implies $r(Bv) \geq r + \epsilon$, which contradicts the maximality of r. Therefore, we must have $Av = rv$. Since $v > 0$, r must be strictly positive (as $A \neq 0$).

Strict positivity of the eigenvector. Suppose $u \geq 0$ is any non-zero eigenvector for r, meaning $Au = ru$. Then $Bu = (1 + r)^{n-1}u$. Since $u \gneq 0$ and $B > 0$, we have $Bu > 0$, forcing $(1 + r)^{n-1}u > 0$, which means u must be strictly positive.

Maximality of r**.** Let λ be any eigenvalue of A with eigenvector $y \in \mathbb{C}^n$, so $Ay = \lambda y$. Taking absolute values component-wise, and using the triangle inequality alongside the non-negativity of A, we get:

$$|\lambda||y| = |\lambda y| = |Ay| \leq A|y|$$

This implies that the non-negative, non-zero vector $|y|$ satisfies $A|y| \geq |\lambda||y|$. By the definition of our function, $r(|y|) \geq |\lambda|$. Since r is the supremum over all such vectors, $r \geq |\lambda|$.

Simplicity of r**.** By applying the exact same argument to A^T (which is also irreducible), there exists a strictly positive left eigenvector $w^T > 0$ such that $w^T A = rw^T$.

To show the geometric multiplicity is 1, suppose v and y are two linearly independent eigenvectors for r. Since $v > 0$, we can choose a real scalar c such that $z = v - cy$ has at least one coordinate equal to zero, but z is not the zero vector. However, $Az = rz$. Taking absolute values, $A|z| \geq r|z|$. This forces $A|z| = r|z|$, making $|z|$ a non-negative eigenvector for r. Then $|z|$ must be strictly positive, contradicting that it has a zero coordinate. Thus, the geometric multiplicity is 1.

To prove the algebraic multiplicity is 1, we utilize the strictly positive left eigenvector w^T. If the algebraic multiplicity were strictly greater than 1, there would exist a generalized eigenvector $g \in \ker(A - rI)^2$ such that $(A - rI)g = v$. Multiplying both sides on the left by w^T yields:

$$w^T(A - rI)g = w^T v$$

Because w^T is a left eigenvector associated with r, we have $w^T A = rw^T$, meaning $w^T(A - rI) = 0$. Thus, the left side is 0. However, because both $w > 0$ and $v > 0$, their inner product is strictly positive ($w^T v > 0$). Therefore, r must be an algebraically simple root. □

4.9 Continuity of Eigenvalues

The basic fact is that roots of a polynomial with continuously varying coefficients cannot suddenly appear far away, and any limit of roots of nearby polynomials must still be a root of the limiting polynomial. It is convenient to formulate this for the unordered multiset of roots.

For two unordered n-tuples $[a_1, \ldots, a_n]$ and $[b_1, \ldots, b_n]$, define

$$d\big([a_1, \ldots, a_n], [b_1, \ldots, b_n]\big) = \min_{\pi \in S_n} \max_i |a_i - b_{\pi(i)}|.$$

This is the natural metric on $\mathbb{C}^n/S_n$.

Theorem 43 (Continuity of Roots)**.** *The unordered multiset of roots of a monic complex polynomial depends continuously on its coefficients.*

Proof. Let

$$p_k(z) = z^n + c^{(k)}_{n-1} z^{n-1} + \cdots + c^{(k)}_0$$

converge coefficientwise to

$$p(z) = z^n + c_{n-1} z^{n-1} + \cdots + c_0.$$

We prove that the root multisets of p_k converge to the root multiset of p.

First, all roots stay in one fixed compact disk. Indeed, if

$$M_k = \max_{0 \le j < n} |c_j^{(k)}|,$$

then every root z of p_k satisfies the Cauchy bound

$$|z| \le 1 + M_k.$$

Since the coefficients converge, the numbers M_k are bounded, so one disk contains every root of every sufficiently large p_k.

Write the roots of p_k, in an arbitrary order, as

$$a_1^{(k)}, \dots, a_n^{(k)}.$$

Consider any subsequence. Because all roots lie in a fixed compact disk, after passing to a further subsequence we may assume

$$a_i^{(k)} \longrightarrow a_i \qquad (i = 1, \dots, n).$$

Then for every z,

$$p_k(z) = \prod_{i=1}^{n} (z - a_i^{(k)}) \longrightarrow \prod_{i=1}^{n} (z - a_i).$$

On the other hand coefficientwise convergence gives $p_k(z) \to p(z)$. Hence

$$p(z) = \prod_{i=1}^{n} (z - a_i).$$

Thus every subsequential limit of the root multiset is exactly the root multiset of p, with multiplicity.

If the root multisets did not converge to the roots of p, there would be an $\varepsilon > 0$ and a subsequence remaining at distance at least ε from that multiset. The preceding compactness argument would produce a further subsequence converging to the root multiset of p, a contradiction. Therefore the roots depend continuously on the coefficients. □

Corollary 44 (Continuity of Eigenvalues)**.** *Let $A \in M_n(\mathbb{C})$. The unordered multiset of eigenvalues of A depends continuously on the entries of A.*

Proof. The coefficients of

$$p_A(z) = \det(zI - A)$$

are polynomials in the entries of A, hence depend continuously on A. The eigenvalues are precisely the roots of p_A, so the result follows from the preceding theorem. □

4.10 Gershgorin's Circle Theorem

Gershgorin's theorem localizes the eigenvalues of a matrix using only its entries.

The basic idea is extremely simple. If

$$Ax = \lambda x,$$

choose a coordinate of the eigenvector which is largest in magnitude. In that coordinate, the diagonal term must almost account for the eigenvalue, because all other coordinates are no larger.

It is essentially no harder to allow a different scale for each coordinate. This gives a useful weighted version of the theorem.

Theorem 45 (Weighted Gershgorin Circle Theorem)**.** *Let*

$$A = (a_{ij}) \in \mathbb{C}^{n\times n},$$

and let

$$w_1, \dots, w_n > 0.$$

For each i, define the weighted row radius

$$R_i(w) = \sum_{j\neq i} |a_{ij}| \frac{w_j}{w_i}$$

and the disk

$$D_i(w) = \{z \in \mathbb{C} : |z - a_{ii}| \leq R_i(w)\}.$$

Then every eigenvalue of A lies in

$$\sigma(A) \subseteq \bigcup_{i=1}^{n} D_i(w).$$

Moreover, suppose $I \subseteq [n]$ is such that

$$\bigcup_{i\in I} D_i(w)$$

is disjoint from

$$\bigcup_{i\notin I} D_i(w).$$

Then the first union contains exactly

$$|I|$$

eigenvalues of A, counted with algebraic multiplicity.

Proof. We first prove the localization statement.

Let

$$Ax = \lambda x, \qquad x \neq 0.$$

Choose i maximizing

$$\frac{|x_i|}{w_i}.$$

In particular, $x_i \neq 0$, and for every j,

$$|x_j| \leq \frac{w_j}{w_i}|x_i|.$$

The i-th coordinate of $Ax = \lambda x$ gives

$$(\lambda - a_{ii})x_i = \sum_{j\neq i} a_{ij}x_j.$$

Taking absolute values,

$$|\lambda - a_{ii}|\,|x_i| \leq \sum_{j\neq i} |a_{ij}|\,|x_j| \leq \sum_{j\neq i} |a_{ij}| \frac{w_j}{w_i}|x_i|.$$

Canceling $|x_i|$,

$$|\lambda - a_{ii}| \leq R_i(w).$$

Thus

$$\lambda \in D_i(w).$$

For the counting statement, write

$$A = D + E,$$

where

$$D = \operatorname{diag}(a_{11}, \ldots, a_{nn})$$

and E has zero diagonal. Consider the (continuous) family

$$A(t) = D + tE, \qquad 0 \le t \le 1.$$

Its weighted Gershgorin disks are

$$D_i(t) = \{z : |z - a_{ii}| \le tR_i(w)\}.$$

Since

$$D_i(t) \subseteq D_i(1),$$

the two unions

$$\mathcal{D}_I(t) = \bigcup_{i \in I} D_i(t), \qquad \mathcal{D}_{\overline{I}}(t) = \bigcup_{i \notin I} D_i(t)$$

remain disjoint for every $t \in [0, 1]$.

By the first part, every eigenvalue of $A(t)$ belongs to one of these two sets.

Because the two original unions are compact and disjoint, they are separated by some positive distance. The eigenvalues depend continuously on the entries of the matrix by Corollary 44, so as t changes, no eigenvalue can pass from one union to the other without crossing the gap between them. Consequently, the number of eigenvalues in $\mathcal{D}_I(t)$, counted with algebraic multiplicity, is constant in t.

At $t = 0$,

$$A(0) = D,$$

whose eigenvalues are exactly

$$a_{11}, \ldots, a_{nn}.$$

The union $\mathcal{D}_I(0)$ contains precisely the diagonal entries

$$a_{ii}, \qquad i \in I,$$

and hence exactly $|I|$ eigenvalues, with multiplicity.

The same is therefore true at $t = 1$, where $A(1) = A$. □

The usual Gershgorin theorem is the special case

$$w_1 = \cdots = w_n = 1.$$

Thus, writing

$$R_i = \sum_{j \ne i} |a_{ij}|,$$

every eigenvalue lies in one of the ordinary Gershgorin disks

$$D_i = \{z : |z - a_{ii}| \le R_i\}.$$

The weighted form can also be viewed as ordinary Gershgorin after a diagonal change of coordinates. Indeed, if

$$W = \operatorname{diag}(w_1, \ldots, w_n),$$

then

$$W^{-1} A W$$

is similar to A, while its ij-entry is

$$a_{ij} \frac{w_j}{w_i}.$$

Thus the weights simply choose the coordinate scaling in which we apply Gershgorin.

Corollary 46 (Second Gershgorin Theorem)**.** *Every connected component of*

$$\bigcup_{i=1}^{n} D_i$$

which consists of k Gershgorin disks contains exactly k eigenvalues, counted with algebraic multiplicity.

Proof. Each disk is connected, so every disk lies entirely within one connected component of the union. A connected component is therefore the union of some subset I of the disks.

Since there are finitely many disks, distinct components are compact and disjoint. Apply the counting part of Theorem 45. □

There is an analogous column version. Define

$$C_i = \sum_{j \neq i} |a_{ji}|.$$

Since A and A^T have the same characteristic polynomial, applying Gershgorin to A^T gives

$$\sigma(A) \subseteq \bigcup_i \{z : |z - a_{ii}| \leq C_i\}.$$

Hence, using both row and column information,

$$\sigma(A) \subseteq \left(\bigcup_i D_i^{\text{row}} \right) \cap \left(\bigcup_i D_i^{\text{col}} \right).$$

For Hermitian matrices all eigenvalues are real, so Gershgorin becomes the interval bound

$$\sigma(A) \subseteq \bigcup_i [a_{ii} - R_i, a_{ii} + R_i].$$

In particular, if A is Hermitian and

$$a_{ii} > R_i \qquad \text{for every } i,$$

then every eigenvalue is positive, and hence

$$A \succ 0.$$

More generally, if

$$|a_{ii}| > R_i \qquad \text{for every } i,$$

then 0 belongs to no Gershgorin disk, so A is nonsingular.

5 Complex Analysis

5.1 Cauchy's Theorem

The key fact of complex integration is that a holomorphic function has zero integral around a closed contour. The essential case is a triangle.

Theorem 47 (Cauchy–Goursat for Triangles)**.** *Let T be a triangle, and suppose that f is holomorphic on an open neighborhood of T. Then*

$$\oint_{\partial T} f(z)\, dz = 0.$$

Proof. Write

$$I(T) = \oint_{\partial T} f(z)\,dz.$$

Divide T into four similar triangles by joining the midpoints of its sides. The oriented boundary integrals of the four smaller triangles sum to $I(T)$, since every interior edge appears twice with opposite orientations. Hence one of the four triangles, call it T_1, satisfies

$$|I(T_1)| \geq \frac{1}{4}|I(T)|.$$

Repeating this construction gives nested triangles

$$T \supset T_1 \supset T_2 \supset \cdots$$

such that

$$|I(T_n)| \geq 4^{-n}|I(T)|, \qquad \operatorname{diam}(T_n) = 2^{-n}\operatorname{diam}(T).$$

The triangles shrink to a single point z_0.

Since f is differentiable at z_0,

$$f(z) = f(z_0) + f'(z_0)(z - z_0) + r(z),$$

where

$$r(z) = o(|z - z_0|).$$

Thus, for every $\varepsilon > 0$, for all sufficiently large n and every $z \in T_n$,

$$|r(z)| \leq \varepsilon|z - z_0| \leq \varepsilon \operatorname{diam}(T_n).$$

The constant and linear terms have zero integral around a closed contour:

$$\oint_{\partial T_n} 1\,dz = 0, \qquad \oint_{\partial T_n} (z - z_0)\,dz = 0.$$

Therefore

$$I(T_n) = \oint_{\partial T_n} r(z)\,dz.$$

Hence

$$\begin{aligned} |I(T_n)| &\leq \max_{z \in T_n} |r(z)| \operatorname{perimeter}(T_n) \\ &\leq \varepsilon \operatorname{diam}(T_n) \operatorname{perimeter}(T_n) \\ &= C\varepsilon\, 4^{-n}, \end{aligned}$$

where

$$C = \operatorname{diam}(T) \operatorname{perimeter}(T).$$

Combining this with

$$4^{-n}|I(T)| \leq |I(T_n)|$$

gives

$$|I(T)| \leq C\varepsilon.$$

Since $\varepsilon > 0$ is arbitrary, $I(T) = 0$. □

The proof has one main idea: repeatedly zoom in while retaining a constant fraction of the integral. At the limiting point, holomorphicity says that f is linear up to an error smaller than the scale of the triangle; the linear part integrates to zero and the error is too small to support a non-zero integral.

Corollary 48 (Cauchy's Theorem for Polygonal Curves)**.** *Let γ be a positively oriented simple closed polygonal curve, and suppose that f is holomorphic on an open neighborhood of γ and its interior. Then*

$$\oint_\gamma f(z)\,dz = 0.$$

Proof. Triangulate the polygon bounded by γ. By Theorem 47, the integral around every triangle is zero. Summing these integrals, every interior edge appears twice with opposite orientations and cancels, leaving exactly

$$\oint_\gamma f(z)\,dz = 0.$$

□

The polygonal statement immediately gives the usual form of Cauchy's theorem.

Theorem 49 (Cauchy's Theorem)**.** *Let $U \subseteq \mathbb{C}$ be simply connected, let $f : U \to \mathbb{C}$ be holomorphic, and let γ be a closed piecewise-C^1 curve in U. Then*

$$\oint_\gamma f(z)\,dz = 0.$$

Proof. First suppose that γ is a simple closed curve. Since U is simply connected, the interior of γ is contained in U.

Approximate γ by simple closed polygonal curves γ_m such that both the curves and their derivatives converge to γ on each C^1 piece. For sufficiently large m, γ_m and its interior lie inside U, so by Corollary 48,

$$\oint_{\gamma_m} f(z)\,dz = 0.$$

Writing the curves on the same parameter interval,

$$\begin{aligned}\left|\int_\gamma f(z)\,dz - \int_{\gamma_m} f(z)\,dz\right| &= \left|\int \left(f(\gamma(t))\gamma'(t) - f(\gamma_m(t))\gamma_m'(t)\right)\,dt\right| \\ &\le \int |f(\gamma) - f(\gamma_m)|\,|\gamma'|\,dt + \int |f(\gamma_m)|\,|\gamma' - \gamma_m'|\,dt.\end{aligned}$$

Since f is uniformly continuous and bounded on a compact neighborhood of the curves, both terms tend to zero. Hence

$$\oint_\gamma f(z)\,dz = 0.$$

A general closed piecewise-C^1 curve can be approximated in the same way by a closed polygonal curve. Splitting the polygon at its finitely many self-intersections decomposes it into finitely many simple closed polygonal curves, each of which has zero integral. Passing to the limit again proves the result. □

Corollary 50 (Cauchy's Theorem with Holes)**.** *Let Ω be a bounded region whose boundary consists of finitely many piecewise-C^1 curves, oriented so that Ω always lies to the left. If f is holomorphic on a neighborhood of $\overline{\Omega}$, then*

$$\oint_{\partial\Omega} f(z)\,dz = 0.$$

Equivalently, if γ is the counterclockwise outer boundary and $C_1, \dots, C_m$ are the counterclockwise boundaries of the holes, then

$$\oint_\gamma f(z)\,dz = \sum_{j=1}^{m} \oint_{C_j} f(z)\,dz.$$

Proof. Join every inner boundary to the outer boundary by non-intersecting cuts. The resulting region is simply connected, so Cauchy's theorem applies. Each cut appears twice in the resulting boundary, once in each direction, and the two integrals cancel. What remains is the displayed boundary integral. □

5.2 Cauchy's Integral Formula

Cauchy's integral formula can be viewed retrospectively as the one-pole case of the residue theorem. We prove it first, however, since it follows directly from Cauchy's theorem and does not require Laurent series.

Theorem 51 (Cauchy's Integral Formula)**.** *Let γ be a positively oriented simple closed piecewise-C^1 curve, let z lie in its interior, and suppose that f is holomorphic on a neighborhood of γ and its interior. Then*

$$f(z) = \frac{1}{2\pi i} \oint_\gamma \frac{f(w)}{w - z}\, dw.$$

Proof. Choose a sufficiently small circle

$$C_\varepsilon = \{w : |w - z| = \varepsilon\}$$

inside γ. The function

$$w \longmapsto \frac{f(w)}{w - z}$$

is holomorphic on the region between γ and C_ε. By Corollary 50,

$$\oint_\gamma \frac{f(w)}{w - z}\, dw = \oint_{C_\varepsilon} \frac{f(w)}{w - z}\, dw.$$

Parameterize the small circle by

$$w = z + \varepsilon e^{it}, \qquad 0 \le t \le 2\pi.$$

Then

$$\begin{aligned} \oint_{C_\varepsilon} \frac{f(w)}{w - z}\, dw &= \int_0^{2\pi} \frac{f(z + \varepsilon e^{it})}{\varepsilon e^{it}} i\varepsilon e^{it}\, dt \\ &= i \int_0^{2\pi} f(z + \varepsilon e^{it})\, dt. \end{aligned}$$

As $\varepsilon \to 0$, continuity of f gives

$$i \int_0^{2\pi} f(z + \varepsilon e^{it})\, dt \longrightarrow 2\pi i f(z).$$

The left-hand side is independent of ε, so it must already equal $2\pi i f(z)$. □

Corollary 52 (Cauchy's Differentiation Formula)**.** *Under the hypotheses of Theorem 51, for every integer $k \ge 0$,*

$$f^{(k)}(z) = \frac{k!}{2\pi i} \oint_\gamma \frac{f(w)}{(w - z)^{k+1}}\, dw.$$

Proof. Differentiate the integral formula k times with respect to z. Since the contour stays a positive distance from z, differentiation under the integral is valid, and

$$\frac{d^k}{dz^k} \frac{1}{w - z} = \frac{k!}{(w - z)^{k+1}}.$$

□

5.3 Laurent Series

Cauchy's integral formula also explains why holomorphic functions are represented by power series, and why isolated singularities are represented by Laurent series.

Theorem 53 (Laurent Expansion)**.** *Suppose that f is holomorphic on the annulus*

$$r < |z-a| < R.$$

Then there are unique coefficients $(c_n)_{n\in\mathbb{Z}}$ such that

$$f(z) = \sum_{n=-\infty}^{\infty} c_n (z-a)^n$$

throughout the annulus, with convergence uniform on compact subannuli.

For every $r < \rho < R$,

$$\boxed{c_n = \frac{1}{2\pi i} \oint_{|w-a|=\rho} \frac{f(w)}{(w-a)^{n+1}}\, dw} \qquad (n \in \mathbb{Z}).$$

Proof. Fix z with

$$r < |z-a| < R,$$

and choose radii

$$r < \rho_- < |z-a| < \rho_+ < R.$$

Applying the same argument as Cauchy's integral formula to the annulus between the two circles gives

$$f(z) = \frac{1}{2\pi i} \oint_{|w-a|=\rho_+} \frac{f(w)}{w-z}\, dw - \frac{1}{2\pi i} \oint_{|w-a|=\rho_-} \frac{f(w)}{w-z}\, dw.$$

For the outer circle, $|z-a| < |w-a|$, so

$$\begin{aligned} \frac{1}{w-z} &= \frac{1}{(w-a)-(z-a)} \\ &= \frac{1}{w-a} \frac{1}{1-\frac{z-a}{w-a}} \\ &= \sum_{n=0}^{\infty} \frac{(z-a)^n}{(w-a)^{n+1}}. \end{aligned}$$

The geometric series converges uniformly on the circle, so we may integrate term-by-term:

$$\frac{1}{2\pi i} \oint_{|w-a|=\rho_+} \frac{f(w)}{w-z}\, dw = \sum_{n=0}^{\infty} \left(\frac{1}{2\pi i} \oint_{|w-a|=\rho_+} \frac{f(w)}{(w-a)^{n+1}}\, dw \right) (z-a)^n.$$

For the inner circle, $|w-a| < |z-a|$, so

$$\begin{aligned} \frac{1}{w-z} &= -\frac{1}{z-a} \frac{1}{1-\frac{w-a}{z-a}} \\ &= -\sum_{m=0}^{\infty} \frac{(w-a)^m}{(z-a)^{m+1}}. \end{aligned}$$

Again the convergence is uniform, hence

$$-\frac{1}{2\pi i} \oint_{|w-a|=\rho_-} \frac{f(w)}{w-z}\, dw = \sum_{m=0}^{\infty} \left(\frac{1}{2\pi i} \oint_{|w-a|=\rho_-} f(w)(w-a)^m\, dw \right) (z-a)^{-m-1}.$$

Together these give a Laurent expansion

$$f(z) = \sum_{n=-\infty}^{\infty} c_n (z-a)^n.$$

For any fixed integer n, the function

$$w \longmapsto \frac{f(w)}{(w-a)^{n+1}}$$

is holomorphic throughout the annulus. By Corollary 50, its integral around two circles centered at a inside the annulus is the same. Thus both the positive and negative coefficients can be written uniformly as

$$c_n = \frac{1}{2\pi i} \oint_{|w-a|=\rho} \frac{f(w)}{(w-a)^{n+1}}\, dw$$

for any $r < \rho < R$.

The geometric-series argument is uniform whenever z ranges over a compact subannulus, which gives locally uniform convergence.

Finally, the coefficient formula determines every c_n, so the expansion is unique. □

Corollary 54 (Taylor Expansion)**.** *If f is holomorphic on $|z-a| < R$, then*

$$f(z) = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!} (z-a)^n.$$

Proof. Apply Theorem 53 with $r = 0$. For $n < 0$, the coefficient integrand

$$\frac{f(w)}{(w-a)^{n+1}}$$

has a non-negative power of $(w-a)$ and is holomorphic inside the circle, so Cauchy's theorem gives $c_n = 0$.

For $n \geq 0$, Corollary 52 gives

$$c_n = \frac{f^{(n)}(a)}{n!}.$$

□

5.4 Residue Theorem

Let a be an isolated singularity of f. By Theorem 53, in a punctured neighborhood of a,

$$f(z) = \sum_{n=-\infty}^{\infty} c_n (z-a)^n.$$

The coefficient

$$\operatorname{Res}(f, a) := c_{-1}$$

is called the *residue* of f at a.

The significance of this coefficient follows from the elementary identity

$$\oint_{|z-a|=\rho} (z-a)^n\, dz = \begin{cases} 2\pi i, & n = -1, \\ 0, & n \neq -1. \end{cases}$$

Indeed, uniform convergence of the Laurent series on the circle gives

$$\boxed{\oint_{|z-a|=\rho} f(z)\, dz = 2\pi i \operatorname{Res}(f, a).}$$

Theorem 55 (Residue Theorem)**.** *Let γ be a positively oriented simple closed piecewise-C^1 curve, and suppose that f is holomorphic on a neighborhood of γ and its interior except at finitely many isolated singularities*

$$a_1, \dots, a_m$$

inside γ. Then

$$\oint_\gamma f(z)\,dz = 2\pi i \sum_{j=1}^{m} \operatorname{Res}(f, a_j).$$

Proof. Choose disjoint small counterclockwise circles C_j around the singularities. On the region inside γ with the interiors of these circles removed, f is holomorphic. Therefore Corollary 50 gives

$$\oint_\gamma f(z)\,dz = \sum_{j=1}^{m} \oint_{C_j} f(z)\,dz.$$

By the Laurent expansion around a_j,

$$\oint_{C_j} f(z)\,dz = 2\pi i \operatorname{Res}(f, a_j).$$

Summing gives the result. □

Corollary 56 (Residue at a Pole)**.** *If a is a pole of order m of f, then*

$$\operatorname{Res}(f, a) = \frac{1}{(m-1)!} \frac{d^{m-1}}{dz^{m-1}} \left((z-a)^m f(z)\right)\Big|_{z=a}.$$

In particular, at a simple pole,

$$\operatorname{Res}(f, a) = \lim_{z \to a} (z-a) f(z).$$

Proof. Write

$$f(z) = \frac{g(z)}{(z-a)^m},$$

where g is holomorphic at a. The residue is the coefficient of $(z-a)^{-1}$, hence it is the coefficient of $(z-a)^{m-1}$ in the Taylor series of g. Therefore

$$\operatorname{Res}(f, a) = \frac{g^{(m-1)}(a)}{(m-1)!}.$$

□

5.5 Cotangent Partial Fractions

The residue theorem gives a partial fraction expansion of the cotangent.

Theorem 57 (Cotangent Partial Fraction Expansion)**.** *For every $z \notin \mathbb{Z}$,*

$$\pi \cot(\pi z) = \frac{1}{z} + 2z \sum_{n=1}^{\infty} \frac{1}{z^2 - n^2}.$$

Equivalently,

$$\pi \cot(\pi z) = \frac{1}{z} + \sum_{n=1}^{\infty} \left(\frac{1}{z-n} + \frac{1}{z+n} \right).$$

Proof. Fix $z \notin \mathbb{Z}$ and consider, as a function of w,

$$f(w) = \frac{\pi \cot(\pi w)}{w^2 - z^2}.$$

Let C_N be the positively oriented square with vertices

$$\left(N + \frac{1}{2}\right)(\pm 1 \pm i).$$

For sufficiently large N, the points $\pm z$ lie inside C_N.

The poles of f inside C_N are the integers

$$-N, -N+1, \ldots, N$$

and the two points $\pm z$.

At an integer n, the function $\pi \cot(\pi w)$ has residue 1, so

$$\operatorname{Res}(f, n) = \frac{1}{n^2 - z^2}.$$

At $w = z$,

$$\operatorname{Res}(f, z) = \frac{\pi \cot(\pi z)}{2z}.$$

Since cot is odd,

$$\operatorname{Res}(f, -z) = \frac{\pi \cot(-\pi z)}{-2z} = \frac{\pi \cot(\pi z)}{2z}.$$

Thus the residues at $\pm z$ sum to

$$\frac{\pi \cot(\pi z)}{z}.$$

It remains to show that the contour integral tends to zero. Writing $w = x + iy$,

$$|\cot(\pi w)|^2 = \frac{\cos^2(\pi x) + \sinh^2(\pi y)}{\sin^2(\pi x) + \sinh^2(\pi y)}.$$

On the vertical sides of C_N, $x = N + \frac{1}{2}$, so

$$|\cot(\pi w)| \leq 1.$$

On the horizontal sides, $|y| = N + \frac{1}{2}$, so $\cot(\pi w)$ is also uniformly bounded independently of N.

Furthermore,

$$|w^2 - z^2| = \Omega(N^2)$$

uniformly on C_N, while the length of C_N is $O(N)$. Hence

$$\oint_{C_N} f(w)\, dw = O(N^{-1}) \to 0.$$

By the residue theorem,

$$0 = \frac{\pi \cot(\pi z)}{z} + \sum_{n \in \mathbb{Z}} \frac{1}{n^2 - z^2}.$$

Therefore

$$\frac{\pi \cot(\pi z)}{z} = \sum_{n \in \mathbb{Z}} \frac{1}{z^2 - n^2}.$$

Separating the $n = 0$ term and pairing n with $-n$ gives

$$\frac{\pi \cot(\pi z)}{z} = \frac{1}{z^2} + 2\sum_{n=1}^{\infty} \frac{1}{z^2 - n^2}.$$

Multiplying by z proves

$$\pi \cot(\pi z) = \frac{1}{z} + 2z\sum_{n=1}^{\infty} \frac{1}{z^2 - n^2}.$$

Finally,

$$\frac{2z}{z^2 - n^2} = \frac{1}{z - n} + \frac{1}{z + n},$$

which gives the equivalent form. □

The pairing of n and $-n$ is important: the paired terms are $O(n^{-2})$, so the series converges on compact subsets of $\mathbb{C} \setminus \mathbb{Z}$.

5.6 Euler's Product for Sine

The sine product follows almost immediately by integrating the cotangent partial fraction expansion.

Theorem 58 (Euler's Product for Sine)**.** *For every $z \in \mathbb{C}$,*

$$\frac{\sin(\pi z)}{\pi z} = \prod_{n=1}^{\infty} \left(1 - \frac{z^2}{n^2}\right).$$

Equivalently,

$$\sin(\pi z) = \pi z \prod_{n=1}^{\infty} \left(1 - \frac{z^2}{n^2}\right).$$

Proof. From Theorem 57,

$$\pi \cot(\pi z) - \frac{1}{z} = 2z\sum_{n=1}^{\infty} \frac{1}{z^2 - n^2}.$$

The left-hand side is

$$\frac{d}{dz} \log\left(\frac{\sin(\pi z)}{\pi z}\right).$$

For every n,

$$\frac{d}{dz} \log\left(1 - \frac{z^2}{n^2}\right) = \frac{2z}{z^2 - n^2}.$$

Hence

$$\frac{d}{dz} \log\left(\frac{\sin(\pi z)}{\pi z}\right) = \sum_{n=1}^{\infty} \frac{d}{dz} \log\left(1 - \frac{z^2}{n^2}\right).$$

The series on the right converges uniformly on compact subsets avoiding the integers, since its terms are $O(n^{-2})$. Thus we may integrate term-by-term.

Near $z = 0$, choose the logarithms so that they vanish at 0. Integrating from 0 to z gives

$$\log\left(\frac{\sin(\pi z)}{\pi z}\right) = \sum_{n=1}^{\infty} \log\left(1 - \frac{z^2}{n^2}\right),$$

because

$$\lim_{z \to 0} \frac{\sin(\pi z)}{\pi z} = 1.$$

Exponentiating,

$$\frac{\sin(\pi z)}{\pi z} = \prod_{n=1}^{\infty} \left(1 - \frac{z^2}{n^2}\right)$$

in a neighborhood of 0.

The infinite product converges uniformly on compact sets, since

$$\sum_{n=1}^{\infty} \frac{|z|^2}{n^2}$$

converges uniformly when z is restricted to a compact set. Thus the product defines an entire function. Since both sides are entire and agree on a neighborhood of 0, the identity theorem extends the equality to all $z \in \mathbb{C}$. □

6 Number Theory and Arithmetic

6.1 Bertrand's Postulate and Products of Small Primes

The central binomial coefficient is exponentially large, but small prime powers account for only a limited part of its factorization. Counting all the remaining prime factors gives a lower bound on the product of small primes. Isolating those in a dyadic interval gives Bertrand's postulate and, more strongly, $\Omega(n/\log n)$ primes in $(n, 2n)$.

For real $x \geq 1$, put

$$P(x) = \prod_{p \leq x} p, \qquad \vartheta(x) = \log P(x) = \sum_{p \leq x} \log p.$$

Lemma 59. *For every real $x \geq 1$, $P(x) \leq 4^x$.*

Proof. It suffices to prove the bound for integers, since $P(x) = P(\lfloor x \rfloor)$. Induct on n, with $n = 1, 2$ immediate. If $n \geq 4$ is even, it is composite, so $P(n) = P(n-1) \leq 4^{n-1} < 4^n$.

If $n = 2m+1$ is odd, every prime $m+1 < p \leq 2m+1$ divides $\binom{2m+1}{m}$. The two middle coefficients in the binomial expansion are equal and sum to at most 2^{2m+1}, so $\binom{2m+1}{m} \leq 2^{2m}$. Therefore

$$P(2m+1) \leq P(m+1)\binom{2m+1}{m} \leq 4^{m+1}2^{2m} = 4^{2m+1}.$$

□

Lemma 60 (Central binomial coefficient bounds)**.** *For every even integer $m \geq 4$,*

$$\boxed{\frac{2^m}{m+1} \leq \binom{m}{m/2} \leq m^{\sqrt{m}} P(m/3) \frac{P(m)}{P(m/2)}.}$$

Moreover, every prime power dividing $\binom{m}{m/2}$ is at most m.

Proof. The central coefficient is the largest of the $m+1$ coefficients whose sum is 2^m, proving the lower bound.

For a prime p, Legendre's formula gives

$$v_p\binom{m}{m/2} = \sum_{j \geq 1} \left(\left\lfloor \frac{m}{p^j} \right\rfloor - 2 \left\lfloor \frac{m/2}{p^j} \right\rfloor \right).$$

Every summand is zero or one and vanishes once $p^j > m$. Thus the exponent is at most $\lfloor \log_p m \rfloor$, and the entire prime-power contribution is at most m.

There are at most $\sqrt{m}$ primes at most $\sqrt{m}$, so together they contribute at most $m^{\sqrt{m}}$. Every larger prime occurs with exponent at most one. Among these larger primes, those in $(m/3, m/2]$ do not occur at all, because

$$\left\lfloor \frac{m}{p} \right\rfloor - 2\left\lfloor \frac{m/2}{p} \right\rfloor = 2 - 2 = 0.$$

The remaining larger primes are at most $m/3$ or belong to $(m/2, m]$. Bounding their products by $P(m/3)$ and $P(m)/P(m/2)$ proves the upper bound. □

The estimate now gives both desired lower bounds. To count all primes, use $P(m/3) \le P(m/2)$. To isolate the primes in $(m/2, m]$, instead use $P(m/3) \le 4^{m/3}$. Define the relative error

$$E(x) = \frac{\log x}{\sqrt{x}} + \frac{\log(2x)}{x}.$$

Taking logarithms in the lemma and using $\log(m+1) \le \log(2m)$ gives, for every even $m \ge 4$,

$$\boxed{\begin{aligned} \vartheta(m) &\ge m\big(\log 2 - E(m)\big), \\ \vartheta(m) - \vartheta(m/2) &\ge m\big(\tfrac{1}{3}\log 2 - E(m)\big). \end{aligned}} \tag{38}$$

Both summands of $E(x)$ decrease for $x \ge 8$: their derivatives have the signs of $1 - \frac{1}{2}\log x$ and $1 - \log(2x)$, respectively. We will use the two explicit values

$$E(64) = \frac{55}{64}\log 2, \qquad E(1024) = \frac{331}{1024}\log 2 < \frac{1}{3}\log 2.$$

Theorem 61 (Bertrand's postulate, quantitatively)**.** *There is an absolute constant $c > 0$ such that, for every integer $n \ge 2$,*

$$\boxed{\#\{p \text{ prime} : n < p < 2n\} \ge c\frac{n}{\log n}.}$$

In particular, every such interval contains a prime.

Proof. For $n \ge 512$, apply the second bound in (38) with $m = 2n$:

$$\vartheta(2n) - \vartheta(n) \ge c_0 n, \qquad c_0 = 2\log 2\left(\frac{1}{3} - \frac{331}{1024}\right) > 0.$$

Since $2n$ is composite, the left side sums $\log p$ over $n < p < 2n$. Each summand is at most $\log(2n) \le 2\log n$, so

$$\#\{p : n < p < 2n\} \ge \frac{\vartheta(2n) - \vartheta(n)}{\log(2n)} \ge \frac{c_0}{2}\frac{n}{\log n}.$$

For $2 \le n < 512$, use the prime sequence

$$2, 3, 5, 7, 13, 23, 43, 83, 163, 317, 631.$$

Each term is less than twice its predecessor. If q is the first term greater than n and q' its predecessor, then $q' \le n < q < 2q' \le 2n$. Thus these intervals also contain a prime. Decreasing $c > 0$ if necessary makes the quantitative bound hold for these finitely many n as well. □

Corollary 62 (Product of small primes)**.** *For every integer $N \ge 2$,*

$$\boxed{\prod_{p \le 16\log N} p > N.}$$

Proof. Put $x = 16\log N$. If $x < 64$, then $N < e^4 < 81$, while $x \geq 16\log 2 > 7$. Hence $P(x) \geq 2\cdot 3\cdot 5\cdot 7 = 210 > N$.

Otherwise, let $m = 2\lfloor x/2 \rfloor$, so $m \geq 64$ and $m > x - 2$. The first bound in (38) gives

$$\begin{aligned}\vartheta(x) &\geq \vartheta(m) \geq m\big(\log 2 - E(64)\big)\\ &= \frac{9m}{64}\log 2 > \frac{3m}{32} > \frac{3(x-2)}{32} > \frac{x}{16} = \log N,\end{aligned}$$

using $\log 2 > 2/3$ and $x \geq 64$. Exponentiating proves the claim. □

6.2 Lucas's Theorem

Theorem 63 (Lucas's Theorem)**.** *Let p be prime, and write the base-p expansions*

$$n = n_0 + n_1 p + \cdots + n_r p^r, \qquad k = k_0 + k_1 p + \cdots + k_r p^r,$$

where $0 \leq n_i, k_i < p$. Then

$$\binom{n}{k} \equiv \prod_{i=0}^{r} \binom{n_i}{k_i} \pmod{p}.$$

Proof. Over $\mathbb{F}_p$, the binomial theorem gives

$$(1+x)^p = \sum_{j=0}^{p} \binom{p}{j} x^j \equiv 1 + x^p \pmod{p},$$

since $p \mid \binom{p}{j}$ for $0 < j < p$. Iterating,

$$(1+x)^{p^i} \equiv 1 + x^{p^i} \pmod{p}.$$

Therefore

$$\begin{aligned}(1+x)^n &= \prod_i (1+x)^{n_i p^i}\\ &\equiv \prod_i (1+x^{p^i})^{n_i} \pmod{p}.\end{aligned}$$

Expanding the right-hand side,

$$\prod_i \left(\sum_{j=0}^{n_i} \binom{n_i}{j} x^{jp^i} \right).$$

Since every $j < p$, the exponent

$$k = \sum_i k_i p^i$$

has a unique base-p representation. Thus the only way to obtain x^k is to choose $x^{k_i p^i}$ from the i-th factor. Hence

$$[x^k](1+x)^n \equiv \prod_i \binom{n_i}{k_i} \pmod{p}.$$

Since $[x^k](1+x)^n = \binom{n}{k}$, the result follows. □

Corollary 64. *With the same notation,*

$$\binom{n}{k} \not\equiv 0 \pmod{p} \iff k_i \le n_i \quad \textit{for every } i.$$

Consequently, the number of entries in the n-th row of Pascal's triangle that are non-zero modulo p is

$$\prod_i (n_i + 1).$$

Proof. Because $0 \le n_i, k_i < p$,

$$\binom{n_i}{k_i} \not\equiv 0 \pmod{p}$$

exactly when $k_i \le n_i$. Lucas's theorem gives the first claim. For the second, each digit k_i may independently be chosen from

$$0, 1, \ldots, n_i.$$

□

6.3 Quadratic Reciprocity

For an odd prime p, define the Legendre symbol by

$$\left(\frac{a}{p}\right) = \begin{cases} 0, & p \mid a, \\ 1, & a \not\equiv 0 \pmod{p} \text{ is a square}, \\ -1, & a \text{ is not a square.} \end{cases}$$

We first prove the fact that will give both quadratic reciprocity and its supplementary laws.

Lemma 65 (Euler's Criterion)**.** *For p an odd prime,*

$$a^{(p-1)/2} \equiv \left(\frac{a}{p}\right) \pmod{p}.$$

Proof. Assume $a \not\equiv 0 \pmod{p}$. There are exactly $(p-1)/2$ non-zero squares modulo p, since x and $-x$ have the same square.

Every square $a = x^2$ satisfies

$$a^{(p-1)/2} = x^{p-1} \equiv 1 \pmod{p}.$$

Thus the $(p-1)/2$ non-zero squares are exactly the roots of

$$x^{(p-1)/2} - 1.$$

If a is not a square, then $a^{(p-1)/2} \neq 1$, while Fermat's little theorem gives

$$\left(a^{(p-1)/2}\right)^2 = a^{p-1} \equiv 1.$$

Hence $a^{(p-1)/2} \equiv -1$. □

Lemma 66 (Gauss's Lemma)**.** *Let p be an odd prime, $p \nmid a$, and set $m = (p-1)/2$. Consider*

$$a, 2a, \ldots, ma$$

modulo p, represented by integers in $\{1, \ldots, p-1\}$. Let N be the number of these residues that lie in $(p/2, p)$. Then

$$\left(\frac{a}{p}\right) = (-1)^N.$$

Proof. For each $1 \le j \le m$, replace the residue of aj by its representative up to sign in

$$\{1, \dots, m\}.$$

These absolute representatives are all distinct: if

$$aj \equiv \pm ak \pmod{p},$$

then

$$j \equiv \pm k \pmod{p},$$

and for $1 \le j, k \le m$ this forces $j = k$.

Thus they are exactly a permutation of $1, \dots, m$. A residue contributes a minus sign precisely when it lies in $(p/2, p)$, so

$$a^m m! \equiv (-1)^N m! \pmod{p}.$$

Canceling $m!$ gives

$$a^{(p-1)/2} \equiv (-1)^N \pmod{p}.$$

Now apply Lemma 65. □

Corollary 67 (First Supplement)**.** *For every odd prime p,*

$$\left(\frac{-1}{p}\right) = (-1)^{(p-1)/2}.$$

Equivalently, -1 is a quadratic residue modulo p if and only if

$$p \equiv 1 \pmod{4}.$$

Proof. Multiplication by -1 sends every

$$1, \dots, \frac{p-1}{2}$$

to a residue greater than $p/2$. Thus $N = (p-1)/2$ in Gauss's lemma. □

Corollary 68 (Second Supplement)**.** *For every odd prime p,*

$$\left(\frac{2}{p}\right) = (-1)^{(p^2-1)/8}.$$

Equivalently, 2 is a quadratic residue modulo p exactly when

$$p \equiv \pm 1 \pmod{8}.$$

Proof. For

$$1 \le j \le \frac{p-1}{2},$$

the numbers $2j$ already lie in $\{1, \dots, p-1\}$. They lie above $p/2$ exactly when

$$j > \frac{p}{4}.$$

Hence

$$N = \frac{p-1}{2} - \left\lfloor \frac{p}{4} \right\rfloor.$$

Checking the four possibilities $p \equiv 1, 3, 5, 7 \pmod{8}$, this number is even for $p \equiv 1, 7 \pmod{8}$ and odd for $p \equiv 3, 5 \pmod{8}$. Thus

$$(-1)^N = (-1)^{(p^2-1)/8}.$$

Now apply Gauss's lemma. □

For quadratic reciprocity, it is useful to rewrite Gauss's lemma as a lattice-point count.

Lemma 69 (Eisenstein's Form of Gauss's Lemma)**.** *Let p be an odd prime and let a be odd with $p \nmid a$. Then*

$$\left(\frac{a}{p}\right) = (-1)^{\displaystyle\sum_{j=1}^{(p-1)/2} \left\lfloor \frac{aj}{p} \right\rfloor}.$$

Proof. Set $m = (p-1)/2$ and write

$$aj = pq_j + r_j, \qquad 1 \le r_j \le p-1.$$

Let

$$s_j = \min(r_j, p - r_j) \in \{1, \dots, m\}.$$

As in the proof of Gauss's lemma, the s_j are a permutation of $1, \dots, m$.

Let $\epsilon_j = 0$ if $r_j \le m$ and $\epsilon_j = 1$ otherwise. Since a and p are odd, reducing

$$aj = pq_j + r_j$$

modulo 2 gives

$$j \equiv q_j + s_j + \epsilon_j \pmod 2.$$

Summing over j, the sums of the j's and the s_j's cancel modulo 2, since (s_j) is a permutation of (j). Therefore

$$\sum_j q_j \equiv \sum_j \epsilon_j = N \pmod 2.$$

Since $q_j = \lfloor aj/p \rfloor$, Gauss's lemma gives the result. □

Theorem 70 (Quadratic Reciprocity)**.** *For distinct odd primes p and q,*

$$\left(\frac{p}{q}\right)\left(\frac{q}{p}\right) = (-1)^{\frac{(p-1)(q-1)}{4}}.$$

Equivalently,

$$\left(\frac{p}{q}\right) = \left(\frac{q}{p}\right)$$

unless

$$p \equiv q \equiv 3 \pmod 4,$$

in which case the two symbols have opposite signs.

Proof. Set

$$m = \frac{p-1}{2}, \qquad n = \frac{q-1}{2}.$$

By Lemma 69,

$$\left(\frac{q}{p}\right) = (-1)^A, \qquad A = \sum_{x=1}^{m} \left\lfloor \frac{qx}{p} \right\rfloor,$$

and

$$\left(\frac{p}{q}\right) = (-1)^B, \qquad B = \sum_{y=1}^{n} \left\lfloor \frac{py}{q} \right\rfloor.$$

Consider the mn lattice points

$$(x, y), \qquad 1 \le x \le m, \quad 1 \le y \le n.$$

The number below the line

$$py = qx$$

is exactly

$$A,$$

because for a fixed x there are

$$\left\lfloor \frac{qx}{p} \right\rfloor$$

possible positive y.

Similarly, the number above the line is exactly B.

No lattice point lies on the line: $py = qx$ would imply $p \mid x$, impossible since $1 \le x < p$. Thus every point is either above or below, and

$$A + B = mn = \frac{(p-1)(q-1)}{4}.$$

Consequently,

$$\left(\frac{q}{p}\right)\left(\frac{p}{q}\right) = (-1)^{A+B} = (-1)^{\frac{(p-1)(q-1)}{4}}.$$

□

6.4 Values of the Zeta Function at Even Integers

The formula for

$$\zeta(2k) = \sum_{n=1}^{\infty} \frac{1}{n^{2k}}$$

comes from expanding the same function in two different ways.

Expanding the partial fractions of $\cot(\pi z)$ produces the sums $\zeta(2k)$.

On the other hand, $\cot z$ can be written using the exponential function, whose expansion is naturally encoded by the Bernoulli numbers.

Define the Bernoulli numbers by

$$\boxed{\frac{t}{e^t - 1} = \sum_{m=0}^{\infty} B_m \frac{t^m}{m!}.}$$

Thus

$$B_0 = 1, \qquad B_1 = -\frac{1}{2}.$$

Theorem 71 (Euler)**.** *For every integer $k \ge 1$,*

$$\boxed{\zeta(2k) = (-1)^{k+1} \frac{B_{2k}(2\pi)^{2k}}{2(2k)!}.}$$

Proof. We compute the power series of

$$\pi z \cot(\pi z)$$

in two ways.

From the poles of cotangent. The partial-fraction expansion

$$\pi \cot(\pi z) = \frac{1}{z} + \sum_{n=1}^{\infty} \frac{2z}{z^2 - n^2}$$

gives

$$\pi z \cot(\pi z) = 1 - 2\sum_{n=1}^{\infty} \frac{z^2}{n^2 - z^2}.$$

For $|z| < 1$,

$$\frac{z^2}{n^2 - z^2} = \frac{z^2/n^2}{1 - z^2/n^2} = \sum_{k=1}^{\infty} \frac{z^{2k}}{n^{2k}}.$$

Hence

$$\begin{aligned} \pi z \cot(\pi z) &= 1 - 2\sum_{n=1}^{\infty}\sum_{k=1}^{\infty} \frac{z^{2k}}{n^{2k}} \\ &= 1 - 2\sum_{k=1}^{\infty} \left(\sum_{n=1}^{\infty} \frac{1}{n^{2k}} \right) z^{2k}. \end{aligned}$$

Therefore

$$\boxed{\pi z \cot(\pi z) = 1 - 2\sum_{k=1}^{\infty} \zeta(2k) z^{2k}.} \tag{1}$$

The interchange of the sums is justified for $|z| < 1$, where the series is absolutely convergent.

From the Bernoulli generating function. Using

$$\cot x = i\frac{e^{2ix} + 1}{e^{2ix} - 1} = i + \frac{2i}{e^{2ix} - 1},$$

we obtain

$$x \cot x = ix + \frac{2ix}{e^{2ix} - 1}.$$

Set

$$t = 2ix.$$

By the Bernoulli generating function,

$$\frac{2ix}{e^{2ix} - 1} = \sum_{m=0}^{\infty} B_m \frac{(2ix)^m}{m!}.$$

Thus

$$x \cot x = ix + \sum_{m=0}^{\infty} B_m \frac{(2ix)^m}{m!}.$$

The $m = 1$ term is

$$B_1(2ix) = -\frac{1}{2}(2ix) = -ix,$$

so it cancels the initial ix.

Moreover, $x \cot x$ is even, so all remaining odd coefficients vanish. Thus

$$x \cot x = \sum_{k=0}^{\infty} B_{2k} \frac{(2ix)^{2k}}{(2k)!}.$$

Since

$$i^{2k} = (-1)^k,$$

this becomes

$$\boxed{x \cot x = \sum_{k=0}^{\infty} (-1)^k \frac{2^{2k} B_{2k}}{(2k)!} x^{2k}.}$$

Substituting $x = \pi z$,

$$\boxed{\pi z \cot(\pi z) = \sum_{k=0}^{\infty} (-1)^k \frac{B_{2k} (2\pi)^{2k}}{(2k)!} z^{2k}.} \tag{2}$$

Finally compare the coefficient of z^{2k} in (1) and (2). For $k \geq 1$,

$$-2\zeta(2k) = (-1)^k \frac{B_{2k} (2\pi)^{2k}}{(2k)!}.$$

Therefore

$$\boxed{\zeta(2k) = (-1)^{k+1} \frac{B_{2k} (2\pi)^{2k}}{2(2k)!}.}$$

□

6.5 Hensel's Lemma

A root modulo p^k already determines the first k base-p digits of a possible root modulo a higher power. To find the next digits, change a to $a + p^k t$. Modulo p^{2k}, every nonlinear term in this correction vanishes, leaving one linear equation for t. We generalize this to commutative rings.

Throughout, R is a commutative ring with identity, and congruence modulo an ideal means that the difference belongs to that ideal.

Theorem 72 (Hensel lifting modulo an ideal)**.** *Let $I \subseteq R$ be an ideal, $f \in R[X]$, and $a \in R$. Suppose $f(a) \in I$ and $f'(a)$ is invertible modulo I. Choose $u \in R$ with $uf'(a) \equiv 1 \pmod{I}$. Then there is a unique residue class $b \pmod{I^2}$ satisfying*

$$f(b) \equiv 0 \pmod{I^2}, \qquad b \equiv a \pmod{I}.$$

It is given by

$$\boxed{b \equiv a - uf(a) \pmod{I^2}.}$$

Proof. For any correction $h \in I$, the binomial theorem gives

$$f(a + h) \equiv f(a) + f'(a)h \pmod{I^2},$$

because every remaining term contains at least two factors of h.

Take $h = -uf(a) \in I$. Then

$$f(a + h) \equiv (1 - uf'(a))f(a) \equiv 0 \pmod{I^2},$$

since both factors belong to I. This proves existence.

Conversely, if $h \in I$ gives a root modulo I^2, multiply its linearized equation by u. Since $(uf'(a) - 1)h \in I^2$, we obtain

$$0 \equiv uf(a) + uf'(a)h \equiv uf(a) + h \pmod{I^2}.$$

Thus $h \equiv -uf(a) \pmod{I^2}$, proving uniqueness. □

Lift the inverse at the same time. The next step needs an inverse of $f'(b)$ modulo I^2. There is no need to compute it from scratch. Since $b \equiv a \pmod{I}$, the old u remains an inverse modulo I. Put $e = 1 - uf'(b) \in I$ and set

$$\boxed{u^+ = u(2 - uf'(b)) = u(1 + e).}$$

Its new error is

$$1 - u^+ f'(b) = 1 - (1 + e)(1 - e) = e^2 \in I^2.$$

Thus both the root and its inverse derivative have been lifted.

Starting with $f(a_0) \in I$ and $1 - u_0 f'(a_0) \in I$, iterate

$$\boxed{\begin{aligned} a_{j+1} &= a_j - u_j f(a_j), \\ u_{j+1} &= u_j\big(2 - u_j f'(a_{j+1})\big). \end{aligned}}$$

Applying the theorem with I^{2^j} in place of I proves inductively that

$$f(a_j) \in I^{2^j}, \qquad 1 - u_j f'(a_j) \in I^{2^j}, \qquad a_{j+1} - a_j \in I^{2^j}.$$

Each iteration therefore doubles the ideal-adic precision. For computations modulo I^N, reduce every operation modulo the next required precision $I^{\min\{2^{j+1}, N\}}$. Starting modulo I, at most $\lceil \log_2 N \rceil$ iterations suffice.

Prime powers. For $R = \mathbb{Z}$ and $I = (p)$ with p prime, invertibility of $f'(a)$ modulo I is exactly $p \nmid f'(a)$, recovering the usual prime-power statement.

For $R = \mathbb{F}[[t]]$ and $I = (t)$, congruence modulo I^k means agreement of coefficients below degree k. The same update doubles the number of known coefficients. More generally, for $R = \mathbb{F}[[t_1, \ldots, t_r]]$ and $I = (t_1, \ldots, t_r)$, it doubles precision in total degree.

6.6 Arithmetic Algorithms

6.6.1 Linear Sieve

The linear sieve is distinguished from the ordinary Eratosthenes sieve by the fact that every composite integer is generated exactly once.

This also makes it convenient for evaluating arbitrary multiplicative functions.

Suppose f is multiplicative. We assume access to two operations:

- PrimePower(p, e) computes $f(p^e)$ at cost G;
- Combine$(f(a), f(b))$ computes $f(ab)$ when $\gcd(a, b) = 1$, at cost F.

For each integer i, let

$$\mathrm{lp}[i]$$

be its least prime factor. We also maintain

$$e[i] = v_{\mathrm{lp}[i]}(i), \qquad q[i] = \mathrm{lp}[i]^{e[i]}.$$

Thus

$$i = q[i] \cdot r \qquad \text{with} \qquad \gcd(q[i], r) = 1.$$

Algorithm 1 Linear Sieve for Multiplicative Functions

Input: Integer n
Output: Primes below n, least prime factors, and $f(1), \ldots, f(n-1)$

```
f[1] ← f(1)
primes ← ∅
Initialize lp, e, q to zero
for i = 2, ..., n − 1 do
    if lp[i] = 0 then
        lp[i] ← i
        e[i] ← 1
        q[i] ← i
        append i to primes
        f[i] ← PrimePower(i, 1)
    for p in primes do
        if ip ≥ n then
            break
        lp[ip] ← p
        if p = lp[i] then
            e[ip] ← e[i] + 1
            q[ip] ← q[i]p
            if q[ip] = ip then
                f[ip] ← PrimePower(p, e[ip])
            else
                f[ip] ← Combine(f[ip/q[ip]], f[q[ip]])
            break
        else
            e[ip] ← 1
            q[ip] ← p
            f[ip] ← Combine(f[i], f[p])
```

Lemma 73. *Every composite integer $x < n$ is generated exactly once by Algorithm 1.*

Proof. Let

$$p = \mathrm{lp}[x], \qquad i = x/p.$$

Then every prime factor of i is at least p, so

$$p \le \mathrm{lp}[i].$$

Therefore the inner loop for i reaches p before it terminates, and generates $x = ip$.

Conversely, the sieve only generates ip while

$$p \le \mathrm{lp}[i].$$

Thus p is the least prime factor of ip. Hence if x is generated as ip, necessarily

$$p = \mathrm{lp}[x], \qquad i = x/p,$$

which is unique. □

Theorem 74. *Algorithm 1 correctly computes any multiplicative function f on $[1, n)$ in*

$$O\left(n + nF + \frac{n}{\log n} G\right)$$

time.

Proof. By the preceding lemma, the total number of inner-loop iterations is $O(n)$.

Whenever $p < \mathrm{lp}[i]$, we have $\gcd(i, p) = 1$, so

$$f(ip) = \mathrm{Combine}(f(i), f(p)).$$

When $p = \mathrm{lp}[i]$, write

$$i = p^e r, \qquad \gcd(p, r) = 1.$$

Then

$$ip = p^{e+1} r,$$

and the algorithm computes

$$f(ip) = \mathrm{Combine}\big(f(r), f(p^{e+1})\big).$$

The value $f(p^{e+1})$ has already been computed unless $ip = p^{e+1}$ itself is a pure prime power; in that case it is computed directly by PrimePower.

Thus PrimePower is called exactly once for each prime power below n.

There are

$$\pi(n) + O(\sqrt{n}\log n) = O\left(\frac{n}{\log n}\right)$$

prime powers below n. Indeed, the powers with exponent at least two contribute $O(\sqrt{n}\log n)$, while

$$\pi(n) = O\left(\frac{n}{\log n}\right)$$

follows already from Lemma 59: since

$$\sum_{p \le n} \log p \le n \log 4,$$

the primes larger than $\sqrt{n}$ number at most

$$\frac{2n \log 4}{\log n},$$

and there are at most $\sqrt{n}$ smaller primes.

Hence there are $O(n)$ calls to Combine and $O(n/\log n)$ calls to PrimePower, giving

$$O\left(n + nF + \frac{n}{\log n} G\right).$$

□

For ordinary multiplicative functions, Combine is simply multiplication. The only function-specific information needed is therefore the prime-power formula. For example,

$$\varphi(p^e) = p^e - p^{e-1}, \qquad \mu(p^e) = \begin{cases} -1, & e = 1, \\ 0, & e \ge 2, \end{cases}$$

and

$$\sigma_k(p^e) = 1 + p^k + p^{2k} + \cdots + p^{ek}.$$

6.6.2 Extended GCD with Preprocessing

Given integers a, b, we want integers x, y satisfying

$$ax + by = \gcd(a, b).$$

Ordinary extended Euclid takes $O(\log N)$ arithmetic operations for $|a|, |b| \le N$. With preprocessing, we can batch many Euclidean steps.

The basic observation is that after scaling a pair (u, v) to a fixed box, we only have to consider $O(T)$ reductions: one coordinate lies on the boundary of the box.

For an integer $T \ge 2$, define

$$\mathcal{B}_T = \{(p, q) \in \mathbb{Z}^2 : \max(|p|, |q|) = T\}.$$

Then

$$|\mathcal{B}_T| = O(T).$$

Lemma 75 (Short boundary reduction)**.** *For every $(p, q) \in \mathcal{B}_T$, there is a unimodular matrix $R \in \mathrm{SL}_2(\mathbb{Z})$ whose entries have magnitude $O(\sqrt{T})$ and such that the first coordinate of*

$$R\begin{pmatrix} p \\ q \end{pmatrix}$$

has magnitude $O(\sqrt{T})$.

Proof. After swapping coordinates and changing signs, assume $|q| = T$.

Set

$$Q = \lceil \sqrt{T} \rceil.$$

Consider the $Q + 1$ fractional parts

$$0, \frac{p}{q}, \frac{2p}{q}, \ldots, \frac{Qp}{q}.$$

Partition $[0, 1)$ into Q intervals of length $1/Q$. Two fractional parts lie in the same interval, so for some $1 \le r \le Q$ and some integer s,

$$|rp - sq| \le \frac{|q|}{Q} = O(\sqrt{T}).$$

Also,

$$|s| \le \left| \frac{rp}{q} \right| + 1 = O(\sqrt{T}).$$

Dividing (r, s) by $\gcd(r, s)$ only makes these bounds smaller, so assume $\gcd(r, s) = 1$. Then the row vector $(r, -s)$ can be completed to a unimodular matrix

$$R = \begin{pmatrix} r & -s \\ u & v \end{pmatrix}.$$

The Bézout coefficients u, v may be chosen with magnitude $O(\max(|r|, |s|)) = O(\sqrt{T})$.

Its first output coordinate is

$$rp - sq = O(\sqrt{T}),$$

as required. □

Precompute one such matrix $R_{p,q}$ for every $(p, q) \in \mathcal{B}_T$. The table contains only $O(T)$ matrices.

Lemma 76 (One lookup shrinks the pair)**.** *Let*

$$z = \begin{pmatrix} u \\ v \end{pmatrix} \neq 0, \qquad B = \max(|u|, |v|).$$

Using one lookup in the boundary table followed by one Euclidean division, we can apply a unimodular transformation sending z to z' with

$$\|z'\|_\infty = O\left(\frac{B}{\sqrt{T}}\right).$$

Proof. Round the normalized pair to the boundary:

$$p = \left\lfloor \frac{Tu}{B} \right\rceil, \qquad q = \left\lfloor \frac{Tv}{B} \right\rceil.$$

Because one of $|u|, |v|$ equals B, one of $|p|, |q|$ equals T, so

$$(p, q) \in \mathcal{B}_T.$$

Moreover,

$$z = \frac{B}{T}\begin{pmatrix} p \\ q \end{pmatrix} + e, \qquad \|e\|_\infty = O(B/T).$$

Let $R = R_{p,q}$. By Lemma 75, the first coordinate of $R(p, q)^T$ is $O(\sqrt{T})$ and every entry of R is $O(\sqrt{T})$. Thus the first coordinate of Rz is

$$O\left(\frac{B}{T}\sqrt{T}\right) + O\left(\sqrt{T}\frac{B}{T}\right) = O\left(\frac{B}{\sqrt{T}}\right).$$

Call this coordinate w_1 and the other coordinate w_2. If $w_1 = 0$, the gcd has already been computed. Otherwise choose

$$Q = \left\lfloor \frac{w_2}{w_1} \right\rceil$$

and apply

$$\begin{pmatrix} 1 & 0 \\ -Q & 1 \end{pmatrix}.$$

The second coordinate becomes

$$w_2 - Qw_1$$

and has magnitude at most $|w_1|/2$. Hence both coordinates are

$$O(B/\sqrt{T}).$$

□

The extended coefficients are maintained simply by accumulating the unimodular transformations.

Algorithm 2 Preprocessed Extended GCD

Input: Integers a, b, boundary table for parameter T
Output: (x, y) satisfying $ax + by = \gcd(a, b)$

$z \leftarrow (a, b)^T$
$P \leftarrow I$
while $z_1 \neq 0$ and $z_2 \neq 0$ **do**
  $B \leftarrow \max(|z_1|, |z_2|)$
  $p \leftarrow \lfloor Tz_1/B \rceil$
  $q \leftarrow \lfloor Tz_2/B \rceil$
  $R \leftarrow R_{p,q}$
  $z \leftarrow Rz$
  $P \leftarrow RP$
  if $z_1 = 0$ or $z_2 = 0$ **then**
    break
  $Q \leftarrow \lfloor z_2/z_1 \rceil$
  $E \leftarrow \begin{pmatrix} 1 & 0 \\ -Q & 1 \end{pmatrix}$
  $z \leftarrow Ez$
  $P \leftarrow EP$
if $z_1 \neq 0$ **then**
  return $\operatorname{sgn}(z_1)$ times the first row of P
else
  return $\operatorname{sgn}(z_2)$ times the second row of P

Theorem 77. *Suppose*

$$0 < \max(|a|, |b|) \leq N.$$

After storing an $O(T)$-entry boundary table, Algorithm 2 uses

$$O\left(\frac{\log N}{\log T}\right)$$

operations.

Proof. At all times,

$$z = P(a, b)^T$$

with P unimodular, so

$$\gcd(z_1, z_2) = \gcd(a, b).$$

By Lemma 76, every nonterminal iteration decreases

$$B = \|z\|_\infty$$

by a factor $\Theta(\sqrt{T})$. Hence after s iterations,

$$B \leq N \cdot O(T^{-s/2}).$$

Since the coordinates are integers, after

$$s = O(\log N / \log T)$$

iterations one coordinate must become zero.

If

$$z = (g, 0)^T,$$

then the first row (x, y) of P satisfies

$$ax + by = g,$$

and $|g| = \gcd(a, b)$. The other case is identical. □

Thus, for example,

$$T = N^{\varepsilon}$$

gives $O(N^{\varepsilon})$ stored table entries and

$$O(1/\varepsilon)$$

query iterations, while

$$T = (\log N)^c$$

gives polylogarithmic space and

$$O\left(\frac{\log N}{\log \log N}\right)$$

query iterations.

6.6.3 Rational Reconstruction

Chinese remaindering or Hensel lifting may give a rational answer only through its residue modulo a large integer. Small numerator and denominator bounds make that residue sufficient to recover the exact fraction. Extended Euclid performs the recovery: its remainder is a candidate numerator, and the corresponding coefficient of the residue is a candidate denominator.

Given $M \geq 2$ and a residue $x \pmod M$, seek integers a, b satisfying

$$|a| \leq A, \qquad 1 \leq b \leq B, \qquad \gcd(a, b) = 1, \qquad a \equiv bx \pmod M,$$

where $A \geq 0$, $B \geq 1$, and $2AB < M$. The congruence and coprimality already imply $\gcd(b, M) = 1$: any common divisor of b and M also divides a. Thus the congruence really says that a/b has residue x.

Lemma 78 (Uniqueness)**.** *Under $2AB < M$, there is at most one such reduced fraction.*

Proof. If a/b and c/d both qualify, then $M \mid ad - bc$, whereas

$$|ad - bc| \leq |a|d + |c|b \leq 2AB < M.$$

Therefore $ad = bc$. Reducedness and positive denominators make the two pairs identical. □

Reduce x to $0 \leq x < M$. Start from $(r_0, t_0) = (M, 0)$ and $(r_1, t_1) = (x, 1)$, and apply the same Euclidean update to both coordinates:

$$(r_0, r_1) \leftarrow (r_1, r_0 - qr_1), \qquad (t_0, t_1) \leftarrow (t_1, t_0 - qt_1), \qquad q = \lfloor r_0/r_1 \rfloor.$$

Every pair satisfies $r_i \equiv t_i x \pmod M$. The nonnegative remainders decrease while the nonzero t_i alternate in sign and increase in absolute value. Stop as soon as the current remainder fits the numerator bound, and check the denominator and coprimality bounds.

Algorithm 3 Rational reconstruction by stopping extended Euclid early

Input: $M \geq 2$, integer x, $A \geq 0$, $B \geq 1$, and $2AB < M$
Output: The unique reduced pair (a, b) in the stated bounds, or Fail

$x \leftarrow x \bmod M$, with $0 \leq x < M$
if $x = 0$ **then**
  return $(0, 1)$
if $A = 0$ **then**
  return Fail
$(r_0, r_1) \leftarrow (M, x)$; $(t_0, t_1) \leftarrow (0, 1)$
while $r_1 > A$ **do**
  $q \leftarrow \lfloor r_0 / r_1 \rfloor$
  $(r_0, r_1) \leftarrow (r_1, r_0 - q r_1)$
  $(t_0, t_1) \leftarrow (t_1, t_0 - q t_1)$
if $|t_1| > B$ or $\gcd(r_1, |t_1|) \neq 1$ **then**
  return Fail
return $(\mathrm{sgn}(t_1) r_1, |t_1|)$

The loop never divides by zero: it performs a step only while $r_1 > A \geq 0$. The coefficient t_1 is always nonzero. Correctness of an accepted pair is immediate from the invariants. The real question is why stopping at the *first* small remainder cannot miss a solution.

A smaller denominator cannot yet have a small remainder. The relevant congruence pairs form the integer lattice

$$\mathcal{L} = \{(u, v) \in \mathbb{Z}^2 : u \equiv vx \pmod{M}\}.$$

Initially $(M, 0), (x, 1)$ are a basis: every pair is $((u - vx)/M)(M, 0) + v(x, 1)$. A Euclidean update replaces a basis by another integer basis, since subtracting an integer multiple and swapping are reversible over $\mathbb{Z}$.

Lemma 79 (Euclidean exclusion bound)**.** *For consecutive pairs $(r_0, t_0), (r_1, t_1)$ in the algorithm, if $(u, v) \in \mathcal{L}$ and $0 < |v| < |t_1|$, then $|u| \geq r_0$.*

Proof. Express $(u, v) = \alpha(r_0, t_0) + \beta(r_1, t_1)$ with integers α, β. We cannot have $\alpha = 0$, because then a nonzero v would have magnitude at least $|t_1|$. Negate the pair if necessary so that $\alpha > 0$; the hypotheses and desired conclusion use absolute values and are unchanged.

The coefficients t_0, t_1 have opposite signs, except that initially $t_0 = 0$. If $\beta < 0$, then αt_0 and βt_1 have the same sign, or the first is zero. Their sum has magnitude at least $|t_1|$, a contradiction. Hence $\beta \geq 0$, and $u = \alpha r_0 + \beta r_1 \geq r_0$. □

Theorem 80 (Rational reconstruction)**.** *Algorithm 3 returns the unique reduced fraction with $|a| \leq A$, $1 \leq b \leq B$, and $a \equiv bx \pmod{M}$, or correctly reports that none exists. It uses $O(\log M)$ Euclidean steps and a constant number of $O(\log M)$-bit working integers.*

Proof. The cases $x = 0$ and $A = 0$ follow from uniqueness and the fact that a reduced fraction with numerator zero has denominator one. Otherwise, at the stopping point $r_0 > A \geq r_1 \geq 0$. Suppose a valid (a, b) exists. The exclusion bound implies $|t_1| \leq b \leq B$, since $b < |t_1|$ would force $|a| \geq r_0 > A$.

Both pairs satisfy the congruence, so $M \mid at_1 - br_1$. Their bounds give

$$|at_1 - br_1| \leq A|t_1| + br_1 \leq 2AB < M.$$

Thus $at_1 = br_1$. Since (a, b) is primitive, this implies $(r_1, t_1) = \lambda(a, b)$ for an integer λ. It is nonzero because $t_1 \neq 0$, and $|t_1| \leq b$ forces $|\lambda| = 1$. Consequently the stopping pair passes both checks and normalization returns exactly (a, b). If either check fails, no valid pair exists.

For the cost, remainders at least halve every two Euclidean divisions: if $r_1 \le r_0/2$ this is immediate, and otherwise the next remainder is $r_0 - r_1 < r_0/2$. Hence the loop has $O(\log M)$ iterations. The final gcd test has the same step bound. Consecutive pairs also satisfy

$$|r_0 t_1 - r_1 t_0| = M, \qquad M = r_0|t_1| + r_1|t_0|,$$

because their determinant is preserved up to sign and the coefficients alternate in sign. Since $r_0 \ge 1$, all stored t_i have magnitude at most M, as do the remainders. This proves the storage claim. □

Bounds and certification. A symmetric choice satisfying the *strict* uniqueness condition is

$$A = B = \left\lfloor \sqrt{\frac{M-1}{2}} \right\rfloor \qquad (M \ge 3).$$

The denominator check must not be omitted. For example, with $M = 5$, $x = 2$, and $A = B = 1$, Euclid stops at $(r_1, t_1) = (1, -2)$. Its fraction $-1/2$ has the correct residue, but its denominator is outside the prescribed range, so the answer is Fail.

When numerator and denominator bounds are known, accumulate CRT moduli or precision using Hensel lifting until $M > 2AB$, then reconstruct.

6.6.4 Modular Square Roots

Let p be an odd prime. Given $a \in \mathbb{F}_p$, we want to find

$$x^2 = a,$$

or determine that no solution exists.

If $p \equiv 3 \pmod 4$, Euler's criterion immediately gives the particularly simple formula

$$x = a^{(p+1)/4},$$

since

$$x^2 = a^{(p+1)/2} = a \cdot a^{(p-1)/2} = a.$$

For general odd primes, Cipolla's algorithm gives an equally simple $O(\log p)$ expected-time algorithm.

Lemma 81. *Let d be a quadratic non-residue modulo p, and let*

$$K = \mathbb{F}_p[\omega]/(\omega^2 - d).$$

Then

$$\omega^p = -\omega.$$

Consequently,

$$(u + v\omega)^p = u - v\omega.$$

Proof. By Euler's criterion,

$$d^{(p-1)/2} = -1.$$

Thus

$$\omega^p = \omega(\omega^2)^{(p-1)/2} = \omega d^{(p-1)/2} = -\omega.$$

Also, in characteristic p,

$$(u + v\omega)^p = u^p + v^p \omega^p = u - v\omega.$$

□

Theorem 82 (Cipolla's Algorithm)**.** *Suppose $a \neq 0$ is a quadratic residue modulo the odd prime p. Choose $t \in \mathbb{F}_p$ such that*

$$d = t^2 - a$$

is a quadratic non-residue. In

$$K = \mathbb{F}_p[\omega]/(\omega^2 - d),$$

define

$$x = (t + \omega)^{(p+1)/2}.$$

Then $x \in \mathbb{F}_p$ and

$$x^2 = a.$$

Proof. By Lemma 81,

$$(t + \omega)^p = t - \omega.$$

Therefore

$$x^2 = (t + \omega)^{p+1} = (t + \omega)(t - \omega) = t^2 - \omega^2 = t^2 - d = a.$$

Furthermore,

$$x^{p-1} = (x^2)^{(p-1)/2} = a^{(p-1)/2} = 1$$

by Euler's criterion. Hence

$$x^p = x,$$

so x is fixed by Frobenius and therefore lies in $\mathbb{F}_p$. □

Arithmetic in the quadratic extension is easily computable:

$$(u + v\omega)(r + s\omega) = (ur + dvs) + (us + vr)\omega.$$

Thus the exponentiation in Theorem 82 is ordinary binary exponentiation, using only $O(\log p)$ operations in $\mathbb{F}_p$.

A random choice of t succeeds with constant probability. In fact, exactly $(p-1)/2$ values of t make $t^2 - a$ a non-residue.

Proof. Let χ denote the Legendre symbol. The number of pairs (t, y) satisfying

$$y^2 = t^2 - a$$

is $p - 1$: indeed,

$$(t - y)(t + y) = a,$$

and every choice of nonzero $u = t - y$ uniquely determines

$$t + y = a/u.$$

On the other hand, for fixed t, the number of choices of y is

$$1 + \chi(t^2 - a).$$

Hence

$$\sum_{t \in \mathbb{F}_p} \chi(t^2 - a) = -1.$$

Since a is a nonzero square, exactly two choices of t give $t^2 - a = 0$. If R and S denote the numbers giving a nonzero residue and a non-residue, then

$$R + S = p - 2, \qquad R - S = -1.$$

Thus

$$S = \frac{p-1}{2}.$$

□

Algorithm 4 Modular Square Root

Input: Odd prime p, $a \in \mathbb{F}_p$
Output: x with $x^2 = a$, or None if no root exists

if $a = 0$ **then**
  return 0
if $a^{(p-1)/2} \neq 1$ **then**
  return None
if $p \equiv 3 \pmod 4$ **then**
  return $a^{(p+1)/4}$
repeat
  Choose $t \in \mathbb{F}_p$ uniformly at random
  $d \leftarrow t^2 - a$
  if $d = 0$ **then**
    return t
until $d^{(p-1)/2} = -1$
Work in $\mathbb{F}_p[\omega]/(\omega^2 - d)$
$x \leftarrow (t + \omega)^{(p+1)/2}$
return x

The expected number of choices of t is $O(1)$, and every test and the final exponentiation take $O(\log p)$ field operations. Thus the expected running time is $O(\log p)$.

6.6.5 Generalized Floor Sum

For nonnegative integers a, b, n, r, s and $m > 0$, define

$$F_{r,s}(a, b; m, n) = \sum_{x=0}^{n-1} x^r \left\lfloor \frac{ax + b}{m} \right\rfloor^s.$$

The ordinary floor sum is

$$F_{0,1}(a, b; m, n).$$

We use

$$P_r(t) = \sum_{x=0}^{t-1} x^r,$$

viewed as the usual degree-$(r + 1)$ power-sum polynomial.

The algorithm follows the same idea as Euclid's algorithm. First remove integer parts of a/m and b/m. Once

$$0 \leq a, b < m,$$

transpose the lattice points underneath the line

$$y = \frac{ax + b}{m}.$$

The useful point is that after the transposition, the new slope is m/a, so the pair (a, m) undergoes an ordinary Euclidean step.

Removing Integer Parts Write

$$a = qm + a_0, \qquad b = tm + b_0,$$

with

$$0 \le a_0, b_0 < m.$$

Then

$$\left\lfloor \frac{ax+b}{m} \right\rfloor = qx + t + \left\lfloor \frac{a_0 x + b_0}{m} \right\rfloor.$$

Expanding the power gives

$$\boxed{F_{r,s}(a,b;m,n) = \sum_{j=0}^{s} \binom{s}{j} \sum_{\ell=0}^{s-j} \binom{s-j}{\ell} q^{\ell} t^{s-j-\ell} F_{r+\ell,j}(a_0, b_0; m, n).}$$

Thus we may reduce immediately to

$$0 \le a, b < m.$$

The Reciprocal Step Assume now

$$0 < a < m, \qquad 0 \le b < m.$$

Set

$$h = \left\lfloor \frac{an+b}{m} \right\rfloor, \qquad c = (an+b) \bmod m.$$

Thus

$$an + b = mh + c.$$

The geometric motivation is to regard

$$q_x = \left\lfloor \frac{ax+b}{m} \right\rfloor$$

as the height of the x-th column and then transpose the resulting lattice diagram. Algebraically, write

$$q_x^s = \sum_{t=1}^{q_x} \left(t^s - (t-1)^s \right).$$

For $t = h - y$, the condition $q_x \ge t$ is

$$ax + b \ge m(h-y).$$

Using $an + b = mh + c$,

$$\begin{aligned} x &\ge \left\lceil \frac{m(h-y) - b}{a} \right\rceil \\ &= n - \left\lfloor \frac{my + c}{a} \right\rfloor. \end{aligned}$$

Define

$$u_y = \left\lfloor \frac{my+c}{a} \right\rfloor.$$

Then the contributing x's are exactly

$$n - u_y, \ldots, n-1.$$

Now define the two polynomials

$$D_s(Y) = (h-Y)^s - (h-Y-1)^s = \sum_{i=0}^{s-1} d_i Y^i,$$

and

$$R_r(U) = \sum_{j=1}^{U} (n-j)^r = P_r(n) - P_r(n-U) = \sum_{j=1}^{r+1} e_j U^j.$$

Theorem 83 (Reciprocal Floor-Sum Recurrence)**.** *For $s > 0$,*

$$\boxed{F_{r,s}(a,b;m,n) = \sum_{i=0}^{s-1} \sum_{j=1}^{r+1} d_i e_j F_{i,j}(m,c;a,h).}$$

Proof. Using the finite-difference expansion of q_x^s and swapping the order of summation,

$$\begin{aligned} F_{r,s}(a,b;m,n) &= \sum_{x=0}^{n-1} x^r \sum_{t=1}^{q_x} (t^s - (t-1)^s) \\ &= \sum_{t=1}^{h} (t^s - (t-1)^s) \sum_{\substack{0 \le x < n \\ q_x \ge t}} x^r. \end{aligned}$$

Substitute $t = h - y$. By the preceding calculation,

$$q_x \ge h - y \iff n - u_y \le x < n,$$

where

$$u_y = \left\lfloor \frac{my + c}{a} \right\rfloor.$$

Hence

$$F_{r,s} = \sum_{y=0}^{h-1} D_s(y) R_r(u_y).$$

Expanding the two polynomials,

$$D_s(y) = \sum_i d_i y^i, \qquad R_r(u_y) = \sum_j e_j u_y^j,$$

gives exactly

$$\sum_{i,j} d_i e_j \sum_{y=0}^{h-1} y^i \left\lfloor \frac{my + c}{a} \right\rfloor^j.$$

The inner sum is

$$F_{i,j}(m,c;a,h).$$

□

The base case is simply

$$F_{r,0}(a,b;m,n) = P_r(n).$$

If $a = 0$ after reduction, then the floor is constant and the answer is immediate.

For the ordinary floor sum, $r = 0, s = 1$,

$$D_1(Y) = 1, \qquad R_0(U) = U,$$

so Theorem 83 reduces to the particularly simple identity

$$\boxed{F_{0,1}(a,b;m,n) = F_{0,1}(m,c;a,h).}$$

After the reciprocal step, reducing the new numerator m modulo the new denominator a performs exactly a Euclidean-algorithm step on (m, a). Thus there are only

$$O(\log m)$$

levels.

For any fixed powers r, s, the running time is therefore $O(\log m)$. If all

$$F_{i,j}, \qquad i + j \le K,$$

are computed together, the straightforward implementation takes

$$O(K^4 \log m)$$

arithmetic operations. The transformations are polynomial convolutions, so this dependence on K can be reduced further with fast polynomial multiplication.

6.6.6 Multiplicative Function Prefix Sums

Let f, g be arithmetic functions and let

$$c = f * g$$

be their Dirichlet convolution:

$$c(n) = \sum_{d|n} f(n/d)g(d).$$

Write their prefix sums as

$$F(x) = \sum_{n \le x} f(n), \qquad G(x) = \sum_{n \le x} g(n), \qquad C(x) = \sum_{n \le x} c(n).$$

Suppose $G(x)$ and $C(x)$ can be evaluated quickly and $g(1)$ is invertible. Then $F(x)$ can be recovered much faster than evaluating every value of f up to x.

Lemma 84 (Prefix Sum from Dirichlet Convolution)**.** *For every x,*

$$C(x) = \sum_{d=1}^{x} g(d) F\left(\left\lfloor \frac{x}{d} \right\rfloor\right).$$

Consequently,

$$\boxed{F(x) = \frac{C(x) - \displaystyle\sum_{d=2}^{x} g(d) F\left(\left\lfloor \frac{x}{d} \right\rfloor\right)}{g(1)}.}$$

Proof. Expand and swap the order of summation:

$$\begin{aligned} C(x) &= \sum_{n \le x} \sum_{d|n} f(n/d)g(d) \\ &= \sum_{d \le x} g(d) \sum_{k \le x/d} f(k) \\ &= \sum_{d \le x} g(d) F\left(\left\lfloor \frac{x}{d} \right\rfloor\right). \end{aligned}$$

Separating the $d = 1$ term gives the recurrence. □

The crucial optimization is that

$$\left\lfloor \frac{x}{d} \right\rfloor$$

takes only $O(\sqrt{x})$ distinct values. If

$$q = \left\lfloor \frac{x}{d} \right\rfloor,$$

then the largest index having the same quotient is

$$r = \left\lfloor \frac{x}{q} \right\rfloor.$$

Therefore the whole interval $d, \dots, r$ contributes

$$\big(G(r) - G(d-1)\big)F(q).$$

Precompute $F(x)$ for

$$x \le B$$

using the linear sieve, and memoize all larger recursive values.

Algorithm 5 Multiplicative Function Prefix Sum

Input: x, prefix functions G, C, table of $F(t)$ for $t \le B$
Output: $F(x)$

```
function PrefixF(x)
    if x ≤ B then
        return F_small[x]
    if x is memoized then
        return memo[x]
    A ← C(x)
    l ← 2
    while l ≤ x do
        q ← ⌊x/l⌋
        r ← ⌊x/q⌋
        A ← A − (G(r) − G(l − 1)) · PrefixF(q)
        l ← r + 1
    A ← A/g(1)
    memo[x] ← A
    return A
```

Theorem 85. *If $G(x)$ and $C(x)$ are evaluable in $O(1)$ arithmetic operations and $F(1), \dots, F(B)$ are precomputed, then Algorithm 5 computes $F(n)$ in*

$$O\left(B + \frac{n}{\sqrt{B}}\right)$$

arithmetic operations. Taking

$$B = n^{2/3}$$

gives

$$O(n^{2/3}).$$

Proof. All recursive arguments larger than B are of the form

$$\left\lfloor \frac{n}{k} \right\rfloor > B,$$

so there are at most

$$\frac{n}{B}$$

such memoized states.

For

$$x = \left\lfloor \frac{n}{k} \right\rfloor,$$

one call performs $O(\sqrt{x})$ quotient blocks. Hence the total work on large states is

$$\begin{aligned}\sum_{k \le n/B} O\left(\sqrt{\frac{n}{k}}\right) &= O\left(\sqrt{n} \sum_{k \le n/B} k^{-1/2}\right) \\ &= O\left(\sqrt{n}\sqrt{\frac{n}{B}}\right) \\ &= O\left(\frac{n}{\sqrt{B}}\right).\end{aligned}$$

Adding the $O(B)$ preprocessing gives the claim. □

Two standard examples are immediate.

Since

$$\varphi * 1 = \mathrm{id},$$

we may take

$$g(n) = 1, \qquad G(x) = x, \qquad C(x) = \sum_{n \le x} n = \frac{x(x+1)}{2}$$

to compute

$$\sum_{n \le x} \varphi(n)$$

in $O(x^{2/3})$ time.

Similarly,

$$\mu * 1 = \varepsilon$$

gives

$$\sum_{n \le x} \mu(n)$$

from

$$G(x) = x, \qquad C(x) = 1.$$

6.6.7 LCM and GCD Convolution

Now take

$$P = \{1, \dots, N\}$$

ordered by divisibility:

$$a \le b \iff a \mid b.$$

Then

$$a \vee b = \mathrm{lcm}(a, b), \qquad a \wedge b = \gcd(a, b).$$

Hence

$$(f *_{\mathrm{LCM}} g)(k) = \sum_{\mathrm{lcm}(a,b)=k} f(a) g(b)$$

is join convolution and is diagonalized by the divisor zeta transform

$$\boxed{(\zeta_{\mathrm{div}} f)(k) = \sum_{d \mid k} f(d).}$$

Likewise

$$(f *_{\mathrm{GCD}} g)(k) = \sum_{\gcd(a,b)=k} f(a) g(b)$$

is meet convolution and is diagonalized by the multiple zeta transform

$$\boxed{(\zeta_{\mathrm{mult}} f)(k) = \sum_{\substack{j \le N \\ k \mid j}} f(j).}$$

Both transforms can be implemented by processing one prime (this can be viewed as a dimension) at a time.

For divisor zeta, for every prime p, process multiples x of p in increasing order:

$$\boxed{F(x) \leftarrow F(x) + F(x/p).}$$

Möbius inversion processes in the reverse direction with subtraction.

The total number of updates is

$$\sum_{p \le N} \left\lfloor \frac{N}{p} \right\rfloor = O(N \log \log N).$$

For the multiple transform, process the same prime chains in the opposite direction:

$$\boxed{F(x) \leftarrow F(x) + F(px).}$$

Again inversion reverses the order and changes addition to subtraction.

Thus GCD and LCM convolution take

$$\boxed{O(N \log \log N)}$$

ring operations.

6.6.8 Erdős–Ginzburg–Ziv

The theorem asks for both a cardinality and a congruence: among $2n-1$ integers, choose exactly n whose sum is divisible by n. For a prime modulus, pairing the input enforces the cardinality automatically. The remaining task looks like subset sum, but we need much less than its full reachable set: one new representable residue per pair suffices. A binary search finds that new residue without assuming any monotonicity of the membership pattern.

Theorem 86 (Constructive Erdős–Ginzburg–Ziv)**.** *Given $2n-1$ integers, one can deterministically find n distinct input positions whose values sum to zero modulo n in $O(n \log(2n))$ arithmetic operations and $O(n)$ space.*

Reduce the input values modulo n first.

Lemma 87 (Finding a new translate)**.** *Let p be prime, let $S \subseteq \mathbb{Z}_p$ contain 0 but not t, and let $d \neq 0$ modulo p. With constant-time membership queries to S, one can find $z \notin S$ with $z - d \in S$ in $O(\log p)$ time.*

Proof. Since d is invertible, the progression $0, d, 2d, \ldots$ visits every residue. Put $b = td^{-1} \bmod p$, so $1 \le b < p$. At the endpoints of the integer interval (with respect to the AP) $[0, b]$, the membership answers are respectively yes and no. Bisect the interval, retaining a left endpoint whose answer is yes and a right endpoint whose answer is no. When they become consecutive, the right endpoint times d is the desired z.

Each query halves the interval length. The modular inverse is computed by the extended Euclidean algorithm in $O(\log p)$ time. □

Algorithm 6 A boundary along a cyclic progression

```
function NewResidue(p, S, d, t)
    a ← 0; b ← (td^{-1} mod p)
    while b − a > 1 do
        c ← ⌊(a + b)/2⌋
        if cd mod p ∈ S then
            a ← c
        else
            b ← c
    return bd mod p
```

Sort the $2p - 1$ residues, writing their values as $b_1 \le \cdots \le b_{2p-1}$. If any p consecutive values are equal, their sum is already divisible by p. Otherwise, keep b_p fixed and choose one item from each pair (b_i, b_{p+i}), $1 \le i < p$. Every such choice uses exactly p positions. Start by taking $b_1, \ldots, b_p$, whose sum is s. Switching pair i increases the sum by $d_i = b_{p+i} - b_i \not\equiv 0 \pmod p$: equality would give at least p consecutive equal residues. We therefore need a subset of the $p - 1$ nonzero differences with sum $t = -s \pmod p$.

Maintain only a set S of known representable residues, initially $\{0\}$. After processing i differences, every member of S must be representable using those differences. Unless t is already present, apply Lemma 87 to d_i. It supplies a new residue $z = (z - d_i) + d_i$ whose predecessor $z - d_i$ was already representable. Add just z, recording that predecessor and the index i. We do not add the rest of $S + d_i$.

Each unsuccessful step adds one new residue. After $p - 1$ steps we would have all p residues, so t must be found. This proves the prime case and explains the speedup: the theorem only needs guaranteed growth, not every sum that became reachable at each step.

Algorithm 7 Finding a zero-sum p-subset for prime p

function PRIMEEGZ($p, a_1, \ldots, a_{2p-1}$)
  Sort the position labels $q_1, \ldots, q_{2p-1}$ by $a_{q_i} \bmod p$
  $b_i \leftarrow a_{q_i} \bmod p$ for every i
  for $i = 1, \ldots, p$ **do**
    if $b_i = b_{i+p-1}$ **then**
      return $\{q_i, \ldots, q_{i+p-1}\}$
  $t \leftarrow -\sum_{i=1}^{p} b_i \bmod p$; $S \leftarrow \{0\}$, stored as a Boolean array
  for $i = 1, \ldots, p-1$ **do**
    if $t \in S$ **then**
      break
    $d_i \leftarrow (b_{p+i} - b_i) \bmod p$
    $z \leftarrow$ NEWRESIDUE(p, S, d_i, t)
    prev$[z] \leftarrow (z - d_i) \bmod p$; pair$[z] \leftarrow i$
    Insert z into S
  $I \leftarrow \{q_1, \ldots, q_p\}$; $z \leftarrow t$
  while $z \neq 0$ **do**
    $i \leftarrow$ pair$[z]$
    Replace q_i by q_{p+i} in I
    $z \leftarrow$ prev$[z]$
  return I

Every predecessor was present strictly earlier, so pair indices decrease along the reconstruction path. No pair is switched twice. The switches sum to t, giving total sum $s + t = 0 \pmod p$. Sorting and the at most $p - 1$ boundary searches cost $O(p \log p)$; the Boolean and predecessor arrays use $O(p)$ space.

Composite Moduli. Suppose $n = pq$, with p prime. Repeatedly take a pool of $2p - 1$ unused positions, use PrimeEGZ to remove a group G_j of p positions whose sum is divisible by p, and retain the $p - 1$ unused positions in the pool. Adding p fresh positions prepares the next call. Starting with $p - 1$ positions and making $2q - 1$ such calls uses exactly $(p - 1) + p(2q - 1) = 2pq - 1$ input positions.

The groups are disjoint. Their integer quotients $c_j = p^{-1} \sum_{i \in G_j} a_i$, $1 \le j \le 2q - 1$, form an EGZ instance for modulus q. Select q of these quotients with sum divisible by q; the union of the corresponding groups has pq positions and sum divisible by pq.

Algorithm 8 EGZ for an arbitrary modulus

function EGZ($n, a_1, \ldots, a_{2n-1}$)
  if $n = 1$ **then**
    return $\{1\}$
  Find the least divisor $p \ge 2$ of n by trial division up to $\sqrt{n}$
  if no divisor was found **then**
    return PRIMEEGZ($n, a_1, \ldots, a_{2n-1}$)
  $q \leftarrow n/p$; $P \leftarrow$ the first $p - 1$ position labels
  for $j = 1, \ldots, 2q - 1$ **do**
    Append the next p fresh position labels to P
    Apply PrimeEGZ to the values at P, obtaining position set $G_j \subseteq P$
    $P \leftarrow P \setminus G_j$; $c_j \leftarrow \left(\sum_{i \in G_j} a_i\right)/p$
  $J \leftarrow$ EGZ($q, c_1, \ldots, c_{2q-1}$)
  return $\bigcup_{j \in J} G_j$

Proof of Theorem 86. The prime construction proves existence and correctness for prime moduli. The

disjoint-group construction reduces a composite modulus to a smaller one, and the modulus-one case is immediate. Strong induction proves correctness for every n.

The least proper divisor is prime. At a composite call the $2q-1$ prime subproblems therefore take $O(qp\log p) = O(n\log p)$ time in total, and assembling the groups and quotients takes $O(n)$. Only the final call at modulus $q = n/p \le n/2$ is recursive. Every level costs at most its current modulus times $O(\log(2n))$, while the successive moduli form a geometrically decreasing sequence. Their sum is at most $2n$, proving $O(n\log(2n))$ total time. Trial division is smaller than this bound. The stored group lists across the recursion also have geometrically decreasing total size, so the space usage is $O(n)$. □

6.6.9 Modular Subset Sum

Given residues $a_1, \ldots, a_n \in \mathbb{Z}_m$, we want every residue obtained by a subset of the input positions. The ordinary recurrence repeatedly adds a translate of the reachable set. Its cost comes from scanning all m residues at every step, even though most of them have not changed. Hashing lets us skip unchanged intervals.

Theorem 88 (Modular subset sum)**.** *For any fixed $c > 0$, all subset sums modulo m can be computed in $O(n + m\log^2(2m))$ word operations with probability at least $1-(m+2)^{-c}$. The algorithm uses $O(m)$ working space besides storage for input-position labels. It also stores predecessors from which a subset representing any reported sum can be recovered. Hash collisions can omit reachable sums, but never create an invalid witness.*

The word size is $O(\log(n+m))$ bits for fixed c, and the input residues are already reduced modulo m. We include the empty subset, of sum zero. The case $m = 1$ is immediate, so assume $m \ge 2$ below. Multiplicities can be stored in an array of length m after an $O(n+m)$ input pass.

Let S be the reachable set before processing an item of value x. Bellman's recurrence is

$$S \leftarrow S \cup (S + x).$$

The genuinely new residues are $C = (S+x) \setminus S$. Comparing the two characteristic strings naturally reports the symmetric difference $D = S \triangle (S+x)$, including both directions of disagreement. This costs only a factor of two: translation preserves cardinality, so $|D| = 2|C|$. Every successful update permanently adds at least one residue. Across the whole execution, $\sum |C| \le m-1$.

We must also bound steps with no new residues, especially when $n \gg m$. Process equal values consecutively. As soon as $S \cup (S+x) = S$, we have $S + x = S$ and every remaining copy of x can be skipped. Thus there are at most $m-1$ successful updates and at most one final unsuccessful update per value. Reordering the input does not change its subset sums.

Two implementation rules are essential. First, find the entire difference against the *old* S before setting any new bits; updating during the comparison could use the same input copy more than once. Second, for each new residue r, record its old predecessor $(r-x) \bmod m$ and the current input-position label. Every predecessor was reached at an earlier step, so reconstruction terminates at zero and never reuses an input position.

Write $s[0], \ldots, s[m-1]$ for the characteristic string of S. The characteristic string of $S + x$ has entry $s[(i-x) \bmod m]$ at position i. To compare these strings on an interval, store the doubled string ss and use a polynomial fingerprint. For a base $\alpha \ne 0$ in a large prime field, the normalized hash of $D[l:r]$ is

$$\operatorname{hash}(D[l:r]) = \alpha^{-l} \sum_{i=l}^{r-1} D[i]\alpha^i.$$

The normalization makes hashes of equally long strings comparable even when their starting positions differ. Precompute powers and inverse powers of α, and keep the weighted values $D[i]\alpha^i$ in a Fenwick tree or Segtree. A bit insertion updates its two copies in $O(\log m)$ time; an interval hash takes two prefix-sum queries and therefore $O(\log m)$ time.

Compare $s[l:r]$ with the length-$(r-l)$ interval of ss beginning at $(l-x) \bmod m$. If their hashes agree, skip the interval. Otherwise bisect it and recurse; at a singleton, inspect the two actual bits. The visited

nodes lie on paths to disagreements, together with their siblings, so d disagreements cost $O((d+1)\log m)$ hash comparisons. This is $O((d+1)\log^2 m)$ time. By $|D| = 2|C|$ and the bound on the number of updates, the total is $O(m \log^2 m)$ after reading the input, provided no unequal compared strings collide.

6.6.10 Modular Subset Sum using Runs

Let c_x be the number of available copies of residue $x \in \mathbb{Z}_m$. We want every attainable subset sum, including zero from the empty subset. Bellman's update for one copy of x is $S \leftarrow S \cup (S + x)$. The important freedom is to change the order in which we represent S: if consecutive positions differ by x, this update just extends each occupied run by one position. The boundaries needed to construct this new representation are exactly the differences between S and $S + x$. Thus discovering the update also prepares a compact representation for the next one.

Theorem 89 (Modular subset sum by runs)**.** *From the multiplicities $c_0, \ldots, c_{m-1}$, all subset sums modulo m can be computed deterministically in $O(m \log(2m))$ word operations and $O(m)$ working space. Predecessors give a witness for any attainable target. For an explicit list of n items, forming the multiplicities adds $O(n + m)$ time.*

The case $m = 1$ is immediate. Henceforth $m \geq 2$, and zero-valued items are discarded.

Runs along an addition cycle. Fix a nonzero direction d. The permutation $z \mapsto z + d$ splits $\mathbb{Z}_m$ into $g = \gcd(d, m)$ cycles of length $L = m/g$. Store the occupied positions of each cycle as maximal cyclic runs, and keep the ordinary membership array of S. Full and empty cycles have separate flags and no stored runs. A partial cycle is neither full nor empty; maintain a list of partial cycles so that the other cycles can be skipped.

For example, when m is prime there is just one cycle, $0, d, 2d, \ldots, (m-1)d$, and the representation is an ordinary sorted list of occupied intervals in this cyclic order. For general m, we use one such list per partial cycle. A wrapping run may be split into two ordinary half-open intervals at coordinate zero.

Write $\rho_d(S)$ for the total number of cyclic runs in partial cycles. Every run has a distinct first unoccupied position after it, so

$$\boxed{\rho_d(S) = |(S + d) \setminus S|.} \tag{39}$$

This identity links the size of the representation to growth under this iteration.

Lemma 90 (Growth pays for the next representation)**.** *For $S' = S \cup (S + x)$ and $\Delta = |S' \setminus S|$,*

$$|S \triangle (S + x)| = 2\Delta, \qquad \rho_x(S') \leq \Delta.$$

Proof. The two sets S and $S + x$ have equal cardinality, giving the first identity. In direction x, the update extends each old run by one; runs may merge or fill a cycle, but cannot split. Thus $\rho_x(S') \leq \rho_x(S) = \Delta$. □

The next update will scan only these runs. Even if it uses a different direction, their number is already paid for by the previous update's new residues.

Change direction using only the boundaries. Suppose S is currently stored in direction d, but the next item is x. We first compare S and $S + x$ in the *old* order, where both are readily available as interval lists. Then we use their differences to reconstruct S in the *new* order.

Indeed, in the x-order a position z is a run start exactly when $z \in S$ and $z - x \notin S$. It is an exclusive run end exactly when $z \notin S$ and $z - x \in S$. Hence the tagged boundaries are

$$P = S \setminus (S + x) \quad \text{(starts)}, \qquad Q = (S + x) \setminus S \quad \text{(ends)}.$$

These sets both have size Δ. Sort their 2Δ elements by their positions in the new cycles. Within a partial cycle, starts and ends alternate and uniquely determine its occupied intervals. A cycle with no boundary is either full or empty, distinguished by one membership lookup. We have reconstructed the entire set from its boundaries, not by sorting all its elements.

Comparisons. Here are explicit formulas implementing the change of coordinates. With $g = \gcd(d, m)$, $L = m/g$, and $u = d/g$, represent cycle $c \in [0, g)$ by

$$\phi_d(c, t) = c + g(ut \bmod L), \qquad 0 \le t < L.$$

The inverse coordinate map is $c = z \bmod g$ and $t = ((z - c)/g)u^{-1} \bmod L$. The inverse exists because $\gcd(u, L) = 1$ and is computed once for each input direction by the extended Euclidean algorithm.

Translation by x sends an old cycle to another old cycle, rotating its interval list:

$$\boxed{c' = (c + x) \bmod g, \qquad t' = t + \frac{c + x - c'}{g} u^{-1} \pmod L.} \tag{40}$$

To check this, substitute in $\phi_d(c', t')$ and use $uu^{-1} \equiv 1 \pmod L$; the result is $\phi_d(c, t) + x \pmod m$. A cyclic shift preserves the order of an interval list up to one cut. Split an interval at zero if necessary and concatenate the wrapped and unwrapped portions. This takes linear time in the list length, without sorting.

For each destination cycle, merge-sweep its old interval list and its translated source list. Between successive endpoints, membership in both sets is constant. Skip equal stretches; on an unequal stretch, emit each position with its start or end tag. The work is linear in the number of scanned intervals plus the number of emitted positions.

If $h = \gcd(x, m)$ equals g, then $g \mid x$ and every old cycle stays in place. Its full and empty states cannot change, so only partial cycles need scanning or rebuilding. Moreover, x/g generates each cycle, so every partial cycle has boundaries in the new order. If $h \ne g$, scan all g old cycles and initialize all h new cycle records.

Lemma 91 (One update)**.** *Given the run representation of S in direction d, one Bellman update with x takes*

$$O(1 + \rho_d(S) + \Delta \log(2 + \Delta) + \mathbf{1}_{g \ne h}(g + h))$$

word operations, where $g = \gcd(d, m)$, $h = \gcd(x, m)$, and $\Delta = |(S + x) \setminus S|$. The resulting set is stored in direction x with at most Δ partial runs.

Proof. The old-order sweep costs $O(1 + \rho_d(S) + \Delta)$ if $g = h$, and an additional $O(g)$ otherwise. Each source and destination list is scanned a constant number of times. Sorting the 2Δ new-coordinate boundary keys costs $O(\Delta \log(2 + \Delta))$; pairing them reconstructs all partial cycles in $O(\Delta)$ time. When $g \ne h$, boundary-free cycles are initialized from their representative membership bits in $O(h)$ time. When $g = h$, retain the existing full/empty flags.

Now extend each reconstructed run by one position and merge touching runs. This computes $S \cup (S + x)$ in $O(\Delta)$ time, since the number of old runs in direction x is exactly Δ. Full and empty cycles are unchanged. The new membership bits are precisely Q; record their old predecessors before setting any of them. The preceding lemma bounds the resulting number of runs. □

Algorithm 9 One Bellman update using addition-cycle runs

function RunUpdate($S, \mathcal{I}_d, x, \text{copy}$)
  $g \leftarrow \gcd(d, m)$; $h \leftarrow \gcd(x, m)$
  $P, Q \leftarrow \emptyset$
  for each old source cycle c (only partial cycles if $g = h$) **do**
    Compute its destination c' and rotation by (40)
    Rotate its sorted interval list by splitting at zero
    Merge-sweep it against the old list in cycle c'
    Emit old-only positions into P and translated-only positions into Q
  Convert tagged P, Q to x-cycle coordinates and sort by cycle and position
  if $g \neq h$ **then**
    Initialize each new cycle c as full or empty from the old bit $S[c]$
  else
    Retain full and empty cycle records
  for each cycle containing boundary events **do**
    Pair consecutive cyclic starts and ends to reconstruct its old runs
    Extend every run by one and merge; flag a cycle if it becomes full
  for each $z \in Q$ **do**
    parent$[z] \leftarrow z - x \pmod m$; item$[z] \leftarrow (x, \text{copy})$
  Set all bits $S[z]$ for $z \in Q$; replace d by x
  return $|Q|$

In the reconstruction, a final start pairs with an initial end across zero; store the wrapping interval as a suffix and a prefix.

Group equal gcds. The potentially expensive cycle initialization happens only when the gcd changes. Process all values with the same gcd consecutively. Each possible gcd is a divisor of m, so their total contribution is bounded by

$$\sum_{g|m} g = m \sum_{q|m} \frac{1}{q} \leq m \sum_{q=1}^{m} \frac{1}{q} = O(m \log(2m)). \tag{41}$$

If an update by x makes no change, then $S + x = S$. Iterating translation shows that S is a union of the cycles generated by x, namely the residue classes modulo $h = \gcd(x, m)$. Every other value with this same gcd preserves those classes. We may therefore skip the rest of the entire gcd class, not only the remaining copies of x.

Algorithm 10 Deterministic modular subset sum

Input: Modulus $m \geq 2$ and multiplicities c_x for $0 \leq x < m$

Group nonzero values with positive multiplicity by $\gcd(x, m)$
Precompute $(x/g)^{-1} \bmod (m/g)$ for every such value, where $g = \gcd(x, m)$
$S \leftarrow \{0\}$; $d \leftarrow 1$; initialize the one run $[0, 1)$
for each nonempty gcd class g **do**
    finished $\leftarrow$ false
    for each value x in this class **do**
        for $r = 1, \ldots, c_x$ **do**
            $\Delta \leftarrow \textsc{RunUpdate}(S, \mathcal{I}_d, x, r)$
            if $\Delta = 0$ or $|S| = m$ **then**
                finished $\leftarrow$ true; **break**
        if finished **then**
            break
    if $|S| = m$ **then**
        break
return the membership and predecessor arrays

Proof of Theorem 89. Every executed update is Bellman's recurrence, and the preceding argument justifies all skipped items. This proves correctness. Let Δ_i be the number of residues first reached by update i. Since each nonzero residue is discovered at most once, $\sum_i \Delta_i \leq m - 1$. There are at most $m - 1$ productive updates and at most one unproductive update per gcd class, hence $O(m)$ updates.

The initial representation has one run. After update i, it has at most Δ_i partial runs, so the sum of the run counts scanned at the next updates is at most $1 + \sum_i \Delta_i = O(m)$. There are exactly $2\Delta_i$ boundary keys in update i, giving total sorting cost

$$\sum_i O\big(\Delta_i \log(2 + \Delta_i)\big) \leq O\left(\log(2m) \sum_i \Delta_i\right) = O(m \log(2m)).$$

Each gcd class is entered once, so its gcd contributes at most once as an old value and once as a new value in the cycle-initialization costs. Equation (41) bounds their sum by $O(m \log(2m))$. Computing gcds and modular inverses for at most $m - 1$ values by the Euclidean algorithm takes $O(m \log(2m))$ additional word operations. All other costs are covered by Lemma 91.

Cycle records, intervals, boundary events, and membership and predecessor arrays each use $O(m)$ space. For a target $z \neq 0$, follow parent$[z]$ while recording item$[z]$ until reaching zero. Every predecessor was reachable strictly before the corresponding item copy was used, because all new bits were set only after recording the whole update. Thus the chain terminates, never reuses an input copy, and its item values sum to the target modulo m. □

6.6.11 Integer 3SUM

Given three sets A, B, C of at most n polynomially bounded integers, we want to decide whether $c \in A + B$ for every $c \in C$. The idea is to hash integers to binary masks. When the masks of a and b are disjoint, their numerical sum is their union, so subset convolution detects $a + b = c$. A second-moment test excludes false matches caused by modular hashing. We resample the prime in every trial; this lets us dispense with the collision-bucket cleanup.

Theorem 92 (Randomized integer 3SUM)**.** *For $A, B, C \subseteq [-n^{O(1)}, n^{O(1)}]$ with $|A|, |B|, |C| \leq n$, whether $c \in A + B$ can be decided for every $c \in C$ by a one-sided Monte Carlo algorithm in expected $O(n^{1.965})$ word operations, with total error probability at most n^{-2}.*

Trimmed subset convolution. For functions $F, G : 2^{[d]} \to R$ supported on sets of size t, define

$$(F * G)(Z) = \sum_{X \subseteq Z} F(X)G(Z \setminus X), \qquad |Z| = 2t.$$

Here R is any fixed commutative ring. We use $\widetilde{O}$ to hide polylogarithmic factors. Write $\binom{u}{\le v} = \sum_{j \le v} \binom{u}{j}$ and $\binom{u}{\ge v} = \sum_{j \ge v} \binom{u}{j}$.

Lemma 93 (Selected-entry subset convolution)**.** *For $t \le s \le 2t$, we can preprocess F, G in $O\big(d(n + \binom{d}{\le s})\big)$ ring operations and subsequently compute $(F * G)(Z)$ for any $|Z| = 2t$ in $O\big(d\binom{2t}{\ge s}\big)$ ring operations.*

Proof. Write $\mathcal{Z}U(X) = \sum_{Y \subseteq X} U(Y)$ and $\mathcal{M}U(X) = \sum_{Y \subseteq X} (-1)^{|X \setminus Y|} U(Y)$ for the zeta and inverse zeta transforms. Let $H(Z) = \sum_{X \cup Y = Z} F(X)G(Y)$ be the covering product. Then

$$\mathcal{Z}H(X) = \mathcal{Z}F(X)\,\mathcal{Z}G(X), \qquad H = \mathcal{M}(\mathcal{Z}F \cdot \mathcal{Z}G).$$

For $|Z| = 2t$, the condition $X \cup Y = Z$ and $|X| = |Y| = t$ forces $X \cap Y = \varnothing$; hence $H(Z) = (F * G)(Z)$.

Both transforms use the same dynamic program. Starting from $U_0(X) = U(X)$, for $i = 1, \dots, d$ compute

$$U_i(X) = U_{i-1}(X) + \varepsilon \mathbf{1}_{i \in X} U_{i-1}(X \setminus \{i\}), \qquad \varepsilon = +1 \text{ or } -1. \tag{42}$$

The final array is $\mathcal{Z}U$ for $\varepsilon = +1$ and $\mathcal{M}U$ for $\varepsilon = -1$. Precompute all intermediate states of the two forward transforms and the inverse transform on $|X| \le s$. This uses $O(d\binom{d}{\le s})$ operations. For a query Z, continue the same three DPs only on $X \subseteq Z$ with $|X| > s$, using the saved states at $|X| = s$ as boundary values. Dependencies in (42) only delete elements, so no other states are needed. There are at most $\binom{2t}{\ge s}$ such states per DP stage. This proves the claimed costs. □

One convolution also verifies the sums. Use the ring of integer triples with coordinatewise addition and multiplication

$$(u_0, u_1, u_2)(v_0, v_1, v_2) = (u_0 v_0,\ u_1 v_0 + u_0 v_1,\ u_2 v_0 + 2u_1 v_1 + u_0 v_2).$$

This is polynomial multiplication truncated after degree two, with the quadratic coefficient stored twice, so it is associative. Assign an integer a the weight $w(a) = (1, a, a^2)$. Then $w(a)w(b) = (1, a + b, (a + b)^2)$. Thus a subset-convolution query returns $M = (M_0, M_1, M_2)$, the count and first two moments of all contributing pairs. We accept a target c precisely when

$$M_0 > 0, \qquad M_2 - 2cM_1 + c^2 M_0 = 0. \tag{43}$$

Indeed, the second expression is $\sum_{(a,b)\ \text{contributing}} (a + b - c)^2$, so the test cannot accept a false target. If several numbers hash to the same mask, simply add their weights; no collision handling is required.

Random hashing. Let lg denote the base-two logarithm, and set

$$t = \lfloor 0.2 \lg n \rfloor, \qquad s = \lfloor 0.367 \lg n \rfloor, \qquad P = 2^d = \Theta(n 2^t \log n), \quad T = \binom{d}{2t}, \quad K = \binom{2t}{t}.$$

Choose the constant in P sufficiently large. In each trial, independently pick a uniform prime $p \in (P, 2P]$, a uniform $\xi \in \mathbb{F}_p^*$, and uniform $\alpha, \beta \in \mathbb{F}_p$. Write

$$x(a) = (\xi a + \alpha) \bmod p, \qquad y(b) = (\xi b + \beta) \bmod p, \qquad z(c) = (\xi c + \alpha + \beta) \bmod p. \tag{44}$$

For $0 \le u < P$, let $\pi(u) \subseteq [d]$ be its one-bit positions, and write $\mathrm{val}(X) = \sum_{i \in X} 2^{i-1}$ for the inverse encoding. In particular, if $a + b = c$ and $\pi(x(a)) \cap \pi(y(b)) = \varnothing$, then $z(c) = x(a) + y(b) < P$ and $\pi(z(c)) = \pi(x(a)) \cup \pi(y(b))$.

Algorithm 11 Randomized 3SUM with trimmed subset convolution

Input: Sets A, B, C of at most n polynomially bounded integers

Set t, s, d, P, T, K as above and $R \leftarrow \lceil c_0 P^2 \log n/(TK) \rceil$ for a sufficiently large constant c_0
Mark every $c \in C$ as unanswered
for $r = 1, \ldots, R$ **do**
    Independently draw p, ξ, α, β and define x, y, z by (44)
    Initialize sparse maps F, G with default value $(0, 0, 0)$
    for $a \in A$ **do**
        if $x(a) < P$ and $|\pi(x(a))| = t$ **then**
            $F(\pi(x(a))) \leftarrow F(\pi(x(a))) + w(a)$
    for $b \in B$ **do**
        if $y(b) < P$ and $|\pi(y(b))| = t$ **then**
            $G(\pi(y(b))) \leftarrow G(\pi(y(b))) + w(b)$
    Build the trimmed convolution index for F, G at cutoff s
    for each unanswered $c \in C$ with $z(c) < P$ and $|\pi(z(c))| = 2t$ **do**
        $(M_0, M_1, M_2) \leftarrow (F * G)(\pi(z(c)))$ using the index
        if $M_0 > 0$ and $M_2 - 2cM_1 + c^2 M_0 = 0$ **then**
            Mark c as answered *yes*
Answer *no* for every remaining unanswered c

Lemma 94 (A fixed witness survives)**.** *For any $a \in A$, $b \in B$, and $c \in C$ with $a + b = c$, one trial of Algorithm 11 accepts c with probability $\Omega(TK/P^2)$.*

Proof. Fix disjoint masks $X, Y \subseteq [d]$ of size t, and condition on $x(a) = \mathrm{val}(X)$ and $y(b) = \mathrm{val}(Y)$. For any fixed prime p, this event has probability exactly p^{-2}: after choosing ξ, there are unique suitable α, β. Conditioning therefore changes the distribution of the random prime by at most a factor four, as $P < p \le 2P$.

For any nonzero integer q of polynomial magnitude, a random prime in $(P, 2P]$ divides q with probability $O(\log n/P)$. Consequently, except with conditional probability $O(n \log n/P)$, neither a nor b shares its residue modulo p with another element of its respective input set. Call this the isolation event.

For an incorrect pair (a', b') with $a' + b' \neq c$ to contribute to the query $\pi(z(c)) = X \cup Y$, it is necessary that $p \mid (a' + b' - c)$. Under isolation, this also implies $a' \not\equiv a \pmod p$; otherwise $a' = a$ and then $b' = b$. For a fixed such prime, conditional on the two prescribed witness hashes, ξ remains uniform in $\mathbb{F}_p^*$, so $x(a') = x(a) + \xi(a' - a)$ is uniform over $\mathbb{F}_p \setminus \{x(a)\}$. There are only $K = \binom{2t}{t}$ possible t-bit submasks of $X \cup Y$. Thus, union-bounding over all incorrect pairs, the probability of any incorrect contribution, conditioned on the witness hashes, is at most

$$O\left(\frac{n \log n}{P} + \frac{n^2 K \log n}{P^2}\right) = o(1), \tag{45}$$

where the last equality uses $K \le 4^t$ and $P = \Theta(n 2^t \log n)$. When there is no incorrect contribution, the chosen witness contributes, every contributing pair is an actual solution, and (43) accepts.

There are TK choices of disjoint ordered mask pairs (X, Y). These events are disjoint, and each occurs with probability at least $1/(4P^2)$. Summing their surviving probabilities proves the lemma. □

Proof of Theorem 92. The moment test proves that no false positive is possible. By Lemma 94, a true target succeeds in one trial with probability $\Omega(TK/P^2)$. The prime and affine hashes are redrawn independently in every trial, so the chosen value $R = \Theta(P^2 \log n/(TK))$ makes the probability of missing a fixed true target at most n^{-3}. A union bound over $|C| \le n$ queries gives total error probability at most n^{-2}.

Let $H = \binom{d}{\le s}$ and $Q = \binom{2t}{\ge s}$. A trial takes $\widetilde{O}(n + H + |C^*|Q)$ operations, where $C^* = \{c \in C : z(c) < P,\ |\pi(z(c))| = 2t\}$. For every fixed p and c, $z(c)$ is uniform in $\mathbb{F}_p$, so $\mathbb{E}|C^*| \le nT/P$. As $K = \Theta(4^t/\sqrt{t})$, we have $R = \widetilde{O}(n^2/T)$. The total expected time is therefore

$$\widetilde{O}\left(\frac{n^3}{T} + \frac{n^2 H}{T} + \frac{n^3 Q}{P}\right). \tag{46}$$

With $t = 0.2 \lg n$, $s = 0.367 \lg n$, and $d = 1.2 \lg n + O(\log \log n)$, the binomial-entropy bound gives

$$T = n^{1.101955+o(1)}, \qquad H = n^{1.065966+o(1)}, \qquad Q = n^{0.164371+o(1)}, \qquad P = n^{1.2+o(1)}.$$

The three terms in (46) have exponents 1.898045, 1.964011, and 1.964371, respectively. All omitted factors are polylogarithmic, so the time is $O(n^{1.965})$. The intermediate values have polynomial magnitude and require only $O(1)$ machine words each. □

6.7 Fast Factorials and Deterministic Factorization

To compute the factorial we will obtain values of

$$F_v(x) = (x+1)(x+2)\cdots(x+v)$$

at the arithmetic progression

$$0, v, 2v, \ldots.$$

Using a product tree followed by multipoint evaluation gives two logarithmic factors. The special structure lets us do better: we never construct the coefficients of F_v at all. Instead, we recursively construct its values on the arithmetic progression.

The main tool is fast shifting of equally spaced polynomial evaluations.

6.7.1 Fast Factorial

Let p be prime and suppose

$$0 \le n < p.$$

We compute

$$n! \pmod p$$

in

$$O(\sqrt{n} \log n)$$

field operations.

Choose a power of two

$$v = \Theta(\sqrt{n}), \qquad v \ge \sqrt{n}.$$

For

$$d = 1, 2, 4, \ldots, v,$$

define

$$F_d(x) = \prod_{j=1}^{d} (x+j).$$

Rather than storing the coefficients of F_d, store the $d+1$ evaluations

$$\boxed{S_d = \big(F_d(0), F_d(v), F_d(2v), \ldots, F_d(dv)\big).}$$

The base case is

$$S_1 = (1, v+1).$$

The doubling identity is

$$\boxed{F_{2d}(x) = F_d(x) F_d(x+d).}$$

Suppose S_d is known. Using Lemma 660, compute

$$A_i = F_d(iv + d), \qquad 0 \le i \le d.$$

Then

$$F_{2d}(iv) = F_d(iv)F_d(iv+d) = S_d[i]A_i$$

gives the first $d+1$ entries of S_{2d}.

For the remaining entries, compute by two more evaluation shifts

$$B_j = F_d((d+1+j)v),$$

and

$$C_j = F_d((d+1+j)v+d), \qquad 0 \le j \le d.$$

Then

$$B_jC_j = F_{2d}((d+1+j)v).$$

Keeping

$$j = 0, \ldots, d-1$$

gives exactly the remaining d entries.

Thus

$$\boxed{S_d \longrightarrow S_{2d}}$$

uses three evaluation shifts and $O(d)$ pointwise multiplications.

Algorithm 12 Fast Rising Factorial Evaluations

Input: Power of two v

Output:

$$S_v[i] = F_v(iv) \qquad (0 \le i \le v)$$

```
1:
   S ← (1, v + 1)
2: d ← 1
3: while d < v do
4:     A ← EvalShift(S, d)                          ▷ A_i = F_d(iv + d)
5:     B ← EvalShift(S, (d + 1)v)
6:     C ← EvalShift(S, (d + 1)v + d)
7:     S' ← empty array
8:     for i = 0, ..., d do
9:         append S[i]A[i] to S'
10:    for j = 0, ..., d − 1 do
11:        append B[j]C[j] to S'
12:    S ← S'
13:    d ← 2d
14: return S
```

Theorem 95 (Fast Factorial)**.** *For prime p and $n < p = n^{O(1)}$, one can compute*

$$n! \pmod p$$

in

$$\boxed{O(\sqrt{n}\log n)}$$

time.

Proof. At a doubling step of size d, the work is

$$O(d \log d).$$

Therefore

$$T(v) = T(v/2) + v \log v = O(v \log v).$$

Since

$$v = \Theta(\sqrt{n}),$$

this is

$$O(\sqrt{n} \log n).$$

At the end we know

$$F_v(0), F_v(v), \ldots, F_v(v^2).$$

Since

$$q = \left\lfloor \frac{n}{v} \right\rfloor \leq v,$$

the complete blocks give

$$(qv)! = \prod_{i=0}^{q-1} F_v(iv).$$

Finally multiply the fewer than v remaining terms

$$qv + 1, \ldots, n.$$

□

6.7.2 Deterministic Integer Factorization

The same construction gives a deterministic $\widetilde{O}(N^{1/4})$-time factorization algorithm.

Let N be composite. If N is even, we immediately find the factor 2, so assume that N is odd.

Let v be the smallest power of two satisfying

$$v \geq N^{1/4}.$$

Thus

$$N^{1/4} \leq v < 2N^{1/4}.$$

Define

$$F_v(x) = \prod_{j=1}^{v} (x + j).$$

Then

$$F_v(iv) = \prod_{j=1}^{v} (iv + j)$$

is the product of the i-th block of v consecutive integers.

Using the fast evaluation construction from Theorem 95, compute

$$\boxed{F_v(0), F_v(v), \ldots, F_v((v-1)v) \pmod{N}}$$

in

$$O(v \log v)$$

arithmetic operations over $\mathbb{Z}/N\mathbb{Z}$.

The only additional issue is that $\mathbb{Z}/N\mathbb{Z}$ is not a field: the evaluation-shift algorithm may try to divide by a nonunit. For factorization, this causes no difficulty.

Batched Inversions Suppose one stage of the algorithm requires the inverses of

$$a_1, \ldots, a_t \in \mathbb{Z}/N\mathbb{Z}.$$

Compute

$$A = \prod_{i=1}^{t} a_i \pmod{N}$$

and

$$g = \gcd(A, N).$$

If

$$g = 1,$$

then every a_i is invertible. One extended gcd computes A^{-1}, after which prefix and suffix products recover all

$$a_i^{-1}$$

using $O(t)$ modular multiplications.

If

$$g > 1,$$

split the product into two halves and compute the gcd of each half-product with N. Repeating this isolates a nonunit denominator. Its gcd with N reveals a factor of N.

Thus every required inversion either succeeds or produces a factor, while the batching adds only linear work to each evaluation-shift stage.

Theorem 96 (Strassen–Pollard Factorization)**.** *A nontrivial factor of a composite integer N can be found deterministically in*

$$\boxed{O\left(N^{1/4}(\log N)^2 \log\log N\right)}$$

bit operations, assuming

$$M(\ell) = O(\ell \log \ell)$$

for multiplication of ℓ-bit integers.

Proof. Let p be the smallest prime divisor of N. Since N is composite,

$$p \le \sqrt{N}.$$

By the choice of v,

$$\sqrt{N} \le v^2.$$

Hence p lies among the integers

$$1, 2, \ldots, v^2.$$

Therefore there are

$$0 \le i < v, \qquad 1 \le j \le v$$

such that

$$p = iv + j.$$

For this i,

$$F_v(iv) = \prod_{t=1}^{v} (iv + t)$$

contains p as a factor, and consequently

$$\gcd(F_v(iv), N) > 1.$$

After computing the v block products, calculate

$$g_i = \gcd(F_v(iv), N) \qquad (0 \le i < v).$$

At least one satisfies

$$g_i > 1.$$

If

$$1 < g_i < N,$$

then g_i is already a nontrivial factor.
The remaining possibility is

$$g_i = N.$$

In this case inspect the v integers in that block:

$$iv + 1, \ldots, iv + v.$$

At least one is divisible by the smallest prime factor p. Since every one of these integers is at most

$$v^2 = O(\sqrt{N}) < N,$$

its gcd with N is a proper nontrivial factor.
It remains to bound the running time. Put

$$L = \lceil \log_2 N \rceil.$$

All arithmetic takes place modulo N, so every ring element has $O(L)$ bits.
The fast evaluation construction uses

$$O(v \log v)$$

ring operations. A modular multiplication costs

$$O(M(L))$$

bit operations, so the sampling phase costs

$$O\big(v \log v \, M(L)\big).$$

Since

$$v = \Theta(N^{1/4})$$

and

$$\log v = \Theta(\log N) = \Theta(L),$$

this is

$$O\big(N^{1/4} L\, M(L)\big).$$

The final $O(v)$ gcd computations cost

$$O\big(v\, M(L) \log L\big),$$

which is smaller than the sampling cost. The batched inversions likewise contribute only lower-order terms.

Therefore the total bit complexity is

$$O\big(N^{1/4} L\, M(L)\big).$$

Substituting

$$M(L) = O(L \log L)$$

gives

$$\boxed{O\left(N^{1/4} (\log N)^2 \log\log N\right).}$$

□

6.8 Miller–Rabin Primality Test

Let $n > 2$ be odd, and write

$$n - 1 = 2^s d, \qquad d \text{ odd}.$$

The Miller–Rabin test studies the squaring sequence

$$a^d, \quad a^{2d}, \quad a^{4d}, \quad \ldots, \quad a^{2^s d} = a^{n-1} \pmod{n}.$$

For a prime modulus, this sequence has a very restricted form.

Lemma 97. *Let p be an odd prime and write*

$$p - 1 = 2^s d$$

with d odd. For every $a \not\equiv 0 \pmod{p}$, either

$$a^d \equiv 1 \pmod{p},$$

or

$$a^{2^r d} \equiv -1 \pmod{p}$$

for some $0 \le r < s$.

Proof. By Fermat's theorem,

$$a^{2^s d} = a^{p-1} \equiv 1 \pmod{p}.$$

If $a^d \equiv 1$, we are done.

Otherwise, let $j \geq 1$ be the smallest index such that

$$a^{2^j d} \equiv 1 \pmod{p}.$$

Then

$$x := a^{2^{j-1} d}$$

satisfies

$$x^2 \equiv 1 \pmod{p} \qquad \text{but} \qquad x \not\equiv 1 \pmod{p}.$$

Since

$$(x-1)(x+1) \equiv 0 \pmod{p}$$

and p is prime,

$$x \equiv -1 \pmod{p}.$$

□

For composite n, a base a satisfying the conclusion of Lemma 97 is called a *strong liar*; otherwise it is a *strong witness* for the compositeness of n.

Algorithm 13 Miller–Rabin Primality Test

Input: Integer $n \geq 2$, number of rounds t
Output: Composite or Probably Prime

```
if n ∈ {2, 3} then
    return Probably Prime
if n < 2 or n is even then
    return Composite
Write n − 1 = 2^s d with d odd
for k = 1, . . . , t do
    Choose a uniformly from {2, . . . , n − 2}
    if gcd(a, n) > 1 then
        return Composite
    x ← a^d mod n
    if x = 1 or x = n − 1 then
        continue
    pass ← false
    for r = 1, . . . , s − 1 do
        x ← x^2 mod n
        if x = n − 1 then
            pass ← true
            break
        if x = 1 then
            return Composite
    if not pass then
        return Composite
return Probably Prime
```

By Lemma 97, a prime is always accepted. The usefulness of the test comes from the fact that very few bases can fool it.

Theorem 98 (Rabin–Monier)**.** *Let n be an odd composite integer. At most one quarter of the nonzero residues modulo n are strong liars.*

Consequently, t independent rounds of the Miller–Rabin test accept a composite n with probability at most

$$\boxed{4^{-t}}.$$

The running time is

$$O\big(t\, M(\log n) \log n\big)$$

bit operations, where $M(\ell)$ is the cost of multiplying two ℓ-bit integers. Using

$$M(\ell) = O(\ell \log \ell),$$

this is

$$\boxed{O\left(t \log^2 n \log\log n\right).}$$

Proof. Write

$$n - 1 = 2^s d, \qquad d \text{ odd},$$

and factor

$$n = \prod_{i=1}^{k} p_i^{e_i}.$$

For each prime divisor p_i, write

$$p_i - 1 = 2^{s_i} u_i, \qquad u_i \text{ odd},$$

and set

$$\nu = \min_i s_i.$$

We first place every strong liar inside a set that is easy to count.
Suppose a is a strong liar. If

$$a^d \equiv 1 \pmod{n},$$

then certainly

$$a^{2^{\nu-1} d} \equiv 1 \pmod{n}.$$

Otherwise, for some $0 \le r < s$,

$$a^{2^r d} \equiv -1 \pmod{n}.$$

Reducing modulo any p_i, the order of a modulo p_i has 2-adic valuation exactly $r + 1$. Since this order divides $p_i - 1$,

$$r + 1 \le s_i$$

for every i, and hence

$$r + 1 \le \nu.$$

Therefore, after squaring if necessary,

$$a^{2^{\nu-1}d} \equiv \pm 1 \pmod{n}.$$

Thus every strong liar belongs to

$$S = \left\{ a \in (\mathbb{Z}/n\mathbb{Z})^\times : a^{2^{\nu-1}d} \equiv \pm 1 \pmod{n} \right\}.$$

We now count S.
By the Chinese remainder theorem,

$$(\mathbb{Z}/n\mathbb{Z})^\times \cong \prod_{i=1}^{k} (\mathbb{Z}/p_i^{e_i}\mathbb{Z})^\times.$$

For odd p_i, the i-th group is cyclic of order

$$N_i = p_i^{e_i-1}(p_i - 1) = p_i^{e_i-1} 2^{s_i} u_i.$$

Set

$$m = 2^{\nu-1} d.$$

Since $p_i \nmid d$,

$$\gcd(m, N_i) = 2^{\nu-1} \gcd(d, u_i).$$

In a cyclic group of order N_i, the equation

$$x^m = 1$$

has exactly $\gcd(m, N_i)$ solutions.
The equation

$$x^m = -1$$

has the same number of solutions. Indeed,

$$\gcd(m, N_i) \mid \frac{N_i}{2},$$

because $s_i \geq \nu$, so -1 lies in the image of the m-th power map.
Hence, writing

$$g_i = \gcd(d, u_i),$$

the CRT gives

$$|S| = 2 \prod_{i=1}^{k} 2^{\nu-1} g_i.$$

Since

$$\varphi(n) = \prod_{i=1}^{k} p_i^{e_i-1} 2^{s_i} u_i,$$

we obtain

$$\frac{\varphi(n)}{|S|} = \frac{1}{2}\prod_{i=1}^{k} p_i^{e_i-1}\left(2^{s_i-\nu+1}\frac{u_i}{g_i}\right).$$

Define

$$R_i = 2^{s_i-\nu+1}\frac{u_i}{g_i}.$$

Each R_i is an even integer, so in particular

$$R_i \geq 2.$$

It remains only to check that

$$\frac{\varphi(n)}{|S|} \geq 4.$$

If $k \geq 3$, then immediately

$$\frac{\varphi(n)}{|S|} \geq \frac{1}{2}\,2^k \geq 4.$$

Suppose $k = 2$. If n is not squarefree, some $e_i \geq 2$, and therefore

$$p_i^{e_i-1} \geq 3.$$

Hence

$$\frac{\varphi(n)}{|S|} \geq \frac{1}{2}\cdot 3\cdot 2\cdot 2 > 4.$$

It remains to consider

$$n = pq, \qquad p < q.$$

We always have $R_p \geq 2$.
If

$$s_q > \nu,$$

then

$$R_q \geq 4.$$

Otherwise $s_q = \nu$, so

$$q - 1 = 2^{\nu}u_q.$$

We claim that $u_q \nmid d$. Indeed, since $s \geq \nu$, if $u_q \mid d$ then

$$q - 1 = 2^{\nu}u_q \mid 2^s d = n - 1.$$

But

$$n - 1 = pq - 1 \equiv p - 1 \pmod{q-1},$$

and

$$0 < p - 1 < q - 1,$$

so $q-1 \nmid n-1$, a contradiction.
Thus

$$\frac{u_q}{\gcd(u_q, d)}$$

is an odd integer greater than 1, hence at least 3. Therefore

$$R_q \geq 6.$$

So in either case,

$$\frac{\varphi(n)}{|S|} \geq \frac{1}{2} \cdot 2 \cdot 4 = 4.$$

Finally, suppose $k = 1$, so

$$n = p^e, \qquad e \geq 2.$$

Since

$$p - 1 \mid p^e - 1 = n - 1,$$

the odd part u_1 divides d. Hence

$$R_1 = 2,$$

and therefore

$$\frac{\varphi(n)}{|S|} = p^{e-1}.$$

This is at least 4 except when

$$n = 3^2 = 9.$$

For $n = 9$, the only strong liars are

$$\pm 1,$$

so there are exactly

$$2 = \frac{9-1}{4}.$$

Thus in every case the number of strong liars is at most

$$\frac{n-1}{4}.$$

Therefore a random Miller–Rabin round accepts a composite number with probability at most $1/4$, and t independent rounds fail with probability at most

$$4^{-t}.$$

For the running time, let

$$\ell = \Theta(\log n).$$

Computing $a^d \bmod n$ by repeated squaring and then following the Miller–Rabin squaring chain uses

$$O(\log n)$$

modular multiplications. Each operates on $O(\log n)$-bit integers, so one round takes

$$O\big(M(\log n)\log n\big)$$

bit operations. Therefore t rounds take

$$O\big(t\, M(\log n)\log n\big).$$

Using

$$M(\ell) = O(\ell \log \ell)$$

gives

$$O\left(t \log^2 n \log\log n\right).$$

□

7 Combinatorics

7.1 Entropy Method

Throughout this section, entropy is measured in bits.

7.1.1 Shearer's Lemma

Lemma 99 (Shearer's Lemma)**.** *Let*

$$X = (X_1, \ldots, X_n)$$

be a random vector, and let

$$\mathcal{A} = \{A_1, \ldots, A_m\}$$

be a collection of subsets of $[n]$ *such that every coordinate* $i \in [n]$ *belongs to at least* k *members of* $\mathcal{A}$*. Then*

$$\boxed{kH(X) \le \sum_{j=1}^{m} H(X_{A_j}).}$$

Proof. Order the coordinates as $1, \ldots, n$. For every $A \subseteq [n]$, the chain rule gives

$$H(X_A) = \sum_{i \in A} H\left(X_i \mid X_j : j \in A,\ j < i\right).$$

Conditioning on more information can only decrease entropy, so

$$H(X_A) \ge \sum_{i \in A} H(X_i \mid X_1, \ldots, X_{i-1}).$$

Summing over $A \in \mathcal{A}$,

$$\begin{aligned}\sum_{A \in \mathcal{A}} H(X_A) &\ge \sum_{A \in \mathcal{A}} \sum_{i \in A} H(X_i \mid X_1, \ldots, X_{i-1}) \\ &\ge k \sum_{i=1}^{n} H(X_i \mid X_1, \ldots, X_{i-1}) \\ &= kH(X).\end{aligned}$$

□

The interpretation is that if several overlapping projections together cover every coordinate k times, then together they contain at least k copies of the information in the whole object.

7.1.2 Entropy Bound for Binomial Sums

Define the binary entropy function

$$H(p) = -p \log_2 p - (1-p)\log_2(1-p).$$

Theorem 100. *For*

$$0 \le k \le \frac{n}{2},$$

$$\boxed{\sum_{j=0}^{k} \binom{n}{j} \le 2^{\,nH(k/n)}.}$$

In particular,

$$\binom{n}{k} \le 2^{\,nH(k/n)}.$$

Proof. Let $X \subseteq [n]$ be chosen uniformly from all subsets of size at most k, and let

$$X_i = \mathbf{1}[i \in X].$$

Then

$$H(X_1, \ldots, X_n) = \log_2 \sum_{j=0}^{k} \binom{n}{j}.$$

By subadditivity of entropy,

$$H(X_1, \ldots, X_n) \le \sum_{i=1}^{n} H(X_i).$$

By symmetry, every X_i has the same probability p of being 1. Moreover,

$$np = \mathbb{E}|X| \le k,$$

so

$$p \le \frac{k}{n} \le \frac{1}{2}.$$

Since the binary entropy function is increasing on $[0, 1/2]$,

$$H(X_i) = H(p) \le H(k/n).$$

Therefore

$$\log_2 \sum_{j=0}^{k} \binom{n}{j} \le nH(k/n),$$

which gives the result. □

7.1.3 Irregular Moore Bound

For a directed edge

$$\vec{e} = (u, v),$$

a *non-returning walk* is a walk that never immediately traverses an edge and then its reverse.

Let $N_i(v)$ be the number of non-returning walks of length i starting at v, and let $N_i(\vec{e})$ be the number of non-returning walks of length $i+1$ whose first edge is $\vec{e}$.

The entropy argument says that even though the degrees are irregular, the average number of such walks grows as though every vertex had the average degree.

Lemma 101 (Non-Returning Walk Growth)**.** *Let G be a graph of minimum degree at least 2 and average degree*

$$\bar{d}.$$

Define the stationary vertex distribution

$$\pi(v) = \frac{d_v}{\bar{d}\, n}.$$

Then for $i \geq 1$,

$$\boxed{\mathbb{E}_{v\sim\pi} N_i(v) \geq \bar{d}(\bar{d}-1)^{i-1},}$$

and if $\vec{e}$ is a uniformly random directed edge, then for $i \geq 0$,

$$\boxed{\mathbb{E}_{\vec{e}} N_i(\vec{e}) \geq (\bar{d}-1)^i.}$$

Proof. We first prove the edge version.

Choose $\vec{e}_0$ uniformly among all directed edges, and then generate

$$\vec{e}_0, \vec{e}_1, \ldots, \vec{e}_i$$

by choosing each next edge uniformly among all non-returning continuations.

Every $\vec{e}_j$ is again uniformly distributed among directed edges, so the head vertex of each $\vec{e}_j$ has distribution π.

Conditioned on $\vec{e}_0 = \vec{e}$, the random continuation

$$(\vec{e}_1, \ldots, \vec{e}_i)$$

has support of size at most $N_i(\vec{e})$. Hence

$$H(\vec{e}_1, \ldots, \vec{e}_i \mid \vec{e}_0 = \vec{e}) \leq \log_2 N_i(\vec{e}).$$

Averaging and using Jensen,

$$\begin{aligned} \log_2 \mathbb{E}_{\vec{e}} N_i(\vec{e}) &\geq \mathbb{E}_{\vec{e}} \log_2 N_i(\vec{e}) \\ &\geq H(\vec{e}_1, \ldots, \vec{e}_i \mid \vec{e}_0). \end{aligned}$$

By the chain rule and the Markov property,

$$H(\vec{e}_1, \ldots, \vec{e}_i \mid \vec{e}_0) = i\, \mathbb{E}_{v\sim\pi} \log_2(d_v - 1).$$

Now

$$\mathbb{E}_{v\sim\pi} \log_2(d_v - 1) = \frac{1}{\bar{d}} \mathbb{E}_{v \text{ uniform}} \left[d_v \log_2(d_v - 1) \right].$$

The function

$$x \longmapsto x \log(x-1)$$

is convex for $x \geq 2$, so Jensen gives

$$\mathbb{E}_{v\sim\pi} \log_2(d_v - 1) \geq \log_2(\bar{d} - 1).$$

Thus

$$\mathbb{E}_{\vec{e}} N_i(\vec{e}) \geq (\bar{d} - 1)^i.$$

For the vertex version, choose $v \sim \pi$, then the first edge uniformly from the d_v choices, and continue non-returningly. Exactly the same argument gives

$$\log_2 \mathbb{E}_{v\sim\pi} N_i(v) \geq \mathbb{E}_{v\sim\pi} \log_2 d_v + (i-1)\mathbb{E}_{v\sim\pi} \log_2(d_v - 1).$$

Since

$$x \longmapsto x \log x$$

is convex,

$$\mathbb{E}_{v\sim\pi} \log_2 d_v = \frac{1}{\bar{d}} \mathbb{E}[d_v \log_2 d_v] \geq \log_2 \bar{d}.$$

Combining the bounds gives

$$\mathbb{E}_{v\sim\pi} N_i(v) \geq \bar{d}(\bar{d} - 1)^{i-1}.$$

□

Theorem 102 (Alon–Hoory–Linial Moore Bound)**.** *Let G be a simple graph with n vertices, minimum degree at least 2, average degree $\bar{d}$, and girth g.*

If

$$g = 2r + 1,$$

then

$$\boxed{n \geq 1 + \bar{d} \sum_{i=0}^{r-1} (\bar{d} - 1)^i.}$$

If

$$g = 2r,$$

then

$$\boxed{n \geq 2 \sum_{i=0}^{r-1} (\bar{d} - 1)^i.}$$

Proof. Suppose first that

$$g = 2r + 1.$$

Fix a vertex v. The endpoints of all non-returning walks of lengths

$$0, 1, \ldots, r$$

starting at v are distinct: two such walks ending at the same vertex would produce a cycle of length at most $2r$.

Hence

$$n \geq 1 + \sum_{i=1}^{r} N_i(v).$$

Average over $v \sim \pi$ and apply Lemma 101:

$$n \geq 1 + \sum_{i=1}^{r} \bar{d}(\bar{d} - 1)^{i-1}.$$

Now suppose

$$g = 2r.$$

Take an edge uv, and orient it both ways:

$$\vec{e} = (u, v), \qquad \overleftarrow{e} = (v, u).$$

The endpoints of the non-returning walks of lengths up to r extending from the two sides of this edge are all distinct; otherwise they would form a cycle of length less than $2r$.

Therefore

$$n \geq \sum_{i=0}^{r-1} N_i(\vec{e}) + \sum_{i=0}^{r-1} N_i(\overleftarrow{e}).$$

Choose $\vec{e}$ uniformly among directed edges. Its reverse is also uniform, so taking expectations gives

$$n \geq 2 \sum_{i=0}^{r-1} (\bar{d} - 1)^i.$$

□

7.2 Matchings

7.2.1 Hall's Marriage Theorem

Theorem 103 (Hall)**.** *Let $G = (L \sqcup R, E)$ be bipartite. There is a matching saturating L if and only if*

$$\boxed{|N(S)| \geq |S| \qquad \text{for every } S \subseteq L.}$$

Proof. Necessity is immediate: if a matching saturates S, its matched neighbors are $|S|$ distinct elements of $N(S)$.

For sufficiency, let M be a maximum matching. Suppose some $x_0 \in L$ is unmatched.

Let $S \subseteq L$ and $T \subseteq R$ be the vertices reachable from x_0 by alternating paths, starting with an edge outside M.

No vertex of T is unmatched, since otherwise the alternating path to it would be an augmenting path, contradicting maximality of M.

Moreover,

$$\boxed{T = N(S).}$$

Indeed, every reached right vertex is clearly a neighbor of S. Conversely, from any $x \in S$, every nonmatching edge can extend the alternating path to its other endpoint; the unique matching edge incident to x, if any, already lies on the alternating path.

Finally, M gives a bijection

$$T \longleftrightarrow S \setminus \{x_0\}.$$

Every $y \in T$ is matched, and its matched partner is reachable and hence lies in $S \setminus \{x_0\}$; conversely every $x \in S \setminus \{x_0\}$ was reached through its matching edge.

Therefore

$$|N(S)| = |T| = |S| - 1,$$

contradicting Hall's condition. Hence M saturates L. □

7.2.2 Gale–Ryser Theorem

Hall's theorem generalizes naturally from choosing one neighbor for every left vertex to prescribing all degrees on both sides.

Theorem 104 (Bipartite Degree Realization)**.** *Let $G = (L \sqcup R, E)$ be bipartite, and let*

$$a : L \to \mathbb{Z}_{\geq 0}, \qquad b : R \to \mathbb{Z}_{\geq 0}$$

satisfy

$$\sum_{u \in L} a_u = \sum_{v \in R} b_v = D.$$

There is a subgraph $H \subseteq G$ with

$$\deg_H(u) = a_u \quad (u \in L), \qquad \deg_H(v) = b_v \quad (v \in R)$$

if and only if for every $X \subseteq L$ and $Y \subseteq R$,

$$\boxed{a(X) \leq b(Y) + |E(X, R \setminus Y)|.}$$

Proof. Construct the network

$$s \longrightarrow L \longrightarrow R \longrightarrow t$$

with capacities

$$c(s, u) = a_u, \qquad c(u, v) = 1 \quad (uv \in E), \qquad c(v, t) = b_v.$$

An integral flow of value D chooses exactly a_u edges incident to every $u \in L$ and exactly b_v edges incident to every $v \in R$. Thus such a flow is exactly the desired subgraph.

By max-flow min-cut, it remains to characterize when every cut has capacity at least D.

Any cut is determined by sets $X \subseteq L$ and $Y \subseteq R$ placed on the source side. Its capacity is

$$a(L \setminus X) + |E(X, R \setminus Y)| + b(Y).$$

Since $a(L) = D$, this is at least D exactly when

$$a(X) \leq b(Y) + |E(X, R \setminus Y)|.$$

Thus the stated inequalities are precisely the min-cut conditions. Integrality of max flow completes the proof. □

7.2.3 Cantor–Schröder–Bernstein

An injection in each direction should let us pair two sets perfectly, but neither injection need be surjective. View the injections as two matchings in a bipartite graph. One already covers the right side; repair its unmatched left vertices by switching matchings along the alternating chains they start. The chains may be infinite, but the switch can be specified explicitly.

Theorem 105 (Cantor–Schröder–Bernstein)**.** *If there are injections $f : A \to B$ and $g : B \to A$, then there is a bijection $h : A \to B$. Explicitly, put*

$$A_0 = A \setminus g(B), \qquad A_{j+1} = g(f(A_j)), \qquad C = \bigcup_{j \geq 0} A_j.$$

Then a bijection is

$$\boxed{h(a) = \begin{cases} f(a), & a \in C, \\ g^{-1}(a), & a \notin C. \end{cases}}$$

Here g^{-1} is defined on $g(B)$.

Proof. The definition of C gives $C = A_0 \sqcup g(f(C))$: its initial part is outside $g(B)$, and all later parts are inside. Consequently, using injectivity of g,

$$\begin{aligned} g(B \setminus f(C)) &= g(B) \setminus g(f(C)) \\ &= (A \setminus A_0) \setminus g(f(C)) = A \setminus C. \end{aligned}$$

Thus f maps C bijectively onto $f(C)$, while g^{-1} maps $A \setminus C$ bijectively onto $B \setminus f(C)$. These two maps have disjoint domains covering A and disjoint images covering B, so their union is the stated bijection. □

Consider disjoint copies of A and B, with edges $a \to f(a)$ and $b \to g(b)$. The g-edges already pair every vertex of B with one of A, leaving exactly A_0 unmatched. From $a_0 \in A_0$, follow

$$a_0 \xrightarrow{f} b_0 \xrightarrow{g} a_1 \xrightarrow{f} b_1 \xrightarrow{g} a_2 \longrightarrow \cdots.$$

Every vertex has at most one incoming edge and exactly one outgoing edge in this directed graph. These chains therefore cannot branch, merge, or repeat a vertex: tracing a repetition backward would give an incoming edge at their starting vertex in A_0. Switch from the g-edges to the f-edges along all these chains. Every affected B vertex stays matched, the starting A vertex becomes matched, and every later A vertex is rematched rather than lost. The affected A vertices are exactly C.

Outside these chains, retain the g-edges. This handles cycles, doubly infinite chains, and chains starting on the B side without choosing an orientation separately for each component. The formula for C performs all the switches at once.

Corollary 106 (Combining two saturating matchings)**.** *In any bipartite graph with sides A, B, possibly infinite, if one matching covers A and another covers B, their union contains a perfect matching.*

Proof. Let $f(a)$ be the partner of a in the first matching and $g(b)$ the partner of b in the second. Both are injections. The bijection above uses either the edge $af(a)$ or the edge $g(b)b$ with $g(b) = a$, so all its matched pairs belong to the union of the two matchings. □

7.2.4 Brégman's Theorem

For a 0–1 matrix A, its permanent counts the perfect matchings in the corresponding bipartite graph.

Theorem 107 (Brégman's Theorem)**.** *Let*

$$A \in \{0,1\}^{n \times n}$$

have row sums

$$r_1, \ldots, r_n.$$

Then

$$\boxed{\operatorname{per}(A) \le \prod_{i=1}^{n} (r_i!)^{1/r_i}.}$$

Proof. Let $\mathcal{M}$ be the set of perfect matchings represented by A, and choose a uniformly random

$$\sigma \in \mathcal{M}.$$

Then

$$H(\sigma) = \log_2 |\mathcal{M}| = \log_2 \operatorname{per}(A).$$

Independently choose a uniformly random ordering of the rows. Reveal the values $\sigma(i)$ in this random order.

When row i is revealed, let R_i be the number of currently unused columns j such that

$$A_{ij} = 1.$$

Conditioned on everything already revealed, $\sigma(i)$ has at most R_i possible values. Thus

$$H(\sigma(i) \mid \text{earlier rows}) \leq \mathbb{E}[\log_2 R_i].$$

By the chain rule,

$$H(\sigma) \leq \sum_{i=1}^{n} \mathbb{E}[\log_2 R_i].$$

Now fix both i and the matching σ. For every allowed column

$$j \in N(i),$$

there is a unique row

$$\sigma^{-1}(j)$$

matched to j. These give exactly r_i rows, one of which is i itself.

A column $j \in N(i)$ is still available when row i is exposed exactly when the row $\sigma^{-1}(j)$ has not appeared earlier. Therefore R_i is the number of these r_i distinguished rows appearing at or after i.

Since the row order is uniformly random, the rank of i among these r_i rows is uniform. Consequently,

$$R_i \quad \text{is uniform on} \quad \{1, 2, \ldots, r_i\}.$$

Hence

$$\mathbb{E}[\log_2 R_i] = \frac{1}{r_i} \sum_{j=1}^{r_i} \log_2 j = \frac{1}{r_i} \log_2(r_i!).$$

Therefore

$$\log_2 \operatorname{per}(A) \leq \sum_{i=1}^{n} \frac{\log_2(r_i!)}{r_i}.$$

Exponentiating gives

$$\operatorname{per}(A) \leq \prod_{i=1}^{n} (r_i!)^{1/r_i}.$$

□

7.2.5 Kahn–Lovász Theorem

Brégman's inequality bounds perfect matchings in bipartite graphs. The same degree bound extends to arbitrary simple graphs, with each vertex contributing half the exponent. The reason for this square root is combinatorial: two perfect matchings superimpose to form even cycles, and directed cycle covers are exactly what the permanent counts.

Write $\operatorname{pm}(G)$ for the number of perfect matchings of G.

Lemma 108 (Squaring the matching count)**.** *Let G be a finite simple graph on an even number of vertices, and let A be its full adjacency matrix. Then*

$$\boxed{\operatorname{pm}(G)^2 \leq \operatorname{per}(A).}$$

More precisely, $\operatorname{pm}(G)^2$ *equals the number of permutations counted by* $\operatorname{per}(A)$ *whose cycles all have even length.*

Proof. The square $\mathrm{pm}(G)^2$ counts ordered pairs (M_1, M_2) of perfect matchings. Retaining two copies of every common edge, their union is a spanning multigraph whose components are doubled edges and even cycles of length at least four.

Fix such a union H, and let $c(H)$ count its cycles of length at least four. Each of these cycles has exactly two ways to assign its edges alternately to M_1 and M_2. A doubled edge must occur in both matchings. Thus exactly $2^{c(H)}$ ordered pairs produce H.

On the other hand,

$$\mathrm{per}(A) = \sum_{\sigma \in S_{|V(G)|}} \prod_{v \in V(G)} A_{v,\sigma(v)}$$

counts permutations for which every v is adjacent to $\sigma(v)$. Such a permutation is a directed cycle cover: a two-cycle uses one undirected edge in both directions, while every longer cycle has two possible orientations.

Consequently, the permutations having only even cycles produce exactly the same unions H, each with the same multiplicity $2^{c(H)}$. Their number is therefore $\mathrm{pm}(G)^2$. The permanent may additionally count cycle covers containing odd cycles, which proves the inequality. □

The key correspondence is that an even cycle has two alternating assignments to the matchings and two orientations. These choices account for the same multiplicity.

Theorem 109 (Kahn–Lovász)**.** *Let G be a finite simple graph on $2n$ vertices, with no isolated vertices, and write $d_v = \deg_G(v)$. Then*

$$\boxed{\mathrm{pm}(G) \le \prod_{v \in V(G)} (d_v!)^{1/(2d_v)}.}$$

The bound is attained by every disjoint union of balanced complete bipartite graphs.

Proof. The row sums of the full adjacency matrix A are the vertex degrees d_v. By Lemma 108 and Brégman's inequality,

$$\mathrm{pm}(G)^2 \le \mathrm{per}(A) \le \prod_{v \in V(G)} (d_v!)^{1/d_v}.$$

Take square roots.

For sharpness, $K_{d,d}$ has $d!$ perfect matchings, while its $2d$ vertices contribute

$$\prod_{v \in V(K_{d,d})} (d!)^{1/(2d)} = d!.$$

Both the matching count and the degree product multiply over disjoint components. □

If G has an isolated vertex, then $\mathrm{pm}(G) = 0$, so that case is immediate.

Corollary 110 (Regular graphs)**.** *Every simple d-regular graph on $2n$ vertices, where $d \ge 1$, satisfies*

$$\boxed{\mathrm{pm}(G) \le (d!)^{n/d}.}$$

When $d \mid n$, equality is attained by the disjoint union of n/d copies of $K_{d,d}$.

Proof. There are $2n$ identical factors in the Kahn–Lovász bound. □

7.2.6 The van der Waerden Permanent Theorem

The permanent of a nonnegative matrix is a weighted sum of perfect matchings, with no signs to cancel them as in the determinant. A doubly stochastic matrix has every row and column sum equal to one. The van der Waerden theorem says that even the least favorable such weights leave at least as much total matching weight as the uniform matrix.

Theorem 111 (van der Waerden)**.** *For every doubly stochastic $A = (a_{ij}) \in \mathbb{R}_{\geq 0}^{n \times n}$, with $n \geq 1$,*

$$\boxed{\operatorname{per} A := \sum_{\sigma \in S_n} \prod_{i=1}^{n} a_{i,\sigma(i)} \geq \frac{n!}{n^n}.}$$

The bound is attained when every entry is $1/n$.

The useful object is the polynomial

$$p_A(x) = \prod_{i=1}^{n} \left(\sum_{j=1}^{n} a_{ij} x_j \right), \qquad [x_1 \cdots x_n] p_A = \operatorname{per} A.$$

Selecting one term from each row uses every variable exactly once precisely when the chosen columns form a permutation. To extract this coefficient, we can repeatedly differentiate in one variable and then set it to zero. After the first step the polynomial need no longer be a product of linear forms. We therefore need a larger class that survives coefficient extraction, and a quantity that does not decrease too much at each step.

Capacity For a nonzero homogeneous polynomial p of degree d in d variables with nonnegative real coefficients, define

$$\operatorname{Cap}(p) = \inf_{x_1, \ldots, x_d > 0} \frac{p(x)}{x_1 \cdots x_d} = \inf_{\substack{x_i > 0 \\ x_1 \cdots x_d = 1}} p(x).$$

The equality follows by scaling all coordinates by the same positive factor. Nonnegativity immediately gives $[x_1 \cdots x_d] p \leq \operatorname{Cap}(p)$: the desired monomial is one of the nonnegative terms in the quotient. We need the reverse inequality up to a controlled factor.

For the polynomial p_A, weighted AM–GM gives, for every positive x,

$$p_A(x) \geq \prod_{i=1}^{n} \prod_{j=1}^{n} x_j^{a_{ij}} = \prod_{j=1}^{n} x_j^{\sum_i a_{ij}} = x_1 \cdots x_n.$$

Since $p_A(1, \ldots, 1) = 1$, we have $\operatorname{Cap}(p_A) = 1$. Thus it suffices to retain a fraction $n!/n^n$ of capacity while extracting all variables.

The following one-variable inequality gives exactly the loss needed at one extraction. Its hypothesis on the roots will be justified afterward.

Lemma 112 (The cost of extracting a linear coefficient)**.** *Let F be a real polynomial of degree $d \geq 2$, with $F(0) > 0$ and all roots strictly negative and real. Then*

$$\boxed{F'(0) \geq \left(\frac{d-1}{d} \right)^{d-1} \inf_{t > 0} \frac{F(t)}{t}.}$$

Proof. Write $F(t) = F(0) \prod_{j=1}^{d} (1 + b_j t)$ with $b_j > 0$, and put $s = \sum_j b_j = F'(0)/F(0)$. AM–GM bounds the product from above:

$$F(t) \leq F(0) \left(1 + \frac{st}{d} \right)^d.$$

Evaluate this bound at $t = d/((d-1)s)$, the minimizer of its right side divided by t. This yields

$$\inf_{t>0} \frac{F(t)}{t} \le F(0) \frac{(d-1)s}{d} \left(\frac{d}{d-1} \right)^d = F'(0) \left(\frac{d}{d-1} \right)^{d-1}.$$

Rearrange. The loss comes from replacing the individual b_j by their average; equality in that replacement holds when they are all equal. □

Root Condition A homogeneous real polynomial is *half-plane stable* here if $p(z) \neq 0$ whenever every $\operatorname{Re} z_i > 0$. A product of nonzero linear forms with nonnegative coefficients has this property: each form has strictly positive real part. In particular, p_A is stable.

For the inductive proof, temporarily impose two stronger conditions: every degree-d monomial has a positive coefficient, and

$$p(z) \neq 0 \quad \text{if all } \operatorname{Re} z_i \ge 0 \text{ and } \sum_i \operatorname{Re} z_i > 0. \tag{47}$$

Call the latter condition *strict half-plane stability*. We will remove both extra assumptions by a simple substitution.

Lemma 113 (Real roots and closure under extraction)**.** *Suppose p has degree $d \ge 2$ in d variables and satisfies these two stronger conditions. Then, for every $y \in \mathbb{R}^{d-1}_{>0}$, $F(t) = p(y,t)$ has degree d and all its roots are strictly negative and real. Moreover,*

$$q(x_1, \dots, x_{d-1}) = \left. \frac{\partial p}{\partial x_d} \right|_{x_d = 0}$$

is homogeneous of degree $d-1$, has every degree-$(d-1)$ coefficient positive, and is strictly half-plane stable.

Proof. The coefficient of t^d is the positive coefficient of x_d^d in p, independent of the other coordinates. Also $F(0) > 0$. If a nonzero root t were not on the negative real axis, write $t = |t|e^{i\theta}$ with $-\pi < \theta < \pi$. Multiplication by $\lambda = e^{-i\theta/2}$ puts both $\lambda y_1, \dots, \lambda y_{d-1}$ and λt in the open right half-plane. Homogeneity would give $p(\lambda y, \lambda t) = \lambda^d p(y,t) = 0$, contradicting stability. Thus all roots are negative and real.

To prove closure, now fix a possibly complex vector z with $\operatorname{Re} z_i \ge 0$ and $\sum_i \operatorname{Re} z_i > 0$. The polynomial $F(t) = p(z,t)$ still has degree d, and the strict condition makes it nonzero whenever $\operatorname{Re} t \ge 0$. All its roots $\lambda_1, \dots, \lambda_d$ therefore have negative real part. Its logarithmic derivative at zero is

$$\frac{q(z)}{p(z,0)} = \frac{F'(0)}{F(0)} = \sum_{j=1}^{d} \frac{1}{-\lambda_j}.$$

Every summand has strictly positive real part, so the sum is nonzero. This proves $q(z) \neq 0$. Its degree and positive coefficients follow because the operation selects exactly the monomials with exponent one in x_d. □

Capacity Losses Telescope Put $G(d) = ((d-1)/d)^{d-1}$ for $d \ge 2$. For p, q as in the preceding lemma, fix $y > 0$ and apply the one-variable bound:

$$q(y) \ge G(d) \inf_{t>0} \frac{p(y,t)}{t} \ge G(d) \operatorname{Cap}(p) \prod_{i=1}^{d-1} y_i.$$

Divide by the product of the coordinates and take the infimum. Thus

$$\boxed{\operatorname{Cap}(q) \ge G(d) \operatorname{Cap}(p).} \tag{48}$$

The polynomial class and the capacity bound both survive the same operation, so we can repeat it.

Theorem 114 (Gurvits's capacity bound)**.** *If p is a nonzero homogeneous half-plane-stable polynomial of degree n in n variables with nonnegative real coefficients, then*

$$\boxed{[x_1 \cdots x_n]p \geq \frac{n!}{n^n} \operatorname{Cap}(p).}$$

Proof. First assume all degree-n coefficients are positive and p is strictly half-plane stable. Repeatedly differentiate in the last variable and set it to zero. The resulting polynomials satisfy the same conditions by Lemma 113. After extracting $x_n, \ldots, x_2$, the last polynomial is ax_1, where $a = [x_1 \cdots x_n]p$, and its capacity is a. Iterating (48) gives

$$a \geq \operatorname{Cap}(p) \prod_{d=2}^{n} \left(\frac{d-1}{d}\right)^{d-1} = \operatorname{Cap}(p) \frac{(n-1)!}{n^{n-1}} = \operatorname{Cap}(p) \frac{n!}{n^n}.$$

For $n = 1$ the equality is immediate and the product is empty.

For the general case, set $S = x_1 + \cdots + x_n$ and, for $\varepsilon > 0$,

$$p_\varepsilon(x) = p(x_1 + \varepsilon S, \ldots, x_n + \varepsilon S).$$

If all real parts of z are nonnegative and their sum is positive, every $z_i + \varepsilon \sum_j z_j$ has positive real part. Thus p_ε is strictly half-plane stable. Expanding any nonzero monomial of p and choosing εS in all its n factors shows that every degree-n coefficient of p_ε is positive. The same expansion retains all original terms with nonnegative additions, so $p_\varepsilon \geq p$ coefficientwise and $\operatorname{Cap}(p_\varepsilon) \geq \operatorname{Cap}(p)$. The proved case therefore gives

$$[x_1 \cdots x_n]p_\varepsilon \geq \frac{n!}{n^n} \operatorname{Cap}(p).$$

Let $\varepsilon \downarrow 0$. Coefficients converge to those of p, proving the bound. □

Proof of Theorem 111. The product of row forms p_A is homogeneous, nonnegative, and half-plane stable. The AM–GM calculation gives $\operatorname{Cap}(p_A) = 1$, and its $x_1 \cdots x_n$ coefficient is $\operatorname{per} A$. Apply Theorem 114. For the uniform matrix, each of the $n!$ permutation terms is n^{-n}, so the bound is attained. □

Corollary 115 (Perfect matchings in regular bipartite graphs)**.** *An r-regular bipartite graph with n vertices on each side and $r \geq 1$ has at least $r^n n!/n^n$ perfect matchings.*

Proof. If B is its bipartite adjacency matrix, B/r is doubly stochastic and $\operatorname{per}(B/r) = r^{-n} \operatorname{per} B$. Apply the theorem. The same statement holds for a multigraph when parallel edges are counted as distinct choices. □

7.3 Concentration Inequalities

Most concentration inequalities come from the same idea:

$$\Pr[X \geq t] = \Pr[e^{\theta X} \geq e^{\theta t}] \leq e^{-\theta t} \mathbb{E} e^{\theta X},$$

followed by a bound on the moment generating function.

7.3.1 Hoeffding

Lemma 116 (Hoeffding's Lemma)**.** *If $\mathbb{E}X = 0$ and $a \leq X \leq b$, then for every $\theta \in \mathbb{R}$,*

$$\boxed{\mathbb{E} e^{\theta X} \leq \exp\left(\frac{\theta^2 (b-a)^2}{8}\right).}$$

Proof. Let

$$\psi(\theta) = \log \mathbb{E}e^{\theta X}$$

be the cumulant generating function. Define the modified expectation

$$\mathbb{E}_\theta[g(X)] = \frac{\mathbb{E}[g(X)e^{\theta X}]}{\mathbb{E}e^{\theta X}}.$$

Differentiating gives

$$\psi'(\theta) = \mathbb{E}_\theta X$$

and

$$\boxed{\psi''(\theta) = \mathbb{E}_\theta X^2 - (\mathbb{E}_\theta X)^2 = \mathrm{Var}_\theta(X).}$$

The modification changes the distribution of X, but not its support, so under the modified distribution we still have $a \le X \le b$. Any random variable $Y \in [a, b]$ satisfies

$$\mathrm{Var}(Y) = \min_c \mathbb{E}(Y - c)^2 \le \mathbb{E}\left(Y - \frac{a+b}{2}\right)^2 \le \frac{(b-a)^2}{4}.$$

Therefore, for every θ,

$$\psi''(\theta) \le \frac{(b-a)^2}{4}.$$

Finally,

$$\psi(0) = 0, \qquad \psi'(0) = \mathbb{E}X = 0.$$

By Taylor's theorem, for some ξ between 0 and θ,

$$\psi(\theta) = \psi(0) + \theta\psi'(0) + \frac{\theta^2}{2}\psi''(\xi).$$

Therefore

$$\psi(\theta) = \frac{\theta^2}{2}\psi''(\xi) \le \frac{\theta^2}{2} \cdot \frac{(b-a)^2}{4} = \frac{\theta^2(b-a)^2}{8}.$$

Exponentiating gives

$$\mathbb{E}e^{\theta X} \le \exp\left(\frac{\theta^2(b-a)^2}{8}\right).$$

□

Theorem 117 (Hoeffding's Inequality)**.** *Let $X_1, \ldots, X_n$ be independent with*

$$a_i \le X_i \le b_i.$$

Then

$$\boxed{\Pr\left[\sum_i (X_i - \mathbb{E}X_i) \ge t\right] \le \exp\left(-\frac{2t^2}{\sum_i (b_i - a_i)^2}\right).}$$

The same bound holds for the lower tail.

Proof. Apply Lemma 116 independently:

$$\mathbb{E}e^{\theta \sum_i (X_i - \mathbb{E}X_i)} \le \exp\left(\frac{\theta^2}{8}\sum_i (b_i - a_i)^2\right).$$

The exponential-moment bound gives

$$\Pr[\cdots] \le \exp\left(-\theta t + \frac{\theta^2}{8}\sum_i (b_i - a_i)^2\right).$$

Optimize over θ. □

7.3.2 Chernoff

Theorem 118 (Chernoff Bounds)**.** *Let*

$$X = \sum_i X_i$$

where the X_i are independent Bernoulli variables, and let

$$\mu = \mathbb{E}X.$$

Then for $\delta \ge 0$,

$$\boxed{\Pr[X \ge (1+\delta)\mu] \le \left(\frac{e^\delta}{(1+\delta)^{1+\delta}}\right)^\mu \le \exp\left(-\frac{\delta^2}{2+\delta}\mu\right).}$$

For $0 \le \delta \le 1$,

$$\boxed{\Pr[X \le (1-\delta)\mu] \le e^{-\delta^2\mu/2}.}$$

Proof. For a Bernoulli variable of mean p,

$$\mathbb{E}e^{\theta X_i} = 1 + p(e^\theta - 1) \le \exp(p(e^\theta - 1)).$$

Hence

$$\mathbb{E}e^{\theta X} \le \exp(\mu(e^\theta - 1)).$$

Therefore

$$\Pr[X \ge (1+\delta)\mu] \le \exp(-\theta(1+\delta)\mu + \mu(e^\theta - 1)).$$

Set

$$e^\theta = 1 + \delta.$$

This gives the exact first bound. The simpler forms follow from elementary calculus.

The lower tail is identical with $\theta < 0$. □

7.3.3 Bernstein

Lemma 119 (Bernstein MGF Bound)**.** *If*

$$\mathbb{E}X = 0, \qquad |X| \le R,$$

then for

$$0 \le \theta < 3/R,$$

$$\boxed{\log \mathbb{E}e^{\theta X} \le \frac{\theta^2 \mathbb{E}X^2}{2(1 - \theta R/3)}.}$$

Proof. For $k \geq 2$,

$$k! \geq 2 \cdot 3^{k-2},$$

and

$$|X|^k \leq R^{k-2} X^2.$$

Thus

$$\begin{aligned} e^{\theta X} &\leq 1 + \theta X + X^2 \sum_{k \geq 2} \frac{\theta^k R^{k-2}}{k!} \\ &\leq 1 + \theta X + \frac{\theta^2 X^2}{2(1 - \theta R/3)}. \end{aligned}$$

Take expectations, use $\mathbb{E}X = 0$, and use

$$1 + x \leq e^x.$$

□

Theorem 120 (Bernstein's Inequality)**.** *Let $X_1, \ldots, X_n$ be independent, mean zero, and satisfy*

$$|X_i| \leq R.$$

Set

$$\sigma^2 = \sum_i \mathbb{E}X_i^2.$$

Then

$$\boxed{\Pr\left[\sum_i X_i \geq t\right] \leq \exp\left(-\frac{t^2}{2(\sigma^2 + Rt/3)}\right).}$$

Proof. By Lemma 119,

$$\Pr\left[\sum_i X_i \geq t\right] \leq \exp\left(-\theta t + \frac{\theta^2 \sigma^2}{2(1 - \theta R/3)}\right).$$

Take

$$\theta = \frac{t}{\sigma^2 + Rt/3}.$$

□

7.4 Martingales

The exponential-moment method extends almost unchanged to martingales. The main difference is that moment-generating-function bounds are applied conditionally at each step.

Lemma 121 (Bounded Optional Stopping)**.** *Let $(M_k)_{k=0}^n$ be a supermartingale and let $\tau \leq n$ be a stopping time. Then*

$$\boxed{\mathbb{E}M_\tau \leq \mathbb{E}M_0.}$$

For a martingale, equality holds.

Proof. Write

$$M_\tau - M_0 = \sum_{i=1}^n \mathbf{1}_{\{\tau \geq i\}}(M_i - M_{i-1}).$$

Since

$$\{\tau \geq i\} \in \mathcal{F}_{i-1},$$

we have

$$\begin{aligned}\mathbb{E}[M_\tau - M_0] &= \sum_{i=1}^{n} \mathbb{E}\left[\mathbf{1}_{\{\tau \geq i\}} \mathbb{E}[M_i - M_{i-1} \mid \mathcal{F}_{i-1}]\right] \\ &\leq 0.\end{aligned}$$

For a martingale, every term is zero. □

A useful consequence is the martingale analogue of the maximal form of Markov's inequality.

Lemma 122 (Ville's Inequality)**.** *Let $(M_k)_{k=0}^n$ be a nonnegative supermartingale. Then for every $a > 0$,*

$$\boxed{\Pr\left[\max_{0 \leq k \leq n} M_k \geq a\right] \leq \frac{\mathbb{E}M_0}{a}.}$$

Proof. Let τ be the first k such that $M_k \geq a$, taking $\tau = n$ if this never occurs. By bounded optional stopping,

$$\mathbb{E}M_\tau \leq \mathbb{E}M_0.$$

On the event

$$\max_{k \leq n} M_k \geq a$$

we have $M_\tau \geq a$, while $M_\tau \geq 0$ always. Hence

$$a \Pr\left[\max_{k \leq n} M_k \geq a\right] \leq \mathbb{E}M_\tau \leq \mathbb{E}M_0.$$

□

7.4.1 Azuma and McDiarmid

Azuma's inequality is Hoeffding's inequality with independence replaced by the martingale property.

Theorem 123 (Maximal Azuma–Hoeffding)**.** *Let*

$$Y_k = \sum_{i=1}^{k} X_i, \qquad Y_0 = 0,$$

where

$$\mathbb{E}[X_i \mid \mathcal{F}_{i-1}] = 0$$

and

$$a_i \leq X_i \leq b_i.$$

Then

$$\boxed{\Pr\left[\max_{0 \leq k \leq n} Y_k \geq t\right] \leq \exp\left(-\frac{2t^2}{\sum_{i=1}^{n}(b_i - a_i)^2}\right).}$$

The same bound holds for

$$\Pr\left[\min_{0 \leq k \leq n} Y_k \leq -t\right].$$

Consequently,

$$\Pr\left[\max_{0 \leq k \leq n} |Y_k| \geq t\right] \leq 2\exp\left(-\frac{2t^2}{\sum_{i=1}^{n}(b_i - a_i)^2}\right).$$

Proof. Set

$$c_i = b_i - a_i.$$

Conditionally on $\mathcal{F}_{i-1}$, Hoeffding's lemma gives

$$\mathbb{E}[e^{\theta X_i} \mid \mathcal{F}_{i-1}] \le e^{\theta^2 c_i^2/8}.$$

Therefore

$$M_k = \exp\left(\theta Y_k - \frac{\theta^2}{8}\sum_{i=1}^{k} c_i^2\right)$$

is a nonnegative supermartingale with $M_0 = 1$.

If $Y_k \ge t$ for some $k \le n$, then

$$M_k \ge \exp\left(\theta t - \frac{\theta^2}{8}\sum_{i=1}^{n} c_i^2\right).$$

Thus Ville's inequality gives

$$\Pr\left[\max_{k\le n} Y_k \ge t\right] \le \exp\left(-\theta t + \frac{\theta^2}{8}\sum_{i=1}^{n} c_i^2\right).$$

Optimizing at

$$\theta = \frac{4t}{\sum_i c_i^2}$$

gives the claimed bound.

Apply the same argument to $-Y_k$ for the lower tail, and union bound the two tails for the final statement. □

Theorem 124 (McDiarmid's Inequality)**.** *Let $Z_1, \dots, Z_n$ be independent, and suppose*

$$|f(z) - f(z')| \le c_i$$

whenever z, z' differ only in coordinate i. Then

$$\boxed{\Pr[f(Z) - \mathbb{E}f(Z) \ge t] \le \exp\left(-\frac{2t^2}{\sum_i c_i^2}\right).}$$

Proof. Consider the Doob martingale

$$M_i = \mathbb{E}[f(Z) \mid Z_1, \dots, Z_i].$$

Fixing $Z_1, \dots, Z_{i-1}$, regard M_i as a function of Z_i. For any two possible values z_i, z_i',

$$|\mathbb{E}[f(Z) \mid Z_i = z_i, Z_1, \dots, Z_{i-1}] - \mathbb{E}[f(Z) \mid Z_i = z_i', Z_1, \dots, Z_{i-1}]| \le c_i,$$

because the two expectations can be coupled using the same values of $Z_{i+1}, \dots, Z_n$.

Thus, conditioned on $\mathcal{F}_{i-1}$, M_i ranges over an interval of length at most c_i. Since

$$M_{i-1} = \mathbb{E}[M_i \mid \mathcal{F}_{i-1}],$$

the martingale difference $M_i - M_{i-1}$ also lies in an interval of length at most c_i.

Now

$$M_n - M_0 = f(Z) - \mathbb{E}f(Z),$$

so Theorem 123 gives the result. □

7.4.2 Freedman's Inequality

Azuma only uses a worst-case bound on each martingale increment. When the increments usually have much smaller conditional variance, Freedman's inequality gives the martingale analogue of Bernstein's inequality.

Let

$$Y_k = \sum_{i=1}^{k} X_i, \qquad V_k = \sum_{i=1}^{k} \mathbb{E}[X_i^2 \mid \mathcal{F}_{i-1}],$$

where

$$\mathbb{E}[X_i \mid \mathcal{F}_{i-1}] = 0.$$

The quantity V_k is the accumulated conditional variance.

We first record the conditional version of the Bernstein MGF bound.

Lemma 125 (Conditional Bernstein MGF Bound)**.** *Suppose*

$$\mathbb{E}[X \mid \mathcal{F}] = 0, \qquad X \le R$$

almost surely, where $R > 0$. Then for

$$0 \le \theta < 3/R,$$

$$\boxed{\mathbb{E}[e^{\theta X} \mid \mathcal{F}] \le \exp\left(\frac{\theta^2 \mathbb{E}[X^2 \mid \mathcal{F}]}{2(1 - \theta R/3)}\right).}$$

Proof. For $\theta \ge 0$ and $x \le R$,

$$e^{\theta x} \le 1 + \theta x + \frac{e^{\theta R} - \theta R - 1}{R^2} x^2.$$

Taking conditional expectation and using

$$\mathbb{E}[X \mid \mathcal{F}] = 0$$

gives

$$\mathbb{E}[e^{\theta X} \mid \mathcal{F}] \le 1 + \frac{e^{\theta R} - \theta R - 1}{R^2} \mathbb{E}[X^2 \mid \mathcal{F}].$$

Since

$$1 + x \le e^x$$

and, for $\theta R < 3$,

$$e^{\theta R} - \theta R - 1 \le \frac{(\theta R)^2}{2(1 - \theta R/3)},$$

the result follows. □

Theorem 126 (Freedman's Inequality)**.** *Suppose*

$$\mathbb{E}[X_i \mid \mathcal{F}_{i-1}] = 0$$

and

$$X_i \le R$$

almost surely. Then

$$\boxed{\Pr\left[\exists k \le n : Y_k \ge t \text{ and } V_k \le \sigma^2\right] \le \exp\left(-\frac{t^2}{2(\sigma^2 + Rt/3)}\right).}$$

Proof. Fix

$$0 \le \theta < 3/R$$

and write

$$g(\theta) = \frac{\theta^2}{2(1 - \theta R/3)}.$$

By Lemma 125,

$$\mathbb{E}[e^{\theta X_i} \mid \mathcal{F}_{i-1}] \le \exp\left(g(\theta)\mathbb{E}[X_i^2 \mid \mathcal{F}_{i-1}]\right).$$

Hence

$$M_k = \exp\left(\theta Y_k - g(\theta)V_k\right)$$

is a nonnegative supermartingale with $M_0 = 1$.

If for some $k \le n$,

$$Y_k \ge t, \qquad V_k \le \sigma^2,$$

then

$$M_k \ge \exp\left(\theta t - g(\theta)\sigma^2\right).$$

Ville's inequality therefore gives

$$\Pr\left[\exists k \le n : Y_k \ge t,\ V_k \le \sigma^2\right] \le \exp\left(-\theta t + g(\theta)\sigma^2\right).$$

Take

$$\theta = \frac{t}{\sigma^2 + Rt/3}.$$

When $\sigma^2 > 0$, this satisfies $\theta R < 3$, and

$$1 - \frac{\theta R}{3} = \frac{\sigma^2}{\sigma^2 + Rt/3}.$$

Therefore

$$\begin{aligned} -\theta t + g(\theta)\sigma^2 &= -\frac{t^2}{\sigma^2 + Rt/3} + \frac{t^2}{2(\sigma^2 + Rt/3)} \\ &= -\frac{t^2}{2(\sigma^2 + Rt/3)}. \end{aligned}$$

The case $\sigma^2 = 0$ follows by taking a limit. □

Thus Freedman has the same basic form as Bernstein:

$$\exp\left(-\Theta\left(\min\left\{\frac{t^2}{\sigma^2}, \frac{t}{R}\right\}\right)\right),$$

but it simultaneously controls every time k at which the accumulated conditional variance is at most σ^2.

If also

$$X_i \ge -R,$$

then applying the theorem to $-Y_k$ gives the corresponding lower-tail bound.

7.4.3 Generalized Hölder and Read-k Concentration

Functions of independent variables need not themselves be independent. When each underlying variable appears in only a bounded number of functions, Hölder's inequality controls the resulting dependence.

Theorem 127 (Generalized Hölder inequality)**.** *Let $X_1, \dots, X_m$ be independent random variables, and let $f_i = f_i(X_{S_i}) \geq 0$, where $S_i \subseteq [m]$. Suppose $0 < \theta_i \leq 1$ and*

$$\sum_{i:\, j \in S_i} \theta_i \leq 1 \qquad (j \in [m]).$$

Then

$$\boxed{\mathbb{E} \prod_i f_i^{\theta_i} \leq \prod_i (\mathbb{E} f_i)^{\theta_i}.}$$

Proof. Induct on the number of underlying variables. Fix all variables except X_m. Apply Hölder with exponents $1/\theta_i$ to the factors whose sets S_i contain m, adding a constant factor one if their weights sum to less than one. This gives

$$\mathbb{E}_{X_m} \prod_i f_i^{\theta_i} \leq \prod_{i:\, m \notin S_i} f_i^{\theta_i} \prod_{i:\, m \in S_i} (\mathbb{E}_{X_m} f_i)^{\theta_i}.$$

The new factors depend only on the remaining variables and satisfy the same weight constraints. Apply induction and use independence to identify the iterated expectations with $\mathbb{E} f_i$. The case with no underlying variables is equality. □

Corollary 128 (Bounded variable reuse)**.** *Suppose $f_1, \dots, f_r \geq 0$ are functions of independent variables, and each underlying variable occurs in at most k functions. Then*

$$\boxed{\mathbb{E} \prod_{i=1}^{r} f_i \leq \prod_{i=1}^{r} \left(\mathbb{E} f_i^k\right)^{1/k}.}$$

Proof. Apply Theorem 127 to f_i^k with $\theta_i = 1/k$. □

Concentration. A family $Y_1, \dots, Y_r$ is *read-k* if each Y_i is a function of independent underlying variables and each underlying variable appears in at most k of these functions.

For $a, p \in [0, 1]$, write

$$D(a \| p) = a \ln \frac{a}{p} + (1 - a) \ln \frac{1 - a}{1 - p},$$

with the usual limiting conventions.

Theorem 129 (Read-k Chernoff bound)**.** *Let $Y_1, \dots, Y_r \in [0, 1]$ form a read-k family, and put $S = \sum_i Y_i$ and $p = \mathbb{E} S / r$. Then*

$$\boxed{\Pr(S \geq ra) \leq \exp\left(-\frac{r}{k} D(a \| p)\right) \qquad (p \leq a \leq 1),}$$

and

$$\boxed{\Pr(S \leq ra) \leq \exp\left(-\frac{r}{k} D(a \| p)\right) \qquad (0 \leq a \leq p).}$$

In particular, for every $\varepsilon > 0$,

$$\Pr\left(|S - \mathbb{E} S| \geq \varepsilon r\right) \leq 2 \exp\left(-\frac{2\varepsilon^2 r}{k}\right).$$

Proof. The cases $p = 0, 1$ are deterministic, so assume $0 < p < 1$. Write $p_i = \mathbb{E}Y_i$. For every real t, Corollary 128 gives

$$\mathbb{E}e^{tS} \le \prod_i \left(\mathbb{E}e^{ktY_i}\right)^{1/k}.$$

Convexity on $[0, 1]$ gives $e^{ktu} \le 1 - u + ue^{kt}$. Therefore

$$\mathbb{E}e^{tS} \le \prod_i (1 - p_i + p_i e^{kt})^{1/k} \le (1 - p + pe^{kt})^{r/k},$$

where the last step uses concavity of $u \mapsto \ln(1 - u + ue^{kt})$.

Exponential Markov with $t \ge 0$ bounds the upper tail, and with $t \le 0$ bounds the lower tail. For $0 < a < 1$, the optimizing value satisfies

$$e^{kt} = \frac{a(1-p)}{p(1-a)}.$$

Substitution yields the relative-entropy bounds; the endpoints follow by limits.

Finally, as a function of a,

$$\frac{\partial^2}{\partial a^2} D(a\|p) = \frac{1}{a(1-a)} \ge 4,$$

while $D(p\|p) = 0$ and its first derivative there is zero. Hence $D(a\|p) \ge 2(a-p)^2$. Apply this to both tails. □

Writing $\mu = \mathbb{E}S$, the same calculation also gives the usual multiplicative forms with an extra factor k in the denominator of the exponent:

$$\Pr(S \ge (1+\delta)\mu) \le \left(\frac{e^\delta}{(1+\delta)^{1+\delta}}\right)^{\mu/k} \qquad (\delta \ge 0),$$

and

$$\Pr(S \le (1-\delta)\mu) \le \exp\left(-\frac{\delta^2\mu}{2k}\right) \qquad (0 \le \delta \le 1).$$

Indeed, before averaging the p_i's, use $1 + u \le e^u$ to obtain $\mathbb{E}e^{tS} \le \exp((\mu/k)(e^{kt} - 1))$, and optimize.

The factor k is necessary in general. Repeat each of several independent Bernoulli variables exactly k times. Their sum is k times a sum with only r/k independent terms.

7.5 Probabilistic Method

7.5.1 Isolation Lemma

Let

$$\mathcal{F} \subseteq 2^{[n]}$$

be any nonempty family of feasible sets. Assign independently chosen random integer weights

$$w_i \in [M]$$

to the ground elements, and define

$$w(S) = \sum_{i \in S} w_i.$$

Theorem 130 (Isolation Lemma)**.** *With probability at least*

$$\boxed{1 - \frac{n}{M},}$$

there is a unique minimum-weight set in $\mathcal{F}$.

In particular, choosing

$$M = 2n$$

gives uniqueness with probability at least $1/2$.

Proof. For each $i \in [n]$, fix all weights except w_i.

Let

$$A_i = \min_{\substack{S \in \mathcal{F} \\ i \in S}} \sum_{j \in S \setminus \{i\}} w_j$$

and

$$B_i = \min_{\substack{S \in \mathcal{F} \\ i \notin S}} w(S),$$

with the convention that a minimum over an empty family is $+\infty$.

Call i *confused* if

$$A_i + w_i = B_i.$$

Once all the other weights have been fixed, there is at most one value of w_i that can satisfy this equality. Therefore

$$\Pr[i \text{ is confused}] \leq \frac{1}{M}.$$

Now suppose the minimum-weight feasible set is not unique. Let S, T be two distinct minimum-weight members of $\mathcal{F}$, and choose

$$i \in S \triangle T.$$

Assume without loss of generality that

$$i \in S \setminus T.$$

Then

$$A_i + w_i \leq w(S), \qquad B_i \leq w(T).$$

But S and T already have globally minimum weight, so equality must hold:

$$A_i + w_i = B_i.$$

Thus some element is confused.

Hence

$$\Pr[\text{minimum is not unique}] \leq \Pr[\exists i \text{ confused}] \leq \sum_i \frac{1}{M} = \frac{n}{M}.$$

□

7.5.2 Random-Order Independent Sets

An independent set can be obtained by ordering the vertices and excluding the last vertex of every edge. The resulting events are dependent, but their dependence has the right sign: conditioned on one vertex's position, all the requirements for retaining it are increasing events in independent choices.

Lemma 131 (Harris's inequality)**.** *Let $X_1, \ldots, X_m$ be independent Bernoulli variables. If $f, g : \{0,1\}^m \to \mathbb{R}$ are both coordinatewise increasing, or both coordinatewise decreasing, then*

$$\mathbb{E}[fg] \geq \mathbb{E}[f]\mathbb{E}[g].$$

Consequently, for increasing events $E_1, \ldots, E_r$,

$$\Pr(E_1 \cap \cdots \cap E_r) \geq \prod_{i=1}^{r} \Pr(E_i).$$

Proof. Induct on m. Condition on X_m, and write $F_a = \mathbb{E}[f \mid X_m = a]$, $G_a = \mathbb{E}[g \mid X_m = a]$, and $p = \Pr(X_m = 1)$. By induction,

$$\mathbb{E}[fg] \geq (1-p)F_0G_0 + pF_1G_1.$$

The difference between this expression and $\mathbb{E}[f]\mathbb{E}[g]$ is $p(1-p)(F_1 - F_0)(G_1 - G_0)$, which is nonnegative. The case $m = 0$ is immediate.

For events, apply the inequality repeatedly to their indicators. The product of increasing indicators is again increasing. □

Theorem 132 (Weighted nonuniform independence bound)**.** *Let $H = (V, \mathcal{E})$ be a finite hypergraph with no empty edge. There is a random independent set I such that, for every $v \in V$,*

$$\boxed{\Pr(v \in I) \geq \int_0^1 \prod_{\substack{e \in \mathcal{E} \\ v \in e}} \bigl(1 - t^{|e|-1}\bigr)\, dt.}$$

In particular, for arbitrary weights $w_v \geq 0$,

$$\boxed{\alpha_w(H) \geq \sum_{v \in V} w_v \int_0^1 \prod_{\substack{e \in \mathcal{E} \\ v \in e}} \bigl(1 - t^{|e|-1}\bigr)\, dt,}$$

where $\alpha_w(H)$ is the maximum weight of an independent set.

Proof. Give the vertices independent uniform priorities $U_v \in [0, 1]$. Retain v precisely when it is not the last vertex of any edge containing it. Every edge loses its last vertex, so the retained set is independent.

Condition on $U_v = t$. For $u \neq v$, the indicators $X_u = \mathbf{1}_{\{U_u > t\}}$ are independent Bernoulli variables. For each incident edge e, retaining v requires

$$\sum_{u \in e \setminus \{v\}} X_u > 0.$$

This is an increasing event of probability $1 - t^{|e|-1}$. Harris's inequality bounds the probability that all these requirements hold by the product of their probabilities. Integrate over t.

Finally, $\alpha_w(H) \geq \mathbb{E} \sum_{v \in I} w_v = \sum_v w_v \Pr(v \in I)$. □

An empty product is one. A singleton edge makes the corresponding factor zero, correctly excluding its vertex.

Corollary 133 (Caro–Tuza)**.** *If H is r-uniform, with $r \geq 2$, then*

$$\boxed{\alpha_w(H) \geq \sum_{v \in V} w_v \binom{d(v) + 1/(r-1)}{d(v)}^{-1}.}$$

Here

$$\binom{d + 1/(r-1)}{d}^{-1} = \prod_{j=1}^{d} \frac{(r-1)j}{(r-1)j + 1},$$

with the empty product equal to one.

Proof. Put $q = r - 1$ and

$$J_d = \int_0^1 (1 - t^q)^d\, dt.$$

Then $J_0 = 1$. For $d \geq 1$, integrate by parts with

$$u = (1-t^q)^d, \qquad dv = dt.$$

Since

$$du = -qd\, t^{q-1}(1-t^q)^{d-1}\, dt, \qquad v = t,$$

and the boundary term $t(1-t^q)^d$ vanishes at both 0 and 1,

$$\begin{aligned} J_d &= qd \int_0^1 t^q (1-t^q)^{d-1}\, dt \\ &= qd \int_0^1 \big(1-(1-t^q)\big)(1-t^q)^{d-1}\, dt \\ &= qd(J_{d-1} - J_d). \end{aligned}$$

Therefore

$$(qd+1)J_d = qdJ_{d-1}, \qquad J_d = \frac{qd}{qd+1} J_{d-1}.$$

Iterating from $J_0 = 1$ gives

$$J_d = \prod_{j=1}^{d} \frac{qj}{qj+1}.$$

Finally,

$$\binom{d+1/q}{d} = \prod_{j=1}^{d} \frac{j+1/q}{j} = \prod_{j=1}^{d} \frac{qj+1}{qj},$$

so

$$J_d = \binom{d+1/q}{d}^{-1} = \binom{d+1/(r-1)}{d}^{-1}.$$

Substitute this value into Theorem 132. □

Corollary 134 (Caro–Wei)**.** *Every finite simple graph satisfies*

$$\boxed{\alpha_w(G) \geq \sum_{v \in V(G)} \frac{w_v}{d(v)+1}.}$$

In particular, if G has n vertices and m edges, then

$$\alpha(G) \geq \sum_v \frac{1}{d(v)+1} \geq \frac{n^2}{n+2m}.$$

Proof. Set $r = 2$. The last inequality is Cauchy–Schwarz. □

Weighted priorities for graphs. For graphs, the priorities can also be chosen to favor particular vertices. Give v an independent exponential priority of rate $a_v > 0$, and retain the local minima. This is an independent set, and

$$\Pr(v \in I) = \int_0^\infty a_v e^{-a_v t} \prod_{u \in N(v)} e^{-a_u t}\, dt = \frac{a_v}{\sum_{u \in N[v]} a_u}.$$

Consequently,

$$\boxed{\alpha_w(G) \geq \sum_v \frac{w_v a_v}{\sum_{u \in N[v]} a_u}.}$$

Taking $a_v = w_v + \varepsilon$ and letting $\varepsilon \downarrow 0$ gives

$$\alpha_w(G) \geq \sum_{v:\, w_v > 0} \frac{w_v^2}{\sum_{u \in N[v]} w_u}.$$

7.5.3 Counting Independent Sets in Regular Graphs

For a graph G, let $\mathcal{I}(G)$ be its independent sets and define

$$Z_G(\lambda) = \sum_{I \in \mathcal{I}(G)} \lambda^{|I|}.$$

Thus $Z_G(1)$ counts independent sets, while other values of λ give different weights to their sizes.

Theorem 135 (Kahn–Zhao)**.** *Let G be a finite simple d-regular graph on n vertices, where $d \geq 1$. For every $\lambda \geq 0$,*

$$\boxed{Z_G(\lambda) \leq \left(2(1+\lambda)^d - 1\right)^{n/(2d)}.}$$

In particular,

$$|\mathcal{I}(G)| \leq (2^{d+1} - 1)^{n/(2d)}.$$

Equality holds for disjoint unions of $K_{d,d}$.

Proof. The case $\lambda = 0$ is immediate, so assume $\lambda > 0$.

Bipartite graphs. Suppose G has parts L, R. Counting edges from either side gives $d|L| = d|R|$, so both parts have size $n/2$.

Fix the part $S \subseteq L$ of an independent set. The vertices that can still be chosen on the right are

$$U(S) = \{v \in R : N(v) \cap S = \varnothing\}.$$

Every subset of $U(S)$ is allowed, since there are no edges within R. Summing the weights of all completions gives

$$\begin{aligned} Z_G(\lambda) &= \sum_{S \subseteq L} \lambda^{|S|} \sum_{T \subseteq U(S)} \lambda^{|T|} \\ &= \sum_{S \subseteq L} \lambda^{|S|} (1+\lambda)^{|U(S)|}. \end{aligned}$$

We now express this sum as an expectation under independent choices. Choose each vertex of L with probability $\lambda/(1+\lambda)$. A particular subset S then has probability

$$\Pr(S) = \left(\frac{\lambda}{1+\lambda}\right)^{|S|} \left(\frac{1}{1+\lambda}\right)^{|L|-|S|} = \frac{\lambda^{|S|}}{(1+\lambda)^{|L|}}.$$

This absorbs the weight of the left part into the sampling distribution. The weight of its right-side completions remains inside the expectation:

$$Z_G(\lambda) = (1+\lambda)^{|L|} \mathbb{E}(1+\lambda)^{|U(S)|}.$$

For $v \in R$, put $f_v(S) = 1 + \lambda \mathbf{1}_{\{v \in U(S)\}}$. Then $(1+\lambda)^{|U(S)|} = \prod_{v \in R} f_v(S)$. The factor f_v depends only on the choices at the d neighbors of v, and each left-vertex choice appears in exactly d factors. By Corollary 128,

$$\mathbb{E} \prod_{v \in R} f_v \leq \prod_{v \in R} \left(\mathbb{E} f_v^d\right)^{1/d}.$$

All neighbors of v are absent with probability $(1+\lambda)^{-d}$. Hence

$$\begin{aligned} \mathbb{E} f_v^d &= \left(1 - (1+\lambda)^{-d}\right) + (1+\lambda)^{-d}(1+\lambda)^d \\ &= 2 - (1+\lambda)^{-d}. \end{aligned}$$

Consequently,

$$\begin{aligned} Z_G(\lambda) &\leq (1+\lambda)^{n/2} \left(2 - (1+\lambda)^{-d}\right)^{n/(2d)} \\ &= \left(2(1+\lambda)^d - 1\right)^{n/(2d)}. \end{aligned}$$

Reduction to the bipartite case. Let $\widetilde{G}$ be the bipartite double cover of G. Its vertices are $V(G) \times \{0,1\}$; every edge uv of G gives the two edges $(u,0)(v,1)$ and $(v,0)(u,1)$. We construct a size-preserving injection

$$\mathcal{I}(G) \times \mathcal{I}(G) \longrightarrow \mathcal{I}(\widetilde{G}).$$

For each $W \subseteq V(G)$ such that $G[W]$ is bipartite, fix a bipartition $W = W_0 \sqcup W_1$, depending only on W. Given independent sets A, B, their union $W = A \cup B$ induces a bipartite graph: every edge joins $A \setminus B$ to $B \setminus A$, and vertices of $A \cap B$ are isolated.

Swap the two memberships at every vertex of W_1:

$$A' = (A \cap W_0) \cup (B \cap W_1), \qquad B' = (B \cap W_0) \cup (A \cap W_1).$$

Each edge of $G[W]$ originally has one endpoint in $A \setminus B$ and the other in $B \setminus A$. Exactly one endpoint is swapped, so afterward both endpoints belong to the same one of A', B'. Thus no edge joins A' to B', and

$$(A' \times \{0\}) \cup (B' \times \{1\})$$

is independent in $\widetilde{G}$.

The union remains W, so the output determines the fixed bipartition used in the construction. Swapping again recovers A, B, proving injectivity. The swap also preserves $|A| + |B|$, and therefore preserves weights. Summing over all input pairs gives

$$\boxed{Z_G(\lambda)^2 \le Z_{\widetilde{G}}(\lambda).}$$

The double cover is d-regular on $2n$ vertices. Apply the bipartite bound and take square roots.

Finally, every independent set of $K_{d,d}$ lies entirely in one side. Each side contributes $(1+\lambda)^d$, with the empty set counted twice, so

$$Z_{K_{d,d}}(\lambda) = 2(1+\lambda)^d - 1.$$

Independence polynomials multiply over disjoint unions, proving the equality statement. □

7.5.4 Triangle-Free Independent Sets and Ramsey Numbers

For lower bounds, we sample an independent set itself, with probability proportional to its weight:

$$\Pr_\lambda(I) = \frac{\lambda^{|I|}}{Z_G(\lambda)}, \qquad a_G(\lambda) = \frac{1}{n}\mathbb{E}_\lambda|I|, \qquad \lambda > 0.$$

Triangle-freeness will let us describe the conditional distribution inside a neighborhood exactly. The same calculation relates the probability of selecting its center to the expected number of selected neighbors.

Available vertices. Call v *available* when $N(v) \cap I = \varnothing$. Condition on the choices at all vertices except v. If v is unavailable, it is forced to be absent. Otherwise both choices are allowed, with respective weights 1 and λ, so its conditional inclusion probability is $\lambda/(1+\lambda)$. Averaging gives

$$\boxed{\Pr_\lambda(v \in I) = \frac{\lambda}{1+\lambda}\Pr_\lambda(v \text{ is available}).} \tag{49}$$

For $x \ge 0$, let $W(x)$ be the unique nonnegative solution of $we^w = x$. Uniqueness follows because $w \mapsto we^w$ is strictly increasing on $[0,\infty)$.

Theorem 136 (Triangle-free occupancy bound)**.** *Let G be a finite simple triangle-free graph on $n \ge 1$ vertices, with maximum degree at most d, where $d \ge 1$. For every $\lambda > 0$,*

$$\boxed{a_G(\lambda) \ge \frac{\lambda}{1+\lambda}\frac{W(d\ln(1+\lambda))}{d\ln(1+\lambda)}.}$$

Proof. Put $r = \lambda/(1+\lambda)$ and $L = \ln(1+\lambda)$.

Fix a vertex v, and expose $J = I \cap (V \setminus N(v))$. This exposes v itself but leaves its neighbors unexposed. Since G is triangle-free, $N(v)$ is independent, and every neighbor of a vertex $u \in N(v)$ lies outside $N(v)$. Thus J determines which vertices in $N(v)$ are available:

$$A_v(J) = \{u \in N(v) : N(u) \cap J = \varnothing\}.$$

Write $U_v = |A_v(J)|$.

The possible completions are exactly $I = J \cup T$ with $T \subseteq A_v(J)$. Their weights are $\lambda^{|J|}\lambda^{|T|}$, so, conditionally on J,

$$\Pr_\lambda(I \cap N(v) = T \mid J) = \frac{\lambda^{|T|}}{(1+\lambda)^{U_v}} \qquad (T \subseteq A_v(J)).$$

In particular, the available neighbors are selected independently with probability r.

The center v is available exactly when $T = \varnothing$. The conditional probability of this event is $(1+\lambda)^{-U_v}$, while the conditional expected number of selected neighbors is rU_v. Averaging over J,

$$\Pr_\lambda(v \text{ is available}) = \mathbb{E}_\lambda e^{-LU_v}, \qquad \mathbb{E}_\lambda|I \cap N(v)| = r\,\mathbb{E}_\lambda U_v.$$

Now average also over a uniformly chosen vertex v, independently of I. Write $U = U_v$ and $a = a_G(\lambda)$. The first identity and (49) give

$$\frac{a}{r} = \mathbb{E}e^{-LU} \geq e^{-L\mathbb{E}U},$$

by Jensen's inequality. The second identity gives

$$\begin{aligned} r\,\mathbb{E}U &= \frac{1}{n}\sum_v \mathbb{E}_\lambda|I \cap N(v)| \\ &= \frac{1}{n}\sum_u d(u)\Pr_\lambda(u \in I) \leq da. \end{aligned}$$

Combining the two bounds,

$$\frac{a}{r} \geq e^{-dLa/r}.$$

Set $z = dLa/r$. The inequality becomes $ze^z \geq dL$, so $z \geq W(dL)$. Rearranging proves the theorem. □

Expected size and the independence polynomial. Differentiating the finite sums defining Z_G and $\mathbb{E}_\lambda|I|$ gives

$$\begin{aligned} \frac{d}{d\lambda}\ln Z_G(\lambda) &= \frac{\mathbb{E}_\lambda|I|}{\lambda} = \frac{n\,a_G(\lambda)}{\lambda}, \\ \frac{d}{d\lambda}\mathbb{E}_\lambda|I| &= \frac{\mathbb{E}_\lambda|I|^2 - (\mathbb{E}_\lambda|I|)^2}{\lambda} = \frac{\mathrm{Var}_\lambda(|I|)}{\lambda} \geq 0. \end{aligned} \tag{50}$$

Thus increasing λ increases the expected size, and integrating a lower bound on that expectation gives a lower bound on Z_G.

Corollary 137 (Asymptotic independence bound)**.** *As $d \to \infty$, every finite simple triangle-free graph on n vertices with maximum degree at most d satisfies*

$$\boxed{\alpha(G) \geq \mathbb{E}_{I \text{ uniform in } \mathcal{I}(G)}|I| \geq (1-o(1))\frac{n\ln d}{d}.}$$

Proof. For $x \geq e$, the identity $W(x) + \ln W(x) = \ln x$ gives

$$\ln x - \ln\ln x \leq W(x) \leq \ln x.$$

Indeed, $W(x) \geq 1$, so first $W(x) \leq \ln x$; substituting this upper bound into the logarithm gives the lower bound.

Choose $\lambda = 1/\ln d$. With $r = \lambda/(1+\lambda)$ and $L = \ln(1+\lambda)$,

$$\frac{r}{L} = 1 - O\left(\frac{1}{\ln d}\right), \qquad dL = (1+o(1))\frac{d}{\ln d}.$$

Hence $W(dL) \geq \ln d - 2\ln\ln d - O(1)$, and the occupancy bound yields

$$a_G(\lambda) \geq \frac{\ln d - 2\ln\ln d - O(1)}{d} = (1-o(1))\frac{\ln d}{d}.$$

For sufficiently large d, $\lambda \leq 1$. By (50), the expectation at $\lambda = 1$, which is the uniform distribution on independent sets, is at least the expectation at this λ. Finally, the maximum size is at least the expected size. □

Corollary 138 (Triangle Ramsey upper bound)**.** *As $t \to \infty$,*

$$\boxed{R(3,t) \leq (1+o(1))\frac{t^2}{\ln t}.}$$

Proof. Let G be a triangle-free graph with no independent set of size t. Every neighborhood is independent, so $\Delta(G) \leq t-1$. Applying Corollary 137 with degree bound $d = t$,

$$t > \alpha(G) \geq (1-o(1))\frac{|V(G)|\ln t}{t}.$$

Thus $|V(G)| \leq (1+o(1))t^2/\ln t$. Any graph with more vertices must contain a triangle or an independent set of size t, proving the claim. □

Corollary 139 (Independent-set count in triangle-free graphs)**.** *Under the hypotheses of Theorem 136, put $w = W(d\ln(1+\lambda))$. Then*

$$\boxed{Z_G(\lambda) \geq \exp\left(\frac{n}{2d}(w^2+2w)\right).}$$

In particular,

$$|\mathcal{I}(G)| \geq \exp\left(\left(\frac{1}{2} - o(1)\right)\frac{n\ln^2 d}{d}\right) \qquad (d \to \infty).$$

Proof. Use (50), $Z_G(0) = 1$, and the occupancy bound:

$$\begin{aligned}
\ln Z_G(\lambda) &= n\int_0^\lambda \frac{a_G(t)}{t}\,dt \\
&\geq \frac{n}{d}\int_0^\lambda \frac{W(d\ln(1+t))}{(1+t)\ln(1+t)}\,dt \\
&= \frac{n}{d}\int_0^{d\ln(1+\lambda)} \frac{W(x)}{x}\,dx,
\end{aligned}$$

where the last step substitutes $x = d\ln(1+t)$. The integrands at zero are interpreted by their continuous limits.

Now substitute $u = W(x)$, so $x = ue^u$ and $dx/x = (1+u)\,du/u$. Therefore

$$\ln Z_G(\lambda) \geq \frac{n}{d}\int_0^w (1+u)\,du = \frac{n}{2d}(w^2+2w).$$

Exponentiating proves the first bound. At $\lambda = 1$, $w = (1-o(1))\ln d$, giving the second. □

7.5.5 Moser–Tardos Resampling

Suppose an object is described by independent random choices, and we want none of a given collection of bad events to occur. Resampling the whole object may discard almost everything that was already correct. Instead, repair a violated event by resampling only the variables on which it depends. This can create other violations, but only nearby ones. The Moser–Tardos theorem shows that sufficiently rare, sparsely dependent bad events cannot sustain this chain of repairs for very long.

Let $X_1, \dots, X_N$ be mutually independent random variables and let $\mathcal{A}$ be a finite collection of bad events. Each $A \in \mathcal{A}$ is determined by a specified set of variable indices $\mathrm{vbl}(A)$. Write $p_A = \Pr[A]$ for its probability under the original product distribution, and define

$$\Gamma(A) = \{B \in \mathcal{A} \setminus \{A\} : \mathrm{vbl}(A) \cap \mathrm{vbl}(B) \neq \emptyset\}, \qquad \Gamma^+(A) = \Gamma(A) \cup \{A\}.$$

The open neighborhood excludes A; the closed neighborhood includes it.

Repair only what changed. Maintain the set Q of currently violated events. Resampling A can change the truth value only of events in $\Gamma^+(A)$, so only these need to be checked again. The dependency lists are assumed available.

Algorithm 14 Moser–Tardos resampling

Input: Independent-variable samplers, bad-event tests, and dependency lists
Output: An assignment in which no bad event occurs, upon termination

```
Sample X_1, ..., X_N independently
Q ← {A ∈ 𝒜 : A occurs}
while Q ≠ ∅ do
    Choose any A ∈ Q
    for each i ∈ vbl(A) do
        Replace X_i by a fresh independent sample from its distribution
    for each B ∈ Γ⁺(A) do
        if B occurs then
            Insert B into Q
        else
            Remove B from Q if present
return (X_1, ..., X_N)
```

Theorem 140 (Moser–Tardos)**.** *Suppose there are numbers $x_A \in (0,1)$ satisfying*

$$\boxed{p_A \le x_A \prod_{B \in \Gamma(A)} (1 - x_B) \qquad (A \in \mathcal{A}).}$$

Then Algorithm 14 terminates with probability one. If N_A is the number of times it resamples A, then

$$\boxed{\mathbb{E}N_A \le \frac{x_A}{1 - x_A}, \qquad \mathbb{E}\left[\sum_{A \in \mathcal{A}} N_A\right] \le \sum_{A \in \mathcal{A}} \frac{x_A}{1 - x_A}.}$$

Any rule for choosing a currently violated event is allowed, provided it does not inspect unused random samples. The numbers x_A certify the guarantee but are not needed by the algorithm.

The assignment at a resampling time is conditioned on previous repairs, so we cannot simply charge p_A for each occurrence of A in the log. Instead, encode a resampling by the earlier repairs relevant to it. This certificate will impose bad-event tests on disjoint random samples.

Witness tree. Let $A_1, A_2, \ldots$ be the resampling log, listing an event immediately before its variables are resampled. For step t, start a tree T_t with root labelled A_t. Scan $A_{t-1}, \ldots, A_1$ backward. If the current event is in the closed neighborhood of any label already in the tree, attach a new node beneath a *deepest* such node; otherwise skip it. Fix any rule for breaking depth ties.

Every child label belongs to its parent's closed neighborhood. More importantly, if two tree nodes use a common variable, the earlier resampling has greater depth: when that earlier node is inserted, the later node is already present, so the chosen parent is at least as deep. Consequently nodes using a common variable have distinct depths. The same argument for equal labels shows that siblings have distinct labels. Call a rooted labelled tree *proper* if every child label is in its parent's closed neighborhood and siblings have distinct labels. Thus every tree produced by the log is proper.

Lemma 141 (Witness-tree probability)**.** *For a fixed finite labelled tree T, the probability that $T = T_t$ for some resampling step t is at most*

$$\prod_{v \in V(T)} p_{\ell(v)},$$

where $\ell(v)$ denotes the event labelling node v.

Proof. Presample an infinite table $R(i,0), R(i,1), \ldots$ for each variable X_i, with all table entries independent and each $R(i,j)$ distributed as X_i. Initialize $X_i = R(i,0)$; each resampling of X_i advances its index by one. Generating entries only when needed has exactly the same law, so these tables describe the algorithm's randomness.

If two nodes of T use the same variable at the same depth, the tree cannot occur. Otherwise, for a node v and $i \in \mathrm{vbl}(\ell(v))$, put

$$k_T(v,i) = \#\{w \in V(T) : \mathrm{depth}(w) > \mathrm{depth}(v),\ i \in \mathrm{vbl}(\ell(w))\}.$$

Test the event $\ell(v)$ using the values $R(i, k_T(v,i))$ for its variables. For each fixed i, distinct depths give distinct indices, so different nodes use disjoint table entries. The tests are therefore independent, and all pass with probability $\prod_v p_{\ell(v)}$.

It remains to show that occurrence of T forces all these tests to pass. Suppose node v came from step s of the log. Every earlier resampling of a variable used by $\ell(v)$ is included in the tree: the backward scan already contains v when it reaches that resampling. Among nodes using X_i, chronological order is the reverse of depth order. Thus $k_T(v,i)$ is exactly the number of times X_i was resampled before step s. The table value used in the test is consequently the value present when the algorithm selected $\ell(v)$, and that event was violated. This holds for every node, proving the probability bound. □

The table is read in *decreasing depth*, not from the root downward. Nodes in different branches may share variables; the table index counts all deeper nodes using that variable, not merely descendants.

Sum all certificates. For an isolated event, proper witness trees are chains, representing repeated failures of the same test. With dependencies, each node may also have one child for each neighboring label. The factor $\prod_{B \in \Gamma(A)}(1 - x_B)$ will pay for these possible branches.

Lemma 142 (Total witness weight)**.** *Under the theorem's hypothesis, the total weight of all finite proper trees rooted at A satisfies*

$$\sum_{T \text{ proper, rooted at } A} \prod_{v \in V(T)} p_{\ell(v)} \le \frac{x_A}{1 - x_A}.$$

Proof. Let $W_h(A)$ be the same sum restricted to trees with at most h levels, so $W_0(A) = 0$. Children are not ordered, and at most one has any given label. For each $B \in \Gamma^+(A)$, either omit that child, contributing 1, or choose its subtree, contributing $W_h(B)$. Therefore

$$W_{h+1}(A) = p_A \prod_{B \in \Gamma^+(A)} (1 + W_h(B)).$$

Put $K_A = x_A/(1-x_A)$, so $1 + K_A = 1/(1-x_A)$. If $W_h(B) \le K_B$ for all B, the hypothesis gives

$$W_{h+1}(A) \le p_A \prod_{B \in \Gamma^+(A)} \frac{1}{1-x_B} \le \frac{x_A}{1-x_A} = K_A.$$

Induction proves this for every h. Every finite proper tree has finite height, so taking the increasing limit proves the claim. □

Proof of Theorem 140. Different resamplings of the same event produce different witness trees. Indeed, the tree of the jth resampling of A contains exactly j nodes labelled A: every earlier occurrence of A is included because it is in the root's closed neighborhood. Hence, on each execution,

$$N_A \le \sum_{T \text{ proper, rooted at } A} \mathbf{1}\{T \text{ occurs in the log}\}.$$

Take expectations and apply the two lemmas. Nonnegative sums allow passage from finite truncations to the entire log, even before termination has been established. We obtain $\mathbb{E}N_A \le x_A/(1-x_A)$ and the stated bound on total resamplings. A nonnegative integer-valued count with finite expectation is finite with probability one. Upon termination Q is empty, so no bad event occurs. □

The symmetric criterion.

Corollary 143 (Symmetric algorithmic local lemma)**.** *Suppose every event has probability at most p and overlaps at most $D \ge 1$ other events. If $ep(D+1) \le 1$, then the algorithm terminates with probability one and uses at most $|\mathcal{A}|/D$ expected resamplings.*

Proof. Choose $x_A = 1/(D+1)$ for every A. Since $(1+1/D)^D \le e$,

$$x_A \prod_{B \in \Gamma(A)} (1-x_B) \ge \frac{1}{D+1}\left(\frac{D}{D+1}\right)^D \ge \frac{1}{e(D+1)} \ge p.$$

The theorem applies, and $x_A/(1-x_A) = 1/D$. □

7.6 Polynomial Method

7.6.1 Triangular Independence

To bound a family of objects, represent its members by functions in a small vector space. If each function has an evaluation that detects it but annihilates every later function, the functions are linearly independent. The bound is then the dimension of the space.

Lemma 144 (Triangular independence)**.** *Let $v_1, \dots, v_m$ belong to a vector space V over a field $\mathbb{F}$. Suppose there are linear functionals $\lambda_1, \dots, \lambda_m \in V^*$ such that*

$$\lambda_i(v_i) \ne 0, \qquad \lambda_i(v_j) = 0 \quad \text{whenever } i < j.$$

Then $v_1, \dots, v_m$ are linearly independent. In particular, $m \le \dim V$ when V is finite-dimensional.

Proof. Suppose $\sum_j c_j v_j = 0$ is a nontrivial dependence, and choose the smallest i with $c_i \ne 0$. Applying λ_i kills all terms with $j > i$, while all coefficients with $j < i$ are zero. Thus $0 = c_i \lambda_i(v_i)$, a contradiction. □

Equivalently, the matrix $M_{ij} = \lambda_i(v_j)$ is lower triangular with nonzero diagonal. Reversing the order of the indices gives the same conclusion when the vanishing condition is $i > j$ instead. We thus do not need every off-diagonal entry to vanish.

7.6.2 Combinatorial Nullstellensatz

For

$$\alpha = (\alpha_1, \ldots, \alpha_n) \in \mathbb{N}^n,$$

write

$$x^\alpha = x_1^{\alpha_1} \cdots x_n^{\alpha_n}.$$

We say that x^α is *maximal in the support* of f if

$$[x^\alpha] f \neq 0$$

and there is no different monomial x^β in the support of f with

$$\beta_i \geq \alpha_i \qquad \text{for every } i.$$

We give an upgraded version of the Combinatorial Nullstellensatz is an exact formula for a coefficient.

Lemma 145 (Univariate Coefficient Formula)**.** *Let*

$$A \subseteq \mathbb{F}, \qquad |A| = d + 1,$$

and define

$$\phi_A(t) = \prod_{a \in A} (t - a).$$

Then for every polynomial g of degree at most d,

$$\boxed{[t^d] g(t) = \sum_{a \in A} \frac{g(a)}{\phi'_A(a)}.}$$

In particular,

$$\sum_{a \in A} \frac{a^j}{\phi'_A(a)} = \begin{cases} 0, & j < d, \\ 1, & j = d. \end{cases}$$

Proof. Lagrange interpolation gives

$$g(t) = \sum_{a \in A} g(a) \frac{\phi_A(t)}{(t - a)\phi'_A(a)}.$$

Each polynomial

$$\frac{\phi_A(t)}{t - a}$$

is monic of degree d. Taking the coefficient of t^d gives the result. □

Theorem 146 (Coefficient Formula)**.** *Let*

$$f \in \mathbb{F}[x_1, \ldots, x_n],$$

and fix

$$d = (d_1, \ldots, d_n).$$

For each i, choose

$$A_i \subseteq \mathbb{F}, \qquad |A_i| = d_i + 1,$$

and define

$$\phi_i(t) = \prod_{a \in A_i} (t - a).$$

Suppose either

1. $x^d = x_1^{d_1} \cdots x_n^{d_n}$ *is maximal in the support of* f*, or*

2. $$\deg f \le d_1 + \cdots + d_n.$$

Then

$$\boxed{[x^d]f = \sum_{a \in A_1 \times \cdots \times A_n} \frac{f(a)}{\prod_{i=1}^n \phi_i'(a_i)}.}$$

In the second case the coefficient on the left is allowed to be zero.

Proof. By linearity, evaluate the right-hand side on a monomial

$$x^\alpha.$$

The sum factors:

$$\sum_a \frac{a^\alpha}{\prod_i \phi_i'(a_i)} = \prod_{i=1}^n \left(\sum_{a_i \in A_i} \frac{a_i^{\alpha_i}}{\phi_i'(a_i)} \right).$$

For $\alpha = d$, every factor equals 1 by Lemma 145.

Now consider any other monomial x^α appearing in f. Under the first hypothesis, maximality of x^d implies that

$$\alpha_i < d_i$$

for some i.

Under the second hypothesis, if $\alpha_i \ge d_i$ for every i, then

$$|\alpha| \ge \sum_i d_i.$$

Since $|\alpha| \le \deg f \le \sum_i d_i$, equality would hold everywhere and $\alpha = d$. Thus again, for every $\alpha \ne d$, some coordinate satisfies

$$\alpha_i < d_i.$$

The corresponding factor is 0 by Lemma 145. Hence every monomial except x^d is annihilated, while x^d contributes exactly its coefficient. □

Theorem 147 (Combinatorial Nullstellensatz)**.** *Suppose*

$$x_1^{d_1} \cdots x_n^{d_n}$$

is a maximal monomial in the support of

$$f \in \mathbb{F}[x_1, \ldots, x_n].$$

If

$$|A_i| > d_i$$

for every i*, then there exists*

$$a \in A_1 \times \cdots \times A_n$$

such that

$$f(a) \ne 0.$$

Proof. Restrict each A_i to any $d_i + 1$ elements. By Theorem 146,

$$0 \ne [x^d]f = \sum_a \frac{f(a)}{\prod_i \phi_i'(a_i)}.$$

Therefore not every evaluation $f(a)$ can be zero. □

The usual formulation, which assumes

$$\deg f = d_1 + \cdots + d_n \qquad \text{and} \qquad [x^d]f \neq 0,$$

is an immediate special case: the total-degree condition automatically makes x^d maximal.

Alon–Füredi Combinatorial Nullstellensatz guarantees one nonzero grid point. Alon–Füredi gives the sharp minimum number of them.

For positive integers $a_1, \ldots, a_n$ define

$$\mu(a_1, \ldots, a_n; d) = \min_{\substack{0 \le d_i \le a_i - 1 \\ \sum_i d_i \le d}} \prod_{i=1}^{n} (a_i - d_i).$$

Theorem 148 (Alon–Füredi)**.** *Let*

$$A = A_1 \times \cdots \times A_n, \qquad |A_i| = a_i,$$

and suppose

$$f \in \mathbb{F}[x_1, \ldots, x_n]$$

is not identically zero on A. If

$$\deg f \le d,$$

then

$$\boxed{|\{x \in A : f(x) \neq 0\}| \ge \mu(a_1, \ldots, a_n; d).}$$

Proof. Reduce f modulo

$$\prod_{a \in A_i} (x_i - a)$$

for every variable x_i. This preserves all values on A, does not increase total degree, and allows us to assume

$$\deg_{x_i} f < a_i.$$

We induct on n.

For $n = 1$, a nonzero polynomial of degree at most d has at most d roots, so it is nonzero on at least

$$a_1 - \min(d, a_1 - 1) = \mu(a_1; d)$$

points.

Now suppose $n > 1$. Let

$$S = \{c \in A_n : f(x_1, \ldots, x_{n-1}, c) \equiv 0\}, \qquad s = |S|.$$

Since f is nonzero somewhere on A,

$$s < a_n.$$

Every $x_n - c$ for $c \in S$ divides f, so

$$f(x) = \left(\prod_{c \in S} (x_n - c) \right) g(x)$$

with

$$\deg g \le d - s.$$

For every

$$c \in A_n \setminus S,$$

the slice

$$g_c(x_1, \dots, x_{n-1}) = g(x_1, \dots, x_{n-1}, c)$$

is nonzero. By induction, it is nonzero on at least

$$\mu(a_1, \dots, a_{n-1}; d - s)$$

points.

Therefore

$$|\{f \neq 0\}| \geq (a_n - s)\mu(a_1, \dots, a_{n-1}; d - s).$$

If

$$d_1, \dots, d_{n-1}$$

achieve the latter minimum, set

$$d_n = s.$$

Then

$$0 \leq d_n \leq a_n - 1, \qquad \sum_i d_i \leq d,$$

so this choice is feasible in the definition of

$$\mu(a_1, \dots, a_n; d).$$

Thus

$$\mu(a_1, \dots, a_n; d) \leq (a_n - s)\mu(a_1, \dots, a_{n-1}; d - s),$$

which proves the theorem. □

The bound is sharp. For any feasible $d_1, \dots, d_n$, choose

$$S_i \subseteq A_i, \qquad |S_i| = d_i,$$

and take

$$f(x) = \prod_{i=1}^{n} \prod_{a \in S_i} (x_i - a).$$

Then

$$\deg f = \sum_i d_i,$$

and f is nonzero on exactly

$$\prod_i (a_i - d_i)$$

grid points.

Ryser's Permanent Formula The coefficient formula can be significantly more useful than the ordinary version, as is shown in the following.

Theorem 149 (Ryser's Formula)**.** *For*

$$A = (a_{ij}) \in \mathbb{F}^{n\times n},$$

$$\boxed{\operatorname{per}(A) = \sum_{S\subseteq[n]} (-1)^{n-|S|} \prod_{i=1}^{n} \left(\sum_{j\in S} a_{ij} \right).}$$

Consequently the permanent can be computed in

$$O(n2^n)$$

arithmetic operations.

Proof. Define

$$P(x_1, \ldots, x_n) = \prod_{i=1}^{n} \left(\sum_{j=1}^{n} a_{ij} x_j \right).$$

Choosing one variable from each factor shows

$$[x_1 x_2 \cdots x_n] P = \operatorname{per}(A).$$

Apply Theorem 146 with

$$A_j = \{0, 1\}, \qquad d_j = 1.$$

For

$$\phi(t) = t(t-1),$$
$$\phi'(0) = -1, \qquad \phi'(1) = 1.$$

Thus

$$\operatorname{per}(A) = \sum_{\varepsilon\in\{0,1\}^n} (-1)^{n-\sum_j \varepsilon_j} P(\varepsilon).$$

Writing

$$S = \{j : \varepsilon_j = 1\}$$

gives exactly

$$\operatorname{per}(A) = \sum_{S\subseteq[n]} (-1)^{n-|S|} \prod_i \sum_{j\in S} a_{ij}.$$

□

Chevalley–Warning The same coefficient identity can turn a degree bound into a congruence for the number of solutions of a polynomial system.

Theorem 150 (Chevalley–Warning)**.** *Let*

$$f_1, \ldots, f_m \in \mathbb{F}_q[x_1, \ldots, x_n]$$

satisfy

$$\sum_{j=1}^{m} \deg f_j < n.$$

Then the number N of common zeros of

$$f_1, \ldots, f_m$$

in $\mathbb{F}_q^n$ is divisible by the characteristic p of $\mathbb{F}_q$.

Proof. Define

$$P(x) = \prod_{j=1}^{m} \left(1 - f_j(x)^{q-1}\right).$$

For every

$$a \in \mathbb{F}_q^n,$$

Fermat's theorem gives

$$P(a) = \begin{cases} 1, & f_1(a) = \cdots = f_m(a) = 0, \\ 0, & \text{otherwise.} \end{cases}$$

Hence in $\mathbb{F}_q$,

$$\sum_{a \in \mathbb{F}_q^n} P(a) = N.$$

Also,

$$\deg P \le (q-1) \sum_j \deg f_j < n(q-1).$$

Therefore

$$[x_1^{q-1} \cdots x_n^{q-1}]P = 0.$$

Use the degree-bounded form of Theorem 146 with

$$A_i = \mathbb{F}_q, \qquad d_i = q - 1.$$

Since

$$\phi_i(t) = \prod_{a \in \mathbb{F}_q} (t - a) = t^q - t,$$

we have

$$\phi_i'(a) = -1$$

for every $a \in \mathbb{F}_q$. Thus

$$0 = [x_1^{q-1} \cdots x_n^{q-1}]P = (-1)^n \sum_{a \in \mathbb{F}_q^n} P(a) = (-1)^n N$$

inside $\mathbb{F}_q$.

Hence

$$N \equiv 0 \pmod{p}.$$

□

7.6.3 Multiplicity Schwartz–Zippel

Ordinary Schwartz–Zippel counts zeros. The multiplicity version counts a zero several times according to how many derivatives vanish there.

For a multi-index

$$\alpha \in \mathbb{N}^n,$$

define the Hasse derivative $D^\alpha f$ by

$$f(x + z) = \sum_{\alpha \in \mathbb{N}^n} D^\alpha f(x) z^\alpha.$$

Definition 151 (Multiplicity)**.** For a nonzero polynomial f and a point a, define

$$\operatorname{mult}(f,a) = \min\{|\alpha| : D^\alpha f(a) \neq 0\}.$$

Thus

$$\operatorname{mult}(f,a) = 0$$

exactly when $f(a) \neq 0$.

We use the elementary fact

$$\operatorname{mult}(D^\alpha f, a) \geq \operatorname{mult}(f,a) - |\alpha|.$$

This follows immediately from the Taylor expansion: taking a Hasse derivative of order $|\alpha|$ can lower the smallest nonzero total degree by at most $|\alpha|$.

Theorem 152 (Multiplicity Schwartz–Zippel)**.** *Let*

$$0 \neq f \in \mathbb{F}[x_1, \ldots, x_n]$$

have total degree at most d, and let

$$S \subseteq \mathbb{F}$$

be finite. Then

$$\boxed{\sum_{a \in S^n} \operatorname{mult}(f,a) \leq d|S|^{n-1}.}$$

Consequently, for every $r \geq 1$,

$$\boxed{|\{a \in S^n : \operatorname{mult}(f,a) \geq r\}| \leq \frac{d}{r}|S|^{n-1}.}$$

Proof. We induct on n.

For $n = 1$, this is the usual fact that the sum of the multiplicities of the roots of a nonzero degree-d polynomial is at most d.

Now write

$$f(x_1, \ldots, x_n) = \sum_{j=0}^{t} f_j(x_1, \ldots, x_{n-1}) x_n^j,$$

where

$$f_t \neq 0.$$

Then

$$\deg f_t \leq d - t.$$

Fix

$$a' = (a_1, \ldots, a_{n-1}) \in S^{n-1}$$

and put

$$r(a') = \operatorname{mult}(f_t, a').$$

Choose a multi-index α of size $r(a')$ such that

$$D^\alpha f_t(a') \neq 0.$$

Then the univariate polynomial

$$g(T) = D^{(\alpha,0)} f(a', T)$$

is nonzero and has degree exactly t.

For every $b \in S$,

$$\operatorname{mult}(f, (a', b)) \le r(a') + \operatorname{mult}(g, b).$$

Summing over b and using the univariate multiplicity bound,

$$\sum_{b \in S} \operatorname{mult}(f, (a', b)) \le |S| r(a') + t.$$

Now sum over a':

$$\begin{aligned}\sum_{a \in S^n} \operatorname{mult}(f, a) &\le |S| \sum_{a' \in S^{n-1}} \operatorname{mult}(f_t, a') + t|S|^{n-1} \\ &\le |S|(d-t)|S|^{n-2} + t|S|^{n-1} \\ &= d|S|^{n-1}.\end{aligned}$$

The second claim follows by Markov's inequality. □

7.6.4 Zero Patterns of Polynomial Maps

Given polynomials $f_1, \dots, f_m$, their zero pattern at a point records which of them vanish. We equivalently record the nonzero set

$$S(x) = \{i \in [m] : f_i(x) \neq 0\}.$$

Although the point x may range over an infinite field, the number of patterns is controlled by the dimension of a polynomial space.

Theorem 153 (Elementary zero-pattern bound)**.** *Let $f_1, \dots, f_m \in K[x_1, \dots, x_n]$, and suppose $\deg f_i \le d_i$, where $d_i \ge 0$. Writing $D = \sum_i d_i$, the number of realized zero patterns is at most*

$$\boxed{\binom{n+D}{n}}.$$

If every degree is at most d, then the number of realized patterns having at most s nonzero entries is at most

$$\boxed{\binom{n+ds}{n}}.$$

Both conclusions remain valid when the points are restricted to an arbitrary subset of K^n.

Proof. For every realized nonzero set S, choose a representative point x_S and define

$$P_S(x) = \prod_{i \in S} f_i(x).$$

Then

$$P_S(x_T) \neq 0 \quad \Longleftrightarrow \quad S \subseteq T.$$

Order the realized patterns by nondecreasing cardinality. The resulting evaluation matrix is triangular with nonzero diagonal, so the P_S's are linearly independent.

Every P_S has degree at most D, and the space of polynomials of total degree at most D in n variables has dimension $\binom{n+D}{n}$.

For the second claim, retain only patterns with $|S| \le s$. Their product polynomials have degree at most ds, and the same argument applies. □

Corollary 154 (Support patterns of low-rank matrices)**.** *Over any field, the number of support patterns of $a \times b$ matrices of rank at most $r \geq 1$ is at most*

$$\boxed{\binom{2ab + r(a+b)}{r(a+b)}}.$$

For $n \times n$ matrices and $1 \leq r \leq n$, this is at most

$$\left(\frac{2en}{r}\right)^{2nr}.$$

Proof. Every rank-at-most-r matrix can be written as UV, where $U \in K^{a\times r}$ and $V \in K^{r\times b}$. Its ab entries are degree-two polynomials in the $r(a+b)$ entries of U, V. Apply the zero-pattern bound.

For the square case, use $\binom{N}{k} \leq (eN/k)^k$ and $r \leq n$. □

7.6.5 Regular Subgraphs from Chevalley–Warning

Over $\mathbb{F}_p$, the expression x^{p-1} records whether x is nonzero. This lets a polynomial system select a subset of objects and impose linear constraints on its incidence vector.

Lemma 155 (A nonempty zero-sum subsequence)**.** *Let p be prime and $a_1, \ldots, a_m \in \mathbb{F}_p^r$. If $m > r(p-1)$, there is a nonempty $S \subseteq [m]$ with*

$$\sum_{i\in S} a_i = 0.$$

Proof. For each coordinate j, impose

$$\sum_{i=1}^{m} a_{i,j} x_i^{p-1} = 0.$$

The sum of the polynomial degrees is at most $r(p-1) < m$. Chevalley–Warning says that the number of common zeros is divisible by p. The all-zero vector is one solution, so another solution exists.

For such a solution, let $S = \{i : x_i \neq 0\}$. Fermat's theorem gives $x_i^{p-1} = 1$ on S and zero elsewhere, so the equations say precisely that $\sum_{i\in S} a_i = 0$. □

The threshold is sharp: $p-1$ copies of each standard basis vector have no nonempty zero-sum subsequence. One may replace r by the dimension of the span of the a_i's.

Theorem 156 (A regular subgraph)**.** *Let p be prime, and let G be a finite loopless graph, allowing parallel edges. If*

$$|E(G)| > (p-1)|V(G)|, \qquad \Delta(G) \leq 2p-1,$$

then G contains a nonempty p-regular subgraph.

Proof. Associate to each edge its endpoint-incidence vector in $\mathbb{F}_p^{V(G)}$. The zero-sum lemma gives a nonempty edge set whose degree at every vertex is divisible by p.

Each selected degree lies between zero and $2p-1$, so it is either zero or p. Delete the isolated vertices. □

Corollary 157 (A cubic subgraph in an almost 4-regular graph)**.** *Adding one edge between distinct vertices of a finite 4-regular graph produces a graph containing a nonempty 3-regular subgraph.*

Proof. On n vertices the new graph has $2n+1$ edges and maximum degree at most five. Apply the theorem with $p = 3$. □

7.6.6 Alon–Tarsi Orientations and List Coloring

A proper coloring is a nonzero evaluation of the graph polynomial $\prod_{uv\in E}(x_u - x_v)$. An orientation identifies a particular coefficient of this polynomial: its exponent vector is the outdegree sequence, and its coefficient counts balanced edge subsets with signs.

Lemma 158 (The orientation coefficient)**.** *Let D orient a finite loopless graph G, and put*

$$P_D(x) = \prod_{u\to v\in E(D)} (x_u - x_v).$$

Call $F \subseteq E(D)$ balanced *if $d_F^+(v) = d_F^-(v)$ at every vertex. Then*

$$\boxed{\left[\prod_v x_v^{d_D^+(v)}\right] P_D = \sum_{\substack{F\subseteq E(D)\\ F \text{ balanced}}} (-1)^{|F|}.}$$

Proof. Choosing x_u from every factor $x_u - x_v$ gives the exponent vector d_D^+. Let F contain the edges on which we instead choose $-x_v$. The exponent of x_v becomes

$$d_D^+(v) - d_F^+(v) + d_F^-(v).$$

This equals $d_D^+(v)$ at every vertex exactly when F is balanced, and the term has sign $(-1)^{|F|}$. □

Theorem 159 (Alon–Tarsi list coloring)**.** *Suppose an orientation D of G has different numbers of even-sized and odd-sized balanced edge subsets. For any lists $L(v)$ satisfying*

$$|L(v)| \geq d_D^+(v) + 1,$$

there is a proper coloring c with $c(v) \in L(v)$ for every v.

Proof. Choose $d_D^+(v) + 1$ elements from each list and identify their union with distinct real numbers. The preceding coefficient is a nonzero integer, and $\deg P_D = |E| = \sum_v d_D^+(v)$. Combinatorial Nullstellensatz supplies a list assignment where $P_D \neq 0$. Every factor is then nonzero, so adjacent vertices receive different colors. □

Corollary 160 (Orientations with no odd directed cycle)**.** *If D has no odd directed cycle, lists of size $d_D^+(v) + 1$ suffice.*

Proof. Every nonempty balanced edge set decomposes into edge-disjoint directed cycles. To obtain the decomposition, follow edges until a directed cycle appears, remove that cycle, and repeat; balance is preserved.

All these cycles have even length, so every balanced edge set has even size. The signed sum in Lemma 158 is therefore positive, since it includes the empty set. □

Obtaining the orientation from a density bound.

Lemma 161 (Prescribed outdegree upper bounds)**.** *Let $G = (V, E)$ be a finite loopless graph, and let b_v be nonnegative integers. An orientation with $d^+(v) \leq b_v$ exists if and only if*

$$\boxed{|E(S)| \leq \sum_{v\in S} b_v \qquad (S \subseteq V).}$$

Proof. Necessity holds because each edge internal to S has one tail in S.

For sufficiency, make a bipartite graph with one left vertex for each edge of G and b_v right slots for each $v \in V$. Connect an edge to every slot of either endpoint.

For a set F of left vertices, let S be its set of endpoints. Its neighborhood has size $\sum_{v\in S} b_v$, which is at least $|E(S)| \geq |F|$. Hall's theorem supplies a matching covering all edges. Orient each edge away from the vertex whose slot receives it. □

Corollary 162 (List coloring bipartite graphs from density)**.** *Let G be bipartite, and suppose*

$$|E(S)| \le \sum_{v \in S} b_v \qquad (S \subseteq V).$$

Then arbitrary lists of size at least $b_v + 1$ at vertex v admit a proper coloring.

Proof. Choose the bounded-outdegree orientation from the lemma. Every directed cycle is even because the underlying graph is bipartite. Apply Corollary 160. □

Corollary 163 (Bipartite planar graphs are 3-choosable)**.** *Every finite simple bipartite planar graph admits a proper coloring from arbitrary lists of size three.*

Proof. Every connected simple bipartite plane graph with at least three vertices has facial boundary walks of length at least four. Thus $2e \ge 4f$, and Euler's formula gives $e \le 2v - 4$. Components with one or two vertices also satisfy $e \le 2v$.

The same applies to every induced subgraph, so $|E(S)| \le 2|S|$ for all $S \subseteq V$. Use $b_v = 2$ in the preceding corollary. □

7.6.7 Restricted Intersections

Intersection restrictions produce low-degree polynomials that separate the members of a set family. The resulting bound depends on the dimension of the polynomial functions on the domain where they are evaluated. On one uniform layer, that dimension is smaller than on the whole Boolean cube.

Low-degree functions on a uniform layer. For $S \subseteq [n]$, write $x_S = \prod_{i \in S} x_i$. On Boolean inputs, replacing every positive power of x_i by x_i preserves values and does not increase degree. Thus polynomial functions of degree at most s on the Boolean cube have dimension at most

$$\sum_{j=0}^{\min\{s,n\}} \binom{n}{j}.$$

Lemma 164 (Dimension on a uniform layer)**.** *Let $0 \le s \le k \le n$, and let K be any field. The restrictions of degree-at-most-s polynomials to incidence vectors of k-subsets of $[n]$ span a space of dimension at most*

$$\boxed{\binom{n}{s}}.$$

Proof. First work over $\mathbb{Q}$. For $|S| = j \le s$, on the k-th layer we have

$$\sum_{\substack{T \supseteq S \\ |T|=s}} x_T = \binom{k-j}{s-j} x_S.$$

Indeed, if the input set contains S, the left side counts its s-subsets containing S; otherwise both sides vanish. The binomial coefficient is positive, so every monomial of degree at most s is in the span of the degree-s monomials. There are $\binom{n}{s}$ of these.

Now consider the evaluation matrix whose rows are k-subsets and whose columns are squarefree monomials of degree at most s. Its entries are integers zero and one. Every minor larger than $\binom{n}{s}$ vanishes over $\mathbb{Q}$, hence vanishes as an integer and therefore over every field. Its rank over K is consequently at most $\binom{n}{s}$. □

Lemma 165 (Intersection-polynomial bound)**.** *Let $A_1, \dots, A_m$ be distinct subsets of $[n]$. Suppose there are polynomials $q_i(t) \in K[t]$, each of degree at most s, such that*

$$q_i(|A_i|) \neq 0, \qquad q_i(|A_i \cap A_j|) = 0 \quad (j < i).$$

Integers in these evaluations are interpreted in K. Then

$$\boxed{m \leq \sum_{j=0}^{\min\{s,n\}} \binom{n}{j}.}$$

If every A_i has size k and $s \leq k$, then

$$\boxed{m \leq \binom{n}{s}.}$$

Proof. Associate to A_i the polynomial function

$$P_i(x) = q_i\left(\sum_{v \in A_i} x_v\right).$$

At the incidence vector of A_j, its value is $q_i(|A_i \cap A_j|)$. The evaluation matrix is triangular with nonzero diagonal, so the functions P_i are linearly independent.

Their Boolean reductions have degree at most s. Use the Boolean-cube dimension bound, or Lemma 164 in the uniform case. □

Theorem 166 (Restricted intersections over the integers)**.** *Let L be a set of s nonnegative integers, and let $\mathcal{F} \subseteq 2^{[n]}$ satisfy*

$$|A \cap B| \in L \qquad (A, B \in \mathcal{F},\ A \neq B).$$

Then

$$\boxed{|\mathcal{F}| \leq \sum_{j=0}^{\min\{s,n\}} \binom{n}{j}.}$$

If $\mathcal{F}$ is k-uniform and $s \leq k$, then

$$\boxed{|\mathcal{F}| \leq \binom{n}{s}.}$$

Proof. Order the sets by nondecreasing cardinality and work over $\mathbb{Q}$. For each A_i, put

$$q_i(t) = \prod_{\substack{\ell \in L \\ \ell < |A_i|}} (t - \ell).$$

Its diagonal value $q_i(|A_i|)$ is nonzero.

If $j < i$, then $|A_j| \leq |A_i|$, and distinctness gives $|A_i \cap A_j| < |A_i|$. This intersection size belongs to L, so it is one of the roots of q_i. Apply Lemma 165. □

Discarding roots at least $|A_i|$ is necessary in the nonuniform case: a set size may itself belong to L, but only the smaller roots are needed to annihilate earlier sets.

The nonuniform bound is sharp for $L = \{0, \dots, s-1\}$ and the family of all subsets of size at most s, when $s \leq n$. For a uniform family, $s = 1$ gives the bound n when all distinct pairs have the same intersection size.

Theorem 167 (Frankl–Wilson modular intersection bound)**.** *Let p be prime and $L \subseteq \mathbb{F}_p$, with $|L| = s$. Suppose $\mathcal{F} \subseteq 2^{[n]}$ satisfies*

$$|A| \bmod p \notin L \quad (A \in \mathcal{F}),$$

and

$$|A \cap B| \bmod p \in L \quad (A, B \in \mathcal{F},\ A \neq B).$$

Then

$$\boxed{|\mathcal{F}| \leq \sum_{j=0}^{\min\{s,n\}} \binom{n}{j}.}$$

If $\mathcal{F}$ is k-uniform and $s \leq k$, then

$$\boxed{|\mathcal{F}| \leq \binom{n}{s}.}$$

Proof. Work over $\mathbb{F}_p$ and take

$$q_i(t) = \prod_{\ell \in L} (t - \ell)$$

for every i. The size condition makes every diagonal evaluation nonzero, and the intersection condition makes every off-diagonal evaluation zero. Apply Lemma 165. □

7.6.8 Finite-Field Kakeya Sets

For a prime power q and an integer $n \geq 1$, a set $K \subseteq \mathbb{F}_q^n$ is *Kakeya* if, for every $v \in \mathbb{F}_q^n \setminus \{0\}$, some affine line $a + \mathbb{F}_q v$ lies in K.

The highest-degree part of a polynomial turns information about a line into information about its direction. If P has degree d and degree-d homogeneous part H, then the coefficient of t^d in $P(a + tv)$ is $H(v)$, independently of the offset a. We apply this observation to Hasse derivatives as well. The target is dictated by multiplicity Schwartz–Zippel: a nonzero degree-d polynomial has average multiplicity at most d/q on $\mathbb{F}_q^n$, so vanishing to order greater than d/q everywhere is impossible.

Lemma 168 (Degree required to vanish on a Kakeya set)**.** *Let $K \subseteq \mathbb{F}_q^n$ be Kakeya, and let P be a nonzero polynomial with* $\operatorname{mult}(P, a) \geq m$ *at every $a \in K$, where $m \geq 1$ is an integer. Then*

$$\deg P \geq \frac{q^2}{2q - 1}\, m.$$

Proof. Write $d = \deg P$ and let H be its degree-d homogeneous part. Since P vanishes on the nonempty set K, we have $d > 0$. Suppose that $d < q^2 m/(2q - 1)$. We will show that every Hasse derivative $D^\alpha H$ with $|\alpha| \leq d/q$ vanishes on $\mathbb{F}_q^n$.

Fix such an α, put $s = |\alpha|$, and choose a contained line $a + \mathbb{F}_q v$ in a nonzero direction v. The univariate polynomial $Q(t) = D^\alpha P(a + tv)$ has degree at most $d - s$ and vanishes to order at least $m - s$ at each $t \in \mathbb{F}_q$. Indeed, differentiation lowers multiplicity by at most s, and restriction to a line cannot lower it further. But

$$q(m - s) - (d - s) = qm - d - (q - 1)s \geq qm - \left(2 - \frac{1}{q}\right) d > 0.$$

Thus the root multiplicities exceed the degree bound, forcing $Q = 0$. Its coefficient of t^{d-s} is $D^\alpha H(v)$, so this coefficient vanishes.

This holds for every nonzero v. It also holds at $v = 0$, since H is homogeneous and $|\alpha| \leq d/q < d$. Consequently $\operatorname{mult}(H, v) > d/q$ at every point of $\mathbb{F}_q^n$, contradicting

$$\sum_{v \in \mathbb{F}_q^n} \operatorname{mult}(H, v) \leq d q^{n-1}.$$

□

Vanishing to order m at a point specifies all Taylor coefficients of total degree below m. Counting these conditions now converts the degree bound into a bound on $|K|$.

Theorem 169 (Finite-field Kakeya bound)**.** *Every Kakeya set $K \subseteq \mathbb{F}_q^n$ satisfies*

$$|K| \geq \left(\frac{q}{2 - 1/q}\right)^n.$$

Proof. Put $c = q^2/(2q-1)$. For a positive integer m, let $d = \lceil cm \rceil - 1$, so $d < cm$. By Lemma 168, the only polynomial of degree at most d vanishing to order m throughout K is zero.

The space of polynomials of degree at most d has dimension $\binom{d+n}{n}$. At one point, vanishing to order m imposes $\binom{m+n-1}{n}$ homogeneous linear conditions, one for each $D^\alpha P(a)$ with $|\alpha| < m$. Since all these conditions together have trivial kernel,

$$\binom{d+n}{n} \leq |K| \binom{m+n-1}{n}.$$

Divide by the binomial coefficient on the right and let $m \to \infty$. Since $d/m \to c$, the ratio tends to c^n, giving the claim. □

7.6.9 Slice Rank and Low-Degree Tensor Methods

The polynomial method becomes particularly effective when a forbidden configuration can be encoded by a tensor that is diagonal on the set of interest. The reason is a simple relationship:

diagonal support forces large slice rank,	low degree forces small slice rank.

Slice rank Let $X_1, \dots, X_k$ be finite sets and $T : X_1 \times \cdots \times X_k \to \mathbb{F}$.

A *slice* is a function of the form

$$f(x_i) g(x_1, \dots, x_{i-1}, x_{i+1}, \dots, x_k)$$

for some distinguished coordinate i. The *slice rank* $\operatorname{srank}(T)$ is the minimum number of slices whose sum is T.

For $k = 2$, this is ordinary matrix rank. For larger k, one factor may depend arbitrarily on all but one coordinate, which makes slice rank much more flexible than ordinary tensor rank.

We need one elementary support lemma.

Lemma 170 (Large support in a subspace)**.** *If $V \leq \mathbb{F}^X$ has dimension d, then some $f \in V$ is nonzero on at least d elements of X.*

Proof. Choose $D \subseteq X$, $|D| = d$, such that restriction $V \to \mathbb{F}^D$ has rank d. It is therefore onto. Choose $f \in V$ whose restriction to D is identically one. □

The analogue of "a nonsingular diagonal matrix has full rank" is the central fact behind the method.

Theorem 171 (Diagonal tensor lemma)**.** *Let A be finite, let $k \geq 2$, and suppose $T : A^k \to \mathbb{F}$ satisfies*

$$T(a_1, \dots, a_k) \neq 0 \quad \Longleftrightarrow \quad a_1 = \cdots = a_k.$$

Then

$$\boxed{\operatorname{srank}(T) = |A|.}$$

Proof. The upper bound follows by slicing in the first coordinate:

$$T(x_1, \dots, x_k) = \sum_{a \in A} 1_{x_1 = a} T(a, x_2, \dots, x_k).$$

For the lower bound, induct on k. The case $k = 2$ is ordinary matrix rank.

Suppose

$$T = \sum_{i=1}^{k} \sum_{j=1}^{r_i} f_{i,j}(x_i) g_{i,j}(x_1, \dots, \widehat{x_i}, \dots, x_k),$$

and put $r = \sum_i r_i$.

Let W be the span of $f_{1,1}, \dots, f_{1,r_1}$ in $\mathbb{F}^A$. Its orthogonal complement has dimension at least $|A| - r_1$, so by Lemma 170 there is $h \in W^{\perp}$ with

$$|\operatorname{supp} h| \geq |A| - r_1.$$

Contract the first coordinate against h:

$$T_h(x_2, \dots, x_k) = \sum_{x_1 \in A} h(x_1) T(x_1, x_2, \dots, x_k).$$

All slices distinguished by the first coordinate vanish. Every other slice remains a slice, so

$$\operatorname{srank}(T_h) \leq r - r_1.$$

The tensor T_h is diagonal, with nonzero diagonal entries precisely on $\operatorname{supp} h$. By induction,

$$r - r_1 \geq \operatorname{srank}(T_h) = |\operatorname{supp} h| \geq |A| - r_1.$$

Thus $r \geq |A|$. □

Low degree gives few slices The second ingredient is trivial. A monomial of small total degree must have small degree in at least one variable block.

For a prime p, write

$$M_p(n, d) = \left| \{\alpha \in \{0, \dots, p-1\}^n : |\alpha| \leq d\} \right|.$$

This is the number of reduced monomials in n variables of total degree at most d.

Lemma 172 (Polynomial slice-rank bound)**.** *Let*

$$P(x^{(1)}, \dots, x^{(k)})$$

be a polynomial function on $(\mathbb{F}_p^n)^k$, represented with degree at most $p-1$ in every scalar variable and total degree at most D. Then

$$\boxed{\operatorname{srank}(P) \leq k M_p\left(n, \left\lfloor \frac{D}{k} \right\rfloor\right).}$$

Proof. Expand P into monomials. For every monomial, at least one of its k blocks has degree at most D/k. Assign the monomial to one such block.

For a fixed block i, group together all assigned monomials having the same monomial in $x^{(i)}$. Such a group is one slice: the chosen monomial is the one-variable-block factor, and everything else is absorbed into the other factor.

There are at most $M_p(n, \lfloor D/k \rfloor)$ choices for the low-degree monomial in each block. □

We will also use the identical Boolean-cube specialization.

Corollary 173 (Multilinear slice-rank bound)**.** *Suppose P is multilinear in each scalar variable, with k blocks of n variables and total degree at most D. Then*

$$\operatorname{srank}(P) \le k \sum_{j=0}^{\lfloor D/k \rfloor} \binom{n}{j}.$$

Combining low slice rank with the diagonal tensor lemma gives the general method.

Corollary 174 (Diagonal-polynomial principle)**.** *Let $\phi_i : [m] \to \mathbb{F}_p^n$ for $i = 1, \dots, k$, and let P be as in Lemma 172. Suppose*

$$P(\phi_1(j_1), \dots, \phi_k(j_k)) \neq 0 \quad \Longleftrightarrow \quad j_1 = \cdots = j_k.$$

Then

$$\boxed{m \le k M_p\left(n, \left\lfloor \frac{D}{k} \right\rfloor\right).}$$

Proof. Restrict P to the k indexed lists. The resulting tensor is diagonal, so its slice rank is m by Theorem 171. Restriction cannot increase the displayed slice decomposition, and Lemma 172 supplies the upper bound. □

Counting the low-degree monomials For $0 < \rho < 1$,

$$\begin{aligned} M_p(n,d)\rho^d &\le \sum_{\alpha \in \{0,\dots,p-1\}^n} \rho^{|\alpha|} \\ &= (1 + \rho + \cdots + \rho^{p-1})^n. \end{aligned}$$

Thus

$$\boxed{M_p(n,d) \le \rho^{-d}(1 + \rho + \cdots + \rho^{p-1})^n.} \tag{51}$$

Define

$$\boxed{\theta_p = \min_{0<\rho<1} (1 + \rho + \cdots + \rho^{p-1})\rho^{-(p-1)/3}.}$$

It is convenient to write

$$\theta_p = p^{1-c_p}, \qquad c_p = 1 - \log_p \theta_p.$$

Then $0 < c_p < 1$.

Tri-colored sum-free sets A *tri-colored sum-free set* is a collection

$$\{(x_i, y_i, z_i) : 1 \le i \le m\}$$

such that

$$x_i + y_j + z_k = 0 \quad \Longleftrightarrow \quad i = j = k.$$

This is exactly a diagonal zero-set problem.

Theorem 175 (Tri-colored sum-free bound)**.** *If*

$$\{(x_i, y_i, z_i) : 1 \le i \le m\} \subseteq (\mathbb{F}_p^n)^3$$

is tri-colored sum-free, then

$$\boxed{m \le \theta_p^n = p^{(1-c_p)n}.}$$

Proof. Use the zero-sum indicator

$$P(x,y,z)=\prod_{\ell=1}^{n}\left(1-(x_\ell+y_\ell+z_\ell)^{p-1}\right).$$

Fermat's theorem gives $P(x,y,z)=1$ exactly when $x+y+z=0$. The total degree is $(p-1)n$, so Corollary 174 gives

$$m\le 3M_p\left(n,\left\lfloor\frac{(p-1)n}{3}\right\rfloor\right)\le 3\theta_p^n. \tag{1}$$

The factor 3 disappears by tensor powering. The r-fold Cartesian power of the collection is tri-colored sum-free of size m^r in $\mathbb{F}_p^{nr}$. Hence

$$m^r\le 3\theta_p^{nr}.$$

Take r-th roots and let $r\to\infty$. □

The exponential rate is sharp.

Theorem 176 (Sharpness of the tri-colored bound; external)**.** *For every fixed prime p, there exist tri-colored sum-free collections in $\mathbb{F}_p^n$ of size*

$$\boxed{p^{(1-c_p)n-o(n)}=\theta_p^{n-o(n)}.}$$

General translation-invariant linear systems The same diagonal-polynomial principle handles many equations at once.

Fix r homogeneous equations in k vector variables,

$$\sum_{i=1}^{k}a_{j,i}x_i=0,\qquad j=1,\dots,r,$$

where $a_{j,i}\in\mathbb{F}_p$. Assume

$$\sum_{i=1}^{k}a_{j,i}=0\qquad(j=1,\dots,r),$$

so every diagonal tuple is a solution.

Define

$$\Gamma_{p,r,k}=\min_{0<\rho<1}(1+\rho+\cdots+\rho^{p-1})\rho^{-r(p-1)/k}.$$

Theorem 177 (Diagonal solutions to linear systems)**.** *Suppose $A\subseteq\mathbb{F}_p^n$ has the property that every solution in A^k to the preceding system is diagonal. Then*

$$\boxed{|A|\le\Gamma_{p,r,k}^n.}$$

If $k>2r$, then $\Gamma_{p,r,k}<p$.

Proof. The system indicator is

$$P(x_1,\dots,x_k)=\prod_{j=1}^{r}\prod_{\ell=1}^{n}\left[1-\left(\sum_{i=1}^{k}a_{j,i}x_{i,\ell}\right)^{p-1}\right].$$

Reduce modulo u^p-u in every scalar variable. This preserves the function on $\mathbb{F}_p$, keeps every scalar degree at most $p-1$, and does not increase total degree. Thus the total degree is at most $r(p-1)n$.

On A^k, the polynomial is nonzero exactly on the diagonal. Hence

$$|A| \le kM_p\left(n, \left\lfloor \frac{r(p-1)n}{k} \right\rfloor\right) \le k\Gamma^n_{p,r,k}.$$

Apply this to the s-fold Cartesian power A^s, take s-th roots, and let $s \to \infty$.

For the last claim, put $\rho = e^{-t}$. At $t = 0$, the expression defining $\Gamma_{p,r,k}$ equals p. The derivative of its logarithm at $t = 0$ is

$$-\frac{p-1}{2} + \frac{r(p-1)}{k},$$

which is negative when $k > 2r$. Hence a sufficiently small positive t makes the value strictly less than p. □

The cap-set theorem is now immediate.

Corollary 178 (Cap-set bound)**.** *If $A \subseteq \mathbb{F}_3^n$ contains no nontrivial solution to $x + y + z = 0$, equivalently no nontrivial three-term arithmetic progression, then*

$$\boxed{|A| \le \theta_3^n,}$$

where

$$\theta_3 = \min_{0<\rho<1}(1 + \rho + \rho^2)\rho^{-2/3} \approx 2.7551046.$$

The minimizer is

$$\rho = \frac{\sqrt{33} - 1}{8}.$$

Proof. Apply Theorem 177 with $p = 3$, $r = 1$, $k = 3$, and equation $x + y + z = 0$.

For the minimizer, differentiating the logarithm gives

$$\frac{1 + 2\rho}{1 + \rho + \rho^2} = \frac{2}{3\rho},$$

or $4\rho^2 + \rho - 2 = 0$. □

For odd p, the same argument applied to $x - 2y + z = 0$ gives the corresponding exponential bound for sets with no nontrivial three-term progression in $\mathbb{F}_p^n$.

Three-petal sunflowers The Boolean-cube specialization gives another application.

Theorem 179 (Slice-rank sunflower bound)**.** *Let $\mathcal{F} \subseteq 2^{[n]}$ contain no three distinct sets forming a sunflower. Then*

$$\boxed{|\mathcal{F}| \le 3(n+1)\sum_{j=0}^{\lfloor n/3 \rfloor}\binom{n}{j} \le 3(n+1)\left(\frac{3}{2^{2/3}}\right)^n.}$$

Proof. Partition $\mathcal{F}$ by cardinality. Fix one uniform layer $\mathcal{A} \subseteq \binom{[n]}{s}$, represented by its indicator vectors in $\{0,1\}^n$.

Over a field of characteristic zero, define

$$P(x, y, z) = \prod_{i=1}^{n}(2 - x_i - y_i - z_i).$$

For Boolean inputs, a factor vanishes exactly when its coordinate has membership pattern 110, 101, or 011, i.e. exactly two of the three sets contain that element.

Suppose first that $x, y, z \in \mathcal{A}$ are three distinct sets. If $P(x, y, z) \neq 0$, then no coordinate lies in exactly two sets. Consequently every pairwise intersection consists exactly of the coordinates of type 111, so the three pairwise intersections are equal. Thus x, y, z form a sunflower, a contradiction.

If exactly two of x, y, z coincide, say $x = y \neq z$, then $|x| = |z|$ implies $x \setminus z \neq \varnothing$. At such a coordinate the pattern is 110, so again $P = 0$.

On the diagonal, every factor equals 2 or -1, so $P \neq 0$. Thus P restricts to a diagonal tensor on $\mathcal{A}^3$.

The polynomial is multilinear and has total degree n. By Corollary 173 and Theorem 171,

$$|\mathcal{A}| \leq 3 \sum_{j \leq n/3} \binom{n}{j}.$$

There are $n + 1$ cardinality layers.

Finally, for $\rho = 1/2$,

$$\begin{aligned} \sum_{j \leq n/3} \binom{n}{j} &\leq \rho^{-n/3} \sum_{j=0}^{n} \binom{n}{j} \rho^j \\ &= 2^{n/3} \left(\frac{3}{2}\right)^n = \left(\frac{3}{2^{2/3}}\right)^n. \end{aligned}$$

□

Arithmetic triangle removal We now use the tri-colored theorem in a different way. Instead of assuming that the only triangles are already isolated, we start with an arbitrary collection of interacting triangles and randomly pass to a small subspace. Bounded local triangle degree then makes a positive fraction of the sampled triangles isolated, at which point Theorem 175 applies.

Let $N = p^n$. For $X, Y, Z \subseteq \mathbb{F}_p^n$, a *triangle* is a triple $(x, y, z) \in X \times Y \times Z$ satisfying $x + y + z = 0$. We regard the three sets as colored copies, so vertices in different color classes are distinct even if they represent the same vector.

Set

$$\boxed{C_p = 1 + \frac{1}{c_p}.}$$

Lemma 180 (Bounded triangle degree)**.** *Suppose $X, Y, Z \subseteq \mathbb{F}_p^n \setminus \{0\}$ are pairwise disjoint and satisfy:*

1. *every two distinct vectors in $X \cup Y \cup Z$ are linearly independent;*
2. *every two-dimensional subspace contains at most one triangle from $X \times Y \times Z$.*

Suppose every point lies in at most ρN triangles, where $0 < \rho \leq 1/(5p^3)$, and suppose there are δN^2 triangles in total. Then

$$\boxed{\delta \leq 125 p^2 \rho^{1+c_p}.}$$

Proof. Choose

$$d = \left\lfloor \log_p \frac{1}{5\rho} \right\rfloor.$$

Then $d \geq 3$ and

$$\frac{1}{5p} < \rho p^d \leq \frac{1}{5}. \tag{2}$$

Choose a uniformly random d-dimensional subspace $U \leq \mathbb{F}_p^n$. Call a sampled triangle *good* if it is the only sampled triangle containing any of its three vertices.

Fix a triangle (x, y, z), and let $W = \operatorname{span}\{x, y\}$. The assumptions give $\dim W = 2$, and $z = -x - y \in W$.

Condition on $W \subseteq U$. Then U/W is a uniformly random $(d-2)$-subspace of the $(n-2)$-dimensional quotient $\mathbb{F}_p^n/W$.

Consider another triangle containing x, say (x, y', z'). By the second genericity condition, $y' \notin W$. Thus $y' + W$ is a fixed nonzero vector of the quotient. By symmetry,

$$\Pr[y' \in U \mid W \subseteq U] = \frac{p^{d-2} - 1}{p^{n-2} - 1}.$$

Moreover

$$\frac{p^{d-2} - 1}{p^{n-2} - 1} \leq \frac{p^{d-2}}{p^{n-2}} = p^{d-n};$$

after cross-multiplication this inequality is simply $p^{d-2} \leq p^{n-2}$.

There are at most ρN triangles containing x, so conditioned on $W \subseteq U$, the probability that x lies in another sampled triangle is at most

$$\rho N p^{d-n} = \rho p^d \leq \frac{1}{5}.$$

The same estimate holds for y and z. A union bound therefore shows that, conditioned on $W \subseteq U$, the triangle is good with probability at least $2/5$.

It remains to sample W. A random d-subspace contains a fixed two-dimensional subspace with probability

$$\begin{aligned} q &:= \Pr[W \subseteq U] \\ &= \frac{(p^d - 1)(p^{d-1} - 1)}{(p^n - 1)(p^{n-1} - 1)}. \end{aligned}$$

Factoring out the leading powers,

$$q = p^{2(d-n)} \frac{(1 - p^{-d})(1 - p^{-(d-1)})}{(1 - p^{-n})(1 - p^{-(n-1)})}.$$

Since $d \geq 3$ and $p \geq 2$, the fraction is at least $1/2$. Hence

$$q \geq \frac{1}{2} p^{2(d-n)}.$$

Therefore a fixed triangle is good with probability at least

$$\frac{2}{5} q \geq \frac{1}{5} p^{2(d-n)}.$$

From the strict inequality in (2), $p^d > 1/(5p\rho)$, so

$$\Pr[\text{fixed triangle is good}] > \frac{1}{125 p^2 \rho^2 N^2}.$$

If T is the number of good triangles, summing over the δN^2 original triangles gives

$$\mathbb{E}T \geq \frac{\delta}{125 p^2 \rho^2}. \tag{3}$$

The good triangles form a tri-colored sum-free collection: a mixed zero-sum triple would be a second sampled triangle containing each of its chosen vertices. Thus

$$T \leq \theta_p^d = p^{(1-c_p)d}.$$

Together with (3),

$$\frac{\delta}{125 p^2 \rho^2} \leq p^{(1-c_p)d}.$$

Since $p^d \le 1/(5\rho) \le 1/\rho$,

$$p^{(1-c_p)d} \le \rho^{-(1-c_p)}.$$

Rearranging gives

$$\delta \le 125p^2\rho^{1+c_p}.$$

□

The genericity assumptions can be enforced without altering the triangle pattern.

Lemma 181 (Generic padding)**.** *Replace colored points $x_i, y_i, z_i \in \mathbb{F}_p^n$ by*

$$x_i' = (x_i, 1, 0), \qquad y_i' = (y_i, -1, 1), \qquad z_i' = (z_i, 0, -1)$$

in $\mathbb{F}_p^{n+2}$.

Then triangles are preserved exactly, the three color classes become disjoint, every two distinct padded points are linearly independent, and every two-dimensional subspace contains at most one triangle.

Proof. The final two coordinates sum to zero, so

$$x_i' + y_j' + z_k' = 0 \quad \iff \quad x_i + y_j + z_k = 0.$$

They also distinguish the three colors.

If two padded points are scalar multiples, their final two coordinates force the scalar to equal one and the points to coincide.

Finally fix a triangle (x_i', y_j', z_k'), and let W be its two-dimensional span. If another Y-point $y_{j'}'$ belonged to W, then

$$y_{j'}' = \alpha x_i' + \beta y_j'.$$

The last two coordinates force $\alpha = 0$ and $\beta = 1$, hence $y_{j'}' = y_j'$. The same argument applies to the X- and Z-points. Thus this is the unique triangle contained in W. □

We next remove the bounded-degree hypothesis. Only the exponent matters at this stage.

The preceding lemma says that if every point lies in at most ρN triangles, then the total triangle density is at most $O_p(\rho^{1+c_p})$. By itself, this is only an upper bound. The key is to make ρ proportional to the triangle density itself.

A family of ηN vertex-disjoint triangles makes the configuration robust under deletions: deleting one point destroys at most one of these distinguished triangles. Thus fewer than ηN point deletions cannot destroy every triangle.

We exploit this by deleting points that belong to disproportionately many triangles. Such a deletion removes many triangles while sacrificing at most one distinguished triangle. When the current triangle density is β, use a degree threshold $g(\beta)\beta N/\eta$, where g is a slowly growing function. During one scale at which the density stays between β and 2β, only $O(\beta N^2)$ triangles are available to be destroyed, so at most $O(\eta N/g(\beta))$ points can be deleted. Taking $g(\beta)$ to be a sufficiently large constant times $\log^2(e/\beta)$ makes these deletion costs summable over all scales.

Consequently, the process stops before half the distinguished triangles are destroyed. The remaining density β is positive, but every point now has degree at most ρN, where $\rho = g(\beta)\beta/\eta$. In the small-density regime, the preceding lemma gives

$$\beta \lesssim_p \left(\frac{g(\beta)\beta}{\eta}\right)^{1+c_p} \quad \Longrightarrow \quad \beta \gtrsim_p \left(\frac{\eta}{g(\beta)}\right)^{C_p}, \qquad C_p = 1 + \frac{1}{c_p}.$$

The upper bound contains β^{1+c_p}, which decreases faster than β. If the remaining triangle count were too small, the upper bound would be smaller than that count itself. Pruning rules out excessive concentration at individual points, while disjointness rules out the empty configuration; together they force many triangles. The logarithmic overhead produces the intermediate exponent $C_p + o(1)$, and Cartesian powering later removes that loss.

Lemma 182 (Disjoint triangles force many triangles, asymptotically)**.** *Suppose $X, Y, Z \subseteq \mathbb{F}_p^n$ contain ηN pairwise vertex-disjoint triangles. Then, for fixed p and $\eta \to 0$,*

$$\boxed{\#\{\text{triangles in } X \times Y \times Z\} \geq \eta^{C_p + o(1)} N^2.}$$

Proof. By Lemma 181, after adjoining two coordinates we may assume the genericity hypotheses of Lemma 180. This changes N, the density of the distinguished triangles, and the total triangle density only by fixed powers of p, which are absorbed into the $o(1)$ exponent. We therefore work directly in the generic situation.

Let the current triangle density be β, so there are βN^2 triangles. Choose

$$g(\beta) = K \log^2(1/\beta),$$

with K a sufficiently large absolute constant. For sufficiently small β, both $g(\beta)$ and the monotonicity properties used below behave in the evident direction.

Repeatedly delete any point lying in at least

$$g(\beta)\frac{\beta}{\eta}N$$

current triangles, recomputing β after each deletion.

We first bound the number of deleted points. During a dyadic scale in which the triangle density decreases from at most 2β to below β, every deletion destroys at least $g(\beta)\beta N/\eta$ triangles, while fewer than $2\beta N^2$ triangles are available. Thus this scale causes at most

$$\frac{2\eta N}{g(\beta)}$$

deletions. Summing over dyadic scales gives fewer than $\eta N/2$ deletions when K is sufficiently large, because

$$\sum_{j \geq 0} \frac{1}{\log^2(2^j/\beta_0)} < \infty.$$

Since the distinguished triangles are vertex-disjoint, fewer than $\eta N/2$ deletions cannot destroy all of them. Thus the process terminates with positive triangle density β, and every remaining point belongs to at most ρN triangles, where

$$\rho = g(\beta)\frac{\beta}{\eta}.$$

If $\rho > 1/(5p^3)$, then

$$\beta > \frac{\eta}{5p^3 g(\beta)}.$$

Taking logarithms shows $\log(1/\beta) \leq (1 + o(1)) \log(1/\eta)$, hence

$$\beta \geq \eta^{1+o(1)} \geq \eta^{C_p + o(1)},$$

since $C_p > 1$.

Otherwise Lemma 180 applies:

$$\beta \leq 125p^2 \left(g(\beta)\frac{\beta}{\eta}\right)^{1+c_p}.$$

Rearranging,

$$\beta^{c_p} \geq \frac{\eta^{1+c_p}}{125p^2 g(\beta)^{1+c_p}}. \tag{4}$$

Put

$$u = \log\frac{1}{\beta}, \qquad v = \log\frac{1}{\eta}.$$

Since $g(\beta) = Ku^2$, taking logarithms of (4) gives

$$c_p u \le (1 + c_p)v + O(\log u).$$

This first implies $u = O(v)$. Hence $O(\log u) = o(v)$, and therefore

$$u \le \left(\frac{1 + c_p}{c_p} + o(1) \right) v = (C_p + o(1))v.$$

Equivalently,

$$\beta \ge \eta^{C_p + o(1)}.$$

The original configuration had at least as many triangles. □

A standard product argument removes the $o(1)$ completely.

Theorem 183 (Disjoint-triangle supersaturation)**.** *If $X, Y, Z \subseteq \mathbb{F}_p^n$ contain ηN pairwise vertex-disjoint triangles, then*

$$\boxed{\#\{\text{triangles in } X \times Y \times Z\} \ge \eta^{C_p} N^2.}$$

Proof. Suppose the triangle density is $\delta < \eta^{C_p}$. Then for some $\alpha > 0$,

$$\delta = \eta^{(1+\alpha)C_p}.$$

Take the r-fold Cartesian power. The new space has size N^r, contains $\eta^r N^r$ distinguished disjoint triangles, and has total triangle density δ^r. By Lemma 182,

$$\delta^r \ge (\eta^r)^{C_p + o(1)}$$

as $r \to \infty$, because $\eta^r \to 0$.

But

$$\delta^r = (\eta^r)^{(1+\alpha)C_p},$$

contradicting the preceding inequality for sufficiently large r. □

Theorem 184 (Arithmetic triangle removal in $\mathbb{F}_p^n$)**.** *Let $0 < \varepsilon < 1$ and*

$$\boxed{\delta = \left(\frac{\varepsilon}{3}\right)^{C_p}.}$$

If $X, Y, Z \subseteq \mathbb{F}_p^n$ contain fewer than δN^2 triangles, then fewer than εN colored points can be deleted so that no triangle remains.

Proof. Greedily choose a maximal family of vertex-disjoint triangles.

If it contains fewer than $\varepsilon N/3$ triangles, delete the three vertices of every chosen triangle. This removes fewer than εN colored points, and maximality ensures that no triangle remains.

Therefore, if εN deletions are necessary, there are at least $\varepsilon N/3$ disjoint triangles. By Theorem 183, there are at least

$$\left(\frac{\varepsilon}{3}\right)^{C_p} N^2$$

triangles in total. This is the contrapositive. □

The exponent C_p is essentially optimal.

Theorem 185 (Sharpness of arithmetic removal)**.** *For every fixed prime p, there are examples with $\varepsilon \to 0$ for which at least εN deletions are necessary, while the triangle density satisfies*

$$\boxed{\delta = \varepsilon^{C_p - o(1)}.}$$

Proof. Take a tri-colored sum-free collection

$$\{(x_i, y_i, z_i) : 1 \le i \le m\} \subseteq (\mathbb{F}_p^n)^3$$

with

$$m = p^{(1-c_p)n-o(n)}$$

from Theorem 176.

For $\ell \ge 1$, replace each point by a fiber:

$$X' = X \times \mathbb{F}_p^\ell,$$
$$Y' = Y \times \mathbb{F}_p^\ell,$$
$$Z' = Z \times \mathbb{F}_p^\ell.$$

Tri-colored sum-freeness forces every triangle to use the same index i. For fixed i, there are $p^{2\ell}$ triangles, because two fiber coordinates determine the third. Hence there are $mp^{2\ell}$ triangles in total.

For a fixed i, at least p^ℓ points must be deleted from its three fibers to remove all triangles. Indeed, if fewer are deleted, the remaining fiber sets $U, V, W \subseteq \mathbb{F}_p^\ell$ satisfy

$$|U| + |V| + |W| > 2p^\ell.$$

Choose $w \in W$. Then $|U| + |V| > p^\ell$, so the sets U and $-w - V$ intersect, giving $u + v + w = 0$.

Thus at least mp^ℓ deletions are necessary. Relative to the ambient size $p^{n+\ell}$,

$$\varepsilon = \frac{m}{p^n}, \qquad \delta = \frac{m}{p^{2n}}.$$

Since $m = p^{(1-c_p)n-o(n)}$,

$$\varepsilon = p^{-c_p n - o(n)}, \qquad \delta = p^{-(1+c_p)n-o(n)} = \varepsilon^{C_p - o(1)}.$$

□

Compressing a sumset The preceding applications used a polynomial whose restriction became diagonal. There is another useful consequence of exactly the same low-degree decomposition: matrices of the form

$$P(s + t)$$

have low ordinary rank. This yields a strong compression theorem for sumsets.

For a prime power q, define

$$M_q(n, d) = |\{\alpha \in \{0, \dots, q-1\}^n : |\alpha| \le d\}|.$$

We need one matrix-space lemma.

Lemma 186 (Pivot positions and matrix rank)**.** *Let $\mathcal{V}$ be a linear space of matrices over a field. Order the matrix entries row by row, and choose an echelon basis whose first nonzero entries, or* pivots*, are all distinct.*

If every matrix in $\mathcal{V}$ has rank at most r, then all pivot positions can be covered by at most r rows and columns.

Proof. Form the bipartite graph whose left vertices are matrix rows, whose right vertices are columns, and whose edges are the pivot positions. By König's theorem, it is enough to show that a matching of t pivot positions forces some matrix in $\mathcal{V}$ to have rank at least t.

Let the matched pivots be

$$p_i = (r_i, c_i), \qquad r_1 < \cdots < r_t,$$

and let B_i be the corresponding basis matrix. Scale B_i so that $B_i(p_i) = 1$.

Let $R = \{r_1, \dots, r_t\}$, $C = \{c_1, \dots, c_t\}$, and define

$$Q(\lambda_1, \dots, \lambda_t) = \det\left(\sum_{i=1}^{t} \lambda_i B_i \Big|_{R \times C} \right).$$

We claim

$$[\lambda_1 \cdots \lambda_t] Q = \pm 1. \tag{5}$$

Consider a determinant contribution to this coefficient. For every i, let q_i be the cell from which the factor λ_i is selected. The cells $q_1, \dots, q_t$ form a transversal of the selected rows and columns.

Since p_i is the first nonzero entry of B_i and $B_i(q_i) \neq 0$, we have $q_i \geq p_i$ in row-major order. Hence the row of q_i is at least r_i. But the rows of the q_i's are exactly $\{r_1, \dots, r_t\}$, so equality of their sums forces

$$\mathrm{row}(q_i) = r_i \qquad \text{for every } i.$$

Now q_i and p_i lie in the same row, so $q_i \geq p_i$ implies

$$\mathrm{col}(q_i) \geq c_i.$$

Again the columns of the q_i's are exactly $\{c_1, \dots, c_t\}$, and equality of their sums forces $\mathrm{col}(q_i) = c_i$ for every i.

Thus the only contribution to $\lambda_1 \cdots \lambda_t$ uses the matched pivots themselves. Its coefficient is $\pm \prod_i B_i(p_i) = \pm 1$, proving (5).

The determinant polynomial has total degree at most t. Therefore inclusion–exclusion on the Boolean cube gives

$$[\lambda_1 \cdots \lambda_t] Q = \sum_{S \subseteq [t]} (-1)^{t-|S|} Q(1_S).$$

Indeed every monomial missing some variable cancels, while a monomial containing every variable must equal $\lambda_1 \cdots \lambda_t$ because its total degree is at most t.

The left side is nonzero, so $Q(1_S) \neq 0$ for some S. Hence $\sum_{i \in S} B_i$ has a nonsingular $t \times t$ minor and therefore rank at least t. □

Define

$$\Lambda_q(n) = \min_{0 \leq d \leq (q-1)n} \left(2M_q\left(n, \left\lfloor \frac{d}{2} \right\rfloor\right) + q^n - M_q(n, d) \right).$$

Theorem 187 (Sumset compression in $\mathbb{F}_q^n$)**.** *For arbitrary $S, T \subseteq \mathbb{F}_q^n$, there are $S_0 \subseteq S$ and $T_0 \subseteq T$ such that*

$$\boxed{|S_0| + |T_0| \leq \Lambda_q(n),}$$

and

$$\boxed{S + T = (S_0 + T) \cup (S + T_0).}$$

If

$$D = \left\lfloor \frac{(q-1)n}{3} \right\rfloor,$$

then

$$\boxed{\Lambda_q(n) \leq 3M_q(n, D).}$$

Consequently $\Lambda_q(n) \leq 3\vartheta_q^n$ for

$$\vartheta_q = \min_{0 < \rho < 1} (1 + \rho + \cdots + \rho^{q-1}) \rho^{-(q-1)/3} < q.$$

Proof. Fix $d \le (q-1)n$, and let $\mathcal{P}_d$ be the space of reduced polynomials on $\mathbb{F}_q^n$ of total degree at most d. Its dimension is $M_q(n,d)$.

Let $V \le \mathcal{P}_d$ consist of polynomials vanishing outside $S+T$. Vanishing at each point outside $S+T$ gives at most one linear condition, so

$$\dim V \ge M_q(n,d) - q^n + |S+T|. \tag{6}$$

For $P \in V$, form the matrix

$$A_P(s,t) = P(s+t), \qquad s \in S,\ t \in T.$$

The map $P \mapsto A_P$ is injective: if $A_P = 0$, then P vanishes on $S+T$, while by definition it vanishes outside $S+T$, so it vanishes on all of $\mathbb{F}_q^n$. A reduced polynomial that vanishes everywhere is zero.

Expand $P(s+t)$. Every term is a monomial in s times a monomial in t, and their two degrees sum to at most d. Hence one side has degree at most $d/2$. Assign each term to such a side and group by its low-degree monomial. Exactly as in the slice-rank argument, this writes A_P as a sum of at most

$$2M_q\left(n, \left\lfloor \frac{d}{2} \right\rfloor\right)$$

rank-one matrices.

Thus every matrix in the image of V has rank at most this number. Choose an echelon basis of that matrix space. By Lemma 186, all of its pivots can be covered by rows $S_1 \subseteq S$ and columns $T_1 \subseteq T$ with

$$|S_1| + |T_1| \le 2M_q\left(n, \left\lfloor \frac{d}{2} \right\rfloor\right). \tag{7}$$

For each $u \in S+T$, let $\phi(u)$ be the lexicographically first pair (s,t) with $s+t=u$. Every pivot is some $\phi(u)$: if a matrix has a nonzero entry at one representation of u, it has the same nonzero entry at every representation of u, so its first such entry is $\phi(u)$.

Therefore distinct pivots correspond to distinct sums. Every pivot covered by a row in S_1 or a column in T_1 corresponds to a sum in

$$(S_1 + T) \cup (S + T_1).$$

Hence this union contains at least $\dim V$ distinct elements of $S+T$.

By (6), at most $q^n - M_q(n,d)$ sums remain uncovered. For each uncovered sum, choose one representation $s+t$ and add its s-coordinate to S_1. This yields S_0, T_0 satisfying

$$S + T = (S_0 + T) \cup (S + T_0)$$

and

$$|S_0| + |T_0| \le 2M_q\left(n, \left\lfloor \frac{d}{2} \right\rfloor\right) + q^n - M_q(n,d).$$

Minimizing over d proves the first bound.

For the simpler bound, put $L = (q-1)n = 3D + r$ with $r \in \{0,1,2\}$. Choose

$$d = \begin{cases} 2D, & r = 0,1, \\ 2D+1, & r = 2. \end{cases}$$

Then $\lfloor d/2 \rfloor = D$.

The involution

$$\alpha_i \longmapsto q - 1 - \alpha_i$$

sends a reduced monomial of degree j to one of degree $L - j$. For the chosen d, every monomial of degree $> d$ is therefore paired with a monomial of degree at most D. Hence

$$q^n - M_q(n,d) \le M_q(n,D),$$

and

$$\Lambda_q(n) \le 3M_q(n, D).$$

The final exponential estimate follows from (51), with p replaced by q. □

Taking $S = T$ gives a useful one-sided form: some $S_0 \subseteq S$ with $|S_0| \le \Lambda_q(n)$ satisfies

$$S_0 + S = S + S.$$

For a three-term-progression-free set in odd characteristic, this gives another proof of an exponential cap-set bound: if $s \in S \setminus S_0$, then $2s \notin S_0 + S$, since an equality $2s = s_0 + s'$ would be a nontrivial three-term progression unless $s_0 = s' = s$. Thus necessarily $S_0 = S$.

7.7 Extremal Set Theory

7.7.1 Bollobás Two Families Theorem

Theorem 188 (Bollobás Two Families Theorem)**.** *Let*

$$(A_1, B_1), \dots, (A_m, B_m)$$

be pairs of finite sets such that

$$A_i \cap B_i = \emptyset$$

for every i, while

$$A_i \cap B_j \neq \emptyset$$

whenever

$$i \neq j.$$

Then

$$\boxed{\sum_{i=1}^{m} \binom{|A_i| + |B_i|}{|A_i|}^{-1} \le 1.}$$

Proof. Choose a uniformly random ordering of all elements appearing in the sets.

For each i, let E_i be the event that every element of A_i appears before every element of B_i.

Among the

$$|A_i| + |B_i|$$

elements of $A_i \cup B_i$, every relative order is equally likely. The event E_i occurs exactly when the first $|A_i|$ positions are occupied by A_i, so

$$\Pr[E_i] = \binom{|A_i| + |B_i|}{|A_i|}^{-1}.$$

We claim that the events E_i are pairwise disjoint. Indeed, suppose both E_i and E_j occurred for

$$i \neq j.$$

Choose

$$x \in A_i \cap B_j, \qquad y \in A_j \cap B_i.$$

Since E_i occurs,

$$x \text{ appears before } y.$$

Since E_j occurs,

$$y \text{ appears before } x,$$

a contradiction.

Therefore

$$\sum_i \Pr[E_i] \le 1,$$

which is exactly the desired inequality. □

Corollary 189. *If*

$$|A_i| = a, \qquad |B_i| = b$$

for every i, then

$$\boxed{m \le \binom{a+b}{a}.}$$

7.7.2 Exterior Algebra and the Skew Two Families Theorem

The uniform bound has an algebraic explanation: represent each first member by a vector in a space of dimension $\binom{a+b}{a}$, and use the second members to certify linear independence. Exterior products turn intersections into zeros. The proof needs intersections in only one direction of the index order, giving the *skew* version of the theorem.

Exterior products as intersection tests. If V has basis $e_1, \dots, e_d$, its ath exterior power $\bigwedge^a V$ has basis $e_I = e_{i_1} \wedge \cdots \wedge e_{i_a}$ indexed by $I = \{i_1 < \cdots < i_a\} \subseteq [d]$. Thus

$$\dim \bigwedge^a V = \binom{d}{a}.$$

The product is bilinear: $e_I \wedge e_J = 0$ when $I \cap J \neq \emptyset$, and otherwise it is $e_{I \cup J}$ multiplied by the sign of the permutation sorting the concatenated indices. In particular, the product is alternating on vectors, including in characteristic 2. Multilinear expansion gives

$$v_1 \wedge \cdots \wedge v_a = \sum_{I \in \binom{[d]}{a}} \det([v_1 \ \cdots \ v_a]_I) e_I.$$

Here the subscript selects the rows in I. The product is nonzero exactly when the vectors are linearly independent, because its coordinates are precisely their maximal minors.

If $U, W \subseteq V$ have dimensions a, b, choose bases and write $\alpha = u_1 \wedge \cdots \wedge u_a$ and $\beta = w_1 \wedge \cdots \wedge w_b$. Their concatenated bases are independent exactly when $U \cap W = \{0\}$, so

$$\boxed{\alpha \wedge \beta \neq 0 \quad \Longleftrightarrow \quad U \cap W = \{0\}.}$$

Theorem 190 (Exterior two families bound)**.** *Let $\dim V = a + b$, and suppose $\alpha_1, \dots, \alpha_m \in \bigwedge^a V$ and $\beta_1, \dots, \beta_m \in \bigwedge^b V$ satisfy*

$$\alpha_i \wedge \beta_i \neq 0, \qquad \alpha_i \wedge \beta_j = 0 \quad (i < j).$$

Then the α_i are linearly independent, and hence

$$\boxed{m \le \binom{a+b}{a}.}$$

The exterior elements need not themselves be products of vectors.

Proof. In a nontrivial dependence $\sum_i c_i \alpha_i = 0$, choose the largest j with $c_j \neq 0$ and wedge with β_j on the right. Every term with $i < j$ vanishes by the hypothesis, while the later coefficients are zero. This leaves $c_j \alpha_j \wedge \beta_j = 0$, a contradiction. The dimension of $\bigwedge^a V$ gives the bound. □

Since $\bigwedge^{a+b} V$ is one-dimensional, taking the coefficient of $e_1 \wedge \cdots \wedge e_{a+b}$ in $\alpha \wedge \beta_j$ defines a scalar linear functional of α. Thus this is exactly the triangular-independence principle, with the index order reversed.

Theorem 191 (Skew subspace two families)**.** *Let $(U_1, W_1), \ldots, (U_m, W_m)$ be pairs of subspaces of a finite-dimensional vector space over any field, with $\dim U_i = a$ and $\dim W_i = b$. Suppose*

$$U_i \cap W_i = \{0\}, \qquad U_i \cap W_j \neq \{0\} \quad (i < j).$$

Then $m \leq \binom{a+b}{a}$.

Proof. Put $d = a+b$. If the ambient dimension is d, wedge the bases of the subspaces and apply Theorem 190. We reduce a larger ambient space to this case without destroying any required intersection or disjointness.

Choose coordinates in the original ambient space $\mathbb{F}^D$. Let T be a $d \times D$ matrix of independent indeterminates, and work over the rational-function field $K = \mathbb{F}(T)$. For each i, concatenate bases of U_i, W_i into a $D \times d$ matrix C_i. Its columns are independent, so there exists a linear map sending them to the standard basis of $\mathbb{F}^d$. Consequently $\det(TC_i)$ is a nonzero polynomial: it takes the value 1 at that map. Over K, all these determinants are nonzero.

Thus T is injective on each diagonal sum $U_i + W_i$ after extending scalars to K. The image subspaces have dimensions a, b and trivial diagonal intersection. A nonzero vector in $U_i \cap W_j$ remains nonzero under T, since it belongs to $U_i \subseteq U_i + W_i$. Hence the required cross-intersections also persist. The image configuration lies in K^d, where the first part applies. The case $d = 0$ is immediate: the cross-intersection condition permits at most one pair. □

The rational-function extension is only a device for choosing one projection that works for every pair.

Corollary 192 (Skew set-pair bound)**.** *Let $(A_1, B_1), \ldots, (A_m, B_m)$ be pairs of finite sets with $|A_i| = a$, $|B_i| = b$, $A_i \cap B_i = \emptyset$, and $A_i \cap B_j \neq \emptyset$ whenever $i < j$. Then*

$$\boxed{m \leq \binom{a+b}{a}.}$$

Proof. Give each ground element u a coordinate basis vector e_u. The coordinate subspaces $U_i = \operatorname{span}\{e_u : u \in A_i\}$ and $W_i = \operatorname{span}\{e_u : u \in B_i\}$ have the required intersection pattern. Apply Theorem 191. □

For sets with $a+b > 0$, there is also an explicit way to start in dimension $a+b$: assign distinct real numbers t_u to the ground elements and use $v_u = (1, t_u, \ldots, t_u^{a+b-1})$. Every $a+b$ distinct vectors are independent by the Vandermonde determinant. Wedge the vectors from A_i and B_i; diagonal products are nonzero, and a shared ground element makes every required cross-product zero.

The bound is sharp: take all pairs $(A, [a+b] \setminus A)$ with $|A| = a$. Distinct a-subsets intersect one another's complements. This exterior argument proves the uniform bound under a weaker, one-sided intersection hypothesis.

7.7.3 Sunflower Lemma

A family $S_1, \ldots, S_r$ is an *r-sunflower* if there is a set C, called the *core*, such that

$$S_i \cap S_j = C \qquad (i \neq j).$$

Equivalently, after deleting the core, the petals $S_1 \setminus C, \ldots, S_r \setminus C$ are pairwise disjoint.

Let $\mathrm{Sun}(k, r)$ be the least N such that every family of N distinct sets of size at most k contains an r-sunflower. The classical Erdős–Rado lemma gives

$$\mathrm{Sun}(k, r) \leq (r-1)^k k! + 1.$$

The modern bound replaces the factorial by only a logarithmic factor in the base:

$$\boxed{\mathrm{Sun}(k, r) \leq (Cr \log(2k))^k}$$

for a universal constant C.

Why a probabilistic strengthening appears. The eventual passage from a probabilistic statement to an ordinary sunflower is very simple. Suppose for the moment that the family is uniform and has common core C. Assign every element outside C, independently and uniformly, to one of $2r$ bins. Each bin is marginally a density-$1/(2r)$ random set.

If such a random set contains an entire petal with probability at least $1/2$, then the expected number of bins containing a complete petal is at least r. Hence some partition has r good bins. Choosing one petal from each gives r pairwise disjoint petals and therefore an r-sunflower.

This suggests proving something stronger: find a subfamily with so many alternative petals that a sparse random set is very likely to contain at least one of them.

Definition 193 (Robust Sunflower)**.** Let $\mathcal{R}$ be a family of at least two finite sets with common intersection

$$C = \bigcap_{S \in \mathcal{R}} S.$$

Let W_γ contain each element outside C, independently, with probability γ. We call $\mathcal{R}$ a (γ, ε)-*robust sunflower* if

$$\boxed{\Pr\left[\exists S \in \mathcal{R} : S \setminus C \subseteq W_\gamma\right] \geq 1 - \varepsilon.}$$

No particular petal need be likely to appear. Robustness says that the family collectively offers enough alternative petals that a sparse random sample almost always contains one.

Theorem 194 (Robust Sunflower Lemma)**.** *There is a universal constant C_0 such that, for $k \geq 1$, $0 < \gamma \leq 1$, and $0 < \varepsilon \leq 1/2$, every family of at least*

$$\boxed{\left(\frac{C_0 \log(2k/\varepsilon)}{\gamma}\right)^k}$$

distinct sets of size at most k contains a (γ, ε)-robust sunflower.

The proof has two conceptual steps.

First, whenever some pattern occurs in too many members of the family, we move that pattern into the common core. What remains is a *spread* family, in which no prescribed set of elements appears too frequently.

Second, we expose the random sample in several rounds. The main task is to define a measure of progress whose failure produces exactly the kind of containment statement that spreadness can count. The resulting halving lemma says that one random round reduces this measure by a factor of two for almost every candidate. Repeating the halving eventually completes a member of the family.

Removing common structure. A nonempty family $\mathcal{F}$ of size M is *R-spread* if

$$\boxed{\#\{S \in \mathcal{F} : Z \subseteq S\} \leq M R^{-|Z|}} \qquad \text{for every nonempty } Z. \tag{52}$$

Thus every prescribed element costs a factor of R: a fixed element occurs in at most a $1/R$-fraction of the family, a fixed pair in at most a $1/R^2$-fraction, and so on.

If the family is not spread, that is not an obstacle but useful information. It means that some nonempty pattern occurs so frequently that it should be regarded as common structure rather than as part of the petals.

Lemma 195 (Extracting a Spread Core)**.** *Let $R > 1$, and let $\mathcal{F}$ contain at least $2R^k$ distinct sets of size at most k. Then there is a subfamily $\mathcal{G} \subseteq \mathcal{F}$ with common intersection C such that*

$$\mathcal{A} = \{S \setminus C : S \in \mathcal{G}\}$$

contains at least two members and is R-spread.

Proof. Start with $C = \emptyset$ and $\mathcal{A} = \mathcal{F}$. If $\mathcal{A}$ is not R-spread, choose a nonempty set Z such that

$$\#\{A \in \mathcal{A} : Z \subseteq A\} > |\mathcal{A}|R^{-|Z|}.$$

Restrict to the members containing Z, add Z to the core, and delete it from every remaining set:

$$C \leftarrow C \cup Z, \qquad \mathcal{A} \leftarrow \{A \setminus Z : A \in \mathcal{A},\ Z \subseteq A\}.$$

The map $A \mapsto A \setminus Z$ is injective on sets containing Z, so the new residual family has more than $|\mathcal{A}|R^{-|Z|}$ members. Meanwhile, every residual set loses $|Z|$ elements.

Repeat until the residual family is spread. If the successive removed patterns are $Z_1, \dots, Z_t$, then $\sum_i |Z_i| \le k$, since all of them have been deleted from every surviving set. Consequently the final family has more than

$$|\mathcal{F}|R^{-\sum_i |Z_i|} \ge 2R^k R^{-k} = 2$$

members.

Its common intersection is empty. Indeed, if some element x belonged to every residual set, then

$$\#\{A \in \mathcal{A} : x \in A\} = |\mathcal{A}| > |\mathcal{A}|R^{-1},$$

contradicting spreadness. Hence the corresponding original sets have common intersection exactly C. □

We may therefore forget the original core and focus on the following problem:

Given a large R-spread family $\mathcal{F}$ of sets of size at most k, prove that a sparse random set contains some member of $\mathcal{F}$ with high probability.

Designing progress backwards from spreadness. Suppose that a set T has already been exposed, and we now reveal a new random block B. We want to prove that only a few members of the family fail to make progress.

Spreadness dictates the form such a proof must take. Its only input is a concrete set Z, and its only conclusion is a bound on the number of family members containing Z. Thus, if $\mathcal{B}$ denotes the members that fail to improve, we would like to associate each $S \in \mathcal{B}$ with a witness $Z \subseteq S$. Grouping failures by their witnesses would then give

$$|\mathcal{B}| \le \sum_Z \#\{S \in \mathcal{F} : Z \subseteq S\} \le M \sum_Z R^{-|Z|}.$$

The larger the witness Z, the more valuable this is. A witness of size b costs the factor R^{-b}.

This identifies the desired measure:

A failure of progress for S should expose a large subset $Z \subseteq S$.

The most obvious progress measure is the global distance

$$d(T) := \min_{Q \in \mathcal{F}} |Q \setminus T|,$$

the number of elements missing from the closest target. This correctly measures how close T is to success, but it has forgotten the index S. If $d(T)$ fails to decrease, there is no failed family member inside which to locate a witness Z, so spreadness has nothing to count.

At the other extreme, one could track the fixed local distance $|S \setminus T|$ for each S. This has the correct interface with spreadness: failure remains localized inside S. But it is much too rigid. If $H = S \setminus T$, then after a density-p random block,

$$\mathbb{E}|H \setminus B| = (1-p)|H|.$$

The sampling probability p will be small, so one cannot expect a single fixed request to shrink by a factor of two.

We need both properties simultaneously:

- progress must be indexed by S, so that failure leaves a subset of S;
- S must have many alternative ways to improve, since a sparse block cannot be expected to help one predetermined target.

This leads to a localized version of the global distance.
For a target $Q \in \mathcal{F}$, call

$$H_T(Q) := Q \setminus T$$

its current *request*: the elements still needed before T contains Q. For each $S \in \mathcal{F}$, define

$$\boxed{d_T(S) := \min\{|Q \setminus T| : Q \in \mathcal{F},\ Q \setminus T \subseteq S\}.}$$

The minimum is always defined, since $Q = S$ is an admissible choice.

Thus $d_T(S)$ is the shortest request, among all targets, that lies entirely inside S. The set S is not itself required to be the target. It acts as a reservoir capable of supplying the remaining elements of whichever target is easiest to finish.

This is precisely a localization of the obvious global distance:

$$d(T) = \min_{S \in \mathcal{F}} d_T(S).$$

It retains the correct notion of closeness to success, but also remembers where the remaining request lives.

Definition 196 (m-Donor)**.** A set $S \in \mathcal{F}$ is an *m-donor for T* if $d_T(S) \le m$. Equivalently, some target $Q \in \mathcal{F}$ has a request $Q \setminus T$ of size at most m, and that entire request lies inside S.

Initially every $S \in \mathcal{F}$ is a k-donor for $T = \emptyset$, witnessed by the target $Q = S$. At the other endpoint, the existence of a 0-donor means that some target has empty request, so T already contains a member of the family.

The definition was chosen so that failure has the form spreadness needs: if S does not become a better donor, the unfulfilled part of an appropriate request remains as a large subset of S.

One random block halves almost every donor. For fixed T and m, let

$$\mathcal{H}_m(T) := \{Q \setminus T : Q \in \mathcal{F},\ |Q \setminus T| \le m\}$$

be the family of *short requests*. Then S is an m-donor if and only if it contains some $H \in \mathcal{H}_m(T)$.

Lemma 197 (Donor Halving)**.** *Let $\mathcal{F}$ be an R-spread family of size M. Fix an exposed set T and an integer $m \ge 1$. Include every element outside T, independently, in a new block B with probability $p \in (0,1)$.*

If $pR \ge 256$, then with probability at least $1/2$, all but at most $M/4^m$ of the m-donors for T become $\lfloor m/2 \rfloor$-donors for $T \cup B$.

Proof. Put $d = \lfloor m/2 \rfloor$ and abbreviate $\mathcal{H} = \mathcal{H}_m(T)$.

From a failed donor to a large obstruction. After the block B is exposed, a request $H \in \mathcal{H}$ has leftover

$$H \setminus B.$$

If one leftover contains another, then the larger one is irrelevant: every donor capable of supplying the larger leftover can also supply the smaller. We therefore retain only the inclusion-minimal leftovers and write

$$\mathcal{M}(B) := \min_{\subseteq}\{H \setminus B : H \in \mathcal{H}\}.$$

Call an m-donor S *bad* if it is not a d-donor for $T \cup B$. Since S was an m-donor, it contains some short request $H \in \mathcal{H}$. Choose an inclusion-minimal leftover $Z \in \mathcal{M}(B)$ contained in $H \setminus B$. Then $Z \subseteq S$.

Moreover,

$$\boxed{|Z| > d.}$$

Indeed, $Z = H' \setminus B$ for some request $H' = Q \setminus T \in \mathcal{H}$. If $|Z| \le d$, then

$$Q \setminus (T \cup B) = Z \subseteq S,$$

so S could donate the at most d remaining elements of Q and would be a d-donor. This contradicts badness.

Thus every bad donor produces exactly the kind of witness for which the progress measure was designed: a large obstruction $Z \subseteq S$.

Spreadness reduces the problem to obstruction weight. Give each obstruction Z weight $R^{-|Z|}$, and define

$$\Phi(B) := \sum_{\substack{Z \in \mathcal{M}(B) \\ |Z| > d}} R^{-|Z|}.$$

A fixed obstruction Z can lie in at most $MR^{-|Z|}$ members of $\mathcal{F}$. Hence, if $D(B)$ is the number of bad donors, then

$$\boxed{D(B) \le M\Phi(B).} \tag{53}$$

The structural part of the proof is complete: failure has been converted into containment of a large concrete set. It remains to show that the random collection of possible obstructions has very small total expected weight.

Repairing a failed sample. The family $\mathcal{M}(B)$ depends on the random block B, so there is no fixed list of obstructions over which to union-bound. Instead, we charge every failure to the outcome obtained by repairing it.

Let V be the universe outside T, and for $A \subseteq V$ let

$$\pi(A) := p^{|A|}(1-p)^{|V|-|A|}$$

be the probability that the new block is exactly A.

Consider a pair (B, Z) with $Z \in \mathcal{M}(B)$ and $|Z| > d$. Since Z is a leftover, it is disjoint from B. Define the *repaired sample*

$$\boxed{Y = B \sqcup Z.}$$

The actual block B omitted every element of the obstruction Z; the repaired block Y additionally selects all of them.

We need two facts: a fixed repaired outcome Y has few possible preimages (B, Z), and each such preimage has small probability after its spreadness weight is included.

A repaired sample has few explanations. For every Y arising from at least one pair (B, Z), choose once and for all a short request $H_Y \in \mathcal{H}$ with $H_Y \subseteq Y$. Such a request exists: if $Z = H \setminus B$, then $H \subseteq B \cup Z = Y$.

Now fix Y and any pair (B, Z) repairing to it. Since $H_Y \subseteq Y = B \sqcup Z$, we have $H_Y \setminus B \subseteq Z$. But $H_Y \setminus B$ is itself a leftover request, while Z is inclusion-minimal among all leftovers. Therefore

$$\boxed{H_Y \setminus B = Z.}$$

In particular, $Z \subseteq H_Y$. Since $|H_Y| \le m$, there are at most 2^m possible choices for Z, and each determines $B = Y \setminus Z$.

This is the second role of minimality. Besides removing redundant leftovers, it traps every obstruction charging to Y inside one fixed request of size at most m.

The cost of one explanation. Let $b = |Z|$. The failed block B and repaired block Y differ only on the elements of Z: they are absent from B and present in Y. Hence

$$\frac{\pi(B)}{\pi(Y)} = \left(\frac{1-p}{p}\right)^b.$$

After incorporating the spreadness weight R^{-b},

$$\boxed{\pi(B)R^{-b} = \pi(Y)\left(\frac{1-p}{pR}\right)^b.} \tag{54}$$

This identity is the heart of the proof. Each obstruction element has two opposing effects:

- the random block omitted it, producing the probability ratio $(1-p)/p$;
- the bad donor had to contain it, producing the spreadness factor $1/R$.

The net cost per obstruction element is therefore approximately $1/(pR)$. The condition $pR \gg 1$ says that the rarity supplied by spreadness more than compensates for the tendency of a sparse random block to miss elements.

Every relevant obstruction has $b > d$, hence $b > m/2$. For a fixed repaired outcome Y, there are at most 2^m possible obstructions, so Equation (54) bounds their total charge by

$$\pi(Y)\, 2^m (pR)^{-m/2} = \pi(Y)\left(\frac{2}{\sqrt{pR}}\right)^m \le \frac{\pi(Y)}{8^m}.$$

Summing over all repaired outcomes Y, whose probabilities add to one, gives

$$\mathbb{E}[\Phi(B)] \le \frac{1}{8^m}.$$

Together with Equation (53),

$$\mathbb{E}[D(B)] \le \frac{M}{8^m}.$$

Finally, Markov's inequality yields

$$\Pr\left[D(B) > \frac{M}{4^m}\right] \le \frac{M/8^m}{M/4^m} = 2^{-m} \le \frac{1}{2}.$$

Thus, with probability at least $1/2$, all but at most $M/4^m$ of the m-donors become d-donors. □

Proof of Theorem 194. The case $\gamma = 1$ is immediate, so assume $0 < \gamma < 1$. Set

$$\ell := \left\lceil 4 \log \frac{2k}{\varepsilon} \right\rceil, \qquad p := 1 - (1-\gamma)^{1/\ell}, \qquad R := \frac{256\ell}{\gamma}.$$

The definition of p ensures that the union of ℓ independent density-p samples is exactly a density-γ sample. Moreover,

$$\gamma = 1 - (1-p)^\ell \le \ell p,$$

so $pR \ge 256$. Since $R = O(\log(2k/\varepsilon)/\gamma)$, choosing C_0 sufficiently large ensures that the hypothesis of the theorem gives

$$|\mathcal{F}| \ge 2R^k.$$

Apply Lemma 195. We obtain a subfamily $\mathcal{G} \subseteq \mathcal{F}$ with common core C whose residual family

$$\mathcal{A} := \{S \setminus C : S \in \mathcal{G}\}$$

is R-spread and has size $M := |\mathcal{A}| \geq 2$.

Choose independent density-p random sets $B_1, \ldots, B_\ell$ on the universe outside C, and let $T_i = B_1 \cup \cdots \cup B_i$.

Initially track every member of $\mathcal{A}$, with donor parameter $m_0 = k$. Every $A \in \mathcal{A}$ is a k-donor for $T_0 = \emptyset$, witnessed by the target $Q = A$. All donor relations will be defined with respect to the full family $\mathcal{A}$, even after we stop tracking some of its members.

At a round with current parameter $m > 0$, call the round *successful* if all but at most $M/4^m$ tracked donors become $\lfloor m/2 \rfloor$-donors. On a successful round, discard the exceptions and replace m by $\lfloor m/2 \rfloor$. On a failed round, retain the tracked family and leave m unchanged. Once $m = 0$, keep it equal to zero.

Conditioned on the past, the newly exposed elements $B_{i+1} \setminus T_i$ form an independent density-p sample of the unused universe. Since $pR \geq 256$, Lemma 197 makes every round with positive parameter successful with conditional probability at least $1/2$.

Failed rounds cause no damage: enlarging T only shrinks every request, so every existing donor relation remains valid.

Some tracked donor survives throughout the process. Indeed, at a successful round with parameter m, at most $M/4^m$ donors are discarded. The positive values of m occurring at successful rounds are distinct, so the total number discarded is at most

$$M \sum_{j \geq 1} 4^{-j} = \frac{M}{3}.$$

At least $2M/3 > 0$ donors therefore survive.

It remains to show that the donor parameter reaches zero. Conditioned on the past, a positive m_i is reduced to at most $\lfloor m_i/2 \rfloor$ with probability at least $1/2$, and otherwise remains unchanged. Therefore

$$\mathbb{E}[m_{i+1} \mid \text{past}] \leq \frac{1}{2} \left\lfloor \frac{m_i}{2} \right\rfloor + \frac{1}{2} m_i \leq \frac{3}{4} m_i.$$

Iterating,

$$\mathbb{E}[m_\ell] \leq k \left(\frac{3}{4} \right)^\ell \leq k e^{-\ell/4} \leq \frac{\varepsilon}{2}.$$

Since m_ℓ is a nonnegative integer,

$$\Pr[m_\ell > 0] \leq \mathbb{E}[m_\ell] \leq \varepsilon.$$

Thus, with probability at least $1 - \varepsilon$, the final parameter is zero. Some tracked donor survives, and a 0-donor means that some target $Q \in \mathcal{A}$ has empty request. Hence $Q \subseteq T_\ell$.

Finally, an element lies outside T_ℓ exactly when all ℓ blocks omit it, an event of probability

$$(1 - p)^\ell = 1 - \gamma.$$

These events are independent across elements, so $T_\ell \overset{d}{=} W_\gamma$. Consequently

$$\Pr[\exists A \in \mathcal{A} : A \subseteq W_\gamma] \geq 1 - \varepsilon.$$

Restoring the common core C, the corresponding original subfamily $\mathcal{G}$ is a (γ, ε)-robust sunflower. □

Corollary 198 (Sunflower Lemma)**.** *There is a universal constant C such that*

$$\boxed{\mathrm{Sun}(k, r) \leq (Cr \log(2k))^k.}$$

Proof. It is enough to consider k-uniform families. For each $S \in \mathcal{F}$, add $k - |S|$ fresh elements that occur in no other member, producing a k-set $\widehat{S}$. For distinct S, T, $\widehat{S} \cap \widehat{T} = S \cap T$, so any sunflower among the padded sets gives one among the originals.

Apply Theorem 194 with $\gamma = 1/(2r)$ and $\varepsilon = 1/2$. For a sufficiently large universal constant, a family of size $(Cr \log(2k))^k$ contains a robust sunflower $\mathcal{R}$ with core $C = \bigcap_{S \in \mathcal{R}} S$.

Because $\mathcal{R}$ is uniform and its members are distinct, every petal $S \setminus C$ is nonempty. Assign every element outside C, independently and uniformly, to one of $2r$ bins $B_1, \ldots, B_{2r}$. Each fixed bin is marginally a density-$1/(2r)$ random set, so robustness gives

$$\Pr[\exists S \in \mathcal{R} : S \setminus C \subseteq B_j] \geq \frac{1}{2}.$$

The expected number of bins containing a complete petal is therefore at least r, so some assignment has at least r good bins.

Choose one complete petal from each of r distinct good bins. The petals are pairwise disjoint because the bins are disjoint, and they belong to distinct family members because they are nonempty. Adding back the common core C gives an r-sunflower. □

Lower Bound. The exponential dependence on r and k cannot be removed.

Proposition 199. *For every $k, r \geq 2$,*

$$\boxed{\mathrm{Sun}(k, r) > (r-1)^k.}$$

Proof. Partition the universe into k disjoint blocks $X_1, \ldots, X_k$, each of size $r-1$, and let $\mathcal{F}$ contain every transversal

$$S = \{x_1, \ldots, x_k\}, \qquad x_i \in X_i.$$

Then $|\mathcal{F}| = (r-1)^k$.

Suppose r distinct members of $\mathcal{F}$ formed a sunflower. Fix a block X_i. The r sets choose r elements from only $r-1$ possibilities, so two choose the same element $x \in X_i$. Any element belonging to two members of a sunflower belongs to the core and hence to every member. Thus all r sets choose the same element from X_i.

Applying this independently to every block shows that all r sets make the same choice in every X_i, so they are identical, a contradiction. Therefore $\mathcal{F}$ contains no r-sunflower. □

7.7.4 Kruskal–Katona and Intersecting Families

For a family $\mathcal{F}$ of r-element sets, its *lower s-shadow* is

$$\partial_s \mathcal{F} := \{B : |B| = s,\ B \subseteq A \text{ for some } A \in \mathcal{F}\}, \qquad 0 \leq s \leq r.$$

Write $\partial \mathcal{F} = \partial_{r-1} \mathcal{F}$ for the immediate shadow. Thus $\partial_0 \mathcal{F} = \{\emptyset\}$ when $\mathcal{F}$ is nonempty, and every shadow of the empty family is empty.

How small can a shadow be if $|\mathcal{F}|$ is fixed? Impose an order on the elements. We should then try to reuse small elements before introducing larger ones. The appropriate ordering compares sets at their *largest* disagreement.

For distinct sets of the same size, define the *colexicographic order* by

$$A <_{\mathrm{colex}} B \quad \Longleftrightarrow \quad \max(A \triangle B) \in B.$$

Let $\mathcal{I}_r(m)$ be the first m members of $\binom{\mathbb{N}}{r}$ in this order, where $\mathbb{N} = \{1, 2, \ldots\}$. In particular, $\mathcal{I}_r(\binom{a}{r}) = \binom{[a]}{r}$.

Theorem 200 (Kruskal–Katona)**.** *Let $\mathcal{F}$ be a finite family of r-element sets, where $r \geq 1$, and put $m = |\mathcal{F}|$. For every $0 \leq s \leq r$,*

$$\boxed{|\partial_s \mathcal{F}| \geq |\partial_s \mathcal{I}_r(m)|.}$$

More explicitly, every positive integer m has a unique expansion

$$m = \binom{a_r}{r} + \binom{a_{r-1}}{r-1} + \cdots + \binom{a_j}{j}, \qquad a_r > a_{r-1} > \cdots > a_j \geq j \geq 1,$$

and then

$$\boxed{|\partial_s \mathcal{F}| \geq \sum_{i=j}^{r} \binom{a_i}{i-r+s}.}$$

Here $\binom{a}{b} = 0$ *for* $b < 0$, *and* $\binom{a}{0} = 1$. *The same family* $\mathcal{I}_r(m)$ *attains equality for every* s.

The proof has two parts. First we move sets earlier in colex without increasing their shadow. Then we count the shadow of the resulting initial segment. Moving one element at a time is insufficient: a family can be stable under every such move without being an initial segment. We therefore allow a whole block to move, but always use a smallest block that still changes the family. This choice is what makes the shadow argument work.

Block compression. For disjoint sets U, V of the same positive size, put

$$c_{U,V}(A) = \begin{cases} (A \setminus V) \cup U, & A \cap (U \cup V) = V, \\ A, & \text{otherwise.} \end{cases}$$

The compressed family $C_{U,V}\mathcal{F}$ replaces A by $c_{U,V}(A)$ only when the latter is not already in $\mathcal{F}$. Equivalently,

$$C_{U,V}\mathcal{F} = \{c_{U,V}(A) : A \in \mathcal{F}\} \cup \{A \in \mathcal{F} : c_{U,V}(A) \in \mathcal{F}\}.$$

For each R disjoint from $U \cup V$, this operation moves $R \cup V$ to $R \cup U$ if only the former is present, and otherwise leaves the pair unchanged. Hence it preserves the number of sets. We call $\mathcal{F}$ (U, V)-*compressed* when $C_{U,V}\mathcal{F} = \mathcal{F}$. The compression with $U = V = \emptyset$ is understood to be the identity.

Lemma 201 (Compress Smaller Blocks First)**.** *Suppose that for every* $u \in U$ *there is a* $v \in V$ *such that* $\mathcal{F}$ *is* $(U \setminus \{u\}, V \setminus \{v\})$-*compressed. Then*

$$\partial(C_{U,V}\mathcal{F}) \subseteq C_{U,V}(\partial\mathcal{F}).$$

In particular, $|\partial(C_{U,V}\mathcal{F})| \leq |\partial\mathcal{F}|$.

Proof. Write $C = C_{U,V}$, and take $B = A \setminus \{a\}$ with $A \in C\mathcal{F}$. We show that $B \in C(\partial\mathcal{F})$.

First suppose that $A \notin \mathcal{F}$. Then $A = R \cup U$ and $R \cup V \in \mathcal{F}$, with $R \cap (U \cup V) = \emptyset$. If $a \in R$, the set $(R \setminus \{a\}) \cup V$ is in $\partial\mathcal{F}$, and its compression is B. If $a = u \in U$, choose $v \in V$ as in the hypothesis. Compressing the smaller block in $R \cup V$ shows that $R \cup (U \setminus \{u\}) \cup \{v\} \in \mathcal{F}$. Thus $B = R \cup (U \setminus \{u\}) \in \partial\mathcal{F}$; moreover, B is fixed by $c_{U,V}$, so it remains in $C(\partial\mathcal{F})$.

Now suppose that $A \in \mathcal{F}$. Then $B \in \partial\mathcal{F}$, so there is nothing to prove unless B is eligible to move. In that case write $B = R \cup V$, with $R \cap (U \cup V) = \emptyset$. It suffices to show that its partner $R \cup U$ also lies in $\partial\mathcal{F}$. If $a \notin U$, then A is eligible to move too. Since it survived in $C\mathcal{F}$, its partner $R \cup U \cup \{a\}$ belongs to $\mathcal{F}$. If $a = u \in U$, choose v as in the hypothesis and compress the smaller block in $A = R \cup V \cup \{u\}$. This gives $R \cup U \cup \{v\} \in \mathcal{F}$. In either case $R \cup U \in \partial\mathcal{F}$, as required. □

Proof of the colex bound in Theorem 200. Relabel the ground set as $[n]$. As long as the current family is not an initial segment, there are A in the family and B outside it with $B <_{\text{colex}} A$. Thus the pair $U = B \setminus A$, $V = A \setminus B$ satisfies $\max U < \max V$ and its compression changes the family.

Among all changing compressions with $\max U < \max V$, choose one minimizing $|U|$. It satisfies the hypothesis of Lemma 201. For $|U| = 1$ the required smaller compression is the identity. For $|U| \geq 2$, given $u \in U$, choose any $v \in V \setminus \{\max V\}$. The pair $(U \setminus \{u\}, V \setminus \{v\})$ still points earlier in colex and has smaller size, so its compression already fixes the family. Consequently the chosen compression does not increase the shadow.

Every nontrivial step strictly decreases the nonnegative integer

$$W(\mathcal{F}) := \sum_{A \in \mathcal{F}} \sum_{a \in A} 2^a,$$

because the largest element of $U \cup V$ lies in V. The process therefore terminates, and it can terminate only at $\mathcal{I}_r(m)$. This proves the immediate-shadow bound.

The next calculation shows that the shadow of a colex initial segment is itself an initial segment. Applying the immediate-shadow bound repeatedly then gives the result for every lower shadow: at each level, the initial segment of the actual shadow's size contains the initial segment of the size already guaranteed. The cases $s = r$, $s = 0$, and $m = 0$ also follow directly from the definitions. □

Counting the extremal shadow. Choose a_r largest with $\binom{a_r}{r} \le m$, subtract this term, and repeat at rank $r-1$ until the remainder vanishes. Pascal's identity gives

$$0 \le m - \binom{a_r}{r} < \binom{a_r}{r-1},$$

so successive upper indices strictly decrease. This proves existence of the expansion in Theorem 200.

It also proves uniqueness. Indeed, if the leading upper index is a, the sum of all possible remaining terms is at most

$$\sum_{i=1}^{r-1} \binom{a-r+i}{i} = \binom{a}{r-1} - 1.$$

The identity follows by repeated use of Pascal's identity; for $r = 1$ the sum is empty. Thus the leading index must be exactly the greedy choice, after which the same argument applies to the remainder.

Put $a = a_r$ and $m' = m - \binom{a}{r}$. The initial segment has the recursive description

$$\mathcal{I}_r(m) = \binom{[a]}{r} \cup \{\{a+1\} \cup B : B \in \mathcal{I}_{r-1}(m')\},$$

where the second family is empty when $m' = 0$. For $1 \le s < r$, its s-shadow is

$$\binom{[a]}{s} \cup \{\{a+1\} \cup B : B \in \partial_{s-1}\mathcal{I}_{r-1}(m')\}.$$

The two parts are disjoint. Recursion shows that this is an initial segment, and counts it as $\sum_{i=j}^{r} \binom{a_i}{i-r+s}$. This completes the proof of the exact formula.

Corollary 202 (Complete Characterization of Face Counts)**.** *For $r \ge 1$, define $K_r(0) = 0$, and, for the binomial expansion above, define*

$$K_r(m) := \sum_{i=j}^{r} \binom{a_i}{i-1}.$$

Nonnegative integers $f_0, f_1, \dots, f_d$, with $f_0 = 1$, are the numbers of faces of sizes $0, 1, \dots, d$ of a finite simplicial complex if and only if

$$K_r(f_r) \le f_{r-1} \qquad (1 \le r \le d).$$

Here a simplicial complex is a family of finite sets closed under taking subsets; its face of size zero is $\emptyset$.

Proof. Necessity is Theorem 200 applied at every rank, since the shadow of the r-faces is contained in the $(r-1)$-faces. For sufficiency, take $\emptyset$ together with $\mathcal{I}_r(f_r)$ at each positive rank. The shadow at rank r is the initial segment of size $K_r(f_r)$, so the stated inequalities say exactly that this union is closed under taking subsets. □

Corollary 203 (Erdős–Ko–Rado)**.** *Let $n \ge 2k$ and $k \ge 1$. If $\mathcal{F} \subseteq \binom{[n]}{k}$ is* intersecting*, meaning that any two of its members meet, then*

$$\boxed{|\mathcal{F}| \le \binom{n-1}{k-1}.}$$

The bound is attained by all k-sets containing one fixed element.

Proof. Suppose instead that $|\mathcal{F}| > \binom{n-1}{k-1}$, and take the family of complements $\mathcal{G} = \{[n] \setminus A : A \in \mathcal{F}\}$. It consists of more than $\binom{n-1}{n-k}$ sets of size $n-k$. Since $k \le n-k$, Kruskal–Katona gives $|\partial_k \mathcal{G}| \ge \binom{n-1}{k}$: the corresponding colex initial segment contains all $(n-k)$-sets of $[n-1]$. But $\mathcal{F} \cap \partial_k \mathcal{G} = \emptyset$, since a member of this intersection would be disjoint from a member of $\mathcal{F}$. Therefore

$$\binom{n}{k} \ge |\mathcal{F}| + |\partial_k \mathcal{G}| > \binom{n-1}{k-1} + \binom{n-1}{k} = \binom{n}{k},$$

a contradiction. □

7.8 Ramsey Theory

7.8.1 Hales–Jewett Theorem

Let

$$[m] = \{0, 1, \dots, m-1\}.$$

A *combinatorial line* in $[m]^N$ is obtained from a word $\tau(*) \in ([m] \cup \{*\})^N$ containing at least one wildcard. For $a \in [m]$, let $\tau(a)$ be the word obtained by replacing every occurrence of $*$ by a. The corresponding line is

$$\{\tau(0), \dots, \tau(m-1)\}.$$

Let $HJ(c, m)$ denote the least N such that every c-coloring of $[m]^N$ contains a monochromatic combinatorial line.

For the quantitative bound, define the hierarchy

$$h_1(n) = 2n, \qquad h_{r+1}(n) = h_r^n(1),$$

where h_r^n denotes n-fold composition. Thus $h_2(n) = 2^n$, $h_3(n)$ is a tower of 2's of height n, and h_4 iterates the tower function.

Theorem 204 (Hales–Jewett)**.** *For every $c, m \ge 1$,*

$$\boxed{HJ(c, m) \le \frac{1}{cm} h_4(c + m + 2).}$$

In particular, $HJ(c, m)$ is finite and has a primitive-recursive upper bound.

We induct on the alphabet size and force the symbols 0 and 1 to become indistinguishable. We may then merge them into a single symbol and invoke Hales–Jewett for an alphabet of size $m-1$.

Proof of the reduction. We induct on m. The case $m = 1$ is trivial.

Assume the theorem for an alphabet of size $m-1$, and set

$$\ell = HJ(c, m-1).$$

We shall construct ℓ blocks

$$\tau_1(*), \dots, \tau_\ell(*)$$

with the following property:

$$\chi\big(\tau_1(\nu_1) \cdots \tau_\ell(\nu_\ell)\big) \tag{55}$$

is unchanged if any collection of coordinates ν_i is switched between 0 and 1.

Once this is achieved, 0 and 1 can be regarded as the same letter, leaving an alphabet of size $m-1$. The induction hypothesis will then finish the proof.

Constructing the Blocks We choose the block lengths large enough for a repeated pigeonhole argument. Set $L_0 = 0$, and for $i = 1, \ldots, \ell$ define

$$n_i = c^{\,m^{L_{i-1}+\ell-i}}, \qquad L_i = L_{i-1} + n_i.$$

We claim that

$$HJ(c, m) \le L_\ell.$$

Let

$$\chi : [m]^{L_\ell} \to [c]$$

be arbitrary, and partition the coordinates into consecutive blocks of lengths $n_1, \ldots, n_\ell$.

We construct the block roots backwards, starting with $\tau_\ell(*)$ and ending with $\tau_1(*)$. Suppose $\tau_{i+1}, \ldots, \tau_\ell$ have already been chosen.

For $0 \le t \le n_i$, let

$$W_t = 0^t 1^{\,n_i - t}.$$

Think of W_t as placing a boundary between a string of 0's and a string of 1's. We shall find two positions of this boundary that are indistinguishable to the coloring.

For each t, consider the function

$$\Phi_t : [m]^{L_{i-1}} \times [m]^{\ell-i} \longrightarrow [c]$$

defined by

$$\Phi_t(z, \nu_{i+1}, \ldots, \nu_\ell) = \chi\big(z\, W_t\, \tau_{i+1}(\nu_{i+1}) \cdots \tau_\ell(\nu_\ell)\big).$$

The arguments of Φ_t describe everything outside the i-th block that can still affect its color: z is an arbitrary assignment to all earlier coordinates, while $\nu_{i+1}, \ldots, \nu_\ell$ specify the already constructed later blocks.

There are

$$m^{L_{i-1}+\ell-i}$$

such contexts. A function assigning one of c colors to every context can therefore take at most

$$c^{\,m^{L_{i-1}+\ell-i}} = n_i$$

different values.

But we have $n_i + 1$ words

$$W_0, W_1, \ldots, W_{n_i}.$$

By the pigeonhole principle, two of them, say W_t and W_u with $t < u$, induce exactly the same function:

$$\Phi_t = \Phi_u.$$

Define

$$\tau_i(*) = 0^t\, *^{\,u-t}\, 1^{\,n_i - u}.$$

Then

$$\tau_i(0) = W_u, \qquad \tau_i(1) = W_t.$$

Thus changing the parameter of the i-th block from 0 to 1, or back again, does not change the color, no matter how the earlier coordinates or later block parameters are chosen.

Proceeding backwards through all ℓ blocks proves Equation (55).

Merging 0 and 1 For $\nu = (\nu_1, \ldots, \nu_\ell) \in [m]^\ell$, write

$$\tau(\nu) = \tau_1(\nu_1) \cdots \tau_\ell(\nu_\ell).$$

By construction, $\chi(\tau(\nu))$ is unchanged whenever any coordinates of ν are switched between 0 and 1. We may therefore identify these two symbols. Let

$$\overline{[m]} = \{\star, 2, \ldots, m-1\},$$

where $\star$ represents either 0 or 1. Define

$$\overline{\chi}(\bar{\nu}) = \chi(\tau(\nu)),$$

where each occurrence of $\star$ in $\bar{\nu}$ may be represented by either 0 or 1 in ν.
The invariance just proved makes $\overline{\chi}$ well defined. It is a c-coloring of

$$\overline{[m]}^\ell,$$

whose alphabet has size $m-1$.
Since $\ell = HJ(c, m-1)$, this coloring contains a monochromatic combinatorial line. Let

$$\sigma(*) \in (\overline{[m]} \cup \{*\})^\ell$$

be a root for such a line.

Lifting the Line We now turn $\sigma(*)$ into a root in the original space $[m]^{L_\ell}$, block by block.
For coordinate i,

- if $\sigma_i = *$, use the entire root $\tau_i(*)$;
- if $\sigma_i = j \in \{2, \ldots, m-1\}$, use the fixed block $\tau_i(j)$;
- if $\sigma_i = \star$, use the fixed block $\tau_i(0)$.

Call the resulting root $\rho(*)$.
For $a = 2, \ldots, m-1$, the word $\rho(a)$ maps exactly to the a-point of the monochromatic line for $\overline{\chi}$. The word $\rho(0)$ maps to its $\star$-point.
The word $\rho(1)$ differs from $\rho(0)$ only by changing some block parameters from 0 to 1. By Equation (55), this does not change its color. Therefore

$$\chi(\rho(0)) = \chi(\rho(1)) = \cdots = \chi(\rho(m-1)).$$

Thus $\rho(*)$ gives a monochromatic combinatorial line in $[m]^{L_\ell}$. □

Quantitative Bound The preceding argument gives an explicit recursive bound: if $B(c, 1) = 1$, and for $m \geq 2$ we set $\ell = B(c, m-1)$ and recursively

$$L_0 = 0, \qquad L_i = L_{i-1} + c^{m^{L_{i-1}+\ell-i}}, \qquad B(c, m) = L_\ell,$$

then

$$HJ(c, m) \leq B(c, m).$$

7.8.2 The Van der Waerden Theorem

A combinatorial line becomes an arithmetic progression simply by summing its coordinates: each wildcard contributes one copy of the same parameter.

Let $W(c, m)$ be the least M such that every c-coloring of $\{1, \dots, M\}$ has a monochromatic arithmetic progression of length m and positive common difference.

Theorem 205 (Van der Waerden)**.** *For every $c, m \geq 1$,*

$$W(c, m) \leq (m-1)HJ(c, m) + 1.$$

In particular, every finite coloring of the positive integers has arbitrarily long monochromatic arithmetic progressions.

Proof. Let $N = HJ(c, m)$ and $M = (m-1)N + 1$. Given a coloring χ of $\{1, \dots, M\}$, color words $w \in [m]^N$ by

$$\widehat{\chi}(w) = \chi\left(1 + \sum_{i=1}^{N} w_i\right).$$

Theorem 204 gives a monochromatic combinatorial line. Let $J \neq \varnothing$ be its wildcard coordinates and b_i its fixed coordinates. The images of its m words are

$$a + dt, \qquad t = 0, 1, \dots, m-1, \qquad a = 1 + \sum_{i \notin J} b_i, \quad d = |J| \geq 1.$$

They lie in $\{1, \dots, M\}$ and have the same color. □

The sum map need not be injective on all words. It only needs to send the selected line to a nondegenerate progression, which is exactly what $|J| > 0$ guarantees.

Theorem 206 (Gallai–Witt)**.** *For every nonempty finite $F \subseteq \mathbb{Z}^d$, every finite coloring of $\mathbb{Z}^d$ contains a monochromatic set $a + tF$, where $a \in \mathbb{Z}^d$ and t is a positive integer.*

Proof. Use F itself as the alphabet and put $N = HJ(c, |F|)$. Color a word $(v_1, \dots, v_N) \in F^N$ by the color of $v_1 + \cdots + v_N$. On a combinatorial line, the fixed coordinates sum to some a, while the $t \geq 1$ wildcard coordinates contribute t copies of the varying letter. The images are exactly $a + tF$. □

7.8.3 Schur's Theorem

Let $R(n_1, \dots, n_c)$ be the least N such that every c-coloring of the edges of K_N has, for some i, a clique of size n_i entirely in color i. Write

$$R_c = R(\underbrace{3, \dots, 3}_{c \text{ entries}}).$$

The identity $x + y = z$ can be built into a triangle: its three positive differences always satisfy it. Ramsey then has only the monochromaticity left to enforce.

Theorem 207 (Schur)**.** *Every c-coloring of $\{1, \dots, R_c - 1\}$ has monochromatic positive integers x, y, z with $x + y = z$. The values x and y are allowed to coincide.*

Proof. On vertices $0, 1, \dots, R_c - 1$, give the edge ij the color of $|i - j|$. A monochromatic triangle with vertices $a < b < d$ gives

$$x = b - a, \qquad y = d - b, \qquad z = d - a,$$

which have the same color and satisfy $x + y = z$. □

Power equations over finite fields. Color field elements by cosets of the subgroup of kth powers; a monochromatic additive triple then has a common multiplicative factor which can be canceled.

Corollary 208. *For every $k \geq 1$ and every prime $p \geq R_k$, there are nonzero $a, b, d \in \mathbb{F}_p$ such that $a^k + b^k = d^k$.*

Proof. The subgroup $H = \{t^k : t \in \mathbb{F}_p^*\}$ has index at most k. Indeed, the homomorphism $t \mapsto t^k$ has kernel of size at most k, since $t^k - 1$ has at most k roots; counting its fibers gives $[\mathbb{F}_p^* : H] = |\ker(t \mapsto t^k)|$.

Color $1, \ldots, R_k - 1$ by their cosets of H in $\mathbb{F}_p^*$. They are nonzero since $p \geq R_k$. Schur gives $x + y = z$ in one coset gH. Write $x = ga^k$, $y = gb^k$, $z = gd^k$ with $a, b, d \neq 0$, and cancel g. □

7.8.4 Szemerédi's Cube Lemma

A *Hilbert cube of dimension d* is a set

$$x + \{0, h_1\} + \cdots + \{0, h_d\} = \left\{ x + \sum_{i=1}^{d} \varepsilon_i h_i : \varepsilon_i \in \{0, 1\} \right\}, \qquad h_1, \ldots, h_d > 0.$$

The steps are positive integers. Different choices of the ε_i may give the same vertex; a cube with 2^d distinct vertices is called *proper*. The lemma below concerns Hilbert cubes without a properness requirement.

To build a cube, find a smaller cube in $A \cap (A - h)$ and then add its translate by h. Thus the useful quantity is the largest number of pairs sharing one positive difference. We estimate it by considering differences between nearby entries in sorted order: their sum is small by telescoping, while bounded repetition forces their sum to be large.

Lemma 209 (Short differences)**.** *Let $A \subseteq \{1, \ldots, N\}$, where $N \geq 2$. Suppose that $|A \cap (A - h)| \leq M$ for every positive integer h, where $M \geq 1$. Then*

$$|A| \leq \sqrt{M(N-1)} + \big(M(N-1)\big)^{1/4} + 1.$$

Proof. Write $A = \{a_1 < \cdots < a_m\}$ and put $X = M(N-1)$. For an integer $1 \leq t < m$, take all differences $a_{i+j} - a_i$ with $1 \leq j \leq t$ and $1 \leq i \leq m - j$. Their number is $L = tm - t(t+1)/2$. Each positive integer occurs at most M times, so the kth smallest difference is at least k/M. Their total sum is therefore at least $L^2/(2M)$.

On the other hand, for each j the sum telescopes:

$$\sum_{i=1}^{m-j} (a_{i+j} - a_i) = \sum_{i=m-j+1}^{m} a_i - \sum_{i=1}^{j} a_i \leq j(N-1).$$

Summing over j gives

$$\frac{L^2}{2M} \leq \frac{(N-1)t(t+1)}{2}, \qquad m \leq \frac{t+1}{2} + \sqrt{X}\sqrt{1 + \frac{1}{t}} \leq \sqrt{X} + \frac{t+1}{2} + \frac{\sqrt{X}}{2t}.$$

Choose $t = \lceil X^{1/4} \rceil$. If $t \geq m$, the claimed bound is immediate. Otherwise the last two terms are at most $X^{1/4} + 1$, which proves the claim. □

For $M = 1$, this already gives the familiar bound $|A| \leq \sqrt{N-1} + (N-1)^{1/4} + 1$ for a set with distinct positive differences. Iterating the same estimate gives the cube lemma with a leading factor tending to one.

Theorem 210 (Szemerédi's cube lemma)**.** *Let $N \geq 16$ and $d \geq 2$ be integers. Every set $A \subseteq \{1, \ldots, N\}$ satisfying*

$$|A| > \big(1 + N^{-1/4}\big)^2 N^{\,1 - 1/2^{d-1}}$$

contains a Hilbert cube of dimension d.

Proof. Let F_d be the largest size of a subset of $\{1,\dots,N\}$ containing no d-dimensional Hilbert cube. Clearly $F_1 = 1$. If A contains no d-cube, then $A \cap (A-h)$ contains no $(d-1)$-cube for any $h > 0$: such a cube together with its translate by h would lie in A. Lemma 209 therefore gives

$$F_d \le \sqrt{NF_{d-1}} + (NF_{d-1})^{1/4} + 1.$$

Put $u = N^{-1/4} \le 1/2$. We prove by induction that $F_d \le (1+u)^2 N^{\,1-1/2^{d-1}}$ for $d \ge 2$. For $d = 2$, the recurrence gives

$$\frac{F_2}{N^{1/2}} \le 1 + u + u^2 \le (1+u)^2.$$

For $d \ge 3$, write $\alpha = 1 - 1/2^{d-1} \ge 3/4$. The induction hypothesis and the recurrence give

$$\frac{F_d}{N^{\alpha}} \le 1 + u + N^{-\alpha/2}\sqrt{1+u} + N^{-\alpha} \le 1 + u + u^{3/2}\sqrt{1+u} + u^3 \le (1+u)^2.$$

The last inequality uses $u^{3/2}\sqrt{1+u} = u\sqrt{u+u^2} \le u$ and $u^3 \le u^2$, both consequences of $0 < u \le 1/2$. □

Equivalently, if $|A| = \delta N$ with $0 < \delta \le 1$, a d-cube is guaranteed whenever

$$2^{d-1} < \frac{\log N}{\log\bigl((1+N^{-1/4})^2/\delta\bigr)}.$$

In particular, fixed positive density gives dimension $\log_2 \log N - O_\delta(1)$. The displayed condition also applies when the density depends on N. For example, if $N \ge 64$ and $|A| \ge N/2$, we may take $d = \lfloor \log_2 \log_2 N \rfloor$: then $N^{1/2^{d-1}} \ge 4$ and $(1+N^{-1/4})^2 < 2$, so the theorem applies.

7.8.5 Infinite Ramsey Theorem

Write $\binom{X}{r}$ for the family of r-element subsets of X. The infinite pigeonhole principle says that a finite coloring of an infinite set has an infinite monochromatic subset. Ramsey's theorem allows us to color not just elements, but all fixed-size subsets.

Theorem 211 (Infinite Ramsey theorem)**.** *Let X be infinite and let $r, c \ge 1$ be integers. Every coloring $\chi : \binom{X}{r} \to \{1,\dots,c\}$ has an infinite subset $H \subseteq X$ on which all r-element subsets have the same color.*

The useful intermediate goal is weaker than monochromaticity: make the color of a subset depend only on its *first chosen member*. Then one more application of pigeonhole removes that last dependence. Each chosen member is handled by restricting all future choices to a suitable infinite set.

Proof. Induct on r. For $r = 1$, use the infinite pigeonhole principle. Suppose $r \ge 2$, and set $X_0 = X$. Choose $x_i \in X_{i-1}$, and color the $(r-1)$-element subsets of $X_{i-1} \setminus \{x_i\}$ by $S \mapsto \chi(\{x_i\} \cup S)$. By induction there is an infinite $X_i \subseteq X_{i-1} \setminus \{x_i\}$ on which this coloring is constant, say with color a_i.

Continue for $i = 1, 2, \dots$. Whenever $i_1 < \cdots < i_r$, all of $x_{i_2}, \dots, x_{i_r}$ lie in X_{i_1}, so

$$\chi(\{x_{i_1}, \dots, x_{i_r}\}) = a_{i_1}.$$

Some color a occurs infinitely often in (a_i). The corresponding x_i form the required set H. □

Corollary 212. *Every infinite graph has an infinite clique or an infinite independent set. Every infinite partially ordered set has an infinite chain or an infinite antichain.*

Proof. For graphs, color a pair according to whether it is an edge. For partially ordered sets, color a pair according to whether its members are comparable. Apply the case $r = 2$. □

Dickson's lemma: finitely many minimal vectors. A more algebraic application turns an infinite sequence of integer vectors into a coordinatewise nondecreasing subsequence. The coloring records which coordinates decrease; homogeneity would force any such coordinate to decrease forever, which is impossible.

Lemma 213 (Dickson)**.** *Every infinite sequence in $\mathbb{Z}^d_{\geq 0}$ has an infinite coordinatewise nondecreasing subsequence. Consequently, every $S \subseteq \mathbb{Z}^d_{\geq 0}$ has finitely many minimal elements under coordinatewise comparison, and every element of S lies above one of them.*

Proof. Given $v_1, v_2, \ldots$, color each pair $i < j$ by the set $\{k : v_i(k) > v_j(k)\}$, using at most 2^d colors. Infinite Ramsey gives an infinite homogeneous set of indices. Its common color must be empty: a coordinate belonging to that color would give an infinite strictly decreasing sequence of nonnegative integers. The corresponding subsequence is therefore coordinatewise nondecreasing.

Distinct minimal elements of S are incomparable, so there cannot be infinitely many. For any $v \in S$, the nonempty set $\{u \in S : u \leq v\}$ is finite; a minimal member of this set is also minimal in S. □

In particular, for any field K, every monomial ideal in $K[x_1, \ldots, x_d]$ is finitely generated. Associate $x_1^{a_1} \cdots x_d^{a_d}$ with its exponent vector a. Divisibility is coordinatewise comparison, so the finitely many minimal exponent vectors supply generators: every monomial in the ideal is divisible by one of them.

An infinite sequence with monochromatic interval sums. Coloring differences converts additive identities into relations between edges. Infinite Ramsey gives more than a single additive triple: all consecutive sums of one infinite sequence can have the same color.

Corollary 214. *Every finite coloring of the positive integers admits a sequence $a_1, a_2, \ldots$ of positive integers such that all sums*

$$a_i + a_{i+1} + \cdots + a_j, \qquad 1 \leq i \leq j,$$

have the same color.

Proof. Color a pair of nonnegative integers $s < t$ by the color of $t - s$. Choose an infinite homogeneous set $b_0 < b_1 < \cdots$, and put $a_i = b_i - b_{i-1}$. Then $a_i + \cdots + a_j = b_j - b_{i-1}$ always has the homogeneous color. □

7.8.6 Kőnig's Infinity Lemma

A *ray* is a one-way infinite simple path $v_0 v_1 v_2 \cdots$. A rooted tree is *finitely branching* if each vertex has finitely many children.

Arbitrarily long partial constructions need not be compatible. Kőnig's lemma makes them compatible when every step has only finitely many choices: at least one choice must leave infinitely many possibilities for the future.

Lemma 215 (Kőnig's infinity lemma)**.** *Every infinite finitely branching rooted tree contains a ray starting at its root.*

Proof. The root has infinitely many descendants. Since it has finitely many children, one child also has infinitely many descendants. Choose that child and repeat. Depth strictly increases, so the chosen vertices form a ray. □

Finite branching is essential even if there are paths of every finite length: attach to one root, for each $n \geq 1$, a separate finite path of length n. The resulting tree has no ray.

Corollary 216 (Rays in locally finite graphs)**.** *Every infinite connected locally finite graph has a ray starting at any prescribed vertex v. In fact, it has a ray $v = v_0, v_1, \ldots$ with $\operatorname{dist}(v, v_i) = i$; in particular, this ray is induced.*

Proof. Every finite-radius ball about v is finite, by induction on its radius. Thus there are vertices arbitrarily far from v. Form the rooted tree of finite shortest paths starting at v, where a path's parent is obtained by deleting its last vertex. It is infinite and finitely branching. A ray in this tree gives compatible shortest paths of all lengths, hence the desired ray in the original graph. An edge joining v_i to v_j with $j \geq i+2$ would contradict $\operatorname{dist}(v, v_j) = j$. □

Why infinite Ramsey implies finite Ramsey. The missing step is compatibility: if finite counterexamples existed at every size, Kőnig's lemma would assemble them into one infinite counterexample.

Corollary 217 (Finite Ramsey theorem)**.** *For all integers $r, c \geq 1$ and $m \geq r$, some N has the property that every c-coloring of $\binom{\{1,\dots,N\}}{r}$ has a monochromatic m-element subset.*

Proof. Suppose otherwise. At level N put all colorings on $\binom{\{1,\dots,N\}}{r}$ without a monochromatic m-element subset, and join each coloring to its restriction at level $N-1$. Include the empty coloring as root. These form a finitely branching tree with a vertex at every level. An infinite branch gives compatible colorings whose union colors $\binom{\mathbb{Z}_{>0}}{r}$ without a monochromatic m-element subset, contradicting infinite Ramsey. □

7.8.7 Compactness for Colorings and Matchings

The same tree argument applies whenever variables have finite choice sets and a violation can be witnessed using finitely many variables. This is a useful way to transfer finite existence results to countably infinite objects.

Lemma 218 (Countable finite-choice compactness)**.** *Let $x_1, x_2, \dots$ be variables, with x_i taking values in a nonempty finite set D_i. Consider constraints each involving only finitely many variables. If every finite collection of constraints is simultaneously satisfiable, then all constraints are simultaneously satisfiable.*

Proof. At level n, take all assignments to $x_1, \dots, x_n$ satisfying every constraint involving only these variables. This level is nonempty: otherwise, for each of the finitely many assignments, choose a constraint it violates. Those finitely many constraints would already be unsatisfiable.

Restriction gives a finitely branching rooted tree. By Kőnig's lemma it has an infinite branch, defining all variables. Every constraint is satisfied once the branch reaches a level containing all its variables. □

Theorem 219 (De Bruijn–Erdős, countable case)**.** *Let G be a countable graph and $k \geq 1$ a finite integer. Then G is k-colorable if and only if every finite subgraph of G is k-colorable.*

Proof. Necessity follows by restriction. Conversely, introduce a variable $x_v \in \{1, \dots, k\}$ for each vertex and a constraint $x_u \neq x_v$ for each edge uv. Any finite collection of constraints lies in a finite subgraph and is satisfiable. Apply finite-choice compactness. □

Theorem 220 (Infinite Hall theorem, countable case)**.** *Let $G = (L, R; E)$ be bipartite, with L countable and each vertex of L having finite degree. There is a matching covering L if and only if*

$$|N(S)| \geq |S| \qquad \text{for every finite } S \subseteq L.$$

Proof. Necessity is immediate. For sufficiency, every finite $F \subseteq L$ can be matched into $N(F)$ by finite Hall. Now choose a variable $x_v \in N(v)$ for each $v \in L$, and impose $x_u \neq x_v$ for distinct $u, v \in L$. Every finite collection of these constraints involves finitely many left vertices, which can be matched as just proved. Finite-choice compactness supplies all choices simultaneously, and the edges vx_v form the required matching. □

Finite neighborhoods cannot be omitted. Consider the sets $D_0 = \mathbb{Z}_{>0}$ and $D_i = \{i\}$ for $i \geq 1$. Every finite subfamily has distinct representatives, but the whole family does not: the singleton sets use every possible choice for D_0.

7.9 Additive Combinatorics

Unless stated otherwise, sets are finite subsets of an abelian group. Write $A \pm B = \{a \pm b : a \in A, b \in B\}$ and $hA = \{a_1 + \cdots + a_h : a_i \in A\}$, with $0A = \{0\}$. Repetition is allowed in hA.

For a multiset $X = \{x_1, \ldots, x_n\}$, occurrences are indexed even when their values agree, and $|X| = n$. Write $\Sigma(X) = \sum_{i=1}^{n} x_i$ for its total weight and $\mathcal{S}(X) = \{\sum_{i \in I} x_i : I \subseteq [n]\}$ for its subset sums, including zero. Thus $\mathcal{S}(X) = \{0, x_1\} + \cdots + \{0, x_n\}$, and each occurrence may be used at most once. Subsets of multisets select occurrences; $X \uplus Y$ adds their multiplicities, so $\mathcal{S}(X \uplus Y) = \mathcal{S}(X) + \mathcal{S}(Y)$ and $\Sigma(X \uplus Y) = \Sigma(X) + \Sigma(Y)$. All sets in denominators are nonempty; $[M] = \{1, \ldots, M\}$, and log denotes the natural logarithm.

Addition either produces growth or exposes structure that prevents it. Kneser's theorem identifies the exact obstruction to elementary growth: periodicity. Ruzsa's and Plünnecke's inequalities propagate small growth, while additive energy detects it inside a large subset. Fourier analysis and almost-periodicity then turn small growth into subspace structure. For Subset Sum and Knapsack, a complementary objective is to find structure *inside the attainable sums*: long progressions allow us to fill targets and force exchanges of equal weight.

7.9.1 Kneser's Addition Theorem

The *stabilizer* of a finite nonempty set S is the finite subgroup $H(S) = \{g : S + g = S\}$. It is a subgroup because translations compose and invert; it is finite because $g \mapsto s + g$ injects it into S for any fixed $s \in S$.

The expected bound $|A + B| \geq |A| + |B| - 1$ fails when, for example, $A = B = H$ is a nontrivial finite subgroup. Here $A + B = H$: adding elements merely moves within an existing coset. Kneser's theorem says that this is the only obstruction, once we pass to the stabilizer of the *sumset*.

Theorem 221 (Kneser's addition theorem)**.** *For finite nonempty subsets A, B of an abelian group, put $H = H(A + B)$. Then*

$$\boxed{|A + B| \geq |A + H| + |B + H| - |H|.}$$

All three sets are unions of H-cosets. Dividing by $|H|$ gives the ordinary sum-of-sizes-minus-one inequality for their images in G/H, where the sumset has trivial stabilizer. The proof has two parts: a union cannot lose too much periodicity without gaining elements, and moving elements between the summands eventually forces periodicity.

Lemma 222 (Union of periodic sets)**.** *If $S = S_1 \cup \cdots \cup S_m$, where the S_i are finite and nonempty, then $|S| + |H(S)| \geq \min_i \big(|S_i| + |H(S_i)|\big)$.*

Proof. It suffices to prove the two-set case and then induct. Put $H_i = H(S_i)$ and $H = H(S_1 \cup S_2)$. Quotient by $H_1 \cap H_2$: this subgroup stabilizes both sets and their union, so every cardinality in the inequality is divided by its order. Thus assume $H_1 \cap H_2 = \{0\}$, and write $h_i = |H_i|$.

Each coset of $K = H_1 + H_2$ is an h_2-by-h_1 grid: its rows are H_1-cosets and its columns are H_2-cosets, with one point at each intersection. Within such a coset, S_1 consists of r whole rows and S_2 of c whole columns. Their local contributions to $S_2 \setminus S_1$ and $S_1 \setminus S_2$ are $c(h_2 - r)$ and $r(h_1 - c)$, respectively.

If some coset has $0 < r < h_2$ and $0 < c < h_1$, the product of these two gains is $r(h_2 - r)c(h_1 - c) \geq (h_2 - 1)(h_1 - 1)$. Consequently either $|S_2 \setminus S_1| \geq h_1 - 1$ or $|S_1 \setminus S_2| \geq h_2 - 1$. Since $|H| \geq 1$, the first inequality gives $|S| + |H| \geq |S_1| + h_1$, and the second gives $|S| + |H| \geq |S_2| + h_2$.

Otherwise, every nonempty coset not completely filled by S meets only one of the two sets. If one such coset meets only S_1 and another only S_2, then $|S_1 \setminus S_2| \geq h_1$ and $|S_2 \setminus S_1| \geq h_2$. If $h_1 \geq h_2$, use the first gain to compensate for the stabilizer of S_2; if $h_2 \geq h_1$, use the second for S_1.

In the remaining case, all partially filled cosets come from the same set, say S_1. Both these cosets and the completely filled cosets are unions of H_1-cosets. Thus $S + H_1 = S$, which means $H_1 \leq H$, and $|S| + |H| \geq |S_1| + |H_1|$. □

Proof of Theorem 221. First we prove the weaker-looking inequality

$$|A + B| + |H(A + B)| \geq |A| + |B|. \tag{56}$$

Fix $b \in B$. Among finite pairs (A', B') with $A \subseteq A'$, $b \in B'$, $A' + B' \subseteq A + B$, and $|A'| + |B'| = |A| + |B|$, choose (A_b, B_b) minimizing $|B_b|$. The original pair is admissible. Put $S_b = A_b + B_b$.

For $t \in A_b - b$, consider the transfusion $A' = A_b \cup (B_b + t)$, $B' = B_b \cap (A_b - t)$. Each removed $y \in B_b$ corresponds to the newly inserted $y + t \in A'$, so total cardinality is preserved, and $b \in B'$. Moreover $A' + B' \subseteq A_b + B_b$: the only new type of sum is $(z + t) + y = z + (y + t)$, where $z \in B_b$ and $y + t \in A_b$. The transformed pair is therefore admissible. Minimality forces $B' = B_b$, or

$$B_b + t \subseteq A_b \qquad (t \in A_b - b). \tag{57}$$

For $c \in B_b - b$ and $a \in A_b$, take $t = a - b$ to obtain $a + c \in A_b$. Hence $A_b + c \subseteq A_b$, and finiteness gives equality. Thus $B_b - b \subseteq H(A_b)$: if such a difference were not a period, the transfusion could still move another element. In particular $S_b = A_b + b$ and $|H(A_b)| \geq |B_b|$, so $|S_b| + |H(S_b)| \geq |A_b| + |B_b| = |A| + |B|$. As b ranges over B, the inclusions $A + b \subseteq S_b \subseteq A + B$ show that $A + B = \bigcup_{b \in B} S_b$. The union lemma proves (56).

Finally put $H = H(A + B)$. The sets $A + H$ and $B + H$ still sum to $A + B$, whose stabilizer is H. Applying (56) to these enlarged summands yields $|A + B| + |H| \geq |A + H| + |B + H|$, as required. □

Corollary 223 (Cauchy–Davenport)**.** *If p is prime and $A, B \subseteq \mathbb{Z}/p\mathbb{Z}$ are nonempty, then $|A + B| \geq \min\{p, |A| + |B| - 1\}$. More generally, for nonempty $A_1, \ldots, A_h \subseteq \mathbb{Z}/p\mathbb{Z}$,*

$$|A_1 + \cdots + A_h| \geq \min\left\{p, \sum_{i=1}^{h} |A_i| - h + 1\right\}.$$

Proof. If $A + B$ is proper, its stabilizer is a proper subgroup of a prime-order group, hence trivial. Apply Kneser; the iterated statement follows by induction. □

Corollary 224 (Growth of modular subset sums)**.** *Let p be prime. For nonzero $a_1, \ldots, a_t \in \mathbb{Z}/p\mathbb{Z}$, let $\mathcal{S}_j = \{\sum_{i \in I} a_i : I \subseteq [j]\}$. Then $|\mathcal{S}_j| \geq \min\{p, j + 1\}$. In particular, any $p - 1$ nonzero input occurrences have subset sums covering the whole group.*

Proof. Use $\mathcal{S}_j = \mathcal{S}_{j-1} + \{0, a_j\}$, starting from $|\mathcal{S}_0| = 1$. Cauchy–Davenport increases the size by at least one at each step until the group is filled. □

7.9.2 The Erdős–Heilbronn Conjecture

For $A \subseteq \mathbb{F}_p$, define the restricted sumset

$$A \dot{+} A = \{a + b : a, b \in A,\ a \neq b\}.$$

Theorem 225 (Erdős–Heilbronn)**.** *For every nonempty $A \subseteq \mathbb{F}_p$,*

$$\boxed{|A \dot{+} A| \geq \min\{p,\ 2|A| - 3\}.}$$

The polynomial proof is only one step beyond the proof of Cauchy–Davenport from the Combinatorial Nullstellensatz. The restriction $a \neq b$ is encoded by the factor $x - y$.

Proof. Write $m = |A|$. The cases $m \leq 1$ are immediate.

First suppose

$$2m - 3 \geq p.$$

For any $c \in \mathbb{F}_p$, the sets A and $c - A$ satisfy

$$|A \cap (c - A)| \geq 2m - p \geq 2.$$

Thus there are distinct $a, b \in A$ with $a + b = c$. Hence

$$A \dot{+} A = \mathbb{F}_p.$$

Now suppose

$$2m - 3 < p.$$

Assume for contradiction that

$$|A \dot{+} A| \leq 2m - 4.$$

Enlarge the restricted sumset, if necessary, to a set $C \subseteq \mathbb{F}_p$ of size exactly $2m - 4$, and consider

$$P(x, y) = (x - y) \prod_{c \in C} (x + y - c).$$

For every $(a, b) \in A^2$, either $a = b$, in which case the first factor vanishes, or $a \neq b$, in which case $a + b \in C$. Thus

$$P(a, b) = 0 \qquad \text{for every } (a, b) \in A^2.$$

The total degree is $2m - 3$. Its top homogeneous part is

$$(x - y)(x + y)^{2m-4}.$$

The coefficient of $x^{m-1}y^{m-2}$ is

$$\binom{2m-4}{m-2} - \binom{2m-4}{m-1} = \frac{1}{m-1}\binom{2m-4}{m-2}.$$

Since $2m - 4 < p$, neither the numerator nor $m - 1$ vanishes modulo p. Hence this coefficient is nonzero in $\mathbb{F}_p$.

The Combinatorial Nullstellensatz, applied with exponents $m - 1$ and $m - 2$, now says that P cannot vanish on all of $A \times A$, since

$$|A| > m - 1, \qquad |A| > m - 2.$$

This contradiction proves the theorem. □

The restriction to distinct summands is paid for by one linear factor $x - y$, and correspondingly the Cauchy–Davenport bound $2|A| - 1$ loses exactly two.

7.9.3 Iterated Modular Sumsets

Sparse convolution computes one sumset quickly, but repeatedly using it can still be expensive: many intermediate sets may be large even when their sum is no larger. In a cyclic group, this failure of growth reveals a period. We quotient out that period immediately. Processing a balanced merge tree depth first keeps only a logarithmic number of completed subtrees waiting to be merged, which makes the total work easy to charge.

We count operations on words of logarithmic bit length, and use the usual fast algorithm for dense integer convolution. All randomized routines are Las Vegas: their answers are exact and their running-time bounds are in expectation.

Theorem 226 (Iterated modular sumsets)**.** *Let $m, r \geq 1$. Given nonempty sets $A_1, \ldots, A_r \subseteq \mathbb{Z}/m\mathbb{Z}$, put $L = \sum_i |A_i|$ and $K = |A_1 + \cdots + A_r|$. Their sumset can be listed in expected $\widetilde{O}(L + K)$ time.*

An empty input set makes the answer empty. The two-set routine used below is $\textsc{ModAdd}(X, Y, m)$: compute the ordinary integer sumset of the representatives in $[0, m - 1]$, reduce modulo m, and deduplicate. Every modular sum has at most two ordinary lifts, so a near-linear sparse sumset algorithm implements this routine in expected $\widetilde{O}(|X + Y|)$ time. Sets returned by the routines are sorted.

Finding a period from the gaps. For $C \subseteq \mathbb{Z}/m\mathbb{Z}$, a period is a residue h such that $C + h = C$. The periods form a subgroup. If $C \neq \varnothing$ and its smallest positive integer period is d, allowing $d = m$, then $d \mid m$ and $C = (C \bmod d) + \{0, d, \ldots, m - d\}$.

Lemma 227 (Cyclic periods)**.** *The smallest positive period of a sorted nonempty set $C = \{c_0 < \cdots < c_{s-1}\} \subseteq [0, m-1]$ can be found in $O(s)$ time.*

Proof. Form the cyclic gaps $g_i = c_{i+1} - c_i$ for $i < s - 1$, and $g_{s-1} = m + c_0 - c_{s-1}$. A translation preserving C preserves its cyclic order, hence rotates this gap word without changing it. Conversely, if the word consists of repetitions of a block of length $q \mid s$, translation by $d = g_0 + \cdots + g_{q-1}$ preserves C. Thus the shortest repeating block gives the desired period.

The prefix-function computation below finds all possible block lengths. Here π_i is the length of the longest proper prefix of $g_0 \cdots g_i$ that is also a suffix thereof. On a mismatch, the next possible match is a border of the current matched prefix, so replacing j by π_{j-1} skips no candidate. Successful extensions increase j by one, whereas every failed comparison decreases it; there are only $O(s)$ comparisons. Following the same border links from π_{s-1} enumerates every proper border. A border of length b gives a repeating block precisely when $s - b$ divides s. This proves correctness and the linear time bound. □

Algorithm 15 CyclicPeriod(C, m)

Form the cyclic gap word $g_0, \ldots, g_{s-1}$; set $\pi_0 \leftarrow 0$
for $i = 1, \ldots, s - 1$ **do**
    $j \leftarrow \pi_{i-1}$
    while $j > 0$ and $g_i \neq g_j$ **do**
        $j \leftarrow \pi_{j-1}$
    If $g_i = g_j$, set $j \leftarrow j + 1$
    $\pi_i \leftarrow j$
$q \leftarrow s$, $b \leftarrow \pi_{s-1}$
while $b > 0$ **do**
    If $s - b$ divides s, set $q \leftarrow \min\{q, s - b\}$
    $b \leftarrow \pi_{b-1}$
return $g_0 + \cdots + g_{q-1}$

Merge until growth fails, then quotient. Kneser's theorem implies that if $|X + Y| < |X| + |Y| - 1$, the sumset has a nontrivial period. Moreover, every period of a partial sumset is a period of the final sumset: $(X + Y + h) + Z = (X + Y) + Z$. Consequently a period found anywhere in the merge tree can be used to reduce the entire problem.

The recursive helper returns either an ordinary set or a tagged value Period(d). Such a tag is propagated immediately, before processing any further subtree. The helper uses the input family and modulus of the current ModularSum call.

Algorithm 16 Computation of an iterated modular sumset

```
function ModularSum(A_1, ..., A_r, m)
    If some A_i is empty, return ∅
    If m = 1, return {0}
    Sort and deduplicate the sets; pad the family with {0} to a power of two
    Z ← Merge(1, r), using the padded value of r
    If Z is a set, return Z
    Let Z = Period(d); form B_i ← A_i mod d
    B ← ModularSum(B_1, ..., B_r, d)
    return B + {0, d, ..., m − d}, listed in sorted order
function Merge(ℓ, u)
    If ℓ = u, return A_ℓ
    v ← ⌊(ℓ + u)/2⌋
    X ← Merge(ℓ, v); if it is a period tag, return it
    Y ← Merge(v + 1, u); if it is a period tag, return it
    C ← ModAdd(X, Y, m)
    if |C| < |X| + |Y| − 1 then
        return Period(CyclicPeriod(C, m))
    return C
```

Proof of Theorem 226. Padding at most doubles the input size; below, L includes the dummy singleton sets.

Correctness. Every intermediate set is the sum of a subfamily. If no period is reported, the root is the required sumset. Otherwise Kneser and Lemma 227 give a proper divisor $d \mid m$ which is a period of the full answer S. Writing π for reduction modulo d, we have $\pi(S) = \pi(A_1) + \cdots + \pi(A_r)$ and $S = \pi^{-1}(\pi(S))$. The recursive answer and its lift are therefore correct. The modulus decreases by at least a factor of two.

Work before the first period. Write $h = \log_2 r$ after padding. Every partial sumset has size at most K: choose one element from each omitted set and translate the partial sumset into the final one. For every completed merge before the first failure of growth, $|C| - 1 \geq (|X| - 1) + (|Y| - 1)$. Hence, in a completed subtree with root set F, the sum of $|C| - 1$ over any one level is at most $|F| - 1$. The total size of all sets in that subtree is at most $(h + 1)(|F| - 1)$ plus its number of nodes.

Just before the first failed merge, the visited leaves are partitioned into at most $h + 1$ completed subtrees: they are the sets on the depth-first stack, including the two operands of that merge. Each root has size at most K, and together these subtrees have $O(r)$ nodes. The sum of all intermediate sizes computed so far is thus $O(K(h + 1)^2 + r)$; the failed merge adds at most another K. If no merge fails, use the single completed root instead. Sparse convolution and period detection therefore cost $\widetilde{O}(L + K)$ at this modulus.

Quotienting. The total projected input size never exceeds the original L, and there are at most $1 + \log_2 m$ moduli. At each proper quotient the answer size drops by at least a factor of two, since the answer is a union of full fibers. Thus the answer sizes over all recursive calls sum to at most $2K$. Projection, sorting, all merges, and all final lifts together take expected $\widetilde{O}(L + K)$ time. □

For ordinary integer sumsets, every merge already satisfies $|X + Y| - 1 \geq (|X| - 1) + (|Y| - 1)$. The same argument then bounds a balanced merge tree directly, without quotienting. The cyclic case needs the period step precisely because this inequality can fail.

7.9.4 Ruzsa's Triangle Inequality and Covering Lemma

Two elementary operations underlie much of sumset calculus: an intermediate set compares two difference sets, and a maximal disjoint family of translates turns a cardinality bound into a covering.

Theorem 228 (Ruzsa's triangle inequality)**.** *For finite nonempty subsets A, B, C of an abelian group,*

$$\boxed{|A - C|\,|B| \le |A - B|\,|B - C|.}$$

Proof. For each $d \in A - C$, fix $d = a_d - c_d$. The map $(d, b) \mapsto (a_d - b, b - c_d)$ into $(A - B) \times (B - C)$ is injective: adding the output coordinates recovers d, hence the chosen a_d, and then the first coordinate recovers b. □

Taking $-A$ as the intermediate set gives $|A - A| \le |A + A|^2/|A|$, so small doubling also controls subtraction.

Theorem 229 (Ruzsa's covering lemma)**.** *For nonempty A, B, if $|A + B| \le K|B|$, some $T \subseteq A$ satisfies $|T| \le K$ and $A \subseteq T + (B - B)$.*

Proof. Choose $T \subseteq A$ maximal with the translates $t + B$ pairwise disjoint. They lie in $A + B$, so $|T||B| \le |A + B|$. For each $a \in A$, maximality gives $a + b = t + b'$ for some $t \in T$ and $b, b' \in B$, which places a in $t + (B - B)$. □

7.9.5 Plünnecke–Ruzsa Inequalities

From $|A + B| \le K|A|$, we would like each further addition of B to cost at most another factor K. The hypothesis says nothing directly about $A + B$, so it cannot simply be iterated. Instead, choose a subset of A with minimum relative growth; minimality then controls every part of that subset.

Lemma 230 (Minimal growth)**.** *Let X, B be nonempty. Suppose $\kappa = |X + B|/|X| \le |Y + B|/|Y|$ for every nonempty $Y \subseteq X$. Then $|X + B + C| \le \kappa|X + C|$ for every finite set C.*

Proof. List $C = \{c_1, \dots, c_m\}$, and let $X_i \subseteq X$ consist of those x for which $x + c_i$ first appears in translate i. Thus the $X_i + c_i$ partition $X + C$, and $|X + C| = \sum_i |X_i|$.

Now add the translates $X + B + c_i$ in the same order. Every point of $(X \setminus X_i) + c_i$ was already covered by earlier translates of X, so every point of $(X \setminus X_i) + B + c_i$ was already covered by earlier translates of $X + B$. Hence translate i contributes at most

$$|X + B| - |(X \setminus X_i) + B| \le \kappa|X| - \kappa|X \setminus X_i| = \kappa|X_i|$$

new points. The inequality also holds when $X \setminus X_i$ is empty. Sum over i. □

The overlaps are useful: minimality ensures that an already covered portion of X accounts for at least its proportional share of $X + B$.

The signs in Ruzsa's triangle inequality can in fact be chosen independently. Reflecting sets in the original inequality gives four of the eight cases; the other four follow from the all-plus case, using the preceding lemma.

Theorem 231 (Ruzsa's triangle inequalities, all sign choices)**.** *For finite nonempty subsets A, B, C of an abelian group and $\varepsilon, \eta, \theta \in \{-1, 1\}$,*

$$|A|\,|B + \varepsilon C| \le |A + \eta B|\,|A + \theta C|.$$

Proof. First prove the all-plus case. Choose a nonempty $X \subseteq A$ minimizing $\kappa = |X + B|/|X|$, so $\kappa \le |A + B|/|A|$. Since $X \neq \varnothing$, the set $X + B + C$ contains a translate of $B + C$. By Lemma 230 and $X \subseteq A$,

$$|B + C| \le |X + B + C| \le \kappa|X + C| \le \frac{|A + B|\,|A + C|}{|A|}.$$

The original triangle inequality, with intermediate set $-A$, also gives $|A|\,|B - C| \le |A + B|\,|A + C|$. Thus either sign is allowed on the left when both signs on the right are plus.

Now replace B, C by $\eta B, \theta C$. The right side becomes $|A + \eta B|\,|A + \theta C|$, while the two possible left-hand sumsets have sizes $|\eta B \pm \theta C| = |B \pm \eta\theta C|$. These give both choices of ε, independently of η and θ, covering all eight cases. □

Theorem 232 (Plünnecke–Ruzsa)**.** *For nonempty A, B, if $|A+B| \le K|A|$, there is a nonempty $X \subseteq A$ with $|X+hB| \le K^h|X|$ for every integer $h \ge 0$. Moreover,*

$$\boxed{|rB - sB| \le K^{r+s}|A| \qquad (r, s \in \mathbb{Z}_{\ge 0}).}$$

In particular, $|A+A| \le K|A|$ implies $|rA - sA| \le K^{r+s}|A|$.

Proof. Choose $X \subseteq A$ minimizing $\kappa = |X+B|/|X|$, so $\kappa \le K$. Iterating the minimal-growth lemma with $C = (h-1)B$ gives $|X+hB| \le \kappa^h|X|$. Ruzsa's triangle inequality, with intermediate set $-X$, then gives

$$|rB - sB|\,|X| \le |rB + X|\,|X + sB| \le \kappa^{r+s}|X|^2.$$

Divide by $|X|$ and use $|X| \le |A|$. □

Corollary 233 (Plünnecke–Ruzsa with either sign)**.** *Let A, B be finite and nonempty, and suppose $|A + \delta B| \le K|A|$ for some $\delta \in \{-1, 1\}$. Then, for either $\varepsilon \in \{-1, 1\}$ and all integers $r, s \ge 0$, we have $|rB + \varepsilon sB| \le K^{r+s}|A|$.*

Proof. This follows from the original theorem: $rB + sB = (r+s)B$, so take its parameters to be $(r+s, 0)$ and reflect B if $\delta = -1$. □

A finite symmetric set S containing zero is an *L-approximate subgroup* if $S + S \subseteq T + S$ for some $|T| \le L$. If $|A+A| \le K|A|$, then $S = A - A$ has $|S| \le K^2|A|$ and $|S+S+A| = |3A - 2A| \le K^5|A|$. Ruzsa's covering lemma, applied to $S+S$ and A, yields $S + S \subseteq T + S$ with $|T| \le K^5$. Thus $A - A$ is a K^5-approximate subgroup.

7.9.6 Multiway Sumset Inequalities

When an algorithm merges many sumsets, we want both to charge the input sizes to the output and to control the full sum from smaller partial sums. The following inequalities serve these two purposes.

Lemma 234 (Growth of a multiway sumset)**.** *For finite nonempty $A_1, \dots, A_k \subseteq \mathbb{Z}^d$,*

$$\sum_{i=1}^{k} (|A_i| - 1) \le |A_1 + \cdots + A_k| - 1.$$

Proof. Order each A_i lexicographically. Start with the sum of the least elements, advance the first summand through its list, then the second, and so on. Each advance strictly increases the total lexicographically, producing $1 + \sum_i (|A_i| - 1)$ distinct sums. □

Theorem 235 (Sumset submultiplicativity)**.** *For finite nonempty subsets $A_1, \dots, A_k$ of a commutative semigroup, where $k \ge 2$, put $B_i = \sum_{j \ne i} A_j$. Then*

$$\boxed{|A_1 + \cdots + A_k|^{k-1} \le \prod_{i=1}^{k} |B_i|.}$$

Proof. Order each A_i arbitrarily, and let V contain the lexicographically first representing tuple of each element of $A_1 + \cdots + A_k$. Thus $|V| = |A_1 + \cdots + A_k|$.

For each i, the partial-sum map on $\pi_{-i}V$ is injective. Indeed, if two distinct projected tuples had the same sum, replace the larger projection in its canonical full tuple by the smaller one, keeping coordinate i fixed. This preserves the full sum but decreases its representing tuple, a contradiction. Consequently, $|\pi_{-i}V| \le |B_i|$.

Each coordinate belongs to exactly $k-1$ of these projections. Shearer's lemma, in its projection form, therefore gives

$$|A_1 + \cdots + A_k|^{k-1} = |V|^{k-1} \le \prod_{i=1}^{k} |\pi_{-i}V| \le \prod_{i=1}^{k} |B_i|.$$

□

Corollary 236 (Controlling a multiway merge)**.** *If every leave-one-out sum has size at most M, then $|A_1 + \cdots + A_k| \le M^{k/(k-1)}$. If also $k - 1 \ge \log_2 M$, this is at most $2M$.*

Proof. Apply sumset submultiplicativity. For the second claim, write $M^{k/(k-1)} = M \cdot M^{1/(k-1)} \le 2M$. □

7.9.7 Short Sums and Long Arithmetic Progressions

For integer sums, a useful conclusion is often that attainable sums *contain* a long progression. The basic construction first reaches every residue modulo an available number, then adds copies of that number to fill the rest of each residue class. With two generators, the residue graph gives an exact answer.

Theorem 237 (Two-generator Frobenius theorem)**.** *For coprime integers $a, b \ge 2$, the largest integer not of the form $ax + by$, with $x, y \in \mathbb{Z}_{\ge 0}$, is $ab - a - b$. Thus every integer at least $(a-1)(b-1)$ is representable, and this threshold is sharp.*

Proof. Make a directed graph on $\mathbb{Z}/a\mathbb{Z}$, with an edge $r \to r + b$ of weight b from each vertex. Since $\gcd(a, b) = 1$, these edges form one directed cycle, visiting $0, b, 2b, \ldots, (a-1)b \pmod a$ in that order. Adding an a-coin is a loop of weight a at the current vertex.

For $0 \le j < a$, any walk from zero to residue jb uses $j + ka$ of the b-edges for some $k \ge 0$. Its weight is therefore at least jb, and the direct j-edge path attains that weight. The a-loops then attain exactly $jb, jb + a, jb + 2a, \ldots$ in this residue class. Its largest unattainable integer is consequently $jb - a$. Taking the maximum over the cycle gives $(a-1)b - a = ab - a - b$. □

The same graph describes the exact threshold for any finite $A \subseteq \mathbb{Z}_{>0}$ with $\gcd(A) = 1$. Put $M = \max A$, give each edge $r \to r + a \pmod M$ weight a, and let w_r be the least weight of a walk from zero to r. Every residue is reachable because A generates $\mathbb{Z}/M\mathbb{Z}$. Minimality and the available M-loops show that the attainable integers in residue r are exactly $w_r + qM$, $q \ge 0$. Thus the sharp threshold is $1 + \max_r w_r - M$. To bound it without finding the shortest weights, Kneser's theorem will bound the number of edges needed to reach each residue; every edge weighs at most M.

Lemma 238 (A diameter bound for a cyclic sumset)**.** *Let $C \subseteq \mathbb{Z}/m\mathbb{Z}$ contain zero and generate the group. If $b = |C|$ and $h = \lceil 2m/b \rceil$, then $hC = \mathbb{Z}/m\mathbb{Z}$.*

Proof. The case $m = 1$ is immediate. If $(j+1)C$ is proper, its stabilizer H is proper. Because $0 \in C$ and C generates the group, C meets at least two H-cosets. Thus $|C + H| \ge \max\{b, 2|H|\}$, and Kneser gives

$$|(j+1)C| \ge |jC| + |C + H| - |H| \ge |jC| + \max\{b - |H|, |H|\} \ge |jC| + b/2.$$

Since zero belongs to C, the sets jC are nested. If hC were proper, each of the first h steps would gain at least $b/2$ elements, giving $|hC| \ge 1 + hb/2 > m$, a contradiction. □

Theorem 239 (A progression of short repeated sums)**.** *Let $\varnothing \ne A \subseteq [M]$, put $b = |A|$, and let $d = \gcd(A)$. Then $\bigcup_{1 \le j \le \lfloor 8M/b \rfloor} jA$ contains M consecutive positive multiples of d.*

Proof. Divide by d, obtaining $D = A/d$, and put $m = \max D$. The b residues of D modulo m are distinct, contain zero, and generate $\mathbb{Z}/m\mathbb{Z}$. By the diameter bound, every residue has a representative s_r which is a sum of at most $h = \lceil 2m/b \rceil$ elements of D, so $0 \le s_r \le hm$.

For any integer $t \ge hm$, choose the representative of its residue. Then $t = s_r + qm$ for an integer $q \ge 0$; append q copies of $m \in D$. In particular, every integer from hm to $hm + M - 1$ uses at most

$$h + q \le 2h + \left\lceil \frac{M}{m} \right\rceil \le \frac{4m}{b} + \frac{M}{m} + 3 \le \frac{5M}{b} + 3 \le \frac{8M}{b}$$

summands, using $b \le m \le M$. The number of summands is an integer, so it is at most $\lfloor 8M/b \rfloor$. Multiply this positive interval by d. □

Corollary 240 (Unbounded Subset Sum threshold)**.** *If $A \subseteq \mathbb{Z}_{>0}$, $M = \max A$, $b = |A|$, and $\gcd(A) = 1$, every integer $t \geq M\lceil 2M/b \rceil$ is a nonnegative integer combination of elements of A. In particular, $2M^2/b + M$ is a sufficient threshold.*

Proof. The preceding residue representatives have value at most $M\lceil 2M/b \rceil$. Append copies of M to the representative of t mod M. □

The argument also recovers a representation. Breadth-first search in the directed graph with edges $r \to r + a \pmod M$, for $a \in A$, finds a path with the fewest edges to each residue. Its weight is at most M times its length, so the diameter bound makes every representative light enough to pad with copies of M.

Algorithm 17 Unbounded Subset Sum above the elementary threshold

Input: $A \subseteq \mathbb{Z}_{>0}$, $\gcd(A) = 1$, $M = \max A$, $t \geq M\lceil 2M/|A| \rceil$
Output: Nonnegative integer multiplicities $(c_a)_{a \in A}$ with $\sum_a c_a a = t$

```
Initialize seen[0 ... M − 1] ← false
seen[0] ← true; initialize a queue with 0
while the queue is nonempty do
    Remove r from the front
    for each a ∈ A do
        u ← (r + a) mod M
        if seen[u] is false then
            seen[u] ← true
            parent[u] ← r, item[u] ← a
            Append u to the queue
Initialize all c_a ← 0; set r ← t mod M, s ← 0
while r ≠ 0 do
    a ← item[r]
    c_a ← c_a + 1, s ← s + a
    r ← parent[r]
c_M ← c_M + (t − s)/M
return (c_a)_{a∈A}
```

Preprocessing uses $O(M|A|)$ arithmetic operations and $O(M)$ space. Reconstruction follows at most $\lceil 2M/|A| \rceil$ parent edges, in addition to initializing the $|A|$ output counts. The counts provide a compact answer even when t is large.

7.9.8 A Subset-Sum Intersection Bound

In a Knapsack exchange argument, bounding the number of changed items does not by itself bound their total weight in another part. The next theorem supplies this second bound: a multiset with sufficiently many occurrences and bounded multiplicity shares a positive subset sum with every multiset of sufficiently large total weight.

Theorem 241 (Subset-sum intersection)**.** *Let X, Y be multisets of integers in $[W]$, with $r = |X| \geq 1$, and suppose every value occurs at most μ times in X. If*

$$r^2 \geq 256\mu W\lceil \log(2r) \rceil, \qquad \Sigma(Y) \geq \frac{128\mu W^2}{r},$$

then $\mathcal{S}(X) \cap \mathcal{S}(Y)$ contains a positive integer.

The useful contrapositive is that, under the density condition on X, having no common positive subset sum forces $\Sigma(Y) < 128\mu W^2/r$. Size and bounded multiplicity produce a progression in $\mathcal{S}(X)$, and the total weight of Y forces it to hit that progression.

Why a progression suffices. Suppose $\mathcal{S}(X)$ contains consecutive positive multiples of d. Group occurrences of Y into small disjoint blocks whose weights are divisible by d. Their running sums stay aligned with the progression; if the progression is long enough, the first running sum to reach its beginning cannot jump past its end.

Lemma 242 (Hitting a progression)**.** *Let Y be a multiset in $[W]$, and let a, d be positive integers with $d \mid a$. If $\Sigma(Y) \geq a + (d-1)W$, then $\mathcal{S}(Y)$ intersects $\{a, a+d, \ldots, a+(W-1)d\}$.*

Proof. Order any d remaining occurrences. Among their $d+1$ prefix sums, two agree modulo d; the intervening nonempty block has weight divisible by d and at most dW. Repeatedly extract such a block until fewer than d occurrences remain.

The resulting disjoint block weights $q_1, \ldots, q_k$ are positive multiples of d, each at most dW, and their total is at least $\Sigma(Y) - (d-1)W \geq a$. Take the shortest prefix whose weight q is at least a. Its predecessor is below a, and its last block weighs at most dW, so $a \leq q < a + dW$. As $d \mid q$, this is one of the desired progression points; disjointness makes it a subset sum of Y. □

From repeated sums to subset sums. The short-sums theorem allows repetition, whereas a subset sum may use each occurrence only once. To bridge the gap, use a set of *popular pair sums*: each available summand has many disjoint representations by two occurrences of X. A short representation then has enough alternatives to avoid every occurrence already used.

Proof of Theorem 241. Regard occurrences as distinct objects, and let m_u be the multiplicity of value u in X. For each integer z, let t_z be the maximum number of pairwise disjoint occurrence pairs with sum z. Thus $t_z \leq \lfloor r/2 \rfloor$. Disjointness is required within each family for a fixed sum; families for different sums may overlap.

Count the available pair representations. For distinct values u, v, we can form $\min\{m_u, m_v\} \geq m_u m_v/\mu$ disjoint pairs. For a single value u, we can form $\lfloor m_u/2 \rfloor \geq m_u(m_u-1)/(2\mu)$ pairs. For a fixed sum z, complementary value classes do not overlap: value u can pair only with $z-u$. We can therefore take all these pairs simultaneously for that sum. Summing over sums gives

$$\sum_z t_z \geq \frac{1}{\mu} \sum_{u<v} m_u m_v + \frac{1}{2\mu} \sum_u m_u(m_u-1) = \frac{r(r-1)}{2\mu} \geq \frac{r^2}{4\mu}. \tag{58}$$

Here $r \geq 2$ follows from the density hypothesis. Bounded multiplicity has converted the $\binom{r}{2}$ occurrence pairs into at least $\binom{r}{2}/\mu$ disjoint representations.

Choose a threshold that provides both variety and alternatives. For $1 \leq v \leq \lfloor r/2 \rfloor$, put $B_v = \{z : t_z \geq v\}$ and $b_v = |B_v|$. We want b_v large, so the short-sums theorem uses few summands, and vb_v large, so those few choices cannot exhaust the v available pairs for any sum.

Set $L_r = \lceil \log(2r) \rceil$. Some threshold satisfies

$$b := b_v \geq \frac{r}{4\mu} + \frac{r^2}{8\mu L_r v}. \tag{59}$$

Indeed, counting each z once for every $v \leq t_z$ gives $\sum_v b_v = \sum_z t_z$. If (59) failed for all v, then

$$\sum_z t_z < \frac{r}{4\mu} \left\lfloor \frac{r}{2} \right\rfloor + \frac{r^2}{8\mu L_r} \sum_{v=1}^{\lfloor r/2 \rfloor} \frac{1}{v} \leq \frac{r^2}{8\mu} + \frac{r^2}{8\mu} = \frac{r^2}{4\mu},$$

contradicting (58). We used $\sum_{v \leq r/2} 1/v \leq 1 + \log(r/2) \leq L_r$. Fix this v. The two useful consequences, together with density, are $b \geq r/(4\mu)$ and $vb \geq r^2/(8\mu L_r) \geq 32W$.

Realize a long progression without reusing occurrences. Apply Theorem 239 to $B_v \subseteq [2W]$. It gives $2W$ consecutive positive multiples of $d = \gcd(B_v)$, forming a progression P, each represented using at most $H = \lfloor 16W/b \rfloor$ elements of B_v. Our threshold gives $H \le 64\mu W/r$ and $2H \le v$.

Fix a representation $z_1 + \cdots + z_h$ of a point of P, where $h \le H$. For each z_i, fix v disjoint occurrence pairs of sum z_i, and choose one pair from each family in order. Before choice i, exactly $2(i-1)$ occurrences have been used. Because the candidates within this family are disjoint, each used occurrence forbids at most one candidate. At most $2(i-1) < 2H \le v$ candidates are forbidden, so an unused pair remains. This works even if some z_i's coincide. Consequently $P \subseteq \mathcal{S}(X)$.

Check that the progression starts early enough. Put $a = \min P$. Every point uses at most H pair sums, each at most $2W$, so $\max P \le 2WH$. Also, B_v contains b distinct positive multiples of d in $[2W]$, whence $bd \le 2W$. Therefore

$$a \le \max P \le \frac{128\mu W^2}{r}, \qquad d \le \frac{2W}{b} \le \frac{8\mu W}{r}. \tag{60}$$

Since P has $2W$ terms, its endpoint is $\max P = a + (2W-1)d \ge a + Wd$. The weight hypothesis therefore gives $\Sigma(Y) \ge \max P \ge a + (d-1)W$. The hitting lemma now finds a positive subset sum of Y among the first W points of P, all of which belong to $\mathcal{S}(X)$. □

7.9.9 Bounded Knapsack

A bounded-Knapsack record (w_i, p_i, u_i) specifies u_i identical items of positive integer weight w_i and rational profit p_i; multiplicities are encoded in binary. Given capacity C, we maximize total profit. Let N be the number of records and $W = \max_i w_i$.

Theorem 243 (Bounded Knapsack)**.** *An optimal selection, including its multiplicities, can be found deterministically in*

$$O\big(N + W^2 \log^3(W+2)\big)$$

arithmetic and comparison operations. When the profits are positive integers at most P, the same bound holds with P in place of W.

A density-greedy selection is close to an optimum in the number of changed items, but not necessarily in their total weight: $O(W)$ changes can weigh $\Theta(W^2)$. A DP of this width, updated once for each of the W item weights, would be cubic. Instead, we partition the changes into parts. A part containing many distinct weights will have a small *local* change bound, certified by the subset-sum intersection theorem. Processing the parts from smaller to larger bounds then keeps every DP update inexpensive.

We prove the more useful assertion that the *entire exact-weight profile* near any density prefix can be computed within this time. Write $\mathrm{wt}(x)$ and $\mathrm{pr}(x)$ for a selection's weight and profit.

Lemma 244 (Exact weights near a density prefix)**.** *For any prefix g in nonincreasing profit-density order, all values*

$$H(z) = \max\{\mathrm{pr}(x) : \mathrm{wt}(x) = z\}, \qquad |z - \mathrm{wt}(g)| < W,$$

can be computed in $O(N + W^2 \log^3(W+2))$ operations. A maximizing selection for any one finite entry can also be recovered within this bound. Unattainable weights have value $-\infty$, and profits may have either sign.

Exchange obstruction and kernelization. Fix a total order of individual occurrences by nonincreasing density, breaking ties arbitrarily, and choose for each attainable z the lexicographically greatest optimal indicator vector x_z. If A consists of occurrences *unselected* by x_z, all preceding a nonempty set B of occurrences *selected* by x_z, then

$$\mathcal{S}(A) \cap \mathcal{S}(B) = \{0\}. \tag{61}$$

Indeed, exchanging equal-weight nonempty subsets of B and A would not decrease profit, because all densities in A are at least those in B; in case of equality it would improve the tie-break. The same rule ensures that x_z chooses a profit prefix within each weight class.

Put $E^- = g \setminus x_z$ and $E^+ = x_z \setminus g$. Every removal precedes every addition. Moreover,

$$\left|\mathrm{wt}(E^+) - \mathrm{wt}(E^-)\right| = |z - \mathrm{wt}(g)| < W. \tag{62}$$

By (61), the signed sequence $\{-w_i : i \in E^-\} \uplus \{w_i : i \in E^+\}$ has no nonempty zero-sum subsequence. This forces few changes.

Lemma 245 (Zero-sum-free walk)**.** *If nonzero integers $a_1, \ldots, a_k \in [-W, W]$ have total sum of absolute value less than W and no nonempty submultiset sums to zero, then $k \le 2W - 1$.*

Proof. Negate the terms if necessary so their total lies in $[0, W)$. Starting at zero, take a positive remaining term when the current sum is below W, and a negative one otherwise. All partial sums stay in $[0, 2W - 1]$. Once the required sign is exhausted, the remaining terms move monotonically toward the final sum, still in this range. No two partial sums coincide, since the intervening terms would sum to zero. Hence $k + 1 \le 2W$. □

Thus $|g \triangle x_z| \le 2W - 1$ for every target. As g and x_z select prefixes within each weight class, all changes occur among the last $2W$ copies of each weight selected by g and the first $2W$ copies outside it. Retain these at most $4W^2$ occurrences as a kernel K; all other choices remain fixed as in g, simultaneously for all targets. There is no need to expand other binary multiplicities: a density threshold and the relevant endpoints can be found by weighted selection in $O(N + W)$ operations. Expanding and sorting K costs $O(W^2 \log(W + 2))$.

If $K = \varnothing$, the lemma is immediate. Otherwise write $n = |K| \le 4W^2$ and $L = \lceil \log(2n) \rceil = O(\log(W + 2))$.

What the partition must guarantee. We will partition K into parts I, with d_I distinct weights in part I and a bound Δ_I such that

$$\mathrm{wt}\big(I \cap (g \triangle x_z)\big) \le \Delta_I \qquad \text{for every } z. \tag{63}$$

We bound the *total changed weight*, not the signed change, so (63) controls every partial correction regardless of the order in which a part is processed.

A weight class has a concave profit profile and can be incorporated into a DP in time linear in its current width. Thus part I costs $O(d_I R)$ if the DP radius is R. We will arrange that

$$\sum_I d_I \Delta_I = O(W^2 L^2), \qquad \sum_{I:\, \Delta_I \le D} \Delta_I = O(LD) \quad (D \ge 1). \tag{64}$$

Then processing parts in increasing order of Δ_I requires radius only $O(L\Delta_I)$ at part I, and total DP work is $O(W^2 L^3)$.

Constructing the parts. Work independently on the two sides of the boundary of g. On one side, let U be the unassigned occurrences. Visit powers of two m in decreasing order, beginning with the smallest power at least n. At level m, repeatedly take

$$J = \{i \in U : m/2 < \mathrm{mult}_U(w_i) \le m\}, \qquad s = |J|,$$

where multiplicities are recomputed within the remaining occurrences on this side. Each represented weight occurs more than $m/2$ times, so J has fewer than $2s/m$ distinct weights.

Fix $c_0 = 2^{14}$. If $s^2 \le c_0 mWL$, assign all of J to one *remainder part*, with bound $\Delta_J = Ws$. Otherwise take the outer $\lceil s/2 \rceil$ occurrences of J in density order as a part I, leaving the inner $\lfloor s/2 \rfloor$ occurrences B unassigned as its *buffer*; outer means farther from the boundary of g. Give I the bound

$$\Delta_I = \left\lceil \frac{c_0 m W^2}{s} \right\rceil. \tag{65}$$

Remove the new part from U and repeat. The buffer is not a separate part: its occurrences will be assigned in later steps. Assigning a part never changes the fixed solutions g or x_z.

The tradeoff is now explicit. A remainder is light because it is small. Otherwise the part has fewer than $2s/m$ distinct weights, and the nearby buffer has enough occurrences of bounded multiplicity to force its changed weight down to $O(mW^2/s)$.

Lemma 246 (The buffer bounds changes)**.** *Every part satisfies* (63)*, for all targets simultaneously.*

Proof. A remainder has total weight at most Ws. Consider a buffered part I and any $x = x_z$. Let X be the larger of $B \cap x$ and $B \setminus x$, and write $r = |X|$. Since B contains $\lfloor s/2 \rfloor$ occurrences, $r \geq s/8$; every weight occurs at most m times in X. Moreover,

$$r^2 \geq s^2/64 > 256mWL \geq 256mW\lceil \log(2r) \rceil.$$

The subset-sum intersection theorem therefore implies that, whenever $\mathcal{S}(X) \cap \mathcal{S}(Y) = \{0\}$,

$$\mathrm{wt}(Y) < \frac{128mW^2}{r} \leq \frac{1024mW^2}{s} =: \tau. \tag{66}$$

Let $D_I = I \cap (g \triangle x)$, and let E_{opp} denote all changes on the side of g opposite to I. Every occurrence of I lies farther from the greedy boundary than every occurrence of B, and both lie on the same side. Suppose first that $I \subseteq g$. The changed occurrences D_I precede the buffer X, which in turn precedes the additions $E_{\mathrm{opp}} = x \setminus g$. If $X = B \cap x$, its occurrences are selected, so the exchange obstruction applies to (D_I, X). If $X = B \setminus x$, its occurrences are unselected, and the obstruction applies to (X, E_{opp}).

If $I \cap g = \varnothing$, the order is instead $E_{\mathrm{opp}} = g \setminus x$, then X, then D_I. The exchange obstruction applies to (E_{opp}, X) when $X = B \cap x$, and to (X, D_I) when $X = B \setminus x$. In each case, the first set consists of occurrences unselected by x and the second of later occurrences selected by x. Thus (61) and (66) imply

$$\mathrm{wt}(D_I) < \tau \quad \text{or} \quad \mathrm{wt}(E_{\mathrm{opp}}) < \tau.$$

In the second case, the balance (62) implies $\mathrm{wt}(D_I) \leq \mathrm{wt}(E_{\mathrm{opp}}) + W$. Thus in both cases $\mathrm{wt}(D_I) < \tau + W$. Finally, $s \leq mW$ (there are at most W item weights), so

$$\tau + W \leq \frac{1025mW^2}{s} \leq \Delta_I.$$

The construction is independent of x_z, proving the claim. □

Why the parts are inexpensive. On entering level m, all remaining multiplicities are at most m, because larger levels have been exhausted. A weight not in J has multiplicity at most $m/2$, and cannot enter J as occurrences are removed. Therefore the next working set has at most half as many occurrences. There are $O(L)$ buffered parts and at most one remainder per level, and $O(L)$ levels per side.

For a buffered part, $d_I < 2s/m$ and $\Delta_I = \lceil c_0 mW^2/s \rceil$, so $d_I \Delta_I = O(W^2)$. For a remainder, $s^2 \leq c_0 mWL$ gives $d_I \Delta_I < 2Ws^2/m = O(W^2 L)$. Summing over the levels establishes the first inequality in (64).

For the second, within any one level the successive buffered working-set sizes at least halve. Their unrounded bounds $c_0 mW^2/s$ therefore at least double, so only $O(1)$ buffered parts lie in any fixed dyadic range of Δ_I; the single remainder contributes at most one more. Across $O(L)$ levels and both sides there are $O(L)$ parts per dyadic range. Summing the geometric sequence of ranges up to D gives

$$\sum_{I:\, \Delta_I \leq D} \Delta_I = O(LD). \tag{67}$$

Scanning the kernel once for each of its $O(L^2)$ parts constructs the partition in $O(nL^2)$ operations.

Concave updates and the dynamic program. Process parts in nondecreasing order of Δ_I. During part I, keep signed weight changes only in $[-R_I, R_I]$, where

$$R_I = \sum_{J \text{ processed through } I} \Delta_J = O(L\Delta_I).$$

Indeed, the absolute partial correction of every x_z is at most the total weight it changes in the parts encountered so far, even when the current part is only partially processed.

Group the occurrences in I by weight. For a group G of a occurrences, let $p_1 \geq \cdots \geq p_a$ be their profits and $P_G(k) = \sum_{j=1}^{k} p_j$, with $P_G(0) = 0$. Since I lies entirely on one side of g, the number r_G initially selected is either 0 or a. Changing this number by q gives the best profit change

$$B_G(q) = P_G(r_G + q) - P_G(r_G), \qquad -r_G \leq q \leq a - r_G.$$

Its successive differences are the sorted profits, so B_G is concave. This concavity is the reason that all copies of a weight can be processed in one update rather than separately.

Let $F(t)$ be the maximum profit change relative to g at signed weight change t, using the groups already processed. Initially, $F(0) = 0$ and all other values are $-\infty$. For a group G of items of weight w, let $B_G(q)$ be the best profit change from choosing q items of this group. The update is

$$F_{\text{new}}(t) = \max_q \{F(t - wq) + B_G(q)\}, \qquad |t| \leq R_I. \tag{68}$$

Since the group changes the weight by a multiple of w, different residue classes modulo w can be processed independently. Fix a residue r, and write the output and input weights as $t = r + wk$ and $j = r + w\ell$. The update becomes

$$F_{\text{new}}(r + wk) = \max_\ell \{F(r + w\ell) + B_G(k - \ell)\},$$

which is max-plus convolution with the concave sequence B_G.

To handle its finite domain $[a, b]$, extend B_G to all integers by the lexicographically ordered pairs

$$\widetilde{B}_G(q) = \big(-\operatorname{dist}(q, [a, b]),\ B_G(\operatorname{clip}_{[a,b]} q)\big).$$

This extension is concave: its successive differences are $(1, 0)$ before the interval, $(0, B_G(q+1) - B_G(q))$ inside, and $(-1, 0)$ afterward, which are nonincreasing in lexicographic order.

For the fixed residue r, form the matrix

$$K^{(r)}_{k,\ell} = \big(-\operatorname{dist}(k - \ell, [a, b]),\ F(r + w\ell) + B_G(\operatorname{clip}_{[a,b]}(k - \ell))\big),$$

using only columns with $F(r + w\ell) > -\infty$ and rows corresponding to retained output weights. Concavity makes this matrix anti-Monge, so SMAWK finds every row maximum in time linear in its number of rows and columns. The first coordinate is zero precisely for feasible transitions; if no such transition exists, the output value is $-\infty$.

Each input and output state belongs to exactly one residue class. Thus the total number of rows and columns across all matrices is $O(R_I)$, and so is the update time. We process only residue classes containing retained states.

Consequently the total DP work is

$$O\left(\sum_I d_I R_I\right) = O\left(L \sum_I d_I \Delta_I\right) = O(W^2 L^3).$$

Every partial correction of x_z remains in the retained range by (63); its choices within each group are among those considered by (68). Induction therefore preserves a DP value at least as large as its profit change. Conversely every finite state represents a legitimate selection. Hence, for each requested z,

$$H(z) = \operatorname{pr}(g) + F(z - \operatorname{wt}(g)).$$

Store maximizing predecessors and backtrack from any one desired entry, restoring the fixed choices outside K. This costs at most the number of group updates. Together with preprocessing and partitioning, it fits the stated bound and proves Lemma 244.

From exact weights to Knapsack.

Proof of Theorem 243. Discard nonpositive-profit items. If all remaining items fit, take them all. Otherwise let g be the longest feasible density prefix. Its unused capacity is less than W, since the next occurrence does not fit. Every optimum also leaves less than W unused capacity: otherwise it could include another positive-profit occurrence. Thus their weights differ by less than W, and the exact-weight profile supplies the optimum.

For positive integer profits bounded by P, the fractional Knapsack bound is strictly less than $\mathrm{pr}(g) + P$. Hence the integer optimum profit belongs to $\{\mathrm{pr}(g), \ldots, \mathrm{pr}(g) + P - 1\}$. Exchange weights and profits, replacing (w_i, p_i) by $(p_i, -w_i)$. Their density order is unchanged, since $-w_i/p_i$ increases with p_i/w_i. Apply Lemma 244 with maximum weight P to compute the minimum original weight attaining each candidate profit; return the greatest candidate whose minimum weight is at most C. The same reconstruction and running-time bounds apply. □

7.9.10 Witnesses for Long Progressions in Repeated Sumsets

An existential progression is insufficient when an algorithm must produce the summands of a requested term. The construction below first finds a progression with a small step d, then fills its missing residue classes using a table modulo d. This table is cheap either in preprocessing or in the query: if $d > |A|$, then $d = O(k)$, so it can be deferred to a query whose answer already permits $O(k)$ summands.

Theorem 247 (Constructive repeated-sum progression)**.** *Let $A \subseteq \{0, \ldots, M\}$, where $M \geq 1$, $0 \in A$, $\gcd(A) = 1$, and $n = |A| \geq 2$. Put $k = \lceil (M+1)/n \rceil$. In expected $\widetilde{O}(n)$ preprocessing time, one can find an integer T and a data structure certifying $\{T, T+1, \ldots, T+M\} \subseteq 328kA$. For any requested term, it returns an explicit list of $328k$ summands in expected $\widetilde{O}(k)$ time. Preprocessing space is $\widetilde{O}(n)$, and query workspace is $\widetilde{O}(n+k)$.*

The constants merely leave room between the estimates. The query bound is for an explicit list of summands. Zeros can instead be recorded as one multiplicity. Under the theorem's normalization, zero padding places the same interval in every hA with $h \geq 328k$. For a general nonsingleton integer set, subtract its minimum and divide by the gcd of the differences; a representation with a fixed number of terms can then be translated back.

A greedy representation succeeds on most targets. For $B \subseteq [0, D]$, define its prefix density up to D by $\rho_D(B) = \min_{1 \leq x \leq D} |B \cap [1, x]|/x$. Given a target, greedily subtract the largest available element not exceeding the remainder. Greedy need not always finish quickly, but positive prefix density makes it finish quickly on most targets in every prefix. Splitting a target randomly into two such easy targets then gives a Las Vegas witness.

Lemma 248 (Greedy splitting)**.** *Suppose $\{0, 1\} \subseteq B \subseteq [0, D]$ and $\rho_D(B) \geq \rho > 0$. For an integer $h \geq 2/\rho$, every $t \in [0, D]$ can be represented by $2h$ elements of B in expected $O(h \log(2|B|))$ time after sorting B.*

Proof. Let G_j be the targets in $[0, D]$ that greedy represents in at most j steps; zero belongs to every G_j. Fix $x \leq D$, list $B \cap [1, x] = \{b_1 < \cdots < b_q\}$, and put $b_{q+1} = x + 1$. Since $b_1 = 1$, the intervals $[b_i, b_{i+1} - 1]$ partition $[1, x]$. In the interval beginning at b_i, greedy first takes b_i, and succeeds precisely when the remaining offset belongs to G_j. Writing $\gamma_j = \rho_D(G_j)$, this gives at least $q + \gamma_j(x - q)$ successful targets. Consequently $\gamma_{j+1} \geq \rho + (1 - \rho)\gamma_j$, and induction gives $\gamma_h \geq 1 - (1 - \rho)^h \geq 1 - e^{-2} > 3/4$.

Choose u uniformly in $\{0, \ldots, t\}$. Each of the tests $u \in G_h$ and $t - u \in G_h$ fails on at most $t/4$ choices, so both succeed with probability at least $1/2$. Run greedy on both parts and retry if either needs more than h steps. Each trial costs $O(h \log(2|B|))$, and a successful trial returns the desired representation, padded with zeros. □

Algorithm 18 Greedy witnesses and random splitting

```
function GREEDY(B, t, h)
    Q ← [ ]
    for j = 1, ..., h do
        Find b = max(B ∩ [0, t]) by binary search
        Append b to Q; set t ← t − b
    If t = 0, return Q; otherwise return ⊥
function SPLIT(B, t, h)
    repeat
        Choose u uniformly from {0, ..., t}
        Q_1 ← GREEDY(B, u, h), Q_2 ← GREEDY(B, t − u, h)
    until neither answer is ⊥
    return the concatenation of Q_1, Q_2
```

From total density to prefix density. A set can be large while leaving a long initial gap. A prefix-count function locates an anchor from which it is dense in one direction. The available elements $0, 1$ let us translate the resulting witnesses back without subtracting any input elements.

Lemma 249 (An interval when zero and one are available)**.** *Suppose* $\{0,1\} \subseteq B \subseteq [0, D]$ *and* $|B| \geq (D+1)/q$*, where* $q \geq 1$ *is an integer. In* $O(|B| \log(2|B|))$ *time one can construct an integer* s *and a witness for* $[s, s + D] \subseteq 32qB$*, with expected* $O(q \log(2|B|))$ *time per query.*

Proof. The obstacle is that a large set need not have dense initial prefixes. We first find a point from which B is dense in one direction. Measuring distances from that point produces a set E to which the greedy-splitting lemma applies. The elements $0, 1 \in B$ then let us replace each E-summand by two B-summands.

Find a dense direction. Define $f(x) = 2q|B \cap [0, x]| - (x + 1)$ for integer $-1 \leq x \leq D$. This measures the surplus of points over density $1/(2q)$: for $x < y$, $f(y) - f(x) = 2q|B \cap [x + 1, y]| - (y - x)$. Thus an interval on which f does not decrease has the desired density. Also, f decreases by at most one per step.

Let u minimize f and v maximize f. Since $f(-1) = 0$ and $f(D) \geq D + 1$, we have $f(u) \leq 0$ and $f(v) \geq D + 1$. Either $u \leq D/2$ or $v > D/2$. Indeed, otherwise $0 \leq v \leq D/2 < u \leq D$, so f would drop by at least $D + 1$ in at most D steps.

If $u \leq D/2$, look to the right of u. Minimality gives $|B \cap [u + 1, u + x]| \geq x/(2q)$ whenever $1 \leq x \leq D - u$. Set $E = \{0\} \cup ((B - u) \cap [1, D])$.

Otherwise $v > D/2$, and we look to the left of $v + 1$. Maximality gives $|B \cap [v + 1 - x, v]| \geq x/(2q)$ whenever $1 \leq x \leq v + 1$. Set $E = \{0\} \cup ((v + 1 - B) \cap [1, D])$.

In either case, the available direction has length at least $D/2$. Up to that length, every prefix of E has density at least $1/(2q)$. A longer prefix still contains at least $D/(4q)$ positive elements, so its density is at least $1/(4q)$. Consequently $\rho_D(E) \geq 1/(4q)$; the length-one prefix also shows that $1 \in E$. By Lemma 248, every integer in $[0, D]$ has a witness using exactly $h = 16q$ elements of E.

Convert the witnesses back to B. In the right-facing case, $\{u + 1\} + E \subseteq B + B$. For $e > 0$, use $u + 1 + e = 1 + (u + e)$; for $e = 0$, use $u + 1 = 0 + (u + 1)$. Both are valid because $u + e \in B$ for positive $e \in E$, and $1 \in E$ implies $u + 1 \in B$. Thus a representation of $j \in [0, D]$ by h elements of E becomes a representation of $h(u + 1) + j$ by $2h$ elements of B. Take $s = h(u + 1)$.

In the left-facing case, $\{v + 1\} - E \subseteq B + B$. For $e > 0$, use $v + 1 - e = 0 + (v + 1 - e)$; for $e = 0$, use $v + 1 = 1 + v$. Here $1 \in E$ ensures $v \in B$. To represent $s + j$, where $s = h(v + 1) - D$, represent $D - j$ by h elements of E and apply these replacements. Their sum is $h(v + 1) - (D - j) = s + j$. Again the result uses exactly $2h = 32q$ elements of B.

Construction and query time. Sort B. Between consecutive elements of B, the function f decreases by one per step, and it jumps upward at each element. Its extrema therefore occur among $-1, D$, the elements $b \in B$, and their predecessors $b-1$. Evaluate these $O(|B|)$ candidates using their sorted ranks; there is no need to scan $[0, D]$. Constructing and sorting E takes $O(|B| \log(2|B|))$ time. Each query makes one greedy-splitting call, taking expected $O(q \log(2|B|))$ time, followed by $O(q)$ replacements. □

Algorithm 19 The interval witness $\textsc{Seed}(B, D, q)$

function $\textsc{Seed}(B, D, q)$
  Sort B; evaluate $f(x) = 2q|B \cap [0, x]| - (x+1)$ at the extrema candidates in the proof
  Choose a minimizing position u
  if $u \le D/2$ **then**
    $\sigma \leftarrow +1$, $s \leftarrow 16q(u+1)$
    $E \leftarrow \{0\} \cup \{b - u : b \in B,\ 1 \le b - u \le D\}$
  else
    Choose a maximizing position v; set $u \leftarrow v + 1$
    $\sigma \leftarrow -1$, $s \leftarrow 16qu - D$
    $E \leftarrow \{0\} \cup \{u - b : b \in B,\ 1 \le u - b \le D\}$
  return the record (E, D, q, u, σ, s), with E sorted
function $\textsc{SeedQuery}(\mathcal{W}, j)$ ▷ *Returns a witness for $s + j$, $0 \le j \le D$*
  Read (E, D, q, u, σ, s) from $\mathcal{W}$
  $z \leftarrow j$ if $\sigma = +1$, and $z \leftarrow D - j$ otherwise
  $Q \leftarrow \textsc{Split}(E, z, 8q)$
  Replace each entry by its pair from the proof of Lemma 249
  return the resulting list of $32q$ elements of B

A small step, followed by a residue table. The closest pair in A supplies a small positive difference d. Adding that pair to a large residue class creates a normalized set containing zero and one. The preceding lemma builds a progression in that class. Its length is chosen to leave enough room for the residue representatives used to fill its gaps: if there are t occupied residues, a representative needs $O(d/t)$ summands, so a progression spanning $\Theta(Md/t)$ suffices.

For the residue table we use the following elementary witness routine. Its dense convolution operations have length equal to the small modulus, not the original numerical range.

Lemma 250 (Dense modular convolution with witnesses)**.** *Given $X, Y \subseteq \mathbb{Z}/d\mathbb{Z}$, one can compute $X + Y$ and one pair $(x_z, y_z) \in X \times Y$ with $x_z + y_z \equiv z \pmod d$ for every output residue z, in expected $\widetilde{O}(d)$ time.*

Proof. A dense cyclic convolution of the indicator arrays gives the support. To obtain witnesses, independently retain each element of X with probability 2^{-j}, for each $j = 0, \dots, \lceil \log_2 d \rceil$. If X' is the retained set, compute the two cyclic convolutions $c = 1_{X'} * 1_Y$ and $w = (x 1_{X'}(x)) * 1_Y$. Whenever $c_z = 1$, its unique representation is $(w_z, (z - w_z) \bmod d)$.

For a residue with $a \ge 1$ representations, the corresponding first coordinates are distinct. One of the probabilities lies in $[1/(2a), 1/a]$. If $a > 1$, it isolates exactly one representation with probability at least $\frac{1}{2}(1 - 1/a)^{a-1} \ge 1/(2e)$; if $a = 1$, retaining everything succeeds. A full pass over the probabilities therefore finds each missing witness with constant probability. Repeat passes until all known output residues have witnesses. A union bound gives $O(\log(2d))$ expected passes. Every discovered witness is exact, and the total cost is $\widetilde{O}(d)$. □

Algorithm 20 Dense modular pair witnesses

```
function PairWitness(X, Y, d)
    Compute Z = supp(1_X * 1_Y) by cyclic convolution modulo d
    Mark the parent pair of every z ∈ Z as missing
    while some parent pair is missing do
        for j = 0, ..., ⌈log_2 d⌉ do
            Retain each x ∈ X independently with probability 2^{-j}, forming X'
            Compute c = 1_{X'} * 1_Y and w = (x 1_{X'}(x)) * 1_Y
            For every z with c_z = 1, store the pair (w_z, (z − w_z) mod d)
    return Z and all stored pairs
```

Lemma 251 (Short residue representatives)**.** *Let $d \geq 1$, $0 \in A \subseteq [0, M]$, $\gcd(A \cup \{d\}) = 1$, and let $C = A \bmod d$ have size t. Let H be the smallest power of two at least $2d/t$. In expected $\widetilde{O}(|A| + d)$ time one can store, for every residue r, a value $w_r \leq HM$ and a representation of $w_r \equiv r \pmod d$ by exactly H elements of A. Each representation can be expanded in $O(H)$ time.*

Proof. First, $HC = \mathbb{Z}/d\mathbb{Z}$. For completeness, if $(j+1)C$ is proper with stabilizer J, then C meets at least two J-cosets, since it contains zero and generates the cyclic group. Kneser gives $|(j+1)C| \geq |jC| + |C+J| - |J| \geq |jC| + t/2$, using $|C + J| \geq \max\{t, 2|J|\}$. Since $0 \in C$, the repeated sumsets are nested. If HC were proper, the first H additions would therefore give more than d elements. The case $d = 1$ is immediate.

Choose one original representative $a_r \in A$ of every $r \in C$. Starting with C, repeatedly square modulo d, using Lemma 250 to store parent pairs at each level. At level zero store weight a_r; at higher levels store the sum of the weights of the two chosen parents. At the final level every residue is present and its weight is at most HM. Recursively following parents produces H leaves and takes $O(H)$ time. There are $\log_2 H = O(\log(2d))$ levels, so the construction takes expected $\widetilde{O}(|A| + d)$ time. □

Algorithm 21 Building and reading the residue table

```
function ResidueTable(A, d, H)
    C_0 ← A mod d; choose a representative a_r ∈ A for each r ∈ C_0
    Set w_0(r) ← a_r and h ← log_2 H
    for j = 1, ..., h do
        (C_j, P_j) ← PairWitness(C_{j−1}, C_{j−1}, d)
        For r ∈ C_j with P_j(r) = (u, v), set w_j(r) ← w_{j−1}(u) + w_{j−1}(v)
    return (a_r, P_1, ..., P_h, w_h)
function Expand(j, r, Q)                    ▷ Append to one shared output list
    if j = 0 then
        Append a_r to Q; return
    Let (u, v) = P_j(r)
    Expand(j − 1, u, Q); Expand(j − 1, v, Q)
```

Proof of Theorem 247.

Build a progression with a small step. Sort A and find a closest pair $a_*, a_* + d$. Then $1 \leq d \leq M/(n-1) \leq 2M/n \leq 2k$. Put $t = |A \bmod d|$, choose a largest residue class $A' \subseteq A$, and let $a_0 = \min A'$. The set $B = ((A' - a_0)/d) + \{0, 1\}$ contains zero and one, and every $b \in B$ has a stored pair of original elements summing to $a_0 + a_* + db$. Also $|B| \geq |A'| + 1 \geq n/t + 1$.

Set $\ell = \lceil 5M/t \rceil$. Since $t \leq d$, we have $\max B \leq M/d + 1 \leq \ell$, and $|B| \geq n/t + 1 \geq (\ell + 1)/(5k)$. The last inequality uses $\ell + 1 \leq 5M/t + 2$ and $kn \geq M + 1$. Apply Lemma 249 to B, with range ℓ and parameter $5k$. It supplies an interval $[s_B, s_B + \ell] \subseteq 160kB$. Replacing each summand by its stored pair gives

$P = \{s, s+d, \ldots, s+\ell d\} \subseteq 320kA$, where $s = 160k(a_0 + a_*) + ds_B$. All this takes $O(n \log(2n))$ time, and a requested term of P has a witness in expected $O(k \log(2n))$ time.

Fill the residue classes. Let H be the smallest power of two at least $2d/t$. Then $H \le 4d/t \le 8k$, and $\ell d \ge 5Md/t \ge HM + M$. The residue table provides $w_r \in HA$, with $0 \le w_r \le HM$ and $w_r \equiv r \pmod d$. Take $T = s + HM$. For $x \in [T, T+M]$, let $r = (x - s) \bmod d$. Then $x - w_r \equiv s \pmod d$, while $s \le x - w_r \le s + \ell d$. Thus $x - w_r \in P$. Combine its $320k$-term witness with the H-term witness for w_r, and pad with zeros to obtain exactly $328k$ terms.

When to build the table. If $d \le n$, build it during preprocessing, in expected $\widetilde{O}(n)$ time. If $d > n$, store the residue lists but build the table inside a query. In that case $n < d \le 2k$, so its expected cost is $\widetilde{O}(k)$. Expansion costs $O(H) = O(k)$, as does handling the output list. These observations give all the claimed time and space bounds. □

Algorithm 22 A repeated-sum progression and its witnesses

```
1:  function ProgressionQuery(A, M)
2:      Sort A; set n ← |A|, k ← ⌈(M + 1)/n⌉
3:      Find a closest pair a_*, a_* + d
4:      Group A by residue modulo d; let t be the number of groups
5:      Choose a largest group A'; set a_0 ← min A'
6:      Form B = ((A' − a_0)/d) + {0, 1}, storing an original pair for every b
7:      ℓ ← ⌈5M/t⌉, W ← Seed(B, ℓ, 5k)
8:      s ← 160k(a_0 + a_*) + ds_B, using the offset s_B in W
9:      H ← the smallest power of two at least 2d/t; T ← s + HM
10:     If d ≤ n, store R ← ResidueTable(A, d, H)
11:     return T and the construction records
12: function ProgressionQuery(D, x)                          ▷ T ≤ x ≤ T + M
13:     Read the construction records from D
14:     Use the stored table R, or build it now if d > n
15:     r ← (x − s) mod d, w ← w_{log_2 H}(r)
16:     j ← (x − s − w)/d, Q ← SeedQuery(W, j)
17:     Replace every b ∈ Q by its stored pair of original elements
18:     Expand(log_2 H, r, Q); append zeros to reach 328k entries
19:     return the resulting list
```

7.9.11 Witnesses for Long Progressions in Subset Sums

Write $\mathcal{S}(X) = \{\sum_{x \in Y} x : Y \subseteq X\}$ and $\Sigma_X = \sum_{x \in X} x$. Repeated-sum witnesses may reuse an element; subset-sum witnesses must remember which items remain available. Disjoint pairs provide a convenient way to enforce this. Start by taking the smaller endpoint of every pair. Using a gap means switching that pair to its larger endpoint. Several pairs with the same gap therefore provide several independent copies of that gap.

This gives a short initial progression using the repeated-sum theorem. To extend it, partition the unused numbers into short intervals and pair numbers with different residues modulo the current step d. Either there are enough such pairs to fill a smaller lattice, or most numbers remain in groups having the same residue. Pairing the extremes of those groups then lengthens the progression without changing d. Each step uses a fresh collection of disjoint pairs.

Theorem 252 (Constructive subset-sum progression)**.** *Let $X \subseteq \{1, \ldots, M\}$ have n distinct elements, put $\Lambda = \lceil \log_2(2n) \rceil$, and let ℓ be an integer with $M \le \ell \le n^2/(C\Lambda)$, where $C = 2^{30}$ suffices. In expected $\widetilde{O}(n)$*

time, one can find $X_* \subseteq X$, integers $s, d > 0$, and a witness for $\{s, s+d, \dots, s+\ell d\} \subseteq \mathcal{S}(X_*)$, with $d \le 5M/n$ and

$$|X_*| = O\left(\frac{\ell + M\log(2n)}{n}\right), \qquad \textit{expected query time } \widetilde{O}\left(\frac{\ell + M\log(2n)}{n}\right).$$

Every query returns distinct original items. In fact, all answers choose exactly one endpoint from each pair in a fixed matching on X_*.

The constant is only for the bookkeeping below. Taking $\ell = M$ gives the usual density requirement $n^2 \ge CM\Lambda$. The hypotheses imply $n \ge C\Lambda$ and $\ell/n \ge 1$; in particular, all rounding estimates below have ample slack. The progression is stored by its three parameters; its terms are never enumerated. For a multiset, the theorem may be applied to one chosen copy of each distinct value, with n counting those distinct values. Its density hypothesis cannot use the total multiplicity in place of n.

Turning repeated gaps into distinct items. All pairs below are oriented as (a, b) with $a < b$, and their endpoints are distinct across the entire collection. A pair carries its original item indices as well as its values. For a collection T, put $B_T = \sum_{(a,b)\in T} a$. If a gap value g has q available pairs, it may be used at most q times: switch that many different pairs. A list of requested gaps then gives a subset of weight $B_T + \sum g$. Unused pairs contribute their smaller endpoints, rather than disappearing.

We need one elementary way to obtain uniform multiplicities. Suppose $p \le n$ objects have labels, with label frequencies f_v. There is a threshold $u \ge 1$ such that, for $F = \{v : f_v \ge u\}$, we have $u|F| \ge p/\Lambda$. Indeed, writing $N_j = |\{v : f_v \ge j\}|$, we have $p = \sum_{j=1}^{p} N_j \le (\max_j jN_j)\sum_{j=1}^{p} 1/j \le \Lambda \max_j jN_j$. Sorting the frequencies finds a maximizing threshold in $O(p\log(2p))$ time. We may then keep any prescribed number at most u of objects of each retained label.

Algorithm 23 Uniform labels and independent endpoint choices

```
function Uniform(T, label)
    Group the objects by label into lists T_v; set f_v ← |T_v|
    Sort the frequencies as f_1 ≤ ··· ≤ f_t
    Choose i maximizing f_i(t − i + 1); set u ← f_i
    return u and the lists T_v with f_v ≥ u
function ChooseEndpoints((T_v)_v, Q)        ▷ Each nonzero label occurs in Q at most |T_v| times
    Set a counter to zero for each list; select every pair's smaller endpoint
    w ← 0
    for each nonzero label v in Q do
        Increment the counter of T_v; let (a, b) be that pair
        Switch its selected endpoint to b; set w ← w + b − a
    return w and the list of selected original items
```

The initial progression. Set $L_0 = \lceil 512\ell/n \rceil$ and $G = \lceil 4M/n \rceil$. Sort X, pair consecutive elements as $(x_1, x_2), (x_3, x_4), \dots$, and retain pairs whose gap is at most G. All the original paired gaps sum to at most M, so fewer than M/G pairs are discarded. At least $n/8$ pairs remain. Uniformize their gap labels, obtaining t distinct gaps, each available at least u times, with $ut \ge n/(8\Lambda)$.

Let d_0 be the gcd of the retained gaps and $A = \{0\} \cup \{g/d_0 : g \text{ is a retained gap}\}$. Then $A \subseteq [0, L_0]$, $|A| = t + 1$, and $\gcd(A) = 1$. Apply Theorem 247 with ambient range L_0. It supplies an interval $[T, T + L_0] \subseteq qA$, where $q = 328\lceil (L_0 + 1)/(t + 1) \rceil$. Since $1 \le t \le G \le L_0$, we have $qt \le 656L_0$. The density hypothesis gives $656L_0 \le n/(8\Lambda) \le ut$; thus $q \le u$.

Keep q pairs of each retained gap, and let B be the sum of their smaller endpoints. Multiplying a repeated-sum witness by d_0 and switching distinct pairs gives a witness for $\{B + d_0(T + j) : 0 \le j \le L_0\} \subseteq \mathcal{S}(X_0)$, with step $d_0 \le G \le 5M/n$. There are at most $656L_0$ pairs, so $|X_0| \le 1312L_0 < 2^{20}\ell/n$. The construction takes expected $\widetilde{O}(n)$ time and its query takes expected $\widetilde{O}(L_0)$ time. Only these retained pairs are removed from the reservoir of unused items.

Algorithm 24 A subset-sum seed

function SUBSETSEED(X, M, L_0)
  Sort X; form disjoint consecutive pairs with gap at most $G = \lceil 4M/|X| \rceil$
  Apply UNIFORM with label $b - a$, obtaining t retained labels
  $d_0 \leftarrow \gcd\{g : g \text{ is a retained label}\}$, $q \leftarrow 328\lceil (L_0+1)/(t+1) \rceil$
  Keep q pairs per label; relabel their lists by g/d_0
  $B \leftarrow \sum a$, over all kept pairs; $A \leftarrow \{0\} \cup \{g/d_0\}$
  $(T, \mathcal{W}) \leftarrow$ PROGRESSION(A, L_0)
  Store the lists, $B, d_0, \mathcal{W}$; return the progression $(B + d_0 T, d_0, L_0)$ and its record
function SUBSETSEEDQUERY(record, z)
  Read the pair lists, $B, d_0, \mathcal{W}$ from the record
  $Q \leftarrow$ PROGRESSIONQUERY($\mathcal{W}, (z - B)/d_0$)
  $(w, Y) \leftarrow$ CHOOSEENDPOINTS($(T_v)_v, Q$)
  return Y

A batch of pairs either changes the step or extends the length. Suppose the current progression is $P = \{s, s+d, \ldots, s+Ld\}$, where $L_0 \le L < \ell$ and $d \le 5M/n$. Let U be the unused items; for now assume $|U| \ge n/2$. We will verify this throughout the construction. Put $\Delta = \lfloor L/32 \rfloor$ and partition $[1, M]$ into consecutive intervals of $\Delta + 1$ integers. There are at most $h \le M/\Delta + 1 \le 64M/L + 1$ intervals, and two numbers in the same interval differ by at most Δ. This choice lets $O(d)$ gap switches move a sum by only a fixed fraction of the current span Ld. If too few switches change the residue, density instead forces many d-spacings inside the intervals.

Within each interval, repeatedly pair two numbers of different residues modulo d. This can be done by grouping by residue and maintaining a list of nonempty groups. Remove the two chosen endpoints from that interval's temporary lists. Stop immediately if $4d\Lambda$ pairs have been found. Otherwise finish all intervals; the unpaired numbers in each interval then have one common residue. These are temporary pairings: only pairs ultimately selected for the witness will be removed from U.

Many different-residue pairs: reduce the step. Uniformize the $4d\Lambda$ pairs by their nonzero gap residues. There are t retained residues, each available u times, with $ut \ge 4d$. Put $d' = \gcd(d, \text{retained residues})$ and $D = d/d'$. Since every residue is nonzero, $d' < d$. The set $C = \{0\} \cup \{r/d' : r \text{ retained}\}$ generates $\mathbb{Z}/D\mathbb{Z}$.

Let H be the smallest power of two at least $2D/(t+1)$. Then $Ht < 4D \le 4d \le ut$, so we can keep H pairs of each retained residue. By Lemma 251, in expected $\widetilde{O}(D)$ time the table RESIDUETABLE(C, D, H) gives an H-term representation of every residue modulo D. Replace each nonzero term r/d' by a different kept pair having gap congruent to r modulo d. For every residue class in $d'\mathbb{Z}/d\mathbb{Z}$, this gives an actual gap sum w in that class, with $0 \le w \le H\Delta \le 4d\Delta$. No pair is reused, even when the same residue occurs repeatedly.

Let B be the sum of the smaller endpoints of all kept pairs. The new progression has start $s' = s + B + 4d\Delta$, step d', and length $L' = (L - 4\Delta)d/d' \ge 7L/4$. For any term z of this progression, choose the table residue $(z - s - B)/d' \bmod D$, expand it, and perform the corresponding switches. Then $z - B - w \equiv s \pmod d$ and $s \le z - B - w \le s + Ld$, so it is a term of P. This growth step consumes fewer than $8d$ items. Its additional query work is $O(Ht) = O(d)$.

Few different-residue pairs: extend with equal residues. There are fewer than $4d\Lambda$ temporary pairs. Thus the number R of unpaired items is at least $n/2 - 8d\Lambda \ge n/3$. Since $L \ge 512M/n$, we have $h \le n/8 + 1 \le n/6$, hence $R \ge 2h$. Within each interval, pair the smallest remaining number with the largest, the second smallest with the second largest, and so on. Every resulting gap is a positive multiple of d and at most Δ.

An interval with q_i remaining numbers supplies total gap at least $d\lfloor q_i^2/4 \rfloor$: the j-th extreme pair spans at least $q_i + 1 - 2j$ gaps of size d. Consequently the total gap W of all these pairs satisfies $W \ge d(R^2/h - h)/4 \ge dR^2/(8h)$, by Cauchy–Schwarz and $R \ge 2h$. In particular $W \ge Ld$, since $72hL \le 4608M + 72L \le 4680\ell \le n^2$.

There are at most $R/2$ pairs, so their average gap is at least $dR/(4h)$. Take pairs in decreasing gap order until their gap sum first reaches Ld. An initial segment has average at least the overall average, so at most $\lceil 4Lh/R \rceil \le 1 + (1024M + 16L)/n$ pairs suffice. Write their gaps as $d\delta_1, \dots, d\delta_q$, and put $B = \sum a_i$. Each $1 \le \delta_i \le L$, so the two intervals $[0, L]$ and $\delta_i + [0, L]$ overlap or touch. Successively choosing the lower or upper endpoints therefore gives the progression with start $s + B$, step d, and length $L' = L + \sum_i \delta_i \ge 2L$.

The witness is equally simple. Starting with the requested progression index, undo the pairs in reverse order. If the index exceeds the length before that pair was added, choose the upper endpoint and subtract δ_i; otherwise choose the lower endpoint. The index then lies in the preceding interval. After all pairs have been undone, query the original progression. The step consumes at most $2 + (2048M + 32L)/n$ items.

Algorithm 25 One growth step using short intervals

function IntervalPairs(U, d, Δ, p)
  Group U by $\lfloor (x-1)/(\Delta+1) \rfloor$, then by residue modulo d
  $T \leftarrow \varnothing$
  for each interval **do**
    Keep a list of its nonempty residue groups
    while at least two groups remain **do**
      Pop one item from each of two groups; orient and append their pair to T
      Remove empty groups from the list
      If $|T| = p$, return (`residues`, T)
  In each remaining group, sort the items and pair its extremes
  return (`stretch`, all these extreme pairs)
function Grow(U, s, d, L, n, Λ)
  $\Delta \leftarrow \lfloor L/32 \rfloor$
  $(\tau, T) \leftarrow$ IntervalPairs($U, d, \Delta, 4d\Lambda$)
  if $\tau =$ `residues` **then**
    Uniformize T with label $(b - a) \bmod d$; let t be the number of retained labels
    $d' \leftarrow \gcd(d, \text{retained labels})$, $D \leftarrow d/d'$
    $H \leftarrow$ the smallest power of two at least $2D/(t+1)$
    Keep H pairs per retained label; relabel their lists by r/d'
    $C \leftarrow \{0\} \cup \{r/d'\}$, $\mathcal{R} \leftarrow$ ResidueTable(C, D, H)
    $B \leftarrow \sum a$ over kept pairs
    $s' \leftarrow s + B + 4d\Delta$, $L' \leftarrow (L - 4\Delta)D$
    Store a residue record with $s, d, L, B, d', D, H, \mathcal{R}$ and the pair lists
  else
    Sort T by decreasing gap; keep the shortest prefix of gap sum at least Ld
    $\delta_i \leftarrow (b_i - a_i)/d$ for the kept pairs, $B \leftarrow \sum_i a_i$
    $s' \leftarrow s + B$, $d' \leftarrow d$, $L' \leftarrow L + \sum_i \delta_i$
    Store a stretch record with s, d, L, B and the kept pairs
  return (s', d', L'), the record, and the endpoints of the kept pairs

Proof of Theorem 252. Build the seed, remove its retained endpoints, and repeatedly apply the growth step until the length reaches ℓ. Truncate the last progression to length ℓ. Every new collection of pairs uses unused items, so recursive reconstruction produces distinct items. Each step retains one endpoint of each of its pairs; the same is true of the seed.

We check that the reservoir always has at least $n/2$ items. Each growth factor is at least $7/4$, so there are at most 2Λ steps. Moreover, the sum of their starting lengths is at most $7\ell/3$. The step d stays unchanged in a stretch step and at least halves in a residue step. Thus the values of d at the residue steps sum to at most $2d_0 \le 10M/n$. The seed uses fewer than $2^{20}\ell/n$ items. Summing the two growth bounds, including the additive constant in each stretch step, gives $|X_*| \le 2^{21}(\ell + M\Lambda)/n \le n/256$. These estimates apply to every prefix of the construction. In particular, a first step leaving fewer than $n/2$ unused items would contradict

the same bound. Every growth step is therefore valid.

Each step sorts or scans at most n items. All residue tables together have moduli summing to $O(d_0)$, and the seed construction costs expected $\widetilde{O}(n)$. Hence the total preprocessing takes expected $\widetilde{O}(n)$ time and space. A query traverses one chain of growth records, reads each retained pair once, and makes one repeated-sum query at the seed. Its expected time is $\widetilde{O}(|X_*|)$. The sharper size bound in the statement follows by summing the geometrically increasing lengths, rather than charging ℓ separately at every step. □

Algorithm 26 Building and querying the subset-sum progression

function SubsetProgression(X, M, ℓ)
  $n \leftarrow |X|$, $\Lambda \leftarrow \lceil \log_2(2n) \rceil$, $L_0 \leftarrow \lceil 512\ell/n \rceil$
  Build the seed record and its progression (s, d, L_0)
  Remove its kept endpoints from X, leaving U; set $L \leftarrow L_0$
  while $L < \ell$ **do**
    $((s, d, L), \mathcal{W}, Y) \leftarrow$ Grow(U, s, d, L, n, Λ)
    Append $\mathcal{W}$ to the record list; remove Y from U
  return (s, d, ℓ), all records, and the union X_* of kept endpoints
function UndoGrowth($\mathcal{W}, z, Q$) ▷ *Append new items to one shared list Q*
  Read the stored parameters and pairs from $\mathcal{W}$
  if $\mathcal{W}$ is a residue record **then**
    $r \leftarrow ((z - s - B)/d' \bmod D$; expand residue r in $\mathcal{R}$ into an H-term list V
    $(w, Y) \leftarrow$ ChooseEndpoints($(T_v)_v, V$)
    Append Y to Q; return $z - B - w$
  else
    $j \leftarrow (z - s - B)/d$, $K \leftarrow L + \sum_i \delta_i$
    for kept pairs (a_i, b_i) in reverse order **do**
      $K \leftarrow K - \delta_i$
      if $j > K$ **then**
        Append b_i to Q; set $j \leftarrow j - \delta_i$
      else
        Append a_i to Q
    return $s + dj$
function SubsetProgressionQuery(z)
  $Q \leftarrow$ an empty output list
  For growth records in reverse order, set $z \leftarrow$ UndoGrowth($\mathcal{W}, z, Q$)
  Append SubsetSeedQuery(seed record, z) to Q
  return Q

7.9.12 Additive Energy

Small sumsets force many pairs to produce the same sum. Additive energy measures these collisions even when the sumset is not known to be small. Define $r_{A+B}(x) = |\{(a, b) \in A \times B : a + b = x\}|$ and $E(A, B) = \sum_x r_{A+B}(x)^2$; write $E(A) = E(A, A)$. Equivalently, energy counts quadruples with $a + b = a' + b'$.

Proposition 253 (Energy bounds)**.** *For finite nonempty sets A, B,*

$$\frac{|A|^2|B|^2}{|A + B|} \le E(A, B) \le |A||B| \min\{|A|, |B|\}.$$

Also $E(A, B) = \sum_x r_{A-A}(x) r_{B-B}(x)$.

Proof. The multiplicities sum to $|A||B|$, so Cauchy–Schwarz gives the lower bound. Each multiplicity is at most $\min\{|A|, |B|\}$, giving the upper bound. Finally, rewrite $a + b = a' + b'$ as $a - a' = b' - b$ and count by this common difference. □

Thus $|A+A| \le K|A|$ implies $E(A) \ge |A|^3/K$. A converse cannot control all of A, since a structured part may contribute most of the energy while the remaining elements make the sumset large. Balog–Szemerédi–Gowers finds such a structured part.

7.9.13 Solymosi's Sum–Product Bound

Write $AA = \{ab : a, b \in A\}$. Ratios place the points of $A \times A$ on rays through the origin. The squared numbers of points on these rays count equal products. Addition connects this multiplicative information to the sumset: sums of points from two distinct rays determine their summands, and sums from consecutive rays occupy disjoint sectors.

Theorem 254 (Solymosi)**.** *Let A be a finite nonempty set of positive real numbers, and put $n = |A|$. We have that*

$$|A+A|^2|AA| \ge \frac{n^4}{1+2\log n}.$$

Consequently,

$$\max\{|A+A|, |AA|\} \ge \frac{n^{4/3}}{(1+2\log n)^{1/3}}.$$

Proof. Put $s = |A+A|$. For each occupied slope $\lambda > 0$, let $R_\lambda = \{(a,b) \in A^2 : b = \lambda a\}$ and $r_\lambda = |R_\lambda|$. The multiplicative energy $E = \sum_\lambda r_\lambda^2$ counts equalities $ad = bc$. Counting the same equalities by the common product and applying Cauchy–Schwarz gives

$$E = \sum_{z \in AA} |\{(a,b) \in A^2 : ab = z\}|^2 \ge \frac{n^4}{|AA|}.$$

We bound E using addition. For $1 \le t \le n$, list the slopes with $r_\lambda \ge t$ as $\lambda_1 < \cdots < \lambda_k$. For each $i < k$, the set $R_{\lambda_i} + R_{\lambda_{i+1}}$ has exactly $r_{\lambda_i} r_{\lambda_{i+1}}$ elements. Indeed, an equality $p+q = p'+q'$ makes $p-p' = q'-q$ lie on both distinct lines, so both differences are zero. The slope of $p+q$ lies strictly between λ_i and λ_{i+1}: it is their weighted average, with the positive first coordinates of p and q as weights. Thus these sumsets are disjoint subsets of $(A+A)^2$, and

$$(k-1)t^2 \le \sum_{i=1}^{k-1} r_{\lambda_i} r_{\lambda_{i+1}} \le s^2.$$

Let $N(t) = |\{\lambda \in A/A : r_\lambda \ge t\}|$. We have proved $N(t) \le 1 + s^2/t^2$ for $1 \le t \le n$. Also $N(t) \le n^2$, and every $r_\lambda \le n$. Integrating over thresholds gives

$$E = \int_0^n 2tN(t)\,dt \le n^2 + \int_1^n 2t\left(1 + \frac{s^2}{t^2}\right)\,dt = 2n^2 - 1 + 2s^2\log n.$$

Finally, $s \ge 2n-1$: if $a_1 < \cdots < a_n$ are the elements of A, then $a_1+a_1, \ldots, a_1+a_n, a_2+a_n, \ldots, a_n+a_n$ increase strictly. Hence $2n^2 - 1 \le (2n-1)^2 \le s^2$, giving $E \le (1+2\log n)s^2$. Combine this with the lower bound on E. □

For a real set A with at least two elements, retain a largest subset of one strict sign and negate it if necessary. Its size is at least $(|A|-1)/2$, and its sumset and product set are no larger than those of A. Thus, including the immediate singleton case, the same bound holds for arbitrary nonempty real sets up to an absolute constant: $\max\{|A+A|, |AA|\} \gtrsim |A|^{4/3}/\log^{1/3}(2|A|)$.

7.9.14 Balog–Szemerédi–Gowers

Large additive energy means that many pairs reuse the same sums. We retain a suitable set of sum labels and view their representations as edges of a bipartite graph. Four-edge walks then express endpoint differences as alternating sums of edge labels. If every difference has many such representations, but there are few possible label tuples, the difference set must be small.

Lemma 255 (Many four-edge walks)**.** *Let Γ be bipartite with parts U, V of size $N \geq 1$, and at least δN^2 edges, where $0 < \delta \leq 1$. There is $U' \subseteq U$, of size at least $\delta N/2$, such that every ordered $u, u' \in U'$ is joined by at least $\delta^5 N^3/2^{12}$ four-edge walks u, v_1, w, v_2, u'. Vertices may repeat.*

Proof. Call an ordered pair $(u, w) \in U^2$ bad if it has fewer than $\theta = \delta^2 N/32$ common neighbors. We seek a large set in which each vertex has few bad partners. Then any two vertices have many common good partners, each supplying many four-edge walks.

Choose $v \in V$ uniformly and put $W = N_\Gamma(v)$. A pair with c common neighbors belongs to W^2 with probability c/N, so sampling a neighborhood favors pairs with many common neighbors. In particular, $\mathbb{E}|W| \geq \delta N$, hence $\mathbb{E}|W|^2 \geq \delta^2 N^2$. If $b(W)$ counts bad ordered pairs in W^2, then $\mathbb{E}b(W) \leq N^2(\theta/N) = \delta^2 N^2/32$.

Consequently, some neighborhood satisfies

$$|W|^2 - 16b(W) \geq \frac{\delta^2 N^2}{2}.$$

Thus $|W| \geq \delta N/\sqrt{2}$ and $b(W) \leq |W|^2/16$. Delete vertices with more than $|W|/4$ bad partners in W. At most $4b(W)/|W| \leq |W|/4$ vertices are deleted, leaving

$$|U'| \geq \frac{3|W|}{4} \geq \frac{3\delta N}{4\sqrt{2}} \geq \frac{\delta N}{2}.$$

For any $u, u' \in U'$, at least $|W|/2$ vertices $w \in W$ are good partners of both endpoints. Each gives at least θ^2 choices of (v_1, v_2), so the number of walks is at least

$$\frac{|W|}{2}\theta^2 \geq \frac{\delta N}{4}\left(\frac{\delta^2 N}{32}\right)^2 = \frac{\delta^5 N^3}{2^{12}}.$$

□

Lemma 256 (Restricted sums give a small difference set)**.** *Let A, B be subsets of an abelian group with $|A| = |B| = N \geq 1$, and let $\Gamma \subseteq A \times B$ have at least δN^2 edges, where $0 < \delta \leq 1$. If $S = \{a + b : (a, b) \in \Gamma\}$ has size at most LN, some $A' \subseteq A$ satisfies*

$$|A'| \geq \frac{\delta N}{2}, \qquad |A' - A'| \leq 2^{12} L^4 \delta^{-5} N.$$

Proof. Choose A' using Lemma 255. For each $d \in A' - A'$, fix endpoints a, a' with $d = a - a'$. Every walk a, b_1, c, b_2, a' supplies four labels in S with

$$d = (a + b_1) - (c + b_1) + (c + b_2) - (a' + b_2).$$

For the fixed endpoints, these labels recover b_1, then c, then b_2, so distinct walks give distinct label tuples. Their alternating sum recovers d, so tuples counted for different differences are disjoint. There are therefore at least $|A' - A'|\delta^5 N^3/2^{12}$ distinct tuples in S^4. Since $|S|^4 \leq L^4 N^4$, the claimed bound follows. □

The remaining choice is which sums to retain. The restricted-sum lemma gives

$$\frac{|A' - A'|}{|A'|} \leq 2^{13}\frac{L^4}{\delta^6} = 2^{13}\left(\frac{L}{\delta^{3/2}}\right)^4.$$

Thus we want few sum labels relative to the density of their representations. We optimize this ratio over popularity thresholds.

Theorem 257 (Balog–Szemerédi–Gowers, energy form)**.** *Let A be a finite nonempty subset of an abelian group. If $|A| = N$ and $E(A) \geq N^3/K$, where $K \geq 1$, some $A' \subseteq A$ satisfies*

$$\boxed{|A'| \geq \frac{N}{4K}, \qquad |A' - A'| \leq 2^{21} K^4 |A'|.}$$

Proof. Put $r(x) = r_{A+A}(x)$. For $0 < t \leq 1$, define $S_t = \{x : r(x) \geq tN\}$, $L_t = |S_t|/N$, and $\delta_t = N^{-2} \sum_{x \in S_t} r(x)$. Thus δ_t is the density of the graph retaining precisely the pairs whose sums lie in S_t. Since every retained sum has at least tN representations, $tL_t \leq \delta_t \leq 1$.

Integrating over thresholds recovers the normalized energy: a sum with $r(x)$ representations contributes for exactly the thresholds $0 < t \leq r(x)/N$. Hence

$$\int_0^1 \delta_t \, dt = \frac{1}{N^2} \sum_x r(x) \int_0^{r(x)/N} dt = \frac{E(A)}{N^3} \geq \frac{1}{K}.$$

Among thresholds with $\delta_t \geq 1/(2K)$, choose one maximizing $\delta_t^{3/2}/L_t$, and denote the maximum by Q. Such thresholds exist because $\delta_t = 1$ for sufficiently small positive t. Only finitely many sets S_t arise, so the maximum is attained.

We claim that $Q \geq 1/(4K)$. For every threshold satisfying the density condition, maximality gives $\delta_t^{3/2} \leq QL_t \leq Q\delta_t/t$, and therefore $\delta_t \leq Q^2/t^2$. At the other thresholds, $\delta_t < 1/(2K)$. Together with $\delta_t \leq 1$, these bounds imply

$$\frac{1}{K} \leq \int_0^1 \delta_t \, dt \leq \frac{1}{2K} + \int_0^\infty \min\left\{1, \frac{Q^2}{t^2}\right\} dt = \frac{1}{2K} + 2Q.$$

The last integral is evaluated by splitting at $t = Q$. This proves the claim.

At the chosen threshold, write $S = S_t$, $L = L_t$, and $\delta = \delta_t$. We have $\delta \geq 1/(2K)$ and $L \leq 4K\delta^{3/2}$. Join two copies of A when their sum lies in S. By Lemma 256, some $A' \subseteq A$ satisfies $|A'| \geq \delta N/2 \geq N/(4K)$ and

$$|A' - A'| \leq 2^{12} L^4 \delta^{-5} N \leq 2^{20} K^4 \delta N \leq 2^{21} K^4 |A'|.$$

□

If $|A' - A'| \leq D|A'|$, Plünnecke–Ruzsa with base set A' and summand set $-A'$ gives $|A' + A'| \leq D^2|A'|$. Here this yields $|A' + A'| \leq 2^{42} K^8 |A'|$.

The proof also gives a deterministic polynomial-time extraction. Compute the representation counts and scan their distinct values to maximize δ_t^3/L_t^2 subject to $\delta_t \geq 1/(2K)$. For the resulting graph, compute common-neighbor counts, choose a neighborhood with the required score, and prune it as in Lemma 255.

7.9.15 Algorithmic BSG

For algorithms, it is useful to cover most relevant pairs by a few Cartesian products whose full sumsets are small. Each product can then be handled as a batch; the uncovered pairs can be inspected individually.

We count group operations and comparisons of group elements as constant-time operations. Sampling uses independent uniform draws with replacement. Write $C_0 = 2^{22}$.

Theorem 258 (BSG decomposition)**.** *Let A, B, S be subsets of an abelian group, each of size at most $N \geq 1$, and let $0 < \varepsilon < 1$. There is a Las Vegas algorithm with expected running time $O(N^2 \log^2(2N))$ which outputs sets $A_i \subseteq A$, $B_i \subseteq B$, their full sumsets $T_i = A_i + B_i$, and the residual pair list*

$$R = \{(a, b) \in A \times B : a + b \in S\} \setminus \bigcup_{i=1}^{k} (A_i \times B_i).$$

The bounds are $k \leq \lceil 32/\varepsilon \rceil$, $|T_i| \leq C_0 \varepsilon^{-5} N$, and $|R| \leq \varepsilon N^2$. Every returned decomposition is correct; only the running-time bound is in expectation.

The products may overlap, and their sumsets may contain elements outside S. The important guarantees are that they cover the relevant pairs and have few possible sums. Three observations make the construction fast: estimate only the graph statistics we need, charge work to the edges removed, and stop using the delicate construction whenever the claimed sumset bound is already trivial.

A sampling test with a gap. We repeatedly need to recognize small probabilities. A constant-factor gap around the threshold makes the sample count proportional to its reciprocal.

Lemma 259 (Testing a probability threshold)**.** *Let $P : \Omega \to \{0,1\}$, let $p = \mathbb{E}_{x\in\Omega} P(x)$, and let $0 < \tau \le 1/2$, $0 < \zeta < 1$. The procedure* $\textsc{Low}(\Omega, P, \tau, \zeta)$ *samples $s = \lceil 8\tau^{-1}\log(2/\zeta)\rceil$ independent points and returns true if their mean is at most τ. Except with probability at most ζ, it returns true whenever $p \le \tau/2$, and false whenever $p \ge 2\tau$. Thus an accurate true answer implies $p < 2\tau$, and an accurate false answer implies $p > \tau/2$.*

Proof. Let X be the number of sampled successes. If $p \le \tau/2$, exponential Markov and independence give $\Pr(X > s\tau) \le (1+p)^s 2^{-s\tau} \le e^{-s\tau/8}$. If $p \ge 2\tau$, applying Markov to 2^{-X} gives $\Pr(X \le s\tau) \le (1-p/2)^s 2^{s\tau} \le e^{-s\tau/4}$. Here we used $1 + x \le e^x$, $\log 2 - 1/2 \ge 1/8$, and $1 - \log 2 \ge 1/4$. The chosen sample count makes each relevant error probability at most ζ. For $\tau/2 < p < 2\tau$, either answer is allowed. □

Finding one structured product. Represent the relevant pairs by a bipartite graph Γ on parts U, V of size N; pad smaller parts with isolated vertices. Let $e_\Gamma(X, Y)$ count edges between X and Y. Suppose Γ has at least δN^2 edges, where $0 < \delta \le 1$. We will find a large U' and a set V' such that every pair in $U' \times V'$ has many three-edge walks between its endpoints. When edges carry sum labels, these walks will bound $|U' + V'|$.

Keep only left vertices of degree at least $\delta N/2$, forming U_0. Choose a right vertex v and examine $W = U_0 \cap N_\Gamma(v)$. Pairs with very few common neighbors rarely occur together in this random neighborhood. We seek a W with few such pairs, remove vertices having too many bad partners, and take right vertices with many remaining neighbors.

The expensive operation would be computing all common-neighbor counts. Instead, choose one random list Q of right vertices and define

$$\theta = \frac{\delta^3}{2^{16}}, \qquad \mathrm{bad}(u, w) = \left[\frac{1}{|Q|}\sum_{z\in Q} \Gamma_{u,z}\Gamma_{w,z} \le 2\theta\right],$$

where $\Gamma_{u,z}$ is the adjacency-matrix entry. This predicate is evaluated only when requested. The list Q is fixed throughout one call, so it defines an actual symmetric relation; fresh samples are used to estimate densities in that relation. The cutoff $\Theta(\delta^3 N)$ comes from the following balance: $\mathbb{E}|W|^2 = \Omega(\delta^2 N^2)$, and we can tolerate an $O(\delta)$ fraction of bad pairs before pruning becomes costly.

Sampling from $W \times W$ means drawing its two coordinates independently. Each call to Low follows the explicit sampling procedure in Lemma 259.

Lemma 260 (Sampled three-walk lemma)**.** *Whenever Algorithm 27 returns sets, they satisfy $|U'| \ge \delta N/8$ and $e_\Gamma(U', V') \ge \delta N|U'|/4$. With probability at least $1 - \eta$, it returns sets and every $(u, b) \in U' \times V'$ has at least $\delta^5 N^2/2^{22}$ three-edge walks u, z, w, b. Vertices may repeat. Writing $\ell = \log(2N/(\delta\eta))$, the running time is $O(N\delta^{-4}\ell^2 + \delta^{-6}\ell^3 + N|U'|)$. The last term is omitted if the procedure returns $\perp$.*

Proof.

The size and edge bounds are deterministic. Any returned U' passes the check $|U'| \ge |W|/2 \ge \delta N/8$. Every $u \in U' \subseteq U_0$ has degree at least $\delta N/2$; discarding right vertices outside V' loses at most $\delta N|U'|/4$ edges. Thus $e_\Gamma(U', V') \ge \delta N|U'|/4$. In particular, neither returned set contains an isolated padding vertex.

The sampled relation is reliable. Apply Lemma 259 with threshold 2θ to each of the N^2 pairs, using the shared list Q. A union bound shows that, with probability at least $1 - \eta/4$, every flagged pair has at most $4\theta N$ common neighbors, and every unflagged pair has at least θN. Condition on such a list Q.

Algorithm 27 BsgPiece(Γ, δ, η): sampling a structured product

Input: Adjacency matrix and left degrees of Γ, density lower bound $0 < \delta \le 1$, failure allowance $0 < \eta < 1$
Output: A pair (U', V'), or $\perp$

$U_0 \leftarrow \{u : \deg_\Gamma(u) \ge \delta N/2\}$, $\theta \leftarrow \delta^3/2^{16}$
Sample a list Q of $\lceil 4\theta^{-1} \log(8N^2/\eta) \rceil$ points of V
Define bad by the displayed formula above
$M \leftarrow \lceil 16\delta^{-2} \log(4/\eta) \rceil$, $\zeta \leftarrow \eta/(4(M+N+1))$
for $j = 1, \ldots, M$ **do**
    Choose $v \in V$; form the list $W \leftarrow U_0 \cap N_\Gamma(v)$
    If $|W| < \delta N/4$, continue
    If Low$(W \times W, \text{bad}, \delta/256, \zeta)$ is false, continue
    $U' \leftarrow \{u \in W : \text{Low}(W, \text{bad}(u, \cdot), \delta/32, \zeta)\}$
    if $|U'| < |W|/2$ **then**
        return $\perp$
    Compute $d(b) = \sum_{u \in U'} \Gamma_{u,b}$ for all $b \in V$, by scanning $U' \times V$
    $V' \leftarrow \{b : d(b) \ge \delta |U'|/4\}$
    return (U', V')
return $\perp$

A useful neighborhood occurs often enough. Deleting the low-degree left vertices removes at most $\delta N^2/2$ edges. For uniform $v \in V$, therefore, $\mathbb{E}|W| \ge \delta N/2$ and $\mathbb{E}|W|^2 \ge \delta^2 N^2/4$. Let $b(W)$ count flagged ordered pairs in W^2. Each flagged pair appears in W^2 with probability at most 4θ, so $\mathbb{E}b(W) \le 4\theta N^2$. Consequently the score $Z = |W|^2 - 512b(W)/\delta$ satisfies $\mathbb{E}Z \ge 7\delta^2 N^2/32$, while $Z \le N^2$. Since $\mathbb{E}Z \le \delta^2 N^2/8 + N^2 \Pr(Z \ge \delta^2 N^2/8)$, a neighborhood with $Z \ge \delta^2 N^2/8$ occurs with probability at least $\delta^2/16$. Such a neighborhood has $|W| \ge \delta N/4$ and $b(W)/|W|^2 \le \delta/512$, so its first Low test accepts whenever accurate. The probability that none of the M trials produces one is at most $\eta/4$.

Pruning makes the conclusion hold for every endpoint. There are at most $M + N$ further sampling tests. Conditional on their inputs and all previous randomness, each uses fresh samples and fails with probability at most ζ. Their total failure probability is at most $\eta/4$. Assume all are accurate. An accepted W has $b(W)/|W|^2 \le \delta/128$. At least half its vertices have at most $\delta|W|/64$ bad partners and hence survive. Every survivor has at most $\delta|W|/16$ bad partners. Thus $|U'| \ge |W|/2 \ge \delta N/8$.

For $u \in U'$ and $b \in V'$, there are at least $\delta|U'|/4 \ge \delta|W|/8$ neighbors $w \in U'$ of b. At least $\delta|W|/16$ of these are unflagged partners of u. Each supplies at least θN choices of z, giving $(\delta|W|/16)\theta N \ge \delta^5 N^2/2^{22}$ walks. The three failure allowances sum to at most $3\eta/4$.

Cost. One bad-pair query scans Q, costing $O(\delta^{-3}\ell)$. A threshold test on W^2 or on one vertex's partners uses $O(\delta^{-1}\ell)$ such queries. There are $O(\delta^{-2}\ell)$ neighborhood trials and at most N partner tests after one is accepted. Forming the trial neighborhoods costs $O(N\delta^{-2}\ell)$; the final exact degree scan costs $O(N|U'|)$. These bounds give the stated time. □

From walks to sums. Suppose $U = A$, $V = B$, and every edge of Γ has label $a + b \in S$, where $|S| \le N$. For each $t \in U' + V'$, fix one pair (a, b) with $a + b = t$. A walk a, z, w, b gives the three labels $(a + z, w + z, w + b) \in S^3$, whose alternating sum is t. For the fixed endpoints, these labels recover z and w. Thus distinct walks give distinct triples, and triples used for different t's are disjoint. The walk bound proves $|U' + V'|\delta^5 N^2/2^{22} \le |S|^3 \le N^3$, or $|U' + V'| \le C_0 \delta^{-5} N$.

We do not need to verify the walk counts. Enumerating $U' \times V'$ and sorting its sums verifies the desired conclusion directly.

Peeling and exact verification. Build the graph $\Gamma_0 = \{(a, b) : a + b \in S\}$, using sorted S for membership tests. Store its adjacency matrix, all left degrees, and its edge count. During a pass, remove the edges covered by each returned product, and update these quantities as the removed entries are scanned. A restart begins again from Γ_0.

Algorithm 28 BSG decomposition

Input: A, B, S, each of size at most N, and $0 < \varepsilon < 1$
Output: The decomposition in Theorem 258

if $C_0\varepsilon^{-5} \geq N$ **then**
  Enumerate $A \times B$; sort and deduplicate its sums to form $T = A + B$
  return the single triple (A, B, T) and $R = \varnothing$
Build Γ_0, padding the parts to size N with isolated vertices
repeat
  $\Gamma \leftarrow \Gamma_0$, $m \leftarrow$ its edge count, $\mathcal{L} \leftarrow [\,]$, *failed* $\leftarrow$ false
  while $m > \varepsilon N^2$ **do**
    $\delta \leftarrow m/N^2$
    $(X, Y) \leftarrow \textsc{BsgPiece}(\Gamma, \delta, \varepsilon/1024)$
    if the call returns $\perp$ **then**
      failed $\leftarrow$ true; **break**
    Enumerate $X \times Y$; sort and deduplicate its sums to form $T = X + Y$
    if $|T| > C_0\delta^{-5}N$ **then**
      failed $\leftarrow$ true; **break**
    Append (X, Y, T) to $\mathcal{L}$
    Delete the edges in $X \times Y$, updating degrees and m
until *failed* = false
return $\mathcal{L}$ and the list R of remaining edges

Proof of Theorem 258. If $C_0\varepsilon^{-5} \geq N$, the one-product answer is valid because $|A + B| \leq N^2 \leq C_0\varepsilon^{-5}N$. Assume henceforth $C_0\varepsilon^{-5} < N$. This is also what makes the sampling costs affordable.

Correctness and number of products. Only products passing exact verification are kept. Every kept sumset has size at most $C_0\delta^{-5}N \leq C_0\varepsilon^{-5}N$, and the remaining edges are exactly the required set R. Let m_i be the edge count before a successful iteration and put $\delta_i = m_i/N^2$. The deterministic graph bounds ensure that at least $\delta_i N|A_i|/4 \geq \delta_i^2 N^2/32$ edges are removed. Thus, while the next density is positive, $\delta_{i+1} \leq \delta_i(1 - \delta_i/32)$, giving $1/\delta_{i+1} \geq 1/\delta_i + 1/32$. Before each iteration $1/\delta_i < 1/\varepsilon$, so there are at most $\lceil 32/\varepsilon \rceil$ kept products. An aborted pass has at most one additional, failed call.

Why exact scans do not accumulate a factor $1/\varepsilon$. The stronger edge-removal guarantee gives $m_{i+1} \leq m_i(1 - |A_i|/(4N)) \leq m_i e^{-|A_i|/(4N)}$. Multiplying these inequalities over all but the final kept product, then using $|A_i| \leq N$ for that final product, shows $\sum_i |A_i| \leq N + 4N\log(1/\varepsilon)$. Indeed, before that final iteration the graph still has more than εN^2 edges. This argument also bounds every verified prefix of an aborted pass; its final failed candidate adds at most one N-vertex scan.

Consequently all exact degree scans, product enumerations, and edge deletions in one pass inspect $O(N^2(1 + \log(1/\varepsilon)))$ pairs in total. Sorting the resulting sum lists adds a factor $O(\log(2N))$. This also accounts for explicitly producing every T_i.

Sampling time. There are $O(1/\varepsilon)$ calls, each with $\delta \geq \varepsilon$ and $\eta = \varepsilon/1024$. Apart from the exact scans already charged, their total time is

$$O\big(N\varepsilon^{-5}\ell^2 + \varepsilon^{-7}\ell^3\big), \qquad \ell = \log(2N/\varepsilon).$$

The nontrivial-case assumption implies $\varepsilon^{-5} < N$, hence $N\varepsilon^{-5} < N^2$, $\varepsilon^{-7} < N^{7/5}$, and $\ell = O(\log(2N))$. The smaller term $N^{7/5}\log^3(2N)$ is $O(N^2\log^2(2N))$, since $\log(2N) = O(N^{3/5})$. Thus every pass costs $O(N^2\log^2(2N))$, including initialization and a possible final failed candidate.

Restarts. Conditional on any preceding history, a fresh call has failure probability at most $\varepsilon/1024$, by Lemma 260 and the label-counting argument. There are fewer than $34/\varepsilon$ calls in a pass, so a union bound makes its failure probability less than $1/16$. Independent passes therefore finish after fewer than two attempts in expectation. Exact verification guarantees correctness even when some sampling tests err, proving the Las Vegas claim. □

The useful distinction from a purely existential BSG statement is that the full lists T_i are produced within the same bound.

7.9.16 Snevily's Conjecture

Let G be an abelian group, written additively. Given two k-element sets

$$A = \{a_1, \ldots, a_k\}, \qquad B = \{b_1, \ldots, b_k\},$$

we would like to match the elements of A and B so that all matched sums are distinct.

Theorem 261 (Snevily)**.** *If G is a finite abelian group of odd order, then there is a permutation $\pi \in S_k$ such that*

$$a_1 + b_{\pi(1)}, \ldots, a_k + b_{\pi(k)}$$

are pairwise distinct.

Characters turn group addition into multiplication:

$$\chi(a+b) = \chi(a)\chi(b).$$

We shall choose characters which are simultaneously independent on A and on B. In characteristic 2, multiplying the two resulting determinants expands into a sum over all matchings π. If every matching repeated a sum, every determinant in this expansion would vanish, contradicting the nonzero product.

There are only two preliminary ingredients.

Characters form a basis. Let m be the exponent of G. Since m is odd, choose a finite field F of characteristic 2 containing an element of order m. Then all characters

$$\widehat{G} = \mathrm{Hom}(G, F^\times)$$

are defined over F, and $|\widehat{G}| = |G|$.

For distinct characters χ, ψ,

$$\sum_{g \in G} \chi(g)\psi(g)^{-1} = 0.$$

Indeed, choose h with $\chi(h) \neq \psi(h)$ and multiply the sum by $\chi(h)\psi(h)^{-1}$; translation by h leaves the sum unchanged. For $\chi = \psi$, the sum is $|G| \neq 0$ in characteristic 2, since $|G|$ is odd. Thus the character table is nonsingular, so the characters form a basis of the vector space of functions $G \to F$, and remain a basis over every extension field of F.

A matching with a unique multiset of sums.

Lemma 262. *There is a permutation $\pi \in S_k$ such that no other permutation σ gives the same multiset*

$$\{a_1 + b_{\pi(1)}, \dots, a_k + b_{\pi(k)}\}.$$

Proof. Induct on k. Put $g = a_1 + b_1$, and let I be the set of indices i for which $a_i + b_j = g$ for some j. Relabel so that $I = \{1, \dots, \ell\}$. For each $i \le \ell$ this j is unique, and the resulting j's are distinct, so fix $\pi(i)$ by

$$a_i + b_{\pi(i)} = g.$$

If $\ell = k$, this determines π uniquely and all its sums equal g. Otherwise apply induction to the remaining elements of A and the unused elements of B.

No remaining element of A can form the sum g with any element of B. Hence any permutation producing the same multiset must use exactly the same ℓ pairs producing g, and induction then forces all the remaining pairs as well. □

The lemma lets us obtain characters which are simultaneously independent on A and B.

Lemma 263 (Common character basis)**.** *There are characters $\chi_1, \dots, \chi_k \in \widehat{G}$ such that both matrices*

$$\big(\chi_i(a_j)\big)_{i,j=1}^{k}, \qquad \big(\chi_i(b_j)\big)_{i,j=1}^{k}$$

are nonsingular.

Proof. Adjoin algebraically independent variables t_g, one for each $g \in G$, and work over

$$K = F(t_g : g \in G).$$

Define $\varphi : G \to K$ by $\varphi(g) = t_g$. Since the characters form a basis,

$$\varphi = \sum_{\chi \in \widehat{G}} \lambda_\chi \chi$$

for suitable $\lambda_\chi \in K$.

Consider

$$L = \big(\varphi(a_i + b_j)\big)_{i,j=1}^{k}.$$

Writing

$$X_{i,\chi} = \lambda_\chi \chi(a_i), \qquad Y_{j,\chi} = \chi(b_j),$$

multiplicativity of the characters gives

$$L = XY^{\top}.$$

Hence Cauchy–Binet gives

$$\det L = \sum_{\substack{U \subseteq \widehat{G} \\ |U| = k}} \left(\prod_{\chi \in U} \lambda_\chi \right) \det\big(\chi(a_i)\big)_{\substack{i \in [k] \\ \chi \in U}} \det\big(\chi(b_j)\big)_{\substack{j \in [k] \\ \chi \in U}}.$$

Thus if no common character basis existed, then $\det L = 0$.

On the other hand,

$$\det L = \sum_{\pi \in S_k} \operatorname{sgn}(\pi) \prod_{i=1}^{k} t_{a_i + b_{\pi(i)}}.$$

A monomial here records exactly the multiset of matched sums. By Lemma 262, one permutation produces a monomial which occurs nowhere else, so it cannot cancel. Therefore $\det L \neq 0$, a contradiction. □

Proof of Theorem 261. Choose the characters from Lemma 263, and put

$$X = (\chi_i(a_j))_{i,j}, \qquad Y = (\chi_i(b_j))_{i,j}.$$

Both determinants are nonzero.

In characteristic 2, signs in a determinant disappear. Expanding the two determinants and reindexing gives the identity

$$\det X \, \det Y = \sum_{\pi \in S_k} \det\bigl(\chi_i(a_j + b_{\pi(j)})\bigr)_{i,j=1}^{k}. \tag{69}$$

Indeed, a term on the right is

$$\prod_j \chi_{\sigma(j)}(a_j)\chi_{\sigma(j)}(b_{\pi(j)})$$

for some $\sigma, \pi \in S_k$. Reindexing the second product by $\ell = \pi(j)$ gives an arbitrary pair of permutation terms from $\det X$ and $\det Y$, and this correspondence is bijective.

If the theorem were false, then for every π two of the elements

$$a_j + b_{\pi(j)}$$

would coincide. The corresponding two columns of the determinant on the right of Equation (69) would be equal, so every term would vanish. This would give

$$\det X \, \det Y = 0,$$

contradicting the choice of the characters. □

7.10 Enumerative and Algebraic Combinatorics

7.10.1 Lindström–Gessel–Viennot Lemma

The LGV lemma is fundamentally a sign-reversing involution. The usual edge-weighted version is a special case of the following slightly more general statement.

Let G be a finite directed acyclic graph and let R be a commutative ring. Assign to every directed path P a weight

$$w(P) \in R.$$

We only require the following *tail-switching property.* Suppose two paths P, Q meet at a vertex v, and write

$$P = P^- P^+, \qquad Q = Q^- Q^+$$

at v. Let

$$P' = P^- Q^+, \qquad Q' = Q^- P^+.$$

Then require

$$\boxed{w(P)w(Q) = w(P')w(Q').}$$

Ordinary multiplicative edge weights

$$w(P) = \prod_{e \in P} w(e)$$

automatically satisfy this condition.

Let

$$a_1, \dots, a_k, \qquad b_1, \dots, b_k$$

be ordered vertices. Define the path matrix

$$M_{ij} = \sum_{P : a_i \to b_j} w(P).$$

For a permutation $\sigma \in S_k$, let $\mathcal{N}_\sigma$ be the set of k-tuples

$$(P_1, \dots, P_k)$$

of pairwise vertex-disjoint directed paths satisfying

$$P_i : a_i \to b_{\sigma(i)}.$$

Define

$$w(P_1, \dots, P_k) = \prod_i w(P_i).$$

Theorem 264 (Lindström–Gessel–Viennot)**.** *With the notation above,*

$$\boxed{\det M = \sum_{\sigma \in S_k} \operatorname{sgn}(\sigma) \sum_{\mathbf{P} \in \mathcal{N}_\sigma} w(\mathbf{P}).}$$

Proof. Expand the determinant:

$$\det M = \sum_{\sigma \in S_k} \operatorname{sgn}(\sigma) \prod_{i=1}^{k} \left(\sum_{P_i : a_i \to b_{\sigma(i)}} w(P_i) \right).$$

Thus $\det M$ is the signed weighted sum over *all* path families

$$(P_1, \dots, P_k),$$

including intersecting ones.

We cancel the intersecting families by an involution.

Fix a topological ordering of the vertices of G. For an intersecting family, choose the earliest vertex v in this order that lies on at least two paths. Among the paths containing v, choose the first pair

$$P_i, P_j$$

according to some fixed ordering of the indices.

Swap their tails after v:

$$P_i = P_i^- P_i^+, \qquad P_j = P_j^- P_j^+$$

becomes

$$P_i' = P_i^- P_j^+, \qquad P_j' = P_j^- P_i^+.$$

Because G is acyclic, the switched objects are still directed paths. The endpoints $b_{\sigma(i)}$ and $b_{\sigma(j)}$ are exchanged, so the new permutation is

$$\sigma \circ (i\, j),$$

which has the opposite sign.

By the tail-switching assumption,

$$w(P_i)w(P_j) = w(P_i')w(P_j'),$$

so the total weight is unchanged.

The chosen first intersection and pair are unchanged by performing the swap again, so this operation is an involution.

Hence every intersecting family cancels with another family of equal weight and opposite sign. The only uncancelled terms are the pairwise vertex-disjoint families, giving the stated formula. □

A particularly common special case occurs when the sources and sinks are *compatible*: no vertex-disjoint path family can connect

$$a_i \to b_{\sigma(i)}$$

for any nonidentity permutation σ. Then

$$\boxed{\det M = \sum_{\substack{P_i : a_i \to b_i \\ P_1,\dots,P_k \text{ pairwise disjoint}}} \prod_i w(P_i).}$$

Thus a determinant directly enumerates nonintersecting path families.

7.10.2 Burnside's Lemma and Pólya Enumeration

Suppose a finite group acts on a family of objects. Objects in the same orbit are regarded as equivalent. Simply dividing the number of objects by the group size is generally wrong, because different objects may have different numbers of symmetries. Burnside's lemma corrects this by counting each orbit through the symmetries that fix its elements.

Pólya enumeration applies this principle when a group permutes a finite set of positions and each position receives an object from some weighted family. A decoration fixed by a permutation must be constant on each cycle. Thus a cycle of length j contributes the inventory of one object with every weight multiplied by j. The cycle index records exactly how these contributions combine.

Burnside's lemma Let a finite group G act on a set Ω. For $x \in \Omega$, write

$$G_x = \{g \in G : gx = x\}$$

for its stabilizer, and for $g \in G$, write

$$\mathrm{Fix}(g) = \{x \in \Omega : gx = x\}.$$

For every orbit $O = Gx$, the orbit–stabilizer theorem gives

$$|O|\,|G_x| = |G|.$$

Theorem 265 (Weighted Burnside lemma)**.** *Let a finite group G act on a set Ω, and let*

$$w : \Omega \to R$$

be constant on orbits, where R is a commutative $\mathbb{Q}$-algebra. Assume all sums below are defined. Then

$$\boxed{\sum_{O \in \Omega/G} w(O) = \frac{1}{|G|} \sum_{g \in G} \sum_{x \in \mathrm{Fix}(g)} w(x).}$$

In particular,

$$\boxed{|\Omega/G| = \frac{1}{|G|} \sum_{g \in G} |\,\mathrm{Fix}(g)|.}$$

Proof. Group the fixed pairs by the orbit of x. If O is one orbit, then every $x \in O$ has the same weight $w(O)$ and stabilizers of the same size. Its total contribution to the double sum is therefore

$$\sum_{x \in O} |G_x| w(x) = |O|\,|G_x|\,w(O) = |G| w(O).$$

Summing over all orbits and dividing by $|G|$ proves the result. □

For generating functions, it is useful to allow Ω to be infinite but graded. If every coefficient receives contributions from only finitely many objects, the same proof applies coefficientwise.

Cycle indices and weighted decorations Let G act on a finite set D of n positions. For $g \in G$, let $c_j(g)$ be the number of j-cycles in its permutation of D. The *cycle index* of the action is

$$\boxed{Z_G(u_1, \dots, u_n) = \frac{1}{|G|} \sum_{g \in G} \prod_{j=1}^{n} u_j^{c_j(g)}.}$$

Now let $\mathcal{A}$ be a weighted family. Give each $a \in \mathcal{A}$ a weight vector

$$\mathrm{wt}(a) = (\alpha_1(a), \dots, \alpha_r(a)) \in \mathbb{N}^r,$$

and define its multivariate generating function

$$A(\mathbf{x}) = A(x_1, \dots, x_r) = \sum_{a \in \mathcal{A}} x_1^{\alpha_1(a)} \cdots x_r^{\alpha_r(a)}.$$

Assume each coefficient of A is finite.

A decoration of D is a function

$$f : D \to \mathcal{A}.$$

Its weight is the product of the weights placed at its positions:

$$\mathrm{wt}(f) = \prod_{v \in D} \mathbf{x}^{\mathrm{wt}(f(v))} = \mathbf{x}^{\sum_{v \in D} \mathrm{wt}(f(v))}.$$

The action of G is

$$(g \cdot f)(v) = f(g^{-1}v).$$

For $j \geq 1$, write

$$\mathbf{x}^j = (x_1^j, \dots, x_r^j).$$

Theorem 266 (Pólya enumeration)**.** *With the preceding notation,*

$$\boxed{\sum_{[f] \in \mathcal{A}^D/G} \mathrm{wt}(f) = Z_G\big(A(\mathbf{x}), A(\mathbf{x}^2), \dots, A(\mathbf{x}^n)\big).}$$

Equivalently,

$$\sum_{[f] \in \mathcal{A}^D/G} \mathrm{wt}(f) = \frac{1}{|G|} \sum_{g \in G} \prod_{j=1}^{n} A(\mathbf{x}^j)^{c_j(g)}.$$

Proof. Fix $g \in G$. A decoration f is fixed by g exactly when it is constant on every cycle of g.

Consider one cycle of length j. If it receives the object $a \in \mathcal{A}$, then a appears at all j positions, so its contribution to the total weight is

$$\left(\mathbf{x}^{\mathrm{wt}(a)}\right)^j = \mathbf{x}^{j\,\mathrm{wt}(a)}.$$

Summing over the possible objects placed on this cycle gives

$$\sum_{a \in \mathcal{A}} \mathbf{x}^{j\,\mathrm{wt}(a)} = A(\mathbf{x}^j).$$

Different cycles may be decorated independently. Hence the weighted inventory of decorations fixed by g is

$$\prod_{j=1}^{n} A(\mathbf{x}^j)^{c_j(g)}.$$

Weighted Burnside now gives the stated average. □

For example, if

$$\mathcal{A} = \{a_1, \ldots, a_r\}, \qquad \mathrm{wt}(a_i) = e_i,$$

then

$$A(x_1, \ldots, x_r) = x_1 + \cdots + x_r,$$

and the coefficient of $x_1^{n_1} \cdots x_r^{n_r}$ counts orbits in which object type a_i occurs exactly n_i times.

If instead $\mathcal{A}$ has two objects of weights 0 and 1, then $A(z) = 1 + z$, and

$$Z_G(1 + z, 1 + z^2, \ldots, 1 + z^n)$$

counts G-orbits of subsets of D by cardinality.

Necklaces Let the cyclic group C_n act by rotation on n cyclically ordered positions. A rotation by s positions has $\gcd(n, s)$ cycles, each of length

$$\frac{n}{\gcd(n, s)}.$$

Equivalently, a rotation of order $d \mid n$ consists of n/d cycles of length d, and there are $\varphi(d)$ such rotations.

Theorem 267 (Necklace cycle index)**.** *For the cyclic action on n positions,*

$$\boxed{Z_{C_n}(u_1, \ldots, u_n) = \frac{1}{n} \sum_{d \mid n} \varphi(d) u_d^{n/d}.}$$

Consequently, for a weighted family with generating function $A(\mathbf{x})$, the generating function of cyclic sequences of n objects from $\mathcal{A}$, modulo rotation, is

$$\boxed{N_n(\mathbf{x}) = \frac{1}{n} \sum_{d \mid n} \varphi(d) A(\mathbf{x}^d)^{n/d}.}$$

Proof. Group the rotations according to their order d. There are $\varphi(d)$ rotations of order d, and each has n/d cycles of length d. This gives the cycle index. The second formula is the substitution

$$u_d \mapsto A(\mathbf{x}^d).$$

□

If $\mathcal{A}$ consists of r unweighted object types, then $A = r$, and the number of necklaces is

$$\boxed{\frac{1}{n} \sum_{d \mid n} \varphi(d) r^{n/d}.}$$

If instead the r types have independent weights $x_1, \ldots, x_r$, then

$$A(\mathbf{x}) = x_1 + \cdots + x_r.$$

For prescribed multiplicities

$$n_1 + \cdots + n_r = n,$$

the coefficient of $x_1^{n_1} \cdots x_r^{n_r}$ is

$$\boxed{\frac{1}{n} \sum_{d \mid \gcd(n_1, \ldots, n_r)} \varphi(d) \frac{(n/d)!}{(n_1/d)! \cdots (n_r/d)!}.}$$

Indeed,

$$[x_1^{n_1} \cdots x_r^{n_r}](x_1^d + \cdots + x_r^d)^{n/d}$$

vanishes unless d divides every n_i, and otherwise is the displayed multinomial coefficient.

The symmetric group and unlabelled multisets For S_n acting on n positions, suppose a permutation has m_j cycles of length j. Then

$$\sum_{j\geq 1} jm_j = n,$$

and the number of permutations with these cycle counts is

$$\frac{n!}{\prod_{j\geq 1} j^{m_j} m_j!}.$$

Therefore

$$\boxed{Z_{S_n} = \sum_{\substack{m_1,m_2,\ldots\geq 0 \\ \sum_j jm_j = n}} \prod_{j\geq 1} \frac{u_j^{m_j}}{j^{m_j} m_j!}.}$$

Summing over all n,

$$\begin{aligned} \sum_{n\geq 0} Z_{S_n}(u_1, u_2, \ldots)t^n &= \prod_{j\geq 1} \sum_{m\geq 0} \frac{1}{m!} \left(\frac{u_j t^j}{j}\right)^m \\ &= \boxed{\exp\left(\sum_{j\geq 1} \frac{u_j t^j}{j}\right).} \end{aligned} \tag{1}$$

Now let

$$A(\mathbf{x}) = \sum_{\alpha\in\mathbb{N}^r\setminus\{0\}} a_\alpha \mathbf{x}^\alpha$$

be the generating function of a family of nonempty unlabelled object types. An unordered multiset of n such objects is precisely an orbit of an ordered n-tuple under S_n. Substituting

$$u_j = A(\mathbf{x}^j)$$

into (1), and then setting $t = 1$, gives the inventory of arbitrary finite multisets:

$$\boxed{M(\mathbf{x}) = \exp\left(\sum_{j\geq 1} \frac{A(\mathbf{x}^j)}{j}\right).}$$

Expanding A,

$$\begin{aligned} \sum_{j\geq 1} \frac{A(\mathbf{x}^j)}{j} &= \sum_\alpha a_\alpha \sum_{j\geq 1} \frac{\mathbf{x}^{j\alpha}}{j} \\ &= -\sum_\alpha a_\alpha \log(1 - \mathbf{x}^\alpha). \end{aligned}$$

Hence

$$\boxed{M(\mathbf{x}) = \prod_\alpha (1 - \mathbf{x}^\alpha)^{-a_\alpha}.}$$

In one variable, if

$$A(z) = \sum_{m\geq 1} a_m z^m,$$

this becomes

$$\boxed{M(z) = \exp\left(\sum_{j\geq 1} \frac{A(z^j)}{j}\right) = \prod_{m\geq 1} (1 - z^m)^{-a_m}.}$$

The condition $A(\mathbf{0}) = 0$ ensures that these infinite sums and products are well-defined coefficientwise.

This is the unlabelled analogue of the exponential formula. For labelled sets of components one obtains $\exp(A(z))$; for unlabelled multisets, repeated identical components produce the extra terms

$$\frac{A(z^2)}{2}, \frac{A(z^3)}{3}, \dots.$$

Application: unlabelled graphs A simple graph on $[n]$ is determined by a subset of

$$E_n = \binom{[n]}{2}.$$

The symmetric group S_n acts on vertices and therefore induces an action on E_n. Two graphs are isomorphic exactly when their edge sets lie in the same orbit of this induced action.

Burnside's lemma reduces the problem to counting the orbits of each vertex permutation on E_n. If a permutation has vertex-cycle lengths

$$\lambda = (\lambda_1, \dots, \lambda_r) \vdash n,$$

let m_j denote the number of parts of λ equal to j, and put

$$z_\lambda = \prod_{j \geq 1} j^{m_j} m_j!.$$

There are

$$\frac{n!}{z_\lambda}$$

permutations of cycle type λ.

We first count the induced cycles on unordered pairs.

Between two distinct vertex cycles of lengths a and b, the orbit of a pair has length

$$\operatorname{lcm}(a, b).$$

Since there are ab such pairs, the number of pair-orbits between the two cycles is

$$\frac{ab}{\operatorname{lcm}(a, b)} = \gcd(a, b).$$

Within a vertex cycle of length a, an unordered pair is determined up to rotation by its cyclic distance

$$1, \dots, \left\lfloor \frac{a}{2} \right\rfloor.$$

Thus there are

$$\left\lfloor \frac{a}{2} \right\rfloor$$

pair-orbits inside that cycle.

Define

$$e(\lambda) = \sum_{i=1}^{r} \left\lfloor \frac{\lambda_i}{2} \right\rfloor + \sum_{1 \leq i < j \leq r} \gcd(\lambda_i, \lambda_j).$$

Theorem 268 (Unlabelled graph count)**.** *The number g_n of isomorphism classes of simple graphs on n vertices is*

$$\boxed{g_n = \sum_{\lambda \vdash n} \frac{2^{e(\lambda)}}{z_\lambda}.}$$

Proof. Fix a permutation of cycle type λ. By the preceding count, its induced action on E_n has exactly $e(\lambda)$ orbits.

An edge set is fixed by the permutation exactly when it is a union of these edge-orbits. Each orbit may independently be included or omitted, so the number of fixed graphs is

$$2^{e(\lambda)}.$$

There are $n!/z_\lambda$ permutations of this cycle type. Burnside's lemma therefore gives

$$\begin{aligned} g_n &= \frac{1}{n!}\sum_{\lambda\vdash n}\frac{n!}{z_\lambda}2^{e(\lambda)} \\ &= \sum_{\lambda\vdash n}\frac{2^{e(\lambda)}}{z_\lambda}. \end{aligned}$$

□

The same calculation can retain the number of edges. If the induced edge-orbits have lengths

$$\ell_1,\dots,\ell_m,$$

then the fixed graphs contribute

$$\prod_{i=1}^{m}(1+z^{\ell_i}),$$

because including the i-th orbit contributes all ℓ_i of its edges.

For two distinct vertex cycles of lengths a, b, there are $\gcd(a,b)$ edge-orbits, each of length

$$\operatorname{lcm}(a,b).$$

Inside a vertex cycle of length a, each distance

$$d < \frac{a}{2}$$

gives one orbit of length a; when a is even, the opposite pairs $d = a/2$ form one orbit of length $a/2$. Thus the same Burnside calculation gives the ordinary generating polynomial for unlabelled graphs on n vertices by number of edges.

7.10.3 Q-analogues

A useful q-analogue replaces subsets by subspaces: cardinality becomes dimension, intersection becomes intersection of subspaces, and disjointness means intersection $\{0\}$. The extra powers of q count free choices of linear maps. The same Gaussian coefficients also enumerate partitions in a rectangle when q is a formal variable.

In statements about $\mathbb{F}_q$, q is a prime power. In generating identities, q is an indeterminate. The finite identities are polynomial identities and hence hold in both interpretations; the infinite products below are interpreted as formal series, or for $|q| < 1$ where appropriate.

Count Bases. Define $[r]_q = 1 + q + \cdots + q^{r-1}$ and $[r]_q! = \prod_{j=1}^{r}[j]_q$, with $[0]_q! = 1$. The Gaussian coefficient is

$$\binom{n}{k}_q = \frac{[n]_q!}{[k]_q![n-k]_q!} = \prod_{i=0}^{k-1}\frac{q^{n-i}-1}{q^{k-i}-1} \qquad (0 \le k \le n).$$

Set it to zero for $k < 0$ or $k > n$. At $q = 1$, its value is $\binom{n}{k}$.

Theorem 269 (Subspace count)**.** *An n-dimensional space over $\mathbb{F}_q$ has exactly $\binom{n}{k}_q$ subspaces of dimension k.*

Proof. An ordered independent k-tuple in $\mathbb{F}_q^n$ has $\prod_{i=0}^{k-1}(q^n - q^i)$ choices: at stage i, exclude the q^i vectors already in the span. Each k-space has $\prod_{i=0}^{k-1}(q^k - q^i)$ ordered bases. Divide these two products and cancel their powers of q. □

Orthogonal complementation gives $\binom{n}{k}_q = \binom{n}{n-k}_q$. Splitting according to containment in a fixed hyperplane gives the two Pascal recurrences

$$\boxed{\binom{n}{k}_q = \binom{n-1}{k}_q + q^{n-k}\binom{n-1}{k-1}_q = \binom{n-1}{k-1}_q + q^k\binom{n-1}{k}_q.} \tag{70}$$

Indeed, a k-space not contained in the hyperplane meets it in a $(k-1)$-space S. In the quotient by S, its remaining choice is a line outside a hyperplane of an $(n-k+1)$-space. There are $[n-k+1]_q - [n-k]_q = q^{n-k}$ such lines. Symmetry gives the other form.

How large is the Gaussian coefficient? For a numerical prime power $q \geq 2$,

$$\boxed{q^{k(n-k)} \leq \binom{n}{k}_q \leq 4q^{k(n-k)}.} \tag{71}$$

Factor out $q^{k(n-k)}$ from the product formula. Each resulting numerator factor is at least its corresponding denominator factor, giving the lower bound. For the upper bound, discard the numerator factors and use

$$\prod_{j=1}^{\infty}(1 - q^{-j}) \geq \frac{1}{2} \cdot \frac{3}{4}\left(1 - \sum_{j=3}^{\infty} 2^{-j}\right) = \frac{9}{32} > \frac{1}{4}.$$

Here $\prod(1 - a_j) \geq 1 - \sum a_j$ follows by induction for finite products and then by limits. Thus a k-space has roughly $k(n-k)$ freely chosen field coordinates.

Lemma 270 (Prescribed intersection dimension)**.** *Fix an a-dimensional subspace $H \leq V = \mathbb{F}_q^n$. The number of k-spaces W with $\dim(W \cap H) = r$ is*

$$\boxed{\binom{a}{r}_q \binom{n-a}{k-r}_q q^{(a-r)(k-r)}.}$$

The expression is understood as zero outside the feasible dimensions.

Proof. Choose $S = W \cap H$, giving $\binom{a}{r}_q$ choices, and fix a complement $V = H \oplus K$. Projection to K sends W to a $(k-r)$-space L, with $\binom{n-a}{k-r}_q$ choices. In $V/S = (H/S) \oplus K$, the image of W is disjoint from H/S and projects bijectively onto L. It is therefore the graph of a unique linear map $L \to H/S$. There are $q^{(a-r)(k-r)}$ such maps, each giving exactly one W. □

Summing over r gives a q-Vandermonde identity:

$$\boxed{\binom{a+b}{k}_q = \sum_r q^{(a-r)(k-r)} \binom{a}{r}_q \binom{b}{k-r}_q.}$$

In particular, there are $q^{ak}\binom{n-a}{k}_q$ k-spaces disjoint from a fixed a-space, and $q^{a(n-a)}$ complements to that space.

Flags and a second q-factorial interpretation. If $d_1 + \cdots + d_s = n$, a flag whose successive dimension increases are $d_1, \ldots, d_s$ can be chosen in successive quotients. Its count is

$$\binom{n}{d_1}_q \binom{n-d_1}{d_2}_q \cdots = \frac{[n]_q!}{\prod_i [d_i]_q!}.$$

In particular, the number of complete flags is $[n]_q!$. The same polynomial counts permutations by inversions: $\sum_{\sigma \in S_n} q^{\mathrm{inv}(\sigma)} = [n]_q!$. To prove this, insert the largest label n into a permutation of $[n-1]$. Its position creates each possible number $0, \ldots, n-1$ of new inversions once, multiplying the generating polynomial by $[n]_q$.

Partitions. For a partition λ fitting inside a $k \times (n-k)$ rectangle, write $|\lambda|$ for its number of cells. Then

$$\boxed{\binom{n}{k}_q = \sum_{\lambda \subseteq k \times (n-k)} q^{|\lambda|}.} \tag{72}$$

For a direct proof, split these partitions into those with fewer than k nonzero rows and those with exactly k. The first fit in a $(k-1) \times (n-k)$ rectangle. From each of the second kind, remove its first column of height k, leaving a $k \times (n-k-1)$ rectangle. Thus their generating polynomials satisfy the second recurrence in (70), with the same boundary values. This proves the identity and also shows that Gaussian coefficients are polynomials with nonnegative integer coefficients. The product and recurrence identities, initially justified for prime powers, hold formally because they agree at infinitely many values of q after denominators are cleared.

Theorem 271 (Finite q-binomial theorem)**.** *For $n \geq 0$,*

$$\boxed{\prod_{i=0}^{n-1} (1 + zq^i) = \sum_{k=0}^{n} q^{\binom{k}{2}} \binom{n}{k}_q z^k.}$$

Proof. The coefficient of z^k chooses $0 \leq i_1 < \cdots < i_k \leq n-1$, with weight $q^{i_1 + \cdots + i_k}$. Subtract $0, 1, \ldots, k-1$ from these indices. The resulting weakly increasing entries lie between zero and $n-k$ and, on reversal, describe a partition in the rectangle of (72). The removed staircase has size $\binom{k}{2}$. □

Taking $z = -1$ gives the cancellation identity

$$\sum_{j=0}^{d} (-1)^j q^{\binom{j}{2}} \binom{d}{j}_q = \begin{cases} 1, & d = 0, \\ 0, & d > 0. \end{cases} \tag{73}$$

This is the subspace-lattice Möbius cancellation identity.

Durfee square identity. For ordinary partitions, marking each row by z and each cell by q gives

$$\boxed{\prod_{j \geq 1} \frac{1}{1 - zq^j} = \sum_{r \geq 0} \frac{z^r q^{r^2}}{\prod_{j=1}^{r} (1-q^j) \prod_{j=1}^{r} (1 - zq^j)}.} \tag{74}$$

The left side chooses the multiplicity of each positive part. On the right, cut at the largest upper-left square, of side r. The square contributes $z^r q^{r^2}$. The part to its right has at most r rows and contributes $\prod_{j=1}^{r} (1-q^j)^{-1}$: these cells create no new rows. The part below has parts at most r and contributes $\prod_{j=1}^{r} (1 - zq^j)^{-1}$. Conversely, attaching any such two parts gives a partition with this largest square. This proves the formula.

Random Flags Prove the q-Sperner Theorem. Order all subspaces of $V = \mathbb{F}_q^n$ by inclusion. A complete flag is a chain $\{0\} = V_0 < V_1 < \cdots < V_n = V$ with $\dim V_j = j$. The linear group acts transitively on each dimension level and permutes complete flags. Hence a uniform complete flag contains any given j-space with probability $1/\binom{n}{j}_q$: it contains exactly one of them, and all have the same probability.

Theorem 272 (q-LYM and q-Sperner)**.** *If $\mathcal{A}$ is an antichain of subspaces of $\mathbb{F}_q^n$, then*

$$\boxed{\sum_{U \in \mathcal{A}} \binom{n}{\dim U}_q^{-1} \leq 1, \qquad |\mathcal{A}| \leq \binom{n}{\lfloor n/2 \rfloor}_q.}$$

More generally, a family with no chain of $s+1$ members has size at most the sum of the s largest dimension levels, for $1 \leq s \leq n+1$. All these size bounds are attained by the corresponding full levels.

Proof. An antichain meets a complete flag at most once. Taking expectations proves the first bound. More generally, the same calculation gives a weighted sum at most s when every flag contains at most s members. Let $N_j = \binom{n}{j}_q$ and a_j be the number of family members of dimension j. Thus $0 \leq a_j/N_j \leq 1$ and $\sum_j a_j/N_j \leq s$. The quantity $\sum_j a_j = \sum_j N_j(a_j/N_j)$ is maximized by assigning this budget first to the s largest N_j: moving any available budget from a smaller coefficient to a larger one cannot decrease the sum. This proves the general size bound.

Finally, $N_{j+1}/N_j = (q^{n-j}-1)/(q^{j+1}-1)$, so the levels increase up to the middle and then decrease, and are symmetric by complementation. The largest is therefore $N_{\lfloor n/2 \rfloor}$. A union of s levels has no chain of $s+1$ members, proving sharpness. □

A shadow bound. If $\mathcal{F}$ consists of k-spaces, let $\partial\mathcal{F}$ contain all their $(k-1)$-subspaces. Double-counting containment pairs gives

$$|\partial\mathcal{F}|[n-k+1]_q \geq |\mathcal{F}|[k]_q, \qquad \boxed{\frac{|\partial\mathcal{F}|}{\binom{n}{k-1}_q} \geq \frac{|\mathcal{F}|}{\binom{n}{k}_q}.}$$

Each k-space has $[k]_q$ hyperplanes, and each $(k-1)$-space is in $[n-k+1]_q$ k-spaces.

Intersecting Subspaces and the q-Erdős–Ko–Rado Theorem. An intersecting family of k-spaces has $U \cap W \neq \{0\}$ for every two distinct members. A natural example consists of all k-spaces containing one fixed line. There are $\binom{n-1}{k-1}_q$ of them, by passing to the quotient by that line. We prove this is optimal for $n \geq 2k$. Unlike the antichain condition, intersection is not visible along one random flag; instead, use the graph whose edges represent forbidden disjoint pairs.

Lemma 273 (Spectral bound for an independent set)**.** *Let a simple D-regular graph on N vertices have adjacency matrix A, where $D > 0$. If every eigenvalue of A is at least $-\beta$, with $\beta > 0$, then every independent set has size at most $N\beta/(D+\beta)$.*

Proof. Write its indicator as $z = \alpha\mathbf{1} + h$, where $\alpha = |S|/N$ and $h \perp \mathbf{1}$. Then $\|h\|_2^2 = N\alpha(1-\alpha)$, while independence gives

$$0 = z^\top A z = DN\alpha^2 + h^\top A h \geq DN\alpha^2 - \beta N\alpha(1-\alpha).$$

For $\alpha > 0$, divide by $N\alpha$ and rearrange; the empty set is immediate. □

Let Ω be the set of k-spaces, with $n \geq 2k \geq 2$. The q-Kneser graph joins $W, Z \in \Omega$ when $W \cap Z = \{0\}$. Its degree, by Lemma 270, is $D = q^{k^2}\binom{n-k}{k}_q$. We need its least eigenvalue.

For a j-space J, define the containment indicator $f_J(W) = \mathbf{1}_{J \subseteq W}$, and let U_j be the real span of all such indicators, with $U_{-1} = \{0\}$. These spaces are nested: $U_0 \subseteq U_1 \subseteq \cdots \subseteq U_k = \mathbb{R}^\Omega$. Indeed, summing f_J over all j-spaces containing a fixed $(j-1)$-space L gives $[k-j+1]_q f_L$; at level k the indicators are the individual coordinate vectors.

For fixed J, W, the quantity $(Af_J)(W)$ counts k-spaces containing J and disjoint from W. It is zero if $J \cap W \neq \{0\}$. Otherwise quotienting by J and applying the disjoint-subspace count gives

$$(Af_J)(W) = c_j \mathbf{1}_{J \cap W = \{0\}}, \qquad c_j = q^{k(k-j)} \binom{n-k-j}{k-j}_q .$$

The cancellation identity (73) expands this disjointness indicator into containment indicators:

$$Af_J = c_j \sum_{L \leq J} (-1)^{\dim L} q^{\binom{\dim L}{2}} f_L .$$

Only the term $L = J$ is at level j; every other term lies in U_{j-1}. Consequently A preserves U_j and acts on it modulo U_{j-1} by

$$\lambda_j = (-1)^j q^{k(k-j)+\binom{j}{2}} \binom{n-k-j}{k-j}_q .$$

Because A is symmetric, the orthogonal complement of U_{j-1} inside U_j is also invariant. On this complement, $(A - \lambda_j I)v$ lies both in U_{j-1} and in its orthogonal complement, so it is zero. These complements exhaust $\mathbb{R}^\Omega$, proving that every eigenvalue is among the λ_j.

Their absolute values strictly decrease, since for $0 \leq j < k$,

$$\frac{|\lambda_{j+1}|}{|\lambda_j|} = q^{j-k} \frac{q^{k-j} - 1}{q^{n-k-j} - 1} \leq q^{j-k} < 1.$$

Thus all eigenvalues are at least $\lambda_1 = -q^{k(k-1)} \binom{n-k-1}{k-1}_q$.

Theorem 274 (q-Erdős–Ko–Rado)**.** *For $n \geq 2k$ and $k \geq 1$, an intersecting family $\mathcal{F}$ of k-dimensional subspaces of $\mathbb{F}_q^n$ satisfies*

$$\boxed{|\mathcal{F}| \leq \binom{n-1}{k-1}_q .}$$

The family of all k-spaces containing a fixed line attains equality.

Proof. The family is an independent set of the q-Kneser graph. Apply the spectral bound with $N = \binom{n}{k}_q$, the degree D above, and $\beta = -\lambda_1$. The formula gives

$$\frac{D}{\beta} = q^k \frac{q^{n-k} - 1}{q^k - 1}, \qquad \frac{\beta}{D + \beta} = \frac{q^k - 1}{q^n - 1}.$$

Therefore $|\mathcal{F}| \leq \binom{n}{k}_q (q^k - 1)/(q^n - 1) = \binom{n-1}{k-1}_q$. The fixed-line construction gives the stated equality example. □

When $n < 2k$, all k-spaces intersect because $\dim(U \cap W) \geq 2k - n > 0$, so the whole level is allowed.

Uniform random subspaces. To sample a uniform k-space, choose an $n \times k$ matrix uniformly over $\mathbb{F}_q$, reject it unless its columns are independent, and return their span. Every k-space has the same number $|\operatorname{GL}_k(q)|$ of ordered bases, so the accepted span is uniform. The acceptance probability is

$$\prod_{i=0}^{k-1} (1 - q^{i-n}) \geq \prod_{j=1}^{\infty} (1 - q^{-j}) > \frac{1}{4}.$$

Thus fewer than four trials are needed in expectation. Gaussian elimination uses $O(nk^2)$ field operations per trial for $k \geq 1$.

7.10.4 Jacobi Triple Product

We seek a product expansion for the bilateral series

$$\Theta(x,y)=\sum_{k\in\mathbb{Z}} x^{k^2}y^k.$$

The derivation starts from the square in the exponent. A rectangular version of the Durfee-square decomposition will separate the signed index into two independent nonnegative indices.

Why rectangles appear. Writing $k=r-s$ with $r,s\ge 0$ gives

$$k^2=r^2+s^2-2rs.$$

The square terms separate into one depending on r and one depending on s. The obstruction is the mixed term $-2rs$, which is minus twice the area of an $r\times s$ rectangle.

This suggests attaching an ordinary partition to each k, giving every cell weight x^2, and cutting the partition around a rectangle with $r-s=k$. The rectangle contributes x^{2rs}, exactly canceling the mixed term. To avoid counting the same partition repeatedly, choose the *largest* rectangle with this difference.

The rectangular Durfee identity. Write $|\lambda|$ for the size of a partition and $\ell(\lambda)$ for its number of nonzero parts. Define

$$P(q)=\sum_{\lambda} q^{|\lambda|}=\prod_{j\ge 1}\frac{1}{1-q^j},\qquad P_r(q)=\sum_{\ell(\lambda)\le r} q^{|\lambda|}=\prod_{j=1}^{r}\frac{1}{1-q^j},$$

with $P_0(q)=1$. The first product chooses the multiplicity of each possible part. For the second, transpose the diagram: partitions with at most r rows correspond to partitions whose parts are at most r.

Lemma 275 (Rectangular Durfee identity)**.** *For every integer k,*

$$\boxed{P(q)=\sum_{\substack{r,s\ge 0\\ r-s=k}} q^{rs}P_r(q)P_s(q).}$$

Proof. In a partition λ, choose the largest upper-left rectangle of height r and width s satisfying $r-s=k$. Zero dimensions are allowed, and we append zero rows when necessary. The candidate rectangles have dimensions

$$\big(\max\{k,0\}+t\big)\times\big(\max\{-k,0\}+t\big),\qquad t=0,1,2,\dots.$$

They are nested, the first always fits, and sufficiently large ones do not. Thus there is a unique largest one.

Remove this rectangle. The piece ρ to its right has at most r rows. The piece below it has at most s columns: otherwise $\lambda_{r+1}\ge s+1$, so the next rectangle of dimensions $(r+1)\times(s+1)$ would fit. Transposing the lower piece gives a partition σ with at most s rows. Their sizes satisfy $|\lambda|=rs+|\rho|+|\sigma|$.

Conversely, choose any such ρ and σ. Attach ρ to the right of the rectangle and $\sigma^{\top}$ below it. After padding ρ with zeros, the first r row lengths are $s+\rho_i$, while every lower row has length at most s. Hence this is a partition, and the next rectangle cannot fit. The construction therefore recovers exactly the prescribed maximal rectangle.

The rectangle contributes q^{rs}, and the two remaining pieces are independent, contributing $P_r(q)$ and $P_s(q)$. Summing over its possible dimensions proves the identity. □

For $k=0$, the rectangles are squares, and we recover the usual Durfee-square identity

$$P(q)=\sum_{r\ge 0} q^{r^2}P_r(q)^2.$$

Separating the two indices. Multiply $\Theta(x,y)$ by $P(x^2)$, thereby attaching a partition whose cells have weight x^2. Apply the rectangular identity separately for each k:

$$\begin{aligned} P(x^2)\Theta(x,y) &= \sum_{k\in\mathbb{Z}} x^{k^2}y^k \sum_{\substack{r,s\geq 0\\ r-s=k}} x^{2rs}P_r(x^2)P_s(x^2) \\ &= \sum_{r,s\geq 0} x^{(r-s)^2+2rs}y^{r-s}P_r(x^2)P_s(x^2) \\ &= \sum_{r,s\geq 0} \big(x^{r^2}y^r P_r(x^2)\big)\big(x^{s^2}y^{-s}P_s(x^2)\big). \end{aligned}$$

The rectangle has canceled the mixed term: $(r-s)^2+2rs = r^2+s^2$. The indices are now independent. Thus, defining $A(x,z) = \sum_{r\geq 0} x^{r^2}z^r P_r(x^2)$, we obtain

$$\boxed{P(x^2)\Theta(x,y) = A(x,y)A(x,y^{-1}).} \tag{75}$$

It remains to recognize the series A.

Odd staircases. The factor $P_r(x^2)$ counts partitions with at most r rows, with their sizes doubled. The remaining factor x^{r^2} suggests adding a fixed diagram of size r^2. The useful choice comes from

$$r^2 = 1+3+\cdots+(2r-1)\,:$$

add an odd staircase.

Lemma 276 (Odd-staircase identity)**.** *For every $r\geq 0$,*

$$\sum_{\substack{\alpha \text{ has exactly } r\\ \text{distinct positive odd parts}}} x^{|\alpha|} = x^{r^2}P_r(x^2) = \frac{x^{r^2}}{\prod_{j=1}^{r}(1-x^{2j})}.$$

Consequently,

$$A(x,z) = \prod_{m\geq 1}(1+zx^{2m-1}).$$

Proof. Take a partition ρ with at most r rows and pad it with zeros to have r entries. Double its entries and add the descending odd staircase:

$$\alpha_i = 2\rho_i + 2(r-i)+1 \qquad (1\leq i\leq r).$$

The entries are positive and odd, and $\alpha_i - \alpha_{i+1} = 2(\rho_i-\rho_{i+1})+2\geq 2$, so they are distinct. Moreover,

$$|\alpha| = 2|\rho| + \sum_{i=1}^{r}\big(2(r-i)+1\big) = 2|\rho| + r^2.$$

Conversely, distinct positive odd parts satisfy $\alpha_i \geq 2(r-i)+1$. Subtracting the staircase and halving therefore gives nonnegative, nonincreasing integers, recovering ρ. This proves the first identity, including the empty partition when $r=0$.

Multiplying by z^r and summing over r counts all partitions into distinct odd parts, with z recording their number of parts. Each positive odd integer may independently occur once or not at all, so the generating function is $\prod_{m\geq 1}(1+zx^{2m-1})$. □

Substituting these two factors into (75) now produces the product expansion.

Theorem 277 (Jacobi triple product)**.** *As a formal power series in x with Laurent-polynomial coefficients in y, and analytically for $|x| < 1$ and $y \neq 0$,*

$$\boxed{\sum_{k\in\mathbb{Z}} x^{k^2} y^k = \prod_{m=1}^{\infty} (1 - x^{2m})(1 + yx^{2m-1})(1 + y^{-1}x^{2m-1}).}$$

Proof. The factorization and odd-staircase identity give

$$P(x^2)\Theta(x, y) = \prod_{m\geq 1} (1 + yx^{2m-1})(1 + y^{-1}x^{2m-1}).$$

Multiply by $P(x^2)^{-1} = \prod_{m\geq 1}(1 - x^{2m})$.

All preceding rearrangements are valid coefficientwise. In particular, the smallest power of x contributed by indices r, s in the double sum is $r^2 + s^2$, so only finitely many pairs contribute to any fixed coefficient. The partition series have constant term one and are therefore invertible as formal power series.

For $|x| < 1$ and $y \neq 0$, the series converges absolutely because the quadratic exponent k^2 dominates the linear growth from y^k. The product also converges absolutely, since

$$\sum_{m\geq 1} \left(|x|^{2m} + (|y| + |y|^{-1})|x|^{2m-1}\right) < \infty.$$

Thus the formal identity also gives the stated analytic identity. □

7.10.5 Pentagonal Number Theorem

To obtain $\prod_{m\geq 1}(1 - q^m)$ from the Jacobi triple product, choose its three families of factors to cover the three residue classes modulo 3. The corresponding substitution on the series side produces the pentagonal exponents.

Theorem 278 (Pentagonal number theorem)**.** *As an identity of formal power series, and analytically for $|q| < 1$,*

$$\boxed{\prod_{m=1}^{\infty} (1 - q^m) = \sum_{k\in\mathbb{Z}} (-1)^k q^{k(3k-1)/2}.}$$

Equivalently,

$$\prod_{m=1}^{\infty} (1 - q^m) = 1 + \sum_{k=1}^{\infty} (-1)^k \left(q^{k(3k-1)/2} + q^{k(3k+1)/2}\right).$$

Proof. To avoid fractional powers in the substitution, write $q = t^2$. For $0 < |t| < 1$, apply Theorem 277 with $x = t^3$ and $y = -t^{-1}$. This gives

$$\begin{aligned}
\sum_{k\in\mathbb{Z}} (-1)^k t^{3k^2-k} &= \prod_{m=1}^{\infty} (1 - t^{6m})(1 - t^{6m-4})(1 - t^{6m-2}) \\
&= \prod_{m=1}^{\infty} (1 - q^{3m})(1 - q^{3m-2})(1 - q^{3m-1}) \\
&= \prod_{m=1}^{\infty} (1 - q^m).
\end{aligned}$$

The three families cover each positive integer exponent of q exactly once. Since $k(3k - 1)$ is even, $t^{3k^2-k} = q^{k(3k-1)/2}$, proving the first identity. The case $q = 0$ is immediate, and equality of the convergent power series also gives the formal identity. Pairing the terms indexed by k and $-k$ gives the second form. □

The expansion starts $1 - q - q^2 + q^5 + q^7 - q^{12} - q^{15} + \cdots$. Its sparsity gives a recurrence for partition numbers.

Corollary 279 (Partition recurrence)**.** *Let $p(n)$ count partitions of n, with $p(0) = 1$ and $p(n) = 0$ for $n < 0$. For every $n \geq 1$,*

$$\boxed{p(n) = \sum_{k \geq 1} (-1)^{k-1} \left(p\left(n - \frac{k(3k-1)}{2} \right) + p\left(n - \frac{k(3k+1)}{2} \right) \right).}$$

Proof. Choosing the multiplicity of each possible part gives $\sum_{n \geq 0} p(n) q^n = \prod_{m \geq 1} (1 - q^m)^{-1}$. Multiply by the pentagonal expansion. The product is 1, so its coefficient of q^n vanishes for $n \geq 1$:

$$p(n) + \sum_{k \geq 1} (-1)^{k} \left(p\left(n - \frac{k(3k-1)}{2} \right) + p\left(n - \frac{k(3k+1)}{2} \right) \right) = 0.$$

Isolating $p(n)$ gives the recurrence. □

Only $O(\sqrt{n})$ summands can be nonzero for a fixed n, so the recurrence computes $p(0), \ldots, p(N)$ using $O(N^{3/2})$ arithmetic operations. Note that this is not as efficient as just inverting the expansion using FPS inversion, which is $O(N \log N)$, but this may be independently useful.

7.10.6 Hook-Length Formula

Let

$$\lambda = (\lambda_1 \geq \lambda_2 \geq \cdots) \vdash n$$

be a Young diagram. For a cell $u = (i, j)$, its hook consists of u, the cells to its right, and the cells below it; let $h(u)$ denote its size.

Let f^λ be the number of standard Young tableaux of shape λ.

Theorem 280 (Hook-Length Formula)**.**

$$\boxed{f^\lambda = \frac{n!}{\prod_{u \in \lambda} h(u)}.}$$

Why should hooks appear? At each cell, the row and column conditions combine into one requirement: its entry must be the smallest in its hook. A filling is standard exactly when this holds at every cell.

Among the $n!$ uniformly random fillings, a fixed cell u is smallest in its hook with probability $1/h(u)$. Multiplying these local probabilities suggests

$$f^\lambda = n! \prod_{u \in \lambda} \frac{1}{h(u)}.$$

For a single row, the hook lengths are $n, n-1, \ldots, 1$, and this simply recovers the familiar $1/n!$ probability of being sorted. In a general diagram, the hooks overlap and their minimum-entry conditions are *not independent*. The remarkable content of the formula is that the probability of satisfying all of them nevertheless equals the product of their individual probabilities.

Let r be the number of rows of the original diagram. Throughout the branching argument, we regard every smaller diagram as also having r rows by appending zero rows if necessary. This is only bookkeeping: it lets us use the same coordinates even when the last nonempty row disappears.

Define the shifted row lengths

$$\ell_i = \lambda_i + r - i.$$

Because $\lambda_i \geq \lambda_{i+1}$,

$$\ell_i - \ell_{i+1} = \lambda_i - \lambda_{i+1} + 1 > 0,$$

so

$$\ell_1 > \ell_2 > \cdots > \ell_r.$$

The point of this shift is that it turns the hook lengths in each row into almost all the integers from 1 up to ℓ_i.

Lemma 281 (Hook Product Identity)**.**

$$\boxed{\prod_{u \in \lambda} h(u) = \frac{\prod_{i=1}^{r} \ell_i!}{\prod_{1 \leq i < j \leq r} (\ell_i - \ell_j)}.}$$

Proof. Fix a row i. Read its cells from right to left.

If there were no lower rows, the hook lengths would simply be

$$1, 2, \ldots, \lambda_i.$$

Each lower row changes this sequence once. When we pass the last column occupied by row $j > i$, the next cell gains not only one new cell to its right but also the cell in row j below it. Thus the hook length jumps by 2 instead of 1, skipping one integer.

There are $r - i$ lower rows, so the λ_i hook lengths in row i are obtained from the $\ell_i = \lambda_i + r - i$ integers

$$1, 2, \ldots, \ell_i$$

by deleting exactly $r - i$ values.

The value skipped when crossing the end of row j is

$$\lambda_i - \lambda_j + j - i = \ell_i - \ell_j.$$

Hence

$$\{h(i,1), \ldots, h(i,\lambda_i)\} = \{1, \ldots, \ell_i\} \setminus \{\ell_i - \ell_j : j > i\},$$

and therefore

$$\prod_{q=1}^{\lambda_i} h(i,q) = \frac{\ell_i!}{\prod_{j>i} (\ell_i - \ell_j)}.$$

Multiplying over i proves the identity. □

Define

$$F(\lambda) = n! \, \frac{\prod_{i<j} (\ell_i - \ell_j)}{\prod_i \ell_i!}.$$

By the preceding lemma,

$$F(\lambda) = \frac{n!}{\prod_{u \in \lambda} h(u)}.$$

The remainder of the proof will show that F satisfies the same recurrence as f^λ. The benefit of the padded coordinates now becomes apparent: removing a corner from row i just changes some value,

$$\ell_i \longmapsto \ell_i - 1,$$

even when that removal makes an entire row disappear.

The number of standard tableaux satisfies the branching recurrence

$$f^\lambda = \sum_{\mu \lessdot \lambda} f^\mu,$$

because the largest entry n must lie in a removable corner. We will show that F satisfies the same recurrence.

Lemma 282. *For distinct numbers $\ell_1, \dots, \ell_r$,*

$$\boxed{\sum_{i=1}^{r} \ell_i \prod_{j \neq i} \frac{\ell_i - \ell_j - 1}{\ell_i - \ell_j} = \sum_{i=1}^{r} \ell_i - \binom{r}{2}.}$$

Proof. Let

$$P(x) = \prod_{j=1}^{r} (x - \ell_j)$$

and define

$$Q(x) = x\big(P(x-1) - P(x)\big) + rP(x).$$

Since both $P(x-1)$ and $P(x)$ have leading term x^r, their difference has degree at most $r-1$. The x^r-term in $x(P(x-1) - P(x))$ is $-rx^r$, which is cancelled by $rP(x)$, so

$$\deg Q \leq r - 1.$$

At $x = \ell_i$, we have $P(\ell_i) = 0$, and hence

$$Q(\ell_i) = \ell_i P(\ell_i - 1) = -\ell_i \prod_{j \neq i} (\ell_i - \ell_j - 1).$$

Since Q has degree at most $r-1$, Lagrange interpolation says that its leading coefficient is

$$[x^{r-1}]Q = \sum_{i=1}^{r} \frac{Q(\ell_i)}{P'(\ell_i)}.$$

But

$$P'(\ell_i) = \prod_{j \neq i} (\ell_i - \ell_j),$$

so

$$[x^{r-1}]Q = -\sum_{i=1}^{r} \ell_i \prod_{j \neq i} \frac{\ell_i - \ell_j - 1}{\ell_i - \ell_j}.$$

It remains to compute the same coefficient directly. Write

$$s_1 = \sum_i \ell_i.$$

Then

$$P(x) = x^r - s_1 x^{r-1} + \cdots .$$

Expanding $P(x-1)$ to the first two terms,

$$P(x-1) = x^r - (r+s_1)x^{r-1} + \left(\binom{r}{2} + (r-1)s_1 + \cdots \right) x^{r-2} + \cdots .$$

Hence

$$[x^{r-1}]\,(x(P(x-1) - P(x))) = \binom{r}{2} + (r-1)s_1,$$

while

$$[x^{r-1}]\,rP(x) = -rs_1.$$

Therefore

$$[x^{r-1}]Q = \binom{r}{2} - s_1.$$

Comparing the two expressions for $[x^{r-1}]Q$ gives

$$\sum_{i=1}^{r} \ell_i \prod_{j \neq i} \frac{\ell_i - \ell_j - 1}{\ell_i - \ell_j} = s_1 - \binom{r}{2},$$

as claimed. □

Now suppose a removable corner is deleted from row i. Then n becomes $n-1$, and among the shifted row lengths only

$$\ell_i \longmapsto \ell_i - 1$$

changes. Therefore

$$\frac{F(\lambda - \square_i)}{F(\lambda)} = \frac{\ell_i}{n} \prod_{j \neq i} \frac{\ell_i - \ell_j - 1}{\ell_i - \ell_j}.$$

This formula also automatically detects rows from which no corner may be removed. For $i < r$, row i is not removable exactly when $\lambda_i = \lambda_{i+1}$. In that case

$$\ell_i - \ell_{i+1} = \lambda_i - \lambda_{i+1} + 1 = 1,$$

so the factor corresponding to $j = i+1$ is

$$\frac{\ell_i - \ell_{i+1} - 1}{\ell_i - \ell_{i+1}} = 0.$$

For the last padded row, if $\lambda_r = 0$, then $\ell_r = 0$, so the prefactor ℓ_r vanishes. Thus we may sum the ratio over all $i = 1, \ldots, r$; non-removable rows simply contribute 0.

Finally,

$$n = \sum_{i=1}^{r} \lambda_i = \sum_{i=1}^{r} (\ell_i - r + i) = \sum_{i=1}^{r} \ell_i - \binom{r}{2}.$$

Hence Lemma 282 gives

$$\sum_{\mu \lessdot \lambda} F(\mu) = F(\lambda) \sum_{i=1}^{r} \frac{\ell_i}{n} \prod_{j \neq i} \frac{\ell_i - \ell_j - 1}{\ell_i - \ell_j} \qquad = F(\lambda), \frac{1}{n}\left(\sum_i \ell_i - \binom{r}{2}\right) = F(\lambda).$$

Thus F satisfies exactly the same branching recurrence as f^λ.
For the empty diagram, $f^\varnothing = 1$. Also

$$F(\varnothing) = \frac{\prod_{i<j}(j-i)}{\prod_{i=1}^{r}(r-i)!} = 1,$$

since

$$\prod_{i<j}(j-i) = \prod_{j=1}^{r}(j-1)!.$$

Therefore, by induction on the number of cells,

$$f^\lambda = F(\lambda).$$

Using Lemma 281,

$$\boxed{f^\lambda = \frac{n!}{\prod_{u \in \lambda} h(u)}.}$$

This proves the hook-length formula.

7.10.7 The Fundamental Theorem of Symmetric Polynomials

A symmetric polynomial depends on a collection of values without depending on their order. The coefficients of $\prod_i (T - x_i)$ are the natural coordinates for such information. The fundamental theorem says that they capture every symmetric polynomial uniquely. Its proof is an elimination algorithm: cancel the largest monomial using a product of elementary symmetric polynomials, then repeat.

Let R be a commutative ring with identity. A polynomial $f \in R[x_1, \dots, x_n]$ is *symmetric* if $f(x_{\sigma(1)}, \dots, x_{\sigma(n)}) = f(x_1, \dots, x_n)$ for every $\sigma \in S_n$. Define the elementary symmetric polynomials by

$$e_0 = 1, \qquad e_k = \sum_{1 \le i_1 < \cdots < i_k \le n} x_{i_1} \cdots x_{i_k} \quad (1 \le k \le n).$$

Thus $e_1 = x_1 + \cdots + x_n$, $e_n = x_1 \cdots x_n$, and

$$\prod_{i=1}^{n}(T - x_i) = T^n - e_1 T^{n-1} + e_2 T^{n-2} - \cdots + (-1)^n e_n.$$

Theorem 283 (Fundamental theorem of symmetric polynomials)**.** *For every symmetric $f \in R[x_1, \dots, x_n]$, there is a unique $F \in R[Y_1, \dots, Y_n]$ such that*

$$\boxed{f(x_1, \dots, x_n) = F(e_1, \dots, e_n).}$$

If $\deg f \le D$, every monomial $Y_1^{b_1} \cdots Y_n^{b_n}$ in F has weighted degree $\sum_{k=1}^{n} k b_k \le D$. If f is homogeneous of degree D, every such monomial has weighted degree exactly D.

Elimination. Order monomials first by total degree and then lexicographically with $x_1 > \cdots > x_n$. Thus among monomials of the same degree, compare exponents from left to right and prefer the larger exponent at the first difference. This order is preserved by multiplication by any fixed monomial.

Suppose $cx_1^{\alpha_1} \cdots x_n^{\alpha_n}$ is the leading term of a nonzero symmetric polynomial. Then $\alpha_1 \geq \cdots \geq \alpha_n$: if $\alpha_i < \alpha_{i+1}$, swapping those two variables produces a strictly larger monomial with the same nonzero coefficient. That contradicts the choice of leading term.

The leading monomial of e_k is $x_1 \cdots x_k$, with coefficient one. Consequently, setting $\alpha_{n+1} = 0$, the product

$$E_\alpha = e_1^{\alpha_1 - \alpha_2} e_2^{\alpha_2 - \alpha_3} \cdots e_n^{\alpha_n} \tag{76}$$

has leading monomial exactly x^α, again with coefficient one. Indeed, the exponent of x_i in the product of leading monomials is $\sum_{j=i}^{n} (\alpha_j - \alpha_{j+1}) = \alpha_i$. All other choices of terms give smaller monomials.

Proof of Theorem 283. For existence, subtract cE_α from the current remainder. This cancels its leading term, preserves symmetry, and introduces no monomial of greater total degree: E_α is homogeneous of degree $\sum_i \alpha_i$. If the remainder is nonzero, its leading monomial strictly decreases. There are only finitely many monomials of total degree at most $D = \deg f$, so repetition reaches zero. Recording $cY_1^{\alpha_1 - \alpha_2} \cdots Y_n^{\alpha_n}$ at every step produces F. Its weighted degree is the degree of the canceled term, proving both degree assertions. The zero input gives $F = 0$.

For uniqueness, a monomial Y^b becomes a product e^b with leading exponent vector

$$\alpha_i = b_i + b_{i+1} + \cdots + b_n.$$

Different b give different vectors, because $b_i = \alpha_i - \alpha_{i+1}$. In a nonzero polynomial F, choose the term whose substituted leading monomial is largest. No other substituted term can cancel that monomial, and its coefficient stays nonzero because the leading coefficient of e^b is one. Thus $F(e_1, \ldots, e_n) \neq 0$. The substitution map is injective, proving uniqueness even if R has zero divisors. □

Algorithm 29 Rewrite a symmetric polynomial in elementary symmetric coordinates

Input: A symmetric polynomial $f \in R[x_1, \ldots, x_n]$ with exact coefficient operations and zero tests
Output: The unique F satisfying $f = F(e_1, \ldots, e_n)$

$r \leftarrow f$; $F \leftarrow 0$; form $e_1, \ldots, e_n$
while $r \neq 0$ **do**
  Let cx^α be the leading term of r in degree-lexicographic order
  $\alpha_{n+1} \leftarrow 0$; $b_i \leftarrow \alpha_i - \alpha_{i+1}$ for $1 \leq i \leq n$
  $F \leftarrow F + c \prod_{i=1}^{n} Y_i^{b_i}$
  $r \leftarrow r - c \prod_{i=1}^{n} e_i^{b_i}$, expanding and collecting coefficients
return F

Power Sums and Newton's Identities. The power sums $p_r = \sum_{i=1}^{n} x_i^r$ are another natural description of an unordered collection. Newton's identities convert between these and the e_k. The logarithmic derivative performs the conversion at once: it turns the product defining the e_k into a sum involving the p_r.

Theorem 284 (Newton's identities)**.** *Put $e_k = 0$ for $k > n$. Over any commutative ring, for every $k \geq 1$,*

$$\boxed{k e_k = \sum_{r=1}^{k} (-1)^{r-1} e_{k-r} p_r.}$$

If $n!$ is invertible in R, every symmetric polynomial is uniquely a polynomial in $p_1, \ldots, p_n$ as well.

Proof. Work in the formal power-series ring over $R[x_1, \dots, x_n]$ and set $E(t) = \prod_i (1 + x_i t) = \sum_{k=0}^{n} e_k t^k$. Since $E(0) = 1$, this series is invertible. The product rule and geometric series give

$$\frac{E'(t)}{E(t)} = \sum_{i=1}^{n} \frac{x_i}{1 + x_i t} = \sum_{r \geq 1} (-1)^{r-1} p_r t^{r-1}.$$

Multiply by $E(t)$ and compare coefficients of t^{k-1} to obtain the identity.

If $n!$ is invertible, so is every $1 \leq k \leq n$, and the identity recursively expresses e_k using $p_1, \dots, p_k$. Conversely, solving for p_k expresses it using $e_1, \dots, e_k$, with leading term $(-1)^{k-1} k e_k$. These are inverse triangular polynomial changes of coordinates with invertible diagonal coefficients. The uniqueness in Theorem 283 therefore transfers to the power-sum coordinates. □

7.10.8 Robinson–Schensted Correspondence

The Robinson–Schensted correspondence gives a bijection

$$\boxed{S_n \longleftrightarrow \bigsqcup_{\lambda \vdash n} \mathrm{SYT}(\lambda) \times \mathrm{SYT}(\lambda).}$$

Its basic operation is row insertion.

Row Insertion To insert x into a standard tableau P:

1. In the first row, find the leftmost entry $y > x$.
2. Replace y by x and insert y into the next row.
3. If there is no entry larger than x, append x to the row and stop.

Lemma 285. *Row insertion preserves the tableau property and adds exactly one corner cell.*

Proof. Replacing the first entry larger than x preserves increasing order within the row.

Suppose an entry y is bumped from column c into the next row. Because the tableau was column-increasing, the entry directly below y, if it exists, is larger than y. Thus the first entry larger than y in the next row occurs in some column $c' \leq c$.

Hence the bumping path moves weakly to the left as it descends. The entry directly above the newly inserted y is therefore smaller than y, so column order is also preserved.

Eventually an entry is appended at the end of some row. The bumping-path property ensures that this new cell is an outer corner. □

The Correspondence Given

$$\pi = \pi_1 \pi_2 \cdots \pi_n \in S_n,$$

start with empty tableaux P, Q.

For $t = 1, \dots, n$:

- row-insert π_t into P;
- exactly one new cell is created;
- put t into that cell of Q.

The insertion tableau P and recording tableau Q always have the same shape.

Since later cells receive larger labels and new cells are always outer corners, Q is also standard.

Reverse Insertion Suppose a corner cell of P contains x. Delete that cell. Moving upward one row at a time, replace the rightmost entry $y < x$ by x and set

$$x \leftarrow y.$$

When the first row has been processed, output the final value of x.

Lemma 286. *Reverse insertion is the inverse of row insertion.*

Proof. Suppose forward insertion placed x into a row and bumped y. By definition, y was the leftmost entry larger than x. Equivalently, after the replacement, x is the rightmost entry smaller than y. Thus the reverse rule exactly undoes this step. Apply this successively up the bumping path. □

Theorem 287 (Robinson–Schensted)**.** *The map*

$$\pi \longmapsto (P, Q)$$

is a bijection between permutations of $[n]$ *and pairs of standard Young tableaux of the same shape.*

Proof. Given (P, Q), find the cell of Q containing n. It is a corner. Remove it and reverse-insert from the corresponding corner of P. The output is π_n.

Repeat with

$$n-1, n-2, \ldots, 1.$$

By the preceding lemma this exactly reverses the forward algorithm. □

One immediate consequence is

$$\boxed{n! = \sum_{\lambda \vdash n} (f^\lambda)^2.}$$

Longest Increasing Subsequences

Theorem 288 (Schensted)**.** *If the insertion tableau of* π *has shape* λ*, then*

$$\boxed{\lambda_1 = \mathrm{LIS}(\pi), \qquad \lambda_1' = \mathrm{LDS}(\pi).}$$

Proof. Consider the first row after processing a prefix of the permutation:

$$r_1 < r_2 < \cdots < r_t.$$

We claim that

$$r_j$$

is the smallest possible final value of an increasing subsequence of length j in the processed prefix.

This is true initially. When a new value x is inserted, it replaces the first $r_j > x$, or is appended if none exists. This is exactly the standard update of the minimum possible tail of an increasing subsequence of each length.

Thus the first row has an entry in position j exactly when an increasing subsequence of length j exists. Therefore

$$\lambda_1 = \mathrm{LIS}(\pi).$$

For decreasing subsequences, reverse the order on the alphabet. Row insertion in the reversed order is column insertion in the original order, so it transposes the shape. An increasing subsequence in the reversed order is exactly a decreasing subsequence in the original order. Hence

$$\lambda_1' = \mathrm{LDS}(\pi).$$

□

7.10.9 MacMahon's Master Theorem

Theorem 289. *Let $A = (a_{ij})$ be an $n \times n$ matrix, and let $X = \operatorname{diag}(x_1, \ldots, x_n)$ be a diagonal matrix of formal variables. For any tuple $k = (k_1, \ldots, k_n) \in \mathbb{N}^n$, let $x^k = x_1^{k_1} \cdots x_n^{k_n}$. Then*

$$[x^k] \prod_{i=1}^{n} \left(\sum_{j=1}^{n} a_{ij} x_j \right)^{k_i} = [x^k] \frac{1}{\det(I - AX)}.$$

Proof. Let $R = \mathbb{C}[a_{11}, \ldots, a_{nn}]$ be the polynomial ring representing the generic matrix A. We work in the ring of formal power series $R[[x_1, \ldots, x_n]]$.

Let V be a free module of rank n over R with basis $v_1, \ldots, v_n$. Let $M = AX$, meaning $M_{ij} = a_{ij} x_j$. We define a linear operator $T : V \to V$ by its action on the standard basis:

$$T(v_i) = \sum_{j=1}^{n} M_{ij} v_j.$$

The operator T extends naturally to the symmetric algebra $\operatorname{Sym}(V) \cong R[v_1, \ldots, v_n]$. The subspace $S^m(V)$ of homogeneous polynomials of degree m has a basis of monomials $v^k = v_1^{k_1} \cdots v_n^{k_n}$ where $|k| = \sum_{i=1}^{n} k_i = m$. The action of T on this basis is:

$$T(v^k) = \prod_{i=1}^{n} T(v_i)^{k_i} = \prod_{i=1}^{n} \left(\sum_{j=1}^{n} a_{ij} x_j v_j \right)^{k_i}.$$

The trace of T acting on $S^m(V)$ is the sum of its diagonal matrix entries in this standard basis:

$$\operatorname{Tr}(S^m(T)) = \sum_{|k|=m} [v^k] T(v^k) = \sum_{|k|=m} [v^k] \prod_{i=1}^{n} \left(\sum_{j=1}^{n} a_{ij} x_j v_j \right)^{k_i}.$$

In the polynomial expansion of $\prod_{i=1}^{n} (\sum_{j=1}^{n} a_{ij} x_j v_j)^{k_i}$, the variables x_j and v_j are strictly coupled; every term chosen from the inner sum contributes exactly one x_j and one v_j. Therefore, the formal coefficient of v^k is exactly x^k multiplied by the scalar coefficient we would obtain if we evaluated at $v_j = 1$. Thus:

$$[v^k] T(v^k) = x^k \cdot [x^k] \prod_{i=1}^{n} \left(\sum_{j=1}^{n} a_{ij} x_j \right)^{k_i}.$$

Summing this trace over all degrees m yields a formal power series:

$$\sum_{m=0}^{\infty} \operatorname{Tr}(S^m(T)) = \sum_{k \in \mathbb{N}^n} x^k \left([x^k] \prod_{i=1}^{n} \left(\sum_{j=1}^{n} a_{ij} x_j \right)^{k_i} \right).$$

We must now prove that this sum evaluates to $\det(I - AX)^{-1}$. Let W be an $n \times n$ matrix of independent formal variables w_{ij}. We will establish the generic trace identity $\sum_{m=0}^{\infty} \operatorname{Tr}(S^m(W)) = \det(I - W)^{-1}$ in $\mathbb{C}[[w_{11}, \ldots, w_{nn}]]$.

By comparing the homogeneous components of degree m, we define a sequence of polynomials:

$$F_m(W) = \operatorname{Tr}(S^m(W)) - [\text{degree } m \text{ of } \det(I - W)^{-1}].$$

We must prove $F_m(W)$ is identically zero in $\mathbb{C}[w_{11}, \ldots, w_{nn}]$.

Let $\Delta(W)$ be the discriminant of the characteristic polynomial of W with respect to λ. Because there exist matrices with distinct eigenvalues, Δ is a non-zero polynomial.

Let $C \in \mathbb{C}^{n\times n}$ be any specific complex matrix such that $\Delta(C) \neq 0$. This non-vanishing discriminant guarantees that C has n distinct roots, meaning C is diagonalizable. Let its eigenvalues be $\lambda_1, \ldots, \lambda_n$, with a basis of eigenvectors $u_1, \ldots, u_n$.

Because C acts diagonally on V, the induced basis of monomials $u^k = u_1^{k_1} \cdots u_n^{k_n}$ is an eigenbasis for $S^m(C)$, with corresponding eigenvalues $\lambda_1^{k_1} \cdots \lambda_n^{k_n}$. Computing the trace in this eigenbasis yields:

$$\sum_{m=0}^{\infty} \operatorname{Tr}(S^m(C)) = \sum_{m=0}^{\infty} \sum_{|k|=m} \lambda_1^{k_1} \cdots \lambda_n^{k_n} = \prod_{i=1}^{n} \left(\sum_{k_i=0}^{\infty} \lambda_i^{k_i} \right).$$

By formally evaluating the independent geometric series, we obtain:

$$\prod_{i=1}^{n} \frac{1}{1-\lambda_i} = \frac{1}{\det(I-C)}.$$

Thus, $F_m(C) = 0$ for all strictly diagonalizable matrices C. We have established that the polynomial F_m evaluates to zero wherever the non-zero polynomial Δ is non-zero, thus $F_m\Delta = 0$, so $F_m = 0$ as $\Delta \neq 0$ and we're over $\mathbb{C}$.

By substituting the variables $w_{ij} = a_{ij}x_j$ (which corresponds to $W = AX$) back into the identity for W, we obtain:

$$\sum_{m=0}^{\infty} \operatorname{Tr}(S^m(T)) = \frac{1}{\det(I-AX)}.$$

□

7.10.10 Ehrhart Polynomials

For a lattice polytope $P \subseteq \mathbb{R}^d$, let

$$L_P(t) = |tP \cap \mathbb{Z}^d|, \qquad t = 0, 1, 2, \ldots.$$

Ehrhart's theorem says that this is actually a polynomial in the dilation factor.

The essential reason is already visible for a simplex. After adding one homogenizing coordinate, dilations of the simplex become horizontal slices of a simplicial cone. Every lattice point of that cone is a unique lattice point in one bounded fundamental parallelepiped plus nonnegative integer combinations of the cone generators. The infinite count therefore contributes only a factor $(1-z)^{-d-1}$.

Theorem 290 (Ehrhart)**.** *Let $P \subseteq \mathbb{R}^d$ be a d-dimensional lattice polytope. Then there is a polynomial $L_P(t)$ of degree d such that*

$$L_P(t) = |tP \cap \mathbb{Z}^d|$$

for every integer $t \geq 0$. Its constant term is 1, and its leading coefficient is the Euclidean volume:

$$\boxed{L_P(t) = \operatorname{vol}(P)t^d + O(t^{d-1}).}$$

Proof. We first treat a lattice simplex

$$S = \operatorname{conv}(v_0, \ldots, v_d).$$

Homogenize its vertices to

$$w_i = (v_i, 1) \in \mathbb{Z}^{d+1}.$$

The w_i are linearly independent, and

$$(x, t) \in \mathbb{Z}^{d+1}$$

satisfies $x \in tS$ exactly when

$$(x,t) = \sum_{i=0}^{d} \lambda_i w_i \qquad \text{for some } \lambda_i \geq 0.$$

Consider the half-open fundamental parallelepiped

$$\Pi = \left\{ \sum_{i=0}^{d} \theta_i w_i : 0 \leq \theta_i < 1 \right\}.$$

Every lattice point y in the cone has a unique representation

$$y = p + \sum_{i=0}^{d} n_i w_i,$$

where $n_i \in \mathbb{Z}_{\geq 0}$ and $p \in \Pi \cap \mathbb{Z}^{d+1}$: simply write $\lambda_i = n_i + \theta_i$ with $n_i = \lfloor \lambda_i \rfloor$.

If $p \in \Pi \cap \mathbb{Z}^{d+1}$, its last coordinate is $\sum_i \theta_i$, hence is an integer in $\{0, \ldots, d\}$. Let h_j^* be the number of such points whose last coordinate is j. The generating function of the horizontal lattice-point counts is therefore

$$\boxed{\sum_{t \geq 0} L_S(t) z^t = \frac{h_0^* + h_1^* z + \cdots + h_d^* z^d}{(1-z)^{d+1}}.} \tag{77}$$

Indeed, each added generator w_i raises the final coordinate by one, giving one geometric factor $1 + z + z^2 + \cdots$.

Since

$$[z^r](1-z)^{-d-1} = \binom{r+d}{d},$$

we obtain

$$\boxed{L_S(t) = \sum_{j=0}^{d} h_j^* \binom{t+d-j}{d}.}$$

Each term is a polynomial in t of degree d, so L_S is a polynomial.

The sum $\sum_j h_j^*$ is the number of lattice points in a fundamental parallelepiped for the lattice generated by $w_0, \ldots, w_d$, hence

$$\sum_j h_j^* = |\det(w_0, \ldots, w_d)|.$$

Subtracting the first column from the others shows

$$|\det(w_0, \ldots, w_d)| = |\det(v_1 - v_0, \ldots, v_d - v_0)| = d! \operatorname{vol}(S).$$

Since every binomial polynomial above has leading coefficient $1/d!$, the leading coefficient of L_S is $\operatorname{vol}(S)$.

Now triangulate P into lattice simplices $S_1, \ldots, S_m$ using vertices of P. Any nonempty intersection of simplices in a triangulation is a common face, hence is itself a lattice simplex. Inclusion–exclusion gives

$$L_P(t) = \sum_{\emptyset \neq I \subseteq [m]} (-1)^{|I|+1} L_{\bigcap_{i \in I} S_i}(t).$$

The same cone argument applies to lower-dimensional faces, so every term is polynomial. Hence L_P is polynomial.

Distinct d-simplices intersect only in lower-dimensional faces, so only the maximal simplices contribute to the coefficient of t^d. That coefficient is

$$\sum_{i=1}^{m} \operatorname{vol}(S_i) = \operatorname{vol}(P).$$

Finally $0P = \{0\}$, so $L_P(0) = 1$. □

Rational polytopes. The same proof explains the rational case essentially for free.

Corollary 291 (Rational Ehrhart theorem)**.** *Suppose every vertex of P lies in $\frac{1}{q}\mathbb{Z}^d$. Then $L_P(t) = |tP \cap \mathbb{Z}^d|$ is a quasipolynomial of period dividing q: for each residue $r \bmod q$, there is a polynomial p_r such that*

$$L_P(t) = p_r(t) \qquad \text{whenever } t \equiv r \pmod{q}.$$

Its degree is $\dim P$*, and its leading coefficient is again* $\mathrm{vol}(P)$.

Proof. For a rational simplex, replace the homogenized generators $(v_i, 1)$ by the integral vectors

$$(qv_i, q).$$

The same fundamental-parallelepiped decomposition now gives a generating function of the form

$$\frac{B(z)}{(1-z^q)^{d+1}}$$

for some polynomial $B(z)$. On each fixed residue class modulo q, its coefficients are polynomial in t. Triangulating a rational polytope into rational simplices completes the proof. □

Ehrhart reciprocity and Pick's theorem. The same cone picture also counts interior lattice points. For a polytope P, write

$$L_{P^\circ}(t) = |tP^\circ \cap \mathbb{Z}^d|.$$

Theorem 292 (Ehrhart reciprocity)**.** *For every d-dimensional lattice polytope P and every integer $t \geq 1$,*

$$\boxed{L_{P^\circ}(t) = (-1)^d L_P(-t).}$$

Proof. For a lattice simplex $S = \mathrm{conv}(v_0, \ldots, v_d)$, recall the homogenized generators $w_i = (v_i, 1)$ and the half-open parallelepiped

$$\Pi = \left\{ \sum_{i=0}^{d} \theta_i w_i : 0 \leq \theta_i < 1 \right\}.$$

If h_j^* counts its lattice points of height j, then

$$L_S(t) = \sum_{j=0}^{d} h_j^* \binom{t+d-j}{d}.$$

For interior points of the cone, use instead

$$\Pi^\circ = \left\{ \sum_{i=0}^{d} \theta_i w_i : 0 < \theta_i \leq 1 \right\}.$$

The map

$$p \longmapsto \sum_{i=0}^{d} w_i - p$$

bijects lattice points of Π of height j with lattice points of Π° of height $d+1-j$. Hence

$$L_{S^\circ}(t) = \sum_{j=0}^{d} h_j^* \binom{t+j-1}{d}.$$

Using

$$\binom{t+j-1}{d} = (-1)^d \binom{-t+d-j}{d},$$

we obtain

$$L_{S^\circ}(t) = (-1)^d L_S(-t).$$

As in the proof of Ehrhart's theorem, a lattice triangulation and inclusion–exclusion extend the identity from simplices to arbitrary lattice polytopes. □

In dimension two, reciprocity immediately determines the middle coefficient of the Ehrhart polynomial.

Corollary 293 (Pick's theorem)**.** *Let P be a lattice polygon of area A, with I lattice points in its interior and B lattice points on its boundary. Then*

$$\boxed{A = I + \frac{B}{2} - 1.}$$

Moreover,

$$\boxed{L_P(t) = At^2 + \frac{B}{2}t + 1.}$$

Proof. By Ehrhart's theorem,

$$L_P(t) = At^2 + ct + 1$$

for some c. Evaluating at $t = 1$ counts all lattice points of P, so

$$I + B = A + c + 1.$$

On the other hand, Ehrhart reciprocity in dimension two gives

$$I = L_{P^\circ}(1) = L_P(-1) = A - c + 1.$$

Subtracting the second identity from the first gives $B = 2c$, hence $c = B/2$. Substituting back yields

$$A = I + \frac{B}{2} - 1.$$

□

7.10.11 The Stanley–Wilf Conjecture

A permutation σ of $[n]$ *contains* a permutation π of $[k]$ if there are indices

$$i_1 < \cdots < i_k$$

whose values occur in the same relative order as $\pi(1), \ldots, \pi(k)$. Otherwise σ *avoids* π.

Let $s_n(\pi)$ be the number of n-permutations avoiding π.

Theorem 294 (Stanley–Wilf, exponential form)**.** *For every fixed permutation π, there is a constant C_π such that*

$$\boxed{s_n(\pi) \le C_\pi^n.}$$

The key change of viewpoint is to replace a permutation by its permutation matrix. Avoiding a permutation then becomes an extremal problem for ordered 0–1 matrices.

For 0–1 matrices A, P, say that A *contains* P if one can choose rows and columns of A, preserving their order, so that every 1-entry of P corresponds to a 1-entry of A. Let

$$f(n, P)$$

be the maximum number of 1's in an $n \times n$ matrix avoiding P.

Lemma 295 (Marcus–Tardos)**.** *For every fixed permutation matrix P,*

$$\boxed{f(n,P) = O_P(n).}$$

Proof. Suppose P is $k \times k$. Set

$$q = k^2.$$

First assume that $q \mid n$, and partition an $n \times n$ P-avoiding matrix A into $q \times q$ blocks.

Contract every nonempty block to a 1 and every empty block to a 0, obtaining an $(n/q) \times (n/q)$ matrix B. Then B avoids P: otherwise choose one 1 from each of the k corresponding blocks of A.

Call a block *wide* if its 1's occupy at least k columns, and *tall* if they occupy at least k rows.

Fix one block-column. There are fewer than

$$k\binom{q}{k}$$

wide blocks in it. Indeed, otherwise choose from each wide block some k occupied internal columns. By pigeonhole, k blocks use the same set

$$c_1 < \cdots < c_k$$

of internal columns. Taking these blocks in increasing block-row order, and in the rth block choosing a 1 in internal column $c_{\pi(r)}$, produces P. The same argument shows that every block-row contains fewer than

$$k\binom{q}{k}$$

tall blocks.

Finally, a nonempty block which is neither wide nor tall has its 1's in at most $k-1$ rows and at most $k-1$ columns, hence contains at most

$$(k-1)^2$$

ones.

There are at most $f(n/q, P)$ nonempty blocks, because B avoids P. Hence

$$f(n,P) \le (k-1)^2 f(n/q,P) + 2k^3\binom{k^2}{k} n.$$

The second term bounds all wide and tall blocks: there are at most

$$\frac{n}{q}\, k\binom{q}{k}$$

of each type, and each block has at most $q^2 = k^4$ entries.

For $n = q^r$, divide by n:

$$\frac{f(n,P)}{n} \le \frac{(k-1)^2}{k^2}\frac{f(n/q,P)}{n/q} + 2k^3\binom{k^2}{k}.$$

Since

$$\frac{(k-1)^2}{k^2} < 1,$$

this geometric recurrence gives $f(n,P) = O_P(n)$.

For arbitrary n, pad the matrix with zero rows and columns to the next power of q, which is less than qn. □

The extremal theorem becomes an enumeration theorem by contracting much smaller 2×2 blocks.

Proof of Theorem 294. Let P be the permutation matrix of π, and let $\mathcal{T}_n(P)$ be the family of all $n \times n$ binary matrices avoiding P.

Map every $A \in \mathcal{T}_{2n}(P)$ to an $n \times n$ matrix B by replacing each 2×2 block by 0 if it is empty and by 1 otherwise. Again B avoids P.

If B has w ones, then it has at most

$$15^w$$

preimages, since each nonempty 2×2 block has $2^4 - 1 = 15$ possibilities. By Lemma 295,

$$w \le f(n, P) \le Cn$$

for some $C = C_P$. Therefore

$$|\mathcal{T}_{2n}(P)| \le |\mathcal{T}_n(P)|\, 15^{Cn}.$$

Iterating for powers of two gives

$$\log |\mathcal{T}_n(P)| = O_P(n),$$

and hence

$$|\mathcal{T}_n(P)| \le D_P^n$$

for some constant D_P. Padding to the next power of two handles general n.

The permutation matrix of every π-avoiding permutation belongs to $\mathcal{T}_n(P)$, so

$$s_n(\pi) \le |\mathcal{T}_n(P)| \le D_P^n.$$

□

7.11 Möbius Inversion on Posets

Often an object has an exact type x, but it is easier to count objects whose type is at most y. If $f(x)$ counts exact types, the accessible quantity is $g(y) = \sum_{x \le y} f(x)$. Möbius inversion recovers the exact counts by coefficients depending only on the partial order.

7.11.1 Incidence Algebra and the Möbius Function

A poset P is *locally finite* if every interval $[x, y] = \{z : x \le z \le y\}$ is finite.

An *incidence function* assigns an integer $\alpha(x, y)$ to every comparable pair $x \le y$. Define convolution by

$$(\alpha * \beta)(x, y) = \sum_{x \le z \le y} \alpha(x, z)\beta(z, y).$$

This is matrix multiplication with indices restricted by the order. Local finiteness makes every entry a finite sum. Associativity follows because both parenthesizations expand to

$$\sum_{x \le u \le v \le y} \alpha(x, u)\beta(u, v)\gamma(v, y).$$

The identity is $\delta(x, y) = 1$ when $x = y$ and 0 otherwise. The *zeta function* is $\zeta(x, y) = 1$ for every $x \le y$. These functions, with pointwise addition and convolution, form the incidence algebra $I(P; \mathbb{Z})$.

Theorem 296 (Möbius function)**.** *Every locally finite poset has a unique integer-valued incidence function μ satisfying*

$$\mu * \zeta = \zeta * \mu = \delta.$$

Equivalently,

$$\sum_{x \le z \le y} \mu(x, z) = \sum_{x \le z \le y} \mu(z, y) = \delta(x, y).$$

It is determined recursively by

$$\boxed{\mu(x,x)=1, \qquad \mu(x,y)=-\sum_{x\le z<y}\mu(x,z) \quad (x<y).}$$

It also has the chain formula

$$\boxed{\mu(x,y)=\sum_{k\ge 0}(-1)^k \#\{x=x_0<x_1<\cdots<x_k=y\}.}$$

The length-zero chain is counted only when $x=y$.

Proof. Put $N=\zeta-\delta$, so $N(x,y)=1$ for $x<y$ and is zero on the diagonal. Repeated convolution shows that $N^{*k}(x,y)$ counts strict chains with k steps from x to y; here $N^{*0}=\delta$. Such a chain uses $k+1$ distinct elements of $[x,y]$, so $N^{*k}(x,y)=0$ for $k\ge|[x,y]|$.

Consequently the geometric series

$$\mu=\delta-N+N^{*2}-N^{*3}+\cdots$$

is a finite sum at each interval. Multiplying on either side by $\zeta=\delta+N$ cancels all terms except δ. This proves both inverse identities and the chain formula. An inverse is unique: if ν were another, then $\nu=\nu*(\zeta*\mu)=(\nu*\zeta)*\mu=\mu$. Finally, separating the $z=y$ term in $\mu*\zeta=\delta$ gives the recursion. □

The chains in the formula need not be saturated: a step may skip intermediate elements. The alternating signs undo repeated counting along chains of intermediate types. In a finite poset, after ordering the elements compatibly with $\le$, this is simply the inverse of an upper triangular matrix with ones on the diagonal.

7.11.2 The Inversion Formula

Theorem 297 (Möbius inversion)**.** *Let P be locally finite and let $f,g:P\to A$, where A is an abelian group. Suppose that, below each fixed element, f and g have only finitely many nonzero values. Then*

$$\boxed{g(y)=\sum_{x\le y}f(x) \quad\Longleftrightarrow\quad f(y)=\sum_{x\le y}\mu(x,y)g(x).}$$

Dually, if f and g have only finitely many nonzero values above each fixed element, then

$$\boxed{g(x)=\sum_{y\ge x}f(y) \quad\Longleftrightarrow\quad f(x)=\sum_{y\ge x}\mu(x,y)g(y).}$$

Proof. Suppose $g(y)=\sum_{x\le y}f(x)$. Substitution and interchange of the finite sums give

$$\begin{aligned}\sum_{x\le y}\mu(x,y)g(x)&=\sum_{x\le y}\mu(x,y)\sum_{z\le x}f(z)\\&=\sum_{z\le y}f(z)\sum_{z\le x\le y}\mu(x,y)\\&=\sum_{z\le y}f(z)\delta(z,y)=f(y).\end{aligned}$$

Conversely, if $f(y)=\sum_{x\le y}\mu(x,y)g(x)$, then

$$\begin{aligned}\sum_{x\le y}f(x)&=\sum_{x\le y}\sum_{z\le x}\mu(z,x)g(z)\\&=\sum_{z\le y}g(z)\sum_{z\le x\le y}\mu(z,x)=g(y).\end{aligned}$$

The rearrangements are finite: only finitely many starting points z contribute, and for each one the intermediate points lie in the finite interval $[z, y]$. The upper-sum version follows by reversing the order, whose Möbius function is $\mu_{P^{\mathrm{op}}}(y, x) = \mu_P(x, y)$ by the chain formula. □

On a finite poset the support conditions are automatic. More generally, one may always apply the theorem inside any finite interval $[a, b]$, using sums over $a \le x \le y$ or $x \le y \le b$.

7.11.3 Products of Posets

The recursion shows that $\mu(x, y)$ depends only on the order structure of $[x, y]$. Intervals of the product thus give a useful factorization.

Lemma 298 (Product formula)**.** *For locally finite posets P, Q, ordered coordinatewise on $P \times Q$,*

$$\mu_{P\times Q}\big((x,u),(y,v)\big) = \mu_P(x,y)\mu_Q(u,v).$$

Proof. The proposed function satisfies the defining cancellation identity:

$$\begin{aligned}\sum_{\substack{x\le z\le y\\ u\le w\le v}} \mu_P(x,z)\mu_Q(u,w) &= \left(\sum_{x\le z\le y} \mu_P(x,z)\right)\left(\sum_{u\le w\le v} \mu_Q(u,w)\right)\\ &= \delta(x,y)\delta(u,v).\end{aligned}$$

The recursion uniquely determines the Möbius function, proving the claim. Repetition gives the formula for any finite product. □

7.11.4 Differences, Inclusion–Exclusion, and Divisor Sums

Chains. On the chain $\{0, 1, \dots, n\}$,

$$\mu(i,j) = \begin{cases} 1, & j = i,\\ -1, & j = i+1,\\ 0, & j \ge i+2.\end{cases}$$

Indeed, after the first two values, the preceding values sum to zero, so the recursion gives zero at every longer interval. Thus $g(j) = \sum_{i=0}^{j} f(i)$ is inverted by $f(j) = g(j) - g(j-1)$, with $g(-1) = 0$.

Subsets. For subsets ordered by inclusion, $[S, T]$ is a product of $|T \setminus S|$ two-element chains. Hence

$$\mu(S,T) = (-1)^{|T\setminus S|}, \qquad g(T) = \sum_{S\subseteq T} f(S) \iff f(T) = \sum_{S\subseteq T} (-1)^{|T\setminus S|} g(S).$$

For inclusion–exclusion, let $A_1, \dots, A_n$ be subsets of a finite universe Ω. Let $E(T)$ count elements belonging to exactly the sets indexed by T, and put $C(S) = |\bigcap_{i\in S} A_i|$, with $C(\varnothing) = |\Omega|$. Since $C(S) = \sum_{T\supseteq S} E(T)$, upper-sum inversion gives

$$\left|\Omega \setminus \bigcup_{i=1}^{n} A_i\right| = E(\varnothing) = \sum_{S\subseteq\{1,\dots,n\}} (-1)^{|S|} C(S).$$

Taking $f(S) = a_{|S|}$ similarly gives binomial inversion:

$$b_n = \sum_{k=0}^{n} \binom{n}{k} a_k \quad\Longleftrightarrow\quad a_n = \sum_{k=0}^{n} (-1)^{n-k} \binom{n}{k} b_k.$$

Divisibility. Order the positive integers by divisibility. If $d \mid n$ and $n/d = \prod_i p_i^{e_i}$, the interval $[d, n]$ is a product of chains $\{0, 1, \dots, e_i\}$, recording prime exponents. Therefore $\mu(d, n) = \mu_{\text{arith}}(n/d)$, where

$$\mu_{\text{arith}}(m) = \begin{cases} 1, & m = 1, \\ (-1)^r, & m \text{ is a product of } r \text{ distinct primes}, \\ 0, & m \text{ is divisible by a prime square.} \end{cases}$$

Poset inversion becomes the arithmetic Möbius inversion formula

$$\boxed{g(n) = \sum_{d \mid n} f(d) \quad \Longleftrightarrow \quad f(n) = \sum_{d \mid n} \mu_{\text{arith}}(n/d) g(d).}$$

The cancellation identity says $\sum_{d \mid m} \mu_{\text{arith}}(d) = 1$ for $m = 1$ and 0 otherwise. In particular,

$$\mathbf{1}_{\gcd(a,b)=1} = \sum_{d \mid \gcd(a,b)} \mu_{\text{arith}}(d).$$

Summing this indicator over $1 \le a \le N$ gives the useful counting identity

$$\#\{1 \le a \le N : \gcd(a, b) = 1\} = \sum_{d \mid b} \mu_{\text{arith}}(d) \left\lfloor \frac{N}{d} \right\rfloor.$$

7.11.5 Subspaces over a Finite Field

Let V be finite-dimensional over $\mathbb{F}_q$, and order its subspaces by inclusion. For $U \subseteq W$, the map $X \mapsto X/U$ identifies $[U, W]$ with the subspace lattice of W/U. Thus its Möbius function depends only on $d = \dim W - \dim U$.

Theorem 299 (Subspace Möbius function)**.** *If $d = \dim W - \dim U$, then*

$$\boxed{\mu(U, W) = (-1)^d q^{\binom{d}{2}}.}$$

Proof. The number of j-dimensional subspaces of a d-dimensional space is

$$\binom{d}{j}_q = \prod_{i=0}^{j-1} \frac{q^d - q^i}{q^j - q^i}.$$

The numerator counts ordered independent j-tuples, and the denominator counts the ordered bases of each j-dimensional subspace. The finite q-binomial theorem gives

$$\sum_{j=0}^{d} \binom{d}{j}_q q^{\binom{j}{2}} t^j = \prod_{i=0}^{d-1} (1 + tq^i).$$

At $t = -1$, the right side is zero for $d > 0$ and one for $d = 0$. Consequently the proposed values satisfy

$$\sum_{U \subseteq X \subseteq W} (-1)^{\dim X - \dim U} q^{\binom{\dim X - \dim U}{2}} = \sum_{j=0}^{d} \binom{d}{j}_q (-1)^j q^{\binom{j}{2}} = \delta(U, W).$$

This is the defining cancellation identity. □

7.11.6 Set Partitions

For $n \geq 1$, let Π_n be the set of partitions of $\{1, \ldots, n\}$, ordered by refinement: $\pi \leq \sigma$ means that every block of π is contained in a block of σ. Write $\widehat{0}$ for the partition into singletons and $\widehat{1}$ for the one-block partition.

Theorem 300 (Partition-lattice Möbius function)**.** *For $n \geq 1$,*

$$\mu_{\Pi_n}(\widehat{0}, \widehat{1}) = (-1)^{n-1}(n-1)!.$$

More generally, if $\pi \leq \sigma$ and b_B is the number of blocks of π contained in $B \in \sigma$, then

$$\boxed{\mu_{\Pi_n}(\pi, \sigma) = \prod_{B \in \sigma} (-1)^{b_B - 1}(b_B - 1)!.}$$

Proof. An intermediate partition between π and σ independently partitions the b_B blocks of π inside each $B \in \sigma$. Thus $[\pi, \sigma] \cong \prod_{B \in \sigma} \Pi_{b_B}$, so the product formula reduces the claim to the value from bottom to top.

Put $a_j = (-1)^{j-1}(j-1)!$ for $j \geq 1$. For a fixed partition ρ, $\prod_{B \in \rho}(|B| - 1)!$ counts permutations whose cycle blocks are exactly ρ: each block can be made into a cycle in $(|B| - 1)!$ ways. The sign of each such permutation is $\prod_{B \in \rho}(-1)^{|B|-1}$. Therefore

$$\sum_{\rho \in \Pi_n} \prod_{B \in \rho} a_{|B|} = \sum_{\tau \in S_n} \operatorname{sgn}(\tau) = \begin{cases} 1, & n = 1, \\ 0, & n \geq 2. \end{cases}$$

For $n \geq 2$, the last sum vanishes by pairing τ with $(1\,2)\tau$, which has opposite sign.

Now induct on n. The assertion holds for $n = 1$. For $\rho < \widehat{1}$, all its blocks have size less than n, so the interval product and induction give $\mu(\widehat{0}, \rho) = \prod_{B \in \rho} a_{|B|}$. The displayed zero-sum identity then forces $\mu(\widehat{0}, \widehat{1}) = a_n$ by the Möbius recursion. The general interval formula follows from the product decomposition. □

7.11.7 Weighted Inversion

The same algebra also inverts weighted cumulative sums. Allow incidence functions to take values in any ring R with identity, using the same convolution. Multiplication in R need not be commutative.

Theorem 301 (Invertibility in the incidence algebra)**.** *An incidence function $\alpha \in I(P; R)$ has a two-sided convolution inverse if and only if every diagonal value $\alpha(x, x)$ is a unit of R.*

Proof. Necessity follows by evaluating both inverse identities at (x, x). For sufficiency, let D agree with α on the diagonal and be zero elsewhere. Its inverse D^{-1} has diagonal entries $\alpha(x, x)^{-1}$. Put $K = D^{-1} * \alpha - \delta$. Then K vanishes on the diagonal and $\alpha = D * (\delta + K)$. As before, $K^{*j}(x, y) = 0$ for $j \geq |[x, y]|$, because every nonzero term requires a strict chain. Hence

$$\alpha^{-1} = \left(\delta - K + K^{*2} - K^{*3} + \cdots\right) * D^{-1}$$

is well-defined entrywise and is a two-sided inverse. □

For R-valued functions with finite support below each fixed element, associativity now gives

$$g(y) = \sum_{x \leq y} f(x)\alpha(x, y) \quad \Longleftrightarrow \quad f(y) = \sum_{x \leq y} g(x)\alpha^{-1}(x, y).$$

The order of the factors is part of the formula. Ordinary Möbius inversion is the special case $\alpha = \zeta$.

7.12 Geometry

A *d-simplex* is the convex hull of $d+1$ affinely independent points. Its faces are the convex hulls of subsets of its vertices, and its *facets* are its $(d-1)$-dimensional faces. A finite *triangulation* divides a polytope into simplices, including all their faces, so that two simplices intersect in a common face or not at all. Its *mesh* is the largest diameter of a simplex.

We repeatedly use the following incidence fact: in a triangulated d-ball, each $(d-1)$-simplex belongs to two d-simplices if it is interior, and one if it is on the boundary.

7.12.1 Sperner's Lemma

Write the standard n-simplex as

$$\Delta^n = \{x \in \mathbb{R}^{n+1} : x_i \geq 0 \ (0 \leq i \leq n), \quad \textstyle\sum_{i=0}^{n} x_i = 1\}.$$

Its vertices are $e_0, \ldots, e_n$. A point belongs to the face spanned by $\{e_i : i \in I\}$ precisely when its positive coordinates all have indices in I.

Theorem 302 (Sperner's lemma)**.** *Let $\mathcal{T}$ be a finite triangulation of Δ^n. Give each vertex v a label $\ell(v) \in \{0, \ldots, n\}$ satisfying*

$$\ell(v) = i \quad \Longrightarrow \quad v_i > 0.$$

Then the number of n-simplices whose vertices have all $n+1$ labels is odd. In particular, at least one such fully labelled *simplex exists.*

The boundary condition says that a point may use only the labels of the original vertices spanning its smallest face. In particular, e_i has label i, and label i is forbidden on the opposite face $x_i = 0$.

Count faces missing one label. In one dimension, the endpoint labels differ, so the label must change an odd number of times along the subdivided interval. In higher dimensions, count the facets using labels $0, \ldots, n-1$. A fully labelled simplex has exactly one such facet. A simplex which uses all these labels but not n has exactly two, because one label is repeated. Thus modulo 2, this facet count detects exactly the fully labelled simplices.

The same facets can also be counted from the boundary. The boundary condition forces every one of them onto the single original face $x_n = 0$, where the lower-dimensional version of the lemma applies.

Proof. Induct on n. For $n = 0$ there is one point, with its required label. For $n \geq 1$, call an $(n-1)$-simplex *special* if its labels are exactly $0, \ldots, n-1$.

An n-simplex has one special facet if it is fully labelled, two if its label set is exactly $\{0, \ldots, n-1\}$, and none otherwise. Indeed, in the second case precisely one label occurs twice, and either of its vertices can be deleted. If N counts fully labelled simplices, R counts simplices of the second type, and I, B count interior and boundary special facets, respectively, counting incidences gives

$$N + 2R = 2I + B, \qquad \text{so} \qquad N \equiv B \pmod 2.$$

A boundary special facet cannot lie on $x_i = 0$ for $i < n$, since it has a vertex labelled i. It therefore lies on $x_n = 0$. Conversely, the special facets there are exactly the fully labelled simplices of the induced triangulation of that $(n-1)$-simplex. Its labelling is again a Sperner labelling, so induction makes B odd. □

One can read the same count as the handshaking lemma. Make a graph whose vertices are the top-dimensional simplices and one exterior vertex; join across special facets. Fully labelled simplices have degree one, all other interior vertices have degree zero or two, and the exterior vertex has odd degree. Another odd-degree vertex must exist.

7.12.2 Brouwer's Fixed-Point Theorem

Theorem 303 (Brouwer)**.** *Every continuous map $f : K \to K$, where K is a nonempty compact convex subset of a finite-dimensional Euclidean space, has a fixed point. In particular, this holds for a simplex, cube, or closed ball.*

Label a coordinate with excess mass. First let $K = \Delta^n$. At any point which is not fixed, some coordinate satisfies $x_i > f_i(x)$: the coordinate differences sum to zero, so if they are not all zero, at least one is positive. Label the point by such an i. This automatically obeys Sperner's boundary condition, since $x_i > f_i(x) \geq 0$.

A fully labelled small simplex now contains, for every i, a point at which coordinate i has positive excess. These are initially *different points*. Shrinking the simplex and taking a common limit makes every excess nonnegative at *one point*. Their sum is still zero, forcing every excess to vanish.

Why arbitrarily fine triangulations exist. The only geometric preparation is a subdivision whose mesh decreases. It will also preserve the antipodal symmetries needed later.

Lemma 304 (Barycentric refinement)**.** *A finite triangulation of dimension at most $n \geq 1$ has a subdivision whose mesh is at most $n/(n+1)$ times its original mesh. This subdivision respects every subcomplex and commutes with $x \mapsto -x$. Consequently, repeated subdivision gives mesh tending to zero while preserving any prescribed boundary or antipodal symmetry.*

Proof. For every nonempty simplex F, introduce its barycentre b_F, the average of its vertices. The new simplices have vertices $b_{F_0}, \ldots, b_{F_k}$ for strictly nested nonempty faces $F_0 \subsetneq \cdots \subsetneq F_k$. This is a triangulation: inductively subdivide each boundary and cone its simplices to the barycentre. The construction agrees on shared faces, and barycentres commute with negation.

For nested faces $F \subsetneq G$ with p and q vertices, respectively, write $b_{G\setminus F}$ for the average of the remaining vertices. Then

$$b_G = \frac{p}{q} b_F + \frac{q-p}{q} b_{G\setminus F}, \qquad \|b_F - b_G\| \leq \frac{q-p}{q} \operatorname{diam}(G) \leq \frac{n}{n+1} \operatorname{diam}(G).$$

Every pair of vertices of a new simplex comes from nested faces. A simplex's diameter equals the largest distance between its vertices, so the mesh contracts by the claimed factor. □

Proof of Theorem 303. First suppose $f : \Delta^n \to \Delta^n$; the case $n = 0$ is immediate. Choose triangulations with mesh tending to zero. If a triangulation vertex is fixed, we are done. Otherwise label every vertex v by an index i with $v_i > f_i(v)$.

By Sperner's lemma, the rth triangulation has a fully labelled simplex with vertices $v_0^{(r)}, \ldots, v_n^{(r)}$, where $v_i^{(r)}$ has label i. Pass to a subsequence along which $v_0^{(r)} \to x$, using compactness. Since the simplex diameters tend to zero, $v_i^{(r)} \to x$ for every i. Continuity gives

$$x_i - f_i(x) = \lim_{r\to\infty} \left((v_i^{(r)})_i - f_i(v_i^{(r)}) \right) \geq 0.$$

The left sides sum to zero, so each is zero and $f(x) = x$. By an affine change of coordinates, the result holds for every simplex.

For general K, choose a simplex Q containing it and let $\pi_K(x)$ be the point of K nearest to x in Euclidean distance. This point exists by compactness and is unique by convexity: if $p \neq q$ were both nearest, their midpoint would be closer, since

$$\left\| x - \frac{p+q}{2} \right\|^2 = \frac{\|x-p\|^2 + \|x-q\|^2}{2} - \frac{\|p-q\|^2}{4}.$$

The projection is continuous. Indeed, if $x_j \to x$, every convergent subsequence of $\pi_K(x_j)$ has a limit $p \in K$ satisfying $\|x-p\| \le \|x-y\|$ for every $y \in K$, by passing to the limit in the minimizing inequalities. Uniqueness forces $p = \pi_K(x)$; compactness then implies $\pi_K(x_j) \to \pi_K(x)$.

Apply the simplex case to $f \circ \pi_K : Q \to K \subseteq Q$. Its fixed point x lies in K, where $\pi_K(x) = x$, and hence $f(x) = x$. □

The limiting argument also quantifies the approximation. Put $g(x) = x - f(x)$ on Δ^n, and let $\omega(\delta) = \sup_{\|x-y\| \le \delta} \|g(x) - g(y)\|_\infty$. In a fully labelled simplex of diameter at most δ, every point x satisfies $g_i(x) \ge -\omega(\delta)$ for all i. Since $\sum_i g_i(x) = 0$, also $g_i(x) \le n\omega(\delta)$, so

$$\|x - f(x)\|_\infty \le n\omega(\delta) \longrightarrow 0.$$

7.12.3 Tucker's Lemma

Use the polyhedral n-ball

$$P_n = \{x \in \mathbb{R}^n : \|x\|_1 \le 1\} = \operatorname{conv}\{\pm e_1, \dots, \pm e_n\}.$$

This *cross-polytope* is an interval in dimension one, a diamond in dimension two, and an octahedron in dimension three. Working with P_n makes the simplices ordinary straight simplices. Radial rescaling identifies it with the closed Euclidean ball and takes antipodal boundary points to antipodal points.

A boundary triangulation is *antipodally symmetric* if σ is a boundary simplex precisely when $-\sigma$ is. A labelling is *antipodal* there if $\ell(-v) = -\ell(v)$. An edge whose labels are $i, -i$ is called *complementary*.

Theorem 305 (Tucker's lemma, ball form)**.** *Let $n \ge 1$, and let $\mathcal{T}$ triangulate P_n, with an antipodally symmetric boundary triangulation. Label its vertices from $\{\pm 1, \dots, \pm n\}$, with $\ell(-v) = -\ell(v)$ on the boundary. Then $\mathcal{T}$ contains a complementary edge.*

Neither the interior triangulation nor the interior labels must be antipodal. In dimension one the lemma says that a sequence beginning with one sign and ending with the other has adjacent opposite signs. In higher dimensions, labels can also change their absolute values, so merely counting positive-to-negative transitions is insufficient: $+1, -2$ is not a complementary edge. We need a pattern which records both the signs and the different indices.

The right pattern is alternation. A labelled k-simplex is *alternating* if its labels, sorted by increasing absolute value, have distinct absolute values and alternating signs. Thus they are

$$+a_0, -a_1, +a_2, \dots \quad \text{or} \quad -a_0, +a_1, -a_2, \dots, \qquad 1 \le a_0 < \cdots < a_k.$$

Its sign is the sign of its smallest label in absolute value. Every vertex is alternating, with the sign of its label.

This is the natural generalization of a non-complimentary edge for higher simplices and reduces to it. The important property is how it behaves under deleting a vertex. An alternating simplex of positive dimension has one positive alternating facet and one negative alternating facet. A nonalternating simplex with no complementary edge has either no alternating facets, or two with the *same* sign. Consequently, counting only the positive facets detects alternating simplices modulo 2, just as counting special facets detected fully labelled simplices in Sperner's lemma.

Lemma 306 (Alternating-facet parity)**.** *Let σ be a k-simplex, where $k \ge 1$, with nonzero integer labels and no complementary edge. Then*

$$\boxed{\#\{\textit{positive alternating facets of } \sigma\} \equiv 1_{\{\sigma \textit{ is alternating}\}} \pmod 2.}$$

Labels may repeat.

Proof. Sort all labels by nondecreasing absolute value. Equal absolute values have equal signs, since opposite labels would form a complementary edge. Thus the simplex is alternating exactly when consecutive signs always differ.

If the sequence alternates, deleting its first or last term gives an alternating facet, and these have opposite signs. Deleting any internal term makes its two neighbours, which have equal signs, adjacent. There is therefore exactly one positive alternating facet.

If exactly one adjacent pair has equal signs, deleting either member of that pair restores alternation. Both resulting facets have the same sign, giving either two positive facets or none. This includes a pair of repeated labels.

If at least two adjacent pairs have equal signs, deleting one term cannot repair them all. Two separated defects cannot both be removed; if both touch the deleted term, three consecutive signs were equal and the two remaining ones are still equal. Hence no alternating facet exists. □

Now sum the identity over a triangulated k-ball B with no complementary edge. Interior facets are counted twice and disappear, leaving

$$\boxed{\#\{\text{alternating } k\text{-simplices in } B\} \equiv \#\{\text{positive alternating } (k-1)\text{-simplices in } \partial B\} \pmod 2.} \tag{78}$$

The count on the left includes *both* signs. Keeping only positive facets on the right is what makes antipodal symmetry useful in the next step: a positive simplex on one hemisphere corresponds to a negative simplex on the other.

Antipodality forces the boundary count to be odd. The boundary of P_{d+1} splits into two hemispheres along $x_{d+1} = 0$; their common equator is ∂P_d. These hemispheres and successive equators are unions of faces of the cross-polytope, so any straight-simplex triangulation of its boundary respects them. Indeed, a simplex lying on the boundary of a convex polytope lies in one of its supporting faces, and intersection with a supporting face is a face of that simplex.

We can therefore repeat the same count on a hemisphere, then on a hemisphere of its equator, until reaching a pair of points. At the bottom, antipodality says exactly one point has a positive label. This one odd contribution propagates back up through every dimension.

Lemma 307 (Ky Fan's parity lemma on the cross-polytope)**.** *Let $d \geq 0$, and let $\mathcal{T}$ be an antipodally symmetric triangulation of ∂P_{d+1}, with an antipodal labelling from $\{\pm 1, \ldots, \pm m\}$ and no complementary edge. The number of positive alternating d-simplices is odd, and equals the number of negative alternating d-simplices. In particular, $m \geq d+1$.*

Proof. Induct on d. For $d = 0$, the sphere consists of two antipodal points with opposite labels, exactly one positive.

For $d \geq 1$, let H be the upper hemisphere $x_{d+1} \geq 0$. Projection onto the first d coordinates identifies it with P_d: the inverse is $x \mapsto (x, 1 - \|x\|_1)$. Thus H is a triangulated d-ball, and its boundary is the equator ∂P_d. By Equation (78) and induction, the number of alternating d-simplices in H, with either sign, is odd.

Antipodality exchanges positive and negative simplices. Hence the positive alternating simplices on the whole sphere consist of the positive ones in H and antipodes of the negative ones in H. Their total equals the number just shown to be odd. The antipodal bijection also makes the global positive and negative counts equal. An alternating d-simplex uses $d+1$ distinct absolute labels, proving the last assertion. □

Proof of Theorem 305. Suppose there were no complementary edge. Apply Fan's lemma to the boundary ∂P_n: it has an odd number of positive alternating $(n-1)$-simplices. By Equation (78), the ball must then have an odd number of alternating n-simplices. But each would require $n+1$ distinct absolute labels, and only n are available. □

Corollary 308 (Tucker's lemma, sphere form)**.** *For $n \geq 1$, an antipodally symmetric triangulation of ∂P_{n+1}, labelled antipodally from $\{\pm 1, \ldots, \pm n\}$, has a complementary edge.*

Proof. Otherwise Fan's lemma, in dimension n, would force an alternating n-simplex, again requiring $n+1$ distinct absolute labels. □

7.12.4 The Borsuk–Ulam Theorem

Write $S^n = \{x \in \mathbb{R}^{n+1} : \|x\|_2 = 1\}$. A map g on S^n is *odd* if $g(-x) = -g(x)$.

Theorem 309 (Borsuk–Ulam)**.** *For every continuous $f : S^n \to \mathbb{R}^n$, where $n \geq 1$,*

$$\boxed{\text{there exists } x \in S^n \text{ with } f(x) = f(-x).}$$

Equivalently, every continuous odd map $g : S^n \to \mathbb{R}^n$ has a zero.

Label the largest coordinate. The difference $g(x) = f(x) - f(-x)$ is odd, so it suffices to find a zero of an odd map. At a point where $g \neq 0$, label by the signed index of a coordinate attaining $\|g\|_\infty$, breaking ties by the smallest index. Opposite points receive opposite labels, exactly as Tucker requires.

A complementary edge now has one endpoint at which coordinate i is positive and largest in magnitude, and another where that same coordinate is negative and largest in magnitude. If the edge is short, continuity says those two coordinate values are close. They have opposite signs, so both are small; because they are largest in magnitude, *all* coordinates at both endpoints are small.

Proof. First prove the odd-map formulation. Put $Q = \partial P_{n+1}$ and let $\rho : Q \to S^n$ be radial projection, $\rho(u) = u/\|u\|_2$. It is an odd homeomorphism, with inverse $x \mapsto x/\|x\|_1$. The continuous map $h = g \circ \rho : Q \to \mathbb{R}^n$ is therefore odd.

Take repeated barycentric subdivisions of the boundary triangulation of P_{n+1}. They remain antipodally symmetric and have mesh tending to zero. If any vertex satisfies $h(v) = 0$, the conclusion follows. Otherwise give every vertex the signed maximal-coordinate label just described. The tie rule chooses the same index at v and $-v$, so this labelling is antipodal.

By Corollary 308, the rth triangulation contains an edge $u_r v_r$ labelled $+i_r, -i_r$, respectively. Maximality of the chosen coordinates gives

$$\begin{aligned} \|h(u_r)\|_\infty + \|h(v_r)\|_\infty &= h_{i_r}(u_r) - h_{i_r}(v_r) \\ &\leq \|h(u_r) - h(v_r)\|_\infty \longrightarrow 0. \end{aligned}$$

The convergence follows from uniform continuity on compact Q and $\|u_r - v_r\| \to 0$. Pass to a convergent subsequence $u_r \to u \in Q$. Then $h(u) = 0$, so $g(\rho(u)) = 0$.

For arbitrary f, apply this result to the odd map $g(x) = f(x) - f(-x)$. Conversely, applying the antipodal-equality formulation to an odd g gives $g(x) = g(-x) = -g(x)$, hence $g(x) = 0$. □

Equivalently, there is no continuous odd map $S^n \to S^{n-1}$. Such a map would be an odd map into $\mathbb{R}^n$ with no zero; conversely, a zero-free odd map into $\mathbb{R}^n$ could be normalized by its Euclidean norm to obtain one into S^{n-1}.

7.13 Discrepancy Theory

7.13.1 Komlós's Theorem

Theorem 310 (Komlós)**.** *Let $v_1, \ldots, v_n \in \mathbb{R}^d$ satisfy $\|v_i\|_2 \leq 1$. There are signs $\varepsilon_1, \ldots, \varepsilon_n \in \{-1, 1\}$ such that*

$$\left\| \sum_{i=1}^{n} \varepsilon_i v_i \right\|_\infty \leq 9\pi.$$

The proof has two steps: one lemma converts overlap of translated distributions into signs, and a second lemma constructs a distribution whose overlap is controlled by Euclidean distance.

Proof sketch. We will construct a mean-zero probability density P supported on $[-1,1]^d$ such that

$$\mathrm{ov}_P(z) \geq 1 - \frac{\pi}{2}\|z\|_2, \tag{79}$$

where $\mathrm{ov}_P(z)$ denotes the common mass of P and its translate by z. We then scale

$$u_i = \frac{v_i}{9\pi}.$$

Since $\|v_i\|_2 \leq 1$, (79) gives

$$\mathrm{ov}_P(6u_i) \geq 1 - \frac{\pi}{2} \cdot \frac{6}{9\pi} = \frac{2}{3}.$$

The signing lemma will show that these overlap inequalities let us reweight P, without ever leaving its support, until its mean becomes

$$\sum_i \varepsilon_i u_i = \frac{1}{9\pi} \sum_i \varepsilon_i v_i.$$

Because every mean of a distribution supported on $[-1,1]^d$ still lies in $[-1,1]^d$, the theorem follows immediately.

Thus there are only two real questions:

1. Why does overlap at distance $6u_i$ let us choose a sign for u_i?
2. Why can one distribution in a fixed cube have large overlap with *every* short Euclidean translate, independently of d?

The first question is combinatorial. For one vector u_i, we move to a temporary state space in which some mass remembers two possible parents, separated by $6u_i$. We recurse there, so the other signs are chosen before the sign of u_i, and then project back to the old support and resolve the postponed choice. The second question is analytic. We take the square root of the density to be a product of cosine waves; translating such a product changes its L_2 energy according to $\|z\|_2$, because the coordinate derivatives are orthogonal.

Why moving a mean helps. A sign is an indivisible choice, but probability mass can be split in arbitrary proportions. This gives a useful relaxation: instead of moving a single point by exactly $\pm u$, we may move different pieces of mass by $\pm 3u$ and arrange that their *average* displacement is exactly $+u$ or $-u$. Once the resulting distribution is still supported in the same cube, convexity converts the mean bound back into a discrepancy bound.

If at least $1/3$ of the mass still has both choices $\pm 3u$ available, then that flexible mass can contribute any coefficient in $[-1,1]$ times u. The remaining mass is at most $2/3$, so even if it is forced arbitrarily to one parent or the other, it contributes some coefficient in $[-2,2]$. Every number in $[-2,2]$ lies within distance one of one of the two signs $\{-1,1\}$, so the flexible mass can correct the coefficient to an exact sign.

Overlap and notation. During the proof we attach auxiliary bits to the distribution. Formally, a probability distribution on $\mathbb{R}^d \times \{0,1\}^q$ is represented by densities $P_b(x)$, $b \in \{0,1\}^q$, with

$$\sum_b \int_{\mathbb{R}^d} P_b(x)\,dx = 1.$$

Its mean is

$$\mu(P) = \sum_b \int (x,b) P_b(x)\,dx.$$

Translations act only on the $\mathbb{R}^d$ coordinates. Define the overlap of P with its translate by $z \in \mathbb{R}^d$ as

$$\mathrm{ov}_P(z) := \sum_b \int \min\{P_b(x), P_b(x-z)\}\,dx.$$

This is exactly the amount of probability mass the two distributions have in common. Equivalently,

$$1 - \mathrm{ov}_P(z) = \frac{1}{2} \sum_b \int |P_b(x) - P_b(x - z)| \, dx.$$

Lemma 311 (Signing from overlap)**.** *Let P be a compactly supported probability distribution on $\mathbb{R}^d \times \{0,1\}^q$, and let $u_1, \dots, u_n \in \mathbb{R}^d$. If*

$$\mathrm{ov}_P(6u_i) \geq \frac{2}{3} \qquad (i = 1, \dots, n),$$

then there are signs $\varepsilon_i \in \{-1, 1\}$ and a probability distribution R, supported wherever P is supported, such that

$$\mu(R) = \mu(P) + \left(\sum_{i=1}^{n} \varepsilon_i u_i, 0 \right).$$

The final zero means that all auxiliary-bit coordinates keep the same mean.

Proof. Induct on n, allowing any number of auxiliary bits. The case $n = 0$ is trivial. Write $u = u_n$. The recursive step has four parts:

1. create mass with two possible parents for the vector u;
2. show that this preparation does not reduce the overlap needed for the other vectors;
3. recurse on $u_1, \dots, u_{n-1}$ while remembering how much flexible mass was created;
4. return to the old support and use that flexible mass to round the remaining displacement to $+u$ or $-u$.

1. Prepare a future choice of parent. Fix an old bit string $b \in \{0,1\}^q$. For a location x, set

$$A = P_b(x + 3u), \qquad B = P_b(x - 3u).$$

These are the masses whose original locations were the two candidate parents $x + 3u$ and $x - 3u$. Append one new bit and define

$$\begin{aligned} Q_{(b,0)}(x) &= \frac{1}{2} \max\{A, B\}, \\ Q_{(b,1)}(x) &= \frac{1}{2} \min\{A, B\}. \end{aligned} \tag{80}$$

If the new bit is 0, at least one parent is available. If it is 1, both parents are available. Thus the bit 1 marks exactly the mass on which we retain a choice for later. Here "available" only means that the parent lies in the support of the old distribution.

The marginal mass at x is

$$Q_{(b,0)}(x) + Q_{(b,1)}(x) = \frac{1}{2}(A + B) = \frac{1}{2}\big(P_b(x + 3u) + P_b(x - 3u)\big).$$

Hence Q is a probability distribution: summing over b and integrating,

$$\sum_b \int \big(Q_{(b,0)} + Q_{(b,1)}\big) = \frac{1}{2}(1 + 1) = 1.$$

Its mean is unchanged. Indeed,

$$\begin{aligned} &\sum_b \int x\big(Q_{(b,0)}(x) + Q_{(b,1)}(x)\big) \, dx \\ &\quad = \frac{1}{2} \sum_b \int x P_b(x + 3u) \, dx + \frac{1}{2} \sum_b \int x P_b(x - 3u) \, dx \\ &\quad = \frac{1}{2}\big(\mu_x(P) - 3u\big) + \frac{1}{2}\big(\mu_x(P) + 3u\big) = \mu_x(P). \end{aligned}$$

For each old bit string b, the total mass with label b is also unchanged, so all old auxiliary-bit means are preserved.

Let β be the probability that the new bit equals 1. From (80),

$$\begin{aligned}\beta &= \frac{1}{2}\sum_b \int \min\{P_b(x+3u), P_b(x-3u)\}\,dx \\ &= \frac{1}{2}\,\mathrm{ov}_P(6u) \geq \frac{1}{3},\end{aligned}$$

where the second equality is the change of variables $y = x + 3u$. Thus

$$\mu(Q) = (\mu(P), \beta).$$

The new coordinate has converted the condition of least β mass still having two choices into a constraint on the mean.

2. Preparing u does not damage the other directions. This is the reason for the particular max–min split. Let $\mathcal{S}_u$ denote the operation in (80), applied also to nonnegative functions of arbitrary total mass. It has two simple properties:

$$\begin{aligned}F \leq G \quad &\Longrightarrow \quad \mathcal{S}_u F \leq \mathcal{S}_u G, \\ \int \mathcal{S}_u F &= \int F.\end{aligned}$$

The first is monotonicity of max and min in each argument; the second is the identity $\max\{A, B\} + \min\{A, B\} = A + B$. It also commutes with spatial translations.

Fix another shift z, and let

$$H_b(x) = \min\{P_b(x), P_b(x - z)\}.$$

Then $H \leq P$ and $H \leq P(\cdot - z)$. Monotonicity and translation invariance give

$$\mathcal{S}_u H \leq Q, \qquad \mathcal{S}_u H \leq Q(\cdot - z).$$

Thus $\mathcal{S}_u H$ is common mass of Q and its translate by z. Since splitting preserves total mass,

$$\mathrm{ov}_Q(z) \geq \int \mathcal{S}_u H = \int H = \mathrm{ov}_P(z).$$

So preparing a choice for u_n cannot consume the overlap required later for $u_1, \ldots, u_{n-1}$.

3. Recurse while preserving the amount of flexible mass. Apply the induction hypothesis to Q and the vectors $u_1, \ldots, u_{n-1}$, extended by zero in every auxiliary coordinate. We obtain signs $\varepsilon_1, \ldots, \varepsilon_{n-1}$ and a distribution R' supported on $\mathrm{supp}(Q)$ such that

$$\mu(R') = \mu(Q) + \sum_{i<n} \varepsilon_i (u_i, 0) = \left(\mu(P) + \sum_{i<n} \varepsilon_i u_i, \beta\right),$$

where the final coordinate is the new bit and $\mu(P)$ already includes all older auxiliary coordinates. Because the last coordinate is binary,

$$\mathbb{E}_{R'}[b_{q+1}] = \Pr_{R'}[b_{q+1} = 1] = \beta.$$

Therefore the recursion may rearrange the flexible mass in space, but it cannot change how much of it remains.

4. Return to the old support and round to a sign. We now remove the new bit by sending each state of R' to one of its available parents in the old support.

First consider the states with new bit 0. Choose one available parent for each such state, for example the $x + 3u$ parent whenever it is available and the $x - 3u$ parent otherwise. Their total mass is $1 - \beta$, and every move is by either $+3u$ or $-3u$. If m_+ and m_- are the masses sent in the two directions, then

$$m_+ + m_- = 1 - \beta$$

and their total contribution to the change in the mean is

$$3(m_+ - m_-)u =: au.$$

Hence

$$|a| = 3|m_+ - m_-| \le 3(m_+ + m_-) = 3(1-\beta) \le 2.$$

Now consider the states with new bit 1. They have total mass β, and every such state has both parents available. Send the same fraction θ of this mass to the $+3u$ parent and the remaining fraction $1-\theta$ to the $-3u$ parent. The contribution to the mean is

$$3\beta\theta u - 3\beta(1-\theta)u = 3\beta(2\theta - 1)u.$$

As θ runs from 0 to 1, the coefficient runs through $[-3\beta, 3\beta]$, which contains $[-1, 1]$ because $\beta \ge 1/3$.

Since $a \in [-2, 2]$, at least one of the signs $\varepsilon_n \in \{-1, 1\}$ satisfies

$$|\varepsilon_n - a| \le 1.$$

Choose such a sign and set

$$\theta = \frac{1}{2} + \frac{\varepsilon_n - a}{6\beta}.$$

The inequality $|\varepsilon_n - a| \le 1 \le 3\beta$ ensures $0 \le \theta \le 1$, and then

$$3\beta(2\theta - 1) = 3\beta \cdot \frac{\varepsilon_n - a}{3\beta} = \varepsilon_n - a.$$

Thus the flexible mass contributes $(\varepsilon_n - a)u$, while the unmarked mass contributes au. Their sum is exactly

$$au + (\varepsilon_n - a)u = \varepsilon_n u.$$

Every transferred point is an available parent, hence lies in the support of P, and all old auxiliary bits are unchanged. This completes the induction. □

Why a cosine has the right overlap. It remains to build one density inside a fixed cube which overlaps strongly with every short Euclidean translate.

A uniform density is the wrong shape. It has a sharp boundary: translating it slightly immediately chops off a slab of mass at each exposed face, and these coordinatewise losses naturally accumulate like an ℓ_1 norm. We instead want a density which dies smoothly at the boundary.

The cosine is the natural one-dimensional wave for this. On $[-1, 1]$,

$$h(t) = \cos(\pi t/2)$$

is one half-wave: it is largest in the middle and vanishes exactly at the two endpoints. If we translate it slightly the new profile still lies almost on top of the old one; unlike a relatively flat distribution, there is no jump at the boundary to create an immediate portion of lost mass. Its derivative is a sine, ninety degrees out of phase with the cosine. Algebraically this is

$$\int hh' = 0.$$

For a product $f(x) = \prod_j h(x_j)$, changing different coordinates therefore produces orthogonal L_2 directions. Their squared costs add, so a translation by z costs $\sqrt{z_1^2 + \cdots + z_d^2}$ rather than $|z_1| + \cdots + |z_d|$.

We use the density $P = f^2$. Squaring has two advantages: it makes a nonnegative probability density, and

$$\partial_z P = 2f\,\partial_z f$$

lets Cauchy–Schwarz convert change in probability mass into the L_2 energy of the translated wave.

Lemma 312 (Overlap in the cube)**.** *There is a mean-zero probability density P supported on $[-1,1]^d$ such that, for every $z \in \mathbb{R}^d$,*

$$\mathrm{ov}_P(z) \geq 1 - \frac{\pi}{2}\|z\|_2.$$

Proof. Define

$$h(t) = \begin{cases} \cos(\pi t/2), & |t| \leq 1, \\ 0, & |t| > 1, \end{cases} \qquad f(x) = \prod_{j=1}^{d} h(x_j), \qquad P(x) = f(x)^2.$$

On $(-1,1)$,

$$h'(t) = -\frac{\pi}{2}\sin(\pi t/2).$$

The endpoint values of h' are irrelevant to the integrals below. We need three one-dimensional identities:

$$\int h^2 = 1, \qquad \int hh' = 0, \qquad \int (h')^2 = \frac{\pi^2}{4}. \tag{81}$$

Indeed,

$$\int_{-1}^{1} \cos^2(\pi t/2)\,dt = 1, \qquad \int_{-1}^{1} \sin^2(\pi t/2)\,dt = 1,$$

and

$$\int hh' = \frac{1}{2}\int (h^2)' = \frac{1}{2}\big(h(1)^2 - h(-1)^2\big) = 0.$$

Since $\int h^2 = 1$, we have

$$\int_{\mathbb{R}^d} f(x)^2\,dx = \prod_{j=1}^{d} \int h(x_j)^2\,dx_j = 1.$$

Thus $P = f^2$ is a probability density. It is even in every coordinate, so its mean is zero, and it is supported on $[-1,1]^d$.

Now fix $z = (z_1, \ldots, z_d)$. Write

$$\partial_z f = \sum_{j=1}^{d} z_j \partial_j f.$$

For each j,

$$\partial_j f(x) = h'(x_j) \prod_{k \neq j} h(x_k).$$

Hence, using (81),

$$\begin{aligned} \|\partial_j f\|_2^2 &= \int h'(x_j)^2\,dx_j \prod_{k \neq j} \int h(x_k)^2\,dx_k \\ &= \frac{\pi^2}{4}. \end{aligned}$$

If $j \neq k$, then

$$\begin{aligned} \langle \partial_j f, \partial_k f \rangle &= \left(\int hh'\right)^2 \prod_{\ell \neq j,k} \int h^2 \\ &= 0. \end{aligned}$$

Thus the coordinate derivatives are pairwise orthogonal. Expanding the square gives

$$\|\partial_z f\|_2^2 = \left\| \sum_j z_j \partial_j f \right\|_2^2 = \sum_j z_j^2 \|\partial_j f\|_2^2 + 2\sum_{j<k} z_j z_k \langle \partial_j f, \partial_k f \rangle = \frac{\pi^2}{4}\sum_j z_j^2 = \frac{\pi^2}{4}\|z\|_2^2.$$

Therefore

$$\|\partial_z f\|_2 = \frac{\pi}{2}\|z\|_2. \tag{82}$$

We finally convert this energy estimate into overlap. The density $P = f^2$ is continuously differentiable on all of $\mathbb{R}^d$: although h' has a jump at ± 1, the derivative of h^2 vanishes there because $h = 0$. For every x,

$$P(x) - P(x - z) = \int_0^1 \partial_z P(x - tz)\, dt,$$

up to an irrelevant overall sign. We have that $\min\{a, b\} = (a + b - |a - b|)/2$ and both P and its translate have total mass one. Hence, by the triangle inequality and translation invariance of Lebesgue measure,

$$\begin{aligned} 1 - \mathrm{ov}_P(z) &= \frac{1}{2}\int |P(x) - P(x - z)|\, dx \\ &\le \frac{1}{2}\int_0^1 \int |\partial_z P(x - tz)|\, dx\, dt \\ &= \frac{1}{2}\int |\partial_z P(x)|\, dx. \end{aligned}$$

Since $P = f^2$, we have $\partial_z P = 2f\, \partial_z f$ almost everywhere, so Cauchy–Schwarz and (82) give

$$\begin{aligned} 1 - \mathrm{ov}_P(z) &\le \int |f(x)\partial_z f(x)|\, dx \\ &\le \|f\|_2\, \|\partial_z f\|_2 \\ &= 1 \cdot \frac{\pi}{2}\|z\|_2. \end{aligned}$$

Rearranging proves the claim. □

Proof of Theorem 310. Take P from Lemma 312 and define

$$u_i = \frac{v_i}{9\pi}.$$

For every i,

$$\begin{aligned} \mathrm{ov}_P(6u_i) &\ge 1 - \frac{\pi}{2}\|6u_i\|_2 \\ &= 1 - \frac{\pi}{2} \cdot \frac{6}{9\pi}\|v_i\|_2 \\ &\ge 1 - \frac{1}{3} = \frac{2}{3}. \end{aligned}$$

Therefore Lemma 311 gives signs ε_i and a probability distribution R supported on $[-1, 1]^d$ with

$$\mu(R) = \sum_i \varepsilon_i u_i = \frac{1}{9\pi}\sum_i \varepsilon_i v_i.$$

Because every coordinate of every point in the support of R lies between -1 and 1, the same is true of every coordinate of its mean:

$$\|\mu(R)\|_\infty \le 1.$$

Substituting the displayed expression for $\mu(R)$ gives

$$\frac{1}{9\pi}\left\|\sum_i \varepsilon_i v_i\right\|_\infty \le 1,$$

and hence

$$\left\|\sum_i \varepsilon_i v_i\right\|_\infty \le 9\pi.$$

□

7.13.2 Rank-One Matrix Discrepancy

For a Hermitian matrix S, its operator norm is $\|S\| = \max_{\|x\|_2=1} |x^*Sx|$. Thus matrix discrepancy asks for one choice of coefficients which controls every quadratic direction simultaneously, including both positive and negative eigenvalues. Here u^* denotes conjugate transpose.

Theorem 313 (Rank-one matrix discrepancy)**.** *Let $A_i = u_i u_i^*$, where $u_i \in \mathbb{C}^d$, and let $\xi_1, \dots, \xi_n$ be independent finitely supported real random variables. Put*

$$\mu_i = \mathbb{E}\xi_i, \qquad \sigma^2 = \left\| \sum_i \mathrm{Var}(\xi_i) A_i^2 \right\|.$$

There are outcomes $\varepsilon_i \in \mathrm{supp}(\xi_i)$ such that

$$\left\| \sum_i (\varepsilon_i - \mu_i) A_i \right\| \le 2\sqrt{2}\,\sigma.$$

The scale σ already controls each fixed direction: for $S = \sum_i (\xi_i - \mu_i) A_i$, independence gives $\mathbb{E}S^2 = \sum_i \mathrm{Var}(\xi_i) A_i^2$, and hence $\mathbb{E}\|Sx\|_2^2 \le \sigma^2$ for every unit vector x. The maximizing direction can depend on the outcome, so these separate second-moment bounds do not themselves give the conclusion. We will instead encode all the eigenvalues in one polynomial and control its largest root.

Encoding and averaging. For Hermitian S, set

$$\chi_S(x) = \det(xI - S)\det(xI + S) = \prod_{k=1}^{d} (x^2 - \lambda_k(S)^2).$$

Its largest root is exactly $\|S\|$. The second determinant is essential: a bound on the largest root of $\det(xI - S)$ alone would not control large negative eigenvalues. Write $\lambda_{\max}(f)$ for the largest root of a real-rooted polynomial f.

The plan is to bound the largest root of $\mathbb{E}\chi_S$ and then select an outcome with no larger root. This implication is not automatic; it will follow because the expected polynomial remains real-rooted under changes to the independent coefficients.

Rank one makes the expectation simple. By multilinearity in the columns, $\det(M + zuu^*)$ is affine in z: a term using two columns from uu^* vanishes because they are proportional. For invertible M, we have by the Matrix Determinant Lemma that

$$\det(M + zuu^*) = \det(M)(1 + zu^* M^{-1} u).$$

Consequently, the product of the two determinants is quadratic in each random coefficient. Centering removes its linear term, and expectation replaces its quadratic term by the variance.

Lemma 314 (Averaging by differentiation)**.** *Let H be Hermitian, let $B_i \succeq 0$ have rank at most one, and let η_i be independent real random variables with mean zero and variance one. Then*

$$\mathbb{E}\,\chi_{H+\sum_i \eta_i B_i}(x) = \prod_i \left(1 - \tfrac{1}{2}\partial_{z_i}^2\right) \left[\det\left(xI - H + \sum_i z_i B_i \right) \det\left(xI + H + \sum_i z_i B_i \right) \right] \Bigg|_{z=0}.$$

Proof. For one coefficient, the two affine factors have the form $(a - \eta b)(c + \eta d)$. Thus

$$\mathbb{E}[(a - \eta b)(c + \eta d)] = ac - bd = \left(1 - \tfrac{1}{2}\partial_z^2\right)[(a + zb)(c + zd)]\big|_{z=0}.$$

Apply this identity to the independent coefficients one at a time. Expectations, derivatives, and evaluations belonging to distinct indices commute. □

Notice that the auxiliary variable z has the *same* sign in both determinants. This will preserve real-rootedness; the operator $1 - \frac{1}{2}\partial_z^2$ supplies the minus sign required by the expectation.

Preserving real roots. Let $\mathbb{H} = \{z \in \mathbb{C} : \operatorname{Im} z > 0\}$. A nonzero polynomial is *stable* if it has no zero when all its variables lie in $\mathbb{H}$; it is *real stable* if its coefficients are real. In one variable, real stability is exactly real-rootedness, since nonreal roots of a real polynomial occur in conjugate pairs. The multivariate definition lets us preserve this property while applying the averaging operators, before setting $z = 0$.

Lemma 315 (Stability rules)**.** *Products preserve stability. Coefficientwise limits of stable polynomials of bounded degree are stable or zero. The same holds after substituting real constants for variables. For real c, the operation $p \mapsto p + c\partial_i p$ preserves real stability, and so does $p \mapsto (1 - \frac{1}{2}\partial_i^2)p$.*

If H is Hermitian and $B_i \succeq 0$, then $\det(xI + H + \sum_i z_i B_i)$ is real stable in $x, z_1, \ldots, z_n$.

Proof. Products are immediate. For the limit assertion, suppose a nonzero limit p vanishes at an interior point $w \in \mathbb{H}^m$. Choose a complex line through w on which p is not identically zero, and a small disk in that line, contained in $\mathbb{H}^m$, whose boundary contains no zero of p. The approximating polynomials are uniformly bounded away from zero on the boundary. The maximum modulus principle applied to their reciprocals gives the same bound inside, contradicting convergence to zero at w. Substitution of a real constant a follows by substituting $a + i\epsilon$ and letting $\epsilon \downarrow 0$.

For the differential rule, fix the other variables in $\mathbb{H}$ and call the resulting polynomial $f(z)$. Its roots ρ_k lie outside $\mathbb{H}$. If f is nonconstant, then

$$\frac{f'(z)}{f(z)} = \sum_k \frac{1}{z - \rho_k} \quad \text{has strictly negative imaginary part for } z \in \mathbb{H}.$$

It cannot equal the real number $-1/c$, so $f + cf'$ has no zero there. Constant f and $c = 0$ are immediate. Now factor $1 - \frac{1}{2}\partial_i^2 = (1 - \partial_i/\sqrt{2})(1 + \partial_i/\sqrt{2})$.

Finally, the matrix in the determinant has positive definite imaginary part $(\operatorname{Im} x)I + \sum_i (\operatorname{Im} z_i)B_i$. It is nonsingular: a kernel vector $v \neq 0$ would give $0 = \operatorname{Im}(v^* M v) > 0$. Its determinant has real coefficients because it is real on real inputs, where the matrix is Hermitian. □

Applying these rules to Lemma 314 shows that its expected polynomial is real-rooted. It remains monic of degree $2d$, so real specialization cannot produce the zero polynomial.

The same conclusion holds for arbitrary independent finitely supported coefficients ζ_i. Indeed, absorb $\sum_i (\mathbb{E}\zeta_i)B_i$ into H, multiply B_i by the standard deviation of ζ_i, and standardize the nonconstant coefficients. Deterministic coefficients are simply absorbed into H.

Choosing an outcome. The relevant property of real-rooted polynomials has a short explanation. In a linear combination, the largest root cannot dip below the largest roots of both endpoints: a horizontal level would have to be crossed twice, although the polynomial's value at that level is affine in the family parameter.

Lemma 316 (Selection from a real-rooted average)**.** *Suppose every convex combination of monic polynomials $f_1, \ldots, f_k$ of the same positive degree is real-rooted. For any convex combination $f = \sum_i \alpha_i f_i$, some i with $\alpha_i > 0$ satisfies $\lambda_{\max}(f_i) \leq \lambda_{\max}(f)$.*

Proof. First consider $f_t = (1 - t)f_0 + tf_1$. Suppose an interior f_t has largest root below those of both endpoints. Choose a real x strictly between these values which is not an endpoint root. The number of roots in (x, ∞) is positive at both endpoints and zero at this interior parameter.

The ordered roots vary continuously when the polynomial retains its degree, thus the root count can change only when $f_t(x) = 0$. It would have to change at least twice, whereas $f_t(x)$ is a nonzero affine function of t and vanishes at most once. This proves the assertion for two polynomials. Grouping all but one positive-weight term and repeating proves the general case. □

Now condition on any already chosen coefficients. The conditional expected polynomial is an average of the polynomials obtained by fixing the next coefficient. Every convex combination of these children corresponds to another law for that coefficient, keeping the remaining coefficients independent; hence it is real-rooted.

The lemma selects a value without increasing the largest root. After all coefficients are fixed, the polynomial is χ_S. Therefore

$$\text{some supported outcome satisfies} \qquad \|S\| \le \lambda_{\max}(\mathbb{E}\chi_S).$$

We have reduced discrepancy to a largest-root bound for one expected polynomial.

Keeping the test point above the roots. For centered unit-variance coefficients, the polynomial to bound is

$$F(x) = \prod_i (1 - \tfrac{1}{2}\partial_{z_i}^2) \det\left(xI + \sum_i z_i B_i\right)^2 \Bigg|_{z=0}. \tag{83}$$

The operators can move roots towards a proposed upper bound $x = L$. We will begin with negative z_i and move z_i to zero when its operator is applied. Each shift must compensate for that operator *without making any remaining coordinate harder to process.* Logarithmic derivatives give exactly this invariant.

For real stable p, define

$$\mathcal{A}(p) = \{a \in \mathbb{R}^m : p(a+h) > 0 \text{ for all } h \in [0,\infty)^m\}, \qquad \Phi_i^p(a) = \frac{\partial_i p(a)}{p(a)}.$$

We call $a \in \mathcal{A}(p)$ *above the roots.* The roots of $t \mapsto p(a + te_i)$ are then strictly negative; if their distances from zero are $\rho_\ell > 0$, then $\Phi_i^p(a) = \sum_\ell 1/\rho_\ell$. A small logarithmic derivative thus means that no root is too close in that direction.

We also need to control Φ_j when a different coordinate i changes. Stability forces the same reciprocal-distance form, now with nonnegative weights. The signs of these weights are the key: comparisons for the individual fractions can be added without cancellation.

Lemma 317 (Positive partial fractions for the barriers)**.** *If p is real stable and $a \in \mathcal{A}(p)$, then for any i, j,*

$$\Phi_j^p(a + te_i) = c + \sum_\ell \frac{b_\ell}{t + \rho_\ell}, \qquad c, b_\ell \ge 0, \quad \rho_\ell > 0.$$

In particular, each Φ_j^p is nonnegative and nonincreasing when any coordinate increases above the roots.

Proof. Call the rational function on the left $R(t)$. Its denominator $p(a + te_i)$ has only negative real roots, so all its poles are negative. Also $R(t) \ge 0$ for real $t \ge 0$, by factoring $s \mapsto p(a + te_i + se_j)$.

For $i = j$ the representation follows directly by logarithmic differentiation. For $i \ne j$, put $P(t,s) = p(a + te_i + se_j)$, which is real stable by real specialization and satisfies $P(0,0) > 0$. Fix $t \in \mathbb{H}$. As a polynomial in s, $P(t,s)$ has all roots outside $\mathbb{H}$, so its logarithmic derivative at $s = i\epsilon$ has nonpositive imaginary part. Letting $\epsilon \downarrow 0$ gives

$$\operatorname{Im} R(t) \le 0 \qquad (t \in \mathbb{H}).$$

The denominator at $s = 0$ is nonzero there, since $P(t,0)$ is stable by specialization and nonzero at $t = 0$.

This sign condition determines the partial fractions. Near a pole λ of order k with leading coefficient $b \ne 0$, write $t = \lambda + re^{i\theta}$. The leading imaginary part is $-br^{-k}\sin(k\theta)$. It stays nonpositive for $0 < \theta < \pi$ only when $k = 1$ and $b > 0$. Thus every pole is simple with positive residue. The same argument along rays tending to infinity shows that the polynomial part has degree at most one, with nonpositive linear coefficient. It has the form $c - bt$, $b \ge 0$. Since $R(t) \ge 0$ for all real $t \ge 0$, we must have $b = 0$ and $c \ge 0$. This gives the claimed representation. □

The quadratic comparison. Suppose the current polynomial has degree at most two in z_i. Along that coordinate its normalized restriction is $P(t) = (1 + rt)(1 + st)$, with $r, s \ge 0$. The averaging operator subtracts the constant rs. We will shift far enough that the resulting quadratic still dominates each original linear factor. The partial-fraction lemma then turns those two comparisons into bounds for *all* the remaining logarithmic derivatives.

Lemma 318 (Quadratic barrier update)**.** *Let p be real stable with $\deg_{z_i} p \le 2$, let $a \in \mathcal{A}(p)$, and suppose $\Phi_i^p(a) \le \delta$ for some $\delta > 0$. For $q = (1 - \frac{1}{2}\partial_i^2)p$,*

$$a + \delta e_i \in \mathcal{A}(q), \qquad \Phi_j^q(a + \delta e_i) \le \Phi_j^p(a) \quad \text{for every } j.$$

Proof. Factor

$$P(t) := \frac{p(a + te_i)}{p(a)} = (1 + rt)(1 + st), \qquad r, s \ge 0, \qquad r + s \le \delta.$$

Allow a zero value for a missing factor. Put $D = P(\delta) - rs = q(a + \delta e_i)/p(a)$. Since $r, s \le \delta$,

$$D - (1 + \delta r) = s\big[(\delta - r) + \delta^2 r\big] \ge 0.$$

Exchanging r, s gives $D \ge \max\{1 + \delta r, 1 + \delta s\} \ge 1$. This proves positivity at the shifted point. For any $b \ge a$, Lemma 317 gives $\Phi_i^p(b) \le \delta$, so the same calculation shows $q(b + \delta e_i) > 0$. Hence $a + \delta e_i \in \mathcal{A}(q)$; q is real stable by Lemma 315.

For any j, the poles in Lemma 317 can only occur at the roots of P, so

$$\Phi_j^p(a + te_i) = c + \frac{b_1}{1 + rt} + \frac{b_2}{1 + st}, \qquad c, b_1, b_2 \ge 0.$$

For a missing factor set its coefficient to zero; for coincident roots put the single residue in either term. Multiplying by P gives

$$\frac{(\partial_j p)(a + te_i)}{p(a)} = cP(t) + b_1(1 + st) + b_2(1 + rt).$$

Apply $1 - \frac{1}{2}\partial_t^2$. The last two terms are linear, so they are unchanged, while P becomes $P - rs$. Since the coordinate derivatives commute, evaluating at $t = \delta$ and dividing by D gives

$$\begin{aligned}\Phi_j^q(a + \delta e_i) &= c + \frac{b_1(1 + \delta s) + b_2(1 + \delta r)}{D} \\ &\le c + b_1 + b_2 = \Phi_j^p(a).\end{aligned}$$

□

The degree bound is preserved when operators are applied in other variables. Thus rank one does two jobs: it makes averaging depend only on the variance, and it leaves a quadratic at every update.

Matching shifts to the variance matrix. Assume temporarily that $V = \sum_i B_i^2 \preceq I$. For nonzero rank-one $B_i \succeq 0$, write $a_i = \operatorname{tr} B_i = \|B_i\|$. Its sole nonzero eigenvalue is a_i, so $B_i^2 = a_i B_i$. Consequently, shifts proportional to a_i have total matrix cost proportional to V, not to the potentially much larger $\sum_i B_i$.

To choose their scale, start at $z_i = -ta_i$ and $x = L > t > 0$. The determinant matrix is then $LI - tV \succeq (L - t)I$. Larger t allows larger compensating shifts, but leaves less positive-definite margin at the starting point. The initial logarithmic derivative is at most $2a_i/(L - t)$; the factor two comes from the two determinants. The update asks that this be at most the available shift ta_i, so it suffices that $L \ge t + 2/t$. Balancing these two terms gives $t = \sqrt{2}$ and $L = 2\sqrt{2}$.

Lemma 319 (Root bound)**.** *Let $B_i \succeq 0$ have rank at most one and satisfy $V = \sum_i B_i^2 \preceq I$. Then F from (83) has largest root at most $2\sqrt{2}$.*

Proof. Discard zero matrices and set $a_i = \operatorname{tr} B_i$, $\delta_i = \sqrt{2}\, a_i$, $L = 2\sqrt{2}$, and

$$p_0(x, z) = \det\left(xI + \sum_i z_i B_i\right)^2, \qquad w_0 = (L, -\delta_1, \dots, -\delta_n).$$

At this point the determinant matrix is $M = LI - \sqrt{2}V \succeq \sqrt{2}I$. Increasing any coordinate adds a positive semidefinite matrix, so $w_0 \in \mathcal{A}(p_0)$. The rank-one determinant formula gives

$$\Phi_{z_i}^{p_0}(w_0) = 2\,\mathrm{tr}(M^{-1}B_i) \le \sqrt{2}\,a_i = \delta_i,$$

since $M^{-1} \preceq I/\sqrt{2}$.

Process $i = 1, \dots, n$: apply $1 - \frac{1}{2}\partial_{z_i}^2$ and move z_i from $-\delta_i$ to 0. Lemma 318 keeps the current point above the roots and does not increase any remaining barrier. Thus every future update still has $\Phi_{z_j} \le \delta_j$ available. After the last step, $(L, 0, \dots, 0)$ is above the roots of the final polynomial. Therefore $F(x) > 0$ for all $x \ge L$. The polynomial F is monic and real-rooted by the averaging identity and stability rules, so its largest root is at most L. □

Proof of Theorem 313. Put $\tau_i = \sqrt{\mathrm{Var}(\xi_i)}$. If $\sigma = 0$, each positive semidefinite summand $\tau_i^2 A_i^2$ is zero, so every nonzero A_i has a deterministic coefficient and the centered sum vanishes. Otherwise discard indices with $A_i = 0$ or $\tau_i = 0$ and put

$$B_i = \frac{\tau_i A_i}{\sigma}, \qquad \eta_i = \frac{\xi_i - \mu_i}{\tau_i}.$$

These coefficients have mean zero and variance one, and $\sum_i B_i^2 \preceq I$. Their expected polynomial is F from Lemma 319, whose largest root is at most $2\sqrt{2}$. The outcome-selection argument gives supported $\widehat{\eta}_i$ with $\|\sum_i \widehat{\eta}_i B_i\| \le 2\sqrt{2}$. Taking $\varepsilon_i = \mu_i + \tau_i \widehat{\eta}_i$ and multiplying by σ proves the result. Discarded indices may receive any supported outcome. □

Signings and fractional rounding. Uniform signs and Bernoulli coefficients give two useful forms of the theorem.

Corollary 320. *For $A_i = u_i u_i^*$, some signs $s_i \in \{-1, 1\}$ satisfy*

$$\left\| \sum_i s_i A_i \right\| \le 2\sqrt{2} \left\| \sum_i A_i^2 \right\|^{1/2}.$$

For any $\theta_i \in [0, 1]$, some subset S satisfies

$$\left\| \sum_{i \in S} A_i - \sum_i \theta_i A_i \right\| \le 2\sqrt{2} \left\| \sum_i \theta_i(1 - \theta_i) A_i^2 \right\|^{1/2}.$$

If also $\sum_i A_i \preceq I$ and $\|A_i\| \le \epsilon$, the latter error is at most $\sqrt{2\epsilon}$.

Proof. Use independent uniform signs for the first assertion, and independent Bernoulli coefficients of means θ_i for the second. For the last bound,

$$\sum_i \theta_i(1 - \theta_i) A_i^2 \preceq \frac{\epsilon}{4} \sum_i A_i \preceq \frac{\epsilon}{4} I,$$

using $A_i^2 \preceq \epsilon A_i$ and $\theta_i(1 - \theta_i) \le 1/4$. □

Thus every fractional sum can be rounded to a subset sum with an error governed by its variance. In particular, when $\sum_i A_i = I$ and $\theta_i = 1/2$, both parts of the resulting partition are within $\sqrt{2\epsilon}$ of $I/2$ in operator norm.

8 Graph Theory

All graphs are finite. Unless stated otherwise, undirected graphs are simple. Multigraphs may have parallel edges; loops are allowed only where explicitly stated, and degrees count incidences with multiplicity. Digraphs have no loops or parallel arcs, but may have both opposite arcs. An oriented graph has at most one arc between each pair of vertices.

A path has distinct vertices, and its length is its number of edges. A single vertex is a path of length zero. Disjoint paths have no vertex in common, including their endpoints; internally disjoint paths may share their specified endpoints. We write $G - X$ for vertex deletion, $G - e$ for edge deletion, and $\alpha(G)$ for the largest size of an independent vertex set. For a digraph, independence is measured in its underlying undirected graph.

8.1 Max Flow–Min Cut and Menger's Theorem

Let $N = (V, E)$ be a directed network with source s, sink t, and capacities $c_e \geq 0$. For a flow f, write $|f|$ for its value.

Theorem 321 (Max-flow min-cut)**.** *The maximum value of an s–t flow equals the minimum capacity of an s–t cut.*

Proof. Take a maximum flow f, and in its residual graph let S be the vertices reachable from s. Since $t \notin S$, this is an s–t cut.

If $u \in S$ and $v \notin S$, then uv has no residual capacity, so

$$f(uv) = c(uv).$$

If $u \notin S$ and $v \in S$, then $f(uv) = 0$, since otherwise the reverse residual edge vu would reach u. Hence

$$|f| = f(S, \bar{S}) - f(\bar{S}, S) = c(S, \bar{S}).$$

Every flow has value at most every cut capacity, so both are optimal. □

For integer capacities, there is an integral maximum flow: start with the zero flow and repeatedly augment along a residual s–t path by its integer bottleneck capacity. Each augmentation increases the flow value by at least one, so the process terminates.

Menger's theorem. The vertex-disjoint form of Menger is exactly max-flow min-cut after giving each vertex capacity one.

Theorem 322 (Menger)**.** *Let G be a finite graph and let $A, B \subseteq V(G)$ be disjoint. Then*

maximum number of pairwise vertex-disjoint A–B paths

$=$

minimum number of vertices meeting every A–B path.

Proof. Replace every vertex v by

$$v^- \longrightarrow v^+$$

of capacity 1. Replace every edge uv by the two arcs

$$u^+ \to v^-, \qquad v^+ \to u^-,$$

of capacity $M := |V(G)| + 1$. Add a source s with arcs $s \to a^-$ for $a \in A$, and a sink t with arcs $b^+ \to t$ for $b \in B$, again of capacity M.

An integral flow of value r decomposes into r vertex-disjoint A–B paths, since at most one unit can pass through each $v^- \to v^+$. Conversely, such paths give a flow of value r.

There is a cut of capacity at most $|V(G)| < M$, so a minimum cut never uses an M-capacity arc. It therefore consists entirely of arcs $v^- \to v^+$ and is exactly a set of vertices meeting every A–B path. Max-flow min-cut and integrality give the result. □

The edge-disjoint version is more direct: simply give every edge capacity 1. Then integral flows are collections of edge-disjoint A–B paths, while cuts are sets of edges meeting every such path.

8.2 Matchings

8.2.1 The Reduced Gallai–Edmonds Decomposition

A nonempty graph K is *factor-critical* if $K - v$ has a perfect matching for every $v \in V(K)$. In particular, K has odd order, and any prescribed vertex can be the one missed by a near-perfect matching.

For $S \subseteq V(G)$, define its *deficiency* by

$$d_G(S) = q(G - S) - |S|.$$

Every matching in G misses at least $d_G(S)$ vertices: at most $|S|$ odd components of $G - S$ can have a matching edge to S, and each remaining odd component has an uncovered vertex. Also,

$$d_G(S) \equiv |V(G)| \pmod 2, \tag{84}$$

since the parity of $q(G - S)$ equals that of $|V(G)| - |S|$.

Theorem 323 (Reduced Gallai–Edmonds Decomposition)**.** *Every graph G has a set $S \subseteq V(G)$ such that:*

1. *Every component of $G - S$ is factor-critical.*
2. *There is a matching covering S whose other endpoints lie in distinct components of $G - S$.*

For any such S, every maximum matching consists of a matching as in the second condition and a near-perfect matching in each component of $G - S$. Conversely, every compatible choice of these matchings is maximum. In particular,

$$\nu(G) = \frac{|V(G)| + |S| - q(G - S)}{2}. \tag{85}$$

One may choose S to maximize $d_G(S)$, breaking ties by maximizing $|S|$.

Proof. First verify the size formula from the two conditions; we will then use it for smaller graphs in the existence proof. Put $n = |V(G)|$ and $q = q(G - S)$, and match S into distinct components. Within every component receiving such an edge, take a perfect matching after deleting its endpoint; within every other component, leave an arbitrary vertex unmatched. Factor-criticality permits all these choices. The resulting matching has size

$$|S| + \sum_{K \text{ component of } G-S} \frac{|V(K)| - 1}{2} = \frac{n + |S| - q}{2}.$$

For an arbitrary matching, there are at most $|S|$ edges touching S, and at most $(|V(K)| - 1)/2$ edges internal to each odd component K. These bounds give the same total. Equality forces every vertex of S to be matched outside S, and forces a near-perfect matching inside each K. Only one vertex in each K is then available for an edge to S, so these edges use distinct components. This proves all the asserted consequences of the two conditions.

We now prove their existence by induction on n. For $n = 0$, take $S = \emptyset$. Otherwise choose S to maximize $d_G(S)$, and among the maximizers choose one of largest size. Any strictly larger set with deficiency at least $d_G(S)$ will contradict this choice.

Every component of $G - S$ is odd. If a component K were even, delete any $v \in V(K)$ as well. The odd-order graph $K - v$ has at least one odd component, so

$$q(G - (S \cup \{v\})) \geq q(G - S) + 1.$$

Thus $d_G(S \cup \{v\}) \geq d_G(S)$, a contradiction.

Every such component is factor-critical. Suppose that K is a component and $H = K - v$ has no perfect matching. Apply the induction hypothesis to H. Its set T satisfies the already established size formula, giving

$$d_H(T) = |V(H)| - 2\nu(H) \geq 2,$$

because $|V(H)|$ is even and H has no perfect matching. The set $S' = S \cup \{v\} \cup T$ removes the odd component K, replaces it by the components of $H - T$, and adds $1 + |T|$ vertices to S. Consequently,

$$d_G(S') = d_G(S) - 2 + d_H(T) \geq d_G(S),$$

again contradicting the choice of S.

The vertices of S can be matched to distinct components. Form a bipartite graph H_S with left part S and one right vertex for each component of $G - S$; join s to a component when s has a neighbor in it. For $X \subseteq S$, every component not in $N_{H_S}(X)$ remains an odd component after we put X back. Hence

$$\begin{aligned} d_G(S \setminus X) &\geq q(G - S) - |N_{H_S}(X)| - |S \setminus X| \\ &= d_G(S) + |X| - |N_{H_S}(X)|. \end{aligned}$$

Maximality of the deficiency implies $|N_{H_S}(X)| \geq |X|$. Hall's theorem gives the required matching in H_S, and choosing one corresponding edge of G for each matched pair proves the second condition. □

Corollary 324 (Tutte–Berge Formula)**.** *For every graph G,*

$$|V(G)| - 2\nu(G) = \max_{S \subseteq V(G)} \big(q(G - S) - |S|\big).$$

Thus the largest odd-component obstruction equals the minimum number of vertices left unmatched.

Proof. Every matching misses at least $d_G(S)$ vertices for every S. For a set supplied by Theorem 323, Equation (85) gives equality. □

Corollary 325 (Tutte's Perfect Matching Theorem)**.** *A graph G has a perfect matching if and only if*

$$q(G - S) \leq |S| \qquad \text{for every } S \subseteq V(G).$$

Proof. A perfect matching must match every odd component of $G - S$ to S, proving necessity. Conversely, the condition makes the maximum in Corollary 324 at most zero. It is at least zero by taking $S = \emptyset$, so $2\nu(G) = |V(G)|$. □

Corollary 326 (Petersen)**.** *Every bridgeless cubic graph has a perfect matching.*

Proof. Fix $S \subseteq V(G)$. If K is an odd component of $G - S$, then the number of edges between K and S is $3|V(K)| - 2|E(K)|$, an odd number. It cannot be one, since that edge would be a bridge. Thus each odd component sends at least three edges to S. At most $3|S|$ edges leave S, giving $3q(G - S) \leq 3|S|$. Apply Tutte's theorem. □

8.2.2 Kőnig's Theorem

For a graph G, let $\nu(G)$ denote the maximum size of a matching and $\tau(G)$ the minimum size of a vertex cover. Since every vertex cover must contain a distinct endpoint of each edge in a matching, always

$$\nu(G) \leq \tau(G).$$

For bipartite graphs, max-flow min-cut gives equality.

Theorem 327 (Kőnig)**.** *If $G = (L, R; E)$ is bipartite, then*

$$\nu(G) = \tau(G).$$

Proof. Construct a network by adding a source s and sink t, with

$$s \to u \quad (u \in L), \qquad u \to v \quad (uv \in E), \qquad v \to t \quad (v \in R).$$

Give the first and last kinds of edges capacity 1, and the middle edges capacity $|V(G)| + 1$.

By integrality of maximum flow, an integral flow of value k is exactly a matching of size k: each unit of flow travels along some

$$s \to u \to v \to t,$$

and the capacity-1 edges incident with s and t ensure that no two chosen edges share a vertex. Thus

$$\max \text{flow} = \nu(G).$$

Now consider an s–t cut $(S, \overline{S})$. A minimum cut never uses an edge $u \to v$ corresponding to an edge of G, since such an edge alone has capacity $|V(G)| + 1$, while cutting all edges out of s costs only $|L| \le |V(G)|$. Hence for every $uv \in E$,

$$u \in S \quad \Longrightarrow \quad v \in S.$$

Therefore

$$C = (L \cap \overline{S}) \cup (R \cap S)$$

is a vertex cover: if $uv \in E$ and $u \notin C$, then $u \in S$, so $v \in S$ and hence $v \in C$.

Moreover, the only edges crossing the cut are

$$s \to u \quad (u \in L \cap \overline{S})$$

and

$$v \to t \quad (v \in R \cap S),$$

so

$$\text{cap}(S, \overline{S}) = |C|.$$

Thus every minimum cut gives a vertex cover of the same size.

Conversely, from any vertex cover C take

$$S = \{s\} \cup (L \setminus C) \cup (R \cap C).$$

Because C meets every edge of G, no middle edge goes from S to $\overline{S}$, and the cut has capacity exactly $|C|$. Hence

$$\min \text{cut} = \tau(G).$$

Max-flow min-cut now gives

$$\nu(G) = \max \text{flow} = \min \text{cut} = \tau(G).$$

□

Matrix formulation. For a 0–1 matrix, make one bipartite vertex for every row and every column, and an edge for every 1-entry. Kőnig's theorem becomes

maximum number of 1's with no two in one row or column

=

minimum number of rows and columns covering every 1.

Dual edge-cover form. If G has no isolated vertices, let $\rho(G)$ be its minimum edge-cover size and $\alpha(G)$ its maximum independent-set size. The identities

$$\alpha(G) = |V(G)| - \tau(G), \qquad \rho(G) = |V(G)| - \nu(G)$$

therefore give, for bipartite G,

$$\rho(G) = \alpha(G).$$

Thus the minimum number of edges meeting every vertex equals the maximum number of pairwise nonadjacent vertices.

8.3 Mader's Density and Connectivity Theorem

High average degree need not make the whole graph highly connected. The right conclusion is that some subgraph is highly connected. To find it, minimize the order of a subgraph that exceeds a linear edge bound with a fixed allowance for a separator. A small separation would split this allowance between two smaller subgraphs, forcing one of them to exceed the same bound.

Theorem 328 (Mader's Density Theorem)**.** *Let $k \geq 1$, and let G have average degree $d \geq 4k$. Then G contains a $(k+1)$-connected subgraph H whose average degree is greater than $d - 2k$.*

Proof. Put $\rho = d/2 \geq 2k$. Choose a subgraph H of minimum order among those satisfying

$$|V(H)| \geq 2k, \qquad |E(H)| > \rho\big(|V(H)| - k\big).$$

Such a graph exists because G itself qualifies. No qualifying graph has exactly $2k$ vertices, since $\binom{2k}{2} < 2k^2 \leq \rho k$. Thus $|V(H)| > 2k$. Deleting a vertex of degree at most ρ would preserve both conditions, so minimality gives $\delta(H) > \rho$. In particular, $|V(H)| > \rho$, and

$$\frac{|E(H)|}{|V(H)|} > \rho - \frac{\rho k}{|V(H)|} > \rho - k.$$

Suppose that H has a proper separation with sides V_1, V_2, where $V_1 \cup V_2 = V(H)$, no edge joins $V_1 \setminus V_2$ to $V_2 \setminus V_1$, and $|V_1 \cap V_2| \leq k$. Each side contains a vertex outside the separator and all its neighbors, so $|V_i| > \rho \geq 2k$. By minimality, both smaller induced subgraphs satisfy $|E(H[V_i])| \leq \rho(|V_i| - k)$. Hence

$$|E(H)| \leq |E(H[V_1])| + |E(H[V_2])| \leq \rho(|V_1| + |V_2| - 2k) \leq \rho(|V(H)| - k),$$

contradicting the choice of H. Therefore H is $(k+1)$-connected, and the preceding density inequality gives its claimed average degree. □

8.4 Long Directed Paths

A longest directed path contains every in-neighbor of its first vertex and every out-neighbor of its last vertex. This immediately gives a path of length at least the minimum in-degree or the minimum out-degree. To add the two bounds, we want the *same* vertex to play both endpoint roles, possibly on two different paths with the same vertex set. Then all its neighbors lie in that set.

Theorem 329 (A Path from the Two Minimum Degrees)**.** *Let D be a nonempty oriented graph. If every vertex has out-degree at least a and in-degree at least b, where a, b are nonnegative integers, then D contains a directed path of length at least $a + b$. In particular, a longest directed path has at least*

$$\delta^+(D) + \delta^-(D) + 1$$

vertices.

Proof. Choose a longest directed path $P = v_1 \cdots v_s$, and put $U = V(P)$. The endpoint observation proves the assertion when $a = 0$ or $b = 0$, so assume $a, b \geq 1$. Suppose for a contradiction that $s \leq a + b$.

Call a vertex a *possible start*, respectively a *possible end*, if some directed path on exactly U starts, respectively ends, there. No vertex can have both roles. Indeed, both paths would be longest paths in D, so all in-neighbors and all out-neighbors of that vertex would lie in $U \setminus \{v\}$. These two neighborhoods are disjoint because D is oriented, giving $a + b \leq s - 1$.

We show that the endpoint roles must nevertheless overlap. Write

$$I = \{i : v_i \to v_1\}, \qquad J = \{j : v_s \to v_j\}, \qquad r = \min J, \qquad t = \max I.$$

Maximality of P gives $|I| \geq b$ and $|J| \geq a$. Moreover, $I \subseteq \{3, \ldots, s\}$ and $J \subseteq \{1, \ldots, s-2\}$, since opposite arcs are forbidden. Consequently,

$$s \leq a + b \leq |I| + |J| \leq (t - 2) + (s - r - 1),$$

so $r < t$.

If $v_s \to v_1$, then P closes to a directed cycle on U. Cutting the cycle before or after any vertex makes that vertex a possible start and a possible end, a contradiction. Hence $r \geq 2$ and $t \leq s - 1$.

For $i \in I$, $j \in J$, and $j \leq i$, reorder the three consecutive blocks of P to obtain the path

$$Q(i, j) = v_{i+1} \cdots v_s \, v_j \cdots v_i \, v_1 \cdots v_{j-1}. \tag{86}$$

The two new joins are exactly $v_s \to v_j$ and $v_i \to v_1$. Thus $Q(i, j)$ visits every vertex of U once, starting at v_{i+1} and ending at v_{j-1}. If $r = 2$, then $Q(t, r)$ ends at v_1, already a possible start. If $t = s - 1$, the same path starts at v_s, already a possible end. We may therefore assume $r \geq 3$ and $t \leq s - 2$.

Now consider the two sets of indices

$$\mathcal{S} = \{i + 1 : i \in I \cap \{r, \ldots, t - 1\}\},$$
$$\mathcal{T} = \{j - 1 : j \in J \cap \{r + 1, \ldots, t\}\}.$$

For $h \in \mathcal{S}$, the path $Q(h - 1, r)$ starts at v_h. For $h \in \mathcal{T}$, the path $Q(t, h + 1)$ ends at v_h. Hence $\mathcal{S}$ and $\mathcal{T}$ must be disjoint subsets of $\{r, \ldots, t\}$.

But at most $r - 3$ indices of I are below r, and only the index t remains above the interval $\{r, \ldots, t - 1\}$. Similarly, at most $s - t - 2$ indices of J are above t, and only r remains below $\{r + 1, \ldots, t\}$. Therefore

$$|\mathcal{S}| \geq b - r + 2, \qquad |\mathcal{T}| \geq a - s + t + 1,$$

and so

$$|\mathcal{S}| + |\mathcal{T}| \geq a + b - s + t - r + 3 \geq t - r + 3 > t - r + 1.$$

This exceeds the size of $\{r, \ldots, t\}$, a contradiction. □

The bound is sharp: a regular tournament on $2r + 1$ vertices has $\delta^+ = \delta^- = r$, and no path can have more than $2r$ edges. Such a tournament is obtained on $\mathbb{Z}_{2r+1}$ by orienting $i \to i + j$ for $1 \leq j \leq r$. The assumption that the graph is oriented matters: two vertices joined in both directions have $\delta^+ = \delta^- = 1$, but their longest path has only one edge.

8.5 Cycles with Many Chords

A *chord* of a cycle joins two nonconsecutive vertices of that cycle. An endpoint of a longest path has no neighbor outside the path. Rotating the path creates more endpoints without changing its vertex set. Choosing the original path suitably forces the neighborhoods of many of these endpoints into one cycle.

Lemma 330 (A cycle containing neighborhoods)**.** *Every finite simple graph of minimum degree $\delta \geq 2$ contains a cycle C and a set $S \subseteq V(C)$ such that*

$$|S| \geq \delta, \qquad N_G(v) \subseteq V(C) \quad (v \in S).$$

Proof. Among all longest paths with a chosen order $P = v_1 \cdots v_m$, choose one minimizing

$$j = \min\{i : v_i v_m \in E(G)\}.$$

Every neighbor of v_m lies on P. Since its degree is at least two, $j \le m-2$, and

$$C = v_j v_{j+1} \cdots v_m v_j$$

is a cycle.

For every neighbor v_i of v_m, put its successor v_{i+1} in S. These successors are distinct, so $|S| = d_G(v_m) \ge \delta$. The successor of v_{m-1} is v_m, whose neighborhood already lies in C.

For another neighbor v_i, rotate the path to

$$v_1 \cdots v_i \, v_m v_{m-1} \cdots v_{i+1}.$$

This is again a longest path, so its endpoint v_{i+1} has no neighbor outside $V(P)$. Its first $j-1$ vertices are unchanged. If v_{i+1} had a neighbor among them, the new path would have a smaller value of j, contradicting our choice. Thus every neighbor of v_{i+1} lies in C. □

Theorem 331 (Chords forced by minimum degree)**.** *Every finite simple graph of minimum degree $\delta \ge 2$ contains a cycle with at least*

$$\left\lceil \frac{\delta(\delta-2)}{2} \right\rceil$$

chords.

Proof. Take C, S from Lemma 330. Every $v \in S$ has all its neighbors on C, and only two of its incident edges belong to the cycle. Hence it is incident with at least $d_G(v) - 2$ chords.

Each chord is counted at most twice, so the number of chords is at least

$$\frac{1}{2} \sum_{v \in S} (d_G(v) - 2) \ge \frac{\delta(\delta-2)}{2}.$$

□

The leading constant $1/2$ cannot be increased: in $K_{\delta+1}$, every cycle has at most

$$\binom{\delta+1}{2} - (\delta+1) = \frac{(\delta+1)(\delta-2)}{2}$$

chords.

Corollary 332 (A cycle with at least as many chords as vertices)**.** *Every graph on $n \ge 1$ vertices with at least $2n^{3/2}$ edges contains a cycle with at least as many chords as vertices.*

Proof. Repeatedly delete a vertex whose current degree is less than $2\sqrt{n}$, keeping the original value of n fixed. If every vertex were deleted, counting each edge when its first endpoint is removed would give fewer than $2n^{3/2}$ edges. Thus a nonempty subgraph H remains, with minimum degree at least $2\sqrt{n}$.

The preceding theorem gives a cycle in H with at least

$$\frac{(2\sqrt{n})(2\sqrt{n}-2)}{2} = 2n - 2\sqrt{n} \ge n$$

chords. The hypothesis is impossible for $1 \le n < 4$, so the last inequality applies. The cycle has at most n vertices, and all its chords in H are also chords in G. □

8.6 Vertex Partitions under Degree Constraints

Moving a vertex between parts changes only its incident internal edges. Adding a prescribed cost for each possible assignment lets one compare this change with the desired degree bounds.

Theorem 333 (Variable degree bounds)**.** *Let G be a graph, let $r \geq 1$, and let $f_1, \dots, f_r : V(G) \to \mathbb{Z}_{\geq 0}$ satisfy*

$$d_G(v) < \sum_{i=1}^{r} f_i(v) \qquad (v \in V(G)).$$

Then $V(G)$ has a partition $V_1, \dots, V_r$, allowing empty parts, such that

$$d_{G[V_i]}(v) < f_i(v) \qquad (v \in V_i).$$

In particular, $f_i(v) = 0$ forbids placing v in V_i.

Proof. Choose a partition minimizing

$$\Phi(V_1, \dots, V_r) = \sum_{i=1}^{r} \left(|E(G[V_i])| - \sum_{v \in V_i} f_i(v) \right).$$

For a vertex v, let $d_i(v)$ count its neighbors in V_i.

Suppose $v \in V_i$ violates the bound, so $d_i(v) \geq f_i(v)$. Since $\sum_j d_j(v) = d_G(v) < \sum_j f_j(v)$, some j satisfies $d_j(v) < f_j(v)$. Moving v from V_i to V_j changes the potential by

$$\big(d_j(v) - f_j(v)\big) - \big(d_i(v) - f_i(v)\big) < 0,$$

contradicting its minimality. □

Whenever a vertex violates its bound, the proof identifies a move that decreases Φ. Repeating such moves terminates, since there are finitely many partitions, and the resulting partition satisfies every bound.

Corollary 334 (Lovász's degree partition theorem)**.** *Let G have maximum degree Δ. If nonnegative integers $b_1, \dots, b_r$ satisfy*

$$\sum_{i=1}^{r} b_i \geq \Delta - r + 1,$$

then $V(G)$ can be partitioned into $V_1, \dots, V_r$ with

$$\Delta(G[V_i]) \leq b_i \qquad (1 \leq i \leq r).$$

Proof. Use the constant thresholds $f_i(v) = b_i + 1$. Their sum is at least $\Delta + 1 > d_G(v)$. □

Thus, for every integer $b \geq 0$, the vertices can be partitioned into

$$\left\lceil \frac{\Delta + 1}{b + 1} \right\rceil$$

parts, each inducing maximum degree at most b. This number is best possible in general: in $K_{\Delta+1}$, each such part has at most $b + 1$ vertices.

Corollary 335 (Vertexwise degree reduction)**.** *For every integer $r \geq 1$, a finite graph has a vertex partition $V_1, \dots, V_r$ satisfying*

$$d_{G[V_i]}(v) \leq \left\lfloor \frac{d_G(v)}{r} \right\rfloor \qquad (v \in V_i).$$

In particular, there is a cut across which every vertex has at least $\lceil d_G(v)/2 \rceil$ neighbors.

Proof. Set $f_i(v) = \lfloor d_G(v)/r \rfloor + 1$ for every i. Then $\sum_i f_i(v) > d_G(v)$. For the final claim, take $r = 2$ and subtract the internal degree from the total degree. □

Lists of permitted parts. The same theorem allows vertex-dependent restrictions on the partition. Suppose $L(v) \subseteq [r]$ lists the permitted parts for v, and $b_i(v) \geq 0$ is its allowed internal degree in part i. If

$$\sum_{i \in L(v)} (b_i(v) + 1) > d_G(v),$$

a suitable partition exists: take $f_i(v) = b_i(v) + 1$ for $i \in L(v)$, and $f_i(v) = 0$ otherwise. In particular, $b_i(v) = 0$ gives proper list coloring from lists of size at least $d_G(v) + 1$.

8.7 Path Covers and Independent Representatives

A *path cover* of a digraph is a partition of its vertices into directed paths, including paths consisting of a single vertex. An *independent transversal* of the cover chooses one vertex from each path, with no arc in either direction between any two chosen vertices.

It is tempting to keep joining paths until their endpoints are independent. Joining alone may get stuck, but an induction can move the representatives away from the endpoints. The condition preserved by this induction is that the set of endpoints cannot be replaced by a proper subset.

For a path cover $\mathcal{P}$, let $T(\mathcal{P})$ be its set of final vertices. Call $\mathcal{P}$ *terminal-minimal* if there is no path cover $\mathcal{Q}$ of the same digraph with $T(\mathcal{Q}) \subsetneq T(\mathcal{P})$. This is minimality under inclusion. In particular, every cover with the minimum number of paths is terminal-minimal.

Theorem 336 (Gallai–Milgram)**.** *Every finite digraph has a path cover with an independent transversal. More strongly, every terminal-minimal path cover has an independent transversal, and hence so does every path cover with the minimum number of paths.*

Proof. We prove the stronger statement by induction on the number of vertices. The empty graph is immediate. Let $\mathcal{P} = \{P_1, \ldots, P_k\}$ be terminal-minimal, with P_i ending at t_i. If $\{t_1, \ldots, t_k\}$ is independent, it is the required transversal. Otherwise, relabel so that $t_2 \to t_1$ is an arc.

The path P_1 has a vertex before t_1. Otherwise we could append t_1 to P_2, delete the singleton path P_1, and replace the endpoint set by its proper subset $T(\mathcal{P}) \setminus \{t_2\}$. Let u be the predecessor of t_1 on P_1. Delete t_1 from the digraph and shorten P_1 to end at u, obtaining a path cover $\mathcal{P}'$ with endpoint set

$$T' = \{u, t_2, \ldots, t_k\}.$$

We claim that $\mathcal{P}'$ is terminal-minimal in the smaller digraph. Suppose instead that there is a cover $\mathcal{Q}$ with $T(\mathcal{Q}) \subsetneq T'$.

- If $u \in T(\mathcal{Q})$, append t_1 to the path ending at u, using $u \to t_1$.
- If $u \notin T(\mathcal{Q})$ but $t_2 \in T(\mathcal{Q})$, append t_1 to the path ending at t_2, using $t_2 \to t_1$.
- If neither u nor t_2 is in $T(\mathcal{Q})$, add the singleton path $\{t_1\}$.

In the first two cases, the new endpoint set is contained in $T(\mathcal{P})$ and has fewer than k elements. In the third case, it is contained in $\{t_1, t_3, \ldots, t_k\}$. Each case contradicts terminal-minimality of $\mathcal{P}$, proving the claim.

By induction, $\mathcal{P}'$ has an independent transversal. Its representative on the shortened path $P_1 - t_1$ also lies on P_1, and all other paths are unchanged. The same vertices therefore form an independent transversal of $\mathcal{P}$. Finally, a terminal-minimal cover exists because singleton paths give a cover and there are only finitely many possible endpoint sets. □

Corollary 337 (The Independence-Number Bound)**.** *Every finite digraph D has a path cover with at most $\alpha(D)$ paths, where $\alpha(D)$ is the largest size of an independent vertex set in its underlying undirected graph. In particular, every nonempty tournament has a directed Hamilton path* (Rédei's theorem)*.*

Proof. An independent transversal has one vertex for each path, so the number of paths is at most $\alpha(D)$. A nonempty tournament has independence number one. □

Corollary 338 (Dilworth)**.** *In a finite partially ordered set, the minimum number of chains in a partition equals the maximum size of an antichain.*

Proof. Construct a digraph on the elements of the poset, with an arc $x \to y$ whenever $x < y$. The vertices of a directed path form a chain, while an independent set is an antichain. Gallai–Milgram therefore gives a chain partition with no more parts than the largest antichain has elements. Conversely, a chain contains at most one member of any antichain, so no chain partition can have fewer parts. □

8.8 Tree Packing and Arboricity

A repeated edge in a family of spanning trees can be exchanged without removing it from their union. A sequence of such exchanges may eventually add an uncovered edge. When no such improvement is possible, the exchange structure separates the graph into parts that admit disjoint spanning trees and a quotient that admits a covering by spanning trees.

Throughout this subsection, graphs are loopless multigraphs. For a vertex partition $\mathcal{P}$, the quotient $G/\mathcal{P}$ contracts each part to one vertex, deletes edges internal to a part, and retains parallel edges.

Exchange chains. Let $T_1, \dots, T_k$ be spanning trees of a connected graph G. For $f \notin T_i$, let $P_i(f)$ be the unique path in T_i between the endpoints of f. Replacing any edge of $P_i(f)$ by f gives another spanning tree.

Form a directed graph whose vertices are the edges of G, with an arc $e \to f$ whenever, for some i,

$$f \notin T_i, \qquad e \in P_i(f).$$

Call an edge *uncovered* if it lies in none of the trees, and *repeated* if it lies in at least two.

Lemma 339 (Augmenting an exchange chain)**.** *If a repeated edge can reach an uncovered edge in the exchange digraph, the trees can be modified so that their union strictly increases.*

Proof. Choose a shortest directed path $e_0, e_1, \dots, e_m$ from a repeated edge to an uncovered edge. For each $j < m$, choose i_j witnessing the arc $e_j \to e_{j+1}$.

Perform the exchanges

$$T_{i_j} \longleftarrow T_{i_j} - e_j + e_{j+1} \qquad (j = 0, \dots, m-1)$$

in this order. We first check that every exchange remains valid after the earlier ones.

The original path $P_{i_j}(e_{j+1})$ contains none of $e_0, \dots, e_{j-1}$. Otherwise an arc $e_h \to e_{j+1}$, with $h < j$, would shorten the chosen exchange path. Thus no earlier exchange removes an edge of this path. The edge e_{j+1} has not previously been inserted, since the exchange-path vertices are distinct. Its original fundamental path therefore still exists in the current tree, and the exchange is valid.

No exchange removes an edge from the union. For $j = 0$, this follows because e_0 was repeated. For $j > 0$, the edge e_j still belongs to its original tree T_{i_j}, and the previous exchange has just inserted it into $T_{i_{j-1}}$. These are different trees: originally e_j was absent from $T_{i_{j-1}}$ but present in T_{i_j}. Deleting one copy therefore leaves another.

The last exchange inserts the previously uncovered edge e_m. Hence the union strictly increases. □

Theorem 340 (Packing–covering theorem)**.** *For every connected multigraph $G = (V, E)$ and integer $k \geq 1$, there is a partition $\mathcal{P}$ of V such that:*

1. *Each $G[U]$, $U \in \mathcal{P}$, contains k edge-disjoint spanning trees.*
2. *The edges of $G/\mathcal{P}$ are covered by k spanning trees.*

A singleton part has the empty tree as each of its spanning trees. The covering trees in the quotient need not be disjoint.

Proof. Choose spanning trees $T_1, \dots, T_k$ whose union has maximum cardinality. Let $D \subseteq E$ consist of the edges that can reach an uncovered edge in their exchange digraph. In particular, every uncovered edge belongs to D. By Lemma 339, no edge of D is repeated.

Let $\mathcal{P}$ be the vertex sets of the components of (V, D). Fix a tree T_i. If $uv \in D \setminus T_i$, every edge e of the u-v path in T_i has an arc $e \to uv$, and therefore belongs to D. If $uv \in D \cap T_i$, the edge itself is such a path.

Consequently $T_i \cap D$ connects every part $U \in \mathcal{P}$. Its restriction to U is a spanning tree of $G[U]$. These spanning trees are edge-disjoint as i varies, because no edge of D is repeated.

Contract these connected restrictions in each T_i. The resulting graphs are spanning trees of $G/\mathcal{P}$. They cover every quotient edge: an uncovered edge belongs to D, so its endpoints lie in the same part and it disappears on contraction. □

The optimization formula. For a partition $\mathcal{P}$ of V, let $e_G(\mathcal{P})$ be the number of edges joining different parts. Let $r_k(G)$ be the maximum number of edges that can be partitioned into k forests, allowing empty forests. The *arboricity* $a(G)$ is the least number of forests needed to partition all edges.

Corollary 341 (Forest packing formula)**.** *For every multigraph $G = (V, E)$ and $k \geq 1$,*

$$r_k(G) = \min_{\mathcal{P}} \left(e_G(\mathcal{P}) + k(|V| - |\mathcal{P}|) \right).$$

Proof. For any partition $\mathcal{P}$, each forest contains at most $\sum_{U \in \mathcal{P}} (|U| - 1) = |V| - |\mathcal{P}|$ edges internal to the parts. There are at most $e_G(\mathcal{P})$ available crossing edges. This proves the upper bound.

Apply Theorem 340 in each connected component. Inside every part, choose the k disjoint spanning trees it supplies; in each component's quotient, choose the k covering trees. For each index i, combine the i-th internal trees with the lifted i-th quotient tree.

The resulting k forests contain every crossing edge and exactly $k(|V| - |\mathcal{P}|)$ distinct internal edges. Assign every edge appearing in several forests to just one of them. The resulting edge partition attains the displayed upper bound for this $\mathcal{P}$. □

Corollary 342 (Nash–Williams–Tutte spanning-tree packing)**.** *A connected multigraph G contains k edge-disjoint spanning trees if and only if*

$$e_G(\mathcal{P}) \geq k(|\mathcal{P}| - 1) \qquad \text{for every partition } \mathcal{P} \text{ of } V(G).$$

Proof. Every spanning tree uses at least $|\mathcal{P}| - 1$ crossing edges, proving necessity.

For sufficiency, the condition and Corollary 341 give $r_k(G) \geq k(|V| - 1)$. Each of the k forests has at most $|V| - 1$ edges, so equality forces all of them to be spanning trees. □

Corollary 343 (Nash–Williams arboricity theorem)**.** *For an integer $k \geq 1$, the edges of a multigraph G can be partitioned into k forests if and only if*

$$|E(G[U])| \leq k(|U| - 1) \qquad (\varnothing \neq U \subseteq V(G)).$$

Consequently its arboricity is

$$a(G) = \max_{\substack{U \subseteq V(G) \\ |U| \geq 2}} \left\lceil \frac{|E(G[U])|}{|U| - 1} \right\rceil,$$

with value zero for an edgeless graph.

Proof. A forest has at most $|U| - 1$ edges inside a nonempty set U, so the inequalities are necessary.

Conversely, for every partition $\mathcal{P}$,

$$|E| = e_G(\mathcal{P}) + \sum_{U \in \mathcal{P}} |E(G[U])| \leq e_G(\mathcal{P}) + k(|V| - |\mathcal{P}|).$$

The forest packing formula gives $r_k(G) = |E|$. Minimizing the permissible integer k yields the arboricity formula. □

Corollary 344 (Edge-connectivity supplies spanning trees)**.** *For every integer $k \geq 1$, a $2k$-edge-connected multigraph contains k edge-disjoint spanning trees.*

Proof. For a partition into $r \geq 2$ parts, each part has at least $2k$ edges leaving it. Summing counts every crossing edge twice, so $e_G(\mathcal{P}) \geq kr \geq k(r-1)$. Apply Corollary 342. □

8.9 Directed Branching Packing

A *branching* is a spanning directed forest whose edges point away from its roots. Equivalently, every vertex has indegree at most one and there are no directed cycles. Its *root set* consists of the vertices of indegree zero. A branching with one root is an *out-arborescence*.

A branching with root set R must enter every nonempty set disjoint from R. Thus, to pack branchings with prescribed root sets $R_1, \dots, R_k$, every vertex set must have enough entering edges for all branchings whose roots lie outside it. Remarkably, these cut conditions are sufficient, even when the root sets differ.

Theorem 345 (Edmonds' disjoint branching theorem)**.** *Let $G = (V, E)$ be a directed multigraph and let $R_1, \dots, R_k \subseteq V$ be nonempty. There exist pairwise edge-disjoint spanning branchings $B_1, \dots, B_k$, with B_i having root set exactly R_i, if and only if*

$$|\delta^-(S)| \geq |\{i : R_i \cap S = \varnothing\}| \qquad (\varnothing \neq S \subseteq V). \tag{87}$$

Proof. Necessity follows because every branching whose roots avoid S must use an edge entering S.

For sufficiency, write

$$d(S) = |\delta^-(S)|, \qquad q(S) = |\{i : R_i \cap S = \varnothing\}|.$$

Call a nonempty set S *tight* if $d(S) = q(S)$. The cut function d is submodular, whereas q is supermodular:

$$\begin{aligned} d(X) + d(Y) &\geq d(X \cap Y) + d(X \cup Y), \\ q(X) + q(Y) &\leq q(X \cap Y) + q(X \cup Y). \end{aligned} \tag{88}$$

For the second inequality, observe that each indicator $\mathbf{1}[R_i \cap S = \varnothing]$ is supermodular.

We induct on $\sum_i |V \setminus R_i|$. If every $R_i = V$, the empty branchings suffice. Otherwise assume $R_1 \neq V$. We will choose one edge for B_1, delete it from the graph, and add its head to R_1. The task is to preserve (87).

Choose an inclusion-minimal tight set W meeting R_1 but not contained in R_1. Such a set exists because V is tight. For $Z = W \setminus R_1$, we have

$$d(Z) \geq q(Z) > q(W) = d(W),$$

since Z avoids R_1 but W does not. Every edge entering Z from outside W also enters W. Therefore some edge $a = u \to v$ has $u \in W \cap R_1$ and $v \in W \setminus R_1$.

Delete a and replace R_1 by $R_1 \cup \{v\}$. A cut can violate the new condition only if a enters it, the cut was previously tight, and it already met R_1: otherwise either its indegree is unchanged or its required indegree decreases together with its actual indegree.

Suppose such a tight set U exists. Since $v \in U \cap W$, the intersection is nonempty. Submodularity, supermodularity, and the cut conditions give

$$d(U \cap W) + d(U \cup W) \leq d(U) + d(W) = q(U) + q(W) \leq q(U \cap W) + q(U \cup W) \leq d(U \cap W) + d(U \cup W).$$

Equality holds throughout. Thus $U \cap W$ is tight. Moreover, equality in the supermodular inequality implies $U \cap W$ meets R_1: otherwise R_1, which meets both U and W, would contribute strictly more to $q(U \cap W) + q(U \cup W)$ than to $q(U) + q(W)$.

But $v \in (U \cap W) \setminus R_1$, while $u \in W \setminus U$. Hence $U \cap W$ is a proper subset of W meeting R_1 but not contained in it, contradicting the minimality of W. The cut conditions are therefore preserved.

Apply induction to the smaller graph with the enlarged root set, obtaining disjoint branchings. In the first branching, both u and v are roots. Adding $u \to v$ joins their two components, removes v from the root set, and creates no directed cycle. This recovers the original root set R_1 and completes the proof. □

Common-root arborescences. Taking $R_1 = \cdots = R_k = \{r\}$ gives the directed analogue of Nash–Williams:

$$\max\{\text{edge-disjoint spanning out-arborescences rooted at } r\} = \min_{\varnothing \neq S \subseteq V \setminus \{r\}} |\delta^-(S)|. \tag{89}$$

In particular, k edge-disjoint paths from r to every other vertex, as guaranteed by Menger's theorem, can be organized into k edge-disjoint spanning arborescences.

The theorem also applies to integral edge capacities by replacing each edge with the corresponding number of parallel copies.

8.10 Packing and Covering Cycles

Let $\nu_{\mathrm{cyc}}(G)$ be the maximum number of pairwise vertex-disjoint cycles in G, and let $\tau_{\mathrm{cyc}}(G)$ be the minimum number of vertices whose deletion leaves a forest.

The inequality $\nu_{\mathrm{cyc}}(G) \le \tau_{\mathrm{cyc}}(G)$ is immediate. The Erdős–Pósa theorem bounds the reverse gap by a logarithmic factor. The proof first controls cubic graphs, then places a maximal subdivision of such graphs inside an arbitrary graph.

Cubic graphs contain many disjoint cycles. In the auxiliary multigraphs below, loops are cycles of length one, parallel edge pairs are cycles of length two, and a loop contributes two to the degree.

Lemma 346 (Cycle packing in cubic multigraphs)**.** *Let H be a nonempty finite cubic multigraph, and put $q = |V(H)|$ and $r = \nu_{\mathrm{cyc}}(H)$. Then*

$$q \le 8r \log_2(8r).$$

Proof. The handshaking identity makes q even, so $q \ge 2$. We first show that H has a cycle of length at most $2\lceil \log_2 q \rceil$. A loop or parallel pair already suffices. Otherwise H is simple. If its girth exceeded $2h$, all walks of lengths at most h from a fixed vertex that never immediately retrace an edge would have distinct endpoints: a repeated endpoint would yield a cycle of length at most $2h$. There are three choices for the first edge and two at every later step, giving

$$q \ge 1 + 3(1 + 2 + \cdots + 2^{h-1}) = 1 + 3(2^h - 1).$$

Taking $h = \lceil \log_2 q \rceil$ is a contradiction.

Next, after deleting a cycle C of length c, the remaining graph contains a subdivision of a cubic multigraph on at least $q - 2c$ vertices; the cubic multigraph may be empty. To see this, initially at most c edges join C to its complement, since each cycle vertex uses two of its three incidences on C.

After deleting C, repeatedly delete vertices of current degree at most one. Moving such a vertex into the deleted set reduces its edge boundary by at least one: in the original cubic graph it has at least two incidences toward the deleted set and at most one toward the remaining graph. If a vertices are removed in this process, at most $c - a$ boundary edges remain.

Every remaining vertex has degree two or three. Each degree-two vertex is incident with one boundary edge, so there are at most $c - a$ such vertices. Suppress the degree-two paths and discard components that are cycles. The resulting cubic multigraph therefore has at least

$$q - c - a - (c - a) = q - 2c$$

vertices. It is represented by a subdivision in $H - V(C)$.

Now repeatedly choose a shortest cycle and perform this reduction. Each later cycle lifts to a cycle avoiding all those previously chosen, so at most r cycles are selected. Every intermediate cubic graph has at most q vertices, and each step reduces its order by at most $4\lceil \log_2 q \rceil$. When the cubic graph becomes empty, we obtain

$$q \le 4r \lceil \log_2 q \rceil.$$

Put $y = q/(4r)$. If $y < 4$, then $q < 16r \le 8r \log_2(8r)$. Otherwise,

$$y \le 1 + \log_2 q = \log_2(8r) + \log_2 y \le \log_2(8r) + \frac{y}{2},$$

using $\log_2 y \le y/2$ for $y \ge 4$. Hence $y \le 2\log_2(8r)$, which gives the result. □

A maximal subgraph of degrees two and three. The obstruction to using the cubic lemma directly is that the original graph may have arbitrarily large degrees and arbitrarily long subdivided paths. A maximal subgraph with degrees two and three separates these issues: its degree-three vertices admit the cubic bound, and the remaining cycles can be covered by one vertex per disjoint cycle.

Lemma 347 (A cycle cover from a maximal subcubic subgraph)**.** *Let G be a finite simple graph, and let $H \subseteq G$ be inclusion-maximal subject to every vertex of H having degree two or three in H. Write*

$$B = \{v \in V(H) : d_H(v) = 3\}.$$

There is a set $X \subseteq V(H) \setminus B$, with

$$|X| \le \nu_{\mathrm{cyc}}(G),$$

such that $B \cup X$ meets every cycle of G.

Proof. Maximality has two immediate consequences. There is no cycle disjoint from H, since it could be added as a component. There is also no path with distinct degree-two endpoints in H, no internal vertex in H, and all edges outside H: adding it would increase the endpoint degrees to three and give every new vertex degree two.

Let Z be the set of degree-two vertices $z \in H$ for which some cycle C_z meets H exactly at z. Choose one such cycle for each $z \in Z$. These chosen cycles are pairwise vertex-disjoint. Indeed, if C_z and $C_{z'}$, with $z \ne z'$, intersected, their union would contain a z-z' path with all internal vertices outside H, contradicting the second consequence of maximality.

Let $\mathcal{D}$ be the components of H that are cycles and avoid Z. They are disjoint from all the chosen C_z's. Thus

$$|Z| + |\mathcal{D}| \le \nu_{\mathrm{cyc}}(G).$$

Take X to consist of Z and one vertex from each component in $\mathcal{D}$. It meets every cycle component of H.

Suppose a cycle C of G avoids $B \cup X$. It must meet H. If it meets H in exactly one vertex z, then $z \in Z \subseteq X$, a contradiction.

If it meets H in at least two vertices but is not contained in H, take a segment of C between consecutive visits to $V(H)$ that uses an edge outside H. This segment may be a single edge; in every case its edges and internal vertices lie outside H. Since C avoids B, both endpoints have degree two in H, again contradicting maximality.

Finally, if $C \subseteq H$, all its vertices have degree two in H. It is therefore a whole cycle component of H, and such a component meets X. □

Theorem 348 (Erdős–Pósa)**.** *For every finite simple graph G containing a cycle,*

$$\tau_{\mathrm{cyc}}(G) \le \nu_{\mathrm{cyc}}(G)\big(8\log_2(8\nu_{\mathrm{cyc}}(G)) + 1\big).$$

If G is a forest, both parameters are zero.

Consequently, for every integer $k \ge 1$, every graph contains either k vertex-disjoint cycles or a set of $O(k\log(k+1))$ vertices meeting all its cycles.

Proof. Choose H, B, X as in Lemma 347. Discard the cycle components of H, and suppress every degree-two path in the remaining components. The result is a cubic multigraph H' with vertex set B.

Vertex-disjoint cycles of H' lift to vertex-disjoint cycles of H, and hence of G. If $B \ne \varnothing$, Lemma 346 therefore gives

$$|B| \le 8\nu_{\mathrm{cyc}}(H')\log_2(8\nu_{\mathrm{cyc}}(H')) \le 8\nu_{\mathrm{cyc}}(G)\log_2(8\nu_{\mathrm{cyc}}(G)).$$

The same upper bound is immediate when $B = \varnothing$.

Since $B \cup X$ meets every cycle and $|X| \le \nu_{\mathrm{cyc}}(G)$,

$$\tau_{\mathrm{cyc}}(G) \le |B| + |X| \le \nu_{\mathrm{cyc}}(G)\bigl(8\log_2(8\nu_{\mathrm{cyc}}(G)) + 1\bigr).$$

The packing–covering alternative follows by considering whether $\nu_{\mathrm{cyc}}(G) \ge k$. □

8.11 Vertex and Edge Coloring

A proper *vertex coloring* assigns different colors to adjacent vertices; the least number of colors is the *chromatic number* $\chi(G)$. A proper *edge coloring* assigns different colors to edges sharing an endpoint; the least number is the *chromatic index* $\chi'(G)$. Thus vertex-color classes are independent sets, while edge-color classes are matchings.

8.11.1 Brooks's Theorem

Greedy vertex coloring uses at most $\Delta(G) + 1$ colors: when a vertex is colored, its neighbors forbid at most $\Delta(G)$ colors. To save the extra color, we arrange for every vertex except the last to have an uncolored neighbor. At the last vertex, we instead arrange for two neighbors to have the same color.

The basic operation is simple along a path. With $k \ge \Delta(G)$ colors, if $u_1, \dots, u_t$ are the uncolored vertices of a path, we can color $u_1, \dots, u_{t-1}$ in this order: when u_i is colored, its neighbor u_{i+1} is still uncolored, so at most $k - 1$ colors are forbidden. We leave u_t for last.

Theorem 349 (Brooks)**.** *If a finite connected simple graph G is neither complete nor an odd cycle, then*

$$\chi(G) \le \Delta(G).$$

Complete graphs and odd cycles have $\chi(G) = \Delta(G) + 1$.

Proof. We first prove the following form: for every integer $k \ge 3$, every graph of maximum degree at most k containing no K_{k+1} is k-colorable. Fix k, and induct on the number n of vertices. The empty graph is immediate. If some vertex has degree less than k, delete it, color the remaining graph by induction, and put it back using a color absent from its neighbors. Hence we may assume that G is k-regular.

Choose a vertex v_2. It has two nonadjacent neighbors v_1, v_3, since otherwise $N[v_2]$ would induce a K_{k+1}. Extend the path $v_1v_2v_3$ at its last vertex until no further extension is possible, obtaining $P = v_1 \cdots v_r$. Every neighbor of v_r lies on P. There are two cases.

The path contains every vertex. Here $r = n$. Since $k \ge 3$, the vertex v_2 has a neighbor v_j other than v_1, v_3, with $j \ge 4$. Give v_1, v_3 the same color, and greedily color the remaining vertices in the order

$$v_4, \dots, v_{j-1}, \quad v_n, v_{n-1}, \dots, v_j, \quad v_2.$$

Empty ranges are omitted. Every vertex before v_2 has an uncolored neighbor: for v_h with $4 \le h < j$, use v_{h+1}; for v_h with $h > j$, use v_{h-1}; and for v_j, use v_2. Finally, the neighbors v_1, v_3 of v_2 have the same color, so its k neighbors forbid at most $k - 1$ colors. The coloring is complete.

Some vertex lies outside the path. Let v_j be the earliest neighbor of v_r on P. The vertices $v_j, v_{j+1}, \dots, v_r$, in this order, form a cycle C, and every neighbor of v_r lies on C. Because $r < n$, both $G - V(C)$ and $G[V(C)]$ have fewer than n vertices.

Color $G - V(C)$ by induction. If there is no edge from C to its complement, color $G[V(C)]$ separately by induction. Otherwise, choose the largest index ℓ for which $v_\ell \in V(C)$ has a neighbor $u \notin V(C)$. We have $\ell < r$, and $v_{\ell+1}$ has no neighbor outside C. We can therefore give $v_{\ell+1}$ the color of u. Continue around the cycle from $v_{\ell+1}$, greedily coloring each subsequent vertex until only v_ℓ remains. Each of these steps has an uncolored next neighbor on C. At the final step, v_ℓ has two neighbors of the same color, namely u and $v_{\ell+1}$, so it too can be colored. This finishes the induction.

Now let G satisfy the statement of the theorem. If $\Delta(G) \le 2$, then G is a path or a cycle, and the claim follows directly. If $\Delta(G) \ge 3$, set $k = \Delta(G)$. A copy of K_{k+1} in G would already use all k incident edges at

each of its vertices; connectedness would force $G = K_{k+1}$. Thus G contains no such clique, and the result just proved applies. □

The common idea in the two cases is to postpone one vertex whose neighbors repeat a color. A path or cycle supplies the order in which the other vertices can be colored while retaining an uncolored neighbor. Only a maximal path is needed, not a longest one.

Arboricity and degeneracy. The same greedy viewpoint gives a useful coloring bound for unions of forests. If the edges of G are the union of a forests, then every nonempty subgraph H satisfies

$$|E(H)| \le a(|V(H)| - 1),$$

so its average degree is strictly less than $2a$. Hence H has a vertex of degree at most $2a - 1$, we call this $(2a - 1)$-degeneracy. Repeatedly deleting such a vertex shows that $\chi(G) \le 2a$.

In particular, the union of three trees is 6-colorable. The bound is best possible: K_{2a} has arboricity a and chromatic number $2a$ (and hence K_6 can be decomposed into three forests, indeed into three spanning trees).

8.11.2 Kőnig's Edge Coloring Theorem

An edge coloring needs at least $\Delta(G)$ colors at a vertex of maximum degree. We start with a recoloring operation that attains this bound for bipartite multigraphs.

Fix a palette of colors. A color is *missing at a vertex* if no colored edge incident with that vertex has that color. For any two distinct colors α, β, the subgraph formed by their edges has maximum degree two; each component is a path, an even cycle, or an isolated vertex. Swapping α and β throughout one component preserves a proper edge coloring. A vertex missing one of the two colors is an endpoint of a path component, possibly an isolated vertex.

For multigraphs, a two-color component may be a cycle consisting of two parallel edges; the same swapping rule still applies. In a bipartite graph, parity prevents the obstructing two-color path from joining the endpoints of the edge we want to color.

Theorem 350 (Kőnig's Edge-Coloring Theorem)**.** *Every finite loopless bipartite multigraph satisfies* $\chi'(G) = \Delta(G)$.

Proof. Induct on the number of edges; the edgeless case is immediate. Use a palette of $\Delta(G)$ colors. After deleting xy, color the remaining edges by induction. Choose a color α missing at x and a color β missing at y. If $\alpha = \beta$, color xy immediately.

Otherwise consider the α, β-component containing x. It cannot contain y: a path between them would start with a β-edge and end with an α-edge, so it would have even length, whereas x and y lie in opposite bipartition classes. Swap the two colors in the component containing x. The color β is now missing at both endpoints of xy, so it can be used on that edge. □

8.11.3 Vizing's Theorem

For a simple graph with an edge, greedy edge coloring gives $2\Delta(G) - 1$ colors, since an edge has at most $2\Delta(G) - 2$ adjacent edges. Vizing saves all but one of the colors above the lower bound $\Delta(G)$. The proof extends a coloring one edge at a time: first move the uncolored edge through a fan, then finish with a two-color swap.

Theorem 351 (Vizing)**.** *Every finite simple graph satisfies*

$$\Delta(G) \le \chi'(G) \le \Delta(G) + 1.$$

Proof. The lower bound is immediate. For the upper bound, induct on the number of edges. There is nothing to prove when there are no edges. Write $\Delta = \Delta(G)$, use the palette $\{1, \dots, \Delta + 1\}$, and color every edge except xy_0 by induction. Denote this partial coloring by c_0. Every vertex has a missing color.

A fan moves the uncolored edge. Choose a maximal sequence of distinct neighbors $y_0, y_1, \dots, y_t$ of x such that

$$c_0(xy_{h+1}) \text{ is missing at } y_h \qquad (0 \le h < t).$$

For each $0 \le i \le t$, define a partial coloring c_i by shifting the colors toward the first edge:

$$c_i(xy_h) = c_0(xy_{h+1}) \quad (0 \le h < i), \qquad xy_i \text{ is uncolored},$$

and leave all other edges as in c_0. This is proper: each inserted color was missing at its new endpoint y_h, and at x we have merely moved the same distinct colors among incident edges. In particular, the colors missing at x are the same in every c_i.

Choose a color α missing at x, and a color β missing at y_t in c_0. The color β is still missing at y_t in c_t. If it is also missing at x, use c_t and color xy_t with β.

Otherwise the unique edge at x colored β in c_0 must be one of the fan edges: if its other endpoint were outside the fan, we could append it. It is neither xy_0, which is uncolored, nor xy_t, since β is missing at y_t. Write this edge as xy_{i+1}, with $i < t$. By the fan condition, β is missing at y_i in c_0. In c_t, the β-edge at x is xy_i.

One of two rotations can be completed. In c_t, let P be the α,β-component containing y_t. If it does not contain x, swap its two colors. Then α is missing at both x and y_t, so xy_t can be colored α.

Suppose instead that P contains x. Because β is missing at y_t and α is missing at x, P is a path from y_t to x, ending with the β-edge y_ix. Return to the coloring c_i, whose uncolored edge is xy_i. The segment of P from y_t to y_i is unchanged: c_i and c_t differ only on edges incident with x. Moreover, β is missing at both ends of this segment in c_i. At y_i, it was missing in c_0 and its spoke is now uncolored; at y_t, none of the incident edges changed from c_0. Therefore this segment is an entire α,β-component in c_i, disjoint from x.

Swap its two colors. Now α is missing at y_i as well as at x, so color xy_i with α. In every case, all edges of G have been colored. □

The extra color is sometimes necessary. For K_{2r+1}, every color class is a matching with at most r edges, while the graph has $r(2r+1)$ edges. For $r \ge 1$, at least $2r+1$ colors are needed, and Vizing's theorem gives exactly this many: $\chi'(K_{2r+1}) = 2r+1 = \Delta(K_{2r+1}) + 1$.

8.11.4 Galvin's List Edge-Coloring Theorem

In a *list coloring*, each vertex or edge must receive a color from its own prescribed finite list. A two-color swap need not respect these lists. Instead, we will color a whole independent set at once, chosen so that every vertex losing a color also loses an out-neighbor. Stable matchings will supply the required independent sets in a line graph.

A *kernel* of a digraph D is an independent set K such that every vertex outside K has an arc to a vertex of K. Opposite arcs are allowed. We call D *kernel-perfect* if every induced subdigraph has a kernel; the empty subdigraph has the empty kernel.

Lemma 352 (Kernel lemma for list coloring)**.** *Let D be a kernel-perfect digraph. If every vertex v has a list $L(v)$ with $|L(v)| \ge d_D^+(v) + 1$, then the underlying undirected graph has a proper coloring from these lists.*

Proof. Induct on the number of vertices. If D is nonempty, choose a color a occurring in some list, and put $W = \{v : a \in L(v)\}$. Choose a kernel K of $D[W]$. Since $W \neq \emptyset$, the kernel is nonempty. Color all vertices of K with a, delete K, and remove a from the remaining lists.

Every vertex whose list shrinks belongs to $W \setminus K$, so it has an out-neighbor in K. Its out-degree therefore decreases by at least one, preserving the inequality between list size and out-degree. For all other vertices the inequality is unchanged or improves. The remaining digraph is kernel-perfect, so induction colors it. No remaining vertex uses a, and K is independent, giving a proper coloring of the whole graph. □

Write $\chi'_\ell(G)$ for the least k such that every assignment of lists of size k to the edges admits a proper edge coloring from those lists.

Theorem 353 (Galvin). *Every loopless bipartite multigraph G satisfies*

$$\chi'_\ell(G) = \chi'(G) = \Delta(G).$$

In particular, lists of size at least $\Delta(G)$ always suffice, even when different edges have different lists.

Proof. The lower bound follows by giving every edge the same list: the edges at a vertex of maximum degree need distinct colors. The edgeless case is immediate, so assume $E(G) \neq \emptyset$ and put $\Delta = \Delta(G)$. Fix arbitrary lists $L(e)$ of size at least Δ. Let X, Y be the bipartition, and fix an ordinary edge coloring $c : E(G) \to \{1, \dots, \Delta\}$ supplied by Theorem 350.

Construct a digraph D whose vertices are the edges of G. At an endpoint in X, direct arcs from larger colors to smaller colors; at an endpoint in Y, direct arcs from smaller colors to larger colors. For parallel edges, these rules give opposite arcs. The underlying graph of D is exactly the line graph of G, so coloring it is equivalent to edge coloring G.

An edge of color i has at most $i - 1$ out-neighbors through its endpoint in X, and at most $\Delta - i$ through its endpoint in Y. Hence

$$d^+_D(e) \leq (i - 1) + (\Delta - i) = \Delta - 1.$$

It remains to prove that D is kernel-perfect.

Fix any $F \subseteq E(G)$. In the bipartite multigraph $(X \cup Y, F)$, let each vertex of X prefer smaller edge colors, and each vertex of Y prefer larger edge colors. All incident edges are strictly ordered, and being matched is preferred to being unmatched. We seek a matching $M \subseteq F$ such that, for every edge $e = xy \notin M$, at least one endpoint is matched and prefers its matching edge to e.

Such a *stable matching* is obtained by Gale-Shapley. The matching M is an independent set in $D[F]$. For every $e \in F \setminus M$, stability supplies a preferred incident matching edge $f \in M$, and the definition of D gives the arc $e \to f$. Thus M is a kernel of $D[F]$. Since F was arbitrary, D is kernel-perfect.

Now $|L(e)| \geq \Delta \geq d^+_D(e) + 1$, so Lemma 352 gives the required edge coloring. □

Corollary 354 (Dinitz's theorem). *If each cell of an $n \times n$ array has a list of at least n symbols, one can choose a symbol from each list so that no symbol repeats in a row or a column.*

Proof. Use one vertex for each row and one for each column, and identify a cell with the corresponding edge of $K_{n,n}$. Apply Theorem 353. □

8.12 Planarity and Euler's Formula

For a connected plane graph, let n, m, f be the numbers of vertices, edges, and faces. Euler's formula and the basic incidence count are

$$n - m + f = 2, \qquad \sum_F \ell(F) = 2m,$$

where $\ell(F)$ is the length of the boundary walk of F.

If every face has length at least $g \geq 3$, then

$$gf \leq 2m.$$

Combining this with Euler gives

$$n - m + \frac{2m}{g} \geq 2,$$

and hence

$$m \leq \frac{g}{g-2}(n - 2).$$

Thus a simple planar graph satisfies

$$m \leq 3n - 6,$$

while a simple bipartite planar graph satisfies

$$m \leq 2n - 4.$$

In particular, every simple planar graph has a vertex of degree at most 5.

8.12.1 Sylvester–Gallai

Theorem 355 (Sylvester–Gallai)**.** *Every finite noncollinear set of points in $\mathbb{R}^2$ determines a line containing exactly two of the points.*

Proof. We use projective duality. Represent a point

$$p = (a, b) \in \mathbb{R}^2$$

by the vector

$$\widetilde{p} = (a, b, 1) \in \mathbb{R}^3,$$

and associate with it the great circle

$$C_p = \{x \in S^2 : \langle x, \widetilde{p} \rangle = 0\}.$$

A line

$$\alpha x + \beta y + \gamma = 0$$

through some of the original points corresponds to the antipodal pair

$$\pm \frac{(\alpha, \beta, \gamma)}{\sqrt{\alpha^2 + \beta^2 + \gamma^2}}$$

lying on all the corresponding great circles. Thus a line containing exactly k original points corresponds to an antipodal pair at which exactly k great circles meet.

The great circles form a simple plane graph on the sphere: its vertices are their intersection points, and its edges are the arcs between consecutive intersections. Since the original points are not collinear, the circles are not all concurrent.

If exactly k circles meet at a vertex, that vertex has degree $2k$. Hence every vertex has even degree at least 4.

But every simple planar graph has a vertex of degree at most 5. Its degree here must therefore be exactly 4, so exactly two circles meet there. Dualizing back gives a line containing exactly two of the original points. □

The same argument gives more than existence. Let t_k denote the number of lines containing exactly k of the given points. On the sphere,

$$V = 2 \sum_{k \geq 2} t_k,$$

while the degree sum gives

$$2E = \sum_{k \geq 2} (2t_k)(2k),$$

so

$$E = 2 \sum_{k \geq 2} k t_k.$$

Since $E \leq 3V - 6$,

$$2 \sum_{k \geq 2} k t_k \leq 6 \sum_{k \geq 2} t_k - 6.$$

Rearranging yields *Melchior's inequality*

$$t_2 \geq 3 + \sum_{k \geq 4} (k - 3) t_k.$$

Thus there are in fact at least three ordinary lines, and every line containing many points forces still more of them.

8.12.2 The Colorful Sylvester–Gallai Theorem

For the two-color version we need one more consequence of Euler's formula. Suppose that the edges around each vertex of a plane graph are read in cyclic order. A *color change* is a corner at which two consecutive edges have different colors.

Lemma 356 (Two-color lemma)**.** *If the edges of a finite simple plane graph are colored with two colors, then some vertex has at most two color changes around it.*

Proof. Suppose otherwise. The number of color changes around any vertex is even, so every vertex has at least four. If c is the total number of changing corners, then

$$c \geq 4n.$$

Now count the same corners by faces. Around a face of length $r \geq 3$, the number of changes is even and at most $2(r-2)$: for $r = 3$ it is at most 2, for $r = 4$ at most 4, and thereafter the bound is weaker than the trivial bound r.

Hence

$$c \leq 2\sum_{F}(\ell(F) - 2) = 2(2m - 2f) = 4(m - f).$$

Euler gives $m - f = n - 2$, so

$$c \leq 4n - 8,$$

contradicting $c \geq 4n$. □

Theorem 357 (Motzkin–Rabin)**.** *Let a finite noncollinear set of points in the plane be colored red and blue. Then there is a line containing at least two points, all of the same color; equivalently, the line contains points of one color and none of the other.*

Proof. Dualize the points to red and blue great circles as above. We must show that some intersection point lies only on circles of one color.

Suppose instead that every intersection contains circles of both colors. Consider the cyclic order of the circle-arcs around such a vertex. The two rays belonging to one great circle are opposite and have the same color. Therefore the cyclic color pattern repeats after half a turn.

Since both colors occur, the first half-turn contains at least two color changes when viewed cyclically. Repeating the pattern gives at least four color changes around the whole vertex.

Thus every vertex has at least four color changes, contradicting Lemma 356. □

8.12.3 Crossings and Large Planar Subgraphs

Planarity can still give useful information when a drawing has many crossings.

Consider a drawing of a simple graph G with $n \geq 3$ vertices, m edges, and X crossings. Form its *crossing graph* H: the vertices of H are the edges of G, and two are adjacent if the corresponding drawn edges cross.

An independent set in H is precisely a collection of pairwise noncrossing edges of G. These edges form a planar subgraph, so

$$\alpha(H) \leq 3n - 6.$$

Caro–Wei gives

$$\alpha(H) \geq \sum_{v \in V(H)} \frac{1}{d_H(v) + 1}.$$

By Cauchy–Schwarz,

$$\alpha(H) \geq \sum_{v \in V(H)} \frac{1}{d_H(v) + 1} \geq \frac{m^2}{\sum_v (d_H(v) + 1)} = \frac{m^2}{m + 2|E(H)|}.$$

Since every adjacent pair in H contributes at least one crossing,

$$|E(H)| \le X.$$

Therefore

$$3n - 6 \ge \frac{m^2}{m + 2X},$$

which rearranges to

$$X \ge \frac{1}{2}\left(\frac{m^2}{3n-6} - m\right).$$

Equivalently, every drawing with X crossings contains at least

$$\frac{m^2}{m+2X}$$

pairwise noncrossing edges.

There is one simple geometric trick. Randomly divide the vertices of G into two parts and retain only the edges crossing the partition. Let M and Y be the numbers of retained edges and retained crossings. The resulting graph is bipartite, so a crossing-free subset has at most $2n - 4$ edges. Applying the same Caro–Wei argument conditionally gives

$$Y \ge \frac{1}{2}\left(\frac{M^2}{2n-4} - M\right).$$

Every original edge survives with probability $1/2$, while every crossing survives with probability $1/4$. Hence

$$\mathbb{E}M = \frac{m}{2}, \qquad \mathbb{E}Y = \frac{X}{4}.$$

Also

$$\mathbb{E}M^2 \ge (\mathbb{E}M)^2 = \frac{m^2}{4}.$$

Taking expectations in the preceding inequality gives

$$\frac{X}{4} \ge \frac{1}{2}\left(\frac{m^2/4}{2n-4} - \frac{m}{2}\right),$$

and therefore

$$X \ge \frac{m^2}{4n-8} - m.$$

This is a stronger estimate at high density. For the crossing lemma, however, a still simpler linear estimate turns out to amplify better.

8.12.4 The Crossing Lemma

The crossing lemma says that once a graph is moderately denser than planar, its number of crossings grows cubically with the number of edges.

The key first step is to exploit bipartiteness once more.

Lemma 358. *Every drawing of a simple graph with n vertices, m edges, and X crossings satisfies*

$$X \ge 2m - 8n.$$

Proof. Randomly partition the vertices into two classes, and retain only edges joining the two classes. Let M be the number of retained edges and Y the number of retained crossings.

Each edge survives with probability $1/2$, so

$$\mathbb{E}M = \frac{m}{2}.$$

A fixed crossing survives exactly when both of its two edges cross the random cut, which occurs with probability $1/4$. Hence

$$\mathbb{E}Y = \frac{X}{4}.$$

For every outcome, delete one edge participating in each remaining crossing. At most Y edges are deleted, leaving a planar bipartite graph with at least $M - Y$ edges. Therefore

$$M - Y \leq 2n - 4,$$

and in particular

$$Y \geq M - 2n.$$

Taking expectations,

$$\frac{X}{4} \geq \frac{m}{2} - 2n.$$

Multiplying by 4 gives

$$X \geq 2m - 8n.$$

□

This linear inequality by itself is weak for dense graphs. The amplification comes from random vertex sampling. An edge survives with probability p^2, whereas a crossing, which involves four vertices, survives with probability p^4.

Theorem 359 (Crossing lemma)**.** *Let G be a simple graph with n vertices and $m \geq 6n$ edges. Then*

$$\mathrm{cr}(G) \geq \frac{1}{54}\frac{m^3}{n^2}.$$

Proof. Take a drawing of G with

$$X = \mathrm{cr}(G)$$

crossings. We may assume that adjacent edges do not cross: any such crossing can be removed locally without creating a new one. Thus every crossing involves four distinct vertices.

Keep every vertex independently with probability p, and retain the induced drawing. Let N_p, M_p, X_p denote its numbers of vertices, edges, and crossings. Then

$$\mathbb{E}N_p = pn, \qquad \mathbb{E}M_p = p^2 m, \qquad \mathbb{E}X_p = p^4 X.$$

The last identity holds because all four endpoints of a crossing must survive.

Apply Lemma 358 to every realization:

$$X_p \geq 2M_p - 8N_p.$$

Taking expectations gives

$$p^4 X \geq 2p^2 m - 8pn.$$

Hence

$$X \geq \frac{2m}{p^2} - \frac{8n}{p^3}.$$

We choose p to maximize the right-hand side. Differentiating,

$$\frac{d}{dp}\left(\frac{2m}{p^2}-\frac{8n}{p^3}\right)=-\frac{4m}{p^3}+\frac{24n}{p^4},$$

so the optimum occurs at

$$p=\frac{6n}{m}.$$

The assumption $m \geq 6n$ ensures $p \leq 1$.

Substituting,

$$\frac{1}{p^2}=\frac{m^2}{36n^2}, \qquad \frac{1}{p^3}=\frac{m^3}{216n^3},$$

and therefore

$$X \geq 2m\frac{m^2}{36n^2}-8n\frac{m^3}{216n^3}=\left(\frac{1}{18}-\frac{1}{27}\right)\frac{m^3}{n^2}=\frac{1}{54}\frac{m^3}{n^2}.$$

□

For comparison, if one starts only from the usual estimate

$$X \geq m-3n,$$

the same sampling argument gives

$$\operatorname{cr}(G) \geq \frac{1}{64}\frac{m^3}{n^2} \qquad (m \geq 4n).$$

Optimizing that same argument gives

$$\operatorname{cr}(G) \geq \frac{4}{243}\frac{m^3}{n^2} \qquad \left(m \geq \frac{9}{2}n\right).$$

8.12.5 Szemerédi–Trotter

The crossing lemma gives a proof of the Szemerédi–Trotter incidence bound. The construction is to turn each line into a path through the points incident to it. Many incidences then give many graph edges, while two such edges can cross only when their supporting lines cross. Thus we have a small upper bound on the crossing number, and the crossing lemma gives a large lower bound.

Theorem 360 (Szemerédi–Trotter)**.** *Let P be a set of p points and $\mathcal{L}$ a set of ℓ lines in $\mathbb{R}^2$. If $I(P,\mathcal{L})$ is the number of incidences between them, then*

$$I(P,\mathcal{L}) \leq 3p^{2/3}\ell^{2/3}+6p+\ell.$$

Proof. Write $I=I(P,\mathcal{L})$. On each line $L \in \mathcal{L}$, list its incident points in their order along the line and join each consecutive pair by the segment between them. Let G be the resulting graph on vertex set P.

If L contains k_L points, it contributes

$$\max\{k_L-1,0\}$$

edges. Hence, writing $e=|E(G)|$,

$$e=\sum_{L\in\mathcal{L}}\max\{k_L-1,0\} \geq \sum_{L\in\mathcal{L}}(k_L-1)=I-\ell.$$

Equivalently,

$$I \leq e+\ell.$$

The graph G is simple. Indeed, two points determine at most one line, so the same pair cannot be joined by two different lines. Also, because we join only consecutive points on each line, no graph edge passes through another vertex of P.

Now consider crossings in this drawing. Edges lying on the same line do not cross in their interiors. Edges lying on two distinct lines can cross at most once, since two lines meet at most once. Thus each pair of lines contributes at most one crossing, and after an arbitrarily small local perturbation at multiple concurrence points if necessary,

$$\operatorname{cr}(G) \le \binom{\ell}{2} < \frac{\ell^2}{2}.$$

There are two cases.

If

$$e < 6p,$$

then immediately

$$I \le e + \ell < 6p + \ell,$$

which is stronger than the claimed bound.

Otherwise $e \ge 6p$, so the crossing lemma (Theorem 359) applies:

$$\operatorname{cr}(G) \ge \frac{1}{54}\frac{e^3}{p^2}.$$

Together with the geometric upper bound,

$$\frac{1}{54}\frac{e^3}{p^2} \le \frac{\ell^2}{2}.$$

Multiplying by $54p^2$ gives

$$e^3 \le 27p^2\ell^2.$$

Taking cube roots,

$$e \le 3p^{2/3}\ell^{2/3}.$$

Therefore

$$I \le e + \ell \le 3p^{2/3}\ell^{2/3} + \ell,$$

again proving the stated bound. □

8.13 The Erdős–Sós Theorem

Theorem 361 (Erdős–Sós)**.** *Every nonempty finite simple graph of average degree greater than $k-2$ contains every tree on $k \ge 2$ vertices as a subgraph.*

The bound is sharp: a disjoint union of copies of K_{k-1} has average degree $k-2$ and contains no connected graph on k vertices.

With minimum degree at least $k-1$, the theorem is immediate by greedy embedding. Root the tree anywhere and expose its vertices away from the root. Whenever we embed a new child, its parent has at most $k-2$ already used vertices to avoid, but at least $k-1$ neighbours.

Average degree is harder because the embedding may arrive at a vertex of very small degree. We therefore need to average the *places from which an embedding could start*, rather than insist that every place works.

The strengthened statement. Give every edge of G both orientations. Thus a graph with n vertices and average degree d has

$$\sum_{u \in V(G)} d_G(u) = dn$$

directed edges $u \to v$.

We shall prove something stronger than merely finding a copy of a tree. Fix a tree T with t vertices, and distinguish an oriented edge

$$r \to s$$

of T. Think of r as the root and rs as the first edge of the embedding.

Now put the vertices of G in a random cyclic order. A directed host edge $u \to v$ will be called *good* if we can embed all of T between u and v around the circle, with

$$r \mapsto u, \qquad s \mapsto v.$$

The main claim will be

$$\text{expected number of bad directed edges} \leq (t-2)n.$$

This immediately proves the theorem: if $d > t-2$, then the graph has $dn > (t-2)n$ directed edges, so for some cyclic order not all of them can be bad.

Why should the loss be $t-2$? A tree consisting of one edge costs nothing: every directed host edge is already a copy. Starting from that edge, each time we glue in more of the tree, the argument below loses at most *one directed edge at each host vertex*. Altogether there are exactly $t-2$ vertices beyond the initial edge.

The cyclic ordering is what makes the gluing work. The endpoint v of $u \to v$ acts as a *deadline*: the entire tree must be embedded before we reach v. When the tree branches at its root, we use the first deadline at which one branch fits. Everything before that deadline is reserved for the first branch; after a measure-preserving rearrangement, everything after it is reserved for the second branch. The two embeddings are then automatically disjoint.

Good directed edges. Let π be a cyclic ordering of $V(G)$. For vertices u, v, let

$$[u, v]_\pi$$

denote the set of vertices encountered when moving from u to v in the direction of π, including both endpoints.

Fix an oriented edge $r \to s$ of a tree T. We say that a directed edge $u \to v$ of G *supports* $(T, r \to s)$ in π if there is an embedding

$$\phi : T \hookrightarrow G$$

such that

$$\phi(r) = u, \qquad \phi(s) = v, \qquad \phi(V(T)) \subseteq [u, v]_\pi.$$

Thus uv plays the distinguished tree edge rs, while v is also the deadline by which every other vertex of the tree must have appeared.

For a fixed host vertex u, let

$$b_{\pi,u}(T, r \to s)$$

be the number of outgoing directed edges $u \to v$ which do not support $(T, r \to s)$, and set

$$b_\pi(T, r \to s) = \sum_{u \in V(G)} b_{\pi,u}(T, r \to s).$$

Lemma 362 (Average number of bad directed edges)**.** *Let G have n vertices, and let $r \to s$ be an oriented edge of a t-vertex tree T. For a uniformly random cyclic ordering π,*

$$\mathbb{E}\, b_\pi(T, r \to s) \leq (t-2)n.$$

Proof. We induct on t.

If $t = 2$, then T consists only of the edge rs. Every directed host edge $u \to v$ supports it, so

$$b_\pi(T, r \to s) = 0.$$

Assume now that $t \geq 3$.

Step 1: move the root if necessary.

Let $\bar{\pi}$ be the cyclic order obtained by reversing π. As sets,

$$[u, v]_\pi = [v, u]_{\bar{\pi}}.$$

Hence the same embedded copy shows

$$u \to v \text{ supports } (T, r \to s) \text{ in } \pi$$

if and only if

$$v \to u \text{ supports } (T, s \to r) \text{ in } \bar{\pi}.$$

Since reversal preserves the uniform distribution on cyclic orders,

$$\mathbb{E}\, b_\pi(T, r \to s) = \mathbb{E}\, b_\pi(T, s \to r).$$

Therefore, if r is a leaf, we may reverse the distinguished edge and root the tree at s instead. Since $t \geq 3$, the new root has degree at least two. Thus from now on we may assume

$$d_T(r) \geq 2.$$

This is the main reason for using cyclic rather than linear orders: there is no preferred beginning, so the two ends of the distinguished edge are perfectly symmetric.

Step 2: split the branches at the root.

Separate the branches incident with r into two nonempty groups, putting the distinguished edge rs into the second group. This writes

$$T = T_1 \cup T_2, \qquad V(T_1) \cap V(T_2) = \{r\}.$$

Choose an edge

$$rr_1 \in E(T_1),$$

and retain

$$rs \in E(T_2)$$

as the distinguished edge of T_2.

Both T_1 and T_2 have fewer vertices than T, and because the root is counted in both,

$$|T_1| + |T_2| = |T| + 1. \tag{90}$$

We now show that gluing the two branches costs at most one additional bad directed edge per host vertex.

Fix $u \in V(G)$ and write the cyclic order starting from u as

$$\pi = (u, v_2, v_3, \ldots, v_n).$$

Scan clockwise from u until we find the first directed edge $u \to v_j$ which supports

$$(T_1, r \to r_1).$$

If there is no such edge, set $j = n$.

The point of choosing the *first* one is that every host edge $u \to v_i$ with $i < j$ is already certified bad for T_1. The vertices

$$u, v_2, \dots, v_j$$

contain a copy of T_1 whenever such a first successful edge exists. We now want to reserve the vertices after v_j for T_2.

Step 3: move the suffix in front of the first branch.

Define a new cyclic order

$$\varphi_u(\pi) = (u, v_{j+1}, v_{j+2}, \dots, v_n, v_j, v_{j-1}, \dots, v_2). \tag{91}$$

Thus the old suffix

$$v_{j+1}, \dots, v_n$$

is moved immediately after u, while the old prefix is placed after it in reverse order.

Why this particular rearrangement? Suppose $i > j$ and the directed edge $u \to v_i$ supports $(T_2, r \to s)$ in $\varphi_u(\pi)$. Then its T_2-copy lies entirely in

$$\{u, v_{j+1}, \dots, v_i\}.$$

Meanwhile the first T_1-copy lies entirely in

$$\{u, v_2, \dots, v_j\}.$$

These two sets meet only in u. Hence the two embedded trees are automatically disjoint away from their common root, and their union is an embedding of T.

Moreover, both sets lie inside

$$[u, v_i]_\pi = \{u, v_2, \dots, v_i\},$$

so $u \to v_i$ supports $(T, r \to s)$ in the *original* order π.

We have therefore divided the outgoing edges at u into three types:

- if $i < j$, then $u \to v_i$ is bad for T_1;
- if $i > j$ and $u \to v_i$ is bad for T, then it must be bad for T_2 in $\varphi_u(\pi)$;
- the single edge $u \to v_j$ may be lost.

Consequently

$$b_{\pi,u}(T, r \to s) \le b_{\pi,u}(T_1, r \to r_1) + b_{\varphi_u(\pi),u}(T_2, r \to s) + 1. \tag{92}$$

If no edge supports T_1, then every outgoing edge is already counted by the first term, so the same inequality remains true.

Step 4: the rearrangement does not bias the random order.

This is essential. We want to apply the induction hypothesis to T_2 in the new order, so $\varphi_u(\pi)$ must still be uniformly distributed when π is uniform.

There is a short way to see this. First define

$$\psi_u(\pi) = (u, v_2, \dots, v_j, v_n, v_{n-1}, \dots, v_{j+1}).$$

Thus ψ_u keeps the prefix through the first successful deadline v_j fixed and merely reverses the suffix after it.

Because this prefix is unchanged, the first successful index is still j after applying ψ_u. Therefore applying ψ_u again reverses the same suffix a second time:

$$\psi_u^2 = \mathrm{id}\,.$$

So ψ_u is a bijection on cyclic orders.

Now reverse the entire cyclic order after applying ψ_u. This gives exactly

$$(u, v_{j+1}, \dots, v_n, v_j, \dots, v_2) = \varphi_u(\pi).$$

Both suffix reversal and full reversal are bijections. Hence φ_u is a bijection, and so

$$\pi \text{ uniform} \quad \Longrightarrow \quad \varphi_u(\pi) \text{ uniform}.$$

This measure-preserving property is the only reason for the somewhat specific-looking rearrangement in Equation (91).

Step 5: take expectations.

Taking expectations in Equation (92) gives, for each u,

$$\mathbb{E}\, b_{\pi,u}(T, r \to s) \le \mathbb{E}\, b_{\pi,u}(T_1, r \to r_1) + \mathbb{E}\, b_{\pi,u}(T_2, r \to s) + 1,$$

because $\varphi_u(\pi)$ is uniform.

Summing over all n choices of u,

$$\mathbb{E}\, b_{\pi}(T, r \to s) \le \mathbb{E}\, b_{\pi}(T_1, r \to r_1) + \mathbb{E}\, b_{\pi}(T_2, r \to s) + n.$$

By induction,

$$\mathbb{E}\, b_{\pi}(T, r \to s) \le (|T_1| - 2)n + (|T_2| - 2)n + n = (|T_1| + |T_2| - 3)n = (|T| - 2)n,$$

where the last equality uses Equation (90). □

Proof of Theorem 361. Let T be a tree on k vertices, and orient any one of its edges as $r \to s$.

If G has n vertices and average degree d, then the number of directed edges of G is

$$\sum_{u \in V(G)} d_G(u) = 2|E(G)| = dn.$$

By Lemma 362, a uniformly random cyclic order satisfies

$$\mathbb{E}\, b_{\pi}(T, r \to s) \le (k-2)n.$$

Since $d > k - 2$,

$$(k-2)n < dn.$$

Thus the expected number of bad directed edges is strictly smaller than the total number of directed edges. In particular, for some cyclic ordering π,

$$b_{\pi}(T, r \to s) < dn.$$

At least one directed edge $u \to v$ is therefore good, and by definition its witnessing embedding is a copy of T in G. □

The proof is an averaged version of greedy embedding, but the averaging is arranged so that different branches never collide. A directed edge $u \to v$ gives an interval in which the tree must fit. At a branching root, the *first* interval which fits one group of branches marks a boundary. A measure-preserving rearrangement places the unused vertices on the other side of that boundary, so the second group of branches can be embedded there independently. The first group pays for all earlier failures, the second group pays for all later failures, and only the boundary edge itself is lost. This single lost edge per host vertex is what accumulates to the threshold $k - 2$.

8.14 Huang's Hypercube Theorem

The *n-dimensional hypercube* Q_n has vertex set $\{0,1\}^n$, with two vertices adjacent when they differ in exactly one coordinate. Write $\Delta(H)$ for the maximum degree of a graph H.

Theorem 363 (Huang)**.** *For every $n \geq 1$ and every $S \subseteq \{0,1\}^n$ with $|S| > 2^{n-1}$,*

$$\Delta(Q_n[S]) \geq \sqrt{n}.$$

Exactly half the vertices can induce no edges: take the strings with an even number of ones. The theorem says that just one vertex beyond this threshold forces a vertex with at least $\lceil \sqrt{n} \rceil$ neighbours inside the set.

An eigenvector certifies the degree. Suppose we assign signs to the edges of a graph and let A be its signed adjacency matrix. If $Ax = \lambda x$, choose a coordinate of x with largest absolute value. Its eigenvalue equation expresses λ times that coordinate as a signed sum of neighbouring coordinates, none larger in absolute value. There must therefore be at least λ nonzero contributions.

To use this observation inside a prescribed set S, we need an eigenvector which vanishes outside S. A single eigenvector is not enough: deleting even one vertex might destroy its support. A large eigenspace gives room to impose the required zeros. Each excluded vertex imposes only one homogeneous linear equation, so fewer equations than the dimension cannot eliminate every vector.

Lemma 364 (An eigenspace survives coordinate restrictions)**.** *Let G be a finite simple graph on N vertices, and let A be a real symmetric matrix with zero diagonal, $A_{uv} = 0$ for nonedges, and $|A_{uv}| \leq 1$ for edges. Suppose A has a positive eigenvalue λ whose eigenspace has dimension r. Then every $S \subseteq V(G)$ with $|S| > N - r$ satisfies $\Delta(G[S]) \geq \lambda$.*

Proof. Within the λ-eigenspace, impose $x_v = 0$ for all $v \notin S$. These $N - |S| < r$ equations leave a nonzero solution x. Choose $u \in S$ maximizing $|x_u|$. Then

$$\lambda |x_u| = \left| \sum_{v \in S \cap N_G(u)} A_{uv} x_v \right| \leq |S \cap N_G(u)| \, |x_u|.$$

Since $|x_u| > 0$, cancellation proves the claim. □

The target is now to construct a signing of Q_n with an eigenvalue $\sqrt{n}$ of multiplicity 2^{n-1}. The *size* of the eigenvalue will give the degree bound; its *multiplicity* will give the half-the-vertices threshold.

Why signs help: cancel the two orders of a two-step walk. The entry $(A^2)_{uv}$ is the sum of the signed weights of two-step walks from u to v. Every vertex has n walks which immediately return, and each has weight 1, regardless of the edge signs. Thus $(A^2)_{uu} = n$.

The only other possible endpoints differ in two coordinates. There are exactly two walks between them, corresponding to the two orders in which those coordinates can be changed. If their signs cancel, all off-diagonal entries disappear and we obtain $A^2 = nI$. This would force every eigenvalue to be either $\sqrt{n}$ or $-\sqrt{n}$. A zero trace would then put exactly half the eigenvalues at each value.

The recursive structure of the cube makes this cancellation easy. It consists of two copies of Q_{n-1} joined by a matching. Give the matching positive signs, use a signing B in the first copy, and reverse every sign in the second. A walk which uses the matching once can take its other step before or after crossing; these two possibilities now have opposite signs.

Lemma 365 (A signing with two eigenvalues)**.** *The hypercube Q_n has a symmetric signed adjacency matrix A_n satisfying $A_n^2 = nI$. For $n \geq 1$, its eigenvalues are $\sqrt{n}$ and $-\sqrt{n}$, each with multiplicity 2^{n-1}.*

Proof. Starting with the one-by-one matrix $A_0 = (0)$, define

$$A_j = \begin{pmatrix} A_{j-1} & I \\ I & -A_{j-1} \end{pmatrix}.$$

The diagonal blocks describe the two copies of Q_{j-1}, and the identity blocks describe their matching. Thus A_j is a symmetric signed adjacency matrix of Q_j. Moreover,

$$A_j^2 = \begin{pmatrix} A_{j-1}^2 + I & A_{j-1} - A_{j-1} \\ A_{j-1} - A_{j-1} & A_{j-1}^2 + I \end{pmatrix} = jI$$

by induction. Its eigenvalues are therefore $\pm\sqrt{j}$. For $j \geq 1$, symmetry gives a basis of eigenvectors, and the zero trace makes the two multiplicities equal. Their sum is 2^j, so each is 2^{j-1}. □

Proof of Theorem 363. Apply Lemma 364 to A_n, with $N = 2^n$, $r = 2^{n-1}$, and $\lambda = \sqrt{n}$. □

Sharpness: a block construction. The bound $\lceil\sqrt{n}\rceil$ is attainable with exactly $2^{n-1}+1$ vertices. Identify a cube vertex with a subset of $[n]$, and partition $[n]$ into $r = \lceil\sqrt{n}\rceil$ nonempty blocks $B_1, \ldots, B_r$, each of size at most r. Let $\mathcal{C}$ consist of the subsets containing no entire block.

We want internal cube edges to occur only when changing one element changes membership in $\mathcal{C}$. This happens if we take the even members of $\mathcal{C}$ and the odd members of its complement, or make the opposite parity choice. Indeed, adjacent cube vertices have opposite parity, so a neighbour of a selected vertex is also selected precisely when its membership in $\mathcal{C}$ differs.

Either choice has maximum induced degree at most r. From a set in $\mathcal{C}$, a change leaving $\mathcal{C}$ must add the last missing element of a block, with at most one possibility per block. From a set outside $\mathcal{C}$, a change entering $\mathcal{C}$ is possible only when exactly one block is full, and must remove an element of that block. There are at most r such possibilities.

It remains to choose the larger parity class. The alternating sum factorizes over the blocks:

$$\sum_{S\in\mathcal{C}}(-1)^{|S|} = \prod_{j=1}^{r}\left(\sum_{T\subsetneq B_j}(-1)^{|T|}\right) = \prod_{j=1}^{r}\left(-(-1)^{|B_j|}\right) = (-1)^{n+r}.$$

Here the sum over all subsets of a nonempty block is zero, so excluding its full subset leaves $-(-1)^{|B_j|}$. Consequently the two parity choices have sizes $2^{n-1}+1$ and $2^{n-1}-1$. The larger gives the claimed sharp example.

9 Convexity and Duality

9.1 Sion's Minimax Theorem

A function g on a convex set is *quasiconvex* if

$$g((1-t)x + ty) \leq \max\{g(x), g(y)\} \qquad (t \in [0,1]),$$

equivalently, if every sublevel set $\{x : g(x) \leq a\}$ is convex.

It is *quasiconcave* if

$$g((1-t)x + ty) \geq \min\{g(x), g(y)\},$$

equivalently, if every superlevel set $\{x : g(x) \geq a\}$ is convex.

Sion's theorem says that quasiconvexity in the minimizing variable and quasiconcavity in the maximizing variable are enough for minimax equality.

The proof is easiest to understand at a fixed threshold a. For a choice y, call x *good for* y if $f(x,y) \leq a$. Quasiconvexity in x says that the good points for each fixed y form a convex set.

The key fact is that if two choices y_0, y_1 have no common good point, then some convex combination of them is bad for *every* x. Quasiconcavity in y is exactly what makes this possible.

Lemma 366 (Two-Point Reduction)**.** *Let X be compact and convex, let $y_0, y_1 \in Y$, and set*

$$y_t = (1-t)y_0 + ty_1.$$

Suppose that $f(\cdot, y)$ is quasiconvex on X for every y, that $f(x, \cdot)$ is quasiconcave on Y for every x, and that f is continuous.

Set

$$M = \min_{x \in X} \max\{f(x, y_0), f(x, y_1)\}.$$

Then for some $t \in [0, 1]$,

$$\boxed{\min_{x \in X} f(x, y_t) \geq M.}$$

Proof. Define

$$U_i = \{x \in X : f(x, y_i) < M\}, \qquad i \in \{0, 1\},$$

and

$$D_t = \{x \in X : f(x, y_t) < M\}.$$

By the definition of M,

$$U_0 \cap U_1 = \emptyset.$$

Indeed, if some x belonged to both, then

$$\max\{f(x, y_0), f(x, y_1)\} < M,$$

contradicting the definition of M.

Now quasiconcavity in y gives

$$f(x, y_t) \geq \min\{f(x, y_0), f(x, y_1)\}.$$

Hence if $f(x, y_t) < M$, then at least one of $f(x, y_0), f(x, y_1)$ is less than M. Therefore

$$D_t \subseteq U_0 \cup U_1.$$

On the other hand, D_t is convex, since it is a strict sublevel set of the quasiconvex function $f(\cdot, y_t)$. In particular, D_t is connected. Since U_0 and U_1 are disjoint open subsets of X, every nonempty D_t must therefore lie entirely in one of them:

$$D_t \subseteq U_0 \qquad \text{or} \qquad D_t \subseteq U_1.$$

If some D_t is empty, then $f(x, y_t) \geq M$ for every x, and we are done. Suppose therefore that D_t is nonempty for every $t \in [0, 1]$.

Define

$$I = \{t : D_t \subseteq U_0\}, \qquad J = \{t : D_t \subseteq U_1\}.$$

Then I and J are disjoint and cover $[0, 1]$. Moreover, $D_0 = U_0$ and $D_1 = U_1$, so $0 \in I$ and $1 \in J$.

We claim that I is closed. Let $t_k \in I$ with $t_k \to t$. Choose $x \in D_t$, which is possible by assumption. Since $f(x, y_t) < M$, continuity gives

$$f(x, y_{t_k}) < M$$

for all sufficiently large k. Hence

$$x \in D_{t_k} \subseteq U_0,$$

so D_t meets U_0. But D_t lies entirely in either U_0 or U_1, and therefore $D_t \subseteq U_0$. Thus $t \in I$.

The same argument shows that J is closed. This writes the connected interval $[0, 1]$ as a disjoint union of two nonempty closed sets, a contradiction.

Hence $D_t = \emptyset$ for some t, and consequently

$$\min_x f(x, y_t) \geq M.$$

□

The lemma says that two incompatible choices of y can be replaced by a single convex combination that is at least as bad for the minimizer as the better of the two choices was pointwise.

Lemma 367 (Finite Reduction)**.** *Let $y_1, \dots, y_m \in Y$ and $a \in \mathbb{R}$. If*

$$\bigcap_{i=1}^{m} \{x \in X : f(x, y_i) \leq a\} = \emptyset,$$

then there is

$$\bar{y} \in \operatorname{conv}\{y_1, \dots, y_m\}$$

such that

$$\boxed{\min_{x \in X} f(x, \bar{y}) > a.}$$

Proof. We induct on m.

For $m = 1$, the hypothesis says $f(x, y_1) > a$ for every $x \in X$. Compactness and continuity give

$$\min_x f(x, y_1) > a.$$

Now suppose $m > 1$, and let

$$X' = \{x \in X : f(x, y_m) \leq a\}.$$

This is compact and convex: it is a closed sublevel set of the continuous quasiconvex function $f(\cdot, y_m)$.

If $X' = \emptyset$, then $\bar{y} = y_m$ works.

Otherwise, within X' the first $m - 1$ sublevel sets have empty intersection. By the inductive hypothesis, there is

$$y' \in \operatorname{conv}\{y_1, \dots, y_{m-1}\}$$

such that

$$\min_{x \in X'} f(x, y') > a.$$

Thus for every $x \in X$,

$$\max\{f(x, y'), f(x, y_m)\} > a.$$

Indeed, if $x \in X'$, then $f(x, y') > a$; while if $x \notin X'$, then $f(x, y_m) > a$.

Since X is compact,

$$M := \min_{x \in X} \max\{f(x, y'), f(x, y_m)\} > a.$$

Applying Lemma 366 to y' and y_m, there is some point $\bar{y}$ on the segment joining them such that

$$\min_x f(x, \bar{y}) \geq M > a.$$

Finally,

$$\bar{y} \in \operatorname{conv}\{y_1, \dots, y_m\}.$$

□

Theorem 368 (Sion's Minimax Theorem)**.** *Let X be a nonempty compact convex set and let Y be a nonempty convex set, both contained in finite-dimensional real vector spaces. Let*

$$f : X \times Y \to \mathbb{R}$$

be continuous.

Suppose that for every $y \in Y$, the function $f(\cdot, y)$ is quasiconvex on X, and for every $x \in X$, the function $f(x, \cdot)$ is quasiconcave on Y. Then

$$\boxed{\min_{x \in X} \sup_{y \in Y} f(x, y) = \sup_{y \in Y} \min_{x \in X} f(x, y).}$$

If Y is also compact, the supremum is attained, and hence

$$\boxed{\min_{x\in X}\max_{y\in Y} f(x,y) = \max_{y\in Y}\min_{x\in X} f(x,y).}$$

Proof. Set

$$\alpha = \sup_{y\in Y}\min_{x\in X} f(x,y).$$

Weak minimax always gives

$$\alpha \le \min_{x}\sup_{y} f(x,y).$$

We prove the reverse inequality.

For every $y \in Y$, define

$$C_y = \{x \in X : f(x,y) \le \alpha\}.$$

Each C_y is closed and convex.

We claim that the family $\{C_y : y \in Y\}$ has the finite intersection property. Suppose instead that for some $y_1, \dots, y_m \in Y$,

$$C_{y_1} \cap \cdots \cap C_{y_m} = \emptyset.$$

By Lemma 367, there would be some

$$\bar{y} \in \operatorname{conv}\{y_1, \dots, y_m\} \subseteq Y$$

such that

$$\min_{x} f(x,\bar{y}) > \alpha.$$

This contradicts the definition

$$\alpha = \sup_{y}\min_{x} f(x,y).$$

Thus every finite subfamily has nonempty intersection. Since X is compact,

$$\bigcap_{y\in Y} C_y \neq \emptyset.$$

Choose x^* in this intersection. Then

$$f(x^*,y) \le \alpha \qquad \text{for every } y \in Y,$$

so

$$\sup_{y} f(x^*,y) \le \alpha.$$

Consequently,

$$\min_{x}\sup_{y} f(x,y) \le \alpha.$$

Together with weak minimax,

$$\sup_{y}\min_{x} f(x,y) \le \min_{x}\sup_{y} f(x,y) \le \alpha = \sup_{y}\min_{x} f(x,y),$$

and therefore equality holds. □

9.2 Linear Programming Duality

Sion's theorem makes LP duality into a direct calculation.

Consider the standard-form primal LP

$$\boxed{\begin{array}{ll} \text{minimize} & c^T x \\ \text{subject to} & Ax = b, \\ & x \geq 0. \end{array}}$$

Its dual is

$$\boxed{\begin{array}{ll} \text{maximize} & b^T y \\ \text{subject to} & A^T y \leq c. \end{array}}$$

The variable y is unrestricted.

Theorem 369 (LP Strong Duality)**.** *If the primal LP is feasible and has a finite optimum, then the dual is feasible, attains its optimum, and*

$$\min_{\substack{Ax=b \\ x\geq 0}} c^T x = \max_{A^T y \leq c} b^T y.$$

Proof. Let x^* be an optimal primal solution, and choose M so large that

$$0 \leq x_i^* < M \qquad \text{for every } i.$$

Set

$$X = [0, M]^n.$$

Define the Lagrangian

$$L(x, y) = c^T x + y^T (b - Ax).$$

For fixed x,

$$\sup_{y \in \mathbb{R}^m} L(x, y) = \begin{cases} c^T x, & Ax = b, \\ +\infty, & Ax \neq b. \end{cases}$$

Therefore

$$\min_{x \in X} \sup_y L(x, y) = c^T x^*.$$

The set X is compact and convex, and L is linear in each variable, so Sion's theorem gives

$$c^T x^* = \sup_y \min_{x \in X} L(x, y).$$

For fixed y, write

$$r = c - A^T y.$$

Then

$$L(x, y) = b^T y + r^T x,$$

and hence

$$q(y) := \min_{x \in X} L(x, y) = b^T y + M \sum_{i=1}^{n} \min(0, r_i).$$

This is a concave polyhedral function of y. Since its supremum is finite, it attains that supremum; let y^* be a maximizer. Thus

$$q(y^*) = c^T x^*.$$

Because x^* is primal feasible,

$$L(x^*, y^*) = c^T x^*.$$

But

$$q(y^*) = \min_{x \in X} L(x, y^*) = c^T x^*,$$

so x^* minimizes $L(\cdot, y^*)$ over the box X.

Now every coordinate satisfies

$$x_i^* < M.$$

If some

$$r_i = c_i - (A^T y^*)_i$$

were negative, increasing x_i^* slightly would remain inside X and would strictly decrease $L(x, y^*)$, contradicting minimality. Therefore

$$r_i \geq 0 \qquad \text{for every } i,$$

or equivalently,

$$A^T y^* \leq c.$$

Thus y^* is dual feasible.

Since $r \geq 0$,

$$q(y^*) = \min_{0 \leq x \leq M} \left(b^T y^* + r^T x \right) = b^T y^*.$$

Consequently,

$$b^T y^* = q(y^*) = c^T x^*,$$

which proves strong duality. □

Weak duality is even simpler: if x is primal feasible and y is dual feasible, then

$$b^T y = x^T A^T y \leq x^T c = c^T x.$$

The signs in the other standard forms can be recovered without memorization.

Free primal variables. If x_j is unrestricted, write

$$x_j = x_j^+ - x_j^-, \qquad x_j^+, x_j^- \geq 0.$$

The two corresponding dual inequalities are

$$(A^T y)_j \leq c_j \qquad \text{and} \qquad -(A^T y)_j \leq -c_j,$$

so together they give

$$(A^T y)_j = c_j.$$

Primal inequality constraints. For a minimization problem,

$$Ax \geq b$$

can be written

$$Ax - s = b, \qquad s \geq 0.$$

The dual constraint corresponding to s is

$$-y \leq 0,$$

hence

$$y \geq 0.$$

Similarly,

$$Ax \le b$$

gives

$$y \le 0.$$

Thus the standard correspondences for a minimization primal are

Primal	Dual
$x_j \ge 0$	$(A^T y)_j \le c_j$
x_j free	$(A^T y)_j = c_j$
$(Ax)_i = b_i$	y_i free
$(Ax)_i \ge b_i$	$y_i \ge 0$
$(Ax)_i \le b_i$	$y_i \le 0.$

Negating the objective gives the familiar maximization form:

$$\boxed{\begin{array}{ll}\text{maximize} & c^T x \\ \text{subject to} & Ax \le b, \\ & x \ge 0\end{array}} \quad \Longleftrightarrow \quad \boxed{\begin{array}{ll}\text{minimize} & b^T y \\ \text{subject to} & A^T y \ge c, \\ & y \ge 0.\end{array}}$$

9.3 Lagrangian Relaxation

Let $F(k)$ be the minimum cost of a feasible object using exactly k units of a resource. Instead of enforcing the count, assign each unit a price λ and compute $G(\lambda) = \min_k(F(k) + \lambda k)$. This can remove a cardinality dimension from an algorithm. Discrete convexity of F is what makes the constrained value recoverable from the penalized values.

Theorem 370 (Discrete Lagrangian duality)**.** *Let $F : I \to \mathbb{R}$, where I is a finite interval of integers, and put $G(\lambda) = \min_{k \in I}(F(k) + \lambda k)$. For every $K \in I$ and $\lambda \in \mathbb{R}$,*

$$G(\lambda) - \lambda K \le F(K).$$

Equality can be attained for every $K \in I$ if and only if F is discretely convex: its differences $\Delta_k = F(k) - F(k-1)$ are nondecreasing.

In that case,

$$\boxed{F(K) = \max_{\lambda \in \mathbb{R}} \big(G(\lambda) - \lambda K\big),}$$

and a price is optimal exactly when

$$\boxed{-\Delta_{K+1} \le \lambda \le -\Delta_K,}$$

with inequalities involving missing endpoint differences omitted.

Proof. The lower bound follows by using $k = K$ in the minimum defining G. Equality holds precisely when K minimizes $F(k) + \lambda k$.

Suppose F is discretely convex. For $j > K$,

$$F(j) + \lambda j - F(K) - \lambda K = \sum_{i=K+1}^{j} (\Delta_i + \lambda) \ge 0$$

when $\lambda \ge -\Delta_{K+1}$. For $j < K$, the same difference is $-\sum_{i=j+1}^{K}(\Delta_i + \lambda) \ge 0$ when $\lambda \le -\Delta_K$. Thus the stated prices are sufficient; comparison with the adjacent counts shows they are necessary.

Conversely, if $2F(K) > F(K-1) + F(K+1)$, then for every λ,

$$G(\lambda) - \lambda K \leq \frac{F(K-1) + F(K+1)}{2} < F(K).$$

So exactness at every count requires discrete convexity. □

The optimal price is the negative slope of a supporting line to F at K. At a fixed price, the minimizing counts form an integer interval: the discretely convex sequence $F(k) + \lambda k$ cannot rise between two minima. Thus a solver may skip K when breaking ties while still returning the value needed to recover $F(K)$. Recovering a witness of count exactly K is a separate task.

Corollary 371 (Integer Lagrangian search)**.** *Suppose F is integer-valued and discretely convex, with $|F(k)| \leq B$, where $B \geq 0$ is an integer. If $G(\lambda)$ can be evaluated in time T, then $F(K)$ can be computed in*

$$\boxed{O(T \log(B+2))}$$

time using only integer prices.

Proof. Every marginal is an integer in $[-2B, 2B]$. The supporting interval in Theorem 370 therefore contains an integer price in $[-2B-1, 2B+1]$.

The function $D(\lambda) = G(\lambda) - \lambda K$ is concave, since G is a minimum of affine functions. Its integer first differences are consequently nonincreasing.

Initialize $\ell = -2B - 1$, $r = 2B + 1$. While $\ell < r$, set $m = \lfloor (\ell + r)/2 \rfloor$. If $D(m) < D(m+1)$, replace ℓ by $m+1$; otherwise replace r by m. The first case discards no maximizer, and the second retains a leftmost maximizer. Return $D(\ell)$ when the interval becomes a singleton. □

Certifying convexity by minimum-cost flow.

Theorem 372 (Convexity of minimum-cost flow)**.** *Fix a directed network with distinguished vertices s, t, finite integral lower and upper capacities, and real edge costs. Let $C(k)$ be the minimum cost of an integral s-t flow of value k.*

The feasible integer values form an interval, and C is discretely convex on that interval:

$$\boxed{2C(k) \leq C(k-1) + C(k+1).}$$

Consequently, if another problem has exact-count value $F(k) = C(k) + \alpha k + \beta$, then F is discretely convex.

Proof. A convex combination of flows of values $a < b$ gives a fractional flow of every intermediate value. At an integer value, flow integrality gives an integral feasible flow. Thus the feasible integer values form an interval.

Now average optimal flows of values $k-1$ and $k+1$. The average is a feasible fractional flow of value k, with cost $(C(k-1) + C(k+1))/2$. By integrality of the fixed-value flow polytope, an integral optimum costs no more. This proves the inequality.

Adding an affine function does not change second differences. □

A reduction to minimum-cost k-flow is therefore useful even when flow is not used to solve the problem: it certifies the convexity needed by the faster penalized dynamic program. The reduction must preserve the optimum for every relevant k using one fixed network.

Several constraints by repeated one-dimensional relaxation. For several integer resources, the same search can be nested. The required condition is that convexity survives each preceding relaxation, not merely that the original value function is convex along its coordinate lines.

Theorem 373 (Repeated Lagrangian relaxation)**.** *Let $\mathcal{X}$ be finite, with integer costs $c(X)$, integer resource counts $k_1(X), \dots, k_d(X)$, and a feasible target $K = (K_1, \dots, K_d)$.*

For each j and each choice of integer prices $\lambda_1, \dots, \lambda_{j-1}$, define the partially relaxed one-dimensional value function

$$H_j(t; \lambda_{<j}) = \min_{\substack{X \in \mathcal{X} \\ k_j(X) = t \\ k_i(X) = K_i \ \ (i>j)}} \left(c(X) + \sum_{i<j} \lambda_i (k_i(X) - K_i) \right).$$

Assume its attainable values of t form an integer interval and $H_j(\,\cdot\,; \lambda_{<j})$ is discretely convex on that interval. Then

$$\boxed{\min_{\substack{X \in \mathcal{X} \\ k(X) = K}} c(X) = \max_{\lambda_1 \in \mathbb{Z}} \cdots \max_{\lambda_d \in \mathbb{Z}} \min_{X \in \mathcal{X}} \left(c(X) + \sum_{i=1}^{d} \lambda_i (k_i(X) - K_i) \right).}$$

Suppose additionally that $|c(X)| \le B$ and $|k_i(X) - K_i| \le R$, where $B \ge 0$, $R \ge 1$ are integers. At search depth j, it suffices to use

$$L_j = (2B+1)(2R+1)^{j-1}, \qquad -L_j \le \lambda_j \le L_j.$$

If the fully penalized minimum takes time T, nested searches compute the constrained optimum in

$$\boxed{O_d \left(T \prod_{j=1}^{d} \log(L_j + 2) \right)}$$

time.

Proof. Let

$$V_j(\lambda_1, \dots, \lambda_j) = \min_{\substack{X \in \mathcal{X} \\ k_i(X) = K_i \ \ (i>j)}} \left(c(X) + \sum_{i=1}^{j} \lambda_i (k_i(X) - K_i) \right).$$

Thus V_0 is the requested optimum and V_d is the fully penalized problem.

Fix the first $j-1$ prices. By definition,

$$V_{j-1} = H_j(K_j; \lambda_{<j}), \qquad V_j = \min_t \big(H_j(t; \lambda_{<j}) + \lambda_j (t - K_j) \big).$$

The profile is integer-valued and discretely convex, so Theorem 370 gives

$$\boxed{V_{j-1}(\lambda_{<j}) = \max_{\lambda_j \in \mathbb{Z}} V_j(\lambda_{<j}, \lambda_j).}$$

Applying these identities successively proves the nested formula.

For the search bounds, suppose the previous prices satisfy $|\lambda_i| \le L_i$. Every finite value of H_j then has absolute value at most

$$B_j = B + R \sum_{i<j} L_i.$$

The stated geometric choice satisfies $L_j = 2B_j + 1$. Hence Corollary 371 permits the j-th search inside $[-L_j, L_j]$.

To evaluate a search value V_j, recursively perform the remaining searches; at depth d, call the fully penalized solver. Each depth uses $O(\log(L_j + 2))$ evaluations of the next depth, giving the product bound. □

To verify the hypothesis, it is enough to reduce each profile $H_j(t; \lambda_{<j})$, with the earlier prices fixed, to minimum-cost t-flow up to an affine term in t. The network and its costs may depend on the fixed earlier prices, but must remain fixed as t varies.

9.4 Seidel's Algorithm

We consider low-dimensional linear programming. A linear program may have many constraints, but in fixed dimension only a few are needed to determine its optimum. Seidel's algorithm exploits this by inserting the constraints in random order. Most insertions leave the current answer unchanged. When a constraint does invalidate it, the new optimum lies on that constraint's boundary, reducing the dimension by one. The analysis runs this observation backward: an insertion can matter only if deleting that constraint changes the answer.

Consider minimizing $c^\top x$ subject to m inequalities $a_i^\top x \le b_i$ in $\mathbb{R}^d$. First impose a fixed box $[-R, R]^d$, where $R > 0$. Every nonempty intermediate feasible region is then compact. Among its objective minimizers, choose the lexicographically smallest coordinate vector. Equivalently, minimize

$$(c^\top x, x_1, \ldots, x_d) \quad \text{lexicographically.}$$

This gives a unique answer. We remove the artificial box at the end.

Lemma 374 (Boundary reduction)**.** *Let x be the canonical optimum before adding $a^\top z \le b$, and suppose $a^\top x > b$. If the new problem is feasible, its canonical optimum lies on $a^\top z = b$. Moreover, the new problem is infeasible exactly when its restriction to that boundary is infeasible.*

Proof. Let y be the new optimum. It is feasible for the old problem and differs from x, so its objective-and-coordinate tuple is lexicographically larger. If $a^\top y < b$, moving a sufficiently small positive distance from y toward x preserves every constraint, including the new one, and strictly decreases that tuple. This contradicts optimality.

If any new feasible point exists, the segment from it to x crosses the boundary while remaining feasible for all old constraints. Thus a feasible boundary point exists exactly when a new feasible point exists. □

The recursive problem therefore uses the old constraints restricted to one hyperplane.

Eliminate one coordinate. Suppose $a \neq 0$, and choose the *last* index j with $a_j \neq 0$. On the new boundary,

$$x_j = \frac{b - \sum_{\ell \neq j} a_\ell x_\ell}{a_j}.$$

Substituting in an inequality $u^\top x \le v$ replaces it by

$$\widetilde{u}_\ell = u_\ell - \frac{u_j}{a_j} a_\ell \quad (\ell \neq j), \qquad \widetilde{v} = v - \frac{u_j}{a_j} b. \tag{93}$$

The reduced objective has coefficients $\widetilde{c}_\ell = c_\ell - c_j a_\ell / a_j$; its omitted constant does not affect optimization. The remaining coordinates retain their box bounds. The two eliminated bounds $x_j \le R$ and $-x_j \le R$ must also be substituted and included as inequalities.

Choosing the last nonzero coordinate preserves the coordinate tie rule. The eliminated x_j depends only on earlier coordinates, since $a_\ell = 0$ for $\ell > j$. If two reduced points first differ before j, that is also their first difference after lifting. If they first differ after j, all earlier coordinates, including the determined x_j, agree. Thus lexicographic order on the remaining coordinates is exactly the order inherited from the original space.

Algorithm 30 Restriction and randomized incremental linear programming

```
function RESTRICT((u, v), (a, b), j)
    λ ← u_j / a_j
    return (u − λa with coordinate j deleted,  v − λb)

function SEIDEL(c, H, R)
    d ← dim c
    if d = 0 then
        return the empty vector if every (u, v) ∈ H has v ≥ 0; otherwise return Infeasible
    x_ℓ ← −R if c_ℓ ≥ 0, and x_ℓ ← R otherwise, for ℓ = 1, ..., d
    Randomly permute H as ((a_1, b_1), ..., (a_m, b_m))
    for i = 1, ..., m do
        if a_i^⊤ x > b_i then
            if a_i = 0 then
                return Infeasible
            j ← max{ℓ : (a_i)_ℓ ≠ 0}
            H' ← {RESTRICT((a_t, b_t), (a_i, b_i), j) : 1 ≤ t < i}
            Append RESTRICT((e_j, R), (a_i, b_i), j) and RESTRICT((−e_j, R), (a_i, b_i), j) to H'
            c' ← c − (c_j/(a_i)_j)a_i, with coordinate j deleted
            y ← SEIDEL(c', H', R)
            if y = Infeasible then
                return Infeasible
            Insert coordinate x_j = (b_i − Σ_{ℓ≠j} (a_i)_ℓ y_ℓ)/(a_i)_j into y to obtain x
    return x
```

In the lifting line, coordinates of y are indexed by the surviving indices $\ell \neq j$. Every recursive call uses a fresh uniform permutation, including the two substituted box inequalities. Initialization is the canonical optimum of the box alone: a zero objective coefficient chooses $-R$ by the tie rule.

Theorem 375 (Seidel's algorithm)**.** *For fixed d, Algorithm 30 solves the boxed linear program, or reports infeasibility, in expected $O_d(m+1)$ arithmetic operations. It is correct for every sequence of random choices. The same bound holds for an unrestricted linear program using the symbolic-box implementation below.*

Why only a few constraints can matter. The following consequence of Helly's theorem replaces a general-position assumption that exactly d constraints meet at the optimum.

Lemma 376 (Small certificates)**.** *In dimension q, with a fixed compact box, a feasible system has at most q inequalities that already determine its canonical optimum. An infeasible system has at most $q+1$ inequalities that are infeasible together with the box.*

Proof. Let x be the canonical optimum, and let U_x consist of points whose objective-and-coordinate tuple is lexicographically smaller than that of x. This set is convex: a positive convex combination of two lexicographically negative difference vectors is still lexicographically negative. The box, all input halfspaces, and U_x have empty intersection. Helly gives an empty subfamily of at most $q+1$ sets. That subfamily must include U_x, since x belongs to every other set. Thus at most q input halfspaces, together with the box, already exclude every better point. They contain x, so they determine exactly its canonical optimum.

For an infeasible system, apply Helly directly to the box and input halfspaces. At most $q+1$ input halfspaces suffice, whether or not the box occurs in the empty subfamily. □

These certificates are used only for analysis; the algorithm does not compute them.

Proof of Theorem 375 for a fixed box. Correctness follows by induction on dimension and on the processed prefix. A satisfied inequality preserves the old optimum. A violated nonconstant inequality reduces to its

boundary by Lemma 374, and (93) describes that problem exactly. Its canonical solution lifts to the new canonical optimum. A violated constant inequality, or an infeasible recursive problem, certifies infeasibility.

For the expected work, condition on the first i inequalities as an *unordered* set. The last of them is uniform among these i choices. If the prefix is feasible, fix a certificate of at most q inequalities. Removing an inequality outside it preserves the optimum, so that inequality cannot trigger recursion when inserted last. If the prefix is infeasible, fix an infeasibility certificate of at most $q+1$ inequalities. Removing anything outside it leaves an infeasible prefix, in which case the algorithm would have stopped before reaching this insertion. Therefore

$$\boxed{\Pr[\text{insertion } i \text{ triggers recursion}] \le \min\{1, (q+1)/i\}.}$$

For a feasible prefix the sharper bound is q/i.

Let $T_q(m)$ bound expected work in dimension q. Scanning and shuffling cost $O(q(m+1))$. At insertion i, restriction costs $O(q(i+1))$ and creates a $(q-1)$-dimensional problem with $i+1$ inequalities. Summing over insertions,

$$T_q(m) \le O(q^2(m+1)) + \sum_{i=1}^{m} \frac{q+1}{i} T_{q-1}(i+1), \qquad T_0(m) = O(m+1).$$

If $T_{q-1}(s) \le K_{q-1}(s+1)$, then $\sum_{i=1}^{m}(i+2)/i \le 3m$ gives $T_q(m) \le K_q(m+1)$ with $K_q = 3(q+1)K_{q-1} + O(q^2)$. This proves linear dependence on m for fixed dimension; for example, it gives the explicit bound $O(3^d(d+1)!(m+1))$. □

Remove the box. Use a formal parameter $R = \Lambda$ and interpret comparisons as holding for all sufficiently large positive Λ. Every right-hand side and every point coordinate remains affine in Λ: normal vectors and objective coefficients never depend on it, so restriction only multiplies affine expressions by ordinary scalars. Store an expression $u + \Lambda v$ as the pair (u, v) and compare two expressions by their v coefficients first, then their u coefficients. Thus every symbolic operation has constant arithmetic overhead.

An Infeasible result means the boxed system is infeasible for all sufficiently large boxes, hence the original system is infeasible. Otherwise write the returned point as $x(\Lambda) = u + \Lambda v$. It is feasible and optimal in the box for all sufficiently large Λ. Consequently $a_i^\top v \le 0$ for every original inequality, and if $a_i^\top v = 0$, then $a_i^\top u \le b_i$. Choose

$$\Lambda_0 = \max\left(\{1\} \cup \left\{ \frac{a_i^\top u - b_i}{-a_i^\top v} : a_i^\top v < 0 \right\}\right).$$

Then $x_0 = u + \Lambda_0 v$ satisfies every original inequality. The objective slope determines the outcome:

$$\begin{cases} c^\top v < 0: & x_0 + tv \ (t \ge 0) \text{ certifies unboundedness below,} \\ c^\top v = 0: & x_0 \text{ is an optimum of the original problem.} \end{cases}$$

Indeed, every fixed feasible point lies in the box for all sufficiently large Λ, so its objective is at least $c^\top u + \Lambda c^\top v$. This rules out a positive slope; for zero slope it proves global optimality, and for negative slope the displayed ray is feasible and improving.

For fixed input there are only finitely many possible recursive executions, and every comparison in them is eventually constant as Λ grows.

9.5 Majorization

Theorem 377 (Hardy-Littlewood-Pólya)**.** *Let $x, y \in \mathbb{R}^n$. If $x \prec_w y$, then there exists a doubly stochastic matrix M such that $x \le My$. If $x \prec y$ then $x = My$.*

Proof. Let $\mathcal{M}$ be the compact, convex set of $n \times n$ doubly stochastic matrices, and let $\mathcal{V} = [0,1]^n$, which is also compact and convex. Consider the bilinear function $f(M, v) = v^T(My - x)$.

For any fixed $v \in \mathcal{V}$, let $P \in \mathcal{M}$ be a permutation matrix that orders the components of y comonotonically with v. Because $x \prec y$, an application of summation by parts yields $v^T x \le v^T P y$. Consequently, we can always choose $M = P$ to see that $\max_{M \in \mathcal{M}} f(M, v) \ge f(P, v) \ge 0$.

By Sion's Minimax Theorem:

$$\max_{M\in\mathcal{M}}\min_{v\in\mathcal{V}} v^T(My-x) = \min_{v\in\mathcal{V}}\max_{M\in\mathcal{M}} v^T(My-x) \geq 0$$

This guarantees the existence of some $M_0 \in \mathcal{M}$ such that $v^T(M_0y - x) \geq 0$ for all possible vectors $v \in \mathcal{V}$.

Applying this inequality with the standard basis vectors $v = e_i \in \mathcal{V}$ implies that $M_0y \geq x$ component-wise. However, because M_0 is doubly stochastic, the sum of the components of M_0y exactly equals the sum of the components of y. By the definition of majorization, $\sum y_i = \sum x_i$ if $x \prec y$, thus $M_0y = x$. □

9.5.1 Muirhead's Inequality

For

$$\alpha = (\alpha_1, \ldots, \alpha_n) \in \mathbb{R}^n_{\geq 0}$$

and

$$x = (x_1, \ldots, x_n) \in \mathbb{R}^n_{\geq 0},$$

define the symmetric monomial mean

$$\boxed{[\alpha](x) = \frac{1}{n!}\sum_{\pi\in S_n} x_1^{\alpha_{\pi(1)}} \cdots x_n^{\alpha_{\pi(n)}}.}$$

We average over all permutations, including repetitions when some α_i's are equal.

Muirhead's inequality says that making the exponent vector more unequal increases its symmetric monomial mean.

Theorem 378 (Muirhead's Inequality)**.** *Let*

$$\alpha, \beta \in \mathbb{R}^n_{\geq 0}$$

satisfy

$$\beta \prec \alpha.$$

Then for every

$$x_1, \ldots, x_n \geq 0,$$

$$\boxed{[\beta](x) \leq [\alpha](x).}$$

Proof. We first assume

$$x_i > 0 \qquad (i = 1, \ldots, n);$$

the general case follows by continuity.

By Hardy–Littlewood–Pólya,

$$\beta = M\alpha$$

for some doubly stochastic matrix M.

By the Birkhoff–von Neumann theorem, every doubly stochastic matrix is a convex combination of permutation matrices, so

$$M = \sum_{k=1}^{N} \theta_k P_k, \qquad \theta_k \geq 0, \qquad \sum_k \theta_k = 1.$$

Therefore

$$\beta = \sum_k \theta_k P_k \alpha.$$

Fix a permutation $Q \in S_n$. Then

$$Q\beta = \sum_k \theta_k Q P_k \alpha.$$

Writing

$$x^\gamma = \prod_{i=1}^{n} x_i^{\gamma_i},$$

we obtain

$$\begin{aligned} x^{Q\beta} &= x^{\sum_k \theta_k Q P_k \alpha} \\ &= \prod_k \left(x^{Q P_k \alpha}\right)^{\theta_k}. \end{aligned}$$

Weighted AM–GM gives

$$\boxed{x^{Q\beta} \le \sum_k \theta_k x^{Q P_k \alpha}.}$$

Now average this inequality over all $Q \in S_n$. The left-hand side gives

$$\frac{1}{n!} \sum_Q x^{Q\beta} = [\beta](x).$$

For each fixed P_k, as Q ranges over S_n, so does QP_k. Hence

$$\frac{1}{n!} \sum_Q x^{Q P_k \alpha} = [\alpha](x).$$

Therefore

$$\begin{aligned} [\beta](x) &\le \sum_k \theta_k \left(\frac{1}{n!} \sum_Q x^{Q P_k \alpha} \right) \\ &= \sum_k \theta_k [\alpha](x) \\ &= [\alpha](x). \end{aligned}$$

This proves Muirhead for positive x_i, and continuity gives the result when some $x_i = 0$. □

9.6 SMAWK and Totally Monotone Matrices

We want the leftmost minimum of every row of an implicit $m \times n$ matrix A, where any entry can be evaluated in constant time. Scanning every row costs $O(mn)$. If optimal columns move only to the right as rows increase, solved rows constrain the search in nearby rows. SMAWK adds a second idea: first discard columns until only as many as there are rows remain. Alternating column reduction with recursion on half the rows gives $O(m + n)$ time.

Total monotonicity. Call A *totally monotone for leftmost row minima* if

$$\boxed{A_{i,a} > A_{i,b} \implies A_{j,a} > A_{j,b} \qquad (i < j,\ a < b).} \tag{94}$$

Thus once a later column becomes strictly better than an earlier one, it stays strictly better in every lower row. The contrapositive says that if the earlier column is at least as good in a lower row, it is at least as good in every upper row.

Lemma 379. *Condition (94) holds exactly when, in every submatrix obtained by selecting rows and columns in their original orders, the leftmost row-minimum columns are nondecreasing.*

Proof. Suppose two rows of such a submatrix had leftmost minimum columns b and a, respectively, with $a < b$. In the upper row i, leftmostness gives $A_{i,a} > A_{i,b}$. Total monotonicity then gives $A_{j,a} > A_{j,b}$ in the lower row, contradicting its minimum at a. Conversely, a failure of (94) gives a 2×2 submatrix whose leftmost minima are in the upper-right and lower-left corners, violating the stated monotonicity. □

Total monotonicity, not merely monotonicity of the original row minima, is needed because the algorithm repeatedly deletes columns.

Monge matrices. A real matrix is *Monge* if

$$A_{i,a} + A_{j,b} \leq A_{i,b} + A_{j,a} \qquad (i < j,\ a < b).$$

Rearranging gives $A_{i,a} - A_{i,b} \leq A_{j,a} - A_{j,b}$, which immediately implies (94).

Column reduction. Consider ordered row and column lists $R = (r_0, \ldots, r_{q-1})$ and $C = (c_0, \ldots, c_{s-1})$, with $q, s > 0$. Indices in these lists start at zero. Scan the columns from left to right and maintain a stack S of at most q candidate columns. Its key invariant is

the column in stack position h cannot be a leftmost minimum in rows $r_0, \ldots, r_{h-1}$.

The stack position counts how many upper rows have already ruled out that column.

If the stack top is in position h, compare it with the new column at row r_h. If the new column is strictly better, the old top loses in every lower row as well, and the invariant has already ruled it out above: discard it. Otherwise, the new column loses, including by ties, in every row through r_h. It can be pushed into position $h + 1$, or discarded if all rows have now been ruled out.

Algorithm 31 Remove columns

```
1:  function REDUCE(R, C)
2:      q ← |R|; S ← an empty stack
3:      for each c ∈ C, in increasing order do
4:          while S ≠ ∅ do
5:              h ← |S| − 1
6:              if A_{r_h,c} < A_{r_h,S[h]} then
7:                  Pop the last column from S
8:              else
9:                  break
10:         if |S| < q then
11:             Push c onto S
12:     return S
```

Lemma 380 (Column reduction)**.** *Algorithm 31 takes $O(|C|)$ time and returns at most $|R|$ columns containing the leftmost minimum of every row of $A[R, C]$.*

Proof. We verify the invariant and show that every discarded column is incapable of being a leftmost minimum anywhere in the original submatrix $A[R, C]$.

Suppose a column d is popped from position h because a later column c is strictly better in row r_h. By total monotonicity, c is strictly better in every row r_i with $i \geq h$. For $i < h$, the invariant already excludes d. Thus popping it is safe.

Suppose instead the new column c is pushed into position $h > 0$. The loop stopped because the preceding stack column d satisfies $A_{r_{h-1},d} \leq A_{r_{h-1},c}$, with $d < c$. The contrapositive of total monotonicity gives $A_{r_i,d} \leq A_{r_i,c}$ for all $i < h$. Because d is earlier, c cannot be the leftmost minimum in those rows, establishing

the invariant. At position zero the invariant is empty. If the stack is already full, the same comparison at its last row rules out c in all q rows, so omitting it is safe.

Every omitted column is therefore excluded from all row optima. Each input column is pushed at most once and popped at most once. Every successful while-loop comparison pops a column, and each input column causes at most one unsuccessful comparison. Hence the total number of comparisons and stack operations is $O(|C|)$. □

Solve half the rows. After reduction, call the remaining columns C'. Recursively solve rows $r_1, r_3, r_5, \ldots$ using only C'. Total monotonicity places the minimum of an unsolved row between the minima of its two solved neighbors. At the first or last row, the missing neighbor is replaced by the first or last column of C'.

Scan each such interval from left to right. The intervals for successive unsolved rows overlap only at their shared endpoint, so their total length, counting repetitions, is $O(|C'| + |R|)$. The crucial point is that reduction has made $|C'| \le |R|$: each recursive level now pays only for its shrinking number of rows.

Use a global answer array p, with $p[r]$ the original column index of row r's minimum, and one workspace array pos indexed by original column indices. The latter translates answers back into positions in the current reduced list.

Algorithm 32 SMAWK for all leftmost row minima

Input: Totally monotone matrix A accessed by entry evaluations; answer array p and workspace pos allocated once

```
1:  function SMAWK(R, C)
2:      if R = ∅ then
3:          return
4:      C ← REDUCE(R, C); q ← |R|; s ← |C|
5:      if q = 1 then
6:          p[r_0] ← c_0; return
7:      SMAWK((r_1, r_3, r_5, ...), C)
8:      for j = 0, ..., s − 1 do
9:          pos[c_j] ← j
10:     for i = 0, 2, 4, ... < q do
11:         L ← 0 if i = 0, otherwise pos[p[r_{i−1}]]
12:         U ← s − 1 if i + 1 = q, otherwise pos[p[r_{i+1}]]
13:         p[r_i] ← c_L
14:         for j = L + 1, ..., U do
15:             if A_{r_i,c_j} < A_{r_i,p[r_i]} then
16:                 p[r_i] ← c_j
```

Call the routine with $R = (0, \ldots, m-1)$ and $C = (0, \ldots, n-1)$, where $n \ge 1$.

Theorem 381 (SMAWK)**.** *For a totally monotone $m \times n$ matrix with $n \ge 1$, Algorithm 32 finds every leftmost row minimum using $O(m+n)$ entry evaluations, $O(m+n)$ additional operations, and $O(m+n)$ space. In particular, constant-time entry access gives linear time.*

Proof. Induct on the number of rows. Reduction preserves every leftmost minimum. With one row, it leaves exactly its minimum column. Otherwise, the recursive call is on a submatrix and hence is still totally monotone. Induction gives the correct minima for its rows. By Lemma 379, each remaining minimum is inside its scanned interval. Scanning left to right and changing the answer only on strict improvement finds its leftmost occurrence.

For the cost, write $T(q, s)$ for one call. Reduction takes $O(s)$, and interpolation plus the position map takes $O(q+s)$, so

$$T(q, s) \le T(\lfloor q/2 \rfloor, \min\{q, s\}) + O(q+s).$$

The first call costs $O(m+n)$. At depth $h \geq 1$, there are at most $m/2^h$ rows and $m/2^{h-1}$ columns. Summing this geometric sequence gives total time $O(m+n)$. The stored row and column lists satisfy the same bounds, and the answer and position arrays use $O(m+n)$ space, proving the space claim. □

9.7 Concave Min-Plus Convolution

9.7.1 Description

Given an array a of length n (which is concave, meaning its forward differences are monotonically decreasing) and an arbitrary array b of length m, the concave min-plus convolution computes:

$$c[p] = \min_{0 \leq x < n,\, 0 \leq y < m,\, x+y=p} (a[x] + b[y])$$

We give an $O(m \log n + n)$ solution by chunking the array b and using a modified Li Chao Tree.

The algorithm processes b in chunks of size up to n. For a chunk starting at index s, we compute its contributions to a window of length $2n$ in the output array c using two passes:

- **Forward Pass:** Evaluates pairs where the relative index in a is greater than or equal to the relative index in b. It inserts functions $f_k(i) = a[i-k] + b[s+k]$ into a Li Chao tree and queries them for $i \geq k$.
- **Reverse Pass:** Symmetrically handles the remaining cross-terms where the relative index in a is strictly less than the relative index in b, sweeping backwards.

9.7.2 Crossings

Lemma 382 (Single Crossing Property)**.** *For a concave array a, define the extended functions for $k \in [0, n)$:*

$$f_k(i) = \begin{cases} -\infty - k & \text{if } i < k \\ a[i-k] + b[s+k] & \text{if } i \geq k \end{cases}$$

For any $j < k$, the difference $D(i) = f_k(i) - f_j(i)$ transitions from negative to positive at most once.

Proof. We evaluate the difference $D(i)$ across three domains:

1. **For $i < j$:** Both are artificial. $D(i) = (-\infty - k) - (-\infty - j) = j - k < 0$.
2. **For $j \leq i < k$:** f_j is finite, but f_k is artificial. $D(i) = -\infty < 0$.
3. **For $i \geq k$:** Both are finite. $D(i) = a[i-k] - a[i-j] + C$. Because a is concave, its slope is decreasing. Since $i - k < i - j$, the difference $a[i-k] - a[i-j]$ strictly monotonically increases with i.

Because $D(i)$ is strictly negative for all $i < k$ and monotonically increases for all $i \geq k$, the functions f_j and f_k intersect at most once globally. □

9.7.3 Boundary Conditions

While Lemma 382 proves the functions cross at most once, their shapes are highly irregular due to the jump at $i = k$.

A standard Li Chao Tree assumes that if a function is better at the left endpoint L and worse at the midpoint M, the single continuous crossing must lie within $[L, M]$, and recursively routes the function leftward.

A function f_k might be artificially better at L (because it's about $-\infty$), worse at M, but then become the true minimum on the right side of the interval.

For a node spanning the discrete interval $[L, R)$, let z be the current resident function and x be the newly inserted function.

- If $x(L) \geq z(L)$ and $x(R-1) \geq z(R-1)$, then x is dominated at both strict boundaries. By Lemma 382, it must be dominated everywhere inside the interval. We safely discard x.
- If $x(L) < z(L)$ and $x(R-1) < z(R-1)$, then x dominates z everywhere in the interval. We replace z with x and discard z.
- Otherwise, a crossing exists within the interval. We swap x and z if necessary to maintain the best function at the midpoint, and recursively push the loser to the appropriate child.

9.8 Rectangular Monotone Min-Plus Product

Let A be $n \times k$ and B be $k \times n$, with nondecreasing rows in B. We want $C_{ij} = \min_t(A_{it} + B_{tj})$. Encoding a weight a as X^a lets ordinary matrix multiplication count candidate sums, but large weights make the polynomials expensive. We will reduce the exponents modulo a small random prime. The product then counts both correct sums and false matches due to the reduction; monotonicity lets us find and subtract the false matches in entire intervals.

Theorem 383 (Row-monotone rectangular min-plus product)**.** *Suppose $B \in \{0, \ldots, U\}^{k \times n}$ has nondecreasing rows, where $U \geq 1$, and A has integer entries or $+\infty$. Write $\mathsf{M} = \mathsf{MM}(n, k, n)$ for an available ordinary matrix multiplication bound, including input/output work. Then $A \star B$ can be computed exactly in expected time*

$$\boxed{\widetilde{O}\left(n^2 + nk + \sqrt{nkU\,\mathsf{M}}\right).}$$

The random choice affects running time, not correctness. The tilde suppresses logarithms in the dimensions and integer magnitudes; word-operation bounds assume polynomially bounded input integers.

Normalize. Subtract the smallest finite entry from each row of A and save it for restoration at the end. The normalized answer is in $[0, U]$, since each row now has a zero entry. Entries of A greater than U cannot improve that answer, so replace them by $+\infty$. Entirely infinite rows are answered separately. Henceforth all finite entries and all answers lie in $[0, U]$, and every row of A has a zero.

Rounding. For $\ell \geq 0$, round the entries:

$$A^{(\ell)} = \left\lfloor A/2^\ell \right\rfloor, \qquad B^{(\ell)} = \left\lfloor B/2^\ell \right\rfloor, \qquad C^{(\ell)} = A^{(\ell)} \star B^{(\ell)}.$$

Infinite entries stay infinite. Every $C^{(\ell)}$ has nondecreasing rows and lies in $[0, \lfloor U/2^\ell \rfloor]$. For $h = \lceil \log_2(U+1) \rceil$, all finite rounded inputs are zero, so $C^{(h)} = 0$. Starting there, recover $C^{(0)}$ one level at a time.

Each finite rounded entry satisfies $A^{(\ell)}_{it} = 2A^{(\ell+1)}_{it} + \alpha_{it}$ with $\alpha_{it} \in \{0, 1\}$, and similarly for B. Every candidate sum therefore doubles and increases by zero, one, or two. Taking minima gives

$$\boxed{2C^{(\ell+1)}_{ij} \leq C^{(\ell)}_{ij} \leq 2C^{(\ell+1)}_{ij} + 2.}$$

The lower bound holds for every candidate; the upper bound uses a minimizer at the preceding level. Thus a refinement needs to determine which of three consecutive integers is the smallest attainable sum.

Count Modulo a Prime, Then Subtract False Matches. Fix a prime p. During one refinement abbreviate $a = A^{(\ell)}$, $b = B^{(\ell)}$, and $D = C^{(\ell+1)}$. Form polynomial matrices

$$\mathcal{A}_{it} = X^{a_{it} \bmod p}, \qquad \mathcal{B}_{tj} = X^{b_{tj} \bmod p}, \qquad \mathcal{H} = \mathcal{A}\mathcal{B} \pmod{X^p - 1},$$

using zero at infinite entries. Reduction modulo $X^p - 1$ folds exponents with the same residue together. Consequently

$$[X^{r \bmod p}]\mathcal{H}_{ij} = \#\{t : a_{it} + b_{tj} \equiv r \pmod p\}.$$

Compute the ordinary polynomial product first and then fold its coefficients. Its degree is at most $2p-2$, so this costs $\widetilde{O}(p\mathsf{M})$. The prime p reduces weights, not coefficient counts.

For $\delta \in \{0,1,2\}$, let $r_{ij}^{\delta} = 2D_{ij} + \delta$. The coefficient at this residue counts the desired exact matches plus indices whose discrepancy is a *nonzero* multiple of p:

$$E_{ij}^{\delta} = \#\{t : a_{it} + b_{tj} - r_{ij}^{\delta} \in p\mathbb{Z} \setminus \{0\}\}.$$

After subtracting them, put $N_{ij}^{\delta} = [X^{r_{ij}^{\delta} \bmod p}]\mathcal{H}_{ij} - E_{ij}^{\delta}$. The refinement is simply

$$\boxed{C_{ij}^{(\ell)} = 2D_{ij} + \min\{\delta \in \{0,1,2\} : N_{ij}^{\delta} \neq 0\}.} \tag{95}$$

At least one of the three counts is nonzero by the rounding bound. The only remaining problem is to subtract false matches without examining all n^2k triples.

False Match Search. For fixed i, t, δ, the discrepancy is $a_{it} + b_{tj} - (2D_{ij} + \delta)$. It is a difference of two nondecreasing sequences. The difference itself need not be monotone, but the endpoints of a column interval bound every value inside it.

More generally, take an integer a and nondecreasing integer sequences $u_1, \ldots, u_N$ and $v_1, \ldots, v_N$. On $[L, R]$, every $a + u_j - v_j$ belongs to

$$[\, a + u_L - v_R,\ a + u_R - v_L \,].$$

If this interval contains no nonzero multiple of p, discard it. If both sequences are constant on the interval, report the entire interval or nothing. Otherwise bisect and repeat.

Algorithm 33 Report intervals of false matches

```
function REPORT(a, u, v, p, L, R)
    q_- ← ⌈(a + u_L − v_R)/p⌉; q_+ ← ⌊(a + u_R − v_L)/p⌋
    if q_- > q_+ or q_- = q_+ = 0 then
        return
    if u_L = u_R and v_L = v_R then
        Emit [L, R]; return
    M ← ⌊(L + R)/2⌋
    REPORT(a, u, v, p, L, M)
    REPORT(a, u, v, p, M + 1, R)
```

Here $1 \le L \le R \le N$. Monotonicity makes equal endpoint values certify constancy, so the emitted intervals are disjoint and cover exactly the desired indices.

Lemma 384 (Expected cost of endpoint pruning)**.** *Fix a, u, v independently of p, and write $\mathcal{V} = (u_N - u_1) + (v_N - v_1)$ for their total variation. For a uniformly chosen prime $p \in [P, 2P]$, with P above an absolute constant, Algorithm 33 runs in expected time*

$$\boxed{\widetilde{O}(1 + \mathcal{V}/P),}$$

including the time to emit its intervals.

Proof. Consider the full balanced interval tree, fixed before choosing p. A nonconstant node $I = [L, R]$ has bounding-interval width

$$w_I = (u_R - u_L) + (v_R - v_L) \ge 1.$$

It can recurse only if p divides some nonzero integer in that bounding interval. There are at most $w_I + 1 \le 2w_I$ such integers. If their magnitudes are bounded by $W \ge 2$, each has at most $\log W / \log P$ prime divisors in $[P, 2P]$. There are $\Omega(P/\log P)$ primes in this range, so a union bound gives

$$\Pr[I \text{ recurses}] = O(w_I \log W / P),$$

with the bound capped at one.

At a fixed depth the intervals are disjoint. By monotonicity, $\sum_I w_I \le \mathcal{V}$: their variations are disjoint portions of the total variation of the two sequences. Summing over the $O(\log(2N))$ depths bounds the expected number of nodes that recurse by $O(\mathcal{V} \log W \log(2N)/P)$. Every visited node except the root is a child of a node that recurses, so the number of calls is at most one plus twice this number. Constant intervals never recurse, even when they contain many matches. Each call and each emitted interval take constant time. □

Apply the reporter to an output row. For each finite (i, t) and each δ, call the reporter with

$$a = a_{it}, \qquad u_j = b_{tj}, \qquad v_j = 2D_{ij} + \delta.$$

A reported interval contributes one false match to every output column in it. Accumulate these contributions in three difference arrays per output row: an interval $[L, R]$ adds 1 at L and -1 at $R + 1$. Prefix sums then give all E^{δ}_{ij} in $O(n^2)$ additional work.

Algorithm 34 Row-monotone min-plus product

Input: Normalized A, B as above and a prime p

$h \leftarrow \lceil \log_2(U + 1) \rceil$; $D \leftarrow 0$
for $\ell = h - 1, h - 2, \ldots, 0$ **do**
  $a \leftarrow \lfloor A/2^{\ell} \rfloor$; $b \leftarrow \lfloor B/2^{\ell} \rfloor$
  Compute $\mathcal{H} = (X^{a \bmod p})(X^{b \bmod p}) \bmod (X^p - 1)$
  Initialize three false-count difference arrays per output row
  for each finite a_{it} and $\delta \in \{0, 1, 2\}$ **do**
    for each $[L, R]$ emitted by Report$(a_{it}, b_{t,*}, 2D_{i,*} + \delta, p, 1, n)$ **do**
      Add 1 at L and -1 at $R + 1$ in array (i, δ)
  Prefix-sum the arrays to obtain E^{δ}_{ij}
  Subtract from the three queried coefficients to obtain N^{δ}_{ij}
  $D'_{ij} \leftarrow 2D_{ij} + \min\{\delta : N^{\delta}_{ij} \ne 0\}$ for all i, j
  $D \leftarrow D'$
return D, restoring the saved row offsets

Correctness and the Running Time.

Proof of Theorem 383. Initially $D = C^{(h)} = 0$. If $D = C^{(\ell+1)}$, the polynomial product counts modular matches and the reporter subtracts exactly those that are not exact matches. The three-value rounding bound then proves (95). Induction gives $D = C^{(0)}$ at termination, for *any* prime p.

Choose one prime uniformly from $[P, 2P]$ and reuse it at every level. All rounded inputs and all exact products $C^{(\ell)}$ are determined by the original matrices, independently of p. Thus the reporting lemma applies directly.

At level ℓ, the reporter's two sequences have total variation at most $2U/2^{\ell}$. Summing its expected cost over i, t, δ gives $\widetilde{O}(nk + nkU/(2^{\ell}P))$. There are $O(\log(U + 1))$ levels, and $\sum_{\ell \ge 0} 2^{-\ell} \le 2$. Together with the polynomial products and output work, the total is

$$\widetilde{O}\left(n^2 + nk + \frac{nkU}{P} + P\mathsf{M}\right).$$

Choosing P within a constant factor of $\sqrt{nkU/\mathsf{M}}$ balances the last two terms. Primes up to $2P$ can be enumerated by a sieve in $\widetilde{O}(P)$ time and a prime in the interval sampled uniformly.

If the balanced scale is below the absolute constant required by the lemma, use the direct monotone sweep instead. Each row of B has at most $U+1$ constant intervals; for every i,t, each interval supplies one range-minimum update $C_{ij} \leftarrow \min\{C_{ij}, A_{it}+B_{tj}\}$. Store each update on the $O(\log n)$ canonical nodes of a segment tree, then propagate minimum tags to its leaves. This costs $\widetilde{O}(n^2+nk+nkU)$, which is within the claimed bound when $nkU/\mathsf{M} = O(1)$. This also covers small U. □

For $k = n^\beta$, $U = O(n^\mu)$, and an admissible exponent $\omega_\beta = \omega(1,\beta,1)$, the bound is $\widetilde{O}(n^2 + n^{1+\beta} + n^{(1+\beta+\mu+\omega_\beta)/2})$. One may always take the better of this bound and the direct $O(n^2k)$ algorithm.

9.8.1 Column-Monotone Right Matrices

If B is instead nondecreasing down its columns, replace each normalized row of A by prefix minima $\widehat{A}_{it} = \min_{s\le t} A_{is}$. This preserves the product: $\widehat{A} \le A$ gives one inequality, and a minimizing $s \le t$ gives

$$\widehat{A}_{it} + B_{tj} = A_{is} + B_{tj} \ge A_{is} + B_{sj} \ge C_{ij}.$$

Thus rows of $\widehat{A}$ are nonincreasing; an initial infinite prefix is skipped. Rounding preserves this order.

Use the same three-value refinement and polynomial products. For each output (i,j) and offset δ, now report *middle-index* intervals, taking

$$a_0 = -(2D_{ij}+\delta), \qquad u_t = b_{tj}, \qquad v_t = -a_{it}.$$

Both sequences are nondecreasing on the finite suffix of row i. A reported interval contributes its length to the single false count E^δ_{ij}, rather than a range update to an output row. Their total variation is again at most $2U/2^\ell$, but there are n^2 reporter calls per offset instead of nk. The same analysis, with the analogous direct sweep over middle-index constant intervals when the prime scale is small, gives

$$\boxed{\widetilde{O}\left(n^2 + nk + \sqrt{n^2 U\,\mathsf{M}}\right).}$$

Transposition gives the corresponding bounds for a monotone left matrix.

9.9 Radon's Theorem

Theorem 385 (Radon's Theorem)**.** *Any $d+2$ points*

$$x_1, \dots, x_{d+2} \in \mathbb{R}^d$$

can be partitioned into two nonempty sets whose convex hulls intersect.

Proof. The lifted vectors

$$(x_1,1), \dots, (x_{d+2},1) \in \mathbb{R}^{d+1}$$

are linearly dependent. Thus there are coefficients λ_i, not all zero, such that

$$\sum_i \lambda_i x_i = 0, \qquad \sum_i \lambda_i = 0.$$

Let

$$P = \{i : \lambda_i > 0\}, \qquad N = \{i : \lambda_i < 0\}.$$

Both are nonempty. Set

$$L = \sum_{i\in P} \lambda_i = -\sum_{i\in N} \lambda_i > 0.$$

Then

$$\sum_{i \in P} \frac{\lambda_i}{L} x_i = \sum_{i \in N} \frac{-\lambda_i}{L} x_i.$$

Each side is a convex combination, so this common point lies in both convex hulls. □

9.10 Helly's Theorem

Theorem 386 (Helly's Theorem)**.** *Let*

$$C_1, \dots, C_m \subseteq \mathbb{R}^d$$

be convex sets, with $m \geq d+1$. If every $d+1$ of them have nonempty intersection, then

$$\bigcap_{i=1}^{m} C_i \neq \emptyset.$$

Proof. We induct on m. The case $m = d+1$ is exactly the hypothesis.

Suppose $m > d+1$. By induction, deleting any one of the sets leaves a family with nonempty intersection. For

$$i = 1, \dots, d+2,$$

choose

$$x_i \in \bigcap_{j \neq i} C_j.$$

Apply Radon's theorem to

$$x_1, \dots, x_{d+2}.$$

There is a partition

$$P \sqcup N = [d+2]$$

and a point

$$x \in \operatorname{conv}\{x_i : i \in P\} \cap \operatorname{conv}\{x_i : i \in N\}.$$

We claim $x \in C_j$ for every j.

If $j \leq d+2$, one of P, N does not contain j. Every x_i in that part belongs to C_j, so by convexity its convex hull, and hence x, lies in C_j.

If $j > d+2$, then every x_i lies in C_j, so again $x \in C_j$.

Thus

$$x \in \bigcap_{j=1}^{m} C_j.$$

□

9.11 Carathéodory's Theorem

Theorem 387 (Carathéodory's Theorem)**.** *If*

$$x \in \operatorname{conv}(S) \subseteq \mathbb{R}^d,$$

then x lies in the convex hull of at most $d+1$ points of S.

Proof. Choose a representation

$$x = \sum_{i=1}^{m} \alpha_i x_i, \qquad \alpha_i > 0, \qquad \sum_i \alpha_i = 1,$$

using as few points as possible.

Suppose

$$m > d + 1.$$

Then the lifted vectors

$$(x_i, 1) \in \mathbb{R}^{d+1}$$

are linearly dependent, so there are λ_i, not all zero, with

$$\sum_i \lambda_i x_i = 0, \qquad \sum_i \lambda_i = 0.$$

Replace

$$\alpha_i$$

by

$$\alpha_i - t\lambda_i.$$

This does not change either

$$\sum_i \alpha_i \quad \text{or} \quad \sum_i \alpha_i x_i.$$

Choose

$$t = \min_{\lambda_i > 0} \frac{\alpha_i}{\lambda_i}.$$

Then all new coefficients remain nonnegative and at least one becomes zero. This gives a convex representation of x using fewer points, a contradiction.

Hence

$$m \leq d + 1.$$

□

9.12 Colorful Carathéodory

Theorem 388 (Colorful Carathéodory)**.** *Let*

$$C_1, \ldots, C_{d+1} \subseteq \mathbb{R}^d$$

satisfy

$$0 \in \operatorname{conv}(C_i) \qquad \textit{for every } i.$$

Then one can choose

$$c_i \in C_i$$

such that

$$0 \in \operatorname{conv}\{c_1, \ldots, c_{d+1}\}.$$

Proof. By Theorem 387, we may first replace each C_i by a finite subset whose convex hull still contains 0.

Among all colorful choices

$$c_i \in C_i,$$

choose one whose convex hull

$$K = \operatorname{conv}\{c_1, \ldots, c_{d+1}\}$$

has minimum distance from 0. Let $z \in K$ be the closest point.

Suppose

$$z \neq 0.$$

The closest-point property implies

$$z^T x \geq \|z\|^2 \qquad \text{for every } x \in K.$$

Indeed, if some $x \in K$ had

$$z^T x < \|z\|^2,$$

then moving slightly from z toward x would decrease the distance to 0.

The minimal face of K containing z therefore lies in the hyperplane

$$H = \{x : z^T x = \|z\|^2\},$$

which has dimension $d-1$. Hence z is a convex combination of at most d of the selected points. Thus some color, say C_j, is absent from this representation.

Since

$$0 \in \operatorname{conv}(C_j),$$

not every $y \in C_j$ can satisfy $z^T y > 0$. Choose

$$y \in C_j$$

with

$$z^T y \leq 0.$$

Replace the selected point of color j by y. The new colorful convex hull contains both z and y, and for small $\varepsilon > 0$,

$$\|(1-\varepsilon)z + \varepsilon y\|^2 = \|z\|^2 + 2\varepsilon z^T(y-z) + O(\varepsilon^2) < \|z\|^2.$$

This contradicts the minimal choice of K.

Therefore

$$z = 0.$$

□

9.13 Tverberg's Theorem

Tverberg's theorem generalizes Radon's theorem from two parts to an arbitrary number of parts.

Theorem 389 (Tverberg's Theorem)**.** *Let $r \geq 2$. Any*

$$(r-1)(d+1)+1$$

points in $\mathbb{R}^d$ can be partitioned into r nonempty sets

$$S_1, \ldots, S_r$$

such that

$$\bigcap_{j=1}^{r} \operatorname{conv}(S_j) \neq \emptyset.$$

Proof. Set

$$D = (r-1)(d+1).$$

Let

$$u_1, \ldots, u_r \in \mathbb{R}^{r-1}$$

be the vertices of a simplex centered at the origin. Thus

$$u_1 + \cdots + u_r = 0,$$

and this is, up to scaling, their only linear dependence.

Let the given points be

$$x_1, \dots, x_{D+1} \in \mathbb{R}^d,$$

and lift them to

$$\bar{x}_i = (x_i, 1) \in \mathbb{R}^{d+1}.$$

For every i, define a color class in

$$\mathbb{R}^{r-1} \otimes \mathbb{R}^{d+1} \cong \mathbb{R}^D$$

by

$$C_i = \{u_1 \otimes \bar{x}_i, \dots, u_r \otimes \bar{x}_i\}.$$

Because

$$u_1 + \cdots + u_r = 0,$$

we have

$$0 \in \operatorname{conv}(C_i)$$

for every i.

There are exactly

$$D + 1$$

color classes in $\mathbb{R}^D$. By colorful Carathéodory, for every i we may choose an index

$$j(i) \in [r]$$

such that

$$0 \in \operatorname{conv}\{u_{j(i)} \otimes \bar{x}_i : i = 1, \dots, D+1\}.$$

Thus there are coefficients

$$\lambda_i \geq 0, \qquad \sum_i \lambda_i = 1,$$

such that

$$\sum_i \lambda_i u_{j(i)} \otimes \bar{x}_i = 0.$$

Define

$$S_\ell = \{i : j(i) = \ell\}$$

and

$$z_j = \sum_{i \in S_j} \lambda_i \bar{x}_i.$$

Then

$$\sum_{j=1}^{r} u_j \otimes z_j = 0.$$

Choose

$$u_1, \dots, u_{r-1}$$

as a basis and write

$$u_r = -\sum_{j=1}^{r-1} u_j.$$

Then

$$0 = \sum_{j=1}^{r-1} u_j \otimes (z_j - z_r).$$

Linear independence of $u_1, \dots, u_{r-1}$ implies

$$z_1 = \cdots = z_r =: z.$$

The last coordinate of z_j is

$$\alpha_j = \sum_{i \in S_j} \lambda_i.$$

Since all the z_j are equal,

$$\alpha_1 = \cdots = \alpha_r.$$

But

$$\sum_{j=1}^{r} \alpha_j = \sum_i \lambda_i = 1,$$

so

$$\alpha_j = \frac{1}{r}.$$

In particular, every S_j is nonempty.

Let y be r times the first d coordinates of z. For every j,

$$y = \sum_{i \in S_j} \frac{\lambda_i}{\alpha_j} x_i,$$

and the coefficients are nonnegative and sum to 1. Therefore

$$y \in \operatorname{conv}\{x_i : i \in S_j\}$$

for every j.

Hence

$$y \in \bigcap_{j=1}^{r} \operatorname{conv}(S_j).$$

□

For more than $(r-1)(d+1)+1$ points, apply the theorem to any subset of the required size and distribute the remaining points arbitrarily among the parts.

9.14 Matroid Intersection

Let

$$M_1 = (E, \mathcal{I}_1), \qquad M_2 = (E, \mathcal{I}_2)$$

be two matroids on the same ground set, with rank functions r_1, r_2. We want a largest set

$$I \in \mathcal{I}_1 \cap \mathcal{I}_2.$$

The basic example is bipartite matching: take one partition matroid requiring at most one edge incident to each left vertex, and another requiring at most one edge incident to each right vertex.

Theorem 390 (Matroid Intersection)**.** *For any two matroids on the same ground set,*

$$\boxed{\max_{I \in \mathcal{I}_1 \cap \mathcal{I}_2} |I| = \min_{U \subseteq E} \big(r_1(U) + r_2(E \setminus U)\big).}$$

The easy direction is immediate. For every common independent set I and every $U \subseteq E$,

$$|I| = |I \cap U| + |I \setminus U| \le r_1(U) + r_2(E \setminus U).$$

We prove the reverse inequality algorithmically.

Exchange Graph Fix a common independent set

$$I \in \mathcal{I}_1 \cap \mathcal{I}_2.$$

Its exchange graph D_I is the directed bipartite graph with sides I and $E \setminus I$.

For $y \in I$ and $x \notin I$, add

$$y \longrightarrow x$$

if

$$I - y + x \in \mathcal{I}_1,$$

and add

$$x \longrightarrow y$$

if

$$I - y + x \in \mathcal{I}_2.$$

Also define

$$X_1 = \{x \notin I : I + x \in \mathcal{I}_1\},$$

and

$$X_2 = \{x \notin I : I + x \in \mathcal{I}_2\}.$$

Thus an augmenting path has the form

$$x_0, y_1, x_1, \ldots, y_k, x_k,$$

where

$$x_0 \in X_1, \qquad x_k \in X_2.$$

If P is such a path, define

$$I' = I \triangle V(P) = I - \{y_1, \ldots, y_k\} + \{x_0, \ldots, x_k\}.$$

Then

$$|I'| = |I| + 1.$$

We use a shortest augmenting path, so it has no directed shortcut.

Lemma 391 (Unique Exchange Matching)**.** *Let $M = (E, \mathcal{I})$ be a matroid, let $I \in \mathcal{I}$, and let J satisfy*

$$|J| = |I|.$$

Construct the bipartite exchange graph between $I \setminus J$ and $J \setminus I$, joining $y \in I \setminus J$ to $x \in J \setminus I$ whenever

$$I - y + x \in \mathcal{I}.$$

If this graph has a unique perfect matching, then

$$J \in \mathcal{I}.$$

Proof. Let the unique perfect matching be

$$(y_1, x_1), \ldots, (y_k, x_k).$$

Orient the matching edges from x_i to y_i and all other exchange edges from y_i to x_j. There can be no directed cycle, since such a cycle would be an alternating cycle and would give a second perfect matching.

Thus we may order the pairs so that

$$y_i x_j$$

is never an exchange edge when $i < j$.

Suppose for contradiction that J is dependent, and let $C \subseteq J$ be a circuit. Choose the smallest i such that

$$x_i \in C.$$

For every other $x_j \in C \setminus I$, we have $j > i$, so

$$I - y_i + x_j$$

is dependent. Hence

$$x_j \in \mathrm{cl}(I - y_i).$$

Every element of $C \cap I$ also lies in $I - y_i$. Therefore

$$C - x_i \subseteq \mathrm{cl}(I - y_i).$$

Since C is a circuit,

$$x_i \in \mathrm{cl}(C - x_i) \subseteq \mathrm{cl}(I - y_i).$$

But (y_i, x_i) is an exchange edge, so

$$I - y_i + x_i$$

is independent, a contradiction. □

Lemma 392 (Augmenting Path)**.** *If P is a shortest directed path from X_1 to X_2 in D_I, then*

$$I \triangle V(P) \in \mathcal{I}_1 \cap \mathcal{I}_2.$$

Proof. Write

$$P = x_0, y_1, x_1, \ldots, y_k, x_k.$$

We prove independence in M_1. Add a new element t which is a coloop, so that

$$I + t$$

is independent.

Between

$$(I \cap V(P)) \cup \{t\}$$

and

$$V(P) \setminus I,$$

the M_1 exchange graph has the perfect matching

$$tx_0, \quad y_1x_1, \quad \ldots, \quad y_kx_k.$$

Because P is shortest, it has no shortcut, so this perfect matching is unique. By Lemma 391,

$$I \triangle V(P)$$

is independent in M_1.

For M_2, use instead the perfect matching

$$y_1x_0, \quad y_2x_1, \quad \ldots, \quad y_kx_{k-1}, \quad tx_k.$$

The same argument proves independence in M_2. □

Algorithm 35 Matroid Intersection

Input: Independence oracles for M_1, M_2
Output: A maximum common independent set I

```
I ← ∅
while true do
    Construct the exchange digraph D_I
    Construct source set X_1 and sink set X_2
    Find a shortest directed path P from X_1 to X_2
    if no such path exists then
        return I
    I ← I△V(P)
```

It remains to prove that failure to find an augmenting path certifies optimality.

Proof of Theorem 390. The upper bound was already proved.

Run Algorithm 35. Every augmentation increases $|I|$ by one and preserves common independence by Lemma 392.

Suppose eventually there is no path from X_1 to X_2. Let

$$U = \{e \in E : e \text{ can reach } X_2 \text{ in } D_I\}.$$

Then

$$X_2 \subseteq U, \qquad X_1 \cap U = \emptyset.$$

We claim

$$r_1(U) = |I \cap U|.$$

If not, some

$$x \in U \setminus I$$

could be added to $I \cap U$ while preserving independence in M_1. Since $x \notin X_1$, the set $I + x$ is dependent. Let C be its fundamental circuit.

Because

$$(I \cap U) + x$$

is independent, C must contain some

$$y \in I \setminus U.$$

Removing any element of the fundamental circuit gives an independent set, so

$$I - y + x \in \mathcal{I}_1.$$

Thus

$$y \longrightarrow x$$

is an arc. But $x \in U$ can reach X_2, hence so can y, contradicting $y \notin U$.

Thus

$$r_1(U) = |I \cap U|.$$

Similarly,

$$r_2(E \setminus U) = |I \setminus U|.$$

Indeed, otherwise there is

$$x \in (E \setminus U) \setminus I$$

such that

$$(I \setminus U) + x$$

is independent in M_2. Since $X_2 \subseteq U$, we have $x \notin X_2$, so $I + x$ has a fundamental circuit containing some

$$y \in I \cap U.$$

Then

$$I - y + x \in \mathcal{I}_2,$$

so

$$x \longrightarrow y$$

is an arc. Since y can reach X_2, so can x, contradicting $x \notin U$.

Therefore

$$r_1(U) + r_2(E \setminus U) = |I \cap U| + |I \setminus U| = |I|.$$

This matches the general upper bound, so I is maximum and proves the min–max theorem. □

With independence oracles, the direct implementation uses $O(|E|^2)$ oracle queries per augmentation and at most $|E|$ augmentations, hence $O(|E|^3)$ oracle queries.

9.15 Symmetric Submodular Minimization

Let $f : 2^V \to \mathbb{R}$ be submodular and symmetric:

$$f(A) + f(B) \geq f(A \cup B) + f(A \cap B), \qquad f(S) = f(V \setminus S).$$

For $n = |V| \geq 2$, we want a nonempty proper set minimizing f. Excluding $\emptyset$ and V is essential: symmetry and submodularity give $2f(S) \geq f(V) + f(\emptyset) = 2f(\emptyset)$, so they are already unconstrained minimizers.

Queyranne's algorithm uses a greedy ordering and contraction. In each phase it finds a pair s, t whose cheapest separating set is the singleton $\{t\}$. Record that singleton and merge the pair. Either we just recorded an optimum, or some optimum does not separate them and survives the merger. The proof uses only submodularity for the ordering and symmetry at its final step.

The Greedy Rule and the Certificate. For disjoint sets, define their interaction by

$$\operatorname{adj}(A, B) = f(A) + f(B) - f(A \cup B) - f(\emptyset) \geq 0.$$

For an undirected cut function with nonnegative edge weights, this is $2w(E(A, B))$: adding the two boundaries counts edges between the sets twice, while the boundary of their union counts them zero times. Thus the natural ordering starts anywhere and repeatedly adds the element most strongly attached to the current prefix.

Write $W_i = \{v_1, \ldots, v_i\}$ and $W_0 = \emptyset$. A *maximum-adjacency ordering* satisfies

$$\boxed{v_{i+1} \in \arg\min_{v \notin W_i} \big(f(W_i \cup \{v\}) - f(\{v\})\big).} \tag{96}$$

The omitted terms in the interaction are constant over the candidates. At $i = 0$ all scores are zero, so the first element is arbitrary; ties at later steps can also be broken arbitrarily.

A pair (s, t) is *pendent* if

$$f(\{t\}) = \min_{t \in X,\ s \notin X} f(X).$$

The point of the ordering is that its *last two* elements form such a pair. The following prefix inequality will certify this:

$$f(W_i) + f(\{u\}) \leq f(W_i \setminus S) + f(S \cup \{u\}), \qquad u \notin W_i, \quad S \subseteq W_{i-1}.$$

It says that moving any part of the earlier prefix over to a future element cannot improve the total value of the resulting two sides. The restriction $S \subseteq W_{i-1}$ keeps the last chosen element v_i on its original side.

Subtracting singleton values will convert (96) into the simpler rule of minimizing the value of the next prefix. We first prove the inequality for that simpler rule. This intermediate function need not be symmetric or nonnegative.

Lemma 393 (Greedy splitting inequality)**.** *Let $g : 2^U \to \mathbb{R}$ be submodular, and order $U = \{u_1, \dots, u_m\}$ by choosing each next element to minimize $g(U_{i-1} \cup \{u\})$, where $U_i = \{u_1, \dots, u_i\}$. For $m \geq 2$, every set X separating u_{m-1} and u_m satisfies*

$$\boxed{g(X) + g(U \setminus X) \geq g(U_{m-1}) + \min_{x \in X} g(\{x\}).}$$

Proof. Induct on m. For $m = 2$, the two possible sets X are singletons, and the assertion follows from $g(\{u_1\}) \leq g(\{u_2\})$. Suppose $m > 2$ and put $a = u_1$.

First assume $a \notin X$. On $U' = U \setminus \{a\}$ define $h(Y) = g(Y \cup \{a\})$. This is submodular, and $u_2, \dots, u_m$ is a greedy ordering for h: every comparison is exactly the corresponding comparison for g after choosing a. Choose $x \in X$ minimizing $h(\{x\})$. Induction gives

$$g(X \cup \{a\}) + g(U \setminus X) \geq g(U_{m-1}) + g(\{a, x\}).$$

Submodularity applied to X and $\{a, x\}$ gives

$$g(X) + g(\{a, x\}) \geq g(X \cup \{a\}) + g(\{x\}).$$

Adding cancels the two terms involving a, leaving $g(X) + g(U \setminus X) \geq g(U_{m-1}) + g(\{x\})$, which is at least the asserted bound.

If $a \in X$, apply the case just proved to $U \setminus X$. The first greedy choice minimizes singleton values, so

$$\min_{x \in U \setminus X} g(\{x\}) \geq g(\{a\}) \geq \min_{x \in X} g(\{x\}).$$

Substituting this in that case's conclusion proves the remaining case. □

Corollary 394 (Greedy-prefix inequality)**.** *For any submodular f and maximum-adjacency ordering, every $1 \leq i < n$, $u \notin W_i$, and $S \subseteq W_{i-1}$ satisfy*

$$\boxed{f(W_i) + f(\{u\}) \leq f(W_i \setminus S) + f(S \cup \{u\}).}$$

Proof. Restrict to $U = W_i \cup \{u\}$ and subtract the modular function of singleton values:

$$g(Y) = f(Y) - \sum_{v \in Y} f(\{v\}) \qquad (Y \subseteq U).$$

A modular function has equality in the submodular inequality, so g is submodular; also $g(\{v\}) = 0$ for every v. The order $v_1, \dots, v_i, u$ greedily minimizes g of the next prefix, because its candidate score is

$$g(W \cup \{v\}) = f(W \cup \{v\}) - f(\{v\}) - \sum_{w \in W} f(\{w\}).$$

The last sum is independent of v, and restricting the candidates preserves every greedy choice. The set $S \cup \{u\}$ separates the last two elements v_i, u. Apply Lemma 393; its singleton minimum is zero, so it gives $g(S \cup \{u\}) + g(W_i \setminus S) \geq g(W_i)$. The modular terms on the left cover $W_i \cup \{u\}$ once, while those on the right cover W_i once. Cancel them to obtain the claim. □

Theorem 395 (Pendent pair)**.** *For symmetric submodular f, the last two elements $s = v_{n-1}$ and $t = v_n$ of a maximum-adjacency ordering satisfy*

$$\boxed{f(\{t\}) = \min_{t \in X,\ s \notin X} f(X).}$$

Proof. Fix X with $t \in X$ and $s \notin X$. Take $S = X \setminus \{t\} \subseteq W_{n-2}$ in the prefix inequality at $i = n - 1$, with future element $u = t$. Since $W_{n-1} = V \setminus \{t\}$ and $W_{n-1} \setminus S = V \setminus X$, it gives

$$f(V \setminus \{t\}) + f(\{t\}) \leq f(V \setminus X) + f(X).$$

Symmetry turns this into $2f(\{t\}) \leq 2f(X)$. The singleton $\{t\}$ itself is allowed, so the lower bound is attained. □

Contract the Pair. Maintain a partition $\mathcal{P}$ of the original ground set into nonempty blocks. A current element is one block $B \in \mathcal{P}$. The contracted function is

$$f_{\mathcal{P}}(\mathcal{X}) = f\left(\bigcup_{B \in \mathcal{X}} B\right) \qquad (\mathcal{X} \subseteq \mathcal{P}).$$

It remains symmetric and submodular: unions, intersections, and complements of block families correspond to the same operations on their unions, because the blocks partition V.

Algorithm 36 A pendent pair of current blocks

```
function PendentPair(P, f)
    for each B ∈ P do
        d_B ← f(B)
    Choose any B_1 ∈ P; L ← [B_1]; W ← B_1
    while |L| < |P| do
        for each B ∈ P not yet in L do
            q_B ← f(W ∪ B) − d_B
        Choose B with smallest q_B; append B to L; W ← W ∪ B
    return the last two blocks (S, T) of L, together with d_T
```

Algorithm 37 Queyranne's symmetric submodular minimization

Input: A finite set V with $n \geq 2$ and a symmetric submodular function f
Output: A nonempty proper set X^* minimizing f

```
P ← {{v} : v ∈ V}; best ← +∞
while |P| ≥ 2 do
    (S, T, c) ← PendentPair(P, f)
    if c < best then
        best ← c; save a copy X* ← T
    P ← (P \ {S, T}) ∪ {S ∪ T}
return X*
```

Theorem 396 (Queyranne's algorithm)**.** *Algorithm 37 finds a nontrivial minimizer using $O(n^3)$ evaluations of f. With the implementation below, it uses $O(n^3)$ additional operations and $O(n)$ words, excluding space used by the function evaluator. If one evaluation costs $T_f(n)$, the total running time is $O(n^3(T_f(n)+1))$.*

Proof. Maintain the invariant that either a global optimum has been saved or some global optimum is a union of current blocks. Initially every set is such a union. Fix a surviving optimum in one phase. If it separates the last two blocks S, T, the pendent-pair theorem for $f_{\mathcal{P}}$ gives $f(T)$ no larger than its value. The block T is itself a nonempty proper subset of V, so it is also globally optimal and is recorded. Otherwise the optimum is still a union of blocks after merging S, T. This proves the invariant. With one block left, no nontrivial union of blocks remains, so an optimum must have been saved.

With m current blocks, the singleton evaluations cost m calls, and the greedy comparisons cost at most $(m-1)+(m-2)+\cdots+1$. Thus a phase uses $O(m^2)$ calls. There are phases of sizes $n, n-1, \ldots, 2$, so the total is $\sum_{m=2}^{n} O(m^2) = O(n^3)$.

For explicit set manipulation, store the blocks as disjoint lists and keep an n-entry membership vector for W, together with one selected flag per block. To evaluate $f(W \cup B)$, temporarily mark the vertices of the unselected block B, call the evaluator on the vector, and then unmark them. Once a block is selected, mark it permanently for this phase. In one greedy step, the candidate blocks are disjoint, so all temporary marking and unmarking costs $O(n)$ in total, not $O(n)$ per candidate. A phase therefore has $O(nm + m^2)$ additional work; summed over phases this is $O(n^3)$. Computing singleton values, merging block lists, clearing

the membership vector, and copying the best block cost at most $O(n)$ per phase. The lists, vectors, scores, and saved answer require $O(n)$ words. □

10 Data Structures

10.1 RMQ

We want to find the index of the minimum element within an arbitrary subarray $A[l \dots r]$. We can achieve $O(1)$ query time with $O(n)$ preprocessing time and space.

1. Blocks We partition the array into blocks of size $B = \Theta(\log n)$. For each index $i \in [0, n-1]$, we maintain a bitmask $mask[i]$ of length up to B. This mask represents a strictly increasing subsequence of values ending at i, extending leftward up to the block boundary or at most B elements.

Specifically, the k-th bit (0-indexed) of $mask[i]$ is set to 1 if and only if $A[i-k]$ is the minimum element in the suffix $A[i-k \dots i]$. As we iterate i from 0 to $n-1$, we compute $mask[i]$ iteratively:

1. Shift the previous mask left by 1: $mask_{\text{curr}} \leftarrow mask_{i-1} \ll 1$.
2. While the current element $A[i]$ is strictly less than the element corresponding to the lowest set bit in $mask_{\text{curr}}$, unset that bit.
3. Set the 0-th bit to 1, representing the element $A[i]$ itself.

2. Sparse Table We construct a Sparse Table over the $\lfloor \frac{n}{B} \rfloor$ blocks. Instead of storing all elements, the 0-th level of the Sparse Table only stores the overall minimum of each block. The minimum of the j-th block (which ends at index $i = j \cdot B + B - 1$) is found using its mask: $mask[i]$ contains suffix minimums, so its highest set bit is the minimum of the entire block. Because there are only n/B blocks, the Sparse Table takes $O((n/B)\log(n/B)) = O(n)$ time and space.

3. Query Resolution To answer a query for the range $[l, r]$:

- **Case 1: The range spans at most B elements.** The range is small enough that the entire answer is encoded within $mask[r]$. We truncate the mask to only consider the last $r - l + 1$ elements by doing a bitwise AND with $(1 \ll (r - l + 1)) - 1$. The index of the highest set bit in this truncated mask corresponds to the minimum in $[l, r]$.
- **Case 2: The range spans multiple blocks.** We decompose the range into three parts:
  1. A partial block on the left: from l to the end of its block.
  2. A set of completely covered intermediate blocks.
  3. A partial block on the right: from the start of its block to r.

  The minimum of the left partial block is found using the highest set bit of the mask at the end of that block. The minimum of the right partial block is found using the highest set bit of $mask[r]$. The minimum of the completely covered intermediate blocks is queried in $O(1)$ time from the Sparse Table. We simply take the minimum of these three candidates.

Algorithm 38 RMQ

```
function Init(A[0 ... n − 1])
    B ← Θ(log n)
    mask ← array of size n
    curr_mask ← 0
    for i ← 0 to n − 1 do
        curr_mask ← curr_mask ≪ 1
        while curr_mask ≠ 0 and A[i] < A[i − trailing_zeros(curr_mask)] do
            curr_mask ← curr_mask & (curr_mask − 1)                ▷ Pops the lowest set bit
        curr_mask ← curr_mask | 1
        mask[i] ← curr_mask
                                                                   ▷ Build Sparse Table over Blocks
    num_blocks ← ⌊n/B⌋
    ST ← 2D array of size log(num_blocks) × num_blocks
    for j ← 0 to num_blocks − 1 do
        end_idx ← j · B + B − 1
        ST[0][j] ← end_idx − ⌊log2(mask[end_idx])⌋
    for k ← 1 to ⌊log2(num_blocks)⌋ do
        for j ← 0 to num_blocks − (1 ≪ k) do
            ST[k][j] ← min(ST[k − 1][j], ST[k − 1][j + (1 ≪ (k − 1))])
function Query(l, r)
    if r − l + 1 ≤ B then
        trunc_mask ← mask[r] & ((1 ≪ (r − l + 1)) − 1)
        return r − ⌊log2(trunc_mask)⌋
                                                                   ▷ Left partial block
    left_end ← (l/B) · B + B − 1
    ans ← left_end − ⌊log2(mask[left_end])⌋
                                                                   ▷ Right partial block
    ans ← min(ans, r − ⌊log2(mask[r])⌋)
                                                                   ▷ Intermediate full blocks
    x ← l/B + 1
    y ← r/B − 1
    if x ≤ y then
        k ← ⌊log2(y − x + 1)⌋
        mid_ans ← min(ST[k][x], ST[k][y − (1 ≪ k) + 1])
        ans ← min(ans, mid_ans)
    return ans
```

10.2 DSU

A disjoint-set union structure maintains a partition under merges and queries for the representative of an element's set. Represent each set by a rooted tree. A *find* follows parent pointers to the root and then makes every other vertex on the path a child of that root; this is *path compression*. A union performs two finds and links the roots if they differ.

Path compression makes the path just traversed cheap to traverse again, but later unions can put new ancestors above its root. To measure lasting progress, we track how much larger a vertex's parent's rank becomes. The inverse Ackermann function will count the number of increasingly coarse ways of measuring this progress.

10.2.1 Rank Facts

Use *union by rank*: every vertex starts with rank 0; when roots of equal rank r are linked, the surviving root gets rank $r+1$; otherwise the smaller-rank root becomes a child of the larger-rank root. Ranks are not decreased by compression. Thus ranks strictly increase along parent pointers, and a vertex's rank is fixed once it becomes a nonroot.

A root of rank r represents at least 2^r elements, by induction on the unions. Consequently every rank is at most $\lfloor \log_2 n \rfloor$. There is also a useful bound on their sum: each rank increase adds exactly 1 to the sum and occurs during a successful union, of which there are at most $n-1$. Hence $\sum_x r(x) \le n-1$ throughout the execution. This suggests an amortization budget: if a rank-r vertex can be charged only $O(r)$ times at each of L levels, all such charges total $O(nL)$. We now choose the levels to make this happen.

10.2.2 Why the Ackermann Hierarchy Appears

Consider an increasing threshold function f with $f(s) > s$. For a vertex of rank r, measure its parent's rank against $f(r), f^{(2)}(r), f^{(3)}(r), \ldots$, where $f^{(j)}$ denotes j-fold iteration. These thresholds are suited to path compression for a simple reason. Suppose x has parent rank p, and a later nonroot y on the find path satisfies $r(p(y)) \ge f(r(y))$. Since $r(y) \ge p$, the root has rank at least $f(p)$. Thus compression advances x by at least one threshold: if $p \ge f^{(j)}(r(x))$, its new parent has rank at least $f^{(j+1)}(r(x))$.

We can afford r such advances per level at a rank-r vertex, because $\sum_x r(x) \le n-1$. Once its parent's rank reaches $f^{(r+1)}(r)$, we therefore move to the next level, whose threshold is precisely $f^{(r+1)}(r)$. Starting with the smallest possible threshold, $r+1$, gives the recurrence

$$A_0(r) = r+1, \qquad A_{k+1}(r) = A_k^{(r+1)}(r) \qquad (r \ge 1).$$

This is an Ackermann hierarchy. The first thresholds are $A_0(r) = r+1$, $A_1(r) = 2r+1$, and $A_2(r) = 2^{r+1}(r+1) - 1$; the next repeatedly applies this exponential growth. Each new level packages together $r+1$ iterations of the previous level.

Define $\alpha = \alpha(n)$ to be the least integer $k \ge 1$ such that $A_k(1) > \log_2 n$. This differs from the usual inverse Ackermann conventions by at most an additive constant. Since no parent rank exceeds $\log_2 n$, only α levels can occur. We will show that each find has at most one uncharged positive-rank vertex per level; every other visit uses up part of that vertex's $O(r)$ budget.

10.2.3 Levels and Progress

For a nonroot x of positive rank $r = r(x)$, define

$$\ell(x) = \max\{k \ge 0 : A_k(r) \le r(p(x))\}, \qquad j(x) = \max\{i \ge 1 : A_{\ell(x)}^{(i)}(r) \le r(p(x))\}.$$

The level specifies the current threshold function, and the index counts how many of its iterations the parent's rank has reached. The functions A_k increase in both their argument and k, so $0 \le \ell(x) < \alpha$. Moreover, maximality of the level gives $r(p(x)) < A_{\ell(x)+1}(r) = A_{\ell(x)}^{(r+1)}(r)$, and hence $1 \le j(x) \le r$.

After x becomes a nonroot, its rank is fixed and its parent's rank never decreases. Therefore its pair $(\ell(x), j(x))$ never decreases lexicographically: the level can increase, and while the level stays fixed the index can only increase. There are at most αr possible pairs, so this pair can strictly advance fewer than αr times over the remainder of the execution. Summing over vertices and using their final ranks, the total number of strict advances is at most $\alpha(n-1)$.

These levels and indices are only for the analysis; the data structure does not compute or store them.

10.2.4 Charging a Find

Consider a find path $x_0, x_1, \ldots, x_t$, with root x_t, and evaluate all levels and indices before compression. Suppose x_i has level k and index j, and some later nonroot x_q, where $i < q < t$, also has level k. Then

$$r(x_t) \ge r(p(x_q)) \ge A_k(r(x_q)) \ge A_k(r(x_{i+1})) \ge A_k^{(j+1)}(r(x_i)).$$

Here the third inequality uses $r(x_q) \geq r(x_{i+1})$, and the last uses the definition of j. After compression the parent of x_i is x_t, so either its level increases or its index increases. Charge the visit to this strict advance.

Thus only the last positive-rank vertex of each level on the path can remain uncharged. The levels need not occur in sorted order: we only use the last occurrence of each. There are at most α such vertices, and at most one rank-0 vertex because ranks strictly increase along the path. Including the root, the find therefore takes $O(\alpha + c)$ time, where c is the number of charged visits.

Across all finds, including those inside unions, there are at most $\alpha(n-1)$ charged visits. Consequently a sequence of m operations, including n singleton creations, takes $O(m\alpha(n))$ total time and $O(n)$ space.

The recurrence explains the bound: a second occurrence of a level on a path forces an extra iteration of that level's threshold; after r iterations we move to a threshold that packages all of them together. Ackermann growth describes these successive thresholds, and its inverse counts how many levels fit below the maximum rank.

10.3 DSU with Potential

The DSU with Potential, maintains disjoint sets while tracking relative differences (potentials) between elements in the same set. The potentials must belong to a group G.

For any node x, the array $\mathtt{p}[x]$ maintains the potential difference from x to its current parent in the DSU tree, which we denote as $\Delta(x, \text{parent}(x))$.

1. Path Compression When querying the root of a node x, the algorithm recursively compresses the path. Suppose the recursive call finds the root R and the potential from the parent to the root, $\Delta(\text{parent}(x), R)$.

To update x to point directly to R, we compose the potential from x to its parent with the potential from its parent to R:

$$\Delta(x, R) = \Delta(x, \text{parent}(x)) \circ \Delta(\text{parent}(x), R)$$

2. Querying Potential To find the potential $\Delta(a, b)$ from node a to node b, we first verify they belong to the same set. Let R be their common root. The `find` operation gives us $v_a = \Delta(a, R)$ and $v_b = \Delta(b, R)$.

The path from a to b can be constructed by traversing from a up to R, and then down from R to b. Thus, the combined potential is:

$$\Delta(a, b) = \Delta(a, R) \circ \Delta(R, b) = v_a \circ v_b^{-1}$$

3. Edge Insertion When merging the sets containing a and b by adding a directed edge from a to b with a given potential P, we require that $\Delta(a, b) = P$.

Let R_a and R_b be the respective roots of a and b. Suppose we link R_a as a child of R_b. We need to compute the required weight $W = \Delta(R_a, R_b)$ for this new edge.

We know the full path $a \to R_a \to R_b \to b$ must equal the target potential P:

$$\Delta(a, R_a) \circ \Delta(R_a, R_b) \circ \Delta(R_b, b) = P$$

Substituting our known variables $v_a = \Delta(a, R_a)$, $W = \Delta(R_a, R_b)$, and $v_b^{-1} = \Delta(R_b, b)$:

$$v_a \circ W \circ v_b^{-1} = P$$

Thus:

$$W = v_a^{-1} \circ P \circ v_b$$

If the union-by-size heuristic determined that R_b should become a child of R_a, the weight of the reversed edge is the inverse W^{-1}.

10.4 Persistent Queue

A fully persistent queue allows us to push and pop elements from any historical state of the queue. By modeling the sequence of pushes as a tree, we can implement it by using Algorithm 48.

1. State Each state in the persistent queue is represented as a node in a tree. A node u stores:

- v: The value inserted at this state.
- p: The parent node (the state of the queue before this element was pushed).
- depth: The depth of the node in the tree (the total number of pushes to reach this state).
- jmp: The jump pointer used for ancestor queries.
- fr: A pointer to the node representing the current front of the queue.

2. Push Operation When pushing a value onto a previous state p, we append a new child node u. The front pointer fr is inherited from p

We compute the new jump pointer jmp$[u]$ in $O(1)$ time using the combination rule from Algorithm 48. We check:

$$\text{depth}[p] - \text{depth}[\text{jmp}[p]] = \text{depth}[\text{jmp}[p]] - \text{depth}[\text{jmp}[\text{jmp}[p]]]$$

If they are equal, we merge them: jmp$[u] \leftarrow$ jmp[jmp$[p]]$. Otherwise, we reset the jump length: jmp$[u] \leftarrow p$.

3. Pop Operation To pop an element from state p, we must advance the front pointer fr to the next element in the queue. Because the queue's elements form a path from fr down to the current tail p, the next element to become the front is the child of fr that lies on the path to p.

This is a Level Ancestor query: we must find the ancestor of p whose depth is exactly depth$[fr] + 1$. We can find this in $O(\log n)$ time and then create a new state that copies p, but updates its fr pointer to this new ancestor.

4. Front Operation Querying the front of the queue simply requires reading the value stored at the node pointed to by fr.

Algorithm 39 Persistent Queue

```
function Push(v, p)
    if p = NULL then
        return QueueNode(v, parent: NULL, fr: nxt, jmp: nxt, depth: 0)
    fr ← nodes[p].fr
    p_j ← nodes[p].jmp
    p_jj ← nodes[p_j].jmp
    if depth[p] − depth[p_j] = depth[p_j] − depth[p_jj] then
        jmp ← p_jj
    else
        jmp ← p
    depth ← nodes[p].depth + 1
    return QueueNode(v, p, fr, jmp, depth)
function LevelAncestor(p, target_depth)
    while depth[p] > target_depth do
        if depth[jmp[p]] ≥ target_depth then
            p ← jmp[p]
        else
            p ← parent[p]
    return p
function Pop(p)
    new_node ← nodes[p].clone()
    target_depth ← nodes[nodes[p].fr].depth + 1
    new_node.fr ← LevelAncestor(p, target_depth)
    return new_node
```

10.5 Online Suffix Max

To answer suffix maximum queries $\max(A[i \dots n-1])$ in a setting where elements are appended, we can combine a monotonic stack with a DSU.

1. Monotonic Stack When a new element x is appended at index p, it invalidates any previous elements smaller than it. We maintain a stack of indices corresponding to a strictly decreasing subsequence of the array. As x is pushed, we pop all indices j from the stack where $A[j] \le x$.

2. DSU Any index whose suffix maximum was previously $A[j]$ now shares a suffix maximum of x. For every popped index j, we union its set into p's set. Finally, we push p onto the stack.

3. Queries To find the maximum in the suffix starting at index i, we simply query the DSU for the root of i.

10.6 Mo's Algorithm

Suppose we have a static array of length n and

$$q$$

offline range queries

$$[L_i, R_i).$$

Assume we can maintain the answer while moving either endpoint by one position, with $O(1)$ work per move.

Mo's algorithm orders the queries so consecutive intervals differ only a small amount.

Choose a block size B. Sort queries by

$$\left(\left\lfloor \frac{L}{B} \right\rfloor, R\right),$$

except that the direction of R alternates between blocks:

$$\boxed{\begin{cases} R \text{ increasing}, & \lfloor L/B \rfloor \text{ even}, \\ R \text{ decreasing}, & \lfloor L/B \rfloor \text{ odd}. \end{cases}}$$

Algorithm 40 Mo's Algorithm

Input: Array of length n, range queries $[L_i, R_i)$

choose block size B
sort queries by

$$\left(\lfloor L/B \rfloor,\ (-1)^{\lfloor L/B \rfloor} R\right)$$

with the second coordinate interpreted as alternating direction
$L \leftarrow 0$, $R \leftarrow 0$
for query $[a, b)$ in this order **do**
    while $L > a$ **do**
        $L \leftarrow L - 1$; ADD(L)
    while $R < b$ **do**
        ADD(R); $R \leftarrow R + 1$
    while $L < a$ **do**
        REMOVE(L); $L \leftarrow L + 1$
    while $R > b$ **do**
        $R \leftarrow R - 1$; REMOVE(R)
    record current answer

Theorem 397 (Mo Movement Bound)**.** *With block size B, the total number of endpoint moves is*

$$\boxed{O\left(qB + \frac{n^2}{B} + n\right).}$$

Choosing

$$B \asymp \frac{n}{\sqrt{q}}$$

gives

$$\boxed{O(n\sqrt{q} + q)}$$

total work when each endpoint update costs $O(1)$.

Proof. First consider the left endpoint.

Inside one block,

$$L$$

always lies in an interval of width B. Thus between consecutive queries in that block it moves by at most B. Over q queries,

$$\text{total movement of } L = O(qB + n).$$

Now consider R. Within one L-block, queries are sorted monotonically by R. Thus R traverses the array at most once inside that block.

There are at most

$$\left\lceil \frac{n}{B} \right\rceil$$

blocks. Therefore

$$\text{total movement of } R = O\left(n\frac{n}{B} + n\right) = O\left(\frac{n^2}{B} + n\right).$$

The odd/even reversal means that when one block finishes near one end of the array, the next begins from that same end rather than forcing an unnecessary reset.

Combining the two bounds,

$$O\left(qB + \frac{n^2}{B} + n\right).$$

Balance the two main terms:

$$qB = \frac{n^2}{B},$$

so

$$B = \frac{n}{\sqrt{q}}.$$

Then

$$qB = \frac{n^2}{B} = n\sqrt{q}.$$

□

10.7 A Fast Succinct Hash Table

We first assume access to the random hash functions used below; the next subsection explains how to replace fully random functions by a small explicit random data structure.

We work on a word RAM with word size

$$w = \Theta(\log n),$$

and consider a fixed polynomial-length sequence of operations independent of the random choices of the data structure. We maintain a capacity parameter

$$N = \Theta(n)$$

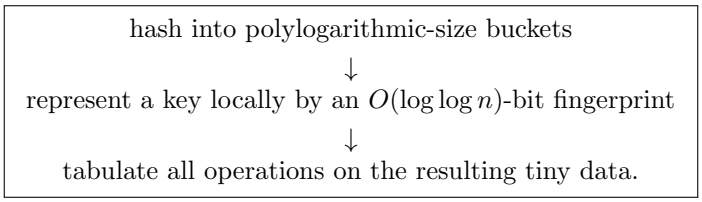
throughout an epoch.

The goal is an exact dynamic dictionary using

$$\boxed{nw + O(n \log\log n)}$$

bits, with constant-time queries and, with high probability, constant time per update after deamortization.

The construction has three layers:



The only updates that can occasionally be expensive are local fingerprint rebuilds. A final buffering step deamortizes these.

10.7.1 Small Buckets

Put

$$L = \lceil (\log N)^3 \rceil$$

and hash the keys uniformly into

$$B = \left\lceil \frac{N}{L} \right\rceil$$

buckets.

We maintain $n \le N$. For any fixed bucket, its load X is binomial with mean

$$\mu = \frac{n}{B} \le L.$$

Thus it exceeds $4L$ with superpolynomially small probability by a Chernoff bound.

At each time in a fixed update sequence, the current set of keys is fixed independently of the random top-level hash function, so the same argument applies. A union bound over $N^{O(1)}$ times still gives

$$N^{O(1)} e^{-\Omega(\log^3 N)} = N^{-\omega(1)}.$$

Henceforth condition on this event. Every bucket contains

$$O(\log^3 N)$$

keys throughout the epoch.

10.7.2 Storing the Full Keys Succinctly

The actual w-bit keys must be stored somewhere, but we do not want a word-sized pointer or other $\Theta(\log n)$-bit overhead for every key.

Within each bucket, divide the keys into blocks of

$$s = \lceil (\log N)^2 \rceil$$

keys. All blocks except possibly the last are full.

A bucket of size $b \le 4L$ therefore uses at most

$$\left\lceil \frac{b}{s} \right\rceil = O(\log N)$$

blocks. Give every bucket a directory containing the addresses of these blocks.

This representation has very little wasted space. Across all buckets, the number of allocated key slots is at most

$$\sum_{i=1}^{B} s \left\lceil \frac{|B_i|}{s} \right\rceil \le \sum_i |B_i| + Bs = n + O\left(\frac{N}{\log N}\right).$$

Since $w = \Theta(\log N)$, the full keys therefore occupy

$$\boxed{nw + O(N)}$$

bits.

Each bucket directory has $O(\log N)$ block addresses, each using $O(\log N)$ bits. Hence all directories together use

$$O\left(\frac{N}{\log^3 N} \log^2 N\right) = O\left(\frac{N}{\log N}\right)$$

bits.

A position within one bucket consists of

$$\text{block number} \in [O(\log N)]$$

and

$$\text{offset} \in [(\log N)^2],$$

so it requires only

$$\boxed{O(\log \log N)}$$

bits.

Insertion appends to the last block, allocating a new block if necessary. Deletion replaces the deleted key by the last key in its bucket and frees the last block if it becomes empty. Thus the key storage itself supports every local update in $O(1)$ time.

The only remaining task is to locate the local position of a queried key.

10.7.3 Local Fingerprints

For each bucket choose a random fingerprint function

$$f : U \to [M], \qquad M = L^3 = \Theta(\log^9 N).$$

A fingerprint therefore occupies

$$\log M = O(\log \log N)$$

bits.

We maintain the invariant that the fingerprints of the keys currently in a bucket are distinct.

For a bucket containing $b \le 4L$ keys, a fresh random fingerprint function fails this condition with probability at most

$$\binom{b}{2}\frac{1}{M} \le \frac{8L^2}{L^3} = \frac{8}{L}.$$

Thus a fresh function is collision-free with probability at least

$$1 - \frac{8}{L}.$$

Whenever a collision occurs, resample the local fingerprint function until all keys in that bucket have distinct fingerprints and rebuild its local metadata.

For a query key x, we compute its bucket and $f(x)$. If the local dictionary contains this fingerprint, it returns a candidate position. We then compare the full w-bit key stored at that position with x.

10.7.4 A Constant-Time Dictionary Inside a Bucket

Each bucket stores the map

$$f(x) \longmapsto \text{position}(x).$$

Both sides use

$$O(\log \log N)$$

bits.

Store this map in a B-tree of fanout

$$F = \lfloor \sqrt{\log N} \rfloor.$$

A node contains $O(F)$ fingerprints, local positions, and local child indices, and therefore has

$$O(F \log \log N) = O(\sqrt{\log N} \log \log N) = o(\log N)$$

bits.

Consequently there are only

$$2^{o(\log N)} = N^{o(1)}$$

possible node states. The input to any elementary node operation also has only $O(\log \log N)$ bits, so all possible node operations—searching within a node, inserting or deleting an entry, splitting, merging, and choosing a child—can be stored in one global

$$N^{o(1)}$$

lookup table.

A bucket contains at most $4L = O(\log^3 N)$ entries. Hence the B-tree depth is

$$O\left(\log_F(\log^3 N)\right) = O(1),$$

since

$$\log_F(\log^3 N) = \frac{3 \log \log N + O(1)}{\frac{1}{2} \log \log N + O(1)} = 6 + o(1).$$

Thus lookup, insertion, deletion, splitting, and merging in the local dictionary all take

$$\boxed{O(1)}$$

time.

There is one dictionary entry per stored key, each using $O(\log \log N)$ bits. B-tree node overhead and bucket roots are smaller. Hence all local dictionaries use

$$\boxed{O(n \log \log N)}$$

bits.

10.7.5 Cost of Fingerprint Rebuilding

An ordinary update costs $O(1)$. The only expensive event is an insertion whose fingerprint collides with an existing fingerprint, forcing a rebuild of its $O(L)$-key bucket.

We first quantify how often this happens.

Suppose the current bucket contains $b \leq 4L$ keys and its current random function is collision-free on those keys. For a new key whose fingerprint has not previously been exposed under this function, its value is uniform and independent of the b current values. Hence

$$\Pr[\text{collision}] = \frac{b}{M} \leq \frac{4L}{L^3} = \frac{4}{L^2}.$$

The same bound follows for a fixed oblivious update sequence by deferred decisions even when keys are deleted and later reinserted: keys that coexisted with x are already known not to collide with it, while fingerprints first exposed while x was absent are uniform subject only to avoiding the $O(L)$ fingerprints then present. This changes the denominator from M to $M - O(L)$, and hence, for sufficiently large N,

$$\Pr[\text{collision} \mid \text{entire past}] \leq \frac{5}{L^2}.$$

We will use

$$p := \frac{5}{L^2}.$$

If a collision occurs, a fresh fingerprint function on the whole bucket fails to be injective with probability at most

$$\frac{8}{L}.$$

Thus the number G of trials needed to obtain a good function satisfies

$$\Pr[G > r] \le \left(\frac{8}{L}\right)^r.$$

Taking

$$r = \lceil \log N \rceil,$$

$$\Pr[G > r] \le \left(\frac{8}{\log^3 N}\right)^{\log N} = \exp\left(-\Omega(\log N \log\log N)\right) = N^{-\Omega(\log\log N)}.$$

A union bound over any polynomial number of rebuilds preserves a $N^{-\omega(1)}$ failure probability.

Conditioned on $G \le \log N$, a rebuild costs

$$O(L \log N) = O(\log^4 N)$$

work.

10.7.6 Deamortizing Rebuilds

An individual rebuild still costs polylogarithmic time. We now turn this into constant time per external operation.

Set

$$q = \lfloor N^{1/4} \rfloor.$$

Maintain two deterministic $\sqrt{N}$-ary tries, each holding at most q pending updates.

The *growing buffer* receives new updates. The *shrinking buffer* contains the previous batch, which is being applied to the hash table. Queries first inspect the growing buffer, then the shrinking buffer, and then the main table, so pending updates are immediately visible.

Every external operation performs a fixed constant amount of work on the shrinking batch. When the growing buffer reaches q updates, we swap the two buffers.

It remains only to prove that a batch of q updates requires $O(q)$ total work with high probability.

Let

$$I_j = \mathbf{1}[\text{the } j\text{-th update causes a fingerprint collision}], \qquad C = \sum_{j=1}^{q} I_j.$$

Let $\mathcal{F}_{j-1}$ contain the entire history before update j. From the preceding calculation,

$$\Pr[I_j = 1 \mid \mathcal{F}_{j-1}] \le p = \frac{5}{L^2}.$$

Although the I_j's need not be independent, the usual Chernoff proof still applies. Indeed, for every $\lambda \ge 0$,

$$\begin{aligned}\mathbb{E}[e^{\lambda I_j} \mid \mathcal{F}_{j-1}] &= 1 + (e^\lambda - 1)\Pr[I_j = 1 \mid \mathcal{F}_{j-1}] \\ &\le 1 + p(e^\lambda - 1).\end{aligned}$$

Therefore, iterating conditional expectation,

$$\mathbb{E}e^{\lambda C} \le \left(1 + p(e^\lambda - 1)\right)^q.$$

This is exactly the MGF bound for a $\mathrm{Bin}(q, p)$ random variable. Hence all the standard binomial Chernoff upper-tail bounds remain valid.

Put

$$\mu = qp = \frac{5q}{L^2} = \frac{5q}{\log^6 N}.$$

Taking the standard $\delta = 1$ Chernoff bound,

$$\Pr[C \geq 2\mu] \leq \left(\frac{e}{4}\right)^{\mu} \leq e^{-\mu/3}.$$

Since $q = N^{1/4}$,

$$e^{-\mu/3} = \exp\left(-\Omega\left(\frac{N^{1/4}}{\log^6 N}\right)\right) = N^{-\omega(1)}.$$

Thus, with probability $1 - N^{-\omega(1)}$,

$$\boxed{C \leq \frac{10q}{\log^6 N}.}$$

We already showed that, with probability $1 - N^{-\omega(1)}$, every fingerprint rebuild requires at most $\log N$ trials. Since one trial rebuilds an $O(L) = O(\log^3 N)$-key bucket, every collision costs at most

$$O(L \log N) = O(\log^4 N)$$

work. Therefore all rebuilds in the batch cost

$$O\left(\frac{q}{\log^6 N} \cdot \log^4 N\right) = O\left(\frac{q}{\log^2 N}\right) = O(q).$$

Together with the $O(q)$ ordinary update work, the entire batch costs $O(q)$ with probability $1 - N^{-\omega(1)}$.

Choosing the constant amount of draining work per external operation larger than this hidden constant ensures that the shrinking buffer is empty before the growing buffer fills.

A union bound over any polynomial number of batches still gives high probability.

10.7.7 The Update Buffers Are Succinct

A $\sqrt{N}$-ary trie reads

$$\frac{1}{2} \log N$$

bits of the key per level. Since $w = \Theta(\log N)$, its depth is $O(1)$.

A deliberately crude representation already suffices. There are $O(q)$ nodes and each node may contain a direct-access array of $\sqrt{N}$ child pointers, each of $O(\log N)$ bits. Thus one buffer uses at most

$$O(q\sqrt{N} \log N) = O(N^{3/4} \log N) = o(N)$$

bits.

The two buffers therefore do not affect the $nw + O(n \log \log n)$ space bound.

10.7.8 Dynamic Resizing

Finally, N must track the current number of keys.

Use the usual constant-factor hysteresis so that

$$N = \Theta(n).$$

When a resize is triggered, initialize a new table with capacity $N' = \Theta(n)$ and migrate a constant number of old keys per external operation.

Crucially, when a key is inserted into the new table, immediately delete its full representation from the old table. Thus a key is never stored in both tables.

If the old and new tables currently contain n_1, n_2 keys, then

$$n_1 + n_2 = n,$$

and their full-key payload occupies

$$n_1 w + n_2 w = nw$$

bits, apart from the $O(n)$ block slack already accounted for.

Both tables together contain only $O(n)$ keys, so their combined metadata uses

$$O(n \log \log n)$$

bits.

Queries inspect both tables during migration. Since resizing is triggered only after a constant-factor change in n, there are $\Theta(n)$ external operations available before another resize can be necessary, enough to finish the migration at constant additional work per operation.

10.7.9 Summary

We obtain, throughout the execution,

$$\boxed{nw + O(n \log \log n)}$$

bits of space.

Queries take worst-case $O(1)$ time. Updates take $O(1)$ external work, and with probability $1 - n^{-\omega(1)}$ every background batch finishes before its buffer is needed again.

10.8 Simulating Fully Random Hashing

A truly random function

$$h : U \to V$$

cannot be stored explicitly when U is large. For the hash-table applications above, however, we only need a function to look fully random on small fixed subsets of U.

The following construction gives exactly that.

10.8.1 Private Random Cells

Fix

$$S \subseteq U, \qquad |S| \le n^{0.8}.$$

Choose a sufficiently large constant c. Independently for $j = 1, \dots, c$, choose a universal hash function

$$g_j : U \to [n^{0.9}]$$

and an array

$$A_j[1, \dots, n^{0.9}]$$

of independent uniform elements of the additive group V.

Define

$$\boxed{h(x) = \sum_{j=1}^{c} A_j[g_j(x)].}$$

Lemma 398 (Full Randomness on a Fixed Small Set)**.** *For every constant K, c can be chosen as a constant depending only on K such that, for every fixed $S \subseteq U$ with $|S| \le n^{0.8}$,*

$$\Pr[h|_S \text{ is a fully random function}] \ge 1 - n^{-K}.$$

Proof. For $x \in S$, call the cell $A_j[g_j(x)]$ *private* if no other $y \in S$ accesses it.

For fixed x, j, universality and a union bound give

$$\begin{aligned}\Pr[A_j[g_j(x)] \text{ is not private}] &\le \sum_{y \in S \setminus \{x\}} \Pr[g_j(x) = g_j(y)] \\ &\le \frac{|S| - 1}{n^{0.9}} \\ &\le n^{-0.1}.\end{aligned}$$

The functions $g_1, \ldots, g_c$ are independent, so

$$\Pr[x \text{ has no private cell}] \le n^{-0.1c}.$$

Union-bounding over the at most $n^{0.8}$ elements of S,

$$\Pr[\exists x \in S \text{ with no private cell}] \le n^{0.8 - 0.1c}.$$

Choosing

$$c \ge 10K + 8$$

makes this at most n^{-K}.

Condition now on the g_j's and on every $x \in S$ having a private cell. Choose one private cell

$$A_{j_x}[g_{j_x}(x)]$$

for each x.

These selected cells are all distinct. Moreover, the selected cell for x is not used in the hash value of any other key in S.

Condition on every array entry except these selected cells. Then, for every $x \in S$,

$$h(x) = A_{j_x}[g_{j_x}(x)] + c_x,$$

where c_x is now fixed.

The selected array entries are mutually independent and uniform in V. Adding fixed constants preserves uniformity, so

$$(h(x))_{x \in S}$$

is a vector of mutually independent uniform elements of V. Thus $h|_S$ is exactly a fully random function. □

The construction uses

$$\widetilde{O}(n^{0.9})$$

random bits when U, V have polynomial size, and evaluates h using a constant number of hash evaluations and table lookups.

10.8.2 Splitting a Large Set into Small Fixed Sets

The lemma only applies to fixed sets of size at most $n^{0.8}$. To use it for n stored keys, first split those keys into much smaller buckets.

Choose an independent function

$$h_1 : U \to [B], \qquad B = \lceil \sqrt{n} \rceil,$$

using the preceding construction.

We first prove that every bucket has

$$(1 + o(1))\sqrt{n}$$

keys.

For the analysis only, partition the n stored keys into

$$G = \lceil n^{0.2} \rceil$$

fixed groups

$$S_1, \ldots, S_G, \qquad |S_j| \le n^{0.8}.$$

Choose the constant c in Lemma 398 large enough that the failure probability for each group is at most n^{-10}. A union bound gives

$$\Pr[h_1 \text{ is fully random on every } S_j] \ge 1 - Gn^{-10} = 1 - n^{-\Omega(1)}.$$

Condition on this event.

For destination bucket i and group S_j, let

$$X_{i,j} = |\{x \in S_j : h_1(x) = i\}|.$$

Then

$$X_{i,j} \sim \mathrm{Bin}\left(|S_j|, \frac{1}{B}\right),$$

with mean

$$\mu_{i,j} = \frac{|S_j|}{B} \le n^{0.3}.$$

Set

$$a = n^{0.15} \log n.$$

The Bernstein inequality gives

$$\Pr[|X_{i,j} - \mu_{i,j}| \ge a] \le 2 \exp\left(-\frac{a^2}{2(\mu_{i,j} + a/3)}\right).$$

For sufficiently large n,

$$a \le n^{0.3},$$

so

$$2(\mu_{i,j} + a/3) \le 3n^{0.3}.$$

Hence

$$\begin{aligned}\Pr[|X_{i,j} - \mu_{i,j}| \ge a] &\le 2 \exp\left(-\frac{n^{0.3} \log^2 n}{3n^{0.3}}\right) \\ &= 2e^{-\log^2 n/3}.\end{aligned}$$

There are at most

$$BG \le 2n^{0.7}$$

pairs (i, j), so a union bound gives

$$\Pr[\exists i, j : |X_{i,j} - \mu_{i,j}| > a] \le 4n^{0.7} e^{-\log^2 n/3} = n^{-\omega(1)}.$$

Therefore, simultaneously for every bucket i,

$$\begin{aligned}|B_i| &= \sum_j X_{i,j} \\ &= \sum_j \mu_{i,j} + O(Ga) \\ &= \frac{n}{B} + O(n^{0.35} \log n) \\ &= \sqrt{n} + o(\sqrt{n}).\end{aligned}$$

Thus

$$\boxed{\max_i |B_i| = (1 + o(1))\sqrt{n}}$$

with high probability.

10.8.3 Randomize After Splitting

Now choose a second, independent function h_2 from the private-cell construction.

Condition on h_1 and on the event that every top-level bucket has $(1 + o(1))\sqrt{n}$ keys. The sets

$$B_1, \ldots, B_B$$

are now fixed, and for sufficiently large n,

$$|B_i| < n^{0.8}.$$

By Lemma 398, for any desired constant K, we may choose the constant number of tables in h_2 so that

$$\Pr[h_2|_{B_i} \text{ is not fully random}] \le n^{-K}$$

for each fixed i.

There are only $O(\sqrt{n})$ buckets. Hence

$$\Pr[\exists i : h_2|_{B_i} \text{ is not fully random}] \le O(\sqrt{n})\, n^{-K}.$$

Taking K sufficiently large gives high probability that

$$\boxed{h_2 \text{ is fully random on every bucket simultaneously.}}$$

10.9 Soft Heaps

A soft heap saves comparisons by allowing a controlled number of keys to increase. This is not an approximation to the numerical value of the minimum: a corrupted key may increase arbitrarily. Instead, the guarantee controls how many items are affected. The implementation below uses ordinary heap-ordered ternary trees, except that high nodes combine two items into one shared-key pool.

Definition 399 (Soft heap)**.** Each item has an original key and a current key that never decreases and is never smaller than its original key. An item whose key has increased is *corrupted*. After N insertions, an ε-soft heap has at most εN corrupted items still present. Here N counts all insertions, not just the items currently present; when heaps are melded, their insertion counts add. Meld consumes two disjoint heaps with the same error parameter.

Find-min and extract-min use the current keys. An extraction returns a minimum *before* its subsequent repairs, together with the items newly corrupted by those repairs. No corruption will occur during insertion or melding. Break ties by insertion identifiers, so the keys can be treated as distinct.

Theorem 400 (Ternary soft heap)**.** *For $0 < \varepsilon \le 1/2$, the construction below supports insert and meld in amortized $O(1)$ time, find-min in worst-case $O(1)$ time, and extract-min in amortized $O(\log(1/\varepsilon))$ time. More precisely, N insertions, D extractions, M melds, and F find-min queries take*

$$O\big(N + M + F + D\log(1/\varepsilon)\big)$$

total time and $O(N)$ space. Reporting newly corrupted items is included.

10.9.1 A Forest of Heaps

A rank-r tree is a perfect ternary tree with 3^r original leaves. Each leaf records one insertion; items subsequently move only upward. A node stores either one representative item or nothing. Whenever a node is empty but its subtree is not, refill it from the child with minimum representative key, then recursively refill that child. An empty node after refilling therefore means that its entire subtree is empty.

Keep the nonempty trees in a doubly linked list of nondecreasing ranks, with at most two roots of each rank. Three rank-r roots are linked under a new rank-$(r+1)$ root, which is filled by one ordinary upward pull. This is carrying in base 3. A rank-r link costs $O(r)$, but at most $N/3^r$ rank-r trees are ever created: their insertion leaves are disjoint. Thus all linking work is $O(N \sum_{r \geq 1} r/3^r) = O(N)$.

The obstacle is extraction. Pulling a separate representative through every level for every deletion can cost $\Theta(\log N)$ each. Above a threshold, we will make each upward representative account for two lower ones.

10.9.2 Combine Two Representatives, Keep the Larger Key

Every representative e carries a linked list $C(e)$ of corrupted items, all assigned current key $\mathrm{key}(e)$. The representative itself retains its original key. Thus a pool is $\{e\} \cup C(e)$, with e the largest original key in that pool. Lists are concatenated in constant time.

Set $h = \lceil \log_3(2/\varepsilon) \rceil$. At ranks at most h, refill in exactly the ordinary way. At a higher rank, pull two consecutive minimum representatives e_1, e_2 from the children, refilling after each pull. Their keys satisfy $\mathrm{key}(e_1) < \mathrm{key}(e_2)$. Keep e_2 and concatenate $C(e_2)$, $\{e_1\}$, and $C(e_1)$ into its corruption list. Only e_1 is newly corrupted; the other list members were already corrupted. If just one representative remains, keep it without combining.

This is the saving: two representatives arrive from below, but only one continues upward as a representative. The discarded ordering information is recorded explicitly as corruption.

Use a Boolean argument soft to distinguish deletion repairs from insertion or meld repairs. The latter always make single pulls, even at high ranks. In the following routines, $\mathcal{R}$ collects newly corrupted items during the current extraction; $\perp$ denotes an absent item.

Algorithm 41 Pulling representatives and refilling an empty node

```
1:  function Pull(v, soft, R)
2:      Choose a child u of v with minimum representative key
3:      if no child has a representative then
4:          return ⊥
5:      e ← u.rep; u.rep ← ⊥
6:      Fill(u, soft, R)
7:      return e                                   ▷ Move its entire pool with it.

8:  procedure Fill(v, soft, R)
9:      e ← Pull(v, soft)
10:     if e ≠ ⊥ and soft and rank(v) > h then
11:         f ← Pull(v, soft, R)
12:         if f ≠ ⊥ then
13:             C(f) ← C(f) ‖ [e] ‖ C(e)
14:             Append e to R; e ← f
15:     v.rep ← e

16: function Link(u1, u2, u3)
17:     Create an empty parent v of rank rank(u1) + 1
18:     Give v children u1, u2, u3
19:     Fill(v, false, ∅)
20:     return v
```

The symbol ‖ denotes linked-list concatenation. Only representatives' list headers are live; after splicing $C(e)$ into $C(f)$, the old header can be cleared. At a leaf, Pull returns $\perp$. Every call to Fill has an empty node as its input.

Lemma 401 (Ordering invariant)**.** *Representatives are heap ordered by their original keys. Every item in a representative's corruption list has original key at most its representative's key, and no item's current key decreases.*

Proof. A single pull removes the smallest child representative and replaces it by one whose key is no smaller. It therefore produces a representative no larger than any left in the children. For a double pull, the second key is no smaller than the first, and every remaining child key is at least the second. Attaching the first pool to the second raises its common key and leaves the second pool's key unchanged. Apply this argument inductively to the recursive repairs. Moving a pool upward without combining it changes no current key. □

10.9.3 Root Lists and Heap Operations

At each root-list position store a pointer to a minimum representative in the suffix beginning there. The first pointer finds the global minimum in constant time. If a root of rank r changes, recompute suffix minima backward from that position; at most $2(r+1)$ positions are affected. A pool member has the same current key as its representative, so find-min may return any member of the minimum pool.

To meld, merge the sorted root lists and carry whenever three equal ranks meet. Stop once one input list and the carry are empty: the other input's remaining suffix is unchanged and can be spliced in. Only the newly written prefix needs new suffix-minimum pointers. An absent list head or carry has rank $+\infty$ for the rank comparison below.

Algorithm 42 Melding and inserting

```
1:  function MELD(L_1, L_2)
2:      P ← empty list; c ← ⊥
3:      while (L_1 ≠ ∅ and L_2 ≠ ∅) or c ≠ ⊥ do
4:          r ← minimum rank among the input heads and c
5:          Remove all rank-r roots from the two heads and carry into a list G
6:          Clear the carry if it was removed
7:          if |G| ≥ 3 then
8:              Remove three roots u_1, u_2, u_3 from G
9:              c ← LINK(u_1, u_2, u_3)
10:         Append the remaining roots of G to P
11:     Splice the remaining input suffix after P
12:     Recompute suffix minima backward on P only
13:     return the resulting root list

14: procedure INSERT(L, e)
15:     C(e) ← ∅; create a rank-zero root v holding e
16:     L ← MELD(L, [v])
```

Algorithm 43 Extracting a current minimum

```
1: function ExtractMin(L)
2:     if L = ∅ then
3:         return Empty
4:     v ← root named by the first suffix-minimum pointer
5:     e ← v.rep; κ ← key(e)
6:     if C(e) ≠ ∅ then
7:         Remove an item f from C(e); return (f, κ, ∅)
8:     v.rep ← ⊥; R ← ∅
9:     Fill(v, true, R)
10:    Remove v from L if its subtree is empty
11:    Repair suffix minima backward through the changed prefix
12:    return (e, κ, R)
```

A rank group has at most five roots: two from each heap and one carry. Consequently one link leaves at most two roots and one new carry. Find-min uses the same first three nonempty-heap steps as ExtractMin, returning a pool member and κ without modifying anything.

10.9.4 Why Only Few Items Are Corrupted

Let c_r bound a corruption list at a rank-r node. Below the threshold there are no corruptions. Above it, a double pull combines two lists and one newly corrupted representative, whereas a single pull only moves a list:

$$c_r = 0 \quad (r \le h), \qquad c_r \le 2c_{r-1} + 1 \quad (r > h).$$

Hence $c_{h+j} \le 2^j - 1$. There are at most $N/3^r$ nodes of rank r, even when all nodes over the whole history are counted. Pools partition the present items, so the number of corrupted items is at most

$$\sum_{j \ge 1} \frac{N}{3^{h+j}}(2^j - 1) \le \frac{N}{3^h} \sum_{j \ge 1} (2/3)^j = \frac{2N}{3^h} \le \varepsilon N.$$

This explains the ternary branching: pool sizes grow by at most a factor 2 per level, while the number of available nodes falls by a factor 3. Binary branching would not make this geometric sum converge.

10.9.5 Amortized Cost

Insertions and melds. A link producing rank $r + 1$ makes only single pulls and costs $O(r + 1)$. Root-list merging and suffix rebuilding cost the scanned prefix length. These costs are paid by a constant multiple of the nonnegative potential

$$\sum_{\text{heaps } H} \left(2 \sum_{v \text{ root of } H} (\text{rank}(v) + 1) + R_H + 1 \right),$$

where R_H is the maximum root rank and is -1 for an empty heap. A link of three rank-r roots decreases the weighted root sum by $4r + 2$, and can increase the maximum-rank term by at most one. This pays for its pull path and for list positions processed by a carry. Before carries, merging two nonempty heaps releases $\min\{R_{H_1}, R_{H_2}\} + 1$ from the maximum-rank terms. This pays for merging the prefixes through the smaller maximum rank. Any further processing is a carry chain already charged to links. Creating a singleton increases the potential by a constant. Thus insert and meld have constant amortized cost. Extractions only remove roots, so cannot increase this potential.

Refilling below the threshold. A maximal single-pull path at ranks at most h costs $O(h+1)$. It is started either by an extraction at a low root or by moving one representative from rank h to rank $h+1$. Distinct such boundary crossings use distinct original items, because representatives only move upward. Every crossed item has either been extracted, remains a representative at a high node, or remains corrupted. There are at most D of the first kind, at most $\sum_{r>h} N/3^r = O(N/3^h)$ of the second, and $O(N/3^h)$ of the third by the preceding calculation. Thus all low-rank work costs $O((h+1)(D+N/3^h)) = O(N + D(h+1))$.

Refilling above the threshold. Every double fill newly corrupts one representative, and an item can lose representative status only once. There are therefore at most N double fills. A high node can have an unpaired final fill at most once, when the last representative leaves its children. Empty final repairs can likewise be charged to nodes. Since only $O(N)$ nodes are ever created, the local work at high nodes is $O(N)$. Recursive low-rank work was counted separately.

Repairing root minima. The first $h+1$ ranks cost $O(h+1)$ per extraction. To charge the additional cost at a high root, let U_r be the number of representatives ever placed at rank r, counting all nodes over the entire execution. We have $U_h \le N$. A usual high-rank placement consumes two representatives from rank $r-1$. There are only $O(N/3^r)$ exceptions: final unpaired fills, and exact fills caused by linking at rank r or higher. For the latter, each rank-s link causes at most one exact placement at rank r, and $\sum_{s\ge r} N/3^s = O(N/3^r)$. Consequently

$$U_r \le \tfrac{1}{2}U_{r-1} + O(N/3^r), \qquad U_{h+j} = O(N/2^j).$$

The recurrence follows by counting consumed representatives; its solution follows by summing the geometric error terms. Root removals contribute at most one additional change per historical node. Charging the extra $O(j)$ root-list repairs to these changes gives

$$O\left(N \sum_{j\ge 1} \frac{j}{2^j}\right) = O(N).$$

Together these estimates prove Theorem 400.

The construction does not need to maintain a nearly sorted order on every item. It keeps exact order on representatives, lets the pools share their largest key, and makes the number of representatives shrink faster than the cost of moving through the upper levels grows.

10.9.6 Selecting from a Binary Min-Heap

Theorem 402 (Heap selection)**.** *Given the root of a binary min-heap of at least k items, its k smallest items can be found, not necessarily in sorted order, in $O(k)$ time.*

Proof. The usual algorithm repeatedly extracts the smallest exposed node and inserts its children, taking $O(k \log k)$ time. Replace its priority queue by a (1/4)-soft heap. To compensate for corruption, expose the children of a node as soon as it is *extracted uncorrupted* or *becomes corrupt*. Keep every exposed node in a candidate list S, and after $k-1$ extractions select the k smallest candidates by ordinary linear-time selection.

Algorithm 44 Selecting the k smallest nodes of a binary min-heap

```
1: function HeapSelect(r, k)
2:     Q ← new (1/4)-soft heap; S ← [r]
3:     Insert(Q, r)
4:     for i = 1, ..., k − 1 do
5:         (e, κ, R) ← ExtractMin(Q)
6:         if e is not corrupt then
7:             R ← R ∪ {e}
8:         for v ∈ R do
9:             for each existing child w of v do
10:                Insert(Q, w); append w to S
11:    return the k smallest items of S by linear-time selection
```

For correctness, let B be the *frontier* of exposed items that are still in Q and uncorrupted. If B is empty, every node has already been exposed, and we are done. Otherwise every unexposed node has an ancestor in B: whenever a frontier node is extracted or corrupted, its children replace it on the frontier. Moreover, the minimum original key β on B never decreases, since newly exposed nodes are children of former frontier nodes. Each extraction removes an item with original key at most β, because every frontier key is still exact. After $k-1$ extractions, the current minimum frontier item and the $k-1$ extracted items therefore give k distinct items with keys at most β. But every unexposed node has key at least β. Thus all k smallest items have been exposed and belong to S.

It remains to bound $|S|$. Let I be the number of insertions and C the number of items ever corrupted. Each node is exposed only once, and only children of the $k-1$ extracted or C corrupted nodes are added:

$$I \le 1 + 2(k - 1 + C), \qquad C \le (k-1) + \tfrac{1}{4} I.$$

The second inequality holds because at most $I/4$ corrupt items remain in Q, while at most $k-1$ have been extracted. Consequently $I < 8k$. All soft-heap operations and the final selection therefore take $O(k)$ time. □

10.9.7 Selecting from Sorted Lists

Theorem 403 (Sorted-list selection)**.** *Given m sorted lists with constant-time indexed access and $1 \le k$ at most their total length, the k smallest elements can be represented by their prefix lengths $(k_1, \dots, k_m)$ in*

$$O\big(m \log(2 + k/m)\big)$$

time. More precisely, the bound can be improved to $O\big(m + \sum_{i=1}^{m} \log(k_i + 1)\big)$. Reporting all k elements explicitly takes an additional $O(k)$ time.

Proof. First, an $O(m+k)$ algorithm follows directly from heap selection. View each list as a heap-ordered path, and join their first elements under a binary tree of $m-1$ dummy nodes with key $-\infty$. Selecting the $m+k-1$ smallest nodes in this tree gives the answer.

To avoid the linear dependence on k, suppose $k > 2m$ and split each remaining list into consecutive blocks of size $b = \lfloor k/(2m) \rfloor$, represented by their last elements. Among the true k smallest items, at most $m(b-1)$ belong to incomplete blocks. Thus there are at least

$$\frac{k - m(b-1)}{b} \ge m$$

complete blocks. Selecting the m smallest block representatives takes $O(m)$ time by the preceding algorithm and identifies m blocks contained entirely among the desired items. Accept these $mb \ge k/4$ items, advance the corresponding list prefixes, and recurse on the remaining $k - mb$ items. Each round costs $O(m)$ and reduces k by a constant factor, proving the first bound.

For the stronger bound, we first record a useful refinement: if the remaining lists have lengths $n_1, \ldots, n_m$, selection takes $O(m + \sum_i \log(n_i + 1))$ time. Indeed, in the preceding block algorithm, let s be the number still to select and, when $s > 4m$, ignore lists shorter than $\lfloor s/(2m) \rfloor$. Together they contain fewer than $s/2$ items. If q lists remain, use blocks of size $b' = \lfloor s/(4q) \rfloor$ and select their q smallest representatives. At least $s/2$ of the desired items lie in these lists, so the same complete-block argument shows that all q selected blocks are safe. We accept $qb' \geq s/8$ items in $O(q)$ time. Since s decreases geometrically, a list of initial length n_i can participate only in the last $O(\log(n_i + 1))$ rounds: participation requires $s < 2m(n_i + 1)$. Bucketing lists by the powers of two of their remaining lengths lets us maintain the eligible lists in time proportional to the number participating. The final round, with $s \leq 4m$, costs $O(m)$.

Now partition each original list into blocks of sizes $1, 2, 4, \ldots$, and again use block maxima as representatives. Put $L = \sum_i \lfloor \log_2(k_i + 1) \rfloor$; exactly L representatives belong to the true k smallest items. Guess an upper bound $\ell \geq L$ and select the ℓ smallest representatives in $O(m + \ell)$ time. If ℓ_i of them come from list i, retain only its first $n_i = \min\{|A_i|, 2^{\ell_i + 1} - 1\}$ items. These prefixes contain the true solution, since $n_i \geq k_i$ whenever $\ell \geq L$, and $\sum_i \log(n_i + 1) = O(m + \ell)$. The length-sensitive algorithm therefore finds the correct prefix lengths in $O(m + \ell)$ time. We can verify them in $O(m)$ time by comparing the largest selected item against the first unselected item of each original list. Trying $\ell = m, 2m, 4m, \ldots$ until verification succeeds costs $O(m + L) = O(m + \sum_i \log(k_i + 1))$ in total. □

10.9.8 Selecting the Smallest Pairwise Sums

Theorem 404 (Selection from $X+Y$)**.** *For unsorted arrays X, Y of lengths m, n, respectively, and $1 \leq k \leq mn$, the k smallest sums $x_i + y_j$, counting pairs with multiplicity, can be found in $O(m + n + k)$ time. If X and Y are already min-heaps, the time is $O(k)$.*

Proof. Heapify both arrays in $O(m + n)$ time. The obstacle is that the $m \times n$ sum matrix is not explicitly available, but its entries form a constant-degree *implicit min-heap*. Identify each pair (i, j) with the key $x_i + y_j$, rooted at $(1, 1)$, and give it children

$$\begin{array}{ll} (2i, 1), (2i + 1, 1), (i, 2), (i, 3), & j = 1, \\ (i, 2j), (i, 2j + 1), & j > 1, \end{array}$$

omitting out-of-range indices. Every pair other than $(1, 1)$ appears exactly once as a child: $(i, 1)$ has parent $(\lfloor i/2 \rfloor, 1)$ for $i > 1$, and (i, j) with $j > 1$ has parent $(i, 1)$ when $j \in \{2, 3\}$ and $(i, \lfloor j/2 \rfloor)$ otherwise. Heap order of X and Y ensures that no child has a smaller sum than its parent.

Apply the preceding heap-selection algorithm, expanding at most four children per node and taking the soft-heap error parameter $\varepsilon = 1/8$. The same analysis gives $I \leq 4(k + C)$ and $C \leq k + \varepsilon I$, hence $I = O(k)$. The implicit heap is never constructed, and generating each child takes constant time. □

10.10 Sampling

Write UniformInt(N) for an exact uniform integer in $\{0, \ldots, N - 1\}$. Continuous-key algorithms below are stated in the real-arithmetic model.

10.10.1 Alias Tables

Let $w_1, \ldots, w_n$ be nonnegative integer weights with $W = \sum_i w_i > 0$. We want to return i with probability w_i/W.

A prefix-sum structure locates a random point in the total mass by searching. An alias table instead rearranges the same mass into n equal columns, each containing at most two outcomes. Sampling then requires only a uniform column and one threshold comparison.

Scale the mass of item i to $b_i = nw_i$, and give every column capacity W. Total scaled mass and total capacity are both nW. If $b_s < W$, fill column s with all b_s units of item s and the remaining $W - b_s$ units from some item ℓ with $b_\ell \geq W$. The latter becomes the alias of s.

Algorithm 45 Building and sampling an integer alias table

```
function BuildAlias(w_1, ..., w_n)
    W ← Σ_i w_i; require W > 0
    b_i ← n w_i for every i
    S ← {i : b_i < W}, L ← {i : b_i ≥ W}
    while S ≠ ∅ and L ≠ ∅ do
        remove s from S and ℓ from L
        cut[s] ← b_s
        alias[s] ← ℓ
        b_ℓ ← b_ℓ − (W − b_s)
        if b_ℓ < W then
            insert ℓ into S
        else
            insert ℓ into L
    for each remaining i ∈ S ∪ L do
        cut[i] ← W
        alias[i] ← i
    return (W, cut, alias)

function AliasSample(W, cut, alias)
    n ← |cut|
    j ← 1 + UniformInt(n)
    r ← UniformInt(W)
    if r < cut[j] then
        return j
    else
        return alias[j]
```

Theorem 405 (Alias sampling)**.** *The alias table uses $O(n)$ preprocessing time and space and returns item i with probability w_i/W using $O(1)$ table operations per sample.*

Proof. If r columns remain, their unassigned masses sum to rW. Filling one column removes exactly W mass and one column, preserving the invariant. Hence a mass below W cannot remain unless some mass is at least W. If no small mass remains, every remaining mass must equal W, since their average is W.

Every item's original nw_i units are therefore placed into the n columns without changing their identity. Sampling a uniform column and then a uniform integer position in that column chooses each of the nW positions with probability $1/(nW)$. Exactly nw_i positions contain item i, so its probability is w_i/W.

Each iteration finishes one column, giving linear preprocessing. □

10.10.2 Exponential Races

Independent exponential clocks convert weighted sampling into taking minimums.

For each positive weight w_i, draw an independent $X_i \sim \mathrm{Exp}(1)$ and put

$$E_i = \frac{X_i}{w_i}.$$

Then E_i is exponential with rate w_i, since $\Pr[E_i > t] = e^{-w_i t}$.

Theorem 406 (Exponential race)**.** *Let $W = \sum_i w_i$. Then*

$$\boxed{\Pr[E_i = \min_j E_j] = \frac{w_i}{W}.}$$

More generally, if $i_1, \dots, i_k$ are the indices of the k smallest clocks in increasing order, then for distinct indices

$$\boxed{\Pr[i_1, \dots, i_k] = \prod_{r=1}^{k} \frac{w_{i_r}}{W - \sum_{\ell<r} w_{i_\ell}}.}$$

Thus sorting the independent clocks gives successive proportional sampling without replacement.

Proof. The density of E_i at t is $w_i e^{-w_i t}$. Hence

$$\begin{aligned}\Pr[E_i = \min_j E_j] &= \int_0^\infty w_i e^{-w_i t} \prod_{j \neq i} e^{-w_j t}\, dt \\ &= \int_0^\infty w_i e^{-W t}\, dt = \frac{w_i}{W}.\end{aligned}$$

After the first clock at time t, condition each remaining clock on exceeding t. Exponentials are memoryless:

$$\Pr[E_j - t > s \mid E_j > t] = e^{-w_j s}.$$

Thus the residual clocks are again independent exponentials with their original rates. Repeating the first-clock argument proves the ordered formula. □

This representation is valuable algorithmically because only the smallest keys matter. In a stream, the k smallest clocks can be maintained in $O(k)$ space and $O(n \log(k+1))$ time with a max-heap.

It is also automatically mergeable.

Proposition 407 (Mergeability of bottom-k keys)**.** *Partition the items into disjoint batches. If each batch retains its k smallest keys, then the global k smallest keys are the k smallest among the retained keys.*

Proof. Any item discarded by its batch already has at least k smaller keys in that same batch, so it cannot belong to the global k smallest. □

10.10.3 Priority Sampling

Exponential races are designed to sample items. A different random-key construction is useful when the goal is to retain only k weighted items while later estimating the total weight of *any* requested subset.

Let items have nonnegative weights w_i. Independently draw U_i uniformly from $(0, 1)$, and assign priority

$$q_i = \frac{w_i}{U_i}.$$

Keep the k largest priorities. Let τ be the $(k+1)$-st largest priority. For a retained item define

$$\widehat{w}_i = \max\{w_i, \tau\},$$

and for an omitted item put $\widehat{w}_i = 0$.

If there are at most k positive-weight items, retain all of them and simply set $\widehat{w}_i = w_i$.

Theorem 408 (Priority sampling)**.** *For every item i,*

$$\boxed{\mathbb{E}[\widehat{w}_i] = w_i.}$$

Consequently, for every subset A,

$$\boxed{\widehat{W}(A) = \sum_{i \in A} \widehat{w}_i}$$

is an unbiased estimator of $W(A) = \sum_{i \in A} w_i$.

Proof. Fix i, and condition on all priorities except q_i. Let τ_i be the k-th largest priority among the other items. Then i is retained exactly when $q_i > \tau_i$. If it is retained, the global $(k+1)$-st priority is exactly τ_i.

If $\tau_i \le w_i$, then $q_i = w_i/U_i \ge w_i \ge \tau_i$ always, so i is retained and $\widehat{w}_i = w_i$.

Suppose instead that $\tau_i > w_i$. Then

$$q_i > \tau_i \iff U_i < \frac{w_i}{\tau_i},$$

which occurs with probability w_i/τ_i. When this occurs, $\widehat{w}_i = \tau_i$; otherwise it is zero. Therefore, conditioned on the other priorities,

$$\mathbb{E}[\widehat{w}_i] = \tau_i \frac{w_i}{\tau_i} = w_i.$$

Removing the conditioning proves the first claim, and linearity of expectation proves the subset-sum claim. □

The estimator has a useful interpretation. Heavy items with $w_i \ge \tau$ are represented by their exact weight. Lighter retained items all receive the common threshold estimate τ. Thus the sample spends its limited capacity on heavy items while randomly representing the tail.

Algorithm 46 Priority sampling

```
function PrioritySample((w_i)_{i=1}^n, k)
    H ← an empty min-heap of capacity k + 1
    for i = 1, ..., n with w_i > 0 do
        draw U_i uniformly from (0, 1)
        q_i ← w_i / U_i
        insert (q_i, i, w_i) into H
        if |H| > k + 1 then
            delete the minimum-priority item from H
    if |H| ≤ k then
        return every retained item with estimate w_i
    remove the minimum-priority item and let its priority be τ
    for each remaining (q_i, i, w_i) ∈ H do
        ŵ_i ← max{w_i, τ}
    return the k retained items, their estimates, and τ
```

The construction uses $O(k)$ space and $O(n \log(k+1))$ time. It is also mergeable: if the items are partitioned into disjoint batches, retaining the $k+1$ largest priorities from each batch is sufficient to recover the global $k+1$ largest priorities and therefore both the final sample and its threshold τ. The random priority of an item must be generated once and preserved; resampling priorities during a merge changes the distribution.

11 Trees

11.1 LCA

1. Pre-Order Reduction Let $\texttt{dfs}[0 \dots n-1]$ be the sequence of nodes visited in a standard pre-order DFS traversal. Let $\texttt{pos}[u]$ be the pre-order index of node u, such that $\texttt{dfs}[\texttt{pos}[u]] = u$. Let $p[u]$ be the parent of node u, and let $\text{depth}[u]$ be its distance from the root.

We construct our $O(n)$ RMQ from Algorithm 38 over an implicit array Z of size n, defined by the depth of the parent of the node at each pre-order step:

$$Z[i] = \text{depth}[p[\texttt{dfs}[i]]]$$

2. Queries To find $LCA(a, b)$, we first ensure that a is visited before b by swapping them if $\texttt{pos}[a] > \texttt{pos}[b]$. If $a = b$, the LCA is simply a. Otherwise, we query the RMQ for the index m that minimizes Z in the range $[\texttt{pos}[a] + 1, \texttt{pos}[b]]$. The LCA is exactly the parent of the node at that minimum index: $p[\texttt{dfs}[m]]$.

3. Correctness There are two cases:

- **Case 1 (a is an ancestor of b):** The pre-order interval $[\texttt{pos}[a] + 1, \texttt{pos}[b]]$ contains only proper descendants of a. The node in this interval with the shallowest parent is the direct child of a that lies on the path to b. The parent of this child is a.
- **Case 2 (a and b are in separate branches):** Let $L = LCA(a, b)$. After the DFS visits a and finishes its subtree, it backtracks to L and eventually descends into the branch containing b. Let v be the direct child of L that roots the subtree containing b. The node v is visited after a and before b, so $\texttt{pos}[v] \in [\texttt{pos}[a] + 1, \texttt{pos}[b]]$. Its parent is L. Because every other node visited in this interval is a descendant of L, their parents have a depth of at least depth$[L]$. Thus, the RMQ identifies v, and returns its parent $p[v] = L$.

Algorithm 47 LCA

```
1:  function InitLCA(n, p, dfs, pos, depth)
                                  ▷ Define a closure to compare the parent depths of two pre-order indices
2:      cmp ← Closure(i, j) returning depth[p[dfs[i]]] < depth[p[dfs[j]]]
                                  ▷ Initialize the RMQ data structure using this closure
3:      rmq ← RMQ_Init(n, cmp)
4:      return LCA_Struct{p, dfs, pos, rmq}
5:  function QueryLCA(a, b)
6:      if a = b then
7:          return a
                                  ▷ Order by entry time
8:      l ← min(pos[a], pos[b])
9:      r ← max(pos[a], pos[b])
10:     m ← rmq.query(l + 1, r)
11:     return p[dfs[m]]
```

11.2 $O(n)$ Binary Lifting

Binary Lifting usually requires $O(n \log n)$ time and space to precompute an ancestor table, but using a modified method we can do it in $O(n)$ preprocessing and space.

1. Preprocessing We define $\texttt{jmp}[v]$ as a pointer to an ancestor of v. The idea is to grow the jump distances in powers of two as we ascend.

During a pre-order traversal, consider a node v with parent p. We look at the jump from the parent $p \to \texttt{jmp}[p]$, and the subsequent jump $\texttt{jmp}[p] \to \texttt{jmp}[\texttt{jmp}[p]]$. If these two jumps cover the exact same change in depth, we combine them. The new jump from v covers the edge (v, p), the first jump of length L, and the second jump of length L. Thus, the new jump pointer points to $\texttt{jmp}[\texttt{jmp}[p]]$ and covers a distance of $1 + 2L$.

If the two preceding jumps do not match in length, we set $\texttt{jmp}[v] = p$, resetting the jump length to 1.

2. Querying The algorithm has two phases:

1. **Equate Depths:** If u and v are at different depths, we identify the deeper node and use our `depth_jmp` function to lift it exactly to the depth of the shallower node, this works basically the same as the LCA function. If the nodes are now identical, we have found the LCA.

2. **Simultaneous Ascent:** If the nodes are distinct but at the same depth, we ascend them simultaneously. We evaluate the jump pointers $\texttt{jmp}[u]$ and $\texttt{jmp}[v]$:

   - If $\texttt{jmp}[u] \neq \texttt{jmp}[v]$, the jump lands strictly below the LCA. We can safely take this maximal jump: $u \leftarrow \texttt{jmp}[u]$ and $v \leftarrow \texttt{jmp}[v]$.
   - If $\texttt{jmp}[u] = \texttt{jmp}[v]$, the jump has either hit the LCA exactly or overshot it into a higher common ancestor. Because we cannot distinguish between these two cases, we take a single step up to the parents: $u \leftarrow p[u]$ and $v \leftarrow p[v]$.

We repeat this until $u = v$, at which point u is the LCA.

3. Complexity Let $L(u) = \text{depth}[u] - \text{depth}[\texttt{jmp}[u]]$. By construction, $L(u)$ is always of the form $2^k - 1$ for some integer $k \geq 1$. We call k the rank of the jump. A jump of rank k implicitly spans a tree of $2^k - 1$ edges.

Lemma 409 (Monotonicity of Jump Sequences)**.** *For any node u, let $J(u) = [L_1, L_2, \ldots, L_m]$ be the sequence of jump lengths encountered by repeatedly following the `jmp` pointers from u to the root. The sequence $J(u)$ is monotonically non-decreasing.*

Proof. The sequence of jumps from a node depends entirely on its depth d. Let J_d denote the sequence of jump lengths from a node at depth d.

By induction, all jump lengths are drawn from the set of skew-binary weights: $\mathcal{W} = \{2^k - 1 \mid k \geq 1\}$.

We will prove by induction on depth d that:

$$J_d \text{ satisfies } L_1 \leq L_2 < L_3 < \cdots < L_m.$$

Base Case ($d = 1$):
Trivial.

Inductive Step:
Assume J_d satisfies the invariant. We consider J_{d+1}:

- **Case A:** J_d does *not* start with identical elements.
  By the invariant, we have $L_1 < L_2 < L_3 < \ldots$
  The algorithm prepends a jump of 1, making $J_{d+1} = [1, L_1, L_2, \ldots]$.
  Because all elements are ≥ 1, we have the invariant.

- **Case B:** J_d starts with two identical elements.
  By the invariant, we have: $J_d = [L, L, L_3, L_4, \ldots]$ with $L < L_3 < L_4 \ldots$
  The algorithm merges the first two jumps, replacing them with $2L{+}1$, making $J_{d+1} = [2L{+}1, L_3, L_4, \ldots]$.
  Because $L \in \mathcal{W}$, the next available weight in $\mathcal{W}$ is $2L{+}1$. Since $L_3 \in \mathcal{W}$ and $L_3 > L$, we have $L_3 \geq 2L{+}1$.
  Therefore, the new sequence satisfies the invariant again.

Thus J_d is always monotonically non-decreasing for any depth d. □

By the lemma, $J(u)$ can contain at most two jumps of any length $2^k - 1$. Thus the entire sequence $J(u)$ consists of at most $2\lfloor \log_2 n \rfloor$ jumps.

During a query, we attempt to follow $J(u)$ to cover a target distance. Every operation falls into one of two categories:

1. Successful Jumps:
When we take $\texttt{jmp}[u]$, we consume one of the elements of $J(u)$, so there are at most $O(\log n)$ of these operations.

2. Penalty Steps:
If taking the jump L would overshoot our target we cannot consume it. Instead, we take a penalty step to the parent p. Because $L > 1$ was built by merging a step of 1 and two jumps of size $(L-1)/2$, stepping to the parent "unpacks" L. The new sequence from the parent, $J(p)$, replaces L with two smaller jumps: $[(L-1)/2, (L-1)/2, \dots]$. We only unpack a jump of size L when it is greater than the remaining distance. Because the remaining distance strictly decreases, we will never need to unpack a jump of size L more than once across the query. Thus, the total number of penalty steps is bounded by the number of unique jump sizes, which is $\leq \log_2 n$.

Algorithm 48 Binary Lifting

```
1:  function BuildJumpTable(n, par, dfs, depth)
2:      jmp ← array of size n
3:      for each v in dfs do
4:          p ← par[v]
5:          if v = p then
6:              jmp[v] ← v
7:          else
8:              p_j ← jmp[p]
9:              p_jj ← jmp[p_j]
10:             if depth[p] − depth[p_j] = depth[p_j] − depth[p_jj] then
11:                 jmp[v] ← p_jj
12:             else
13:                 jmp[v] ← p
14:     return jmp
15: function QueryLCA(u, v, par, jmp, depth)
16:     if depth[u] > depth[v] then
17:         Swap u and v
18:     v ← DepthJump(v, depth[u], par, jmp, depth)
19:     while u ≠ v do
                                                  ▷ We would overshot the LCA.
20:         if jmp[u] = jmp[v] then
21:             u ← par[u]
22:             v ← par[v]
23:         else
24:             u ← jmp[u]
25:             v ← jmp[v]
26:     return u
```

11.3 Virtual Tree

Let T be a rooted tree. Suppose a query specifies a set

$$S = \{v_1, \dots, v_k\}$$

of important vertices.

A *virtual tree* is the smallest compressed tree containing all vertices of S while preserving their ancestor relationships and distances in T.

Preprocess:

- DFS entry times $\text{tin}(v)$;

- depths or root distances;
- an LCA data structure.

Construction Sort the query vertices by DFS order:

$$v_1, \dots, v_k.$$

Add

$$\mathrm{LCA}(v_i, v_{i+1}) \qquad (1 \le i < k).$$

Sort and deduplicate the resulting set U by DFS order.

Now scan U with a stack. Before inserting v, pop until the stack top is an ancestor of v. Connect v to that stack top and push v.

Algorithm 49 Virtual Tree

Input: Query vertices S
Output: Virtual tree on S and necessary LCAs

```
sort S by tin
U ← S
for i = 1, ..., |S| − 1 do
    add LCA(S_i, S_{i+1}) to U
sort and deduplicate U by tin
stack ← empty
for v ∈ U in DFS order do
    while stack nonempty and top is not an ancestor of v do
        pop stack
    if stack nonempty then
        add virtual edge (top, v)
        assign its weight
            dist_T(top, v)
    push v
```

Lemma 410. *The set U contains every branching vertex required by the minimal subtree spanning S.*

Proof. Any branching vertex x of the spanning subtree is the LCA of two marked vertices lying in two different child subtrees of x.

Take the last marked vertex in DFS order from one such child subtree and the first marked vertex from the next nonempty child subtree. They are consecutive among the marked vertices lying across that boundary, and their LCA is exactly x.

Thus every necessary branching LCA occurs among LCAs of consecutive marked vertices. □

Lemma 411. *After sorting U, the parent of a vertex v in the virtual tree is the deepest earlier vertex of U that is an ancestor of v.*

Proof. DFS order makes the descendants of every vertex contiguous. The stack contains exactly the chain of currently open ancestors. Popping until the top contains v therefore leaves precisely its deepest ancestor in U. □

There are only

$$k - 1$$

added LCAs, so

$$|U| \le 2k - 1.$$

Therefore construction takes

$$O(k \log k)$$

time with $O(1)$ LCA queries, or

$$O(k \log k + k \log n)$$

with binary-lifting LCA.

11.4 XOR-Linked Traversal

XOR-Linked Traversal is a method to traverse a tree bottom-up iteratively with better constant factor.

We store:

- $d[u]$: The degree of node u.
- $p[u]$: The XOR sum of the indices of all neighbors of node u. Initially, $p[u] = \bigoplus_{v \in \text{neighbors}(u)} v$.

If a node u is a leaf, its degree is 1. Therefore, $p[u]$ is simply the index of its only neighbor, v. We can process the edge (u, v), and then delete the edge from the graph using $p[v] \leftarrow p[v] \oplus u$ and decrementing its degree.

If decrementing v's degree makes it a new leaf, we set $u \leftarrow v$ and continue.

Algorithm 50 XOR-Linked Traversal

Input: Number of nodes N, Initial XOR-sum array p, Initial degree array d
Output: Updated arrays p and d, processing nodes bottom-up

```
for i ← 0 to N − 1 do
    u ← i
    while d[u] = 1 do
        d[u] ← 0
        v ← p[u]
        Process(u, v)
        p[v] ← p[v] ⊕ u
        d[v] ← d[v] − 1
        u ← v
return p, d
```

11.5 DFS

The algorithm constructs a DFS order, alongside subtree sizes and depths, with a better constant factor.

Algorithm 51 DFS

Input: Number of nodes N, Parent array p, Degree array d
Output: Arrays $p, ss, pos, dfs, depth$

Initialize $ss \leftarrow [1, \ldots, 1]$ of size N
Initialize $dfs \leftarrow [0, \ldots, 0]$ of size N
Initialize $depth \leftarrow [0, \ldots, 0]$ of size N
$idx \leftarrow N - 1$

for all edge (u, v) processed via XOR-Linked Traversal **do**
    $ss[v] \leftarrow ss[v] + ss[u]$
    $dfs[idx] \leftarrow u$
    $idx \leftarrow idx - 1$

$pos \leftarrow ss$
for $i \leftarrow 1$ **to** $N - 1$ **do**
    $u \leftarrow dfs[i]$
    $v \leftarrow p[u]$
    $depth[u] \leftarrow depth[v] + 1$
    $temp \leftarrow pos[v]$
    $pos[v] \leftarrow pos[v] - pos[u]$
    $pos[u] \leftarrow temp$

for $i \leftarrow 0$ **to** $N - 1$ **do**
    $pos[i] \leftarrow pos[i] - 1$
    $dfs[pos[i]] \leftarrow i$
return $p, ss, pos, dfs, depth$

1. $ss[v] \leftarrow ss[v] + ss[u]$ calculates the subtree size for every node.
2. We iterate top-down using the order in dfs. We initialize a pos array with the subtree sizes ss.
    - Imagine node v owns a contiguous block of indices ending at $pos[v]$.
    - It gives its child u a chunk of this block equal to u's subtree size. u's block now ends where v's block ended.
    - v's remaining block end pointer is shifted back by u's size ($pos[v] - pos[u]$).
    - As this repeats for all children of v, the children partition the space reserved for v's descendants, and v's pointer ends up pointing to the beginning of its block.
3. $pos[i]$ points to the 1-based index where node i should appear. Subtracting 1 makes it a 0-indexed DFS array, and placing i at $dfs[pos[i]]$ computes dfs.

11.6 Diameter

With the bottom-up sequence from XOR-Linked traversal, we compute the diameter of the tree using DP.

Algorithm 52 Tree Diameter

Input: Number of nodes N, Parent array p, Degree array d
Output: Integer representing the tree diameter

Initialize $max_depth \leftarrow [0, \ldots, 0]$ of size N
Initialize $diameter \leftarrow [0, \ldots, 0]$ of size N

for all edge (u, v) processed via XOR-Linked traversal **do**
    ▷ *Path through v connecting u's deepest leaf and previously processed children's deepest leaf*
    $path_through_v \leftarrow max_depth[v] + max_depth[u] + 1$
    $diameter[v] \leftarrow \max(\{diameter[v], diameter[u], path_through_v\})$
    $max_depth[v] \leftarrow \max(max_depth[v], max_depth[u] + 1)$
return $\max_{i \in [0, N-1]} diameter[i]$

The algorithm maintains two variables for each vertex v:

- $max_depth[v]$: The length of the longest path from v down to a leaf in its subtree already processed.
- $diameter[v]$: The maximum diameter completely contained within the subtree rooted at v already processed.

Because the traversal processes children before their parents, when we evaluate the edge between child u and parent v, $max_depth[u]$ and $diameter[u]$ are already computed. We update v's state by checking if joining u's deepest path to v's currently known deepest path creates a new maximum diameter for v's subtree. Finally, we update v's maximum depth to include u.

11.7 Shallowest Decomposition Tree

The Shallowest Decomposition Tree possesses the same properties as a Centroid Decomposition but can be constructed in $O(n)$ time with optimal depth.

Definition 412 (Valid Tree Labeling)**.** A valid tree labeling is an assignment of integer labels $\ell : V \to \mathbb{N}$ such that for any two distinct vertices u, v sharing the same label ($\ell(u) = \ell(v)$), the simple path between them contains at least one vertex w with a strictly greater label ($\ell(w) > \ell(u)$).

The depth of the resulting decomposition tree is exactly the maximum label assigned.

1. The DP We compute the optimal labeling in $O(n)$ time using a bottom-up DP. For each node u, we maintain a bitmask $\mathcal{F}(u)$ representing the set of forbidden labels. The k-th bit of $\mathcal{F}(u)$ is 1 if assigning label k to u would violate the labeling condition with its descendants.

The optimal label $\ell(u)$ is the smallest non-forbidden integer, corresponding to the index of the lowest unset bit in $\mathcal{F}(u)$.

When a child v passes its constraints up to its parent u, the newly assigned label $\ell(v)$ acts as a "shield." Any forbidden labels strictly smaller than $\ell(v)$ are shielded and are no longer forbidden for u. However, $\ell(v)$ itself becomes forbidden for u, and any constraints strictly greater than $\ell(v)$ pass through unaffected.

It is easy to see that the propagation is exactly:

$$\mathcal{P}(v) = \mathcal{F}(v) + 1$$

When u aggregates the propagated masks $\mathcal{P}(v)$ from all its children, it must resolve collisions:

1. **Union:** If a label is forbidden by any child, it is forbidden for u. Let $\mathcal{O}_{\text{once}} = \bigvee \mathcal{P}(v)$.
2. **Collision:** If a label k is forbidden by two or more distinct children, u must act as a shield for them by taking a label strictly greater than k. Let $\mathcal{O}_{\text{twice}}$ be the mask of bits set by at least two children.

If $\mathcal{O}_{\text{twice}} > 0$, let $k = \lfloor \log_2(\mathcal{O}_{\text{twice}}) \rfloor$ be its highest set bit. To force $\ell(u) > k$, we saturate all bits up to k by applying a mask $M_k = (1 \ll (k+1)) - 1$. The final forbidden state for u is exactly:

$$\mathcal{F}(u) = \mathcal{O}_{\text{once}} \vee M_k$$

From this, we evaluate $\ell(u) = \text{trailing_zeros}(\mathcal{F}(u) + 1)$.

2. Decomposition Chains Let $\text{anc}(x)$ denote the lowest ancestor of x in the original rooted tree such that $\ell(\text{anc}(x)) > \ell(x)$. In the decomposition tree, u becomes the direct parent of all vertices x where $\text{anc}(x) = u$.

Two vertices x and y sharing the same $\text{anc}(x) = u$ must reside in the subtree of the same child v of u. Furthermore, they must have distinct labels (if they shared a label, a higher label would exist between them, violating the premise that u is their lowest higher-labeled ancestor).

Thus, the set of vertices in v's subtree that attach to u form a strict total order: $x_1 > x_2 > \cdots > x_k$. When u is removed during decomposition, x_1 becomes the highest-labeled root of the newly isolated component, x_2 becomes its child, and so on. This vertical parent-child line in the decomposition tree is called a Decomposition Chain.

The labels present in the chain from child v to parent u are exactly the labels exposed in v's subtree that are strictly less than $\ell(u)$. Because $\mathcal{P}(v)$ encodes all exposed labels, we can extract this chain mask:

$$\text{ChainMask}(v \to u) = \mathcal{P}(v) \wedge ((1 \ll \ell(u)) - 1)$$

2. Global Chain Consider the set of vertices $S \subseteq V$ that have no higher-labeled ancestor in the original rooted tree. These vertices represent the sequence of maximum labels in the decomposition process.

By the same proof as before, the labels in S are distinct, let them be $x_1 > x_2 > \cdots > x_k$.

In the decomposition tree, the vertex x_1 (which possesses the maximum label in the entire tree) naturally becomes the root, r_{decomp}. When x_1 is removed during decomposition, x_2 becomes the highest-labeled root of its newly isolated component, thus a child of x_1. Thus the vertices in S form a chain.

3. Vertex Retrieval Knowing the labels in a chain is insufficient; we must retrieve the vertices in $O(1)$ time.

We maintain a global array of stacks, allocating one stack per possible label. During the DFS, immediately after computing $\ell(u)$, we push u onto stacks$[\ell(u)]$.

Because multiple subtrees share this global stack, we must manage the traversal order.

- **Forward:** In the DFS, we iterate through the children of u in forward order $(v_1, v_2, \ldots, v_m)$. Their vertices are recursively evaluated and pushed onto the stacks in that order. The vertices belonging to the final child v_m reside at the top of the stacks.
- **Reversed:** To extract the chains to form edges, we iterate through the children of u in reverse $(v_m, \ldots, v_1)$. Because we process the last child v_m first, the vertices defined by its ChainMask are guaranteed to be exactly at the top of their respective stacks.

For a given ChainMask, we iterate through its set bits in decreasing order (extracting the highest bit $L = \lfloor \log_2(\text{mask}) \rfloor$ first). This ensures that higher-labeled vertices become ancestors of lower-labeled vertices in the decomposition tree. We pop the vertex w from stacks$[L]$, add the directed edge $u \to w$, and update $u \leftarrow w$ to continue forming the chain downwards.

Algorithm 53 Shallowest Decomposition Tree

```
function ExtractChain(mask, u)
    while mask ≠ 0 do
        L ← ⌊log2(mask)⌋
        mask ← mask ⊕ (1 ≪ L)
        if stacks[L] ≠ ∅ then
            v ← stacks[L].pop()
            Add edge u → v to decomposition tree
            u ← v
function DFS(u, p)
    once ← 0; twice ← 0
    for all v ∈ adj[u] do
        if v ≠ p then
            DFS(v, u)
            prop ← forbidden[v] + 1
            twice ← twice ∨ (once ∧ prop)
            once ← once ∨ prop
    if twice = 0 then
        M ← 0
    else
        k ← ⌊log2(twice)⌋
        M ← (1 ≪ (k + 1)) − 1
    forbidden[u] ← once ∨ M
    label_u ← trailing_zeros(forbidden[u] + 1)
    stacks[label_u].push(u)
    for all v ∈ reversed(adj[u]) do
        if v ≠ p then
            extract_mask ← (forbidden[v] + 1) ∧ ((1 ≪ label_u) − 1)
            ExtractChain(extract_mask, u)
function ShallowestDecomp(adj, root)
    Initialize forbidden array to 0 for all nodes
    Initialize stacks as an array of empty stacks
    DFS(root, null)
    max_label ← ⌊log2(forbidden[root] + 1)⌋
    decomp_root ← stacks[max_label].pop()
    final_mask ← (forbidden[root] + 1) ∧ ((1 ≪ max_label) − 1)
    ExtractChain(final_mask, decomp_root)
    return decomp_root
```

11.8 Static Top Tree

A static top tree is best viewed as a tree expressing a rooted tree DP as a balanced expression.

The input tree itself never changes. We choose heavy children once, break the tree into heavy paths, and build a binary expression whose leaves are the original vertices. A point update changes one leaf and therefore requires recomputing only the $O(\log n)$ expression nodes above it.

The useful feature is that the expression distinguishes two kinds of partial trees:

Path	and	Point.

A Path object represents a connected piece whose distinguished direction follows one heavy path.

A Point object represents one or more completed child pieces hanging from the same vertex.

Only four operations are needed:

$$\begin{aligned} \mathsf{Vertex}_u &: \mathsf{Point} \longrightarrow \mathsf{Path}, \\ \mathsf{AddEdge} &: \mathsf{Path} \longrightarrow \mathsf{Point}, \\ \mathsf{Rake} &: \mathsf{Point} \times \mathsf{Point} \longrightarrow \mathsf{Point}, \\ \mathsf{Compress} &: \mathsf{Path} \times \mathsf{Path} \longrightarrow \mathsf{Path}. \end{aligned}$$

We also require an identity point

$$\mathbf{0}_{\mathsf{P}}$$

for Rake.

11.8.1 The Four Operations

Suppose the original tree is rooted.

For a vertex u, all light-child subtrees are first converted to Point objects and combined. Inserting u turns the resulting point object into a path object:

$$\mathsf{Vertex}_u : \mathsf{Point} \to \mathsf{Path}.$$

A completed child path becomes something hanging from its parent after we cross the parent–child edge:

$$\mathsf{AddEdge} : \mathsf{Path} \to \mathsf{Point}.$$

Several light-child contributions hanging from the same parent are combined by

$$\mathsf{Rake}.$$

Finally, consecutive pieces of one heavy path are concatenated by

$$\mathsf{Compress}.$$

Thus the recursion around a vertex u has the form

$$\begin{array}{c} \text{light child paths} \\ \downarrow\ \mathsf{AddEdge} \\ \text{point contributions} \\ \downarrow\ \mathsf{Rake} \\ \text{all light children of } u \\ \downarrow\ \mathsf{Vertex}_u \\ \text{one heavy-path piece.} \end{array}$$

Consecutive heavy-path pieces are then joined with Compress.

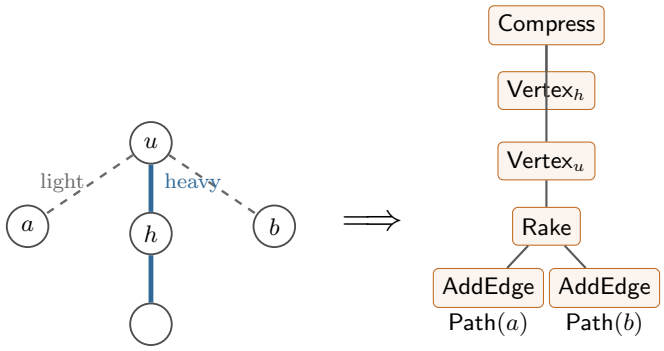


Figure 1: A vertex's light-child subtrees are converted to points and raked together; the vertex turns them back into a path piece. Heavy-path pieces are then compressed.

11.8.2 Heavy Paths

Let

$$\mathrm{sz}(u)$$

be the number of vertices in the rooted subtree of u.

For every nonleaf u, choose as its heavy child

$$h(u) \in \arg \max_{v \text{ child of } u} \mathrm{sz}(v).$$

All other child edges are light.

We use the standard fact:

Lemma 413 (Light-Edge Halving)**.** *If v is a light child of u, then*

$$\boxed{\mathrm{sz}(v) \le \frac{\mathrm{sz}(u) - 1}{2}.}$$

Consequently every root–leaf path contains at most

$$\lfloor \log_2 n \rfloor$$

light edges.

Proof. Since v is not the heavy child, there is another child $h(u)$ satisfying

$$\mathrm{sz}(h(u)) \ge \mathrm{sz}(v).$$

Hence

$$\mathrm{sz}(u) \ge 1 + \mathrm{sz}(v) + \mathrm{sz}(h(u)) \ge 1 + 2\,\mathrm{sz}(v).$$

Whenever a root–leaf path crosses a light edge, the subtree size therefore drops by at least a factor 2. Starting from at most n, this can happen at most $\lfloor \log_2 n \rfloor$ times. □

11.8.3 The Expression Tree

For every original vertex u, define

$$\lambda(u) = 1 + \sum_{\substack{v \text{ child of } u \\ v \neq h(u)}} \mathrm{sz}(v).$$

This is the number of original vertices represented by the local Vertex_u piece: u itself together with all of its light-child subtrees.

We construct the expression recursively.

For every light child v of u, recursively construct the heavy-path expression beginning at v, then wrap it in an $\mathsf{AddEdge}$ node.

Merge all such point objects with Rake, or use $\mathbf{0}_{\mathsf{P}}$ if there are none. Applying Vertex_u gives one path object of weight $\lambda(u)$.

Now follow the heavy chain

$$u,\ h(u),\ h^2(u), \ldots$$

and construct one such path object for every vertex on it. Merge them, in top-to-bottom order, with $\mathsf{Compress}$.

Thus if

$$u = u_0, u_1, \ldots, u_k$$

is a maximal heavy path,

$$\mathrm{BuildPath}(u) = \mathsf{Compress}\big(\mathsf{Vertex}_{u_0}(\cdots), \ldots, \mathsf{Vertex}_{u_k}(\cdots)\big).$$

The root expression is

$$\mathrm{BuildPath}(r).$$

Lemma 414 (Correctness of the Construction)**.** *For every heavy-path head u,*

$$\text{BuildPath}(u)$$

represents exactly the rooted subtree T_u.

In particular, the root of the expression tree represents the entire input tree.

Proof. We induct on $\text{sz}(u)$.

Let

$$u = u_0, u_1, \ldots, u_k$$

be the maximal heavy path beginning at u.

For every u_i, each light child v has strictly smaller subtree. By induction,

$$\text{BuildPath}(v)$$

represents exactly T_v. After AddEdge, it is a point contribution attached to u_i. Raking these contributions therefore represents exactly all light-child subtrees of u_i.

Applying Vertex_{u_i} adds u_i, so the resulting path object contains

$$u_i \quad + \quad \text{all light-child subtrees of } u_i.$$

The only missing descendants of u_i are those continuing through its heavy child u_{i+1}. Compressing the path pieces in order therefore adds precisely these heavy continuations.

Every vertex of T_u lies either on the heavy path or in the subtree of a unique light child of a heavy-path vertex. Hence the resulting object contains exactly T_u. □

11.8.4 Weight-Balanced Merging

A naive left-associated merge of a heavy path could have linear depth. We therefore balance every Compress and Rake expression according to the number of original vertices represented by its children.

Suppose we must merge ordered objects

$$C_1, \ldots, C_q$$

of positive weights

$$s_1, \ldots, s_q.$$

Let

$$S = \sum_i s_i.$$

Scan from left to right, maintaining

$$z = S - 2\sum_{j<i} s_j.$$

Put C_i on the left while

$$z > s_i;$$

once this fails, put C_i and all later objects on the right. Recursively merge the two sides.

Equivalently, C_i goes left precisely when the midpoint of its weighted interval lies to the left of $S/2$.

Lemma 415 (Weighted-Merge Depth)**.** *Consider an item of weight s inside a weighted merge of total weight S.*

Its depth in the resulting binary merge tree is

$$\boxed{O\left(1 + \log \frac{S}{s}\right).}$$

Proof. At a recursive call of total weight W, let C be the weight of the child containing our item.

If

$$C \le \frac{2W}{3},$$

then after one level the containing weight has shrunk by a factor at least $3/2$.

Suppose instead that

$$C > \frac{2W}{3}.$$

Assume first that the heavy child is the right side. Let $P = W - C$ be the weight of the left side, and let x be the first item placed on the right, with weight s_x.

The condition for x to be placed on the right is

$$W - 2P \le s_x.$$

Since $P = W - C$,

$$s_x \ge 2C - W.$$

Because $C > 2W/3$,

$$s_x > \frac{C}{2}.$$

When the right side is recursively split, x is its first item and has more than half its total weight. Hence x becomes a singleton child, and all remaining items lie in a child of weight

$$C - s_x \le C - (2C - W) = W - C < \frac{W}{3}.$$

Thus after at most one additional level, our item is either isolated or lies in a subproblem of weight less than $W/3$.

The case in which the left side has weight $> 2W/3$ is symmetric: the last item on the left has weight $> C/2$, becomes isolated at the next split, and all other items lie in a subproblem of weight $< W/3$.

Therefore, unless the item has already become a leaf, every two levels reduce the containing total weight by at least a constant factor.

Starting from S, this can happen only

$$O\left(\log \frac{S}{s}\right)$$

times before the containing weight is $O(s)$, at which point the item is isolated after $O(1)$ further levels. □

Theorem 416 (Static Top-Tree Height)**.** *The expression tree constructed above has*

$$\boxed{O(\log n)}$$

height and $O(n)$ nodes.

Proof. First consider the node count.

There is exactly one Vertex node for each original vertex. Every light tree edge contributes one AddEdge node.

A binary merge of q items contributes $q - 1$ internal nodes. Summed over all rake lists, this is $O(n)$, because every raked item corresponds to a distinct light child.

Likewise, summed over all heavy paths, the number of Compress nodes is

$$n - \#\{\text{heavy paths}\} = O(n).$$

Hence the expression contains $O(n)$ nodes.

Now fix an original vertex x, and follow the expression-parent chain from its Vertex node to the root.

Every time this chain passes from a child heavy path into its parent's local piece, it crosses a light represented-tree edge. By Lemma 413, this happens only $O(\log n)$ times.

Between two such light transitions, the chain passes through one weighted rake tree and one weighted compress tree. By Lemma 415, the number of merge nodes contributed is bounded by a logarithm of the ratio between the enclosing represented size and the size of the child piece.

These size ratios telescope as we move upward through nested represented pieces. Their total logarithm is at most

$$\log \frac{n}{1} = O(\log n).$$

The nonmerge constructors contribute only $O(1)$ nodes per light transition. Thus the whole expression-parent chain has length $O(\log n)$. □

Theorem 417 (Construction Time)**.** *The recursive construction above takes*

$$\boxed{O(n \log n)}$$

time and $O(n)$ space.

Once the expression structure has been built, evaluating all DP values bottom-up takes $O(n)$ time.

Proof. Computing subtree sizes and heavy children takes $O(n)$ time.

The expression itself has $O(n)$ nodes. The only superlinear work comes from the simple weighted-merge routine, which scans the current list at every recursive level.

By Lemma 415, every item occurs in $O(\log n)$ such scans. Thus all merge construction costs sum to $O(n \log n)$.

A bottom-up evaluation performs $O(1)$ work at each expression node, hence takes $O(n)$. □

11.8.5 The DP Interface

After combining the two vertex cases, a static-top-tree DP needs

$$\boxed{\begin{aligned} \mathsf{vertex} &: (\mathsf{Point}, u) \mapsto \mathsf{Path}, \\ \mathsf{compress} &: (\mathsf{Path}, \mathsf{Path}) \mapsto \mathsf{Path}, \\ \mathsf{rake} &: (\mathsf{Point}, \mathsf{Point}) \mapsto \mathsf{Point}, \\ \mathsf{addEdge} &: \mathsf{Path} \mapsto \mathsf{Point}, \end{aligned}}$$

together with $\mathbf{0}_{\mathsf{P}}$.

At a vertex node u, if there is no rake child we evaluate

$$\mathsf{vertex}(\mathbf{0}_{\mathsf{P}}, u).$$

Otherwise we evaluate

$$\mathsf{vertex}(\mathsf{point}(\mathrm{left}(u)), u).$$

11.8.6 Point Updates

Store the value of every expression node.

If the value of an original vertex u changes, start at its Vertex node and recompute every expression ancestor.

Theorem 418 (Static Top-Tree Update)**.** *A vertex update takes*

$$\boxed{O(\log n)}$$

time.

Proof. Only ancestors of the corresponding expression leaf can depend on the updated value.

By Theorem 416, there are $O(\log n)$ such ancestors, and every constructor is recomputed in $O(1)$ time. □

11.8.7 Rooted-Subtree Queries

The representation also gives subtree queries.

The Vertex node for u already contains

$$u + \text{every light-child subtree of } u.$$

The only descendants missing from T_u are therefore the vertices below u on its heavy path, together with their light subtrees.

Those are exactly the later pieces of the same Compress expression.

Let

$$x = u, \qquad R = \mathsf{path}[u].$$

While the expression parent $p(x)$ is a Compress node:

$$\begin{cases} R \leftarrow \mathsf{compress}\big(R, \mathsf{path}[r(p(x))]\big), & x = l(p(x)), \\ R \text{ unchanged}, & x = r(p(x)). \end{cases}$$

Then set

$$x \leftarrow p(x)$$

and continue.

Stop at the first non-Compress parent.

Theorem 419 (Static Rooted-Subtree Query)**.** *The preceding procedure returns the* Path *value of exactly the rooted subtree T_u, in*

$$\boxed{O(\log n)}$$

time.

Proof. The compress expression for a heavy path preserves the top-to-bottom order.

At any compress node, the left child represents an earlier, higher segment of the heavy path, and the right child represents a later, lower segment.

While climbing from the leaf corresponding to u:

If the current object is the left child, then the right sibling lies strictly below it on the same heavy path and therefore belongs to T_u. It must be appended.

If the current object is the right child, then the left sibling lies strictly above u and must be excluded.

Consequently the procedure accumulates exactly the suffix of the heavy path beginning at u. Each vertex piece on that suffix already contains all of its light-child subtrees.

Thus the accumulated represented vertices are exactly T_u.

When we reach a noncompress parent, we have reached the head of this heavy path. Crossing the next AddEdge would move to its represented parent, which lies outside T_u, so the query must stop there.

The number of expression ancestors visited is $O(\log n)$. □

11.8.8 Path Queries

The weight-balanced compress and rake trees also give

$$\boxed{O(\log n)}$$

path queries, rather than $O(\log^2 n)$.

Recall that for a vertex u,

$$\lambda(u) = 1 + \sum_{\substack{v \text{ child of } u \\ v \neq h(u)}} \mathrm{sz}(v).$$

Equivalently, if $h(u)$ exists,

$$\lambda(u) = \mathrm{sz}(u) - \mathrm{sz}(h(u)).$$

These $\lambda(u)$'s are exactly the weights used to balance the Compress tree of each heavy path.

We first record how to query a prefix of one heavy path.

Lemma 420 (Heavy-Path Prefix)**.** *Let H be a maximal heavy path with head h, and let $x \in H$. The aggregate on the path segment*

$$P(h, x)$$

can be obtained in

$$\boxed{O\left(1 + \log \frac{\mathrm{sz}(h)}{\lambda(x)}\right)}$$

time.

Proof. The leaves of the compress expression for H occur in top-to-bottom order along the heavy path.

Start at the leaf corresponding to x, with the aggregate of x itself, and climb toward the root of this compress tree.

Whenever the current node is a right child, its left sibling lies entirely before it on the heavy path, so prepend the aggregate of that left sibling. Whenever the current node is a left child, the right sibling lies below x and is ignored.

When the root is reached, the accumulated value is exactly the prefix

$$P(h, x).$$

The work is proportional to the depth of the leaf x. The total weight of the compress tree is

$$\sum_{u \in H} \lambda(u) = \mathrm{sz}(h),$$

because the differences telescope along the heavy path.

By Lemma 415, the leaf of weight $\lambda(x)$ has depth

$$O\left(1 + \log \frac{\mathrm{sz}(h)}{\lambda(x)}\right).$$

□

For noncommutative aggregates, store both orientations of every Path value. Then the same procedure returns the segment in either direction.

Why the costs telescope. Consider walking upward from a vertex x_0 toward the root by the usual heavy-light decomposition.

Let

$$h_i$$

be the head of the heavy path containing x_i, and after processing the segment

$$P(h_i, x_i)$$

set

$$x_{i+1} = \mathrm{parent}(h_i).$$

Thus the edge

$$x_{i+1} h_i$$

is light.

The cost of the i-th heavy-path segment is

$$O\left(1+\log\frac{\mathrm{sz}(h_i)}{\lambda(x_i)}\right).$$

Now observe the reason for having balanced the compress tree using the weights λ. For $i \geq 1$, the previous heavy-path head h_{i-1} is a light child of x_i. Therefore its entire subtree is included in the local weight of x_i:

$$\boxed{\lambda(x_i) \geq \mathrm{sz}(h_{i-1}).}$$

Consequently,

$$\begin{aligned}\sum_{i=0}^{k}\log\frac{\mathrm{sz}(h_i)}{\lambda(x_i)} &\leq \log\frac{\mathrm{sz}(h_0)}{\lambda(x_0)}+\sum_{i=1}^{k}\log\frac{\mathrm{sz}(h_i)}{\mathrm{sz}(h_{i-1})}\\ &= \log\frac{\mathrm{sz}(h_k)}{\lambda(x_0)}\\ &\leq \log n.\end{aligned}$$

There are also only $O(\log n)$ segments, because moving from h_i to x_{i+1} crosses a light edge, and every root–leaf path has only $O(\log n)$ light edges.

Hence all heavy-path prefixes on one upward walk cost

$$\boxed{O(\log n)}$$

in total.

Theorem 421 (Static Path Query)**.** *An associative aggregate on an arbitrary represented-tree path*

$$P(u,v)$$

can be queried in

$$\boxed{O(\log n)}$$

time.

Proof. Use the ordinary heavy-light path decomposition.

While u and v lie on different heavy paths, process the endpoint whose heavy-path head is deeper. If h is the head of the heavy path containing that endpoint x, query

$$P(h,x)$$

using Lemma 420, append or prepend it with the appropriate orientation, and replace

$$x \leftarrow \mathrm{parent}(h).$$

The segments processed from the u-side form one upward heavy-light walk, and those processed from the v-side form another. By the telescoping calculation above, each side costs

$$O(\log n)$$

in total.

Eventually u and v lie on the same heavy path. The remaining segment between them is one interval in a single weight-balanced compress tree.

An interval in a binary expression tree can be obtained by climbing from its two endpoint leaves toward their least common ancestor, accumulating the siblings lying inside the interval. Its cost is at most the sum of the two endpoint depths, hence

$$O(\log n)$$

by Lemma 415.

Combining the two upward portions and the final heavy-path interval gives

$$O(\log n)$$

total time. □

Theorem 422 (Static Top-Tree Queries)**.** *After preprocessing, and after any point updates, the static top tree supports*

$$\boxed{\begin{array}{l} \textit{rooted-subtree query} : O(\log n), \\ \textit{arbitrary path query} : O(\log n). \end{array}}$$

Path aggregates may be noncommutative by storing both orientations of every Path *value.*

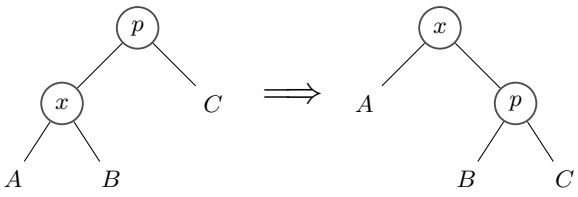
11.9 Splay Tree

A splay tree is a binary tree with no explicit balancing information. Whenever a node is accessed, rotations move it to the root.

Individual operations may take linear time, but the amortized cost of a splay is logarithmic.

11.9.1 Rotation

Suppose x is the left child of p.



The inorder order

$$A, x, B, p, C$$

is unchanged.

The other rotation is symmetric.

11.9.2 Splaying

To splay x, repeatedly inspect its parent p and, when it exists, its grandparent g.

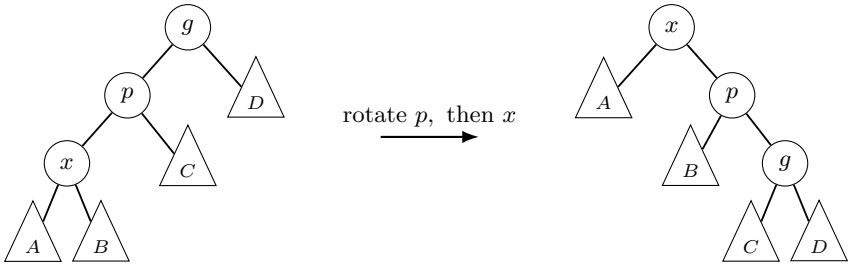
A final *zig* rotates x once.

In a *zig–zig*, x and p have the same orientation; rotate p, then x.

In a *zig–zag*, they have opposite orientations; rotate x twice.

Before rotating, all lazy tags on the auxiliary root–x path must be pushed from top to bottom.

Zig–Zig The only non-obvious part of splaying is the zig–zig step. In the left–left case it performs



The point of zig–zig is that splaying should improve more than just the accessed vertex.

If we blindly rotate x upward twice, x reaches the top, but both vertices above it are pushed down:

$$g, p, x \quad \longrightarrow \quad x, g, p.$$

In particular, p, which was immediately next to the accessed vertex, actually becomes deeper.

Zig–zig instead reverses the whole three-vertex piece:

$$g, p, x \quad \longrightarrow \quad x, p, g.$$

Thus x gains two levels while p stays close to it; only the more distant ancestor g is pushed far downward. Vertices that were close to x remain relatively close to the top as well. This is the improvement that makes the amortized bound possible.

11.9.3 Access Lemma

Give every node v an arbitrary positive analysis weight

$$w(v) > 0.$$

For a splay subtree T_v, define

$$s(v) = \sum_{u \in T_v} w(u), \qquad r(v) = \log_2 s(v),$$

and use the potential

$$\Phi = \sum_{v} r(v).$$

We need one elementary inequality.

Lemma 423. *If $a, b > 0$ and*

$$a + b \le c,$$

then

$$\boxed{\log_2 a + \log_2 b \le 2 \log_2 c - 2.}$$

Proof. By AM–GM,

$$ab \le \left(\frac{a+b}{2} \right)^2 \le \frac{c^2}{4}.$$

Take base-2 logarithms. □

Theorem 424 (Splay Access Lemma)**.** *Suppose x is splayed in a tree of total analysis weight W. Its amortized number of rotations is at most*

$$\boxed{1 + 3 \log_2 \frac{W}{w(x)}.}$$

Proof. We prove that every double rotation has amortized cost at most

$$3(r'(x) - r(x)),$$

while the final zig has amortized cost at most

$$1 + 3(r'(x) - r(x)).$$

For a zig, only x and its parent p change rank, and

$$r'(x) = r(p).$$

Thus

$$\begin{aligned} 1 + \Delta\Phi &= 1 + r'(x) + r'(p) - r(x) - r(p) \\ &= 1 + r'(p) - r(x) \\ &\le 1 + r'(x) - r(x). \end{aligned}$$

Now consider zig–zig:

$$x \subset p \subset g.$$

Again

$$r'(x) = r(g).$$

The old subtree of x and the new subtree of g are disjoint and both lie inside the new subtree of x. Hence

$$s(x) + s'(g) \le s'(x).$$

By Lemma 423,

$$r(x) + r'(g) \le 2r'(x) - 2.$$

Also,

$$r'(p) \le r'(x), \qquad r(p) \ge r(x).$$

Therefore

$$\begin{aligned} 2 + \Delta\Phi &= 2 + r'(p) + r'(g) - r(x) - r(p) \\ &\le 2 + r'(x) + r'(g) - 2r(x) \\ &\le 3(r'(x) - r(x)). \end{aligned}$$

For zig–zag, the new subtrees rooted at p and g are disjoint inside the new subtree rooted at x, so

$$r'(p) + r'(g) \le 2r'(x) - 2.$$

Thus

$$\begin{aligned} 2 + \Delta\Phi &= 2 + r'(p) + r'(g) - r(x) - r(p) \\ &\le 2r'(x) - r(x) - r(p) \\ &\le 2(r'(x) - r(x)) \\ &\le 3(r'(x) - r(x)). \end{aligned}$$

Summing over the whole splay telescopes:

$$\sum (r'(x) - r(x)) = r_{\text{final}}(x) - r_{\text{initial}}(x).$$

Finally,

$$r_{\text{final}}(x) \le \log_2 W, \qquad r_{\text{initial}}(x) \ge \log_2 w(x).$$

□

With unit weights,

$$W = n,$$

so a splay costs

$$O(\log n)$$

amortized.

11.9.4 Augmentation

Suppose values lie in a monoid

$$(\mathcal{M}, \circ, e).$$

Store

$$\mathrm{agg}(v) = \mathrm{agg}(L_v) \circ \mathrm{val}(v) \circ \mathrm{agg}(R_v).$$

A rotation changes only constantly many child pointers, so the aggregates are restored in $O(1)$ time.

For sequence reversal, maintain a lazy reversal bit. If the monoid is noncommutative, store both

$$\mathrm{fwd}(v) \qquad \text{and} \qquad \mathrm{rev}(v).$$

Reversal swaps the children and these two aggregates.

11.10 Link-Cut Tree

A link-cut tree is a data structure for maintaining a forest while supporting operations on arbitrary tree paths in amortized

$$\boxed{O(\log n)}$$

time.

The main difficulty is that a tree path can be long. Therefore, we decompose it into paths, store each path in a splay tree, and change this path decomposition whenever a new path is accessed.

It is useful to distinguish three objects:

1. the *represented tree*, which is the actual tree we care about;
2. the *auxiliary trees*, which are splay trees representing disjoint paths of the represented tree;
3. the *tree of auxiliary trees*, obtained by contracting each such path to one object and keeping the represented edges between different paths.

11.10.1 Preferred Paths

At every vertex, at most one edge to a child is designated *preferred*. These choices are not determined by subtree size and are not permanent; they change during accesses.

Since every vertex has at most one preferred child, the preferred edges form a collection of vertex-disjoint downward paths.

Each such preferred path is stored in one *auxiliary splay tree*.

All remaining represented-tree edges are called *virtual*. They join one preferred path to another. Thus if we contract every preferred path, the virtual edges form a tree whose nodes are the auxiliary trees.

For example, consider the following represented tree.

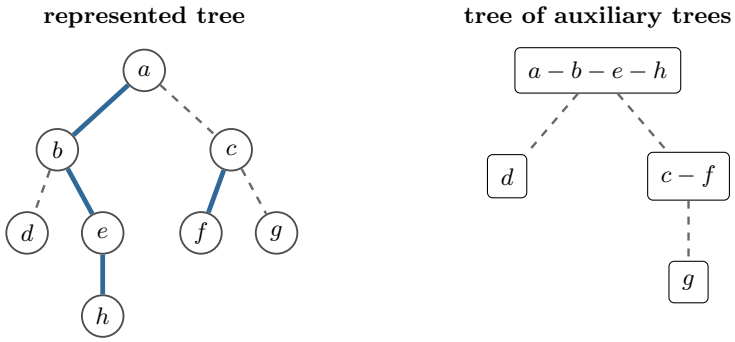


Figure 2: On the left, preferred edges partition the represented tree into the paths $a - b - e - h$, $c - f$, d, and g. Each path is stored in one auxiliary splay tree. Contracting those paths gives the tree of auxiliary trees on the right.

This second view is particularly useful for understanding Access: an access starts in one auxiliary tree and repeatedly moves through a virtual edge to the auxiliary tree above it.

11.10.2 How a Preferred Path Is Stored

Suppose a preferred path is

$$v_1, v_2, \ldots, v_k$$

from top to bottom in the represented tree.

Its auxiliary tree is a splay tree whose *inorder order* is

$$v_1, v_2, \ldots, v_k.$$

The shape of the splay tree has nothing to do with the shape of the represented tree.

For example, the represented path

$$a - b - e - h$$

could currently be stored as follows.

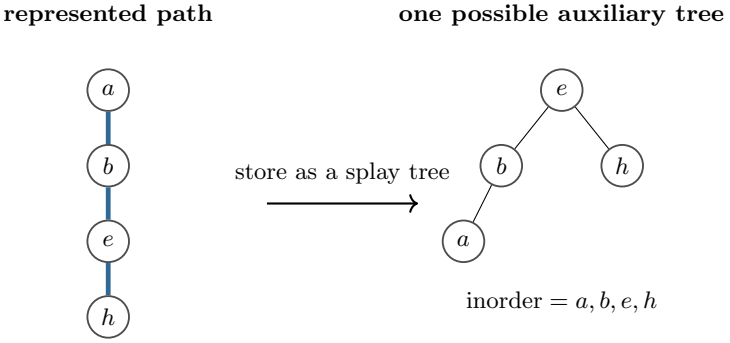


Figure 3: An auxiliary tree does not imitate the represented-tree parent pointers. Its only structural requirement is that inorder order agrees with top-to-bottom order along the preferred path.

Splaying may completely rearrange an auxiliary tree, but it preserves inorder order, so it continues to represent the same tree path.

It also means that ordinary binary-tree augmentation works immediately. If every splay node stores an aggregate of its subtree, then the root of an auxiliary tree stores the aggregate of the entire preferred path.

11.10.3 Connecting the Auxiliary Trees

The auxiliary trees must still remember how the preferred paths are joined inside the represented tree.

Suppose a preferred path begins at x, and the represented parent of x is p. Then the edge

$$px$$

is virtual and connects this auxiliary tree to the auxiliary tree containing p.

This connection is called the *path-parent* relation.

Conceptually, we could store two kinds of pointers:

- ordinary splay-tree parent pointers inside an auxiliary tree;
- a path-parent pointer from the whole auxiliary tree to the auxiliary tree above it.

A link-cut tree can encode both with a single field

$$\text{parent}(x).$$

If x is not the root of its auxiliary tree, this is its ordinary splay parent.

If x *is* the root of its auxiliary tree, then its parent pointer instead stores the path-parent connection of the whole auxiliary tree.

Thus x is an auxiliary root precisely when

$$\text{parent}(x) = \bot$$

or

$$x \notin \{\text{left}(\text{parent}(x)), \text{right}(\text{parent}(x))\}.$$

Notice that if x is an auxiliary root, $\text{parent}(x)$ need not be the represented parent of x itself. The pointer represents the virtual edge immediately above the top of the path stored by the entire auxiliary tree.

This is what allows us to climb from one auxiliary tree to the next.

11.10.4 Access

The fundamental operation is

$$\mathsf{Access}(v).$$

Its purpose is simple:

make the entire represented root–v path one preferred path.

Equivalently, after an access, the root–v path is stored in one auxiliary splay tree.

Before the operation, that path may pass through many auxiliary trees. In the tree-of-auxiliary-trees picture, $\mathsf{Access}(v)$ walks upward through these auxiliary trees and splices them together one at a time.

The key local operation is easiest to understand after splaying a vertex x.

Since inorder order is top-to-bottom represented-tree order,

- the left subtree of x consists of the part of its current preferred path *above* x;
- the right subtree of x consists of the part *below* x.

Suppose we have already exposed the portion from x downward toward v, stored in an auxiliary tree called *last*.

We then replace

$$\text{right}(x)$$

by *last*.

This does exactly what we want:

- the old preferred continuation below x stops being preferred and becomes a separate auxiliary tree;
- the path from x toward v becomes preferred;
- hence the exposed suffix toward v is merged with the preferred path containing x.

We then follow the path-parent pointer to the auxiliary tree immediately above and repeat.

Algorithm 54 Link-Cut Tree Access

```
1: function Access(v)
2:     last ← ⊥
3:     x ← v
4:     while x ≠ ⊥ do
5:         Splay(x)
6:         right(x) ← last
7:         Pull(x)
8:         last ← x
9:         x ← parent(x)
10:    Splay(v)
```

Here and below, the assignment

$$\text{right}(x) \leftarrow \mathit{last}$$

includes the corresponding parent-pointer bookkeeping.

Lemma 425 (Correctness of Access)**.** *After* Access(v)*, the represented root–v path is one preferred path, represented by the auxiliary splay tree rooted at v.*

Proof. Consider an iteration after x has been splayed.

The right subtree of x is exactly the portion of the current preferred path strictly below x. Replacing it by *last* therefore discards the old preferred continuation and makes the already exposed route from x toward v preferred.

Consequently, after the splice, the auxiliary tree rooted at x represents a preferred path ending at v.

Its path-parent pointer leads to the represented edge immediately above the top of this preferred path. The next loop iteration enters the auxiliary tree above this edge and splices that path onto the front.

Inductively, *last* always represents an increasingly long suffix of the represented root–v path. When there is no path-parent left, that suffix begins at the represented root and hence is the entire path.

Finally, splaying v makes v the root of the resulting auxiliary tree. □

11.10.5 Why Access Is Fast

One access can pass through many auxiliary trees, so it can perform many splays. Thus, a single access need not take $O(\log n)$ worst-case time.

There are two pieces to the analysis:

1. if an access encounters k auxiliary trees, all of its splaying costs only

   $$O(k + \log n)$$

   amortized time;

2. over a sequence of accesses, the number of auxiliary-tree boundaries crossed is only $O(\log n)$ amortized per access.

The first part comes from the splay-tree potential. The second is a heavy–light accounting argument.

Importantly, *heavy and light edges are used only for the analysis.* The link-cut tree does not choose its preferred edges according to heavy–light decomposition.

11.10.6 The Analysis Tree

For the splay analysis, expand the tree of auxiliary trees back to the vertex level.

Keep every splay left/right edge. Whenever an auxiliary tree has a path-parent pointer, also keep that connection.

Ignoring which edges are splay edges and which are path-parent edges gives one ordinary rooted tree on the same vertex set. Call this the *analysis tree*.

For a vertex x, let

$$\sigma(x) = \#\{\text{descendants of } x \text{ in the analysis tree}\},$$

including x, and set

$$\rho(x) = \log_2 \sigma(x).$$

Use the potential

$$\boxed{\Phi = \sum_x \rho(x).}$$

The useful feature of this viewpoint is that the splice performed by Access does not change the underlying analysis tree.

When the old right subtree of x stops being a splay child, its root becomes the root of a separate auxiliary tree whose path-parent is x. Thus it was a child of x in the analysis tree before the splice and remains a child afterward.

Likewise, *last* was connected to x through a path-parent relation before the splice and becomes its right splay child afterward.

Thus a splice merely changes how an analysis-tree edge is represented: as a splay edge or as a path-parent edge.

Only splay rotations change the analysis tree and therefore the potential.

Lemma 426 (Total Splay Cost During Access)**.** *Suppose an access performs k loop iterations. The total amortized rotation cost of its splays is*

$$\boxed{O(k + \log n).}$$

Proof. Consider one splay of x.

All auxiliary trees hanging virtually from vertices of the current preferred path may be regarded as fixed positive weights attached to those vertices. Then the weighted size of a splay subtree is exactly the corresponding analysis-tree subtree size σ.

Hence the weighted splay access lemma gives

$$\text{amortizedCost}(\mathsf{Splay}(x)) \le 1 + 3\big(\rho_{\text{after}}(x) - \rho_{\text{before}}(x)\big).$$

Let

$$x_1, x_2, \ldots, x_k$$

be the successive loop vertices, starting near v and moving toward the represented root.

After processing x_i, the exposed structure rooted at x_i lies below the next loop vertex x_{i+1} in the analysis tree. Therefore

$$\sigma_{\text{after}}(x_i) \le \sigma_{\text{before}}(x_{i+1}),$$

so

$$\rho_{\text{after}}(x_i) \le \rho_{\text{before}}(x_{i+1}).$$

The rank increases therefore telescope:

$$\begin{aligned}\sum_{i=1}^{k} \big(\rho_{\text{after}}(x_i) - \rho_{\text{before}}(x_i)\big) &\le \rho_{\text{after}}(x_k) - \rho_{\text{before}}(x_1) \\ &\le \log_2 n.\end{aligned}$$

Thus the total amortized splay cost is

$$O(k + \log n).$$

The final splay of v contributes another $O(\log n)$, which is absorbed into the same bound. □

11.10.7 How Many Auxiliary Trees Does Access Cross?

It remains to account for k, the number of preferred-path pieces encountered.

For this argument only, fix the represented tree and classify a child edge uv as *heavy* when

$$\mathrm{sz}(v) > \frac{1}{2}\,\mathrm{sz}(u),$$

and *light* otherwise.

Each vertex has at most one heavy child, and every represented root–leaf path contains at most

$$\log_2 n$$

light edges.

Now consider what an access does to preferred edges.

Every virtual edge on the newly exposed root–v path becomes preferred. Among these, only

$$O(\log n)$$

can be light.

What about heavy edges?

A heavy preferred edge ux can become nonpreferred only if some other child y of u becomes preferred. Since u has at most one heavy child, the new preferred edge uy must then be light.

Thus every event that destroys a heavy preferred edge can be charged to the creation of a light preferred edge.

Furthermore, after a heavy edge has become preferred once, making it preferred again requires it to have become nonpreferred in between.

Hence, over q accesses,

$$\#\{\text{heavy preferred creations}\} \le n + \#\{\text{light preferred creations}\}.$$

Since each access creates only $O(\log n)$ light preferred edges,

$$\#\{\text{light preferred creations}\} = O(q \log n).$$

Therefore the total number of preferred-path boundaries crossed or changed is

$$O(n + q \log n).$$

Combining this with Lemma 426, the total time for q accesses is

$$O(n + q \log n).$$

Thus access takes

$$\boxed{O(\log n)}$$

amortized time.

Theorem 427 (Link-Cut Access)**.**

$$\boxed{\mathsf{Access}(v) \text{ takes } O(\log n) \text{ amortized time.}}$$

11.10.8 Rerooting

Once we can expose a root–vertex path, rerooting is immediate.

After

$$\mathsf{Access}(v),$$

the represented root–v path is exactly one auxiliary tree, in top-to-bottom inorder order.

To make v the represented root, simply reverse this path.

We therefore define

$$\boxed{\mathsf{MakeRoot}(v): \quad \mathsf{Access}(v); \quad \mathsf{Reverse}(v).}$$

The reversal is stored lazily in the splay tree.

Reversing the order of the exposed path reverses every represented parent relation along it, so v becomes the represented root.

11.10.9 Dynamic-Forest Operations

Once Access and MakeRoot are available, the usual dynamic-tree operations become short.

Finding the root. Perform

$$\mathsf{Access}(v).$$

The entire represented root–v path is now one auxiliary tree. Its leftmost vertex is therefore the represented root.

Walk left while pushing lazy reversals, and splay the resulting vertex.

Link. Suppose u and v lie in different represented trees.

First perform

$$\mathsf{MakeRoot}(u).$$

Now u has no represented parent. Make its auxiliary tree a virtual child of v, thereby adding the represented edge

$$uv.$$

Cut. Suppose uv is a represented edge.

Perform

$$\mathsf{MakeRoot}(u), \qquad \mathsf{Access}(v).$$

The auxiliary tree rooted at v now represents exactly

$$P(u, v).$$

Since u and v are adjacent in the represented tree, this path contains exactly their one edge. Detaching that exposed connection removes uv from the represented forest.

11.10.10 Path Queries and Updates

The most useful consequence of access is that an arbitrary represented path can be turned into one auxiliary tree.

To expose

$$P(u, v),$$

perform

$$\boxed{\mathsf{MakeRoot}(u), \qquad \mathsf{Access}(v).}$$

After MakeRoot(u), the path $P(u, v)$ is a root–v path. The access therefore makes exactly this path one preferred path.

Hence the entire auxiliary tree rooted at v consists precisely of $P(u, v)$.

If every splay node stores an aggregate of its subtree, then

$$\boxed{\text{PathQuery}(u, v) = \text{agg}(v).}$$

Similarly, any update that can be represented by a compatible lazy tag can be applied to the root v, thereby updating the entire path at once.

Theorem 428 (Link-Cut Tree)**.** *A link-cut tree supports*

$$\text{makeRoot}, \quad \text{findRoot}, \quad \text{link}, \quad \text{cut},$$

together with path aggregates and compatible lazy path updates, in

$$\boxed{O(\log n)}$$

amortized time per operation and $O(n)$ space.

Proof. Every operation consists of a constant number of calls to Access, together with $O(1)$ additional splays, lazy reversals, and pointer changes.

By Theorem 427, each access costs $O(\log n)$ amortized time.

Splay-tree aggregates and lazy information are maintained by Pull and Push in $O(1)$ time per rotation, so augmenting the auxiliary trees does not change the asymptotic bound. □

11.11 Top Tree

The cleanest way to arrive at a top tree is to start from the one thing a link-cut tree does not maintain well.

A link-cut tree knows everything about the current preferred path.

It knows almost nothing about the collection of virtual children hanging from that path.

Suppose u has virtual child subtrees

$$T_1, T_2, \ldots, T_k.$$

An ordinary LCT may maintain their total aggregate explicitly, but if

$$k = \Theta(n),$$

there is no balanced structure representing those children individually.

This becomes a problem as soon as we want subtree queries, subtree lazy updates, or more complicated DP information.

The top-tree idea is:

keep the LCT preferred-path splay tree, and also put every collection of virtual children in a splay tree.

The preferred-path splay is the *compress tree*.

The virtual-child splay is the *rake tree*.

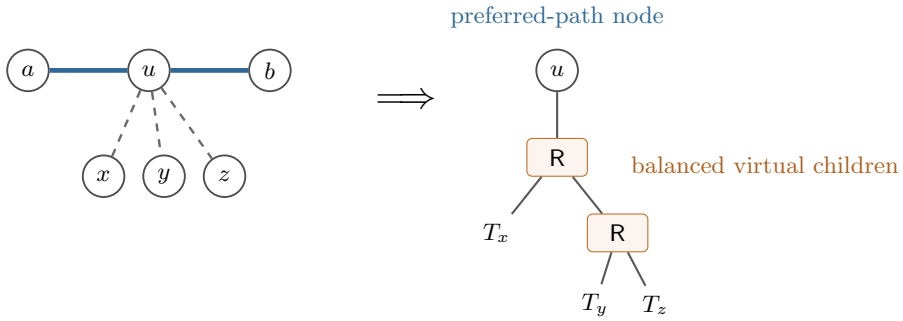


Figure 4: A top tree adds a balanced rake structure for the virtual children that an ordinary link-cut tree leaves implicit.

11.11.1 Real and Fake Nodes

A real represented-tree vertex stores its ordinary two splay children

$$\text{ch}[0], \qquad \text{ch}[1].$$

These are the preferred-path or compress children.

We also give it two rake slots

$$\text{ch}[2], \qquad \text{ch}[3].$$

A represented vertex may have arbitrarily many virtual children, so the rake slots contain a binary tree of *fake nodes*.

Fake nodes contain no represented-tree vertex value. Their sole purpose is to combine two virtual-child aggregates.

Whenever a virtual child is inserted or deleted, ordinary splay rotations within the rake tree keep this collection self-adjusting.

Since every fake node is an internal node of a binary tree whose leaves are represented virtual pieces, the total number of fake nodes is $O(n)$.

11.11.2 The Same Path/Point Interface Reappears

Recall the static-top-tree interface.

A preferred path is a

$$\mathsf{Path}$$

object.

A collection of virtual children attached at one vertex is a

$$\mathsf{Point}$$

object.

The four operations are again

$$\boxed{\begin{aligned} \mathsf{Vertex}_u &: \mathsf{Point} \to \mathsf{Path}, \\ \mathsf{AddEdge} &: \mathsf{Path} \to \mathsf{Point}, \\ \mathsf{Rake} &: \mathsf{Point}^2 \to \mathsf{Point}, \\ \mathsf{Compress} &: \mathsf{Path}^2 \to \mathsf{Path}. \end{aligned}}$$

The difference is only that the binary expression is now maintained dynamically by splay rotations.

11.11.3 Access with Virtual-Child Trees

Consider one iteration of ordinary

$$\mathsf{Access}(v)$$

at a vertex x.

After splaying x, let

$$old = \mathrm{right}(x).$$

This is the old preferred continuation below x.

In an ordinary LCT we simply execute

$$\mathrm{right}(x) \leftarrow last.$$

With virtual-child structures, we must also maintain where those represented subtrees live.

The old preferred continuation becomes virtual, so if $old \neq \bot$, convert it to a point cluster with

$$\mathsf{AddEdge}(old)$$

and insert that point into x's rake tree.

Conversely, *last* was previously a virtual child contribution of x. Remove its point representation from the rake tree and install it as the new right preferred child.

Thus one splice is

$$\boxed{\begin{array}{ll} \text{if } old \neq \bot: & \mathsf{InsertVirtual}(x, old), \\ \text{if } last \neq \bot: & \mathsf{EraseVirtual}(x, last), \\ & \mathrm{right}(x) \leftarrow last. \end{array}}$$

11.11.4 Aggregates

For a real node u, its auxiliary subtree consists of a piece of a preferred path together with arbitrary represented subtrees hanging off that path. Thus its represented material has a natural partition into two parts:

$$\boxed{\text{preferred-path spine} \quad \sqcup \quad \text{material hanging off the spine.}}$$

We maintain one aggregate for each part.

For a real node u, let

$$\boxed{\begin{array}{l} \mathsf{path}(u) = \text{aggregate of the represented vertices on its preferred-path spine,} \\ \mathsf{sub}(u) = \text{aggregate of everything hanging off that spine,} \\ \mathsf{all}(u) = \text{aggregate of the entire represented cluster.} \end{array}}$$

Compress children versus rake children. Recall that a real node u has two different kinds of auxiliary children.

The children

$$\mathrm{ch}_0(u), \mathrm{ch}_1(u)$$

are *compress children*. They continue the same preferred-path spine as u.

The children

$$\mathrm{ch}_2(u), \mathrm{ch}_3(u)$$

are *rake children*. They represent entire clusters attached virtually to u, and hence lie completely off the preferred path containing u.

The path aggregate. Only the compress children lie on the same preferred path as u. Therefore

$$\boxed{\mathsf{path}(u) = \mathsf{path}(\mathrm{ch}_0\, u) \oplus \mathrm{val}(u) \oplus \mathsf{path}(\mathrm{ch}_1\, u).}$$

Inorder order in the compress tree is represented-tree path order.

The rake children do not appear here because none of their represented vertices lies on this preferred path.

The off-path aggregate. Now consider everything in the cluster that is *not* on the spine.

There are two sources.

First, each compress child may itself have represented material hanging from its own part of the spine. We therefore take

$$\mathsf{sub}(\mathrm{ch}_0\, u) \qquad \text{and} \qquad \mathsf{sub}(\mathrm{ch}_1\, u).$$

Second, each rake child lies entirely off the current spine. Consequently we want the *whole* cluster of each rake child, not merely its own off-path part. Hence

$$\mathsf{all}(\mathrm{ch}_2\, u) \qquad \text{and} \qquad \mathsf{all}(\mathrm{ch}_3\, u).$$

Putting these together,

$$\boxed{\begin{aligned}\mathsf{sub}(u) = \mathsf{sub}(\mathrm{ch}_0\, u) &\oplus \mathsf{sub}(\mathrm{ch}_1\, u) \\ &\oplus \mathsf{all}(\mathrm{ch}_2\, u) \oplus \mathsf{all}(\mathrm{ch}_3\, u).\end{aligned}}$$

The whole cluster. Since the path and off-path pieces are disjoint and together form the entire cluster,

$$\boxed{\mathsf{all}(u) = \mathsf{path}(u) \oplus \mathsf{sub}(u).}$$

Missing children contribute the identity element.

Fake rake nodes. A fake rake node does not correspond to a represented-tree vertex and does not lie on a preferred path. Its only purpose is to balance a collection of virtual child clusters.

Thus it simply combines the represented material of its two children. If x is a fake rake node with children a, b, then

$$\boxed{\mathsf{all}(x) = \mathsf{all}(a) \oplus \mathsf{all}(b).}$$

Ordered versus unordered aggregates. Along a preferred path there is a canonical order, so path may represent a noncommutative aggregate. As in an ordinary link-cut tree, in that case we store both

$$\mathsf{path}^{\rightarrow} \qquad \text{and} \qquad \mathsf{path}^{\leftarrow}$$

and rerooting can reverse the path.

Rake children, on the other hand, have no meaningful order. Any aggregate that combines independent virtual subtrees must therefore be commutative.

11.11.5 Two Kinds of Lazy Tags

The same path/off-path distinction must also be remembered by lazy updates.

Suppose an auxiliary cluster represents

$$\boxed{\text{spine} \sqcup \text{material hanging from the spine.}}$$

An update might be intended for only the first part or only the second. We therefore maintain two tags:

$$\boxed{\mathsf{plazy} \qquad \text{and} \qquad \mathsf{slazy}.}$$

Here

$$\mathsf{plazy} = \text{pending update to the preferred-path spine,}$$

while

$$\mathsf{slazy} = \text{pending update to the material off that spine.}$$

Pushing a path tag. A path tag affects u and the parts of its compress children lying on the same spine. It does not affect any rake child.

Thus when a path tag is pushed through a real node, it is propagated as a path tag only to

$$\mathrm{ch}_0(u), \qquad \mathrm{ch}_1(u).$$

The rake children are untouched.

This is exactly what we would expect: a path update follows the preferred path and nowhere else.

Pushing an off-path tag. An off-path tag is slightly more subtle.

Consider first a compress child c. The path of c is part of the parent's path as well. Therefore it is *not* part of the parent's off-path region. Only the material hanging from that child should receive the update.

Hence an off-path tag pushed into a compress child remains

$$\boxed{\mathsf{slazy}.}$$

Now consider a rake child c.

The entire represented cluster of c hangs off the parent's path. Therefore *every represented vertex inside c* must receive the update.

But internally c again divides its material into

$$\mathsf{path}(c) \qquad \text{and} \qquad \mathsf{sub}(c).$$

Both pieces belong to the parent's off-path region.

Consequently, when an off-path update crosses a rake edge, it becomes an update to the whole child cluster, plazy and slazy.

11.11.6 Push Before Changing the Decomposition

There is one important consequence of this interpretation.

Lazy tags describe a set of *represented vertices*, not merely a set of auxiliary nodes. We must therefore never change which represented material is classified as compress or rake while an unpushed tag above it still depends on the old classification.

In particular, before removing a virtual child y, push all tags along the rake-root–y path.

After the removal, recompute aggregates on the way back up and suppress any fake rake node left with only one child.

11.11.7 Path Operations

Path operations are exactly as in an ordinary link-cut tree.

To expose

$$P(u, v),$$

perform

$$\boxed{\mathsf{MakeRoot}(u), \qquad \mathsf{Access}(v).}$$

Afterward every edge of $P(u, v)$ is preferred, so the path is precisely the preferred-path spine of the auxiliary cluster rooted at v.

Hence

$$\boxed{\mathrm{PathQuery}(u, v) = \mathsf{path}(v).}$$

Likewise, a lazy update to $P(u, v)$ is recorded as a path update:

$$\boxed{\mathsf{plazy}(v).}$$

11.11.8 Subtree Operations

Now fix a represented root r, and suppose we want the rooted subtree T_u.

Perform

$$\boxed{\mathsf{MakeRoot}(r), \qquad \mathsf{Access}(u).}$$

Afterward the preferred path ending at u is exactly

$$P(r, u).$$

Since u is the bottom endpoint of this path, it has no preferred child below it.

Therefore every proper descendant of u lies in a virtual child cluster attached at u.

Meanwhile, the compress material on the other side of u is precisely the ancestor path

$$r \rightsquigarrow u,$$

which must not be included in T_u.

Thus the rooted subtree has the decomposition

$$\boxed{T_u = \{u\} \sqcup \{\text{all virtual-child clusters attached at } u\}.}$$

If the two rake children of u are $\mathrm{ch}_2\, u$ and $\mathrm{ch}_3\, u$, then

$$\boxed{\mathrm{SubtreeQuery}(r, u) = \mathrm{val}(u) \oplus \mathsf{all}(\mathrm{ch}_2\, u) \oplus \mathsf{all}(\mathrm{ch}_3\, u).}$$

For a lazy update to the whole rooted subtree, apply the update to

$$\mathrm{val}(u)$$

and to the complete clusters represented by the rake children.

11.11.9 Link, Cut, and Reroot

The dynamic-forest operations retain the same interpretation as in the underlying link-cut tree.

To link two represented trees by an edge uv, reroot the first tree at u, expose v, and attach the complete represented tree rooted at u as a new virtual child cluster of v.

To cut an existing represented edge uv, perform

$$\mathsf{MakeRoot}(u), \qquad \mathsf{Access}(v).$$

The edge uv is now exposed in the compress structure and can be detached exactly as in the ordinary link-cut tree.

Rerooting remains

$\mathsf{Access}(u)$	followed by a lazy reversal of the exposed path.

11.11.10 Amortized Analysis

Regard every real and fake auxiliary node as belonging to one forest for analysis. For a node x, let

$$\sigma(x) = \text{represented mass contained below } x,$$

and define its rank by

$$\rho(x) = \log_2 \sigma(x).$$

Use the usual splay potential

$$\boxed{\Phi = \sum_x \rho(x).}$$

A rotation inside a compress tree and a rotation inside a rake tree are both ordinary splay tree rotations. The weighted splay access lemma therefore gives, for either kind of splay,

$$\text{amortizedCost}(\mathsf{Splay}(x)) \le 1 + 3\big(\rho_{\text{after}}(x) - \rho_{\text{before}}(x)\big).$$

Now consider the successive clusters reached during an expose,

$$x_1, x_2, \ldots, x_k.$$

After x_i has been processed, its represented cluster is contained in the larger cluster reached at the next step. Consequently

$$\sigma_{\text{after}}(x_i) \le \sigma_{\text{before}}(x_{i+1}),$$

and hence

$$\rho_{\text{after}}(x_i) \le \rho_{\text{before}}(x_{i+1}).$$

The rank increases telescope exactly as in the ordinary link-cut-tree analysis:

$$\sum_i \big(\rho_{\text{after}}(x_i) - \rho_{\text{before}}(x_i)\big) = O(\log n).$$

The important point is that moving a represented cluster between the compress and rake structures does not alter the represented material. The same vertices are still present; only the auxiliary decomposition used to store them has changed.

The additive +1 terms in the splay access bound are charged to the same preferred-path changes as in the ordinary link-cut tree. Inserting or removing a cluster from a rake tree happens precisely because a represented edge has changed between preferred and virtual status, so these are the same.

It follows that one expose costs

$$\boxed{O(\log n)}$$

amortized time.

Aggregate recomputation and lazy propagation take $O(1)$ work per visited real or fake auxiliary node and are therefore absorbed into the same bound.

Theorem 429 (Self-Adjusting Top Tree)**.** *The augmented link-cut structure above uses*

$$O(n)$$

space and supports

$$\mathsf{link}, \quad \mathsf{cut}, \quad \mathsf{reroot},$$

together with compatible path and rooted-subtree queries and lazy updates, in

$$\boxed{O(\log n)}$$

amortized time.

Proof. There are n real nodes. Every fake node is an internal node of a binary rake tree, so the total number of fake nodes is also $O(n)$.

By the preceding potential argument, every expose costs $O(\log n)$ amortized time.

Each supported operation performs only a constant number of exposes, together with a constant number of extra cluster insertions, deletions, reversals, or lazy-tag operations.

Therefore every operation takes $O(\log n)$ amortized time and the data structure uses $O(n)$ space. □

11.12 Lazy Anti-Monopoly Tree

The Lazy AM-Tree maintains an incremental MSF in $O(\log n)$ amortized time per insertion. It maintains a Transformed MST (T-MST), a rooted tree that minimizes depth while preserving the results of Path-Max queries.

TW-Transformations and Promotion A fundamental property of T-MSTs is the *TW-Transformation*: given two adjacent edges (x, y, w_1) and (y, z, w_2) with $w_1 \geq w_2$, replacing the edge (x, y, w_1) with (x, z, w_1) preserves all Path-Max queries in the tree.

The Promote(x) operation leverages this to move a node x up the tree while preserving Path-Max queries. Let $y = \mathit{parent}[x]$ and $z = \mathit{parent}[y]$, with edge weights $w_1 = \mathit{weight}(x \to y)$ and $w_2 = \mathit{weight}(y \to z)$.

- **Shortcut** ($w_1 > w_2$): x is directly promoted to become z's child with weight w_1. Node y becomes a sibling of x.
- **Rotate** ($w_1 < w_2$ or y is root): y is pushed down to become a child of x with weight w_1. If y is not the root, x is attached to z with weight w_2.

Stitch-Based Insertion Instead of finding the heaviest edge on the cycle, the Stitch-based algorithm moves the new edge (u, v, w) to its correct position using TW-transformations.

If $weight(u \to parent[u]) < w$, the edge (u, v, w) is TW-transformed to $(parent[u], v, w)$ and recursively moved upwards. Once both $weight[u \to parent[u]] > w$ and $weight[v \to parent[v]] > w$, we attach the smaller subtree to the larger one (e.g., u becomes a child of v with weight w). This displaces u's old parent edge (u, u', w'). Because $w' > w$, we apply another TW-transformation, moving the displaced edge to (u', v, w'), and recursively route it.

If the original edge closed a cycle, the relocation forces the two endpoints to eventually meet at their LCA $(u = v)$. At this point, we discard the edge.

Algorithm 55 Lazy AM-Tree

```
function Promote(x)
    y ← parent[x]
    z ← parent[y]
    w_1 ← weight(x → y); w_2 ← weight(y → z)
    if z ≠ null and w_1 > w_2 then
                                          ▷ Shortcut: x bypasses y to become its sibling
        size[y] ← size[y] − size[x]
        parent[x] ← z
        weight(x → parent) ← w_1
    else
                                          ▷ Rotate: y is pushed down as a child of x
        size[y] ← size[y] − size[x]
        size[x] ← size[x] + size[y]
        parent[x] ← z
        weight(x → parent) ← w_2
        parent[y] ← x
        weight(y → parent) ← w_1
function UpwardCalibrate(x)
    while parent[x] ≠ null do
        y ← parent[x]
        if size[x] ≤ (2/3) size[y] then
            x ← y
        else
            Promote(x)
function LinkByStitch(u, v, w)
    if u = v or u = null or v = null then
        return
    if parent[u] ≠ null and w > weight(u → parent) then
        LinkByStitch(parent[u], v, w)
    else if parent[v] ≠ null and w > weight(v → parent) then
        LinkByStitch(u, parent[v], w)
    else
        if size[u] > size[v] then
            Swap u ↔ v
        u_old_parent ← parent[u]
        w_old ← weight(u → parent)
        parent[u] ← v
        weight(u → parent) ← w
        size[v] ← size[v] + size[u]
                                          ▷ Route the displaced heavier edge
        LinkByStitch(u_old_parent, v, w_old)
function LazyInsert(u, v, w)
    UpwardCalibrate(u)
    UpwardCalibrate(v)
    LinkByStitch(u, v, w)
```

Correctness and Complexity

Theorem 430. *The Lazy AM-Tree maintains a valid T-MST and supports insertions in $O(\log n)$ amortized time.*

Proof. **Correctness:** Every operation in LinkByStitch is a valid TW-transformation. By continually replacing edges with equivalent higher-reaching edges, the sequence preserves Path-Max queries. If u and v were previously connected, the endpoints of the heaviest cycle-edge move to their LCA, where $u = v$ triggers the base case that drops the redundant edge. Therefore, the tree remains valid.
Complexity: We use the potential function $\Phi = \sum_{i=1}^{n} \log(size[i])$.

1. **Calibrate:** The Promote function is only called when x is a heavy child ($size[x] > \frac{2}{3} size[y]$). Under this condition, rotating x upwards decreases y's size by at least a factor of $1/3$, meaning Φ decreases by an amount strictly greater than the $O(1)$ actual cost of the rotation. Therefore, Promote has an amortized cost of zero. Because the loop only advances upward when the AM-rule holds, size increases by a factor of $3/2$ per advancement, limiting iterations to $O(\log n)$.

2. **Link:** The LinkByStitch recursion traverses upward, performing $O(d(u) + d(v))$ actual operations. Structural size changes only occur when the smaller subtree is attached to the larger one. Because the smaller subtree's size at most doubles, the potential Φ increases by at most 1 per topological attachment. Across the $O(d(u) + d(v))$ affected nodes, the total potential increase is bounded by $O(\log n)$.

Thus the total amortized cost per insertion is $O(\log n)$. □

12 Graph Algorithms

12.1 Shortest Paths

12.1.1 Bellman–Ford–Dijkstra

Dijkstra's algorithm efficiently computes shortest paths under nonnegative edges. Bellman–Ford handles negative edges but has a bad running time.

The following hybrid separates the two jobs: after every use of a negative edge, run Dijkstra to exhaustion on all nonnegative edges.

Let

$$E^+ = \{e : w(e) \geq 0\}, \qquad E^- = \{e : w(e) < 0\}.$$

For $i \geq 1$, define

$$d_i(v) = \min\{w(P) : P \text{ is an } s\text{-}v \text{ path containing fewer than } i \text{ negative edges}\}.$$

Thus d_1 is simply shortest-path distance in the nonnegative subgraph.

Also define

$$d'_i(v) = \min\left\{d_i(v), \min_{(u,v)\in E^-} d_i(u) + w(u,v)\right\}.$$

The crucial recurrence is

$$d_{i+1}(v) = \min\left\{d_i(v), \min_u \left(d'_i(u) + \mathrm{dist}_{G^+}(u,v)\right)\right\},$$

where it is enough to consider vertices u whose label actually decreased in the negative-edge relaxation.

Indeed, take a shortest path using at most i negative edges and cut it after its last negative edge. Everything after that edge lies in G^+.

Algorithm 56 Bellman–Ford–Dijkstra

Input: Directed weighted graph G, source s
Output: Shortest-path distances, assuming no reachable negative cycle

```
d[s] ← 0
d[v] ← ∞ for v ≠ s
insert s into priority queue Q
repeat
    A ← ∅
                                          ▷ Dijkstra through nonnegative edges
    while Q is nonempty do
        remove a minimum-label vertex u from Q
        add u to A
        for each (u, v) ∈ E⁺ do
            if d[u] + w(u, v) < d[v] then
                d[v] ← d[u] + w(u, v)
                decrease-key v in Q
                                          ▷ Allow one additional negative edge
    for u ∈ A do
        for each (u, v) ∈ E⁻ do
            if d[u] + w(u, v) < d[v] then
                d[v] ← d[u] + w(u, v)
                decrease-key v in Q
until Q is empty
return d
```

Theorem 431. *After the i-th Dijkstra phase, $d[v] = d_i(v)$ for every vertex v.*

After the following negative-edge phase, $d[v] = d'_i(v)$. Consequently, if the graph contains no reachable negative cycle, the algorithm terminates with the true shortest-path distances.

Proof. We induct on i.

For $i = 1$, the first phase is ordinary Dijkstra on G^+, so it computes d_1.

Suppose a negative phase has produced d'_i. The only newly improved vertices are inserted into Q. Running Dijkstra from these labels computes

$$\min_u \left(d'_i(u) + \mathrm{dist}_{G^+}(u, v) \right),$$

while retaining any old smaller value $d_i(v)$. By the last-negative-edge decomposition, this is exactly $d_{i+1}(v)$.

Now suppose the Dijkstra phase has produced d_i. Relaxing every negative outgoing edge from a vertex whose value became newly relevant computes d'_i. A negative edge from a vertex whose d_i value did not change was already relaxed when that value was first obtained, so rescanning it is unnecessary.

Thus the invariant holds in every round.

If no reachable negative cycle exists, every shortest path may be chosen simple and therefore contains at most $n - 1$ negative edges. Hence eventually $d_i(v) = \mathrm{dist}(s, v)$ for all v, after which no negative edge improves a label and Q becomes empty. □

Let $\eta(v)$ be the minimum number of negative edges among shortest s–v paths. Then v can become newly active in at most $\eta(v) + 1$ Dijkstra phases.

With a binary heap, the running time is therefore

$$O\left(\sum_v (\deg(v) + \log n)(\eta(v) + 1) \right).$$

If every shortest path uses at most h negative edges or we only consider such paths, this is

$$O((m + n \log n)(h + 1)).$$

Thus the algorithm interpolates naturally between Dijkstra ($h = 0$) and Bellman–Ford ($h = \Theta(n)$).

12.1.2 Eppstein's Algorithm

Eppstein's algorithm elegantly computes the k shortest paths from a source s to a destination t in a directed graph in $O((m + n) \log n + k \log k)$ time. Rather than searching for full paths from scratch, the algorithm characterizes every possible $s \to t$ path as the base shortest path plus a sequence of deviations, or sidetracks.

First, we compute the shortest path from every vertex u to t using a reverse Dijkstra's search. This yields the shortest path distance $d[u]$ for all vertices and implicitly defines a shortest path tree T rooted at t.

For any directed edge $e = (u, v)$ with weight w, its sidetrack cost represents the distance penalty incurred by taking e instead of following the shortest path tree T from u. It is defined as:

$$\delta(e) = w + d[v] - d[u] \geq 0$$

Any path P from s to t can be uniquely identified by the sequence of non-tree edges (sidetracks) it traverses: $e_1, e_2, \ldots, e_p$. The total length of P is exactly the base distance plus the sum of its sidetrack penalties:

$$\text{length}(P) = d[s] + \sum_{i=1}^{p} \delta(e_i)$$

State Space and Persistent Heaps To efficiently find the k lightest sidetrack sequences, we construct a min-heap $H[u]$ for every vertex u. This heap contains all possible sidetrack edges originating from u and all ancestors of u in T, ordered by $\delta(e)$.

Because $H[u]$ shares almost all its elements with the heap of its parent in T, we use a Persistent Skew Heap. This allows us to implicitly build the heaps for all vertices in $O(m \log n)$ time. Once the heaps are built, enumerating the k-shortest paths reduces to a Dijkstra-like search over the structure of these heaps, transitioning either by swapping to heavier sidetracks within the same heap or appending new sidetracks from a destination vertex.

Algorithm 57 Eppstein's k-Shortest Paths

Input: Graph $G = (V, E)$, source s, destination t, integer k
Output: Lengths of the k shortest $s \to t$ paths

```
function EPPSTEIN(G, s, t, k)
    d, T ← REVERSEDIJKSTRA(G, t)
    if d[s] = ∞ then
        return ∅
    Order V by d[u] ascending
    for u ∈ V do
        root ← heap root of u's parent in T (or null if none)
        for each outgoing edge e = (u, v) ∈ E \ T do
            δ ← weight(e) + d[v] − d[u]
            root ← PERSISTENTINSERT(root, δ, v)
        H[u] ← root
    Q ← empty min-priority queue
    Q.push(length = d[s], node = H[s])
    R ← empty list
    while Q is not empty and |R| < k do
        (L, node) ← Q.pop()
        Append L to R
        if node.left ≠ null then
            Q.push(L − node.δ + node.left.δ, node.left)
        if node.right ≠ null then
            Q.push(L − node.δ + node.right.δ, node.right)
        next_heap ← H[node.dest]
        if next_heap ≠ null then
            Q.push(L + next_heap.δ, next_heap)
    return R
```

Complexity Analysis

Theorem 432. *Eppstein's algorithm correctly enumerates the k shortest paths in $O((m+n)\log n + k \log k)$ time.*

Proof. The base distances and shortest path tree are found in $O((m+n)\log n)$ time. Constructing the persistent heaps requires inserting at most m non-tree edges. Each persistent insertion takes $O(\log n)$ time, leading to $O(m \log n)$ time and space for the heap construction.

During the path extraction phase, every valid transition in the state space explores paths in non-decreasing order of total weight because the heaps are strictly min-ordered ($\delta(\text{child}) \geq \delta(\text{parent})$) and $\delta(e) \geq 0$. Since each state expands to at most 3 subsequent states, the priority queue size never exceeds $3k$. Performing k extractions from this queue takes $O(k \log k)$ time. Summing these phases yields the final bounds. □

12.1.3 Deterministic Negative Weight Shortest Paths

Let $G = (V, E, w)$ be a directed graph with integral arc weights $w(e) \geq -W$, where $W \geq 1$. We seek shortest paths from a specified source, or a negative-weight cycle. A potential $p : V \to \mathbb{R}$ reweights an arc $u \to v$ to $w(u, v) + p(u) - p(v)$. This leaves every cycle weight unchanged and shifts the weight of every s–t path by the same amount. If all reduced weights are nonnegative, Dijkstra finishes the problem. We discard nonnegative self-loops (a negative self-loop is already a certificate), and for each ordered pair keep only its cheapest parallel arc. This preserves all shortest paths and negative cycles, and gives $m = O(n^2)$.

It is enough to make the reduced weights progressively less negative. Given a lower bound $-W$, define $c(e) = w(e) + W/2$. A potential satisfying

$$p(v) \le p(u) + c(u, v) \qquad (u \to v \in E) \tag{97}$$

raises the lower bound on the reduced w-weights from $-W$ to $-W/2$. Repeating this operation and using integrality at the end will recover exact distances. We concentrate first on one such *scaling step*: find a feasible c-potential or a negative w-cycle.

The recursion combines two ideas. We cover a vertex set by smaller, possibly overlapping pieces, solve them separately, and merge their potentials. The overlaps are deliberate: a *padded* boundary forces a short walk to spend a definite distance before it is forced to move from an earlier piece to a later piece. Thus even a walk containing many negative arcs can be processed with few Dijkstra sweeps. If we cannot make every piece small, we instead find a piece of small weak diameter. That second kind of progress allows a failed merge to certify a negative cycle. The rest of the algorithm is devoted to obtaining these two properties without paying too much for the vertices duplicated in the overlaps.

Why a diameter bound permits failure. The costs c are not necessarily nonnegative, so they cannot measure the distance needed to return from the end of a walk to its start. Define the *clipped length*

$$\ell(e) = \max\{c(e), 0\}, \qquad H = (V, E, \ell). \tag{98}$$

We use c for feasibility and ℓ for distances. Both can be related to the original weights by

$$w(e) \le \ell(e), \qquad w(e) \le 2c(e) - \ell(e). \tag{99}$$

The second inequality is immediate when $c(e) \ge 0$; when $c(e) < 0$, it is equivalent to $w(e) \ge -W$. Consequently, a walk P satisfying

$$c(P) < 0, \qquad \ell(P) > d \tag{100}$$

has $w(P) \le 2c(P) - \ell(P) < -d$. If its end can reach its start along a path of ℓ-length at most d, then adjoining that path gives a negative closed walk, and hence a negative cycle.

This explains the diameter requirement. A set S has weak ℓ-diameter at most d if every ordered pair of vertices in S can be connected by a path of ℓ-length at most d *in the whole graph* H. These connecting paths need not stay in S. Our recursion will maintain a slightly more convenient version: except at the initial distance scale, there is an anchor a with

$$\mathrm{dist}_H(a, v),\ \mathrm{dist}_H(v, a) \le d/2 \quad \text{for every } v \in S. \tag{101}$$

The two anchor paths concatenate to the return route needed above. At the initial scale we can instead choose $d \ge nW$: a walk of w-weight less than $-d$ cannot become a simple path after removing only nonnegative cycles, since a simple path has at most $n - 1$ arcs, each of weight at least $-W$.

We consider the terminal case. Once $d < W/2$, either all arcs of $G[S]$ have $c(e) \ge 0$, in which case the zero potential is feasible, or some arc $u \to v$ has $w(u, v) = c(u, v) - W/2 < -W/2$. The anchor gives a return route from v to u of ℓ-length, and therefore w-weight, at most $d < W/2$. The arc and return route form a negative cycle. There is no need to decompose further at this scale.

Why padding makes pieces mergeable. Consider an ordered pair of overlapping children (A_1, A_2) covering a set A. A vertex in $A_1 \cap A_2$ can be treated as belonging to either child. If we process child 2 before child 1, a walk can move freely from color 2 to color 1, but a move from color 1 to color 2 may force us to restart the processing. Only a walk from $A_1 \setminus A_2$ to $A_2 \setminus A_1$ is *forced* to make this unfavorable change: an occurrence in the overlap can be colored to avoid it.

We therefore require the two exclusive regions to be separated by clipped distance δ in precisely that direction. Call an ordered split (A_1, A_2) δ-*padded* if

$$\mathrm{dist}_{H[A]}(u, v) \ge \delta \quad (u \in A_1 \setminus A_2,\ v \in A_2 \setminus A_1). \tag{102}$$

The graph $H[A]$ is induced on A. There is no corresponding condition in the reverse direction, because reverse changes are already compatible with the processing order.

An ordered cover $\mathcal{Y} = (Y_1, \dots, Y_k)$ can be built by repeatedly replacing one member by a padded ordered split. A *coloring* of a walk assigns each vertex occurrence an index of a child containing it; a coloring is nonincreasing if these indices never increase. Occurrences of the same vertex may have different colors.

Lemma 433 (Padding under refinement)**.** *If an ordered cover is obtained by δ-padded refinements, then every walk in the original set of ℓ-length less than δ has a nonincreasing coloring by the final cover.*

Proof. Start with the constant coloring of the one-member cover. Suppose a nonincreasing coloring is available before splitting a member A into (A_1, A_2). The occurrences colored A form one consecutive interval. In this interval use color 2 until the first occurrence in $A_1 \setminus A_2$, then use color 1. If no such occurrence exists, use color 2 throughout. After the first A_1-exclusive occurrence there can be no A_2-exclusive occurrence: the intervening subwalk lies in A and would have length at least δ. Thus every new color is valid and the sequence is nonincreasing. Replacing one color by two consecutive colors does not change the order of the other intervals. Repeat for every refinement. □

The strength of this lemma is that the width is not lost when many balls are removed. We need not pay a separate crossing bound for each split; any walk shorter than δ is compatible with the *entire* final cover.

Merge with reverse Dijkstra sweeps. Suppose $\mathcal{Y} = (Y_1, \dots, Y_k)$ is such a cover of S, and we have feasible c-potentials ϕ_i on $G[Y_i]$. Initialize $D(v) = 0$ for every $v \in S$, representing a length-zero walk from an arbitrary starting vertex. We decrease labels only through actual c-walks, but we do not insist on finding the exact minimum.

One *reverse sweep* processes $Y_k, Y_{k-1}, \dots, Y_1$. Within Y_i, run multi-source Dijkstra using the nonnegative reduced costs $c_{\phi_i}(u, v) = c(u, v) + \phi_i(u) - \phi_i(v)$, starting with labels $D(v) - \phi_i(v)$, and then add ϕ_i back. Equivalently, the update performs the exact closure

$$D(v) \leftarrow \min_{u \in Y_i} \big(D(u) + \mathrm{dist}_{c, G[Y_i]}(u, v)\big) \qquad (v \in Y_i). \tag{103}$$

After that Dijkstra call, relax every arc of $G[S]$ whose tail lies in Y_i, even if its head is outside Y_i.

A reverse sweep handles a nonincreasing-colored walk: Dijkstra handles its successive portions inside each child, and outgoing relaxations move to the next, smaller child index. The global label of a vertex automatically transfers between children containing it. The final outgoing relaxation may also handle *one arbitrary edge*, regardless of the next child's index. This is the only property of a sweep that we need.

A longer walk can be divided into short portions and a few separating edges. Scan it in order, and whenever the next edge would bring the current portion's ℓ-length to at least δ, set that edge aside and start a new portion. Each portion has length less than δ and hence a nonincreasing coloring; each portion together with its following set-aside edge has length at least δ. These disjoint blocks show that a walk of length at most d has at most $\lfloor d/\delta \rfloor$ set-aside edges. One sweep handles a portion and its following edge, so

$$q := 1 + \left\lfloor \frac{d}{\delta} \right\rfloor \tag{104}$$

sweeps dominate the cost of every walk of ℓ-length at most d. In particular, a walk can have arbitrarily many negative-cost edges of clipped length zero without requiring extra sweeps.

After the sweeps, check all original feasibility inequalities on $G[S]$. If they hold, D is a feasible potential. Otherwise, some $u \to v$ has $D(u) + c(u, v) < D(v)$. Since $D(u)$ is the cost of a represented walk, appending this edge gives a walk P with $c(P) < D(v) \le 0$. If $\ell(P) \le d$, the preceding domination property would imply $D(v) \le c(P)$, a contradiction. Thus P satisfies (100). This is why approximate short-walk labels suffice: a single scan for feasibility converts any remaining error into the failure certificate we want.

Algorithm 58 Merging a padded ordered cover

```
function MERGE(S, d, δ, (Y_i, φ_i)_{i=1}^k)
    q ← 1 + ⌊d/δ⌋; D(v) ← 0 for every v ∈ S
    for t = 1, ..., q do
        for i = k, k − 1, ..., 1 do
            Run multi-source Dijkstra on G[Y_i] with costs c_{φ_i} and starting labels D − φ_i; restore labels by adding φ_i
            Relax every u → v in G[S] with u ∈ Y_i
    if D(v) ≤ D(u) + c(u, v) for every arc u → v of G[S] then
        return D
    else
        return a violating arc and the predecessor pointers
```

Predecessor pointers also suffice to recover a certificate without expanding the possibly very long represented walk. Whenever a label is strictly improved, retain the predecessor used in that update. For every final predecessor arc $u \to v$, $D(v) \geq D(u) + c(u, v)$: the label of u may only have decreased since the pointer was set. A predecessor cycle therefore has c-weight at most zero and strictly negative w-weight. If the predecessor graph is acyclic, tracing the pointers from a violated edge yields an actual walk of cost at most its tail label; appending the violated edge gives the required long negative walk.

For the running time, use the vertex mass

$$\omega(v) = \deg_G(v) + \lambda, \quad \lambda = \lceil \log_2 \log_2(4n) \rceil, \qquad M(A) = \sum_{v \in A} \omega(v). \tag{105}$$

Degrees count both incoming and outgoing arcs in the original graph. With integer-priority-queue Dijkstra, processing $G[A]$ takes $O(M(A))$ time. Thus if $I = \sum_i M(Y_i)$ is the *incidence mass* of the cover, one sweep costs $O(I)$ and the merge costs

$$O\left(\left(1 + \frac{d}{\delta}\right) I\right). \tag{106}$$

The remaining task is to construct a cover with large padding width δ, small incidence mass, and enough progress to pay for its recursive calls.

Credits for progress in the recursion. A vertex may participate in several recursive instances before its piece becomes small enough, or before the distance scale reaches the terminal case. We must charge the current merge against a decrease in the remaining amount of work, not simply against the number of children: many small pieces would otherwise be expensive to analyze.

A mass-b piece has roughly $\log_2 b$ possible size-halving levels left; a scale-d piece has roughly $\log_2(4d/W)$ possible halvings before $d < W/2$. Assigning each unit of mass a credit for each remaining level suggests the potential

$$\Psi(b, d) = b\left(\log_2 b + \log_2 \frac{4d}{W}\right), \qquad \Psi(0, d) = 0. \tag{107}$$

We do not require every recursive call to halve mass or scale. Instead, the exact decrease in this potential will pay for work even when a split extracts a very small piece.

Halving the scale of a child of mass b saves exactly b credits. For a disjoint same-scale split of mass r into b and $r - b$, the saving is

$$\begin{aligned} \kappa(r, b) &= r \log_2 r - b \log_2 b - (r - b) \log_2 (r - b) \\ &= b \log_2 \frac{r}{b} + (r - b) \log_2 \frac{r}{r - b}. \end{aligned} \tag{108}$$

The first term counts the size levels skipped by the extracted mass: each of its b units belonged to a size-r problem and now belongs to a size-b one, saving $\log_2(r/b)$ levels. The remaining $r - b$ units also save a little,

since their problem shrinks. In particular, for $0 < b \le r/2$,

$$b \log_2(r/b) \le \kappa(r,b) \le b \log_2(r/b) + \frac{b}{\ln 2}, \qquad \kappa(r,b) \ge 2b. \tag{109}$$

The last inequality follows from concavity of binary entropy. The smaller the extracted core relative to its parent, the more credits it earns *per vertex*. This matters because a tiny ball may have a boundary too large to be paid for by merely $O(b)$ work.

To make a split padded, retain a surrounding shell of mass σ in *both* children. A duplicated unit is processed again, and may participate in later recursive calls. Its extra potential is at most $O(L)$, where

$$M_0 = M(V), \qquad L = \lceil \log_2(2M_0) \rceil . \tag{110}$$

Indeed, in the relevant range $W/4 \le d' \le 2nW$,

$$\Psi(b+\sigma, d') - \Psi(b, d') \le 6L\sigma \quad (0 \le b \le b+\sigma \le M_0). \tag{111}$$

This follows by differentiating Ψ with respect to its mass; the derivative is $\log_2 b + (\ln 2)^{-1} + \log_2(4d'/W) \le 6L$. Thus a light core of mass $b < r/2$, whose separation earns $\kappa(r,b) = \Theta(b(1+\log_2(r/b)))$, can afford a shell of mass

$$\sigma \le \frac{\theta}{L}\, b \left(1 + \log_2 \frac{r}{b}\right), \qquad \theta = 1/256. \tag{112}$$

The $1/L$ compensates for the future work of boundary copies; the $\log(r/b)$ comes from the saved size levels. We will use the same rule for large balls, where the additive 1 supplies a shell allowance even when b is close to r.

Finding thin shells by growing balls. Consider an instance (S, d) with $M = M(S)$ and a current residual $R \subseteq S$ of mass $r > M/2$. In the nonnegative graph $H[R]$, choose a center z and grow either its outward balls $C_t^+ = \{v : \mathrm{dist}_{H[R]}(z, v) \le t\}$ or inward balls $C_t^- = \{v : \mathrm{dist}_{H[R]}(v, z) \le t\}$. For either orientation, at radius t take the core $C = C_t^{\pm}$ and the pad $B = C_{t+\delta}^{\pm}$. Its shell is $B \setminus C$; write $b = M(C)$ and $\sigma = M(B \setminus C)$. Accept it when (112) holds, with r fixed during the search. Otherwise replace C by B and try the next radius. Start with radius zero, including every vertex at zero distance. A ball is *light* if $b < r/2$, and *heavy* otherwise.

The same boundary allowance that pays for overlap also bounds the number of unsuccessful increments. While the ball is light, set $x = \log_2(r/b) > 1$, its remaining number of size-halving levels. If the shell test fails, the ball grows from mass b to

$$b' > b\left(1 + \frac{\theta(1+x)}{L}\right), \qquad x = \log_2(r/b).$$

Since $1 < x \le L$ and $\theta \le 1/2$, we have $0 \le \theta(1+x)/L \le 2\theta \le 1$. For $0 \le u \le 1$, concavity gives $\log_2(1+u) \ge u$, by comparing with the chord joining $(0,0)$ and $(1,1)$. Therefore

$$x' = \log_2(r/b') = x - \log_2(b'/b) < x - \log_2\left(1 + \frac{\theta(1+x)}{L}\right) \le x - \frac{\theta(1+x)}{L} \le \left(1 - \frac{\theta}{L}\right) x. \tag{113}$$

Thus each failed test reduces the logarithmic size deficit x by a factor of at least $1 - \theta/L$. Starting from $x \le L$, after $O(L \log L)$ failures we must have $x \le 1$, meaning that the ball has become heavy. Once heavy, every further rejection grows its mass by at least $1 + \theta/L$, so there can be only $O(L)$ additional rejections before the mass would exceed r.

Fix a sufficiently large constant C_0 and use

$$Q = 8\,\lceil C_0 L \log_2(2L) \rceil , \qquad \delta = \frac{d}{Q}. \tag{114}$$

Every search finds a light thin shell or reaches half the residual within radius $d/8$. If continued after becoming heavy, it finds a thin shell within another $d/8$. Thus every returned pad has radius at most $d/4$. Our earlier merge bound now gives $q = Q + 1 = O(L \log L)$ sweeps for each recursive instance. For a heavy ball, (112) also gives $\sigma \le 2\theta r/L$.

One residual and two directions. The ball direction determines the order of its split. An outward ball (C, B) yields

$$(B,\ R \setminus C). \tag{115}$$

Its exclusive regions are C and $R \setminus B$; going from the former to the latter costs at least δ in $H[R]$, by the triangle inequality for distance from z. Thus this split is δ-padded. For an inward ball, the separated direction is from $R \setminus B$ into C, so the valid order is $(R \setminus C, B)$.

Maintain one residual R, initially S, inside one ordered cover, initially (S). While $M(R) > M/2$, choose $z \in R$ and race the two ball searches in $H[R]$, advancing them with comparable work budgets. If either finds a thin *light* ball first, replace R by its ordered padded split and remove C from the residual. Its padded child stays in the cover. The cores removed in successive rounds are disjoint. Inward and outward choices may alternate because each operation is an ordered padded refinement.

If neither orientation finds a light ball, both searches have reached at least half the residual. Continue them to thin heavy shells (C^+, B^+) and (C^-, B^-) around the same center z. Instead of extracting a light piece, use the two balls to split R into three ordered children:

$$\big(B^+ \setminus C^-,\quad B^+ \cap B^-,\quad R \setminus C^+\big). \tag{116}$$

This is the outward split $(B^+, R \setminus C^+)$ followed by the inward split $(B^+ \setminus C^-, B^+ \cap B^-)$ inside its first member. The second split remains padded because restricting to B^+ can only increase induced distances. Each side child omits one of the heavy cores and so has mass at most $r/2$.

The middle child has a different advantage. Every vertex in it lies in both pads, whose radii are at most $d/4$. Hence it can be reached from z and can reach z in H within length $d/4$. It receives the smaller recursive scale $d/2$, with z as its anchor. It need not be small: the two-sided reachability gives the progress that a mass reduction would otherwise provide.

Algorithm 59 Constructing a padded ordered cover

```
function Cover(S, d)
    R ← S; 𝒴 ← (S)
    while M(R) > M(S)/2 do
        Choose z ∈ R; race inward and outward searches in H[R]
        if one search finds a thin light core C and pad B then
            Replace R by (B, R \ C) if outward, or (R \ C, B) if inward
            R ← R \ C
        else
            Continue both heavy searches to thin core–pad pairs
            Replace R by the triple in (116)
            Give the middle child scale d/2; other children scale d
            return the ordered cover and child scales
    return the cover with scale d on every child
```

Each refinement pays for its own overlap. We now prove that the children created by the ball searches are not too expensive. A split earns the potential decrease it would have without padding; any duplicated shell is charged against that saving. Because the algorithm always refines the current residual, these local charges telescope without reassigning vertices to the final children.

For a light core of mass $b < r/2$, the ideal saving is $\kappa(r, b)$. With $x = \log_2(r/b) > 1$, the shell satisfies

$$\sigma \le \frac{\theta}{L} b(1 + x) \le \frac{2\theta}{L}\, bx \le \frac{2\theta}{L} \kappa(r, b). \tag{117}$$

Duplicating σ units costs at most $6L\sigma$, leaving at least $(1 - 12\theta)\kappa(r, b) \ge b$ actual credits.

For a heavy split, first assign vertices disjointly to $C^+ \setminus C^-$, $C^+ \cap C^-$, and $R \setminus C^+$. Each side core belongs to a child of mass at most $r/2$, so every unit in it earns at least one size credit. The central core belongs to a

child with half the scale, so every unit in it earns one *distance* credit. The ideal saving is therefore at least r. The two heavy shells together contain at most $4\theta r/L$ mass; by (111) they consume at most $24\theta r$ credits. The actual decrease is at least $r/2$.

Let $M = M(S)$ and write Π for the decrease in Ψ from the parent to the final overlapping children. If the construction ends after light balls, they removed total core mass at least $M/2$. If it ends with a heavy split on residual mass r, previous light balls earn at least $M - r$ credits and the heavy split earns $r/2$. In either case,

$$\Pi = \Psi(M, d) - \sum_i \Psi(M(Y_i), d_i) \geq M/2. \tag{118}$$

Moreover every shell has mass $O(1/L)$ times the actual saving of its refinement. Summing gives the total duplicated mass and child incidence mass:

$$\mathcal{D} = \sum_i M(Y_i) - M = O(\Pi/L), \qquad I = \sum_i M(Y_i) = O(M). \tag{119}$$

These inequalities show both why the recursion progresses and why padding does not erase that progress.

The cover itself is built in $O(M)$ time. Each search is an incremental Dijkstra exploration, pausing at successive radii rather than restarting; a successful light search examines $O(M(B)) = O(M(C))$ mass by the shell rule. Balancing the work of the two orientations keeps the unsuccessful search within the same bound. Accepted cores are disjoint, while the final heavy case costs $O(M(R))$ only once. Every same-scale child has mass at most $(1/2 + O(\theta))M$, and only a half-scale middle child may be larger.

The recursive scaling step. Choose a power-of-two initial scale d_0 with $nW \leq d_0 < 2nW$. Solve (V, d_0) recursively. A same-scale child inherits an existing anchor when there is one; the half-scale middle child of a heavy split obtains an anchor from its center. Thus every instance satisfies either $d = d_0$ or (101). As discussed at the start, a failed merge returns either a nonpositive c-cycle (immediately a negative w-cycle) or a walk with (100). Below the root scale, the anchor closes that walk with a return path of w-weight at most d; at the root scale, removing nonnegative cycles would otherwise leave a simple path of weight below $-d_0 \leq -nW$, which is impossible. The terminal case $d < W/2$ was also handled at the beginning. All failures therefore yield a negative cycle in the original weights.

Algorithm 60 One negative-weight scaling step

```
function SOLVE(S, d)
    if S = ∅ then
        return the empty potential
    if d < W/2 then
        if every c-arc in G[S] is nonnegative then
            return the zero potential
        else
            return a negative w-cycle using the anchor
    (𝒴, (d_i)) ← COVER(S, d)
    for each child Y_i do
        φ_i ← SOLVE(Y_i, d_i); propagate any negative-cycle certificate
    return MERGE(S, d, d/Q, (Y_i, φ_i)_i), converting a failure to a negative cycle
```

Every same-scale child has a constant-factor smaller mass; a potentially large middle child halves d. Thus the recursion terminates. At an instance of mass M, the cover costs $O(M)$, its incidence mass is $O(M)$, and the merge costs $O(QM)$. Since the potential decreases by $\Pi = \Omega(M)$, this work can be charged to $O(Q\Pi)$.

Summing these charges telescopes the potential:

$$\sum_{\text{nonterminal } S} \Pi_S = \Psi(M_0, d_0) - \sum_{\text{terminal } S} \Psi(M(S), d(S)) \leq \Psi(M_0, d_0) = O(M_0 L),$$

where terminal potentials are nonnegative. Consequently, one scaling step takes

$$O(M_0QL) = O(M_0L^2 \log(2L)). \tag{120}$$

Terminal scans are charged to their parents' incidence mass, and one final clipped-graph Dijkstra search recovers an anchor return path if necessary.

Exact distances by scaling and rounding. Maintain a potential p and a power-of-two bound $K > 0$ such that the current reduced weights $a(u, v) = w(u, v) + p(u) - p(v)$ satisfy $a(e) \geq -K$. Start with $p = 0$ and $K \geq W$. Apply the scaling-step solver to a using bound K. If it finds a potential h feasible for $a + K/2$, replace p by $p + h$ and K by $K/2$. If it finds a negative cycle, its weight is negative under the original weights as well.

After $O(\log(nW))$ halvings, choose K with $nK < 1/2$. There can be no negative cycle: any negative simple cycle has integral original weight at most -1, whereas summing the reduced weights around a cycle gives its original weight, and every reduced arc is at least $-K$, so the cycle would have weight greater than $-nK > -1/2$.

Run Dijkstra from the desired source s using nonnegative costs $a(e) + K$, and let $D(v)$ be the resulting distances. For each reachable v, the exact distance $\delta_a(v)$ under the reduced costs satisfies

$$\delta_a(v) \leq D(v) < \delta_a(v) + nK. \tag{121}$$

Indeed, adding K per edge cannot shorten any path; applied to an optimal simple path, it increases its weight by less than nK. Since $\delta_a(v) = \mathrm{dist}_w(s, v) + p(s) - p(v)$ and original distances are integral, we recover them exactly as

$$\mathrm{dist}_w(s, v) = \lfloor D(v) - p(s) + p(v) \rfloor . \tag{122}$$

Unreachable vertices remain unreachable. All dyadic quantities can be scaled to integers for the Dijkstra computations.

Theorem 434 (Deterministic negative-weight shortest paths)**.** *For a directed graph with integral weights at least $-W$, one can find a negative-weight cycle or compute exact single-source shortest paths deterministically in*

$$O\big((m + n \log\log n) \log^2 n \log\log n \log(nW)\big)$$

time.

The $\log\log n$ factor here can be removed with significantly more effort.

12.2 Low-Diameter Decompositions

12.2.1 Undirected Low-Diameter Decompositions

Let $G = (V, E, \ell)$ be an undirected graph with nonnegative edge lengths. Given $D > 0$, we want a partition into clusters of *strong* diameter at most D, while cutting each edge e with probability $O(\ell(e) \log n/D)$. The idea is to give each vertex a random head start and assign every vertex to the center that reaches it first. Exponential head starts make close competitions rare; bounding the largest head start bounds the cluster diameters.

Set $a = D/2$ and $\beta = 4 \log(2n)/D$. Independently sample $\delta_s \sim \mathrm{Exp}(\beta)$ for every $s \in V$, where $\Pr[\delta_s > t] = e^{-\beta t}$, and resample if any $\delta_s > a$. Assign v to a center

$$c(v) \in \arg\min_{s \in V} \{d(s, v) - \delta_s\}.$$

This takes one Dijkstra computation: add a source r with an arc $r \to s$ of length $a - \delta_s \geq 0$ for each s, and label each vertex by the first vertex after r on its shortest-path-tree path.

Theorem 435 (Undirected low-diameter decomposition)**.** *The resulting clusters have strong diameter at most D, and every edge e is cut with probability $O(\ell(e)\log n/D)$. The expected running time is $O(m + n\log n)$.*

Proof. The sampling is accepted with probability at least $1 - ne^{-\beta a} \geq 3/4$, so its expected cost is $O(n)$. If $c(v) = s$, comparison with the candidate center v gives $d(s,v) - \delta_s \leq -\delta_v$, hence $d(s,v) \leq a$. Every shortest s–v path stays in the cluster of s: if x lies on such a path, then for any center t,

$$\delta_s - d(s,x) = \delta_s - d(s,v) + d(x,v) \geq \delta_t - d(t,v) + d(x,v) \geq \delta_t - d(t,x).$$

Thus each cluster has strong radius at most a, and strong diameter at most $2a = D$.

For the cutting bound, first use unconditioned exponential shifts. Fix v and let $X_s = \delta_s - d(s,v)$; write $g(v)$ for the gap between the largest and second-largest scores. Exponential memorylessness gives

$$\Pr[g(v) \leq t] \leq 1 - e^{-\beta t}. \tag{123}$$

Indeed, fix s and condition on all other shifts. Winning requires $\delta_s \geq b$ for a fixed threshold b; conditional on winning, the chance that $\delta_s - b \leq t$ is at most $1 - e^{-\beta t}$ (also when $b < 0$). Sum this bound over the possible winners.

For an edge $e = uv$ of length w, each score $\delta_s - d(s,\cdot)$ changes by at most w between u and v. If their winning centers differ, then $g(u) \leq 2w$. Conditioning on acceptance of the shifts therefore gives

$$\Pr[e \text{ is cut}] \leq \frac{1 - e^{-2\beta w}}{3/4} \leq \frac{8}{3}\beta w = O\left(\frac{w\log n}{D}\right).$$

Only one Dijkstra computation is needed after sampling, taking $O(m + n\log n)$ time with an appropriate heap. □

12.2.2 Directed Low-Diameter Decompositions and Negative Weight Shortest Paths

Let $G = (V, E, w)$ be a directed graph with nonnegative arc lengths. A *low-diameter decomposition* (LDD) removes a random set of arcs F such that every strongly connected component of $G - F$ has *weak diameter* at most D: for u, v in the same component, $d_G(u,v) \leq D$, where the connecting path may leave the component. Its *loss* is the least ℓ satisfying

$$\Pr[e \in F] \leq \min\left\{1, \frac{\ell w(e)}{D}\right\} \qquad (e \in E). \tag{124}$$

We obtain loss $O(\log n \log\log n)$ in expected time $O((m + n\log\log n)\log n)$ for polynomially bounded integral lengths.

An outward ball has short paths from its center, but need not have small directed diameter. We can nevertheless separate it from the rest of the graph by deleting only its outgoing boundary, then recurse inside it. There are two kinds of progress: a ball may be small enough to recurse on, or a region may have short paths in *both* directions and already satisfy the diameter bound. We use these alternatives to construct smaller instances, and charge the search work and cutting probabilities to the size reduction.

Discard self-loops and retain only the shortest arc of each ordered pair, assigning its cut decision to all parallel copies. Hence $m \leq n^2$. If $D < 1$, delete every positive-length arc; below assume $D \geq 1$. We process initial SCCs independently and ignore isolated vertices.

Degree volume and one-sided carving. Fix a recursive instance H with M arcs. Throughout an attempt on H, use its original degrees to define

$$T = 2M, \qquad \mu(X) = \sum_{v \in X} \deg_H(v). \tag{125}$$

An arc internal to X contributes two to $\mu(X)$, and an arc crossing its boundary contributes one. Therefore

$$|E(H[X])| \le \frac{\mu(X)}{2}, \qquad |E(H[V(H) \setminus X])| \le M - \frac{\mu(X)}{2}. \tag{126}$$

Thus a small-volume set contains few arcs, while every component outside a large-volume set contains few arcs. Keeping degrees fixed as vertices are removed makes volume suitable for sampling as well.

Write $B_J^+(s,r) = \{v : d_J(s,v) \le r\}$ and $B_J^-(s,r) = \{v : d_J(v,s) \le r\}$. A *$\sigma$-pass*, $\sigma \in \{+,-\}$, fixes a graph snapshot J, some centers, and a common radius r. It processes the centers in random order. If U is the current remainder, it carves $P = B_J^\sigma(s,r) \cap U$. For $\sigma = +$ it deletes arcs from P to $U \setminus P$; for $\sigma = -$ it deletes arcs from $U \setminus P$ to P. Then P is removed from U.

Only one boundary direction is deleted. In the outward case a surviving directed cycle cannot leave P; in the inward case it cannot enter P. Hence no SCC can meet both P and $U \setminus P$. Successive pieces and the final remainder can be treated independently. Throughout the pass distances are measured in the *fixed* graph J, even along paths through previously carved vertices. Moreover all sampled centers are processed, including those already carved. These conventions are essential: a center sampled inside a ball must still reach its target, regardless of earlier carving.

Lemma 436 (Random-order carving)**.** *Suppose a σ-pass uses k fixed centers in random order and a common radius uniform in $[a,b]$, where $a < b$. It cuts an arc e with probability at most $H_k w(e)/(b-a)$, where $H_k = \sum_{j=1}^{k} 1/j \le 1 + \ln(k+1)$. Its searches can be implemented so that the number R_v of scans at a vertex v satisfies*

$$\mathbb{E}R_v \le H_k, \qquad \mathbb{E}[2^{R_v}] \le k+1. \tag{127}$$

A scan costs $O(\deg_J(v) + \log\log n)$ time.

Proof. Consider an outward pass and $e = u \to v$. A center s can cut e only if $d_J(s,u) \le r < d_J(s,v) \le d_J(s,u) + w(e)$, so the permissible radii have total length at most $w(e)$. Rank the centers by their distance to u, breaking ties consistently. If the center of rank j cuts e, none of the $j-1$ closer centers can have been processed first: at radius $r \ge d_J(s,u)$, any such center would already have carved u. The probability is therefore at most $1/j$, and summing over ranks proves the cutting bound. The inward case reverses the arcs.

For the searches, store at each vertex the smallest distance reached by an earlier center. If a new search reaches v at a distance no smaller than this record, it need not continue through v: every radius-r continuation would already have been reached by the earlier center. Thus a vertex is scanned only at record minima among the center distances. In random order, the record indicators are independent with means $1, 1/2, \dots, 1/k$ (zero or infinite distances can only reduce the scan count). Hence, since for an indicator I we have $2^I = 1 + I$, $\mathbb{E}2^{R_v} \le \prod_{j=1}^{k}(1 + 1/j) = k+1$. □

To sample centers at rate ρ, take $\lceil \rho \rceil$ independent draws from $\Pr[s = v] = \deg_H(v)/T$, discard duplicates and vertices outside the current snapshot, and use every nonisolated vertex instead if $\rho \ge T$. For any fixed set X in the snapshot,

$$\Pr[S \cap X = \varnothing] \le \left(1 - \frac{\mu(X)}{T}\right)^{\lceil \rho \rceil} \le e^{-\rho\mu(X)/T}. \tag{128}$$

One alias table, constructed in $O(M)$ time, supports all these draws in $O(1)$ time each.

The main use of the sampling bound is *opposite-direction coverage*. If $s \in B_J^+(u,r)$, then $u \in B_J^-(s,r)$, and conversely. Hence an inward pass of radius at least r removes u whenever it samples any vertex of $B_J^+(u,r)$; an outward pass similarly covers a large inward ball. In particular, a vertex whose ball in the opposite direction has volume greater than T/q survives with probability at most $e^{-\rho/q}$. The ball and the snapshot must be fixed before the centers of this pass are sampled.

Pivot, peel, and bootstrap. A decomposition attempt either uses a two-sided pivot immediately or peels small one-sided balls and then runs a sequence of CKR passes. We call the first pass after peeling the *bootstrap pass*; the subsequent, doubly exponential passes are numbered $1, 2, \ldots$. The bootstrap uses constant sampling and has a weaker size bound. Its purpose is to make the first numbered pass safe.

Draw K degree-weighted samples, which we call probes, for a sufficiently large constant K. For $\sigma \in \{+, -\}$, call s *σ-small* when $\mu(B_H^\sigma(s, D/4)) \le T/4$.

If some probe s is large in both directions, independently choose r_+, r_- uniformly from $[D/4, D/2]$ and cut the outgoing boundary of $B_H^+(s, r_+)$ and the incoming boundary of $B_H^-(s, r_-)$. Every resulting SCC is either contained in or disjoint from each ball. An SCC in their intersection has weak diameter at most D: the route $u \to s \to v$ has length at most $r_- + r_+ \le D$. Every other SCC avoids a ball of volume greater than $T/4$, so it contains fewer than $3M/4$ arcs by Equation (126). We accept this without further passes.

Otherwise every probe is small in at least one direction, so some σ has at least $K/2$ σ-small probes. Process their σ-balls in random order using one radius uniform in $[D/8, D/4]$. Each carved piece has volume at most $T/4$, although their union need not be small. On the remainder, run the *bootstrap pass*: for a sufficiently large constant b we draw $\lceil b \rceil$ fresh degree-weighted centers, carve in direction $-\sigma$ at one radius uniform in $[D/12, D/8]$, and reject the entire attempt if any piece has volume exceeding $3T/4$.

The peel makes such a rejection unlikely. Fix u and a direction σ, and suppose $W = B_H^{-\sigma}(u, D/8)$ has $\mu(W) > 3T/4$. Any selected σ-small center in W has u in its radius-$D/8$ σ-ball and hence removes u. Thus, if u survives the peel in direction σ, at least $K/2$ of the original probes lie outside W, a set of sampling mass below $1/4$. Consequently,

$$\Pr[u \text{ survives the peel and } \mu(W) > 3T/4] \le \Pr[\mathrm{Bin}(K, 1/4) \ge K/2] \le e^{-K/12}. \tag{129}$$

A bootstrap piece can exceed $3T/4$ only if its center has such a large $(-\sigma)$-ball. Taking a union bound over both choices of σ and the $\lceil b \rceil$ fresh centers gives rejection probability at most $2\lceil b \rceil e^{-K/12}$.

Alternating multiscale passes. The bootstrap carves $(-\sigma)$-balls. Its purpose is to make large σ-balls unlikely among the surviving vertices, so the next pass can carve in direction σ. Repeating this argument lets us alternate directions, using just one CKR pass at each scale.

Throughout a pass, distances are still measured in the graph induced by the vertices present at its beginning, its snapshot. Centers are processed in random order, including centers assigned to earlier pieces during the same pass. Each piece is the intersection of its ball with the vertices still unassigned. We delete its outgoing boundary for an outward pass, or its incoming boundary for an inward pass. Consequently, no remaining SCC meets two different pieces.

For $M > 16$, set

$$\begin{aligned} &J = \lceil \log_2 \log_2 M \rceil - 1, \qquad s_i = 2^{2^i} \quad (1 \le i \le J), \\ &a_0 = D/12, \qquad \delta = \frac{D}{32(J+1)}, \qquad a_i = a_{i-1} - \delta. \end{aligned} \tag{130}$$

Pass i carves in the direction opposite to its predecessor, starting with direction σ after the bootstrap. It chooses a common radius uniformly in $[a_i, a_{i-1}]$ and requires every carved piece P to satisfy

$$\mu(P) \le T/s_i, \qquad T = 2M. \tag{131}$$

If this fails, we discard the entire attempt and resample. Since every continuing SCC inside P has at most $\mu(P)/2 \le M/s_i$ arcs, acceptance guarantees a logarithmic size reduction of at least $\log_2 s_i = 2^i$.

The radius intervals are decreasing: every radius in the preceding pass is at least every radius in the next pass. Their total width is less than $D/32$. This allows the preceding pass to eliminate potentially oversized centers of the next pass.

Lemma 437 (Coverage by the preceding pass)**.** *Suppose pass i carves in direction σ_i, and the preceding pass makes q independent draws in direction $-\sigma_i$. If the preceding pass has minimum radius at least a_{i-1}, then,*

for every fixed vertex u,

$$\Pr\left[\begin{array}{c} u \text{ survives the preceding pass, and} \\ \mu(B^{\sigma_i}_{H_i}(u, a_{i-1})) > T/s_i \end{array}\right] \le e^{-q/s_i},$$

where H_i is the snapshot of pass i.

Proof. Fix the snapshot H_{i-1} before the preceding pass and consider

$$W = B^{\sigma_i}_{H_{i-1}}(u, a_{i-1}).$$

The next snapshot is an induced subgraph of H_{i-1}. Hence, if u survives and its ball in H_i has volume greater than T/s_i, then $\mu(W) > T/s_i$.

Every $s \in W$ satisfies $u \in B^{-\sigma_i}_{H_{i-1}}(s, a_{i-1})$. Sampling such an s in the preceding pass therefore removes u, unless it was removed already. This holds even if s was assigned to an earlier piece during that pass, since all sampled centers are processed using the fixed snapshot.

The set W is fixed before the preceding samples are drawn. Each draw belongs to W with probability $\mu(W)/T > 1/s_i$. Thus survival requires all q draws to miss W, and has probability at most

$$\left(1 - \frac{\mu(W)}{T}\right)^q \le e^{-q/s_i}.$$

□

We now choose the sampling rates. If pass i makes N_i fresh degree-weighted draws, the lemma and a union bound give

$$\Pr[\text{pass } i \text{ rejects}] \le N_i e^{-q_{i-1}/s_i}, \tag{132}$$

where q_{i-1} is the number of draws in the preceding pass. For a final pass that processes all remaining vertices, the same bound holds by a union bound over the at most T possible centers.

There are two competing requirements. With ρ_i sampled centers, random-order CKR scans a vertex $O(\log(\rho_i + 1))$ times in expectation. We want this to be $O(\log s_i)$, so it can be paid for by the size reduction. At the same time, the preceding sampling rate must make $\rho_i e^{-\rho_{i-1}/s_i}$ very small.

Both requirements are met by

$$\rho_i = A 2^{i+1} s_i^2 = A 2^{i+1} 2^{2^{i+1}} \qquad (1 \le i \le J), \tag{133}$$

where A is a sufficiently large constant. Since $s_i = s_{i-1}^2$, for $i \ge 2$ we have

$$\frac{\rho_{i-1}}{s_i} = A 2^i, \qquad \log(\rho_i + 1) = O(2^i), \qquad \Pr[\text{pass } i \text{ rejects}] = e^{-\Omega(2^i)}. \tag{134}$$

For pass 1, whose predecessor is the bootstrap with $\lceil b \rceil$ draws, the rejection probability is at most $\lceil \rho_1 \rceil e^{-b/4}$.

Normally pass i makes $\lceil \rho_i \rceil$ independent draws, ignoring centers absent from its snapshot. If $\rho_i \ge T$, it instead processes every nonisolated vertex of the snapshot in random order and terminates the sequence of passes. There are at most T such vertices, so this does not increase the work or the rejection bound. It also empties the remainder, since every remaining vertex is a center. Such a pass occurs by $i = J$: $2^J \ge \frac{1}{2} \log_2 M$ implies $s_J^2 \ge M$, and hence $\rho_J \ge T$.

Algorithm 61 Directed low-diameter decomposition

```
function DirectedLDD(H, D)
    if |E(H)| ≤ 16 then
        return the constant-size two-sided-pivot decomposition
    M ← |E(H)|; construct the sampler
    Set J, (s_i, ρ_i), (a_i) as above
    repeat
        Start with the original graph H and no deleted arcs
        Draw K probes and test their inward and outward radius-D/4 balls
        if some probe is large in both directions then
            Cut its inward and outward balls
            Mark SCCs inside their intersection terminal
        else
            Choose σ with at least K/2 σ-small probes
            Peel their σ-balls at a common radius in [D/8, D/4]
            Bootstrap: carve (−σ)-balls using ⌈b⌉ fresh draws and radius in [D/12, D/8]
            Reject if some bootstrap piece has volume > 3T/4
            for i = 1, ..., J while the attempt survives do
                Reverse the carving direction
                Choose the radius uniformly in [a_i, a_{i−1}]
                if ρ_i < T then
                    Carve using ⌈ρ_i⌉ fresh draws
                else
                    Carve using all remaining nonisolated centers in random order
                Reject if some carved piece P has μ(P) > T/s_i
                if ρ_i ≥ T then
                    break
    until the attempt is accepted
    Recurse on every nonterminal SCC of the accepted graph
    return the union of deleted arcs
```

Each attempt is discarded completely if a size test fails, including all tentative cuts. In the pivot case, an SCC inside the intersection has weak diameter at most D, using the route $u \to s \to v$ through the pivot. Every other SCC omits one ball of volume greater than $T/4$ and therefore has fewer than $3M/4$ arcs. In the peel case, peeled pieces have volume at most $T/4$, bootstrap pieces at most $3T/4$, and numbered pieces satisfy Equation (131). Thus every nonterminal recursive child has at most $3M/4$ arcs.

For $M \le 16$, there are at most 32 nonisolated vertices. Choose a center in a nonterminal SCC and cut its independent inward and outward balls at radii uniform in $[D/4, D/2]$. Every SCC in their intersection is terminal, while each remaining nonterminal SCC omits the chosen center. At most 32 such pivots are needed, and the cutting probability of each arc is $O(w(e)/D)$.

Expected running time. The bootstrap rejects with probability at most $2\lceil b\rceil e^{-K/12}$, by the peel's sampling bound. Pass 1 rejects with probability at most $\lceil \rho_1\rceil e^{-b/4}$, and later rejection probabilities are $e^{-\Omega(2^i)}$ by Equation (134). Choose A, then b, then K sufficiently large. Every attempt is accepted with probability at least $p = 3/4$.

Let η_i denote the rejection bounds for numbered passes, and put $\tau_i = \sum_{j\ge i} \eta_j$. Their rapid decay gives

$$\sum_{i\ge 1} 2^i \tau_i = O(1). \tag{135}$$

We will use this to pay for the work of attempts that are rejected after reaching an expensive pass.

Fix a vertex v and let $R_i(v)$ count its record-minimum Dijkstra scans in pass i, taking $R_i(v) = 0$ if the pass is not reached. There are at most $N_i = \min\{\lceil \rho_i\rceil, T\}$ centers, so the random-order record bound Lemma 436

gives $\mathbb{E}[2^{R_i(v)}] \le N_i + 1$. For a sufficiently large constant B, define the excess $Z_i(v) = (R_i(v) - B2^i)_+$. A tail sum yields

$$\mathbb{E}Z_i(v) \le 2(N_i+1)2^{-B2^i} = e^{-\Omega(2^i)}, \qquad \sum_{i\ge 1} \mathbb{E}Z_i(v) = O(1). \tag{136}$$

Thus pass i uses at most $B2^i$ scans at v, apart from a summably small expected excess.

First consider an accepted attempt. If v enters a recursive child with M' arcs after pass j, its total scan allowance is $\sum_{i\le j} B2^i = O(2^j)$. But the size guard gives

$$\log_2 \frac{M}{M'} \ge \log_2 s_j = 2^j. \tag{137}$$

The pivot, peel, and bootstrap require only constant allowance and give a constant-factor size reduction. If v becomes terminal, its allowance is at most $O(2^J) = O(\log M)$. The excess has $O(1)$ expectation per accepted attempt, since conditioning on acceptance increases its expectation by at most $1/p$.

Next consider a rejected attempt. If pass i is reached, rejection can occur only in that or a later pass. Consequently,

$$\mathbb{E}\left[\mathbf{1}_{\text{reject}} \sum_{i\ge 1} R_i(v)\right] \le B\sum_{i\ge 1} 2^i\tau_i + \sum_{i\ge 1}\mathbb{E}Z_i(v) = O(1). \tag{138}$$

The initializer and bootstrap also contribute constant expected scans. Since each attempt succeeds with probability at least $3/4$, restarts increase these bounds by only a constant factor.

Accepted scan allowances telescope along each vertex's recursive chain through the decreases in $\log M$. Every continuing child has at most $3M/4$ arcs, so the $O(1)$ excess and initialization costs are paid at most $O(\log m)$ times. Hence every vertex has $O(\log m)$ expected scans in total.

There is also $O(M)$ non-Dijkstra work per attempt. The sampling rates grow geometrically, and sampling stops as soon as the rate reaches T; therefore only $O(M)$ centers are generated. Sampling takes $O(1)$ time per draw, and piece volumes are checked by summing parent degrees over disjoint assigned sets. Timestamped arrays avoid resetting vertex state between passes. Each scan of v costs $O(\deg_H(v) + \log\log n)$ using integer-priority-queue Dijkstra. Thus

$$\mathbb{E}[T_{\text{LDD}}] = O((m + n\log\log n)\log n). \tag{139}$$

Cutting loss. We need to account for the fact that an accepted attempt is conditioned on passing its size tests in our analysis. The same rejection tails control this conditioning.

Fix an arc e, and let $\mathcal{A}$ denote acceptance, with $p = \Pr(\mathcal{A}) \ge 3/4$. Let E_i be the event that pass i is reached with both endpoints of e still in its remainder. Conditional on the preceding passes, the random-order CKR bound is

$$\Pr[e \text{ cut in pass } i \mid E_i,\ \text{preceding history}] \le \alpha_i(e) := \frac{H_{N_i} w(e)}{\delta} = O\left(\frac{w(e)(J+1)2^i}{D}\right). \tag{140}$$

Here $H_k = \sum_{j=1}^k 1/j$; replacing the actual number of centers by its deterministic upper bound N_i is harmless.

Since an attempt reaching pass i can be rejected only in that or a later pass, $\Pr(E_i \cap \mathcal{A}^c) \le \tau_i$. Therefore

$$\Pr[e \text{ cut in pass } i \mid \mathcal{A}] \le \frac{\alpha_i(e)\Pr(E_i)}{p} \le \alpha_i(e)\left(\Pr(E_i \mid \mathcal{A}) + \frac{\tau_i}{p}\right). \tag{141}$$

Summing the correction terms over all numbered passes gives

$$O\left(\frac{w(e)(J+1)}{D}\sum_{i\ge 1} 2^i\tau_i\right) = O\left(\frac{w(e)(J+1)}{D}\right).$$

For the main terms, let j be the last numbered pass to which e is exposed in an accepted attempt. Their total contribution is $O(w(e)(J+1)2^j/D)$. If e survives into a recursive child with M' arcs, Equation (137) charges this to

$$O\left(\frac{w(e)(J+1)}{D}\log\frac{M}{M'}\right).$$

Otherwise recursion stops charging e, and $2^j \le 2^J = O(\log M)$. The pivot, peel, and bootstrap contribute $O(w(e)/D)$ per invocation, including conditioning on acceptance.

The logarithmic size decreases telescope along the recursive chain of e. Every continuing child contains at most $3M/4$ arcs, so the conditioning correction is charged at most $O(\log m)$ times. Since $J = O(\log\log m)$, we obtain

$$\Pr[e \in F] = O\left(\min\left\{1, \frac{w(e)\log n\log\log n}{D}\right\}\right). \tag{142}$$

An arc of length zero is never cut: every outward ball is closed under outgoing zero-length arcs, and every inward ball is closed under incoming zero-length arcs.

Theorem 438 (Directed low-diameter decomposition)**.** *For nonnegative polynomially bounded integral arc lengths, a directed low-diameter decomposition of weak diameter at most D and loss $O(\log n\log\log n)$ can be constructed in expected time*

$$O((m + n\log\log n)\log n).$$

This also implies a randomized $O((m+n\log\log n)\log n\log\log n\log(nW))$ time algorithm for negative-weight SSSP through interlacing the construction with scaling methods similar to Section 12.1.3; our $\log\log n$ savings on construction is not useful here.

12.3 Minimum Cut

12.3.1 Directed Minimum Cut in Almost-Linear Time

Let $G = (V, E, w)$ be a directed multigraph with positive integral arc weights at most W. Fix a root r, and write

$$\delta^-(S) = E(V \setminus S, S), \qquad \lambda = \min_{\varnothing \ne S \subseteq V\setminus\{r\}} w(\delta^-(S)).$$

We first find this *rooted* minimum cut. The global minimum cut is the smaller of the rooted minima in G and its reversal: a minimum-cut side either avoids r, or its complement avoids r in the reversed graph. If some vertex is unreachable from r, the unreachable set is a zero cut. Otherwise, fix an *r-arborescence* T, a spanning tree directed away from r.

A certifying tree update. Every r-arborescence enters every nonempty set avoiding r, so it uses at least one edge of every rooted cut. Given a proposed lower bound $L > 0$ on λ, normalize the capacities to $q(e) = w(e)/L$. If $L > \lambda$ and S^* is a minimum rooted cut, then

$$q(\delta^-(S^*)) = \lambda/L < 1,$$

whereas every arborescence T satisfies $\mathbf{1}_T(\delta^-(S^*)) \ge 1$. In particular, no distribution on arborescences can have inclusion probabilities bounded by q.

We will not try to construct such a distribution directly. Instead, starting from any arborescence T, we repeatedly perform one of two operations:

$$\begin{aligned}&\text{find a rooted cut } S \text{ with } w(\delta^-(S)) < L,\\ &\text{or sample } T' \text{ with } \mathbb{E}[\mathbf{1}_{T'} \mid T] \le \tfrac{1}{2}(\mathbf{1}_T + q).\end{aligned} \tag{143}$$

The second outcome moves the edge marginals halfway toward q. When $L > \lambda$, these updates cannot continue for long: on S^*, the expected number of tree edges decreases toward a value below one, which no arborescence can attain. We will show that an update then fails *constructively*, yielding the cut in the first outcome. This gives a one-sided test for $L \le \lambda$, which can be used in binary search.

Why two trees are easier than one. Suppose we could sample another arborescence with edge marginals at most q. We could average it with T, but obtaining that second tree would amount to constructing the fractional arborescence packing we wanted to avoid. We instead choose an arbitrary set F of $n-1$ arcs, not necessarily forming a tree, such that $T+F$ is the union of two arc-disjoint r-arborescences. Choosing one of these two trees uniformly gives

$$\mathbb{E}[\mathbf{1}_{T'} \mid T] = \frac{\mathbf{1}_T + \mathbb{E}[\mathbf{1}_F]}{2}. \tag{144}$$

It therefore suffices to construct F with $\mathbb{E}[\mathbf{1}_F] \le q$ and ensure that $T+F$ contains two disjoint arborescences. By Theorem 345, the latter condition is equivalent to

$$(\mathbf{1}_T + \mathbf{1}_F)(\delta^-(S)) \ge 2 \quad (\varnothing \ne S \subseteq V \setminus \{r\}). \tag{145}$$

The tree T already enters every rooted cut at least once. Thus only cuts entered *exactly once* by T need another edge. Moreover, two arborescences together have indegree two at each nonroot vertex, so F must choose exactly one incoming arc per vertex $v \ne r$. For a cut S entered once by T, the underlying tree $T[S]$ is connected and has $|S|-1$ edges, while $|S|$ selected arcs of F have their heads in S. To ensure that not all of them are internal to S, assign each selected arc $e=(u,v)$ to a *distinct* tree edge on the undirected u–v path $P_T(e)$. If e is internal to S, its entire path lies in $T[S]$, which has only $|S|-1$ edges. Therefore at least one selected arc enters S from outside, as desired.

This assignment has the interpretation of a tree exchange: adding e to the underlying tree creates a fundamental cycle, and any $f \in P_T(e)$ could be exchanged for e. We do not perform all these exchanges. We use distinct possible exchanges as *witnesses* that the $n-1$ selected arcs supply the missing connectivity. The assignment is a perfect matching between the selected incoming arcs and the edges of T, with allowed pair (e,f) whenever $f \in P_T(e)$.

A capacitated exchange matching. To enforce the marginal bounds, fractionalize this matching. Give every vertex $v \ne r$ one unit of supply, and every tree edge $f \in T$ one unit of demand. The unit at v can use an original arc $e=(u,v)$ up to capacity $q(e)$ and then reach any demand $f \in P_T(e)$. Equivalently, construct a flow network with three layers:

$$s_v \xrightarrow{q(e)} y_e \xrightarrow{\infty} t_f, \qquad e=(u,v), \quad f \in P_T(e). \tag{146}$$

Here s_v must send one unit for each $v \ne r$, and t_f must receive one for each $f \in T$. The flow value is $n-1$ if all supplies and demands can be satisfied. Let $h(e)$ be the total flow using $s_v \to y_e$; then

$$h(e) \le q(e), \qquad \sum_{e \in \delta^-(\{v\})} h(e) = 1 \ (v \ne r), \qquad \sum_e h(e) = n-1. \tag{147}$$

The key property is that *any* demand-satisfying flow provides the two-arborescence cut condition, even if the capacities $q(e)$ are ignored.

Lemma 439 (A matching certifies two arborescences)**.** *If the exchange network routes all $n-1$ units, then*

$$(\mathbf{1}_T + h)(\delta^-(S)) \ge 2 \qquad (\varnothing \ne S \subseteq V \setminus \{r\}). \tag{148}$$

Consequently, every integral demand-satisfying flow selects $n-1$ arcs F such that $T+F$ partitions into two r-arborescences.

Proof. Only cuts entered once by T require attention. Fix one, S. Every arc internal to S can send flow only to tree edges of $T[S]$. Since these $|S|-1$ demands receive one unit each, $h(E(S,S)) \le |S|-1$. Every vertex of S supplies one unit, so $h(E(V,S)) = |S|$. Subtracting yields

$$h(\delta^-(S)) = h(E(V,S)) - h(E(S,S)) \ge 1.$$

For an integral flow, each vertex chooses one arc and every tree edge is matched exactly once. Applying Theorem 345 to $T+F$ gives two disjoint r-arborescences. They use all $2(n-1)$ arcs of $T+F$. □

What if the matching is infeasible? This is where the tree-path restriction pays a second dividend: a failed auxiliary flow already identifies an original rooted cut below L. To see why, consider any set $J \subseteq E(T)$ of tree-edge demands. Let

$$E_J = \{e \in E : P_T(e) \cap J \neq \varnothing\}, \qquad b_v(J) = \min\left\{1, \sum_{e \in E_J \cap \delta^-(\{v\})} q(e)\right\}.$$

The vertex v can supply at most $b_v(J)$ units to the demands in J. Capacitated Hall's theorem (equivalently, max-flow/min-cut) says that all demands can be met precisely when

$$\sum_{v \neq r} b_v(J) \geq |J| \qquad \text{for every } J \subseteq E(T). \tag{149}$$

Lemma 440 (Flow or cut)**.** *If the exchange flow is infeasible, a minimum-cut certificate in its network yields a rooted set S with $w(\delta^-(S)) < L$. In particular, for $L \leq \lambda$ the flow is always feasible.*

Proof. An infeasible flow gives a set J violating Equation (149). Delete J from the underlying tree T, obtaining components $C_0, C_1, \ldots, C_{|J|}$, with $r \in C_0$. Every arc entering a component C_j, $j \geq 1$, has its tree path crossing J, so it belongs to E_J. For $v \in C_j$, let a_v be the q-capacity of arcs entering C_j at v. Then $b_v(J) \geq \min\{1, a_v\}$. Since truncating a sum at one cannot exceed the sum of its truncated terms,

$$\sum_{v \in C_j} b_v(J) \geq \sum_{v \in C_j} \min\{1, a_v\} \geq \min\left\{1, \sum_{v \in C_j} a_v\right\} = \min\{1, q(\delta^-(C_j))\}.$$

The components are disjoint, so their contributions involve disjoint vertices. If every nonroot component had incoming weight at least L, each would contribute at least one unit to the left side of Equation (149), a contradiction. Thus some C_j satisfies $q(\delta^-(C_j)) < 1$, or $w(\delta^-(C_j)) < L$. For $L \leq \lambda$, no such cut exists, so every Hall inequality holds and the flow is feasible. □

If the flow is feasible, decompose it into paths $s_v \to y_e \to t_f$, with weights $\gamma(v, e, f)$. Aggregate these over the arc labels: $x_{vf} = \sum_e \gamma(v, e, f)$. Each nonroot vertex supplies one unit and each tree edge receives one, so x is a fractional perfect matching. Sample an integral perfect matching preserving its pair marginals $\Pr[(v, f) \text{ matched}] = x_{vf}$, by randomized bipartite matching rounding. For each matched pair (v, f), choose its arc label e with probability $\gamma(v, e, f)/x_{vf}$. Thus, for every original arc $e = (u, v)$,

$$\Pr[e \in F] = \sum_{f : x_{vf} > 0} x_{vf} \frac{\gamma(v, e, f)}{x_{vf}} = \sum_f \gamma(v, e, f) = h(e) \leq q(e).$$

Every selected arc enters a different nonroot vertex, so F has exactly $n - 1$ arcs and the rounded assignment remains a valid perfect matching. The rounded matching still satisfies the demand constraints, so Lemma 439 applies, even though its individual selected arcs may exceed the original fractional capacities. Partition $T + F$ into two arborescences using Theorem 345, and choose one uniformly. We do this in $O(n \log n)$ time by Tarjan's two-arborescence algorithm. Then Equation (144) gives exactly the desired update:

$$\mathbb{E}[\mathbf{1}_{T'} \mid T] \leq \tfrac{1}{2}(\mathbf{1}_T + q). \tag{150}$$

An explicit network could have $\Theta(mn)$ arcs, since an original arc $e = (u, v)$ may be eligible for every tree edge on $P_T(e)$. We compress these relations using binary lifting. Root T and create a node for each dyadic ancestor path: the first 2^j tree edges above a vertex v. A block of length one represents its tree edge directly; every larger block has arcs to its two halves, so its node can reach exactly the tree edges it represents.

For each original arc $e = (u, v)$, find the lowest common ancestor of u, v and partition $P_T(e)$ into $O(\log n)$ dyadic blocks. Connect the node representing e to these blocks. It can now reach precisely the tree edges eligible for e, preserving the feasible flows. There are $O(n \log n)$ block nodes and $O((m + n) \log n)$ auxiliary arcs.

Thus one exchange step uses a single maximum-flow computation on a near-linear-size network. The additional work is $\widetilde{O}(m)$.

Why a false lower bound is exposed quickly. Fix $L > \lambda$ and a minimum rooted cut S^*, and let

$$X_t = \mathbf{1}_{T_t}(\delta^-(S^*)), \qquad c = q(\delta^-(S^*)) = \lambda/L < 1.$$

Every X_t is a positive integer. When a flow-or-mix step succeeds, Equation (150) implies

$$\mathbb{E}[X_{t+1} - 1 \mid T_t] \le \tfrac{1}{2}(X_t + c) - 1 \le \tfrac{1}{2}(X_t - 1). \tag{151}$$

In fact, when $X_t = 1$, the next flow *must* fail: Lemma 439 would give $h(\delta^-(S^*)) \ge 1$, but the capacities allow at most $c < 1$. Thus every successful step occurs while $X_t - 1 \ge 1$.

Stop at the first failed flow. Put $Y_t = X_t - 1$ if the first t steps succeeded and $Y_t = 0$ otherwise. The preceding contraction, including the possibility of stopping, gives $\mathbb{E}Y_t \le (n-2)2^{-t}$. If the first $t+1$ flows all succeed, then $X_t \ge 2$ and $Y_t \ge 1$. By Markov's inequality,

$$\Pr[\text{the first } t+1 \text{ flows all succeed}] \le (n-2)2^{-t}. \tag{152}$$

Hence $O(\log(n/\varepsilon))$ calls either find a cut below L or certify $L \le \lambda$ with one-sided error at most ε. The bound does not depend on the gap $L - \lambda$.

Exact minimum cut. For integral weights, we can binary-search L. Maintain an integer $a \le \lambda$ and an explicit rooted cut S of weight $C = w(\delta^-(S)) \ge \lambda$. Initially take $a = 0$ and $S = V \setminus \{r\}$, so $C \le mW$. At each iteration test $L = \lceil (a + C)/2 \rceil$. If the flow-or-mix process finds a cut S' below L, replace S by S' and C by its weight; otherwise set $a = L$. Each correct test halves the search interval. When $a = C$, the stored cut is minimum.

Algorithm 62 Rooted directed minimum cut

```
function Test(G, r, L, t)
    Choose an r-arborescence T
    for j = 0, ..., t do
        Compute the exchange flow for (T, q = w/L)
        if the flow is infeasible then
            return the rooted cut extracted from its certificate
        Round the flow to F; split T + F into two arborescences
        Replace T by one of them uniformly at random
    return NoCutBelow(L)

function RootedMinCut(G, r, ε)
    if some vertex is unreachable from r then
        return the zero cut of the unreachable vertices
    S ← V \ {r}; C ← w(δ⁻(S)); a ← 0
    h ← ⌈log₂(mW + 1)⌉; t ← ⌈log₂((n + 1)h/ε)⌉
    while a < C do
        L ← ⌈(a + C)/2⌉; R ← Test(G, r, L, t)
        if R is a cut then
            S ← R; C ← w(δ⁻(S))
        else
            a ← L
    return S
```

Theorem 441 (Directed minimum cut in almost-linear time)**.** *For positive integral weights at most W, directed minimum cut can be found with probability at least $1 - \varepsilon$ using*

$$O\big(\log(mW)\log(n\log(mW)/\varepsilon)\big)$$

maximum-flow computations on graphs with $O((m+n)\log n)$ arcs, and $\widetilde{O}(m\log(mW)\log(n/\varepsilon))$ additional work. For polynomially bounded W and inverse-polynomial ε, the running time is $m^{1+o(1)}$.

Proof. A failed flow always gives a genuine cut below L by Lemma 440. When $L \le \lambda$, every flow is feasible. When $L > \lambda$, the probability that a test with $t+1$ flows returns NoCutBelow is at most $(n-2)2^{-t} \le \varepsilon/h$ by Equation (152). There are at most h binary-search iterations, so all lower-bound updates are correct with probability at least $1-\varepsilon$. Then termination at $a = C$ proves optimality. The call count follows from the chosen t and the network size from tree-path compression. To compute global min-cut, run the rooted algorithm in G and its reversal with half the error probability, and return the lighter cut. Multiplying the auxiliary flow capacities and demands by L makes them integral and polynomially bounded, so almost-linear maximum flow gives the claimed running time. □

12.4 Low-Stretch Spanning Trees

Let $G = (V, E, w)$ be a connected undirected graph with positive edge lengths, $n = |V| \ge 2$, and $m = |E|$; parallel edges are allowed, and self-loops are discarded. For a spanning tree T, its total stretch is

$$\mathrm{str}_G(T) = \sum_{uv \in E} \frac{d_{T,w}(u,v)}{w(uv)}.$$

We construct a spanning tree with total stretch $O(m \log n \log\log(2n))$ deterministically, in $O(m \log n)$ time for unweighted graphs and $O(m \log n \log\log(2n))$ time for polynomially bounded edge-length ratios (on the word RAM).

We recursively cut off connected sets, called *petals*, from the far side of a rooted cluster. Reattaching their trees creates detours. To pay for these detours, each new petal receives a root path whose *working* lengths are halved, following the highway idea of Abraham–Neiman. A short geometric induction bounds the resulting tree distances. Independently, an entropy potential chooses the cut boundary so that the stretch charges telescope. Working lengths guide the construction; the output tree always uses original edges and original lengths.

Recursive state. A call has a connected induced subgraph X, a root x, and a terminal t. Its working lengths ℓ equal w, except along one marked shortest x–t path, whose edges have working length $w/2$. The output must contain this marked path. Initially $t = x$, so it is empty. Set

$$h(v) = d_{X,\ell}(x, v), \qquad \Delta = \max_{v \in X} h(v).$$

Petals are removed one at a time from a surviving set Y, initially X. Every surviving vertex will retain an x–v shortest path, so its root distance $h(v)$ and a single root shortest-path tree can be reused throughout the call.

Petals and their geometry. Choose $s \in Y \setminus \{x\}$, a shortest x–s path P in Y, and $a = h(s)$. A route from x through $p \in P$ to v exceeds the shortest root distance by

$$e_p(v) = h(p) + d_{Y,\ell}(p, v) - h(v) \ge 0.$$

Moving backward from s to p costs $a - h(p)$. A detour of excess $e_p(v)$ uses twice that amount of the radius budget: halving the new highway will later compensate for it. Define

$$\tau(v) = \min_{p \in P}\{a - h(p) + 2e_p(v)\}, \qquad W_r = \{v \in Y : \tau(v) \le r\}, \quad 0 < r < a. \tag{153}$$

For $p \in P$, $\tau(p) = a - h(p)$: the witness p attains this value, and another point on the geodesic cannot improve it. Hence $W_r \cap P$ is exactly the suffix beginning at a vertex u and ending at s. Write yu for its first edge, where $y \notin W_r$. We recurse on W_r rooted at u and ultimately attach its tree using the *portal edge* yu. There are no fractional portal vertices, even if r falls inside that edge.

Lemma 442 (Petal geometry)**.** *Both W_r and $Y \setminus W_r$ are connected. Removing W_r preserves root distances h on the complement, and for every $vz \in E(Y)$,*

$$|\tau(v) - \tau(z)| \le 4\ell(vz). \tag{154}$$

Let $b = a - h(u) \le r$. Every $v \in W_r$ has a witness p on the u–s segment, with $k = a - h(p) \le b$, such that the u–p segment and some shortest p–v path lie in W_r and

$$h(p) + d_{Y,\ell}(p, v) \le h(v) + \frac{r-k}{2}. \tag{155}$$

In particular,

$$h(v) \ge a - r, \qquad d_{W_r,\ell}(u, v) \le \Delta - a + \frac{3}{2}r. \tag{156}$$

Proof. Both $h(\cdot)$ and $d_{Y,\ell}(p, \cdot)$ change by at most $\ell(vz)$ across an edge vz. Thus e_p is $2\ell(vz)$-Lipschitz, proving (154). Also $e_p \ge 0$ implies $\tau(v) \ge a - h(v)$.

Choose p attaining (153) for $v \in W_r$. Then $k + 2e_p(v) \le r$, which gives (155) and $k \le r$. Since $\tau(p) = k$, p lies on the suffix beginning at u. Along a shortest p–v path, every intermediate vertex q satisfies $e_p(q) \le e_p(v)$; thus its $\tau(q) \le k + 2e_p(v) \le r$. These paths connect the petal to its highway suffix.

Conversely, on an outward root geodesic $q \to q'$, $e_p(q') \le e_p(q)$ for every p, so $\tau(q') \le \tau(q)$. A root geodesic cannot leave a petal after entering it. Hence every survivor keeps a root geodesic and the complement remains connected. Finally the path from u through p to v lies in W_r and has length

$$d_{W_r,\ell}(u, v) \le b - k + d_{Y,\ell}(p, v) \le h(v) - a + b + \frac{r-k}{2} \le \Delta - a + \frac{3}{2}r.$$

The depth lower bound follows from $\tau(v) \ge a - h(v)$. □

Computing petals. To compute the labels $\tau(v)$, orient every edge both ways and give the arc $v \to z$ cost

$$c(v \to z) = 2\big(\ell(vz) + h(v) - h(z)\big) \ge 0. \tag{157}$$

Add a rootward arc of cost $\ell(vz)$ on each edge of P. Starting Dijkstra from s, those special arcs reach $p \in P$ at cost $a - h(p)$, and ordinary arcs then add $2e_p(v)$. Conversely, τ satisfies the relaxation inequalities for both kinds of arcs, so these are exactly its shortest-path labels. A root shortest-path tree represents P implicitly: an arc is special precisely when it moves to the parent of a vertex ancestral to s. DFS ancestor queries identify such arcs in constant time.

Choosing petal boundaries. An edge whose endpoints a petal separates contributes its tree distance divided by its *original* length. Accordingly write

$$B_Y(W) = \sum_{e \in \partial_Y W} \frac{1}{w(e)},$$

where $\partial_Y W$ is the set of edges in $G[Y]$ crossing the cut. Fix vertex masses $\mu(v) = \deg_G(v)$, including parallel edges, and let $\mu(S) = \sum_{v \in S} \mu(v)$. The potential is $\Phi(S) = \mu(S) \log |S|$ for $S \ne \varnothing$, and $\Phi(\varnothing) = 0$. Degrees pay for boundary-edge incidences; cardinality inside the logarithm ensures a $\log n$, not $\log m$, bound.

For $\varnothing \ne W \subsetneq Y$, put $q = |Y|$, $j = |W|$, $M = \mu(Y)$, and $A = \mu(W)$. Splitting Y into W and its complement decreases Φ by

$$D_Y(W) = A \log \frac{q}{j} + (M - A) \log \frac{q}{q-j}. \tag{158}$$

The gains are nonnegative. Every split replaces one set by two, so they telescope over all petals and recursive calls:

$$\sum_{\text{splits}} D_Y(W) \le \Phi(V) = 2m \log n. \tag{159}$$

Lemma 443 (Entropy stopping rule)**.** *For any interval I of radii for which W_r is nonempty and proper, some $r \in I$ satisfies*

$$B_Y(W_r) \le \frac{C(1+\log\log(2|Y|))}{|I|}\, D_Y(W_r) \tag{160}$$

for an absolute constant C.

Proof. We bound $\int B_Y(W_r)/D_Y(W_r)\,dr$ by $O(1+\log\log(2q))$, over nontrivial radii. Vertices enter in increasing order of τ. An edge can cross for at most $4\ell(e) \le 4w(e)$ units of radius by (154); its integral contribution without the denominator is at most four.

First suppose the petal mass $A \le M/2$. The log-sum inequality applied to (158) gives

$$D_Y(W_r) \ge H_M(A) := A\log\frac{M}{A} + (M-A)\log\frac{M}{M-A}.$$

Moreover, since $j \ge 1$ and $M - A \ge M/2$, the second term of (158) is at least $M/(2q)$. Consequently

$$D_Y(W_r) \ge F(A) := \max\left\{H_M(A), \frac{M}{2q}\right\}, \qquad 0 \le A \le M/2.$$

The function F is nondecreasing. Order vertices by entry time and write A_i for the mass of the first i. Charge a crossing edge to its *earlier* endpoint v_i. While that edge crosses, $D_Y(W_r) \ge F(A_i)$, and at most $\mu(v_i)$ edges are charged to v_i. Monotonicity therefore gives

$$\int_{A\le M/2} \frac{B_Y(W_r)}{D_Y(W_r)}\,dr \le 4\sum_{A_i \le M/2} \frac{\mu(v_i)}{F(A_i)} \le 4\int_0^{M/2} \frac{dz}{F(z)} \le 8 + 4\int_{M/q}^{M/2} \frac{dz}{z\log(M/z)} = O(1+\log\log(2q)). \tag{161}$$

For $A \ge M/2$, read the entry order backward, using the complementary mass and charging each edge to its *later* endpoint. The same bound applies. Averaging the integral over I proves (160). □

An inherited marked highway is already halved and cannot be discounted again. If it extends beyond depth $3\Delta/4$, first peel its distant suffix (the *protected petal*). The surviving marked prefix then ends before depth $3\Delta/4$, whereas subsequent petals are wholly beyond that depth. Their highways can therefore be halved without touching a previously halved edge.

Recursive decomposition. Compute h, Δ and a root shortest-path tree containing the marked path; a singleton returns immediately. Choose all radii by (160).

1. *Protected petal.* If $h(t) > 3\Delta/4$, cut a petal from $s = t$ with $r \in [h(t)/4, h(t)/3]$. Give it root u and terminal t, and give the residual cluster terminal y: the old marked path splits into an x–y prefix and a u–t suffix. Otherwise retain terminal t.

2. *Fresh petals.* Scan all s with $h(s) > 7\Delta/8$. If s survives, peel a petal with $r \in [\Delta/16, \Delta/8]$, root u, and terminal s, halving its u–s highway.

3. *Reassemble.* Recurse on each petal and the residual cluster (rooted at x), and add the portal edges.

All pieces are connected by Lemma 442. A portal endpoint y remains in a later piece, so adding one portal per removed petal joins the child trees into a spanning tree. Both radius intervals have width at least $\Delta/16$.

Tree-radius invariant.

Lemma 444 (Recursive invariants)**.** *Each child has one marked shortest root–terminal path, precisely its half-length edges, and working root-radius at most $7\Delta/8$. The returned tree contains the original marked path and satisfies*

$$d_{T,w}(x, v) \le 3h(v) \qquad (v \in X). \tag{162}$$

Proof. Radius and marked paths. A protected petal has $a = h(t) > 3\Delta/4$ and $r \le a/3$; by (156) its working radius is at most $\Delta - a + 3r/2 \le \Delta - a/2 < 5\Delta/8$. The remaining marked prefix ends at y, with $h(y) < a - r \le 3\Delta/4$. If no protected petal is needed, $h(t) \le 3\Delta/4$ already. A fresh petal has $a > 7\Delta/8$ and $r \le \Delta/8$. Its vertices have depth $> 3\Delta/4$, so it misses the surviving marked path, and its radius is at most $\Delta - a + 3r/2 \le 5\Delta/16$. Once all fresh petals have been removed, the residual cluster has radius at most $7\Delta/8$. Shortening a new highway only helps this bound. It remains shortest: every competing route had length at least the highway's old length L, and halving some edges cannot reduce its length below $L/2$, the new highway length. Inherited marked segments remain shortest by geodesic preservation.

Tree distances. Induct on $|X|$. Fix a petal with source depth a and portal yu; put $b = a - h(u) \le r$. For v in this petal choose its witness p from (155), and set $k = a - h(p) \le b$. We analyze petals in reverse removal order, so the tree path to y has already been bounded by $3h(y)$.

For a *fresh* petal, yu is not on the old marked path and $\ell(yu) = w(yu)$. As $h(u) = h(y) + w(yu)$ and $0 \le r - b < w(yu)$,

$$d_{T,w}(x,u) \le 3h(y) + w(yu) = 3h(u) - 2w(yu) \le 3h(u) - \tfrac{3}{2}(r-b). \tag{163}$$

The u–p highway is now half-length. Applying induction within the petal and then (155),

$$\begin{aligned} d_{T,w}(x,v) &\le 3h(u) - \tfrac{3}{2}(r-b) + 3\left(\frac{b-k}{2} + d_{Y,\ell}(p,v)\right) \\ &= 3\big(h(p) + d_{Y,\ell}(p,v)\big) - \tfrac{3}{2}(r-k) \le 3h(v). \end{aligned}$$

Thus the portal supplies credit $\frac{3}{2}(r-b)$ and the shortened highway supplies $\frac{3}{2}(b-k)$, exactly covering the witness excess.

For the *protected* petal, the old marked x–u prefix is kept in the output and has original length $2h(u)$; its u–p segment is already discounted and cannot be halved again. Hence

$$\begin{aligned} d_{T,w}(x,v) &\le 2h(u) + 3\big(b - k + d_{Y,\ell}(p,v)\big) \\ &= 3\big(h(p) + d_{Y,\ell}(p,v)\big) - h(u) \\ &\le 3h(v) + \tfrac{3}{2}(r-k) - h(u) \le 3h(v), \end{aligned}$$

where $h(u) \ge a - r \ge 2r$. The residual child satisfies the bound by induction. Its marked prefix and the protected child's marked suffix preserve the inherited path. □

Stretch. An edge is charged when its endpoints first separate. For a petal cut in a cluster of radius Δ, the final tree path between its endpoints has length at most 6Δ by (162). Each allowed radius interval has width $\Omega(\Delta)$, so the stretch charged at that cut is at most

$$6\Delta B_Y(W_r) \le O(1 + \log\log(2n)) D_Y(W_r).$$

Summing (159) proves the bound.

Work in one call. Compute h and its shortest-path tree once: all survivors keep their root geodesics. Grow each petal by the directed search (157), stopping at the first radius in the permitted interval that satisfies (160). Between vertex-entry events the petal and its boundary do not change. Process tied labels together, and update $|W_r|$, $\mu(W_r)$, and $B_Y(W_r)$ by scanning incident edges as each vertex enters. The entropy lemma guarantees a stopping radius.

Only settled vertices are removed. An unsettled heap entry created by a settled vertex is charged to that incident edge; each such edge is scanned at most once in this call, because the settled endpoint leaves Y. Lazy heap initialization and clearing thus cost $O(m_X + n_X)$ over all petals, apart from priority-queue operations. We find each portal by walking backward along its highway; those vertices belong to the removed petal and are charged once. A single initial scan gives the list of far sources.

For unweighted graphs, multiply distances by two: all arc costs are integers in $[0, 8]$, so constant-range buckets suffice. Empty-bucket advances also have linear aggregate cost: a search stopped at radius r removes a highway suffix of working length $b > r - 1$, containing $\Omega(r)$ vertices (working edges have length at most one). Thus its $O(r + 1)$ bucket advances are paid for by removed vertices. Total work in a call is $O(m_X + n_X)$.

For polynomially bounded ratios, round lengths upward to powers of two and normalize the minimum to one. After scaling by two, all working and arc lengths are polynomially bounded integers. A deterministic integer priority queue supports constant insert/decrease-key time and $O(\log\log(2n))$ deletion time, giving $O(m_X + n_X \log\log(2n))$ work per call. Since $w(e) \le \widetilde{w}(e) < 2w(e)$, the tree's stretch measured with original lengths is at most twice its stretch with rounded lengths.

Depth and running time. The working radius falls by a factor $7/8$ per level; it starts at $n^{O(1)}$ (at most n when unweighted), and every nonsingleton instance has radius at least $1/2$ after normalization. Thus the depth is $O(\log n)$. At each level the clusters are vertex-disjoint, so summing the per-call work proves the claimed times.

Weight spread. Only the running-time argument requires polynomially bounded ratios: geometry and entropy work for arbitrary positive lengths. Removing the ratio restriction would require multiscale bookkeeping, which we leave outside this construction.

Theorem 445 (Entropy-guided petal trees)**.** *Let G be a connected undirected graph with n vertices and m edges. There is a deterministic algorithm constructing a spanning tree T with $\mathrm{str}_G(T) = O(m \log n \log\log(2n))$. For unweighted graphs it runs in $O(m \log n)$ time; for graphs whose positive edge-length ratio is $n^{O(1)}$, it runs in $O(m \log n \log\log(2n))$ time on the word RAM.*

12.5 Low-Stretch Spanning Trees from Perturbed Dijkstra

A shortest-path tree preserves distances from its root, but need not preserve distances between nearby vertices: on an unweighted cycle, the endpoints of the omitted edge are distance $n - 1$ apart in the tree. We seek a distribution over spanning trees for which $\mathbb{E}[d_T(u, v)] \le \gamma w(uv)$ for every edge uv. This implies $\mathbb{E}[d_T(x, y)] \le \gamma d_G(x, y)$ for all vertices x, y, by applying the triangle inequality along a shortest x–y path.

The obstacle is that two nearby vertices can have shortest paths from the root that diverge far away. To prevent this, consider a cluster C of small diameter containing both u and v. If the root-to-u and root-to-v paths in a shortest-path tree have a common ancestor $x \in C$, their tree distance is small: because every subpath of a shortest path is shortest,

$$d_{\widetilde{T}}(u, v) \le d_{\widetilde{G}}(x, u) + d_{\widetilde{G}}(x, v) \le 2\,\mathrm{diam}(\widetilde{G}[C]). \tag{164}$$

Here $\widetilde{G}$ denotes a graph with perturbed edge lengths and $\widetilde{T}$ its shortest-path tree. In particular, if the root is outside C, it is enough for the two root paths to use a common edge of $\delta(C)$: the endpoint of that edge inside C is a common ancestor.

We construct nested random clusters, with diameter $O(D)$ at scale D, and perturb the edge lengths so that Dijkstra paths to nearby vertices usually cross a cluster boundary through the same edge. Every boundary edge receives a small positive penalty and an independent exponential discount. The discounts make the cheapest way of entering a cluster unlikely to be nearly tied with a route using completely different boundary edges. Indeed, if the two root paths to u and v used disjoint boundary edges, appending uv to one path would give such a near-tie, of size at most $2\widetilde{w}(uv)$.

There is a competing requirement: boundaries at scale D must be penalized enough to distinguish different entrances, but not enough to destroy the small diameter of a cluster. Random partitions make an edge of length w a scale-D boundary with probability $O(w/D)$. This lets us afford a penalty proportional to D. To control the resulting distances inside clusters, we will reroute paths through individual carved balls rather than try to concentrate the penalties on a fixed path. We will obtain $\Pr[d_T(u, v) > 4D] = O(bw(uv)/(\eta D))$ at each hierarchy scale, up to negligible failure probabilities. Summing these tails over geometrically spaced scales gives the expected stretch.

Nested partitions and boundary penalties. Let $G = (V, E, w)$ be connected, with positive integral weights at most $W = n^{O(1)}$. Delete loops and retain only the cheapest parallel edge. Put

$$b = K \log(2nW), \qquad \alpha = 100b, \qquad D_i = \alpha^i, \qquad L = \lceil \log_\alpha(2nW) \rceil, \qquad \eta = \frac{1}{100Lb}, \tag{165}$$

where K is a sufficiently large absolute constant. Thus $D_L \geq 2nW$, so the whole graph has diameter at most D_L. The parameter b gives high-probability diameter bounds, η is the penalty budget per scale, and α separates scales enough to absorb the errors accumulated at smaller scales.

At scale D, *exponential ball carving* partitions a graph as follows. Choose any vertex of the remaining graph, sample $R \sim \mathrm{Exp}(b/D)$, remove the ball of radius R in the remaining induced graph, and repeat. If $R \leq D/2$, any two vertices of the ball can be connected through its center by a path of length at most D entirely inside the ball. A radius exceeds $D/2$ with probability $e^{-b/2}$.

An edge $e = xy$ can be cut only when the growing ball reaches one of x, y but stops before reaching the other. Their distances from the ball's center differ by at most $w(e)$, so the radius spends at most $w(e)$ units in this interval. By the memoryless property of the exponential distribution, the probability of stopping during the interval is at most

$$1 - e^{-bw(e)/D} \leq \frac{bw(e)}{D}. \tag{166}$$

The edge can be exposed in this way for at most one carved ball: once a ball reaches either endpoint, it removes that endpoint or both. In particular, there is no factor for the number of balls. Sequential carving is useful here because, at the instant a ball is removed, every newly cut edge is visible on its growing boundary.

Start with $\mathcal{C}_L = \{V\}$. For $i = L-1, \ldots, 1$, carve each cluster of $\mathcal{C}_{i+1}$ at scale D_i, independently of all later refinements, to obtain $\mathcal{C}_i$. Let $\mathcal{C}_0$ consist of singletons. The partitions are nested, and every cluster of $\mathcal{C}_i$ has strong diameter at most D_i, simultaneously for all levels, except with probability at most $nLe^{-b/2}$. If the endpoints of e are separated in $\mathcal{C}_i$, they were first separated at some level $j \geq i$. Summing (166) over these geometrically increasing scales,

$$\Pr[u, v \text{ separated in } \mathcal{C}_i] \leq \min\left\{1, \frac{2bw(uv)}{D_i}\right\}. \tag{167}$$

For each edge e, let $\ell(e)$ be the *largest* index at which its endpoints belong to different clusters; thus $\ell(e) = 0$ if both endpoints remain in one level-1 cluster. Define $h_e = \eta D_{\ell(e)}$ and, independently conditional on the entire hierarchy, draw $X_e \sim \mathrm{Exp}(b/h_e)$. Set

$$\widetilde{w}(e) = w(e) + (h_e - X_e)_+. \tag{168}$$

Run Dijkstra from an arbitrary fixed root r using the lengths $\widetilde{w}$, and output its tree T with the *original* lengths w. The penalty h_e is attached to the coarsest boundary separating the endpoints of e. Consequently, inside a level-i cluster no edge has a penalty from level i or higher; this will allow us to repair paths within finer balls without paying any coarser penalty.

Although the penalty of an individual edge can be much larger than its original length, this happens rarely. An edge first separated at level $j \geq 1$ receives $h_e = \eta D_j$, and the probability of that event is at most $bw(e)/D_j$. Hence

$$\mathbb{E}[\widetilde{w}(e) - w(e)] \leq \mathbb{E}[h_e] \leq \eta + \sum_{j=1}^{L-1} \eta D_j \frac{bw(e)}{D_j} \leq \eta(1 + Lb)w(e) \leq \frac{w(e)}{8}. \tag{169}$$

Here we used $w(e) \geq 1$. We will also use the conditional version: if the endpoints of e belong to one cluster of $\mathcal{C}_i$, then, after fixing $\mathcal{C}_i, \ldots, \mathcal{C}_L$, only the finer levels can separate them, and the same bound applies to their remaining expected perturbation. In particular, for such an edge,

$$\mathbb{E}[\widetilde{w}(e) \mid \mathcal{C}_i, \ldots, \mathcal{C}_L] \leq \frac{9}{8} w(e).$$

Exponential discounts force a common entrance. Fix a level-i cluster C and vertices $u, v \in C$ joined by an edge. Assume $r \notin C$, since otherwise r is already a common ancestor in C. Every root-to-u path crosses $F = \delta(C)$. For $f \in F$, its endpoints are separated at level i, so $h_f \geq \eta D_i$ and the rate of its discount satisfies

$$\frac{b}{h_f} \leq \lambda := \frac{b}{\eta D_i}. \tag{170}$$

Thus a scale-D_i boundary edge has an exponential discount of mean at least $1/\lambda = \eta D_i/b$. What matters is not the gap between the two cheapest paths overall—they may share almost all their boundary edges—but the gap to a path using *none* of the winning path's boundary edges.

Lemma 446 (Exponential gap)**.** *Consider a finite family of edge-simple paths, each containing an edge of F. Give each $f \in F$ an independent discount $X_f \sim \mathrm{Exp}(\lambda_f)$, where $\lambda_f \leq \lambda$; all other edge costs are fixed. Let P be a minimum-cost path and let Q be the cheapest path using none of the edges of $P \cap F$ (take $\mathrm{cost}(Q) = \infty$ if no such path exists). Then for every $t \geq 0$,*

$$\Pr[\mathrm{cost}(Q) - \mathrm{cost}(P) \leq t] \leq 1 - e^{-\lambda t} \leq \lambda t. \tag{171}$$

Proof. Order F and let f^* be the first edge of F on the winning path P. We bound the event $f^* = f$ for each $f \in F$, then sum over the disjoint possibilities.

Fix f and expose every discount except X_f. Among paths using f, the cheapest path P_f has cost $a_f - X_f$; its identity and a_f are independent of X_f, since an edge-simple path uses f exactly once. Let c_f be the cheapest cost of a path avoiding f. If no such path exists, the desired gap is infinite. Otherwise put $\tau_f = a_f - c_f$.

For f to be the first boundary edge of the global winner, f must be the first boundary edge of P_f (a condition independent of X_f), and P_f must beat every path avoiding f, which is equivalent to $X_f > \tau_f$, up to ties. On this event $P = P_f$, and Q avoids f. Therefore

$$\mathrm{cost}(Q) - \mathrm{cost}(P) \geq c_f - (a_f - X_f) = X_f - \tau_f.$$

If $\tau_f \geq 0$, conditional on $X_f > \tau_f$, the excess $X_f - \tau_f$ is exponential with rate λ_f. If $\tau_f < 0$, the event $X_f > \tau_f$ is automatic and the excess is at least X_f. In either case the conditional probability that the gap is at most t is at most $1 - e^{-\lambda_f t} \leq 1 - e^{-\lambda t}$. Summing over the mutually exclusive events $f^* = f$ introduces no factor of $|F|$. □

To apply the lemma, let P_u, P_v be the root paths in the Dijkstra tree. If they share an edge of F, its endpoint in C is their common ancestor. Otherwise P_v followed by vu gives a root-to-u walk avoiding every edge of $P_u \cap F$. Removing cycles makes it a path with no greater cost, and shortest-path optimality gives

$$\mathrm{cost}(P_v) + \widetilde{w}(uv) \leq \mathrm{cost}(P_u) + 2\widetilde{w}(uv). \tag{172}$$

Thus different boundary entrances imply that the gap of Lemma 446 is at most $2\widetilde{w}(uv)$.

The truncation in (168) only ensures that the edge lengths never decrease. For the gap calculation, temporarily use the linear costs $\widehat{w}(e) = w(e) + h_e - X_e$ instead. The two cost systems agree unless $X_e > h_e$ for some edge, an event of probability at most me^{-b}. The gap lemma applies to the formal costs of edge-simple paths even outside this event; on its complement, these are exactly the Dijkstra paths and all edge costs are positive.

Condition on the hierarchy and on all discounts outside F. Because uv is internal to C, its perturbed length is fixed by this conditioning. The boundary discounts are still independent, with rates at most λ by (170). The gap lemma and (172) therefore bound the conditional probability of distinct entrances by $2\lambda\widetilde{w}(uv)$, apart from the truncation event. Finally average over the finer hierarchy and the discounts outside F. The conditional expected-noise bound for $uv \in C$ gives

$$\Pr[\text{no common ancestor in } C \mid \mathcal{C}_i, \ldots, \mathcal{C}_L] \leq O\left(\frac{bw(uv)}{\eta D_i}\right) + me^{-b}. \tag{173}$$

The bound depends on the length of uv, not on the number of possible entrances into C.

Preserving cluster diameters by repairing paths. The coalescence argument is useful only if the clusters retain small diameter in $\widetilde{G}$. Consider instead the larger edge lengths $w + h$, obtained by charging every boundary its full penalty and ignoring the discounts. It suffices to prove that every level-i cluster C has diameter at most $2D_i$ under these lengths.

It would be tempting to take a shortest path in $G[C]$ and concentrate the sum of its boundary penalties. This fails because a single carved ball may cut many edges of the path simultaneously. But such a ball also supplies a shortcut: if the path repeatedly enters and leaves a ball of diameter at most D, replace the part between its first and last visits by a path entirely inside the ball. The shortcut costs at most D and leaves at most two of the ball's boundary crossings. Thus a ball with many crossings costs $O(D)$, not its number of crossings times ηD.

We make this precise for one refinement level. Suppose all penalties from coarser levels have been assigned, and fix a path P of length a under the current weights. We now carve the current parent clusters at scale D; every edge newly separated by this refinement receives an additional penalty ηD. For each removed ball B, let k_B be the number of edges of the *original path* P that are newly separated when B is removed. Each newly penalized edge of P is counted for exactly one ball.

Call B expensive when $\eta D k_B > 3D$. Traverse P from start to finish. At the first encounter with an expensive ball B, jump to the last occurrence of B on P, using a path inside B of original length at most D; then continue along P. This shortcut is valid even if the intervening subwalk left and reentered the parent cluster, since both of its endpoints belong to B. Each expensive ball is used for at most one shortcut, and the traversal may skip other expensive balls altogether. Every shortcut lies inside one new cluster, so its edges receive neither an old penalty nor a new penalty at this level. An expensive ball that is used for a shortcut has at most two boundary edges remaining on the resulting path. Hence the resulting path P' has length after the refinement at most

$$a + \sum_B \min\{\eta D k_B, 3D\}. \tag{174}$$

Indeed, a non-expensive ball contributes at most $\eta D k_B$ in new penalties. A shortcut contributes at most D in old lengths and $2\eta D$, which is at most $3D$; an expensive ball skipped entirely by another shortcut contributes nothing. The inequality holds whenever the carved balls have strong diameter at most D.

The capped sum in (174) concentrates even though the individual k_B may be large. Since weights are integral, we may use $\lfloor R \rfloor$ as the carving radius. The exponential radius can be revealed one integer unit at a time: conditional on reaching any radius, the probability of stopping there is

$$p = 1 - e^{-b/D} \le b/D.$$

At each such step t, let N_t be the number of edges of the fixed path P on the current growing ball's boundary, within the remaining parent graph. If the ball stops at this step, then $k_B = N_t$. Thus, writing

$$Y_t = \mathbf{1}_{\{\text{stop at } t\}} \min\{\eta D N_t, 3D\},$$

we have $0 \le Y_t \le 3D$ and $\mathbb{E}[Y_t \mid \text{past}] \le \eta b N_t$. Moreover,

$$\sum_t N_t \le w(P) \le a. \tag{175}$$

For the first inequality, each occurrence of an edge xy of P can be exposed only while the radius lies between the two integral distances to x and y, an interval of at most $w(xy)$ integer steps. Once the ball reaches the nearer endpoint, it either stops and removes that endpoint or continues until both endpoints have been reached; the edge cannot be exposed by another ball. Consequently, over all balls and all parents,

$$\sum_t \mathbb{E}[Y_t \mid \text{past}] \le \eta b a. \tag{176}$$

We use the following elementary concentration bound. If adapted random variables $0 \le Y_t \le H$ satisfy $\sum_t \mathbb{E}[Y_t \mid \text{past}] \le \mu$ on every history, then

$$\Pr\left[\sum_t Y_t > (e-1)\mu + Hz\right] \le e^{-z} \qquad (z \ge 0). \tag{177}$$

To prove it, apply $e^{x/H} \le 1+(e-1)x/H$ for $0 \le x \le H$ to each conditional exponential moment. The product of the resulting bounds, with the conditional means subtracted from the exponent, is a supermartingale of expectation at most one. Markov's inequality gives (177). With $H = 3D$, $\mu = \eta ba$, and $z = b$, the capped sum is at most $(e-1)\eta ba + 3bD$ except with probability e^{-b}. Thus, provided the balls have diameter at most D, the repaired path satisfies

$$a' \le (1+\rho)a + 3bD, \qquad \rho = (e-1)\eta b. \tag{178}$$

This estimate also holds when P depends on all coarser refinements, since the next level uses fresh randomness.

Now fix u, v in a level-i cluster C. Before revealing levels below i, choose a path between them inside C of original length at most D_i. Refine levels $i-1, i-2, \dots, 1$ in that order, repairing the current path after every refinement. All shortcuts stay inside C. If a_j is the path length after the penalties from levels at least j have been assigned, then

$$a_i \le D_i, \qquad a_j \le (1+\rho)a_{j+1} + 3bD_j \quad (1 \le j < i).$$

At level 0 every remaining edge receives at most η in additional length; since $w(e) \ge 1$, this increases any path's length by a factor of at most $1+\eta$. Iterating the recurrence,

$$d_{w+h}(u,v) \le (1+\eta)(1+\rho)^L \left(D_i + 3b\sum_{j=1}^{i-1} D_j\right) \tag{179}$$

$$\le (1+\eta)e^{\rho L}\left(1 + \frac{3b}{\alpha - 1}\right) D_i \le 2D_i, \tag{180}$$

where $\rho L = (e-1)/100$ and $\alpha = 100b$. For any fixed pair and starting cluster, there are at most L repair steps, each failing with probability e^{-b}. There are at most $n^2 L$ pairs and choices of starting level, and at most nL carved balls. A union bound, using $b = K\log(2nW)$ with sufficiently large K, shows that (180) holds simultaneously for all clusters and vertex pairs with failure probability

$$\delta = (nW)^{-\Omega(1)}.$$

Since $\widetilde{w} \le w + h$, this also bounds the diameters of $\widetilde{G}[C]$.

Expected stretch and running time. Fix an edge uv of original length $w = w(uv)$. At scale D_i, its Dijkstra-tree distance is at most $4D_i$ whenever u, v lie in the same level-i cluster, their root paths have a common ancestor in that cluster, and its perturbed diameter is at most $2D_i$. By (167), (173), and (180), for every $1 \le i < L$ and some absolute constant c,

$$\Pr[d_T(u,v) > 4D_i] \le \min\left\{1, \frac{cbw}{\eta D_i}\right\} + \delta, \tag{181}$$

Here $d_T \le d_{\widetilde{T}}$ because perturbations only increase edge lengths. For $i = 0$ we use the trivial probability bound 1, which is also at most $bw/(\eta D_0)$ because $w \ge 1$.

The distance $d_T(u,v)$ is at most $(n-1)W < D_L$, since T is a spanning tree with original edge lengths. Summing the tail probabilities over the geometric intervals $[4D_i, 4D_{i+1})$ therefore gives

$$\mathbb{E}[d_T(u,v)] \le 4D_0 + 4\sum_{i=0}^{L-1}(D_{i+1}-D_i)\Pr[d_T(u,v) > 4D_i] \tag{182}$$

$$\le O\left(\frac{\alpha L b}{\eta}\, w\right) + O(D_L\delta) \tag{183}$$

$$= O\left(\frac{\log^5(2nW)}{(\log\log(2nW))^2}\, w\right). \tag{184}$$

The error $D_L\delta$ is negligible by taking K sufficiently large. Each carving level can be implemented by truncated Dijkstra searches in the remaining induced graphs. Every vertex is removed once per level, so the total work is $O((m+n)\log n)$ per level, followed by one final Dijkstra run. As $L = O(\log(nW)/\log\log(nW))$, the total time is $O(m\log^2 n/\log\log n)$ for polynomially bounded W.

Theorem 447 (Low-stretch spanning trees via perturbed Dijkstra)**.** *For a connected graph with positive integral weights bounded by $n^{O(1)}$, there is a nonnegative random edge-weight perturbation, conditionally independent given a nested random hierarchy, whose expected increase on each edge e is at most $w(e)/8$. A shortest-path tree of the perturbed graph from any fixed root, evaluated using the original weights, satisfies*

$$\mathbb{E}[d_T(u,v)] = O\left(\frac{\log^5 n}{(\log\log n)^2}\, w(uv)\right) \qquad \text{for every } uv \in E.$$

The randomized construction takes $O(m\log^2 n/\log\log n)$ time in the comparison/arithmetic model.

12.6 Dynamic Distances

Let G be an undirected multigraph with positive edge lengths. We want to maintain approximate distances under edge insertions, deletions, and length changes, and to return the corresponding walks. A walk can be much longer than the time available for a query, so we return a description that can later be expanded. For a walk expression P, let $\ell(P)$ be its *occurrence length*: every traversal is counted, including an edge traversed once in each direction. A distance answer consists of P and a bound D satisfying

$$\ell(P) \le D \le \Gamma\,\mathrm{dist}_G(u,v).$$

Equal endpoints use the empty walk, and disconnected endpoints return infinity.

Theorem 448 (Dynamic distances)**.** *For graphs of size at most N with $\log(\ell_{\max}/\ell_{\min}) = \log^{O(1)} N$, there is a deterministic dynamic distance oracle with approximation $N^{o(1)}$, total maintenance work $(N+q_{\mathrm{upd}})N^{o(1)}$ over q_{upd} changes, and $N^{1+o(1)}$ space. An implicit query costs $\log^{O(1)} N$; reporting s edge occurrences costs $(s+1)\log^{O(1)} N$. The time bounds count arithmetic operations.*

The construction repeatedly replaces the instance by a smaller physical graph H. Stable balls give exact local information; first-exit decomposition reduces long paths to constant-complexity terminal words; a small family of routing forests preserves each word in one common forest; and auxiliary search graphs maintain the required routes while capacity closures keep repairs and the child sparse. Only H enters the distance reduction; the search graphs serve only route construction and repair.

Records and epochs. Graph size counts vertices and edges; parallel edges have distinct identifiers, and self-loops are allowed. Records are immutable and reference-counted. Changing a label or expansion makes the old record inactive, recursively deletes its child users, and reclaims it only after those users release their references; the replacement is a new record. All induced changes finish before the next query.

Let N bound both the live size and the number of top-level changes in one operation block. A declared range $[\ell_{\min}, \ell_{\max}]$ contains the top-level length labels in that block. Longer sequences rebuild between blocks of N changes. Each recursive instance has its own size cap $M \le N$; its input edges may represent walks in its parent. It retains its preprocessing for an *epoch*, with integer cutoff C controlling the local search size. Analyze an epoch with B input changes before choosing its duration, C, and the recursion depth. Thus N remains fixed through the recursion, while M decreases. All changes may be chosen adaptively. We use $\widetilde{O}$ to suppress fixed powers of $\log(2N)$.

12.6.1 Local Information That Survives Changes

Fix the epoch's initial graph G^0. A ball should be small enough to scan, while a slightly larger prefix should be large enough to meet a small terminal set. Vertex count alone cannot provide both properties at a high-degree vertex. Instead give each vertex weight $\omega(v) = 1 + \deg_{G^0}(v)$.

From each initial vertex u, run Dijkstra until the total settled weight first reaches C. Retain the crossing vertex without scanning its adjacency. Let P_u be the retained prefix and S_u its scanned vertices. Then $|P_u| \le C$ and $\sum_{v \in S_u} \omega(v) < C$. If the component has weight below C, scan it completely. Store $d_u(v) = \mathrm{dist}_{G^0}(u, v)$ and a predecessor path $\pi_u(v)$ for every $v \in P_u$. In particular, if $\omega(u) \ge C$, the prefix is just $\{u\}$.

Greedily hit all crossing prefixes to obtain the initial terminal set $\mathcal{A}$. If $\mathcal{U}$ is the family still uncovered and c_v counts its prefixes containing v, then

$$\sum_v \omega(v) c_v = \sum_{P \in \mathcal{U}} \sum_{v \in P} \omega(v) \ge C|\mathcal{U}|.$$

Since $\sum_v \omega(v) \le 2M$,

$$\max_v c_v \ge \frac{\sum_v \omega(v) c_v}{\sum_v \omega(v)} \ge \frac{C|\mathcal{U}|}{2M}.$$

The greedy rule therefore leaves at most $M(1 - C/(2M))^s \le Me^{-sC/(2M)}$ prefixes after s choices, so $O(M \log N/C)$ terminals suffice. Prefix incidence lists and a heap implement the rule: when a prefix is hit, decrement the counts of its vertices. Each of the $O(MC)$ incidences is processed once, so all searches and terminal selection cost $O(MC \log N)$.

Before changing an edge, promote both endpoints to terminals. For each u, keep the closest terminal in P_u in the initial distances: its *pivot* p_u and radius $r_u = d_u(p_u)$. Use $(p_u, r_u) = (\perp, \infty)$ if there is none. An inverse index stores $(u, z, d_u(z))$ for every $z \in P_u$. When z is promoted, inspect these entries and improve each affected pivot, retaining the old choice in a tie. Initial terminals use the same rule. All promotions together inspect $O(MC)$ entries. A new vertex is terminal immediately, with itself as pivot, radius zero, and empty pivot path. Internal routing operations never create terminals; they only add forest marks below.

Lemma 449 (Stable balls)**.** *The radius is the current distance to the terminals, and*

$$B(u) := \{v \in P_u : d_u(v) < r_u\} = \{v : \mathrm{dist}_G(u, v) < r_u\}.$$

A finite pivot path survives and is shortest. No edge incident to $B(u)$ has changed during the epoch, and all stored paths to its vertices remain shortest. The balls only shrink and have size at most C, counting vertices and edges. An absent pivot means that the entire component was scanned and is unchanged.

Proof. There are two points. First, the chosen pivot is no farther than the crossing vertex of P_u. Hence every initial vertex at distance below r_u was actually scanned; if there is no crossing vertex, the whole component was scanned. Thus $B(u)$ is exactly the initial strict ball of radius r_u, and by definition it contains no terminal.

Second, an update cannot enter this strict ball. If an edge incident to $B(u)$ had changed, one of its endpoints $z \in B(u)$ would have been promoted before the change. The pivot rule would then give $r_u \le d_u(z)$, contradicting $d_u(z) < r_u$. Therefore the whole ball and all stored paths inside it are unchanged. In particular, for $0 < r_u < \infty$ the stored pivot path survives and has length r_u. Conversely, any current path shorter than

r_u cannot leave the unchanged ball, while a path that leaves it first reaches a vertex with initial distance at least r_u. Hence the current distance to the terminals is exactly r_u, and all stored paths to vertices of $B(u)$ remain shortest. Radius zero is immediate; radius infinity means the whole component was scanned and no vertex in it was ever promoted, so the component is unchanged.

Promotions can only decrease r_u. Finally, a crossing ball lies inside S_u, whose incident-edge volume is below C, and a completed component has the same bound. This proves monotonicity and the size claim. □

12.6.2 What the Terminal Graph Must Preserve

Consider a path that starts at u and first leaves $B(u)$ on edge wv. Write this *first-exit piece* as $Q = Q' + (wv)$, where Q' stays in the ball. At a terminal start take just the next edge. Including the exit edge is essential: it ensures $r_u \le \ell(Q)$, so this piece can pay for the path from u to its pivot.

Replace the local prefix by its stored shortest path and attach pivots:

$$W_Q = -\pi_u(p_u) + \pi_u(w) + (wv) + \pi_v(p_v). \tag{185}$$

This walk runs from p_u to p_v and consists of three cached paths and one epoch-initial edge. If the exit edge was inserted during the epoch, its endpoints are terminals and we use that edge itself instead. An exit from a pivot-free component is impossible, and the stable-ball lemma ensures that every displayed cached path survives.

Split a terminal-to-terminal path successively at first exits. There is no unfinished final prefix: a terminal cannot lie strictly inside the ball from which the last piece starts. The original path edges contribute total length D. Every intermediate radius appears twice in the attached pivot paths—once at the end of one piece and once at the start of the next—and the two endpoint radii are zero. Since the starting radius of a piece is at most that piece's length, the total attachment cost is at most $2D$. Thus

$$\sum_Q \ell(W_Q) \le D + \sum_Q (r_u + r_v) = D + 2\sum_Q r_{\mathrm{start}(Q)} \le 3D. \tag{186}$$

The same accounting applies to a cyclic concatenation.

Thus it suffices to preserve the terminal words. Their replacements may contain opposite traversals that cancel. This cancellation is what lets us compress the remaining forest paths, so we state the requirement for signed chains. Fix a reference orientation and write $[P]$ for the signed edge vector of a walk. For a chain z, let ∂z be its divergence, positive for net outflow, and put $\|z\|_\ell = \sum_e \ell_e |z_e|$. Cancellation gives $\|[P]\|_\ell \le \ell(P)$.

Let H be a multigraph on a subset of the input vertices containing all terminals. Every edge expands to a current input walk between its endpoints, with exactly the same occurrence length; write lift for the linear extension of these expansions. Suppose every word W has a chain y_W in H with the same boundary and $\|y_W\|_\ell \le \alpha \ell(W)$. A chain with unit boundary contains a path of no greater length, giving the following reduction.

Proposition 450 (Distance reduction)**.** *If the child answers distances in H within a factor β, the parent answers within a factor $15\alpha\beta$.*

Proof. Split a shortest path between terminals a, b into first-exit pieces. The chains y_{W_Q} concatenate at their pivots. Their sum has unit a–b boundary and norm at most $3\alpha \operatorname{dist}_G(a, b)$ by (186). Orient its coefficients by their signs and decompose it into paths and cycles. The a–b path coefficients sum to one, so some path has length at most that norm. Every H-path expands to an input walk of the same length. Hence

$$\operatorname{dist}_G(a, b) \le \operatorname{dist}_H(a, b) \le 3\alpha \operatorname{dist}_G(a, b).$$

For arbitrary x, y, test only whether $y \in B(x)$. If so, return the stored exact path. Otherwise $r_x \le d := \operatorname{dist}_G(x, y)$ whenever the endpoints are connected. If p_x is absent, the stable-ball lemma says that $B(x)$ is the entire unchanged component of x, so the failed test certifies disconnection. Otherwise p_x is a terminal, and the nearest terminal to y satisfies

$$r_y \le \operatorname{dist}_G(y, p_x) \le d + r_x \le 2d.$$

Thus p_y exists and $\mathrm{dist}_G(p_x, p_y) \le r_x + d + r_y \le 4d$. Query the child between p_x, p_y and attach the two pivot paths. The child walk costs at most $12\alpha\beta d$, while the attachments cost at most $3d$, for a total of at most $15\alpha\beta d$. Disconnected pivots cannot be joined in H because every H-path has a physical expansion. Equal endpoints use the empty walk. □

12.6.3 Weighted Routing Forests

The preceding reduction leaves one static epoch problem: build a small physical graph that carries a short chain for every terminal word. A tree is ideal for the later compression step, but one tree can stretch some words badly. We first prove a weighted routing-tree method; the next subsection boosts it to a small family so that every word has one common good tree.

A routing tree may contain several copies of an input vertex. Each tree edge is a copy of an input edge, with the same length and a specified orientation under projection. Tree paths therefore project to input walks of the same occurrence length. Keeping copies distinct lets us join shortest-path trees at successive scales without creating cycles.

Lemma 451 (Weighted routing forests)**.** *For nonnegative edge importances a_e, a graph of size at most M and length ratio Δ has a routing forest T with one tree per input component and a designated representative $\iota(v)$ of each input vertex. Each input vertex and edge has at most $\chi = 2 + \lceil \log_2(2M\Delta) \rceil$ tree copies, the forest has size $O(M\chi)$, and*

$$\sum_e a_e\big(\ell_e + \mathrm{dist}_T(\iota(e^-), \iota(e^+))\big) \le \gamma \sum_e a_e \ell_e, \qquad \gamma = O(\log(2M)\chi). \tag{187}$$

Construction takes $O(M\chi \log(2M))$ arithmetic operations.

Proof. Omit self-loops and work componentwise. The only estimate needed at one scale is this: if a cluster U has strong diameter at most R and weighted length $A_U := \sum_{e \in E(U)} a_e \ell_e$, then it can be cut into clusters of diameter at most $R/2$ while separating edges of total importance $O((\log M)A_U/R)$. Recursing over scales then gives the stretch bound.

Start with a dyadic $R_0 \in [M\ell_{\max}, 2M\ell_{\max})$. In a recursive call on a connected induced cluster U of strong diameter at most R, build a shortest-path tree S_U from the specified root. If U is a singleton or $A_U = 0$, return S_U.

Otherwise repeatedly carve a ball from the remaining induced graph $G[Y]$. For a center $x \in Y$, let $B_r = \{v : \mathrm{dist}_{G[Y]}(x, v) < r\}$ and define

$$b(r) = a(\partial_Y B_r), \qquad V(r) = A_U/M + \int_0^r b(t)\,dt, \qquad k_* = \lceil \log_2(2M) \rceil.$$

The additive seed A_U/M only keeps the volume away from zero. During the sweep, an edge contributes to the integral for at most the difference of its endpoint distances, hence for at most ℓ_e. Therefore $A_U/M \le V(r) \le A_U/M + A_U$. Since $V'(r) = b(r)$ away from distance keys, some $r \in [R/8, R/4]$ must satisfy

$$b(r) \le \frac{8k_*}{R} V(r).$$

Otherwise integrating $V'/V > 8k_*/R$ across the interval would give $V(R/4)/V(R/8) > e^{k_*} > M + 1$, contradicting the preceding bounds. Remove this ball and continue. Shortest paths from its center stay inside it, so every carved cluster has strong diameter below $R/2$.

Now sum these boundary charges over all balls carved in the call. The seed A_U/M is paid at most M times, hence contributes at most A_U. An edge contributes to the integral term only until the first carved ball removes one of its endpoints, and then by at most $a_e\ell_e$. Thus $\sum V(r) \le 2A_U$, and

$$a(\text{edges separated in } U) \le \frac{16k_*}{R} A_U. \tag{188}$$

Use distinct vertex copies in different recursive calls. Attach each child tree to S_U only at the child's root, so the union remains a tree. The representative of a vertex is its copy in the deepest call containing it. From the root of a scale-R call to any designated descendant, the successive tree pieces have lengths at most $R, R/2, R/4, \ldots$, hence total at most $2R$. Therefore an edge first separated in this call has designated endpoints at tree distance at most $4R$. Multiplying (188) by $4R$ charges the whole call by at most $64k_*A_U$.

There are at most χ scales before the diameter drops below $\ell_{\min}$, at which point a cluster is a singleton. Clusters at one scale are disjoint, so their A_U sum to at most the original weighted length. Summing the call charges over all scales and adding the original edge lengths proves (187). Each input object appears in at most one shortest-path tree per scale, proving the copy and size bounds as well.

For the implementation, truncated Dijkstra sweeps distance keys. Between keys, b is constant and V is linear, so it suffices to test the displayed inequality at tied batches and interval endpoints. Every scanned vertex is removed with its chosen ball. Thus the searches in one call are linear up to a logarithm, and summing over the χ scales gives the stated work. All tests are rational inequalities; no numerical logarithm is evaluated. □

12.6.4 From Terminal Words to the Physical Child Graph

We now build the physical child graph. A terminal word has at most four fixed *atoms*—cached paths $\pi_u(v)$ and one initial edge. We choose a small family in which every atom is good in more than three quarters of the forests, so every word has one common good forest; using one forest for the whole word is what lets its internal divergences cancel. We then compress the marked Steiner hull of each forest into physical edges of H.

Let $\mathcal{P}$ contain every nonempty cached path $\pi_u(v)$ with $v \in P_u$, together with every one-edge walk consisting of an initial edge. There are $O(MC)$ atoms, hence $O(M^2)$ in an internal instance where $C < M$. They are kept implicitly through the cached predecessor trees; we never expand all root paths into edge occurrences. For a routing forest T, put

$$\mathrm{cost}_T(P) = \sum_{e \text{ occurring in } P} \big(\ell_e + \mathrm{dist}_T(e^-, e^+)\big).$$

Lemma 452 (Forest cover)**.** *In $MC\log^{O(1)} N$ work, construct $\lambda = O(\log M)$ routing forests such that every current terminal word W built from epoch-initial records has a forest T_j satisfying*

$$\sum_{e \text{ occurring in } W} \big(\ell_e + \mathrm{dist}_{T_j}(e^-, e^+)\big) \le 16\gamma\ell(W).$$

The additional workspace is $MC\log^{O(1)} N$ beyond the cached paths and the forests themselves.

Proof. For each atom choose dyadic L_P with $\ell(P) \le L_P < 2\ell(P)$. For weights z_P, use the standard root-path difference trick on every cached predecessor tree: add z_P/L_P at the endpoint of atom P and take subtree sums. Together with the one-edge atoms this forms

$$a_e = \sum_{P \in \mathcal{P}} z_P \,\mathrm{mult}_P(e)/L_P$$

in $O(MC)$ work. Lemma 451 returns T with

$$\sum_P z_P \frac{\mathrm{cost}_T(P)}{L_P} \le \gamma \sum_P z_P.$$

A root-prefix-sum pass in the same predecessor trees computes every $\mathrm{cost}_T(P)$ in another $O(MC)$ work. Starting with $z_P = 1$, repeat this weighted call and double precisely the atoms with $\mathrm{cost}_T(P) > 8\gamma L_P$. Markov's inequality on the weighted average above says that such atoms carry less than one eighth of the current weight. Standard multiplicative-weights counting then implies that after $O(\log M)$ rounds every atom

is bad in fewer than one quarter of the retained forests. A word has at most four atoms, so the union of their four bad sets still misses at least one forest. In that common forest all four atoms are good, and there

$$\mathrm{cost}_{T_j}(W) \le 8\gamma \sum_{P \text{ in } W} L_P < 16\gamma \ell(W).$$

The weights remain polynomially bounded. If there are no nonempty atoms, one arbitrary forest suffices. □

Fix the root block's units so its input lengths lie in $[1, A_0]$. Put

$$b = \lceil \log_2(2N) \rceil, \qquad K_* = 2^{2b}, \qquad \chi_* = 4 + \lceil \log_2(2A_0) \rceil + (4L+6)b, \qquad \delta_* = K_*^{-(L+2)}.$$

Thus $K_* = \Theta(N^2)$ is dyadic. The tiny dyadic δ_* only separates search chords from forest travel. The range calculation below gives at most $\chi_* = \log^{O(1)} N$ copies of each input object and fewer than K_* total objects in every routing forest; bounded values of N may be absorbed into the explicit base case. In tree distances and paths we suppress ι_j: the endpoints u, v always mean their designated representatives.

The family is fixed for the epoch; no terminal-word catalogue is stored.

For forest j, write $T_j[e]$ for the initial tree path oriented as e. The comparison length $w_j(e)$ counts e together with that path. With child distance factor β, set

$$w_j(e) = \ell_e + \ell(T_j[e]), \qquad r = 10b, \qquad h = 2(r+1)\beta, \qquad \alpha = 16\gamma h.$$

Here r is the search threshold, h the route budget, and α the forest-cover loss. The comparison lengths remain fixed during the epoch. Each initial pair (e, j) is routed or direct; a failed search makes only that pair direct, setting $R_j(e) = e$, marking its endpoints, and creating one physical copy in H and one search chord. Epoch-inserted records instead have terminal endpoints and one direct copy in H; again write $R_j(e) = e$.

For each routed initial pair (e, j), let $\widetilde{R}_j(e)$ join the designated endpoints using tree copies and search chords in forest j, and let $R_j(e)$ be its projection to an input walk. This route is simple before projection and avoids the source chord e; its projected walk may repeat input edges. It will satisfy

$$\ell_e + \ell(R_j(e)) \le h\, w_j(e). \tag{189}$$

The next subsection constructs these routes.

Root each T_j once and preprocess DFS order and LCA. Mark every initial tree root and terminal representative, and mark both designated endpoints whenever an initial pair (e, j) becomes direct. Before closing or physically deleting a copied tree edge, mark both of its endpoints as well. Maintain the standard virtual tree of these monotonically inserted marks. Its edges are the physical forest shortcuts; one new mark changes only $O(1)$ virtual edges, and marking both endpoints of a tree edge makes that edge a singleton shortcut that no later shortcut crosses. An isolated marked root creates no record.

The physical graph H uses only input-vertex names: a virtual edge becomes an edge between the projections of its endpoints and carries its tree path as the expansion. Direct-pair and epoch-inserted records use their ordinary input endpoints. Keep parallel records and self-loops, since their expansions may matter to the cycle oracle. Forest copies therefore survive only inside the artificial search graphs below.

An individual copied route $\widetilde{R}_j(e)$ need not be a path in H, since its endpoints may be unmarked. Compression works only after routes are combined into a chain whose divergence lies on terminals. After removing the direct chords, the residual forest chain has divergence only at marks and therefore compresses exactly onto the virtual tree.

Lemma 453 (Replacement)**.** *For every input chain z with terminal-supported divergence, each forest supplies a chain $L_j z$ in H with*

$$\mathrm{lift}(L_j z) = R_j z, \qquad \partial L_j z = \partial z.$$

For every current terminal word W whose endpoints are terminals, some forest j satisfies

$$\|L_j[W]\|_\ell \le \alpha \ell(W). \tag{190}$$

Epoch-inserted terminal edges and empty words satisfy the same statement directly.

Proof. Fix j and first substitute the copied route for every routed term of z. Separate the resulting chain into its nonforest terms and its copied-tree terms. Every nonforest term already has a physical record in H: direct initial terms and search chords use their pair's direct copy, while epoch-inserted terms use their direct record.

What remains is a chain on the fixed routing forest. At an unmarked copied vertex, every incident nonforest term has already been removed and the original chain has no boundary there; route substitution also preserves boundary. Hence the residual forest chain can have divergence only at marks (terminals, direct-pair endpoints, and endpoints exposed before a closure or deletion). On a tree, such a chain is uniquely a signed sum of paths between these marked boundary vertices. Replace those paths by the corresponding virtual-tree edges. This is exact after lifting, does not increase the norm, and projection of the marked endpoints commutes with boundary. Restoring the nonforest records gives $\mathrm{lift}(L_j z) = R_j z$ and $\partial L_j z = \partial z$.

For $\xi = [W]$, compression can only decrease length, so

$$\|L_j \xi\|_\ell \le \sum_e |\xi_e| \ell(R_j(e)) \le h \sum_e |\xi_e| w_j(e),$$

where identity routes use $\ell_e \le w_j(e)$. Choose the common good forest from Lemma 452. Since cancellation can only reduce $|[W]_e|$ relative to the number of occurrences of e in W,

$$h \sum_e |[W]_e| w_j(e) \le h \sum_{e \text{ occurring in } W} w_j(e) \le 16\gamma h \ell(W) = \alpha \ell(W).$$

□

This is exactly the chain required by Proposition 450. The cycle section will later keep track of the differences $e - R_j(e)$ in order to preserve gradients.

12.6.5 Physical Routes and Search Graphs

The replacement lemma reduces the remaining dynamic problem to maintaining the route certificate (189) for every routed pair. The search graphs serve only this purpose: they look for a short detour through currently usable forest pieces and previously retained direct pairs. If no such detour exists, that pair becomes direct. Later we show that these failures form a sparse greedy skeleton, while capacity closures keep one update from invalidating too many installed routes.

The comparison forest T_j is fixed. A monotone closed-edge set controls new searches: closure removes only the search copy, whereas physical deletion also removes the tree copy and triggers repairs. A direct pair keeps its physical copy for its lifetime, while its search chord persists only until closure or source retirement.

The graph H measures physical length. Search length instead counts chords: every open virtual-tree edge has length δ_* and every search chord has length one. Along a simple routed path the virtual edges expand to edge-disjoint nonempty tree paths, so there are fewer than K_* of them and their total search length is below $K_*\delta_* < 1$. This separation also identifies chord-containing recursive records from their existing length labels. Reusing the virtual tree avoids contracting components or moving chord endpoints when an open forest edge is cut. Both kinds of graph use the same child oracle on disjoint vertex sets. The cancellation below converts a bound on the chord count into a physical route certificate.

Work in the copied graph, keeping every source chord distinct from the tree copies. Suppose $\widetilde{R}_j(e)$ is simple, avoids its source chord e, and uses search chords $f_1, \dots, f_s$. Then

$$e - \widetilde{R}_j(e) = (e - T_j[e]) - \sum_{t=1}^{s} \mathrm{sgn}(f_t)(f_t - T_j[f_t]).$$

To see the identity, remove the displayed source and search chords from both sides. What remains is a zero-boundary chain supported on one tree, and the only such chain is zero. Simplicity and avoidance of the source chord make the norm on the left exactly the route's occurrence length plus ℓ_e. Projection preserves

occurrence length even if opposite projected traversals later cancel as a signed chain. If every chord has comparison length below $2w_j(e)$, then

$$\begin{aligned}\ell_e + \ell(R_j(e)) &= \|e - \widetilde{R}_j(e)\|_\ell \\ &\le \|e - T_j[e]\|_\ell + \sum_{t=1}^{s} \|f_t - T_j[f_t]\|_\ell \\ &\le w_j(e) + \sum_{t=1}^{s} w_j(f_t) \le (1+2s)w_j(e).\end{aligned}$$

For $s \le \beta r$ and $\beta \ge 1$,

$$1 + 2s \le 1 + 2\beta r \le 2\beta(r+1) = h,$$

which proves the route certificate. This includes a route in one open component, with $s = 0$. A self-loop uses the empty route and has certificate at least ℓ_e. Initial tree paths may contain deleted edges; those edges cancel from the identity and are never traversed by the physical route.

Searches with fixed endpoints. Partition the fixed comparison lengths $w_j(e)$ into dyadic classes $[a, 2a)$, with J classes in total. For each forest and class, let $X_{j,a}$ contain a length-δ_* copy of every open virtual-tree edge and one unit edge per surviving class-a direct-pair chord in forest j, with corresponding endpoints in $X_{j,a}$'s own vertex set. Each chord stores its source identifier and orientation, with its fixed designated endpoints in T_j. Search chords stay within their initial tree; epoch-inserted edges stay directly in H. Search lengths are independent of physical lengths.

Maintain the roots of the current open components with any standard incremental nearest-marked-ancestor structure on the fixed rooted tree: regard each tree root as marked initially and mark the child endpoint c whenever its parent edge closes. The nearest such ancestor of v is exactly the root of v's open component.

When closing a forest edge, first mark its two copied endpoints in the virtual tree. It is then a singleton virtual edge; remove its search copies and mark its child endpoint in the component structure. A capacity closure keeps the physical singleton in H, while a physical deletion removes it there as well. The endpoint marks remain forever, so no later virtual edge crosses the cut. Closing a chord only removes its search record. Installed physical routes are unchanged by closures. Since every component root is a tree root or a child endpoint of a closed edge, every component root is a virtual-tree vertex, so searches between distinct components have represented endpoints.

Installing a route. Assume the distance oracle is available one level below. Its input is the disjoint union of H and all search graphs. Initialize it on the virtual edges induced by the terminal marks, including their search copies; there are initially no chords. An installation queries the child's current graph before adding any new search chord, then completes all induced child changes before the next query. The volume bound below will justify the smaller child cap. Thus route searches and terminal queries use the same child, and all its updates are charged together.

The procedure Install(e, j) skips inactive sources and pairs that are already direct, and otherwise acts as follows.

1. If the designated endpoints have the same open-component root, use their unique forest path, including the empty path for a self-loop.

2. If the roots differ, query $X_{j,a}$ between them, where $[a, 2a)$ contains $w_j(e)$. Accept an answer only if its returned bound is at most βr. Report only its chord occurrences; the search-length separation below makes this output-sensitive. There are at most βr. Erase repeated-component portions, then join consecutive chords and the original endpoints by their unique open forest paths.

3. If the test fails, make (e, j) direct, mark its two designated endpoints in T_j, and set $R_j(e) = e$. Create its direct-pair copy of e in H and insert its search chord in $X_{j,a}$. The chord is new because this pair

has never been direct. The new marks only refine this forest's representation; they do not change its open components or existing chords.

Register each nonidentity route for repairs, as described below. Initialization calls this procedure on every initial edge–forest pair in a fixed order; later calls repair routes after physical changes. Each pair becomes direct at most once. Its physical direct copy lasts until the pair retires; its search chord may disappear earlier at a capacity closure. Promotions alone require no route repair at this level: they change the virtual tree and pivots, while installed routes keep the same fixed tree bridges. Virtual-edge replacements retire their recursive users before the next query, according to the record-lifetime rule below.

An accepted answer has $O(h)$ chords. After repeated-component portions are removed, the route visits each open component once; its simple forest bridges lie in distinct components. The resulting route is simple in the copied graph and avoids its routed source chord e. Every reported chord lies in the source's comparison-length class, so the copied-graph identity proves (189) for the projected physical walk.

The pair slot stores only the ordered oriented chord sequence and handles to its repair registrations. The source and chord endpoints are fixed in the initial tree, so the sequence uniquely determines all intervening forest bridges; reconstruct those paths on demand. The static quantity $h\, w_j(e)$ already certifies the route by (189), so neither its exact occurrence length nor separate path handles are stored. Before replacing or retiring the route, delete its registrations. Search closures leave the physical route and registrations unchanged.

12.6.6 Route Repair and Sparse Selection

A physical deletion must find exactly the installed routes that used the deleted object, without scanning all routes. Capacity-limited registrations provide that output-sensitive repair interface. The same closures that bound repair fanout also cut the search forests; failed searches then create a sparse greedy skeleton. Together these two facts are what keep the recursive child small.

Live, capacity-limited registrations. Repairs use standard centroid-decomposition path stabbing on each fixed tree. Store a bridge $u \rightsquigarrow v$ in the one or two branch buckets at the centroid-tree LCA of u, v; every path in a bucket contains the same first tree edge $\rho(c, B)$. A bridge therefore creates at most two registrations, while a point query scans one bucket per centroid ancestor and reports every stored path containing the queried edge.

A nonidentity route has at most $h + 1$ forest bridges and $O(h)$ search chords, so register its bridges as above and also register it in the bucket of every direct pair whose chord it traverses. Keep an $O(1)$-deletion handle to every entry. A simple route contributes at most one entry to a bucket and creates $O(h)$ registrations. Retiring or replacing it removes them all.

When a tree-branch bucket first reaches C live entries, close its representative edge $\rho(c, B)$ before the next installation. Every current route in the bucket uses that edge, and no later open bridge can contain it, so the bucket can only shrink. A direct-pair bucket is identical except that closing it removes the pair's artificial search chord. Buckets never reopen and therefore always contain at most C live entries. Closing changes no installed physical route and causes no route repair; tree-edge closures may add the local endpoint marks and the constant number of virtual-edge changes charged below.

On deleting an initial input edge g, first retire its own routes and mark its source inactive. In every forest perform this centroid-stabbing query for all its at most χ_* tree copies, and inspect all of its direct-pair chord buckets, at most one per forest. The copies have explicit inverse lists. Thus only $\widetilde{O}(C)$ current route entries are inspected and returned. Deduplicate their source pairs and retire those routes. Complete endpoint promotions, structural retirement, and deletion of all copies of g, then reinstall the surviving pairs. A length change inserts a new direct record after retiring the old one. Removing a route also removes its bucket entries, but never triggers a closure.

Every broken route is found: immediately before the physical deletion, its bridge or chord occurrence containing g has a live registration in one of the inspected buckets. Conversely every returned entry names a current installed route, so there are no stale registrations, route versions, or liveness tests. In particular, a physical deletion does not close unrelated search edges.

Lemma 454 (Repair accounting)**.** *An epoch with B input changes has $\widetilde{O}(M + BC)$ route installations and*

$$R = \widetilde{O}\big(h(M/C + B)\big)$$

capacity closures. If $B \le M/C$ and $h \le C$, all registration work is $\widetilde{O}(MC)$.

Proof. Initialization installs $\widetilde{O}(M)$ routes, and each physical deletion reports only $\widetilde{O}(C)$ live routes to reinstall. Thus all installations create $\widetilde{O}(h(M + BC))$ registrations. Each closed bucket contains C distinct live registrations and closes only once, giving the bound on R; under the stated epoch bounds the same insertion count is $\widetilde{O}(MC)$. □

Why the child stays small. Registrations limit repair work, but they also trigger search cuts. The direct-pair search chords are just a dynamic greedy spanner on each quotient, so their sparsity follows from the standard high-girth bound.

We use the standard greedy-spanner fact: inserting an edge only when no $(r-1)$-hop path exists preserves girth above r, even under later deletions and vertex splits. For $r \ge 4\log_2(2K_*)$, the usual breadth-first growth argument then gives $|E| \le 2|V|$ on at most K_* vertices.

For a fixed forest j and length class a, contract the components of the open tree edges and retain their direct-pair chords. When a failed search makes (e, j) direct, a quotient path of at most $r-1$ old chords would lift to a search path of length below r: its $r-1$ chords cost at most $r-1$ and all forest bridges together cost below $K_*\delta_* < 1$. The child would then return a bound at most βr and the search would have succeeded. Hence every search chord satisfies the greedy insertion rule above; search closures and physical deletions only delete chords or split quotient vertices.

If forest j has had s_j open cuts, discard the isolated vertices of uncut initial trees, which have no search chords. At most $2s_j$ quotient vertices remain: the first cut of an initial tree creates two and every later cut one. Since the forest has fewer than K_* vertices, the standard girth bound gives

$$|E(Q)| \le 2|V(Q)| \le 4s_j. \tag{191}$$

Thus each search cut permits only constantly many open search chords per class. Lifetime direct-pair creations are bounded by also charging chords that have since been removed.

Lemma 455 (Child volume)**.** *During an epoch with B input changes, the total number of records created or changed in H and all search graphs is at most*

$$V \le Fh(M/C + B),$$

where $F = \log^{O(1)} N$ is independent of C, h, B.

Proof. Let R be as in Lemma 454. The range calculation below verifies that J, λ, χ_* are fixed polylogarithms independently of C; we suppress only these factors.

There are $\widetilde{O}(B + R)$ open forest cuts, caused only by input deletions and tree-edge closures. The girth bound gives the same bound on open search chords across all forests and classes. Chords already removed number at most $B + R$, so lifetime direct-pair creations also total $\widetilde{O}(B + R)$.

Thus the initial terminal marks number $\widetilde{O}(M/C)$. Later endpoint promotions contribute $\widetilde{O}(B)$ terminal marks, while local cut and direct-pair marks, direct-pair records, and physical forest deletions contribute $\widetilde{O}(B + R)$. The endpoint promotions are already covered by the prefix-incidence bound. By the virtual-tree invariant, each such event changes only $O(1)$ physical virtual-edge records; each shortcut has at most J search copies. Chords have fixed endpoints, so their only record changes are insertion and removal. Hence

$$V = \widetilde{O}(M/C + B + R) = \widetilde{O}\big(h(M/C + B)\big),$$

using $h \ge 1$. This is the claimed bound for a fixed polylogarithm F.

For $B \le M/C$ and $h \le C$, Lemma 454 already bounds the registration work and space by $\widetilde{O}(MC)$. The copied trees and inverse lists take $\widetilde{O}(M)$ space. Route retirement is charged to its installation, and each direct-pair state change takes constant local work beyond its failed installation. Promotions inspect $O(MC)$ prefix incidences in total. This bookkeeping takes $\widetilde{O}(MC)$; chord reporting is charged separately below. □

12.6.7 Implicit Paths, Reporting, and Record Lifetimes

It remains to query, selectively expand, and retire the compact descriptions without materializing their full walks.

A query returns a cached path or attaches two pivot paths to a child answer, as in Proposition 450; leaves store exact shortest-path trees. Each level adds constantly many descriptors, so depth L gives $O(L+1)$ descriptors and $O((L+1)\log^{O(1)} N)$ query time. Store each descriptor's exact occurrence length by adding its parts' lengths.

Reporting uses only static tree indices. In each routing tree, mark an edge whose parent input record has search length at least one and store nearest-marked-ancestor links; a tree path reports these edges by two pointer walks to its LCA. Expansion pointers report all occurrences, and root-prefix sums give lengths. Link–cut trees appear only later in maximum flow.

Every physical output edge expands to one parent input edge or the projection of a nonempty copied-forest path, with additive occurrence length. Each tree-edge copy forwards to one parent input record. To report at a requested input level, traverse descriptions in order and stop at its input edges. Every visited edge occurrence has at least one reported descendant. An output of s occurrences therefore visits at most s occurrences at any level, taking $O((L+1+sL)\log^{O(1)} N)$ work.

Search answers require only chord occurrences. A chord-free search record has root length below $K_*^{L+1}\delta_* < 1$, whereas one chord contributes length one; thus chord-containing records are exactly those of search length at least one. Static marked-ancestor links skip all subunit tree records, so a description containing s chords is reported in $O((L+1)(s+1)\log^{O(1)} N)$ work. An accepted search has $s \le \beta r = O(h)$, giving the claimed output-sensitive bound.

For later applications, no "reaches the root" bit or record tag is needed. The caller already knows whether a recursive instance descends from the physical graph H or from an artificial search graph, and routing forests never join distinct input components. Only descendants of H ever receive level-0 flow fields; search descendants are used solely for internal queries.

Retirement. Reference counting enforces the required recursive order; at an epoch boundary retire the child suffix before discarding the parent forests. Route retirement first removes its live bucket entries. Inserting a mark or closing a forest edge may replace constantly many virtual-edge records, but it leaves the fixed forest, chord endpoints, and every installed route description unchanged. Thus buckets never expose stale routes, and closures require no route repair.

12.6.8 Closing the Recursion

Everything above can now be summarized by a one-level interface. Up to fixed polylogarithms, an epoch of size M costs MC locally, sends at most $Fh(M/C+B)$ records to the child after B changes, and turns a child factor β into a parent factor $15\alpha\beta = O(\gamma(r+1)\beta^2)$. The square in β is structural: one factor determines how long a detour we are willing to install, and the second is paid when the parent later queries the child. We therefore choose the parameters in that order: approximation budgets first, then the common cutoff C, then the epoch length.

At an explicit leaf of size b, run Dijkstra to exhaustion from every vertex after each change. This gives exact distances and their shortest-path trees in $b^2\log^{O(1)} N$ work and $O(b^2)$ space. No terminals or route construction are needed there.

Choose depth $1 \le L \le \log_2(2N)$ and let k be the least power of two at least $N^{1/L}$. We want each child to be smaller by a factor k. With an epoch of $B \le M/C$ changes, its volume is at most $2FhM/C$, so we need C to dominate kFh. Since h depends on the child's approximation, choose all approximation budgets first and use one cutoff large enough for every level. Let p be a sufficiently large dyadic polylogarithm, with a fixed exponent, and set

$$\Gamma_L = 1, \qquad \Gamma_i = p\Gamma_{i+1}^2 \quad (0 \le i < L), \qquad C = kp\Gamma_0. \tag{192}$$

Use this same cutoff at every level. At depth i, take $\beta = \Gamma_{i+1}$, so $h_i = 2(r+1)\Gamma_{i+1}$ and $\alpha_i = 16\gamma h_i$. Since $15\alpha_i\Gamma_{i+1} = 480\gamma(r+1)\Gamma_{i+1}^2$, taking $p \geq 480\gamma(r+1)$ makes this at most Γ_i. Exact leaves start this induction.

Use a leaf when $M \leq C$. Otherwise let an epoch last $\lfloor M/C \rfloor$ incoming changes and give its child cap $\lceil M/k \rceil$. This epoch length balances the initialization term M/C against the number of changes. Since $\Gamma_{i+1} \leq \Gamma_0$, the volume lemma gives

$$\begin{aligned} V_{\text{child}} &\leq Fh_i(M/C + B) \leq \frac{4F(r+1)\Gamma_{i+1}M}{C} \\ &= \frac{4F(r+1)M}{kp}\,\frac{\Gamma_{i+1}}{\Gamma_0} \leq \frac{M}{2k}, \end{aligned} \tag{193}$$

provided $p \geq 8F(r+1)$. The count includes initialization and every later child record, so it bounds live size even when isolated child vertices remain until the epoch ends. Repeated ceilings satisfy $\lceil \lceil M/k \rceil / k \rceil = \lceil M/k^2 \rceil$; since $k^L \geq N$, the cap at depth L is at most one. Thus the size rule alone stops the recursion by depth L.

The range bounds are now direct. A physical output record expands through fewer than K_* input records. Thus at global depth i a component descended from the root physical graph has labels in $[1, K_*^i A_0]$, while one descended from a search graph has labels in $[\delta_*, K_*^i]$. Therefore

$$2 + \lceil \log_2(2M\Delta_i) \rceil \leq 3 + \log_2(2N) + \log_2 A_0 + (i + L + 2)\log_2 K_* \leq \chi_*$$

for $i \leq L$, because $\log_2 K_* = 2b$. Lemma 451 then gives forest size at most $M\chi_* \leq N\chi_* < K_*$ for all sufficiently large N; the remaining bounded cases use the explicit solver. This closes the induction. A tree route has fewer than K_* edges, so the ratio between comparison lengths over all physical and search-descended components is at most $A_0 K_*^{2L+3}$ up to a constant. Hence

$$J = O(\log A_0 + L \log N) = \log^{O(1)} N.$$

Also $r = O(\log N)$, and γ, χ_* and every factor in F are fixed polylogarithms independent of C and p. One sufficiently large fixed dyadic polylogarithm p pays for them, ensures $h_i \leq C$, and absorbs the local work.

An internal epoch uses at most $F_{\text{loc}}MC$ local work, for a fixed polylogarithm F_{loc}. The initial searches, prefix incidences, and implicit atom cover all fit this allowance. With $B \leq M/C$, there are $M \log^{O(1)} N$ installations; each reports $O(h_i)$ chords and performs $O(h_i)$ component-root, bridge, and registration operations up to logarithms. Since $h_i \leq C$, these also total $MC \log^{O(1)} N$. Shortcut activations and their search copies fit the same allowance.

Let U_i bound the amortized cost per incoming change, with MU_i allowed to initialize an instance of cap M. An internal epoch costs at most $F_{\text{loc}}MC$ locally. Its child cap and total incoming record volume are both $O(M/k)$, so child initialization and updates cost $O((M/k)U_{i+1})$. Spread over $\Omega(M/C)$ incoming changes, the local bill is $O(F_{\text{loc}}C^2)$ per change, and the child receives $O(C/k)$ changes per parent change. Thus

$$U_i \leq F_{\text{loc}}C^2 + c(C/k)U_{i+1} \tag{194}$$

for an absolute constant c. Prepay the first epoch's allowance from the initial cap, and each later one from the completed epoch before it. This covers a truncated last epoch even when its work occurs near its start. A leaf has $b \leq C$ and rebuilds in $O(C^2)$ per change, within the same allowance. Because C is uniform, unrolling gives

$$\begin{aligned} U_0 &\leq F_{\text{loc}}C^2 \sum_{j=0}^{L} (cC/k)^j \\ &\leq (L+1)F_{\text{loc}}C^2(1 + cC/k)^L = k^2(p\Gamma_0)^{O(L+1)}. \end{aligned}$$

Similarly, the approximation recurrence gives

$$\log \Gamma_0 = \sum_{j=0}^{L-1} 2^j \log p = (2^L - 1)\log p.$$

Theorem 456 (Parameterized distance oracle)**.** *The distance approximation is* $\Gamma_0 = p^{2^L-1}$*, and total maintenance work is*

$$(N + q_{\mathrm{upd}})k^2(p\Gamma_0)^{O(L+1)}.$$

An implicit query costs $O((L+1)\log^{O(1)} N)$*; reporting* s *input occurrences costs* $O((L+1+sL)\log^{O(1)} N)$*. The construction also retains each route's oriented chord sequence and the static certificate* $\ell_e + \ell(R_j(e)) \le h\, w_j(e)$*. At one recursive level, any input edge has at most* $\lambda\chi_*$ *routing-tree copies and at most* $\kappa := 1 + \lambda(\chi_* + 1) = \log^{O(1)} N$ *current physical output occurrences, counted with multiplicity: each copied forest edge lies in at most one current shortcut, each forest contributes at most one direct-pair copy, and an epoch-inserted edge contributes one direct copy.*

For bounded instances use the explicit solver. Otherwise choose $L = \max\{1, \lfloor \frac{1}{2}\log_2\log_2 N \rfloor\}$. Then

$$\log k \le \frac{\log N}{L} + \log 2 = O\left(\frac{\log N}{\log\log N}\right),$$
$$(L+1)\log(p\Gamma_0) = (L+1)2^L \log p = O\left(\sqrt{\log N}(\log\log N)^2\right).$$

Both are $o(\log N)$, so the maintenance factor, C, and Γ_0 are $N^{o(1)}$. Each instance's workspace is bounded by $MC\log^{O(1)} N$ for the cached atoms and live auxiliary structures, plus the routing forests and child records already covered by the same $N^{1+o(1)}$ bound. Summing over the shrinking caps gives $N^{1+o(1)}$ space. These estimates prove Theorem 448. KMPG additionally deamortizes to worst-case $N^{o(1)}$ updates; the total-work form above is all we need here. The next two sections use only three extra pieces of the interface: implicit physical representations, the oriented chord sequence and certificate of each installed route, and the per-level physical-overlap bound κ.

12.7 Minimum-Ratio Cycles

Give every input edge an arbitrary real gradient g_e, using the reference orientation and weighted length from the distance construction. A circulation is a signed edge vector z satisfying $\partial z = 0$. For a nonzero circulation, define

$$\mathrm{gain}(z) = \frac{|g^{\mathsf{T}} z|}{\|z\|_\ell}, \qquad \mathrm{OPT} = \max_{z\neq 0,\ \partial z = 0} \mathrm{gain}(z).$$

Orient a circulation by its coefficient signs and repeatedly subtract directed cycles. The resulting decomposition $z = \sum_K t_K[K]$, with $t_K > 0$, has no cancellation, so

$$\|z\|_\ell = \sum_K t_K \ell(K),$$
$$|g^{\mathsf{T}} z| \le \sum_K t_K |g(K)| \le \big(\max_K \mathrm{gain}(K)\big)\|z\|_\ell.$$

Thus OPT is also the maximum over simple cycles. Set $\mathrm{OPT} = 0$ when there is no nonzero circulation. Reversing z changes the sign of its numerator, so maximizing this gain is equivalent to finding a most negative ratio for descent. The gradients need not be bounded by, or related to, the lengths.

An answer is a signed physical expression P whose chain $[P]$ is a nonzero circulation, with exact gradient $g(P) = g^{\mathsf{T}}[P]$. Its certificate satisfies $\|[P]\|_\ell \le \ell(P) \le D$. The *certified gain* $|g(P)|/D$ can therefore be smaller than the actual gain of $[P]$. We use it because applying a path and its reversal can require work even when their net vector is zero. A factor-Λ oracle returns certified gain at least OPT/Λ, or certifies $\mathrm{OPT} = 0$. The gradient numerator is exact throughout.

The distance reduction already tells us how to replace every physical cycle by local cached pieces, installed routes, and finally a child circulation. The only new issue is that replacement can erase gradient even when it preserves length. There are exactly two corresponding error terms: route corrections $e - R_j(e)$ for substitutions outside the balls, and cached-tree corrections inside a ball. We store one candidate for each

kind in a single heap. If all candidates have small certified gain, the total gradient lost during the distance replacement is small, so the child must retain the rest. This gives circulation factor $C\Gamma_i$ at a level whose distance factor is Γ_i, with the same cutoff and maintenance bound.

12.7.1 Route Corrections

Assign each physical output record the oriented gradient sum of its input expansion; direct copies inherit the input gradient, while artificial search components have gradient zero. Gradient changes follow the same promote/delete/insert protocol as lengths, so record labels remain immutable.

For every initial source–forest pair precompute the fixed *tree discrepancy*

$$\mu_j(e) = g_e - g(T_j[e]).$$

This is the gradient of the fundamental copied-tree cycle $e - T_j[e]$. One rooted prefix-sum pass per routing forest forms all these scalars during epoch preprocessing. If a routed pair (e, j) uses oriented search chords $f_1, \dots, f_s$, the same tree identity used to prove its length certificate gives

$$\nu_j(e) := g(e - R_j(e)) = \mu_j(e) - \sum_{t=1}^{s} \operatorname{sgn}(f_t)\mu_j(f_t). \tag{195}$$

Thus its correction $C_{j,e} = e - R_j(e)$ has exact gradient $\nu_j(e)$ and occurrence certificate $h\, w_j(e)$ by (189). Identity routes have zero correction and no key. The formula remains valid after tree-edge deletions: it is an algebraic identity in the immutable initial copied tree, and all deleted tree terms cancel before projection.

For any terminal-supported chain z,

$$g^{\mathsf{T}} z - g_H^{\mathsf{T}} L_j z = \sum_e z_e \nu_j(e), \tag{196}$$

where g_H is the additive child gradient. Indeed, $\operatorname{lift}(L_j z) = R_j z$ and each edge's gradient loss is $\nu_j(e)$. Suppose all current correction keys are at most a. For a terminal word W, choose a good forest and put $Y_W = L_j[W]$. The cover and (190) give

$$\begin{aligned} |g(W) - g_H^{\mathsf{T}} Y_W| &\le ah \sum_e |[W]_e| w_j(e) \le a\alpha\ell(W), \\ \|Y_W\|_\ell &\le \alpha\ell(W), \qquad \partial Y_W = \partial[W]. \end{aligned} \tag{197}$$

Thus the same length budget controls the lost gradient and the chain sent to the child. Epoch-inserted terminal edges use their direct copies and have no gradient loss.

At installation, (195) computes $\nu_j(e)$ using only the stored chord sequence, in $O(h)$ scalar operations. If it is nonzero, insert the key $|\nu_j(e)|/(hw_j(e))$ in the global heap and store its deletion handle with the route. Delete that key before replacing or retiring the route. Search closures leave both the route and its key unchanged.

12.7.2 Local Corrections

We must also account for the local prefixes replaced when forming terminal words, and for cycles that never leave a ball. The cached predecessor trees supply both kinds of correction. For every edge $e = (x, y)$ whose endpoints lie in the current ball of u, use the rooted expression

$$C_{u,e} = \pi_u(x) + e - \pi_u(y), \qquad D_{u,e} = d_u(x) + \ell_e + d_u(y).$$

Its numerator is $g(\pi_u(x)) + g_e - g(\pi_u(y))$, so cached root-gradient sums give the score directly. Keeping the two root paths rather than cancelling their common prefix avoids any additional tree preprocessing; the

slightly larger certificate is already strong enough below. Zero-gradient expressions need not be retained, and a nonzero-gradient expression necessarily has nonzero chain. Loops and parallel edges require no special case.

Suppose every such candidate has certified gain at most a. Let P run from u to w inside its ball with s edge occurrences. The root-gradient terms telescope, so

$$\begin{aligned} |g(P) - g(\pi_u(w))| &= \left| \sum_e [P]_e g(C_{u,e}) \right| \\ &\le a \sum_{e \text{ occurring in } P} D_{u,e} \le 2as\ell(P). \end{aligned} \tag{198}$$

For the last inequality, an edge reached after length t has $D_{u,e} \le t + \ell_e + (t + \ell_e) \le 2\ell(P)$. A simple local prefix has $s \le C$. For a simple cycle K of length W based at u, there is a sharper bound. Delete an edge $e = (x, y)$ of length q. The remaining cycle path contains a u–x arc of length b and a u–y arc of length c, with $b + c = W - q$. Since the cached distances are shortest,

$$D_{u,e} = d_u(x) + q + d_u(y) \le b + q + c = W.$$

Thus a cycle contained in $B(u)$ has $|g(K)| \le a \sum_{e \in K} D_{u,e} \le CaW$.

Ball membership only shrinks. For an initial candidate $e = (x, y)$ at u, set $\rho_{u,e} = \max\{d_u(x), d_u(y)\}$. It is current exactly while $\rho_{u,e} < r_u$; then the stable-ball lemma fixes its edge, root paths, length, and gradient. Inserted records create no candidates because their endpoints are promoted first.

Insert all positive-score initial candidates into the same max-heap as the route corrections, storing their fixed scores and expiry radii. After maintenance discard expired local maxima until the top is current. Each local entry is removed once, giving $MC \log^{O(1)} N$ heap work per epoch; route keys are inserted and deleted with their routes. Thus the cleaned maximum is the best current physical correction, or the empty heap certifies zero.

These are exactly the two corrections created by the distance reduction. It remains only to decompose a cycle in the same way and charge the lost gradient to these corrections or to the child.

12.7.3 Why These Candidates Suffice

A cycle need not itself contain a terminal, which is the only mismatch with the terminal-to-terminal decomposition used for distances. Attach one pivot path in each direction. The two copies cancel as a signed chain, so gradient and circulation are unchanged, but the resulting closed walk starts at a terminal and can be decomposed with no exceptional final piece.

Proposition 457 (One circulation reduction)**.** *Suppose every local candidate in a current ball and every current nonidentity route correction has certified gain at most a, and the child satisfies $\mathrm{OPT}_H \le Ra$. Then*

$$\mathrm{OPT}_G \le [18C + 9\alpha(1 + R)]a.$$

Proof. Fix a simple oriented cycle K of length W and a vertex u on it. If $K \subseteq G[B(u)]$, the sharper whole-cycle bound above gives $|g(K)| \le CaW$.

Otherwise the component contains a terminal and $r_u \le W$. Let $P = \pi_u(p_u)$ and form the closed walk

$$\widehat{K} = -P + K + P.$$

It starts at the terminal p_u, has the same signed chain and gradient as K, and has occurrence length $D \le W + 2r_u \le 3W$. Moreover every input edge occurs in $\widehat{K}$ at most three times: once in the simple cycle and once in each copy of P.

Split $\widehat{K}$ successively at first exits exactly as in Section 12.6.2. Because it is a terminal-based closed walk, every piece has both pivot attachments and there is no unfinished piece. Hence (186) gives

$$\sum_Q \ell(W_Q) \le 3D \le 9W.$$

Each local prefix lies in one ball, containing at most C distinct edges. Because every input edge occurs in $\widehat{K}$ at most three times, such a prefix has at most $3C$ occurrences. Applying (198) to all prefixes and summing their lengths therefore loses at most $6CaD \le 18CaW$ in gradient.

Replace every terminal word in a good forest and sum the resulting child chains to obtain a circulation Y. The pivot terms cancel around the closed walk, while (197) and the preceding length bound give

$$|g(K) - g_H^{\mathsf{T}} Y| \le (18C + 9\alpha)aW, \qquad \|Y\|_\ell \le 9\alpha W.$$

Finally $|g_H^{\mathsf{T}} Y| \le Ra\|Y\|_\ell$ proves the claim. The argument also covers self-loops and parallel-edge cycles. □

12.7.4 Reusing the Distance Recursion

The proposition has exactly the recursive shape of the distance hierarchy. We therefore reuse its levels, epochs, cutoff, and child graphs, adding only gradient labels and the global candidate heap.

At a leaf of size $b \le C$, take no terminals: the exact distance cache contains whole components, so every radius is infinite and the same local candidates apply. The whole-cycle bound gives factor C. Treat each leaf cache as an epoch; rebuilding the exact cache after a change also rebuilds its $O(b^2)$ candidate data. There is no separate leaf cycle algorithm.

Let a be the global heap maximum, or zero if it is empty, and use the distance recursion's common cutoff C and factors $\Gamma_L = 1$, $\Gamma_i = p\Gamma_{i+1}^2$. The heap invariant bounds every required gain at every level by a. At a leaf, $\mathrm{OPT} \le Ca = C\Gamma_L a$. Inductively the proposition gives

$$\mathrm{OPT} \le [18C + 9\alpha_i(1 + C\Gamma_{i+1})]a \le 24C\alpha_i\Gamma_{i+1}a \le C\Gamma_i a.$$

The last inequality only enlarges the fixed constant in the distance recursion's polylogarithm p: since $\alpha_i = 32\gamma(r+1)\Gamma_{i+1}$, it suffices to take $p \ge 768\gamma(r+1)$. The local error is additive, so its factor C does not accumulate. At the root, the heap maximum therefore has certified gain at least $\mathrm{OPT}/(C\Gamma_0)$; an empty heap certifies zero optimum.

Theorem 458 (Adding circulation queries)**.** *The distance structure supports minimum-ratio circulation queries with approximation $\Lambda = C\Gamma_0$ and total maintenance work*

$$(N + q_{\mathrm{upd}})k^2(p\Gamma_0)^{O(L+1)}.$$

With the distance construction's depth choice, the approximation and maintenance factor are $N^{o(1)}$, and the space is $N^{1+o(1)}$. An implicit circulation query takes $O(1)$ time after maintenance. Its answer retains its level of origin and either a source edge with an $O(h_i)$ oriented chord sequence, whose tree bridges are reconstructed from fixed endpoints, or one local edge with two cached root paths. The resulting occurrence length is at most the stored certificate. These bounds count arithmetic operations and allow arbitrary gradients.

Proof. The preceding induction gives the approximation. The added work fits the existing allowance: initial local-candidate insertion and all lazy expiry pops cost $MC\log^{O(1)} N$ per internal epoch. Each route installation adds $O(h)$ scalar discrepancy operations and one heap insertion when its correction is nonzero; retirement performs the matching deletion. Exact additive gradients on new physical records use the same path indices as their lengths. The distance structure already bounds all installations, promotions, and record retirements. Thus the maintenance bound is unchanged up to fixed polylogarithmic factors. After the lazy top cleanup, the heap maximum is current, so a query returns its description and certificate in constant time. □

The maximum-flow algorithm below uses exactly this interface: a constant-time query, an exact stored-label numerator, an upper certificate on occurrence length, and an implicit physical description whose updates are already charged to the distance hierarchy.

12.8 Flows

12.8.1 Highest Label Push-Relabel

We analyze the highest-level selection rule for the preflow-push algorithm.

Let $K = \sqrt{m}$. For any node v, define $d'(v) = |\{w : d(w) \leq d(v)\}|$. Thus $d'(v)$ is the number of nodes with a distance label no greater than v's label. We define the potential function Φ as:

$$\Phi = \sum_{v \text{ active}} \frac{d'(v)}{K}$$

Initially, $\Phi \leq n^2/K$, and $\Phi \geq 0$ throughout the execution. Let us examine how the operations affect Φ:

- **Relabeling:** A relabeling of a node v increases Φ by at most n/K, because $d'(v)$ may increase to at most n, while $d'(w)$ for all other nodes $w \neq v$ does not increase.
- **Saturating push:** This operation creates at most one new active node, increasing Φ by at most n/K.
- **Nonsaturating push:** A nonsaturating push from v to w deactivates v and activates w (if it is not already active). Since the push only happens on eligible edges where $d(w) < d(v)$, we know $d'(w) \leq d'(v)$, meaning Φ does not increase.

To bound the number of nonsaturating pushes, we divide the execution into phases. A phase consists of all pushes between two consecutive changes of the maximum active node label, $d^* = \max\{d(v) : v \text{ is active}\}$. We classify a phase as *cheap* if it contains at most K nonsaturating pushes, and *expensive* otherwise.

Claim 459. *The number of phases is at most $4n^2$, and the number of nonsaturating pushes in cheap phases is at most $4n^2K$.*

Proof. Initially, $d^* = 0$, and any increase in d^* is strictly due to a relabeling. The number of relabelings is bounded by $2n^2$. Thus, d^* increases at most $2n^2$ times and consequently decreases at most $2n^2$ times, limiting the total number of changes (and thus phases) to $4n^2$. The bound on cheap pushes follows immediately since each cheap phase has at most K pushes. □

Claim 460. *An expensive phase containing $Q > K$ nonsaturating pushes decreases Φ by at least Q.*

Proof. During an expensive phase, d^* remains constant, meaning all Q nonsaturating pushes originate from nodes at level d^*. The phase ends either when all nodes at level d^* are deactivated, or a node is relabeled to $d^* + 1$. Therefore, level d^* must contain $Q > K$ nodes (active or inactive) at all times during the phase. For any nonsaturating push along an edge (u, v), because there are at least K nodes at level $d(u)$, each push decreases Φ by at least one. □

The total increase of Φ is bounded by the sum of increases from relabelings and saturating pushes. There are at most $2n^2$ relabelings and $2nm$ saturating pushes, so the total increase is at most $(2n^2 + 2nm)(n/K)$. Because $\Phi \geq 0$, the total decrease is bounded by the total increase plus the initial Φ value (n^2/K).

This bounds the number of nonsaturating pushes in expensive phases by $(2n^3 + 2n^2m + n^2)/K \leq (3n^3 + 2n^2m)/K$. Adding the cheap pushes, the total number of nonsaturating pushes is at most $(3n^3 + 2n^2m)/K + 4n^2K$. By setting $K = \sqrt{m}$ and noting $n = O(m)$, this evaluates to $O(n^2\sqrt{m})$.

Theorem 461. *The running time of the preflow-push algorithm using the highest-level selection rule is $O(n^2\sqrt{m})$.*

Proof. Each push and relabel operation can be implemented in $O(1)$ time. The highest-level selection rule can be implemented using a bucket data structure where each bucket i holds all nodes w with $d(w) = i$. Maintaining this structure incurs a total running time overhead of $O(nm)$. Therefore, the algorithm's running time is dominated by the $O(n^2\sqrt{m})$ nonsaturating pushes, resulting in an overall time complexity of $O(n^2\sqrt{m})$. □

12.8.2 Height-Bounded Push–Relabel

Height-bounded push–relabel maintains a bounded-height pseudoflow. Unrouted surplus is confined to height H; after deletions, the symmetric *Pull* operation repairs negative balance and confines any remaining deficit to height zero. The height labels certify both residual distance and sparse level cuts.

Let $G_0 = (V_0, E_0)$ be directed, with n vertices, m edges, and capacities $c_e > 0$. Supply $\sigma(v) \geq 0$ may increase, absorption capacities $\tau(v) \geq 0$ are fixed, and vertices may be deleted. Write $G = (V, E)$ for the surviving graph and $\Lambda = \log(n + m + 2)$. A routing consists of $a \leq \sigma$, $z \leq \tau$, and $0 \leq f \leq c$ satisfying $\operatorname{div} f = a - z$.

Push and Pull. Maintain a possibly nonconserving flow $0 \leq f \leq c$ and balances

$$x(v) = \sigma(v) + \operatorname{in}_f(v) - \operatorname{out}_f(v).$$

The interval $[0, \tau(v)]$ can be absorbed locally; define

$$e^+(v) = (x(v) - \tau(v))_+, \qquad e^-(v) = (-x(v))_+.$$

Each vertex has an integer height $\ell(v) \in \{0, \dots, H\}$. Every residual arc satisfies

$$\ell(u) \leq \ell(v) + 1, \qquad u \to v \text{ admissible} \iff \ell(u) = \ell(v) + 1. \tag{199}$$

A completed state satisfies

$$e^+(v) > 0 \Longrightarrow \ell(v) = H, \qquad e^-(v) > 0 \Longrightarrow \ell(v) = 0. \tag{200}$$

Push sends surplus along admissible arcs. If its current root has no admissible outgoing arc and is below H, raise it by one. Pull is symmetric: it sends flow along admissible arcs into a deficit and lowers a root with no admissible incoming arc. These relabels preserve (199). If v has no admissible outgoing arc, integrality gives $\ell(v) \leq \ell(w)$ for every residual $v \to w$, so raising v by one remains feasible. The Pull case is symmetric. Augmentation can create only the reverse of an admissible arc, whose height difference is -1.

Searches are maintained as forests of admissible arcs. A Push path from surplus u to root r augments by

$$\min\{e^+(u),\ \tau(r) - x(r),\ \text{path bottleneck}\}$$

when $x(r) < \tau(r)$; otherwise r is extended through an admissible arc or raised. Symmetrically, a deficit v in Pull receives

$$\min\{e^-(v),\ x(r),\ \text{path bottleneck}\}$$

from a positive-balance root r; otherwise r is extended or lowered. Forest edges are removed upon saturation or loss of admissibility.

Every represented path consists of admissible arcs and hence has at most H edges. Put

$$\lambda_H := \min\{H, \Lambda\}.$$

A forest path operation can therefore be implemented in $O(\lambda_H)$ time: if $H \leq \Lambda$, store parent pointers and walk the path explicitly; otherwise use a link–cut tree. Initialization, current-arc scans, and scalar balance updates use ordinary adjacency lists and take constant time per touched item.

Initially $f = 0$ and $\ell = 0$, and we run Push. A supply increase resumes Push. To delete S, remove its incident edges and, using the old flow, update every survivor by

$$x(v) \leftarrow x(v) + f(v, S) - f(S, v). \tag{201}$$

Run Pull and then Push. Pull creates no surplus and Push creates no deficit, so (200) is restored.

Distance and level cuts. For every residual path $P : u \rightsquigarrow v$, summing (199) gives

$$\ell(u) - \ell(v) \le |P|, \tag{202}$$

with equality on admissible paths. Every augmentation therefore has at most H edges, while every residual path from surplus to deficit in a completed state has at least H edges.

Without deletions, deficits never occur and heights only increase. Unused absorption stays at height zero, so the maintained flow is a routing. If its value is F, then

$$\mathrm{hcost}(f) := \sum_e f_e \le HF. \tag{203}$$

Let g be any feasible routing of value $G > F$. In the usual source–sink augmentation network, decompose the residual difference $g - f$ into augmenting paths and cycles. The augmenting paths carry total value $G - F$ from unused supply at height H to unused absorption at height zero. On each physical edge, their total residual usage is at most

$$|g_e - f_e| \le g_e + f_e,$$

so their total hop cost is at most $\mathrm{hcost}(g) + \mathrm{hcost}(f)$. By (202),

$$H(G - F) \le \mathrm{hcost}(g) + \mathrm{hcost}(f) \le \mathrm{hcost}(g) + HF,$$

and hence

$$2F \ge G - \mathrm{hcost}(g)/H. \tag{204}$$

For $i = 1, \dots, H$, set

$$S_i = \{v : \ell(v) \ge i\}, \qquad T_i = V \setminus S_i,$$

and write

$$r_f(S, T) = c(S, T) - f(S, T) + f(T, S).$$

Summing the balance equation over S gives

$$x(S) = \sigma(S) - f_{\mathrm{net}}(S, V \setminus S).$$

Since every edge contributes once positively and once negatively to the total balance,

$$x(V) = \sigma(V).$$

Thus for $T = V \setminus S$,

$$f_{\mathrm{net}}(S, T) = \sigma(S) - x(S) = x(T) - \sigma(T).$$

In a completed state, $x(S_i) \ge 0$ and $x(T_i) \le \tau(T_i)$, so

$$f_{\mathrm{net}}(S_i, T_i) \le \min\{\sigma(S_i), \tau(T_i)\}. \tag{205}$$

Every residual arc crossing $S_i \to T_i$ has tail at height i and head at height $i - 1$. Thus each original edge contributes to at most one level. Define

$$d_c(v) = \sum_{e \ni v} c_e, \qquad D = d_c(V), \qquad d_{\min} = \min_{d_c(v) > 0} d_c(v).$$

For $L_i = \{v : \ell(v) = i\}$,

$$r_f(S_i, T_i) \le d_c(L_i).$$

Order the nonempty level sets from high to low, let their masses be μ_j, and put $M_j = \sum_{k \le j} \mu_k$. Then

$$\sum_i \frac{r_f(S_i, T_i)}{d_c(S_i)} \le \sum_j \frac{\mu_j}{M_j} \le 1 + \ln \frac{D}{d_{\min}},$$

since for $j > 1$,

$$\frac{\mu_j}{M_j} = 1 - \frac{M_{j-1}}{M_j} \le \ln \frac{M_j}{M_{j-1}}.$$

Charging each crossing arc to its lower endpoint and reversing the order gives the same bound with denominator $d_c(T_i)$. Therefore

$$\sum_{i=1}^{H} \frac{r_f(S_i, T_i)}{\min\{d_c(S_i), d_c(T_i)\}} \le 2\left(1 + \ln \frac{D}{d_{\min}}\right). \tag{206}$$

Theorem 462 (Weighted flow or sparse cut)**.** *Let b satisfy*

$$\sigma, \tau \le b, \qquad d_c \le \kappa b.$$

For $D > 0$, $\phi > 0$, and $0 < \varepsilon \le 1$, run Push with capacities c/ϕ and

$$H = \max\left\{1, \left\lceil \frac{2\kappa}{\varepsilon\phi}\left(1 + \ln \frac{D}{d_{\min}}\right)\right\rceil\right\}.$$

Either it routes $\min\{\sigma(V), \tau(V)\}$ with congestion $1/\phi$, or some nontrivial level cut (S, T) satisfies

$$r_f(S, T) \le \varepsilon \min\{b(S), b(T)\}, \qquad c(S, T) \le (1 + \varepsilon)\phi \min\{b(S), b(T)\}. \tag{207}$$

The running time is $O(n + m + mH\lambda_H)$.

If $\sigma(V) \le \tau(V)$ and C is the side of smaller b-mass, there is $\widehat{\sigma} \le \sigma|_{V\setminus C}$ routable inside $G[V \setminus C]$ such that

$$b(C) \le \tfrac{1}{2} b(V), \qquad \sigma(V \setminus C) - \widehat{\sigma}(V \setminus C) \le (1 + \varepsilon) b(C). \tag{208}$$

Proof. Suppose the smaller terminal total is not fully routed. Some unused supply is then at height H, while some unused absorption is at height zero. Hence every intermediate level has

$$b(S_i) > 0, \qquad b(T_i) > 0.$$

After scaling the capacities by $1/\phi$,

$$d_c/\phi \le (\kappa/\phi) b,$$

while $D/d_{\min}$ is unchanged. Thus (206) gives

$$\sum_{i=1}^{H} \frac{r_f(S_i, T_i)}{\min\{b(S_i), b(T_i)\}} \le \frac{2\kappa}{\phi}\left(1 + \ln \frac{D}{d_{\min}}\right).$$

Consequently some level satisfies

$$\frac{r_f(S_i, T_i)}{\min\{b(S_i), b(T_i)\}} \le \frac{2\kappa}{H\phi}\left(1 + \ln \frac{D}{d_{\min}}\right) \le \varepsilon,$$

which is the first inequality in (207). For this level,

$$\begin{aligned} \frac{c(S, T)}{\phi} &= f_{\text{net}}(S, T) + r_f(S, T) \le \min\{\sigma(S), \tau(T)\} + \varepsilon \min\{b(S), b(T)\} \\ &\le (1 + \varepsilon) \min\{b(S), b(T)\}, \end{aligned}$$

using (205) and $\sigma, \tau \le b$. Multiplying by ϕ proves the second inequality.

For retention, decompose the routed flow into source–sink paths and let

$$U = f(T, S) \le r_f(S, T).$$

If $C = S$, all source supply in T is routed, since unused supply lies at height $H \subseteq S$. A routed path starting in T can fail to remain inside T only by crossing from T to S, and such paths carry total flow at most U. Deleting them leaves at least

$$(\sigma(T) - U)_+$$

units routed entirely inside $G[T]$.

Now suppose $C = T$. Unused absorption lies at height zero $\subseteq T$, so every absorption vertex in S is saturated. Paths terminate in S with total value $\tau(S)$, of which at most $U = f(T, S)$ entered S from T. Hence at least $(\tau(S) - U)_+$ units both start and end in S. Keeping these paths discards at most

$$\big(\sigma(S) - (\tau(S) - U)_+\big)_+ \le (\sigma(S) - \tau(S) + U)_+$$

units of source supply from S. Since $\sigma(V) \le \tau(V)$,

$$\sigma(S) - \tau(S) \le \tau(T) - \sigma(T) \le \tau(T),$$

and therefore

$$(\sigma(S) - \tau(S) + U)_+ \le \tau(T) + U \le b(T) + \varepsilon b(T) = (1 + \varepsilon)b(T).$$

This proves (208). □

Corollary 463 (Static and incremental routing)**.** *After q supply increases and no deletions, Push maintains a routing of value F satisfying (203) and (204), with residual distance at least H from unused supply to unused absorption, in total time*

$$O(n + m + q + (q + mH)\lambda_H)\,.$$

Deletion amortization. Represent negative balance by movable deficit mass $\nu(v) \ge (-x(v))_+$, initially zero. Deleting S creates $f(S, v)$ units at each survivor v. When Pull sends δ units from donor r to deficit v, transfer δ units of deficit mass from v to r. When a vertex is deleted, freeze its current mass. Let $N(v)$ denote this frozen mass, or its final mass if it survives. Then

$$\sum_v N(v) = \sum_{\text{deletions } S} f(S, V \setminus S) \le \sum_{e \in E_0} c_e. \tag{209}$$

Assume $\tau(v) > 0$ for every originally nonisolated vertex. For each vertex define

$$\Phi_v = \nu(v) + \min\{x(v), \tau(v)/2\}.$$

The potential is nondecreasing. Push decreases x only while $x \ge \tau$. When deletion or Pull decreases x below this regime, the same amount of deficit mass is added to or transferred onto v. When Pull repairs a deficit at v, it increases $x(v)$ and removes the same amount of deficit mass, so $\nu(v) + x(v)$ is unchanged while $x(v) \le 0$. All other increases in x leave ν unchanged. At the end of the lifetime of v,

$$0 \le \Phi_v \le N(v) + \tau(v)/2.$$

A raising phase requires $x \ge \tau$, whereas a lowering phase requires $x \le 0$. Suppose first that a raising phase is followed by a lowering phase. Push cannot decrease x below τ, so while x falls from τ to $\tau/2$ every unit decrease is caused by deletion or Pull and is accompanied by an equal increase in ν. Throughout this interval $\min\{x, \tau/2\} = \tau/2$, and therefore

$$\Delta\Phi_v \ge \tau(v)/2.$$

Conversely, after a lowering phase, Pull can repair a deficit only up to $x = 0$. Before another raising phase begins, x must subsequently rise from 0 to $\tau/2$ through operations that do not decrease ν, so again

$$\Delta\Phi_v \ge \tau(v)/2.$$

It follows that v changes between raising and lowering at most

$$O\left(1+\frac{N(v)}{\tau(v)}\right) \tag{210}$$

times.

During each such phase, $\ell(v)$ is monotone and changes at most H times. Current-arc pointers therefore scan the incidences of v at most

$$O\left(H\deg_0(v)\left(1+\frac{N(v)}{\tau(v)}\right)\right)$$

times.

Each scan creates at most one forest link, and every cut caused by a relabel is charged to a preceding link. A path augmentation either saturates a forest edge, exhausts its active endpoint, or fills or empties its root. A saturated residual orientation cannot be used again before an endpoint relabel: to create residual capacity in that orientation, flow must first be sent through the reverse orientation, whose admissibility has the opposite height difference.

An exhausted surplus or deficit becomes active again only after a supply increase or an incident deletion. Consider two root fills $x=\tau$ at the same vertex that are not already separated by such an activation or by a charged edge saturation. Between them the balance must first decrease below τ; the Pull operation responsible for this decrease either exhausts its deficit origin, saturates an edge, or continues until the root reaches $x=0$. The first two cases are already charged, while the last changes the vertex from the Push to the Pull regime. The argument for two root emptyings is symmetric. Thus uncharged repeated root events are bounded by the phase changes in (210).

Consequently, apart from $O(n+m+q)$ initialization and scalar-update work, the number of forest path operations is

$$O\left(q+H\sum_{v\in V_0}\deg_0(v)\left(1+\frac{N(v)}{\tau(v)}\right)\right).$$

The q term accounts for newly activated endpoints; unlike the scalar supply update itself, the resulting augmentation may require a forest path operation. Each such operation costs $O(\lambda_H)$. If an arc belongs to both forests, then its tail has positive balance and its head negative balance, so it is root-adjacent on every represented path containing it; its two copies can therefore be synchronized before a path update.

Theorem 464 (Supply increases and vertex deletions)**.** *Assume $\tau(v)>0$ for every originally nonisolated vertex. After q supply increases or vertex-set deletions, Push–Pull maintains (200) and (199) in total time*

$$O\left(n+m+q+\left[q+H\sum_{v\in V_0}\deg_0(v)\left(1+\frac{N(v)}{\tau(v)}\right)\right]\lambda_H\right) \tag{211}$$

and $O(m+n)$ space.

If

$$\Delta=\sum_v N(v),\qquad \rho=\max_{\deg_0(v)>0}\frac{\deg_0(v)}{\tau(v)},$$

then

$$\sum_v\deg_0(v)\left(1+\frac{N(v)}{\tau(v)}\right)=2m+\sum_v\deg_0(v)\frac{N(v)}{\tau(v)}\le 2m+\rho\Delta,$$

while (209) gives

$$\Delta\le\sum_e c_e.$$

Hence a convenient coarser bound is

$$O(n+m+q+(q+H(m+\rho\Delta))\lambda_H).$$

Certificates after deletions. Let $z(v)$ be $x(v)$ clipped to $[0, \tau(v)]$. Then

$$x = z + e^+ - e^-, \qquad \operatorname{div} f = \sigma + e^- - z - e^+.$$

Thus f may be viewed as a flow whose sources are the genuine supply σ together with fictitious supply e^-, and whose sinks are the genuine absorption $z \le \tau$ together with extra sinks e^+. Decompose it into source–sink paths and cycles. Discard all paths starting from fictitious supply, and then all remaining paths ending at an extra sink. The latter paths remove at most $e^+(V)$ units of genuine source flow. The remaining paths therefore form a genuine routing of value at least

$$\max\{0, \sigma(V) - e^+(V)\}. \tag{212}$$

The sweep (206) remains valid on the surviving graph. If the height-H and height-zero sets both have positive b-mass, it yields a nontrivial sparse level cut. For a high-side cut S, at least

$$(\sigma(T) - f(T, S))_+$$

supply remains genuinely routable inside $G[T]$, and $f(T, S) \le r_f(S, T)$. The corresponding low-side statement uses the static fact that absorption in S is full.

If $1 \le c_e \le W$, then $D/d_{\min} \le 2mW$, so

$$H = O\left(\frac{\kappa \log(mW + 2)}{\varepsilon\phi}\right)$$

suffices in Theorem 462. Arbitrary real capacities use $D/d_{\min}$ directly.

All bounds count arithmetic and comparison operations; integral capacities and terminal data preserve integral flows.

12.8.3 Flow Rounding

Flow rounding converts a fractional circulation into an integral one without increasing a linear cost. We allow signed edge values, so the same procedure will also round matrices with negative entries.

Theorem 465 (Flow rounding)**.** *Let $f \in \mathbb{R}^E$ be a circulation on a directed graph, and let $a \in \mathbb{R}^E$ be linear costs. An integral circulation x satisfying*

$$\lfloor f_e \rfloor \le x_e \le \lceil f_e \rceil \quad \text{for every } e, \qquad a^{\mathsf{T}} x \le a^{\mathsf{T}} f$$

can be found in $O(n + m \log(2n))$ arithmetic operations.

In particular, any integral lower and upper capacity bounds satisfied by f remain satisfied. After isolated vertices are omitted, the work is $O(m \log(2n))$.

Availability and Cycle Cancelling Fix $L_e = \lfloor f_e \rfloor$ and $U_e = \lceil f_e \rceil$. At a current flow x, the forward and reverse availabilities are

$$r_e^+(x) = U_e - x_e, \qquad r_e^-(x) = x_e - L_e.$$

Traversing e forward has cost a_e, while traversing it backward has cost $-a_e$. Thus every cycle in the undirected fractional support can be oriented to have nonpositive cost. Push around it by the minimum availability in that orientation. This preserves conservation and the fixed bounds, does not increase cost, and makes at least one edge integral. Once an edge becomes integral, it is fixed.

Link-Cut Trees Process the initially fractional edges once, maintaining a forest containing exactly the processed edges that remain fractional. A Link-Cut Tree supports the signed cost sum, minimum directional availability, and flow addition along an oriented path in $O(\log(2n))$ amortized time. Paths are represented implicitly. Store both directional availabilities: reversing a path swaps them and negates its cost, while a push subtracts its amount from the availability in the traversal direction and adds it to the opposite availability.

After a push, FindZero(P) returns a tree edge of P whose availability in the traversal direction is zero, or null if none exists. Collect all such edges before cutting any, since the first cut would destroy the original path. Masking an extracted edge excludes it from further availability queries without changing its flow. Before cutting it, record its current flow value, including any lazy updates.

An empty path has cost zero and minimum availability $+\infty$, and has no edge to return. This convention also handles self-loops.

Algorithm 63 Rounding Fractional Circulations

Input: Graph $G = (V, E)$, circulation f, linear costs a
Output: Integral circulation x with $\lfloor f_e \rfloor \le x_e \le \lceil f_e \rceil$ and $a^\mathsf{T} x \le a^\mathsf{T} f$

```
function RoundFlow(G, f, a)
    x ← f
    Fix L_e ← ⌊f_e⌋ and U_e ← ⌈f_e⌉ for every edge
    Initialize an empty Link-Cut Tree forest
    for each edge e = (u, v) ∈ E with f_e ∉ ℤ do
        if FindRoot(u) ≠ FindRoot(v) then
            Link(e)
        else
            P ← the oriented tree path from v to u
            γ ← a_e + PathCost(P)
            Choose σ ∈ {+1, −1} with σγ ≤ 0
            Q ← P if σ = +1, and the reverse of P otherwise
            δ ← min{r_e^σ(x), MinAvailable(Q)}
            PushPath(Q, δ)
            x_e ← x_e + σδ
            S ← empty list
            while (h ← FindZero(Q)) ≠ null do
                Append h to S
                Mask(h)
            for each h ∈ S do
                Cut(h)
            if x_e ∉ ℤ then
                Link(e)
    Materialize the current edge values
    return x
```

Correctness

Proof. Every update is a circulation and stays within the fixed coordinate bounds. Its cost change is $\delta\sigma\gamma \le 0$.

After each iteration, the forest contains exactly the processed edges that remain fractional. Indeed, all newly integral tree edges are removed. If the new edge remains fractional, its availability was not exhausted, so some edge of the old tree path became integral and was removed. Its endpoints are therefore disconnected, and linking it preserves the forest.

At termination, all fractional edges belong to this forest. If any fractional edge remained, its forest component would have a leaf with exactly one fractional incident edge. All other incident values are integral, so conservation would force that edge's value to be integral as well, a contradiction.

Each edge is processed once, linked at most once, and cut at most once. Each successful zero-availability query is charged to an edge that is then cut, and there is at most one unsuccessful query per cycle move. Thus there are $O(m)$ dynamic-tree operations, in addition to $O(n+m)$ initialization. □

Lemma 466 (Split flow rounding)**.** *Let f be an integral circulation on a digraph D and d a positive integer. Then f can be written as a sum $g_1 + \cdots + g_d$ of integral circulations such that, for every j and every edge e,*

$$\left\lfloor \frac{f_e}{d} \right\rfloor \le (g_j)_e \le \left\lceil \frac{f_e}{d} \right\rceil.$$

Proof. Induct on d, with $d = 1$ immediate. For $d > 1$, round f/d with zero costs to an integral circulation g_1, then apply induction to the integral circulation $f - g_1$ with $d - 1$ parts.

Fix an edge and write $f_e = dq + r$, where $0 \le r < d$. Then $(g_1)_e = q + \varepsilon$ for $\varepsilon \in \{0, 1\}$, with $\varepsilon = 0$ if $r = 0$. Hence

$$\frac{f_e - (g_1)_e}{d-1} = q + \frac{r - \varepsilon}{d-1}.$$

If $r = 0$, this equals q. Otherwise $0 \le r - \varepsilon \le d - 1$, so it lies in $[q, q+1]$. Consequently, rounding this quantity either up or down stays between $\lfloor f_e/d \rfloor$ and $\lceil f_e/d \rceil$, as required. □

Corollary 467 (Split matrix rounding)**.** *Let $A \in \mathbb{Z}^{m \times n}$.*

1. *For every $\alpha \in \mathbb{R}$, there is an integral matrix B such that every entry, row sum, column sum, and the total sum of B is the corresponding quantity of αA, rounded either up or down.*
2. *For every positive integer d,*

   $$A = B_1 + \cdots + B_d$$

   for integral matrices $B_1, \ldots, B_d$, where for each B_k every entry, row sum, column sum, and total sum is the corresponding quantity of A/d, rounded either up or down.

Proof. Represent $A = (a_{ij})$ by a circulation on the digraph with vertices

$$s, \quad r_1, \ldots, r_m, \quad c_1, \ldots, c_n, \quad t$$

and edges

$$s \to r_i, \qquad r_i \to c_j, \qquad c_j \to t, \qquad t \to s.$$

Give these edges respectively the values

$$\sum_j a_{ij}, \qquad a_{ij}, \qquad \sum_i a_{ij}, \qquad \sum_{i,j} a_{ij}.$$

Flow conservation holds at every vertex, so this is an integral circulation f. In any circulation on this graph, the outer edge values are the row sums, column sums, and total sum of the matrix carried by the edges $r_i \to c_j$.

For the first claim, round αf with zero costs to an integral circulation, with every edge rounded up or down, and read off the values on the edges $r_i \to c_j$.

For the second, apply the preceding lemma to write

$$f = g_1 + \cdots + g_d, \qquad \left\lfloor \frac{f_e}{d} \right\rfloor \le (g_k)_e \le \left\lceil \frac{f_e}{d} \right\rceil.$$

Let $(B_k)_{ij} = g_k(r_i, c_j)$. The decomposition of the middle edge values gives $A = B_1 + \cdots + B_d$. Since each g_k is a circulation, its other edge values are exactly the row sums, column sums, and total sum of B_k, so all the required rounding bounds follow. □

12.8.4 Almost-Linear Maximum Flow

Adding a return arc lets us increase the flow value by circulation updates. A smooth penalty discourages capacity violations, while a growing price rewards flow value. For a fixed price, the minimum-ratio-cycle oracle then has exactly the required optimization role: a high-gain circulation is a descent direction, while a low optimum certifies approximate stationarity. The only algorithmic difficulty is that the gradient and metric depend on the current flow, whereas the oracle maintains compact recursive descriptions. We therefore update those descriptions lazily, prove that the stored labels remain accurate enough for descent and stopping, then raise the price in phases and finally use cost-controlled integral rounding. These three layers are independent: optimization at fixed labels, lazy maintenance of those labels, and the price/rounding wrapper.

Consider a directed graph with distinct terminals s, t and integral capacities $0 \le c_e \le U$. Discard zero-capacity arcs and isolated nonterminals, and handle the empty graph directly. Let W be the least power of two at least the total capacity. Add an unconstrained return arc $e_\star = t \to s$ and henceforth count it in m. Then $n = O(m)$ and every original capacity lies in $[1, W]$. Use the underlying undirected multigraph in the oracle, with the arc directions as reference orientations and $N = \Theta(m)$.

An augmented circulation restricted to the original arcs is a signed s–t flow of value $Z = f_{e_\star}$, and conversely every such flow becomes a circulation after adding its value on the return arc. Fix an optimal augmented circulation f^* and a minimum original cut of common integral value F; these are used only in the proof.

Penalty and lengths. We allow arbitrary signed circulations during descent and penalize capacity violations on the original arcs. Write $x_+ = \max\{x, 0\}$ and let

$$V(f) = \sum_{e \ne e_\star} v_e(f_e), \qquad v_e(x) = (-x)_+ + (x - c_e)_+.$$

The return coordinate is unconstrained. Conservation across the fixed minimum cut S_* gives the useful bound

$$Z - F = \sum_{e \in \delta^+(S_*)} (f_e - c_e) - \sum_{e \in \delta^-(S_*)} f_e \le V(f), \tag{213}$$

where the cut sums contain only original arcs. It holds for every augmented circulation. Thus the error

$$E(f) = F - Z + 2V(f) \ge V(f) \ge 0 \tag{214}$$

vanishes exactly at a feasible optimum. For an integral circulation it is an integer, so our goal is to make it less than one and preserve that bound during rounding.

We choose a penalty whose derivatives are exactly dyadic at dyadic coordinates. Cap the reciprocal below a small threshold, and linearly interpolate it between successive powers of two. Write $\lfloor a \rfloor_2$ for the largest power of two at most $a > 0$, and set

$$d = 2^{-\lceil \log_2(64m) \rceil}, \qquad A(s) = \begin{cases} 1/d, & s < d, \\ (3r - s)/(2r^2), & s \ge d, \quad r = \lfloor s \rfloor_2. \end{cases}$$

Thus $1/(128m) < d \le 1/(64m)$. The interpolant is positive, continuous, and nonincreasing. Convexity of the reciprocal puts its chord above the curve; on $[r, 2r]$ we also have $A(s) \le 1/r \le 2/s$. Hence, for $u = \max\{s, d\}$,

$$\frac{1}{u} \le A(s) \le \min\left\{\frac{2}{u}, \frac{1}{d}\right\}, \qquad |A'(s)| \le \frac{2}{u^2}$$

where the derivative exists. Define $\varphi(s) = \int_s^W A(t)\,dt$. This is a convex, continuously differentiable, piecewise quadratic penalty with $\varphi' = -A$. Put $b_c(x) = \varphi(x) + \varphi(c - x)$ and $B(f) = \sum_{e \ne e_\star} b_{c_e}(f_e)$. On $[0, c]$, both

terms are nonnegative. For $x \geq c \geq 1$, $b'_c(x) = d^{-1} - A(x) \geq d^{-1} - 2 \geq (2d)^{-1}$; symmetry gives the same outward slope below zero. Consequently

$$b_{c_e}(f_e) \geq \frac{v_e(f_e)}{2d}, \qquad B(f) \geq \frac{V(f)}{2d}.$$

For the prices $0 < \mu < 8m$ used below, the analysis potential satisfies

$$\Psi_\mu(f) = B(f) + \mu(F - Z) \geq B(f) - \mu V(f) \geq 0,$$

since $\mu < (2d)^{-1}$. The unknown F is only an additive constant; the algorithm uses, on original arcs,

$$g_e(f) = A(c_e - f_e) - A(f_e), \qquad \ell_e(f) = A(f_e) + A(c_e - f_e), \tag{215}$$

and gives the return arc the fixed labels $g_{e_\star} = -\mu$ and $\ell_{e_\star} = 1/W$. All lengths are positive. Constant stationarity accuracy controls both the remaining value gap and the violations.

Lemma 468 (Stationarity bounds the rounding penalty)**.** *For $0 < \mu < 8m$, if $g(f)^\mathsf{T} x \geq -\frac{1}{2}\|x\|_{\ell(f)}$ for every circulation x, then*

$$\mu E(f) \leq 3m + 1. \tag{216}$$

Proof. Compare f with the fixed feasible optimum by taking $x = f^* - f$. Then x is a circulation and $x_{e_\star} = F - Z$. We first bound one original coordinate. Suppose $x_e \geq 0$. Since $0 \leq f_e^* \leq c_e$, $0 \leq x_e \leq c_e - f_e$, and therefore $A(c_e - f_e)x_e \leq 2$. If $f_e < 0$, moving toward f_e^* also repairs at least the lower violation, so $A(f_e)x_e = x_e/d \geq v_e(f_e)/d$; if $f_e \geq 0$, that lower violation is zero. Using $b'_{c_e} = A(c_e - f_e) - A(f_e)$ gives

$$b'_{c_e}(f_e)x_e + \tfrac{1}{2}\ell_e(f)|x_e| = \tfrac{3}{2}A(c_e - f_e)x_e - \tfrac{1}{2}A(f_e)x_e \leq 3 - \frac{v_e(f_e)}{2d}.$$

For $x_e \leq 0$ the symmetric argument exchanges the two slacks and charges the upper violation. Thus every original coordinate contributes at most three, while collectively paying $V(f)/(2d)$ toward the current infeasibility.

It remains to handle the return coordinate. Across the fixed minimum cut, $Z \leq F + V(f)$ by (213). For the other direction, every outgoing arc is at least minus its lower violation and every incoming arc is at most its capacity plus its upper violation, so $Z \geq -W - V(f)$. Since $F \leq W$, we have $|F - Z| \leq 2W + V(f)$. The return edge therefore contributes at most $-\mu(F - Z) + |F - Z|/(2W)$. Summing the stationarity inequality over all coordinates and using $\mu < 8m$ and $(2d)^{-1} \geq 32m$ yields

$$\begin{aligned}
0 &\leq g(f)^\mathsf{T} x + \tfrac{1}{2}\|x\|_{\ell(f)} \\
&\leq 3m - \frac{V(f)}{2d} - \mu(F - Z) + \frac{|F - Z|}{2W} \\
&\leq 3m + 1 - \mu E(f) - \left(\frac{1}{2d} - 2\mu - \frac{1}{2W}\right) V(f) \leq 3m + 1 - \mu E(f).
\end{aligned}$$

□

The same estimate will later pay for price increases and final rounding. It remains to maintain cycle-oracle labels close enough to the current flow without expanding every implicit update.

Implicit updates and lazy labels. The cycle oracle returns compact descriptions, so updates are applied to their stored tree paths and chords without expanding to root edges. Pending values move toward level 0 only when a field crosses its threshold.

For every original arc, let $\bar{f}_e$ be the coordinate value at its last label refresh. The oracle uses the exact values $\bar{g}_e = g_e(\bar{f}_e)$ and $\bar{\ell}_e = \ell_e(\bar{f}_e)$ until the next refresh. For the return arc the labels are flow-independent; its committed scalar supplies the return component of $\bar{f}$. Set

$$\Lambda = C\Gamma_0, \qquad q = \frac{1}{8\Lambda}, \qquad \theta = \frac{q}{16}.$$

For every original arc, one of $\bar{f}_e, c_e - \bar{f}_e$ is at most $c_e/2 \le W/2$, so $1/W \le \bar{\ell}_e \le 2/d$; the return length is exactly $1/W$.

View the physical hierarchy as levels $0, 1, \ldots, L$. A level-0 field belongs to an original arc; a positive-level field belongs to a tree-edge copy in an initial physical routing forest. The return arc has no field: anything reaching it is committed immediately to the scalar $\bar{f}_{e_\star}$. Every other field a stores a signed pending coefficient x_a, initially zero, on its immutable root expansion P_a, and $\ell_a = \bar{\ell}(P_a)$. Positive-level fields are oriented down their rooted forests and know the input handle at the preceding level; direct copies reuse that handle. Search records have no fields. Thus

$$f = \bar{f} + \sum_a x_a[P_a]. \tag{217}$$

Flushing a positive-level field applies its coefficient to that preceding handle and resets it to zero, so forwarding always decreases the level.

Let $\kappa = \log^{O(1)} N$ be the distance construction's per-level physical-overlap bound from Theorem 456, and let Q be the least power of two at least $2\kappa^{L+1}$. Hence every original arc has at most Q ancestor-field occurrences over all levels. For a field let $\widehat{\ell}_a$ be the least power of two at least ℓ_a and set

$$\eta_a = \frac{\theta}{Q\widehat{\ell}_a}, \qquad \frac{\theta}{2Q} < \ell_a \eta_a \le \frac{\theta}{Q}. \tag{218}$$

Maintain $|x_a| < \eta_a$ for every field. Each ancestor occurrence then contributes less than θ/Q to an original arc's weighted error, and there are fewer than Q such occurrences, so $\bar{\ell}_e |f_e - \bar{f}_e| < \theta$. The return labels are flow-independent.

Link–cut path updates. Only the flow application needs dynamic path aggregates. Reuse the link–cut method from Flow Rounding on every physical routing tree, with an edge node for each copied tree edge; keep the original rooting separately. An edge node with field a stores the two headrooms $\eta_a - x_a$ and $\eta_a + x_a$, and each auxiliary tree stores their minima with the usual lazy additive tag. Vertex nodes are neutral.

To add δ along an oriented tree path $u \to v$, let z be its LCA in the fixed original rooting. Apply $-\delta$ to the link–cut path from u to z and $+\delta$ from z to v; thus every edge field receives the signed coefficient of its fixed downward orientation. After either path addition, while a headroom minimum is nonpositive, locate a violating edge node, extract and reset its field, and deposit that value through its preceding-level handle. Re-expose the affected path if necessary. Forwarding strictly decreases the oracle level, so recursive normalization terminates, and no global heap or queue is needed. A physical tree-edge deletion drains its field before cutting its edge node; a search closure does not change the physical link–cut tree.

At level 0, crossing the threshold instead appends the coordinate to a root refresh list, once. Refreshes wait until all handles of the current circulation have been deposited because they change oracle records. To refresh e, retire its old input record while keeping its field alive; recursive retirement first drains every old ancestor contribution. Then commit

$$\bar{f}_e \leftarrow \bar{f}_e + x_e, \qquad x_e \leftarrow 0,$$

recompute its labels and threshold, insert the new record, and clear the listed bit. A later drain may list it again. If cancellation made a listed field small before processing, refreshing it anyway is charged to the threshold crossing that listed it. When the list empties, the single normalization invariant holds at every level before the next query.

Retirement and work. A virtual shortcut has no field of its own. Before it retires, its recursive users drain into the same earlier forest path. Inserting a mark may replace constantly many virtual shortcuts, but this only changes their descriptors; the underlying copy fields stay on the fixed routing-tree edges. Before physically deleting a tree-edge copy, drain its recursive users, extract and forward its pending value, and set its node headrooms to $+\infty$. The resulting earlier-level deposits are normalized immediately; any original field that crosses threshold is put on the root refresh list. At an epoch boundary, retire the recursive suffix first,

then drain the epoch's copy fields while their input handles remain alive. No retired field receives a later deposit, and every new copy field starts at zero.

Whenever the root hierarchy is rebuilt, retire it from deepest to shallowest, drain every copy field into an original field or the committed return scalar, and commit all original fields. Recompute the labels and thresholds and rebuild with zero pending fields. This $O(m)$ materialization is absorbed by the rebuild. We use the same operation at price changes below, so there is only one global reset rule.

Charge a deposit δ its absolute weighted amount $|\delta|\ell_a$, summed over the affected fields. An application contributes at most $|\tau|D$. Forwarding preserves this amount before cancellation, since an input handle's occurrence length equals the copy field's ℓ_a. It goes strictly to earlier levels, so for total external amount $S = \sum_t |\tau_t| D_t$, the total absolute inflow at each level is at most S, including forced drains.

Between resets, a field starts below threshold and can trigger again only after receiving fresh absolute weighted inflow at least $\ell_a \eta_a > \theta/(2Q)$. Charge the trigger to that inflow interval at the instant of crossing, before later cancellation can erase it. For one field these charged intervals are disjoint, and forwarding only moves the same weighted amount to an earlier level. Since the total external amount is at most S per level, summing over the $L+1$ levels gives the bound below. Forced retirement flushes are not threshold events and are charged directly to retirement.

Lemma 469 (Lazy flow maintenance)**.** *For updates of total certified amount $S = \sum_t |\tau_t| D_t$, cleanup keeps $\bar{\ell}_e |f_e - \bar{f}_e| < \theta$ on every original arc; the return labels are flow-independent. It initiates at most*

$$O\Big(\frac{Q(L+1)S}{\theta}\Big)$$

threshold flushes and coordinate refreshes. Each application handle and copy-field flush takes $\log^{O(1)} N$ local work. Forced flushes are charged to record retirement; original-field refreshes additionally incur the distance oracle's maintenance cost.

Initializing or retiring a copy-field node takes logarithmic work, within the existing forest-work bound. Root-list operations are constant time and are charged to the threshold crossing that first lists the coordinate.

Thus every implicit update is normalized without expanding its root walk. We can now run the fixed-price optimization using $\bar{g}, \bar{\ell}$. Query the cycle oracle with these stored labels. A positive answer (z, D) has exact stored-label numerator $\bar{g}^{\mathsf{T}} z$ and $D \geq \|z\|_{\bar{\ell}}$. If its certified gain exceeds q, choose the sign so that $\bar{g}^{\mathsf{T}} z < -qD$. Let $\widehat{D}$ be the least power of two at least D and take

$$\tau = \theta/\widehat{D}, \qquad f \leftarrow f + \tau z, \qquad \theta/2 < \tau D \leq \theta. \tag{219}$$

Deposit all of the circulation's handles before processing the root refresh list or issuing another query. The certificate gives $\bar{\ell}_e \tau |z_e| \leq \theta$ at every coordinate. If the returned gain is at most q, the oracle factor implies that every circulation has stored-label gain at most $\Lambda q = 1/8$; the next paragraph transfers this certificate to the true labels.

Label stability, descent, and stopping. The lazy structure guarantees a coordinatewise weighted error bound; this is now the only fact about the data structure used by the optimization proof. Write $f^+ = f + \tau z$. For every original arc and either endpoint $y \in \{f, f^+\}$, the lazy-update invariant and step certificate give

$$\bar{\ell}_e |y_e - \bar{f}_e| \leq \bar{\ell}_e \big(|f_e - \bar{f}_e| + \tau |z_e|\big) \leq 2\theta.$$

For the return arc, both labels are independent of its flow value while μ is fixed.

Fix either stored slack $s \in \{\bar{f}_e, c_e - \bar{f}_e\}$, let s' be its value at y, and put $u = \max\{s, d\}$. The stored length contains $A(s) \geq 1/u$, so the weighted error above implies $|s' - s| \leq 2\theta u$: each slack changes by only a small multiplicative fraction of its clipped size. Every clipped slack between s and s' is therefore at least $(1 - 2\theta)u$. On that interval the slope bound for A is at most $2/((1-2\theta)u)^2$, and hence

$$|A(s') - A(s)| \leq \frac{4\theta}{(1-2\theta)^2 u} \leq 8\theta A(s).$$

Continuity makes the same estimate valid when the interval crosses d or a dyadic breakpoint. Applying it to the two directional slacks gives

$$|g_e(y) - \bar{g}_e| \le 8\theta\bar{\ell}_e = (q/2)\bar{\ell}_e, \qquad \bar{\ell}_e \le 2\ell_e(y). \tag{220}$$

The second inequality follows from $\ell_e(y) \ge (1 - 8\theta)\bar{\ell}_e$; here $\theta \le 1/128$. For the return arc the gradient error is zero and the two lengths are equal, so (220) holds there as well. At a stopping query the same calculation uses only $y = f$ and the lazy invariant.

For an accepted direction, use the supporting-hyperplane inequality for the convex potential at the new point f^+; this is why the label-stability bound was proved at both endpoints. We obtain

$$\begin{aligned}\Psi_\mu(f^+) - \Psi_\mu(f) &\le \tau g(f^+)^\mathsf{T} z \\ &\le \tau\big(\bar{g}^\mathsf{T} z + (q/2)\|z\|_{\bar{\ell}}\big) \\ &\le -\tfrac{1}{2}q\tau D \\ &< -\tfrac{1}{4}q\theta = -\frac{q^2}{64}.\end{aligned}$$

If the returned gain is at most q, counting a zero-optimum certificate as gain zero, the oracle gives $|\bar{g}^\mathsf{T} x| \le \frac{1}{8}\|x\|_{\bar{\ell}}$ for every circulation x. The same estimate yields

$$\begin{aligned}|g(f)^\mathsf{T} x| &\le (\tfrac{1}{8} + q/2)\|x\|_{\bar{\ell}} \\ &\le 2(\tfrac{1}{8} + q/2)\|x\|_{\ell(f)} < \tfrac{1}{2}\|x\|_{\ell(f)}.\end{aligned}$$

Thus gain above q gives a definite potential decrease, while gain at most q certifies the stationarity required by Lemma 468. Neither decision requires evaluating the potential.

Increasing the price. Now double the price only at stationary points. The final price $4m$ makes the rounding error less than one, while starting at $\mu = 1/W$ avoids an initial $4mF$ potential term; the stationarity bound pays for each doubling.

Initialize $f = \bar{f} = 0$ and $\mu = 1/W$, compute the stored labels, and build the oracle with zero pending fields. Repeat the following, finishing normalization, root refreshes, and maintenance before every query:

1. Query the cycle oracle. If its certified gain exceeds q, take the step (219) and repeat.
2. Otherwise the stopping test holds. If $\mu < 4m$, double μ and repeat; if $\mu \ge 4m$, materialize the flow and round it.

Only the return gradient depends directly on μ. To double the price, perform the global rebuild above: materialize the current flow, double μ, and rebuild the distance/cycle oracle on the same root circulation with zero pending fields. The flow itself does not change. There are only $O(\log(mU))$ such rebuilds, so their near-linear initialization cost is absorbed in the final bound. The last price lies in $[4m, 8m)$, and every price is a power of two.

At a doubling from $\mu < 4m$, stationarity gives

$$\Psi_{2\mu}(f) - \Psi_\mu(f) = \mu(F - Z) \le \mu E(f) \le 3m + 1.$$

Initially $F \le W$ gives $\Psi_{1/W}(0) \le B(0) + 1$, with $B(0) \le 2m\varphi(0)$. Since $A(s) \le 2/\max\{s, d\}$,

$$\varphi(0) \le 1 + 2\log(W/d) = O(\log(mU)).$$

There are $O(\log(mU))$ doublings because $W \le 2(m-1)U$, and each costs at most $3m + 1$ in potential. Thus the initial potential plus all price increases is $O(m\log(mU))$. The potential is nonnegative and each accepted step decreases it by at least $q^2/64$, so

$$T = O\big(m\Lambda^2 \log(mU)\big). \tag{221}$$

This iteration bound also pays for label maintenance. Each step adds at most θ to $S = \sum_t |\tau_t| D_t$, so Lemma 469 bounds threshold field flushes and coordinate refreshes by $O(Q(L+1)T)$. Price changes contribute $O(\log(mU))$ full rebuilds. Final materialization uses the same deepest-to-shallowest drain and then commits the root coordinates without rebuilding.

Rounding to an integral optimum. At termination $\mu \geq 4m$, stationarity gives $E(f) \leq (3m+1)/(4m) < 1$ because the nonempty augmented graph has $m \geq 2$. The error in (214) is a nonnegative integer at integral circulations. It therefore suffices to round without increasing the known cost $\mathcal{C}(f) = 2V(f) - Z = E(f) - F$.

Fully materialize f and retire the oracle fields. Fix the coordinate box $\prod_e [\lfloor f_e \rfloor, \lceil f_e \rceil]$. For every original arc the breakpoints $0, c_e$ are integers, so no unit interval crosses one in its interior; the return term is already linear. Thus $\mathcal{C}(x) = a^{\mathsf{T}} x + b$ on this box, with $|a_e| \leq 3$ on nondegenerate coordinates. The Flow Rounding theorem with linear costs a returns an integral circulation f' in $O(m \log n)$ time with $\mathcal{C}(f') \leq \mathcal{C}(f)$. Now $E(f')$ is an integer in $[0, 1)$, hence

$$0 \leq V(f') \leq E(f') = 0, \qquad F - f'_{e_\star} = E(f') - 2V(f') = 0.$$

Thus f' is feasible and optimal. Remove the return arc; residual reachability from s supplies a minimum cut in the original graph.

During rounding, each directional slack is a distance to an integer boundary, so taking its minimum and adding the move preserve the dyadic grid. Rounding uses no gradient or length labels.

Work and numerical precision. Finally account for the work and exact arithmetic. Since $A(s) \leq 1/d = O(m)$ everywhere and $\mu < 8m$, stored lengths lie in $[1/W, O(m)]$ and gradient magnitudes are $O(m)$, independently of the iterates and oracle parameters. Choose L about $\frac{1}{2} \log_2 \log N$ as in Theorem 456 and Theorem 458, so $C, \Gamma_0, Q, \Lambda = N^{o(1)}$.

There are $O(Q(L+1)T)$ threshold refreshes and $O(\log(mU))$ price rebuilds. Theorem 456 bounds oracle maintenance, including those rebuilds, by

$$\big(m \log(mU) + Q(L+1)T\big) k^2 (p\Gamma_0)^{O(L+1)}.$$

The static algorithm needs only this total-maintenance guarantee. Each circulation application uses $N^{o(1)}$ link–cut path additions; triggered flushes take polylogarithmic work each, and forced flushes are charged to retiring fields. All factors besides m and T are $N^{o(1)}$. By (221), this is $m^{1+o(1)}$ arithmetic operations for polynomial capacities. The flow data add only the link–cut trees and one listed bit per original coordinate, so active records and cover workspaces still occupy $m^{1+o(1)}$ space; rounding uses one additional forest.

For bit complexity, use the common budget

$$\mathcal{B}_* = \log(mU) + (L+1) \log N + \log Q + \log(1/\theta) = \log^{O(1)} m.$$

Every physical description contains at most $K_*^{O(L+1)}$ root occurrences, so every nonzero field length and certificate lies between $1/W$ and $m^{O(1)} K_*^{O(L+1)}$. Hence $\widehat{D}$ and the dyadic step $\tau = \theta/\widehat{D}$ have $O(\mathcal{B}_*)$-bit exponents, and all iterates, references, and pending coefficients remain on one dyadic grid.

Magnitudes are equally bounded. Forwarding and commits do not increase $\sum_a |x_a| \ell_a \leq S \leq T\theta$; since $\ell_a \geq 1/W$, pending coefficients are at most WS, and the same bound holds for original coordinates, the return scalar, old references, and lazy tags. The iteration bound puts their integer parts in $O(\mathcal{B}_*)$ bits.

Finally, for $s \geq d$ the formula $A(s) = (3r - s)/(2r^2)$ uses only the dyadic scale $r = \lfloor s \rfloor_2$, so refreshed labels are $O(\mathcal{B}_*)$-bit dyadics. The same is true of field lengths, $Q, \theta, \widehat{\ell}_a, \eta_a$, headrooms, and the forest-cover sums; artificial search labels use only $O((L+1) \log N)$ bits and MWU weights stay polynomial. Candidate gains are compared by cross multiplication. Thus exact arithmetic costs only a polylogarithmic factor, with no numerical precision parameter or label-rounding error.

Theorem 470 (Almost-linear maximum flow)**.** *A directed graph with m arcs, $n = O(m)$ vertices, and nonnegative integral capacities $U \leq m^{O(1)}$ admits a deterministic algorithm computing an integral maximum flow and a minimum cut in $m^{1+o(1)}$ bit operations using $m^{1+o(1)}$ bits of space.*

12.8.5 Cut-Matching Game and Sparse Cuts

For an undirected graph with capacities c_e, define its sparsity by $\psi(G) = \min_{\emptyset \neq S \neq V} c(\partial S)/\min\{|S|, |V \setminus S|\}$. Cut-matching certifies expansion by constructing a multicommodity flow that sends a small amount from every vertex to every other vertex. A sparse cut would have too little capacity to carry all the flow between its two sides, so such a routing with low congestion gives a lower bound on $\psi(G)$.

Initially, each vertex v holds one unit of its own commodity. In each round, we route a matching and let its endpoints exchange half of everything they hold. Both endpoints then hold the average of their previous mixtures. For ordinary matchings, after one round v's commodity is shared equally by two vertices. If each holder next meets a vertex with none of that commodity, it is shared equally by four vertices, then eight, and so on. This suggests that repeated averaging can spread every commodity throughout the graph in logarithmically many rounds. The difficulty is to keep bringing different mixtures together: exchanging identical mixtures makes no progress. We choose a bisection that forces useful averaging across it, even though the flow computation chooses the matching. The analysis below guarantees $O(\log^2 n)$ rounds with high probability. Assume $n = |V|$ is even.

Theorem 471 (Khandekar–Rao–Vazirani)**.** *Given $\psi > 0$ and $0 < \delta < 1$, a randomized algorithm using $T = O(\log n \log(n/\delta))$ max-flow computations either returns a cut of sparsity less than ψ, or, with failure probability at most δ, establishes $\psi(G) \geq \psi/(2T)$. In particular, polynomially small failure probability costs $O(\log^2 n)$ flows.*

A *fractional perfect matching* across a bisection assigns nonnegative weights to crossing pairs, with total weight one at each vertex. Write W_t for its symmetric adjacency matrix. Vertex i keeps half of each commodity it holds and sends a fraction $W_t(i,j)/2$ of each to vertex j. The corresponding averaging matrix is $A_t = (I + W_t)/2$. Ordinary perfect matchings are a special case; fractional ones allow arbitrary input capacities.

To choose a bisection, give each commodity a fresh independent Gaussian label and score each vertex by the weighted sum of the labels in its mixture. We cut between the lower and upper halves of these scores. Every crossing matching then pairs low scores with high scores; the proof shows that this forces enough averaging of different mixtures. The scores can be computed by applying the previous averaging steps to the vector of labels, so we never need to store the full multicommodity flow explicitly.

Algorithm 64 Cut-Matching Test

Input: Capacitated graph G, target $\psi > 0$, number of rounds T

for $t = 1, \ldots, T$ **do**
  Sample $h \sim N(0, I_n)$ and set $g \leftarrow h - (\sum_i h_i/n)\mathbf{1}$
  $u \leftarrow A_{t-1} \cdots A_1 g$ ▷ *Replay the previous averaging steps*
  Let S contain the $n/2$ smallest coordinates of u
  Give each original edge capacity c_e/ψ; add unit-capacity arcs $s \to v$ for $v \in S$ and $v \to z$ for $v \notin S$
  Compute a maximum s–z flow and a minimum cut
  if the flow value is less than $n/2$ **then**
    return the original vertices on the source side of the cut
  Cancel opposite flows, discard cycles, and store a decomposition into weighted paths
  Give each endpoint pair its total path weight to obtain W_t, and set $A_t \leftarrow (I + W_t)/2$
return the matchings and their path routings ▷ *They encode the multicommodity flow*

Proof. First check the flow step. If a minimum cut has original vertex set U on its source side, write $a = |S \cap U|$ and $b = |U \setminus S|$. Its capacity is $n/2 - a + b + c(\partial U)/\psi$: the first two terms count the terminal arcs crossing the cut. A value below $n/2$ therefore gives

$$\frac{c(\partial U)}{\psi} < a - b \leq \min\{|U|, n - |U|\}.$$

Here $a - b \le a + b = |U|$, and $a - b \le n - a - b = n - |U|$ because $a \le n/2$. Thus U is nontrivial and has sparsity less than ψ. Otherwise every terminal arc is saturated. After cancelling opposite flows and removing cycles, path decomposition gives a fractional perfect matching across the bisection, routed with total load at most c_e/ψ on each original edge e.

Now interpret these matchings as exchanges of commodities. Let $P_t(i, v)$ be the amount of commodity v held at vertex i after t rounds. Initially $P_0 = I$, and the exchange rule gives

$$P_t(i, v) = \frac{1}{2} P_{t-1}(i, v) + \frac{1}{2} \sum_j W_t(i, j) P_{t-1}(j, v), \qquad P_t = A_t \cdots A_1.$$

The columns of P_t sum to one because each commodity has total amount one. Its rows also sum to one: each vertex sends half a unit and receives half a unit, so it always holds one unit in total.

Suppose all the single-commodity flows succeed. A routing path of matching weight a can implement its exchange by carrying $a/2$ of each endpoint's mixture to the other endpoint. Since both mixtures have total amount one, this puts total load a on the path, counting both directions. Thus one round places load at most c_e/ψ on edge e, and all T rounds together place load at most Tc_e/ψ. Tracing each portion of a commodity through the successive exchanges gives a route from its original vertex to its final holder. These routes form a multicommodity flow with congestion at most T/ψ; any loops in a route may be deleted without increasing the load.

Our target is $P_T(i, v) \ge 1/(2n)$ for every i, v. By discarding excess flow separately for each source and destination, this gives a uniform multicommodity flow of $1/(2n)$ units per ordered pair. For any nontrivial cut U, the commodities originating in U send at least $|U|(n - |U|)/(2n)$ units outside, and those originating outside send at least the same amount into U. All this flow must cross the cut, so the congestion bound gives

$$\frac{|U|(n - |U|)}{n} \le \frac{T}{\psi} c(\partial U).$$

Since $|U|(n - |U|)/n \ge \frac{1}{2} \min\{|U|, n - |U|\}$, this directly certifies $\psi(G) \ge \psi/(2T)$.

It remains to show that the exchanges distribute every commodity sufficiently evenly, regardless of the flow choices. For this analysis, continue with arbitrary crossing matchings if a flow fails; the actual algorithm would already have returned a valid cut. The ideal final matrix is $J = \mathbf{1}\mathbf{1}^\top/n$: every vertex holds $1/n$ of every commodity. We measure the discrepancy from this ideal by $\Phi_t = \|P_t - J\|_F^2$, initially $\Phi_0 = n - 1$. It suffices to make $\Phi_T \le 1/(4n^2)$, because then $|P_T(i, v) - 1/n| \le \sqrt{\Phi_T} \le 1/(2n)$ for every i, v.

Fix the previous rounds and write $X = P_{t-1} - J$, with rows x_i. Thus x_i records how vertex i's mixture differs from the uniform mixture, and $x_i - x_j$ records the difference between two mixtures. All probabilities and expectations in the one-round analysis below are conditional on these previous rounds, so X is fixed. Since $A_t J = J$, the new centered matrix is $A_t X$. For an ordinary matching, each matched pair of rows is replaced by its midpoint, reducing their combined squared norm by $\|x_i - x_j\|^2/2$. The fractional case has the same lower bound: row i becomes the convex combination $\sum_j W_t(i, j)(x_i + x_j)/2$, so convexity of squared norm gives

$$\begin{aligned} \Phi_t &\le \sum_{i,j} W_t(i, j) \left\| \frac{x_i + x_j}{2} \right\|^2 \\ &= \Phi_{t-1} - \frac{1}{2} \sum_{\{i,j\}} W_t(i, j) \|x_i - x_j\|^2. \end{aligned}$$

The equality follows by expanding the squared norm and using that every row and column of W_t sums to one; the last sum counts each unordered pair once. Thus averaging always decreases the potential, and the decrease is large when the matching pairs sufficiently different mixtures.

The Gaussian labels give a scalar measure of these differences. Let h be the fresh standard Gaussian vector before centering. The score of the mixture at i is $\sum_v P_{t-1}(i, v) h_v$. Subtracting the common mean label centers these scores without changing their order. Since $P_{t-1} J = J$, the algorithm therefore computes

$u = P_{t-1}(I-J)h = Xh$, so $u_i = \langle x_i, h\rangle$ and $\sum_i u_i = 0$. Choose μ between the two middle coordinates of u. Every pair with positive matching weight has one endpoint on each side of μ. For $u_i \le \mu \le u_j$,

$$(u_i - u_j)^2 = \big((\mu - u_i) + (u_j - \mu)\big)^2 \ge (u_i - \mu)^2 + (u_j - \mu)^2.$$

Each vertex has total matching weight one, so summing charges its entire squared distance from μ to the matching edges. Consequently, writing $Z = \sum_i u_i^2$, we obtain

$$\sum_{\{i,j\}} W_t(i,j)(u_i - u_j)^2 \ge \sum_i (u_i - \mu)^2 = Z + n\mu^2 \ge Z.$$

This is why we cut at the median: every crossing matching must account for all the spread in the scores.

To turn separation of scores into a potential decrease, we need to relate it to differences between the actual mixtures. For a fixed pair, $u_i - u_j = \langle x_i - x_j, h\rangle$ is Gaussian with variance $\|x_i - x_j\|^2$. Its tail satisfies $\Pr((u_i - u_j)^2 > L\|x_i - x_j\|^2) \le 2e^{-L/2}$ (and the bound is immediate if $x_i = x_j$). Take $L = 2\log(12n^2)$. A union bound over fewer than $n^2/2$ pairs gives an event $\mathcal{E}$ with probability at least $11/12$ on which $(u_i - u_j)^2 \le L\|x_i - x_j\|^2$ for every pair. The bound must hold simultaneously for all pairs because the matching is chosen after seeing u. On $\mathcal{E}$, the averaging and median bounds therefore imply

$$\Phi_{t-1} - \Phi_t \ge \frac{1}{2L} \sum_{\{i,j\}} W_t(i,j)(u_i - u_j)^2 \ge \frac{Z}{2L}.$$

The total squared score equals the potential in expectation: $\mathbb{E}u_i^2 = \|x_i\|^2$, so $\mathbb{E}Z = \Phi_{t-1}$. It remains to check that restricting to $\mathcal{E}$ retains a constant fraction of this expectation. The Gaussian fourth moment gives $\mathbb{E}u_i^4 = 3\|x_i\|^4$; hence Cauchy–Schwarz, applied to each coordinate, yields

$$\mathbb{E}[Z\mathbf{1}_{\mathcal{E}^c}] = \sum_i \mathbb{E}[u_i^2 \mathbf{1}_{\mathcal{E}^c}] \le \sum_i \sqrt{\mathbb{E}u_i^4 \Pr(\mathcal{E}^c)} \le \frac{1}{2}\sum_i \|x_i\|^2 = \frac{\Phi_{t-1}}{2}.$$

Thus at least half the expected total squared score lies on $\mathcal{E}$. Since the potential never increases, even on $\mathcal{E}^c$, we conclude that

$$\mathbb{E}[\Phi_{t-1} - \Phi_t] \ge \frac{\mathbb{E}[Z\mathbf{1}_{\mathcal{E}}]}{2L} \ge \frac{\Phi_{t-1}}{4L}.$$

Each round therefore removes an $\Omega(1/\log n)$ fraction of the remaining potential in conditional expectation.

Removing the conditioning and iterating gives $\mathbb{E}\Phi_T \le (n-1)(1 - 1/(4L))^T \le (n-1)e^{-T/(4L)}$. For $T = \lceil 4L\log(4n^2(n-1)/\delta)\rceil$, this is at most $\delta/(4n^2)$. Markov's inequality now gives $\Phi_T \le 1/(4n^2)$ with probability at least $1-\delta$. On this event every vertex holds at least $1/(2n)$ of every commodity, giving the multicommodity flow certificate above whenever all T single-commodity flows succeed. □

12.9 Laplacian Solving

Eliminate a constant fraction of the vertices at a time, sample the Schur complement, and record how to recover the eliminated variables. Stop early: the remaining graph has few vertices but may still have many edges. A small *pilot* sample supplies deliberately coarse resistance estimates for compressing it. The same elimination routine, run to completion, solves the two auxiliary graphs. No low-stretch tree is needed. The resulting solver takes

$$O\Big(m \log^{7/4}(2n)\,[\log\log(4n)]^{1/4}\log(2/\varepsilon)\Big) \tag{222}$$

expected time for relative energy error ε, with high-probability correctness. A longer chain leaves fewer vertices, allowing coarser resistance estimates and therefore fewer probes.

Let G be connected with $n \ge 2$ vertices, m positive-weight edges, and Laplacian L. Discard loops; parallel edges are allowed. Fix a root o, never eliminate it, and ground all matrices at o. Write $\bar{L}$ for the grounded input Laplacian, $\Lambda = \log(2n)$, and $A \approx_\delta B$ for $e^{-\delta}B \preceq A \preceq e^{\delta}B$. Edge leverage is $\tau_e(K) = w_e b_e^\top K^{-1} b_e$, with grounded incidence vector b_e. We assume matrix Chernoff, leverage sampling, and Richardson iteration from the notes; all costs count exact arithmetic and elementary random draws. The walk primitive below is from [SZ23], and the pilot idea from [CLMMPS15]; the final tradeoff is derived here.

1. One elimination step. Choose $F \subseteq V \setminus \{o\}$ with $|F| \geq (|V|-1)/20$ and $d_F(v) \leq d(v)/5$ for $v \in F$, using weighted degrees in the full current graph. This is deterministic and linear-time: for $U \subseteq V \setminus \{o\}$, the quantity $\Phi(U) = |U| - 5\sum_{v\in U} d_U(v)/d(v)$ lower-bounds the number surviving pruning of vertices with $d_U(v) > d(v)/5$. Independent inclusion with probability $1/10$ gives $\mathbb{E}\Phi \geq (|V|-1)/20$. Fix indicators by conditional expectation, updating only neighbors' coefficients in this quadratic polynomial, then prune once.

Put $C = V \setminus F$. In grounded coordinates,

$$K = \begin{pmatrix} A & -W \\ -W^\top & D \end{pmatrix}, \qquad A = X + Y, \qquad S = D - W^\top A^{-1} W, \tag{223}$$

where Y is the internal Laplacian on F and X is the diagonal of weights from F to C, including the root. The degree test gives $Y \preceq 2\,\mathrm{diag}(d_F) \preceq X/2$; it also makes each random-walk step from F exit to C with probability at least $4/5$. Thus the block inverse is easy to approximate and the random walks are short. Completing the square gives

$$y^\top S y = \min_z \begin{pmatrix} z \\ y \end{pmatrix}^\top K \begin{pmatrix} z \\ y \end{pmatrix}. \tag{224}$$

In particular, Schur complementation preserves Loewner order. We approximate A^{-1} and sample S without forming either dense matrix.

Sample the terminal graph. Retain each of the M current multiedges $e = uv$ independently with probability $p \in (0,1]$. If retained, run independent conductance-weighted walks from u, v until they first hit C. Reverse the first walk and concatenate it with e and the second walk. If the resulting terminal walk P has distinct ends s, t, output st with weight $1/(p\,r(P))$, where $r(P) = \sum_{f\in P} 1/w_f$ counts occurrences. Only the endpoints and resistance sum are needed. Alias tables give expected $O(M)$ work; the output count is at most M, with expectation at most pM.

The independent output matrices sum to S in expectation: the Neumann expansion of A^{-1} expresses S as terminal walks with conductance $c(P) = \prod_{f\in P} w_f / \prod_{v\in \mathrm{int}(P)} d(v)$; products count occurrences. Orient each terminal pair consistently. With $p = 1$, generating P with a specified middle-edge occurrence e has probability $c(P)/w_e$. Summing its expected weight over occurrences gives

$$\sum_{\text{middle occurrences } e\in P} \frac{c(P)}{w_e}\,\frac{1}{r(P)} = c(P).$$

Retention and division by p preserve expectation. If every input edge has leverage at most a, every output edge has leverage at most a/p relative to S, since

$$R_S(s,t) = R_K(s,t) \leq \sum_{f\in P} R_K(f) \leq a\,r(P). \tag{225}$$

Here resistances are preserved by Schur complementation. Their triangle inequality follows by reducing to three vertices: conductances a, b, c on uv, vz, uz give $R(u,v) + R(v,z) - R(u,z) = 2c/(ab+ac+bc) \geq 0$.

Recover the eliminated variables. Approximate A^{-1} by an even number q of Jacobi steps:

$$z_0 = 0, \quad z_{j+1} = X^{-1}(b - Y z_j)\ (j < q), \quad Zb = z_q; \qquad 0 \preceq E := Z^{-1} - A \preceq \eta Y. \tag{226}$$

The last bound holds for $q = O(\log(2/\eta))$: with $T = X^{-1/2} Y X^{-1/2} \preceq I/2$, the polynomial $p_q(t) = \sum_{j<q} (-t)^j$ satisfies

$$0 \leq p_q(t)^{-1} - (1+t) = \frac{(1+t)t^q}{1-t^q} \leq 2^{2-q} t \qquad (0 \leq t \leq 1/2).$$

Conjugating this scalar bound by $X^{1/2}$ proves (226). Thus Z is fixed and symmetric positive definite, and needs only sparse products. For a fixed terminal inverse $Q_C \approx_\delta S^{-1}$, use

$$u = Z b_F, \qquad x_C = Q_C(b_C + W^\top u), \qquad x_F = u + ZW x_C. \tag{227}$$

The resulting map Q is the congruence of $\operatorname{diag}(Z, Q_C)$ by $R = \left(\begin{smallmatrix} I & ZW \\ 0 & I \end{smallmatrix}\right)$. To bound its error, first take $Q_C = S^{-1}$ and put $\widehat{K} = K + \operatorname{diag}(E, 0)$, $\widehat{S} = \mathrm{SC}(\widehat{K}, C)$. Since $\operatorname{diag}(E, 0) \preceq \eta \operatorname{diag}(Y, 0) \preceq \eta K$,

$$K \preceq \widehat{K} \preceq (1+\eta)K, \qquad S \preceq \widehat{S} \preceq (1+\eta)S. \tag{228}$$

The second comparison follows from (224). Block elimination gives $\widehat{K}^{-1} = R \operatorname{diag}(Z, \widehat{S}^{-1}) R^\top$; replacing $\widehat{S}^{-1}$ by S^{-1} therefore gives $\widehat{K}^{-1} \preceq Q \preceq (1+\eta)\widehat{K}^{-1}$. Combining with (228) proves $Q \approx_{\log(1+\eta)} K^{-1}$. A general terminal inverse adds at most δ by the same congruence, so $Q \approx_{\delta+\eta} K^{-1}$.

2. A short chain and its full-length baseline. Fix a horizon $J \geq 1$. Set $\eta = c/J$ for a small constant c, and split each input edge into $\rho = \lceil C_0 \Lambda J^2 \rceil$ equal-weight copies. Repeatedly select F, store X, Y, W, and sample the terminal graph with $p = 1$. When the chosen terminal count is reached, use $p = J^{-2}$ in that last round. The final round is recognized after selecting F, before sampling.

After i successful ordinary rounds, every edge has leverage at most $e^{i\eta}/\rho$ in the current graph. Initially this follows from $\tau_e \leq 1$ before splitting. Equation (225) gives the same bound relative to the next exact Schur complement. In $S^{-1/2}$ coordinates, each sample has norm at most $e^{i\eta}/\rho$, and their sum has expectation I. Chernoff therefore yields $\widehat{S} \approx_\eta S$ with conditional failure at most n^{-50}, since $\rho\eta^2 = \Theta(\Lambda)$. Inverse monotonicity multiplies leverage by at most e^η, proving the induction. These scores are used only in the proof, never computed. In the final round, the sample bound is $O(1/(p\rho)) = O(1/\Lambda)$, giving any prescribed constant error by increasing C_0, while $\mathbb{E}|E(\widehat{S})| \leq m\rho/J^2 = O(m\Lambda)$.

Bottom-up use of (227) adds at most η per block solve and per ordinary sampled complement: at most $2J\eta$, plus the final sampling and terminal-inverse errors. There are never more than $m\rho$ multiedges. Hence the chain costs $O(m\Lambda J^3)$ to construct in expectation and $O(m\Lambda J^3 \log(2J))$ to apply, plus its terminal call.

Running this same chain to at most two vertices, with $J = \lceil 80\Lambda \rceil$ and an exact terminal solve, defines $\mathsf{Base}(K)$. On any connected input with M edges and r vertices, construction and application cost at most $O((M+r)\Lambda^5)$ in expectation. Its fixed positive definite map approximates K^{-1} to any fixed constant accuracy, with failure at most $\pi := n^{-40}$. It never calls the compression below. After sampling, check connectivity; on failure, abort that inverse construction and return the identity on its input coordinates. All outcomes then define a fixed positive definite map, and work bounds are unconditional.

3. Compress the small terminal graph. Let H have r vertices and M edges; choose $e^2 \leq s \leq 2n$. Keep an arbitrary spanning tree and sample each other edge independently with probability $1/s$, without rescaling. The pilot B is connected, satisfies $B \preceq H$, and has expected $O(M/s + r)$ edges. Define $t_e = w_e b_e^\top B^{-1} b_e$ and $a_e = \min\{1, t_e\}$. These upper-bound $\tau_e(H)$, and

$$\mathbb{E} \sum_e a_e \leq 2s(r-1). \tag{229}$$

For a non-tree edge, condition on all other sampling decisions. If its score before insertion is t, its leverage after insertion is $\lambda = t/(1+t)$ by the rank-one inverse formula. The inequality $\min\{1, t\} \leq 2t/(1+t)$ bounds its capped score in either outcome. Since its inclusion coin is independent of λ,

$$\mathbb{E} a_e \leq 2\mathbb{E}\lambda = 2s\, \mathbb{E}[\mathbf{1}_{e \in B} \tau_e(B)].$$

Tree-edge scores equal their leverage in B; summing and using $\sum_{e \in B} \tau_e(B) = r - 1$ proves (229).

An overestimate only keeps extra edges. Allowing score inflation by s reduces the number of probes to $k = \lceil C_1 \Lambda / \log s \rceil$. Build $Q_B = \mathsf{Base}(B)$ once and, with fresh independent $g_{e,j} \sim N(0,1)$, form

$$\xi_j = \sum_{e \in B} \sqrt{w_e}\, g_{e,j} b_e, \qquad y_j = Q_B \xi_j, \qquad \widehat{t}_e = \frac{w_e}{k} \sum_{j=1}^{k} (b_e^\top y_j)^2. \tag{230}$$

Independently retain edge e of H with probability $p_e = \min\{1, 4C_2\Lambda s\widehat{t}_e\}$ and rescale its weight by $1/p_e$; let K be the resulting graph. Use $\mathsf{Base}(K)$ as the terminal inverse. A zero sampling probability simply discards that edge.

Conditional on a successful fixed Q_B, squaring $B^{1/2}Q_BB^{1/2}$ gives

$$\tfrac{1}{4}B^{-1} \preceq \mathrm{Cov}(y_j) = Q_BBQ_B \preceq 4B^{-1}.$$

Thus $\widehat{t}_e/t_e$ is χ^2_k/k times a factor in $[1/4, 4]$. Markov applied to $e^{-s\chi^2_k/2}$ gives $\Pr[\chi^2_k/k < 1/s] \le (e/s)^{k/2} = n^{-\Omega(C_1)}$. A union bound over at most r^2 endpoint pairs therefore gives $\widehat{t}_e \ge t_e/(4s)$ simultaneously; parallel edges share the same normalized error. Hence $p_e \ge \min\{1, C_2\Lambda\tau_e(H)\}$, and leverage sampling makes K a constant-factor approximation to H with high probability. Each probe uses one fixed-inverse application, not an accurate solve.

For size, concavity gives $\mathbb{E}[p_e \mid B, Q_B] \le \min\{1, 16C_2\Lambda st_e\} \le 16C_2\Lambda sa_e$ on a good pilot inverse, whereas $p_e \le 1$ always. Consequently

$$\mathbb{E}|E(K)| = O(rs^2\Lambda) + \pi M. \tag{231}$$

This uses (229) without conditioning on successful sketches. Score evaluation costs $O(Mk)$. Since $k = O(\Lambda)$, all pilot and terminal baseline work, including the pilot's k applications and one terminal application, is at most

$$O\big((M/s + rs^2 + \pi M)\Lambda^6\big) \tag{232}$$

in expectation. The failure term follows from capped sampling probabilities; no sample-size test or retry is needed.

4. Assemble and balance the costs. Choose $h \ge C\log(2\Lambda)$ for a sufficiently large constant C, and set $s = e^h$, $J = \lceil 80h \rceil$. Stop the chain at $R = \max\{2, \lceil n/s^3 \rceil\}$ vertices, thinning in its last round. Each round reduces the nonroot count by at least $1/20$, so J rounds suffice. Solve directly when at most two vertices remain. Otherwise $r = O(n/s^3)$, $s < n^{1/3}$, and $\mathbb{E}M = O(m\Lambda)$; apply the compression above and attach its inverse to the stored blocks. An application goes down computing $u = Zb_F$ and passing $b_C + W^\top u$, applies the terminal inverse once, then recovers $x_F = u + ZWx_C$ on the way up.

The score scan costs $O(m\Lambda^2/h)$. Substitution into (232) bounds all auxiliary baseline work by $O(m\Lambda^7/s + \pi m\Lambda^7) = O(m)$. Combining this with the chain costs proves the following theorem; all errors fit within a fixed constant budget, and a union bound covers the polynomially many spectral events.

Theorem 472 (Sampling-based Laplacian solver)**.** *The construction returns a fixed symmetric positive definite map $Q \asymp \bar{L}^{-1}$ with high probability, with expected costs*

$$T_{\mathrm{build}} = O\left(m\Lambda h^3 + \frac{m\Lambda^2}{h}\right), \qquad T_{\mathrm{apply}} = O\big(m\Lambda h^3 \log(2h)\big). \tag{233}$$

For $b \perp \mathbf{1}$ and $0 < \varepsilon \le 1/2$, relative energy error $\|x - L^\dagger b\|_L \le \varepsilon\|L^\dagger b\|_L$ is obtained in expected time

$$O\left(\frac{m\Lambda^2}{h} + m\Lambda h^3 \log(2h)\log(2/\varepsilon)\right). \tag{234}$$

Choose the constant error budgets so that $\frac{1}{2}\bar{L}^{-1} \preceq Q \preceq 2\bar{L}^{-1}$. Starting from zero, Richardson steps $x \leftarrow x + \frac{1}{2}Q(b - \bar{L}x)$ contract energy-norm error by at most $3/4$. Here b is grounded; restore the root value zero afterward. All randomness is in preprocessing; every application uses the same linear map. Choose $h = \max\{C\log(2\Lambda), (\Lambda/\log(2\Lambda))^{1/4}\}$. Asymptotically the second term balances the two costs and gives (222); no separate small-input algorithm is needed. There is no weight-ratio dependence; disconnected inputs with componentwise zero-sum demands are handled componentwise. Correctness is high-probability, not Las Vegas; no finite-precision guarantee is asserted.

References. [SZ23] S. Sachdeva and Y. Zhao, *A Simple and Efficient Parallel Laplacian Solver*, arXiv:2304.14345, 2023. [CLMMPS15] M. B. Cohen, Y. T. Lee, C. Musco, C. Musco, R. Peng, and A. Sidford, *Uniform Sampling for Matrix Approximation*, ITCS 2015, arXiv:1408.5099.

12.10 Spectral Sparsification

After preconditioning, the Laplacian is $S = \sum_i M_i \asymp I$. We build a sparse positive sum A between moving barriers, $\ell S \prec A \prec uS$. Writing $U = uS - A$, $V = A - \ell S$ for the gaps, we permit an update εD only when $D \preceq b(U), b(V)$. The function b below gives a block budget $\mathcal{B} = b(U) \oplus b(V)$ with

$$\mathcal{B} \succeq \nu I, \qquad \operatorname{Tr} \mathcal{B}^{-1} = O(n), \qquad \nu = \Theta\left(\frac{1}{\log(2n)\log^2(2\log(2n))}\right). \tag{235}$$

The first bound admits the dense direction νS, since $S \preceq I$. The inverse-trace bound lets a capped packing obtain the needed expected reward from only $k = \lceil \nu n \rceil$ sampled edges. Each round advances the barrier center by $\Theta(\varepsilon\nu)$ but widens the barriers by only $\Theta(\varepsilon^2\nu)$. Thus $T = \Theta(\varepsilon^{-2}/\nu)$ rounds leave relative width $O(\varepsilon)$ and use $Tk = O(n/\varepsilon^2)$ edges.

Theorem 473 (Optimal-size spectral sparsification)**.** *Given a weighted undirected graph with n vertices and m edges and $0 < \varepsilon \le 1/2$, a randomized algorithm returns, with constant probability, a reweighted subgraph H with $O(n/\varepsilon^2)$ edges and $(1-\varepsilon)L_G \preceq L_H \preceq (1+\varepsilon)L_G$. Let $\Lambda = \log(2n)$ and $\Xi = \log(2\Lambda)$. If a constant-factor approximate Cholesky factor with S_R nonzeros is constructed in time T_{build}, the arithmetic work is*

$$O(T_{\text{build}} + (S_R + m + n)N), \qquad N = \frac{\Lambda^3\Xi^{9/2}}{\varepsilon^2} + \frac{\Lambda^2\Xi^3}{\varepsilon^3}.$$

Combine parallel edges and delete loops in $O(m\Lambda)$ time. Return the input if $m \le n/\varepsilon^2$; otherwise $m < n^2$ implies $\log(2/\varepsilon) = O(\Lambda)$. Below, ε is a sufficiently small constant multiple of the requested accuracy. Write $C : D = \operatorname{Tr}(CD)$; unsubscripted matrix norms are operator norms.

Gaps and budgets. Ground one vertex per connected component, let L be the resulting positive definite Laplacian, and scale the approximate Cholesky factor so that

$$\tfrac{1}{3} I \preceq S = R^{-1} L R^{-\top} \preceq I, \qquad v_e = \sqrt{w_e} R^{-1} b_e, \qquad M_e = v_e v_e^\top, \qquad \sum_e M_e = S.$$

Here b_e is the grounded incidence vector. Write $U = uS - A$ and $V = A - \ell S$. Their positivity is the desired sandwich, and $U + V = (u - \ell)S$ bounds both gaps once the width is bounded. Choose

$$b(x) = x^2 e^{-1/x}, \qquad \phi(x) = e^{e^{1/x}}, \qquad K(x) = -\phi'(x) = \phi(x)/b(x), \qquad J(x) = \sqrt{K(x)}. \tag{236}$$

The potential ϕ penalizes small gaps, K measures marginal cost, and J expresses scores as squared norms. In b, the factor x^2 cancels the inverses in $D(X^{-1})[H] = -X^{-1}HX^{-1}$, while $e^{-1/x}$ leaves contracting exponentials. The identity $bK = \phi$ controls relative potential change. Set

$$\Phi = \operatorname{Tr} \phi(U) + \operatorname{Tr} \phi(V), \qquad \mathcal{B} = b(U) \oplus b(V), \qquad D \oplus D \preceq \mathcal{B}. \tag{237}$$

Matrix functions are spectral. The last condition is the sufficient batch constraint $D \preceq b(U), b(V)$; the algorithm never evaluates Φ.

For $y = e^{1/x} > 1$ we have

$$bK = \phi, \qquad b^{-1} = y(\log y)^2 \le Ce^y = C\phi, \qquad \phi(x) \le C_* F \implies e^{1/x} \le \log(C_* F). \tag{238}$$

Fix $F = C_F n$, with C_F sufficiently large, and choose

$$a = \frac{1}{\log(C\Lambda)}, \qquad \nu = c\, b(a) = \Theta((\Lambda\Xi^2)^{-1}). \tag{239}$$

Allowing the fixed within-epoch growth factor C_* proved below, $\Phi \le C_* F$ gives the eigenvalue and inverse-trace bounds

$$U, V \succeq aI, \qquad \mathcal{B} \succeq \nu I, \qquad \operatorname{Tr} \mathcal{B}^{-1} = O(n), \qquad a^{-1} = \Theta(\Xi), \quad e^{1/a} = \Theta(\Lambda). \tag{240}$$

The local lemma controls potential cost and score-factor motion.

Lemma 474 (Local barrier stability)**.** *If $0 \prec X \preceq 9I$ and $0 \preceq Y \preceq \theta b(X)$ for sufficiently small θ, then*

$$\operatorname{Tr}\phi(X-Y) \le \operatorname{Tr}\phi(X) + (1+C\theta)K(X) : Y, \tag{241}$$

$$\operatorname{Tr}\phi(X+Y) \le \operatorname{Tr}\phi(X) - (1-C\theta)K(X) : Y, \tag{242}$$

$$\|J(X \pm Y) - J(X)\|_F \le C\theta\|J(X)\|_F. \tag{243}$$

Proof. First prove $Y \preceq 2\theta b(X_t)$ along $X_t = X \pm tY$; then integrate the derivatives in this moving budget.

Finite budget comparison. For the exponential factor, fix symmetric $R \succeq 0, R'$ and put $E = e^{R/2}(R'-R)e^{R/2}$ and $F(s) = e^{sR'/2}e^{-sR/2}$. Then

$$F'(s) = \tfrac{1}{2}e^{sR'/2}(R'-R)e^{-sR/2} = \tfrac{1}{2}F(s)e^{-(1-s)R/2}Ee^{-(1+s)R/2}.$$

The exponentials after $F(s)$ are contractions for $0 \le s \le 1$; $F(0) = I$ and Gronwall therefore give

$$\|e^{R'/2}e^{-R/2}\| \le e^{\|E\|/2}. \tag{244}$$

Now fix $t \in [0,1]$ and take $R = X^{-1}$, $R_t = X_t^{-1}$. The bound $Y \preceq \theta b(X) \preceq 9\theta X$ keeps the path positive and below $10I$. For the inverse comparison put

$$C = Y^{1/2}RY^{1/2}, \qquad B = e^{R/2}RY^{1/2}, \qquad \|C\| \le 9\theta, \quad \|B\|^2 \le \theta, \quad \kappa = (1-9\theta)^{-1}.$$

These norm bounds restate $Y \preceq 9\theta X$ and the original budget. The resolvent identities are

$$R_tY^{1/2} = RY^{1/2}(I \pm tC)^{-1}, \qquad E_t := e^{R/2}(R_t - R)e^{R/2} = \mp tB(I \pm tC)^{-1}B^\top, \quad \|E_t\| \le \kappa\theta.$$

Multiply by X_t for the first identity and use $R_t - R = \mp tR_tYR$ for the second. Then Equation (244) gives

$$\|b(X_t)^{-1/2}Y^{1/2}\| = \|(e^{R_t/2}e^{-R/2})B(I \pm tC)^{-1}\| \le e^{\kappa\theta/2}\sqrt{\theta}\,\kappa,$$

and squaring proves

$$Y \preceq \kappa^2 e^{\kappa\theta}\theta b(X_t) \preceq 2\theta b(X_t) \qquad (0 \le t \le 1). \tag{245}$$

Derivative control. For both derivatives use that, on $(0, 10]$, b increases with $b' = O(1)$ and $K = \phi/b$ decreases. For $s < t$, the identity $bK = \phi$ gives

$$b(t)(K(s) - K(t)) = (b(t) - b(s))K(s) + \int_s^t K(r)\,dr \le C(t-s)K(s).$$

Writing $f^{[1]}$ for a divided difference, with f' on the diagonal, we obtain

$$b(t)|K^{[1]}(s,t)| \le CK(s) \quad (s \le t). \tag{246}$$

In an eigenbasis of X_t, put $b_a = b(\lambda_a)$. For $r(t) = K(X_t) : Y$, symmetrizing this bound gives

$$|r'(t)| = \left|\sum_{a,b} K^{[1]}(\lambda_a, \lambda_b)|Y_{ab}|^2\right| \le 2C\operatorname{Tr}(K(X_t)Yb(X_t)^{-1}Y) \le 4C\theta r(t),$$

since Equation (245) implies $Yb(X_t)^{-1}Y \preceq 2\theta Y$. For $J^2 = K$, the same scalar estimate yields

$$\sqrt{b(s)b(t)}\,|J^{[1]}(s,t)| \le C\sqrt{\frac{b(s)}{b(t)}}\frac{K(s)}{J(s)+J(t)} \le CJ(s) \qquad (s \le t). \tag{247}$$

Write $Y = b(X_t)^{1/2}Z_tb(X_t)^{1/2}$, so $\|Z_t\| \le 2\theta$. The spectral derivative formula, with $J_a = J(\lambda_a)$, now gives

$$\begin{aligned}\left\|\tfrac{d}{dt}J(X_t)\right\|_F^2 &= \sum_{a,b} b_ab_b|J^{[1]}(\lambda_a,\lambda_b)|^2|(Z_t)_{ab}|^2 \\ &\le C\sum_{a,b}(J_a+J_b)^2|(Z_t)_{ab}|^2 \le 4C\|Z_t\|^2\sum_a J_a^2.\end{aligned}$$

The last step bounds each squared row and column norm by $\|Z_t\|^2$. Gronwall gives $r(t) = e^{O(\theta t)}r(0)$ and $\|J(X_t)\|_F \le e^{C\theta t}\|J(X)\|_F$. Integrating $\mp r(t)$ gives the potential bounds; integrating $\frac{d}{dt}J(X_t)$ gives the Frobenius bound. □

Sampling a profitable batch. The three conclusions are safety, curvature control, and progress.

Lemma 475 (Capped batch oracle)**.** *Assume $\Phi \le C_* F$. Given a symmetric G, scores $g_i = G : M_i$, and estimates with total error $e = \sum_i |\widehat{g}_i - g_i|$, one capped packing computation returns a batch supported on $k = \lceil \nu n \rceil$ sampled occurrences satisfying*

$$D \oplus D \preceq \mathcal{B}, \qquad \mathbb{E}D \preceq C_0 \nu S, \qquad \mathbb{E}(G : D) \ge \sigma_0 \nu G : S - C_0 \nu e, \tag{248}$$

for absolute $\sigma_0, C_0 > 0$. Expectations condition on the current state and estimates; only rare final-certificate failures are replaced by zero.

Proof. Let $\beta = \sum_i (\widehat{g}_i)_+$. If $\beta = 0$, return zero and use $G : S \le e$. Otherwise draw $i_1, \dots, i_k$ independently from $p_i = (\widehat{g}_i)_+ / \beta$ and set

$$\omega_j = \frac{\nu}{Ckp_{i_j}}, \qquad Y_j = \omega_j M_{i_j}, \qquad \mathcal{Y}_j = Y_j \oplus Y_j.$$

Every occurrence has reward $\omega_j \widehat{g}_{i_j} = \nu\beta/(Ck)$; maximizing estimated reward therefore amounts to maximizing mass:

$$\mathrm{OPT} = \max \left\{ \sum_j z_j : 0 \le z_j \le 1, \quad \sum_j z_j \mathcal{Y}_j \preceq \mathcal{B} \right\}. \tag{249}$$

Return $D = \sum_j z_j Y_j$; the implementation below gives expected mass $\Omega(\mathbb{E}\,\mathrm{OPT})$.

A fractional witness. For existence only, start at $H_0 = \mathcal{B}/2$ and release hI, $h = \nu/(2k)$, before each proposal:

$$Q_j = H_{j-1} + hI, \qquad \rho_j = \mathrm{Tr}(Q_j^{-1} \mathcal{Y}_j), \qquad \alpha_j = (1 + \rho_j)^{-1}, \qquad H_j = Q_j - \alpha_j \mathcal{Y}_j.$$

Shrinkage preserves positive slack; inverse trace will bound its average loss. Put

$$Z = Q^{-1/2} Y Q^{-1/2} \succeq 0, \qquad \rho = \mathrm{Tr}\, Z.$$

Every eigenvalue $\lambda \le \rho$ satisfies

$$(1 - \lambda/(1+\rho))^{-1} = 1 + \lambda/(1 + \rho - \lambda) \le 1 + \lambda.$$

Thus $H_j \succ 0$. Conditional on earlier proposals, $\mathbb{E}_j \mathcal{Y}_j \preceq \nu I/(Ck)$, and the released slack gives

$$\begin{aligned} (Q - Y/(1+\rho))^{-1} &\preceq Q^{-1} + Q^{-1} Y Q^{-1}, \\ \mathbb{E}_j H_j^{-1} &\preceq Q_j^{-1} + \frac{\nu}{Ck} Q_j^{-2} \preceq Q_j^{-1} + h Q_j^{-2} \preceq H_{j-1}^{-1}. \end{aligned}$$

The last step uses $(x+h)^{-1} + h(x+h)^{-2} \le x^{-1}$. Therefore

$$\mathbb{E}\rho_j \le \frac{\nu}{Ck} \mathbb{E}\,\mathrm{Tr}\, Q_j^{-1} \le \frac{2\nu}{Ck}\,\mathrm{Tr}\,\mathcal{B}^{-1} = O(1/C), \qquad \mathbb{E}\alpha_j \ge 1 - \mathbb{E}\rho_j \ge \tfrac{1}{2}$$

for large C. Telescoping proves feasibility and expected mass:

$$\sum_j \alpha_j \mathcal{Y}_j = \mathcal{B}/2 + (\nu/2) I - H_k \preceq \mathcal{B}, \qquad \mathbb{E}\,\mathrm{OPT} \ge \mathbb{E} \sum_j \alpha_j \ge k/2.$$

The output has expected estimated reward $\Omega(\nu\beta)$. To pass to true reward, the caps give $\mathbb{E}D \preceq \mathbb{E}\sum_j Y_j \preceq (\nu/C)S$ and $\mathbb{E}\sum_j \omega_j z_j |\widehat{g}_{i_j} - g_{i_j}| \le (\nu/C)e$. Thus, since $\beta \ge \sum_i \widehat{g}_i \ge G : S - e$,

$$\mathbb{E}(G : D) \ge \sigma_0 \nu \beta - C\nu e \ge \sigma_0 \nu G : S - C_0 \nu e.$$

□

The outer iteration. The oracle gives expected first-order progress $\sigma_0 \nu G : S$. We therefore move both barriers by the matching amount

$$\delta = \sigma_0 \varepsilon \nu, \tag{250}$$

and widen them by only an additional $\gamma\varepsilon\delta$ on each side to pay for curvature. Starting from $A_0 = 0, u_0 = 3, \ell_0 = -3$, run $T = \lfloor (\gamma\varepsilon\delta)^{-1} \rfloor = \Theta(\varepsilon^{-2}/\nu)$ rounds. A round moves the barriers first, so the convexity bound and the batch reward use the same gradient:

$$\bar{u} = u + (1+\gamma\varepsilon)\delta, \qquad \bar{\ell} = \ell + (1-\gamma\varepsilon)\delta, \qquad A^+ = A + \varepsilon D. \tag{251}$$

Evaluate scores at the moved gaps $\bar{U} = \bar{u}S - A, \bar{V} = A - \bar{\ell}S$:

$$G = K(\bar{V}) - K(\bar{U}), \qquad Q = (K(\bar{U}) + K(\bar{V})) : S.$$

Use Lemma 475 with these gaps and estimates of $G : M_i$; set $u^+ = \bar{u}, \ell^+ = \bar{\ell}$. The width never exceeds $6 + 2\gamma T\varepsilon\delta \le 8$, so positive gaps are at most $8I$. The translations are $O(\varepsilon)$-budget steps by Equation (240).

Convexity, using the gradient at the moved endpoint, gives

$$\bar{\Phi} - \Phi \le -(1+\gamma\varepsilon)\delta K(\bar{U}) : S + (1-\gamma\varepsilon)\delta K(\bar{V}) : S = \delta G : S - \gamma\varepsilon\delta Q.$$

The batch changes the gaps by $-\varepsilon D, +\varepsilon D$, so the local lemma gives $\Phi^+ - \bar{\Phi} \le -\varepsilon G : D + C\varepsilon^2(K(\bar{U}) + K(\bar{V})) : D$. Conditioning on the state and score estimates, the oracle gives

$$\begin{aligned} \mathbb{E}_t(\Phi^+ - \Phi) &\le \delta G : S - \gamma\varepsilon\delta Q - \varepsilon(\sigma_0 \nu G : S - C_0 \nu e_t) + CC_0\varepsilon^2\nu Q \\ &= -(\gamma\sigma_0 - CC_0)\varepsilon^2\nu Q + C_0\varepsilon\nu e_t \le -c\varepsilon^2\nu Q + C\varepsilon\nu e_t. \end{aligned} \tag{252}$$

Thus the common barrier displacement cancels the oracle's linear progress exactly, and the slight asymmetry leaves the negative $\Theta(\varepsilon^2\nu Q)$ drift that pays for curvature. The only remaining requirement is aggregate score error $e_t = O(\varepsilon Q)$.

Tracking the edge scores. Recomputing all scores to that accuracy every round would use $\Theta(\varepsilon^{-2})$ probes. Instead refresh once every $q = \lceil 1/\varepsilon \rceil$ rounds and estimate only successive changes; the paired estimator then pays for how far $J(U), J(V)$ move, not for their absolute size. The drift needs only aggregate error, never coordinatewise confidence intervals. Within an epoch let U_t, V_t be the moved gaps. For an independent standard Gaussian pair $\xi = (\xi_U, \xi_V)$, set

$$h_{i,t}(\xi) = (\xi_V^\top J(V_t) v_i)^2 - (\xi_U^\top J(U_t) v_i)^2, \qquad \mathbb{E}_\xi h_{i,t}(\xi) = g_{i,t}.$$

Use $N_0 = \lceil C_{\mathrm{tr}}\varepsilon^{-2} \rceil$ shared pairs at each epoch start and $N_1 = \lceil C_{\mathrm{tr}}\varepsilon^{-1} \rceil$ fresh shared pairs between evaluations:

$$\widehat{g}_{i,0} = N_0^{-1} \sum_j h_{i,0}(\xi_j), \tag{253}$$

$$\widehat{g}_{i,t+1} = \widehat{g}_{i,t} + N_1^{-1} \sum_j \big(h_{i,t+1}(\xi_j) - h_{i,t}(\xi_j)\big). \tag{254}$$

Pairs are shared across edges and endpoints, but drawn fresh after both states are fixed. The amortized cost is $N_0/q + N_1 = O(\varepsilon^{-1})$.

How much do the score factors move? Condition on the epoch-start history and put

$$f_{i,t} = (J(U_t)v_i, J(V_t)v_i), \quad a_{i,t} = \|f_{i,t}\|^2, \quad d_{i,t} = \|f_{i,t+1} - f_{i,t}\|^2, \quad Q_t = \sum_i a_{i,t}.$$

Each transition is a batch update followed by a barrier translation, both $O(\varepsilon)$-budget steps. With $\Delta J_{U,t} = J(U_{t+1}) - J(U_t)$ and similarly for V, Equation (243) and $I/3 \preceq S \preceq I$ give

$$\sum_i d_{i,t} = \mathrm{Tr}(\Delta J_{U,t}^2 S) + \mathrm{Tr}(\Delta J_{V,t}^2 S) \le \|\Delta J_{U,t}\|_F^2 + \|\Delta J_{V,t}\|_F^2 = O(\varepsilon^2 Q_t),$$

$$|Q_{t+1} - Q_t| \le \|\Delta J_{U,t}\|_F (\|J(U_{t+1})\|_F + \|J(U_t)\|_F) + (U \leftrightarrow V) = O(\varepsilon Q_t).$$

Hence $Q_t \asymp Q_0$ and $\sum_{i,t} d_{i,t} = O(q\varepsilon^2 Q_0)$. To normalize errors without a shrinking denominator, use the analysis-only running maxima $w_{i,t} = \max_{s\le t} a_{i,s}$. Cauchy–Schwarz gives

$$w_{i,t} \le 2a_{i,0} + 2q\sum_{s<t} d_{i,s}, \qquad \sum_i w_{i,t} \le 2Q_0 + 2q\sum_{i,s} d_{i,s} = O(Q_0).$$

Convert factor motion into score accuracy. For a standard Gaussian, the paired-difference variance is

$$\mathrm{Var}\big((\xi^\top y)^2 - (\xi^\top x)^2\big) = 2\|yy^\top - xx^\top\|_F^2 \le 4\|y-x\|^2(\|x\|^2 + \|y\|^2).$$

Apply this to each component of f_i. The error $Z_{i,t} = \widehat{g}_{i,t} - g_{i,t}$ starts with variance $O(a_{i,0}^2/N_0)$ and obeys $Z_{i,t+1} = Z_{i,t} + \eta_{i,t}$. Let $\mathbb{E}_{\mathrm{new}}$ condition after both endpoints are fixed and before the fresh probes. The noise has mean zero and variance $O(d_{i,t}(a_{i,t+1} + a_{i,t})/N_1)$. Since $w_{i,t+1}$ is fixed and dominates both endpoint energies,

$$\mathbb{E}_{\mathrm{new}}\frac{Z_{i,t+1}^2}{w_{i,t+1}} = \frac{Z_{i,t}^2 + \mathbb{E}_{\mathrm{new}}\eta_{i,t}^2}{w_{i,t+1}} \le \frac{Z_{i,t}^2}{w_{i,t}} + \frac{Cd_{i,t}}{N_1}.$$

Zero denominators contribute zero. Summing, telescoping, and applying Cauchy–Schwarz proves

$$\mathbb{E}\sum_i Z_{i,t}^2/w_{i,t} \le C\left(Q_0/N_0 + q\varepsilon^2 Q_0/N_1\right) = O(\varepsilon^2 Q_0/C_{\mathrm{tr}}),$$

$$\mathbb{E}e_t = \mathbb{E}\sum_i |Z_{i,t}| \le \left(CQ_0\,\mathbb{E}\sum_i Z_{i,t}^2/w_{i,t}\right)^{1/2} = O(\varepsilon Q_0/\sqrt{C_{\mathrm{tr}}}). \tag{255}$$

Since $Q_t \asymp Q_0$, a sufficiently large constant C_{tr} makes the error term in Equation (252) smaller than half of its negative drift. Hence, conditional on the epoch-start history, $\mathbb{E}(\Phi_{\mathrm{end}} \mid \mathcal{F}_{\mathrm{start}}) \le \Phi_{\mathrm{start}}$.

Why the local estimates remain valid. Initially $U = V = 3S$ and $\Phi_0 \le 2ne^e$. Every substep satisfies $|\Delta \operatorname{Tr}\phi(X)| \le C\varepsilon K(X) : b(X) = C\varepsilon \operatorname{Tr}\phi(X)$, so an epoch grows Φ by at most a fixed factor C_*. Stop the analysis at the first epoch boundary with $\Phi \ge F$. Earlier epochs stay below C_*F, so the estimates apply by induction. Conditional on each starting history their expected terminal potential is no larger; thus the stopped boundary sequence is a nonnegative supermartingale and crosses F with probability at most Φ_0/F. Choosing C_F large makes this small.

On the complementary event, apart from the rare feasibility-certificate failures treated below,

$$Tk = O((n+\nu^{-1})/\varepsilon^2) = O(n/\varepsilon^2), \qquad u_T + \ell_T = 2T\delta = \Theta(\varepsilon^{-1}), \quad u_T - \ell_T \le 8.$$

Return $2A_T/(u_T+\ell_T)$: its eigenvalue sandwich relative to S is $1 \pm (u_T-\ell_T)/(u_T+\ell_T) = 1 \pm O(\varepsilon)$. Congruence by R proves the graph guarantee.

Implicit matrix functions. For the current edge sum L_A,

$$U = R^{-1}(uL - L_A)R^{-\top}, \qquad V = R^{-1}(L_A - \ell L)R^{-\top}, \qquad W(x) = b(x)^{-1/2} = x^{-1}e^{1/(2x)}.$$

A gap product costs $O(S_R + m + n)$. The constraint $D \preceq b(X)$ is $W(X)DW(X) \preceq I$; a packing matrix product or a batch of its edge quadratic forms therefore uses $O(1)$ applications of W, triangular solves, and $O(m+n)$ further work.

We need constant relative accuracy for W and relative $O(\varepsilon)$ accuracy for J. On $[a, 10]$, Lemma 595 and Equation (318) give

$$\deg W = O(\Xi^{3/2}), \qquad \deg J = O(\Lambda\Xi).$$

For W, a complex-angle strip of width $c\sqrt{a}$ stays in $\operatorname{Re} z \ge a/2$ and has $\log|W(z)| = O(a^{-1})$. For $J(z) = W(z)\exp(e^{1/z}/2)$, width ca gives $\operatorname{Re}(1/z) \le a^{-1} + O(1)$ and $\log|J(z)| = O(a^{-1} + e^{1/a}) = O(\Lambda)$. The latter degree also uses $\log(2/\varepsilon) = O(\Lambda)$.

Both functions are bounded below by a constant. An upward-shifted approximation $W(X) \preceq P_X \preceq 2W(X)$ makes $P_X D P_X \preceq I$ conservative, with $\frac{1}{4}b(X) \preceq P_X^{-2} \preceq b(X)$; the batch bounds change only by constants. Relative $c\varepsilon$ approximation of J adds $O(c\varepsilon Q)$ aggregate score bias. The triangle inequality preserves its $O(\varepsilon)$ Frobenius increments, so the same tracking proof applies; small c absorbs the bias into Equation (252).

Implementing the capped packing. The outer proof needs only a feasible vector with expected mass $\Omega(\mathrm{OPT})$; it does not need the packing dual or a high-probability objective guarantee. This lets us specialize the exponential update of Theorem 656 into four simple stages: prune oversized sampled inputs, normalize them uniformly, run an exact-power update, and certify one random primal iterate. Encode the caps and gap constraints by

$$\mathsf{C}_j = \operatorname{diag}\big(\omega_j P_U M_{i_j} P_U,\ \omega_j P_V M_{i_j} P_V,\ e_j e_j^\top\big), \qquad r_j = \operatorname{Tr}\mathsf{C}_j - 1.$$

Using $O(\Lambda)$ shared Gaussian probes, estimate all $r_j \le \widehat{r}_j \le 2r_j$ with failure probability n^{-C_1}. Retain $\mathcal{I} = \{j : \widehat{r}_j \le 1\}$, returning zero if it is empty. Accurate estimates imply

$$j \in \mathcal{I}: \quad \|\mathsf{C}_j\| = 1, \quad 1 \le \operatorname{Tr}\mathsf{C}_j \le 2; \qquad j \notin \mathcal{I}: \quad r_j > \tfrac{1}{2}.$$

The purpose of the pruning is solely to make every retained packing matrix have the same operator norm. It loses little expected value:

$$\mathbb{E}\sum_j r_j \le \frac{\nu}{C}\operatorname{Tr}((P_U^2 + P_V^2)S) \le \frac{4\nu}{C}\operatorname{Tr}\mathcal{B}^{-1} = O(k/C), \qquad \mathbb{E}|\mathcal{I}^c| \le 2\mathbb{E}\sum_j r_j + kn^{-C_1}.$$

Deleting a coordinate costs at most one in the capped objective, hence $\mathbb{E}\,\mathrm{OPT}_{\mathcal{I}} = \Omega(k)$ for large C.

Normalize and initialize. After pruning, no per-coordinate scaling is needed. Let $d = O(n)$ be the packing dimension, set $\mathsf{A}_j = d\mathsf{C}_j$, and write $\tau = \mathrm{OPT}_{\mathcal{I}}/d$. Then $\|\mathsf{A}_j\| = d$ and $d \le \operatorname{Tr}\mathsf{A}_j \le 2d$, so $1/d \le \tau \le 1$. Choose odd $p \ge \max\{64, 64\log(2d)\}$ minimally and set

$$w_{0,j} = \frac{c}{d\Lambda}, \qquad H = \log(\Lambda/c) = \Theta(\Xi), \qquad T_{\mathrm{p}} = \lceil 2pH \rceil. \tag{256}$$

The initialization is safe with high probability. Indeed the independent truncations $\mathsf{C}_j^0 = \mathsf{C}_j \mathbf{1}_{\{r_j \le 1\}}$ are positive contractions and

$$\mathbb{E}\sum_j \mathsf{C}_j^0 \preceq \mathbb{E}\sum_j \mathsf{C}_j \preceq \operatorname{diag}(4I/C, 4I/C, I_k),$$

since, for example, $\mathbb{E}\sum_j \omega_j P_U M_{i_j} P_U \preceq (\nu/C)P_U S P_U \preceq (4\nu/C)b(U)^{-1} \preceq 4I/C$. Matrix Chernoff gives $\|\sum_j \mathsf{C}_j^0\| = O(\Lambda)$. On accurate estimates $\sum_{j\in\mathcal{I}} \mathsf{C}_j \preceq \sum_j \mathsf{C}_j^0$, even with shared probes. Thus small c gives $P_0 := \sum_j w_{0,j}\mathsf{A}_j \preceq I/2$ and $Q_0 := \operatorname{Tr} P_0^p \le d2^{-p} \le 1/(2d) \le \tau$.

Exact-power update. The inner process has two analysis quantities: $Q_t = \operatorname{Tr} P_t^p$ controls spectral growth, while $R_t = M_t/Q_t^{1/p}$ is the feasible primal value we ultimately want. Condition on the retained instance and successful initialization. For round t, use four fresh standard Gaussians and compute

$$P_t = \sum_j w_{t,j}\mathsf{A}_j, \quad v_{t,a} = P_t^{(p-1)/2}\xi_{t,a}\ (1 \le a \le 4), \quad B_t = \tfrac{1}{2}\sum_{a=1}^{4} v_{t,a}v_{t,a}^\top,$$
$$b_{t,j} = \mathsf{A}_j : B_t, \qquad w_{t+1,j} = w_{t,j}\exp((1-b_{t,j})/p), \qquad Q_t = \operatorname{Tr} P_t^p, \quad M_t = \mathbf{1}^\top w_t,$$
$$R_t = M_t/Q_t^{1/p} \le \tau.$$

For $a_j = \mathsf{A}_j : P_t^{p-1}$, the packing proof's Gaussian calculation gives $\mathbb{E}_t e^{-b_{t,j}} \le (1+a_j)^{-2}$. Its Lieb–Thirring inequality and $ea/(1+a)^2 \le e/4 < 1$ therefore give

$$\mathbb{E}_t Q_{t+1} \le \sum_j w_{t,j}\frac{ea_j}{(1+a_j)^2} \le M_t \le \alpha R_t + Q_t/p, \qquad \alpha = 1 - 1/p. \tag{257}$$

The last inequality is $x \le \alpha + x^p/p$, applied to $x = Q_t^{1/p}$, using $R_t \le \tau \le 1$. From $Q_0 \le \tau$, induction gives $\mathbb{E}Q_t \le \tau$. Since $dw_{T_{\mathrm{p}},j} \le Q_{T_{\mathrm{p}}}^{1/p}$, Jensen bounds weight growth, and telescoping the update bounds average

score:

$$\mathbb{E}\log\frac{w_{T_{\mathrm{p}},j}}{w_{0,j}} \le H + p^{-1}\log\mathbb{E}Q_{T_{\mathrm{p}}} \le H,$$
$$T_{\mathrm{p}}^{-1}\sum_{t<T_{\mathrm{p}}}\mathbb{E}(\mathsf{A}_j : B_t) = 1 - \frac{p}{T_{\mathrm{p}}}\mathbb{E}\log\frac{w_{T_{\mathrm{p}},j}}{w_{0,j}} \ge \tfrac{1}{2}.$$

For optimal $w^\star$, feasibility gives $\tau/2 \le T_{\mathrm{p}}^{-1}\sum_t \mathbb{E}(P(w^\star) : B_t) \le T_{\mathrm{p}}^{-1}\sum_t \mathbb{E}\operatorname{Tr} B_t$. Conversely, power mean and $u^\alpha \le \alpha u + 1/p$ give

$$\mathbb{E}_t \operatorname{Tr} B_t = 2\operatorname{Tr} P_t^{p-1} \le 2d^{1/p}Q_t^\alpha \le 2d^{2/p}\left(\alpha Q_t + \frac{1}{dp}\right) \le 4Q_t + 4\tau/p.$$

The argument now closes in two steps. The lower bound on average sampled trace forces $\bar{Q} = \Omega(\tau)$; the moment recurrence then says that Q_t cannot stay large unless the feasible value R_t is large on average. Writing $\bar{Q} = T_{\mathrm{p}}^{-1}\sum_t \mathbb{E}Q_t$ and $\bar{R} = T_{\mathrm{p}}^{-1}\sum_t \mathbb{E}R_t$ gives

$$\bar{Q} \ge (1/8 - 1/p)\tau, \qquad \alpha T_{\mathrm{p}}(\bar{Q} - \bar{R}) \le Q_0 - \mathbb{E}Q_{T_{\mathrm{p}}} \le Q_0, \qquad \bar{R} \ge \tau/16.$$

For the last bound use $Q_0 \le \tau$, $p \ge 64$, and $T_{\mathrm{p}} \ge 2p$.

Certify one primal iterate. Since $\bar{R} = \Omega(\tau)$, a uniformly random iterate already has the required expected feasible value after normalization. Keep w_J, $0 \le J < T_{\mathrm{p}}$, uniformly at random, and let $X = P_J$. With an independent standard Gaussian ξ, set

$$s = \lceil C_2\Lambda \rceil, \qquad \widehat{\lambda} = 2\|X^s\xi\|^{1/s}, \qquad z_j = dw_{J,j}/\widehat{\lambda} \quad (j \in \mathcal{I}),$$

with zero on discarded coordinates; return zero if the estimate vanishes. Gaussian small-ball and norm bounds imply $\|X\| \le \widehat{\lambda} \le 4\|X\|$ except with probability n^{-C_3}. On this event $\sum_j z_j\mathsf{C}_j = X/\widehat{\lambda} \preceq I$ and $\mathbf{1}^\top z \ge dR_J/4$, so its expected value is $\Omega(d\tau) = \Omega(\mathrm{OPT}_{\mathcal{I}})$ after zeroing certificate failures in the analysis. Preprocessing and initialization failures lose at most k times their probability. Only the certificate is union-bounded across outer calls: it protects feasibility even on suboptimal runs. The input shortcut makes the number of calls polynomial in n.

Work. Each packing call uses $T_{\mathrm{p}} = O(\Lambda\Xi)$ rounds of $O(\Lambda)$ products and a constant number of quadratic-form batches. Pruning and certification cost only $O(\Lambda)$ more. With $O(\varepsilon^{-1})$ score probes per outer round, the total product count is

$$N = O\Big(T\left[\Lambda^2\Xi\cdot\Xi^{3/2} + \varepsilon^{-1}\Lambda\Xi\right]\Big) = O\left(\frac{\Lambda^3\Xi^{9/2}}{\varepsilon^2} + \frac{\Lambda^2\Xi^3}{\varepsilon^3}\right).$$

12.11 Connectivity

12.11.1 Connectivity Augmentation and Splitting Off

Let $G = (V, E)$ be an undirected multigraph with nonnegative integral edge weights, and let $\tau \ge 2$. We seek a minimum-cardinality multiset of unit edges whose addition makes G τ-edge-connected. Write $\delta_G(X) = w_G(X, V \setminus X)$, usually omitting the subscript.

We first choose the new edge endpoints and only then pair them. If $b(v)$ is the number of new incidences placed at v, put

$$f_b(X) := \delta_G(X) + b(X).$$

We first find a minimum integral $b \ge 0$ satisfying

$$f_b(X) \ge \tau \qquad (\varnothing \ne X \subsetneq V), \tag{258}$$

and then pair these incidences into edges while preserving the vector b. If F is a pairing and $H = G + F$, then

$$f_b(X) = \delta_H(X) + 2w_F(E[X]). \tag{259}$$

Every incidence in X either crosses the cut or is paired inside X, proving Equation (259). Thus the gap $f_b(X) - \delta_H(X)$ is exactly twice the augmentation mass stored inside X, and re-pairing only moves this stored capacity.

We first construct the minimum feasible star b, exposing inclusion-minimal low regions simultaneously and closing their extensions by isolating cuts. We then pair the fixed incidences: a deterministic cyclic exchange handles all nonconstant scales, and a one-unit repair handles the remaining constant range.

Both stages use the same low-cut method. At each scale take a constant-error sampled cut multigraph, rescaled so that the minimum proper terminal cut is $C = \Theta(\log n)$ and every nonzero sampled capacity is at least one. Cap sampled capacities at C; this does not change whether a cut has value below C. We also use the standard batched masked fixed-tree method, which evaluates the canonical 1- and 2-respecting candidates below, and after contractions the analogous masked isolating candidates, in $O(m \log n + n \log^2 n)$ time.

Atoms and guide trees. A *d-atom* is an inclusion-minimal nonempty proper shore A with $\delta(A) < d$. These are the low regions used in both stages.

Lemma 476 (Atom structure)**.** *Distinct d-atoms are disjoint, every shore of value below d contains one, and every d-atom A satisfies*

$$2\delta_{G[A]}(U) = \delta_G(U) + \delta_G(A \setminus U) - \delta_G(A) \geq d + 1 \qquad (\varnothing \neq U \subsetneq A). \tag{260}$$

Proof. Every deficient shore contains an inclusion-minimal one, and distinct atoms cannot contain one another. If two atoms A, B crossed, posimodularity would give $\delta(A) + \delta(B) \geq \delta(A \setminus B) + \delta(B \setminus A) \geq 2d$, contradicting $\delta(A), \delta(B) < d$; hence atoms are disjoint. For the last claim, both U and $A \setminus U$ are proper nonempty subsets of A, hence each has cut value at least d, whereas $\delta(A) \leq d - 1$; the cut identity gives Equation (260). □

Consider a recursive domain S. Everything outside S is contracted to one outside vertex, and every proper shore in the domain has value at least L. Let κ be the capped sampled capacities at this scale, and put $W = \sum_e \kappa_e$.

Lemma 477 (Greedy guide-tree packing)**.** *For any fixed $\varepsilon > 0$, $R = O(C \log W) = O(\log^2 n)$ applications of stopped Kruskal produce terminal-spanning trees $T_1, \ldots, T_R$ such that*

$$\frac{1}{R} \sum_{j=1}^{R} \mathbf{1}[e \in T_j] \leq \frac{2 + \varepsilon}{C} \kappa_e \qquad \forall e. \tag{261}$$

Proof. For arbitrary nonnegative costs x_e, run Kruskal and stop once all terminals lie in one component. At threshold t, let k be the number of terminal-containing components formed by edges of cost at most t. Their sampled boundaries total at least kC and count every edge of cost greater than t at most twice, so

$$\kappa(\{e : x_e > t\}) \geq kC/2.$$

There is at most one additional component containing only the outside vertex; hence stopped Kruskal uses at most k edges of cost greater than t. Using the layer-cake identity $x(T) = \int_0^\infty |\{e \in T : x_e > t\}|\, dt$ and the preceding bound gives

$$x(T) \leq \frac{2}{C} \sum_e \kappa_e x_e. \tag{262}$$

Now let ℓ_e be the current load, choose a constant $a > 1$ close to one, and call the oracle with $x_e = a^{\ell_e/\kappa_e}$. For $\Phi = \sum_e \kappa_e a^{\ell_e/\kappa_e}$, one tree increases the potential by at most

$$\sum_{e \in T} \kappa_e a^{\ell_e/\kappa_e}(a^{1/\kappa_e} - 1) \leq (a-1)x(T) \leq \frac{2(a-1)}{C}\Phi,$$

where $\kappa(a^{1/\kappa}-1) \le a-1$ for $\kappa \ge 1$. Hence $\Phi_R \le W \exp(2(a-1)R/C)$. On the other hand, the contribution of a single edge gives $a^{\ell_e/\kappa_e} \le \Phi_R$, so

$$\frac{\ell_e}{R\kappa_e} \le \frac{\log W}{R \log a} + \frac{2}{C}\frac{a-1}{\log a}.$$

Choose a so that $2(a-1)/\log a \le 2 + \varepsilon/2$, and then choose the constant in $R = \Theta(C \log W)$ so that the first term is at most $\varepsilon/(2C)$. This yields Equation (261). □

For sufficiently small fixed sampling error and ε, every original cut of value at most $4L/3$ has

$$\frac{1}{R}\sum_T |\delta_T(X)| \le \frac{2+\varepsilon}{C}\,\kappa(\delta(X)) < 3.$$

Since every tree crosses a proper terminal shore at least once and crossing numbers are integral, some guide tree crosses X at most twice. The packing has therefore reduced low-cut recovery to finding the relevant 1- and 2-respecting shores of a fixed tree.

Lemma 478 (Canonical candidates)**.** *For a rooted tree T, $O(n)$ candidate shores contain every root-avoiding d-atom that 2-respects T.*

Proof. Write A_u for the subtree rooted at u. A root-avoiding 2-respecting shore is a subtree, a difference $A_u \setminus A_v$ with v below u, or a union $A_u \cup A_v$ of incomparable subtrees.

All subtrees are candidates. For a difference, fix v and retain only the deepest ancestor u for which $A_u \setminus A_v$ is deficient. Any shallower deficient difference strictly contains this one, so it cannot be inclusion-minimal.

For a union that is an atom, neither constituent subtree can itself be deficient. Hence

$$2w(A_u, A_v) = \delta(A_u) + \delta(A_v) - \delta(A_u \cup A_v) > \delta(A_u).$$

For fixed u, two feasible incomparable partners would contribute disjoint portions of $\delta(A_u)$, each heavier than $\delta(A_u)/2$, impossible. Thus feasible partners lie on one chain, and only the deepest can give an inclusion-minimal union. Keeping all subtrees, one difference per v, and one union per u gives $O(n)$ candidates. □

Assume $d-1 \le 4L/3$. Root every guide tree at an arbitrary terminal r. Every deficient candidate contains a d-atom avoiding r, and every such atom is itself a candidate, so the inclusion-minimal deficient candidates are exactly the atoms avoiding r. If none appears, no deficient shore exists: the shore of any deficient cut avoiding r would contain such an atom. Otherwise reroot at a vertex of any recovered atom A. At most one atom contains r, and by disjointness that atom avoids the new root, so the second pass finds it as well. These two passes form the *low-cut method*.

There are $O(\log^2 n)$ guide trees, hence on a domain with m supported edges the method costs

$$O(m \log^3 n + n \log^4 n). \tag{263}$$

The minimum star. A recursive domain S comes with a certified lower bound L on every proper terminal shore. We expose all atoms below a larger threshold d, solve the resulting children, then close every deficient extension of an atom by one batched isolating-cut step.

Work independently inside every connected component of the positive-weight support. A nontrivial component starts with $L = 1$; a singleton has no internal cut. If a domain already has $L \ge \tau$, it is solved. Otherwise put

$$d := L + \min\left\{ \left\lceil \frac{L}{3} \right\rceil, \left\lceil \frac{\tau - L}{2} \right\rceil \right\}. \tag{264}$$

Then

$$d - 1 \le \frac{4}{3}L, \qquad 2(\tau - d + 1) \ge \tau - L. \tag{265}$$

The first inequality exposes every d-atom; the second will rule out a deficient shore containing two solved atoms. If no d-atom exists, continue on S with lower bound d. Otherwise let the atoms be $A_1, \dots, A_r$ and $R = S \setminus \bigcup_i A_i$. Every proper shore inside an atom or inside R has value at least d (otherwise it contains another d-atom), so recurse on the nonempty sets among $A_1, \dots, A_r, R$ with lower bound d.

Process bottom-up. When a nonroot child Q returns, its proper shores are feasible; if $f_b(Q) < \tau$, add $\tau - f_b(Q)$ incidences at one vertex of Q. Then Q itself is tight, so it is a proper tight witness for every positive coordinate in Q.

After the children are solved, a remaining deficient shore cannot lie inside any one child and therefore must contain an atom and extend beyond it. Contract each A_i to a terminal a_i, let o be the outside vertex, add an edge of capacity $b(v)$ from each uncontracted vertex v to o, and aggregate $b(A_i)$ on the edge $a_i o$. Then

$$\delta_{\widehat{G}}(X) = \delta_G(X) + b(X) = f_b(X) \tag{266}$$

for every shore $X \subsetneq S$. Let m_i be the minimum a_i-isolating shore value (containing a_i but neither o nor another atom terminal), and increase $b(A_i)$ by $(\tau - m_i)_+$. This raises every a_i-isolating shore by that amount and affects no a_j-isolating shore, so the updates are simultaneous.

Lemma 479 (Domain closure)**.** *After these isolating updates, every proper shore of S has value at least τ.*

Proof. Suppose an inclusion-minimal deficient shore $X \subsetneq S$ remains. It cannot cross an atom A_i: then minimality of X and A_i gives

$$f_b(X \setminus A_i) \geq \tau, \qquad \delta_G(A_i \setminus X) \geq d,$$

and posimodularity yields

$$f_b(X) + \delta_G(A_i) \geq f_b(X \setminus A_i) + \delta_G(A_i \setminus X) \geq \tau + d,$$

contradicting $f_b(X) < \tau$ and $\delta_G(A_i) \leq d - 1$.

Thus X contains or avoids every atom. If it avoided them all, then X would lie inside the solved residual child, so it contains at least one. Because the child A_i itself was made feasible, $b(A_i) \geq \tau - \delta_G(A_i) \geq \tau - d + 1$. If X contained two atoms, then by Equation (265)

$$b(X) \geq 2(\tau - d + 1) \geq \tau - L,$$

while $\delta_G(X) \geq L$, contradicting deficiency. Thus X contains exactly one atom and, after contraction, is an isolating shore for its terminal; its value was raised to at least τ, again a contradiction. □

No new low-cut structure is needed for the isolating minima. A deficient a_i-isolating shore X contains the feasible returned atom A_i, so

$$b(X) \geq b(A_i) \geq \tau - d + 1.$$

Together with $f_b(X) \leq \tau - 1$, this gives $\delta_G(X) \leq d - 2$, already within the guide-tree range; the masked fixed-tree method therefore finds the isolating minimum.

Along a root-to-leaf path, either L grows by a factor at least $4/3$ or the gap $\tau - L$ halves. Hence there are $O(\log \tau)$ levels, and domains at one level are disjoint.

At the top level, every proper connected component C has $\delta_G(C) = 0$, so top up $b(C)$ to τ if necessary. The final star is feasible and every positive coordinate lies in a proper tight shore.

For each v with $b(v) > 0$, choose an inclusion-minimal tight shore containing it. These witnesses are laminar: if tight X, Y crossed, then

$$\begin{aligned} f_b(X) + f_b(Y) = f_b(X \setminus Y) + f_b(Y \setminus X) \\ + 2b(X \cap Y) + 2w_G(X \cap Y, V \setminus (X \cup Y)). \end{aligned}$$

The differences are nonempty proper shores, so feasibility makes the first two terms at least τ each. Since the left side equals 2τ, equality forces both differences tight and $b(X \cap Y) = 0$. In particular, the positive

coordinate for which X was chosen lies in the smaller tight shore $X \setminus Y$, contradicting minimality. Thus maximal witnesses $W_1, \dots, W_k$ are disjoint and cover $\operatorname{supp} b$, and

$$b(V) = \sum_i \big(\tau - \delta_G(W_i)\big).$$

Every feasible b' must pay at least these deficits on the disjoint W_i, so b has minimum total mass. Write $B_0 := b(V)$.

Pairing the star. Now keep the star degrees fixed and only re-pair incidences. By Equation (259), every low paired cut stores its missing capacity in internal augmentation edges; we move that capacity across low cuts.

Assume first that $b(V)$ is even. Choose an initial pairing F for which $H = G + F$ is connected. If $C_1, \dots, C_k$ are the components of G, then $b(C_i) \ge \tau \ge 2$; for $k > 1$, reserve two incidences per component, connect the components in a cycle (two parallel links when $k = 2$), and pair the remaining incidences arbitrarily, allowing temporary loops. Thus H is connected and $\lambda(H) \ge 1$.

Every operation preserves $\deg_F = b$ and Equation (259). For a d-atom A of the current graph H, Lemma 476 gives the first reserve below, while $\tau \le f_b(A) = \delta_H(A) + 2w_F(E[A]) \le d - 1 + 2w_F(E[A])$ gives the second:

$$\delta_{H[A]}(U) \ge \frac{d+1}{2} \quad (\varnothing \ne U \subsetneq A), \qquad 2w_F(E[A]) \ge \tau - d + 1. \tag{267}$$

The first reserve protects cuts that split an atom; the second is movable augmentation capacity.

Bulk repair. Suppose $\lambda(H) \ge L < \tau$ and put

$$q := 2\left\lfloor \frac{\min\{L, \tau - L\}}{18} \right\rfloor, \qquad d := L + 3q. \tag{268}$$

If $q = 0$, use the unit repair below. Otherwise

$$q \ge \frac{\min\{L, \tau - L\}}{18}, \qquad d \le \frac{4}{3}L, \qquad \tau - d + 1 \ge 2q, \qquad d + 1 - 2q = L + q + 1. \tag{269}$$

Find the d-atoms of H. If none exists, then $\lambda(H) \ge d > L + q$. Otherwise every atom contains at least q internal augmentation edges by Equations (267) and (269).

Arrange the atoms cyclically. From each atom delete q internal augmentation edges and assign $q/2$ of these *edges* to each cyclic neighbor. Each bundle exposes q incidences toward that neighbor; pair the two bundles along the cycle link to create q new augmentation edges. For two atoms use two parallel links. Crucially, the two endpoints of every deleted edge go to the same neighbor. Thus a cut splitting one atom is protected locally, while a cut separating whole atoms must cross a cycle link.

Lemma 480 (Cyclic bulk repair)**.** *One cyclic exchange raises the connectivity from at least L to at least $L + q$.*

Proof. If a cut X splits at least two atoms, then deleting the q chosen internal edges can reduce the internal crossing contribution of each split atom by at most q. By Equation (267), each therefore still contributes at least $(d+1)/2 - q$, so two split atoms alone contribute $d + 1 - 2q = L + q + 1$.

If X splits at most one atom, the exchange cannot decrease it. Deleted edges in whole atoms did not cross X; in the split atom, a deleted crossing edge has endpoints on opposite shores and both are reconnected to the same whole neighbor, so exactly one replacement edge crosses. A deleted noncrossing edge can only add capacity.

If $\delta_H(X) < L + q < d$, then both X and its complement are d-deficient and hence each contains a d-atom. Since X splits at most one atom, each shore contains a whole atom. Remove the unique split atom from the

atom-cycle if necessary; the remaining cycle or path is connected and has whole atoms on both shores, so some whole-to-whole cycle link crosses X and contributes q new edges. Since $\delta_H(X) \geq L$,

$$\delta_{H^+}(X) \geq \delta_H(X) + q \geq L + q.$$

□

Since $q \geq \min\{L, \tau - L\}/18$, there are $O(\log \tau)$ bulk phases.

Unit repair. When $\min\{L, \tau - L\} < 18$, at most 17 unit increases are needed before reaching τ or returning to the bulk regime. For one increase, delete one internal augmentation edge from each $(L+1)$-atom and rematch the exposed endpoints.

Let $A_1, \ldots, A_r$ be the $(L+1)$-atoms. If none exists, level $L+1$ is already certified. By Equation (267), every atom contains an internal augmentation edge; delete one from each and call the resulting graph R.

Deletion preserves level L. Consider any cut. If it splits at least two atoms, then by Equation (267) each split atom still contributes at least $L/2$ after one deletion. If it splits exactly one atom, it loses at most one edge, while before deletion its value was at least $L+1$: an $(L+1)$-atom cannot cross a minimum L-cut C. Indeed, if $A \cup C \neq V$, submodularity gives $\delta(A) + \delta(C) \geq \delta(A \cap C) + \delta(A \cup C) \geq L + 1 + \delta(C)$, contradicting $\delta(A) \leq L$; if $A \cup C = V$, posimodularity on the two differences gives the same contradiction. A cut splitting no atom does not decrease. Hence $\lambda(R) \geq L$.

Call the $2r$ exposed incidences *labels*; a pair x, y is *bad* if some L-cut of R contains exactly those labels.

Lemma 481 (Two bad partners)**.** *Every label has at most two bad partners.*

Proof. Suppose x had bad partners y_1, y_2, y_3, witnessed by L-cuts X_1, X_2, X_3. In particular no additional label is colocated with x. The union $X_1 \cup X_2$ is proper because it omits y_3; since both cuts have value L and $\lambda(R) = L$, submodularity forces both $X_1 \cap X_2$ and $X_1 \cup X_2$ to have value L. Thus there is an L-cut containing x and no other label. Choose an inclusion-maximal such shore S.

For each i, the union $S \cup X_i$ is proper because it omits another y_j; uncrossing therefore lets us replace X_i by $S \cup X_i$ and assume $S \subseteq X_i$ without changing its two labels. Then $X_i \cap X_j$ is an x-only L-cut containing S, so maximality gives $X_i \cap X_j = S$. Thus $B_i := X_i \setminus S$ are nonempty and disjoint. Since both S and $S \cup B_i$ are minimum cuts,

$$w_R(S, B_i) = \frac{\delta_R(B_i)}{2} \geq \frac{L}{2}.$$

The three disjoint B_i would contribute at least $3L/2$ to $\delta_R(S) = L$, a contradiction. □

Choose a uniform random perfect matching M of the labels and let s be the number of matched bad pairs. By Lemma 481, there are at most $2r$ bad pairs, while every pair appears in M with probability $1/(2r-1)$, so

$$\mathbb{E}\, s \leq \frac{2r}{2r-1}. \tag{270}$$

Let r' be the number of surviving $(L+1)$-atoms, or 0 if the level is certified. For a survivor A, integrality and $\lambda(R) \geq L$ imply $\delta_R(A) = L$ and no matching edge crosses A; hence its labels are a union of matched pairs. It contains at least two labels: otherwise its cut was unchanged from H, so the resulting old L-cut contained an $(L+1)$-atom and therefore that atom's two exposed labels.

If a survivors contain exactly two labels, those two labels must be matched to each other and are bad by definition, so $a \leq s$. Every other survivor contains at least four labels. Since distinct atoms are disjoint,

$$2a + 4(r' - a) \leq 2r, \qquad r' \leq \frac{r}{2} + \frac{s}{2}.$$

Therefore

$$\mathbb{E}[r' \mid r] \leq \frac{r}{2} + \frac{r}{2r-1} \leq \frac{5}{6} r \qquad (r \geq 2). \tag{271}$$

If another round is needed, an L-cut remains and each shore contains a current $(L+1)$-atom, so $r' \geq 2$ and the contraction estimate continues to apply. Iterating Equation (271) gives $\mathbb{E}\, r_t \leq (5/6)^t r_0$; after a sufficiently large constant times $\log n$ rounds, Markov's inequality implies $r_t = 0$ with high probability. Only $O(1)$ levels use unit repair.

Aggregate exposed incidence mass by vertex before each exchange, so every round adds only $O(n)$ supported weighted edges and the maintained support is $m + O(n(\log \tau + \log n))$. With Equation (263), $O(\log \tau)$ star/bulk depths, and $O(\log n)$ unit rounds, the total time is $\widetilde{O}(m+n)$; when $\log \tau = O(\log n)$ the displayed costs give

$$O(m \log^4 n + n \log^5 n). \tag{272}$$

Inverse-polynomial sparsifier failure probabilities and a sufficiently large constant in the $O(\log n)$ unit rounds give the high-probability guarantee.

Optimality and splitting off. If B_0 is odd, add one arbitrary incidence; the star now has even mass $2\lceil B_0/2 \rceil$ and the pairing procedure applies. Conversely every feasible augmentation F^* induces the feasible star $b^*(v) = \deg_{F^*}(v)$, so

$$2|F^*| = b^*(V) \geq B_0.$$

Thus ours is optimal. If temporary loops remain, discard them; this preserves every cut. The resulting augmentation is still feasible, so the same lower bound forces it to contain exactly $\lceil B_0/2 \rceil$ edges.

Theorem 482 (Connectivity augmentation)**.** *With high probability, a minimum-cardinality integral augmentation making G τ-edge-connected can be found in $\widetilde{O}(m+n)$ time; under $\log \tau = O(\log n)$ the explicit bound is $O(m \log^4 n + n \log^5 n)$.*

The same pairing gives a complete splitting-off sequence. If an auxiliary vertex s has $b(v)$ parallel edges sv and $P \subseteq F$ is the set of pairs already split, then every proper $X \subsetneq V$ satisfies

$$\delta_{\text{partial}}(X) = \delta_{G+F}(X) + 2w_{F \setminus P}(E[X]) \geq \tau. \tag{273}$$

An unsplit pair contributes two star edges when both endpoints lie in X, and one edge in either representation when exactly one does; summing gives Equation (273). Hence every prefix is admissible, with the same running time.

12.11.2 All-Pairs Connectivity

Let

$$G = (V, E)$$

be a directed graph with

$$m = |E|.$$

For vertices s, t, let

$$\lambda(s,t)$$

be the maximum number of pairwise edge-disjoint directed s–t paths.

There is a direct algebraic representation of all these values at once.

A Walk Generating Matrix. Index rows and columns by directed edges.

For every composable pair

$$e, f \in E, \qquad \text{head}(e) = \text{tail}(f),$$

introduce a variable

$$x_{ef}.$$

Define

$$X \in \mathbb{F}[x]^{E \times E}$$

by

$$X[e,f] = \begin{cases} x_{ef}, & \text{head}(e) = \text{tail}(f), \\ 0, & \text{otherwise.} \end{cases}$$

Work first over the formal power-series ring. Then

$$\Gamma = (I - X)^{-1} = I + X + X^2 + \cdots .$$

If

$$W = (e_1, \ldots, e_\ell)$$

is a directed walk, define

$$\xi(W) = \prod_{j=1}^{\ell-1} x_{e_j, e_{j+1}}.$$

Lemma 483 (Walk Enumeration)**.** *For edges e, f,*

$$\Gamma[e,f] = \sum_{W : e \rightsquigarrow f} \xi(W),$$

where the sum ranges over directed walks beginning with e and ending with f.

Proof. The entry

$$X^j[e,f]$$

is exactly the sum of the monomials of walks consisting of $j+1$ edges, starting at e and ending at f. Sum over $j \geq 0$. □

Determinants Enumerate Edge-Disjoint Walks. Let

$$S = \{e_1, \ldots, e_r\}, \qquad T = \{f_1, \ldots, f_r\}$$

be equally sized edge sets.

Expand

$$\det \Gamma[S,T].$$

Each term chooses

- a permutation $\sigma \in S_r$;
- for every i, a walk from e_i to $f_{\sigma(i)}$.

Thus it is a signed generating function for families of walks.

We now work in characteristic 2.

Lemma 484 (Walk Cancellation)**.** *Over characteristic 2,*

$$\det \Gamma[S,T]$$

is the generating function of pairwise edge-disjoint walk families connecting S to T. In particular,

$$\det \Gamma[S,T] \not\equiv 0$$

if and only if there exist r pairwise edge-disjoint directed paths joining the starting edges in S to the ending edges in T in some pairing.

Proof. Consider a family in which two walks share an edge. Fix an ordering of walks and positions, and choose canonically the first shared edge

$$h$$

and the first pair of walks containing it.

Write the two walks as

$$P = P^- h P^+, \qquad Q = Q^- h Q^+.$$

Swap the suffixes:

$$P' = P^- h Q^+, \qquad Q' = Q^- h P^+.$$

The endpoint permutation is composed with a transposition. More importantly, the multiset of transition variables

$$x_{ef}$$

is unchanged, so the two families have exactly the same monomial.

The operation is an involution. In characteristic 2, the two identical contributions cancel.

Thus only pairwise edge-disjoint families remain.

For an edge-disjoint family, its transition monomial together with its starting edges reconstructs every walk, so distinct surviving families have distinct monomials and cannot cancel.

Finally, any edge-disjoint walk can have directed cycles deleted without creating an edge intersection, giving edge-disjoint paths. □

Connectivity is Matrix Rank. Let

$$E^{\text{out}}(s) = \{e : \text{tail}(e) = s\}$$

and

$$E^{\text{in}}(t) = \{e : \text{head}(e) = t\}.$$

Theorem 485 (Connectivity via Rank)**.** *Over the field of rational functions,*

$$\lambda(s,t) = \operatorname{rank} \Gamma[E^{\text{out}}(s), E^{\text{in}}(t)].$$

Proof. A matrix has rank at least r exactly when it has a nonzero $r \times r$ minor.

By Lemma 484, such a minor is nonzero exactly when there are r pairwise edge-disjoint directed s–t paths.

The largest possible r is therefore

$$\lambda(s,t).$$

□

Thus all-pairs connectivity is encoded by one matrix:

$$(I - X)^{-1}.$$

Random Evaluation. We cannot manipulate the rational functions explicitly. Instead choose a sufficiently large finite field

$$\mathbb{F}_{2^q}, \qquad q = \Theta(\log n),$$

and substitute independent uniformly random field elements for all variables x_{ef}.

Let

$$A$$

be the resulting numerical matrix.

By Schwartz–Zippel, with high probability,

- $I - A$ is invertible;

- every minor relevant to any pair (s,t) is zero after evaluation exactly when it was identically zero.

Hence simultaneously for every s,t,

$$\lambda(s,t) = \operatorname{rank}\left((I-A)^{-1}[E^{\mathrm{out}}(s), E^{\mathrm{in}}(t)]\right)$$

with high probability.

Algorithm 65 All-Pairs Directed Edge Connectivity

```
Choose random transition weights A[e, f] ∈ F_{2^q} whenever head(e) = tail(f)
M ← (I − A)^{-1}
for s, t ∈ V do
    λ(s, t) ← rank M[E^out(s), E^in(t)]
return all λ(s, t)
```

Theorem 486. *Algorithm 65 computes every directed edge-connectivity with high probability in*

$$\widetilde{O}(m^{\omega})$$

time.

Proof. The matrix inversion costs

$$O(m^{\omega})$$

field operations.

The rank of an $a \times b$ matrix can be computed in

$$O(ab^{\omega-1})$$

operations after exchanging a, b if necessary. Summing these costs over all pairs (s,t) and using

$$\sum_s \deg^+(s) = \sum_t \deg^-(t) = m$$

is bounded by

$$O(m^{\omega}).$$

The field operations cost only polylogarithmic time. □

Bounded All-Pairs Connectivity. Suppose now that we only want

$$\lambda_k(s,t) = \min\{k, \lambda(s,t)\}$$

for every pair s,t.

The algorithm above uses an $m \times m$ inverse. For small k, this contains much more information than necessary. We can compress the transition matrix to rank $O(kn)$ while preserving all minors of size at most k.

There are two steps:

1. modify the graph so every queried source has outdegree k and every queried target has indegree k;
2. factor the edge-transition matrix through only k copies of each vertex.

Degree Reduction Construct a graph G' as follows. For every original vertex v, create two additional vertices v_{in} and v_{out}.

Replace every original edge

$$(u, v)$$

by

$$(u_{\text{out}}, v_{\text{in}}).$$

Also add k parallel copies of

$$v \to v_{\text{out}} \qquad \text{and} \qquad v_{\text{in}} \to v.$$

Thus every original vertex has exactly k outgoing edges and exactly k incoming edges in G'.

The new graph has

$$n' = 3n, \qquad m' = m + 2kn.$$

Lemma 487 (Degree Reduction)**.** *For original vertices s, t,*

$$\lambda_{G'}(s, t) = \min\{k, \lambda_G(s, t)\}.$$

Proof. Any directed s–t path in G' projects to a directed s–t path in G by retaining only the original vertices. Furthermore, edge-disjoint paths in G' project to edge-disjoint paths in G, because every transformed original edge $u_{\text{out}} \to v_{\text{in}}$ corresponds to a unique original edge $u \to v$.

Hence

$$\lambda_{G'}(s, t) \le \lambda_G(s, t).$$

Also s has only k outgoing edges in G', so

$$\lambda_{G'}(s, t) \le k.$$

For the reverse inequality, let

$$r = \min\{k, \lambda_G(s, t)\},$$

and choose r edge-disjoint s–t paths $P_1, \ldots, P_r$ in G.

Lift P_i to G' by replacing every original edge $u \to v$ by

$$u \to u_{\text{out}} \to v_{\text{in}} \to v,$$

using the i-th parallel copy of each gadget edge $u \to u_{\text{out}}$ and $v_{\text{in}} \to v$.

The transformed original edges remain disjoint because the P_i were edge-disjoint, and the gadget edges are disjoint because the paths use different copy indices i. Thus the lifted paths are edge-disjoint in G', proving

$$\lambda_{G'}(s, t) \ge r.$$

□

A Low-Rank Transition Matrix We now apply the walk-generating construction to G'.

Instead of giving every composable pair of edges e, f an independent variable x_{ef}, factor this transition through one of k copies of their common vertex.

For every edge e, vertex v, and $j \in [k]$, introduce variables $y_{e,j}$ and $z_{j,e}$. Define

$$Y \in \mathbb{F}[y]^{E' \times (V' \times [k])}$$

by

$$Y[e,(v,j)] = \begin{cases} y_{e,j}, & \text{head}(e) = v, \\ 0, & \text{otherwise}, \end{cases}$$

and define

$$Z \in \mathbb{F}[z]^{(V' \times [k]) \times E'}$$

by

$$Z[(v,j),f] = \begin{cases} z_{j,f}, & \text{tail}(f) = v, \\ 0, & \text{otherwise}. \end{cases}$$

Then

$$(YZ)[e,f] = \begin{cases} \displaystyle\sum_{j=1}^{k} y_{e,j} z_{j,f}, & \text{head}(e) = \text{tail}(f), \\ 0, & \text{otherwise}. \end{cases}$$

Thus YZ is obtained from the old transition matrix X by the substitution

$$x_{ef} \longmapsto \sum_{j=1}^{k} y_{e,j} z_{j,f}.$$

The advantage is that

$$Y : \mathbb{F}^{kn'} \to \mathbb{F}^{m'}, \qquad Z : \mathbb{F}^{m'} \to \mathbb{F}^{kn'},$$

so

$$\text{rank}(YZ) \le kn' = O(kn).$$

Lemma 488 (Low-Rank Path Detection)**.** *Let $S, T \subseteq E'$ have*

$$|S| = |T| = r \le k.$$

Then

$$\det(I - YZ)^{-1}[S,T] \not\equiv 0$$

if and only if there are r pairwise edge-disjoint directed paths joining S to T.

Proof. Start with the exact generating function

$$\det(I - X)^{-1}[S,T].$$

By Lemma 484, this is nonzero exactly when there are r edge-disjoint paths from S to T.

If no such paths exist, this formal power series is identically zero. Substituting

$$x_{ef} = \sum_{j=1}^{k} y_{e,j} z_{j,f}$$

cannot turn the zero series into a nonzero one. Since this substitution sends X to YZ, we obtain

$$\det(I - YZ)^{-1}[S, T] \equiv 0.$$

Conversely, suppose

$$P_1, \ldots, P_r$$

are edge-disjoint paths from S to T. Since $r \leq k$, assign a different label i to each path P_i.
If

$$P_i = (e_{i,1}, e_{i,2}, \ldots, e_{i,\ell_i}),$$

then in every transition $e_{i,a} \to e_{i,a+1}$, choose the $j = i$ term from

$$\sum_{j=1}^{k} y_{e_{i,a},j} z_{j,e_{i,a+1}}.$$

This produces the monomial

$$\mu = \prod_{i=1}^{r} \prod_{a=1}^{\ell_i - 1} y_{e_{i,a},i} z_{i,e_{i,a+1}}.$$

Because the paths are edge-disjoint and use distinct labels, this monomial records the edges belonging to each individual path. The starting and ending edges are fixed by S and T, so the corresponding path family is recovered uniquely. Hence μ cannot be cancelled by another surviving family.

Therefore its coefficient is nonzero, and

$$\det(I - YZ)^{-1}[S, T] \not\equiv 0.$$

□

Reducing the Matrix Inversion Now use the inverse identity

$$(I - YZ)^{-1} = I + Y(I - ZY)^{-1}Z$$

from Theorem 24.

The left side is an $m' \times m'$ matrix, but the only matrix we actually need to invert on the right is

$$I - ZY,$$

whose dimension is

$$kn' = 3kn.$$

This is the source of the speedup.

Random Evaluation Choose a sufficiently large field $\mathbb{F}_{2^q}$, with $q = \Theta(\log n)$, and independently evaluate all y- and z-variables at uniformly random field elements.

Let the resulting numerical matrices be

$$B \in \mathbb{F}^{m' \times kn'}, \qquad C \in \mathbb{F}^{kn' \times m'}.$$

Let E^{out} be the union of the k edges leaving each *original* vertex, and let E^{in} be the union of the k edges entering each original vertex. Define

$$\widetilde{B} = B[E^{\text{out}}, \cdot], \qquad \widetilde{C} = C[\cdot, E^{\text{in}}],$$

and compute

$$M = \widetilde{B}(I - CB)^{-1}\widetilde{C}.$$

For original vertices s, t, the sets $E^{\text{out}}(s)$ and $E^{\text{in}}(t)$ are disjoint, so the identity term in

$$(I - BC)^{-1} = I + B(I - CB)^{-1}C$$

vanishes on the corresponding submatrix. Hence

$$(I - BC)^{-1}[E^{\text{out}}(s), E^{\text{in}}(t)] = M[E^{\text{out}}(s), E^{\text{in}}(t)].$$

By Lemma 488 and Schwartz–Zippel, with high probability every minor of size at most k needed below has the same zero/nonzero status after the random evaluation as it had formally. Therefore, simultaneously for every original s, t,

$$\operatorname{rank} M[E^{\text{out}}(s), E^{\text{in}}(t)] = \min\{k, \lambda_G(s, t)\}.$$

Algorithm 66 k-Bounded All-Pairs Directed Edge Connectivity

```
Construct the degree-reduced graph G'
Randomly evaluate the low-rank matrices Y, Z to B, C
M ← B̃(I − CB)^{-1}C̃
for s, t ∈ V do
    λ_k(s, t) ← rank M[E^out(s), E^in(t)]
return all λ_k(s, t)
```

Theorem 489 (Bounded All-Pairs Connectivity)**.** *For a simple directed graph on n vertices and any $1 \le k < n$, all values*

$$\min\{k, \lambda(s, t)\}$$

can be computed with high probability in

$$\widetilde{O}((kn)^{\omega})$$

time.

Proof. Correctness follows from Lemmas 487 and 488 and the random-evaluation argument above.

It remains to bound the running time.

The matrix CB has dimension

$$kn' \times kn' = 3kn \times 3kn.$$

Its entries have the form

$$(CB)[(u,i),(v,j)] = \sum_{\substack{e \in E' \\ \mathrm{tail}(e)=u \\ \mathrm{head}(e)=v}} c_{i,e} b_{e,j}.$$

Because of the special structure of G', this matrix can be assembled in $O((kn)^2)$ field operations: between non-original gadget vertices there is at most one transformed original edge, while the only parallel classes have exactly k edges.

We can therefore compute

$$(I - CB)^{-1}$$

in

$$O((kn)^\omega)$$

field operations. Multiplying on the left by $\widetilde{B}$ and on the right by $\widetilde{C}$ has the same asymptotic cost.

Finally, each queried block

$$M[E^{\mathrm{out}}(s), E^{\mathrm{in}}(t)]$$

is only $k \times k$. Computing its rank costs $O(k^\omega)$ field operations, so all n^2 ranks cost

$$O(k^\omega n^2) \le O((kn)^\omega).$$

All field elements have $O(\log n)$ bits, giving the stated $\widetilde{O}((kn)^\omega)$ bit complexity. □

12.12 Hamiltonicity

12.12.1 Undirected Hamiltonicity

Let $G = (V, E)$ be a simple undirected graph with $n \ge 3$ vertices. Fix a root s. We can generate closed walks

$$W = (v_0 = s, v_1, \ldots, v_{n-1}, v_n = s)$$

by dynamic programming, allowing repeated vertices but forbidding an intermediate return to s. Such a walk is a Hamiltonian cycle exactly when $v_0, \ldots, v_{n-1}$ are distinct.

Two simple tests. The direct test assigns a variable y_v to each vertex and uses the monomial $\prod_{i=0}^{n-1} y_{v_i}$, counting the root only once. Distinctness is exactly multilinearity, but the degree is n, so the preceding detector costs 2^n evaluations.

Undirectedness offers another approach. Consider the first repeated vertex before the final return to s. The intervening closed segment has distinct internal vertices. If it has at least three edges, reversing it gives a different walk with the same undirected-edge multiset. In characteristic 2, weights that preserve this reversal make the two contributions cancel. A two-edge backtrack u, v, u is its own reversal, but traverses the same edge twice, so an edge variable detects it by a square.

Thus we could track vertices and reject repetitions directly, or track edges and cancel the longer repeated-vertex segments. Either full test uses n factors. We will mix them so that some edges need not be tracked at all. We will also arrange auxiliary weights so that our desired cycles survive the cancellation.

Use a different test in each part. Choose a partition $V = A \sqcup B$ with $s \in A$. Track visits to vertices of A by variables y_v, and traversals of edges inside B by variables z_e. A repeated vertex in A produces a square, so after the multilinearity test only repeated vertices in B need attention. Their closed segments of length at least three will cancel by reversal.

For a two-edge backtrack u, v, u with $u \in B$, a middle vertex $v \in B$ squares the tracked variable $z_{\{u,v\}}$. If $v \in A$, neither test produces a square, so we exclude this one pattern locally:

$$u, v, u \qquad \text{with } u \in B,\ v \in A.$$

Call a walk *legitimate* if it contains no such consecutive triple. This excludes no Hamiltonian cycle and requires remembering only the last two vertices. Thus vertices in A, edges inside B, and one local restriction handle all the obstructions to reversal cancellation.

Where the saving comes from. Put $E_B = E(G[B])$. The tracked monomial of a walk is

$$M_W = \prod_{\substack{0 \le i < n \\ v_i \in A}} y_{v_i} \prod_{\substack{1 \le i \le n \\ v_{i-1}, v_i \in B}} z_{\{v_{i-1}, v_i\}}.$$

Products count occurrences, not just distinct vertices or edges. There is no tracked factor for a vertex of B or an edge crossing between A and B.

On a Hamiltonian cycle, break the B vertices into maximal consecutive runs. Since the root belongs to A, every run has an A vertex at both ends. A run with r vertices contributes only $r - 1$ internal edges, so replacing its vertex tests by edge tests saves one factor. If there are t runs, then

$$\deg M_W = |A| + |E(W) \cap E_B| = n - t. \tag{274}$$

The crossing edges let us move between the two kinds of tests without paying an additional tracked factor. We now make the cancellation argument precise.

Distinguish desired cycles. The tracked monomials alone do not distinguish different Hamiltonian cycles. Introduce auxiliary edge variables x_{uv}: away from s, use the same variable in both directions, $x_{uv} = x_{vu}$; on edges incident to s, use distinct variables x_{sv} and x_{vs}. All variables are otherwise distinct. Set

$$X_W = \prod_{i=1}^{n} x_{v_{i-1} v_i}.$$

An internal closed segment never visits s, so reversing it preserves X_W. Reversing an entire Hamiltonian cycle instead exchanges the incoming and outgoing root edges and changes X_W. This small asymmetry prevents the desired cycles from canceling with their own reversals.

Work over a field $\mathbb{F}$ of characteristic 2. For a chosen degree d, define

$$F_d(x, y, z) = \sum_{\substack{W \text{ legitimate closed } n\text{-edge walk} \\ v_0 = v_n = s,\ v_i \neq s\ (0 < i < n) \\ \deg M_W = d}} X_W M_W.$$

We test multilinearity only in (y, z), not in x. Restricting to degree d is important: the detector for degree d requires degree at most d in its tracked variables. A counter in the dynamic program will perform this restriction without expanding the polynomial.

Lemma 490 (Hybrid cancellation)**.** *The monomials of F_d that are multilinear in (y, z) are precisely the monomials of the rooted, oriented Hamiltonian cycles with d tracked factors. Each has coefficient* 1.

Proof. First consider a legitimate walk whose tracked monomial is multilinear but which is not Hamiltonian. Choose the smallest $j < n$ such that $v_j = v_i$ for some $i < j$. All vertices before j are distinct, so i is unique. Also $i > 0$, because intermediate visits to s are forbidden. The repeated vertex lies in B, or else its y variable would be squared.

The segment $(v_i, \ldots, v_j = v_i)$ cannot have two edges: a middle vertex in A would violate legitimacy, while a middle vertex in B would square an internal-edge variable. There are no loops, so the segment has at least three edges.

Reverse this segment. Its internal vertices are distinct, so the walk changes. Its vertex and undirected-edge multisets do not change, and the segment avoids s, so the full monomial $X_W M_W$ is preserved. The first repetition is still at the same positions i, j, making this operation an involution.

Legitimacy is preserved as well. Triples inside the segment are reversed, and the forbidden pattern is invariant under reversal. Triples outside are unchanged. A changed triple crossing a boundary of the segment has its middle vertex at $v_i = v_j \in B$, whereas a forbidden triple has its middle vertex in A. Thus these nonsimple walks cancel in pairs in characteristic 2.

A Hamiltonian cycle, on the other hand, is legitimate and has a multilinear tracked monomial. Its auxiliary monomial records its undirected edge set and its outgoing edge at s. These determine its entire traversal: after leaving s, the next unused edge is forced at every vertex. No other walk has the same auxiliary monomial, so its coefficient is 1. All remaining walks have a squared tracked variable and cannot contribute a multilinear monomial. □

Evaluate walks. For a numerical assignment of the variables, F_d has a polynomial-time evaluation. Store the total weight of partial walks by their length, their last two vertices, and their tracked degree. These records suffice because legitimacy is a local condition.

For a step $v \to w$ with $w \neq s$, its tracked factor $g(v, w)$ and its tracked-degree increment $c(v, w)$ are

$$(g(v,w), c(v,w)) = \begin{cases} (y_w, 1), & w \in A, \\ (z_{\{v,w\}}, 1), & v, w \in B, \\ (1, 0), & v \in A,\ w \in B. \end{cases}$$

The full step weight is $x_{vw} g(v, w)$. The symbol $\perp \notin V$ marks the absent predecessor at the root.

Algorithm 67 Evaluating the hybrid walk polynomial

```
function WalkValue(G, s, A, B, d, x, y, z)
    Set all entries of D to zero; set D(⊥, s, 1) ← y_s
    for ℓ = 1, ..., n − 1 do
        Set all entries of D′ to zero
        for each (u, v, j) with 0 ≤ j ≤ d and D(u, v, j) ≠ 0 do
            for each w ∈ N(v) \ {s} do
                if not (u = w ∈ B and v ∈ A) then
                    j′ ← j + c(v, w)
                    if j′ ≤ d then
                        D′(v, w, j′) ← D′(v, w, j′) + D(u, v, j) x_vw g(v, w)
        D ← D′
    return Σ_{v∈N(s)} Σ_u D(u, v, d) x_vs
```

At layer ℓ, the entry $D(u, v, j)$ sums the weights of legitimate ℓ-vertex prefixes with the indicated last vertices and degree. Initialization counts s once; each transition appends exactly one vertex and edge. After $n - 1$ extensions, the last line closes the walk. The closing edge adds no tracked factor because $s \in A$ and has already been counted. It cannot create a forbidden triple ending at s, since the ends of a forbidden triple belong to B. This proves the invariant and the evaluation formula.

There are $O(dn^2)$ states per layer and at most n extensions per state. Thus one evaluation uses $O(dn^4)$ field operations and $O(dn^2)$ field elements of storage. In particular, it has polynomial cost.

The multilinear test. Regard the combined variables (y, z) as one vector q, indexed by the disjoint union $A \sqcup E_B$. By Theorem 693, the polynomial

$$H_d(x, \xi_1, \dots, \xi_d) = \sum_{J \subseteq [d]} F_d \left(x, \sum_{j \in J} \xi_j \right)$$

is nonzero exactly when F_d has a monomial multilinear in q. Here $\xi_1, \dots, \xi_d$ are vectors with those same coordinates. The cancellation lemma therefore makes this a test for Hamiltonian cycles with the chosen tracked degree.

The degree of H_d is at most $n + d \le 2n$: every walk contributes n auxiliary edge factors and d tracked factors. Choose a characteristic-2 field with $4n \le |\mathbb{F}| < 8n$ and assign all auxiliary variables and all coordinates of the ξ_j independently and uniformly. If $H_d \not\equiv 0$, Schwartz–Zippel gives $\Pr[H_d \neq 0] \ge 1/2$. If it is identically zero, every evaluation is zero. Computing this test takes 2^d calls to Algorithm 67, performed sequentially with the *same* auxiliary assignment and direction vectors.

How much degree can a random partition save? Put s in A and place every other vertex independently in A or B with equal probability. Fix a Hamiltonian cycle. Reading its partition labels from s back to s gives a word that begins and ends with A, with n boundaries between consecutive labels. There are 2^{n-1} equally likely words.

Each run of B has exactly two changing boundaries, one entering and one leaving. Conversely, choosing any $2t$ of the n boundaries uniquely determines a word with t runs: start with A and switch labels exactly at the chosen boundaries. Therefore

$$\pi_t := \Pr[\text{the fixed cycle has } t \text{ runs of } B] = \frac{\binom{n}{2t}}{2^{n-1}}, \qquad 0 \le t \le \lfloor n/2 \rfloor.$$

Such a partition lets us use degree $d = n - t$. Each trial then costs $O^*(2^{n-t})$ and succeeds with probability at least $\pi_t/2$, where O^* suppresses factors polynomial in n. Thus the relevant cost is

$$\frac{2^{n-t}}{\pi_t} = \frac{2^{2n-1}}{2^t \binom{n}{2t}}.$$

More runs make the sieve cheaper, but a prescribed unusually large number of runs makes the partition harder to obtain. We choose t to maximize the denominator, balancing these two effects.

Algorithm 68 Hamiltonicity by a hybrid multilinear sieve

Input: Simple undirected graph $G = (V, E)$
Output: No false positives; false-negative probability at most $1/3$

if $n < 3$ **then**
  return No
Fix $s \in V$; choose a characteristic-2 field $\mathbb{F}$ with $4n \le |\mathbb{F}| < 8n$
Choose $t \in \arg\max_{0 \le j \le \lfloor n/2 \rfloor} 2^j \binom{n}{2j}$
$d \leftarrow n - t$, $\pi \leftarrow 2^{-(n-1)} \binom{n}{2t}$, $T \leftarrow \lceil 2 \log(3)/\pi \rceil$
for $i = 1, \dots, T$ **do**
  Put s in A; assign every other vertex independently and uniformly to A or B
  Choose independent uniform values for the distinct auxiliary variables x
  Choose independent uniform vectors $\xi_1, \dots, \xi_d \in \mathbb{F}^{A \sqcup E_B}$
  $R \leftarrow 0$
  for each $J \subseteq [d]$ **do**
    $(y, z) \leftarrow \sum_{j \in J} \xi_j$
    $R \leftarrow R + \textsc{WalkValue}(G, s, A, B, d, x, y, z)$
  if $R \neq 0$ **then**
    return Yes
return No

Theorem 491 (Hybrid Hamiltonicity sieve)**.** *Algorithm 68 decides Hamiltonicity in $O^*((4/(\sqrt{2}-1))^n) = O^*(1.657^n)$ time and polynomial space, with no false positives and false-negative probability at most $1/3$.*

Proof. If there is no Hamiltonian cycle, every tested polynomial is identically zero. Otherwise fix one cycle. Each independent trial succeeds with probability at least $\pi/2$, so the probability that all trials fail is at most $(1 - \pi/2)^T \le e^{-\pi T/2} \le 1/3$.

For the running time, the even part of the binomial theorem gives

$$\sum_{t=0}^{\lfloor n/2 \rfloor} 2^t \binom{n}{2t} = \frac{(1+\sqrt{2})^n + (1-\sqrt{2})^n}{2} = \Theta\big((1+\sqrt{2})^n\big).$$

The largest of these $O(n)$ summands is at least their average. Consequently the total cost is

$$O^*\left(\frac{2^{2n-1}}{\max_t 2^t \binom{n}{2t}} \right) = O^*\left(\left(\frac{4}{1+\sqrt{2}} \right)^n \right).$$

Since $4/(1+\sqrt{2}) = 4(\sqrt{2}-1)$, this is the claimed bound. The algorithm stores only one trial, its direction vectors, and two layers of the walk dynamic program at a time, so its space usage is polynomial. Field elements have $O(\log n)$ bits. □

12.12.2 Directed Hamiltonicity

The cancellation in the undirected algorithm used reversal of a closed segment. A directed graph has no corresponding involution. Instead, we will assign random algebraic weights to its arcs so that an existing Hamiltonian path is unlikely to disappear modulo two. The difficulty is to evaluate the resulting path sum without returning to 2^n time.

Work with a simple loopless directed graph (loops and parallel copies can be discarded for decision). Fix distinct vertices s, t and consider Hamiltonian paths from s to t. This also solves the cycle problem: split any vertex r into a source s inheriting its outgoing arcs and a sink t inheriting its incoming arcs. Hamiltonian cycles correspond to Hamiltonian s–t paths in the resulting $(n+1)$-vertex graph. In what follows, n denotes the number of vertices *after* this reduction, $I = V \setminus \{s, t\}$, and $N = |I| = n - 2$. There are no arcs entering s or leaving t.

Preventing parity cancellation. A plain parity count can vanish even when Hamiltonian paths exist. Independent variables on all arcs would prevent cancellation, but would leave too many distinct incoming weights for the fast determinant evaluation. We need only a small algebraic fingerprint for each arc. Give each arc uv a random quadratic $B_{uv}(T_v) \in \mathbb{F}_2[T_v]_{<3}$, with the *same variable for all arcs entering* v, and form

$$F(\mathbf{T}) = \sum_{P\colon s \rightsquigarrow t \text{ Hamiltonian}} \prod_{uv \in P} B_{uv}(T_v). \tag{275}$$

Every Hamiltonian path enters $v \neq s$ exactly once. Thus each path contributes at most one factor depending on T_v, and $\deg_{T_v} F \leq 2$. The basis $1, T_v, T_v^2$ gives three independent coefficients for all incoming arcs at v, but requires only one evaluation point at that vertex.

Choose the arc quadratics independently, with

$$\Pr[B_{uv} = 0] = \delta = \frac{1}{50}, \qquad \Pr[B_{uv} = B] = \rho = \frac{7}{50} \quad \text{for each nonzero } B \in \mathbb{F}_2[T]_{<3}. \tag{276}$$

The small probability of zero is chosen to balance two possible ways a first-edge expansion can vanish: its only viable branch might be zero, or two viable branches might cancel. Indeed, $\delta^2 + \rho^2 = \delta$.

To bound this cancellation, consider the expansion according to the first arc sv:

$$F = \sum_{v \in I\colon sv \in E} B_{sv}(T_v) g_v(\mathbf{T}), \tag{277}$$

where g_v counts Hamiltonian paths from v to t in $G - s$, with the same random arc quadratics. The crucial property is that g_v does *not* involve T_v: such a path starts at v and never enters it. This independence severely limits relations among the summands.

Lemma 492 (Independent blocks)**.** *Let $g_1, \dots, g_p$ be nonzero polynomials, with g_i independent of T_i. The polynomials $T_i^j g_i$, for $i \in [p]$ and $0 \leq j < d$, span a space of dimension at least $(d-1)p + 1$.*

Proof. Work over an infinite extension field, which does not change the rank. Choose an evaluation $\mathbf{a}$ with every $g_i(\mathbf{a}) \neq 0$. For a relation $\sum_i A_i(T_i) g_i = 0$, where $\deg A_i < d$, the values $u_i = A_i(a_i)$ satisfy the nontrivial equation $\sum_i g_i(\mathbf{a}) u_i = 0$. Moreover, these values determine the relation: if every $u_i = 0$, specializing all variables other than T_i to $\mathbf{a}$ leaves $A_i(T_i) g_i(\mathbf{a}) = 0$, so $A_i = 0$. Thus the relation space injects into a $(p-1)$-dimensional hyperplane. Its dimension is at most $p - 1$, giving the claim. □

Condition on all the quadratics except those leaving s, and suppose exactly $p \geq 1$ of the g_v in (277) are nonzero. The lemma says that the map from their $3p$ binary coefficients to F has kernel dimension at most $p - 1$. This kernel contains the all-zero tuple and at most $2^{p-1} - 1$ other tuples. Since each nonzero tuple has probability at most ρ^p, we get

$$\Pr[F = 0 \mid (g_v)_v] \leq \delta^p + (2^{p-1} - 1)\rho^p \leq \delta. \tag{278}$$

For $p = 1$ the bound is δ; for $p = 2$ it is $\delta^2 + \rho^2 = \delta$. For $p \geq 2$ the expression decreases with p, since $\rho < 1/3$.

If a Hamiltonian path exists, fix one and expose its first arc. By induction, its suffix polynomial is nonzero with probability at least $(1 - \delta)^{n-2}$. Conditional on that event, some g_v is nonzero, and (278) applies. Therefore

$$\Pr[F \not\equiv 0] \geq (1 - \delta)^{n-1} = (49/50)^{n-1}. \tag{279}$$

Choose a characteristic-two field K with $200n \leq |K| < 400n$, sample $a_v \in K$ independently for $v \neq s$, and give arc uv weight $w(uv) = B_{uv}(a_v)$. Put $H = \sum_{P: s \rightsquigarrow t \text{ Hamiltonian}} \prod_{e \in P} w(e) = F(\mathbf{a})$. Because $\deg F \leq 2(n-1)$, Schwartz–Zippel and (279) imply

$$\Pr[H \neq 0] \geq \frac{99}{100}(49/50)^{n-1} \tag{280}$$

on a YES instance; on a NO instance $H = 0$ identically. It remains to evaluate H in time below 2^n.

Replace paths by spanning trees. For a weighted directed graph D, let $L(D)$ be its column Laplacian: $L_{uv} = -w(uv)$ for $u \neq v$, and $L_{vv} = \sum_{u:\,uv\in E(D)} w(uv)$. Write $L_s(D)$ for the matrix with row and column s deleted. The directed Matrix–Tree Theorem expresses $\det L_s(D)$ as the weighted sum of spanning out-arborescences rooted at s.

For $X \subseteq I$, delete all original arcs leaving $I \setminus X$, obtaining G_X. We may also add *arbitrary* arcs leaving t; call the result G_X^+. In characteristic two,

$$H = \sum_{X\subseteq I} \det L_s(G_X^+). \tag{281}$$

Indeed, an out-arborescence with k leaves among the internal vertices occurs in exactly 2^k determinants: every nonleaf must belong to X, while each leaf may or may not. All terms with $k > 0$ cancel. If $k = 0$, only t can be a leaf, so the tree is an s–t Hamiltonian path and uses no added arc leaving t.

The identity still has 2^N determinants. The freedom to add arcs leaving t lets us force many of them to vanish in a form that can be tested *before computing their determinants*.

A test for zero determinants. Write $b_{uv} \in \mathbb{F}_2^3$ for the coefficients of B_{uv}, with $b_{uv} = 0$ for absent arcs. Independently choose $r_v \in \mathbb{F}_2^3$ for each $v \in I$ and add an arc $t \to v$ of weight $r_{v,0} + r_{v,1}a_v + r_{v,2}a_v^2$. For $X \subseteq I$ let $x_u = \mathbf{1}[u \in X]$ and define

$$h_v(x) = r_v + b_{sv} + \sum_{u\in I} x_u b_{uv} \in \mathbb{F}_2^3. \tag{282}$$

When $x_v = 0$, all off-diagonal entries of row v in $L_s(G_X^+)$ vanish. Its diagonal is the evaluation of the quadratic with coefficient vector $h_v(x)$, so the entire row is zero whenever $h_v(x) = 0$. We therefore need only the subsets satisfying

$$(x_v, h_v(x)) \neq 0 \in \mathbb{F}_2^4 \qquad \text{for every } v \in I. \tag{283}$$

For each fixed x, the $h_v(x)$ are independent uniform three-bit vectors, because of the r_v. An inactive vertex passes the test with probability $7/8$, while an active vertex passes automatically. Thus the expected number of admissible subsets is

$$\sum_{X\subseteq I} (7/8)^{|I\setminus X|} = (15/8)^N. \tag{284}$$

This gives the desired exponential base, but only as an expectation. We still need to *list* the admissible subsets without examining all 2^N possibilities.

A random equation replaces a nonzero test. For each $v \in I$, independently choose a uniformly random nonzero $c_v = (\alpha_v, \beta_v) \in \mathbb{F}_2 \times \mathbb{F}_2^3$ and impose

$$\alpha_v x_v + \langle \beta_v, h_v(x)\rangle = 1. \tag{285}$$

Since $h_v(x)$ is affine in x, these conditions form an affine system over $\mathbb{F}_2$. Every solution satisfies (283). Conversely, for any nonzero four-bit vector, exactly eight of the fifteen nonzero linear forms take value one. Therefore every fixed admissible x solves a random system with probability $(8/15)^N$.

Why is solving many such systems affordable? For a fixed choice of $c = (c_v)$, the *expected number of solutions is exactly one*. Indeed, if $\beta_v = 0$, then $\alpha_v = 1$ and the equation forces $x_v = 1$. For every other v, the bit $\langle \beta_v, r_v\rangle$ is uniform and independent. If there are k of these other vertices, exactly 2^k assignments obey the first kind of constraint, and each satisfies the remaining equations with probability 2^{-k}. Hence

$$\mathbb{E}_r |\{x : (285) \text{ holds}\}| = 1. \tag{286}$$

Take $M = \lceil 4n(15/8)^N \rceil$ independent systems, enumerate each solution space by Gaussian elimination, and collect the distinct solutions in S. A union bound shows that the probability of missing *any* admissible subset is at most

$$2^N \big(1 - (8/15)^N\big)^M \le 2^N e^{-4n}. \tag{287}$$

Outside this event, (281) gives $H = \sum_{x \in S} \det L_s(G_X^+)$, since every unreported determinant has a zero row. By (286), the expected total number of reported solutions is just M. The linear systems and reported determinants therefore take $O^*((15/8)^n)$ expected time.

Algorithm and running time. A single compressed polynomial survives with exponentially small probability, so we repeat the encoding $R = \lceil 4(50/49)^n \rceil$ times. Because (287) is already exponentially small, there is no need to repeat the determinant evaluator or take a majority. To enforce a worst-case time bound, keep a *global* count of reported solutions, and abort with NO if it exceeds $40RM$.

Algorithm 69 Directed Hamiltonicity by random quadratic labels

Input: A simple directed graph G
Output: A Hamiltonicity decision with error probability at most $1/3$

Split a vertex into s, t; let $n \leftarrow |V|$, $I \leftarrow V \setminus \{s, t\}$, $N \leftarrow |I|$
Choose a characteristic-two field K with $200n \le |K| < 400n$
$R \leftarrow \lceil 4(50/49)^n \rceil$, $M \leftarrow \lceil 4n(15/8)^N \rceil$, $Q \leftarrow 0$
for $j = 1, \ldots, R$ **do**
    Sample arc quadratics B_e by (276) and independent $a_v \in K$; set $w(uv) \leftarrow B_{uv}(a_v)$
    Sample independent $r_v \in \mathbb{F}_2^3$; form $h_v(x)$ by (282); set $S \leftarrow \varnothing$
    for $k = 1, \ldots, M$ **do**
        Sample independent nonzero $c_v = (\alpha_v, \beta_v) \in \mathbb{F}_2^4$
        Enumerate all x solving (285) by Gaussian elimination
        for each solution x **do**
            $S \leftarrow S \cup \{x\}$; $Q \leftarrow Q + 1$
            if $Q > 40RM$ **then**
                return NO
    $H \leftarrow \sum_{x \in S} \det L_s(G_X^+)$, using the current r, w
    if $H \neq 0$ **then**
        return YES
return NO

Theorem 493 (Directed Hamiltonicity)**.** *Algorithm 69 decides directed Hamiltonicity in* $O^*((375/196)^n) = O^*(1.9133^n)$ *time with error probability at most* $1/3$.

Proof. On a YES instance, (280) makes the probability that all R weighted path sums are zero at most $\exp(-\frac{99}{100} R(49/50)^{n-1}) < 1/40$. For either type of instance, the probability that any determinant enumeration misses an admissible subset is at most $R2^N e^{-4n} < 1/40$. Finally, (286) gives expected total reported solutions at most RM, so the global cutoff is reached with probability at most $1/40$ by Markov's inequality. Outside these events every determinant sum is exact, and some sum is nonzero precisely on a YES instance. The total error is below $3/40$. There are RM affine systems and at most $40RM$ reported solutions before the cutoff, each requiring polynomial work. Since $RM = O^*((50/49)^n (15/8)^n)$, the stated bound follows. Splitting one vertex changes the number of vertices by only one. □

The two exponential factors explain the choice of three coefficients: more coefficients reduce cancellation but make zero rows rarer. With d coefficients the same analysis gives deletion probability $1/(1 + (2^d - 1)^2)$ and determinant base $2 - 2^{-d}$; their combined base is minimized at $d = 3$. Uniformly choosing among the seven *nonzero* quadratics simplifies the first part further, giving an $O^*((245/128)^n) = O^*(1.9141^n)$ algorithm. The slight bias in (276) recovers 1.9133^n. The implementation above stores the distinct reported subsets and therefore uses exponential space.

12.13 Directed Minimum Spanning Tree

Let

$$G = (V, E)$$

be a directed graph with edge weights $w : E \to \mathbb{R}$, and fix a root $r \in V$.

An *r-arborescence* is a set of $n - 1$ edges such that every $v \neq r$ has exactly one incoming edge and every vertex is reachable from r.

We want a minimum-weight r-arborescence.

12.13.1 The Two Reductions

We first isolate the two transformations on which the algorithm is based.

Lemma 494 (Incoming-Edge Reduction)**.** *Let X be a non-root vertex of the current contracted graph, and let*

$$\delta = \min_{e \in \delta^-(X)} w(e).$$

Subtract δ from the weight of every edge entering X.

Then the minimum arborescence weight decreases by exactly δ.

Proof. Every rooted arborescence contains exactly one edge entering X. Thus every feasible solution loses exactly δ in weight. □

After the reduction, at least one incoming edge to X has weight zero.

Choose one such zero-weight edge for every non-root vertex. If the chosen edges are acyclic, they form an optimal arborescence.

Otherwise they contain a directed cycle.

Lemma 495 (Zero-Cycle Contraction)**.** *Suppose*

$$C = X_1 X_2 \cdots X_k X_1$$

is a directed cycle of chosen zero-weight edges. Contract C to one supervertex.

The minimum residual arborescence weight is unchanged.

Proof. Consider any arborescence of the original graph. At least one edge enters C from outside.

Suppose an entering edge enters X_i. Delete all other edges entering vertices of C and instead include every zero-weight cycle edge except the cycle edge entering X_i. This preserves connectivity and indegree one at every vertex of C and does not increase the weight.

Hence some optimum contains exactly one external edge entering C.

Such solutions correspond exactly to arborescences after contracting C: if the contracted solution enters the supervertex through an edge whose original head is X_i, expand C and add every zero cycle edge except the one entering X_i. The weights are identical. □

12.13.2 Meldable Incoming-Edge Heaps

For each current contracted component X, maintain a heap

$$H_X$$

containing the original edges whose heads lie in X.

An edge

$$e = (u, v)$$

stored in H_X may later become internal because its tail is also contracted into X. We do not eagerly delete such edges. Instead, whenever an internal edge reaches the minimum of the heap, we discard it.

Thus we need the following heap operations:

operation	meaning
$\mathsf{Min}(H)$	minimum reduced-cost edge
$\mathsf{Pop}(H)$	delete the minimum
$\mathsf{AddAll}(H, -\delta)$	subtract δ from every key
$\mathsf{Meld}(H_1, H_2)$	merge two incoming-edge heaps.

A skew heap with one lazy additive tag per heap node supports these operations in amortized $O(\log m)$ time, with AddAll taking $O(1)$ time.

When the minimum incoming edge of X has reduced weight δ, we perform

$$\mathsf{AddAll}(H_X, -\delta).$$

When a cycle is contracted,

$$H_C = \mathsf{Meld}\left(H_{X_1}, \ldots, H_{X_k}\right).$$

Edges that have become internal are discarded lazily from the top.

12.13.3 Detecting Cycles with DSU

Maintain the already chosen zero edges as an undirected forest in a disjoint set union structure D_{cyc}.

The important invariant is that before a current component X chooses its incoming edge, X is the unique vertex of its chosen-edge component with no chosen incoming edge. Thus the chosen edges in that component form an arborescence directed away from X.

Suppose the chosen minimum edge is

$$e = (Y, X).$$

If X and Y lie in different components of D_{cyc}, adding e cannot create a cycle, so we simply union them.

If they already lie in the same component, the unique chosen-edge path from X to Y is directed

$$X \rightsquigarrow Y.$$

Adding

$$Y \to X$$

therefore creates exactly one directed cycle.

Hence an ordinary DSU detects the cycles.

12.13.4 Maintaining Contractions

A second DSU,

$$D_{\text{ctr}},$$

represents the currently contracted vertices.

For every original vertex v,

$$D_{\text{ctr}}.\mathsf{find}(v)$$

is its present supervertex.

When a chosen edge

$$e = (u, v)$$

is inspected, its current tail is therefore

$$D_{\text{ctr}}.\mathsf{find}(u).$$

When a cycle is found, union all its vertices in D_{ctr} and meld their heaps.

We make D_{ctr} rollback-capable. Before contracting a cycle, record the current rollback time. This lets us expand all contractions in reverse order once the optimum weight has been found.

12.13.5 Fast Directed MST Algorithm

For a current component X, let

$$e_X$$

denote the zero reduced-cost incoming edge currently chosen for X.

The algorithm processes a component as follows.

Algorithm 70 Tarjan's Directed MST Algorithm

Input: Directed graph $G = (V, E)$ and root r
Output: Minimum r-arborescence, or None

```
put every edge (u, v) into the meldable heap H_v
initialize D_cyc
initialize rollback DSU D_ctr
answer ← 0
C ← ()                                                  ▷ contraction records
for each original vertex s ≠ r do
    X ← s
    loop
        while H_X ≠ ∅ and tail(Min(H_X)) is currently inside X do
            Pop(H_X)
        if H_X = ∅ then
            return None
        e_X ← Min(H_X)
        δ ← w_red(e_X)
        answer ← answer + δ
        AddAll(H_X, −δ)
        Y ← D_ctr.find(tail(e_X))
        if D_cyc.union(X, Y) succeeds then
            break
        record rollback time τ of D_ctr
        find the directed chosen-edge cycle containing X
        contract all its components into a new component Z
        meld all their incoming-edge heaps into H_Z
        append the contraction information and τ to C
        X ← Z
expand the contractions in reverse order
return answer and the resulting parent edges
```

12.13.6 Correctness

Theorem 496. *Algorithm 70 returns a minimum-weight rooted arborescence whenever one exists.*

Proof. Maintain the invariant

$$\mathrm{OPT}(G_{\text{original}}) = \mathit{answer} + \mathrm{OPT}(G_{\text{current}}).$$

Initially this is immediate.

Suppose we process a current non-root component X, whose cheapest external incoming edge has reduced weight δ. Subtracting δ from every edge entering X decreases every feasible arborescence by exactly δ by Lemma 494. Adding δ to *answer* therefore preserves the invariant.

The chosen edge now has reduced weight zero.

If no cycle is created, no contraction is required.

If a zero-weight directed cycle is created, Lemma 495 shows that contracting it leaves the residual optimum unchanged. Thus the invariant again survives.

Eventually all current non-root components have chosen incoming edges and no chosen cycle remains. Those edges form an arborescence of residual cost zero. Therefore

$$\mathrm{OPT}(G_{\mathrm{current}}) = 0,$$

and

$$\mathit{answer} = \mathrm{OPT}(G_{\mathrm{original}}).$$

It remains to recover the actual edges. Expand contractions in reverse order. Suppose a contracted cycle is entered by the already reconstructed edge f, whose original head lies in cycle component X_i. Restore every selected zero cycle edge except the one entering X_i, and use f instead. By the proof of Lemma 495, this preserves an arborescence and its weight.

Repeating through all contractions recovers an optimum arborescence of the original graph. □

12.13.7 Rollback Reconstruction

When contracting a cycle, save its DSU snapshot τ.

During reverse processing, the cycle is still one supervertex. Its currently selected incoming edge tells us which original member of the cycle receives the external edge.

Roll back to τ. The cycle members are now distinct again. If the external edge enters X_i, restore the old cycle edge for every other member and replace precisely the old edge entering X_i.

Thus every contraction requires only its rollback snapshot and the chosen edges already stored by the algorithm; the contracted graph never needs to be explicitly rebuilt.

12.13.8 Running Time

Theorem 497 (Tarjan)**.** *Using lazy skew heaps and disjoint-set union, Algorithm 70 runs in*

$$O(m \log n)$$

time and $O(m+n)$ space.

Proof. Each original edge is inserted into exactly one heap.

Once an edge becomes internal to a contracted component, it remains internal forever. Thus every edge is popped at most once.

A contraction permanently reduces the number of current components, so there are fewer than n contractions and only $O(n)$ successful contraction-DSU unions.

Every heap is formed only by melding heaps whose components are contracted. Hence there are $O(m+n)$ heap operations in total.

Lazy subtraction costs $O(1)$. Meld and pop in a skew heap cost $O(\log m) = O(\log n)$ amortized on the relevant sparse representation, and the DSU work is also at most this bound.

Therefore the total running time is

$$O(m \log n).$$

The reverse rollback pass performs only $O(n)$ unions/rollbacks and edge changes. □

After reconstruction, if $e_v = (u, v)$ is the selected incoming edge of $v \neq r$, one may return simply

$$\mathrm{parent}[v] = u, \qquad \mathrm{parent}[r] = r.$$

12.14 Matching and Coloring

12.14.1 Bipartite Regular Matching

Given a d-regular bipartite graph $G = (P, Q, E)$ with $|P| = |Q| = n$, a perfect matching can be found in $O(n \log n)$ expected time.

The algorithm iteratively augments a partial matching M by discovering augmenting paths via random walks. If M currently leaves k nodes unmatched in both P and Q, we implicitly construct a directed matching graph H:

1. Orient all edges of G from P to Q.
2. Contract each matched edge pair $(u, v) \in M$ into a single supernode.
3. Add a source vertex s connected by d parallel edges to each unmatched node in P, directed outward.
4. Add a sink vertex t connected by d parallel edges from each unmatched node in Q, directed inward.

Any $s \to t$ path in H corresponds directly to a valid augmenting path in G with respect to M. The algorithm simply performs a standard random walk starting from s, choosing an outgoing edge uniformly at random at each step, until it hits t.

Lemma 498 (Random Walk Hitting Time)**.** *Given a partial matching leaving k vertices unmatched, the expected number of steps before a random walk started at s reaches t is at most $2 + \frac{n}{k}$.*

Proof. Construct a modified graph H^* by identifying the source s and sink t into a single unified node s^*. In H^*, the out-degree of every vertex equals its in-degree, making it an Eulerian digraph.

The out-degree of the unified node s^* is kd, while the out-degree of every other node is $d-1$. The expected time for a random walk in H^* starting at s^* to return to s^* is exactly the inverse of its stationary probability π_{s^*}. Because H^* is balanced, this stationary probability is simply the ratio of the degree of s^* to the sum of all degrees in H^*. Evaluating this yields an expected return time of:

$$\frac{1}{\pi_{s^*}} = \frac{n(d-1) + 2kd + kd}{kd} \leq 2 + \frac{n}{k}$$

Running a walk from s^* until it returns to s^* in H^* is identical to running a walk from s until it hits t in the original matching graph H. □

To construct a perfect matching, the algorithm starts with an empty matching ($k = n$) and repeatedly runs untruncated random walks to augment the matching n times until $k = 0$. By the lemma, the total expected number of steps taken across all random walks is bounded by:

$$\sum_{k=1}^{n} \left(2 + \frac{n}{k}\right) = 2n + nH_n = O(n \log n)$$

The same runtime with high probability can easily be acheived by terminating the walks if they take too long and repeating. We then use Markov's inequality and a Chernoff bound.

12.14.2 Kasteleyn Signing and Planar Perfect Matchings

For a general graph, the determinant or Pfaffian expansion contains one term for every perfect matching, but the terms have different signs. For planar graphs, Kasteleyn's observation is that the edges can be signed so that all perfect matchings receive the same sign. Counting perfect matchings then reduces to one Pfaffian, or one determinant in the bipartite case.

We work with a fixed plane embedding. For an oriented edge on the boundary of a face, say that it is *clockwise* if its direction agrees with a clockwise traversal of that face. Boundary walks may repeat edges; occurrences are counted separately.

Definition 499 (Kasteleyn orientation)**.** An orientation of a connected plane graph is *Kasteleyn* if every bounded face has an odd number of clockwise-oriented boundary-edge occurrences.

The outer face is deliberately left unconstrained.

Lemma 500 (Existence of a Kasteleyn orientation)**.** *Every connected plane graph has a Kasteleyn orientation. Given the embedding, one can construct one in $O(|E|)$ time.*

Proof. Take a spanning tree of the dual graph, rooted at the outer face. Initially orient every primal edge whose dual edge is not in this tree arbitrarily.

Process the remaining dual-tree edges from the leaves toward the root. When a bounded face F is processed, exactly one still-unoriented primal edge on its boundary is the edge dual to its parent edge in the dual tree. Orient this edge in whichever direction makes the number of clockwise edges around F odd.

Once F is processed, its parity will never change. Every bounded face is eventually processed, while the outer face is the root and requires no condition. Each edge is handled only constantly many times. □

The face condition is useful because it implies the correct parity not only for faces, but for every even cycle that can occur as the symmetric difference of two perfect matchings.

Lemma 501 (Parity inside a cycle)**.** *Let C be a simple even cycle in a Kasteleyn-oriented plane graph, and let v_{in} be the number of vertices strictly inside C. If $c(C)$ is the number of edges oriented clockwise around C, then*

$$c(C) \equiv v_{\text{in}} + 1 \pmod 2.$$

Proof. Let H consist of C together with everything drawn inside C. Since the original graph is connected, H is connected. Let E_{in} be the number of edges of H not on C, and let F be the number of faces of H inside C.

Sum the number of clockwise-oriented edges over these F faces. Every interior edge occurs twice, once in each direction, so exactly one of its two boundary occurrences agrees with its orientation. Every edge of C occurs once and contributes exactly when it is clockwise around C. Hence

$$F \equiv E_{\text{in}} + c(C) \pmod 2,$$

because every bounded face has odd clockwise parity.

Euler's formula for H, with F bounded faces and one outer face, gives $V - E + F = 1$. Since $V = v_{\text{in}} + |C|$ and $E = E_{\text{in}} + |C|$,

$$F \equiv 1 + v_{\text{in}} + E_{\text{in}} \pmod 2.$$

Combining the two congruences gives $c(C) \equiv v_{\text{in}} + 1 \pmod 2$. □

Call an even cycle C *nice* if deleting its vertices leaves a graph with a perfect matching. For such a cycle, the number of vertices strictly inside C is even: after deleting C, no edge can join an inside vertex to an outside vertex without crossing C, so the remaining perfect matching pairs all inside vertices among themselves. Thus Lemma 501 immediately gives:

Corollary 502 (Nice cycles are oddly oriented)**.** *Every nice cycle in a Kasteleyn orientation has an odd number of edges oriented in either direction around the cycle.*

We now connect this parity to Pfaffian signs. For an oriented weighted graph, define the skew-symmetric matrix K by

$$K_{uv} = \begin{cases} w_{uv}, & u \to v, \\ -w_{uv}, & v \to u, \\ 0, & uv \notin E. \end{cases}$$

Thus $K_{vu} = -K_{uv}$. Parallel edges, if present, may simply be summed in the corresponding matrix entry.

Recall that the Pfaffian expansion of K has one term for each perfect matching:

$$\mathrm{Pf}(K) = \sum_{M} \mathrm{sgn}_K(M) \prod_{e \in M} w_e,$$

where $\mathrm{sgn}_K(M) \in \{\pm 1\}$ combines the usual Pfaffian pairing sign with the orientation signs of its edges.

Lemma 503 (Flipping an alternating cycle)**.** *Let two perfect matchings agree except on one even cycle C. Their Pfaffian terms have the same sign if C is oddly oriented.*

Proof. Relative signs are unchanged by simultaneously relabelling the vertices, so write

$$C = (1, 2, \ldots, 2r, 1)$$

in cyclic order and place all other vertices afterward.

Write $\varepsilon_{ij} = 1$ when the edge is oriented $i \to j$ and $\varepsilon_{ij} = -1$ otherwise. The two alternating matchings on C are

$$M_0 = \{12, 34, \ldots, (2r-1)(2r)\}$$

and

$$M_1 = \{23, 45, \ldots, (2r)1\}.$$

In this ordering the ordinary Pfaffian pairing sign of each displayed pairing is positive. Their orientation-sign ratio is therefore

$$\begin{aligned} \frac{\mathrm{sgn}_K(M_1)}{\mathrm{sgn}_K(M_0)} &= \frac{\varepsilon_{1,2r}\varepsilon_{23}\varepsilon_{45}\cdots\varepsilon_{2r-2,2r-1}}{\varepsilon_{12}\varepsilon_{34}\cdots\varepsilon_{2r-1,2r}} \\ &= -\prod_{i=1}^{2r} \varepsilon_{i,i+1}, \end{aligned}$$

where $2r + 1 = 1$.

If C is oddly oriented, then an odd number of its edges disagree with either traversal direction, so $\prod_i \varepsilon_{i,i+1} = -1$. The ratio is therefore 1. □

Theorem 504 (Kasteleyn–Pfaffian formula)**.** *Let G be a weighted planar graph with a Kasteleyn orientation, and let K be its signed skew adjacency matrix. Then*

$$|\mathrm{Pf}(K)| = \sum_{M \ \mathit{perfect}} \prod_{e \in M} w_e.$$

In particular, for an unweighted planar graph,

$$\#\{\textit{perfect matchings}\} = |\mathrm{Pf}(K)| = \sqrt{\det K}.$$

Proof. Fix two perfect matchings M, M'. Their symmetric difference is a disjoint union of alternating even cycles

$$M \triangle M' = C_1 \cup \cdots \cup C_s.$$

Each C_i is nice: the edges of M outside C_i form a perfect matching after the vertices of C_i are removed.

Hence every C_i is oddly oriented by Corollary 502. Starting from M, flip the alternating cycles one at a time. By Lemma 503, none of these flips changes the Pfaffian sign. Thus every perfect matching occurs in $\mathrm{Pf}(K)$ with the same sign.

Taking the absolute value therefore removes the common sign and leaves exactly the weighted perfect-matching sum. Finally $\det K = \mathrm{Pf}(K)^2$. □

Thus, once a planar embedding is available, the signing is constructed in linear time and we can take the square root of the determinant in $O(n^3)$ arithmetic operations to recover the Pfaffian up to sign.

Bipartite graphs If $G = (L \cup R, E)$ is bipartite with $|L| = |R| = n$, order all vertices of L before those of R. Then the Kasteleyn matrix has block form

$$K = \begin{pmatrix} 0 & B \\ -B^{\mathsf{T}} & 0 \end{pmatrix},$$

where B is a signed weighted bipartite adjacency matrix. The standard Pfaffian block identity gives

$$\mathrm{Pf}(K) = (-1)^{n(n-1)/2} \det B.$$

Hence

$$\sum_{M \text{ perfect}} \prod_{e \in M} w_e = |\det B|.$$

This is the determinant form of Kasteleyn signing: planarity lets one choose signs on the bipartite adjacency matrix so that the determinant has no cancellation between perfect matchings.

Domino tilings of a rectangle. The Kasteleyn matrix of the rectangular grid can be diagonalized almost by inspection. This gives the classical closed form for the number $T_{m,n}$ of domino tilings of an $m \times n$ rectangle.

Regard the mn squares as the vertices of the grid graph $P_m \square P_n$, with its checkerboard bipartition. Give every horizontal edge weight 1 and every vertical edge weight i, and let B be the resulting black-to-white matrix. This is a Kasteleyn matrix up to harmless unit complex scalings. Indeed, if a square lies in row r, multiply its black row of B by i^{-r} and its white column by i^r. Horizontal entries remain 1, while the two vertical edges of every elementary face become 1 and -1. Thus every bounded square has the required odd sign, and the preceding Kasteleyn theorem gives

$$T_{m,n} = |\det B|.$$

It is convenient to recover B as one block of a matrix on all mn squares. Let A_r denote the adjacency matrix of the path P_r, and put

$$K = A_m \otimes I_n + i I_m \otimes A_n.$$

If the vertices are reordered with all black vertices first, then

$$K = \begin{pmatrix} 0 & B \\ B^{\mathsf{T}} & 0 \end{pmatrix}.$$

When mn is even, the two parts have size $mn/2$, so

$$|\det K| = |\det B|^2 = T_{m,n}^2. \tag{1}$$

The path matrix A_r has an especially simple eigenbasis. It's clear to see that for the infinite path the natural eigenfunctions are $e^{i\theta x}$, with eigenvalues of $2\cos\theta$. For a finite path pretend we have two ghost vertices where the eigenvector must vanish, $f(0) = f(r+1) = 0$. We can cancel out waves of opposite frequencies since they agree at 0: $e^{i\theta x} + e^{-i\theta x} = 2i\sin(\theta x)$. Thus we need $\sin((r+1)\theta) = 0$. For $1 \le j \le r$, set

$$v_j(a) = \sin\left(\frac{\pi j a}{r+1}\right), \qquad 1 \le a \le r.$$

Writing $\theta_j = \pi j/(r+1)$ and using

$$\sin((a-1)\theta_j) + \sin((a+1)\theta_j) = 2\cos\theta_j \sin(a\theta_j),$$

together with $v_j(0) = v_j(r+1) = 0$, gives

$$A_r v_j = 2\cos\left(\frac{\pi j}{r+1}\right) v_j.$$

The r eigenvalues are distinct, so these vectors form a basis.

Consequently $v_j \otimes v_k$ is an eigenvector of K with eigenvalue

$$2\cos\left(\frac{\pi j}{m+1}\right) + 2i\cos\left(\frac{\pi k}{n+1}\right).$$

The mn tensor products form a basis, and hence

$$\begin{aligned}|\det K| &= \prod_{j=1}^{m}\prod_{k=1}^{n}\left|2\cos\left(\frac{\pi j}{m+1}\right) + 2i\cos\left(\frac{\pi k}{n+1}\right)\right| \\ &= \prod_{j=1}^{m}\prod_{k=1}^{n}\left(4\cos^2\left(\frac{\pi j}{m+1}\right) + 4\cos^2\left(\frac{\pi k}{n+1}\right)\right)^{1/2}.\end{aligned}$$

Combining this with (1) yields the classical formula

$$T_{m,n} = \prod_{j=1}^{m}\prod_{k=1}^{n}\left(4\cos^2\frac{\pi j}{m+1} + 4\cos^2\frac{\pi k}{n+1}\right)^{1/4}.$$

If mn is odd, there is no perfect matching and $T_{m,n} = 0$; indeed the product above contains the zero factor corresponding to $j = (m+1)/2$ and $k = (n+1)/2$.

When both m and n are even, pairing j with $m+1-j$ and k with $n+1-k$ gives the form

$$T_{m,n} = \prod_{j=1}^{m/2}\prod_{k=1}^{n/2}\left(4\cos^2\frac{\pi j}{m+1} + 4\cos^2\frac{\pi k}{n+1}\right).$$

12.14.3 Bipartite Edge Coloring

Let

$$G = (L \sqcup R, E)$$

be a bipartite multigraph with maximum degree

$$\Delta.$$

König's line-coloring theorem states that

$$\chi'(G) = \Delta.$$

We give an algorithmic proof which constructs a proper Δ-edge-coloring in

$$O(m \log m)$$

time.

The algorithm has three ingredients:

1. reduce to a regular bipartite multigraph of only $O(m)$ edges;
2. split any even-regular multigraph into two equal-degree regular subgraphs using Euler tours;
3. use the splitting operation to find a perfect matching, and then recursively color the graph.

Regularization The first reduction is useful because a proper edge-coloring of a graph obtained by identifying vertices on the same side remains proper when those vertices are separated again.

Lemma 505 (Regularization)**.** *Let G be a nonempty bipartite multigraph with m edges and maximum degree Δ. In $O(m)$ time we can construct a Δ-regular bipartite multigraph G' with*

$$O(m)$$

edges such that any proper Δ-edge-coloring of G' restricts to a proper Δ-edge-coloring of G.

Proof. Consider one side of the bipartition. Greedily combine original vertices into groups, keeping the sum of degrees in each group at most Δ. When the next vertex would make the current group exceed Δ, start a new group.

Let the resulting group loads be

$$d_1, \dots, d_q \le \Delta.$$

Whenever group $i+1$ was started,

$$d_i + d_{i+1} > \Delta,$$

because the first vertex placed in group $i+1$ already failed to fit in group i. Hence, pairing consecutive groups,

$$q = O\left(\frac{m}{\Delta} + 1\right).$$

Do this independently on both sides.

Identify all vertices belonging to one group. Every resulting vertex still has degree at most Δ. A coloring of the identified graph is also valid before identification, because identifying vertices only introduces additional constraints.

Add isolated dummy vertices to the smaller side until both sides contain the same number N of vertices. Since the degree sum on each side is m, both sides have the same total deficit

$$N\Delta - m.$$

Repeatedly join a deficient left vertex to a deficient right vertex until all degrees equal Δ.

The final graph has

$$N\Delta = O(m + \Delta) = O(m)$$

edges, since $\Delta \le m$. Deleting the dummy edges and undoing the identifications gives a coloring of the original graph. □

Euler Splitting The fundamental operation is especially simple.

Lemma 506 (Euler Split)**.** *Let H be a $2d$-regular multigraph. Then its edges can be partitioned into two d-regular spanning subgraphs*

$$H_0, H_1$$

in linear time.

Proof. Every connected component of H is Eulerian. Take an Euler tour of each component and alternately color its edges red and blue.

At every vertex, every entrance in the Euler tour is paired with the following exit, and the two edges receive opposite colors. Thus exactly half of the $2d$ incident edges are red and half are blue. □

The same operation works efficiently when an edge e is stored with a multiplicity $m(e)$ rather than explicitly storing all copies.

Lemma 507 (Euler Split with Multiplicities)**.** *Suppose a $2d$-regular multigraph is represented by multiplicities*

$$m(e)$$

on a set F of distinct edges. It can be split into two d-regular spanning multigraphs in

$$O(|F|)$$

time.

Proof. For every edge e, place

$$\left\lfloor \frac{m(e)}{2} \right\rfloor$$

copies in each side.

One copy remains precisely when $m(e)$ is odd. At every vertex, the number of remaining copies is even, because the original degree is even and we removed an even number of incident copies. Hence the graph consisting of the remaining copies is Eulerian.

Take Euler tours of this residual graph and alternate its edges between the two sides. Each side therefore receives exactly half of every vertex's residual degree, and consequently has degree d. Only the distinct edges need to be inspected. □

Lemma 508 (Perfect Matching in a Regular Bipartite Multigraph)**.** *Let G be a k-regular bipartite multigraph with N vertices on each side and*

$$m = Nk$$

edges. A perfect matching can be found in

$$O(m \log m)$$

time.

Proof. Let

$$Q = 2^t$$

be the smallest power of two satisfying

$$Q \geq m.$$

Write

$$Q = \alpha k + \beta, \qquad 0 \leq \beta < k.$$

Choose an arbitrary bijection between the two vertex classes, and let M_0 be the corresponding perfect matching in the complete bipartite graph. The edges of M_0 need not belong to G.

Construct a multigraph H by

- replacing every edge of G by α parallel copies;
- adding β copies of every edge of M_0.

Call the latter copies *bad*. Every vertex now has degree

$$\alpha k + \beta = Q,$$

so H is Q-regular.

The initial number of bad edges is

$$N\beta < Nk = m \leq Q.$$

Since Q is a power of two, repeatedly apply Lemma 507. At each split, retain the half containing fewer bad edges. Thus after j splits, the retained graph is

$$Q/2^j$$

regular and contains fewer than

$$Q/2^j$$

bad edges.

After t splits it is 1-regular and contains fewer than one bad edge. Hence it contains no bad edge at all.

A 1-regular spanning bipartite graph is a perfect matching, and all of its edges are copies of edges of G. Discarding multiplicities therefore gives a perfect matching of G.

At every level there are only

$$O(m)$$

distinct edges to process, and

$$t = O(\log m).$$

□

It is tempting to take Q to be merely the smallest power of two with $Q \geq k$, pad G to a Q-regular graph, and then repeatedly halve. The problem is that the padding introduces bad edges at *every* vertex.

Indeed, writing $Q = \alpha k + \beta$, the construction adds β bad matching edges at each of the N vertices on one side, for a total of $N\beta$ bad edges. At each Euler split we can keep the half containing fewer bad edges, so after j splits we only know that the number of bad edges has fallen by a factor 2^j. When the degree has finally fallen from Q to 1, this gives only

$$\#\text{bad edges} \leq \frac{N\beta}{Q}.$$

If Q is only $O(k)$, this can still be $\Theta(N)$.

Coloring a Power-of-Two Regular Graph

Lemma 509. *If G is 2^t-regular, then it can be properly edge-colored with exactly 2^t colors in*

$$O(m \log 2^t)$$

time.

Proof. Repeatedly Euler-split every current regular graph into two equal-degree subgraphs. After t levels, every leaf is 1-regular, hence a matching. Give each leaf a different color.

At each level the subgraphs partition the original edge set, so the total work per level is $O(m)$. □

The General Recursion We can now color a k-regular graph using exactly k colors.

Suppose first that k is even, and put

$$d = k/2.$$

Euler-split

$$G = G_1 \sqcup G_2,$$

where both graphs are d-regular.

Recursively color G_1 with colors

$$1, \ldots, d.$$

Each color class is a perfect matching.

Let

$$q$$

be the smallest power of two satisfying

$$q \geq d.$$

Since $q < 2d = k$ unless $d = q$, we need exactly

$$q - d$$

additional perfect matchings to make G_2 q-regular.

Take any $q - d$ color classes from G_1 and temporarily move them into G_2. The resulting graph

$$H$$

is q-regular, so Lemma 509 colors it with q colors.

The part of G_1 that was not moved contains

$$d - (q - d) = 2d - q = k - q$$

of its old color classes. Give these $k - q$ matchings colors disjoint from those used on H.

Altogether we use exactly

$$q + (k - q) = k$$

colors.

If k is odd, first find a perfect matching using Lemma 508, give it one color, remove it, and apply the even-degree procedure to the remaining $(k - 1)$-regular graph.

Algorithm 71 Edge Coloring a Regular Bipartite Multigraph

```
function ColorRegular(G, k)
    if k = 0 then
        return
    if k = 1 then
        give all edges of G one new color
        return
    if k is odd then
        M ← PerfectMatching(G)
        give M one new color
        G ← G − M
        k ← k − 1
    d ← k/2
    (G1, G2) ← EulerSplit(G)
    ColorRegular(G1, d)
    q ← least power of two with q ≥ d
    move any q − d color classes of G1 into G2
    H ← the resulting q-regular graph
    color H with q fresh colors by repeated Euler splitting
    retain the remaining k − q old color classes of G1
```

Theorem 510 (König's Line-Coloring Theorem, Algorithmic Form)**.** *Every bipartite multigraph of maximum degree Δ has a proper Δ-edge-coloring. Such a coloring can be found in*

$$O(m \log m)$$

time.

Proof. Apply Lemma 505, and let $M = O(m)$ be the number of edges after regularization.

Correctness follows from the construction above.

For the running time, consider a recursive call on a k-regular graph with M' edges. Everything outside the recursive call costs

$$O(M' \log M) :$$

Euler splitting is linear, power-of-two coloring costs $O(M' \log k)$, and when k is odd the perfect matching costs $O(M' \log M)$.

The recursive call is made on a graph with half as many edges. Thus

$$T(M') \leq T(M'/2) + O(M' \log M),$$

and therefore

$$T(M) = O(M \log M) = O(m \log m).$$

Finally discard dummy edges and undo the identified vertices. □

12.14.4 Chromatic Number

Let

$$G = (V, E), \qquad |V| = n.$$

For

$$S \subseteq V,$$

let

$$I(S) = |\{T \subseteq S : T \text{ is independent}\}|.$$

The key inclusion–exclusion identity is extremely simple.

Theorem 511 (Coloring by Independent-Set Counts)**.** *For $k \geq 1$, define*

$$C_k = \sum_{S \subseteq V} (-1)^{n-|S|} I(S)^k.$$

Then

$$G \text{ is } k\text{-colorable} \iff C_k > 0.$$

Proof. The quantity

$$I(S)^k$$

counts ordered tuples

$$(A_1, \ldots, A_k)$$

of independent subsets of S.

Consider one fixed tuple of independent subsets of V. Its contribution to C_k is

$$\sum_{\substack{S \subseteq V \\ A_1 \cup \cdots \cup A_k \subseteq S}} (-1)^{n-|S|}.$$

This equals

$$\begin{cases} 1, & A_1 \cup \cdots \cup A_k = V, \\ 0, & \text{otherwise}, \end{cases}$$

by inclusion–exclusion.

Hence C_k counts ordered k-tuples of independent sets whose union is all of V.

Such a tuple exists exactly when G is k-colorable: given a coloring, take its color classes; conversely, from a covering tuple, assign each vertex to any independent set containing it. □

Define an array

$$e(S)$$

initially by

$$e(\{u,v\}) = 1 \qquad \text{for every edge } \{u,v\} \in E,$$

and 0 elsewhere.

Apply the subset zeta transform:

$$E(S) = \sum_{T \subseteq S} e(T).$$

Then

$$E(S)$$

is exactly the number of graph edges contained in S.

Thus

$$a(S) = \begin{cases} 1, & E(S) = 0, \\ 0, & E(S) > 0 \end{cases}$$

is the indicator that S is independent.

Apply the subset zeta transform a second time:

$$I(S) = \sum_{T \subseteq S} a(T).$$

This computes every independent-subset count in

$$O(n2^n)$$

arithmetic operations.

Algorithm 72 Exact Chromatic Number

```
initialize e(S) ← 0 for all S ⊆ V
for each edge {u, v} ∈ E do
    e({u, v}) ← 1
E ← ζ↓e
for S ⊆ V do
    a(S) ← 1[E(S) = 0]
I ← ζ↓a
P(S) ← 1 for all S
for k = 1, . . . , n do
    for S ⊆ V do
        P(S) ← P(S)I(S)
    C_k ← Σ_{S⊆V} (−1)^{n−|S|} P(S)
    if C_k ≠ 0 then
        return k
```

Theorem 512. *The chromatic number of an n-vertex graph can be computed in*

$$O(n2^n)$$

arithmetic operations and

$$O(2^n)$$

space.

12.14.5 The Tutte Polynomial

The preceding inclusion–exclusion algorithm detects whether independent sets can cover the vertex set. Its quantity C_k is not the number of proper k-colorings, since those sets may overlap. To count colorings, we will use subset convolution to make the color classes disjoint. Allowing edges inside a class, and recording how many there are, extends the same calculation to the Tutte polynomial.

Throughout this section, $G = (V, E)$ is a finite undirected multigraph with n vertices and m edges. Loops and parallel edges are allowed; they arise naturally under contraction. For $A \subseteq E$, let $c(A)$ be the number of connected components of the spanning subgraph (V, A), including isolated vertices. Put $c = c(E)$ and $r = n - c$.

The definition. The two quantities $c(A) - c$ and $|A| - n + c(A)$ measure how far (V, A) is from being a spanning forest with one tree in each component of G. The first counts its excess components; the second counts its edges beyond a spanning forest. Both are nonnegative. Define

$$T_G(x, y) = \sum_{A \subseteq E} (x-1)^{c(A)-c} (y-1)^{|A|-n+c(A)}.$$

Thus $x - 1$ records disconnection and $y - 1$ records excess edges. Adding an isolated vertex changes neither exponent, so it does not change T_G.

Theorem 513 (Deletion–contraction)**.** *If $E = \varnothing$, then $T_G = 1$. Otherwise, for any edge e,*

$$T_G(x, y) = \begin{cases} y T_{G \setminus e}(x, y), & e \text{ is a loop}, \\ x T_{G/e}(x, y), & e \text{ is a bridge}, \\ T_{G \setminus e}(x, y) + T_{G/e}(x, y), & \text{otherwise}. \end{cases}$$

Here G/e identifies the endpoints of a nonloop edge and removes that edge, retaining all other edges and any loops or parallel edges created. In particular, T_G has nonnegative integer coefficients.

Proof. Pair each $B \subseteq E \setminus \{e\}$ with $B \cup \{e\}$. If e is a loop, its inclusion leaves the component count unchanged and increases the second exponent by one. The pair therefore contributes a factor $1 + (y - 1) = y$.

Suppose e is neither a loop nor a bridge. Deleting e does not change $c(G)$, so the terms omitting e give $T_{G \setminus e}$. For terms containing e, contraction reduces both the vertex count and the edge count by one, preserves the component count, and hence preserves both exponents. These terms give $T_{G/e}$.

Finally, suppose e is a bridge. Its endpoints lie in different components of (V, B) for every $B \subseteq E \setminus \{e\}$. Relative to the term indexed by B in $T_{G/e}$, omitting e introduces one excess component and contributes a factor $x - 1$, while including e contributes a factor 1. Their sum is x. The coefficient assertion follows by induction from the recurrence. □

For example, a forest with m edges has Tutte polynomial x^m, while a graph consisting of one vertex and m loops has polynomial y^m. The exponents in the defining sum also give $\deg_x T_G \le r$ and $\deg_y T_G \le m - r$: adding edges cannot decrease $|A| - n + c(A)$.

Lemma 514 (Products)**.** *The Tutte polynomial is multiplicative under disjoint union: $T_{G_1 \sqcup G_2} = T_{G_1} T_{G_2}$. The same equality holds if one identifies a vertex of G_1 with a vertex of G_2.*

Proof. Every edge subset splits uniquely into one subset in each factor, and both exponents add. For the vertex identification, the vertex count and the component count of every spanning subgraph both decrease by one, leaving the exponents unchanged. □

Basic counting evaluations. The values at 1 and 2 select which of the two defects are allowed.

Theorem 515. *The following evaluations count spanning subgraphs of G:*

Evaluation	*Count*
$T_G(1,1)$	*one spanning tree in each component of G,*
$T_G(2,1)$	*all spanning forests, including isolated vertices,*
$T_G(1,2)$	*all spanning subgraphs with exactly c components,*
$T_G(2,2)$	*all edge subsets, namely 2^m.*

For a connected graph, the first and third rows count spanning trees and connected spanning subgraphs, respectively.

Proof. At $x = 1$, precisely the terms with $c(A) = c$ survive. At $y = 1$, precisely the terms with $|A|-n+c(A) = 0$ survive, and these are the forests. Setting either variable to 2 gives weight 1 to every value of its exponent. As usual, a zeroth power evaluates to 1 even when its base is zero. □

Theorem 516 (Acyclic orientations)**.** *The number of acyclic orientations of G is $T_G(2,0)$.*

Proof. Let $a(G)$ be this number. An edgeless graph has one orientation, a graph with a loop has none, and a bridge can be oriented either way, giving $a(G) = 2a(G/e)$.

For an edge $e = uv$ that is neither a loop nor a bridge, consider an acyclic orientation of $G \setminus e$. If it contains a directed path between u and v, exactly one direction of e avoids a directed cycle. Otherwise both directions work. The orientations in the second case are exactly those remaining acyclic after identifying u and v: a new directed cycle after identification would give a directed path between them before identification. They are therefore in bijection with the acyclic orientations of G/e. Consequently $a(G) = a(G \setminus e) + a(G/e)$, which is precisely the deletion–contraction recurrence for $T_G(2,0)$. □

Planar duality. For plane graphs, disconnection in one graph corresponds to excess edges in its dual, explaining the symmetry between the variables.

Theorem 517 (Planar duality)**.** *If G is connected and embedded in the plane, with plane dual G^*, then $T_{G^*}(x,y) = T_G(y,x)$.*

Proof. For $A \subseteq E$, let A^* consist of the dual edges corresponding to $E \setminus A$. Deleting the edges outside A merges precisely the faces joined by A^*, so the components of $(V(G^*), A^*)$ correspond to the faces of (V, A). Euler's formula therefore gives

$$c_{G^*}(A^*) = |A| - n + c(A) + 1.$$

Also $|V(G^*)| = m - n + 2$. Since both G and G^* are connected, the two exponents associated with A^* are

$$c_{G^*}(A^*) - 1 = |A| - n + c(A), \qquad |A^*| - |V(G^*)| + c_{G^*}(A^*) = c(A) - 1.$$

They are the exponents for A in the opposite order. Sum over the bijection $A \longleftrightarrow A^*$. □

Recording components and edges separately. For computation, it is convenient to keep the component count and the edge count as separate exponents. Define

$$Z_G(q,z) = \sum_{A \subseteq E} q^{c(A)} z^{|A|} = \sum_{k=0}^{n} \sum_{j=0}^{m} a_{k,j} q^k z^j.$$

Thus $a_{k,j}$ counts spanning subgraphs with k components and j edges. The defining sum for T_G gives the polynomial identity

$$T_G(x,y) = \sum_{\substack{k,j \\ a_{k,j} \neq 0}} a_{k,j} (x-1)^{k-c} (y-1)^{j+k-n}. \tag{288}$$

Every exponent here is nonnegative, because $k \geq c$ and $j \geq n-k$. Equivalently,

$$Z_G\big((x-1)(y-1), y-1\big) = (x-1)^c (y-1)^n T_G(x,y). \tag{289}$$

The coefficient conversion in (288) avoids division and applies without exceptions at $x=1$ and $y=1$.

The chromatic polynomial. Let $\chi_G(q)$ denote the number of proper colorings with q labelled colors when q is a nonnegative integer.

Theorem 518 (Coloring specialization)**.** *The coloring counts agree with the polynomial*

$$\chi_G(q) = Z_G(q,-1) = (-1)^{n-c} q^c T_G(1-q, 0).$$

Proof. Apply inclusion–exclusion to the events that an edge has equal colors at its endpoints. Requiring these events for all edges of A means that every component of (V, A) is monochromatic, leaving $q^{c(A)}$ colorings. Hence $\chi_G(q) = \sum_{A \subseteq E} (-1)^{|A|} q^{c(A)}$. The second equality follows from (289) with $x = 1-q$ and $y = 0$. This argument also handles loops: a loop makes every coloring improper. □

In particular, Theorem 516 also gives $a(G) = (-1)^n \chi_G(-1)$.

The flow polynomial. Fix an arbitrary orientation of G, and let Γ be a finite abelian group of order q. A Γ-flow assigns $f(e) \in \Gamma$ to each edge so that the sum entering each vertex equals the sum leaving it. The flow is *nowhere-zero* if $f(e) \neq 0$ for every edge. Reversing an edge and negating its value is a bijection of flows, so the count is independent of the chosen orientation.

Theorem 519 (Flow specialization)**.** *The number of nowhere-zero Γ-flows depends only on $q = |\Gamma|$ and equals*

$$\Phi_G(q) = \sum_{A \subseteq E} (-1)^{m-|A|} q^{|A|-n+c(A)} = (-1)^{m-n+c} T_G(0, 1-q).$$

Proof. First count flows that are zero outside a fixed set A. Choose a spanning forest of (V, A) and assign arbitrary group elements to its $|A| - n + c(A)$ nonforest edges. Conservation determines the forest-edge values uniquely, by processing leaves towards each root. Conservation at the root then follows by summing the equations over its component. Thus there are exactly $q^{|A|-n+c(A)}$ flows supported inside A.

Inclusion–exclusion over the events $f(e) = 0$ gives the displayed sum. Substituting $x = 0$ and $y = 1-q$ in the definition of T_G gives the same sum after multiplication by $(-1)^{m-n+c}$. □

For prime q, one may take $\Gamma = \mathbb{F}_q$, in which case these are solutions to the incidence-matrix equations with every coordinate nonzero.

Computing the polynomial through vertex subsets. The defining sum has 2^m terms. We instead organize the computation over the 2^n vertex subsets and use set power series power-projection.

Write $d(S) = |E(G[S])|$. As in the preceding chromatic-number algorithm, all values $d(S)$ can be computed by one subset zeta transform: initialize an array to zero, add one at the endpoint set of each edge, and transform. A loop contributes at a singleton and parallel edges contribute with multiplicity. This takes $O(m + n2^n)$ arithmetic operations.

Use $*$ for subset convolution and ε for its identity, as in the SPS section. Over a characteristic-zero coefficient field, define the polynomial-valued set series

$$F_z(S) = (1+z)^{d(S)}, \qquad H_z = F_z - \varepsilon.$$

In particular $H_z(\varnothing) = 0$, so $H_z^{*k} = 0$ for $k > n$. The coefficient $(H_z^{*k})(V)$ sums over ordered partitions of V into k nonempty sets, weighting each block S by $(1+z)^{d(S)}$.

Lemma 520 (The power-projection identity)**.** *For $b_k(z) = (H_z^{*k})(V)$, including $H_z^{*0} = \varepsilon$, one has*

$$Z_G(q, z) = \sum_{k=0}^{n} \binom{q}{k} b_k(z).$$

Here $\binom{q}{k} = q(q-1)\cdots(q-k+1)/k!$ is an ordinary polynomial in q, with $\binom{q}{0} = 1$.

Proof. For a nonnegative integer q, the binomial theorem gives

$$\sum_{k=0}^{n} \binom{q}{k} b_k(z) = (F_z^{*q})(V) = \sum_{\sigma: V \to \{1,\dots,q\}} \prod_{\substack{uv \in E \\ \sigma(u)=\sigma(v)}} (1+z).$$

The convolution assigns every vertex to exactly one of the q labelled color classes, allowing empty classes. The product weights every monochromatic edge by $1 + z$.

Expand this product by choosing a set A of monochromatic edges that contribute z; all other edges contribute 1. A coloring permits this choice exactly when it is constant on every component of (V, A). There are $q^{c(A)}$ such colorings. Summing first over A yields $\sum_A q^{c(A)} z^{|A|} = Z_G(q, z)$. The two expressions agree at every nonnegative integer q, and hence are equal as polynomials. □

The logarithm gives another interpretation of the same identity. If $C_z(\varnothing) = 0$ and, for nonempty S,

$$C_z(S) = \sum_{\substack{A \subseteq E(G[S]) \\ (S,A) \text{ connected}}} z^{|A|},$$

then $F_z = \exp(C_z)$. Thus $C_z = \log(F_z)$ and $Z_G(q, z) = (\exp(q \log(F_z)))(V)$.

Using SPS power projection. The power projection algorithm computes $\langle W, H^{*k} \rangle$ for all requested k. Set $W(S) = \mathbf{1}[S = V]$ and call SPSPowerProj$(H_z, W, n, n+1)$. Its outputs are exactly $b_0(z), \dots, b_n(z)$. All coefficients are now univariate polynomials in z, but the subset-convolution and transposed-convolution steps are unchanged. Since the final degrees in z are at most m, every intermediate polynomial may be reduced modulo z^{m+1}.

Convert from the binomial basis to the monomial basis using $D_0(q) = 1$ and $D_{k+1}(q) = D_k(q)(q-k)/(k+1)$, accumulating $Z_G(q, z) = \sum_k D_k(q) b_k(z)$. Once its coefficients $a_{k,j}$ are known, (288) gives the answer.

Algorithm 73 Tutte polynomial by SPS power projection

function TuttePolynomial($G = (V, E)$)
  $n \leftarrow |V|$, $m \leftarrow |E|$, $c \leftarrow$ number of components of G
  if $n = 0$ **then**
    return 1
  Initialize $a(S) \leftarrow 0$ for every $S \subseteq V$
  for each edge $e \in E$ **do**
    $U \leftarrow$ set of endpoints of e ▷ *A singleton for a loop*
    $a(U) \leftarrow a(U) + 1$
  $d \leftarrow \zeta_{\downarrow} a$
  for $S \subseteq V$ **do**
    $H(S) \leftarrow (1 + z)^{d(S)}$, $W(S) \leftarrow \mathbf{1}[S = V]$
  $H(\varnothing) \leftarrow 0$
  $(b_0, \ldots, b_n) \leftarrow$ SPSPowerProj$(H, W, n, n + 1)$ ▷ *Coefficients in* $\mathbb{Q}[z]/(z^{m+1})$
  $Z(q, z) \leftarrow 0$, $D(q) \leftarrow 1$
  for $k = 0, \ldots, n$ **do**
    $Z(q, z) \leftarrow Z(q, z) + D(q) b_k(z)$
    if $k < n$ **then**
      $D(q) \leftarrow D(q)(q - k)/(k + 1)$
  $P(u, v) \leftarrow 0$
  for each nonzero coefficient $a_{k,j} = [q^k z^j] Z(q, z)$ **do**
    $P(u, v) \leftarrow P(u, v) + a_{k,j} u^{k-c} v^{j+k-n}$
  return $P(x - 1, y - 1)$ ▷ *Shift each variable to obtain ordinary coefficients*

Theorem 521 (Tutte polynomial computation)**.** *For $n \geq 1$, all coefficients of T_G can be computed in*

$$O\big(n^2 2^n (m + 1) \log(m + 2)\big)$$

arithmetic operations and $O(n(m + 1)2^n)$ field elements of space.

Proof. The SPS power projection algorithm takes $O(n^2 2^n)$ ring operations for with $n + 1$ outputs. Here addition costs $O(m + 1)$ field operations and multiplication modulo z^{m+1} costs $O(m + 1 \log(m + 2))$. The ranked arrays used for subset convolution require $O(n2^n)$ coefficient-ring elements. All divisions are by integers at most $\max\{n, m\}$, so the same calculation is valid in the stated positive characteristics.

Precompute factorials and inverse factorials through $\max\{n, m\}$. For each S, the coefficients of $(1 + z)^{d(S)}$ can then be written in $O(m + 1)$ operations using binomial coefficients. Thus initialization costs $O(m + n2^n + (m + 1)2^n)$ operations. Building the polynomials D_k and accumulating Z costs $O(n^2(m + 1))$ operations. Changing the exponent indices costs $O(n(m + 1))$.

Finally, shift the u-variable of P by the binomial formula, at cost $O(n^2(m + 1))$, and shift the v-variable separately for each u-coefficient. A univariate Taylor shift of degree at most m takes $O((m + 1) \log(m + 2))$ operations by factorial scaling and one convolution: if $p(t) = \sum_j p_j t^j$, then $[t^i]p(t - 1) = \frac{1}{i!} \sum_{j \geq i} j! p_j (-1)^{j-i}/(j - i)!$. The second set of shifts therefore costs $O(n(m + 1) \log(m + 2))$. All these costs are absorbed by the stated bounds. □

12.15 Node-Weighted Triangle

Let $G = (A \sqcup B \sqcup C, E)$ be tripartite, with a weight

$$w(v)$$

on every vertex. The *Node-Weighted Triangle* problem asks whether there is a triangle

$$a \in A, \qquad b \in B, \qquad c \in C$$

such that

$$w(a) + w(b) + w(c) = 0.$$

There is no loss in assuming the graph is tripartite. Given an arbitrary graph, make three copies of its vertex set and put an edge between two copies exactly when the corresponding original vertices are adjacent. Triangles are preserved up to a constant multiplicity.

The main point is that unequal multiplicities of the vertex weights do not make the problem harder than ordinary triangle detection.

12.15.1 One Fixed Weight

The basic case is immediate.

Lemma 522 (One Fixed Weight)**.** *Suppose every vertex of A has the same weight u. Then Node-Weighted Triangle can be solved in*

$$O(\mathrm{MM}(|A|, |B|, |C|))$$

time, up to linear input-processing time.

Proof. For every edge $bc \in E(B, C)$, keep it only if

$$u + w(b) + w(c) = 0.$$

Then a triangle

$$a, b, c$$

in the remaining graph satisfies

$$w(a) + w(b) + w(c) = u + w(b) + w(c) = 0.$$

Conversely every zero-weight triangle survives.

Thus the problem has become ordinary tripartite triangle detection, which is one rectangular matrix multiplication. □

So the whole problem is to partition the vertices into instances in which one side has a fixed weight, without paying too much for the partition.

12.15.2 The Square Case

Assume

$$|A|, |B|, |C| \le n.$$

For a weight u, write

$$f_A(u) = |\{a \in A : w(a) = u\}|,$$

and define f_B similarly.

We first process triangles for which the A-weight is at least as frequent as the B-weight.

Fix an A-weight u, and put

$$k = f_A(u).$$

Only consider B-weights v satisfying

$$f_B(v) \le k.$$

Greedily partition these weights into groups P such that

$$\sum_{v \in P} f_B(v) \le 2k,$$

and every group except possibly the last has total frequency at least k. This is possible because every individual weight has frequency at most k.

For a group P, define

$$A_u = \{a \in A : w(a) = u\},$$

$$B_P = \{b \in B : w(b) \in P\},$$

and

$$C_P = \{c \in C : -u - w(c) \in P\}.$$

Every zero-weight triangle whose A-vertex has weight u and whose B-weight lies in P is contained in

$$A_u \times B_P \times C_P.$$

Moreover,

$$|A_u| = k, \qquad |B_P| \le 2k.$$

Split C_P into blocks of at most $2k$ vertices and apply Lemma 522 to every block.

Theorem 523 (Node-Weighted Triangle)**.** *Node-Weighted Triangle on a tripartite graph with $O(n)$ vertices in each part can be solved in*

$$O(n^\omega))$$

time, the same time as ordinary triangle detection.

Proof. Fix an A-weight u of frequency

$$k = f_A(u).$$

Because every completed B-group contains at least k vertices, there are

$$O(n/k)$$

groups.

Also, every $c \in C$ belongs to at most one set C_P, because its required complementary B-weight

$$-u - w(c)$$

belongs to at most one group. Hence

$$\sum_P |C_P| \le n.$$

After splitting the C_P's into $O(k)$-sized blocks, the total number of blocks is therefore

$$O(n/k).$$

Each block has dimensions

$$O(k) \times O(k) \times O(k),$$

so it costs

$$O(k^\omega)$$

to process. Thus the total cost for the fixed weight u is

$$O(n\, k^{\omega-1}).$$

Now sum over all distinct weights u in A. Since

$$\sum_u f_A(u) = |A| \le n$$

and $\omega \geq 2$,

$$k^{\omega-1} \leq n^{\omega-2}k.$$

Therefore

$$\begin{aligned}\sum_u n\, f_A(u)^{\omega-1} &\leq n^1 n^{\omega-2} \sum_u f_A(u) \\ &= n^\omega.\end{aligned}$$

This detects every zero-weight triangle satisfying

$$f_A(w(a)) \geq f_B(w(b)).$$

Run the same construction with A and B interchanged. Every triangle satisfies one of

$$f_A(w(a)) \geq f_B(w(b)), \qquad f_B(w(b)) > f_A(w(a)),$$

so every zero-weight triangle is detected.

The second orientation has the same running time. □

The balancing principle is simple. A weight occurring k times gives a matrix dimension of order k, but there can be only $O(n/k)$ relevant blocks. Since matrix multiplication grows at least quadratically with the dimension, these effects balance.

12.15.3 Rectangular Version

Suppose the three parts have sizes

$$|A| \leq n^a, \qquad |B| \leq n^b, \qquad |C| \leq n^c,$$

and let

$$\omega = \omega(a, b, c).$$

For an A-weight u, write

$$f_A(u) = |\{x \in A : w(x) = u\}|,$$

and similarly define $f_B(v)$.

Since A and B have different sizes, absolute frequencies should not be compared directly. Define their normalized frequencies by

$$\rho_A(u) = f_A(u)^{1/a}, \qquad \rho_B(v) = f_B(v)^{1/b}.$$

Thus

$$f_A(u) = \rho_A(u)^a, \qquad f_B(v) = \rho_B(v)^b.$$

The point is that a normalized frequency ρ naturally gives matrix dimensions

$$\rho^a, \qquad \rho^b, \qquad \rho^c,$$

whose multiplication cost is

$$O(\rho^\omega).$$

Theorem 524 (Rectangular Node-Weighted Triangle)**.** *Node-Weighted Triangle on a tripartite graph with part sizes*

$$n^a, \qquad n^b, \qquad n^c$$

can be solved in

$$O(n^{\omega(a,b,c)})$$

time.

Proof. We first find every zero-weight triangle satisfying

$$\rho_A(w(x)) \geq \rho_B(w(y)).$$

Fix an A-weight u, and set

$$k = f_A(u), \qquad \rho = k^{1/a}.$$

Thus

$$k = \rho^a.$$

Consider exactly those B-weights v for which

$$f_B(v) \leq \rho^b.$$

Greedily partition these weights into groups P such that

$$\sum_{v \in P} f_B(v) \leq 2\rho^b,$$

and every completed group has total frequency at least ρ^b.

For each group P, define

$$A_u = \{x \in A : w(x) = u\},$$
$$B_P = \{y \in B : w(y) \in P\},$$

and

$$C_P = \{z \in C : -u - w(z) \in P\}.$$

Then

$$|A_u| = \rho^a, \qquad |B_P| = O(\rho^b).$$

Split every C_P into blocks of size at most

$$\rho^c.$$

On each resulting instance, all vertices of A_u have the same weight, so Lemma 522 reduces it to ordinary triangle detection with dimensions

$$O(\rho^a) \times O(\rho^b) \times O(\rho^c).$$

By the definition of $\omega(a, b, c)$, each such instance costs

$$O(\rho^{\omega(a,b,c)}).$$

We now count the number of instances for this fixed u.

Since every completed B-group contains at least ρ^b vertices, there are

$$O\left(\frac{n^b}{\rho^b}\right)$$

groups.

Moreover, for fixed u, the sets C_P are disjoint: a vertex z determines the unique complementary weight

$$-u - w(z).$$

Hence

$$\sum_P |C_P| \leq n^c.$$

After splitting into ρ^c-sized blocks, the total number of blocks is therefore

$$O\left(\frac{n^b}{\rho^b} + \frac{n^c}{\rho^c}\right).$$

Thus the total cost associated with u is

$$O\left(n^b \rho^{\omega(a,b,c)-b+\varepsilon} + n^c \rho^{\omega(a,b,c)-c+\varepsilon}\right).$$

Now sum this directly over all A-weights u.

Since

$$\rho^a = f_A(u)$$

and

$$\sum_u \rho^a = \sum_u f_A(u) = |A| \le n^a,$$

we can charge both terms to this sum.

First, the trivial input-size lower bound

$$\omega(a,b,c) \ge a + b$$

gives

$$\omega(a,b,c) - a - b \ge 0.$$

Since $\rho \le n$,

$$\rho^{\omega(a,b,c)-b} = \rho^a \rho^{\omega(a,b,c)-a-b} \le \rho^a n^{\omega(a,b,c)-a-b}.$$

Therefore

$$\begin{aligned}\sum_u n^b \rho^{\omega(a,b,c)-b} &\le n^b n^{\omega(a,b,c)-a-b} \sum_u \rho^a \\ &\le n^b n^{\omega(a,b,c)-a-b} n^a \\ &= n^{\omega(a,b,c)}.\end{aligned}$$

Likewise,

$$\omega(a,b,c) \ge a + c,$$

so

$$\begin{aligned}\sum_u n^c \rho^{\omega(a,b,c)-c} &\le n^c n^{\omega(a,b,c)-a-c} \sum_u \rho^a \\ &\le n^{\omega(a,b,c)}.\end{aligned}$$

Hence this orientation costs

$$O(n^{\omega(a,b,c)}).$$

It finds every zero-weight triangle x, y, z satisfying

$$\rho_A(w(x)) \ge \rho_B(w(y)),$$

because when $u = w(x)$, the weight $w(y)$ is among the allowed B-weights, and the group containing it also contains z in its corresponding set C_P.

Run the same construction with A and B interchanged. This finds all remaining triangles. Rectangular matrix multiplication has the elementary symmetry

$$\omega(b,a,c) = \omega(a,b,c),$$

so the second run has the same complexity. □

12.15.4 Minimum Node-Weighted Triangle

Suppose now that the vertex weights are integers satisfying

$$|w(v)| \le W,$$

and we want a triangle of minimum total weight rather than merely a zero-weight triangle.

Only a logarithmic number of NWT calls are needed.

Lemma 525 (Testing a Target)**.** *Suppose $G = (A \sqcup B \sqcup C, E)$ is tripartite. For any integer t, we can test whether G contains a triangle of total weight exactly t using one call to NWT.*

Proof. Replace the weights on A by

$$w'(a) = w(a) - t$$

and leave the weights on $B \cup C$ unchanged.

Then

$$w'(a) + w(b) + w(c) = 0$$

if and only if

$$w(a) + w(b) + w(c) = t.$$

□

Theorem 526 (Minimum Node-Weighted Triangle from NWT)**.** *Suppose NWT on a given family of n-vertex instances can be solved in time $T(n)$. Then Minimum Node-Weighted Triangle with integer weights*

$$|w(v)| \le W$$

can be solved in

$$O(T(n) \log(W + 2))$$

time.

Proof. We recursively reduce the magnitude of the weights.

Replace every weight $w(v)$ by

$$\widehat{w}(v) = \left\lfloor \frac{w(v)}{2} \right\rfloor.$$

Recursively compute the minimum triangle weight

$$s$$

under the rounded weights.

For every integer x,

$$x = 2\left\lfloor \frac{x}{2} \right\rfloor + \varepsilon_x, \qquad \varepsilon_x \in \{0, 1\}.$$

Therefore for every triangle T,

$$w(T) = 2\widehat{w}(T) + \varepsilon_T, \qquad \varepsilon_T \in \{0, 1, 2, 3\}.$$

Let M be the minimum triangle weight under the original weights.

Since

$$\widehat{w}(T) \ge s$$

for every triangle,

$$w(T) \ge 2s,$$

and hence

$$M \geq 2s.$$

On the other hand, let T_0 be a triangle with

$$\widehat{w}(T_0) = s.$$

Then

$$w(T_0) \leq 2s + 3,$$

so

$$M \leq 2s + 3.$$

Thus

$$M \in \{2s, 2s+1, 2s+2, 2s+3\}.$$

Using Lemma 525, make at most four NWT calls to determine which of these four values is attained, and return the smallest one.

The rounded weights satisfy

$$|\widehat{w}(v)| \leq \left\lceil \frac{W}{2} \right\rceil.$$

Thus, if $T_{\min}(W)$ denotes the running time,

$$T_{\min}(W) \leq T_{\min}(\lceil W/2 \rceil) + 4T(n) + O(n).$$

After $O(\log(W+2))$ levels the weights are bounded by a constant, at which point the minimum can be found with constantly many target queries.

Hence

$$T_{\min}(W) = O(T(n)\log(W+2)).$$

□

12.16 Incremental SCCs

Suppose directed edges

$$e_1, e_2, \ldots, e_m$$

are inserted in this order, and let G_t denote the graph containing the first t edges.

Strongly connected components only merge as t increases. Thus for every pair u, v, there is a threshold

$$\tau(u,v) = \min\{t : u, v \text{ lie in the same SCC of } G_t\},$$

with $\tau(u,v) = m+1$ if they never become strongly connected.

In particular, if $e_t = (u_t, v_t)$, then e_t strictly changes the SCC partition exactly when

$$\tau(u_t, v_t) = t.$$

Indeed, if the endpoints were already strongly connected before e_t arrived, then adding e_t cannot change any reachability relation: whenever a path uses the new edge $u_t \to v_t$, we may replace it by the old path from u_t to v_t inside their SCC. Conversely, if the endpoints first become strongly connected at time t, the SCC partition has clearly changed.

Thus it is enough to compute these m threshold values simultaneously.

Divide and Conquer on Time The basic idea is binary search, with one important optimization.

Suppose we are considering a time interval $[l, r]$, and let $h = \lfloor (l + r)/2 \rfloor$. Compute the SCCs of the graph at time h.

For a query pair u, v,

- if u, v are already in the same SCC at time h, then $\tau(u, v) \leq h$, so the query goes left;
- otherwise $\tau(u, v) > h$, so the query goes right.

If we did this independently at every recursive node, we would repeatedly process the same old edges. The crucial observation is that we can simplify the graph separately for the two recursive calls.

For the left half, an edge joining two different SCCs at time h is irrelevant. Such an edge cannot participate in any directed cycle at any earlier time: if it did, its endpoints would already belong to the same SCC at time h.

For the right half, every SCC present at time h will remain strongly connected forever. We may therefore contract each such SCC to a single vertex.

Thus every relevant edge is sent to only one child.

Algorithm 74 Offline Incremental SCCs

Input: Vertices V, chronological edges $E = \langle e_1, \ldots, e_m \rangle$
Output: The edges whose insertion strictly changes the SCC partition

```
Q ← ⟨(i, u_i, v_i) : e_i = (u_i, v_i)⟩
τ_i ← m + 1 for every i
A ← ⟨(i, u_i, v_i) : e_i = (u_i, v_i)⟩
SOLVE(0, m + 1, V, A, Q, τ)
M ← ⟨⟩
for i = 1, ..., m do
    if τ_i = i then
        append e_i to M
return M
function SOLVE(l, r, V', A, Q, τ)
    if l = r then
        for (i, u, v) ∈ Q do
            τ_i ← l
        return
    h ← ⌊(l + r)/2⌋
                                  ▷ SCCs at the midpoint using edges already present by time h
    C ← SCC(V', ⟨(u, v) : (t, u, v) ∈ A, t ≤ h⟩)
    A_L, A_R, Q_L, Q_R ← ⟨⟩
    for (t, u, v) ∈ A do
        if t ≤ h and C[u] = C[v] then
            append (t, u, v) to A_L
        else if C[u] ≠ C[v] then
            append (t, C[u], C[v]) to A_R
                        ▷ If t > h and C[u] = C[v], this future edge is already internal and can be discarded
    for (i, u, v) ∈ Q do
        if C[u] = C[v] then
            append (i, u, v) to Q_L
        else
            append (i, C[u], C[v]) to Q_R
    V_R ← {C[v] : v ∈ V'}
    SOLVE(l, h, V', A_L, Q_L, τ)
    SOLVE(h + 1, r, V_R, A_R, Q_R, τ)
```

Why the Edge Reduction is Correct The query routing is immediate from monotonicity of SCCs. The only issue is showing that the reduced edge sets still contain all information needed by the two recursive calls.

Let C be the SCC partition at time h.

For the left call, consider an edge $e = (u, v)$ present by time h with $C[u] \neq C[v]$. This edge cannot affect the SCC partition at any time $t \leq h$. Indeed, if it belonged to a directed cycle at such a time, then u and v would be mutually reachable at time t, and hence also at time h, contradicting $C[u] \neq C[v]$. More generally, the edges between different C-components form the condensation DAG, and deleting edges of this DAG does not alter the SCCs themselves.

Hence the left recursion needs only the edges already present by time h whose endpoints lie in the same C-component.

For the right call, every C-component is already strongly connected at time h, so it remains strongly connected at every later time. Contracting it is therefore exact. Edges already internal to a C-component can be discarded. Every other edge is represented simply by the edge between the corresponding contracted components.

Notice also that if an edge arrives only after time h but its endpoints already satisfy $C[u] = C[v]$, then it can be discarded permanently: when it eventually arrives, its endpoints are already strongly connected, so it can never change the SCC partition.

These are precisely the rules used by Algorithm 74.

Theorem 527. *Algorithm 74 returns exactly the edges whose insertion strictly changes the strongly connected component partition, in*

$$O(m \log m)$$

time, apart from the initial $O(|V|)$ cost of reading isolated vertices.

Proof. We prove recursively that every query (i, u, v) passed to a call $\textsc{Solve}(l, r, \dots)$ satisfies

$$\tau(u, v) \in [l, r],$$

where the vertices are interpreted after undoing the contractions made on the path from the root.

At a midpoint h, the preceding discussion shows that the SCC partition computed from the reduced graph is exactly the SCC partition of G_h, modulo the contractions already made.

If $C[u] = C[v]$, then the two query vertices are strongly connected by time h, so their threshold lies in $[l, h]$. Otherwise their threshold is strictly larger than h, so it lies in $[h + 1, r]$. Thus every query is sent to the child containing its threshold.

The edge-reduction argument above proves that the reduced graph passed to each child has exactly the same SCC partitions, throughout that child's time interval, as the corresponding original graphs. Hence the invariant continues to hold recursively.

At a leaf $l = r$, the only possible threshold is l, so assigning $\tau_i = l$ is correct.

Finally, as observed at the beginning, $e_i = (u_i, v_i)$ changes the SCC partition exactly when $\tau(u_i, v_i) = i$. Therefore the returned list M is correct.

For the running time, implement each recursive call only on vertices appearing as endpoints of its active edges or queries. Computing SCCs and partitioning the lists then takes linear time in the size of that call.

Every active edge is passed to at most one child, and every query is passed to exactly one child. Hence the total size of all calls at any fixed recursion depth is $O(m)$. The recursion has $O(\log m)$ levels, giving total time

$$O(m \log m).$$

□

12.17 Arboricity

Let $G = (V, E)$ be a simple undirected graph with n vertices and m edges. Its *arboricity* $a(G)$ is the minimum number of forests whose edge sets partition E. We compute $a(G)$ in $O(n + m^2)$ time and $O(n + m)$ space, using only graph searches and edge reassignments.

Write $E(U)$ for the edges with both endpoints in U. For an integer $k \geq 0$, the Nash–Williams forest decomposition theorem gives

$$a(G) \leq k \quad \Longleftrightarrow \quad |E(U)| \leq k(|U| - 1) \quad \text{for every nonempty } U \subseteq V. \tag{290}$$

Assigning each edge to one of its endpoints, with capacity k at each vertex, gives the weaker bound $|E(U)| \leq k|U|$: each edge inside U uses one unit of capacity in U. To obtain $k(|U| - 1)$, choose a vertex r and remove its capacity. This tests the stronger inequality for every set containing r. We will perform these tests for all r, reusing one assignment so that only m edges need to be reinserted across all tests.

12.17.1 Assigning Edges to Endpoints

For nonnegative integer capacities $b(v)$, an *assignment* gives each edge $e = uv$ an owner $o(e) \in \{u, v\}$. Its load at v is $\ell(v) = |\{e : o(e) = v\}|$, and it is feasible if $\ell(v) \le b(v)$ for every v. During the algorithm some edges are unassigned, written $o(e) = \perp$.

Suppose we want to assign an edge uv. Direct each already assigned edge from its owner to its other endpoint. A directed path from u or v to a vertex with spare capacity tells us how to make room: move the last edge's ownership to the last vertex, then work backwards along the path. Each move fills the space just freed by the preceding move. Finally the starting endpoint has room for uv.

Lemma 528 (Augmenting an endpoint assignment)**.** *Given a feasible partial assignment and an unassigned edge uv, a search from u, v either extends the assignment to include uv, or finds a set R containing u, v such that*

$$|E(R)| > \sum_{x \in R} b(x). \tag{291}$$

The search and any reassignments take $O(n + m)$ time.

Proof. Search along the directed edges described above. If a reached vertex has spare capacity, reassign along the search path backwards and then assign uv to the starting endpoint. Only the final vertex of the path gains load; intermediate vertices exchange one owned edge for another.

Otherwise let R be the set of reached vertices. Every vertex of R is full, and every edge owned by a vertex of R has its other endpoint in R, since the search would otherwise reach that endpoint. Conversely, every assigned edge inside R has its owner in R. Thus exactly $\sum_{x \in R} b(x)$ assigned edges lie inside R. The additional unassigned edge uv proves (291). A search scans each vertex and edge at most a constant number of times, and the reassignment path has at most $n - 1$ edges. □

In particular, the same search proves the complete feasibility criterion: an assignment exists if and only if $|E(U)| \le \sum_{v \in U} b(v)$ for every $U \subseteq V$. Necessity follows by counting owners. For sufficiency, start with no assigned edges and insert them one at a time; a failed search would contradict the assumed inequalities.

Algorithm 75 Assign one edge, or certify insufficient capacity

Input: An unassigned edge $e = uv$, capacities b, and a feasible partial assignment (o, ℓ)
Output: Assign e and return true, or leave (o, ℓ) unchanged and return false

```
function AssignEdge(e = uv, b, o, ℓ)
    Mark all vertices unseen; set t ← ⊥
    Mark u, v seen; set pred(u), pred(v) ← ⊥
    Initialize a queue Q containing u, v
    while Q ≠ ∅ do
        Remove a vertex x from Q
        if ℓ(x) < b(x) then
            t ← x; break
        for each edge h = xy incident to x do
            if o(h) = x and y is unseen then
                Mark y seen; set pred(y) ← (x, h); append y to Q
    if t = ⊥ then
        return false
    while pred(t) ≠ ⊥ do
        (x, h) ← pred(t)
        o(h) ← t; ℓ(t) ← ℓ(t) + 1; ℓ(x) ← ℓ(x) − 1
        t ← x
    o(e) ← t; ℓ(t) ← ℓ(t) + 1
    return true
```

The assignment and load arrays in AssignEdge are modified in place. The adjacency lists are those of the original undirected graph; checking $o(h) = x$ selects the directed edges available from x. No adjacency lists need to be changed when an owner changes.

12.17.2 The Minimum Uniform Capacity

Let $p(G)$ be the least p for which every edge can be assigned with capacity p at each vertex. This lower bound can be computed while inserting the edges. Start with $p = 0$. When a search fails at capacity p, its reached set certifies that this capacity is insufficient. Increasing p by one then makes room at every vertex, so the new edge can be assigned immediately to either endpoint.

Lemma 529. *Algorithm 76 computes $p(G)$ and a feasible assignment of capacity $p(G)$, using exactly one call to* AssignEdge *per edge.*

Proof. Induct on the inserted edges, maintaining that p is their minimum uniform capacity. A successful search preserves feasibility at p; a smaller capacity remains impossible because it was already impossible before the insertion. If the search fails, its reached set R contains more than $p|R|$ edges among the previously inserted edges and the new edge, so capacity p is impossible. The old loads were at most p; after increasing the capacity to $p + 1$, assigning the new edge to either endpoint is feasible. Thus the new minimum capacity is exactly $p + 1$. □

Algorithm 76 Compute the minimum uniform endpoint capacity

Input: An undirected graph $G = (V, E)$
Output: The minimum uniform capacity p and a feasible assignment (o, ℓ)

```
1: function MinimumCapacity(G)
2:     p ← 0; set o(e) ← ⊥ for every e ∈ E
3:     Set b(v) ← 0 and ℓ(v) ← 0 for every v ∈ V
4:     for each edge e = uv ∈ E do
5:         if not AssignEdge(e, b, o, ℓ) then
6:             p ← p + 1; set b(v) ← p for every v ∈ V
7:             o(e) ← u; ℓ(u) ← ℓ(u) + 1
8:     return (p, o, ℓ)
```

For a simple graph, this capacity leaves only two possible arboricities.

Lemma 530. *For every simple graph G, $p(G) \le a(G) \le p(G) + 1$.*

Proof. Root every component of each forest in a partition into $a(G)$ forests, and assign each edge to its endpoint farther from the root. Each vertex owns at most one edge from each forest, giving $p(G) \le a(G)$.

For the upper bound, put $p = p(G)$ and take a nonempty $U \subseteq V$ of size s. If $s \ge p + 1$, an assignment of capacity p gives $|E(U)| \le ps \le (p + 1)(s - 1)$. If $1 \le s < p + 1$, simplicity gives $|E(U)| \le \binom{s}{2} \le (p + 1)(s - 1)$. Now apply (290). □

12.17.3 Testing the Missing Unit of Capacity

Fix a minimum-capacity assignment o_0, write ℓ_0 for its loads, and set $p = p(G)$. For each vertex r, test capacities $b_r(r) = 0$ and $b_r(v) = p$ for $v \ne r$. Start from o_0, unassign *all* edges owned by r, and then reinsert these edges with AssignEdge. All other edges are already feasibly assigned.

Every test starts from the same saved assignment, not from the result of the preceding test. Therefore an edge is removed in exactly one test: the test for its original owner. Although there are up to n tests, together they require only m reinsertion searches.

Lemma 531. *All root tests succeed if and only if $a(G) = p$. A failed test produces a set R with $|E(R)| > p(|R| - 1)$.*

Proof. Suppose the test for r fails. The failed edge was originally owned by r, so it is incident to r. The search starts from both endpoints; its reached set R therefore contains r. By Lemma 528, $|E(R)| > \sum_{v \in R} b_r(v) = p(|R| - 1)$. Now (290) gives $a(G) > p$.

Conversely, suppose all tests succeed. For any nonempty $U \subseteq V$, choose $r \in U$ and use the successful assignment for r. Every edge in $E(U)$ is owned inside U, where the total capacity is $p(|U| - 1)$. Hence all inequalities in (290) hold, so $a(G) \leq p$. The reverse inequality follows from Lemma 530. □

Algorithm 77 Exact arboricity of a simple graph

Input: A simple undirected graph $G = (V, E)$
Output: The arboricity value $a(G)$

```
function ArboricityCapacity(G)
    if E = ∅ then
        return 0
    Remove isolated vertices and relabel the remaining vertices consecutively
    (p, o_0, ℓ_0) ← MinimumCapacity(G)
    Build the lists L_r = {e ∈ E : o_0(e) = r} for all r ∈ V
    for each r ∈ V with L_r ≠ ∅ do
        Copy o ← o_0 and ℓ ← ℓ_0; set b(v) ← p for every v ∈ V
        for each e ∈ L_r do
            o(e) ← ⊥
        ℓ(r) ← 0; b(r) ← 0
        for each e ∈ L_r do
            if not AssignEdge(e, b, o, ℓ) then
                return p + 1
    return p
```

Theorem 532. *Algorithm 77 computes the arboricity of a simple undirected graph in $O(n + m^2)$ time and $O(n + m)$ space.*

Proof. Removing isolated vertices does not change arboricity. If a saved owner list L_r is empty, o_0 already satisfies b_r, so its test may be skipped. The remaining tests are correct by Lemma 531, and Lemma 530 justifies returning $p + 1$ on failure.

Preprocessing takes $O(n + m)$ time. Thereafter at most $2m$ vertices remain, so every search, including clearing its search arrays, costs $O(m)$. Computing p performs m searches. Across the root tests, the number of searches is at most

$$\sum_{r \in V} |L_r| = m.$$

There are at most m nonempty owner lists, and restoring the saved assignment and capacities costs $O(m)$ per test. Capacity increases in MinimumCapacity also cost at most $O(m)$ per inserted edge. The total time is therefore $O(n + m^2)$. Adjacency lists, the saved and working assignments, owner lists, and search arrays use $O(n + m)$ space. □

12.18 An Algebraic Criterion for Bipartite Perfect Matching

Let $G = (L \sqcup R, E)$ be bipartite with $L = R = [n]$, and let $M \in \{0, 1\}^{n \times n}$ be its adjacency matrix. By Hall's theorem, G has no perfect matching exactly when some $I \subseteq L$ satisfies $|N(I)| < |I|$. We will encode this condition as a nontrivial kernel of an explicit matrix.

The idea is to give every left vertex the same number of polynomial coefficients, and to impose the same number of linear constraints at every right vertex. A deficient set then leaves more variables than constraints. For the converse, we must prevent the right-vertex constraints from becoming dependent. We do so by forcing certain polynomials to vanish to high order at distinct points.

High-multiplicity zeros in a small space. An s-dimensional polynomial space always contains a nonzero polynomial vanishing to order at least $s-1$ at any prescribed point: vanishing of the first $s-1$ derivatives imposes only $s-1$ homogeneous linear conditions. Greater multiplicity places additional constraints on the space. The Wronskian shows that these extra orders of vanishing cannot occur too often.

For a polynomial space V, write $N_V(\gamma) = \max_{0 \neq f \in V} \operatorname{mult}_\gamma(f)$ and $t_+ = \max\{t, 0\}$.

Lemma 533 (Wronskian multiplicity bound)**.** *If V has dimension s and degree at most d over a field of characteristic greater than d, then*

$$\sum_{\gamma \in \mathbb{F}} \big(N_V(\gamma) - s + 1\big)_+ \leq s(d - s + 1). \tag{292}$$

Proof. Choose a basis $f_1, \dots, f_s$ with distinct degrees $d_1 < \cdots < d_s \leq d$, by eliminating leading coefficients, and form its Wronskian

$$\mathcal{W}(X) = \det\big(f_j^{(i-1)}(X)\big)_{i,j=1}^{s}.$$

Its leading coefficient is the product of the leading coefficients of the f_j and the Vandermonde $\prod_{i<j}(d_j - d_i)$, which is nonzero. Moreover,

$$\deg \mathcal{W} = \sum_{j=1}^{s} d_j - \binom{s}{2} \leq s(d - s + 1).$$

Fix γ and choose $f \in V$ vanishing there to order $t = N_V(\gamma)$. Make f the first basis element. This changes $\mathcal{W}$ only by a nonzero scalar. Since each of $f, f', \dots, f^{(s-1)}$ is divisible by $(X-\gamma)^{(t-s+1)_+}$, so is the first column of its Wronskian. Thus $\operatorname{mult}_\gamma(\mathcal{W}) \geq (t - s + 1)_+$. Summing over the distinct roots of the nonzero polynomial $\mathcal{W}$ proves the claim. □

The polynomial system. Set

$$a = (n-1)^2, \qquad \ell = a + n = n^2 - n + 1, \qquad k = n(a+1). \tag{293}$$

Work over a field of characteristic greater than $k + a$, with $2n$ distinct elements $\alpha_1, \dots, \alpha_n, \beta_1, \dots, \beta_n$. For each left vertex i, introduce $a+1$ variables $z_{i,0}, \dots, z_{i,a}$ and the polynomial

$$P_i(X) = \sum_{r=0}^{a} z_{i,r}(X - \alpha_i)^{k+r}. \tag{294}$$

A nonzero P_i has a zero of multiplicity at least k at its private point α_i. For every right vertex j, put $Q_j(X) = \sum_i M_{ij} P_i(X)$ and require

$$(X - \beta_j)^\ell \mid Q_j(X). \tag{295}$$

Each right vertex contributes at most ℓ homogeneous linear equations. Hence a set of s active left vertices supplies $s(a+1)$ variables, while $s-1$ neighbors impose at most $(s-1)\ell$ equations.

The parameters reflect the two inequalities needed below: $s(a+1) > (s-1)\ell$ for every $s \leq n$, and $\ell > a + n - 1$. Taking $\ell = a + n$ permits the smallest integral a, namely $(n-1)^2$; taking $k = n(a+1) = O(n^3)$ makes the private polynomials independent.

Theorem 534 (Polynomial Hall criterion)**.** *The system (295) has a nonzero solution if and only if G has no perfect matching.*

Proof. Suppose $I \subseteq L$ has $s = |I| > |N(I)|$. Set $P_i = 0$ outside I. Only the constraints at $N(I)$ remain, so there are at most $(s-1)\ell$ equations in $s(a+1)$ variables. Their difference is

$$s(a+1) - (s-1)\ell = \ell - (n-1)s \geq 1,$$

and hence a nonzero solution exists.

Conversely, suppose there is a nonzero solution, and let $I = \{i : P_i \neq 0\}$. We first show that its $t = |I|$ polynomials are linearly independent. Otherwise their span V has dimension $r < t$. It has degree at most $k + a$ and contains, at each of the t distinct points α_i, a nonzero polynomial vanishing to order at least k. By the Wronskian bound,

$$t(k - r + 1) \leq r(k + a - r + 1).$$

As $t \geq r + 1$, this implies $k + 1 \leq r(a+1) \leq (n-1)(a+1)$, contrary to $k = n(a+1) > (n-1)(a+1)$.

This independence ensures that every neighbor of I gives a nonzero polynomial, so its vanishing constraint has content. Write $s = |I| = \dim V$. If G has a perfect matching, Hall's condition gives $|N(I)| \geq s$. For every $j \in N(I)$, the polynomial Q_j is nonzero: it is a nontrivial linear combination of the independent P_i. Thus V contains s nonzero polynomials with a k-fold zero at distinct points α_i, and at least s nonzero polynomials with an ℓ-fold zero at distinct points β_j. The multiplicity bound yields

$$s(k - s + 1) + s(\ell - s + 1) \leq s(k + a - s + 1),$$

so $\ell \leq a + s - 1 \leq a + n - 1$, contradicting $\ell = a + n$. Therefore a nonzero solution excludes a perfect matching. □

The rank test. For a polynomial f, let $D_t f(\beta) = [Z^t] f(\beta + Z)$ be its t-th Hasse derivative. Divisibility by $(X - \beta)^\ell$ is equivalent to $D_t f(\beta) = 0$ for $0 \leq t < \ell$. For the shifted powers in (294), the coefficients are simply

$$D_t (X - \alpha_i)^{k+r}\big|_{X=\beta_j} = \binom{k+r}{t} (\beta_j - \alpha_i)^{k+r-t}.$$

Consequently, define an $n(a+1) \times n\ell$ matrix, with rows indexed by (i, r) and columns by (j, t), by

$$F_{(i,r),(j,t)} = M_{ij} \binom{k+r}{t} (\beta_j - \alpha_i)^{k+r-t}, \quad 0 \leq r \leq a,\ 0 \leq t < \ell. \tag{296}$$

The entire polynomial system is $zF = 0$, where $z = (z_{i,r})_{i,r}$. By Theorem 534,

$$G \text{ has a perfect matching} \iff \operatorname{rank}(F) = n(a+1).$$

All entries of F can be computed directly after precomputing binomial coefficients and powers; there are $O(n^6)$ entries. The rank test takes $O(n^{3\omega})$ field operations on this $O(n^3) \times O(n^3)$ matrix.

13 Algebraic and Spectral Graph Theory

13.1 The Matrix–Tree Theorem

Lemma 535 (Spanning Tree Minors)**.** *For*

$$F \subseteq E, \qquad |F| = n - 1,$$

$$\det B_r[F, :] = \begin{cases} \pm 1, & F \text{ is a spanning tree}, \\ 0, & \text{otherwise}. \end{cases}$$

Proof. If F contains a cycle, the corresponding incidence rows are dependent. If F is disconnected, its incidence matrix has rank less than $n-1$.

Thus only spanning trees can give a nonzero determinant. For a tree, repeatedly remove a leaf other than r. The corresponding column contains a single nonzero entry ± 1, so expansion along that column reduces the determinant to a smaller tree. Induction gives ± 1. □

A more useful form of Matrix–Tree is the following asymmetric statement.

Theorem 536 (Generalized Matrix–Tree Identity)**.** *Let*

$$W = \operatorname{diag}(w_e)$$

and let

$$A \in \mathbb{F}^{E \times (n-1)}$$

be arbitrary. Then

$$\boxed{\det(B_r^\top W A) = \sum_{T \text{ spanning tree}} \det(B_r[T,:]) \det(A[T,:]) \prod_{e \in T} w_e.}$$

Proof. Apply Cauchy–Binet:

$$\begin{aligned} \det(B_r^\top W A) &= \sum_{\substack{F \subseteq E \\ |F| = n-1}} \det(B_r[F,:]^\top W_F) \det(A[F,:]) \\ &= \sum_{F} \det(B_r[F,:]) \det(A[F,:]) \prod_{e \in F} w_e. \end{aligned}$$

By Lemma 535, every non-tree term vanishes. □

Taking

$$A = B_r$$

gives the usual weighted Matrix–Tree theorem:

$$\boxed{\det(B_r^\top W B_r) = \sum_{T \text{ spanning tree}} \prod_{e \in T} w_e.}$$

The arbitrary second matrix is often useful because it can impose additional combinatorial conditions while the first incidence matrix enforces that the chosen edge set is a tree.

13.1.1 Counting Hamiltonian Paths

Let $G = (V, E)$ be an unweighted graph,

$$|V| = n,$$

and fix vertices s, t.

Give an edge

$$e = \{u, v\}$$

weight

$$w_e = x_u x_v.$$

By Matrix–Tree,

$$P_G(x) := \det(B_r^\top W(x) B_r) = \sum_{T \text{ tree}} \prod_{v \in V} x_v^{\deg_T(v)}.$$

Theorem 537 (Hamiltonian Paths from Matrix–Tree)**.**

$$\boxed{\#\,\mathrm{HamPath}(G;s,t) = \frac{1}{2^n} \sum_{x\in\{-1,1\}^V} x_s x_t P_G(x).}$$

Consequently s–t Hamiltonian paths can be counted in

$$O^*(2^n)$$

time and polynomial space.

Proof. Consider one spanning tree T. Its contribution after multiplying by $x_s x_t$ is

$$x_s^{\deg_T(s)+1} x_t^{\deg_T(t)+1} \prod_{v\notin\{s,t\}} x_v^{\deg_T(v)}.$$

For an integer a,

$$\sum_{x\in\{-1,1\}} x^a = \begin{cases} 2, & a \text{ even}, \\ 0, & a \text{ odd}. \end{cases}$$

Thus T survives the outer sum exactly when

$$\deg_T(s), \deg_T(t)$$

are odd and every other degree is even.

All degrees are positive, and

$$\sum_v \deg_T(v) = 2n-2.$$

The minimum possible sum under these parity conditions is already

$$1+1+2(n-2) = 2n-2.$$

Therefore equality must hold everywhere:

$$\deg_T(s) = \deg_T(t) = 1$$

and

$$\deg_T(v) = 2 \qquad (v \neq s,t).$$

Such a tree is precisely an s–t Hamiltonian path.

Every surviving tree contributes 2^n. □

13.1.2 Counting k-Matchings in a Bipartite Graph

Let

$$G = (U \sqcup V, E), \qquad |U| = n_1 \le n_2 = |V|.$$

Add a new vertex s adjacent to every vertex of $U \cup V$, obtaining H.

Assign weights

$$w(uv) = x_u \qquad (u \in U,\ v \in V),$$

$$w(su) = y_1 \qquad (u \in U),$$

and

$$w(sv) = y_2 \qquad (v \in V).$$

Let

$$P_H(X, y_1, y_2)$$

be the resulting spanning-tree polynomial.

Also define the elementary symmetric polynomial

$$e_k(X) = \sum_{\substack{S \subseteq U \\ |S|=k}} \prod_{u \in S} x_u.$$

Theorem 538 (k-Matching Count)**.**

$$\boxed{\#M_k(G) = \frac{1}{2^{n_1}} \sum_{x \in \{-1,1\}^U} [y_1^{n_1} y_2^{n_2-k}] \left(e_k(X) P_H(X, y_1, y_2)\right).}$$

Hence all size-k matchings can be counted in

$$O^*(2^{n_1}) \le O^*(2^{n/2})$$

time and polynomial space.

Proof. The coefficient

$$[y_1^{n_1} y_2^{n_2-k}]$$

restricts attention to spanning trees containing

- all n_1 edges from s to U;
- exactly $n_2 - k$ edges from s to V.

Such a tree therefore contains exactly k original edges of G.

Fix a term

$$x_S = \prod_{u \in S} x_u, \qquad |S| = k,$$

from $e_k(X)$. For a spanning tree T, the exponent of x_u in

$$x_S P_H$$

is

$$d_T^G(u) + \mathbf{1}[u \in S],$$

where $d_T^G(u)$ counts original G-edges incident to u.

Averaging over

$$x_u \in \{-1, 1\}$$

kills the term unless all these exponents are even.

Since there are exactly k original G-edges,

$$\sum_{u \in U} d_T^G(u) = k.$$

Also

$$|S| = k.$$

The parity condition therefore forces

$$d_T^G(u) = \begin{cases} 1, & u \in S, \\ 0, & u \notin S. \end{cases}$$

The k original edges hence have distinct endpoints in U. They must also have distinct endpoints in V: if

$$u_1v,\ u_2v$$

were both chosen, then because the tree already contains

$$su_1,\ su_2,$$

the four vertices would contain the cycle

$$s - u_1 - v - u_2 - s.$$

Thus the k original edges form a matching.

Conversely, every k-matching extends uniquely to one such spanning tree by adding

- every edge su for $u \in U$;
- every edge sv for unmatched $v \in V$.

Taking S to be the saturated vertices of U, its parity term contributes exactly 2^{n_1}. □

13.1.3 The BEST Theorem

An Euler tour gives an order to the outgoing edges at every vertex. Conversely, prescribing these local orders tells us how to walk: always take the next unused outgoing edge. The only obstruction is that the walk might close before using all edges.

The remedy is to reserve one outgoing edge at each nonroot vertex for its last exit. If these reserved edges form a tree directed toward the root, any region containing unused edges still has a route back to the root. This turns a tour into one arborescence and independent local permutations.

Let $G = (V, E)$ be a finite strongly connected directed multigraph with at least one edge, and suppose every vertex is balanced: $d^+(v) = d^-(v) = d_v$. Parallel edges are distinct; loops are allowed and contribute one to both degrees. Fix an edge e_0 leaving a root r. An Euler tour here is a sequence of all edges, each used once, forming a directed closed walk beginning with e_0. Let τ_r count spanning arborescences directed toward r. For a one-vertex graph, the empty arborescence is counted once.

Theorem 539 (BEST)**.** *The number of Euler tours beginning with the prescribed edge e_0 is*

$$\boxed{\tau_r \prod_{v \in V} (d_v - 1)!.}$$

Proof. We construct a bijection between such tours and an arborescence together with one ordering of $d_v - 1$ edges at every vertex.

A tour determines its last-exit tree. For each $v \neq r$, select the last outgoing edge e_v used at v. If e_v leads to $w \neq r$, the tour must leave w afterward, since it ends at r. Thus the last-exit time of w is strictly later than that of v. These times cannot strictly increase around a directed cycle. Every nonroot vertex has one selected outgoing edge, so following selected edges must terminate at r. They therefore form an arborescence toward r.

Record the outgoing-edge order at each vertex. At $v \neq r$, its last edge is now fixed as e_v, while the other $d_v - 1$ edges may be listed in any order. At r, the first edge is fixed as e_0, leaving its other $d_r - 1$ edges to order.

The last-exit tree prevents premature stopping. Conversely, choose an arborescence T toward r and such local orders. At each nonroot vertex place its tree edge last; at r place e_0 first. Start at r and repeatedly take the next edge in the current vertex's list, stopping when that list is exhausted. No edge is repeated, so the process stops after at most $|E|$ steps.

It cannot stop at $v \neq r$. At such an endpoint, the used trail has entered v once more than it has left. If all d_v outgoing edges had been used, at least $d_v + 1$ incoming edges would have been used, contradicting $d^-(v) = d_v$. Thus it stops at r and the used trail is closed.

Suppose some edges are still unused. The unused graph is balanced, because removing a closed trail subtracts equally many incoming and outgoing edges at every vertex. Choose a vertex v with an unused outgoing edge. It is not r, where the walk stopped. Its tree edge is still unused, because that edge was placed last. The head of this edge has an unused incoming edge, and hence, by balance, an unused outgoing edge. Repeating along T reaches r and forces an unused outgoing edge there, a contradiction.

Thus all edges are used. Their order at each vertex is exactly the prescribed list, and their last-exit tree is T. The two constructions are inverse. There are τ_r choices of T and $(d_v - 1)!$ independent choices of local order at each vertex, proving the formula. □

The bijection. Given the tree and local permutations, recovering the tour needs only one cursor per outgoing-edge list.

Algorithm 78 Recovering an Euler tour from last exits and local orders

Input: G, r, e_0 as above; arborescence T toward r; for each v, a permutation σ_v of its nonfixed outgoing edges

Output: The unique Euler tour with these last exits and local orders

```
for each v ∈ V do
    if v = r then
        L_v ← [e_0] ∥ σ_v
    else
        Let e_v be the outgoing edge of v in T
        L_v ← σ_v ∥ [e_v]
    c_v ← 1
v ← r; E ← an empty list
while c_v ≤ d_v do
    e ← L_v[c_v]; c_v ← c_v + 1
    Append e to E; v ← head(e)
return E
```

Here $\|$ denotes list concatenation. Each edge is stored once and read once, giving $O(|V| + |E|)$ time and space. Strong connectivity supplies an inward arborescence, for example by searching from r in the graph with all arcs reversed and then reversing the search tree's arcs.

Counting tours by a determinant. Let a_{uv} be the number of arcs $u \to v$, including a_{vv} loops, and put

$$L_{\text{out}} = \operatorname{diag}(d_v) - (a_{uv})_{u,v \in V}.$$

By the Matrix-Tree theorem,

$$\#\{\text{Euler tours beginning with } e_0\} = \det L_{\text{out}}[\bar{r}, \bar{r}] \prod_{v \in V} (d_v - 1)!, \qquad \bar{r} = V \setminus \{r\}.$$

Thus construct the Laplacian, delete the root row and column, evaluate one determinant, and multiply the factorials. For $n = |V|$ and $m = |E|$, This gives $O(m + n^\omega)$ arithmetic operations.

If only the starting vertex r is fixed, there are d_r possible first edges, so multiply the theorem's answer by d_r. If tours are identified up to cyclic rotation of their edge sequences, no extra factor is needed: every cyclic tour has exactly one representative beginning with the particular edge e_0. Parallel edges remain distinct under both conventions.

This also shows that τ_r is independent of the root in a balanced strongly connected digraph. The number of cyclic tours does not depend on the edge chosen to mark its beginning, and the factorial product does not depend on the root. Applying BEST with an edge leaving each root therefore gives the same τ_r.

13.2 Cheeger Inequality

Let

$$G = (V, E, w)$$

be an undirected graph with nonnegative edge weights. Define

$$d_v = \sum_u w_{uv}, \qquad \mathrm{vol}(S) = \sum_{v \in S} d_v,$$

and

$$w(S, \bar{S}) = \sum_{\substack{u \in S \\ v \notin S}} w_{uv}.$$

The conductance of S is

$$\phi(S) = \frac{w(S, \bar{S})}{\min\{\mathrm{vol}(S), \mathrm{vol}(\bar{S})\}},$$

and

$$\phi(G) = \min_{\emptyset \neq S \neq V} \phi(S).$$

Let

$$D = \mathrm{diag}(d_v), \qquad L = D - A, \qquad \mathcal{L} = D^{-1/2} L D^{-1/2}$$

be the normalized Laplacian.

Equivalently, its second eigenvalue satisfies

$$\lambda_2 = \min_{\substack{f \neq 0 \\ \sum_v d_v f_v = 0}} \frac{\sum_{\{u,v\} \in E} w_{uv}(f_u - f_v)^2}{\sum_v d_v f_v^2}.$$

Theorem 540 (Weighted Cheeger Inequality)**.** *For every connected weighted graph,*

$$\boxed{\frac{\lambda_2}{2} \leq \phi(G) \leq \sqrt{2\lambda_2}.}$$

The Easy Direction

Lemma 541.

$$\lambda_2 \leq 2\phi(G).$$

Proof. Take any set S with

$$s = \mathrm{vol}(S) \leq \frac{1}{2}\,\mathrm{vol}(V).$$

Let

$$T = \mathrm{vol}(V)$$

and define

$$f_v = \begin{cases} 1 - s/T, & v \in S, \\ -s/T, & v \notin S. \end{cases}$$

Then

$$\sum_v d_v f_v = 0.$$

Only crossing edges contribute to the numerator, and the difference across such an edge is 1. Thus

$$f^T L f = w(S, \bar{S}).$$

Meanwhile,

$$\begin{aligned} f^T D f &= s(1 - s/T)^2 + (T - s)(s/T)^2 \\ &= \frac{s(T-s)}{T} \\ &\geq \frac{s}{2}. \end{aligned}$$

Therefore

$$\lambda_2 \leq \frac{w(S, \bar{S})}{s/2} = 2\phi(S).$$

Minimize over S. □

The Sweep Lemma

Lemma 542 (Sweep Cut)**.** *Let $g : V \to \mathbb{R}_{\geq 0}$ be nonzero and suppose*

$$\operatorname{vol}(\operatorname{supp} g) \leq \frac{1}{2} \operatorname{vol}(V).$$

Then some threshold set

$$S_t = \{v : g_v^2 \geq t\}$$

satisfies

$$\boxed{\phi(S_t) \leq \sqrt{2 \frac{\sum_{\{u,v\}} w_{uv}(g_u - g_v)^2}{\sum_v d_v g_v^2}}.}$$

Proof. We have that

$$\int_0^\infty \operatorname{vol}(S_t)\, dt = \sum_v d_v g_v^2.$$

Similarly,

$$\int_0^\infty w(S_t, \bar{S}_t)\, dt = \sum_{\{u,v\}} w_{uv} |g_u^2 - g_v^2|.$$

Every S_t lies inside $\operatorname{supp} g$, so

$$\operatorname{vol}(S_t) \leq \frac{1}{2} \operatorname{vol}(V).$$

Thus

$$w(S_t, \bar{S}_t) \geq \phi(G) \operatorname{vol}(S_t).$$

Integrating,

$$\phi(G) \leq \frac{\sum_{\{u,v\}} w_{uv} |g_u^2 - g_v^2|}{\sum_v d_v g_v^2}.$$

Now

$$|g_u^2 - g_v^2| = |g_u - g_v|(g_u + g_v),$$

so Cauchy–Schwarz gives

$$\sum_{\{u,v\}} w_{uv}|g_u^2 - g_v^2| \le \left(\sum_{\{u,v\}} w_{uv}(g_u - g_v)^2\right)^{1/2} \left(\sum_{\{u,v\}} w_{uv}(g_u + g_v)^2\right)^{1/2}.$$

Since

$$(g_u + g_v)^2 \le 2g_u^2 + 2g_v^2,$$

the second factor is at most

$$\sqrt{2\sum_v d_v g_v^2}.$$

The result follows. □

The Hard Direction

Proof that $\phi(G) \le \sqrt{2\lambda_2}$. Let f be a λ_2 eigenvector of $D^{-1}L$, which is similar to $\mathcal{L}$:

$$Lf = \lambda_2 Df.$$

Define

$$f_+ = \max(f, 0), \qquad f_- = \max(-f, 0).$$

Their supports are disjoint, so at least one has volume at most half of the total volume. Call that function g.

We claim

$$\frac{g^T Lg}{g^T Dg} \le \lambda_2.$$

For example, if $g = f_+$, then

$$f^T Lf_+ = \lambda_2 f^T Df_+ = \lambda_2 f_+^T Df_+.$$

For each edge,

$$(f_u - f_v)(f_{+,u} - f_{+,v}) \ge (f_{+,u} - f_{+,v})^2,$$

so

$$f^T Lf_+ \ge f_+^T Lf_+.$$

Hence

$$f_+^T Lf_+ \le \lambda_2 f_+^T Df_+.$$

The same proof applies to f_-.

Apply Lemma 542 to g:

$$\phi(G) \le \sqrt{2\,\frac{g^T Lg}{g^T Dg}} \le \sqrt{2\lambda_2}.$$

□

13.2.1 An Explicit Constant-Degree Expander

We give an explicit expander construction on every number of vertices. The basic graph lives on $\mathbb{Z}_m^2$ and uses two translations and two shears.

Recall that for a d-regular graph we use normalized conductance

$$\Phi(G) = \min_{0<|A|\le|V|/2} \frac{|\partial A|}{d|A|}.$$

With the normalized Laplacian convention, Cheeger's inequality is

$$\frac{\Phi(G)^2}{2} \le \lambda_2 \le 2\Phi(G).$$

We will use both directions.

The finite graphs. For $m \ge 2$, let $V_m = \mathbb{Z}_m^2$, and define

$$U(x,y) = (x+1, y), \quad V(x,y) = (x, y+1),$$
$$S(x,y) = (x+y, y), \quad T(x,y) = (x, x+y).$$

All coordinates are taken modulo m.

Let G_m be the undirected multigraph obtained by adding, for every $z \in V_m$, the four edges

$$\{z, Uz\}, \qquad \{z, Vz\}, \qquad \{z, Sz\}, \qquad \{z, Tz\}.$$

Loops and parallel edges are retained, with a loop contributing two to the degree. Since each of the four maps is a permutation, every vertex has two incident edges from each map, so G_m is 8-regular. The translations already make it connected.

Loops cause no difficulty below: they contribute neither to edge boundaries nor to Dirichlet energy, and the proof of Cheeger's inequality is unchanged for multigraphs with loops.

The shear graph on the integer lattice. Consider first the infinite graph Γ on $\mathbb{Z}^2 \setminus \{0\}$ with edges $\{z, Sz\}$ and $\{z, Tz\}$. Thus its neighbors are $S^{\pm1}z, T^{\pm1}z$, and Γ is 4-regular.

Lemma 543 (Expansion of the lattice shear graph)**.** *Every finite $A \subseteq \mathbb{Z}^2 \setminus \{0\}$ satisfies*

$$\boxed{|\partial_\Gamma A| \ge |A|.}$$

Proof. Partition the nonzero lattice into

$$Q_1 = \{(x,y) : x > 0,\ y \ge 0\}, \quad Q_2 = \{(x,y) : x \le 0,\ y > 0\},$$
$$Q_3 = \{(x,y) : x < 0,\ y \le 0\}, \quad Q_4 = \{(x,y) : x \ge 0,\ y < 0\}.$$

On Q_1, both S and T preserve the region, and their images are disjoint. Indeed, if $(X,Y) = S(x,y)$, then $X > Y$, whereas if $(X,Y) = T(x,y)$, then $Y \ge X$.

Hence, for $A_1 = A \cap Q_1$, the $2|A_1|$ edges $z \to Sz$ and $z \to Tz$ have $2|A_1|$ distinct targets in Q_1. At most $|A_1|$ of these targets belong to A_1, so at least $|A_1|$ of the edges leave A.

The same argument works on Q_3 using S, T. On Q_2 and Q_4, use S^{-1}, T^{-1}. These maps preserve the corresponding region, and their images are disjoint: on Q_2, the two images are distinguished by $X + Y \le 0$ and $X + Y > 0$, while on Q_4 they are distinguished by $X + Y \ge 0$ and $X + Y < 0$.

The chosen edges for different regions remain inside different regions, so they are distinct. Summing gives

$$|\partial_\Gamma A| \ge \sum_{i=1}^{4} |A \cap Q_i| = |A|.$$

□

Thus the normalized conductance of the 4-regular graph Γ is at least $1/4$. Although Γ is infinite, the proof of the lower bound in Cheeger's inequality applies verbatim to finitely supported functions, since every level set appearing in that proof is finite. Hence every finitely supported $a : \mathbb{Z}^2 \setminus \{0\} \to \mathbb{C}$ satisfies

$$\frac{\sum_z \big(|a(z) - a(Sz)|^2 + |a(z) - a(Tz)|^2\big)}{4\sum_z |a(z)|^2} \geq \frac{(1/4)^2}{2} = \frac{1}{32}.$$

Equivalently,

$$\boxed{\sum_z \big(|a(z) - a(Sz)|^2 + |a(z) - a(Tz)|^2\big) \geq \frac{1}{8}\sum_z |a(z)|^2.} \tag{297}$$

The same inequality holds for complex-valued a by applying the real-valued statement to its real and imaginary parts.

Finite Fourier transform. Put $\omega = e^{2\pi i/m}$. For $f : \mathbb{Z}_m^2 \to \mathbb{C}$, define

$$\widehat{f}(a,b) = \frac{1}{m^2} \sum_{x,y \in \mathbb{Z}_m} f(x,y)\omega^{-ax-by}.$$

Everything we need follows from

$$\frac{1}{m}\sum_{x=0}^{m-1} \omega^{kx} = \begin{cases} 1, & k = 0 \pmod m, \\ 0, & k \neq 0 \pmod m. \end{cases}$$

For $k \neq 0$, this is just the geometric-series identity $(1 - \omega^{km})/(1 - \omega^k) = 0$.

Lemma 544 (Finite Fourier identities)**.** *For every $f : \mathbb{Z}_m^2 \to \mathbb{C}$,*

$$\boxed{\sum_{\xi \in \mathbb{Z}_m^2} |\widehat{f}(\xi)|^2 = \frac{1}{m^2} \sum_{z \in \mathbb{Z}_m^2} |f(z)|^2.} \tag{298}$$

Moreover,

$$\widehat{f \circ U}(a,b) = \omega^a \widehat{f}(a,b), \qquad \widehat{f \circ V}(a,b) = \omega^b \widehat{f}(a,b),$$

and if M is an invertible linear map on $\mathbb{Z}_m^2$, then

$$\widehat{f \circ M}(\xi) = \widehat{f}(M^{-\mathsf{T}}\xi).$$

Proof. Expanding the left side of (298) gives

$$\sum_\xi |\widehat{f}(\xi)|^2 = \frac{1}{m^4} \sum_{z,w} f(z)\overline{f(w)} \sum_\xi \omega^{-\langle \xi, z-w\rangle}.$$

The inner sum is m^2 when $z = w$ and zero otherwise, by the geometric-series identity in each coordinate. This proves Parseval.

The remaining identities follow directly from the definition by changing variables in the finite sum. □

The matrices of S and T satisfy $S^{\mathsf{T}} = T$. Consequently Parseval, followed by a reindexing of the frequencies, gives

$$\begin{aligned} \frac{1}{m^2}\sum_z |f(z) - f(Sz)|^2 &= \sum_\xi |\widehat{f}(\xi) - \widehat{f}(T\xi)|^2, \\ \frac{1}{m^2}\sum_z |f(z) - f(Tz)|^2 &= \sum_\xi |\widehat{f}(\xi) - \widehat{f}(S\xi)|^2. \end{aligned} \tag{299}$$

For the translations,

$$\begin{aligned}\frac{1}{m^2}\sum_z \left(|f(z)-f(Uz)|^2+|f(z)-f(Vz)|^2\right)\\ =\sum_{(a,b)\in\mathbb{Z}_m^2}\rho(a,b)|\widehat{f}(a,b)|^2,\end{aligned} \tag{300}$$

where

$$\rho(a,b)=|1-\omega^a|^2+|1-\omega^b|^2.$$

Cutting open the torus. The Fourier frequencies live in $\mathbb{Z}_m^2$, while (297) lives on the ordinary lattice $\mathbb{Z}^2$. We connect them by choosing one representative of every frequency.

Let

$$I_m=\{-\lfloor m/2\rfloor,\dots,\lceil m/2\rceil-1\},\qquad Q_m=I_m^2.$$

Identify $\widehat{f}$ with a function $h:Q_m\to\mathbb{C}$, and extend it by zero outside Q_m, obtaining $\widetilde{h}:\mathbb{Z}^2\to\mathbb{C}$.

If f has mean zero, then $\widehat{f}(0,0)=m^{-2}\sum_z f(z)=0$, so $\widetilde{h}(0,0)=0$. Thus $\widetilde{h}$ is a finitely supported function on $\mathbb{Z}^2\setminus\{0\}$, exactly as required in (297).

If an integer shear edge has both endpoints in Q_m, its contribution to the energy of $\widetilde{h}$ is exactly the corresponding shear term in (299). We only have to pay for shear edges crossing the boundary of Q_m.

Lemma 545 (Translations pay for wraparound)**.** *With the preceding notation,*

$$\begin{aligned}\sum_{\xi\in\mathbb{Z}^2}\left(|\widetilde{h}(\xi)-\widetilde{h}(S\xi)|^2+|\widetilde{h}(\xi)-\widetilde{h}(T\xi)|^2\right)\\ \le\frac{1}{m^2}\sum_{R\in\{U,V,S,T\}}\sum_{z\in V_m}|f(z)-f(Rz)|^2.\end{aligned}$$

Proof. Suppose $(a,b)\in Q_m$ is incident to a shear edge that leaves Q_m. Then one of $a+b$ or $a-b$ lies outside I_m, and hence

$$|a|+|b|\ge\frac{m}{2}.$$

Set $\alpha=\pi|a|/m$ and $\beta=\pi|b|/m$. Then $0\le\alpha,\beta\le\pi/2$ and $\alpha+\beta\ge\pi/2$. Since $\beta\ge\frac{\pi}{2}-\alpha$ and $\sin^2$ is increasing on this interval,

$$\sin^2\alpha+\sin^2\beta\ge 1.$$

Using the elementary identity

$$|1-e^{i\theta}|^2=2-2\cos\theta=4\sin^2(\theta/2),$$

we have

$$\rho(a,b)=4\sin^2\alpha+4\sin^2\beta\ge 4. \tag{1}$$

A vertex of Q_m is incident to at most four crossing shear edges: one in each of the S,S^{-1},T,T^{-1} directions. Since $\widetilde{h}$ vanishes outside Q_m, their total contribution charged to (a,b) is at most $4|h(a,b)|^2$, which by (1) is at most $\rho(a,b)|h(a,b)|^2$.

Thus the internal shear edges are paid for by (299), while all crossing shear edges are paid for by (300). Summing proves the claim. □

This is the reason the translation generators are useful in the argument: a modular shear may wrap around the boundary of the chosen square, but wraparound can occur only when at least one frequency coordinate is large, and translations have correspondingly large Fourier energy there.

Theorem 546 (Expansion of the square family)**.** *The graph G_m is an explicit 8-regular multigraph on m^2 vertices satisfying*

$$\boxed{|\partial A| \geq \frac{1}{16}|A| \qquad \text{whenever } |A| \leq \frac{m^2}{2}.}$$

Its adjacency lists can be constructed in $O(m^2)$ time.

Proof. Let $f : V_m \to \mathbb{C}$ have mean zero, and let $\widetilde{h}$ be the lifted Fourier transform above. Combining (297) with Lemma 545 and Parseval gives

$$\begin{aligned}\frac{1}{m^2} \sum_{R \in \{U,V,S,T\}} \sum_{z} |f(z) - f(Rz)|^2 &\geq \frac{1}{8} \sum_{\xi \in \mathbb{Z}^2} |\widetilde{h}(\xi)|^2 \\ &= \frac{1}{8m^2} \sum_{z} |f(z)|^2.\end{aligned}$$

Hence

$$\sum_{R \in \{U,V,S,T\}} \sum_{z} |f(z) - f(Rz)|^2 \geq \frac{1}{8} \sum_{z} |f(z)|^2. \tag{2}$$

The left side is the unnormalized Dirichlet energy of G_m. Since G_m is 8-regular, dividing (2) by $8 \sum_z |f(z)|^2$ shows that its normalized spectral gap satisfies

$$\lambda_2(G_m) \geq \frac{1}{64}.$$

The upper bound in Cheeger's inequality, $\lambda_2 \leq 2\Phi(G_m)$, therefore gives

$$\Phi(G_m) \geq \frac{1}{128}.$$

Multiplying by the degree 8 yields

$$\frac{|\partial A|}{|A|} \geq 8\Phi(G_m) \geq \frac{1}{16}$$

for every A of size at most $m^2/2$.

Each vertex and its four generator images are computed using $O(1)$ modular additions, so the graph is constructed in $O(m^2)$ time. □

Exactly N vertices. The square restriction is easily removed. Contracting blocks of size one or two preserves the edge-expansion constant exactly; only the degree increases from 8 to 16.

Theorem 547 (Explicit expanders on every number of vertices)**.** *For every $N \geq 2$, one can construct in $O(N)$ time a 16-regular multigraph H_N on exactly N vertices such that*

$$\boxed{|\partial A| \geq \frac{1}{16}|A| \qquad \text{whenever } |A| \leq \frac{N}{2}.}$$

Proof. Let $m = \lceil \sqrt{N} \rceil$ and $M = m^2$. Then $N \leq M \leq 2N$. Indeed $N \geq (m-1)^2 + 1$, so

$$2N - m^2 \geq 2(m-1)^2 + 2 - m^2 = (m-2)^2 \geq 0.$$

Construct G_m, which has M vertices. Partition its vertices into N nonempty blocks, each of size one or two: take $M - N$ blocks of size two and $2N - M$ blocks of size one. Contract each block to a single vertex, retaining loops and parallel edges.

A block B has degree $8|B|$ after contraction, since degrees add and internal edges become loops. Thus contracted vertices have degree either 8 or 16. Add four loops to each degree-8 vertex. Loops contribute eight to its degree and do not change any cut, so the resulting multigraph H_N is 16-regular.

It remains to check expansion. Let $A \subseteq V(H_N)$ with $|A| \le N/2$, and let $\widetilde{A} \subseteq V(G_m)$ be the union of the blocks corresponding to A. Contraction preserves exactly the edges crossing the cut, so

$$|\partial_{H_N} A| = |\partial_{G_m} \widetilde{A}|.$$

The expansion theorem for G_m, applied to the smaller of $\widetilde{A}$ and its complement, gives

$$|\partial_{G_m} \widetilde{A}| \ge \frac{1}{16} \min\{|\widetilde{A}|, M - |\widetilde{A}|\}.$$

Every block is nonempty, so $|\widetilde{A}| \ge |A|$. The complement contains $N - |A|$ nonempty blocks, and therefore $M - |\widetilde{A}| \ge N - |A| \ge |A|$. Hence

$$|\partial_{H_N} A| \ge \frac{1}{16}|A|.$$

Finally $M \le 2N$, so constructing G_m, forming the blocks, contracting them, and adding loops all take $O(N)$ time. □

13.3 Expander Products

The preceding construction gives a constant-degree expander directly. Graph products let us improve expansion, change the number of vertices, and control the degree in separate steps. The basic operation is squaring: two independent steps of a D-regular walk replace its nontrivial eigenvalues by their squares, but increase the degree to D^2. Repeated squaring quickly makes the graph dense.

We will use a small expander on the D edge labels to reduce this cost. Derandomized squaring uses it to correlate the two edge choices, giving degree Dd instead of D^2. The zig-zag product makes the edge label part of the vertex and needs only two choices from $[d]$, giving degree d^2 on a larger vertex set. Both use the decomposition into averages over edge labels and deviations from those averages.

13.3.1 Lifting a Walk to Edge Labels

We allow regular undirected multigraphs and count each loop twice in the degree. Assign the D incident edge ends at each vertex labels in $[D]$, called *ports*. The rotation map $\mathrm{Rot}_G : V \times [D] \to V \times [D]$ sends (v, a) to (w, b) when these are the two ends of the same edge. It is an involution without fixed points; a loop exchanges two distinct ports at the same vertex. Recording the arrival port matters because the next step will use it to select another edge.

Write $M_G = A_G/D$ and $\mathcal{L}_G = I - M_G$. We measure contraction by $\eta(G) = \|M_G|_{\mathbf{1}^\perp}\|$. Thus $\|M_G x\| \le \eta(G)\|x\|$ for every mean-zero x, and $\eta(G^2) = \eta(G)^2$. The absolute value in this definition is necessary: an eigenvalue near -1 also becomes near 1 on squaring. To obtain absolute contraction from a normalized Laplacian gap, make the walk lazy. The matrix $(I + M_G)/2$ has nonnegative eigenvalues and contraction $1 - \lambda_2(\mathcal{L}_G)/2$. If D is even, adding $D/2$ loops at each vertex realizes it as a $2D$-regular graph.

The set $\{v\} \times [D]$ is the *cloud* above v. Lift a vector on vertices to a cloud-constant vector by the isometry $Q : \mathbb{R}^V \to \mathbb{R}^{V \times [D]}$, defined by $(Qx)_{v,a} = x_v/\sqrt{D}$. Its adjoint satisfies $(Q^\top y)_v = D^{-1/2} \sum_a y_{v,a}$, so $Q^\top Q = I$ and $P = QQ^\top$ replaces the entries in each cloud by their average. Every port vector has the orthogonal decomposition $y = Py + (I - P)y$: its cloud averages and its within-cloud fluctuations.

Let R be the permutation matrix of the rotation. Reversing an edge twice does nothing, so $R = R^\top$ and $R^2 = I$. Lifting a vertex vector, following each port, and averaging at the other end is exactly one step of the original walk:

$$Q^\top R Q = M_G.$$

The rotation itself preserves norm. The contraction of M_G occurs when the final averaging discards variation among ports of the same vertex. This observation will account for the gain in both products.

Now let H be a d-regular graph on $[D]$, and put $W = I_V \otimes M_H$. This takes an H-step independently within each cloud. It preserves cloud averages and contracts fluctuations by at most $\beta = \eta(H)$.

More explicitly, decompose a port vector as $y = Py + (I - P)y$. The first part is constant within each cloud, while the second has mean zero in every cloud. The walk on H fixes the first part and contracts the second by at most β. It also preserves each cloud's average, so $WP = PW = P$.

Thus, setting $E = W - P$, we have $W = P + E$ and $EP = PE = 0$. Moreover,

$$\|Ey\| = \|W(I - P)y\| \le \beta\|(I - P)y\| \le \beta\|y\|,$$

13.3.2 Derandomized Squaring

An ordinary two-step walk first follows a uniformly chosen port of G, then chooses a fresh uniform port at the intermediate vertex. On the space of ports, this fresh choice is the averaging operator P. Indeed, $M_G^2 = Q^\top RPRQ$. Replace that averaging by one step of H: after arriving at port b, choose one of its d neighbors in H and use that port for the second G-step. The resulting walk is $N = Q^\top RWRQ$. It uses one choice from $[D]$ and one from $[d]$, so its graph has the original vertex set and degree Dd.

Algorithm 79 Rotation in the Derandomized Square

Input: Vertex v and edge label $(a, i) \in [D] \times [d]$
Output: The opposite endpoint and its reverse edge label

1: $(w, b) \leftarrow \mathrm{Rot}_G(v, a)$
2: $(c, j) \leftarrow \mathrm{Rot}_H(b, i)$
3: $(u, e) \leftarrow \mathrm{Rot}_G(w, c)$
4: **return** $(u, (e, j))$

Reversing the path follows the same three rotations in reverse order. A fixed port of the product would force the middle H-rotation to have a fixed port, so the product respects our convention for loops.

The approximation error only sees the fluctuations discarded by P. For $y = RQx$, its squared norm is $\|x\|^2$, while its cloud-constant part has squared norm $\|Q^\top y\|^2 = \|M_G x\|^2$. The available error is therefore proportional to $\|x\|^2 - \|M_G x\|^2$. This is smaller than $\|x\|^2$ precisely when the first walk step preserves most of the vector's norm.

Theorem 548 (Derandomized Squaring)**.** *If $\eta(H) \le \beta \le 1$, then, for every real vector x,*

$$|x^\top (N - M_G^2)x| \le \beta\, x^\top (I - M_G^2)x.$$

Consequently, $(1 - \beta)(I - M_G^2) \preceq I - N \preceq (1 + \beta)(I - M_G^2)$, and

$$\boxed{\eta(G \operatorname{dsq} H) \le \eta(G)^2 + \beta\big(1 - \eta(G)^2\big).}$$

Proof. Put $y = RQx$. Since $W - P$ vanishes on cloud constants,

$$\begin{aligned} |x^\top (N - M_G^2)x| &= |y^\top (W - P)y| \\ &\le \beta\|(I - P)y\|^2 = \beta\big(\|x\|^2 - \|M_G x\|^2\big). \end{aligned}$$

Rearranging gives the two Loewner inequalities. For $x \perp \mathbf{1}$, the same estimate and $x^\top M_G^2 x = \|M_G x\|^2$ give $|x^\top N x| \le \beta\|x\|^2 + (1 - \beta)\|M_G x\|^2 \le [\beta + (1 - \beta)\eta(G)^2]\|x\|^2$. Since N is symmetric, maximizing over unit x proves the contraction bound. □

In particular, if $\gamma = 1 - \eta(G)$, then the new absolute gap is at least $(1 - \beta)\gamma(2 - \gamma)$. For a sufficiently small γ and a fixed $\beta < 1/2$, one step increases the gap by a constant factor. An additive error estimate $\|N - M_G^2\| \le \beta$ would lose this conclusion when $\gamma \ll \beta$. The relative estimate also says that every quadratic form of the normalized Laplacian of G^2 is preserved within a factor $1 \pm \beta$.

13.3.3 The Zig-Zag Product

Derandomized squaring still pays for one arbitrary choice among the D ports. Zig-zag avoids that choice by retaining the port as part of the vertex. Its vertex set is $V \times [D]$. From (v, a), take an H-step to select a nearby port, follow the corresponding edge of G, and take another H-step in the arrival cloud. The middle step is determined by the current port; only the two small steps require choices. Thus G zigzag H has degree d^2 and walk matrix WRW.

Algorithm 80 Rotation in the Zig-Zag Product

Input: Vertex (v, a) and edge label $(i, j) \in [d]^2$
Output: The opposite endpoint and its reverse edge label
1: $(a', i') \leftarrow \mathrm{Rot}_H(a, i)$
2: $(w, b') \leftarrow \mathrm{Rot}_G(v, a')$
3: $(b, j') \leftarrow \mathrm{Rot}_H(b', j)$
4: **return** $((w, b), (j', i'))$

The reverse label uses the two small steps in reverse order. As with derandomized squaring, this gives an involution without fixed ports; here a fixed port would force the middle G-rotation to be fixed.

The two small steps have different roles. The first contracts any variation already present within clouds. If a mean-zero vector is instead constant on clouds, the first step leaves it alone. But its cloud averages after the rotation are governed by G: for such a vector u, $\|PRu\| \le \alpha\|u\|$, where $\alpha = \eta(G)$. Since R preserves norm, at least a $1 - \alpha^2$ fraction of its squared norm has become within-cloud variation. The second small step contracts this newly created variation.

Theorem 549 (Zig-Zag Contraction)**.** *If $\eta(G) \le \alpha \le 1$ and $\eta(H) \le \beta \le 1$, then*

$$\eta(G \text{ zigzag } H) \le f(\alpha, \beta) := \frac{(1-\beta^2)\alpha + \sqrt{(1-\beta^2)^2\alpha^2 + 4\beta^2}}{2} \le \alpha + \beta - \alpha\beta.$$

In particular, the product has absolute gap at least $(1-\alpha)(1-\beta)$, and hence at least this normalized Laplacian gap.

Proof. Work throughout on $\mathbf{1}^\perp$. The compression bound $\|PRP|_U\| \le \alpha$ means that $\|PRu\| \le \alpha\|u\|$ for every $u \in U$.

First replace W by $K = P + \beta(I - P)$, which fixes cloud constants and contracts every fluctuation by exactly β. If $\beta = 0$, then $W = P$ and the desired bound is simply $\|PRP\| \le \alpha$, so assume $\beta > 0$. For any x, put $y = Px + Ex/\beta$. Then $Ky = Wx$, while

$$\|y\|^2 = \|Px\|^2 + \frac{\|Ex\|^2}{\beta^2} \le \|Px\|^2 + \|(I-P)x\|^2 = \|x\|^2.$$

Thus $|x^\top WRWx| = |y^\top KRKy| \le \|KRK\|\,\|x\|^2$, proving $\|WRW\| \le \|KRK\|$.

It remains to bound the eigenvalues of KRK. Since $K^{-1}(KRK)K = RK^2$, every such eigenvalue λ has a nonzero vector z satisfying $RK^2z = \lambda z$. The advantage is that $K^2 = \beta^2 I + (1-\beta^2)P$.

Put $u = Pz$. If $u = 0$, then the eigenvalue equation reduces to $\beta^2 Rz = \lambda z$. Since R preserves norms, this gives $|\lambda| = \beta^2 \le \beta \le f(\alpha, \beta)$.

Now suppose $u \neq 0$. Projecting $RK^2z = \lambda z$ onto U gives the first identity below. Multiplying by R before projecting, and using $R^2 = I$, gives the second:

$$\lambda u = \beta^2 PRz + (1-\beta^2)PRu, \qquad \lambda PRz = u.$$

Multiply the first identity by λ and substitute the second to eliminate PRz. This gives $(\lambda^2 - \beta^2)u = \lambda(1-\beta^2)PRu$.

Since $Pu = u$, we have $|u^\top Ru| = |u^\top PRu| \le \|u\|\,\|PRu\| \le \alpha\|u\|^2$. Taking the inner product of the preceding identity with u therefore yields

$$(\lambda^2 - \beta^2)\|u\|^2 = \lambda(1-\beta^2)u^\top Ru \le |\lambda|(1-\beta^2)\alpha\|u\|^2.$$

Canceling $\|u\|^2 > 0$ gives $|\lambda|^2 - (1-\beta^2)\alpha|\lambda| - \beta^2 \le 0$. Hence $|\lambda|$ is at most the positive root $f(\alpha,\beta)$. Since KRK is symmetric, this proves $\|WRW\| \le \|KRK\| \le f(\alpha,\beta)$.

Finally, $a \mapsto f(a,\beta)$ is convex: its square-root term is the Euclidean norm of $((1-\beta^2)a, 2\beta)$. Using $f(0,\beta) = \beta$ and $f(1,\beta) = 1$, we obtain

$$f(\alpha,\beta) \le (1-\alpha)f(0,\beta) + \alpha f(1,\beta) = \alpha + \beta - \alpha\beta.$$

□

13.4 Alon–Boppana

We now return to simple graphs and unnormalized adjacency matrices. Write $\theta_1(G) \ge \theta_2(G) \ge \cdots$ for the adjacency eigenvalues. A d-regular graph has $\theta_1 = d$; good expansion asks that the other eigenvalues be much smaller. Alon–Boppana gives a limit on how far this can go: for fixed d and growing order, θ_2 cannot stay below $2\sqrt{d-1}$.

The constant comes from a tree. After the first step away from a root, a d-regular tree has $d-1$ forward choices at each vertex. On vectors constant on levels, normalization turns this branching into a half-infinite path with edge weight $\sqrt{d-1}$. We first see this on the binary tree, then solve the finite path to obtain the right test vector. Finally, we place two such vectors in distant graph balls. The balls may contain cycles; the covering tree will let us transfer the vectors without losing their Rayleigh quotient.

13.4.1 The Infinite Binary Tree

We only need finitely supported vectors; the norm below is the supremum of their absolute Rayleigh quotients. Let L_j be level j of the rooted infinite binary tree. It has 2^j vertices. To put one unit of squared norm on a level, give each of its vertices value $2^{-j/2}$; thus $e_j = 2^{-j/2}\mathbf{1}_{L_j}$ form an orthonormal sequence. There are 2^{j+1} edges between levels j and $j+1$, so $\langle e_{j+1}, Ae_j\rangle = \sqrt{2}$. Consequently, for $f = \sum_j c_j e_j$ of finite support,

$$\|f\|^2 = \sum_j c_j^2, \qquad f^\top Af = 2\sqrt{2}\sum_j c_j c_{j+1}.$$

These are exactly the norm and quadratic form of a half-infinite path with edge weight $\sqrt{2}$. In particular, the profile $c_0 = \cdots = c_r = 1$ has Rayleigh quotient $2\sqrt{2}\,r/(r+1)$, approaching $2\sqrt{2}$.

To bound arbitrary vectors, orient the edges from parent to child and use $2|ab| \le ta^2 + t^{-1}b^2$ for $t > 0$. Each non-root vertex occurs as a parent twice and as a child once, so its total coefficient is $2t + t^{-1}$. The best choice balances these contributions: $t = 1/\sqrt{2}$, giving $2\sqrt{2}$. The root receives a smaller coefficient. Summing over edges yields $|f^\top Af| \le 2\sqrt{2}\|f\|^2$ for every finitely supported f. Together with the level-constant vectors above, this proves that the adjacency operator has norm exactly $2\sqrt{2}$.

The same argument explains the general tree bound.

Lemma 550 (Tree Norm Bound)**.** *A finite tree of maximum degree at most $d \ge 2$ has adjacency norm at most $2\sqrt{d-1}$.*

Proof. Root the tree and put $q = d-1$. Every non-root vertex has at most q children, so the preceding edge inequality gives coefficient at most $qt + t^{-1}$. Choose $t = 1/\sqrt{q}$, making this $2\sqrt{q}$. The root has at most d children and receives coefficient at most $d/\sqrt{q} \le 2\sqrt{q}$. Thus $|x^\top A_T x| \le 2\sqrt{q}\|x\|^2$ for all x. □

Lemma 551 (Finite Branching Tree)**.** *For integers $q \ge 1$ and $r \ge 0$, let $T_{q,r}$ be the rooted tree of depth r in which every non-leaf vertex has q children. Then*

$$\rho(T_{q,r}) = 2\sqrt{q}\cos\frac{\pi}{r+2}.$$

Proof. The vectors $e_j = q^{-j/2}\mathbf{1}_{L_j}$ identify the level-constant subspace with a path on $r+1$ vertices whose edge weights are $\sqrt{q}$. Maximizing its quadratic form subject to $\sum_j c_j^2 = 1$ gives the eigenvalue equation

$$\sqrt{q}\,(c_{j-1} + c_{j+1}) = \rho c_j \quad (0 \le j \le r), \qquad c_{-1} = c_{r+1} = 0.$$

If it was infinite we could select complex phases; we need to cancel two of these to get $c_{-1} = c_{r+1} = 0$, producing sines. For $\rho = 2\sqrt{q}\cos\vartheta$, the recurrence is the addition formula for sine and cosine. The left boundary selects $c_j = \sin((j+1)\vartheta)$ up to scale; the right boundary requires $\sin((r+2)\vartheta) = 0$. Taking the smallest positive angle, $\vartheta = \pi/(r+2)$, makes all c_j positive. Thus $f = \sum_j c_j e_j$ is a positive eigenvector of the whole tree with the claimed eigenvalue ρ.

The positive eigenvector f also certifies that ρ is the full adjacency norm. Indeed, let $A_T x = \lambda x$ with $x \neq 0$, and choose u maximizing $|x(u)|/f(u)$. Then

$$|\lambda|\,|x(u)| \le \sum_{v\sim u} |x(v)| \le \frac{|x(u)|}{f(u)} \sum_{v\sim u} f(v) = \rho|x(u)|.$$

Since $|x(u)| > 0$, we obtain $|\lambda| \le \rho$. As ρ itself is an eigenvalue, $\|A_T\| = \rho$. □

This loses only $O(\sqrt{q}/r^2)$ from $2\sqrt{q}$, since $1 - \cos t \le t^2/2$.

13.4.2 From a Tree to a Graph Ball

A large minimum degree supplies many possible continuations of a walk, but they need not reach distinct vertices. We therefore first keep the walks distinct, obtaining a tree, and only then identify walks with the same endpoint. To preserve the denominator of the Rayleigh quotient, combine their values by adding squares and taking a square root. Local injectivity of the covering map makes the numerator increase under this operation.

Lemma 552 (A Spectral Witness in Every Ball)**.** *For every vertex v of a graph of minimum degree at least $d \ge 2$ and every integer $r \ge 0$, there is a nonzero nonnegative vector f supported in $B_G(v,r)$ such that $f^\top A_G f \ge 2\sqrt{d-1}\cos(\pi/(r+2))\|f\|^2$.*

Proof. Form a rooted tree whose vertices are the nonbacktracking walks starting at v, including the empty walk. Here nonbacktracking means that a step never immediately reverses the preceding step; later visits to an old graph vertex are allowed. The parent of a nonempty walk deletes its last step. The root has at least d children, and every other vertex has at least $d-1$ children. Keep exactly $d-1$ children at each vertex and stop at depth r. This leaves $T = T_{d-1,r}$, with its positive eigenvector g from Lemma 551.

Map each tree vertex a to its walk's endpoint $\pi(a)$. Define $f(u) = (\sum_{a:\,\pi(a)=u} g(a)^2)^{1/2}$. Then $\|f\|^2 = \|g\|^2$ and $\operatorname{supp} f \subseteq B_G(v,r)$. To compare quadratic forms, fix a graph edge uv. Its preimages form a matching between the two fibers: at a tree vertex above u, there is at most one incident tree edge above the specified edge uv. For a non-root vertex, that edge is either its parent edge or its unique allowed extension toward v, but cannot be both. Cauchy–Schwarz therefore gives

$$\sum_{ab\in E(T):\,\pi(a)=u,\,\pi(b)=v} g(a)g(b) \le \left(\sum_{\pi(a)=u} g(a)^2\right)^{1/2} \left(\sum_{\pi(b)=v} g(b)^2\right)^{1/2} = f(u)f(v).$$

Summing over graph edges and multiplying by two proves $f^\top A_G f \ge g^\top A_T g = 2\sqrt{d-1}\cos(\pi/(r+2))\|f\|^2$. □

One such vector only lower-bounds θ_1. To reach θ_2, we need a large Rayleigh quotient after removing the top eigendirection. Two witnesses with disjoint, nonadjacent supports provide a two-dimensional space of large Rayleigh quotients; some vector in that space is orthogonal to a top eigenvector. Vertices far apart provide exactly these supports.

Theorem 553 (Alon–Boppana from Diameter)**.** *If a connected graph of minimum degree at least $d \geq 2$ has diameter $D \geq 2r+2$ for an integer $r \geq 0$, then*

$$\boxed{\theta_2(G) \geq 2\sqrt{d-1}\cos\frac{\pi}{r+2}.}$$

More generally, k vertices at pairwise distance at least $2r+2$ give $\theta_k(G) \geq 2\sqrt{d-1}\cos(\pi/(r+2))$. For fixed $d \geq 3$, this implies $\theta_2(G) \geq 2\sqrt{d-1} - O_d((\log n)^{-2})$ for d-regular graphs as $n \to \infty$.

Proof. Choose centers at distance at least $2r+2$. Their radius-r balls are disjoint and no edge joins two balls: such an edge would give a path of length at most $2r+1$ between their centers. Let f_i be the witnesses from Lemma 552. Both $f_i^\top f_j$ and $f_i^\top A_G f_j$ vanish for $i \neq j$. Thus the Rayleigh quotient of any nonzero $\sum_i a_i f_i$ is

$$\frac{\sum_i a_i^2 f_i^\top A_G f_i}{\sum_i a_i^2 \|f_i\|^2} \geq 2\sqrt{d-1}\cos\frac{\pi}{r+2}.$$

The min–max principle now gives the kth-eigenvalue bound, including $k=2$ for two diameter endpoints.

A connected graph of maximum degree d and diameter D has at most $1 + d\sum_{j=0}^{D-1}(d-1)^j$ vertices: count a breadth-first search from one vertex. For fixed $d \geq 3$, this gives $D \geq \log_{d-1} n - O_d(1)$. Take $r = \lfloor (D-2)/2 \rfloor$ and use $1 - \cos t \leq t^2/2$ to obtain the asymptotic bound. If the regular graph is disconnected, d is an eigenvalue at least twice, so $\theta_2 = d$ and the bound also holds. □

The same argument gives many large eigenvalues. Greedily choose centers at pairwise distance at least $2r+2$. Each chosen center excludes at most $1 + d\sum_{j=0}^{2r}(d-1)^j$ vertices, so at least $n/(1 + d\sum_{j=0}^{2r}(d-1)^j)$ eigenvalues satisfy the displayed lower bound. For any fixed d and $\epsilon > 0$, choosing r large enough shows that a positive fraction of the eigenvalues are at least $2\sqrt{d-1} - \epsilon$.

A connected d-regular graph is called *Ramanujan* if every adjacency eigenvalue other than d and, in the bipartite case, $-d$, has absolute value at most $2\sqrt{d-1}$. Alon–Boppana explains the threshold: no smaller uniform bound can hold for an infinite family of fixed-degree graphs.

13.5 Lower Bound on Spectral Sparsification

Theorem 554 (Weighted Spectral Sparsifier Lower Bound)**.** *Let H be a nonnegatively weighted graph on $n \geq 2$ vertices with m positive-weight edges. If $(1-\epsilon)L_{K_n} \preceq L_H \preceq (1+\epsilon)L_{K_n}$ for $0 \leq \epsilon < 1$, then*

$$\boxed{m \geq \frac{n(n-1)(1-\epsilon^2)}{2\big(1+(n-1)\epsilon^2\big)}.}$$

In particular, for every $0 < \epsilon < 1$, one needs $m = \Omega\big(\min\{n^2, n/\epsilon^2\}\big)$.

Proof. Merging parallel edges and discarding loops only decreases m and does not change the Laplacian, so assume H is simple. Put $\Pi = I - \mathbf{1}\mathbf{1}^\top/n$ and $E = L_H - L_{K_n} = L_H - n\Pi$.

We measure the error by its squared Frobenius norm because this is both the sum of squared eigenvalues and the sum of squared entries. The first description makes the approximation assumption useful; the second exposes the missing edges.

Indeed, E vanishes on constants, and its eigenvalues on $\mathbf{1}^\perp$ lie in $[-n\epsilon, n\epsilon]$. Hence

$$\|E\|_F^2 \leq n^2(n-1)\epsilon^2.$$

Now consider the entries. Every missing edge $\{u,v\}$ contributes $E_{uv} = E_{vu} = 1$, since the corresponding entries of L_H and L_{K_n} are 0 and -1. Thus $k = n(n-1) - 2m$ entries of E equal 1. Their sum is therefore k, but the sum of all entries of E is zero, since every row sums to zero. Thus the remaining entries must sum to $-k$.

There are only $n + 2m$ remaining entries: n on the diagonal and two for each edge of H. By Cauchy–Schwarz, their sum of squares is at least $k^2/(n + 2m)$. This is the additional cost of having few edges: the remaining entries must compensate for all the missing-edge errors. Consequently,

$$\|E\|_F^2 \geq k + \frac{k^2}{n + 2m} = \frac{n^2 k}{n + 2m},$$

where the last equality uses $k + n + 2m = n^2$.

Combining this with the spectral upper bound and substituting $k = n(n-1) - 2m$ gives

$$(n-1)\epsilon^2(n + 2m) \geq n(n-1) - 2m.$$

Collecting the terms involving m, we obtain

$$2m\big(1 + (n-1)\epsilon^2\big) \geq n(n-1)(1 - \epsilon^2),$$

which proves the displayed bound.

For $0 < \epsilon \leq 1/2$, this implies

$$m = \Omega\left(\frac{n^2}{1 + n\epsilon^2}\right) = \Omega\big(\min\{n^2, n/\epsilon^2\}\big).$$

For $1/2 < \epsilon < 1$, the lower spectral bound forces H to be connected. Hence $m \geq n - 1 = \Omega(n/\epsilon^2)$, completing the proof. □

14 Electrical Flows

Let

$$G = (V, E, c)$$

be a connected undirected graph with positive edge conductances

$$c_e > 0, \qquad r_e = \frac{1}{c_e}$$

the corresponding resistances.

Fix an arbitrary orientation of the edges. For an oriented edge $e = (u, v)$, write

$$b_e = \mathbf{1}_u - \mathbf{1}_v.$$

Let

$$B \in \mathbb{R}^{E \times V}$$

have rows $b_e^\top$, and let

$$C = \operatorname{diag}(c_e).$$

Then the weighted Laplacian is

$$\boxed{L = B^\top C B.}$$

For a flow $\theta \in \mathbb{R}^E$,

$$B^\top \theta$$

is its divergence vector, and its energy is

$$\boxed{\mathcal{E}(\theta) = \theta^\top C^{-1} \theta = \sum_{e \in E} r_e \theta_e^2.}$$

14.1 Electrical Flows

More generally than s–t flows, let

$$b \in \mathbb{R}^V, \qquad \mathbf{1}^\top b = 0.$$

A flow realizing demand b satisfies

$$B^\top \theta = b.$$

The familiar unit s–t case is

$$b = \mathbf{1}_s - \mathbf{1}_t.$$

The form of the solution to electrical flows can be seen by applying Lagrange multipliers to the optimization problem. Thereby we can see that there exists some $\phi \in \mathbb{R}^n$ such that $\theta = CB\phi$, then we know $L\phi = b$ so $\phi = L^+ b$ is natural.

Theorem 555 (Thomson's Principle)**.** *For every demand $b \perp \mathbf{1}$, the unique minimum-energy flow satisfying*

$$B^\top \theta = b$$

is

$$\boxed{\theta^\star = CBL^+ b.}$$

Its energy is

$$\boxed{\mathcal{E}(\theta^\star) = b^\top L^+ b.}$$

Proof. First,

$$B^\top \theta^\star = B^\top CBL^+ b = LL^+ b = b,$$

since $b \perp \mathbf{1} = \ker L$.

Now let θ be any other feasible flow and write

$$\theta = \theta^\star + z.$$

Then

$$B^\top z = 0.$$

The energy expands as

$$\mathcal{E}(\theta) = \mathcal{E}(\theta^\star) + 2(\theta^\star)^\top C^{-1} z + \mathcal{E}(z).$$

The cross term vanishes:

$$(\theta^\star)^\top C^{-1} z = (CBL^+ b)^\top C^{-1} z = (L^+ b)^\top B^\top z = 0.$$

Therefore

$$\mathcal{E}(\theta) = \mathcal{E}(\theta^\star) + \mathcal{E}(z) \geq \mathcal{E}(\theta^\star),$$

with equality only for $z = 0$.

Finally,

$$\begin{aligned} \mathcal{E}(\theta^\star) &= b^\top L^+ B^\top C C^{-1} CBL^+ b \\ &= b^\top L^+ LL^+ b \\ &= b^\top L^+ b. \end{aligned}$$

□

14.1.1 Potentials and Ohm's Law

A potential

$$\phi : V \to \mathbb{R}$$

induces the flow

$$\theta = CB\phi,$$

or edgewise

$$\boxed{\theta_{uv} = c_{uv}(\phi_u - \phi_v).}$$

Thus Theorem 555 says that the electrical flow for demand b is induced by the potential

$$\boxed{\phi = L^+ b.}$$

Potentials are unique up to adding a constant.

14.1.2 Effective Resistance

For vertices s, t, define

$$b_{st} = \mathbf{1}_s - \mathbf{1}_t.$$

Definition 556 (Effective Resistance)**.**

$$\boxed{R_{\text{eff}}(s,t) = b_{st}^\top L^+ b_{st}.}$$

By Thomson's principle,

$$\boxed{R_{\text{eff}}(s,t) = \min_{\theta:\, B^\top \theta = b_{st}} \mathcal{E}(\theta).}$$

If

$$\phi = L^+ b_{st}$$

is the unit-current potential, then

$$\boxed{R_{\text{eff}}(s,t) = \phi_s - \phi_t = \mathcal{E}(\theta^\star).}$$

Indeed,

$$\phi_s - \phi_t = b_{st}^\top L^+ b_{st}.$$

14.1.3 Maximum Principle

Lemma 557 (Maximum Principle)**.** *Let ϕ be harmonic on*

$$V \setminus S,$$

meaning

$$(L\phi)_v = 0 \qquad (v \notin S).$$

Then every maximum and minimum of ϕ is attained on S.

Proof. At a harmonic vertex v,

$$0 = \sum_{u \sim v} c_{uv}(\phi_v - \phi_u),$$

so

$$\phi_v = \frac{\sum_{u \sim v} c_{uv}\phi_u}{\sum_{u \sim v} c_{uv}}.$$

Thus ϕ_v is a weighted average of its neighbors.

If an interior vertex achieves the global maximum, all of its neighbors must have the same value. By connectedness this propagates until reaching the boundary. The minimum case is identical. □

Because of the maximum principle and nonnegativity of effective resistances we have, for the unit s–t electrical potential,

$$\boxed{\phi_s \geq \phi_v \geq \phi_t \qquad \text{for every } v.}$$

14.1.4 Dirichlet's Principle

Dirichlet's principle is most naturally stated as a boundary-value problem. Fix potentials on some set of boundary vertices; among all extensions to the rest of the graph, the minimum-energy extension is exactly the harmonic one.

Let $G = (V, E)$ be connected, with weighted Laplacian L, and define

$$\mathcal{E}(\phi) = \phi^\top L \phi = \sum_{uv \in E} w_{uv}(\phi_u - \phi_v)^2.$$

Theorem 558 (Dirichlet's Principle)**.** *Let $\emptyset \neq S \subseteq V$, and prescribe boundary values $b \in \mathbb{R}^S$. Among all potentials satisfying*

$$\phi|_S = b,$$

there is a unique minimizer h of $\mathcal{E}(\phi)$. It is characterized by

$$\boxed{(Lh)_v = 0 \qquad (v \in V \setminus S),}$$

that is, h is harmonic at every non-boundary vertex.

Proof. Let $U = V \setminus S$. The harmonicity equations are

$$L_{U,U} h_U = -L_{U,S} b.$$

The matrix $L_{U,U}$ is positive definite. Indeed, if $x \in \mathbb{R}^U$, extend it by 0 on S. Then

$$x^\top L_{U,U} x = \sum_{uv \in E} w_{uv}(x_u - x_v)^2.$$

If this is 0, the extended vector is constant on every edge. Since G is connected and it vanishes on the nonempty set S, it must vanish everywhere. Hence $x = 0$.

Thus there is a unique harmonic extension h.

Now let ϕ be any other extension of the same boundary and write

$$\phi = h + \psi, \qquad \psi|_S = 0.$$

Then

$$\phi^\top L \phi = h^\top L h + 2\psi^\top L h + \psi^\top L \psi.$$

Since h is harmonic on U, the vector Lh is supported on S. Since ψ vanishes on S,

$$\psi^\top L h = 0.$$

Therefore

$$\boxed{\mathcal{E}(\phi) = \mathcal{E}(h) + \mathcal{E}(\psi) \geq \mathcal{E}(h).}$$

Equality forces $\mathcal{E}(\psi) = 0$, so ψ is constant; because it vanishes on S, it is identically zero. Thus h is the unique minimizer. □

Boundary Current and Power For the harmonic extension h, let

$$j = Lh.$$

The coordinate j_v is the net current injected at v. Harmonicity says that $j_v = 0$ away from S, so current enters and leaves the network only through the boundary.

Moreover,

$$\boxed{\mathcal{E}(h) = h^\top Lh = h^\top j = \sum_{v \in S} b_v j_v.}$$

Thus the energy is exactly the power supplied through the boundary: potential times injected current, summed over the boundary vertices.

Define the effective conductance

$$C_{\text{eff}}(s,t) = \frac{1}{R_{\text{eff}}(s,t)}.$$

Corollary 559 (Two-Terminal Dirichlet Principle)**.** *For distinct vertices s, t, let h be harmonic on $V \setminus \{s,t\}$ with*

$$h_s = 1, \qquad h_t = 0.$$

Then

$$\boxed{C_{\text{eff}}(s,t) = \min_{\substack{\phi_s = 1 \\ \phi_t = 0}} \phi^\top L\phi.}$$

Proof. By Theorem 558, h is the unique minimizer.

Let

$$I = (Lh)_s$$

be the total current leaving s. Since Lh vanishes away from s, t, and

$$\mathbf{1}^\top Lh = 0,$$

we have

$$(Lh)_t = -I.$$

Therefore

$$\begin{aligned} h^\top Lh &= h_s(Lh)_s + h_t(Lh)_t \\ &= 1 \cdot I + 0 \cdot (-I) \\ &= I. \end{aligned}$$

Equivalently, the energy is

$$(\text{voltage drop}) \cdot (\text{current}) = 1 \cdot I.$$

Now scale the potential by $1/I$. Currents scale by the same factor, so the new potential sends one unit of current from s to t. Its voltage drop is

$$\frac{1}{I}.$$

Now, by the definition of effective resistance,

$$R_{\text{eff}}(s,t) = \frac{1}{I}.$$

Hence

$$I = \frac{1}{R_{\text{eff}}(s,t)} = C_{\text{eff}}(s,t),$$

and therefore

$$h^\top Lh = C_{\text{eff}}(s,t).$$

□

	current	voltage drop	energy
unit current	1	R_{eff}	R_{eff}
unit voltage	C_{eff}	1	C_{eff}.

14.1.5 Series and Parallel Laws

If two networks of resistances R_1, R_2 are connected in series, every unit flow must pass through both. Hence Thomson's principle immediately gives

$$\boxed{R_{\text{series}} = R_1 + R_2.}$$

If they are connected in parallel and a fraction α of the unit current uses the first branch, the minimum possible energy for this split is

$$\alpha^2 R_1 + (1-\alpha)^2 R_2.$$

Minimizing gives

$$\alpha R_1 = (1-\alpha) R_2$$

and hence

$$\boxed{R_{\text{parallel}} = \frac{R_1 R_2}{R_1 + R_2}.}$$

Equivalently,

$$\boxed{C_{\text{parallel}} = C_1 + C_2.}$$

More generally, parallel currents divide proportionally to conductance.

14.1.6 Rayleigh Monotonicity

Theorem 560 (Rayleigh Monotonicity)**.** *Increasing edge resistances can only increase effective resistance. Equivalently, increasing conductances can only decrease effective resistance.*

Proof. Suppose

$$r'_e \geq r_e$$

for every edge. For every unit s–t flow θ,

$$\sum_e r'_e \theta_e^2 \geq \sum_e r_e \theta_e^2.$$

Taking the minimum over all unit flows and applying Thomson's principle gives

$$R'_{\text{eff}}(s,t) \geq R_{\text{eff}}(s,t).$$

□

Thus:

- deleting edges increases resistance;
- adding edges decreases resistance;
- contracting or shorting vertices decreases resistance.

14.1.7 Effective Resistance is a Metric

Theorem 561. *On the vertices of a connected graph,*

$$R_{\text{eff}}$$

is a metric.

Proof. Nonnegativity, symmetry, and

$$R_{\text{eff}}(s,t) = 0 \iff s = t$$

are immediate from

$$R_{\text{eff}}(s,t) = b_{st}^{\top} L^{+} b_{st}.$$

For the triangle inequality, let

$$p = L^{+}(\mathbf{1}_s - \mathbf{1}_t).$$

By the maximum principle,

$$p_u \geq p_t.$$

Now

$$R_{\text{eff}}(s,t) + R_{\text{eff}}(t,u) - R_{\text{eff}}(s,u) = 2(p_u - p_t) \geq 0.$$

Hence

$$R_{\text{eff}}(s,u) \leq R_{\text{eff}}(s,t) + R_{\text{eff}}(t,u).$$

□

14.2 Random Walks and Electrical Networks

Define the weighted random walk by

$$P(u,v) = \frac{c_{uv}}{c_u}, \qquad c_u = \sum_v c_{uv}.$$

Its stationary distribution is

$$\pi(u) = \frac{c_u}{\text{vol}(G)}, \qquad \text{vol}(G) = \sum_u c_u.$$

This walk is reversible:

$$\boxed{c_u P(u,v) = c_{uv} = c_v P(v,u).}$$

14.2.1 Voltages are Hitting Probabilities

Let

$$T_A = \inf\{j \geq 0 : X_j \in A\}.$$

Theorem 562. *Let*

$$q(u) = \Pr_u[T_s < T_t].$$

Then q is exactly the unit-voltage electrical potential:

$$q(s) = 1, \qquad q(t) = 0,$$

and

$$Lq = 0 \qquad \text{on } V \setminus \{s,t\}.$$

Proof. Conditioning on the first random-walk step gives, for

$$u \notin \{s, t\},$$

$$q(u) = \sum_v P(u, v)q(v).$$

Multiplying by c_u gives

$$\sum_v c_{uv}(q(u) - q(v)) = 0,$$

which is precisely

$$(Lq)_u = 0.$$

The boundary values are clearly 1 and 0. Uniqueness follows from the maximum principle. □

If

$$p = L^+(\mathbf{1}_s - \mathbf{1}_t)$$

is instead the unit-current potential, then

$$\boxed{q(u) = \frac{p_u - p_t}{p_s - p_t} = \frac{p_u - p_t}{R_{\text{eff}}(s, t)}.}$$

14.2.2 Currents are Expected Net Traversals

Run a random walk from s until its first visit to t. For $v \neq t$, let

$$G(v) = \mathbb{E}_s[\#\text{ visits to } v \text{ before } T_t],$$

and set

$$G(t) = 0.$$

Define

$$g(v) = \frac{G(v)}{c_v}.$$

Also let $N_{u\to v}$ be the number of times we walk on the edge $u \to v$ in that direction.

Theorem 563.

$$\boxed{Lg = \mathbf{1}_s - \mathbf{1}_t.}$$

Consequently,

$$g = L^+(\mathbf{1}_s - \mathbf{1}_t) + \alpha\mathbf{1}$$

for some constant α, and the electrical current on (u, v) is

$$\boxed{c_{uv}(g(u) - g(v)) = \mathbb{E}_s[N_{u\to v} - N_{v\to u}].}$$

Proof. For $v \neq t$, every visit to v is either the initial visit when $v = s$ or results from a transition into v:

$$G(v) = \mathbf{1}[v = s] + \sum_u G(u)P(u, v).$$

We have

$$G(u)P(u, v) = c_{uv}g(u).$$

Hence

$$c_v g(v) - \sum_u c_{uv}g(u) = \mathbf{1}[v = s],$$

so

$$(Lg)_v = \mathbf{1}[v = s] \qquad (v \neq t).$$

Because the entries of Lg sum to zero, its t-coordinate is -1.

Also

$$\mathbb{E}N_{u\to v} = G(u)P(u,v) = c_{uv}g(u),$$

so subtraction gives the final identity. □

14.2.3 Hitting a Vertex Through a Given Edge

Corollary 564. *Suppose $\{s,t\} \in E$. A walk starting at s first enters t through the edge $s \to t$ with probability*

$$\boxed{c_{st}R_{\text{eff}}(s,t).}$$

Proof. Before T_t, the edge $s \to t$ can be traversed at most once, and traversing it ends the walk. Thus its expected number of traversals is exactly the desired probability.

By Theorem 563,

$$\mathbb{E}N_{s\to t} = c_{st}(g(s) - g(t)) = c_{st}R_{\text{eff}}(s,t).$$

□

In particular,

$$c_{st}R_{\text{eff}}(s,t) \leq 1.$$

14.2.4 Hitting Times

Let

$$H_{u,t} = \mathbb{E}_u[T_t].$$

For fixed t, define

$$h_t(u) = H_{u,t}.$$

For $u \neq t$,

$$h_t(u) = 1 + \sum_v P(u,v)h_t(v).$$

Multiplying by c_u gives

$$(Lh_t)_u = c_u.$$

Let

$$\mathbf{c} = (c_v)_{v\in V}, \qquad \text{vol}(G) = \sum_v c_v,$$

and define

$$b_t = \mathbf{c} - \text{vol}(G)\mathbf{1}_t.$$

Since both Lh_t and b_t have coordinate sum zero,

$$\boxed{Lh_t = b_t.}$$

Theorem 565 (Hitting-Time Potential)**.** *Let*

$$p_t = L^+ b_t.$$

Then

$$\boxed{H_{u,t} = p_t(u) - p_t(t).}$$

Proof. Both h_t and p_t solve

$$Lx = b_t.$$

Their difference therefore lies in $\ker L = \operatorname{span}\{\mathbf{1}\}$. Since

$$h_t(t) = 0,$$

subtracting the value at t gives the formula. □

14.2.5 Commute Time

Define

$$C_{u,v} = H_{u,v} + H_{v,u}.$$

Theorem 566 (Commute-Time Identity)**.**

$$\boxed{C_{u,v} = \operatorname{vol}(G) R_{\text{eff}}(u, v).}$$

Proof. Using the preceding theorem,

$$p_v - p_u = L^+(b_v - b_u) = \operatorname{vol}(G) L^+(\mathbf{1}_u - \mathbf{1}_v).$$

Therefore

$$\begin{aligned} H_{u,v} + H_{v,u} &= (\mathbf{1}_u - \mathbf{1}_v)^\top (p_v - p_u) \\ &= \operatorname{vol}(G)(\mathbf{1}_u - \mathbf{1}_v)^\top L^+(\mathbf{1}_u - \mathbf{1}_v). \end{aligned}$$

□

14.2.6 Tetali's Hitting-Time Formula

Theorem 567 (Tetali's Formula)**.** *For all s, t,*

$$\boxed{H_{s,t} = \frac{1}{2} \sum_{x \in V} c_x \left(R_{\text{eff}}(s,t) + R_{\text{eff}}(t,x) - R_{\text{eff}}(s,x) \right).}$$

Proof. Write

$$R(a,b) = L^+_{aa} + L^+_{bb} - 2L^+_{ab}.$$

Then

$$\begin{aligned} &\frac{1}{2} \sum_x c_x \big(R(s,t) + R(t,x) - R(s,x) \big) \\ &= \sum_x c_x (L^+_{sx} - L^+_{tx}) - \operatorname{vol}(G)(L^+_{st} - L^+_{tt}) \\ &= (\mathbf{1}_s - \mathbf{1}_t)^\top L^+ \left(\mathbf{c} - \operatorname{vol}(G) \mathbf{1}_t \right) \\ &= p_t(s) - p_t(t) \\ &= H_{s,t}. \end{aligned}$$

□

14.2.7 Cover-Time Bound

Let

$$\mathrm{cov}_u(G)$$

be the expected time for an unweighted random walk starting at u to visit every vertex.

Theorem 568. *For every connected unweighted graph with n vertices and m edges,*

$$\boxed{\mathrm{cov}_u(G) \le 2m(n-1).}$$

Proof. Fix any spanning tree T. Take an Euler tour of T, which traverses every tree edge once in each direction.

Starting from u, wait until the random walk visits the successive vertices of this tour. Once this process finishes, every vertex has been visited. Its expected duration is at most

$$\sum_{\{x,y\}\in T} (H_{x,y} + H_{y,x}) = \sum_{\{x,y\}\in T} C_{x,y}.$$

For a tree edge $\{x, y\}$, Rayleigh monotonicity gives

$$R_{\mathrm{eff}}(x, y) \le 1.$$

Since

$$\mathrm{vol}(G) = 2m,$$

the commute-time identity yields

$$C_{x,y} \le 2m.$$

There are $n - 1$ tree edges. □

14.3 Kron Reduction

Electrical networks remain electrical networks after eliminating internal vertices.

Partition the vertices into a boundary S and interior I, and write

$$L = \begin{pmatrix} L_{SS} & L_{SI} \\ L_{IS} & L_{II} \end{pmatrix}.$$

Theorem 569 (Kron Reduction)**.** *The network induced on the boundary after eliminating the interior has Laplacian*

$$\boxed{L_{\mathrm{Kron}} = L_{SS} - L_{SI} L_{II}^{-1} L_{IS}.}$$

It preserves all effective resistances between vertices of S.

Proof. Fix boundary potentials x_S. The energy is

$$\begin{pmatrix} x_S \\ x_I \end{pmatrix}^{\top} L \begin{pmatrix} x_S \\ x_I \end{pmatrix}.$$

The minimizing interior potential satisfies

$$L_{IS} x_S + L_{II} x_I = 0,$$

so

$$x_I = -L_{II}^{-1} L_{IS} x_S.$$

Substituting gives

$$x_S^{\top} \left(L_{SS} - L_{SI} L_{II}^{-1} L_{IS} \right) x_S.$$

Dirichlet's principle therefore shows that all boundary effective resistances are unchanged. □

There is also a random-walk interpretation.

Let

$$P = D^{-1}A$$

be the transition matrix. Partition it in the same way.

Theorem 570 (Trace Chain)**.** *Observe the random walk only at visits to S. Its transition matrix is*

$$\boxed{Q = P_{SS} + P_{SI}(I - P_{II})^{-1}P_{IS}.}$$

Moreover,

$$\boxed{L_{\text{Kron}} = D_S(I - Q).}$$

Proof. The term

$$P_{SI}P_{II}^{k}P_{IS}$$

is the probability of leaving S, taking exactly k internal steps, and returning to S. Summing over $k \geq 0$ gives the formula for Q.

Also,

$$L_{II} = D_I(I - P_{II}),$$
$$L_{SI} = -D_S P_{SI}, \qquad L_{IS} = -D_I P_{IS},$$

and

$$L_{SS} = D_S(I - P_{SS}).$$

Substitution into the Schur complement gives

$$L_{\text{Kron}} = D_S\left(I - P_{SS} - P_{SI}(I - P_{II})^{-1}P_{IS}\right) = D_S(I - Q).$$

□

Thus for distinct boundary vertices a, b, the reduced conductance is

$$\boxed{\widetilde{c}_{ab} = c_a Q(a, b).}$$

As a special case, eliminating the center of a star whose incident conductances are

$$c_1, \ldots, c_k$$

creates between leaves i, j the conductance

$$\boxed{\widetilde{c}_{ij} = \frac{c_i c_j}{\sum_\ell c_\ell}.}$$

14.4 Infinite Graphs, Recurrence, and Transience

Now let G be a connected locally finite infinite graph.

Choose finite connected sets

$$K_1 \subseteq K_2 \subseteq \cdots, \qquad \bigcup_n K_n = V,$$

containing a fixed vertex o.

Wire every vertex outside K_n into one sink ∂_n, and let

$$R_n = R_{\text{eff}}(o, \partial_n).$$

Because the n-th wired network is obtained from the $(n+1)$-st by additional contractions, Rayleigh monotonicity gives

$$R_n \leq R_{n+1}.$$

Define

$$\boxed{R_{\text{eff}}(o \leftrightarrow \infty) = \lim_{n\to\infty} R_n \in [0, \infty].}$$

14.4.1 Infinite Thomson Principle

A unit flow from o to infinity satisfies

$$\operatorname{div}\theta(o) = 1$$

and

$$\operatorname{div}\theta(v) = 0 \qquad (v \neq o).$$

Equivalently, one unit of net current crosses every finite cut separating o from infinity.

Theorem 571 (Infinite Thomson Principle)**.**

$$\boxed{R_{\text{eff}}(o \leftrightarrow \infty) = \inf_{\theta:\ \textit{unit flow to infinity}} \mathcal{E}(\theta).}$$

In particular,

$$R_{\text{eff}}(o \leftrightarrow \infty) < \infty$$

if and only if there exists a finite-energy unit flow to infinity.

Proof. Let θ be any unit flow to infinity. Restrict it to the edges inside K_n and crossing its boundary. After wiring the exterior, this is a feasible unit flow from o to ∂_n. Therefore finite Thomson gives

$$R_n \leq \mathcal{E}(\theta).$$

Taking $n \to \infty$ gives

$$R_\infty \leq \mathcal{E}(\theta).$$

Conversely, suppose

$$R_\infty < \infty.$$

Let θ_n be the electrical unit flow from o to ∂_n, extended by zero outside K_n. Then

$$\mathcal{E}(\theta_n) = R_n \leq R_\infty.$$

For every fixed edge e,

$$r_e \theta_n(e)^2 \leq R_\infty,$$

so the sequence $\theta_n(e)$ is bounded.

Since E is countable, enumerate its edges as

$$E = \{e_1, e_2, \dots\}.$$

For each fixed edge e, the sequence $(\theta_n(e))_n$ is bounded. By Bolzano–Weierstrass, first pass to a subsequence on which $\theta_n(e_1)$ converges; then to a further subsequence on which $\theta_n(e_2)$ also converges; and so on. Taking the diagonal subsequence of these nested subsequences gives indices n_j such that, simultaneously for every fixed edge e,

$$\theta_{n_j}(e) \to \theta(e).$$

Local finiteness lets us pass the limit through every divergence sum, so θ is a unit flow to infinity.

Finally Fatou's lemma gives

$$\mathcal{E}(\theta) \leq \liminf_j \mathcal{E}(\theta_{n_j}) = R_\infty.$$

Together with the opposite inequality, equality holds. □

14.4.2 Escape Probability

Let

$$\tau_o^+ = \inf\{j \geq 1 : X_j = o\}.$$

Theorem 572.

$$\boxed{C_{\text{eff}}(o \leftrightarrow \infty) = c_o \Pr_o[\tau_o^+ = \infty].}$$

Consequently,

$$\boxed{G \text{ is transient} \iff R_{\text{eff}}(o \leftrightarrow \infty) < \infty.}$$

Proof. In the finite wired network, impose voltage 1 at o and 0 at ∂_n. The voltage at v is

$$h(v) = \Pr_v[T_o < T_{\partial_n}].$$

Thus the total current leaving o is

$$\begin{aligned} C_{\text{eff}}(o, \partial_n) &= \sum_v c_{ov}(1 - h(v)) \\ &= c_o \sum_v P(o, v) \Pr_v[T_{\partial_n} < T_o] \\ &= c_o \Pr_o[T_{\partial_n} < \tau_o^+]. \end{aligned}$$

Letting $n \to \infty$,

$$C_{\text{eff}}(o \leftrightarrow \infty) = c_o \Pr_o[\tau_o^+ = \infty].$$

The right-hand side is positive exactly when the walk is transient. □

Corollary 573 (Green Function)**.** *Let*

$$G(o, o) = \sum_{j \geq 0} \Pr_o[X_j = o]$$

be the expected number of visits to o. Then in a transient network,

$$\boxed{G(o, o) = c_o R_{\text{eff}}(o \leftrightarrow \infty).}$$

Proof. After every visit to o, the walk escapes forever with probability

$$p = \Pr_o[\tau_o^+ = \infty].$$

Thus the number of visits is geometric with mean $1/p$. Apply Theorem 572. □

14.4.3 Nash–Williams Recurrence Criterion

Theorem 574 (Nash–Williams)**.** *Let*

$$\Pi_1, \Pi_2, \ldots$$

be pairwise edge-disjoint cutsets separating o from infinity. Then

$$\boxed{R_{\text{eff}}(o \leftrightarrow \infty) \geq \sum_i \frac{1}{\sum_{e \in \Pi_i} c_e}.}$$

Proof. Let θ be any unit flow to infinity. The net current crossing each Π_i is 1. Hence by weighted Cauchy–Schwarz,

$$1 = \left(\sum_{e \in \Pi_i} \theta_e \right)^2 \leq \left(\sum_{e \in \Pi_i} c_e \right) \left(\sum_{e \in \Pi_i} \frac{\theta_e^2}{c_e} \right).$$

Therefore

$$\sum_{e \in \Pi_i} \frac{\theta_e^2}{c_e} \geq \frac{1}{\sum_{e \in \Pi_i} c_e}.$$

The cutsets are edge-disjoint, so summing gives the claimed lower bound on $\mathcal{E}(\theta)$. Now minimize using infinite Thomson. □

For the unit grid $\mathbb{Z}^2$, take square shells. The r-th shell has

$$O(r)$$

edges, so

$$R_{\text{eff}}(0 \leftrightarrow \infty) \geq \sum_{r \geq 1} \Omega(1/r) = \infty.$$

Thus the two-dimensional random walk is recurrent.

14.4.4 Trees

For a finite binary tree of depth n with all leaves wired together, let R_n be the resistance from the root to the boundary.

Then

$$R_0 = 0$$

and the two branches each have resistance

$$1 + R_n.$$

They occur in parallel, so

$$\boxed{R_{n+1} = \frac{1 + R_n}{2}.}$$

Hence

$$R_n = 1 - 2^{-n}$$

and

$$\boxed{R_{\text{eff}}(o \leftrightarrow \infty) = 1.}$$

The minimizing flow sends

$$2^{-j}$$

through every edge at level j, and its energy is

$$\sum_{j \geq 1} 2^j (2^{-j})^2 = 1.$$

14.5 Random Spanning Trees

Fix a root vertex r. Let

$$B_r \in \mathbb{R}^{E \times (V \setminus \{r\})}$$

be the incidence matrix with the column of r deleted.

14.5.1 Weighted Random Spanning Trees

The weighted spanning-tree distribution is

$$\Pr[T] = \frac{\prod_{e\in T} c_e}{Z},$$

where

$$Z = \sum_{T} \prod_{e\in T} c_e = \det(B_r^\top C B_r).$$

Define the transfer-current matrix

$$\boxed{K = C^{1/2} B L^{+} B^\top C^{1/2}.}$$

Thus

$$K_{ef} = \sqrt{c_e c_f}\, b_e^\top L^{+} b_f.$$

For one edge,

$$K_{ee} = c_e R_{\mathrm{eff}}(e).$$

Theorem 575 (Burton–Pemantle)**.** *For every set of edges*

$$F \subseteq E,$$

$$\boxed{\Pr[F \subseteq T] = \det K[F,F].}$$

Proof. Write

$$F = \{e_1, \dots, e_k\}.$$

Introduce variables

$$z_1, \dots, z_k$$

and multiply the conductance of e_i by

$$1 + z_i.$$

Let

$$U = \begin{pmatrix} \sqrt{c_{e_1}}\, b_{e_1,r} & \cdots & \sqrt{c_{e_k}}\, b_{e_k,r} \end{pmatrix}, \qquad Z_F = \mathrm{diag}(z_1, \dots, z_k),$$

where $b_{e,r}$ denotes the incidence vector with the root coordinate deleted.

The reduced Laplacian becomes

$$L_r(z) = L_r + U Z_F U^\top.$$

Matrix–Tree gives

$$\frac{\det L_r(z)}{\det L_r} = \mathbb{E}_T\left[\prod_{e_i\in T} (1 + z_i)\right].$$

Therefore the coefficient of

$$z_1 z_2 \cdots z_k$$

is exactly

$$\Pr[F \subseteq T].$$

The matrix determinant lemma gives

$$\begin{aligned} \frac{\det L_r(z)}{\det L_r} &= \det\left(I + Z_F U^\top L_r^{-1} U\right) \\ &= \det(I + Z_F K[F,F]). \end{aligned}$$

The coefficient of

$$\prod_{i=1}^{k} z_i$$

in this determinant is the principal minor

$$\det K[F, F].$$

Equating coefficients proves the theorem. □

Corollary 576 (Edge Marginal)**.**

$$\boxed{\Pr[e \in T] = c_e R_{\text{eff}}(e).}$$

For an unweighted graph this is simply

$$\Pr[e \in T] = R_{\text{eff}}(e).$$

Corollary 577 (Foster's Identity)**.**

$$\boxed{\sum_{e \in E} c_e R_{\text{eff}}(e) = n - 1.}$$

Proof. The matrix

$$K = C^{1/2} B L^{+} B^{\top} C^{1/2}$$

is an orthogonal projection:

$$\begin{aligned} K^2 &= C^{1/2} B L^{+} (B^{\top} C B) L^{+} B^{\top} C^{1/2} \\ &= C^{1/2} B L^{+} L L^{+} B^{\top} C^{1/2} \\ &= K. \end{aligned}$$

Its rank is $n - 1$, so

$$\operatorname{tr} K = n - 1.$$

But

$$\operatorname{tr} K = \sum_{e} K_{ee} = \sum_{e} c_e R_{\text{eff}}(e).$$

□

Corollary 578 (Negative Correlation)**.** *For every set of edges $F \subseteq E$,*

$$\boxed{\Pr[F \subseteq T] \le \prod_{e \in F} \Pr[e \in T].}$$

Proof. The principal submatrix $K[F, F]$ is positive semidefinite, so Burton–Pemantle and Hadamard's inequality give

$$\Pr[F \subseteq T] = \det K[F, F] \le \prod_{e \in F} K_{ee} = \prod_{e \in F} \Pr[e \in T].$$

□

For two distinct edges, the deficit is $\Pr[e \in T] \Pr[f \in T] - \Pr[e, f \in T] = K_{ef}^2$.

14.6 Localization of Electrical Flows

Let $G = (V, E, c)$ be a connected undirected graph without self-loops, with $n \geq 2$ vertices and positive conductances c_e. Fix an orientation of the edges, let B be the edge–vertex incidence matrix, and put $C = \operatorname{diag}(c)$, $K = C^{1/2}B$, and $L = K^\top K$. Recall that

$$\Pi = KL^+K^\top$$

is the orthogonal projection onto $\operatorname{im} K$, so $\|\Pi\|_{2\to 2} = 1$. Localization asks how much the norm can increase when we discard the signs of the entries. Write $\overline{\Pi} = |\Pi|$ for the entrywise absolute value.

For an unweighted graph, column e of Π is the electrical flow sending one unit between the endpoints of e. Its ℓ_1-norm measures total edge usage: current follows decreasing potential, so the flow decomposes into paths carrying nonnegative amounts of flow that sum to one, and its ℓ_1-norm is the average length of these paths. An individual flow can use long routes. The following bound limits how much spreading can occur across all source edges together.

Theorem 579 (Localization of Electrical Flows)**.** *For every connected weighted graph on n vertices,*

$$\|\overline{\Pi}\|_{2\to 2} \leq 2\log n.$$

In particular, an unweighted graph with m edges satisfies

$$\frac{1}{m}\sum_{e\in E} \|\Pi \mathbf{1}_e\|_1 = \frac{1}{m}\mathbf{1}^\top \overline{\Pi}\mathbf{1} \leq 2\log n.$$

We will prove the stronger estimate

$$|w^\top \overline{\Pi} w| \leq 2H(\mu_w)\|w\|_2^2, \qquad \mu_w(x) = \frac{\sum_{e\ni x} w_e^2}{2\|w\|_2^2}, \tag{301}$$

for every nonzero $w \in \mathbb{R}^E$. Here $H(\mu) = \sum_x \mu(x)\log(1/\mu(x))$, logarithms are natural, and $0\log 0 = 0$. The norm bound follows from $H(\mu_w) \leq \log n$.

14.6.1 Controlling One Trajectory

Fix $w_e > 0$ for every edge and write $W = \|w\|_2^2$. We need to bound $w^\top \overline{\Pi} w = \sum_{e,f} w_e w_f |\Pi_{ef}|$. An outer product separates the two edge indices:

$$\sum_{e,f} w_e w_f |(rr^\top)_{ef}| = \left(\sum_e w_e |r_e|\right)^2.$$

Since Π projects onto $\operatorname{im} K$, we seek a representation by outer products with $r = Kh$. Their contribution is

$$Q(h) = \left(\sum_{e=xy} w_e \sqrt{c_e}\,|h_x - h_y|\right)^2.$$

We will first find an evolution for which the integral of $Q(h)$ is controlled by the initial function, then choose starting functions whose gradient outer products recover Π.

For $h > 0$, weighted Cauchy–Schwarz puts the w_e^2's in one factor. We want to control this factor by preserving a weighted average of the vertex values. Use the arithmetic mean of the two endpoint values as the edge weight to make the factor linear in h:

$$Q(h) \leq \left(\sum_{e=xy} w_e^2 \frac{h_x + h_y}{2}\right)\left(\sum_{e=xy} \frac{2c_e(h_x - h_y)^2}{h_x + h_y}\right).$$

Each w_e^2 is then split equally between its endpoints:

$$\sum_{e=xy} w_e^2 \frac{h_x + h_y}{2} = \frac{1}{2} \sum_x d_w(x) h_x, \qquad d_w(x) = \sum_{e \ni x} w_e^2.$$

Since $\sum_x d_w(x) = 2W$, choose $\mu(x) = d_w(x)/(2W)$. This is the distribution obtained by choosing edge e with probability w_e^2/W and then choosing one endpoint uniformly. It is positive because $w > 0$ and the graph is connected.

The first factor is now $W\mu^\top h$. To keep it equal to W throughout the evolution, we will preserve the weighted mass $\mu^\top h = 1$. For such functions, the logarithmic-mean inequality

$$\frac{2(a-b)^2}{a+b} \le (a-b)(\log a - \log b) \qquad (a, b > 0)$$

gives

$$Q(h) \le W \sum_{e=xy} c_e (h_x - h_y)(\log h_x - \log h_y) = W (\log h)^\top L h. \tag{302}$$

To integrate this bound, we want to replace Lh by a time derivative while preserving the weighted mass. Motivated by continuous random walks, put $M = \operatorname{diag}(\mu)$ and evolve h by

$$M h_t' = -L h_t, \qquad h_0 \ge 0, \qquad \mu^\top h_0 = 1. \tag{303}$$

At vertex x, this says

$$h_t'(x) = \frac{1}{\mu(x)} \sum_{e=xy} c_e \big(h_t(y) - h_t(x)\big).$$

Each vertex moves toward its neighbors' values. After multiplying by $\mu(x)$, each edge contributes equal and opposite changes at its endpoints, so the weighted mass stays one: $\mu^\top h_t' = -\mathbf{1}^\top L h_t = 0$.

The solution is positive for $t > 0$. Indeed, for $\alpha \ge \max_x L_{xx}/\mu(x)$, the matrix $N = \alpha I - M^{-1} L$ is entrywise nonnegative, and

$$h_t = e^{-\alpha t} \sum_{k \ge 0} \frac{t^k}{k!} N^k h_0.$$

A path from a vertex with $h_0 > 0$ to any other vertex gives a positive term in that coordinate.

The pairing in (302) is now $-\sum_x \mu(x) \log h_t(x)\, h_t'(x)$. The primitive of $\log r$ is $r \log r - r$; since the weighted mass is fixed, the linear term can be dropped. Thus set

$$F(h) = \sum_x \mu(x) h_x \log h_x.$$

The numbers $p_x = \mu(x) h_x$ form a probability distribution, and $F(h) = \sum_x p_x \log(p_x/\mu(x))$ is its relative entropy with respect to μ. Jensen's inequality gives $F(h) \ge 0$, while

$$-\frac{d}{dt} F(h_t) = (\log h_t)^\top L h_t \ge \frac{Q(h_t)}{W}.$$

Integrating from ε to τ and discarding the nonnegative terminal value gives $\int_\varepsilon^\tau Q(h_t)\, dt \le W F(h_\varepsilon)$. Letting $\varepsilon \downarrow 0$ and $\tau \to \infty$ yields

$$\boxed{\int_0^\infty Q(h_t)\, dt \le W F(h_0).} \tag{304}$$

Continuity of $r \log r$ at zero justifies the initial endpoint. The total contribution is bounded by the initial relative entropy.

14.6.2 Choosing the Trajectories

We now choose a random initial density so that the average of the integrated gradient outer products gives Π, up to a constant factor. The gradient evolution tells us which initial weights are needed. Writing $r_t = Kh_t$, we have

$$r_t' = -KM^{-1}Lh_t = -Tr_t, \qquad T = KM^{-1}K^\top.$$

The matrix T is positive semidefinite with $\operatorname{im} T = \operatorname{im} K$, the space onto which Π projects.

Fix a unit eigenvector u of T with eigenvalue $\lambda > 0$. The scalar component $z_t = u^\top r_t$ evolves as $z_t = e^{-t\lambda} z_0$. Thus

$$u^\top \left(\int_0^\infty \mathbb{E}[r_t r_t^\top]\, dt \right) u = \int_0^\infty \mathbb{E}[z_t^2]\, dt = \frac{\mathbb{E}[z_0^2]}{2\lambda}.$$

To make every positive eigendirection contribute equally and obtain the projection, choose its initial second moment to be λ: the larger initial weight exactly offsets the faster decay. We therefore seek the initial average outer product $\mathbb{E}[r_0 r_0^\top] = T$. With this choice, each positive eigendirection contributes $1/2$, so doubling the integral gives the projection.

This requirement tells us how to choose the initial density. The mass at vertex x is $\mu(x)h_0(x)$. Consider one vertex v; take $h_0 = \mathbf{1}_v/\mu(v)$ and thus $r_0 = Kh_0 = K\mathbf{1}_v/\mu(v)$. Then,

$$T = KM^{-1}K^\top = \sum_v \mu(v) \left(\frac{K\mathbf{1}_v}{\mu(v)} \right) \left(\frac{K\mathbf{1}_v}{\mu(v)} \right)^\top.$$

Thus sampling v with probability $\mu(v)$ gives exactly the required average outer product. After this initial choice, the evolution is deterministic.

Since $r_t = e^{-tT} r_0$, the scalar calculation above gives

$$\boxed{2\mathbb{E} \int_0^\infty r_t r_t^\top\, dt = 2 \int_0^\infty e^{-tT} T e^{-tT}\, dt = \Pi.} \tag{305}$$

Every positive eigendirection receives coefficient one; the zero eigendirections contribute nothing.

Taking entrywise absolute values and using the outer-product calculation at the start,

$$w^\top \overline{\Pi} w \le 2\mathbb{E} \int_0^\infty Q(h_t)\, dt \le 2W\, \mathbb{E} F(h_0).$$

For the initial point mass, $F(\mathbf{1}_v/\mu(v)) = \log(1/\mu(v))$. Therefore

$$w^\top \overline{\Pi} w \le 2W \sum_v \mu(v) \log \frac{1}{\mu(v)} = 2H(\mu_w)\|w\|_2^2.$$

For arbitrary nonzero w, first use $|w^\top \overline{\Pi} w| \le |w|^\top \overline{\Pi} |w|$, apply the positive-vector bound to $|w| + \varepsilon \mathbf{1}$, and let $\varepsilon \downarrow 0$. The vertex distribution and its entropy are continuous in this limit. Finally, concavity of the logarithm implies $H(\mu_w) \le \log n$, and symmetry of $\overline{\Pi}$ gives

$$\|\overline{\Pi}\|_{2\to 2} = \sup_{w \ne 0} \frac{|w^\top \overline{\Pi} w|}{\|w\|_2^2} \le 2 \log n,$$

as claimed.

15 Matrix Analysis

15.1 Sampling Matrices without Replacement

We can show that sampling without replacement is more concentrated than sampling with replacement, this works for both scalars and matrices.

Lemma 580 (Hoeffding's Coupling)**.** *Let $\mathcal{C}$ be a finite set of Hermitian matrices. Let $X = (X_1, \ldots, X_m)$ be a sequence of m matrices sampled uniformly with replacement from $\mathcal{C}$. Let $Y = (Y_1, \ldots, Y_m)$ be a sequence sampled uniformly without replacement from $\mathcal{C}$.*

There exists a randomized surrogate function $Z(Y) = (Z_1, \ldots, Z_m)$ such that:

1. *The unconditional distribution of $Z(Y)$ is identical to X.*
2. *The conditional expectation satisfies $\mathbb{E}_{Z|Y}\left[\sum_{i=1}^m Z_i(Y)\right] = \sum_{i=1}^m Y_i$.*

Proof. We construct the sequence $Z_1, \ldots, Z_m$ conditionally from the drawn set Y via the following sequential recipe.

For the k-th step, let D_k be the subset of matrices from Y that have already been chosen in the sequence $Z_1, \ldots, Z_{k-1}$. We draw Z_k as follows:

- With probability $\frac{|D_k|}{|\mathcal{C}|}$, we sample Z_k uniformly from the already-drawn elements D_k.
- With probability $1 - \frac{|D_k|}{|\mathcal{C}|}$, we sample Z_k uniformly from the remaining un-drawn elements $Y \setminus D_k$.

It is straightforward to verify that the joint probability of drawing any sequence $x \in \mathcal{C}^m$ is $|\mathcal{C}|^{-m}$.

For the second property, observe that the conditional expectation $\mathbb{E}_{Z|Y}[\sum_{i=1}^m Z_i]$ is a linear combination of the variables $Y_1, \ldots, Y_m$ and invariant under a permutation of the indices of Y.

It thus must take the form $K \sum_{i=1}^m Y_i$ for some scalar constant K. To determine K, we take the expectation over Y:

$$\mathbb{E}_Y\left[\mathbb{E}_{Z|Y}\left[\sum_{i=1}^m Z_i\right]\right] = m\mathbb{E}[Y_1] \quad \text{and} \quad \mathbb{E}_Y\left[K\sum_{i=1}^m Y_i\right] = Km\mathbb{E}[Y_1]$$

So $K = 1$. □

15.2 Schur and Kronecker Product

Lemma 581 (Kronecker Product Preserves Positivity)**.** *Let A and B be Hermitian matrices. If $A, B \succeq 0$, then $A \otimes B \succeq 0$. Furthermore, if $A, B \succ 0$, then $A \otimes B \succ 0$.*

Proof. Assume $A \succeq 0$ and $B \succeq 0$. Every PSD matrix has a unique PSD square root. We can factor the Kronecker product:

$$A \otimes B = (A^{1/2}A^{1/2}) \otimes (B^{1/2}B^{1/2}) = (A^{1/2} \otimes B^{1/2})(A^{1/2} \otimes B^{1/2})$$

Because the Kronecker product of two Hermitian matrices is Hermitian, $A^{1/2} \otimes B^{1/2}$ is Hermitian. Therefore, we have expressed $A \otimes B$ as the square of a Hermitian matrix, which implies $A \otimes B \succeq 0$.

Now assume $A \succ 0$ and $B \succ 0$. In this case, both A and B are invertible. The Kronecker product of invertible matrices is invertible, with its inverse given by $(A \otimes B)^{-1} = A^{-1} \otimes B^{-1}$. □

The Schur product theorem has an especially transparent interpretation in terms of Gram matrices. A positive semidefinite matrix is a table of inner products; entrywise multiplication simply tensors the vectors producing those inner products.

Lemma 582 (Schur Product Theorem)**.** *Let $A, B \in M_n(\mathbb{C})$ be Hermitian. If $A, B \succeq 0$, then*

$$A \circ B \succeq 0.$$

If $A, B \succ 0$, then $A \circ B \succ 0$.

Proof. Because $A \succeq 0$ and $B \succeq 0$, there are vectors $u_1, \dots, u_n$ and $v_1, \dots, v_n$ such that

$$A_{ij} = \langle u_i, u_j \rangle, \qquad B_{ij} = \langle v_i, v_j \rangle.$$

Therefore

$$(A \circ B)_{ij} = \langle u_i, u_j \rangle \langle v_i, v_j \rangle = \langle u_i \otimes v_i, u_j \otimes v_j \rangle.$$

Thus $A \circ B$ is the Gram matrix of the vectors

$$u_1 \otimes v_1, \dots, u_n \otimes v_n,$$

and every Gram matrix is positive semidefinite.

Now suppose $A, B \succ 0$. Then the Gram vectors $u_1, \dots, u_n$ are linearly independent, and every v_i is nonzero. If

$$\sum_i c_i u_i \otimes v_i = 0,$$

choose, for each j, a linear functional φ_j on $\operatorname{span}\{u_i\}$ satisfying $\varphi_j(u_i) = \delta_{ij}$. Applying $\varphi_j \otimes I$ gives $c_j v_j = 0$, hence $c_j = 0$. So the vectors $u_i \otimes v_i$ are linearly independent, and their Gram matrix $A \circ B$ is positive definite. □

15.3 Differentiating Matrix Functions

For scalar functions, one writes

$$\frac{d}{dt} f(x(t)) = f'(x(t)) x'(t).$$

The analogous formula is generally false for matrices because X and X' need not commute. There are two simple replacements.

For algebraic expressions, differentiate by the ordinary product rule while preserving the order of the factors. For functions defined by spectral calculus, scalar derivatives are replaced by *divided differences*. Inside a trace, only the diagonal divided differences survive, and the usual scalar derivative rule reappears.

For a differentiable matrix-valued map F, write

$$DF(X)[H] = \left. \frac{d}{dt} F(X + tH) \right|_{t=0}.$$

Thus for a differentiable path $X(t)$,

$$\frac{d}{dt} F(X(t)) = DF(X(t))[X'(t)].$$

15.3.1 Product, Inverse, and Power Rules

The ordinary product rule remains valid because multiplication is bilinear; the only care is that the order of the factors cannot be changed.

Lemma 583 (Basic matrix differential rules)**.** *Let $A(t), B(t)$ be differentiable matrix-valued functions.*

Then

$$\frac{d}{dt}(AB) = A'B + AB'.$$

If $A(t)$ is invertible, then

$$\frac{d}{dt} A^{-1} = -A^{-1} A' A^{-1}.$$

For every integer $k \geq 1$,

$$\frac{d}{dt} A^k = \sum_{r=0}^{k-1} A^r A' A^{k-1-r}.$$

Proof. The product rule follows by inserting and subtracting $A(t+h)B(t)$:

$$A(t+h)B(t+h) - A(t)B(t) = \big(A(t+h) - A(t)\big)B(t) + A(t+h)\big(B(t+h) - B(t)\big).$$

Divide by h and let $h \to 0$.

For the inverse, differentiate

$$AA^{-1} = I.$$

The product rule gives

$$A'A^{-1} + A(A^{-1})' = 0.$$

Multiplying on the left by A^{-1} yields

$$(A^{-1})' = -A^{-1}A'A^{-1}.$$

Finally, repeatedly applying the product rule to $A^k = A \cdots A$ gives one term for each factor at which the derivative may land:

$$(A^k)' = A'A^{k-1} + AA'A^{k-2} + \cdots + A^{k-1}A'.$$

□

When A commutes with A', all k terms are equal and this reduces to the formula

$$(A^k)' = kA^{k-1}A'.$$

15.3.2 Divided Differences

Let $f : I \to \mathbb{R}$ be differentiable. Its first divided difference is

$$f^{[1]}(a,b) = \begin{cases} \dfrac{f(a) - f(b)}{a - b}, & a \neq b, \\ f'(a), & a = b. \end{cases}$$

By the fundamental theorem of calculus,

$$f^{[1]}(a,b) = \int_0^1 f'\big((1-s)b + sa\big)\, ds.$$

Thus a divided difference is simply the average scalar derivative between the two eigenvalues.

For a Hermitian matrix

$$X = U \operatorname{diag}(\lambda_1, \ldots, \lambda_n) U^*,$$

recall that spectral calculus defines

$$f(X) = U \operatorname{diag}\big(f(\lambda_1), \ldots, f(\lambda_n)\big) U^*.$$

Theorem 584 (Derivative of a spectral matrix function)**.** *Let X be Hermitian with spectrum in the interior of an interval I, and let $f \in C^1(I)$. For a Hermitian direction H, put*

$$\widetilde{H} = U^* H U.$$

Then

$$U^* Df(X)[H] U = \big(f^{[1]}(\lambda_i, \lambda_j)\widetilde{H}_{ij}\big)_{i,j}.$$

Equivalently,

$$Df(X)[H] = U\left(\big(f^{[1]}(\lambda_i, \lambda_j)\big)_{i,j} \circ \widetilde{H}\right) U^*.$$

Proof. First take $f(x) = x^k$. By Lemma 583,

$$D(X^k)[H] = \sum_{r=0}^{k-1} X^r H X^{k-1-r}.$$

In an eigenbasis of X, its ij-entry is

$$\widetilde{H}_{ij} \sum_{r=0}^{k-1} \lambda_i^r \lambda_j^{k-1-r}.$$

If $\lambda_i \neq \lambda_j$, the geometric-sum identity gives

$$\sum_{r=0}^{k-1} \lambda_i^r \lambda_j^{k-1-r} = \frac{\lambda_i^k - \lambda_j^k}{\lambda_i - \lambda_j}.$$

If $\lambda_i = \lambda_j$, the sum is $k\lambda_i^{k-1}$. Thus in either case the coefficient is $f^{[1]}(\lambda_i, \lambda_j)$. Linearity proves the formula for polynomials.

For a general C^1 function, let $\theta_1, \dots, \theta_r$ be the distinct eigenvalues of X. Choose a Hermite interpolation polynomial p satisfying

$$p(\theta_a) = f(\theta_a), \qquad p'(\theta_a) = f'(\theta_a) \qquad (a = 1, \dots, r).$$

Put $g = f - p$. Then

$$g(\theta_a) = g'(\theta_a) = 0,$$

so uniformly over the finitely many eigenvalues,

$$g(\theta_a + s) = o(|s|) \qquad (s \to 0).$$

If E is Hermitian and small, every eigenvalue of $X + E$ lies within $\|E\|$ of an eigenvalue of X. Indeed,

$$-\|E\| I \preceq E \preceq \|E\| I$$

gives

$$X - \|E\| I \preceq X + E \preceq X + \|E\| I,$$

and hence the same bound on the ordered eigenvalues. Therefore

$$\|g(X + E)\| = \max_{\mu \in \sigma(X+E)} |g(\mu)| = o(\|E\|).$$

Since $g(X) = 0$,

$$f(X + E) - f(X) = p(X + E) - p(X) + o(\|E\|).$$

Thus f and p have the same Fréchet derivative at X.

Finally, p agrees with f at every eigenvalue of X, and their derivatives agree there as well, so

$$p^{[1]}(\lambda_i, \lambda_j) = f^{[1]}(\lambda_i, \lambda_j).$$

The polynomial formula therefore gives the stated result. □

Thus we use the ordinary derivative on one eigenspace and divided difference between two eigenspaces. In particular, if X and H commute, they can be simultaneously diagonalized and

$$Df(X)[H] = f'(X)H.$$

15.3.3 Traces Recover the Scalar Derivative Rule

Although $Df(X)[H]$ is not generally $f'(X)H$, its trace behaves as though the matrices commuted.

Corollary 585 (Derivative of a trace spectral function)**.** *Let*

$$F(X) = \operatorname{Tr} f(X).$$

Then

$$DF(X)[H] = \operatorname{Tr}\big(f'(X)H\big).$$

With the trace inner product, therefore,

$$\nabla F(X) = f'(X).$$

Proof. Work in an eigenbasis of X. Trace keeps only the diagonal entries of $Df(X)[H]$, and on the diagonal

$$f^{[1]}(\lambda_i, \lambda_i) = f'(\lambda_i).$$

Hence

$$\begin{aligned} DF(X)[H] &= \sum_i f'(\lambda_i)\widetilde{H}_{ii} \\ &= \operatorname{Tr}\big(f'(X)H\big). \end{aligned}$$

□

There is also a second-order version.

Corollary 586 (Hessian of a trace spectral function)**.** *Let $f \in C^2(I)$, and let*

$$F(X) = \operatorname{Tr} f(X).$$

In an eigenbasis of $X = \operatorname{diag}(\lambda_1, \dots, \lambda_n)$,

$$D^2F(X)[H, H] = \sum_{i,j} (f')^{[1]}(\lambda_i, \lambda_j)|H_{ij}|^2.$$

Proof. By Corollary 585,

$$\nabla F(X) = f'(X).$$

Therefore

$$D^2F(X)[H, H] = \operatorname{Tr}\big(H\, D(f')(X)[H]\big).$$

Apply Theorem 584 to f':

$$\big(D(f')(X)[H]\big)_{ij} = (f')^{[1]}(\lambda_i, \lambda_j)H_{ij}.$$

Since H is Hermitian, $H_{ji} = \overline{H_{ij}}$, and hence

$$\begin{aligned} \operatorname{Tr}\big(H\, D(f')(X)[H]\big) &= \sum_{i,j} H_{ji}(f')^{[1]}(\lambda_i, \lambda_j)H_{ij} \\ &= \sum_{i,j} (f')^{[1]}(\lambda_i, \lambda_j)|H_{ij}|^2. \end{aligned}$$

□

Examples. For $f(x) = x^{-1}$,

$$f^{[1]}(a, b) = -\frac{1}{ab},$$

including the limiting value $-1/a^2$ when $a = b$. Thus

$$D(X^{-1})[H] = -X^{-1}HX^{-1},$$

recovering the inverse rule.

For matrix entropy,

$$h(X) = \operatorname{Tr}(X \log X),$$

take $f(x) = x \log x$. Since

$$f'(x) = 1 + \log x,$$

Corollary 585 gives immediately

$$\nabla h(X) = I + \log X.$$

The Hessian is

$$D^2h(X)[H, H] = \sum_{i,j} (\log)^{[1]}(\lambda_i, \lambda_j)|H_{ij}|^2.$$

Likewise,

$$\log\det X = \operatorname{Tr}\log X,$$

so

$$D(\log\det X)[H] = \operatorname{Tr}(X^{-1}H).$$

Exponentiating gives Jacobi's formula

$$D(\det X)[H] = \det(X)\operatorname{Tr}(X^{-1}H).$$

Finally, when the *exponent* rather than the matrix varies, no noncommutativity issue arises. For $X \succ 0$,

$$X^s = U \operatorname{diag}(\lambda_i^s) U^*,$$

so differentiating the scalar eigenvalues gives

$$\frac{d}{ds} X^s = X^s \log X = (\log X) X^s.$$

In particular,

$$\left.\frac{d}{ds} X^s\right|_{s=0} = \log X, \qquad \left.\frac{d}{ds} X^{1-s}\right|_{s=0} = -X \log X.$$

15.3.4 The Matrix Exponential

There is also another useful formula for the derivative of the matrix exponential.

Proposition 587 (Derivative of the matrix exponential)**.** *If $X(t)$ is differentiable, then*

$$\frac{d}{dt} e^{X(t)} = \int_0^1 e^{\alpha X(t)} X'(t) e^{(1-\alpha)X(t)}\, d\alpha.$$

Equivalently,

$$De^X[H] = \int_0^1 e^{\alpha X} H e^{(1-\alpha)X}\, d\alpha.$$

Proof. We first prove the finite-difference identity

$$e^A - e^B = \int_0^1 e^{\alpha A}(A - B)e^{(1-\alpha)B}\, d\alpha.$$

Define

$$F(\alpha) = e^{\alpha A}e^{(1-\alpha)B}.$$

Since A commutes with $e^{\alpha A}$ and B commutes with $e^{(1-\alpha)B}$,

$$\begin{aligned} F'(\alpha) &= Ae^{\alpha A}e^{(1-\alpha)B} - e^{\alpha A}Be^{(1-\alpha)B} \\ &= e^{\alpha A}(A - B)e^{(1-\alpha)B}. \end{aligned}$$

Thus the fundamental theorem of calculus gives

$$\begin{aligned} e^A - e^B &= F(1) - F(0) \\ &= \int_0^1 e^{\alpha A}(A - B)e^{(1-\alpha)B}\, d\alpha. \end{aligned}$$

Now take

$$A = X(t + h), \qquad B = X(t).$$

Then

$$\frac{e^{X(t+h)} - e^{X(t)}}{h} = \int_0^1 e^{\alpha X(t+h)} \frac{X(t + h) - X(t)}{h} e^{(1-\alpha)X(t)}\, d\alpha.$$

As $h \to 0$,

$$X(t + h) \to X(t), \qquad \frac{X(t + h) - X(t)}{h} \to X'(t),$$

so continuity of the matrix exponential lets us pass the limit through the integral. Hence

$$\frac{d}{dt}e^{X(t)} = \int_0^1 e^{\alpha X(t)}X'(t)e^{(1-\alpha)X(t)}\, d\alpha.$$

Taking $X(t) = X + tH$ gives the Fréchet derivative formula. □

If X commutes with H, the integrand is independent of α and we recover the scalar formula.

15.4 Polynomial Approximation of Matrix Functions

A low-degree scalar approximation gives a fast way to apply a matrix function: its degree counts matrix–vector products. We construct approximations to powers, logarithms, exponentials, and general analytic functions. Two elementary arguments do most of the work: positivity of Chebyshev coefficients controls additive error, and an exact reciprocal interpolation identity controls relative error. All constructions are deterministic. Work bounds count arithmetic operations, with scalar function evaluations stated separately; they are not bit-complexity bounds.

Scalar error and matrix evaluation. If A is Hermitian with spectrum in $[a, b]$, the spectral theorem gives

$$\sup_{x\in[a,b]} |p(x) - f(x)| \le \eta \quad \Longrightarrow \quad \|p(A) - f(A)\|_{\mathrm{op}} \le \eta. \tag{306}$$

Pointwise bounds $p_- \le f \le p_+$ give $p_-(A) \preceq f(A) \preceq p_+(A)$, and relative error $|p - f| \le \delta f$ for $f \ge 0$ gives $\|(p(A) - f(A))v\| \le \delta\|f(A)v\|$. The same statements apply on an invariant subspace U; sandwiching with its orthogonal projector gives the zero extension. This permits inverse functions on the orthogonal complement of a kernel.

The Chebyshev polynomials satisfy

$$C_j(\cos\theta) = \cos(j\theta), \qquad C_0(t) = 1, \quad C_1(t) = t, \quad C_{j+1}(t) = 2tC_j(t) - C_{j-1}(t).$$

Thus $|C_j(t)| \le 1$ on $[-1, 1]$. For $a < b$, put $c = (a + b)/2$ and $r = (b - a)/2$. The recurrence evaluates $p(A)v$, for $p(x) = \sum_{j=0}^{d} \alpha_j C_j((x - c)/r)$, using at most d products by A, $O(nd)$ other operations, and $O(n)$ vector storage, in addition to the coefficients. A one-point interval needs only a constant. Throughout, $\|\cdot\|_\infty$ denotes the uniform norm on the stated interval.

Constructing polynomials from values. One interpolation rule suffices for all the special functions below.

Lemma 588 (Chebyshev interpolation)**.** *Let $f(c + rt) = \sum_{j\ge0} \alpha_j C_j(t)$ with $\sum_j |\alpha_j| < \infty$. For $d \ge 0$, put $N = d + 1$ and let $I_d f$ interpolate f at*

$$x_\ell = c + r\cos\frac{(\ell + 1/2)\pi}{N}, \qquad 0 \le \ell < N.$$

Its coefficients require N function values and $O(N \log(N + 1))$ arithmetic operations. If each supplied value has error at most ζ, the resulting polynomial satisfies

$$\|\widetilde{I}_d f - f\|_\infty \le 2\sum_{j>d} |\alpha_j| + 2N\zeta. \tag{307}$$

Proof. At these nodes, each C_j agrees with either $\pm C_{j'}$ for some $0 \le j' \le d$, or zero: reduce j modulo $2N$ and reflect around N. Interpolation preserves degrees at most d, while each higher term changes by at most $2|\alpha_j|$. Sum using absolute convergence. The coefficients are a discrete cosine transform of the values. Each is a cosine-weighted average with absolute weights summing to at most two, so value errors contribute at most $2N\zeta$ in total. The transform is computed by an FFT. □

We will repeatedly use the following error budget: a coefficient tail at most $\eta/8$ and value accuracy $\eta/(8N)$ give error at most $\eta/2$; adding $\eta/2$ gives $0 \le q - f \le \eta$. Further errors in the computed coefficients contribute at most their ℓ_1 norm. This controls supplied values and coefficients, not roundoff in later matrix–vector products.

A coefficient-tail bound. The elementary identity

$$(\cos\theta)^j = 2^{-j} \sum_{h=0}^{j} \binom{j}{h} \cos((j - 2h)\theta) \tag{308}$$

shows that every monomial has nonnegative Chebyshev coefficients summing to one. Evaluating a positive expansion at $\cosh s > 1$ bounds its tail, because $C_\ell(\cosh s) = \cosh(\ell s)$ grows with ℓ.

Lemma 589 (Power-series majorants)**.** *Let $F(t) = \sum_{j\ge0} b_j t^j$ with real coefficients and $H(u) := \sum_{j\ge0} |b_j| u^j$. If $H(1) < \infty$, the Chebyshev coefficients α_ℓ of F satisfy $\sum_\ell |\alpha_\ell| \le H(1)$ and, whenever $s > 0$ and $H(\cosh s) < \infty$,*

$$B_d := \sum_{\ell>d} |\alpha_\ell| \le \frac{H(\cosh s)}{\cosh((d + 1)s)} \le 2e^{-sd} H(\cosh s). \tag{309}$$

If $b_j \ge 0$, then $\alpha_\ell \ge 0$ and $\sum_\ell \alpha_\ell = F(1)$.

Proof. Write $t^j = \sum_\ell p_{j\ell} C_\ell(t)$ using (308), where $p_{j\ell} \ge 0$ and $\sum_\ell p_{j\ell} = 1$. Then $\alpha_\ell = \sum_j b_j p_{j\ell}$ and

$$\sum_\ell |\alpha_\ell| \cosh(\ell s) \le \sum_j |b_j| \sum_\ell p_{j\ell} C_\ell(\cosh s) = H(\cosh s).$$

Nonnegativity justifies the majorant sums, and every tail term has weight at least $\cosh((d + 1)s)$. The remaining claims follow at $t = 1$. □

For a nonnegative Chebyshev expansion, replacing its omitted terms by 1 gives, on $[-1,1]$, the one-sided truncation

$$F(t) \le Q_d(t) := B_d + \sum_{\ell=0}^{d} \alpha_\ell C_\ell(t) \le F(1), \qquad 0 \le Q_d(t) - F(t) \le 2B_d. \tag{310}$$

Integer powers.

Lemma 590 (One-sided power approximation)**.** *For an integer $k \ge 1$, $L > 0$, and $0 < \delta \le 1$, one can construct q of degree at most*

$$d = \min\left\{k, \left\lceil \sqrt{k \log(8/\delta)} \right\rceil\right\}$$

such that, on $[0, L]$,

$$x^k \le q(x) \le L^k, \qquad 0 \le q(x) - x^k \le \tfrac{\delta}{2} L^k, \qquad 0 \le q(x)^2 - x^{2k} \le \delta L^{2k}.$$

Its coefficients in the basis $C_j(2x/L - 1)$ take $O(k)$ operations.

Proof. Set $F(t) = ((1+t)/2)^k$. Its coefficients are nonnegative, and $F(\cosh s) = \cosh(s/2)^{2k} \le e^{ks^2/4}$; here $\log\cosh u \le u^2/2$ follows by integrating $\tanh u \le u$. Taking $s = 2d/k$ in (309) gives $B_d \le 2e^{-d^2/k} \le \delta/4$ when $d < k$; otherwise $B_d = 0$. Use $q(x) = L^k Q_d(2x/L - 1)$ in (310). The squared-error bound follows from $q + x^k \le 2L^k$. Expanding $\cos^{2k}(\theta/2)$ gives

$$\alpha_0 = 4^{-k}\binom{2k}{k}, \qquad \alpha_j = 2 \cdot 4^{-k}\binom{2k}{k-j} \quad (1 \le j \le k),$$

which successive binomial ratios compute in $O(k)$ work. □

In particular, for odd $p = 2k + 1 \ge 3$, $L \ge 1$, and $0 < \Delta \le 1$, taking $\delta = \Delta/L^{2k}$ gives

$$0 \le q(x)^2 - x^{p-1} \le \Delta \quad (0 \le x \le L), \qquad \deg q = O\left(\min\{p, \sqrt{p \log(8L^p/\Delta)}\}\right). \tag{311}$$

For $p = 1$, take $q = 1$.

Exponentials.

Lemma 591 (Exponential approximation)**.** *Let $L > 0$, $0 < \delta \le 1$, and $u = \log(16/\delta)$. There is a polynomial q with $0 \le q(x) - e^{-x} \le \delta$ on $[0, L]$ and*

$$\deg q \le d := \left\lceil \frac{2u}{\operatorname{arcosh}(1 + 2u/L)} \right\rceil = O\left(1 + \sqrt{Lu} + \frac{u}{\log(e + u/L)}\right). \tag{312}$$

Construction uses $d+1$ exponential evaluations, each to accuracy $\delta/(8(d+1))$, and $O(d \log(d+1))$ operations.

Proof. Write $F(t) = e^{(L/2)(t-1)}$, so $F(1 - 2x/L) = e^{-x}$. Its power-series coefficients are nonnegative. For $s = \operatorname{arcosh}(1 + 2u/L)$, we have $F(\cosh s) = e^u$, so (309) gives $B_d \le 2e^{u - sd} \le \delta/8$. Interpolate with the stated accuracy, add $\delta/2$, and rescale. The degree estimate uses $\operatorname{arcosh}(1+z) = \Theta(\sqrt{z})$ for $z \le 1$ and $\Theta(\log(1+z))$ for $z \ge 1$. □

This implies $d = O(\sqrt{Lu} + u)$, and improves to $O(u/\log u)$ for fixed L and $u \to \infty$.[1] On $[a, b]$, use $e^x = e^b e^{-(b-x)}$ for one-sided error δe^b. Relative error δ for either exponential follows by requesting normalized error $\delta e^{-(b-a)}$, giving degree $O(b - a + \log(8/\delta))$.

[1] For optimal exponential degree bounds, see A. Aggarwal and J. Alman, *Optimal-Degree Polynomial Approximations for Exponentials and Gaussian Kernel Density Estimation*, CCC 2022. Only the upper bound is used here.

Additive inverse powers.

Lemma 592 (Inverse-power approximation)**.** *For $0 < a < b$, $\beta > 0$, $0 < \delta \le 1$, and $\kappa = b/a$, there is a polynomial with*

$$0 \le q(x) - x^{-\beta} \le \delta a^{-\beta} \quad (a \le x \le b)$$

of degree

$$d = O\Big(\sqrt{\beta(\kappa-1)\log(8/\delta)} + \sqrt{\kappa}\log(8/\delta)\Big). \tag{313}$$

Construction uses $O(d)$ scalar power evaluations and $O(d\log(d+1))$ operations.

Proof. Set $\rho = (\kappa-1)/(\kappa+1)$ and $F(t) = ((1-\rho)/(1-\rho t))^{\beta} = a^{\beta}(c-rt)^{-\beta}$. The binomial series has nonnegative coefficients. Put

$$u = \log(16/\delta), \quad v = \min\{1, u/\beta\}, \quad s = \operatorname{arcosh}\left(1 + \frac{v}{\kappa-1}\right), \quad d = \left\lceil \frac{2u}{s} \right\rceil.$$

Then $F(\cosh s) = (1-v/2)^{-\beta} \le e^{\beta v} \le e^{u}$, so $B_d \le \delta/8$. Interpolate F with value accuracy $\delta/(8(d+1))$, add $\delta/2$, rescale, and multiply by $a^{-\beta}$. Finally, $1/\operatorname{arcosh}(1+z) = O(1+z^{-1/2})$ gives $d = O(u + u\sqrt{(\kappa-1)/v})$, which is the stated bound. □

Additive error $\eta \le a^{-\beta}$ follows with $\delta = \eta a^{\beta}$. For relative error ε, one may take $\delta = \varepsilon\kappa^{-\beta}$, but the next argument avoids the resulting $\log\kappa$ loss for fractional powers.

Relative approximation by interpolating reciprocals. The same nodes approximate every shifted reciprocal relatively. Positive mixtures therefore inherit the bound, without computing any integrals.

Lemma 593 (Positive mixtures of reciprocals)**.** *Let $0 < a < b$, $\kappa = b/a$, and*

$$f(x) = \gamma + \int_0^{\infty} \frac{d\nu(t)}{x+t}, \qquad \gamma \ge 0, \quad \nu \ge 0, \quad 0 < f(a) < \infty.$$

The interpolant P at the N roots of $W_N(x) := C_N((c-x)/r)$ satisfies

$$|P(x) - f(x)| \le \frac{f(x)}{C_N(c/r)} \le 2e^{-N\theta_\kappa} f(x), \qquad \theta_\kappa := \log\frac{\sqrt{\kappa}+1}{\sqrt{\kappa}-1} \ge \frac{2}{\sqrt{\kappa}}. \tag{314}$$

Consequently, for $0 < \delta \le 1$ there are degree $O(\sqrt{\kappa}\log(2/\delta))$ polynomials satisfying

$$(1-\delta)f \le q_- \le f \le q_+ \le (1+\delta)f.$$

Specifically, use $N = \lceil \theta_\kappa^{-1}\log(16/\delta)\rceil$ values, each to accuracy $\delta f(b)/(16N)$, and $O(N\log(N+1))$ arithmetic operations.

Proof. If P_t interpolates $(x+t)^{-1}$, both sides of

$$1 - (x+t)P_t(x) = \frac{W_N(x)}{W_N(-t)}$$

are degree-at-most-N polynomials, vanish at the nodes, and equal one at $x = -t$. On $[a,b]$, $|W_N(x)| \le 1$ and $W_N(-t) = C_N((c+t)/r) \ge C_N(c/r) > 0$. Thus the reciprocal has relative error at most $C_N(c/r)^{-1}$. Interpolation is a finite linear combination of values, so it commutes with the integral; integrating the bound proves the first inequality. The second follows from $c/r = \cosh\theta_\kappa$, $C_N(c/r) = \cosh(N\theta_\kappa) \ge e^{N\theta_\kappa}/2$, and $\log((1+z)/(1-z)) \ge 2z$.

The chosen N makes the exact relative error at most $\delta/8$. Value errors add at most $\delta f(b)/8 \le \delta f(x)/8$. Divide the computed interpolant by $1+\delta/4$ and $1-\delta/4$ to obtain q_- and q_+, respectively. A known positive lower bound on $f(b)$ suffices in the requested value accuracy. □

Corollary 594 (Real powers)**.** *For $0 < \delta \le 1$ and $[a, b] \subset (0, \infty)$, all x^β with $-1 \le \beta \le 1$ have one-sided relative-δ approximations of degree $O(\sqrt{b/a}\log(2/\delta))$, with an absolute constant. For arbitrary real $\beta \neq 0$, the degree is $O(m\sqrt{b/a}\log(2m/\delta))$, where $m = \lceil|\beta|\rceil$; the polynomial may be kept factored.*

Proof. The reciprocal is a point mass at $t = 0$. For $0 < \beta < 1$, substituting $t = xu$ gives

$$x^{-\beta} = \frac{1}{J_\beta}\int_0^\infty \frac{t^{-\beta}}{x+t}\,dt, \qquad J_\beta = \int_0^\infty \frac{u^{-\beta}}{1+u}\,du \in (0,\infty).$$

Apply the mixture lemma; J_β is not computed. Positive fractional powers follow by multiplying the approximations to $x^{-(1-\beta)}$ by x. The endpoint powers are exact or reciprocal. For general β, approximate $x^{\beta/m}$ with relative error $\delta/(2m)$ and raise both bounding polynomials to the mth power. Use $(1-\delta/(2m))^m \ge 1-\delta$ and $(1+\delta/(2m))^m \le e^{\delta/2} \le 1+\delta$. □

In particular, the reciprocal interpolant is explicitly

$$q(x) = \frac{1 - C_N((c-x)/r)/C_N(c/r)}{x}.$$

Its numerator vanishes at zero, and $N = \lceil \frac{1}{2}\sqrt{\kappa}\log(2/\delta)\rceil$ gives $|1 - xq(x)| \le \delta$. For inverse square roots, let $0 < a \le 1$ and $0 < \eta \le 1/2$. The upper relative approximation with $\delta = \eta\sqrt{a}$ gives

$$0 \le p(x) - x^{-1/2} \le \eta \quad (a \le x \le 2), \qquad \deg p = O\left(a^{-1/2}\log\frac{2}{\eta\sqrt{a}}\right). \tag{315}$$

Both use $O(d)$ scalar evaluations and $O(d\log(d+1))$ operations for degree bound d. A factored power uses at most its total degree in matrix–vector products, without expanding its coefficients.

Logarithms. The same relative bound applies to $\log(x/a)$ on $[a, b]$, including exact equality at $x = a$. Indeed, partial fractions give

$$h(x) := \frac{\log(x/a)}{x-a} = \int_0^\infty \frac{dt}{(a+t)(x+t)}, \qquad h(a) = 1/a.$$

Apply the mixture lemma to h, then multiply its bounding polynomials by $x - a \ge 0$. The degree is $O(\sqrt{\kappa}\log(2/\delta))$. For additive error η in $\log x$, take $\delta = \min\{1, \eta/\log\kappa\}$ and add $\log a$.

General analytic functions. Here a contour shift supplies the error bound. A short cosine filter removes the extra $\log(1/s)$ that would arise from summing the individual coefficient bounds. The filter is needed only in this proof.

Lemma 595 (Analytic approximation)**.** *Let $c \in \mathbb{R}$, $r > 0$, and $s > 0$. Suppose f is real on $[c-r, c+r]$ and analytic on a neighborhood of*

$$\{c + r\cos z : |\operatorname{Im} z| \le s\}, \qquad |f(c + r\cos(\theta \pm is))| \le M \quad (\theta \in \mathbb{R}).$$

For $0 < \eta \le M$, set $\ell = \max\{1, \lceil s^{-1}\log(64M/\eta)\rceil\}$. Using 8ℓ real function values, each to accuracy $\eta/6$, and $O(\ell\log(\ell+1))$ operations, one can construct p such that

$$\|p - f\|_\infty \le \eta, \qquad \deg p \le 4\ell - 1 = O\left(1 + \frac{1}{s}\log\frac{2M}{\eta}\right).$$

Proof. Put $g(z) = f(c + r\cos z)$, an even 2π-periodic function, and $\rho = e^{-s}$. Shifting the Fourier coefficient integral downward gives, for $k \ge 0$,

$$\widehat{g}_k = \frac{e^{-ks}}{2\pi}\int_0^{2\pi} g(\theta - is)e^{-ik\theta}\,d\theta. \tag{316}$$

The vertical sides cancel by periodicity; negative frequencies use the upper boundary. In particular $|\widehat{g}_k| \le M\rho^{|k|}$.

Use the nonnegative Fejér kernels and their filtered combination

$$F_m(\theta) = \frac{1}{m}\left|\sum_{j=0}^{m-1} e^{ij\theta}\right|^2, \qquad V_m = 2F_{2m} - F_m.$$

The Fourier multiplier v_k of V_ℓ equals 1 for $|k| \le \ell$, $2 - |k|/\ell$ for $\ell < |k| < 2\ell$, and 0 otherwise. Let $Q = V_\ell * g$, with convolution normalized by $1/(2\pi)$. It is an even trigonometric polynomial of degree at most $2\ell - 1$. By (316), its positive-frequency remainder is the convolution of $g(\cdot - is)$ with

$$K(\theta) = \sum_{k>\ell}(1 - v_k)\rho^k e^{ik\theta} = \frac{z^{\ell+1}(1 - z^\ell)}{\ell(1 - z)^2}, \qquad z = \rho e^{i\theta}.$$

The negative-frequency remainder uses the reflected kernel and the upper boundary. The geometric-series identity $(2\pi)^{-1}\int_0^{2\pi} |1 - \rho e^{i\theta}|^{-2} d\theta = (1 - \rho^2)^{-1}$ therefore gives

$$\|g - Q\|_\infty \le \frac{4Me^{-(\ell+1)s}}{\ell(1 - e^{-2s})} \le 8Me^{-\ell s}. \tag{317}$$

The last step uses $1 - e^{-2s} \ge \frac{1}{2}\min\{s, 1\}$ and $\ell \min\{s, 1\} \ge 1$.

To construct a polynomial from real values, sample at $\theta_j = 2\pi j/(8\ell)$ and form

$$(\mathcal{A}g)(\theta) = \frac{1}{8\ell}\sum_{j=0}^{8\ell-1} g(\theta_j)V_{2\ell}(\theta - \theta_j).$$

For $m \in \{2\ell, 4\ell\}$ the discrete average of $F_m(\theta - \theta_j)$ is one, since its nonconstant Fourier modes cancel. Thus $\|\mathcal{A}\| \le 3$, including for errors in the samples. Moreover $\mathcal{A}Q = Q$: the multiplier is one on the frequencies of Q, and no aliases occur between them and those of $V_{2\ell}$ on the 8ℓ-point grid. Consequently, with value error $\zeta = \eta/6$,

$$\|\widetilde{\mathcal{A}}g - g\|_\infty \le 4\|Q - g\|_\infty + 3\zeta \le 32Me^{-\ell s} + \eta/2 \le \eta.$$

Symmetrize the supplied values under $j \mapsto 8\ell - j$; this does not increase their errors. The result is an even trigonometric polynomial of degree at most $4\ell - 1$, hence a polynomial in $\cos\theta$. A discrete Fourier transform followed by multiplication by the filter constructs its coefficients in $O(\ell \log(\ell + 1))$ operations. Only the analysis uses Q and complex values. □

For a positive interval with $a = c - r$ and $0 < s \le 1$, the simple estimate

$$\operatorname{Re}(c + r\cos(\theta + it)) \ge a - r(\cosh s - 1) \ge a - rs^2 \qquad (|t| \le s) \tag{318}$$

can verify the domain hypothesis: $rs^2 \le a/2$ keeps the image in $\operatorname{Re} z \ge a/2$. One must also bound f there.

For finite smoothness, a simpler estimate suffices. If $g(\theta) = f(c + r\cos\theta)$ is p times continuously differentiable, $p \ge 2$, and $K = (2\pi)^{-1}\int_0^{2\pi} |g^{(p)}(\theta)|\, d\theta$, integration by parts gives $|\widehat{g}_j| \le Kj^{-p}$. Since the Chebyshev coefficients are $2\widehat{g}_j$ for $j \ge 1$, Lemma 588 yields

$$\|\widetilde{I}_d f - f\|_\infty \le 4K\sum_{j>d} j^{-p} + 2(d + 1)\zeta \le \frac{4K}{(p - 1)d^{p-1}} + 2(d + 1)\zeta \qquad (d \ge 1).$$

15.5 Integral Representations and Fractional Powers

15.5.1 The Logarithm

The logarithm has a simpler resolvent representation than the fractional powers, so it is best proved directly.

Lemma 596 (Resolvent representation of the logarithm)**.** *For every $A \succ 0$,*

$$\log A = \int_0^\infty \left(\frac{1}{1+t} I - (A + tI)^{-1} \right) dt.$$

The integral converges in operator norm.

Proof. Start with the scalar identity. For $x > 0$ and $R > 0$,

$$\begin{aligned} \int_0^R \left(\frac{1}{1+t} - \frac{1}{x+t} \right) dt &= \left[\log(1+t) - \log(x+t) \right]_0^R \\ &= \log(1+R) - \log(x+R) + \log x \\ &= \log x + \log \frac{1+R}{x+R}. \end{aligned}$$

As $R \to \infty$,

$$\frac{1+R}{x+R} \to 1,$$

so

$$\log x = \int_0^\infty \left(\frac{1}{1+t} - \frac{1}{x+t} \right) dt.$$

Now diagonalize

$$A = U \operatorname{diag}(\lambda_1, \ldots, \lambda_n) U^*.$$

Then

$$(A + tI)^{-1} = U \operatorname{diag} \left(\frac{1}{\lambda_1 + t}, \ldots, \frac{1}{\lambda_n + t} \right) U^*.$$

Applying the scalar identity to each eigenvalue gives

$$\begin{aligned} \int_0^\infty \left(\frac{1}{1+t} I - (A + tI)^{-1} \right) dt &= U \operatorname{diag}(\log \lambda_1, \ldots, \log \lambda_n) U^* \\ &= \log A. \end{aligned}$$

Since there are finitely many positive eigenvalues and the scalar integrand is $O(t^{-2})$ as $t \to \infty$, the integral converges in operator norm. □

Differentiating this representation gives another useful form of the Fréchet derivative. By the inverse rule,

$$D\big((X + tI)^{-1}\big)[H] = -(X + tI)^{-1} H (X + tI)^{-1}.$$

Therefore

$$D \log(X)[H] = \int_0^\infty (X + tI)^{-1} H (X + tI)^{-1} \, dt.$$

Differentiation under the integral is justified locally uniformly for $X \succ 0$, since the differentiated integrand is $O((1+t)^{-2})$.

15.5.2 Fractional Powers

Many matrix inequalities for fractional powers come from one scalar formula. For $0 < \alpha < 1$, let

$$c_\alpha = \frac{\sin(\pi\alpha)}{\pi}.$$

For $0 < \alpha < 1$, set

$$I_\alpha = \int_0^\infty \frac{x^{\alpha-1}}{1+x}\,dx.$$

Consider

$$f(z) = \frac{z^{\alpha-1}}{1+z},$$

using the branch

$$0 < \arg z < 2\pi,$$

so the branch cut is the positive real axis. Integrate around a keyhole contour enclosing the slit plane.

On the upper side of the positive real axis,

$$z^{\alpha-1} = x^{\alpha-1},$$

so the contribution tends to I_α.

On the lower side, $\arg z = 2\pi$, and the contour runs from ∞ back to 0. Hence

$$z^{\alpha-1} = e^{2\pi i(\alpha-1)}x^{\alpha-1} = e^{2\pi i\alpha}x^{\alpha-1},$$

and its contribution tends to

$$-e^{2\pi i\alpha}I_\alpha.$$

The two circular arcs vanish: on the outer circle their contribution is

$$O(R^{\alpha-1}) \to 0,$$

while on the inner circle it is

$$O(\varepsilon^\alpha) \to 0.$$

The only pole inside the contour is $z = -1$. Since $\arg(-1) = \pi$,

$$\operatorname{Res}_{z=-1} f(z) = (-1)^{\alpha-1} = e^{i\pi(\alpha-1)} = -e^{i\pi\alpha}.$$

Therefore the residue theorem gives

$$(1 - e^{2\pi i\alpha})I_\alpha = -2\pi i e^{i\pi\alpha}.$$

Using

$$1 - e^{2\pi i\alpha} = -2ie^{i\pi\alpha}\sin(\pi\alpha),$$

we obtain

$$I_\alpha = \frac{\pi}{\sin(\pi\alpha)}.$$

Consequently, after the substitution $t = xs$,

$$\int_0^\infty t^{\alpha-1}\frac{x}{t+x}\,dt = x^\alpha\int_0^\infty \frac{s^{\alpha-1}}{1+s}\,ds = x^\alpha\frac{\pi}{\sin(\pi\alpha)}.$$

Thus

$$x^\alpha = \frac{\sin(\pi\alpha)}{\pi}\int_0^\infty t^{\alpha-1}\frac{x}{t+x}\,dt.$$

Applying the scalar identity to every eigenvalue gives the matrix formula.

Lemma 597 (Formula for Fractional Powers)**.** *For $A \succ 0$ and $0 < \alpha < 1$,*

$$A^{\alpha} = c_{\alpha} \int_0^{\infty} t^{\alpha-1} A (tI + A)^{-1} \, dt.$$

Equivalently,

$$A^{\alpha} = c_{\alpha} \int_0^{\infty} t^{\alpha} \left(\frac{1}{t} I - (tI + A)^{-1} \right) dt.$$

Two other useful forms follow immediately. If $-1 < p < 0$, then

$$A^{p} = \frac{\sin(-\pi p)}{\pi} \int_0^{\infty} t^{p} (tI + A)^{-1} \, dt.$$

If $1 < p < 2$, applying the preceding formula to A^{p-1} and multiplying by A gives

$$A^{p} = c_{p-1} \int_0^{\infty} t^{p-2} A^2 (tI + A)^{-1} \, dt.$$

Using

$$A^2 (tI + A)^{-1} = A - tI + t^2 (tI + A)^{-1}$$

recovers the more expanded resolvent formula when that form is convenient.

We also have the scalar identity

$$|x|^{\alpha} = c'_{\alpha} \int_0^{\infty} \frac{1 - \cos(tx)}{t^{\alpha+1}} \, dt, \qquad 0 < \alpha < 2,$$

for a positive constant c'_{α}.

15.5.3 Operator Convexity of Inversion

The formulas are useful because inversion has a very simple matrix-convexity property.

Lemma 598 (Operator Convexity of Inversion)**.** *For $A, B \succ 0$ and $0 \le \lambda \le 1$,*

$$(\lambda A + (1-\lambda) B)^{-1} \preceq \lambda A^{-1} + (1-\lambda) B^{-1}.$$

Proof. For every $A \succ 0$,

$$\begin{pmatrix} A^{-1} & I \\ I & A \end{pmatrix} = \begin{pmatrix} A^{-1/2} \\ A^{1/2} \end{pmatrix} \begin{pmatrix} A^{-1/2} & A^{1/2} \end{pmatrix} \succeq 0.$$

Average this matrix with its analogue for B:

$$\begin{pmatrix} \lambda A^{-1} + (1-\lambda) B^{-1} & I \\ I & \lambda A + (1-\lambda) B \end{pmatrix} \succeq 0.$$

Taking the Schur complement of the lower-right block gives exactly the claimed inequality. □

We also repeatedly use that inversion reverses the Loewner order:

$$0 \prec A \preceq B \qquad \Longrightarrow \qquad B^{-1} \preceq A^{-1}.$$

Indeed,

$$I \preceq A^{-1/2} B A^{-1/2}.$$

Inverting reverses the inequality,

$$A^{1/2} B^{-1} A^{1/2} \preceq I,$$

and conjugating by $A^{-1/2}$ gives $B^{-1} \preceq A^{-1}$.

15.5.4 Löwner–Heinz and Operator Concavity of Powers

The same formula now gives two fundamental facts about fractional powers essentially for free.

Theorem 599 (Löwner–Heinz)**.** *If*

$$0 \preceq A \preceq B$$

and $0 \le \alpha \le 1$*, then*

$$A^\alpha \preceq B^\alpha.$$

Thus $x \mapsto x^\alpha$ *is operator monotone on the positive cone.*

Proof. It is enough to consider $0 < \alpha < 1$ and positive definite A, B; the semidefinite case follows by replacing A, B with $A + \varepsilon I, B + \varepsilon I$ and taking $\varepsilon \downarrow 0$.

For every $t > 0$,

$$A \preceq B \quad \Longrightarrow \quad (tI + A)^{-1} \succeq (tI + B)^{-1}.$$

Therefore

$$A(tI + A)^{-1} = I - t(tI + A)^{-1} \preceq I - t(tI + B)^{-1} = B(tI + B)^{-1}.$$

Integrating this inequality against the positive weight $c_\alpha t^{\alpha-1}$ proves $A^\alpha \preceq B^\alpha$. □

Lemma 600 (Operator Concavity of Fractional Powers)**.** *For* $0 \le \alpha \le 1$*, the map*

$$A \longmapsto A^\alpha$$

is operator concave on the positive cone.

Proof. Again take $0 < \alpha < 1$. For fixed $t > 0$,

$$A(tI + A)^{-1} = I - t(tI + A)^{-1}.$$

By Lemma 598, the map

$$A \longmapsto (tI + A)^{-1}$$

is operator convex. Hence its negative, and therefore $A \mapsto A(tI + A)^{-1}$, is operator concave.

The representation Lemma 597 is a positive weighted integral of these maps, so

$$A \longmapsto A^\alpha$$

is operator concave. □

15.6 Golden–Thompson Inequality

The proof below has one basic mechanism: repeated Cauchy–Schwarz makes a word in A and A^* more alternating. The completely alternating word is therefore the extremal one. The Lie–Trotter formula then turns this "word-straightening" statement into Golden–Thompson.

Lemma 601 (Word Straightening)**.** *Let* $A \in M_d(\mathbb{C})$*, and let*

$$A_1, \dots, A_{2n} \in \{A, A^*\}.$$

Then

$$|\mathrm{Tr}(A_1 \cdots A_{2n})| \le \mathrm{Tr}((A^*A)^n).$$

Proof. If $A = A^*$ the claim is immediate. Otherwise regard A and A^* as two distinct symbols.

Among all length-$2n$ words in A, A^*, choose one

$$P = A_1 \cdots A_{2n}$$

maximizing $|\operatorname{Tr} P|$, and among all such maximizers choose one with the largest number of transitions between A and A^* in cyclic order.

If every adjacent pair is different, the word alternates, so it is a cyclic shift of either

$$(A^*A)^n \qquad \text{or} \qquad (AA^*)^n.$$

These have the same trace, and we are done.

Otherwise two adjacent symbols agree. By cyclically rotating the word, we may arrange that the equal pair occurs across the middle. Write

$$P = QR, \qquad Q = A_1 \cdots A_n, \qquad R = A_{n+1} \cdots A_{2n},$$

with

$$A_n = A_{n+1}.$$

Frobenius Cauchy–Schwarz gives

$$|\operatorname{Tr}(QR)|^2 \le \operatorname{Tr}(Q^*Q)\operatorname{Tr}(R^*R).$$

Both Q^*Q and R^*R are again length-$2n$ words in A, A^*. Since P was a maximizer, neither trace magnitude can exceed $|\operatorname{Tr} P|$. Hence equality must hold throughout, and both Q^*Q and R^*R are themselves maximizers.

It remains to compare transitions. Let t_Q, t_R be the numbers of transitions internal to Q, R. The chosen middle pair contributes no transition, so P has at most

$$t_Q + t_R + 1$$

cyclic transitions.

In Q^*Q, the transitions internal to Q occur once in each half, and the two junctions are automatically transitions because a letter meets its adjoint. Thus Q^*Q has exactly

$$2t_Q + 2$$

transitions, and similarly R^*R has $2t_R + 2$. Their average is

$$t_Q + t_R + 2,$$

strictly larger than the number of transitions of P. Therefore one of Q^*Q, R^*R is a maximizing word with more transitions than P, a contradiction.

So every maximizing word alternates, proving the claim. □

Lemma 602 (Disentangling)**.** *Let U, V be Hermitian and let $k \ge 1$. Then*

$$\begin{aligned}\left|\operatorname{Tr}((UV)^{2^k})\right| &\le \operatorname{Tr}((U^2V^2)^{2^{k-1}}) \\ &\le \operatorname{Tr}((U^4V^4)^{2^{k-2}}) \\ &\le \cdots \le \operatorname{Tr}(U^{2^k}V^{2^k}).\end{aligned}$$

Proof. Apply Lemma 601 to the matrix $A = UV$ and to the word consisting of 2^k copies of A. Since $A^* = VU$,

$$\left|\operatorname{Tr}((UV)^{2^k})\right| \le \operatorname{Tr}((VU^2V)^{2^{k-1}}).$$

Cyclicity of trace turns the right-hand side into

$$\operatorname{Tr}((U^2V^2)^{2^{k-1}}).$$

Now repeat with U^2, V^2, then U^4, V^4, and so on. □

Theorem 603 (Golden–Thompson)**.** *For Hermitian A, B,*

$$\operatorname{Tr}(e^{A+B}) \le \operatorname{Tr}(e^{A}e^{B}).$$

Proof. The Lie–Trotter product formula says

$$e^{A+B} = \lim_{N\to\infty} \left(e^{A/N}e^{B/N}\right)^{N}.$$

For completeness, this follows from

$$e^{A/N}e^{B/N} = I + \frac{A+B}{N} + O(N^{-2})$$

in operator norm. For sufficiently large N the matrix on the left lies near I, so its matrix logarithm is defined there and

$$\log(e^{A/N}e^{B/N}) = \frac{A+B}{N} + O(N^{-2}).$$

Thus

$$\left(e^{A/N}e^{B/N}\right)^{N} = \exp\left(A + B + O(N^{-1})\right) \longrightarrow e^{A+B}.$$

Take $N = 2^k$ and set

$$U = e^{A/2^k}, \qquad V = e^{B/2^k}.$$

These are positive definite, hence Hermitian. By Lemma 602,

$$\left|\operatorname{Tr}\left((e^{A/2^k}e^{B/2^k})^{2^k}\right)\right| \le \operatorname{Tr}(e^{A}e^{B}).$$

Letting $k \to \infty$, the matrix inside the trace converges to e^{A+B}. Since $e^{A+B} \succ 0$, its trace is positive, and the limit gives

$$\operatorname{Tr}(e^{A+B}) \le \operatorname{Tr}(e^{A}e^{B}).$$

□

15.7 Trace Rearrangement and Schatten Hölder Inequalities

The matrix analogue of the scalar rearrangement inequality is von Neumann's trace inequality. Once this is established, the two-factor Schatten Hölder inequality is literally scalar Hölder applied to the singular values. Exterior powers then give the corresponding product inequality and hence the general multi-factor trace form.

Lemma 604 (Trace Rearrangement for Hermitian Matrices)**.** *Let X, Y be Hermitian with eigenvalues*

$$\lambda_1(X) \ge \cdots \ge \lambda_n(X), \qquad \lambda_1(Y) \ge \cdots \ge \lambda_n(Y).$$

Then

$$\operatorname{Tr}(XY) \le \sum_{i=1}^{n} \lambda_i(X)\lambda_i(Y).$$

Proof. Write

$$X = \sum_i \lambda_i(X) u_i u_i^*, \qquad Y = \sum_j \lambda_j(Y) v_j v_j^*.$$

Then

$$\operatorname{Tr}(XY) = \sum_{i,j} \lambda_i(X)\lambda_j(Y)|\langle u_i, v_j\rangle|^2.$$

The matrix

$$M_{ij} = |\langle u_i, v_j\rangle|^2$$

is doubly stochastic, because both (u_i) and (v_j) are orthonormal bases.

By the Birkhoff–von Neumann theorem, M is a convex combination of permutation matrices. Therefore $\operatorname{Tr}(XY)$ is a convex combination of quantities

$$\sum_i \lambda_i(X)\lambda_{\pi(i)}(Y).$$

The scalar rearrangement inequality says that each is at most the aligned sum

$$\sum_i \lambda_i(X)\lambda_i(Y).$$

□

Theorem 605 (Von Neumann's Trace Inequality)**.** *For arbitrary $A, B \in M_n(\mathbb{C})$,*

$$|\operatorname{Tr}(A^*B)| \le \sum_{i=1}^{n} \sigma_i(A)\sigma_i(B),$$

where the singular values are sorted in decreasing order.

Proof. For a matrix A, form its Hermitian dilation

$$\mathcal{H}(A) = \begin{pmatrix} 0 & A \\ A^* & 0 \end{pmatrix}.$$

If

$$A = U\Sigma V^*$$

is a singular-value decomposition, then the eigenvalues of $\mathcal{H}(A)$ are

$$\sigma_1(A), \dots, \sigma_n(A), -\sigma_n(A), \dots, -\sigma_1(A).$$

Let

$$z = \operatorname{Tr}(A^*B).$$

If $z = 0$ there is nothing to prove. Otherwise multiply B by a complex phase so that z becomes positive real; this does not change the singular values of B. Then

$$\operatorname{Tr}(\mathcal{H}(A)\mathcal{H}(B)) = 2\operatorname{Re}\operatorname{Tr}(A^*B) = 2|z|.$$

Apply Lemma 604 to the two Hermitian dilations. Pairing their ordered eigenvalues gives

$$2|z| \le 2\sum_{i=1}^{n} \sigma_i(A)\sigma_i(B).$$

Divide by 2. □

Corollary 606 (Schatten Hölder, Trace Form)**.** *If*

$$\frac{1}{p} + \frac{1}{q} = 1, \qquad 1 \le p, q \le \infty,$$

then

$$|\operatorname{Tr}(A^*B)| \le \|A\|_{S_p}\|B\|_{S_q}.$$

Proof. By von Neumann's inequality and scalar Hölder,

$$\begin{aligned}|\operatorname{Tr}(A^*B)| &\le \sum_i \sigma_i(A)\sigma_i(B) \\ &\le \left(\sum_i \sigma_i(A)^p\right)^{1/p} \left(\sum_i \sigma_i(B)^q\right)^{1/q} \\ &= \|A\|_{S_p}\|B\|_{S_q}.\end{aligned}$$

The endpoint cases have the usual interpretation. □

The same exterior-power idea gives the stronger product form. Recall that $\bigwedge^k A$ acts on $\bigwedge^k \mathbb{C}^n$, satisfies

$$\bigwedge^k (AB) = (\bigwedge^k A)(\bigwedge^k B),$$

and has operator norm

$$\left\|\bigwedge^k A\right\| = \prod_{i=1}^k \sigma_i(A).$$

The last identity follows immediately from an SVD: the singular values of $\bigwedge^k A$ are all k-fold products of singular values of A, and the largest such product uses the k largest singular values.

Lemma 607 (Singular Values of a Product)**.** *For every k,*

$$\prod_{i=1}^k \sigma_i(AB) \le \prod_{i=1}^k \sigma_i(A)\sigma_i(B).$$

Equivalently,

$$\sigma(AB) \prec_{w,\log} \sigma(A)\sigma(B),$$

where the product on the right is coordinatewise.

Proof. By multiplicativity of exterior powers and submultiplicativity of the operator norm,

$$\begin{aligned}\prod_{i=1}^k \sigma_i(AB) &= \left\|\bigwedge^k (AB)\right\| \\ &= \left\|(\bigwedge^k A)(\bigwedge^k B)\right\| \\ &\le \left\|\bigwedge^k A\right\| \left\|\bigwedge^k B\right\| \\ &= \prod_{i=1}^k \sigma_i(A)\sigma_i(B).\end{aligned}$$

□

Theorem 608 (Schatten Hölder, Product Form)**.** *Let*

$$1 \le p, q, r \le \infty, \qquad \frac{1}{r} = \frac{1}{p} + \frac{1}{q}.$$

Then

$$\|AB\|_{S_r} \le \|A\|_{S_p}\|B\|_{S_q}.$$

Consequently, if $p_1, \dots, p_m \in [1,\infty]$ *satisfy*

$$\sum_{j=1}^{m} \frac{1}{p_j} = 1,$$

then

$$|\operatorname{Tr}(A_1 \cdots A_m)| \le \prod_{j=1}^{m} \|A_j\|_{S_{p_j}}.$$

Proof. Assume first that A, B are invertible; the general case follows by regularization and continuity.

By Lemma 607, the logarithms of the singular values of AB are weakly majorized by

$$\log \sigma_i(A) + \log \sigma_i(B).$$

Applying the increasing convex function

$$x \longmapsto e^{rx}$$

gives

$$\sum_i \sigma_i(AB)^r \le \sum_i \big(\sigma_i(A)\sigma_i(B)\big)^r.$$

Scalar Hölder with conjugate exponents p/r and q/r now gives

$$\sum_i \big(\sigma_i(A)\sigma_i(B)\big)^r \le \left(\sum_i \sigma_i(A)^p\right)^{r/p} \left(\sum_i \sigma_i(B)^q\right)^{r/q}.$$

Taking the r-th root proves

$$\|AB\|_{S_r} \le \|A\|_{S_p}\|B\|_{S_q}.$$

For the multi-factor form, define

$$\frac{1}{r_j} = \sum_{i=1}^{j} \frac{1}{p_i}.$$

Since the total reciprocal sum is 1, each intermediate $r_j \ge 1$. Repeatedly applying the product inequality gives

$$\|A_1 \cdots A_m\|_{S_1} \le \prod_{j=1}^{m} \|A_j\|_{S_{p_j}}.$$

Finally,

$$|\operatorname{Tr} X| \le \|X\|_{S_1}$$

by Theorem 605 applied to I and X. □

15.8 Generalized Klein Inequality

Lemma 609 (Generalized Klein Inequality)**.** *Let* $f_i, g_i : I \to \mathbb{R}$ *be functions on an interval* I *such that* $\sum_i f_i(x)g_i(y) \ge 0$ *for all* $x, y \in I$*. If* A *and* H *are Hermitian matrices with eigenvalues in* I*, then* $\sum_i \operatorname{Tr}[f_i(A)g_i(H)] \ge 0$*.*

Proof. Consider the eigenvalue decompositions $A = \sum_j \lambda_j u_j u_j^*$ and $H = \sum_k \mu_k v_k v_k^*$. We evaluate the sum of traces:

$$\sum_i \operatorname{Tr}[f_i(A) g_i(H)] = \sum_i \operatorname{Tr}\left[\left(\sum_j f_i(\lambda_j) u_j u_j^*\right)\left(\sum_k g_i(\mu_k) v_k v_k^*\right)\right]$$

Using the linearity and cyclic property of the trace, this simplifies to:

$$\sum_{j,k}\left[\sum_i f_i(\lambda_j) g_i(\mu_k)\right] \operatorname{Tr}(u_j u_j^* v_k v_k^*) = \sum_{j,k}\left[\sum_i f_i(\lambda_j) g_i(\mu_k)\right] |\langle u_j, v_k\rangle|^2$$

By assumption, the bracketed sum is non-negative for all eigenvalue pairs (λ_j, μ_k), and the squared inner product $|\langle u_j, v_k\rangle|^2 \geq 0$. Thus, the entire sum is non-negative. □

15.9 Wielandt Minimax Formula

Theorem 610 (Wielandt Minimax Formula)**.** *Let A be an $n \times n$ Hermitian matrix with eigenvalues $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n$. For any sequence of indices $1 \leq i_1 < i_2 < \cdots < i_k \leq n$,*

$$\sum_{j=1}^{k} \lambda_{i_j}(A) = \max_{\substack{V_1 \subset \cdots \subset V_k \\ \dim(V_j) = i_j}} \min_{W \in X(V_1, \ldots, V_k)} \operatorname{tr}_W(A)$$

where $X(V_1, \ldots, V_k)$ denotes the set of k-dimensional subspaces W that admit a basis $v_1, \ldots, v_k$ with $v_j \in V_j$.

Proof. **The $\leq$ case:** Let $V_j = \operatorname{span}(e_1, \ldots, e_{i_j})$, where e_i are the eigenvectors of A. For any $W \in X(V_1, \ldots, V_k)$, it remains to show that $\sum_{j=1}^k \lambda_{i_j}(A) \leq \operatorname{tr}_W(A)$.

To show this, we construct an orthonormal set of vectors $v_1, \ldots, v_k$ such that $v_j \in V_j \cap W$. Since $\dim(V_1 \cap W) \geq 1$, we pick any unit vector $v_1 \in V_1 \cap W$. Next, since $\dim(V_2 \cap W) \geq 2$, we pick any unit vector $v_2 \in V_2 \cap W$ that is perpendicular to v_1, and so on. Then we have

$$\operatorname{tr}_W(A) \geq \sum_{j=1}^{k} \langle v_j, A v_j\rangle \geq \sum_{j=1}^{k} \lambda_{i_j}(A),$$

where the last inequality follows from the Rayleigh quotient minimum property on the spaces V_j.

The $\geq$ case: For any sequence of subspaces $V_1 \subset \cdots \subset V_k$ with $\dim(V_j) = i_j$, we must find some $W \in X(V_1, \ldots, V_k)$ such that $\sum_{j=1}^k \lambda_{i_j}(A) \geq \operatorname{tr}_W(A)$.

We prove this by induction on n. The $n = 1$ case is trivially the Courant-Fischer theorem. Assume now $n \geq 2$.

If $i_1 \geq 2$, then we can apply induction. Let $E = \operatorname{span}(e_{i_1}, \ldots, e_n)$. We construct a partial flag within E from the intersection of E with $V_1, \ldots, V_k$. We begin by picking an $(i_k - (i_1 - 1))$-dimensional subspace $W_k' \subset E \cap V_k$, which exists by counting dimensions (it has codimension $i_1 - 1$ within V_k). Then we go down by one space, picking an $(i_{k-1} - (i_1 - 1))$-dimensional subspace $W_{k-1}' \subset W_k' \cap V_{k-1}$. This still exists. We repeat this to form a flag $W_1' \subset \cdots \subset W_k'$. Now, since $\dim(E) \leq n - 1$, we can apply the induction hypothesis: there exists some $W \in X(W_1', \ldots, W_k')$ such that

$$\sum_{j=1}^{k} \lambda_{i_j - (i_1 - 1)}(A|_E) \geq \operatorname{tr}_W(A).$$

Note that $\lambda_{i_j-(i_1-1)}(A|_E)$ is the $(i_j - (i_1 - 1))$-th eigenvalue of A orthogonally projected down to E. By the Cauchy interlacing theorem, $\lambda_{i_j-(i_1-1)}(A|_E) \leq \lambda_{i_j}(A)$. Since $X(W_1', \ldots, W_k') \subset X(V_1, \ldots, V_k)$, we are done.

If $i_1 = 1$, we perform a similar construction. Let $E = \operatorname{span}(e_2, \ldots, e_n)$. If $V_k \subset E$, then we can induct immediately. Otherwise, we construct a partial flag sequence $W_2' \subset \cdots \subset W_k'$ inside E. By induction, there exists some $W' \in X(W_2', \ldots, W_k') \subset X(V_2, \ldots, V_k)$ such that

$$\sum_{j=2}^{k} \lambda_{i_j - 1}(A|_E) \geq \operatorname{tr}_{W'}(A).$$

Thus, by Cauchy interlacing, $\sum_{j=2}^{k} \lambda_{i_j}(A) \geq \operatorname{tr}_{W'}(A)$. It remains to find some vector v such that $W' \oplus \operatorname{span}(v) \in X(V_1, \ldots, V_k)$. If $V_1 \not\subset W'$, then any $v \in V_1 \setminus W'$ would work. Otherwise, if $V_2 \not\subset W'$, then any $v \in V_2 \setminus W'$ would work, and so on. If none of these work, it implies $V_k \subset E$, which is a contradiction. This completes the induction. □

Corollary 611 (Weyl's Inequality)**.** *Let A and B be $n \times n$ Hermitian matrices with eigenvalues sorted in descending order: $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n$. For all $1 \leq i, j \leq n$ such that $i + j - 1 \leq n$, we have:*

$$\lambda_{i+j-1}(A + B) \leq \lambda_i(A) + \lambda_j(B)$$

Theorem 612 (Lidskii's Inequality)**.** *Let A and B be $n \times n$ Hermitian matrices with their eigenvalues sorted in descending order, $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n$. For any sequence of indices $1 \leq i_1 < \cdots < i_k \leq n$, it holds that*

$$\sum_{j=1}^{k} \lambda_{i_j}(A + B) \leq \sum_{j=1}^{k} \lambda_{i_j}(A) + \sum_{j=1}^{k} \lambda_j(B)$$
$$\sum_{j=1}^{k} \lambda_{i_j}(A + B) \geq \sum_{j=1}^{k} \lambda_{i_j}(A) + \sum_{j=1}^{k} \lambda_{n-j+1}(B)$$

Corollary 613 (Extremal Partial Trace)**.** *Let A be a Hermitian matrix. We have:*

$$\sum_{j=1}^{k} \lambda_j(A) = \sup_{\dim(V)=k} \operatorname{tr}_V(A)$$
$$\sum_{j=1}^{k} \lambda_{n-j+1}(A) = \inf_{\dim(V)=k} \operatorname{tr}_V(A)$$

Corollary 614 (Schur-Horn Inequality)**.** *The sum of the largest k eigenvalues is a convex function, and the sum of the smallest k eigenvalues is a concave function. Furthermore, for any subset of indices $i_1 < \cdots < i_k$,*

$$\sum_{j=1}^{k} \lambda_{n-j+1}(A) \leq \sum_{j=1}^{k} A_{i_j, i_j} \leq \sum_{j=1}^{k} \lambda_j(A).$$

Corollary 615 (Hadamard's Determinant Inequality)**.** *For every positive definite matrix A,*

$$\det A \leq \prod_{i=1}^{n} A_{ii}.$$

Proof. Write

$$A = V^* V$$

for an invertible matrix V, and let $v_1, \ldots, v_n$ be the columns of V. Then

$$A_{ij} = \langle v_i, v_j \rangle, \qquad A_{ii} = \|v_i\|_2^2.$$

Moreover,

$$\det A = |\det V|^2.$$

Geometrically, $|\det V|$ is the volume of the parallelepiped spanned by $v_1, \dots, v_n$. Gram–Schmidt writes this volume as the product of the successive perpendicular heights:

$$|\det V| = \prod_{i=1}^{n} \operatorname{dist}\big(v_i, \operatorname{span}(v_1, \dots, v_{i-1})\big) \le \prod_{i=1}^{n} \|v_i\|_2.$$

Squaring gives

$$\det A \le \prod_i \|v_i\|_2^2 = \prod_i A_{ii}.$$

□

15.10 The Operator Jensen Inequality

Ordinary Jensen says that applying a convex function after averaging is better than averaging after applying the function. The operator version allows the "weights" themselves to be matrices.

The clean way to understand the proof is in two steps. First embed the matrix convex combination as the compression of a larger block matrix. Then average that block matrix with a sign-flipped copy to erase its off-diagonal blocks.

Definition 616 (Matrix Convex Combination)**.** Let A_1, A_2 be Hermitian and suppose

$$K_1^* K_1 + K_2^* K_2 = I.$$

Then

$$K_1^* A_1 K_1 + K_2^* A_2 K_2$$

is called a matrix convex combination of A_1, A_2.

Theorem 617 (Operator Jensen)**.** *Let f be operator convex on an interval I, let A_1, A_2 be Hermitian with spectra in I, and suppose*

$$K_1^* K_1 + K_2^* K_2 = I.$$

Then

$$f(K_1^* A_1 K_1 + K_2^* A_2 K_2) \preceq K_1^* f(A_1) K_1 + K_2^* f(A_2) K_2.$$

For operator-concave f, the inequality reverses.

Proof. Put

$$\mathcal{A} = \begin{pmatrix} A_1 & 0 \\ 0 & A_2 \end{pmatrix}, \qquad V = \begin{pmatrix} K_1 \\ K_2 \end{pmatrix}.$$

The condition on K_1, K_2 says precisely that

$$V^* V = I,$$

so V is an isometry. Extend its columns to a unitary matrix

$$Q = (V\ W).$$

Then

$$M := Q^* \mathcal{A} Q = \begin{pmatrix} C & * \\ * & D \end{pmatrix},$$

where

$$C = V^* \mathcal{A} V = K_1^* A_1 K_1 + K_2^* A_2 K_2.$$

Now let

$$U = \begin{pmatrix} I & 0 \\ 0 & -I \end{pmatrix}.$$

Averaging M with U^*MU deletes the off-diagonal blocks:

$$\frac{1}{2}(M + U^*MU) = \begin{pmatrix} C & 0 \\ 0 & D \end{pmatrix}.$$

Operator convexity therefore gives

$$\begin{aligned} \begin{pmatrix} f(C) & 0 \\ 0 & f(D) \end{pmatrix} &= f\left(\frac{1}{2}(M + U^*MU)\right) \\ &\preceq \frac{1}{2}\big(f(M) + U^*f(M)U\big). \end{aligned}$$

Because functional calculus commutes with unitary conjugation,

$$f(M) = Q^*f(\mathcal{A})Q.$$

Taking the upper-left block gives

$$f(C) \preceq V^*f(\mathcal{A})V = K_1^*f(A_1)K_1 + K_2^*f(A_2)K_2.$$

The concave version follows by applying the convex statement to $-f$. □

15.11 Lieb's Concavity Theorem

In one variable,

$$x^q y^{1-q}$$

is the weighted geometric mean of x and y. The operator perspective gives the canonical noncommutative version of this expression, and Operator Jensen gives its joint concavity.

Definition 618 (Operator Perspective)**.** Let $f : (0, \infty) \to \mathbb{R}$ and let $A, B \succ 0$. Define

$$\mathcal{P}_f(A, B) = B^{1/2} f(B^{-1/2} A B^{-1/2}) B^{1/2}.$$

If A and B commute, this reduces to

$$\mathcal{P}_f(A, B) = B f(AB^{-1}).$$

In particular, for $f(t) = t^q$,

$$\mathcal{P}_q(A, B) = B^{1/2}(B^{-1/2} A B^{-1/2})^q B^{1/2},$$

the weighted matrix geometric mean.

Lemma 619 (Perspective Preserves Operator Concavity)**.** *If f is operator concave, then*

$$(A, B) \longmapsto \mathcal{P}_f(A, B)$$

is jointly operator concave on positive-definite pairs.

Proof. Take two positive-definite pairs

$$(A_1, B_1), \qquad (A_2, B_2)$$

and a weight $\lambda \in [0, 1]$. Put

$$A = \lambda A_1 + (1 - \lambda)A_2, \qquad B = \lambda B_1 + (1 - \lambda)B_2,$$

and define

$$K_1 = \sqrt{\lambda}\, B_1^{1/2} B^{-1/2}, \qquad K_2 = \sqrt{1-\lambda}\, B_2^{1/2} B^{-1/2}.$$

Then

$$K_1^* K_1 + K_2^* K_2 = B^{-1/2}\big(\lambda B_1 + (1-\lambda) B_2\big) B^{-1/2} = I.$$

Set

$$X_i = B_i^{-1/2} A_i B_i^{-1/2}.$$

A direct calculation gives

$$K_1^* X_1 K_1 + K_2^* X_2 K_2 = B^{-1/2} A B^{-1/2}.$$

The concave form of Operator Jensen therefore gives

$$f(B^{-1/2} A B^{-1/2}) \succeq K_1^* f(X_1) K_1 + K_2^* f(X_2) K_2.$$

Conjugating by $B^{1/2}$ yields

$$\mathcal{P}_f(A, B) \succeq \lambda \mathcal{P}_f(A_1, B_1) + (1-\lambda)\mathcal{P}_f(A_2, B_2).$$

□

Corollary 620 (Joint Concavity of Tensor Powers)**.** *For $0 \le q \le 1$,*

$$(A, B) \longmapsto A^q \otimes B^{1-q}$$

is jointly operator concave.

Proof. Take

$$f(t) = t^q,$$

which is operator concave by Lemma 600. Apply Lemma 619 to

$$X = A \otimes I, \qquad Y = I \otimes B.$$

Since X and Y commute,

$$\begin{aligned} \mathcal{P}_q(X, Y) &= (I \otimes B^{1/2})(A \otimes B^{-1})^q (I \otimes B^{1/2}) \\ &= A^q \otimes B^{1-q}. \end{aligned}$$

Both X and Y depend linearly on A, B, so the claimed joint concavity follows. □

The same argument gives all exponents whose total is at most one.

Lemma 621 (Joint Concavity Below Total Exponent One)**.** *If*

$$q, r \ge 0, \qquad q + r \le 1,$$

then

$$(A, B) \longmapsto A^q \otimes B^r$$

is jointly operator concave.

Proof. The cases $q + r = 0$ and $q + r = 1$ are immediate from the preceding corollary. Suppose

$$s = q + r \in (0, 1).$$

Define

$$G(A, B) = A^{q/s} \otimes B^{r/s}.$$

The exponents in G sum to one, so G is jointly operator concave. Also

$$A^q \otimes B^r = G(A,B)^s.$$

Now $X \mapsto X^s$ is both operator monotone by Theorem 599 and operator concave by Lemma 600. Hence

$$\begin{aligned} G(\lambda z_1 + (1-\lambda)z_2)^s &\succeq \big(\lambda G(z_1) + (1-\lambda)G(z_2)\big)^s \\ &\succeq \lambda G(z_1)^s + (1-\lambda)G(z_2)^s. \end{aligned}$$

This is exactly the desired joint concavity. □

The last step in Lieb's theorem is easiest to understand by viewing a matrix as a vector.
For $K \in \mathbb{C}^{m\times n}$, define

$$\operatorname{vec}(K) = \sum_{i=1}^{m}\sum_{j=1}^{n} K_{ij}\, e_i \otimes e_j.$$

Thus the row index becomes the first tensor coordinate and the column index the second. Consequently, left multiplication acts on the first factor and right multiplication on the second:

$$\operatorname{vec}(XKY) = (X \otimes Y^T)\operatorname{vec}(K).$$

This can easily be checked on $K = e_i e_j^*$ which implies it for all matrices. It is clear that $\operatorname{vec}(A)^* \operatorname{vec}(B) = \operatorname{Tr}(A^*B)$, thus we can now deal in terms of inner products rather than traces.

Lemma 622 (Lieb's Concavity Theorem)**.** *Let $K \in \mathbb{C}^{m\times n}$ be fixed. If*

$$q, r \geq 0, \qquad q + r \leq 1,$$

then

$$F(A,B) = \operatorname{Tr}(K^* A^q K B^r)$$

is jointly concave for $A \succ 0$ and $B \succ 0$.

Proof. Vectorization turns the trace into a fixed quadratic form. Using the preceding formulas, we obtain

$$F(A,B) = \operatorname{vec}(K)^*\big((B^T)^r \otimes A^q\big)\operatorname{vec}(K).$$

The transpose map is linear and preserves positive definiteness. By Lemma 621,

$$(A,B) \longmapsto (B^T)^r \otimes A^q$$

is jointly operator concave. Applying the fixed positive linear functional

$$M \longmapsto \operatorname{vec}(K)^* M \operatorname{vec}(K)$$

preserves concavity. □

15.11.1 Joint Convexity of Matrix Relative Entropy

The first important consequence is obtained by differentiating Lieb's concavity at exponent zero.

Lemma 623 (Joint Convexity of Matrix Relative Entropy)**.** *For $A, B \succ 0$, define*

$$S(A\|B) = \operatorname{Tr}\big(A\log A - A\log B - A + B\big).$$

Then $S(A\|B)$ is jointly convex in (A,B).

Proof. For $t \in (0,1)$ define

$$J_t(A,B) = \frac{1}{t}\left(\operatorname{Tr}(B^t A^{1-t}) - \operatorname{Tr} A\right).$$

By Lemma 622, the first trace is jointly concave, while the second term is linear. Hence J_t is jointly concave.

As $t \downarrow 0$,

$$\frac{d}{dt} B^t \Big|_{t=0} = \log B, \qquad \frac{d}{dt} A^{1-t} \Big|_{t=0} = -A \log A.$$

Therefore

$$\begin{aligned} \lim_{t \downarrow 0} J_t(A,B) &= \operatorname{Tr}\big((\log B)A - A \log A\big) \\ &= \operatorname{Tr}\big(A \log B - A \log A\big). \end{aligned}$$

A finite pointwise limit of concave functions is concave. Hence

$$(A,B) \longmapsto \operatorname{Tr}(A \log B - A \log A)$$

is jointly concave. Negating it and adding the linear term

$$\operatorname{Tr}(B - A)$$

gives the claimed joint convexity. □

15.11.2 A Variational Formula for the Trace

The next identity is simply nonnegativity of relative entropy, rearranged as a variational principle.

Lemma 624 (Variational Formula for Trace)**.** *For $M \succ 0$,*

$$\operatorname{Tr} M = \max_{T \succ 0} \operatorname{Tr}\big(T \log M - T \log T + T\big).$$

Equality is attained at $T = M$.

Proof. The scalar inequality

$$a \log a - a \log h - a + h \geq 0 \qquad (a, h > 0)$$

is the tangent-line inequality for the convex function

$$x \longmapsto x \log x - x.$$

By the generalized Klein inequality,

$$S(T \| M) = \operatorname{Tr}\big(T \log T - T \log M - T + M\big) \geq 0.$$

Rearranging gives

$$\operatorname{Tr} M \geq \operatorname{Tr}\big(T \log M - T \log T + T\big).$$

Equality holds for $T = M$. □

15.11.3 Lieb's Trace-Exponential Theorem

Now the trace exponential becomes a partial maximization of a jointly concave function. This is the form of Lieb's theorem used throughout matrix concentration.

Theorem 625 (Lieb)**.** *For fixed Hermitian H, the map*

$$A \longmapsto \operatorname{Tr} \exp(H + \log A)$$

is concave on the positive-definite cone.

Proof. Apply Lemma 624 to

$$M = e^{H+\log A}.$$

Then

$$\begin{aligned}\operatorname{Tr} e^{H+\log A} &= \max_{T\succ 0} \operatorname{Tr}\big(T(H+\log A) - T\log T + T\big) \\ &= \max_{T\succ 0} \left\{\operatorname{Tr}(TH) - S(T\|A) + \operatorname{Tr} A\right\}.\end{aligned}$$

By Lemma 623, $S(T\|A)$ is jointly convex. Therefore the quantity inside braces is jointly concave in (T, A).

Partial maximization preserves concavity. Indeed, take A_1, A_2 and nearly optimal T_1, T_2. At

$$A = \lambda A_1 + (1-\lambda)A_2$$

the point

$$T = \lambda T_1 + (1-\lambda)T_2$$

is admissible. Joint concavity then gives

$$F(A,T) \geq \lambda F(A_1,T_1) + (1-\lambda)F(A_2,T_2).$$

Taking the approximations to the maxima arbitrarily close proves concavity. □

Corollary 626 (Lieb's Trace Inequality)**.** *For fixed Hermitian H and a random Hermitian matrix X for which the expectations exist,*

$$\mathbb{E}\operatorname{Tr} e^{H+X} \leq \operatorname{Tr}\exp\left(H + \log \mathbb{E} e^X\right).$$

Proof. Apply Jensen's inequality to the concave map from Theorem 625 with the random positive-definite matrix

$$A = e^X.$$

Then

$$\mathbb{E}\operatorname{Tr} e^{H+\log(e^X)} \leq \operatorname{Tr} e^{H+\log \mathbb{E} e^X}.$$

Since X is Hermitian,

$$\log(e^X) = X.$$

□

15.12 Lieb–Thirring Inequalities

For powers of two, the word-straightening argument from Golden–Thompson gives the inequality almost for free. For arbitrary real exponents, the main ingredient is the Cordes inequality, itself a one-line consequence of Löwner–Heinz. Exterior powers then upgrade an operator-norm inequality to all eigenvalues at once.

Theorem 627 (Lieb–Thirring Inequality)**.** *For $A, B \succ 0$ and every real $q \geq 1$,*

$$\operatorname{Tr}\big((BAB)^q\big) \leq \operatorname{Tr}(B^q A^q B^q).$$

The powers-of-two case. Suppose

$$q = 2^m.$$

Cyclicity gives

$$\operatorname{Tr}((BAB)^q) = \operatorname{Tr}((B^2A)^q).$$

Applying Lemma 602 to

$$U = B^2, \qquad V = A$$

yields

$$\operatorname{Tr}((B^2A)^q) \leq \operatorname{Tr}(B^{2q}A^q) = \operatorname{Tr}(B^qA^qB^q).$$

Lemma 628 (Cordes Inequality)**.** *If $C, D \succ 0$ and $0 \le s \le 1$, then*

$$\|C^s D^s\| \le \|CD\|^s.$$

Here $\|\cdot\|$ is the operator norm.

Proof. Let

$$c = \|CD\|.$$

Then

$$DC^2D = (CD)^*(CD) \preceq c^2 I.$$

Conjugating by D^{-1} gives

$$C^2 \preceq c^2 D^{-2}.$$

Apply Löwner–Heinz, Theorem 599, with exponent s:

$$C^{2s} \preceq c^{2s} D^{-2s}.$$

Conjugating by D^s,

$$D^s C^{2s} D^s \preceq c^{2s} I.$$

Finally,

$$\|C^s D^s\|^2 = \|D^s C^{2s} D^s\|,$$

so

$$\|C^s D^s\| \le c^s = \|CD\|^s.$$

□

Proof of Theorem 627 for general q. Set

$$s = \frac{1}{q}.$$

Apply Lemma 628 to

$$C = B^q, \qquad D = A^{q/2}.$$

Since

$$C^s = B, \qquad D^s = A^{1/2},$$

we obtain

$$\|BA^{1/2}\| \le \|B^q A^{q/2}\|^{1/q}.$$

Raise to the power $2q$. Since

$$\|BA^{1/2}\|^2 = \|BAB\|$$

and

$$\|B^q A^{q/2}\|^2 = \|B^q A^q B^q\|,$$

we get

$$\|(BAB)^q\| \le \|B^q A^q B^q\|.$$

This controls only the largest eigenvalue. Exterior powers apply the same inequality simultaneously to every product of the largest eigenvalues.

Apply the operator-norm inequality above to the positive matrices

$$\bigwedge^k A, \qquad \bigwedge^k B.$$

Exterior powers commute with products and functional calculus, so

$$\left\| \bigwedge^k (BAB)^q \right\| \leq \left\| \bigwedge^k (B^q A^q B^q) \right\|.$$

Since the largest eigenvalue of $\bigwedge^k X$ is the product of the largest k eigenvalues of X, we obtain, for every k,

$$\prod_{i=1}^{k} \lambda_i((BAB)^q) \leq \prod_{i=1}^{k} \lambda_i(B^q A^q B^q).$$

At $k = n$ there is equality, because

$$\begin{aligned} \det((BAB)^q) &= \det(B)^{2q} \det(A)^q \\ &= \det(B^q A^q B^q). \end{aligned}$$

Hence, after taking logarithms,

$$\left(\log \lambda_i((BAB)^q)\right)_{i=1}^{n}$$

is majorized by

$$\left(\log \lambda_i(B^q A^q B^q)\right)_{i=1}^{n}.$$

Applying Karamata to the convex function

$$x \longmapsto e^x$$

gives

$$\sum_i \lambda_i((BAB)^q) \leq \sum_i \lambda_i(B^q A^q B^q),$$

which is precisely

$$\operatorname{Tr}((BAB)^q) \leq \operatorname{Tr}(B^q A^q B^q).$$

□

The following "extended" inequality looks more complicated, but its proof is much easier. Once A is diagonal, the entire theorem reduces to the scalar weighted AM–GM inequality.

Theorem 629 (Extended Lieb–Thirring Inequality)**.** *For $A \succ 0$, Hermitian B, and $\alpha \in [0, 1]$,*

$$\operatorname{Tr}(BA^\alpha BA^{1-\alpha}) \leq \operatorname{Tr}(B^2 A).$$

Proof. Work in an eigenbasis of A:

$$A = \operatorname{diag}(\lambda_1, \ldots, \lambda_n), \qquad \lambda_i > 0.$$

Since B is Hermitian,

$$B_{ji} = \overline{B_{ij}}.$$

Therefore

$$\operatorname{Tr}(BA^\alpha BA^{1-\alpha}) = \sum_{i,j} |B_{ij}|^2 \lambda_i^{1-\alpha} \lambda_j^\alpha.$$

Symmetrizing the pair (i, j),

$$\operatorname{Tr}(BA^\alpha BA^{1-\alpha}) = \frac{1}{2} \sum_{i,j} |B_{ij}|^2 \left(\lambda_i^{1-\alpha} \lambda_j^\alpha + \lambda_j^{1-\alpha} \lambda_i^\alpha\right).$$

Weighted AM–GM gives

$$\lambda_i^{1-\alpha} \lambda_j^\alpha \leq (1-\alpha)\lambda_i + \alpha \lambda_j,$$

while swapping i, j gives

$$\lambda_j^{1-\alpha}\lambda_i^{\alpha} \le (1-\alpha)\lambda_j + \alpha\lambda_i.$$

Adding,

$$\lambda_i^{1-\alpha}\lambda_j^{\alpha} + \lambda_j^{1-\alpha}\lambda_i^{\alpha} \le \lambda_i + \lambda_j.$$

Hence

$$\begin{aligned}\operatorname{Tr}(BA^{\alpha}BA^{1-\alpha}) &\le \frac{1}{2}\sum_{i,j}|B_{ij}|^2(\lambda_i+\lambda_j)\\ &= \sum_{i,j}|B_{ij}|^2\lambda_i\\ &= \operatorname{Tr}(B^2A).\end{aligned}$$

□

15.13 Matrix Concentration

The scalar Chernoff/Bernstein method has two steps:

$$\Pr[S \ge t] \le e^{-\theta t}\mathbb{E}e^{\theta S},$$

and, for independent scalars,

$$\mathbb{E}e^{\theta\sum_i X_i} = \prod_i \mathbb{E}e^{\theta X_i}.$$

For matrices, the first step survives after replacing the scalar exponential by

$$\operatorname{Tr} e^{(\cdot)}.$$

The second step fails because noncommuting exponentials do not factor. Lieb's theorem is exactly the replacement: it lets us move out one random matrix at a time while putting its matrix MGF inside a logarithm.

After this one substitution, Chernoff and Bernstein are similar to their scalar proofs.

Let all matrices below be $d \times d$ self-adjoint matrices.

Lemma 630 (Matrix Laplace Transform)**.** *For independent self-adjoint random matrices $X_1, \dots, X_n$,*

$$\Pr\left[\lambda_{\max}\left(\sum_i X_i\right) \ge t\right] \le \inf_{\theta>0} e^{-\theta t}\operatorname{Tr}\exp\left(\sum_i \log\mathbb{E}e^{\theta X_i}\right).$$

Proof. Let

$$Y = \sum_i X_i.$$

Since

$$e^{\theta\lambda_{\max}(Y)} \le \operatorname{Tr} e^{\theta Y},$$

Markov's inequality gives

$$\Pr[\lambda_{\max}(Y) \ge t] \le e^{-\theta t}\mathbb{E}\operatorname{Tr} e^{\theta Y}.$$

It remains to control the trace MGF.

Condition on

$$X_1, \dots, X_{n-1}$$

and apply Corollary 626 with

$$H = \theta\sum_{i<n} X_i, \qquad X = \theta X_n.$$

Then

$$\mathbb{E}_{X_n} \operatorname{Tr} e^{\theta \sum_{i=1}^{n} X_i} \le \operatorname{Tr} \exp\left(\theta \sum_{i<n} X_i + \log \mathbb{E} e^{\theta X_n} \right).$$

Now condition on

$$X_1, \dots, X_{n-2}$$

and apply the same inequality to X_{n-1}, treating the deterministic matrix

$$\log \mathbb{E} e^{\theta X_n}$$

as part of H. Continuing backwards gives

$$\mathbb{E} \operatorname{Tr} e^{\theta \sum_i X_i} \le \operatorname{Tr} \exp\left(\sum_i \log \mathbb{E} e^{\theta X_i} \right).$$

Substitute this into the Markov bound and optimize over $\theta > 0$. □

Theorem 631 (Matrix Chernoff)**.** *Let X_i be independent and*

$$0 \preceq X_i \preceq RI.$$

Set

$$\mu_{\max} = \lambda_{\max}\left(\sum_i \mathbb{E} X_i \right), \qquad \mu_{\min} = \lambda_{\min}\left(\sum_i \mathbb{E} X_i \right).$$

Then

$$\Pr[\lambda_{\max}(\sum_i X_i) \ge (1+\delta)\mu_{\max}] \le d \left(\frac{e^{\delta}}{(1+\delta)^{1+\delta}} \right)^{\mu_{\max}/R}.$$

For $0 \le \delta < 1$,

$$\Pr[\lambda_{\min}(\sum_i X_i) \le (1-\delta)\mu_{\min}] \le d \left(\frac{e^{-\delta}}{(1-\delta)^{1-\delta}} \right)^{\mu_{\min}/R}.$$

Proof. For $0 \preceq X \preceq RI$, convexity of $e^{\theta x}$ on $[0, R]$ gives by functional calculus

$$e^{\theta X} \preceq I + \frac{e^{\theta R} - 1}{R} X.$$

Hence

$$\log \mathbb{E} e^{\theta X} \preceq \frac{e^{\theta R} - 1}{R} \mathbb{E} X.$$

Insert this into Lemma 630 and optimize exactly as in the scalar Chernoff proof. The lower tail uses $\theta < 0$. □

Theorem 632 (Matrix Bernstein)**.** *Let X_i be independent with*

$$\mathbb{E} X_i = 0, \qquad \lambda_{\max}(X_i) \le R.$$

Set

$$\sigma^2 = \left\| \sum_i \mathbb{E} X_i^2 \right\|.$$

Then

$$\Pr\left[\lambda_{\max}\left(\sum_i X_i \right) \ge t \right] \le d \exp\left(-\frac{t^2}{2(\sigma^2 + Rt/3)} \right).$$

If

$$\|X_i\| \le R,$$

then

$$\Pr\left[\left\|\sum_i X_i\right\| \ge t\right] \le 2d \exp\left(-\frac{t^2}{2(\sigma^2 + Rt/3)}\right).$$

Proof. The scalar inequality used for Freedman holds under functional calculus:

$$e^{\theta X} \preceq I + \theta X + g(\theta) X^2.$$

Taking expectations,

$$\mathbb{E}e^{\theta X} \preceq I + g(\theta)\mathbb{E}X^2 \preceq \exp(g(\theta)\mathbb{E}X^2).$$

Hence

$$\log \mathbb{E}e^{\theta X} \preceq g(\theta)\mathbb{E}X^2.$$

Apply Lemma 630:

$$\Pr[\cdots] \le d \exp(-\theta t + g(\theta)\sigma^2),$$

and optimize as before.

Apply the same bound to $-\sum_i X_i$ for the lower tail. □

Theorem 633 (Matrix Freedman)**.** *Let X_k be a self-adjoint matrix martingale difference sequence:*

$$\mathbb{E}[X_k \mid \mathcal{F}_{k-1}] = 0, \qquad \lambda_{\max}(X_k) \le R.$$

Define

$$Y_k = \sum_{i=1}^{k} X_i$$

and

$$W_k = \sum_{i=1}^{k} \mathbb{E}[X_i^2 \mid \mathcal{F}_{i-1}].$$

Then

$$\Pr\left[\exists k : \lambda_{\max}(Y_k) \ge t \text{ and } \lambda_{\max}(W_k) \le \sigma^2\right] \le d \exp\left(-\frac{t^2}{2(\sigma^2 + Rt/3)}\right).$$

Proof. Fix $\theta \ge 0$ and write

$$g(\theta) = \frac{e^{\theta R} - 1 - \theta R}{R^2}.$$

The scalar inequality

$$e^{\theta x} \le 1 + \theta x + g(\theta) x^2 \qquad (x \le R)$$

gives, by functional calculus,

$$e^{\theta X_k} \preceq I + \theta X_k + g(\theta) X_k^2.$$

Taking conditional expectation and using $\mathbb{E}[X_k \mid \mathcal{F}_{k-1}] = 0$,

$$\mathbb{E}[e^{\theta X_k} \mid \mathcal{F}_{k-1}] \preceq I + g(\theta) V_k \preceq e^{g(\theta) V_k},$$

where

$$V_k = \mathbb{E}[X_k^2 \mid \mathcal{F}_{k-1}].$$

Since the matrix logarithm is operator monotone,

$$\log \mathbb{E}[e^{\theta X_k} \mid \mathcal{F}_{k-1}] \preceq g(\theta) V_k.$$

Now define the scalar process

$$Z_k = \operatorname{tr} \exp\left(\theta Y_k - g(\theta) W_k\right).$$

We claim that Z_k is a positive supermartingale.

Indeed, conditioned on $\mathcal{F}_{k-1}$, the matrix V_k is fixed, and

$$W_k = W_{k-1} + V_k.$$

Lieb's trace inequality gives

$$\begin{aligned}
\mathbb{E}[Z_k \mid \mathcal{F}_{k-1}] &= \mathbb{E} \operatorname{tr} \exp\left(\theta Y_{k-1} - g(\theta) W_k + \theta X_k\right) \\
&\le \operatorname{tr} \exp\left(\theta Y_{k-1} - g(\theta) W_k + \log \mathbb{E}[e^{\theta X_k} \mid \mathcal{F}_{k-1}]\right) \\
&\le \operatorname{tr} \exp\left(\theta Y_{k-1} - g(\theta) W_k + g(\theta) V_k\right) \\
&= Z_{k-1}.
\end{aligned}$$

Also

$$Z_0 = \operatorname{tr} I = d.$$

Suppose that for some k,

$$\lambda_{\max}(Y_k) \ge t, \qquad \lambda_{\max}(W_k) \le \sigma^2.$$

Then

$$\begin{aligned}
\lambda_{\max}\left(\theta Y_k - g(\theta) W_k\right) &\ge \theta \lambda_{\max}(Y_k) - g(\theta) \lambda_{\max}(W_k) \\
&\ge \theta t - g(\theta) \sigma^2.
\end{aligned}$$

Hence

$$Z_k \ge \exp\left(\theta t - g(\theta) \sigma^2\right).$$

Ville's inequality for the positive supermartingale Z_k therefore gives

$$\Pr\left[\exists k : \lambda_{\max}(Y_k) \ge t,\ \lambda_{\max}(W_k) \le \sigma^2\right] \le d \exp\left(-\theta t + g(\theta) \sigma^2\right).$$

Finally, for

$$0 \le \theta < \frac{3}{R},$$

the scalar Bernstein estimate

$$g(\theta) \le \frac{\theta^2}{2(1 - \theta R/3)}$$

holds. Choose

$$\theta = \frac{t}{\sigma^2 + Rt/3}.$$

Then

$$1 - \frac{\theta R}{3} = \frac{\sigma^2}{\sigma^2 + Rt/3},$$

and so

$$-\theta t + g(\theta) \sigma^2 \le -\frac{t^2}{2(\sigma^2 + Rt/3)}.$$

This proves the result. □

Lemma 634 (Matrix Azuma MGF)**.** *Let X be a random self-adjoint matrix with*

$$\mathbb{E}X = 0, \qquad X^2 \preceq A^2,$$

where A is deterministic. Then for every self-adjoint H,

$$\mathbb{E}\operatorname{tr} e^{H+\theta X} \le \operatorname{tr} e^{H+2\theta^2 A^2}.$$

The same statement holds conditionally when H, A are measurable.

Proof. The reason for symmetrization is that our hypothesis controls X^2, rather than X itself. Introducing an independent random sign turns the exponential into an even function of X, which can be controlled by X^2.

We first use the standard trace symmetrization inequality

$$\mathbb{E}\operatorname{tr} e^{H+\theta X} \le \mathbb{E}_{X,\varepsilon} \operatorname{tr} e^{H+2\theta\varepsilon X},$$

where ε is an independent Rademacher sign.

For completeness, let X' be an independent copy of X. Since $\mathbb{E}X' = 0$, convexity of the trace exponential gives

$$\mathbb{E}_X \operatorname{tr} e^{H+\theta X} \le \mathbb{E}_{X,X'} \operatorname{tr} e^{H+\theta(X-X')}.$$

The matrix $X - X'$ is symmetric in distribution, so we may multiply it by an independent Rademacher sign ε. Split

$$H + \theta\varepsilon(X - X') = \left(\frac{H}{2} + \theta\varepsilon X\right) + \left(\frac{H}{2} - \theta\varepsilon X'\right).$$

Golden–Thompson and Cauchy–Schwarz for the trace give

$$\begin{aligned}
&\operatorname{tr} e^{H+\theta\varepsilon(X-X')} \\
&\quad\le \operatorname{tr}\left(e^{H/2+\theta\varepsilon X} e^{H/2-\theta\varepsilon X'}\right) \\
&\quad\le \sqrt{\operatorname{tr} e^{H+2\theta\varepsilon X} \operatorname{tr} e^{H-2\theta\varepsilon X'}}.
\end{aligned}$$

Taking expectation and applying scalar Cauchy–Schwarz, the two factors have the same distribution, so

$$\mathbb{E}\operatorname{tr} e^{H+\theta X} \le \mathbb{E}_{X,\varepsilon} \operatorname{tr} e^{H+2\theta\varepsilon X}.$$

Now condition on X. Lieb's inequality gives

$$\mathbb{E}_\varepsilon \operatorname{tr} e^{H+2\theta\varepsilon X} \le \operatorname{tr}\exp\left(H + \log \mathbb{E}_\varepsilon e^{2\theta\varepsilon X}\right).$$

But

$$\mathbb{E}_\varepsilon e^{2\theta\varepsilon X} = \cosh(2\theta X).$$

The scalar inequality

$$\cosh z \le e^{z^2/2}$$

gives by functional calculus

$$\cosh(2\theta X) \preceq e^{2\theta^2 X^2}.$$

Taking logarithms,

$$\log\cosh(2\theta X) \preceq 2\theta^2 X^2 \preceq 2\theta^2 A^2.$$

Therefore

$$\mathbb{E}_\varepsilon \operatorname{tr} e^{H+2\theta\varepsilon X} \le \operatorname{tr} e^{H+2\theta^2 A^2}.$$

□

Theorem 635 (Matrix Azuma–Hoeffding)**.** *Let X_k be self-adjoint martingale differences and suppose*

$$X_k^2 \preceq A_k^2$$

for deterministic self-adjoint matrices A_k. Set

$$\sigma^2 = \left\| \sum_k A_k^2 \right\|.$$

Then

$$\Pr\left[\lambda_{\max}\left(\sum_k X_k \right) \geq t \right] \leq d \exp\left(-\frac{t^2}{8\sigma^2} \right).$$

Consequently,

$$\Pr\left[\left\| \sum_k X_k \right\| \geq t \right] \leq 2d \exp\left(-\frac{t^2}{8\sigma^2} \right).$$

Proof. The matrix Laplace bound gives

$$\Pr\left[\lambda_{\max}\left(\sum_k X_k \right) \geq t \right] \leq e^{-\theta t} \mathbb{E} \operatorname{tr} e^{\theta \sum_k X_k}.$$

Apply Lemma 634 conditionally to the last martingale difference, then the next-to-last, and so on. This gives

$$\mathbb{E} \operatorname{tr} e^{\theta \sum_k X_k} \leq \operatorname{tr} \exp\left(2\theta^2 \sum_k A_k^2 \right).$$

Since

$$\sum_k A_k^2 \preceq \sigma^2 I,$$

we obtain

$$\mathbb{E} \operatorname{tr} e^{\theta \sum_k X_k} \leq d e^{2\theta^2 \sigma^2}.$$

Hence

$$\Pr\left[\lambda_{\max}\left(\sum_k X_k \right) \geq t \right] \leq d e^{-\theta t + 2\theta^2 \sigma^2}.$$

The exponent is minimized at

$$\theta = \frac{t}{4\sigma^2},$$

giving

$$d e^{-t^2/(8\sigma^2)}.$$

Apply the same result to $-X_k$ and take a union bound to obtain the spectral-norm estimate. □

Theorem 636 (Matrix McDiarmid)**.** *Let $Z_1, \dots, Z_n$ be independent and let*

$$H(Z_1, \dots, Z_n)$$

be self-adjoint. Suppose that whenever z, z' differ only in coordinate i,

$$\left(H(z) - H(z')\right)^2 \preceq A_i^2$$

for a deterministic self-adjoint matrix A_i. Set

$$\sigma^2 = \left\| \sum_i A_i^2 \right\|.$$

Then

$$\Pr\left[\lambda_{\max}\big(H(Z) - \mathbb{E}H(Z)\big) \geq t\right] \leq d e^{-t^2/(8\sigma^2)}.$$

Proof. Take the Doob martingale

$$M_i = \mathbb{E}[H(Z) \mid Z_1, \ldots, Z_i], \qquad X_i = M_i - M_{i-1}.$$

Then

$$\mathbb{E}[X_i \mid \mathcal{F}_{i-1}] = 0$$

and

$$\sum_i X_i = H(Z) - \mathbb{E}H(Z).$$

We only need to show

$$X_i^2 \preceq A_i^2.$$

Condition on $Z_1, \ldots, Z_i$. Let Z_i' be an independent copy of Z_i, and use the same random future

$$Z_{i+1}, \ldots, Z_n$$

on both sides. Then

$$X_i = \mathbb{E}\left[H(Z_1, \ldots, Z_i, \ldots, Z_n) - H(Z_1, \ldots, Z_i', \ldots, Z_n) \mid Z_1, \ldots, Z_i\right].$$

The map

$$X \longmapsto X^2$$

is operator convex, so Jensen gives

$$\begin{aligned} X_i^2 &\preceq \mathbb{E}\left[\left(H(Z_1, \ldots, Z_i, \ldots, Z_n) - H(Z_1, \ldots, Z_i', \ldots, Z_n)\right)^2 \mid Z_1, \ldots, Z_i\right] \\ &\preceq A_i^2. \end{aligned}$$

Thus the Doob differences satisfy the hypotheses of Theorem 635, which proves the claim. □

16 Convex Optimization

A gradient predicts the change in an objective under a small displacement. To choose a step, we must also control the error in this prediction. For a differentiable convex function f, the error is its *Bregman divergence*

$$D_f(y, x) = f(y) - f(x) - \langle \nabla f(x), y - x \rangle.$$

The second argument is the point where we linearize; the first is the point where we evaluate the error. Convexity says exactly that $D_f(y, x) \geq 0$. Unlike a distance, this error need not be symmetric or satisfy a triangle inequality.

The identity $f(y) = f(x) + \langle \nabla f(x), y - x \rangle + D_f(y, x)$ gives a way to design an iteration: replace $D_f(y, x)$ by an upper bound that is easier to minimize, then minimize the resulting model. Quadratic bounds lead to gradient descent and preconditioning. Bounds supplied by a more general reference function lead to Bregman proximal steps. The analysis will use the same reference function to measure progress toward a solution.

Unless specified otherwise, optimization is over $\mathbb{R}^n$; a minimizer is assumed to exist whenever one is used. Constraint sets are convex. Logarithms are natural, $\|v\|_M^2 = v^\top M v$ for $M \succeq 0$, and vector powers and products are coordinatewise when indicated. We count arithmetic operations, treating scalar logarithms, exponentials, and elementary random draws as unit-cost operations. Dependence polynomial in $1/\varepsilon$ is called *low accuracy*; logarithmic dependence is called *high accuracy*.

Lemma 637 (Integrated curvature)**.** *If f is C^2 along a segment and $d = y - x$, then*

$$D_f(y, x) = \int_0^1 (1 - t) d^\top \nabla^2 f(x + td) d \, dt.$$

Proof. For $\varphi(t) = f(x+td)$, integrate φ'' twice: $\varphi(1)-\varphi(0)-\varphi'(0) = \int_0^1 \int_0^s \varphi''(t) \, dt \, ds = \int_0^1 (1-t)\varphi''(t) \, dt$. □

In particular, a bound on the Hessian throughout the segment gives a bound on the linearization error. A Hessian bound only at the starting point does not suffice.

16.1 Linear Systems and Preconditioned Richardson Iteration

For $A \succ 0$, solving $Ax = b$ is equivalent to minimizing $f(x) = \frac{1}{2} x^\top A x - b^\top x$. With $g = Ax - b$,

$$f(x + \Delta) - f(x) = \langle g, \Delta \rangle + \frac{1}{2} \|\Delta\|_A^2, \qquad f(x) - f(x^*) = \frac{1}{2} \|x - x^*\|_A^2.$$

Minimizing this expression gives $\Delta = -A^{-1} g$, which requires the solve we are trying to perform. Instead, suppose a positive definite matrix B is easier to solve in and satisfies $\mu B \preceq A \preceq LB$, where $0 < \mu \leq L$. Replacing the quadratic term by its upper bound $L\|\Delta\|_B^2/2$ gives the step

$$\Delta = -\frac{1}{L} B^{-1} (Ax - b).$$

Because we minimize an upper bound that agrees with f at the current point, this step cannot increase f. The lower comparison will quantify how much progress it makes: it prevents the model from overestimating the curvature by more than a factor L/μ. The iteration is called Richardson iteration when $B = I$ and preconditioned Richardson iteration for general B.

Algorithm 81 Preconditioned Richardson iteration

Input: $\mu B \preceq A \preceq LB$, $A, B \succ 0$, right-hand side b, accuracy $0 < \varepsilon < 1$
Output: $\|x - A^{-1} b\|_A \leq \varepsilon \|A^{-1} b\|_A$

```
function Richardson(A, B, μ, L, b, ε)
    x ← 0, T ← ⌈(L/μ) log(1/ε)⌉
    for t = 1, ..., T do
        Solve Bz = Ax − b
        x ← x − z/L
    return x
```

Theorem 638 (Preconditioned contraction)**.** *One Richardson step contracts the solution error in the A-norm, and the residual in the B^{-1}-norm, by a factor at most $1 - \mu/L$. Consequently Algorithm 81 has its stated guarantee and also $\|Ax - b\|_{B^{-1}} \leq \varepsilon \|b\|_{B^{-1}}$.*

Proof. Change coordinates so that the model's quadratic form becomes Euclidean: put $C = B^{-1/2} A B^{-1/2}$ and $e = B^{1/2}(x - x^*)$. The matrix comparison becomes $\mu I \preceq C \preceq LI$, and the update becomes $e^+ = (I - C/L)e$. In an orthonormal eigenbasis of C, the component with eigenvalue λ is multiplied by $1 - \lambda/L \in [0, 1 - \mu/L]$. The same factor therefore contracts $\|e\|_C = \|x - x^*\|_A$ and $\|Ce\|_2 = \|Ax - b\|_{B^{-1}}$. Iterating and using $(1 - \mu/L)^T \leq e^{-\mu T/L}$ proves both bounds. □

The square root of B is only used in the proof; each iteration needs one application of A and one solve in B. With a constant-factor preconditioner, high accuracy therefore costs only $O(\log(1/\varepsilon))$ applications of A and solves in B. If $A, B \succeq 0$ have the same kernel and b is orthogonal to it, the same algorithm and proof apply on its orthogonal complement, with every solve taken in that subspace.

16.2 Smooth Gradient Descent

For a nonlinear convex function, a uniform quadratic bound on the linearization error plays the role of the spectral comparison above. Write $q(x) = \|x\|_2^2/2$, so $D_q(y,x) = \|y-x\|_2^2/2$. The function is *L-smooth* if $D_f(y,x) \le L D_q(y,x)$ and *μ-strongly convex* if $D_f(y,x) \ge \mu D_q(y,x)$ for all x, y. For $L > 0$, minimizing the smoothness upper bound gives the gradient step $x^+ = x - \nabla f(x)/L$ and the decrease $f(x^+) \le f(x) - \|\nabla f(x)\|_2^2/(2L)$. To turn this decrease into a convergence rate, we need a quantity that can pay for the objective gap over many iterations. Convexity gives $f(x) - f(x^*) \le \langle \nabla f(x), x - x^* \rangle$; expanding the squared distance after the update produces exactly this inner product. The resulting distance differences telescope.

Algorithm 82 Smooth gradient descent

Input: Convex L-smooth f with $L > 0$, initial point x_0, $T \ge 1$

for $t = 0, \dots, T-1$ **do**
  $x_{t+1} \leftarrow x_t - \nabla f(x_t)/L$
return x_T

Theorem 639 (Smooth gradient descent)**.** *For every minimizer x^*, $f(x_T) - f(x^*) \le L\|x_0 - x^*\|_2^2/(2T)$. If f is also μ-strongly convex with $\mu > 0$, then $f(x_{t+1}) - f(x^*) \le (1 - \mu/L)(f(x_t) - f(x^*))$.*

Proof. Put $g = \nabla f(x)$. Add convexity at x to the descent bound:

$$f(x^+) - f(x^*) \le \langle g, x - x^* \rangle - \frac{\|g\|_2^2}{2L} = \frac{L}{2}\big(\|x - x^*\|_2^2 - \|x^+ - x^*\|_2^2\big).$$

The equality follows by expanding $x^+ = x - g/L$. Summing telescopes the distances. Since objective values decrease, each of the T gaps on the left is at least the final gap, proving the first claim.

With strong convexity, we can instead compare the gradient directly with the objective gap. The quadratic lower bound at x gives

$$f(x^*) \ge f(x) + \langle g, x^* - x \rangle + \frac{\mu}{2}\|x^* - x\|_2^2 \ge f(x) - \frac{\|g\|_2^2}{2\mu}.$$

The second inequality minimizes the quadratic expression over all possible displacements $x^* - x$. It implies $\|g\|_2^2 \ge 2\mu(f(x) - f(x^*))$, so the guaranteed decrease removes at least a μ/L fraction of the current gap. □

Lemma 640 (Equivalent Euclidean curvature conditions)**.** *For differentiable convex $f : \mathbb{R}^n \to \mathbb{R}$ and $L > 0$, smoothness is equivalent to convexity of $Lq - f$ and to $\|\nabla f(y) - \nabla f(x)\|_2 \le L\|y - x\|_2$. Strong convexity is equivalent to convexity of $f - \mu q$ and to $\langle \nabla f(y) - \nabla f(x), y - x \rangle \ge \mu\|y - x\|_2^2$. For C^2 functions these are respectively $\nabla^2 f \preceq LI$ and $\nabla^2 f \succeq \mu I$. Moreover, an L-smooth convex function satisfies*

$$\langle \nabla f(x) - \nabla f(y), x - y \rangle \ge \frac{1}{L}\|\nabla f(x) - \nabla f(y)\|_2^2.$$

Proof. Since $D_{Lq-f} = L D_q - D_f$ and $D_{f-\mu q} = D_f - \mu D_q$, the Bregman bounds are equivalent to the stated convexity conditions. For a C^2 function, convexity of these shifted functions is exactly the stated Hessian comparison.

For any differentiable function k, convexity is equivalent to monotonicity of its gradient. Convexity gives $\langle \nabla k(y) - \nabla k(x), y - x \rangle \ge 0$ by adding the supporting inequalities at x and y. Conversely, with $d = y - x$, gradient monotonicity gives $\langle \nabla k(x + td) - \nabla k(x), d \rangle \ge 0$ for $t > 0$; integrating gives $D_k(y,x) \ge 0$. Taking $k = f - \mu q$ proves the strong-convexity gradient condition.

An L-Lipschitz gradient gives the smoothness bound by integrating $\langle \nabla f(x + td) - \nabla f(x), d \rangle \le Lt\|d\|_2^2$.

For the converse, subtract a linear function so that y becomes an optimum. Namely, let $g = \nabla f(x) - \nabla f(y)$ and $\phi(z) = f(z) - \langle \nabla f(y), z \rangle$. This convex function is minimized at y, and its gradient at x is g. Its descent bound gives $\phi(y) \le \phi(x - g/L) \le \phi(x) - \|g\|_2^2/(2L)$, hence $D_f(x,y) \ge \|g\|_2^2/(2L)$. Swap x, y and add, using $D_f(x,y) + D_f(y,x) = \langle g, x - y \rangle$. This proves the displayed inequality; Cauchy–Schwarz then gives gradient Lipschitzness, with the case $g = 0$ immediate. □

16.3 Bregman Proximal Steps and Relative Curvature

A fixed quadratic model must accommodate the largest curvature that occurs anywhere in the domain. If the curvature varies substantially, this can make the model unnecessarily restrictive at most points. We can instead choose a convex reference function h whose curvature varies with the objective's, and use $D_h(y, x)$ as the movement penalty. The choice must meet two requirements: D_h should bound the relevant linearization error, and minimizing a linear function plus D_h should be tractable.

For a convex set K, define

$$\operatorname{Prox}_x^h(v) \in \arg\min_{z \in K} \{\langle v, z\rangle + D_h(z, x)\}.$$

The base point x determines the movement penalty, and v determines the linear objective. We assume the required minimizers exist and h is differentiable at the base points and proximal outputs; comparators may lie on the boundary if h extends continuously there. Without constraints, first-order optimality reads $\nabla h(x^+) = \nabla h(x) - v$. Thus the update subtracts v in gradient coordinates. A quadratic h gives a preconditioned additive step; entropy will give a multiplicative step.

For a quadratic reference, expanding a squared distance was enough to analyze gradient descent. The following identity supplies the same cancellation for a general h.

Lemma 641 (Three-point identity and proximal inequality)**.** *For differentiable convex h,*

$$\langle \nabla h(y) - \nabla h(x), u - y\rangle = D_h(u, x) - D_h(u, y) - D_h(y, x).$$

Consequently, if $x^+ = \operatorname{Prox}_x^h(\eta g)$, then every $u \in K$ satisfies

$$\eta\langle g, x^+ - u\rangle \le D_h(u, x) - D_h(u, x^+) - D_h(x^+, x).$$

Proof. Expand the three divergences: their function values cancel, leaving $\langle \nabla h(y) - \nabla h(x), u - y\rangle$. Optimality of x^+ in the feasible direction $u - x^+$ gives $\langle \eta g + \nabla h(x^+) - \nabla h(x), u - x^+\rangle \ge 0$. Rearrange and apply the identity with $y = x^+$. □

The proximal inequality can be read as $\eta\langle g, x^+ - u\rangle + D_h(x^+, x) \le D_h(u, x) - D_h(u, x^+)$. The decrease in the potential $D_h(u, x)$ pays for both the new point's linear loss relative to u and its movement from x. Retaining the movement term is essential: it will pay for the error in linearizing the objective, or for the error in evaluating a loss at a different point.

16.3.1 Relative Descent

We say that f is *L-smooth relative to h* if $D_f(y, x) \le L D_h(y, x)$, and *$\mu$-strongly convex relative to h* if $D_f(y, x) \ge \mu D_h(y, x)$. Here $L > 0$ and $0 \le \mu \le L$. Equivalently, $Lh - f$ and $f - \mu h$ are convex. For C^2 functions on an open convex domain, this is equivalent to $\mu\nabla^2 h \preceq \nabla^2 f \preceq L\nabla^2 h$. On an affine constraint set the Hessian comparison is restricted to feasible directions. The factors μ, L remain uniform even though both Hessians may vary with position. The upper comparison gives the model $f(x) + \langle \nabla f(x), z - x\rangle + L D_h(z, x)$. Dividing by L and removing terms independent of z shows that its minimizer is $\operatorname{Prox}_x^h(\nabla f(x)/L)$. The coefficient L also makes the proof work: the model's error term $L D_h(x^+, x)$ cancels the movement term in the proximal inequality.

Algorithm 83 Relative gradient descent

Input: Convex f that is L-smooth relative to h on K, point x_0, $T \ge 1$

for $t = 0, \dots, T-1$ **do**

  $x_{t+1} \leftarrow \operatorname{Prox}_{x_t}^h(\nabla f(x_t)/L)$

return x_T

Theorem 642 (Relative descent)**.** *If f is L-smooth and μ-strongly convex relative to h, then one step satisfies*

$$f(x^+) - f(x^*) + LD_h(x^*, x^+) \le (L-\mu)D_h(x^*, x).$$

Thus $D_h(x^*, x^+) \le (1-\mu/L)D_h(x^*, x)$. For $\mu > 0$ and $T \ge 1$, this implies $f(x_T) - f(x^*) \le L(1-\mu/L)^T D_h(x^*, x_0)$. For $\mu = 0$, the last iterate after T steps satisfies $f(x_T) - f(x^*) \le LD_h(x^*, x_0)/T$.

Proof. Put $g = \nabla f(x)$. Split the model's linear term at x^*, so that the proximal inequality controls the part from x^* to x^+:

$$\begin{aligned} f(x^+) &\le f(x) + \langle g, x^* - x\rangle + \langle g, x^+ - x^*\rangle + LD_h(x^+, x) \\ &\le f(x) + \langle g, x^* - x\rangle + LD_h(x^*, x) - LD_h(x^*, x^+) \\ &\le f(x^*) + (L-\mu)D_h(x^*, x) - LD_h(x^*, x^+). \end{aligned}$$

The movement terms cancel in the second line; relative strong convexity gives the third. Dropping the nonnegative objective gap proves contraction of the divergence. To bound the gap at step T, retain it on that step and apply the contraction to the preceding $T-1$ steps.

For $\mu = 0$, sum the one-step inequality to obtain $\sum_{t=1}^{T}(f(x_t) - f(x^*)) \le LD_h(x^*, x_0)$. The model is minimized at x^+ and agrees with f at x, so $f(x^+) \le f(x)$; each gap in the sum is therefore at least the final gap. This proves the last-iterate bound. □

16.3.2 Where Norms Enter

For a norm $\|\cdot\|$, define $\|g\|_* = \max_{\|u\|\le 1}\langle g, u\rangle$. Then $|\langle g, u\rangle| \le \|g\|_*\|u\|$. The norm measures displacements and its dual measures linear losses. For ℓ_p, the dual is ℓ_q with $1/p + 1/q = 1$, including ℓ_1/ℓ_∞; this is Hölder's inequality with equality in the direction proportional to $\operatorname{sgn}(g)|g|^{q-1}$ for $1 < p < \infty$.

A Bregman divergence supplies the potential differences in the proximal inequality, while norm duality bounds the linear effect of a displacement. These are separate roles, so choosing a norm does not determine the reference function, or conversely. Norm steepest descent penalizes $\|y-x\|^2/2$, whereas a Bregman step penalizes $D_h(y, x)$. Even for $h(x) = \|x\|^2/2$, these need not agree: a general squared norm does not expand as an inner-product quadratic. For example, with $h(x) = \|x\|_4^2/2$, we have $D_h(e_2, e_1) = 1$ but $\|e_2 - e_1\|_4^2/2 = 1/\sqrt{2}$. To connect the two roles, mirror descent will use the comparison $D_h(y, x) \ge (\sigma/2)\|y - x\|^2$, called σ-strong convexity of h in that norm. It allows the telescoping Bregman potential to pay for movement measured by the norm.

16.4 Mirror Descent, Entropy, and Multiplicative Weights

Suppose we choose $x_t \in K$, then receive a loss vector g_t and incur loss $\langle g_t, x_t\rangle$. The loss can change every round, so there need not be a single objective to decrease. Instead, we compare our total loss with that of a fixed feasible point u. Their difference, $\sum_t \langle g_t, x_t - u\rangle$, is the *regret* against u.

After seeing g_t, take the proximal step $x_{t+1} = \operatorname{Prox}_{x_t}^h(\eta g_t)$. The proximal inequality bounds $\langle g_t, x_{t+1} - u\rangle$, but our incurred loss uses x_t. To account for this difference, write $\langle g_t, x_t - u\rangle = \langle g_t, x_{t+1} - u\rangle + \langle g_t, x_t - x_{t+1}\rangle$. The extra term is the cost of acting before seeing the loss. Its size is at most $\|g_t\|_*\|x_t - x_{t+1}\|$. If h is strongly convex in the displacement norm, the movement divergence pays a quadratic cost in $\|x_t - x_{t+1}\|$. Completing the square bounds what remains by a term proportional to $\eta\|g_t\|_*^2$.

Algorithm 84 Mirror descent

Input: Reference function h, initial $x_0 \in K$, step size $\eta > 0$, $T \ge 1$

for $t = 0, \dots, T-1$ **do**
  Play x_t and receive g_t
  $x_{t+1} \leftarrow \operatorname{Prox}_{x_t}^h(\eta g_t)$

Theorem 643 (Mirror descent)**.** *If $D_h(y,x) \geq (\sigma/2)\|y-x\|^2$ for $\sigma > 0$, then*

$$\sum_{t<T} \langle g_t, x_t - u\rangle \leq \frac{D_h(u,x_0)}{\eta} + \frac{\eta}{2\sigma}\sum_{t<T}\|g_t\|_*^2 \qquad (u \in K).$$

Proof. The proximal inequality bounds the part involving $x_{t+1} - u$. For the remaining part, duality and $ab \leq \eta a^2/(2\sigma) + \sigma b^2/(2\eta)$ give

$$\langle g_t, x_t - x_{t+1}\rangle \leq \frac{\eta}{2\sigma}\|g_t\|_*^2 + \frac{1}{\eta}D_h(x_{t+1}, x_t).$$

This cancels the negative movement divergence, leaving $\langle g_t, x_t - u\rangle \leq [D_h(u,x_t) - D_h(u,x_{t+1})]/\eta + \eta\|g_t\|_*^2/(2\sigma)$. Sum and discard the final nonnegative divergence. □

Corollary 644 (Subgradient descent)**.** *Let f be convex, let $g_t \in \partial f(x_t)$ satisfy $\|g_t\|_2 \leq G$, and set $x_{t+1} = x_t - \eta g_t$. For $\bar{x} = T^{-1}\sum_{t<T} x_t$ and $R = \|x_0 - x^*\|_2$, $f(\bar{x}) - f(x^*) \leq R^2/(2\eta T) + \eta G^2/2$. With $R, G > 0$, the choice $\eta = R/(G\sqrt{T})$ gives $RG/\sqrt{T}$.*

Proof. Take $h(x) = \|x\|_2^2/2$ in mirror descent, use $f(x_t) - f(x^*) \leq \langle g_t, x_t - x^*\rangle$, and average by convexity. □

This guarantee concerns the average of $x_0, \ldots, x_{T-1}$, the points at which the subgradients were evaluated. Unlike smooth gradient steps, subgradient steps need not decrease the objective; averaging turns a bound on the sum of gaps into a bound at one feasible point. For a G-Lipschitz convex function on all of $\mathbb{R}^n$, every subgradient has norm at most G: apply the subgradient inequality and Lipschitzness in its own unit direction.

16.4.1 Entropy on the Simplex

Suppose each coordinate of g_t is bounded in magnitude. Measuring the loss in ℓ_∞ uses this bound directly, so the dual displacement norm in mirror descent should be ℓ_1. We therefore want a reference function strongly convex in ℓ_1, with a small initial divergence. On the simplex $\Delta_n = \{p \geq 0 : \mathbf{1}^\top p = 1\}$, the mass constraint suggests how to obtain the required curvature. For positive p, weighted Cauchy–Schwarz gives

$$\|z\|_1^2 = \left(\sum_i \sqrt{p_i}\frac{|z_i|}{\sqrt{p_i}}\right)^2 \leq \left(\sum_i p_i\right)\left(\sum_i \frac{z_i^2}{p_i}\right) = \sum_i \frac{z_i^2}{p_i}.$$

The useful cancellation is $\sum_i p_i = 1$: it removes any dimension factor from the bound. Thus a reference with Hessian $\mathrm{diag}(1/p_i)$ has the curvature we need. Integrating twice gives negative entropy $h(p) = \sum_i p_i \log p_i$, up to affine terms that do not affect its Bregman divergence. Applying the integrated curvature formula along the segment from p to q gives $D_h(q,p) \geq \|q-p\|_1^2/2$. The divergence itself is

$$D_h(q,p) = \sum_i q_i \log(q_i/p_i), \qquad D_h(q, \mathbf{1}/n) = \log n + \sum_i q_i \log q_i \leq \log n.$$

The first identity uses $\sum_i (q_i - p_i) = 0$ to cancel the affine terms. We interpret $0 \log 0$ as zero and extend the bounds to boundary comparators q by continuity, keeping the base point p positive. Entropy therefore meets both requirements: its strong-convexity constant is one, and the uniform starting point has divergence at most $\log n$ from every comparator.

Its proximal step is also explicit. To minimize $\langle v, q\rangle + \beta D_h(q,p)$ for $\beta > 0$, introduce a multiplier for $\sum_i q_i = 1$. The coordinate equations are $v_i + \beta\log(q_i/p_i) + \lambda = 0$, absorbing the constant derivative term into λ. Hence every coordinate is multiplied by $e^{-v_i/\beta}$, then a common normalization restores total mass one: $q_i = p_i e^{-v_i/\beta}/\sum_j p_j e^{-v_j/\beta}$. This positive vector satisfies the first-order condition; convexity therefore proves its optimality over the entire simplex.

Algorithm 85 Entropy proximal step and multiplicative weights

function EntropyStep(p, v, β) ▷ $p_i > 0$, $\beta > 0$
  $s_i \leftarrow \log p_i - v_i/\beta$ for every i
  $s \leftarrow s - (\max_i s_i)\mathbf{1}$
  return $\exp(s)/(\mathbf{1}^\top \exp(s))$

procedure MultiplicativeWeights(n, η, T)
  $p \leftarrow \mathbf{1}/n$
  for $t = 0, \dots, T-1$ **do**
    Play p and receive g_t
    $p \leftarrow$ EntropyStep($p, \eta g_t, 1$)

Subtracting the largest log-weight leaves the normalized vector unchanged and keeps every exponent nonpositive. Positive initial weights stay positive in exact arithmetic. The resulting regret is at most $\log n/\eta + (\eta/2)\sum_{t<T} \|g_t\|_\infty^2$. If $n \geq 2$ and $\|g_t\|_\infty \leq G$ with $G > 0$, choosing $\eta = \sqrt{2\log n}/(G\sqrt{T})$ balances the two terms and gives regret at most $G\sqrt{2T\log n}$.

16.4.2 Soft Maximum

Entropy also explains a standard smooth approximation to the maximum. The identity $\max_i z_i = \max_{p\in\Delta_n} \langle p, z\rangle$ expresses a maximum as a linear optimization problem, whose optimizer can jump between vertices when the largest coordinate changes. Subtracting negative entropy rewards spreading the mass and makes the optimizer unique. For $\lambda > 0$, define

$$s_\lambda(z) = \max_{p\in\Delta_n} \left\{ \langle p, z\rangle - \frac{1}{\lambda} h(p) \right\} = \frac{1}{\lambda} \log \sum_i e^{\lambda z_i}.$$

To verify the last equality, put $p_i^* = e^{\lambda z_i}/\sum_j e^{\lambda z_j}$. Expansion gives $\langle p, z\rangle - h(p)/\lambda = \lambda^{-1} \log \sum_i e^{\lambda z_i} - D_h(p, p^*)/\lambda$, whose maximum is attained at $p = p^*$. Since $0 \leq -h(p) \leq \log n$, the regularization changes the maximum by at most $\log n/\lambda$: $\max_i z_i \leq s_\lambda(z) \leq \max_i z_i + \log n/\lambda$.

Differentiation gives $\nabla s_\lambda(z) = p^*$ and $\nabla^2 s_\lambda(z) = \lambda(\operatorname{diag}(p^*) - p^*(p^*)^\top)$. Thus the gradient is exactly the entropy-regularized maximizing distribution. Its curvature has a probabilistic form: for any direction u, $u^\top \nabla^2 s_\lambda(z) u = \lambda \operatorname{Var}_{i\sim p^*}(u_i) \leq \lambda \|u\|_\infty^2$. Applying the construction to $(z, -z)$ approximates $\|z\|_\infty$ within $\log(2n)/\lambda$ and has the same directional curvature bound. Increasing λ reduces the approximation error but increases the curvature bound, and hence reduces the step size permitted by smoothness. The desired approximation accuracy determines this tradeoff.

16.5 Matrix Multiplicative Weights

Scalar multiplicative weights mixes among finitely many choices. To mix among all unit directions in $\mathbb{R}^n$, suppose a symmetric matrix G assigns loss $u^\top G u$ to direction u. A distribution assigning probability p_i to direction u_i has expected loss $\sum_i p_i u_i^\top G u_i = \operatorname{Tr}(GX)$, where $X = \sum_i p_i u_i u_i^\top$. The distribution therefore matters only through the matrix X.

Such a matrix satisfies $X \succeq 0$ and $\operatorname{Tr} X = 1$. Conversely, the spectral decomposition writes every matrix with these properties as a mixture of unit directions. These are the *density matrices*, whose convex feasible set we denote by $\mathcal{D}_n$. Because the loss is linear in X, a regret bound against every fixed unit direction also bounds regret against every density matrix.

The matrix analogues of the scalar ℓ_1 and ℓ_∞ norms are the trace and operator norms. They are dual under the trace inner product: diagonalizing a symmetric H gives $|\operatorname{Tr}(GH)| \leq \|G\|_{\mathrm{op}} \sum_i |\lambda_i(H)|$; equality in the dual characterization is attained by $G = \operatorname{sgn}(H)$.

The eigenvalues of a density matrix form a probability vector, which suggests applying negative entropy to them: $h(X) = \operatorname{Tr}(X \log X)$, with $0 \log 0 = 0$. To obtain the proximal update, we need its convexity and

derivative. Both follow from the scalar tangent inequality, even though two matrices may have different eigenbases. Write $X = \sum_i x_i u_i u_i^\top$ with $x_i > 0$ and $Y = \sum_j y_j v_j v_j^\top$ with $y_j \geq 0$, and put $c_{ij} = \langle u_i, v_j \rangle^2$. Every row and column of c sums to one, so we can express the trace terms in a common double sum. The scalar tangent inequality for $\phi(s) = s \log s$ then gives

$$\begin{aligned} h(Y) - h(X) - \operatorname{Tr}((I + \log X)(Y - X)) \\ = \sum_{i,j} c_{ij} \big[\phi(y_j) - \phi(x_i) - \phi'(x_i)(y_j - x_i)\big] \geq 0. \end{aligned}$$

Thus $I + \log X$ defines a supporting affine function at every positive definite X. This proves convexity on the positive definite cone and, by continuity, on its boundary. Applying the inequality in both directions as $Y \to X$ sandwiches $h(Y) - h(X)$ between the corresponding linear terms. Continuity of $\log X$ therefore gives $\nabla h(X) = I + \log X$. On $\mathcal{D}_n$, the trace constraint cancels the affine terms in the Bregman divergence, leaving $D_h(Y, X) = \operatorname{Tr}(Y \log Y) - \operatorname{Tr}(Y \log X)$, which is well defined even for singular Y.

The proximal objective is $\eta \operatorname{Tr}(GX^+) + D_h(X^+, X)$. Its first-order condition requires $\eta G + \log X^+ - \log X$ to be a scalar multiple of I, the normal to the trace-one constraint. Exponentiating and normalizing gives

$$X^+ = \frac{\exp(\log X - \eta G)}{\operatorname{Tr} \exp(\log X - \eta G)}. \tag{319}$$

Indeed, writing Z for the denominator gives $\eta G + \log X^+ - \log X = -(\log Z) I$. Its inner product with every feasible displacement is zero, so convexity proves optimality, including against singular density matrices. The order of operations matters: in general $\exp(\log X - \eta G) \neq X e^{-\eta G}$, since the matrices need not commute and their product need not even be symmetric.

Starting from I/n, successive logarithmic updates give $X_t = e^{-\eta S_t} / \operatorname{Tr} e^{-\eta S_t}$, where $S_t = \sum_{s<t} G_s$; scalar multiples of I disappear in normalization.

Algorithm 86 Matrix multiplicative weights

Input: Dimension n, step size $\eta > 0$, number of rounds T

$S \leftarrow 0$
for $t = 0, \ldots, T-1$ **do**
  Compute $-\eta S = U \operatorname{diag}(\lambda) U^\top$
  $p_i \leftarrow e^{\lambda_i - \max_j \lambda_j}$; normalize $p \leftarrow p / (\sum_i p_i)$
  $X_t \leftarrow U \operatorname{diag}(p) U^\top$
  Play X_t, receive symmetric G_t, and set $S \leftarrow S + G_t$

For the regret proof, track the logarithm of the normalizing factor $Z_t = \operatorname{Tr} e^{-\eta S_t}$. Its change records the algorithm's loss, while its final value is bounded below by the weight of the best fixed direction. The only obstruction to the scalar argument is that e^{P+Q} need not equal $e^P e^Q$. The Golden–Thompson inequality gives the comparison needed after taking the trace: $\operatorname{Tr} e^{P+Q} \leq \operatorname{Tr}(e^P e^Q)$ for symmetric P, Q, even when they do not commute.

Theorem 645 (Matrix multiplicative weights)**.** *If $\eta \|G_t\|_{\mathrm{op}} \leq 1$ for every t, then every $U \in \mathcal{D}_n$ satisfies*

$$\sum_{t<T} \operatorname{Tr}(G_t(X_t - U)) \leq \frac{\log n}{\eta} + \eta \sum_{t<T} \operatorname{Tr}(X_t G_t^2).$$

If $\|G_t\|_{\mathrm{op}} \leq G$, the right side is at most $\log n/\eta + \eta T G^2$. For $n \geq 2$, $G > 0$, and $T \geq \log n$, choosing $\eta = \sqrt{\log n/(TG^2)}$ gives regret at most $2G\sqrt{T \log n}$.

Proof. Put $W_t = e^{-\eta S_t}$ and $Z_t = \operatorname{Tr} W_t$, so $Z_0 = n$. First bound how much one update can change this potential. For $|a| \leq 1$, the exponential series gives $e^{-a} \leq 1 - a + a^2 \sum_{k \geq 2} 1/k! \leq 1 - a + a^2$. Apply this in an

eigenbasis of G_t, then use Golden–Thompson:

$$\begin{aligned} Z_{t+1} &\le \operatorname{Tr}(W_t e^{-\eta G_t}) \\ &\le Z_t\big(1 - \eta \operatorname{Tr}(X_t G_t) + \eta^2 \operatorname{Tr}(X_t G_t^2)\big). \end{aligned}$$

The second line uses the fact that taking the trace against $W_t \succeq 0$ preserves positive semidefinite inequalities. Divide by Z_t, use $\log(1+a) \le a$, and sum:

$$\log Z_T \le \log n - \eta \sum_{t<T} \operatorname{Tr}(X_t G_t) + \eta^2 \sum_{t<T} \operatorname{Tr}(X_t G_t^2).$$

For the lower bound, Z_T includes the exponential corresponding to a minimum eigenvector of S_T, so $\log Z_T \ge -\eta \lambda_{\min}(S_T)$. Every density matrix has $\operatorname{Tr}(U S_T) \ge \lambda_{\min}(S_T)$: take the trace of $S_T \succeq \lambda_{\min}(S_T) I$ against U. Combining the bounds and dividing by η proves the first claim. Finally, $G_t^2 \preceq G^2 I$ implies $\operatorname{Tr}(X_t G_t^2) \le G^2$. The stated step size satisfies $\eta G \le 1$ and balances the two regret terms. □

16.6 Sparsification and Sensitivity

Richardson is useful once we have a preconditioner that is both easy to solve in and close to the original matrix. For $A^\top A$, a way to seek one is to retain a small weighted set of rows of A, preserving $\|Ax\|_2^2$ for every x. Uniform preservation is essential: the queries made by an optimization algorithm can depend on the sampled matrix, so concentration at a query fixed before sampling is insufficient.

The same sampling question arises for any sum of nonnegative losses $F(x) = \sum_i f_i(x)$. Uniform sampling can fail if a single term dominates at some query. We should instead measure the largest fraction of the objective that each term can contribute.

Definition 646 (Sensitivity)**.** The sensitivity of term i is

$$\tau_i = \sup_{x : F(x) > 0} \frac{f_i(x)}{F(x)}.$$

If term i is kept with probability p_i, we multiply it by $1/p_i$ to keep the estimate unbiased. Its largest possible relative contribution becomes τ_i / p_i. Thus the choice $p_i = \min\{1, q\tau_i\}$ ensures that each randomly retained term contributes at most $F(x)/q$, uniformly over x. Terms kept with probability one create no sampling error. The expected number of retained terms is at most $q \sum_i \tau_i$, so the total sensitivity controls the cost of making every random contribution small.

16.6.1 Leverage Scores

For $A \in \mathbb{R}^{m \times n}$, write its ith row as $a_i^\top$, so $a_i \in \mathbb{R}^n$ is a column vector and $A^\top A = \sum_i a_i a_i^\top$. Put $M = A^\top A$ and assume first that A has full column rank. For $F(x) = \|Ax\|_2^2$, the change of variables $y = M^{1/2} x$ gives

$$\begin{aligned} \tau_i &= \sup_{x \ne 0} \frac{\langle a_i, x \rangle^2}{x^\top M x} \\ &= \sup_{y \ne 0} \frac{\langle M^{-1/2} a_i, y \rangle^2}{\|y\|_2^2} = \|M^{-1/2} a_i\|_2^2 = a_i^\top M^{-1} a_i. \end{aligned}$$

Cauchy–Schwarz proves the upper bound in the middle supremum; choosing $y = M^{-1/2} a_i$ attains it when the row is nonzero. These sensitivities are the *leverage scores*. The coordinate change, called whitening, turns the denominator into squared Euclidean length. The extremal query then points along the transformed row, so the sensitivity is exactly that row's squared length.

The crucial property is that their sum is exactly the dimension. Writing the scalar quadratic forms as traces makes the sum collapse:

$$\begin{aligned}\sum_i \tau_i &= \sum_i \operatorname{Tr}(M^{-1} a_i a_i^\top) \\ &= \operatorname{Tr}\left(M^{-1} \sum_i a_i a_i^\top\right) = \operatorname{Tr}(M^{-1} M) = \operatorname{Tr}(I) = n.\end{aligned}$$

For rank-deficient A, work on its row space and replace M^{-1} by $M^\dagger$; the sum is then $\operatorname{rank}(A)$. Zero rows have zero sensitivity and can be discarded. This identity is why the sample size can depend on the dimension instead of the number of rows.

16.6.2 The Sampling Algorithm

The formula for leverage scores uses the inverse of $A^\top A$, but the sampling algorithm needs only upper estimates $s_i \geq \tau_i$. Constant-factor upper estimates preserve the sample-size bound. Let $0 < \varepsilon < 1$ and $0 < \delta < 1$, and take $p_i = \min\{1, q s_i\}$. Each random contribution is still bounded by $F(x)/q$. The proof below will show that this gives a relative-error tail of order $n \exp(-\Theta(q\varepsilon^2))$, uniformly over all queries. This motivates $q = \Theta(\varepsilon^{-2} \log(n/\delta))$.

Algorithm 87 Leverage-score sampling

Input: Matrix A, upper scores $s_i \geq \tau_i$, accuracy ε, failure probability δ

$\widetilde{A} \leftarrow$ an empty matrix
for each nonzero row $a_i^\top$ **do**
  $p_i \leftarrow \min\{1, 16 s_i \log(4n/\delta)/\varepsilon^2\}$
  Independently sample $\xi_i \sim \text{Bernoulli}(p_i)$
  if $\xi_i = 1$ **then**
    Append $a_i^\top / \sqrt{p_i}$ to $\widetilde{A}$
return $\widetilde{A}$

For a fixed x, the estimate $\widetilde{F}(x) = \sum_i (\xi_i/p_i) \langle a_i, x\rangle^2$ is unbiased. Whenever $p_i < 1$, sensitivity gives

$$\frac{\langle a_i, x\rangle^2}{p_i} \leq \frac{\varepsilon^2}{16 \log(4n/\delta)} F(x).$$

Rows with $p_i = 1$ create no error. Scalar Bernstein therefore gives concentration for each fixed query. To obtain a single event on which every query is preserved, we bound the sampled matrix's error in operator norm after whitening. This norm already takes the supremum over all directions, so matrix concentration supplies the uniform guarantee.

Theorem 647 (Quadratic row sparsification)**.** *Let $s_i \geq \tau_i$ and $S = \sum_i s_i$. Then Algorithm 87 returns, with probability at least $1 - \delta$, a matrix with $O(S\varepsilon^{-2} \log(2n/\delta))$ rows satisfying*

$$(1 - \varepsilon) A^\top A \preceq \widetilde{A}^\top \widetilde{A} \preceq (1 + \varepsilon) A^\top A.$$

In particular, constant-factor upper estimates give $O(n\varepsilon^{-2} \log(2n/\delta))$ rows.

Proof. Discard zero rows and work on the row space of A; the zero-rank case is immediate. Put $M = A^\top A$ on that space and $v_i = M^{-1/2} a_i$. The identities $\sum_i v_i v_i^\top = I$ and $\|v_i\|_2^2 = \tau_i$ express, respectively, the total covariance and each row's contribution in these coordinates. The sampled error is

$$Z = \sum_i Z_i, \qquad Z_i = (\xi_i/p_i - 1) v_i v_i^\top.$$

Put $q = 16\log(4n/\delta)/\varepsilon^2$, so $p_i = \min\{1, qs_i\}$. Rows with $p_i = 1$ contribute $Z_i = 0$. For the remaining rows, $\tau_i/p_i \le 1/q$, and the rank-one matrix $v_i v_i^\top$ has norm $\|v_i\|_2^2 = \tau_i$. Consequently

$$\|Z_i\|_{\mathrm{op}} = |\xi_i/p_i - 1|\,\tau_i \le \tau_i/p_i \le 1/q.$$

To apply matrix Bernstein, we also need a variance bound. Rank one again makes the calculation scalar: $(v_i v_i^\top)^2 = (v_i^\top v_i) v_i v_i^\top = \tau_i v_i v_i^\top$. Also $\mathbb{E}(\xi_i/p_i - 1)^2 = p_i^{-1} - 1$, so

$$\begin{aligned}\sum_i \mathbb{E} Z_i^2 &= \sum_{i:p_i<1} (p_i^{-1} - 1)\tau_i v_i v_i^\top \\ &\preceq \frac{1}{q}\sum_{i:p_i<1} v_i v_i^\top \preceq \frac{1}{q} I.\end{aligned}$$

Matrix Bernstein, with both the summand bound and variance bound at most $1/q$, now gives

$$\begin{aligned}\Pr[\|Z\|_{\mathrm{op}} \ge \varepsilon] &\le 2n\exp\left(-\frac{\varepsilon^2}{2(q^{-1} + \varepsilon/(3q))}\right) \\ &\le 2n\exp(-3q\varepsilon^2/8) \le \delta/2,\end{aligned}$$

where $0 < \varepsilon < 1$ and the last inequality follows from the chosen q. Finally, $I + Z = M^{-1/2}\widetilde{A}^\top \widetilde{A} M^{-1/2}$. The inequality $\|Z\|_{\mathrm{op}} \le \varepsilon$ is equivalent to $-\varepsilon I \preceq Z \preceq \varepsilon I$. Adding I and taking a congruence by $M^{1/2}$ gives

$$(1-\varepsilon)A^\top A \preceq \widetilde{A}^\top \widetilde{A} \preceq (1+\varepsilon)A^\top A.$$

Taking the quadratic form against an arbitrary x gives the desired simultaneous preservation of all queries. On the kernel of A, both forms vanish identically.

The expected row count is at most qS. For the sum N of the independent row indicators, the scalar Chernoff bound gives $\Pr[N > 2qS] \le e^{-qS/3} \le \delta/2$, since $S \ge \mathrm{rank}(A) \ge 1$. A union bound proves both claims simultaneously. □

For regression, apply the theorem to the augmented matrix $[A\ -b]$. Its quadratic form at $(x, 1)$ is $\|Ax - b\|_2^2$. Thus sampling and rescaling its rows preserves the regression objective uniformly over x, using at most one additional dimension in the sample bound.

16.6.3 Why Higher Powers Can Require More Rows

The dimension bound on total sensitivity relied on the quadratic form becoming Euclidean after whitening. For higher powers, many rows can each dominate at a different query. To arrange this, choose nearly orthogonal rows: a query in one row's direction has inner product one with that row and small inner products with all others. Taking a power $p > 2$ suppresses these cross terms enough to make every row indispensable, even if the retained rows may be rescaled arbitrarily.

Lemma 648 (An obstruction for $p > 2$)**.** *For fixed $p > 2$ and sufficiently large n, there is a matrix with $m = \Omega_p(n^{p/2}/\log^{p/2}(2n))$ rows such that every row must be retained by any weighted row sample preserving $\|Ax\|_p^p$ within a factor $1 \pm 1/2$ for all x.*

Proof. Choose m independent uniform vectors $a_i \in \{-1/\sqrt{n}, 1/\sqrt{n}\}^n$. For distinct i, j, Hoeffding gives $\Pr[|\langle a_i, a_j\rangle| > \rho] \le 2e^{-n\rho^2/2}$. Take $\rho = C_p\sqrt{\log(2n)/n}$, with C_p sufficiently large, and $m = \lfloor c_p(n/\log(2n))^{p/2}\rfloor$, with c_p sufficiently small. A union bound makes the probability of any bad pair less than one, and the constants can also ensure $m\rho^p \le 1/8$. Fix a realization satisfying both properties.

At $x = a_i$, row i contributes 1 and all other rows together contribute at most $1/8$. In particular, each sensitivity is at least $8/9$. To rule out compensation by reweighting, query $x = e_1$: every row contributes $n^{-p/2}$, so the upper approximation forces the sum of the retained weights to be at most $3m/2$. If row i is absent, the sampled value at a_i is then at most $(3m/2)\rho^p \le 3/16$, whereas the original value is at least 1. This violates the lower approximation. □

The coordinate query e_1 is what rules out compensation by large weights: all rows contribute equally there, bounding the total rescaling that can be used to replace a missing row.

16.7 Saddle Points and Mirror-Prox

Let X, Y be convex sets and let Φ be differentiable and convex–concave on $K = X \times Y$. The saddle gap is

$$\operatorname{gap}(x, y) = \sup_{v \in Y} \Phi(x, v) - \inf_{u \in X} \Phi(u, y).$$

It is the sum of the improvements available to the two players when each changes only their own choice. Equivalently, it is the supremum of $\Phi(x, u_y) - \Phi(u_x, y)$ over joint comparators $u = (u_x, u_y) \in K$.

Write $z = (x, y)$ and define the saddle operator $F(z) = (\nabla_x \Phi(x, y), -\nabla_y \Phi(x, y))$. The minus sign accounts for maximization in the second variable. Adding the first-order convexity and concavity inequalities gives

$$\Phi(x, u_y) - \Phi(u_x, y) \le \langle F(z), z - u \rangle. \tag{320}$$

For fixed u, the left side is convex in z. Thus a bound $\sum_{t<T} \langle F(z'_t), z'_t - u \rangle \le C$, uniform over u and T, gives gap at most C/T at the average of the z'_t. We will obtain such a bound by making each comparison at most the decrease of a nonnegative potential. The comparator stays fixed while the potentials telescope; only afterward do we take the supremum over u.

16.7.1 From an Implicit Step to Prediction and Correction

Let r be a convex reference. As usual, assume the proximal points exist and take reference gradients only where they are defined. The saddle-gap comparison needs the operator evaluated at the point being compared. An implicit step $w = \operatorname{Prox}_z^r(\eta F(w))$ would therefore give, by Lemma 641,

$$\eta \langle F(w), w - u \rangle \le D_r(u, z) - D_r(u, w) - D_r(w, z).$$

Taking w as the next iterate makes the potential differences telescope. The obstacle is computational: the linear term in the proximal problem depends on the unknown output.

Using the available vector $F(z)$ instead gives the prediction $z' = \operatorname{Prox}_z^r(\eta F(z))$. Its proximal comparison involves $F(z)$, whereas the saddle-gap bound requires $F(z')$. Replacing one by the other introduces $\eta \langle F(z') - F(z), z' - u \rangle$. Even when a Lipschitz bound makes the operator change small, $z' - u$ need not be small. The negative divergence $-D_r(z', z)$ controls the computed step, not the distance to an arbitrary comparator. We need to remove that comparator from the error.

Introduce a correction z^+ and split $z' - u = (z' - z^+) + (z^+ - u)$. The prediction comparison can handle the first part, using z^+ as its comparator. For the second part, compute a proximal step using $F(z')$, so that no operator replacement is needed. The mismatch is then only $\eta \langle F(z') - F(z), z' - z^+ \rangle$: both factors concern computed displacements. The correction must also start from z, because its comparison should supply $D_r(u, z) - D_r(u, z^+)$, the change in potential from the current base to the next one.

Allow the correction size to be η/γ, with $\gamma \ge 1$. Multiplying its comparison by γ restores the coefficient η on the linear term. This supplies more negative divergence from z to z^+, but also multiplies the initial potential by γ. We will choose γ according to which divergences are needed to absorb the operator error.

Lemma 649 (Two-step comparison)**.** *Let $\eta > 0$, $\gamma \ge 1$, and*

$$z' = \operatorname{Prox}_z^r(\eta F(z)), \qquad z^+ = \operatorname{Prox}_z^r((\eta/\gamma) F(z')).$$

Then every $u \in K$ satisfies

$$\begin{aligned} \eta \langle F(z'), z' - u \rangle \le{}& \gamma \big(D_r(u, z) - D_r(u, z^+) \big) \\ & - D_r(z', z) - D_r(z^+, z') - (\gamma - 1) D_r(z^+, z) \\ & + \eta \langle F(z') - F(z), z' - z^+ \rangle. \end{aligned} \tag{321}$$

Proof. Apply Lemma 641 to prediction with comparator z^+ and to correction with comparator u, multiplying the second inequality by γ:

$$\eta\langle F(z), z' - z^+\rangle \le D_r(z^+, z) - D_r(z^+, z') - D_r(z', z),$$
$$\eta\langle F(z'), z^+ - u\rangle \le \gamma\big(D_r(u, z) - D_r(u, z^+) - D_r(z^+, z)\big).$$

Add the inequalities and then add $\eta\langle F(z') - F(z), z' - z^+\rangle$ to both sides. □

The left side compares the prediction with u, while the potential changes from z to the correction. Thus we average predictions and retain corrections as the next bases. Equal step sizes ($\gamma = 1$) leave two negative step divergences; a half-size correction ($\gamma = 2$) leaves all three with coefficient one.

The proof uses only the two proximal comparisons, not exact minimization itself. If those comparisons have additive errors $E'(u)$ and $E^+(u)$, the right side of (321) gains $E'(z^+) + \gamma E^+(u)$. The different comparators will be important: prediction needs an error bound only at the computed correction, whereas correction needs one for every fixed u.

16.7.2 Lipschitz Operators

The remaining error is a product of two step lengths when F is Lipschitz. Strong convexity then converts those lengths into the available divergences. Suppose r is 1-strongly convex in $\|\cdot\|$ and $\|F(z) - F(w)\|_* \le L\|z - w\|$. Norm duality, $ab \le (a^2 + b^2)/2$, and strong convexity give

$$\begin{aligned}\eta\langle F(z') - F(z), z' - z^+\rangle &\le \eta L\|z' - z\|\,\|z' - z^+\| \\ &\le \eta L\big(D_r(z', z) + D_r(z^+, z')\big).\end{aligned}$$

Thus $\eta L \le 1$ absorbs the error into the two corresponding negative divergences. Equal step sizes, $\gamma = 1$, suffice.

Algorithm 88 Mirror-Prox

Input: Saddle operator F; reference r that is 1-strongly convex in $\|\cdot\|$

Input: $\|F(z) - F(w)\|_* \le L\|z - w\|$, $L > 0$; initial $z_0 \in K$, $0 < \eta \le 1/L$, $T \ge 1$

$s \leftarrow 0$
for $t = 0, \dots, T - 1$ **do**
  $z'_t \leftarrow \mathrm{Prox}^r_{z_t}(\eta F(z_t))$
  $z_{t+1} \leftarrow \mathrm{Prox}^r_{z_t}(\eta F(z'_t))$
  $s \leftarrow s + z'_t$
return s/T

Theorem 650 (Mirror-Prox gap bound)**.** *If $\Omega = \sup_{u \in K} D_r(u, z_0) < \infty$, the returned average satisfies $\mathrm{gap}(\bar{x}, \bar{y}) \le \Omega/(\eta T)$. In particular, $\eta = 1/L$ gives $L\Omega/T$.*

Proof. The two-step comparison and the preceding error bound give $\eta\langle F(z'_t), z'_t - u\rangle \le D_r(u, z_t) - D_r(u, z_{t+1})$. Summing and discarding the nonnegative final divergence yields

$$\frac{1}{T}\sum_{t<T}\langle F(z'_t), z'_t - u\rangle \le \frac{D_r(u, z_0)}{\eta T}. \tag{322}$$

For fixed $u = (u_x, u_y)$, convexity–concavity and (320) imply

$$\Phi(\bar{x}, u_y) - \Phi(u_x, \bar{y}) \le \frac{1}{T}\sum_{t<T}\langle F(z'_t), z'_t - u\rangle \le \frac{D_r(u, z_0)}{\eta T}.$$

Take the supremum over u. □

The proof separates the roles of the reference: its step divergences must control the operator error, and its initial divergence must be small. A norm and strong convexity provide one way to meet these requirements, but the two-step comparison itself does not require that particular route.

16.8 Solving Box–Simplex Games

Consider minimizing $P(x) = c^\top x + \max_j (A^\top x - b)_j$ over $[-1,1]^n$. A maximum of coordinates is a maximization over a simplex, giving the game

$$\min_{x\in[-1,1]^n} \max_{y\in\Delta_d} \Phi(x,y), \qquad \Phi(x,y) = x^\top A y - b^\top y + c^\top x,$$

where $A \in \mathbb{R}^{n\times d}$ and $n, d \geq 1$. The simplex variable also supplies a lower bound on the optimum: minimizing over the box gives $D(y) = -b^\top y - \|Ay + c\|_1$, and $D(y) \leq \Phi(x,y) \leq P(x)$ for every feasible pair. Thus $\mathrm{gap}(x,y) = P(x) - D(y)$ is a computable certificate of primal suboptimality. The saddle operator is $g(x,y) = (Ay + c, b - A^\top x)$.

Let $L = \|A\|_{1\to 1} = \max_j \sum_i |A_{ij}|$. If $L = 0$, the variables separate: return $x = -\operatorname{sgn}(c)$ and $y = e_j$ for $j \in \arg\min_k b_k$, with $\operatorname{sgn}(0) = 0$. Otherwise divide A, b, c by L. Until the final algorithm, we use these normalized data and write $B = |A|$ entrywise, so

$$B \geq 0, \qquad \sum_i B_{ij} \leq 1 \quad \text{for every } j. \tag{323}$$

Original gaps are L times normalized gaps.

Our goal is $O(\log(2d)/\varepsilon)$ rounds for the normalized game, with a constant number of matrix products per round. There are two obstacles. First, the usual norm-based analysis charges too much initial divergence on the box. We will instead choose curvature directly from the matrix interaction. Second, the resulting reference couples the variables, making its joint proximal minimization nontrivial. We will obtain the needed comparison from a single sweep, without solving that minimization accurately.

16.8.1 The Interaction Determines the Curvature

Column normalization gives $|\xi^\top A \upsilon| \leq \|\xi\|_\infty \|\upsilon\|_1$, suggesting ℓ_∞ geometry on the box. However, a reference strongly convex in this norm supplies curvature in every coordinate, regardless of which coordinates matter to the matrix. This forces large initial divergence.

Lemma 651 (Strong convexity on a cube)**.** *If h is 1-strongly convex in ℓ_∞ on $[-1,1]^n$, then every point x_0 where h is differentiable satisfies $\sup_{u\in[-1,1]^n} D_h(u, x_0) \geq n/2$.*

Proof. Reveal independent uniform signs one at a time, writing $X_k = (\sigma_1, \ldots, \sigma_k, 0, \ldots, 0)$. The two conditional values of X_k are distance 2 apart and have midpoint X_{k-1}, so strong convexity gives $\mathbb{E}[h(X_k) \mid X_{k-1}] \geq h(X_{k-1}) + 1/2$. Summing yields $\mathbb{E}h(X_n) - h(0) \geq n/2$. Since $\mathbb{E}X_n = 0$,

$$\mathbb{E}D_h(X_n, x_0) = \mathbb{E}h(X_n) - h(0) + D_h(0, x_0) \geq n/2.$$

□

This obstructs the fixed ℓ_∞ analysis, not a reference adapted to the game. Return to the error that the two-step proof actually needs to control. Write $z = (x,y)$, $z' = (x',y')$, and $z^+ = (x^+, y^+)$. The affine terms b, c cancel from the operator difference, and the remaining bilinear terms satisfy $\langle g(z') - g(z), z' - z\rangle = 0$. Consequently,

$$\begin{aligned} \mathcal{E}(z, z', z^+) &:= \langle g(z') - g(z), z' - z^+\rangle \\ &= (x' - x)^\top A(y^+ - y) - (x^+ - x)^\top A(y' - y). \end{aligned} \tag{324}$$

Both terms use displacements from z. This will let the same simplex point y supply the weights in both bounds, matching the divergences based at z. It also explains why the iteration bound need not depend on b, c.

Consider one interaction $\xi^\top A(p-q) = \langle p-q, f\rangle$, where $f = A^\top \xi$, $|\xi_i| \le 2$, and $p, q \in \Delta_d$ with $q > 0$. To choose the reference, first ask which quadratic forms separate these two displacements. Weighted Cauchy–Schwarz gives

$$|\langle p-q, f\rangle| \le \left(\sum_j \frac{(p_j-q_j)^2}{q_j}\right)^{1/2} \left(\sum_j q_j f_j^2\right)^{1/2}.$$

We use this calculation to identify the curvature, not as the final estimate. It suggests how the two blocks should be weighted; a separate argument will establish a global bound in terms of the resulting Bregman divergences.

For the box factor, f_j is the change in payoff against column j. Column normalization and Cauchy–Schwarz give

$$|f_j| \le 2, \qquad f_j^2 \le \left(\sum_i B_{ij}\right) \sum_i B_{ij}\xi_i^2 \le \sum_i B_{ij}\xi_i^2.$$

Averaging under q yields

$$\|f\|_\infty \le 2, \qquad \sum_j q_j f_j^2 \le \sum_i (Bq)_i \xi_i^2 =: Q_q(\xi). \tag{325}$$

Thus coordinate i receives weight $(Bq)_i$, its absolute coefficients averaged under the current distribution. These weights sum to at most one, so $Q_q(\xi) \le 4$ independently of n. Unlike a fixed box norm, this quadratic assigns curvature according to the matrix and the current simplex point. The second-moment inequality itself holds for arbitrary ξ, and also with B in place of A.

The simplex factor has reciprocal weights $1/q_j$. A separable reference $h(y) = \sum_j \varphi(y_j)$ has local quadratic $\frac{1}{2}\sum_j \varphi''(q_j)(p_j-q_j)^2$, suggesting $\varphi''(s) = 1/s$. Integrating and ignoring affine terms gives

$$h(y) = \sum_j y_j \log y_j, \qquad H(p,q) := D_h(p,q) = \sum_j p_j \log(p_j/q_j).$$

We use $0 \log 0 = 0$. Simplex bases are positive, but comparators may lie on the boundary.

Matching this quadratic does not yet prove the desired interaction bound. For example, from $q = \mathbf{1}/d$ to a vertex p, the quadratic simplex term is $d-1$, whereas $H(p,q) = \log d$. We cannot replace the former by the latter in Cauchy–Schwarz. Instead, we return to the linear interaction and use boundedness of f to prove a new estimate involving $H(p,q)$ directly.

16.8.2 From Local Curvature to a Global Bound

We want to bound $\langle p-q, f\rangle$ by an entropy term plus a weighted quadratic in f. Subtract the proposed entropy term: what remains is the largest linear gain obtainable after paying that penalty. This is an entropy-regularized linear optimization, whose value and minimizer can be computed exactly.

For $\tau > 0$, combine a linear cost with the logarithm:

$$\langle f, p\rangle + \tau H(p,q) = \tau \sum_j p_j \log \frac{p_j}{q_j e^{-f_j/\tau}}.$$

The denominator is positive but need not sum to one. Normalize it:

$$p_j^* = \mathrm{EntropyStep}(q, f, \tau)_j = \frac{q_j e^{-f_j/\tau}}{Z}, \qquad Z = \sum_k q_k e^{-f_k/\tau}. \tag{326}$$

Because $\sum_j p_j = 1$, normalization contributes only a constant:

$$\langle f, p\rangle + \tau H(p, q) = -\tau \log Z + \tau H(p, p^*). \tag{327}$$

This is the entropy analogue of completing the square. The minimizer is p^*, its value is $-\tau \log Z$, and the excess at any p is exactly $\tau H(p, p^*)$. Adding a linear cost therefore changes the base of the entropy divergence. We will use the minimum value to bound interactions, and the change-of-base identity later to account for residuals.

Lemma 652 (Entropy comparison)**.** *If $p, q \in \Delta_d$, $q > 0$, and $\tau \geq \|f\|_\infty$ with $\tau > 0$, then*

$$|\langle p - q, f\rangle| \leq \tau H(p, q) + \frac{3}{4\tau} \sum_j q_j f_j^2. \tag{328}$$

Proof. Apply (327) with cost $-f$. For $Z = \sum_j q_j e^{f_j/\tau}$, it gives $\langle p, f\rangle - \tau H(p, q) \leq \tau \log Z$. For $|t| \leq 1$, the exponential series gives

$$e^t - 1 - t \leq \sum_{k\geq 2} \frac{|t|^k}{k!} \leq \frac{t^2}{2} \sum_{k\geq 0} 3^{-k} = \tfrac{3}{4} t^2,$$

where $k! \geq 2 \cdot 3^{k-2}$ for $k \geq 2$. Since $|f_j/\tau| \leq 1$, using $\log Z \leq Z - 1$ gives

$$\tau \log Z \leq \langle q, f\rangle + \frac{3}{4\tau} \sum_j q_j f_j^2.$$

Combine the two bounds and rearrange. Replacing f by $-f$ proves the opposite sign. □

This is the global estimate suggested by the earlier curvature calculation. It controls the linear interaction without bounding the potentially much larger quadratic simplex term. Boundedness is required only here: the entropy update and its exact identity hold for arbitrary costs, including the unrestricted terms b, c that appear in the algorithm.

Combining the lemma with (325) gives, for $\tau \geq 2$,

$$|\xi^\top A(p - q)| \leq \frac{3}{4\tau} Q_q(\xi) + \tau H(p, q). \tag{329}$$

Apply this to the two terms of (324), using $q = y$ in both. Each interaction couples the box displacement of one point to the simplex displacement of the other. After bounding them, regroup the box and simplex costs belonging to the same point:

$$\begin{aligned} |\mathcal{E}(z, z', z^+)| \leq & \left[\frac{3}{4\tau} Q_y(x' - x) + \tau H(y', y)\right] \\ & + \left[\frac{3}{4\tau} Q_y(x^+ - x) + \tau H(y^+, y)\right]. \end{aligned} \tag{330}$$

We now know what we need: the divergence from z to each computed point should dominate that point's term in the bracket. The parameter τ trades the required box curvature against entropy.

16.8.3 Choosing One Reference with Adaptive Curvature

For fixed y, the desired box curvature is a weighted quadratic. We cannot simply choose a new reference at each round according to y: divergences belonging to different reference functions would not automatically telescope. Instead, put the changing weights inside a single fixed function of both variables:

$$r_a(x, y) = \tfrac{1}{2}\langle By, x^2\rangle + a h(y), \tag{331}$$

where squares are coordinatewise and $a \geq 0$ will be chosen below. Its box curvature adapts with y, while the reference itself remains fixed.

For a fixed simplex point, the box divergence is $\frac{1}{2}Q_y$. When both variables move, changing the weights creates a mixed term, so this quadratic and the entropy divergence do not simply add. The mixed term is another difference of expectations, now of quadratic values. The factor $1/2$ makes $t^2/2$ 1-Lipschitz on $[-1, 1]$, so these value changes are no larger than the underlying box displacements. The entropy comparison can therefore control this term as well.

Lemma 653 (Curvature of the coupled reference)**.** *For $\tau \geq 2$ and $z = (x, y), w = (\widetilde{x}, p)$ in $[-1, 1]^n \times \Delta_d$ with $y > 0$,*

$$D_{r_a}(w, z) \geq \left(\tfrac{1}{2} - \frac{3}{4\tau}\right) Q_y(\widetilde{x} - x) + (a - \tau)H(p, y). \tag{332}$$

In particular, r_a is convex for every $a \geq 2$.

Proof. Put $\delta = \widetilde{x} - x$ and $\zeta = \frac{1}{2}(\widetilde{x}^2 - x^2)$, so $|\zeta_i| \leq |\delta_i|$ and $|\zeta_i| \leq 1/2$. For $f = B^\top \zeta$, direct expansion gives

$$D_{r_a}(w, z) = \tfrac{1}{2}Q_y(\delta) + aH(p, y) + \langle p - y, f\rangle. \tag{333}$$

The second-moment bound, applied to B and ζ, gives $\sum_j y_j f_j^2 \leq Q_y(\zeta) \leq Q_y(\delta)$; column normalization also gives $\|f\|_\infty \leq 1/2$. The entropy comparison bounds the mixed term below by $-\tau H(p, y) - 3Q_y(\delta)/(4\tau)$, proving the claim. With $\tau = 2$, the divergence is nonnegative for $a \geq 2$. This proves convexity on the positive simplex, and continuity extends it to the closed simplex. □

We must leave enough curvature after controlling the changing weights to pay for the game interaction. Using the same τ in both estimates, this requires

$$\frac{3}{4\tau} \leq \frac{1}{2} - \frac{3}{4\tau}, \qquad \tau \leq a - \tau.$$

Choose $\tau = 3$ and $a = 6$. The original quadratic coefficient $1/2$ is split as $1/4 + 1/4$: half controls the mixed term and half remains for the interaction. Each use costs $3H$, accounting for the six entropy units. Substitution into (330) yields

$$|\langle g(z') - g(z), z' - z^+\rangle| \leq D_{r_6}(z', z) + D_{r_6}(z^+, z). \tag{334}$$

With $\eta = 1$, their negative coefficients in the two-step comparison are 1 and $\gamma - 1$. Thus choose $\gamma = 2$: prediction has size 1, correction has size $1/2$, and both start from z.

For small initial divergence, choose the box center, where the quadratic and its gradient vanish, and the uniform simplex point. The latter minimizes the worst-case entropy divergence: $H(e_j, q) = -\log q_j$ forces a value at least $\log d$ for every positive base, while $H(p, \mathbf{1}/d) \leq \log d$ for every p. Thus $z_0 = (0, \mathbf{1}/d)$ satisfies

$$D_{r_a}(u, z_0) = \tfrac{1}{2}\langle Bu_y, u_x^2\rangle + aH(u_y, \mathbf{1}/d) \leq \tfrac{1}{2} + a \log d. \tag{335}$$

Exact proximal steps with r_6 would now give gap at most $(1 + 12 \log d)/T$. This settles the geometry and iteration count but it does not yet make a round cheap. The remaining problem is the joint proximal minimization.

16.8.4 One Sweep Instead of a Joint Minimization

Each block of the proximal objective is explicit when the other is fixed. Rather than alternate until the joint problem is nearly solved, we will take one sweep and bound its first-order residual. The three-point identity in Lemma 641 will then give the proximal comparison with that residual retained. We deliberately end with a box minimization: its first-order contribution is nonpositive, leaving only a simplex error for entropy to control. Keep $r = r_a$ with $a \geq 2$ for now.

Fix a proximal base $z = (x, y)$ with $y > 0$ and a linear term $v = (v_x, v_y)$. Put $\ell = v_x - x \odot (By)$, where $\odot$ denotes coordinatewise product. Expanding the proximal objective and dropping terms independent of (w, p) gives

$$\begin{aligned} J(w,p) &= \phi_p(w) + \langle v_y - \tfrac{1}{2}B^\top x^2, p\rangle + aH(p,y), \\ \phi_p(w) &:= \tfrac{1}{2}\langle Bp, w^2\rangle + \langle \ell, w\rangle. \end{aligned} \tag{336}$$

Both blocks are minimized here; the players' opposite signs are already encoded in the saddle operator used to choose v.

For fixed p, the box problem separates into one-variable quadratics. Minimize each and clip to the feasible interval:

$$X(p)_i = \mathrm{clip}_{[-1,1]}\left(-\frac{\ell_i}{(Bp)_i}\right). \tag{337}$$

If $(Bp)_i = 0$, that coordinate's objective is linear; take $X(p)_i = -\operatorname{sgn}(\ell_i)$, with $\operatorname{sgn}(0) = 0$. The function X belongs to this fixed proximal subproblem: z, v, and hence ℓ, do not change during the sweep.

Eliminating the box would leave $\min_w \phi_p(w)$ as the only non-explicit part of the simplex objective. This is a minimum of affine functions of p, hence concave. Each fixed box point supplies an affine upper bound. Choose a positive distribution q and take $\widetilde{x} = X(q)$; then $\min_w \phi_p(w) \le \phi_p(\widetilde{x})$, with equality at $p = q$. Replace only this difficult term by its touching affine upper bound, keeping the entropy term. Its minimizer is an explicit entropy step. Finally, minimize over the box at the new simplex point:

$$\begin{aligned} \widetilde{x} &= X(q), \\ \widehat{y} &= \mathrm{EntropyStep}(y, v_y + \tfrac{1}{2}B^\top(\widetilde{x}^2 - x^2), a), \\ \widehat{x} &= X(\widehat{y}). \end{aligned} \tag{338}$$

Note that the entropy step remains centered at y, even when $q \neq y$.

The last box update restores box optimality but changes the simplex gradient by

$$e = \tfrac{1}{2}B^\top(\widehat{x}^2 - \widetilde{x}^2). \tag{339}$$

We stop here not because the joint minimizer has been approximated to high accuracy, but because its remaining error lies entirely in the simplex block and we can control it. The following lemma gives the comparison needed by the outer proof.

Lemma 654 (What one sweep guarantees)**.** *Let $\widehat{z} = (\widehat{x}, \widehat{y})$ be the sweep from a positive q. For every feasible $u = (u_x, u_y)$,*

$$\langle v, \widehat{z} - u\rangle \le D_r(u, z) - D_r(u, \widehat{z}) - D_r(\widehat{z}, z) + \langle e, \widehat{y} - u_y\rangle. \tag{340}$$

Moreover, for every $p \in \Delta_d$,

$$\langle e, \widehat{y} - p\rangle \le 2H(p, q). \tag{341}$$

Proof. For the first claim, evaluate the gradient of J at $\widehat{z} = (\widehat{x}, \widehat{y})$. Since $\widehat{x} = X(\widehat{y})$ minimizes the box objective, first-order optimality gives

$$\langle \nabla_w J(\widehat{x}, \widehat{y}), \widehat{x} - u_x\rangle \le 0.$$

For the simplex block, $\widehat{y}$ minimizes the entropy-regularized objective with the box frozen at $\widetilde{x}$. For the entropy update, its first-order condition on the simplex says

$$\nabla_p J(\widetilde{x}, \widehat{y}) = \alpha \mathbf{1}$$

for some scalar α. Changing the box point from $\widetilde{x}$ to $\widehat{x}$ changes this gradient by exactly

$$\tfrac{1}{2}B^\top(\widehat{x}^2 - \widetilde{x}^2) = e,$$

so

$$\nabla_p J(\widehat{x}, \widehat{y}) = \alpha \mathbf{1} + e.$$

Because $\widehat{y}$ and u_y both lie in the simplex, $\langle \mathbf{1}, \widehat{y} - u_y \rangle = 0$. Combining the two blocks,

$$\langle \nabla J(\widehat{z}), \widehat{z} - u \rangle \le \langle e, \widehat{y} - u_y \rangle.$$

Finally, $\nabla J(\widehat{z}) = v + \nabla r(\widehat{z}) - \nabla r(z)$, while the three-point identity gives

$$\langle \nabla r(\widehat{z}) - \nabla r(z), u - \widehat{z} \rangle = D_r(u, z) - D_r(u, \widehat{z}) - D_r(\widehat{z}, z).$$

Rearranging proves (340).

For the residual bound, first quantify the loss from keeping the box at $\widetilde{x} = X(q)$ after the simplex point changes to p. The function

$$G(w, p) := \phi_p(w) + 2h(p)$$

differs from the convex reference r_2 only by a linear term, so it is jointly convex. Its supporting-plane inequality at $(\widetilde{x}, q)$ is

$$\phi_p(w) + 2h(p) \ge \phi_q(\widetilde{x}) + 2h(q) + \langle \nabla_w \phi_q(\widetilde{x}), w - \widetilde{x} \rangle + \left\langle \tfrac{1}{2} B^\top \widetilde{x}^2 + 2\nabla h(q), p - q \right\rangle.$$

The box-gradient term is nonnegative because $\widetilde{x} = X(q)$ minimizes ϕ_q over the box. Moreover, $\phi_p(\widetilde{x})$ is affine in p, so

$$\phi_q(\widetilde{x}) + \left\langle \tfrac{1}{2} B^\top \widetilde{x}^2, p - q \right\rangle = \phi_p(\widetilde{x}).$$

Hence

$$\phi_p(w) + 2h(p) \ge \phi_p(\widetilde{x}) + 2h(q) + 2\langle \nabla h(q), p - q \rangle,$$

or equivalently

$$\phi_p(\widetilde{x}) - \phi_p(w) \le 2H(p, q) \qquad \text{for every box point } w. \tag{342}$$

It remains to relate this approximation error to the residual e. Using the definition of ϕ_p,

$$\begin{aligned} \left[\phi_{\widehat{y}}(\widehat{x}) - \phi_{\widehat{y}}(\widetilde{x})\right] + \left[\phi_p(\widetilde{x}) - \phi_p(\widehat{x})\right] &= \tfrac{1}{2}\langle B\widehat{y}, \widehat{x}^2 - \widetilde{x}^2 \rangle + \tfrac{1}{2}\langle Bp, \widetilde{x}^2 - \widehat{x}^2 \rangle = \tfrac{1}{2}\left\langle B(\widehat{y} - p), \widehat{x}^2 - \widetilde{x}^2 \right\rangle \\ &= \left\langle \tfrac{1}{2} B^\top (\widehat{x}^2 - \widetilde{x}^2), \widehat{y} - p \right\rangle = \langle e, \widehat{y} - p \rangle. \end{aligned}$$

The first bracket is nonpositive because $\widehat{x} = X(\widehat{y})$ minimizes $\phi_{\widehat{y}}$. Applying (342) with $w = \widehat{x}$ bounds the second bracket by $2H(p, q)$. Therefore

$$\langle e, \widehat{y} - p \rangle \le 2H(p, q),$$

which proves (341). □

16.8.5 Prediction and Correction

The same sweep will implement both steps. Its residual is bounded against every simplex comparator, but the two-step proof uses different comparators in the two calls. This determines both where to start each sweep and how to account for its error.

Prediction. The prediction is compared only with the computed correction $z^+ = (x^+, y^+)$. Its error is at most $2H(y^+, q)$. Start this sweep at $q = y$, making the error $2H(y^+, y)$: it is now a local divergence with the same base as the step divergences already available.

The interaction uses the curvature of r_6. To cover the prediction error as well, add two entropy units and use $r = r_8 = r_6 + 2h$ for both calls. Then

$$\mathcal{E}(z, z', z^+) + 2H(y^+, y) \le D_{r_6}(z', z) + D_{r_8}(z^+, z) \le D_r(z', z) + D_r(z^+, z). \tag{343}$$

These are the two divergences retained by the two-step comparison with a half-size correction.

Correction. This step is compared with an arbitrary fixed u. Merely adding $2H(u_y, q)$ in every round would accumulate error, since the comparator need not approach q. We instead retain q between rounds and turn the residual into a change in its entropy potential.

The crucial information is that (341) holds for *every* p. Equivalently,

$$\langle e, \widehat{y} \rangle \le \langle e, p \rangle + 2H(p, q) \qquad \text{for every } p \in \Delta_d.$$

Choose the strongest of these upper bounds by minimizing the right side. Its minimizer is

$$q^+ = \mathrm{EntropyStep}(q, e, 2). \tag{344}$$

By (327), with $f = e$, $\tau = 2$, and $p^* = q^+$,

$$\langle e, p \rangle + 2H(p, q) = -2 \log Z + 2H(p, q^+).$$

In particular,

$$\min_p \{\langle e, p \rangle + 2H(p, q)\} = -2 \log Z.$$

Evaluating the same identity at the comparator u_y therefore gives

$$\min_p \{\langle e, p \rangle + 2H(p, q)\} = \langle e, u_y \rangle + 2H(u_y, q) - 2H(u_y, q^+).$$

Combining this with

$$\langle e, \widehat{y} \rangle \le \min_p \{\langle e, p \rangle + 2H(p, q)\}$$

yields

$$\langle e, \widehat{y} - u_y \rangle \le 2H(u_y, q) - 2H(u_y, q^+). \tag{345}$$

Thus the correction residual is paid for by a decrease in the auxiliary entropy potential. Retaining q^+ for the next round makes these decreases telescope, so there is no need to drive the residual itself to zero by an inner iteration. The point q is therefore bookkeeping rather than another game iterate: explicitly,

$$q_{t,j} \propto q_{0,j} \exp\left(-\frac{1}{2} \sum_{s<t} e_{s,j} \right),$$

so it records the accumulated correction residuals.

16.8.6 Algorithm and Convergence

Prediction uses linear term $g(z)$ and sweep start y; correction uses $g(z')/2$ and the persistent start q. Both calls use r_8 and retain the same proximal base z until correction is complete. Only correction is followed by the auxiliary entropy update. Vector squares, products denoted by $\odot$, and quotients below are coordinatewise. An asterisk marks an unused return value.

Algorithm 89 Explicit box–simplex sweep

function EntropyStep(q, f, τ)
  $s_j \leftarrow \log q_j - f_j/\tau$ for every j; $M \leftarrow \max_j s_j$
  $w_j \leftarrow e^{s_j - M}$ for every j
  return $w/(\sum_j w_j)$

function BoxResponse(B, ℓ, p)
  $w \leftarrow Bp$
  for $i = 1, \dots, n$ **do**
    if $w_i = 0$ **then**
      $u_i \leftarrow -\operatorname{sgn}(\ell_i)$
    else
      $u_i \leftarrow \max\{-1, \min\{1, -\ell_i/w_i\}\}$
  return u

function BoxSweep($B, z = (x, y), v = (v_x, v_y), q$)
  $\ell \leftarrow v_x - x \odot (By)$
  $\widetilde{x} \leftarrow$ BoxResponse(B, ℓ, q)
  $d_0 \leftarrow v_y + \frac{1}{2} B^\top (\widetilde{x}^2 - x^2)$
  $\widehat{y} \leftarrow$ EntropyStep($y, d_0, 8$)
  $\widehat{x} \leftarrow$ BoxResponse($B, \ell, \widehat{y}$)
  return $((\widehat{x}, \widehat{y}), \widetilde{x})$

Algorithm 90 Box–simplex game solver

Input: $A \in \mathbb{R}^{n \times d}$, $b \in \mathbb{R}^d$, $c \in \mathbb{R}^n$, target gap $\varepsilon > 0$
Output: A feasible pair $(\bar{x}, \bar{y})$ with gap at most ε

$L \leftarrow \max_j \sum_i |A_{ij}|$
if $L = 0$ **then**
  return $(-\operatorname{sgn}(c), e_j)$, where $j \in \arg\min_k b_k$
$(A, b, c) \leftarrow (A/L, b/L, c/L)$; $B \leftarrow |A|$
$T \leftarrow \lceil (1 + 20 \log d) L/\varepsilon \rceil$
$z = (x, y) \leftarrow (0, \mathbf{1}/d)$; $q \leftarrow \mathbf{1}/d$; $s \leftarrow 0$
for $t = 0, \dots, T - 1$ **do**
  $v \leftarrow (Ay + c,\ b - A^\top x)$
  $(z' = (x', y'), *) \leftarrow$ BoxSweep(B, z, v, y)
  $s \leftarrow s + z'$
  $v^+ \leftarrow \frac{1}{2}(Ay' + c,\ b - A^\top x')$
  $(z^+ = (x^+, y^+), \widetilde{x}) \leftarrow$ BoxSweep(B, z, v^+, q)
  $e \leftarrow \frac{1}{2} B^\top ((x^+)^2 - \widetilde{x}^2)$
  $q^+ \leftarrow$ EntropyStep($q, e, 2$)
  $z = (x, y) \leftarrow z^+$; $q \leftarrow q^+$
return s/T

Theorem 655 (Solving box–simplex games)**.** *After $T \geq 1$ rounds on the normalized game, the prediction average satisfies*

$$\operatorname{gap}\left(\frac{1}{T}\sum_{t<T} z'_t\right) \leq \frac{1 + 20 \log d}{T}. \tag{346}$$

Consequently, Algorithm 90 returns a feasible pair with additive saddle gap at most ε in time

$$O\left((\mathrm{nnz}(A)+n+d)\left(1+\frac{\|A\|_{1\to 1}\log(2d)}{\varepsilon}\right)\right).$$

Proof. All box responses are feasible, and normalized exponentiation from positive initial distributions keeps all simplex bases positive in exact arithmetic. Thus every comparison above applies.

Write $r = r_8$. The correction comparison is multiplied by two to restore the coefficient $g(z')$. Its outer divergence difference therefore has coefficient 2, and its auxiliary entropy difference has coefficient $2 \cdot 2 = 4$. These determine the potential

$$\mathcal{P}_u(z,q) = 2D_{r_8}(u,z) + 4H(u_y,q). \tag{347}$$

Apply the two-step comparison with $\eta = 1$, $\gamma = 2$, and the two sweep residuals. The prediction residual is at most $2H(y^+, y)$; together with the operator error, it is absorbed by $D_r(z', z) + D_r(z^+, z)$ through (343). The correction residual contributes the auxiliary potential difference in (345). The only remaining step divergence has a negative sign, giving

$$\langle g(z'), z' - u\rangle \le \mathcal{P}_u(z,q) - \mathcal{P}_u(z^+,q^+) - D_r(z^+,z'). \tag{348}$$

At $z_0 = (0, \mathbf{1}/d)$ and $q_0 = \mathbf{1}/d$, (335) gives

$$\mathcal{P}_u(z_0,q_0) = \langle Bu_y, u_x^2\rangle + 20H(u_y, \mathbf{1}/d) \le 1 + 20\log d. \tag{349}$$

Sum over rounds, discard the nonnegative final potential and step divergences, and use the averaging argument in Theorem 650. This proves (346).

Rescaling multiplies gaps by L, so the chosen T gives the requested accuracy. Each round uses a constant number of products by A, B and their transposes, together with $O(n+d)$ scalar work. Computing L, initializing, and writing the output also take $O(\mathrm{nnz}(A)+n+d)$ work. This gives the running time; the $L = 0$ branch returns an exact saddle point. □

16.9 Positive Semidefinite Packing

For nonzero $A_1, \dots, A_n \succeq 0$ in $\mathbb{R}^{m\times m}$, primal–dual SDP duality gives

$$\tau = \max\{\mathbf{1}^\top w : w \ge 0,\ \textstyle\sum_i w_i A_i \preceq I\} = \min\{\mathrm{Tr}\, Y : Y \succeq 0,\ \langle A_i, Y\rangle \ge 1\ \forall i\},$$

where $\langle B, C\rangle = \mathrm{Tr}(BC)$. A zero A_i makes the primal unbounded. Delete coordinates on which all inputs vanish; for a PSD matrix, a zero diagonal entry forces the corresponding row and column to vanish. Thus, if $S = \sum_i \mathrm{nnz}(A_i)$ and U counts the distinct input positions, we may assume $m \le U \le S$. Put $r = \max_i \mathrm{rank}\, A_i$; throughout, one may replace r by m.

Theorem 656 (Positive SDP packing)**.** *For the normalized instance used below, suppose $w_0 > 0$ and $H \ge \log 2$ satisfy*

$$\sum_i w_{0,i} A_i \preceq I/2, \qquad w_{0,i}\|A_i\|_{\mathrm{op}} \ge e^{-H} \quad \forall i.$$

For $0 < \varepsilon \le 1/2$, with constant probability the algorithm returns primal–dual feasible y, Y such that

$$\mathbf{1}^\top y \le \tau \le \mathrm{Tr}\, Y = (1+\varepsilon)\mathbf{1}^\top y.$$

Writing $L_m = \log(2m)$ and $\Gamma = \log(2mH/\varepsilon)$, this takes $O(HL_m/\varepsilon^2)$ rounds and

$$O\left(\frac{SHL_m}{\varepsilon^2} + \frac{UHL_m^{3/2}\sqrt{\Gamma}}{\varepsilon^{7/2}}\right) \tag{350}$$

arithmetic work. The dual is represented as a sum of sampled outer products.

Write $P(w) = \sum_i w_i A_i$, $M(w) = \sum_i w_i$, and, for the odd p chosen below, $Q(w) = \operatorname{Tr} P(w)^p$. The proof is organized around the matrix gradient $P^{p-1} = \nabla_P(\operatorname{Tr} P^p/p)$: a one-sided polynomial filter approximates it through $q(P)^2$, Gaussian sketches estimate the resulting dual scores, and a stopping-time argument for Q both certifies the approximation range and controls the final primal–dual gap.

For normalization, compute $f_i = \|A_i\|_F$ in $O(S)$ work and scale all inputs by a common factor so that $\min_i f_i = m$. Since $A_i \succeq 0$, $\|A_i\|_{\mathrm{op}} \le f_i \le \operatorname{Tr} A_i$, and therefore

$$\frac{1}{m} \le \tau \le 1: \quad f_i = m \Longrightarrow A_i/m \preceq I, \qquad mM(w) \le \sum_i w_i f_i \le \operatorname{Tr} P(w) \le m \quad (P(w) \preceq I).$$

Moreover $\frac{1}{p}\,\partial_{w_i} Q(w) = \langle A_i, P(w)^{p-1}\rangle$ is both the marginal power penalty and the tentative ith dual constraint value.

Algorithm. Use any warm start w_0, H satisfying the theorem hypotheses. Without a supplied warm start, take $w_{0,i} = 1/(2nf_i)$ and $H = \log(2n\sqrt{r})$. Indeed, $A_i \preceq f_i I$ and $\|A_i\|_F \le \sqrt{r}\|A_i\|_{\mathrm{op}}$ give $P_0 \preceq I/2$ and $w_{0,i}\|A_i\|_{\mathrm{op}} \ge 1/(2n\sqrt{r}) = e^{-H}$.

Let $h = \varepsilon/C$ for a sufficiently large absolute constant C, and let p be the least odd integer at least L_m/h. Set

$$\ell = \lceil 1/h \rceil + 2, \qquad T = \left\lceil \frac{2pH}{h} \right\rceil, \qquad L = (T/h)^{1/p}.$$

Here p controls the Schatten proxy, ℓ the Gaussian sketch, T the weight-growth certificate, and L the analysis-only spectral threshold.

By Equation (311), choose a polynomial satisfying

$$0 \le q(x)^2 - x^{p-1} \le h/m^2 \qquad (0 \le x \le L). \tag{351}$$

Its degree is $d = O(\sqrt{p \log(m^2 L^p/h)})$; the lower side gives one-step progress and the upper side is used only in recovery.

Since $P_0 \preceq I/2$, $Q_0 \le m2^{-p} \le 1/(2m) \le \tau$ for sufficiently small h. For $t = 0, \dots, T-1$, draw independent $z_{t,1}, \dots, z_{t,\ell} \sim N(0, I)$ and compute

$$\begin{aligned} P_t = P(w_t), \qquad v_{t,j} = q(P_t) z_{t,j}, \qquad B_t = \frac{1}{\ell - 2} \sum_{j=1}^{\ell} v_{t,j} v_{t,j}^\top, \\ b_{t,i} = \langle A_i, B_t \rangle, \qquad w_{t+1,i} = w_{t,i} \exp\left(\frac{1 - b_{t,i}}{p} \right). \end{aligned} \tag{352}$$

Here B_t sketches P_t^{p-1} and $b_{t,i}$ is its ith dual score; scores below one increase the weight. The factor $1/(\ell - 2)$, rather than the unbiased $1/\ell$, is chosen so that the Gaussian Laplace bound exactly compensates this exponential update.

After the T rounds, choose J uniformly from $\{0, \dots, T-1\}$, independently of the samples, put $W = w_J$, and return

$$Y = \frac{1}{T(1-h)} \sum_{t<T} B_t, \qquad D = \operatorname{Tr} Y, \qquad y = \frac{DW}{(1+\varepsilon)M(W)}. \tag{353}$$

The averaged B_t's certify the dual, while the random iterate avoids a separate convexity argument for the primal. If the common normalization factor was c, multiply both returned objects by c to undo it.

One-step progress. Lieb–Thirring converts the noncommutative simultaneous update into a sum of coordinate contributions. Writing $P = P(w)$,

$$Q(w \odot s) \le \sum_i w_i s_i^p \langle A_i, P^{p-1}\rangle \qquad (s_i \ge 0). \tag{354}$$

Stack $X_i = (w_i A_i)^{1/2}$ into X and set $\mathcal{D} = \operatorname{diag}(s_1 I, \dots, s_n I)$. Then $X^\top X = P$ and $X^\top \mathcal{D} X = \sum_i w_i s_i A_i$, so Lieb–Thirring gives

$$\begin{aligned} Q(w \odot s) = \operatorname{Tr}(X^\top \mathcal{D} X)^p &\le \operatorname{Tr}\big(\mathcal{D}^p (X X^\top)^p\big) = \operatorname{Tr}\big(\mathcal{D}^p X P^{p-1} X^\top\big) \\ &= \sum_i w_i s_i^p \langle A_i, P^{p-1} \rangle. \end{aligned}$$

For the sampled scores, write $a_i = \langle A_i, P^{p-1} \rangle$ and $\widetilde{a}_i = \langle A_i, q(P)^2 \rangle \ge a_i$. If $G \succeq 0$ has trace a, eigenvalues λ_s, and $b = (\ell-2)^{-1} \sum_{j=1}^{\ell} z_j^\top G z_j$, then independence and $\prod_s (1+x_s) \ge 1 + \sum_s x_s$ give

$$\mathbb{E} e^{-b} = \prod_s \left(1 + \frac{2\lambda_s}{\ell - 2}\right)^{-\ell/2} \le \left(1 + \frac{2a}{\ell - 2}\right)^{-\ell/2}.$$

The case $a = 0$ is immediate. For $a > 0$, let $g(a) = ea(1 + 2a/(\ell-2))^{-\ell/2}$. Then $g'(a)/g(a) = (\ell-2)(1-a)/(a(\ell - 2 + 2a))$, so g is maximized at $a = 1$. Indeed, using $\log(1+x) \ge x/(1+x)$, $(\ell/2)\log(\ell/(\ell-2)) = (\ell/2)\log(1 + 2/(\ell-2)) \ge (\ell/2)\frac{2/(\ell-2)}{1+2/(\ell-2)} = 1$, so $g(1) = e(\ell/(\ell-2))^{-\ell/2} \le 1$. Hence

$$ea\, \mathbb{E} e^{-b} \le 1. \tag{355}$$

This is the scalar identity that compensates the exponential weight growth.

Let $\mathbb{E}_t$ condition on the past. Whenever $Q_t \le L^p$, we have $\|P_t\|_{\mathrm{op}} \le L$, so Equation (351) applies. For $G_i = q(P_t) A_i q(P_t)$,

$$b_{t,i} = \frac{1}{\ell - 2} \sum_{j=1}^{\ell} z_{t,j}^\top G_i z_{t,j}, \qquad \operatorname{Tr} G_i = \widetilde{a}_i = \langle A_i, q(P_t)^2 \rangle \ge a_i.$$

Applying Equations (354) and (355) with $s_i = \exp((1 - b_{t,i})/p)$, hence $s_i^p = e^{1 - b_{t,i}}$, gives

$$\mathbb{E}_t Q_{t+1} \le \sum_i w_{t,i} a_i e\, \mathbb{E}_t e^{-b_{t,i}} \le \sum_i w_{t,i} \frac{a_i}{\widetilde{a}_i} \le \sum_i w_{t,i} = M_t. \tag{356}$$

If $\widetilde{a}_i = 0$, then $a_i = 0$ by the filter lower bound, so that term is omitted before taking the ratio.

The stopped potential. Set $R(w) = M(w)/Q(w)^{1/p}$, write $R_t = R(w_t)$, and put $\alpha = 1 - 1/p$. This $R(w)$ is an explicit feasible primal value: since $\|P(w)\|_{\mathrm{op}} \le Q(w)^{1/p}$, the vector $w/Q(w)^{1/p}$ is feasible, so $R(w) \le \tau \le 1$. Applying $x \le \alpha + x^p/p$ to $x = Q(w)^{1/p}$ gives

$$\begin{aligned} M(w) = R(w)x &\le \alpha R(w) + R(w)x^p/p = \alpha R(w) + R(w)Q(w)/p \\ &\le \alpha R(w) + Q(w)/p. \end{aligned}$$

Equivalently,

$$Q(w) - M(w) \ge \alpha\big(Q(w) - R(w)\big) \ge \alpha\big(Q(w) - \tau\big). \tag{357}$$

The algorithm never tests the filter range. For analysis let σ be the first $t \le T$ with $Q_t > L^p$ (or ∞), and set $\mathcal{E} = \{\sigma > T\}$. Before σ, Equation (356) holds, and $Q_{(t+1)\wedge\sigma} - Q_{t\wedge\sigma}$ equals $Q_{t+1} - Q_t$ on $\{t < \sigma\}$ and vanishes otherwise,

$$\mathbb{E}\big[Q_{(t+1)\wedge\sigma} - Q_{t\wedge\sigma}\big] \le -\mathbb{E}\big[\mathbf{1}_{\{t<\sigma\}}(Q_t - M_t)\big].$$

Summing over $t < T$ gives the balance inequality used twice below:

$$\frac{\mathbb{E} Q_{T\wedge\sigma}}{T} + \frac{1}{T}\mathbb{E}\sum_{t<T} \mathbf{1}_{\{t<\sigma\}}(Q_t - M_t) \le \frac{Q_0}{T}. \tag{358}$$

By Equation (357), $Q_t - M_t \ge -\alpha\tau$, so $\mathbb{E} Q_{T\wedge\sigma}/T \le Q_0/T + \alpha\tau = O(\tau)$. If an exit occurs then $Q_{T\wedge\sigma} > L^p = T/h$, and therefore

$$\Pr(\mathcal{E}^c) \le \frac{h}{T}\, \mathbb{E} Q_{T\wedge\sigma} = O(h\tau) = O(h). \tag{359}$$

Thus the filter is valid throughout with probability $1 - O(h)$. We retain the second term in Equation (358) because it will pay for the final gap.

Primal–dual recovery. On $\mathcal{E}$, $Q_T \le L^p$, so $w_{T,i}\|A_i\|_{\mathrm{op}} \le \|P_T\|_{\mathrm{op}} \le L$ and therefore

$$\frac{w_{T,i}}{w_{0,i}} \le \frac{L}{w_{0,i}\|A_i\|_{\mathrm{op}}} \le e^H L.$$

Also $T/h \le 3pH/h^2 \le e^{pH}$ for sufficiently small h (since $pH = \Omega(1/h)$), so $\log L = p^{-1}\log(T/h) \le H$ and $\log(w_{T,i}/w_{0,i}) \le 2H$. Telescoping the update gives

$$\log \frac{w_{T,i}}{w_{0,i}} = \frac{1}{p}\sum_{t<T}(1-b_{t,i}), \qquad \frac{1}{T}\sum_{t<T} b_{t,i} = 1 - \frac{p}{T}\log\frac{w_{T,i}}{w_{0,i}} \ge 1 - \frac{2pH}{T} \ge 1 - h.$$

Consequently Y in Equation (353) is dual-feasible.

For the cost comparison, the upper filter bound enters here. For $t < \sigma$, Hölder on the eigenvalues λ_j of P_t, followed by $u^\alpha \le \alpha u + 1/p$, gives in one chain

$$\begin{aligned}\operatorname{Tr} P_t^{p-1} = \sum_j \lambda_j^{p-1} &\le m^{1/p}\left(\sum_j \lambda_j^p\right)^{(p-1)/p} = m^{1/p} Q_t^\alpha \\ &= m^{2/p-1}(mQ_t)^\alpha \le m^{2/p-1}\left(\alpha m Q_t + \frac{1}{p}\right) = m^{2/p}\left(\alpha Q_t + \frac{1}{mp}\right).\end{aligned}$$

Also $\mathbb{E}_t \operatorname{Tr} B_t = (\ell/(\ell-2))\operatorname{Tr} q(P_t)^2 \le (\ell/(\ell-2))(\operatorname{Tr} P_t^{p-1} + h/m)$. Since $m^{2/p} \le e^{2h}$, $\ell/(\ell-2) = 1 + O(h)$, and $1/(mp) \le h/m \le h\tau$, for an absolute constant c,

$$\frac{\mathbb{E}_t \operatorname{Tr} B_t}{1-h} \le (1+ch)\alpha Q_t + ch\tau.$$

By Equation (357),

$$\alpha Q_t - R_t = \alpha(Q_t - R_t) - \frac{R_t}{p} \le Q_t - M_t, \qquad \alpha Q_t \le Q_t - M_t + \alpha\tau.$$

Therefore, enlarging the absolute constant c once,

$$\begin{aligned}\frac{\mathbb{E}_t \operatorname{Tr} B_t}{1-h} - R_t &\le (\alpha Q_t - R_t) + ch\,\alpha Q_t + ch\tau \\ &\le (1+ch)(Q_t - M_t) + ch\tau. \end{aligned} \tag{360}$$

This is exactly the second term retained in Equation (358).

The event $\{t < \sigma\}$ is fixed before B_t is sampled, so Equation (360) may be conditioned on it. Replacing $\mathbf{1}_{\mathcal{E}}$ by $\mathbf{1}_{\{t<\sigma\}}$ can only lose the negative term $-R_t$, whose magnitude is at most τ. Since J is uniform and independent,

$$\begin{aligned}\mathbb{E}[\mathbf{1}_{\mathcal{E}}(D - R(W))] &= \frac{1}{T}\sum_{t<T}\mathbb{E}\left[\mathbf{1}_{\mathcal{E}}\left(\frac{\operatorname{Tr} B_t}{1-h} - R_t\right)\right] \\ &\le \frac{1}{T}\sum_{t<T}\mathbb{E}\left[\mathbf{1}_{\{t<\sigma\}}\left(\frac{\operatorname{Tr} B_t}{1-h} - R_t\right)\right] + \tau\Pr(\mathcal{E}^c) \\ &\le \frac{1+ch}{T}\mathbb{E}\sum_{t<T}\mathbf{1}_{\{t<\sigma\}}(Q_t - M_t) + ch\tau + \tau\Pr(\mathcal{E}^c) \\ &\le (1+ch)Q_0/T + O(h\tau) = O(h\tau),\end{aligned}$$

where the last line uses Equations (358) and (359) and $Q_0/T \le h\tau$. On $\mathcal{E}$, weak duality gives $D \ge \tau \ge R(W)$, so the random variable $\mathbf{1}_{\mathcal{E}}(D - R(W))$ is nonnegative. Markov's inequality together with Equation (359)

therefore implies that, for a sufficiently large absolute constant K, with constant probability $D - R(W) \le Kh\tau \le KhD$. Choosing $C \ge 2K$ gives $D/R(W) \le (1-Kh)^{-1} \le 1 + 2Kh \le 1+\varepsilon$, so the returned vector is primal-feasible:

$$\|P(y)\|_{\mathrm{op}} \le \frac{D\,Q(W)^{1/p}}{(1+\varepsilon)M(W)} = \frac{D}{(1+\varepsilon)R(W)} \le 1,$$

and by construction $\operatorname{Tr} Y = (1+\varepsilon)\mathbf{1}^\top y$.

Implementation and work. Neither P_t^{p-1} nor B_t is formed densely. By Lemma 590, q costs $O(p)$ work to construct, and the Chebyshev recurrence from Section 15.4 evaluates $q(P_t)z$ with $O(d)$ matrix–vector products and $O(md) = O(Ud)$ other work. The f_i cost $O(S)$, absorbed in (350).

Precompute the union of input positions. Each P_t costs $O(S)$ to assemble, and filtering the ℓ probes costs $O(\ell dU)$. Compute

$$(B_t)_{ab} = \frac{1}{\ell - 2}\sum_j (v_{t,j})_a (v_{t,j})_b$$

only at the U input positions, in $O(\ell U)$ work; reusing them, all $\langle A_i, B_t\rangle$ cost $O(S)$. Store the $v_{t,j}$ to represent the dual and sum their squared norms for its trace. Scalar updates cost $O(n)$, absorbed by S, so a round costs $O(S + \ell dU)$.

Finally, $p = \Theta(L_m/h)$ and $T = O(HL_m/h^2)$. Also $\log(m^2L^p/h) = \log(Tm^2/h^2) = O(\Gamma)$, so $\ell d = O(h^{-3/2}\sqrt{L_m\Gamma})$. Multiplying by T gives

$$O\left(\frac{SHL_m}{h^2} + \frac{UHL_m^{3/2}\sqrt{\Gamma}}{h^{7/2}}\right),$$

which is (350) because $h = \Theta(\varepsilon)$.

16.10 Optimal Online Discrepancy Minimization

Suppose vectors $v_1, \dots, v_T \in \mathbb{R}^d$ with $\|v_t\|_2 \le 1$ arrive one at a time. We must choose $\varepsilon_t \in \{\pm 1\}$ before seeing the next vector, and wish to minimize the largest prefix discrepancy $\max_{t \le T} \|S_t\|_\infty$, where $S_t = \sum_{s \le t} \varepsilon_s v_s$. The sequence is fixed in advance (the adversary is *oblivious*). We will obtain the optimal $O(\sqrt{\log(2T)})$ bound in linear time.

The idea is to keep three auxiliary vectors $G_{t,1}, G_{t,2}, G_{t,3}$ such that each is marginally $N(0, I_d)$ and $S_t = G_{t,1} + G_{t,2} + G_{t,3}$. They need not be independent: Hölder's inequality already makes their sum 3-subgaussian. Upon receiving v_t, each auxiliary vector will move by $\delta_{t,j} v_t$ for some $\delta_{t,j} \in \{-1, 0, 1\}$. The problem is to preserve its Gaussian marginal while coupling the three moves so that $\sum_j \delta_{t,j} \in \{\pm 1\}$.

16.10.1 A Gaussian-Preserving Step

Write γ for the density of $N(0, I_d)$. We want to move a Gaussian point by $\pm v$ while preserving its distribution. For a move from x to $x + v$, balance requires the probability flow in each direction to agree: $\gamma(x)p_v^+(x) = \gamma(x+v)p_v^-(x+v)$. A natural choice is to make this common flow $\frac{1}{3}\min\{\gamma(x), \gamma(x+v)\}$. Thus define

$$\begin{aligned} p_v^+(x) &= \tfrac{1}{3}\min\left\{1, \frac{\gamma(x+v)}{\gamma(x)}\right\} = \tfrac{1}{3}\min\{1, e^{-\langle x, v\rangle - \|v\|_2^2/2}\}, \\ p_v^-(x) &= \tfrac{1}{3}\min\left\{1, \frac{\gamma(x-v)}{\gamma(x)}\right\} = \tfrac{1}{3}\min\{1, e^{\langle x, v\rangle - \|v\|_2^2/2}\}, \\ p_v^0(x) &= 1 - p_v^+(x) - p_v^-(x). \end{aligned} \tag{361}$$

The factor $1/3$ ensures that both move probabilities are at most $1/3$, as needed for coupling three such steps. Given $G \sim N(0, I_d)$, sample $\delta \in \{-1, 0, 1\}$ with probabilities $p_v^-(G), p_v^0(G), p_v^+(G)$, respectively.

Lemma 657 (Balanced Gaussian step)**.** *The update $G + \delta v$ is again $N(0, I_d)$. If $\|v\|_2 \le 1$, then, for every x,*

$$0 \le p_v^+(x), p_v^-(x) \le \tfrac{1}{3}, \qquad p_v^+(x) + p_v^-(x) \ge \tfrac{1}{3}. \tag{362}$$

Proof. By construction,

$$\gamma(x)p_v^+(x) = \tfrac{1}{3}\min\{\gamma(x), \gamma(x+v)\} = \gamma(x+v)p_v^-(x+v).$$

Hence the probability density arriving at x is

$$\gamma(x-v)p_v^+(x-v) + \gamma(x+v)p_v^-(x+v) + \gamma(x)p_v^0(x) = \gamma(x),$$

proving stationarity.

The upper bounds are immediate. For the lower bound, put $z = \langle x, v\rangle$ and $r = \|v\|_2^2/2 \le 1/2$. If $|z| \ge r$, one of the two move probabilities is $1/3$. Otherwise neither exponential is clipped, and

$$p_v^+(x) + p_v^-(x) = \tfrac{2}{3}e^{-r}\cosh z \ge \tfrac{2}{3}e^{-1/2} > \tfrac{1}{3}.$$

□

16.10.2 Coupling Three Moves into One Sign

We have three distributions on $\{-1, 0, 1\}$, each assigning probability at most $1/3$ to either sign and at least $1/3$ to moving at all. We want to preserve these marginals while making the sum of the three moves equal ± 1.

Such an outcome has either exactly one nonzero coordinate, or all three nonzero with both signs present. We will first choose which coordinates move, then couple their signs.

Lemma 658 (Three-way coupling)**.** *Suppose $0 \le a_j, b_j \le 1/3$ and $a_j + b_j \ge 1/3$ for $j \in [3]$. In constant time one can sample $\delta \in \{-1, 0, 1\}^3$ satisfying*

$$\Pr(\delta_j = 1) = a_j, \qquad \Pr(\delta_j = -1) = b_j, \qquad \sum_{j=1}^{3} \delta_j \in \{-1, 1\}.$$

Proof. Put $s_j = a_j + b_j$, the required probability that coordinate j moves. Let τ be the probability that all three move. Since every other outcome has exactly one moving coordinate, necessarily

$$\sum_j s_j = 3\tau + (1 - \tau) = 1 + 2\tau, \qquad \tau = \frac{\sum_j s_j - 1}{2}.$$

As $1/3 \le s_j \le 2/3$, we have $0 \le \tau \le 1/2$ and $2\tau \le s_j + 1/3 \le 2s_j$. Thus we can choose all three coordinates with probability τ, and only coordinate j with probability $s_j - \tau$. These probabilities are nonnegative and sum to one.

It remains to assign signs. Assume $\tau > 0$, and let q_j be the conditional probability that coordinate j is positive when all three move. To obtain the desired marginals, the singleton outcomes $+e_j$ and $-e_j$ must then have unconditional probabilities

$$a_j - \tau q_j, \qquad b_j - \tau(1 - q_j).$$

Their nonnegativity is equivalent to

$$\ell_j := \max\{0, 1 - b_j/\tau\} \le q_j \le u_j := \min\{1, a_j/\tau\}. \tag{363}$$

We also need $1 \le \sum_j q_j \le 2$: then the all-three case can have either one or two positive signs, but never zero or three.

These requirements are compatible. First, $\ell_j \le u_j$ because $\tau \le a_j + b_j$. Next, writing $A = \sum_j a_j$ and $B = \sum_j b_j$, we have $A, B \le 1$ and $A + B = 1 + 2\tau$, so $A, B \ge 2\tau$. Consequently,

$$\sum_j u_j = \sum_j \min\{1, a_j/\tau\} \ge 1, \qquad \sum_j \ell_j = 3 - \sum_j \min\{1, b_j/\tau\} \le 2.$$

Choose $q_j = \ell_j$ initially. If their sum is below one, increase the coordinates, without exceeding u_j, until the sum reaches one. This is possible by the preceding inequalities, and gives $1 \le Q := \sum_j q_j \le 2$.

We can now sample the signs in the all-three case by a single uniform random variable. Draw $U \sim \mathrm{Unif}[0, 1)$ and set

$$c_0 = 0, \qquad c_j = \sum_{i \le j} q_i, \qquad Z_j = \lfloor U + c_j \rfloor - \lfloor U + c_{j-1} \rfloor, \qquad \delta_j = 2Z_j - 1. \tag{364}$$

Since $0 \le q_j \le 1$, each $Z_j \in \{0, 1\}$. Moreover, $\mathbb{E}\lfloor U + c \rfloor = c$ for every $c \ge 0$, so $\Pr(Z_j = 1) = \mathbb{E}Z_j = q_j$. Yet the total number of positive signs is always

$$\sum_j Z_j = \lfloor U + Q \rfloor \in \{1, 2\},$$

and therefore $\sum_j \delta_j = 2 \sum_j Z_j - 3 \in \{-1, 1\}$.

Finally, sample this all-three outcome with probability τ. Otherwise sample a singleton $\pm e_j$, assigning conditional probabilities

$$\frac{a_j - \tau q_j}{1 - \tau}, \qquad \frac{b_j - \tau(1 - q_j)}{1 - \tau}$$

to $+e_j$ and $-e_j$, respectively. These are nonnegative and sum to one, and the resulting marginal probabilities are exactly a_j, b_j. If $\tau = 0$, simply sample the singleton outcomes with probabilities a_j, b_j. □

The rounding in (364) preserves each marginal while forcing the number of positive signs to be one or two. Thus the entire coupling requires only scalar arithmetic and one uniform random variable.

16.10.3 Algorithm and Concentration

To start with three standard Gaussian marginals whose sum vanishes, sample independent $H_1, H_2, H_3 \sim N(0, I_d)$ and set

$$\bar{H} = \tfrac{1}{3} \sum_{j=1}^{3} H_j, \qquad G_{0,j} = \sqrt{\tfrac{3}{2}}\,(H_j - \bar{H}). \tag{365}$$

Indeed, $\sum_j G_{0,j} = 0$ and $\mathrm{Cov}(H_j - \bar{H}) = \frac{2}{3} I_d$, so every $G_{0,j} \sim N(0, I_d)$. The shared initialization is deliberately correlated; independence of the three walks is neither needed nor preserved.

Algorithm 91 Online Gaussian triplet walk

Input: Online vectors $v_1, \dots, v_T \in \mathbb{R}^d$ with $\|v_t\|_2 \le 1$
Output: Signs $\varepsilon_1, \dots, \varepsilon_T \in \{\pm 1\}$

```
Initialize G_1, G_2, G_3 by (365)
for t = 1, ..., T do
    Receive v_t; set a_j ← p^+_{v_t}(G_j) and b_j ← p^-_{v_t}(G_j) for j ∈ [3]
    (δ_1, δ_2, δ_3) ← ThreeWayCoupling((a_j, b_j)_{j=1}^3)        ▷ Lemma 658
    ε_t ← δ_1 + δ_2 + δ_3
    G_j ← G_j + δ_j v_t for j ∈ [3]
return (ε_1, ..., ε_T)
```

Theorem 659 (Optimal online discrepancy in linear time)**.** *For any fixed sequence of $T \geq 1$ vectors of Euclidean norm at most one, Algorithm 91 satisfies, for $0 < \delta \leq 1/2$,*

$$\Pr\left[\max_{t\leq T}\left\|\sum_{s\leq t}\varepsilon_s v_s\right\|_\infty \leq \sqrt{18\log\frac{2T\min\{d,T\}}{\delta}}\right] \geq 1-\delta. \tag{366}$$

Its running time is $O(d+T+\sum_t \operatorname{nnz}(v_t))$ with sparse vector representations, and $O(dT)$ for dense vectors.

Proof. By Lemma 658, each update gives a genuine sign $\varepsilon_t \in \{\pm 1\}$. It also gives the prescribed conditional marginals $p^{\pm}_{v_t}(G_{t-1,j})$ for each $\delta_{t,j}$, even though the three increments are coupled. Inductively, Lemma 657 therefore ensures $G_{t,j} \sim N(0, I_d)$ for all t, j. Since their initial sum is zero and their increments sum to $\varepsilon_t v_t$,

$$S_t = \sum_{s\leq t}\varepsilon_s v_s = \sum_{j=1}^{3} G_{t,j}. \tag{367}$$

Hölder's inequality and the Gaussian moment-generating function give, without requiring independence,

$$\mathbb{E}e^{\langle\theta,S_t\rangle} \leq \prod_{j=1}^{3}\left(\mathbb{E}e^{3\langle\theta,G_{t,j}\rangle}\right)^{1/3} = e^{9\|\theta\|_2^2/2}. \tag{368}$$

A direct union bound over coordinates would introduce $\log d$. To avoid it, let P_t be the orthogonal projection onto $E_t = \operatorname{span}\{v_1,\dots,v_t\}$, of dimension $r_t \leq \min\{d,t\}$. Since $S_t \in E_t$, we can replace θ by $P_t\theta$ in (368). Put $h_i = \|P_t e_i\|_2^2 \in [0,1]$. The Gaussian tail bound gives

$$\Pr(|(S_t)_i| > u) \leq 2\exp\left(-\frac{u^2}{18h_i}\right),$$

where the right side is zero when $h_i = 0$. For $u^2 \geq 18$ and $0 < h_i \leq 1$, the inequality $1/h_i - 1 \geq \log(1/h_i)$ implies $e^{-u^2/(18h_i)} \leq h_i e^{-u^2/18}$. Therefore

$$\Pr(\|S_t\|_\infty > u) \leq 2e^{-u^2/18}\sum_{i=1}^{d} h_i = 2r_t e^{-u^2/18}. \tag{369}$$

The projection is used only in the analysis: the apparent dependence on d has become the rank of the vectors seen so far. Union bounding over prefixes yields

$$\Pr\big(\max_{t\leq T}\|S_t\|_\infty > u\big) \leq 2\sum_{t\leq T} r_t\, e^{-u^2/18} \leq 2T\min\{d,T\}e^{-u^2/18}.$$

The choice of u in (366) is at least $\sqrt{18}$ and makes the last expression δ.

Initialization costs $O(d)$; each round uses three inner products, a norm calculation, three vector updates, and the constant-time coupling. Processing a sparse v_t touches only its nonzero coordinates, proving the time bounds. □

Offline consequence. For a matrix $A \in \mathbb{R}^{d\times T}$ whose columns have Euclidean norm at most one, apply the algorithm to its columns and retain the final sum. The single-prefix version of (369) gives, with probability at least $1-\delta$,

$$\|A\varepsilon\|_\infty \leq \sqrt{18\log\frac{2\min\{d,T\}}{\delta}}.$$

The online guarantee applies to obliviously fixed sequences; Gaussian marginal stationarity need not persist against an adversary that chooses future vectors from previous signs.

17 Polynomial Algorithms

Coefficient spaces and precision. Coefficients lie in a field K, except where a commutative ring is specified. The space $K[x]_{<n}$ is represented by arrays of length n, including trailing zeros. Precision modulo x^n means that coefficients $0, \dots, n-1$ are known. Write $F_{<n} = F \bmod x^n$, $F_{\geq n} = F - F_{<n}$, and

$$\operatorname{rev}_d F = x^d F(x^{-1}) \qquad (\deg F \leq d).$$

Reversal always uses the stated degree bound, with zero padding when necessary. We set $\deg 0 = -\infty$ and interpret out-of-range coefficients as zero in each specified coefficient space.

Complexities count coefficient operations unless a bit bound is stated. Division requires the indicated denominator to be a unit. Logarithms, exponentials, and factorial-based transforms require their integer divisors to be invertible; characteristic zero or characteristic larger than the working precision suffices. Results over small-characteristic fields or general rings state their assumptions separately.

17.1 Formal Power Series Operations

Let $D = d/dx$ be the formal derivative. If an error is $E = O(x^m)$, then $E^2 = O(x^{2m})$. Newton updates cancel the linear error, thereby doubling precision. A constant number of products at each doubling costs $\sum_{m=1,2,4,\dots<N} O(m \log(m+2)) = O(N \log(N+2))$. Truncate the last update to the requested precision.

17.1.1 Inverse

Let $A(0) \neq 0$, and suppose $AF \equiv 1 \pmod{x^m}$. Writing $E = 1 - AF$, the update $F_{\text{new}} = F(1+E)$ gives $AF_{\text{new}} = (1-E)(1+E) = 1 - E^2$. Therefore

$$\boxed{F_{\text{new}} = F(2 - AF) \bmod x^{2m}.}$$

Starting from $F = A(0)^{-1}$, these updates compute the inverse in $O(N \log(N+2))$ operations. If $AF = AG = 1$, multiplication by either inverse gives uniqueness.

Only the upper half changes. If $AF = 1 + x^m H \pmod{x^{2m}}$, then $F_{\text{new}} = F - x^m(FH \bmod x^m)$. To avoid a separate negation at each step, maintain the negative inverse $R = -F$. With $H = (AR+1)/x^m \bmod x^m$, its update is $R_{\text{new}} = R + x^m(RH \bmod x^m)$.

Algorithm 92 Formal power-series inverse

Input: $A \bmod x^N$, $A(0) \neq 0$, $N \geq 1$
Output: $A^{-1} \bmod x^N$

```
F ← A(0)^{-1}, m ← 1
while m < N do
    r ← min{2m, N}
    F ← F(2 − A_{<r} F) mod x^r
    m ← r
return F
```

17.1.2 Logarithm

For a unit A, its normalized logarithm is $\log(A/A(0))$, with constant coefficient zero. Its derivative is A'/A, so

$$\boxed{\log \frac{A}{A(0)} = \int_0^x \frac{A'(t)}{A(t)}\, dt.}$$

To obtain precision x^N, compute the integrand modulo x^{N-1}. One inverse and one product give $O(N \log(N+2))$ operations. No scalar logarithm of $A(0)$ is needed. The notation $\log A$ below is used when $A(0) = 1$.

17.1.3 Square Root

Assume 2 is invertible and choose $f_0 \neq 0$ with $f_0^2 = A(0)$. There is a unique square root with constant coefficient f_0. Maintain $F^2 \equiv A \pmod{x^m}$ and $G \equiv F^{-1} \pmod{x^m}$. Since $F^2 - A = O(x^m)$, the known precision of G already suffices for the correction

$$\boxed{F_{\text{new}} = F - \tfrac{1}{2}G(F^2 - A) \bmod x^{2m}, \qquad G_{\text{new}} = G(2 - F_{\text{new}}G) \bmod x^{2m}.}$$

The linear term in $F_{\text{new}}^2 - A$ cancels the residual; the squared correction and the inverse-approximation error start in degree $2m$. The second update lifts the inverse. Starting from $(F, G) = (f_0, f_0^{-1})$ gives total cost $O(N \log(N+2))$. Both corrections are confined to the upper half.

For a nonzero series $A = x^v U$ with $U(0) \neq 0$, a square root exists exactly when $v = 2r$ is even and $U(0)$ has a square root; it is $x^r\sqrt{U}$. Precision must be shifted as well: to determine this root modulo x^N for $N > r$, one needs $U \bmod x^{N-r}$, that is, $A \bmod x^{N+r}$. Knowing only $A \bmod x^N$, with $2r < N$, determines the chosen root only modulo x^{N-r}. Its remaining coefficients do not affect the square modulo x^N. A positive-valuation root has no power-series inverse. For a fixed unit root coefficient, uniqueness follows from $(F-G)(F+G) = 0$, since $F + G$ has unit constant coefficient.

17.1.4 Inverse Square Root

Assume two is invertible. For a unit A, maintain $AF^2 \equiv 1 \pmod{x^m}$, starting from a chosen $F(0)$ with $A(0)F(0)^2 = 1$. The update is

$$\boxed{F_{\text{new}} = \tfrac{1}{2}F(3 - AF^2) \bmod x^{2m}.}$$

If $AF^2 = 1 - E$, then $AF_{\text{new}}^2 = (1-E)(1+E/2)^2 = 1 + O(E^2)$. Thus the same doubling analysis gives $O(N \log(N+2))$ operations, without maintaining a separate inverse.

17.1.5 Exponential

Assume $A(0) = 0$. The equation $\log F = A$ gives the Newton update $F_{\text{new}} = F(1 + A - \log F)$. To evaluate the correction without first extending the inverse of the old F, use $\theta = xD$, whose inverse on zero-constant series divides the coefficient of x^j by j. Unlike D, θ does not lower order.

Suppose $F \equiv \exp A \pmod{x^m}$, $G \equiv F^{-1} \pmod{x^m}$, and $\deg F, \deg G < m$. The residual $V = (\theta A_{<m})F - \theta F$ has order at least m. Since $\deg \theta F < m$, it is obtained through degree $2m-1$ by keeping only the upper half of $(\theta A_{<m})F$. Now $\theta \log F = \theta A_{<m} - V/F$, and the known inverse already gives $V/F \equiv VG \pmod{x^{2m}}$. Consequently

$$\log F \equiv A_{<m} - \theta^{-1}(VG) \pmod{x^{2m}}.$$

Set $C = A_{<2m} - A_{<m} + \theta^{-1}(VG)$. Then $C = O(x^m)$ and $\exp C \equiv 1 + C \pmod{x^{2m}}$, giving

$$\boxed{F_{\text{new}} = F(1 + C) \bmod x^{2m}, \qquad G_{\text{new}} = G(2 - F_{\text{new}}G) \bmod x^{2m}.}$$

The old inverse remains valid for the new F through its first m coefficients, so one inverse update suffices. A constant number of products per level gives $O(N \log(N+2))$ operations.

Algorithm 93 Formal power-series exponential

Input: $A \bmod x^N$, $A(0) = 0$, $N \geq 1$
Output: $\exp A \bmod x^N$

$F \leftarrow 1$, $G \leftarrow 1$, $m \leftarrow 1$
while $m < N$ **do**
  $r \leftarrow \min\{2m, N\}$
  $V \leftarrow ((\theta A_{<m})F)_{\geq m} \bmod x^r$
  $C \leftarrow A_{<r} - A_{<m} + \theta^{-1}(VG \bmod x^r)$
  $F \leftarrow F(1 + C) \bmod x^r$
  $G \leftarrow G(2 - FG) \bmod x^r$
  $m \leftarrow r$
return F

17.1.6 Powers

For a unit F, a scalar α, and a prescribed constant c, define $H = c\exp(\alpha \log(F/F(0)))$. The constant is supplied separately, since $F(0)^\alpha$ need not be defined in the coefficient field. It costs $O(N \log(N+2))$ operations through precision x^N. For an integer exponent k, take $c = F(0)^k$; obtaining that scalar adds $O(\log(|k| + 2))$ operations. For a nonnegative integer power of $F = x^v U$, first remove the valuation: $F^k = x^{vk}U^k$, and return zero modulo x^N if $vk \geq N$. The zeroth power is one, including for zero input.

A direct recurrence is preferable for low-degree input. Writing $F = \sum f_j x^j$, $H = \sum h_j x^j$, the identity $F\theta H = \alpha H \theta F$ gives

$$\boxed{n f_0 h_n = \sum_{j=1}^{n} \big((\alpha + 1)j - n\big) f_j h_{n-j}, \qquad h_0 = c.}$$

Indeed, isolate the $j = 0$ term on the left and combine the remaining convolution terms. If $\deg F = d$, the sum stops at $\min\{d, n\}$, giving $O(N(d + 1))$ operations.

17.2 Polynomial Arithmetic

17.2.1 Division

Let $A = QB + R$, where $B \neq 0$ and $\deg R < \deg B = m$. For $n = \deg A \geq m$, put $d = n - m$. Reversing with these specified bounds gives

$$\operatorname{rev}_n A = \operatorname{rev}_d Q \operatorname{rev}_m B + x^{d+1} \operatorname{rev}_{m-1} R.$$

If $m = 0$, the remainder is zero and the last term is omitted. The reversed divisor has nonzero constant coefficient, so

$$\boxed{Q = \operatorname{rev}_d \left((\operatorname{rev}_n A)(\operatorname{rev}_m B)^{-1} \bmod x^{d+1}\right), \qquad R = A - QB.}$$

This reduces division to series inversion and multiplication, costing $O((n + 1)\log(n + 2))$ operations. Over a commutative ring the same argument works when the leading coefficient of B is a unit; in particular, it works for every monic divisor.

Algorithm 94 Polynomial division

```
function DivMod(A, B)
    Require B ≠ 0
    if A = 0 or deg A < deg B then
        return (0, A)
    n ← deg A, m ← deg B, d ← n − m
    U ← rev_n A mod x^{d+1}
    V ← rev_m B mod x^{d+1}
    Q ← rev_d(U V^{-1} mod x^{d+1})
    return (Q, A − QB)
```

17.2.2 Taylor Shift

For $P(x) = \sum_{i=0}^{d} p_i x^i$, coefficient extraction gives

$$[x^j]P(x+c) = \frac{1}{j!}\sum_{i=j}^{d}(p_i i!)\frac{c^{i-j}}{(i-j)!}.$$

Reverse the factorial-scaled coefficients and convolve with $(c^i/i!)$. With $A(x) = \sum_{i=0}^{d} p_i i! x^{d-i}$ and $B(x) = \sum_{i=0}^{d} c^i x^i/i!$, the answer is $\sum_{j=0}^{d}[x^{d-j}](AB)x^j/j!$. Thus a Taylor shift costs $O((d+1)\log(d+2))$ operations when $1, \ldots, d$ are invertible. Generate powers of c, factorials, and inverse factorials successively in linear time.

Algorithm 95 Taylor shift

```
function TaylorShift(P, c, d)
    Require deg P ≤ d, with the integers 1, ..., d invertible
    A ← Σ_{i=0}^{d} p_i i! x^{d−i}, B ← Σ_{i=0}^{d} c^i x^i/i!
    C ← AB mod x^{d+1}
    return Σ_{j=0}^{d} C_{d−j} x^j/j!
```

A division-free shift. Over an arbitrary commutative ring, use a balanced coefficient split $P = P_0 + x^h P_1$. Then $P(x+c) = P_0(x+c) + (x+c)^h P_1(x+c)$. Pad the input length to a power of two and precompute $(x+c)^{2^j}$ by repeated squaring. Recursing on the two coefficient halves gives $O((d+1)\log^2(d+2))$ operations, with no factorial inverses. At each depth the total coefficient length is $O(d+1)$. This shift applies in small characteristic and over rings with nilpotents.

17.2.3 Evaluation Shift

Suppose $0, 1, \ldots, d$ are distinct in the coefficient field and we know $f(0), \ldots, f(d)$, where $\deg f \le d$. We want $f(s), \ldots, f(s+d)$ without first interpolating the coefficients.

Lemma 660 (Polynomial evaluation shift)**.** *These values can be computed in $O((d+1)\log(d+2))$ operations. The target points may overlap the original points.*

Proof. Let $P(x) = \prod_{j=0}^{d}(x-j)$. Lagrange interpolation gives

$$f(x) = P(x)\sum_{i=0}^{d}\frac{a_i}{x-i}, \qquad a_i = \frac{(-1)^{d-i}f(i)}{i!(d-i)!}.$$

Define $b_j = (s-d+j)^{-1}$ for $0 \le j \le 2d$, except that $b_j = 0$ when its denominator is zero. One convolution computes $v_k = \sum_{i=0}^{d} a_i b_{d+k-i}$ for $0 \le k \le d$. For a target outside $\{0, \ldots, d\}$, its value is $P(s+k)v_k$. An

overlapping target is copied directly from the input; the artificial zero entries of b affect only such targets. For a zero slot j, the overlapping targets are given by $i = d + k - j \in [0, d]$. There are at most two such slots, since the characteristic is zero or exceeds d; all copies take linear time.

It remains to compute all $P(s + k)$ without dividing by a zero. They are products of consecutive windows of length $d + 1$ in $s - d, s - d + 1, \ldots, s + d$. Maintain the product of the nonzero entries and the number of zero entries in the current window. Delete the outgoing nonzero factor by division and multiply the incoming nonzero factor; update the zero count separately. The window product is zero exactly when this count is positive. The window updates take $O(d + 1)$ operations, including in positive characteristic. □

For known values at $a, a + h, \ldots, a + dh$, with $h \neq 0$, apply the lemma to $g(x) = f(a + hx)$; translating the points by δ corresponds to the shift δ/h.

17.2.4 Half-GCD and the Euclidean Quotient Sequence

A Euclidean step with quotient q sends (A, B) to $(B, A - qB)$. Write its forward and inverse matrices as

$$E(q) = \begin{pmatrix} 0 & 1 \\ 1 & -q \end{pmatrix}, \qquad W(q) = E(q)^{-1} = \begin{pmatrix} q & 1 \\ 1 & 0 \end{pmatrix}.$$

The matrices have determinant -1, so every product is unimodular and preserves the ideal generated by the two polynomials. Half-GCD computes the initial Euclidean steps up to the first pair (C, D) satisfying $\deg C \geq m > \deg D$, where $m = \lceil \deg A/2 \rceil$. Its speed comes from determining these steps from high coefficients only.

Lemma 661 (Degrees of Euclidean matrices)**.** *For nonconstant $q_1, \ldots, q_j$, with $j \geq 1$, write $W = W(q_1) \cdots W(q_j) = \left(\begin{smallmatrix} P & Q \\ S & T \end{smallmatrix}\right)$. Then $\deg P = \sum_i \deg q_i$, both $\deg Q, \deg S < \deg P$, and $\deg T \leq \min\{\deg Q, \deg S\}$. All entries of W and W^{-1} have degree at most $\deg P$. If $(a, b)^{\mathsf{T}} = W(c, d)^{\mathsf{T}}$, with $\deg c > \deg d$, then $\deg P = \deg a - \deg c$.*

Proof. For one factor the assertions are immediate. On appending $W(q)$, the entries become $Pq + Q, P, Sq + T, S$. The strict degree inequalities prevent leading-term cancellation and prove the claims by induction. The inverse has the same entries, permuted and signed, because $\det W = \pm 1$. Finally, $a = Pc + Qd$ has leading term from Pc, proving the last identity. □

Lemma 662 (Lifting a high-coefficient Euclidean prefix)**.** *Let $t \geq 0$, and write $A = x^t a + a_0$, $B = x^t b + b_0$, where $a \neq 0$, $\deg a > \deg b$, and $\deg a_0, \deg b_0 < t$. Put $n = \deg a$. Suppose a Euclidean prefix on (a, b) has forward matrix R and output (c, d), with $\deg c \geq \lceil n/2 \rceil > \deg d$. Then the same quotient list is a Euclidean prefix on (A, B). Its output $(C, D) = R(A, B)$ satisfies*

$$\boxed{\deg C = t + \deg c, \qquad \deg D \leq \max\{t + \deg d,\ t + n - \deg c - 1\} < \deg C.}$$

The empty prefix is allowed.

Proof. For the empty prefix the assertions follow directly from the truncation. Otherwise $R^{-1} = W(q_1) \cdots W(q_j)$, and the preceding lemma bounds every entry of R by $n - \deg c$. Consequently

$$(C, D)^{\mathsf{T}} = x^t (c, d)^{\mathsf{T}} + R(a_0, b_0)^{\mathsf{T}},$$

where both error coordinates have degree at most $t + n - \deg c - 1$. Since $2 \deg c \geq n$, these errors have smaller degree than $x^t c$, proving the degree bounds.

To identify the quotients, start from $\deg C > \deg D$ and reconstruct the input by applying $W(q_j), \ldots, W(q_1)$ in that order. At every step $(U, V) \mapsto (q_i U + V, U)$, where $\deg V < \deg U$. Thus division of the reconstructed first polynomial by the second has quotient exactly q_i and remainder V. The reconstructed input is (A, B), so the entire list consists of its actual initial Euclidean quotients. □

Algorithm 96 Half-GCD

```
function HalfGCD(A, B)
    Require A ≠ 0 and deg A > deg B; allow B = 0
    n ← deg A, m ← ⌈n/2⌉
    if deg B < m then
        return I
    R ← HalfGCD(A div x^m, B div x^m)
    (C, D)^T ← R(A, B)^T
    if deg D < m then
        return R
    (q, H) ← DivMod(C, D)
    R ← E(q)R
    if deg H < m then
        return R
    t ← 2m − deg D
    S ← HalfGCD(D div x^t, H div x^t)
    return SR
```

Theorem 663 (Half-GCD)**.** *For* $\deg A > \deg B$*,* $A \neq 0$*, the algorithm returns the matrix of a Euclidean prefix with output* (C, D) *satisfying* $\deg C \geq \lceil \deg A/2 \rceil > \deg D$*. It costs* $O((n+1)\log^2(n+2))$ *operations for* $n = \deg A$*.*

Proof. Induct on n. The immediate return is correct. Otherwise put $a = n - m = \lfloor n/2 \rfloor$ and $h = \lceil a/2 \rceil$. The first recursive call has degree a; the lifting lemma makes its quotients valid for the full input and gives $\deg C \geq m$, $\deg D \leq m + h - 1$. If $\deg D < m$, we are done.

Otherwise one exact division gives the pair (D, H), with $c = \deg D \geq m$. Returning when $\deg H < m$ is correct. In the remaining case, $t = 2m - c > 0$, and the next truncated first polynomial has degree $c - t = 2(c - m)$. Its half threshold is $c - m$. The lifting lemma therefore gives a final first degree at least $t + (c - m) = m$, and a second degree at most $m - 1$. The lifted quotients continue the full Euclidean sequence.

The second recursive degree is at most $2h - 2 \leq a$; the first is a. All nonrecursive work consists of constantly many polynomial products and divisions of size $O(n+1)$. Thus $T(n) \leq 2T(\lfloor n/2 \rfloor) + O((n+1)\log(n+2))$, giving the stated bound. □

Extended GCD and quotient recovery. After half-GCD, perform one additional Euclidean step unless the second polynomial is zero. Both remaining degrees are then below the old half threshold, so recurse and multiply the transformation matrices. An initial Euclidean step handles equal input degrees; swapping handles reversed degrees. For zero second input return the first polynomial and its trivial Bézout coefficients; handle $(0, 0)$ separately. Normalize the final nonzero gcd to be monic, scaling its Bézout row by the same scalar. The total cost remains $O((n+1)\log^2(n+2))$.

To recover the quotient list, concatenate the first recursive list, the explicit quotient, and the second recursive list at each call. The lifting lemma shows that this is the full-input quotient sequence. The quotients have total degree at most n and total stored length $O(n+1)$. The full remainders, whose total length may be quadratic, are not constructed.

Algorithm 97 Extended polynomial GCD

```
function FullGCD(A, B)
    if B = 0 then
        if A = 0 then
            return (0, I)
        c ← lc(A)
        return (A/c, diag(c^{-1}, 1))
    if A = 0 or deg A < deg B then
        (g, R) ← FullGCD(B, A)
        return (g, R ( 0 1 ))
                     ( 1 0 ))
    if deg A = deg B then
        (q, H) ← DivMod(A, B)
        (g, R) ← FullGCD(B, H)
        return (g, RE(q))
    R ← HalfGCD(A, B)
    (C, D)^T ← R(A, B)^T
    if D = 0 then
        c ← lc(C)
        return (C/c, diag(c^{-1}, 1)R)
    (q, H) ← DivMod(C, D)
    (g, S) ← FullGCD(D, H)
    return (g, SE(q)R)
```

The returned matrix satisfies $R(A,B)^{\mathsf{T}} = (g,0)^{\mathsf{T}}$; its first row contains the Bézout coefficients. Swaps and any initial constant quotient are recorded separately when the exact quotient list is requested.

17.2.5 Modular Inversion

For a nonconstant modulus M, first reduce A modulo M. Extended GCD produces $uM + vA = g$. An inverse exists exactly when g is a nonzero constant, in which case

$$\boxed{A^{-1} \bmod M = (v/g) \bmod M.}$$

Necessity follows because any common divisor would divide one; sufficiency is the displayed Bézout identity. The cost is $O(n \log^2(n+2))$, with $n = \deg M$, after the initial reduction.

17.2.6 Resultants and Discriminants

For nonzero polynomials $A = a\prod_{i=1}^{n}(x-\alpha_i)$, $B = b\prod_{j=1}^{m}(x-\beta_j)$, define

$$\operatorname{Res}(A,B) = a^m \prod_i B(\alpha_i) = a^m b^n \prod_{i,j}(\alpha_i - \beta_j).$$

Roots are counted with multiplicity. The expression is symmetric in each root family, so it lies in the coefficient field; it vanishes exactly when the polynomials have a common root in a splitting field. The empty product gives $\operatorname{Res}(A,b) = b^n$ for constant $b \neq 0$.

Lemma 664 (Euclidean resultant identity)**.** *If $R = A \bmod B \neq 0$, with $\deg A = n$, $\deg B = m$, $\deg R = r$, and $b = \operatorname{lc}(B)$, then*

$$\boxed{\operatorname{Res}(A,B) = (-1)^{nm} b^{n-r} \operatorname{Res}(B,R).}$$

Proof. Swapping the root families contributes $(-1)^{nm}$. At every root β_j of B, $A(\beta_j) = R(\beta_j)$. The root-product definitions of $\operatorname{Res}(B,A)$ and $\operatorname{Res}(B,R)$ consequently differ only by b^{n-r}. □

Theorem 665 (Fast resultants)**.** *A resultant of polynomials of degrees at most n is computable in $O((n+1)\log^2(n+2))$ field operations.*

Proof. For constants use the definition; swap inputs if necessary and extract one division if their degrees are equal. In the remaining case recover the Euclidean quotient list by half-GCD, keeping the unnormalized remainders implicit. The list determines their degrees and leading coefficients: whenever $r_{i-1} = q_{i+1}r_i + r_{i+1}$, $\operatorname{lc}(r_i) = \operatorname{lc}(r_{i-1})/\operatorname{lc}(q_{i+1})$.

If the gcd has positive degree, return zero. Otherwise apply the lemma successively, ending with the constant-remainder formula. Only the remainder degrees and leading coefficients are needed. The sum of the exponents $\deg r_{i-2} - \deg r_i$ is $O(n)$, so the scalar powers cost $O(n)$ additional multiplications by binary powering. Half-GCD dominates the cost. □

For a monic polynomial $F = \prod_{i=1}^{n}(x - \alpha_i)$, with $n \geq 1$, its discriminant is

$$\boxed{\operatorname{disc}(F) = \prod_{i<j}(\alpha_i - \alpha_j)^2 = (-1)^{n(n-1)/2}\operatorname{Res}(F, F').}$$

For distinct roots, multiply $F'(\alpha_i) = \prod_{j\neq i}(\alpha_i - \alpha_j)$; repeated roots make both sides zero. For a general degree-n polynomial with leading coefficient a, first normalize and use $\operatorname{disc}(F) = a^{2n-2}\operatorname{disc}(F/a)$. This computes the discriminant within the same bound. Normalization also covers positive characteristic, where F' may have smaller degree. If $F' = 0$ and $\deg F > 0$, return zero.

17.2.7 Root Finding over a Prime Field

Let $f \in \mathbb{F}_p[x]$ be nonzero of degree n. First compute $g = \gcd(f, x^p - x)$, using modular powering. Since $x^p - x = \prod_{a\in\mathbb{F}_p}(x - a)$ has distinct roots, g is exactly the product of the distinct linear factors of f. Only g, whose degree $r \leq \min\{n, p\}$, needs splitting. For $p = 2$, test zero and one.

Assume p is odd. For a factor h and a uniform random $b \in \mathbb{F}_p$, compute $u = (x + b)^{(p-1)/2} \bmod h$ and split by $\gcd(h, u - 1)$. This separates roots with quadratic character $+1$ from the others. For distinct roots α, β, put $c = \beta - \alpha \neq 0$. As $v = \alpha + b$ ranges over $\mathbb{F}_p^\times$, $1 + c/v$ ranges bijectively over $\mathbb{F}_p \setminus \{1\}$. Its $(p-1)/2$ nonsquare values correspond to nonzero $v, v + c$ with opposite quadratic characters. Thus each pair is separated with probability at least $(p-1)/(2p) \geq 1/3$.

Theorem 666 (Randomized root finding)**.** *All distinct roots of a nonzero degree-n polynomial over $\mathbb{F}_p$ can be found by a Las Vegas algorithm in expected*

$$\boxed{O((n+1)\log^2(n+2)\log(p+1))}$$

field operations. Every reported root is correct.

Proof. Organize splitting into rounds, giving every current nonlinear factor one fresh trial, including factors whose preceding trial failed. A fixed pair remains together after j rounds with probability at most $(2/3)^j$, even conditioned on the earlier history. If L is the number of rounds, a union bound gives $\Pr(L > j) \leq \min\{1, r^2(2/3)^j\}$, hence $\mathbb{E}L = O(\log(r+2))$.

A split at degree $d \leq p$ costs $O(d\log(d+2)\log(p+1) + d\log^2(d+2)) = O(d\log(d+2)\log(p+1))$, including its gcd. The factors at a round have total degree at most r. This proves the splitting bound. Extracting g initially costs $O((n+1)\log(n+2)(\log(p+1) + \log(n+2)))$, which is covered by the displayed total. Each split uses exact gcds and divisions, so the algorithm is Las Vegas. □

Algorithm 98 Distinct roots over a prime field

Input: Nonzero $f \in \mathbb{F}_p[x]$
Output: All its roots in $\mathbb{F}_p$

```
if p = 2 then
    return the elements of {0, 1} at which f vanishes
if deg f = 0 then
    return the empty set
g ← gcd(f, (x^p mod f) − x)
P ← {g}, R ← ∅
while P ≠ ∅ do
    Q ← ∅
    for h ∈ P do
        if deg h = 1 then
            Add −h_0/h_1 to R
        else if deg h > 1 then
            Choose uniform b ∈ F_p
            u ← (x + b)^((p−1)/2) mod h
            v ← gcd(h, u − 1)
            if 0 < deg v < deg h then
                Add v, h/v to Q
            else
                Add h to Q
    P ← Q
return R
```

17.2.8 Minimum Linear Recurrence

An order-d recurrence for a finite prefix $s_0, \dots, s_{N-1}$ is a monic polynomial $P(Z) = \sum_{j=0}^{d} p_j Z^j$ satisfying $\sum_{j=0}^{d} p_j s_{i+j} = 0$ for $0 \le i < N - d$. Its connection polynomial is $C(x) = \operatorname{rev}_d P$, with $C(0) = 1$. The order is d, even if $\deg C < d$; zero roots of P describe initial transients.

Reverse the input prefix with its full length: $S^R(x) = \sum_{j=0}^{N-1} s_j x^{N-1-j}$. Direct coefficient comparison shows

$$\boxed{P \text{ is a recurrence of order } d \quad \Longleftrightarrow \quad \deg(PS^R \bmod x^N) < d.}$$

Indeed, the coefficient at x^{N-1-i} is exactly the recurrence sum at index i.

Run Euclid on $r_{-1} = x^N, r_0 = S^R$, and retain the coefficient t_i of S^R:

$$r_i = r_{i-2} - q_i r_{i-1}, \qquad t_i = t_{i-2} - q_i t_{i-1}, \qquad t_{-1} = 0, \quad t_0 = 1.$$

Then $t_i S^R \equiv r_i \pmod{x^N}$ and $\deg t_i + \deg r_{i-1} = N$. The latter follows inductively from $\deg q_i = \deg r_{i-2} - \deg r_{i-1}$: the degree of $q_i t_{i-1}$ strictly dominates that of t_{i-2}.

Theorem 667 (Minimum-order recurrence)**.** *If the input prefix is not zero, let i be the first Euclidean index with $\deg r_i < \deg t_i$, including a final zero remainder. Then $P = t_i / \operatorname{lc}(t_i)$ is a minimum-order recurrence. Such a recurrence is computable in $O(N \log^2(N+2))$ operations. Its coefficients need not be unique.*

Proof. The congruence proves validity. Suppose a smaller-degree nonzero U has remainder $V = US^R \bmod x^N$ with $\deg V < \deg U$. Cross-multiplication with the preceding Euclidean pair gives $Ur_{i-1} - t_{i-1}V \equiv 0 \pmod{x^N}$. Both products have degree less than N, since $\deg U < \deg t_i = N - \deg r_{i-1}$ and $\deg t_{i-1} \le \deg r_{i-1}$. Therefore they are equal. But $\deg(Ur_{i-1}) > \deg(t_{i-1}V)$; if $V = 0$, the left product is still nonzero. Either case is a contradiction.

Recover the quotient list by half-GCD. Its degree drops determine every remainder degree without constructing the remainders: for a nonfinal r_j, $\deg r_j = \deg r_{j-1} - \deg q_{j+1}$, and the final remainder is zero.

This identifies the crossing index in linear time. A balanced product of the first i matrices $E(q_j)$, in reverse multiplication order, gives t_i as its lower-right entry. Their total degree is at most N; each tree level costs $O(N \log(N+2))$, and there are $O(\log(N+2))$ levels. □

Algorithm 99 Minimum recurrence of a finite prefix

Input: $s_0, \ldots, s_{N-1}$
Output: A monic minimum-order forward recurrence P and connection polynomial C

if the prefix is zero, including $N = 0$ **then**
  return $(1, 1)$, of order zero
$S^R \leftarrow \sum_{j=0}^{N-1} s_j x^{N-1-j}$
Obtain the exact quotient list $q_1, \ldots, q_\ell$ for (x^N, S^R) by half-GCD
$d_{-1} \leftarrow N$, $d_0 \leftarrow \deg S^R$
for $j = 1, \ldots, \ell - 1$ **do**
  $d_j \leftarrow d_{j-1} - \deg q_{j+1}$
$d_\ell \leftarrow -\infty$
Find the first $i \in \{1, \ldots, \ell\}$ with $d_i < N - d_{i-1}$
$R \leftarrow E(q_i) \cdots E(q_1)$, using a balanced product tree
$P \leftarrow R_{22}/\operatorname{lc}(R_{22})$, $d \leftarrow \deg P$
return $(P, \operatorname{rev}_d P)$, retaining order d

For example, $(1, 0, 1)$ has minimum order two, but every $P(Z) = Z^2 - tZ - 1$ fits the prefix. The prefix alone does not determine which recurrence its continuation satisfies.

Corollary 668 (Recovery with an order bound)**.** *If an infinite sequence satisfies a recurrence of order at most d from its initial state, its first $2d$ terms determine its unique monic minimum recurrence, recovered by the preceding algorithm.*

Proof. The true generating function is A/B, where $B(0) = 1$, $\deg B \le d$, and $\deg A < d$. A recovered recurrence of order $e \le d$ gives U/C, where $C(0) = 1$, $\deg C \le e$, and $\deg U < e$. The two series agree modulo x^{2d}, so $AC - UB$ is divisible by x^{2d}. Its degree is less than $d + e \le 2d$, hence it is zero. The recovered recurrence therefore holds for the whole sequence and has minimum order.

For a nonzero rational series in reduced form A/B, with $B(0) = 1$, its minimum forward order is $e = \max\{\deg B, 1 + \deg A\}$, and its monic minimum recurrence is $\operatorname{rev}_e B$. Any other representation multiplies both A, B by a polynomial and cannot decrease this order. This also proves uniqueness, including zero-root transients. □

17.3 Rational Functions and Transposition

17.3.1 Bostan–Mori

For a polynomial H, define the coefficient-decimation maps $\mathcal{E}H = \sum_j h_{2j} x^j$ and $\mathcal{O}H = \sum_j h_{2j+1} x^j$. They select coefficients; no division by two is part of their definition.

Let $V(0) \neq 0$. The product $V(x)V(-x)$ is even, so write it as $W(x^2)$. Multiplying numerator and denominator by $V(-x)$ gives

$$\boxed{[x^N]\frac{U}{V} = [x^{\lfloor N/2 \rfloor}]\frac{\mathcal{P}_{N \bmod 2}(U(x)V(-x))}{\mathcal{E}(V(x)V(-x))}, \quad \mathcal{P}_0 = \mathcal{E}, \quad \mathcal{P}_1 = \mathcal{O}.}$$

Only the numerator determines the coefficient parity, since the denominator is a series in x^2. This identity also holds in characteristic two: there $V(x)V(-x) = V(x)^2$ is still even.

Theorem 669 (A distant rational coefficient)**.** *For* $\deg U < d$*,* $\deg V \le d$*,* $V(0) \neq 0$*, the value* $[x^N]U/V$ *is computable in* $O(d \log(d+2) \log(N+2))$ *operations.*

Proof. Each step preserves the numerator degree bound $< d$ and denominator bound $\le d$, while halving N. After $O(\log(N+2))$ steps the answer is the ratio of constant coefficients. Each step uses two products of size $O(d)$. □

Algorithm 100 Bostan–Mori coefficient extraction

Input: $\deg U < d$, $\deg V \le d$, $V(0) \neq 0$, $N \ge 0$
Output: $[x^N]U/V$

```
while N > 0 do
    A ← U(x)V(−x), B ← V(x)V(−x)
    U ← P_{N mod 2} A, V ← EB
    N ← ⌊N/2⌋
return U(0)/V(0)
```

For an improper rational function, first divide the numerator by the denominator. Add the requested coefficient of the polynomial part to the coefficient extracted from the proper fraction.

Truncation to the target index. Before a step, the numerator and denominator may be reduced modulo x^{N+1}, where N is the current target index. These prefixes determine the quotient through that index. Once $N < d$, the operand lengths therefore decrease geometrically along with N, rather than remaining at the original bound d. The remaining arithmetic costs $O((N+1)\log(N+2))$. Power projection uses the same truncation in its bivariate recurrence.

When $V(0) = 1$ and $V(x) = \prod_i (1 - \alpha_i x)$, its next value is $\prod_i (1 - \alpha_i^2 x)$. Thus coefficient decimation squares the exponential bases of the underlying recurrence while keeping the number of bases bounded. No roots need to be computed.

Antipodal transform values. To form W with $W(x^2) = V(x)V(-x)$, a transform of length $2L$, with $L > \deg V$, supplies paired values $V(\omega^j), V(-\omega^j)$. Their products are the values of W at the L-th roots ω^{2j}. In bit-reversed order these antipodal entries are adjacent, so multiplying each adjacent pair produces a length-L transform directly. The degree bound $L > \deg W$ prevents cyclic aliasing.

17.3.2 The Transposition Principle

Fix coefficient bases and use the bilinear dot product $\langle u, v \rangle = \sum_i u_i v_i$; no complex conjugation is used. For a linear map $L : K^n \to K^m$, its transpose is characterized by $\langle Lu, v \rangle = \langle u, L^{\mathsf{T}} v \rangle$.

Theorem 670 (Transposition of a linear algorithm)**.** *A straight-line linear algorithm of size* T*, using additions and multiplications by fixed scalars, gives an algorithm for its transpose of size* $O(T + n + m)$*.*

Proof. Give every intermediate scalar a separate name, and attach to it an accumulated coefficient, initially zero. Seed the output coefficients by v; their weighted sum is $\langle Lu, v \rangle$. Now substitute each defining instruction backward into this linear form. For $z = a + b$, replace the term $\bar{z}z$ by $\bar{z}a + \bar{z}b$, adding $\bar{z}$ to the two input accumulators. For $z = ca$, add $c\bar{z}$ to the accumulator of a. The processed variable z then disappears from the linear form. Copies and coordinate selections are treated in the same way.

Each substitution preserves the value of the linear form. At the end it is a weighted sum of the original inputs, equal to $\langle Lu, v \rangle$ for every u. Its coefficient vector is therefore $L^{\mathsf{T}} v$. Each reversed instruction takes constant work, in addition to seeding and reading the endpoint arrays. □

Auxiliary data remain fixed during transposition. For example, in Bostan–Mori with fixed V, N, precompute the denominator sequence and transpose only the numerator operations.

17.3.3 Transposed Multiplication and Reversal Pairings

For fixed P, let $M_P : K[x]_{<a} \to K[x]_{<b}$ send Q to $PQ \bmod x^b$. Its dot-product transpose is

$$\boxed{M_P^{\mathsf{T}}(Z) = \sum_{j=0}^{a-1} x^j \sum_{\ell \geq 0} p_\ell z_{j+\ell}.}$$

Here $z_i = 0$ outside $0 \leq i < b$. The formula follows by expanding $\sum_{i<b} z_i \sum_j p_{i-j} q_j$ and collecting the coefficient of q_j. We write this middle product as $(Z \circ P)_{<a}$, with the input length b understood. It is one ordinary product after reversing the array Z, with the required coefficient interval extracted afterward.

The reversal pairing is $\langle U, Z \rangle_a^{\mathrm{rev}} = [x^{a-1}]UZ$, which is the dot product after reversing one length-a array. With this pairing, for $b \geq a$, the same multiplication map has adjoint

$$\boxed{M_P^*(Z) = \left(PZ \operatorname{div} x^{b-a}\right) \bmod x^a.}$$

Indeed, for every input monomial x^j, the two pairings both read the coefficient $[x^{b-1-j}]PZ$. Division by x^{b-a} here discards the lower coefficients.

Under the reversal pairings of lengths $2d$ and d, the parity selectors have adjoints

$$\boxed{\mathcal{E}^* Z = xZ(x^2), \qquad \mathcal{O}^* Z = Z(x^2).}$$

For example, $[x^{d-1}](\mathcal{E}U)Z = [x^{2d-1}]U\, xZ(x^2)$. Under dot-product pairings the formulas are instead $\mathcal{E}^{\mathsf{T}} Z = Z(x^2)$ and $\mathcal{O}^{\mathsf{T}} Z = xZ(x^2)$. The change in parity comes from the reversal pairing.

Transposed transforms and bit reversal. Let $\mathcal{F}_- = (\omega^{-ij})_{i,j}$, $\mathcal{F}_+ = (\omega^{ij})_{i,j}$, and let P permute indices by bit reversal. Natural-order transforms are symmetric. An output-bit-reversed forward transform is $\mathcal{B} = P\mathcal{F}_-$, with inverse $\mathcal{B}^{-1} = n^{-1}\mathcal{F}_+ P$. Writing J for the permutation $i \mapsto -i \pmod n$, we have

$$\mathcal{B}^{\mathsf{T}} = nJ\mathcal{B}^{-1}, \qquad (\mathcal{B}^{-1})^{\mathsf{T}} = n^{-1}\mathcal{B}J.$$

To transpose a padded convolution, reverse its stages: zero-pad the output weights, apply the transpose of the inverse transform, multiply by the fixed transformed factor, apply the transpose of the forward transform, then select the original input coordinates.

17.3.4 Transposed Bostan–Mori and Modular Powers

Fix V with $\deg V \leq d$, $V(0) \neq 0$. We seek the length-d reversal-pairing state A satisfying

$$[x^{d-1}]U(x)A(x) = [x^N]\frac{U(x)}{V(x)} \qquad (\deg U < d).$$

At $N = 0$, it is $A = x^{d-1}/V(0)$. For a recursive step, first recurse with $V_1 = \mathcal{E}(V(x)V(-x))$ and index $\lfloor N/2 \rfloor$. Lift the returned state by the appropriate parity adjoint, multiply by $V(-x)$, and extract degrees $d, \ldots, 2d-1$. The dimensions remain d even if $\deg V_1 < d$.

Algorithm 101 Transposed rational coefficient extraction

```
function TransposedProj(N, V, d)
    if N = 0 then
        return x^{d-1}/V(0)
    V_1 ← E(V(x)V(−x))
    A ← TransposedProj(⌊N/2⌋, V_1, d)
    if N is even then
        Z ← xA(x^2)
    else
        Z ← A(x^2)
    return (ZV(−x) div x^d) mod x^d
```

The adjoint identities prove correctness. The cost is $O(d \log(d+2) \log(N+2))$.

Theorem 671 (Recovering a modular power)**.** *Let $P = \operatorname{rev}_d V$, so $\deg P = d$, and let A be the state just computed. Then*

$$\boxed{x^N \bmod P = \operatorname{rev}_{d-1}(AV \bmod x^d).}$$

Proof. Write $x^N = QP + R$, with $\deg R < d$. Substitute x^{-1}, multiply by U/V, and take constant terms. The quotient term is $x^{-d}Q(x^{-1})U(x)$, which has only negative powers. Therefore

$$[x^N]\frac{U}{V} = [x^0]\frac{R(x^{-1})U(x)}{V(x)} = [x^{d-1}]U(x)\frac{\operatorname{rev}_{d-1} R(x)}{V(x)}.$$

By nondegeneracy of the coefficient pairing, $A \equiv \operatorname{rev}_{d-1} R/V \pmod{x^d}$, proving the formula. □

17.4 Composition and Compositional Inversion

17.4.1 Power Projection

For $P, W \in K[x]_{<n}$, power projection asks for $v_j = \sum_{i<n} w_i [x^i] P(x)^j$, $0 \le j < n$. Its generating function is a coefficient of a bivariate rational function:

$$\boxed{\sum_{j \ge 0} v_j y^j = [x^{n-1}]\frac{\operatorname{rev}_{n-1} W(x)}{1 - yP(x)}.}$$

This identity is formal in y, even when $P(0) \neq 0$. First reduce to $P(0) = 0$, so the final denominator is one. If $c = P(0)$, project onto $P - c$, obtaining $\widetilde{v}_j$. Then $v_j = \sum_{i=0}^{j} \binom{j}{i} c^{j-i} \widetilde{v}_i$, a single convolution after factorial scaling.

Assume now $P(0) = 0$. Pad P, W to length m, the least power of two at least n, and reverse W with bound $m - 1$. Padding before reversal preserves the coefficient functional. Initialize $A = \operatorname{rev}_{m-1} W$, $B = 1 - yP$, $h = m$, $k = 1$.

At a stage maintain

$$\deg_x A, \deg_x B < h, \qquad \deg_y A < k, \quad \deg_y B \le k, \qquad hk = m, \qquad B(0, y) = 1.$$

The desired generating function is $[x^{h-1}]A/B$. Since $h - 1$ is odd for $h > 1$, one Bostan–Mori step is

$$\boxed{A_{\text{new}} = \mathcal{O}_x(AB(-x, y) \bmod x^h), \qquad B_{\text{new}} = \mathcal{E}_x(BB(-x, y) \bmod x^h).}$$

Replace (h, k) by $(h/2, 2k)$. Higher x-coefficients cannot affect the target. Truncating them halves the x-dimension, while the y-degree bounds double. The number of stored coefficients remains $O(m)$, and $B(0, y) = 1$ is preserved. At $h = 1$, the answer is $A(0, y)$.

Theorem 672 (Fast power projection)**.** *The first n projections are computable in $O(n \log^2(n+2))$ coefficient operations. For $P(0) = 0$, the algorithm uses only additions and multiplications and works over any commutative coefficient ring. The constant-term shift additionally uses factorial inverses.*

Proof. The rational coefficient identity and each parity step preserve the required generating function. If $P(0) = 0$, then $P^j = 0 \bmod x^m$ for $j \geq m$, so the terminal generating function is a polynomial of degree less than m.

At a stage, ordinary bivariate products fit in a rectangle of size $O(hk) = O(m)$, including constant-factor padding in both dimensions. Kronecker substitution converts them to univariate products of length $O(m)$. There are $\log_2 m$ stages. The initial constant-term shift, when needed, costs $O(n \log(n+2))$. □

Algorithm 102 Power projection with explicit precision bounds

function PowerProj(W, P, n)
  if $n = 0$ **then**
    return the empty vector
  $c \leftarrow P(0)$
  if $c \neq 0$ **then**
    $u \leftarrow \text{PowerProj}(W, P - c, n)$
    $v_j \leftarrow j![y^j]\big((\sum_{i<n} u_i y^i/i!)(\sum_{i<n} c^i y^i/i!)\big)$
    return $(v_j)_{j<n}$
  $m \leftarrow 2^{\lceil \log_2 n \rceil}$; pad P, W to length m
  $A \leftarrow \operatorname{rev}_{m-1} W$, $B \leftarrow 1 - yP$, $h \leftarrow m$, $k \leftarrow 1$
  while $h > 1$ **do**
    $C \leftarrow B(-x, y)$
    $A \leftarrow \mathcal{O}_x(AC \bmod x^h)$
    $B \leftarrow \mathcal{E}_x(BC \bmod x^h)$
    $h \leftarrow h/2$, $k \leftarrow 2k$
  return the first n coefficients of $A(0, y)$

17.4.2 Composition by Transposition

For fixed P, let M_P have columns $\operatorname{coeff}(P^j \bmod x^n)$, $0 \leq j < n$. Composition is $M_P \operatorname{coeff}(F)$, while power projection is $M_P^{\mathsf{T}} \operatorname{coeff}(W)$. Only the numerator states in power projection depend on W, so transposition gives composition within the same time bound.

For a bivariate array Z, define

$$\mathrm{MP}_{h,k}(Z, C) = \sum_{i<h,\, j<k} x^i y^j \sum_{a,b \geq 0} c_{a,b} z_{i+a, j+b},$$

with out-of-range entries of Z set to zero. This is the transpose of multiplication by fixed C, followed by restriction to the appropriate output rectangle. Reversing both axes reduces it to one bivariate convolution.

At a forward stage of dimensions (h, k), the numerator map is $A \mapsto \mathcal{O}_x(AC \bmod x^h)$, with $C = B(-x, y)$. Its dot-product transpose is

$$\boxed{Z \longmapsto \mathrm{MP}_{h,k}(xZ(x^2, y), C).}$$

After upsampling and the middle product, retain x-length h and y-length k.

Theorem 673 (Fast truncated composition)**.** *For $F, P \in K[x]_{<n}$, the coefficients of $F(P(x)) \bmod x^n$ are computable in $O(n \log^2(n+2))$ operations. If $P(0) = 0$, only those prefixes of formal input series are needed. If $P(0) \neq 0$, the outer input must be a finite polynomial or be specified by its expansion about $P(0)$.*

Proof. Precompute the denominator states from power projection. Start the reverse pass with the coefficient weights of F, zero-padded to the terminal length m. Apply the displayed adjoint at every level. Finally reverse the length-m numerator state and retain its first n coefficients, transposing the initial padded reversal. The pairing identity at each stage proves that the result is $M_P \operatorname{coeff}(F)$.

All reversed products have $O(m)$ coefficients, as in the forward algorithm. For $P(0) = c \neq 0$, first replace $F(t)$ by $F(t + c)$ and P by $P - c$. The Taylor shift has smaller cost. □

Algorithm 103 Truncated composition

```
function Compose(F, P, n)
    if n = 0 then
        return zero
    c ← P(0)
    if c ≠ 0 then
        F(t) ← F(t + c), P ← P − c
    m ← 2^⌈log₂ n⌉; pad both inputs to length m
    B ← 1 − yP, h ← m, k ← 1, S ← ∅
    while h > 1 do
        Append (h, k, B(−x, y)) to S
        B ← E_x(B(x, y)B(−x, y) mod x^h)
        h ← h/2, k ← 2k
    Z(y) ← F(y)
    for (h, k, C) in S, in reverse order do
        Z ← MP_{h,k}(xZ(x^2, y), C)
    return rev_{m−1}(Z(x, 0)) mod x^n
```

Storing all denominator levels uses $O(n \log(n + 2))$ space. For a nonzero constant substitution, high coefficients of the outer polynomial can affect the output constant term. Shift the entire outer polynomial before truncating it.

Frequency-domain implementation. The transform implementation includes the same coefficient truncation. Put $H = \mathcal{O}_x(A(x, y)B(-x, y))$ *before* truncation. When two is invertible,

$$H(\omega^2, y) = \frac{A(\omega, y)B(-\omega, y) - A(-\omega, y)B(\omega, y)}{2\omega}, \qquad A_{\text{new}} = H \bmod x^{h/2}.$$

Since $\deg_x A, \deg_x B < h$, an x-transform of length $2h$ represents the ordinary product without cyclic aliasing, and the paired values represent H with coefficient length h. The final crop retains only its first $h/2$ coefficients.

In the transposed computation, first embed the incoming length-$h/2$ coefficient state into length h by appending zeros; this is the transpose of that crop. Transpose the inverse transform, then transpose the displayed two-input combination. An incoming frequency weight Z contributes $ZB(-\omega, y)/(2\omega)$ and $-ZB(\omega, y)/(2\omega)$ to the antipodal entries. Finally transpose the forward transform and crop to the original coefficient rectangle. Bit-reversed transforms make the antipodal entries adjacent. The transform normalizations and the analogous y-dimension crops remain part of the map.

The pointwise calculation yields H; the coefficient truncation then gives A_{new}.

17.4.3 Lagrange Inversion

Lemma 674 (Lagrange inversion)**.** *Let $\Phi, H \in K[[x]]$, and let $f \in xK[[x]]$ be the unique solution of $f = x\Phi(f)$. In characteristic zero, for $n \geq 1$,*

$$\boxed{[x^n]H(f(x)) = \frac{1}{n}[x^{n-1}]H'(x)\Phi(x)^n.}$$

The constant coefficient is $H(0)$.

Proof. The equation determines each coefficient of f from earlier ones, proving existence and uniqueness. By linearity it suffices to prove $[x^n]f^k = (k/n)[x^{n-k}]\Phi^n$ for monomials $H = x^k$. Induct on n, simultaneously for all k. The cases $k = 0$, $k > n$, and $k = n$ follow from constant coefficients and $\operatorname{ord} f \geq 1$. For $1 \leq k < n$, use the induction hypothesis at index $n - k$ with outer series Φ^k:

$$\begin{aligned}[x^n]f^k &= [x^{n-k}]\Phi(f)^k \\ &= \frac{k}{n-k}[x^{n-k-1}]\Phi'\Phi^{n-1} \\ &= \frac{k}{n(n-k)}[x^{n-k-1}](\Phi^n)' = \frac{k}{n}[x^{n-k}]\Phi^n.\end{aligned}$$

Only finitely many coefficients of each outer series contribute, so the linearity reduction applies to formal series as well as polynomials. □

Corollary 675 (Lagrange inversion for a compositional inverse)**.** *If $F(0) = 0$, $F'(0) \neq 0$, and $G = F^{\langle -1\rangle}$, then for $n \geq 1$,*

$$[x^n]H(G(x)) = \frac{1}{n}[x^{n-1}]H'(x)\left(\frac{x}{F(x)}\right)^n.$$

Proof. The series $\Phi = x/F$ is a unit, and $G = x\Phi(G)$ is equivalent to $F(G) = x$. Apply the lemma. □

17.4.4 Compositional Inversion by One Power Projection

Suppose $F = x + O(x^2)$, $G = F^{\langle -1\rangle}$, and write $H = x/G$, so $H(0) = 1$. Applying Lagrange inversion with the roles of F, G exchanged gives

$$[x^m]F(x)^k = \frac{k}{m}[x^{m-k}]H(x)^m \qquad (1 \leq k \leq m).$$

Thus one power projection at coefficient x^m determines the first m coefficients of H^m:

$$\boxed{u_j = \frac{m}{m-j}[x^m]F(x)^{m-j}, \qquad 0 \leq j < m.}$$

Recover $G = x(\sum_{j<m} u_j x^j)^{-1/m} \bmod x^{m+1}$ by one normalized power. All scalar divisors here lie in $1, \ldots, m$.

For target precision x^N, take $m = N - 1$. The input $F \bmod x^N$ then suffices. If $F'(0) = c \neq 1$, normalize $\widetilde{F} = F/c$. Then $F^{\langle -1\rangle}(x) = \widetilde{F}^{\langle -1\rangle}(x/c)$.

Theorem 676 (Fast compositional inverse)**.** *Given $F \bmod x^N$, with $F(0) = 0$, $F'(0) \neq 0$, its compositional inverse modulo x^N is computable in $O(N \log^2(N + 2))$ operations, provided $1, \ldots, N - 1$ are invertible.*

Proof. The displayed Lagrange identity determines H^m to the required precision, and its constant coefficient is one. The normalized power therefore recovers the uniquely chosen H^{-1}. Power projection dominates the cost; normalization and the final coefficient scaling are linear-time operations. □

Algorithm 104 Compositional inverse

Input: $F \bmod x^N$, $F(0) = 0$, $F'(0) \neq 0$
Output: $F^{\langle -1 \rangle} \bmod x^N$

1: **if** $N \leq 1$ **then**
2: ⌊ **return** zero
3: $m \leftarrow N - 1$, $c \leftarrow F'(0)$, $\widetilde{F} \leftarrow F/c$
4: $v \leftarrow \textsc{PowerProj}(x^m, \widetilde{F}, N)$
5: $U \leftarrow \sum_{j=0}^{m-1} \frac{m}{m-j} v_{m-j} x^j$
6: $\widetilde{G} \leftarrow x \exp(-\frac{1}{m} \log U) \bmod x^N$
7: **return** $\widetilde{G}(x/c)$

17.5 Evaluation and Interpolation

17.5.1 Multipoint Evaluation by Transposition

Given $F \in K[x]_{<n}$ and points $\alpha_1, \ldots, \alpha_r$, we want all $F(\alpha_i)$. The points need not be distinct. The transpose of this Vandermonde map takes weights c_i to the first n coefficients of

$$\sum_{i=1}^{r} \frac{c_i}{1 - \alpha_i x} = \sum_{j \geq 0} \left(\sum_i c_i \alpha_i^j \right) x^j.$$

Evaluation requires no division by point differences.

Build a balanced tree on the points. For a node u containing s_u points, store $D_u = \prod_{i \in u} (1 - \alpha_i x)$, retaining degree bound s_u even when some $\alpha_i = 0$. The weighted rational sum has numerator N_u of degree less than s_u. At leaves $N_i = c_i$, and at an internal node

$$N_u = N_L D_R + N_R D_L, \qquad D_u = D_L D_R.$$

At the root, output $N_u / D_u \bmod x^n$. This computes the transposed evaluations in $O((n + r) \log^2(n + r + 2))$ operations: each tree level has total polynomial length $O(r)$, and the final inverse and product have length $O(n + r)$.

Transposing these linear numerator operations gives ordinary evaluation. Initialize $T_{\text{root}} = (F \circ D_{\text{root}}^{-1})_{<r}$, where the reciprocal is known modulo x^n. At a node, pass

$$\boxed{T_L = (T_u \circ D_R)_{<s_L}, \qquad T_R = (T_u \circ D_L)_{<s_R}.}$$

The coefficient $[x^0]T_i$ at leaf i is $F(\alpha_i)$. Each state has length equal to its node's point count.

Theorem 677 (Multipoint evaluation)**.** *A degree-$< n$ polynomial can be evaluated at r arbitrary points in $O((n + r) \log^2(n + r + 2))$ coefficient operations. The transposed evaluation map has the same complexity. Both constructions work over a commutative ring, since all series denominators have constant coefficient one.*

Proof. The upward rational-sum identity proves the transposed algorithm. At each internal node, $\langle N_L D_R + N_R D_L, T_u \rangle = \langle N_L, T_u \circ D_R \rangle + \langle N_R, T_u \circ D_L \rangle$, with the stated child dimensions. The root transpose and leaf extraction therefore prove evaluation. The degree sums at each level give the same cost for both directions. □

Algorithm 105 Multipoint evaluation

Input: $F \in K[x]_{<n}$, points $\alpha_1, \ldots, \alpha_r$
Output: $(F(\alpha_i))_{i=1}^{r}$
1: Build the balanced tree with $D_i = 1 - \alpha_i x$ and $D_u = D_L D_R$
2: $I \leftarrow D_{\text{root}}^{-1} \bmod x^n$
3: DESCEND(root, $(F \circ I)_{<r}$)

4: **function** DESCEND(u, T)
5: | **if** u is leaf i **then**
6: | | Output $T(0)$ at position i
7: | | **return**
8: | DESCEND($L, (T \circ D_R)_{<s_L}$)
9: | DESCEND($R, (T \circ D_L)_{<s_R}$)

An empty point list returns no values, and the zero polynomial returns zeros without constructing a tree. Storing the tree takes $O(r \log(r+2))$ coefficients. The same tree can be reused for several polynomials or weight vectors.

17.5.2 Interpolation

For distinct points $\alpha_1, \ldots, \alpha_n$, put $P(x) = \prod_i (x - \alpha_i)$. The unique degree-$< n$ interpolant for values y_i is

$$\boxed{F(x) = \sum_{i=1}^{n} \frac{y_i}{P'(\alpha_i)} \frac{P(x)}{x - \alpha_i}.}$$

Every summand vanishes at the other points and has value y_i at α_i; uniqueness follows because a nonzero polynomial of degree less than n cannot have n distinct roots.

Construct the ordinary monic product tree P_u, evaluate P' at all points, and set $c_i = y_i / P'(\alpha_i)$. At leaves put $N_i = c_i$; upward, use $N_u = N_L P_R + N_R P_L$. Inductively $N_u = \sum_{i \in u} c_i P_u / (x - \alpha_i)$, so the root is F. Both passes cost $O(n \log^2(n+2))$. The reversed tree required by the evaluation algorithm is obtained as $D_u = \operatorname{rev}_{s_u} P_u$, preserving the degree bounds.

Algorithm 106 Polynomial interpolation

Input: Distinct points $\alpha_1, \ldots, \alpha_n$, values $y_1, \ldots, y_n$
Output: Their degree-$< n$ interpolant
1: Build $P_i = x - \alpha_i$, $P_u = P_L P_R$
2: Evaluate P'_{root} at all α_i, obtaining v_i
3: Set $N_i \leftarrow y_i / v_i$ at each leaf
4: **for** internal nodes u, bottom-up **do**
5: | $N_u \leftarrow N_L P_R + N_R P_L$
6: **return** N_{root}

Over a commutative ring the same construction works when all differences $\alpha_i - \alpha_j$ are units; then every $P'(\alpha_i)$ is a unit.

17.5.3 Chirp-Z: Geometric Evaluation

Let $F(x) = \sum_{i=0}^{d-1} f_i x^i$, let $z \neq 0$, and request $F(1), F(z), \ldots, F(z^{k-1})$. The identity $ij = \binom{i+j}{2} - \binom{i}{2} - \binom{j}{2}$ gives

$$F(z^j) = z^{-\binom{j}{2}} \sum_{i=0}^{d-1} \bigl(f_i z^{-\binom{i}{2}}\bigr) z^{\binom{i+j}{2}}.$$

Set $A_i = f_i z^{-\binom{i}{2}}$ and $B_t = z^{\binom{t}{2}}$ for $0 \le t \le d+k-2$. The required sum is $[x^j](B \circ A)$. One middle product and two diagonal scalings evaluate all points in $O((d+k)\log(d+k+2))$ operations. Distinctness of the powers of z is unnecessary.

The factors $z^{\binom{i}{2}}$ are generated in linear time. Start with $u = v = 1$; after using u at index i, replace (u, v) by (uv, vz). Their inverse factors use z^{-1}.

Algorithm 107 Geometric evaluation

```
function ChirpZ(F, z, k)
    Require z ≠ 0; return the empty vector if k = 0
    Let d be the supplied coefficient length of F
    if d = 0 then
        return k zeros
    A_i ← f_i z^{-binom(i,2)} for 0 ≤ i < d
    B_t ← z^{binom(t,2)} for 0 ≤ t ≤ d + k − 2
    Y ← (B ∘ A)_{<k}
    return (z^{-binom(j,2)} Y_j)_{j=0}^{k-1}
```

Evaluation at $a, az, \ldots, az^{k-1}$ first scales f_i by a^i. The zero polynomial is handled directly.

17.5.4 Inverse Chirp-Z

Assume $z \neq 0$ and $1, z, \ldots, z^{k-1}$ are distinct, equivalently $z^j \neq 1$ for $1 \le j < k$. Given $y_i = F(z^i)$, with $\deg F < k$, let $D(x) = \prod_{j=0}^{k-1}(x - z^j)$. Factoring its Lagrange denominators gives

$$D'(z^i) = z^{i(k-1)} \prod_{m=1}^{i} (1 - z^{-m}) \prod_{m=1}^{k-1-i} (1 - z^m).$$

Thus all weights $c_i = y_i / D'(z^i)$ are obtained from two prefix product arrays and successive powers of $z^{-(k-1)}$, in linear time.

Let $Q(x) = \prod_{j=0}^{k-1}(1 - z^j x)$. Reversing the interpolant in the Lagrange formula yields

$$\frac{\operatorname{rev}_{k-1} F(x)}{Q(x)} = \sum_{i=0}^{k-1} \frac{c_i}{1 - z^i x}.$$

The first k coefficients on the right are $s_m = \sum_i c_i z^{im}$, one Chirp-Z evaluation of the polynomial with coefficients c_i. Hence $\operatorname{rev}_{k-1} F = (\sum_{m<k} s_m x^m) Q \bmod x^k$.

The needed coefficients of Q are also linear-time computable. From $(1-x)Q(zx) = (1 - z^k x)Q(x)$, coefficient comparison gives

$$\boxed{q_0 = 1, \qquad q_j = -z^{j-1} \frac{1 - z^{k-j+1}}{1 - z^j} q_{j-1} \quad (1 \le j < k).}$$

Every denominator is nonzero; $z^k = 1$ is allowed, since $1 - z^k$ occurs only in a numerator.

Theorem 678 (Geometric interpolation)**.** *A degree-$< k$ polynomial is reconstructed from its values at $1, z, \dots, z^{k-1}$ in $O(k \log(k+2))$ operations under the preceding distinctness assumptions.*

Proof. Compute the Lagrange weights, perform one Chirp-Z transform, construct $Q \bmod x^k$ by its recurrence, multiply modulo x^k, and reverse with bound $k-1$. The established identities prove correctness. □

Algorithm 108 Geometric interpolation

Input: $k \geq 1$, $y_i = F(z^i)$, $0 \leq i < k$, $\deg F < k$, distinct nonzero nodes
Output: F

1: $P_0^+ = P_0^- \leftarrow 1$
2: **for** $j = 1, \dots, k-1$ **do**
3: $\quad P_j^+ \leftarrow P_{j-1}^+/(1-z^j)$, $P_j^- \leftarrow P_{j-1}^-/(1-z^{-j})$
4: $c_i \leftarrow y_i z^{-i(k-1)} P_i^- P_{k-1-i}^+$ for $0 \leq i < k$
5: $s \leftarrow \textsc{ChirpZ}(\sum_{i<k} c_i x^i, z, k)$
6: Compute $Q \bmod x^k$ by the coefficient recurrence above
7: **return** $\mathrm{rev}_{k-1}((\sum_{i<k} s_i x^i) Q \bmod x^k)$

17.5.5 Hermite Interpolation

Hermite interpolation specifies several Taylor coefficients at each point. Hasse derivatives make this independent of the characteristic: define $D^{(j)} f(a) = [z^j] f(a+z)$. When $j!$ is invertible, this is $f^{(j)}(a)/j!$, but the definition never requires that division.

Lemma 679 (Taylor data as a remainder)**.** *Over any commutative ring R,*

$$f(x) \equiv \sum_{j=0}^{s-1} D^{(j)} f(a)(x-a)^j \pmod{(x-a)^s}.$$

Proof. Substitute $z = x - a$ in $f(a+z) = \sum_j D^{(j)} f(a) z^j$, then omit the terms divisible by $(x-a)^s$. □

Lemma 680 (Coprimality of the local moduli)**.** *If $a - b$ is a unit, the ideals $((x-a)^r)$ and $((x-b)^s)$ are comaximal for all $r, s \geq 1$.*

Proof. The ideals $I = (x-a)$, $J = (x-b)$ contain elements u, v with $u+v = 1$, because $(x-a)-(x-b) = b-a$ is a unit. Every term of $(u+v)^{r+s-1} = 1$ lies in I^r or J^s. □

Theorem 681 (Hermite interpolation)**.** *Let $a_1, \dots, a_k \in R$ have unit pairwise differences, and let $s_i \geq 1$, $n = \sum_i s_i$. For arbitrary prescribed $y_{i,j} \in R$, $0 \leq j < s_i$, there is a unique polynomial of degree less than n satisfying $D^{(j)} f(a_i) = y_{i,j}$.*

Proof. Set $P_i = (x-a_i)^{s_i}$ and $T_i = \sum_{j<s_i} y_{i,j}(x-a_i)^j$. The Taylor conditions are $f \equiv T_i \pmod{P_i}$. The preceding lemma and the Chinese remainder theorem give a unique class modulo the monic degree-n polynomial $P = \prod_i P_i$, which has a unique representative of degree less than n. □

Product-tree construction. The interpolation formula is

$$f = \sum_i C_i \frac{P}{P_i}, \qquad C_i = T_i \left(\frac{P}{P_i}\right)^{-1} \bmod P_i.$$

Each summand has degree less than n. To compute every local inverse, build the product tree P_u, and propagate $q_u = (P/P_u) \bmod P_u$ downward. At the root $q_u = 1$, and

$$\boxed{q_L = q_u P_R \bmod P_L, \qquad q_R = q_u P_L \bmod P_R.}$$

The parent residue suffices because both child moduli divide P_u. At leaf i, shift q_i to $a_i + z$, invert modulo z^{s_i}, and multiply by $\sum_{j<s_i} y_{i,j} z^j$. Shift back to obtain C_i. Its inverse exists because $q_i(a_i) = \prod_{j \neq i} (a_i - a_j)^{s_j}$ is a unit.

Finally set $F_i = C_i$ at leaves and propagate upward by

$$\boxed{F_u = F_L P_R + F_R P_L.}$$

Induction gives $F_u = \sum_{i \in u} C_i P_u / P_i$, so the root is the required CRT sum.

Theorem 682 (Fast Hermite evaluation and interpolation)**.** *With total multiplicity n, Hermite interpolation takes $O(n \log^2(n+2))$ ring operations and inversions of the stated units. The first s_i Taylor coefficients of a given degree-$< n$ polynomial at each a_i can be computed within the same bound. No factorial denominators are required.*

Proof. Build each P_i by powering and the product tree by balanced multiplication. In each upward or downward pass, node degrees sum to n at every depth, so all products and monic reductions cost $O(n \log^2(n+2))$. At the leaves, use division-free Taylor shifts, followed by ordinary unit-series inversion. Their total cost is at most $\sum_i O(s_i \log^2(s_i+2)) = O(n \log^2(n+2))$.

For evaluation, propagate $f \bmod P_u$ downward through the monic product tree, then shift each leaf remainder by a_i. These are exactly its Taylor coefficients by Lemma 679. The same degree-sum argument applies. □

Algorithm 109 Hermite interpolation over a commutative ring

Input: Points a_i with unit differences, multiplicities s_i, Taylor data $y_{i,j}$
Output: The interpolant of degree less than $n = \sum_i s_i$

Build a balanced product tree with leaves $P_i = (x - a_i)^{s_i}$
$q_{\text{root}} \leftarrow 1$
for internal nodes u, top-down **do**
    $q_L \leftarrow q_u P_R \bmod P_L$, $q_R \leftarrow q_u P_L \bmod P_R$
for leaves i **do**
    $u_i(z) \leftarrow q_i(a_i + z)$, by a division-free Taylor shift
    $v_i(z) \leftarrow (\sum_{j<s_i} y_{i,j} z^j)/u_i(z) \bmod z^{s_i}$
    $F_i(x) \leftarrow v_i(x - a_i)$, by a division-free Taylor shift
for internal nodes u, bottom-up **do**
    $F_u \leftarrow F_L P_R + F_R P_L$
return F_{root}

17.6 Multivariate Evaluation by Integer Lifting

Over a finite field or a residue ring, let $f(x_1, \ldots, x_m)$ have individual degree less than d, so its dense representation has d^m coefficients. Given arbitrary points $a_1, \ldots, a_N$, we want all $f(a_i)$. Reduce the modulus until the complete residue grid is small, evaluate on that grid, then reconstruct the original answers by Chinese remaindering. Using prime powers and nilpotent Taylor expansion removes a factor m from the grid side length. For extension fields, the same Taylor expansion supplies data for Hermite interpolation.

The degree bound $d = 1$ is the constant-polynomial case; empty query lists are also immediate. In the recursive reductions assume $d \geq 2$, $m \geq 1$, and $N \geq 1$. All iterated logarithms are clipped below at one. Soft-O bounds below hide logarithms of the parameters and their bit lengths, but not factors exponential in m.

17.6.1 Product Grids and Nilpotent Taylor Expansion

Lemma 683 (Product-grid evaluation)**.** *Let $A_i \subseteq R$ be nonempty finite sets in a commutative ring, with $q_i = |A_i|$ and $Q = \prod_i q_i$. An m-variate polynomial of individual degree less than d can be evaluated on $A_1 \times \cdots \times A_m$ in*

$$\boxed{O\left(m(d^m + Q)\log^2(d + \max_i q_i + 2)\right)}$$

ring operations. The points in a coordinate set need not have unit differences.

Proof. Order the variables so that $q_1 \le \cdots \le q_m$. After evaluating the first i variables, the coefficient/evaluation array has size $D_i = q_1 \cdots q_i d^{m-i}$, with $D_0 = d^m$ and $D_m = Q$. The next stage evaluates $q_1 \cdots q_{i-1} d^{m-i}$ univariate polynomials of degree less than d at q_i points. By Theorem 677, it costs $O((D_{i-1} + D_i)\log^2(d + q_i + 2))$.

The ratios $D_i/D_{i-1} = q_i/d$ are nondecreasing, so the array sizes first decrease and then increase. None exceeds $\max\{d^m, Q\}$. Summing the stage costs proves the bound. The computation uses only evaluation, so point differences need not be units. □

For a multi-index $\alpha \in \mathbb{N}^m$, define the multivariate Hasse derivative by

$$f(x+z) = \sum_{\alpha \ge 0} (\partial^\alpha f)(x) z^\alpha.$$

If $f = \sum_\beta c_\beta x^\beta$, then $\partial^\alpha f = \sum_{\beta \ge \alpha} \binom{\beta}{\alpha} c_\beta x^{\beta-\alpha}$. The integer binomial coefficients make the formula valid in every characteristic, without factorial inverses.

Lemma 684 (Nilpotent Taylor expansion)**.** *Let I be an ideal of a commutative ring R, with $I^s = 0$. If $a, b \in R^m$ differ coordinatewise by elements of I, then*

$$\boxed{f(a) = \sum_{|\alpha| < s} (\partial^\alpha f)(b)(a-b)^\alpha.}$$

Proof. Substitute $z = a - b$ in the Hasse–Taylor identity. For $|\alpha| \ge s$, the factor $(a-b)^\alpha$ lies in $I^{|\alpha|} \subseteq I^s = 0$. □

We will always have $s \le m$. The number of needed derivatives is

$$\binom{m+s-1}{s-1} \le \binom{2m}{m} < 4^m.$$

They can be constructed from the coefficient formula in $O(4^m m d^m)$ ring operations. Pascal's recurrence supplies the needed binomial coefficients without division. For each query, powers $(a-b)^\alpha$ can be generated by extending one smaller multi-index at a time, rather than exponentiating each monomial independently.

17.6.2 Recursive Prime-Power Reduction

Reduction modulo primes. For coefficients and points modulo r, lift their representatives to $\{0, \ldots, r-1\}$. The integer value of the lifted polynomial is less than $d^m r^{dm}$. It is therefore determined by its residues modulo primes whose product exceeds this bound. The small-prime-product theorem permits primes of size $O(md(1 + \log r))$. Repeating the reduction decreases the modulus, but a terminal full grid inherits a side length proportional to md, up to logarithmic factors. Its extra m^m factor is expensive when there are many variables.

Using prime powers increases the reconstruction modulus without enlarging the base residue grid.

Lemma 685 (One prime-power reduction)**.** *Let $f \in (\mathbb{Z}/r^s\mathbb{Z})[x_1, \ldots, x_m]$, with $r \geq 2$, $1 \leq s \leq m$, and individual degree less than $d \geq 2$. Evaluation at N points reduces to fewer than 4^m families of evaluations over rings $\mathbb{Z}/p^m\mathbb{Z}$, where*

$$p \leq Cd(1 + \log r)$$

for an absolute constant C. Each family contains one problem per chosen prime, on the same number N of points.

Proof. Let $\bar{a}_i \in \{0, \ldots, r-1\}^m$ be the coordinatewise residue of a_i modulo r. The ideal generated by r in $\mathbb{Z}/r^s\mathbb{Z}$ has s-th power zero, so

$$f(a_i) = \sum_{|\alpha|<s} (\partial^\alpha f)(\bar{a}_i)(a_i - \bar{a}_i)^\alpha.$$

For each needed derivative, choose its integer lift F_α with coefficients in $[0, r^s)$. Then

$$0 \leq F_\alpha(\bar{a}_i) < d^m r^{s+m(d-1)} \leq (dr^d)^m.$$

Choose distinct primes $p \leq Cd(1 + \log r)$ with $\prod p > dr^d$, using the small-prime-product bound. Their m-th powers satisfy

$$\prod p^m > (dr^d)^m > F_\alpha(\bar{a}_i).$$

Thus evaluating F_α at $\bar{a}_i$ modulo every p^m and applying CRT recovers the integer value exactly. Reduce it modulo r^s, then use the Taylor formula. There are fewer than 4^m derivative indices. □

The contrast is between $\prod p > (dr^d)^m$ and $\prod p^m > (dr^d)^m$. The second inequality needs only $\prod p > dr^d$, so the base primes no longer carry the factor m. The recursive points are reduced modulo p^m, not merely modulo p; the next Taylor expansion handles their higher digits.

Base case and recursion. At depth zero, reduce every query point modulo the current base r. Evaluate all needed Hasse derivatives on the full grid $\{0, \ldots, r-1\}^m$, over the ring $\mathbb{Z}/r^s\mathbb{Z}$, and reconstruct the query values by nilpotent Taylor expansion. Grid evaluation works even when r is composite.

Algorithm 110 Recursive multivariate evaluation over a prime-power ring

```
function EvaluatePrimePower(f, (a_i)_{i=1}^N, r, s, t)
    ā_i ← a_i mod r, with coordinates in {0, ..., r − 1}
    for multi-indices α with |α| < s do
        Compute ∂^α f by its Hasse coefficient formula
    if t = 0 then
        for |α| < s do
            Evaluate ∂^α f on {0, ..., r − 1}^m over Z/r^s Z
            Read its values at ā_i, obtaining v_{α,i}
    else
        Choose distinct primes p ≤ Cd(1 + log r) with ∏ p > dr^d
        for |α| < s do
            Lift ∂^α f to F_α ∈ Z[x_1, ..., x_m] with coefficients in [0, r^s)
            for each chosen prime p do
                v_{p,i} ← EvaluatePrimePower(F_α mod p^m, (ā_i mod p^m)_i, p, m, t − 1)
            Recover F_α(ā_i) as an integer by CRT modulo ∏ p^m
            v_{α,i} ← F_α(ā_i) mod r^s
    return (∑_{|α|<s} v_{α,i}(a_i − ā_i)^α mod r^s)_{i=1}^N
```

Modulus size and branching. Each reduction replaces the base modulus by at most $Cd(1+\log r)$. Count both the derivative indices and the auxiliary primes when bounding the number of children, retaining their dependence on m, d.

Lemma 686 (Modulus shrinkage)**.** *After a fixed number $t \geq 1$ of reductions, every base modulus is*

$$O_t\big(d(1+\log d+\log^{\circ t} r)\big).$$

The product of the branching bounds contributed by the auxiliary primes alone is $\widetilde{O}_t(d^t(1+\log r))$.

Proof. Set $R_0 = r$ and $R_{j+1} = \lceil Cd(1+\log R_j)\rceil$. Induction gives $R_j = O_j(d(1+\log d+\log^{\circ j} r))$. There are at most R_{j+1} prime choices at level j. The first factor is $O(d(1+\log r))$; each later factor is d times a polynomial in $\log(d+2)$ and $\log(\log r+2)$, for fixed t. Multiplying gives the claim. □

Theorem 687 (Evaluation over prime-power rings)**.** *For fixed $t \geq 1$, the preceding algorithm evaluates $f \in (\mathbb{Z}/r^s\mathbb{Z})[x_1,\dots,x_m]$, with $1 \leq s \leq m$, at N points in*

$$\boxed{\widetilde{O}_t\left(2^{O((t+1)m)}m^3d^t(1+\log r)^2\left([d(1+\log d+\log^{\circ t} r)]^m+N\right)\right)}$$

bit operations.

Proof. With the bounds R_j from the lemma, the number of leaf instances is at most $4^{mt}\prod_{j=1}^{t} R_j$. A leaf evaluates fewer than 4^m derivatives on a grid of at most R_t^m points. Product-grid evaluation, derivative construction, and the final Taylor sums therefore have total cost bounded by

$$\widetilde{O}_t\left(4^{m(t+1)}m^3(1+\log r+\log d)\left(\prod_{j=1}^{t} R_j\right)(d^m+R_t^m+N)\right)$$

bit operations: a residue at level j has $O(m\log R_j)$ bits, and fast integer arithmetic supplies the polynomial products and reductions.

The same expression covers the internal nodes. Their derivative and query arrays have $O(4^m(d^m+mN))$ entries per node. Reducing them into all child rings costs quasi-linear time in their input and output bit lengths. Balanced CRT combines the child residues in quasi-linear time in the aggregate modulus bit length. The reconstruction cost is therefore charged to the aggregate bit length of the child data.

There are only t internal depths. Substitute the bounds on R_t and $\prod_j R_j$ from Lemma 686, absorbing powers of logarithms of d, m, and of bit lengths in soft-O. The remaining two factors $1+\log r$ give the stated bound. □

17.6.3 Extension Fields

Suppose $\mathbb{F}_q = \mathbb{F}_p[z]/(E(z))$, with $e = \deg E$. Represent each field element by a degree-$< e$ polynomial whose integer coefficients lie in $\{0,\dots,p-1\}$. Lift the input polynomial to $F(x,z) \in \mathbb{Z}[x_1,\dots,x_m,z]$ and each query to $\widetilde{a}_i(z) \in \mathbb{Z}[z]^m$. The integer polynomial

$$G_i(z) = F(\widetilde{a}_i(z), z)$$

satisfies $f(a_i) = G_i \bmod (p,E)$, and

$$\deg G_i < edm, \qquad \|G_i\|_1 < d^m(ep)^{m(d-1)+1} \leq \big(d(ep)^d\big)^m.$$

The norm bound follows by submultiplicativity of the coefficient ℓ_1-norm. All lifted coefficients are nonnegative.

Recover each G_i from its first m Taylor coefficients at ed points by Hermite interpolation.

Lemma 688 (Extension-field reduction)**.** *There is an integer r' with*

$$d(ep)^d \le r' < 2ed^2(ep)^d$$

such that the extension-field evaluation problem reduces to fewer than $e4^m$ evaluations over $\mathbb{Z}/(r')^m\mathbb{Z}$, each with edN query points. The additional work is

$$\widetilde{O}\left(e4^m m d^m + e^2 d\, 4^m m^2 N + Nedm\right)$$

operations on auxiliary residues, together with the corresponding coefficient lifts and reductions.

Proof. Choose a prime P with $ed \le P < 2ed$, and let r' be its smallest power at least $d(ep)^d$. This gives the stated interval for r'. Work in $S = \mathbb{Z}/(r')^m\mathbb{Z}$. The ed points $j = 0, \dots, ed-1$ have unit pairwise differences in S.

Write $F(x,z) = \sum_{u=0}^{e-1} f_u(x) z^u$, and set $b_{i,j} = \widetilde{a}_i(j) \in S^m$. Let $A_{i,j}(t) = \widetilde{a}_i(j+t) - b_{i,j} \bmod t^m$. Every coordinate of this vector is divisible by t. Applying nilpotent Taylor expansion in $S[t]/(t^m)$ gives

$$\boxed{G_i(j+t) = \sum_{u=0}^{e-1} (j+t)^u \sum_{|\alpha|<m} (\partial^\alpha f_u)(b_{i,j}) A_{i,j}(t)^\alpha \pmod{t^m}.}$$

Thus fewer than $e4^m$ derivative polynomials, each evaluated at the edN points $b_{i,j}$, supply all the required Taylor data.

For each i, Hermite interpolation at the ed points, each of multiplicity m, recovers $G_i \bmod (r')^m$, since $\deg G_i < edm$. Also $\|G_i\|_1 < (r')^m$, so canonical residue representatives recover its integer coefficients exactly. Finally reduce modulo p and E.

Constructing the derivatives costs $O(e4^m m d^m)$. Obtain the coordinate jets $\widetilde{a}_i(j+t) \bmod t^m$ by Hermite evaluation, and generate the products $A_{i,j}^\alpha$ by extending multi-indices. Their assembly costs $\widetilde{O}(e^2 d\, 4^m m^2 N)$ operations. Hermite reconstruction costs $O(edm \log^2(edm+2))$ per output. All local polynomial operations use monic division and the unit point differences, not factorial inverses. □

Algorithm 111 Reduce extension-field queries to prime-power rings

Input: f and $a_1, \dots, a_N$ over $\mathbb{F}_p[z]/(E)$, recursion depth ℓ
Output: $f(a_1), \dots, f(a_N)$

```
Form the canonical integer lifts F(x,z) = Σ_{u<e} f_u(x) z^u and ã_i(z)
Choose P, r' as in Lemma 688; set S = Z/(r')^m Z
Compute b_{i,j} = ã_i(j) and A_{i,j}(t) = ã_i(j+t) − b_{i,j} mod t^m for 0 ≤ j < ed
for u = 0, ..., e−1 and |α| < m do
    Evaluate ∂^α f_u at all b_{i,j} using EvaluatePrimePower(∂^α f_u, (b_{i,j}), r', m, ℓ)
for i = 1, ..., N do
    Assemble G_i(j+t) mod t^m for each j by the nilpotent Taylor formula
    Reconstruct G_i mod (r')^m by Hermite interpolation
    Lift its coefficients to [0, (r')^m) and reduce modulo p, E
```

Theorem 689 (Elementary multivariate evaluation)**.** *Let $d \ge 2$, and suppose $q \le \exp^{\circ h}(d)$ for a fixed $h \ge 1$. Given $\mathbb{F}_q = \mathbb{F}_p[z]/(E(z))$, with $e = \deg E$, a dense m-variate polynomial of individual degree less than d can be evaluated deterministically at N points in*

$$\boxed{\widetilde{O}_h\left(2^{O(hm)} e m^3 d^{h+3} (1+\log q)^2 \left([d\log(d+2)]^m + edN\right)\right)}$$

bit operations. In particular, the bound has the form $(d^m+N)^{1+o(1)} \operatorname{poly}_h(m, d, \log q)$ as $d \to \infty$.

Proof. Apply the extension-field reduction, then use $t = h+1$ recursive prime-power levels in each of its fewer than $e4^m$ instances. Its auxiliary modulus satisfies $\log r' = O(d(1+\log q)+\log d)$, so $\log^{\circ(h+1)} r' = O(1+\log d)$. The terminal grid size is therefore at most $[C_h d \log(d+2)]^m$.

In Theorem 687, the factor $d^t(1+\log r')^2$ is $\widetilde{O}_h(d^{h+3}(1+\log q)^2)$. Multiply by the number $e4^m$ of reduced instances, retaining their edN query count. Their local Taylor computations, Hermite interpolations, and residue bit lengths are covered by the same bound. This gives the displayed expression.

Finally, $2^{O(hm)} \log(d+2)^m = d^{o(m)}$ for fixed h, and $e \le \log_2 q$. These observations give the last formulation. The field representation is supplied; constructing its defining irreducible polynomial is not included. □

17.7 Multilinear Polynomials

17.7.1 Multilinear-Term Detection

Let $\mathbb{F}$ be a field of characteristic 2, and let $f \in \mathbb{F}[x_1, \dots, x_n]$ have total degree at most k.

For $r = 1, \dots, k$, introduce a vector of new variables

$$z_r = (z_{r,1}, \dots, z_{r,n}).$$

For $J \subseteq [k]$, write $z_J = \sum_{r \in J} z_r$.

Define

$$\boxed{\phi_k f(z_1, \dots, z_k) = \sum_{J \subseteq [k]} f(z_J).}$$

Thus $\phi_k f$ is a polynomial in the kn variables $z_{r,j}$.

This operator is a mixed forward difference.

Forward-Difference Interpretation For a direction $h \in \mathbb{F}^n$, define

$$\Delta_h f(x) = f(x+h) - f(x).$$

Iterating gives

$$(\Delta_{z_1} \cdots \Delta_{z_k} f)(0) = \sum_{J \subseteq [k]} (-1)^{k-|J|} f(z_J).$$

Since $\operatorname{char} \mathbb{F} = 2$, the signs disappear, so

$$\boxed{\phi_k f(z_1, \dots, z_k) = (\Delta_{z_1} \cdots \Delta_{z_k} f)(0).}$$

Thus ϕ_k is exactly the k-fold mixed finite difference of f.

To determine its action on monomials, introduce scalar variables $t_1, \dots, t_k$ and define

$$P_z(t_1, \dots, t_k) = f\left(\sum_{r=1}^{k} t_r z_r\right).$$

For $J \subseteq [k]$, let $1_J \in \{0,1\}^k$ be its indicator vector. Then

$$P_z(1_J) = f\left(\sum_{r \in J} z_r\right) = f(z_J),$$

and consequently

$$\phi_k f(z_1, \dots, z_k) = \sum_{J \subseteq [k]} P_z(1_J).$$

Lemma 690 (Mixed Difference as Coefficient Extraction)**.** *If* $\deg f \le k$*, then*

$$\boxed{\phi_k f(z_1, \dots, z_k) = [t_1 \cdots t_k]\, f\left(\sum_{r=1}^{k} t_r z_r\right).}$$

Proof. Since $\deg f \le k$, the polynomial $P_z(t)$ has total degree at most k.

Consider a monomial

$$c_\alpha(z) t_1^{\alpha_1} \cdots t_k^{\alpha_k}$$

of P_z. When we sum over $t = 1_J$, this monomial contributes $c_\alpha(z)$ exactly for those J containing every r with $\alpha_r > 0$. Hence its total contribution is

$$2^{k - |\operatorname{supp}(\alpha)|} c_\alpha(z).$$

In characteristic 2, this is zero unless

$$|\operatorname{supp}(\alpha)| = k.$$

A surviving monomial must involve all $t_1, \dots, t_k$. Since P_z has degree at most k, its k variables must each have exponent one: their total degree is already k. The only surviving monomial is

$$t_1 t_2 \cdots t_k.$$

Taking its coefficient proves the identity. □

Equivalently, the k-fold finite difference extracts the part of $f(t_1 z_1 + \cdots + t_k z_k)$ in which every direction z_r is used exactly once.

Lemma 691. *Let*

$$M = x_{i_1} \cdots x_{i_d}$$

be a monomial of degree $d \le k$*.*

1. *If* $d < k$*, then* $\phi_k M = 0$*.*
2. *If* $d = k$*, then*

$$\boxed{\phi_k M = \operatorname{perm} Z_M = \det Z_M,}$$

where

$$Z_M = \begin{pmatrix} z_{1,i_1} & z_{1,i_2} & \cdots & z_{1,i_k} \\ z_{2,i_1} & z_{2,i_2} & \cdots & z_{2,i_k} \\ \vdots & \vdots & & \vdots \\ z_{k,i_1} & z_{k,i_2} & \cdots & z_{k,i_k} \end{pmatrix}.$$

In particular, $\phi_k M = 0$ *if* M *is nonmultilinear, while* $\phi_k M \not\equiv 0$ *if* M *is multilinear.*

Proof. By Lemma 690,

$$\phi_k M = [t_1 \cdots t_k] \prod_{\ell=1}^{d} \left(\sum_{r=1}^{k} t_r z_{r,i_\ell}\right).$$

If $d < k$, this product has degree d, so it cannot contain $t_1 \cdots t_k$. Hence $\phi_k M = 0$.

Now suppose $d = k$. To obtain $t_1 \cdots t_k$, the k factors must choose the k variables $t_1, \dots, t_k$ exactly once each. These choices are indexed by permutations $\sigma \in S_k$, so

$$\phi_k M = \sum_{\sigma \in S_k} \prod_{\ell=1}^{k} z_{\sigma(\ell), i_\ell} = \operatorname{perm} Z_M.$$

In characteristic 2, $1 = -1$, so the signs in the determinant disappear:

$$\operatorname{perm} Z_M = \det Z_M.$$

If M is nonmultilinear, then $i_a = i_b$ for some $a \neq b$. The corresponding columns of Z_M are identical, so $\det Z_M = 0$.

If M is multilinear, then $i_1, \ldots, i_k$ are distinct. The determinant is nonzero: for example, the identity-permutation term

$$z_{1,i_1} z_{2,i_2} \cdots z_{k,i_k}$$

does not occur from any other permutation. □

Finite differences annihilate degrees below k. In characteristic 2, the surviving permanent is a determinant, so repeated original variables give repeated columns and are annihilated as well.

Theorem 692 (Multilinear Detection Operator)**.** *Let $f \in \mathbb{F}[x_1, \ldots, x_n]$ have degree at most k, where $\operatorname{char} \mathbb{F} = 2$. Then*

$$\boxed{\phi_k f \not\equiv 0 \iff f \text{ contains a degree-}k \text{ multilinear monomial.}}$$

Proof. Write

$$f = \sum_{I \in \binom{[n]}{k}} c_I x_I + g,$$

where g contains no degree-k multilinear monomial.

By Lemma 691, every term of degree less than k and every nonmultilinear term of degree k is annihilated. Therefore

$$\phi_k f = \sum_{I \in \binom{[n]}{k}} c_I \det Z_I.$$

Different I's cannot cancel. Indeed, every monomial in $\det Z_I$ contains exactly one variable from each row and has second indices precisely the elements of I. Thus if $I \neq I'$, the polynomials $\det Z_I$ and $\det Z_{I'}$ have disjoint monomial supports.

Hence $\phi_k f \equiv 0$ exactly when every $c_I = 0$. □

Black-Box Detection Evaluating $\phi_k f$ requires only evaluations of f. Evaluate f at the 2^k vertices

$$z_J = \sum_{r \in J} z_r \qquad (J \subseteq [k])$$

of the k-dimensional parallelepiped generated by $z_1, \ldots, z_k$.

Theorem 693 (Black-Box Multilinear-Term Detection)**.** *Let*

$$f(x, y) \in \mathbb{F}[x_1, \ldots, x_n, y_1, \ldots, y_m],$$

where $\operatorname{char} \mathbb{F} = 2$. Suppose f has degree at most k in the x-variables and degree at most d in the y-variables. There is a randomized one-sided-error algorithm deciding whether f contains a monomial that is multilinear of degree k in x.

One test uses exactly 2^k black-box evaluations of f. If every random coordinate is chosen independently and uniformly from a finite $S \subseteq \mathbb{F}$, then whenever such a monomial exists, the test succeeds with probability at least

$$\boxed{1 - \frac{d+k}{|S|}.}$$

Proof. Define the operator acting only on x by

$$\phi_k^x f(z_1, \ldots, z_k, y) = \sum_{J \subseteq [k]} f(z_J, y).$$

Write the degree-k multilinear part in x as

$$f(x, y) = \sum_{I \in \binom{[n]}{k}} \lambda_I(y) x_I + g(x, y),$$

where g contains no degree-k multilinear monomial in x. Then

$$\phi_k^x f(z, y) = \sum_I \lambda_I(y) \det Z_I.$$

Since the determinant supports for distinct I's are disjoint,

$$\phi_k^x f \not\equiv 0$$

if and only if at least one coefficient polynomial $\lambda_I(y)$ is nonzero.
Each determinant has degree k in the z-variables, while each λ_I has degree at most d in y. Hence

$$\deg(\phi_k^x f) \leq d + k.$$

Schwartz–Zippel therefore gives

$$\Pr[\phi_k^x f(z_1, \ldots, z_k, y) = 0] \leq \frac{d+k}{|S|}$$

whenever the polynomial is nonzero.
For fixed $z_1, \ldots, z_k, y$, its value is

$$\phi_k^x f(z_1, \ldots, z_k, y) = \sum_{J \subseteq [k]} f\left(\sum_{r \in J} z_r, y\right),$$

using one black-box query for each of the 2^k subsets $J \subseteq [k]$.
If no desired monomial exists, then $\phi_k^x f \equiv 0$, so the algorithm never produces a false positive. □

Algorithm 112 Black-Box Multilinear-Term Detection

Input: Black-box access to $f(x, y)$, target degree k, finite set S
Output: Whether f contains a degree-k multilinear monomial in x

```
Choose every coordinate of z_1, ..., z_k and y independently and uniformly from S
R ← 0
for every J ⊆ [k] do
    x^(J) ← Σ_{r∈J} z_r
    R ← R + f(x^(J), y)
if R ≠ 0 then
    return Yes
else
    return No
```

Independent repetitions decrease the false-negative probability geometrically. A larger finite extension of characteristic 2 may be used when the evaluation procedure accepts extension-field inputs, as an arithmetic circuit does. This is an additional requirement for a black box restricted to base-field inputs.

17.8 Algebraic Series, D-Finiteness, and Recurrence Algorithms

Rational series correspond to constant-coefficient recurrences, and D-finite series to polynomial-coefficient recurrences. Algebraic equations and diagonal representations provide ways to derive these recurrences. Over finite fields, rational diagonals also permit coefficient extraction by processing the digits of the index.

Throughout, complexity counts field operations. The coefficient field has characteristic zero except in the explicitly finite-field part. In evaluation bounds, differential orders, recurrence orders, and matrix dimensions are fixed unless stated otherwise. Evaluation algorithms take the defining equation and the coefficient data selecting its solution as input.

Write $D = d/dx$, $\theta = xD$, and $z_{\underline{i}} = z(z-1)\cdots(z-i+1)$, with $z_{\underline{0}} = 1$.

17.8.1 Differential Equations and Coefficient Recurrences

A series $F \in K[[x]]$ is *D-finite* if

$$\sum_{i=0}^{r} p_i(x) F^{(i)}(x) = 0$$

for polynomials p_i, not all zero.

Equivalently, its derivatives span a finite-dimensional space over $K(x)$. A finite-dimensional span gives an equation by dependence. Conversely, solve an equation with $p_r \neq 0$ for $F^{(r)}$ and differentiate repeatedly; all later derivatives remain in the span of $F, \ldots, F^{(r-1)}$.

A sequence $(a_n)_{n\geq 0}$ is *P-recursive* if

$$\sum_{j=0}^{s} q_j(n) a_{n+j} = 0$$

for all sufficiently large n, where the q_j's are polynomials, not all zero. Reindexing permits backward shifts instead. We interpret $a_n = 0$ for $n < 0$.

The coefficient dictionary. The Euler operator satisfies $\theta x^n = n x^n$, so

$$\boxed{[x^n]Q(\theta)F = Q(n)a_n, \qquad [x^n]x^j Q(\theta)F = Q(n-j)a_{n-j}.} \tag{370}$$

In the opposite operator order, $[x^n]Q(\theta)x^j F = Q(n)a_{n-j}$.

Also

$$\boxed{x^i D^i = \theta_{\underline{i}},}$$

because both operators send x^n to $n_{\underline{i}} x^n$. Operator products mean composition, with the rightmost operation performed first.

Theorem 694 (D-finite if and only if P-recursive)**.** *For $F(x) = \sum_{n\geq 0} a_n x^n$, the series F is D-finite if and only if (a_n) is P-recursive.*

Proof. Suppose $\mathcal{L}F = 0$, where $\mathcal{L} = \sum_{i=0}^{r} p_i(x) D^i \neq 0$. Multiplying by x^r,

$$x^r \mathcal{L} = \sum_i p_i(x) x^{r-i} \theta_{\underline{i}} = \sum_j x^j Q_j(\theta).$$

The Q_j's are not all zero: after grouping by powers of x, the falling factorials of distinct orders are linearly independent. Taking coefficients gives

$$\sum_j Q_j(n-j) a_{n-j} = 0.$$

Conversely, write a P-recurrence as $\sum_{j=0}^{s} q_j(n) a_{n-j} = 0$ for $n \geq n_0$. Multiply every q_j by $n_{\underline{n_0}}$, making the recurrence valid for every $n \geq 0$. The coefficient dictionary gives

$$\boxed{\left(\sum_{j=0}^{s} q_j(\theta) x^j \right) F = 0.}$$

The operator is nonzero: on x^m, its output is $\sum_j q_j(m+j) x^{m+j}$. The powers are distinct, and a nonzero polynomial cannot vanish at all sufficiently large integers in characteristic zero. □

In the converse, x^j shifts the sequence and $q_j(\theta)$ weights the resulting coefficient index.

Sequential recovery and exceptional indices.

Lemma 695 (Coefficient recovery)**.** *Let $\mathcal{L} = \sum_{i=0}^{r} p_i(x) D^i \neq 0$, and suppose $\mathcal{L}H = R$, where $H = \sum_{n\geq 0} h_n x^n$. For fixed order and coefficient degrees, $h_0, \ldots, h_N$ can be computed in $O(N+1)$ operations from the required coefficients of R and at most r prescribed coefficients of H.*

The prescribed indices are the nonnegative integer roots of an explicit nonzero polynomial of degree at most r. All supplied data are assumed consistent with a solution.

Proof. Expand

$$x^r \mathcal{L} = \sum_{j=\nu}^{J} x^j Q_j(\theta), \qquad Q_\nu \neq 0,$$

where ν is the smallest occurring exponent. Taking the coefficient of $x^{n+\nu}$,

$$\boxed{Q_\nu(n) h_n = [x^{n+\nu-r}] R - \sum_{j=\nu+1}^{J} Q_j(n+\nu-j) h_{n+\nu-j}.} \tag{371}$$

Negative-index coefficients are zero. Every coefficient of H on the right has smaller index than n.

When $Q_\nu(n) \neq 0$, solve for h_n. Otherwise use its prescribed value. Since $\deg Q_\nu \leq r$, there are at most r exceptional indices. The recurrence width is fixed, so every ordinary step costs $O(1)$.

If $d = \max_i \deg p_i$, then $J \leq r + d$, so coefficients of R through degree $N + d$ suffice. □

The exceptional coefficients need not form an initial consecutive block. For example, $(\theta - m)F = 0$ has solutions $F = cx^m$: the free coefficient occurs at index m, although the equation has order one.

The rational special case. A series is rational exactly when its coefficients satisfy an eventual constant-coefficient recurrence. Indeed, if $F = P/Q$, with $Q(0) = 1$, then $QF = P$ gives such a recurrence beyond $\deg P$. Conversely, a constant-coefficient recurrence makes QF polynomial for the corresponding denominator Q.

We use the forward-shift convention for characteristic polynomials: $P(Z) = Z^r + \sum_{j=0}^{r-1} p_j Z^j$ annihilates a sequence when $a_{n+r} + \sum_{j<r} p_j a_{n+j} = 0$. The corresponding generating-function connection polynomial is $Q(x) = x^r P(1/x)$. When a recurrence routine returns Q, reverse it with the prescribed bound r before treating its roots as exponential bases. Keep that bound even if $\deg Q < r$.

Constant-coefficient recurrences admit logarithmic-in-the-index evaluation by Bostan–Mori or matrix powering. For polynomial-coefficient recurrences, the block-product algorithm below has square-root dependence on the index.

17.8.2 Algebraic Series and Closure Properties

Theorem 696 (Algebraic series are D-finite)**.** *If $F \in K[[x]]$ is algebraic of degree $d = [K(x, F) : K(x)]$, then it satisfies a homogeneous differential equation of order at most d.*

Proof. Let $P(x, y) \in K(x)[y]$ be its minimal polynomial. Characteristic zero makes P separable, so $P_y(x, F) \neq 0$. Differentiating,

$$F' = -\frac{P_x(x, F)}{P_y(x, F)} \in K(x, F).$$

The product and quotient rules show that differentiation preserves $K(x, F)$. Thus $F, F', \ldots, F^{(d)}$ lie in a d-dimensional $K(x)$-space. A dependence, followed by clearing denominators, gives the equation. □

To construct the equation, work in $K(x)[y]/(P)$ with basis $1, y, \ldots, y^{d-1}$. Extended Euclid computes $P_y^{-1} \bmod P$. Set $H = -P_x P_y^{-1} \bmod P$; differentiation of a representative $R(x, y)$ is then

$$\boxed{R \longmapsto R_x + R_y H \pmod{P}.}$$

Starting from $y \bmod P$, compute the vectors for $F, F', \ldots, F^{(d)}$ and find a dependence by elimination over $K(x)$.

The minimal polynomial describes possible branches; initial coefficient data specify which branch is being computed.

Theorem 697 (Closure of D-finite series)**.** *D-finite series are closed under linear combinations, products, differentiation, integration, and Hadamard products*

$$(F \odot G)(x) = \sum_{n \geq 0} ([x^n]F)([x^n]G)x^n.$$

If A is algebraic with $A(0) = 0$, then $F(A(x))$ is D-finite whenever F is D-finite.

Proof. Let V_F, V_G be the finite-dimensional derivative spans over $K(x)$. Linear combinations lie in $V_F + V_G$; derivatives of a product lie, by Leibniz' rule, in the span of products of basis elements. Differentiation preserves a finite derivative span, and adjoining one antiderivative increases its dimension by at most one.

For substitution, the case $A = 0$ is immediate. Otherwise A has positive valuation. If a nonzero polynomial $p(t)$ has lowest nonzero term ct^j, then $p(A)$ has lowest term cA^j and is not zero. Thus substituting A in the differential equation for F does not destroy its leading coefficient. That equation reduces the functions $F^{(j)}(A)$ to a finite spanning family over $K(x, A)$. The chain rule keeps all derivatives in that span, and $K(x, A)$ is a finite extension of $K(x)$.

For the Hadamard product, write $F = \sum f_n x^n$, $G = \sum g_n x^n$. If either sequence is eventually zero, the conclusion is immediate. Otherwise choose P-recurrences of positive orders r, s. Beyond their exceptional indices, every forward shift of f_n is a $K(n)$-linear combination of $f_n, \ldots, f_{n+r-1}$, and similarly for g_n. Thus every shifted product $f_{n+\ell} g_{n+\ell}$ is a combination of the same rs products $f_{n+i} g_{n+j}$. A dependence among $rs + 1$ coefficient vectors gives a P-recurrence. □

Each closure construction represents derivatives or shifts in a finite spanning family and finds a linear dependence. The resulting equation need not have minimum order. Equation construction is separate from coefficient evaluation.

17.8.3 Root Power Sums and Products of Rational Sequences

For a monic polynomial $f(X) = \prod_{i=1}^{m}(X - \alpha_i)$, define $s_0(f) = m$, $s_k(f) = \sum_i \alpha_i^k$, and $f^*(z) = z^m f(1/z)$. Then

$$\boxed{\begin{aligned}\sum_{k\ge 0} s_k(f) z^k &= m - z\frac{(f^*)'(z)}{f^*(z)},\\ f^*(z) &= \exp\left(-\sum_{k\ge 1}\frac{s_k(f)}{k} z^k\right).\end{aligned}} \tag{372}$$

Both formulas follow from $\log f^* = \sum_i \log(1 - \alpha_i z)$.

Thus polynomial coefficients and root power sums can be converted in $O(D\log(D+2))$ operations through index D. Zero roots are accounted for by the specified degree and s_0.

Theorem 698 (Composed sums and products)**.** *Given monic polynomials of positive degrees m, n,*

$$f(X) = \prod_{i=1}^{m}(X - \alpha_i), \qquad g(X) = \prod_{j=1}^{n}(X - \beta_j),$$

one can compute

$$f \oplus g = \prod_{i,j}(X - \alpha_i - \beta_j), \qquad f \otimes g = \prod_{i,j}(X - \alpha_i\beta_j)$$

in

$$\boxed{O(mn\log(mn+2))}$$

field operations, without finding individual roots.

Proof. Compute both power-sum sequences through $D = mn$. For products,

$$s_k(f \otimes g) = s_k(f)s_k(g).$$

For sums, the binomial theorem gives

$$s_k(f \oplus g) = \sum_{\ell=0}^{k}\binom{k}{\ell} s_\ell(f)s_{k-\ell}(g).$$

This is one convolution after factorial scaling. Recover the degree-D outputs using (372). □

Algorithm 113 Composed root sums or products

Input: Monic f, g, operation $\circ \in \{+, \times\}$
Output: $\prod_{i,j}(X - (\alpha_i \circ \beta_j))$

```
D ← (deg f)(deg g)
Compute s_0, ..., s_D and t_0, ..., t_D by logarithmic differentiation
if ∘ = × then
    u_k ← s_k t_k
else
    Convolve (s_k/k!) and (t_k/k!)
    u_k ← k! times the k-th convolution coefficient
H(z) ← exp(−Σ_{k=1}^{D} u_k z^k/k) mod z^{D+1}
return X^D H(1/X)
```

Hadamard and binomial products of recurrence sequences. Suppose (a_k), (b_k) satisfy constant-coefficient recurrences from their initial states, with characteristic polynomials f, g of positive degrees m, n. Then

$$c_k = a_k b_k, \qquad d_k = \sum_{j=0}^{k} \binom{k}{j} a_j b_{k-j}$$

are annihilated respectively by $f \otimes g$ and $f \oplus g$.

Use companion-matrix states $a_k = u^{\mathsf{T}} A^k v$, $b_k = s^{\mathsf{T}} B^k t$. The two sequences are matrix coefficients of

$$(A \otimes B)^k, \qquad (A \otimes I + I \otimes B)^k.$$

Triangularizing A, B over a splitting field shows that the two characteristic polynomials are $f \otimes g$ and $f \oplus g$. Cayley–Hamilton proves the claims.

The annihilating recurrences and their first mn terms can therefore be constructed in $O(mn \log(mn+2))$ operations. The recurrences need not be minimal.

The Hadamard product of an algebraic series with a rational series is algebraic in characteristic zero. After partial fractions, a rational coefficient sequence is a finite sum of $P(n)\lambda^n$, apart from finitely many initial terms. Hadamard multiplication by such a sequence sends F to $P(\theta)F(\lambda x)$, which is algebraic. Finite changes add only a polynomial. Partial fractions may be taken in a finite splitting field; algebraicity over that finite extension implies algebraicity over $K(x)$ by transitivity of algebraic extensions.

17.8.4 Sparse Interpolation through Rational Recurrences

Sparse interpolation recovers a polynomial from a bound on its number of nonzero terms. Geometric evaluations turn the unknown exponents into the bases of exponential sequences, which can be recovered from a recurrence. The construction works over any field containing the required element ω.

Suppose

$$f(x) = \sum_{i=1}^{t} c_i x^{e_i}, \qquad t \le s, \quad 0 \le e_i \le D,$$

with distinct exponents and nonzero coefficients. Choose $\omega \in K^{\times}$ of multiplicative order greater than D. Then

$$b_j = f(\omega^j) = \sum_{i=1}^{t} c_i \gamma_i^j, \qquad \gamma_i = \omega^{e_i},$$

is a sum of exponentials with distinct nonzero bases.

Lemma 699 (Prony recurrence)**.** *The minimal recurrence polynomial of (b_j) is*

$$P(Z) = \prod_{i=1}^{t} (Z - \gamma_i).$$

It is determined by $b_0, \dots, b_{2s-1}$.

Proof. The displayed polynomial annihilates each exponential sequence. Conversely, an annihilator Q gives

$$\sum_i c_i Q(\gamma_i) \gamma_i^j = 0.$$

For $j = 0, \dots, t-1$, Vandermonde nonsingularity forces $Q(\gamma_i) = 0$ for every i. Thus no smaller-degree annihilator exists.

The leading $t \times t$ Hankel matrix factors as $V \operatorname{diag}(c_i) V^{\mathsf{T}}$, so it is nonsingular. The coefficient sequence has order $t \le s$, so Corollary 668 shows that $2s$ terms suffice to recover the infinite sequence's minimum recurrence. □

Once the roots γ_i are known, let

$$Q(z) = z^t P(1/z), \qquad R(z) = Q(z) \sum_{j=0}^{t-1} b_j z^j \bmod z^t.$$

Then

$$\sum_{j \geq 0} b_j z^j = \sum_i \frac{c_i}{1 - \gamma_i z} = \frac{R(z)}{Q(z)},$$

so

$$\boxed{c_i = -\gamma_i \frac{R(\gamma_i^{-1})}{Q'(\gamma_i^{-1})}.}$$

Multipoint evaluation recovers all amplitudes in $O(t \log^2(t+2))$ operations.

Theorem 700 (Sparse interpolation)**.** *The polynomial f is determined by $2s$ geometric evaluations. Beyond those evaluations, reconstruction costs $O(s \log^2(s+2))$ field operations, plus finding the roots of the recovered polynomial and decoding e_i from $\gamma_i = \omega^{e_i}$.*

Proof. Recover the recurrence, find its distinct roots, recover amplitudes by the displayed formula, and decode the exponents. □

Over a finite field, exponent decoding is a bounded discrete-logarithm problem. A table $1, \omega, \ldots, \omega^D$ gives direct lookup after $O(D)$ preprocessing, in addition to the stated bound.

Algorithm 114 Sparse interpolation

Input: Evaluation access to f, bounds s, D, $\operatorname{ord}(\omega) > D$
Output: The nonzero monomials of f

```
Evaluate b_j = f(ω^j) for 0 ≤ j < 2s
Recover the monic minimal recurrence polynomial P in the forward-shift convention
if P = 1 then
    return the zero polynomial
Find its roots γ_1, ..., γ_t
Q(z) ← z^t P(1/z)
R(z) ← Q(z) Σ_{j=0}^{t-1} b_j z^j mod z^t
Evaluate R, Q' at all γ_i^{-1}
for i = 1, ..., t do
    c_i ← −γ_i R(γ_i^{-1}) / Q'(γ_i^{-1})
    Decode e_i ∈ [0, D] with ω^{e_i} = γ_i
return Σ_i c_i x^{e_i}
```

For individual multivariate degree bounds $D_1, \ldots, D_r$, use mixed-radix substitution

$$x_i \longmapsto x^{\prod_{j<i}(D_j+1)}.$$

This preserves sparsity and encodes exponent vectors injectively. The univariate degree bound becomes $\prod_i (D_i + 1) - 1$.

If evaluations of $x_\ell \partial f / \partial x_\ell$ are additionally available, their exponential bases are the same γ_i, while their amplitudes are $e_{i,\ell} c_i$. Recovering these amplitudes and dividing by c_i gives the exponents in the field. When the characteristic exceeds the individual degree bounds, this replaces discrete logarithms by direct decoding. This variant requires access to those derivative evaluations.

17.8.5 Distant Polynomial and Geometric Recurrences

Write a polynomial-coefficient recurrence as a matrix transition. Suppose $r \geq 1$ and $\sum_{j=0}^{r} q_j(n) a_{n+j} = 0$, with $q_r \neq 0$, on the indices being processed. Use the state $u_n = (a_n, \ldots, a_{n+r-1})^{\mathsf{T}}$. Let $A(x)$ have $q_r(x)$ on its superdiagonal, last row $(-q_0(x), \ldots, -q_{r-1}(x))$, and all other entries zero. At every nonsingular index,

$$\boxed{u_{n+1} = \frac{A(n)}{q_r(n)} u_n.}$$

The first $r-1$ rows shift the known coefficients; the last row solves for the new one. This description also covers $r = 1$. A recurrence polynomial in γ^n gives the same construction with that quantity as the matrix parameter.

Group consecutive transitions into polynomial blocks. First accumulate the scalar denominators in one extra coordinate:

$$u_{j+1} = \frac{A(\xi_j)}{q(\xi_j)} u_j \quad \Longrightarrow \quad z_{j+1} = T(\xi_j) z_j, \qquad T(x) = \operatorname{diag}(A(x), q(x)), \qquad z_0 = (u_0, 1)^{\mathsf{T}}.$$

Here $Q_j = \prod_{i<j} q(\xi_i)$ and $z_j = (Q_j u_j, Q_j)^{\mathsf{T}}$. Multiplication by $\operatorname{diag}(A(\xi_j), q(\xi_j))$ updates both the numerator and the denominator. Divide by the last coordinate at the end.

Consider either $\xi_j = j$, corresponding to polynomial-coefficient recurrences, or $\xi_j = \gamma^j$, corresponding to q-holonomic recurrences, with $\gamma \neq 0$. Let $\sigma(x) = x + 1$ or $\sigma(x) = \gamma x$, respectively. Define the block polynomial

$$B_b(x) = T(\sigma^{b-1}(x)) \cdots T(x).$$

In both cases,

$$\boxed{B_{2b}(x) = B_b(\sigma^b(x)) B_b(x).}$$

For additive shifts, computing $B_b(x + b)$ is a Taylor shift. For multiplicative shifts, computing $B_b(\gamma^b x)$ only scales coefficients. In either case, a block of length b is constructed in $O(b \log(b+2))$ operations by doubling.

Evaluating the block starting points costs $O(b \log^2(b+2))$ by ordinary multipoint evaluation, or $O(b \log(b+2))$ for a geometric progression.

Theorem 701 (Distant recurrence terms)**.** *For fixed matrix dimension and polynomial degrees, a target state at index $N \geq 1$ can be computed in*

$$\boxed{\begin{array}{ll} O(\sqrt{N} \log^2(N+2)), & \xi_j = j, \\ O(\sqrt{N} \log(N+2)), & \xi_j = \gamma^j, \quad \gamma \neq 0. \end{array}}$$

All scalar denominators used by the target computation must be nonzero. The initial state is supplied separately.

Proof. Choose a power of two $b \in [\sqrt{N}, 2\sqrt{N})$. Construct B_b, then evaluate it at the $L = \lfloor N/b \rfloor$ block starting points $\xi_0, \xi_b, \ldots, \xi_{(L-1)b}$. Apply these numerical matrices and process the remaining fewer than b transitions directly.

The construction and numerical applications cost $O(\sqrt{N} \log(N+2))$. Use ordinary or geometric multipoint evaluation for the stated bounds. □

Algorithm 115 A distant recurrence term

Input: $u_0, A(x), q(x), N$, and either $\xi_j = j$ or $\xi_j = \gamma^j$
Output: u_N, assuming nonzero required denominators

if $N = 0$ **then**
  return u_0
$T(x) \leftarrow \operatorname{diag}(A(x), q(x))$
$z \leftarrow (u_0, 1)^{\mathsf{T}}$
Choose a power of two $b \in [\sqrt{N}, 2\sqrt{N})$
$B(x) \leftarrow T(x)$, $a \leftarrow 1$
while $a < b$ **do**
  $B(x) \leftarrow B(\sigma^a(x))B(x)$
  $a \leftarrow 2a$
$L \leftarrow \lfloor N/b \rfloor$
Evaluate B at ξ_{jb}, $0 \le j < L$
for $j = 0, \dots, L-1$ **do**
  $z \leftarrow B(\xi_{jb})z$
for $j = Lb, \dots, N-1$ **do**
  $z \leftarrow T(\xi_j)z$
return the numerator coordinates divided by the last coordinate

The geometric evaluation uses

$$\lambda^{ij} = \lambda^{\binom{i+j}{2}}\lambda^{-\binom{i}{2}}\lambda^{-\binom{j}{2}},$$

which turns the evaluation into a convolution. Repeated geometric evaluation points cause no difficulty; only the interpolation algorithm in Section 17.5.4 needs them to be distinct.

Examples are the factorial recurrence $u_{j+1} = (j+1)u_j$ and the q-Pochhammer product $(a;\gamma)_N = \prod_{j=0}^{N-1}(1 - a\gamma^j)$.

For a polynomial-index recurrence, a fixed nonzero denominator polynomial has only finitely many singular integer indices. If the coefficients needed at those indices are supplied, process them directly and apply the block algorithm to the intervening nonsingular intervals, shifting the polynomial parameter at the start of each interval. With fixed recurrence complexity, there are only constantly many such intervals, so the asymptotic bound is unchanged. The coefficients at singular indices must still be supplied.

Products along a geometric progression. For scalar series, logarithms reduce a product of geometrically scaled arguments to coefficientwise multiplication. Let $f(0) = 1$, and write $\log f(x) = \sum_{r \ge 1} \ell_r x^r$. Then

$$\boxed{\log \prod_{j=0}^{k-1} f(\gamma^j x) = \sum_{r \ge 1} \ell_r S_k(\gamma^r) x^r, \qquad S_k(a) = \sum_{j=0}^{k-1} a^j.} \tag{373}$$

Indeed, logarithms turn the product into a sum, and scaling the argument multiplies its r-th coefficient by γ^{jr}.

For $k \ge 1$, evaluate the scalar weight by

$$S_k(a) = \begin{cases} (a^k - 1)/(a - 1), & a \ne 1, \\ k, & a = 1. \end{cases}$$

The second case covers roots of unity γ, including $\gamma = 1$.

Corollary 702 (A truncated geometric product)**.** *Given $f \bmod x^n$, with $n \ge 1$, $f(0) = 1$, and an integer $k \ge 0$, the product $\prod_{j=0}^{k-1} f(\gamma^j x) \bmod x^n$ is computed in*

$$\boxed{O(n \log(n+2) + \log(k+1))}$$

field operations. No distinctness condition on the powers of γ is needed.

Proof. For $k = 0$, return one. Otherwise compute $\log f$, the scalar γ^k, and the successive powers of γ and γ^k. These give all $S_k(\gamma^r)$, $1 \le r < n$, in linear time. Scale the logarithm coefficients using (373), then exponentiate. □

If $f(0) = c \neq 0$, apply the normalized construction to f/c and multiply the result by c^k. This scalar power adds $O(\log(k+1))$ work.

17.8.6 Batched Recurrences with a Varying Parameter

For several queries, share transition products even when both the stopping index and the parameter value vary. Suppose

$$v_m(x) = \frac{A_m(x)}{q_m(x)} v_{m-1}(x), \qquad v_0(x) = \frac{b_0(x)}{q_0(x)},$$

where $q_0, \dots, q_L$ are nonzero polynomials, and query (m_i, x_i) requests $v_{m_i}(x_i)$, with $0 \le m_i \le L$. Use a product tree for the transitions and a remainder tree for the query values. Matrices act on column states from the left, so later transitions appear on the left.

Set

$$Q_m = q_0 \prod_{j=1}^{m} q_j, \qquad z_m = (Q_m v_m, Q_m)^{\mathsf{T}}.$$

Then $z_m = T_m z_{m-1}$, where $T_m = \operatorname{diag}(A_m, q_m)$ and $z_0 = (b_0, q_0)^{\mathsf{T}}$. The cumulative denominator $Q_{m_i}(x_i)$ must be nonzero for every query.

Build a balanced interval tree on $[1, L]$. At $I = [\ell, r]$, store

$$P_I = T_r \cdots T_\ell, \qquad R_I(x) = \prod_{i:\, m_i \in I} (x - x_i).$$

For children $I_L = [\ell, h]$, $I_R = [h+1, r]$, $P_I = P_{I_R} P_{I_L}$ and $R_I = R_{I_L} R_{I_R}$.

During descent, maintain

$$u_I \equiv z_{\ell-1} \pmod{R_I}, \qquad \deg u_I < \deg R_I.$$

The two child states are

$$\boxed{u_{I_L} = u_I \bmod R_{I_L}, \qquad u_{I_R} = P_{I_L} u_I \bmod R_{I_R}.}$$

At a leaf $\{m\}$, evaluate $T_m u_I$ at its query points. Repeated points are allowed; no interpolation is used.

Theorem 703 (Batched recurrence evaluation)**.** *Suppose the matrix dimension is fixed. Let s be the number of queries, d_0 bound the degrees of b_0, q_0, and d_m bound the degrees of A_m, q_m. If*

$$L + s + d_0 + \sum_{m=1}^{L} d_m = O(N),$$

all valid queries can be answered in

$$\boxed{O(N \log^2(N+2))}$$

operations and $O(N \log(N+2))$ space.

Proof. The root invariant holds after reducing z_0 modulo its query polynomial. Each child modulus divides the parent modulus, and multiplication by P_{I_L} advances the right child's starting state through the left interval. Thus the leaf evaluations are correct.

At any fixed tree depth, intervals are disjoint. The aggregate degrees of their transition products are $O(N)$, as are the aggregate degrees of their query polynomials. Building one level, or carrying out its remainder descent, therefore costs $O(N \log(N+2))$. There are $O(\log(N+2))$ depths. The within-leaf product trees and evaluations satisfy the same total bound. □

Algorithm 116 Batched rational recurrence evaluation

Input: Transitions, initial state, queries (m_i, x_i)
Output: All query values with nonzero cumulative denominators

```
Handle m_i = 0 queries by evaluating b_0, q_0; remove them
Stop if no queries remain
Form T_m, z_0 and build an interval tree on [1, L]
for each leaf I = {m} do
    P_I ← T_m
    R_I ← ∏_{i: m_i = m} (x − x_i), using a product tree
for each internal node I, bottom-up do
    P_I ← P_{I_R} P_{I_L}
    R_I ← R_{I_L} R_{I_R}
DESCEND([1, L], z_0 mod R_{[1,L]})

function DESCEND(I, u)
    if R_I = 1 then
        return
    if I is a leaf then
        Evaluate P_I u mod R_I at its query points
        Divide numerator coordinates by the last coordinate
        return
    if R_{I_L} ≠ 1 then
        DESCEND(I_L, u mod R_{I_L})
    if R_{I_R} ≠ 1 then
        DESCEND(I_R, P_{I_L} u mod R_{I_R})
```

A query-free interval may be skipped during descent, but its transition product may still be needed to reach later queries.

Binomial-prefix queries. Put $c_m(x) = \binom{x}{m}$ and $s_m(x) = \sum_{j=0}^{m} \binom{x}{j}$. Then

$$\boxed{\begin{pmatrix} c_m \\ s_m \end{pmatrix} = \frac{1}{m} \begin{pmatrix} x-m+1 & 0 \\ x-m+1 & m \end{pmatrix} \begin{pmatrix} c_{m-1} \\ s_{m-1} \end{pmatrix}, \qquad \begin{pmatrix} c_0 \\ s_0 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.}$$

The matrices have degree one and the cumulative denominator is $m!$. Therefore N queries $\sum_{j=0}^{m_i} \binom{n_i}{j}$, with $0 \le n_i, m_i \le N$, are answered in $O(N \log^2(N+2))$ operations.

17.8.7 Exponential and Logarithmic Substitution

The substitutions $e^x - 1$ and $\log(1+x)$ are compositional inverses. Transposed evaluation and interpolation compute their full coefficient transforms without expanding the individual powers.

For a polynomial $F(t) = \sum_{j=0}^{m-1} f_j t^j$,

$$\boxed{[x^k]F(e^x) = \frac{1}{k!} \sum_{j=0}^{m-1} f_j j^k.} \tag{374}$$

Here $0^0 = 1$. The sums on the right are the transpose of the map that evaluates a degree-$< n$ polynomial at $0, 1, \dots, m-1$. Transposed multipoint evaluation followed by factorial scaling therefore computes $F(e^x)$ mod x^n in $O((m+n)\log^2(m+n+2))$ operations.

The other two substitutions require only square transforms of size n. Let $V_{ij} = i^j$, for $0 \le i, j < n$, let $D = \operatorname{diag}(1/0!, \ldots, 1/(n-1)!)$, and let S_c be the coefficient map $F(t) \mapsto F(t+c)$ on degree-$< n$ polynomials. With coefficient vectors written as columns,

$$F(e^x) \bmod x^n \quad \text{has map} \quad DV^{\mathsf{T}}.$$

To substitute $e^x - 1$, first shift the input polynomial by -1. The resulting map is $DV^{\mathsf{T}}S_{-1}$.

Theorem 704 (Fast exponential and logarithmic substitutions)**.** *For $F \in K[t]_{<n}$, each of*

$$F(e^x), \qquad F(e^x - 1), \qquad F(\log(1+x)) \pmod{x^n}$$

can be computed in $O(n \log^2(n+2))$ field operations. The transposes of these coefficient maps have the same complexity. More explicitly,

$$\boxed{\mathcal{C}_{e^x-1} = DV^{\mathsf{T}}S_{-1}, \qquad \mathcal{C}_{\log(1+x)} = S_1 V^{-\mathsf{T}} D^{-1}.} \tag{375}$$

Proof. The first formula follows from (374). The integers $0, \ldots, n-1$ are distinct, so V is invertible. Since $e^x - 1$ and $\log(1+x)$ have valuation one and are compositional inverses, their substitution maps on $K[[x]]/(x^n)$ are inverse linear maps. Inverting $DV^{\mathsf{T}}S_{-1}$ gives $S_1 V^{-\mathsf{T}} D^{-1}$.

Apply a Taylor shift, transposed evaluation or transposed interpolation, and diagonal scaling in the indicated order. These operations give the claimed bound. Reversing their order and transposing each factor gives the transposed maps within the same bound. □

Algorithm 117 Exponential and logarithmic substitution

Input: $F \in K[t]_{<n}$, precision $n \ge 1$, substitution $\psi \in \{e^x, e^x - 1, \log(1+x)\}$
Output: $F(\psi(x)) \bmod x^n$

```
Precompute factorials and inverse factorials through n − 1
if ψ = log(1 + x) then
    u_j ← j![t^j]F for 0 ≤ j < n
    Apply transposed interpolation at 0, ..., n − 1: v ← V^{−T} u
    return Σ_{j=0}^{n−1} v_j (x + 1)^j, by Taylor shift
else
    if ψ = e^x − 1 then
        F(t) ← F(t − 1)
    Apply transposed evaluation at 0, ..., n − 1: v ← V^T coeff(F)
    return Σ_{j=0}^{n−1} v_j x^j / j!
```

For a formal input series, $F(e^x - 1)$ and $F(\log(1+x))$ require only $F \bmod t^n$. In contrast, $F(e^x)$ is not a formal substitution at zero: every input coefficient can affect its constant term. The first transformation therefore requires a polynomial input, or a separately specified expansion of the function at $t = 1$.

Stirling transforms and power projection. The coefficient identities

$$\frac{(e^x - 1)^j}{j!} = \sum_{k \ge j} \left\{ {k \atop j} \right\} \frac{x^k}{k!}, \qquad \frac{\log(1+x)^j}{j!} = \sum_{k \ge j} s(k, j) \frac{x^k}{k!}$$

show that these are factorial-scaled Stirling transforms; $s(k, j)$ denotes the signed Stirling number of the first kind. Their inverse relation follows from compositional inversion.

For a linear functional $\ell(H) = \sum_{k<n} w_k [x^k] H$, transposing substitution returns $\ell(\psi^0), \ldots, \ell(\psi^{n-1})$. Thus the same factorization also computes all these power projections in $O(n \log^2(n+2))$ operations.

17.8.8 Linear-Time Translation and Single-Coefficient Queries

For a fixed D-finite series, one coefficient after exponential substitution can be computed in linear time. The main step is a linear-time Taylor shift of its truncation.

Applying the original differential equation to the truncation leaves an error in a bounded-width interval of degrees. An additional first-order operator annihilates this error, giving a homogeneous equation of bounded order and coefficient degrees. Translate that equation to recover the shifted polynomial. The error must be annihilated before translation, since translation can move it into low degrees.

An equation for the truncation. Suppose $\mathcal{L}A = 0$, where $\mathcal{L} = \sum_{i=0}^{r} p_i(z)D_z^i$, $p_r \neq 0$, and $\deg p_i \leq d$. Let $T = A \bmod z^{k+1}$.

Lemma 705 (A bounded-order equation for a truncation)**.** *Put $\ell = \max\{0, k+1-r\}$. Either $\mathcal{L}T = 0$, or*

$$\mathcal{L}T = z^\ell R(z), \qquad \deg R \leq r + d - 1,$$

and

$$\boxed{\big(zRD_z - \ell R - zR'\big)\mathcal{L}T = 0.}$$

The displayed operator is nonzero, has order $r+1$, and coefficient degrees at most $r + 2d$.

Proof. Since $T - A = O(z^{k+1})$, the error has no term below degree $k+1-r$, and applying $\mathcal{L}$ to a degree-k polynomial gives degree at most $k + d$. Factoring out z^ℓ proves the degree bound.

For $E = z^\ell R$, $zRE' = (\ell R + zR')E$. This gives the annihilating operator. Its leading differential coefficient is $zRp_r \neq 0$, and the product rule gives the coefficient-degree bound. □

The annihilator comes from $E'/E = \ell/z + R'/R$ after clearing denominators. The exponent ℓ appears only as a scalar, while R has bounded degree.

Translation of the equation. Let $\mathcal{D}T = 0$ be the bounded-size equation from the lemma. For $H(t) = T(t+\beta)$, replace z by $t+\beta$ in the polynomial coefficients of $\mathcal{D}$. This produces an equation for H of the same bounded order and degree.

Its recurrence computes successive $h_n = [t^n]H$. At the at most $r+1$ exceptional indices, use

$$\boxed{h_n = \sum_{j=n}^{k} a_j \binom{j}{n} \beta^{j-n}.} \tag{376}$$

After precomputing factorials and powers of β, each such sum costs $O(k+1)$.

Theorem 706 (Translation of a truncated D-finite series)**.** *For fixed differential order and coefficient degrees, given $\mathcal{L}A = 0$, $a_0, \ldots, a_k$, and β, every coefficient of*

$$H(t) = \sum_{j=0}^{k} a_j (t+\beta)^j$$

can be computed in $O(k+1)$ operations and $O(k+1)$ space.

Proof. Compute $E = \mathcal{L}T$ in $O(k+1)$ operations. Construct the bounded-order equation, translate it, and apply Lemma 695. Each ordinary step costs $O(1)$, and a constant number of exceptional coefficients are computed by (376). □

The construction translates the polynomial T; it does not require the infinite substitution $A(t+\beta)$ to be defined.

Algorithm 118 Translate a truncated D-finite series

Input: $\mathcal{L}A = 0$, $a_0, \ldots, a_k$, shift β
Output: Coefficients of $H(t) = \sum_{j=0}^{k} a_j (t+\beta)^j$

```
T ← Σ_{j=0}^{k} a_j z^j, E ← LT
if E = 0 then
    D ← L
else
    ℓ ← max{0, k + 1 − r}, R ← E/z^ℓ
    D ← (zRD_z − ℓR − zR′)L
Translate the coefficients of D by β
Derive its coefficient recurrence by (371)
Precompute factorials, inverse factorials, and powers of β
for n = 0, ..., k do
    if the coefficient of h_n in the recurrence is nonzero then
        Solve the recurrence for h_n
    else
        Compute h_n using (376)
return h_0, ..., h_k
```

One coefficient after exponential substitution. Let $A(z) = \sum a_j z^j$. The composition $A(e^x - 1)$ is formally defined because $e^x - 1$ has zero constant term. For a function $F(t)$, supply $A(z) = F(1+z)$, its Taylor series about 1.

Since $(e^x - 1)^j$ has valuation j, only $a_0, \ldots, a_k$ affect the target coefficient. Put $T = A \bmod z^{k+1}$ and $H(t) = T(t-1) = \sum h_j t^j$. Then, for $k \geq 1$,

$$\boxed{[x^k]A(e^x - 1) = [x^k]H(e^x) = \frac{1}{k!}\sum_{j=1}^{k} h_j j^k.} \tag{377}$$

Lemma 707 (A table of equal-exponent powers)**.** *For $k \geq 1$, all j^k, $1 \leq j \leq k$, can be computed in $O(k)$ field operations and $O(k)$ elementary integer operations.*

Proof. Use a linear sieve. Compute p^k by binary exponentiation only for primes $p \leq k$; for a composite $j = pa$, use $j^k = p^k a^k$.

Composites cost $O(k)$. Primes cost $O(\pi(k)\log(k+1)) = O(k)$. The elementary estimate $\pi(k) = O(k/\log(k+1))$ follows from the primorial upper bound: primes above $\sqrt{k}$ each contribute at least $\frac{1}{2}\log k$ to $\sum_{p \leq k} \log p = O(k)$. □

Algorithm 119 Equal-exponent power table

```
function PowerTable(k)
    Initialize least-prime-factor array lp to zero
    P ← ∅, v_1 ← 1
    for j = 2, ..., k do
        if lp[j] = 0 then
            lp[j] ← j; append j to P
            v_j ← j^k by binary exponentiation
        for primes p ∈ P, while p ≤ lp[j] and pj ≤ k do
            lp[pj] ← p, v_pj ← v_p v_j
    return v_1, ..., v_k
```

Each composite is generated exactly once, using its least prime factor.

Theorem 708 (One coefficient after exponential substitution)**.** *Given a fixed-order, fixed-degree equation for A, together with the necessary coefficient data specifying its solution,*

$$\boxed{[x^k]A(e^x-1)}$$

can be computed in $O(k+1)$ operations and $O(k+1)$ space. The cost of obtaining the prescribed input coefficients is additional.

Proof. Recover $a_0, \dots, a_k$, translate the truncation by -1, generate $1^k, \dots, k^k$, and apply (377). For $k = 0$, return a_0. □

Algorithm 120 One coefficient after exponential substitution

Input: $\mathcal{L}A = 0$, solution data, target $k \geq 0$
Output: $[x^k]A(e^x-1)$

```
Recover a_0, ..., a_k using Lemma 695
if k = 0 then
    return a_0
Translate the truncation by −1 using Algorithm 118, obtaining h_0, ..., h_k
(v_1, ..., v_k) ← PowerTable(k)
return (1/k!) Σ_{j=1}^{k} h_j v_j
```

Moments and classical sequences. For a polynomial $F(t) = \sum f_j t^j$,

$$\sum_j f_j j^k = k![x^k]F(e^x).$$

For an infinite sum, the identity requires convergence sufficient for termwise differentiation at $t = 1$.

For binomial moments,

$$\sum_{j=0}^{N} \binom{N}{j} j^k = k![x^k](1+e^x)^N.$$

Use $A(z) = (2+z)^N$, which satisfies

$$(2+z)A' = NA, \qquad a_0 = 2^N, \qquad a_{j+1} = \frac{N-j}{2(j+1)}a_j.$$

One moment costs $O(k+\log(N+2))$, including the initial power 2^N.

For Bell numbers, $B_k = k![x^k]e^{e^x-1}$, so $A(z)=e^z$. Here

$$H(t)=\sum_{j=0}^{k}\frac{(t-1)^j}{j!},\qquad H'-H=-\frac{(t-1)^k}{k!}.$$

Thus

$$h_0=\sum_{j=0}^{k}\frac{(-1)^j}{j!},\qquad (j+1)h_{j+1}=h_j-\frac{(-1)^{k-j}}{j!(k-j)!}.$$

This gives a direct linear-time implementation, followed by $B_k=\sum_{j=1}^{k}h_j j^k$ for $k\ge 1$.

For Bernoulli numbers,

$$\mathcal{B}_k=k![x^k]\frac{x}{e^x-1},\qquad A(z)=\frac{\log(1+z)}{z}=\sum_{j\ge 0}\frac{(-1)^j}{j+1}z^j.$$

The displayed series defines the value at the removable singularity, and

$$z(1+z)A''+(2+3z)A'+A=0.$$

For alternating permutations, work in a field containing i, and use

$$\sec x+\tan x=\frac{e^{ix}+i}{1+ie^{ix}}.$$

With $A(z)=(1+z+i)/(1+i+iz)$, if $c_k=[x^k]A(e^x-1)$, the desired number is $k!i^kc_k$.

Each computation uses a translated polynomial chosen for the target index; it does not compute the full prefix of the composed series.

17.8.9 Diagonals, Constant Terms, and Residues

For $R(s,t)=\sum_{i,j\ge 0}r_{ij}s^it^j$, define

$$\operatorname{Diag}R(x)=\sum_{n\ge 0}r_{nn}x^n.$$

Substituting $t=x/s$ records the difference of the indices:

$$\boxed{\operatorname{Diag}R(x)=[s^0]R\left(s,\frac{x}{s}\right).}$$

Here one first expands R at $(0,0)$, then substitutes. For each fixed power x^j, the resulting Laurent series has no powers of s below $-j$, so its constant term is defined.

If R is analytic for $|s|<a$, $|t|<b$, choose $0<\rho<a$. For $|x|<\rho b$, uniform absolute convergence on $|s|=\rho$ permits termwise integration:

$$\boxed{\operatorname{Diag}R(x)=\frac{1}{2\pi i}\oint_{|s|=\rho}R\left(s,\frac{x}{s}\right)\frac{ds}{s}.}\tag{378}$$

For rational R, the integral is the sum of all enclosed residues. For example, $R=1/(1-t)$ has diagonal one, while its integrand is $1/(s-x)$; the enclosed pole is at $s=x$.

Central Delannoy numbers. Paths with steps $(1,0),(0,1),(1,1)$ have generating function

$$\sum_{m,n\geq 0} D(m,n)s^m t^n = \frac{1}{1-s-t-st}.$$

For $D_n = D(n,n)$,

$$D(x) = \frac{1}{2\pi i}\oint \frac{ds}{-s^2+(1-x)s-x}.$$

The denominator roots are

$$\alpha(x) = \frac{1-x-\sqrt{1-6x+x^2}}{2}, \qquad \beta(x) = \frac{1-x+\sqrt{1-6x+x^2}}{2},$$

where the square root has constant term one. Only $\alpha(x) \to 0$, so only it lies inside a small fixed contour. The derivative of the denominator there is $\sqrt{1-6x+x^2}$. Hence

$$\boxed{D(x) = \frac{1}{\sqrt{1-6x+x^2}}.}$$

Differentiation and coefficient extraction give

$$(1-6x+x^2)D' = (3-x)D$$

and

$$\boxed{(n+2)D_{n+2} = 3(2n+3)D_{n+1} - (n+1)D_n, \qquad D_0 = 1, \quad D_1 = 3.}$$

Theorem 709 (Bivariate rational diagonals are algebraic)**.** *If $R(s,t) \in \mathbb{C}(s,t)$ is regular at the origin, then* Diag $R(x)$ *is algebraic over $\mathbb{C}(x)$.*

Proof. The contour integrand $H(s,x) = R(s,x/s)/s$ is rational in s over $\mathbb{C}(x)$. In a finite splitting field of its denominator, write

$$H(s,x) = B(s,x) + \sum_{\alpha}\sum_{j=1}^{m_\alpha} \frac{c_{\alpha,j}(x)}{(s-\alpha(x))^j}.$$

The pole locations and coefficients are algebraic over $\mathbb{C}(x)$.

Near a sufficiently small nonzero parameter value avoiding pole collisions and degeneracies, choose analytic branches. The fixed contour encloses a fixed collection of poles. Its integral is the sum of their residues $c_{\alpha,1}(x)$, hence is algebraic.

This integral agrees with the analytic diagonal. After clearing denominators, its polynomial relation holds on an open set, and therefore throughout a neighborhood of zero by analyticity. It is consequently an identity of power series. □

17.8.10 Coefficients in Finite Fields by Reading Index Digits

Work over $\mathbb{F}_q$. In characteristic p, every series has zero p-th ordinary derivative, so D-finiteness imposes no restriction. Algebraicity instead permits coefficient extraction by section operators that preserve a fixed rational denominator.

Section operators. For $0 \leq r < q$, define

$$\Lambda_r\left(\sum_{n\geq 0} a_n x^n\right) = \sum_{n\geq 0} a_{qn+r}x^n.$$

If $N = r_0 + qr_1 + \cdots + q^\ell r_\ell$, then

$$a_N = \big(\Lambda_{r_\ell} \cdots \Lambda_{r_0} F\big)(0).$$

Process the least significant digit first.

For several variables,

$$\Lambda_{\mathbf{r}} \left(\sum_{\boldsymbol{\alpha} \geq 0} c_{\boldsymbol{\alpha}} X^{\boldsymbol{\alpha}} \right) = \sum_{\boldsymbol{\alpha} \geq 0} c_{q\boldsymbol{\alpha}+\mathbf{r}} X^{\boldsymbol{\alpha}}.$$

A diagonal query uses the same digit in every coordinate.

Lemma 710 (Sections preserve a denominator)**.** *For polynomials P, B over $\mathbb{F}_q$, with $B(0) \neq 0$,*

$$\boxed{\Lambda_{\mathbf{r}} \left(\frac{P}{B} \right) = \frac{\Lambda_{\mathbf{r}}(PB^{q-1})}{B}.}$$

Proof. The identity

$$\Lambda_{\mathbf{r}}(C(X)H(X^q)) = \Lambda_{\mathbf{r}}(C(X))H(X)$$

follows by comparing exponent residues. Frobenius gives $B(X)^q = B(X^q)$, so $P/B = PB^{q-1}/B(X^q)$. Apply the identity. □

Choose individual degree bounds $D_i \geq \max\{\deg_{X_i} P, \deg_{X_i} B\}$. The product PB^{q-1} has degree at most qD_i in X_i, and sectioning returns this to at most D_i. Thus all numerators lie in the same finite-dimensional space.

Theorem 711 (Rational coefficient queries over finite fields)**.** *For fixed number d of variables, put*

$$S = \prod_{i=1}^{d} (D_i + 1), \qquad V = \prod_{i=1}^{d} (qD_i + 1).$$

After computing $C = B^{q-1}$, a coefficient $[X_1^{N_1} \cdots X_d^{N_d}]P/B$ is obtained in

$$\boxed{O\Big(S^2(1 + \log_q(1 + \max_i N_i))\Big)}$$

operations. The preprocessing costs $O(V \log(V+2) \log(q+1))$ operations and $O(V)$ space.

Proof. For one digit vector, update the S numerator coefficients by

$$P'_{\boldsymbol{\alpha}} = \sum_{\boldsymbol{\beta}} P_{\boldsymbol{\beta}} C_{q\boldsymbol{\alpha}+\mathbf{r}-\boldsymbol{\beta}},$$

interpreting missing coefficients as zero. This costs $O(S^2)$. After all digits are removed, return $P'(0)/B(0)$.

Binary powering constructs B^{q-1}. All intermediate products fit in a coefficient box of size $O(V)$; Kronecker substitution gives the preprocessing bound. □

Algorithm 121 Finite-field rational coefficient extraction

Input: P, B, $B(0) \neq 0$, target vector $\mathbf{N}$
Output: $[X^{\mathbf{N}}]P/B$

```
C ← B^{q-1}
while some coordinate of N is positive do
    r ← N mod q, coordinatewise
    P ← Λ_r(PC)
    N ← ⌊N/q⌋, coordinatewise
return P(0)/B(0)
```

The coefficient formula in the proof implements the update without forming the full product PC. For a fixed field and rational function, the query cost is logarithmic in the indices. Preprocessing has the parameter dependence stated in the theorem.

Over $\mathbb{F}_q$, the numerator's S coefficients have at most q^S possible values. Each digit acts by a fixed linear map on this coefficient vector. Storing the vector avoids enumerating all reachable states.

Algebraic series as rational diagonals.

Theorem 712 (Rational diagonal of a simple branch)**.** *Over any field, suppose $F \in xK[[x]]$ satisfies*

$$P(x,F) = 0, \qquad P(0,0) = 0, \qquad P_y(0,0) \neq 0.$$

Then

$$\boxed{F(x) = \operatorname{Diag} \frac{v^2 P_y(uv,v)}{P(uv,v)}.}$$

After cancelling a factor v, the rational function is regular at the origin.

Proof. Factor over $K[[x]]$:

$$P(x,y) = (y - F(x))U(x,y), \qquad U(0,0) = P_y(0,0) \neq 0.$$

Then

$$\frac{v^2 P_y(uv,v)}{P(uv,v)} = \frac{v^2}{v - F(uv)} + v^2 \frac{U_y(uv,v)}{U(uv,v)}.$$

Every monomial of the second term has larger v-degree than u-degree, so its diagonal is zero.

Expand the first term as

$$\sum_{j \geq 0} v^{1-j} F(uv)^j.$$

This is formal because $F(uv)/v$ is divisible by u. Only $j = 1$ has equal degrees in u, v; it contributes $F(uv)$.

Finally, $P(uv,v)$ is divisible by v, and the quotient has constant term $P_y(0,0)$. □

Combining this with section operators gives logarithmic coefficient queries for simple algebraic branches over $\mathbb{F}_q$. For example, apply it to $F = C - 1$ for the Catalan equation $C = 1 + xC^2$.

Preparing a singular branch. Suppose $P(x,F) = 0$, $P_y(x,F) \neq 0$, and let $\nu = \operatorname{ord}_x P_y(x,F)$. Choose $t > \nu$, and write

$$F = h + x^t(g_0 + G), \qquad \deg h < t, \qquad G(0) = 0.$$

Then

$$Q(x,Y) = x^{-(t+\nu)} P\big(x, h + x^t(g_0 + Y)\big)$$

is polynomial, $Q(0,0) = 0$, and $Q_Y(0,0) \neq 0$.

Indeed, the coefficient of Y before division has valuation $t + \nu$; coefficients of Y-degree at least two have valuation at least $2t > t + \nu$; and the constant term has valuation at least $t + \nu + 1$, since $h + x^t g_0$ agrees with F modulo x^{t+1}.

For a finite-field algebraic series, a polynomial of minimum positive y-degree in $\mathbb{F}_q[x,y]$ annihilating it is separable in y. Otherwise, in characteristic p, write

$$P(x,y) = \sum_{r=0}^{p-1} x^r R_r(x,y)^p.$$

After substituting F, the summands occupy distinct exponent classes modulo p, so every $R_r(x,F)$ vanishes. A nonzero R_r has smaller y-degree, contradicting minimality. It follows that $P_y \neq 0$; its smaller y-degree also implies $P_y(x,F) \neq 0$. Thus the finite valuation ν used above exists.

This prepares any fixed algebraic branch from a sufficiently long initial coefficient prefix. The coefficients through index t are already known; for $N > t$, return $[x^{N-t}]G$ by the rational-diagonal algorithm.

Limitations. D-finite series are not closed under arbitrary composition or reciprocals. The single-coefficient method avoids requiring such closure by working with a target-dependent polynomial.

17.9 Fast Formal-Series Equations and Transformations

Work over a characteristic-zero field. Newton iteration solves formal equations by doubling precision; causal divide-and-conquer computes coefficients in the order imposed by their dependencies.

17.9.1 Newton Iteration for Scalar Differential Equations

Consider

$$y' = \Phi(x, y), \qquad y(0) = a,$$

where Φ is a polynomial of fixed degree in y, with power-series coefficients in x.

The coefficient of x^n determines $(n+1)y_{n+1}$ from earlier coefficients, so the formal solution is unique. Newton iteration groups the convolutions in this recurrence into precision-doubling steps.

Suppose $Y \equiv y \pmod{x^m}$, and put

$$E = \Phi(x, Y) - Y', \qquad A = \Phi_y(x, Y).$$

Choose h to solve

$$h' - Ah = E, \qquad h(0) = 0.$$

With $J = \int_0^x A$, multiplying by e^{-J} changes the equation to $(e^{-J}h)' = e^{-J}E$. The initial condition therefore gives

$$\boxed{h = e^{J} \int_0^x e^{-J} E.}$$

Since $E = O(x^{m-1})$, we have $h = O(x^m)$, and

$$\begin{aligned}(Y+h)' - \Phi(x, Y+h) &= Y' - \Phi(x, Y) + h' - Ah + O(h^2) \\ &= O(x^{2m}).\end{aligned}$$

A residual $O(x^r)$, together with the correct constant term, implies agreement with the solution modulo x^{r+1}. Otherwise the first nonzero term of the difference would differentiate to a lower-degree term than any term produced by $\Phi(x, Y) - \Phi(x, y)$. Thus the correction at least doubles precision. For the implementation, truncate the correction at $M = \min\{2m, N\}$. Only A and E modulo x^{M-1} are needed, since integration raises precision by one. The stored approximation has residual zero modulo x^{M-1}.

Theorem 713 (Fast formal ODE solution)**.** *The solution can be computed modulo x^N in*

$$\boxed{O(N \log(N+2))}$$

field operations and $O(N)$ space, for fixed degree of Φ in y, given its required coefficient prefixes.

Proof. Start from $Y = a \bmod x$. At precision M, evaluating Φ, Φ_y, then forming the integrating-factor correction, costs $O(M \log(M+2))$. Summing over geometrically increasing precisions proves the bound. □

Algorithm 122 Newton solution of a formal ODE

Input: $y' = \Phi(x, y)$, $y(0) = a$, precision $N \geq 1$
Output: $y \bmod x^N$

$Y \leftarrow a$, $m \leftarrow 1$
while $m < N$ **do**
  $M \leftarrow \min\{2m, N\}$
  $E \leftarrow \Phi(x, Y) - Y' \pmod{x^{M-1}}$
  $A \leftarrow \Phi_y(x, Y) \pmod{x^{M-1}}$
  $J \leftarrow \int_0^x A \pmod{x^M}$
  $U \leftarrow \exp(J) \pmod{x^M}$
  $h \leftarrow U \int_0^x (U^{-1}E) \pmod{x^M}$
  $Y \leftarrow Y + h \pmod{x^M}$
  $m \leftarrow M$
return Y

For example, $y' = 1 + y^2$, $y(0) = 0$, gives the tangent series. The same algorithm handles $y' = g(x) + y^2$ for a supplied series g.

The same algorithm applies to expressions Φ for which $\Phi(x, Y)$ and $\Phi_y(x, Y)$ can be evaluated within the stated bound, provided the required inverses and initial constants are defined. No D-finiteness assumption is needed.

17.9.2 CDQ Convolution and Causal Power-Series Equations

When input coefficients depend on earlier outputs, convolution must respect their order of determination. Use the *strictly causal* convolution

$$c_0 = 0, \qquad c_n = [x^{n-1}]A(x)B(x) = \sum_{i+j=n-1} a_i b_j \quad (n \geq 1).$$

Before choosing a_n, b_n, every term needed for c_n involves only earlier coefficients. A prescribed local rule may therefore read c_n and the completed prefixes, then choose a_n, b_n.

CDQ divide-and-conquer solves the left half, propagates its contributions to the right half, then solves the right half. The shift in $c_n = [x^{n-1}]AB$ makes this possible for arbitrary a_0, b_0.

Propagation between intervals. Consider a recursive interval $[l, r)$, with midpoint $m = \lfloor (l + r)/2 \rfloor$. After solving $[l, m)$, all coefficients with index below m are known. We must add to targets in $[m, r)$ exactly those pairs whose larger input index lies in $[l, m)$.

Use local block polynomials $A_{[l,m)} = \sum_{i=l}^{m-1} a_i x^{i-l}$, and similarly for B. Put $p = \min\{m, r - l\}$, $q = \min\{l, r - l\}$, and form

$$\boxed{U = A_{[l,m)} B_{[0,p)} + B_{[l,m)} A_{[0,q)}.} \tag{379}$$

Add $[x^{n-l-1}]U$ to c_n for $m \leq n < r$.

The first product handles pairs whose A-index is in the new block and whose B-index is below m. The second handles pairs whose B-index is in the new block and whose A-index is below l. Thus pairs with both indices in the new block are included only in the first product. The bounds p, q restrict the products to known coefficients and length $O(r - l)$. Prefix indices at least $r - l$ cannot contribute to targets below r.

Theorem 714 (Causal convolution by CDQ)**.** *For a rule that fixes a_n, b_n after receiving c_n, the first N stages of the preceding causal convolution can be generated in*

$$\boxed{O(N \log^2(N + 2))}$$

field operations, plus the work of the local rules, and $O(N)$ space. A fixed number of such convolutions can share the same recursion and local update within the same asymptotic bound.

Proof. Fix a contribution $a_i b_j$ to target $n = i + j + 1 < N$. Let $v = \max\{i, j\}$, so $v < n$. There is a unique lowest recursion node whose two children separate v from n. At this node $v \in [l, m)$, $n \in [m, r)$, and both input coefficients are known after the left recursion. Formula (379) includes this pair exactly once. No other node includes it, because no other split places its larger input index in the left child and its target in the right child. Every contribution is therefore accumulated before its target leaf is processed.

A node of length h performs two products of length $O(h)$, costing $O(h \log(h+2))$. At one recursion depth the interval lengths sum to N, and there are $O(\log(N+2))$ depths. Temporary products may be released before recursing right, giving $O(N)$ peak space. Repeating the products for constantly many convolutions changes only the constant factor. □

Algorithm 123 CDQ generation from a strictly causal convolution

Input: Length N, a local procedure that fixes a_n, b_n after receiving c_n
Output: $c_n = [x^{n-1}]AB$ for $1 \le n < N$, with $c_0 = 0$

```
Initialize a, b, c as zero arrays of length N
if N > 0 then
    SOLVE(0, N)

function SOLVE(l, r)
    if r − l = 1 then
        Run the local procedure at l, passing c_l and the completed prefixes; store a_l, b_l
        return
    m ← ⌊(l + r)/2⌋
    SOLVE(l, m)
    p ← min{m, r − l}, q ← min{l, r − l}
    U ← A_[l,m) B_[0,p) + B_[l,m) A_[0,q)
    for n = m, ..., r − 1 do
        c_n ← c_n + [x^{n−l−1}]U
    Release the temporary product U
    SOLVE(m, r)
```

Empty blocks represent zero polynomials. The local procedure never changes coefficients with smaller indices; it may maintain other state or schedule data for later indices.

Ordinary online convolution is the same method after a shift. Put $A_+ = (A - a_0)/x$ and $B_+ = (B - b_0)/x$. For $n \ge 2$,

$$[x^n]AB = a_0 b_n + a_n b_0 + [x^{n-2}]A_+ B_+.$$

At stage $n-1$, the causal engine for A_+, B_+ supplies the last term before receiving a_n, b_n. Add the two current-index terms after these inputs arrive; handle indices zero and one directly. Thus the routine also produces the usual convolution online within the same bound.

Exponential, inverse, logarithm, and powers. The differential identities reduce each operation to a fixed number of causal convolutions. After producing h_n, the local rule may choose f_{n+1}; thus f_n is known when h_n is computed. The input coefficients may also be supplied in advance.

Write $f = f_0 + xB$, so $B_n = f_{n+1}$, and put $\theta = xD$. The following formulas apply for $n \ge 1$.

For $h = \exp f$, assume $f_0 = 0$, set $h_0 = 1$, and use

$$h' = hf', \qquad nh_n = [x^{n-1}]hf'.$$

Feed $a_n = h_n$, $b_n = (n+1)f_{n+1}$ into one CDQ engine; its output is $c_n = nh_n$.

For $h = f^{-1}$, assume $f_0 \neq 0$, set $h_0 = f_0^{-1}$, and use

$$f_0 h_n = -[x^{n-1}]hB.$$

Feed $a_n = h_n$, $b_n = f_{n+1}$, and return $h_n = -c_n/f_0$ at each noninitial leaf.

For $\ell = \log(f/f_0)$, set $u = \theta\ell$, so $u_0 = 0$. The identity $fu = \theta f$ gives

$$f_0 u_n = n f_n - [x^{n-1}]Bu, \qquad \ell_n = u_n/n.$$

Use $a_n = u_n$, $b_n = f_{n+1}$, compute $u_n = (nf_n - c_n)/f_0$, and then $\ell_n = u_n/n$. Supply the initial coefficient $\ell_0 = 0$ separately.

Finally let $h = h_0(f/f_0)^\alpha$, with $f_0 \neq 0$ and chosen h_0. From $f\theta h = \alpha h \theta f$,

$$\boxed{n f_0 h_n = \alpha[x^{n-1}]hf' - [x^{n-1}]B\theta h.}$$

Use two engines, sharing the recursion. At leaf n, first compute $h_n = (\alpha c_n - d_n)/(nf_0)$. After the rule chooses f_{n+1}, feed the pairs

$$(h_n, (n+1)f_{n+1}), \qquad (nh_n, f_{n+1})$$

into the two engines. Their outputs are exactly c_n, d_n.

Each construction costs $O(N \log^2(N+2))$, in addition to the local rules. When all inputs are known in advance, use the faster offline algorithms.

Rooted unlabelled trees. Let $T(x) = \sum_{n\geq 1} t_n x^n$ count rooted unlabelled trees. A root carries a multiset of rooted trees, so

$$T(x) = x \exp\left(\sum_{j\geq 1} \frac{T(x^j)}{j}\right).$$

Logarithmic differentiation gives

$$xT' = T(1+G), \qquad G(x) = \sum_{n\geq 1} g_n x^n, \qquad g_n = \sum_{d|n} d\, t_d.$$

Thus $t_1 = 1$, and

$$(n-1)t_n = \sum_{j=1}^{n-1} t_j g_{n-j} \qquad (n \geq 2).$$

Use the CDQ engine with $a_i = t_{i+1}$, $b_i = g_{i+1}$. Its output at index $i \geq 1$ is $c_i = \sum_{j=1}^{i} t_j g_{i+1-j} = i\, t_{i+1}$. At leaf zero, set $t_1 = 1$; at every later leaf, set $t_{i+1} = c_i/i$. After finding t_n, add nt_n to $g_n, g_{2n}, g_{3n}, \ldots$ through the requested limit. At that point all divisors of n have been processed, so g_n is complete and can be fixed as the next input coefficient.

The divisor updates cost $\sum_{n\leq N} O(N/n) = O(N \log N)$. Consequently the first N rooted-tree counts take $O(N \log^2(N+2))$ operations and $O(N)$ space.

17.10 Fractional-Linear and Cubic Substitution

Fractional-linear substitution factors into Taylor shifts, scaling, and reversal. Combined with insertion or extraction of even coefficients, it gives a change of basis for symmetric Laurent polynomials and a near-linear algorithm for cubic substitution.

Throughout this subsection the field has characteristic zero. Degree bounds used for reversal and homogenization are retained even when the actual leading coefficients vanish.

17.10.1 Homogeneous Fractional-Linear Substitution

For a matrix $M = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$ and a polynomial of degree at most n, define

$$\mathcal{H}_M^{(n)} F(x) = (cx+d)^n F\left(\frac{ax+b}{cx+d}\right).$$

This is a polynomial of degree at most n. The superscript records the chosen degree bound, not necessarily the actual degree of F.

Theorem 715 (Fast fractional-linear substitution)**.** *If $ad - bc \neq 0$, then $\mathcal{H}_M^{(n)} F$ can be computed in $O((n+1)\log(n+2))$ field operations. No nonvanishing assumption on the individual entries b, c, d is needed.*

Proof. For $c = 0$, we have $d \neq 0$, and the answer is $d^n F((a/d)x + b/d)$, an affine substitution. For $c \neq 0$, write

$$\frac{ax+b}{cx+d} = \frac{a}{c} + \frac{\kappa}{cx+d}, \qquad \kappa = \frac{bc-ad}{c}.$$

Compute $G(t) = F(t + a/c) = \sum_{j=0}^{n} g_j t^j$, then form

$$W(X) = \sum_{j=0}^{n} g_j \kappa^j X^{n-j}$$

by scaling and reversal. The answer is $W(cx+d)$. Two Taylor shifts and linear-time coefficient operations suffice. Division by d is not needed when $c \neq 0$. □

Algorithm 124 Homogeneous fractional-linear substitution

Input: $\deg F \leq n$, scalars a, b, c, d, $ad - bc \neq 0$
Output: $(cx+d)^n F((ax+b)/(cx+d))$

```
if c = 0 then
    G(t) ← F(t + b/d) by Taylor shift
    return d^n G((a/d)x)
G(t) ← F(t + a/c) by Taylor shift
κ ← (bc − ad)/c
W(X) ← Σ_{j=0}^{n} [t^j]G κ^j X^{n−j}
Compute U(X) = W(X + d) by Taylor shift
return U(cx)
```

If $d \neq 0$, the ordinary series $F((ax+b)/(cx+d)) \bmod x^N$ is obtained by dividing this output by $(cx+d)^n$. The total cost is $O((n+N+1)\log(n+N+2))$. If $d = 0$, the rational function may instead have a pole at zero; the homogeneous polynomial remains well-defined.

The outer input is a finite polynomial. An infinite series requires its expansion about the substituted constant term, rather than a truncation at zero.

Quadratic polynomial substitution. Constant substitution is ordinary evaluation, and affine substitution is a Taylor shift and a scaling. For $g(x) = ax^2 + bx + c$, $a \neq 0$, write

$$g(x) = a\left(x + \frac{b}{2a}\right)^2 + c - \frac{b^2}{4a}.$$

First make the affine change in the argument of F, then insert zeros to substitute x^2, and finally Taylor-shift x. The cost of computing $F(g(x))$ is near-linear in the output degree.

17.10.2 A Reversible Symmetric Laurent Substitution

Define

$$\mathcal{J}_n F(z) = z^n F(z + z^{-1}), \qquad \deg F \le n.$$

The output has degree at most $2n$ and satisfies $P(z) = z^{2n} P(1/z)$: it is reciprocal with respect to $2n$, including when its leading or trailing coefficients vanish.

Theorem 716 (Fast Joukowski transform and its inverse)**.** *The map $\mathcal{J}_n$ is a linear isomorphism from degree-$\le n$ polynomials to polynomials reciprocal with respect to $2n$. Both directions take $O((n+1)\log(n+2))$ operations.*

Proof. The forward map follows from the identity

$$z + z^{-1} = \frac{2(u+1)}{u-1}, \qquad u = \left(\frac{z+1}{z-1}\right)^2.$$

Compute

$$U(u) = (u-1)^n F\left(\frac{2(u+1)}{u-1}\right), \qquad V(v) = U(v^2).$$

Then

$$\boxed{\mathcal{J}_n F(z) = 4^{-n}(z-1)^{2n} V\left(\frac{z+1}{z-1}\right).} \tag{380}$$

Indeed, $((z+1)/(z-1))^2 - 1 = 4z/(z-1)^2$. Both fractional-linear substitutions are covered by the preceding theorem.

For the inverse, let P be reciprocal with respect to $2n$, and form

$$V(v) = (v-1)^{2n} P\left(\frac{v+1}{v-1}\right).$$

Reciprocity gives $V(-v) = V(v)$, so V has only even powers. Write $V(v) = U(v^2)$ by extracting its even coefficients, and set

$$\boxed{F(x) = 4^{-n}(x-2)^n U\left(\frac{x+2}{x-2}\right).} \tag{381}$$

To verify inversion, for $P = \mathcal{J}_n F$ the intermediate value is

$$V(v) = (v^2-1)^n F\left(\frac{2(v^2+1)}{v^2-1}\right).$$

Thus the recovered U is exactly the forward U, and (381) recovers F. The map $\mathcal{J}_n$ is injective, since the top nonzero power of F gives a distinct top power of z. Its domain and the reciprocal subspace both have dimension $n+1$, so it is onto. The formulas use two fractional-linear substitutions and one insertion or extraction of even coefficients in each direction. □

Extraction of even coefficients is valid because the transformed polynomial is even; no general substitution of $x^{1/2}$ is used.

Algorithm 125 Joukowski transform and inverse

function JOUKOWSKI(F, n)
  $U \leftarrow \mathcal{H}^{(n)}_{\left(\begin{smallmatrix}2 & 2\\ 1 & -1\end{smallmatrix}\right)} F$
  $V(v) \leftarrow U(v^2)$
  return $4^{-n}\mathcal{H}^{(2n)}_{\left(\begin{smallmatrix}1 & 1\\ 1 & -1\end{smallmatrix}\right)} V$
function INVERSEJOUKOWSKI(P, n)
  $V \leftarrow \mathcal{H}^{(2n)}_{\left(\begin{smallmatrix}1 & 1\\ 1 & -1\end{smallmatrix}\right)} P$
  $U(u) \leftarrow \sum_{j=0}^{n} [v^{2j}]V \, u^j$
  return $4^{-n}\mathcal{H}^{(n)}_{\left(\begin{smallmatrix}1 & 2\\ 1 & -2\end{smallmatrix}\right)} U$

Each $\mathcal{H}$ in this algorithm is computed by Algorithm 124 with the displayed degree bound, including any zero padding.

17.10.3 Chebyshev and Cubic Substitution

Let $C_0(x) = 2$, $C_1(x) = x$, and $C_{d+1}(x) = xC_d(x) - C_{d-1}(x)$. Induction gives

$$\boxed{C_d(z + z^{-1}) = z^d + z^{-d}.}$$

These are the rescaled Chebyshev polynomials $C_d(x) = 2T_d(x/2)$; in particular, $C_3(x) = x^3 - 3x$.

Theorem 717 (Fast composition with a Chebyshev polynomial)**.** *For $\deg F \le n$ and $d \ge 1$, the polynomial $F(C_d(x))$ can be computed in*

$$\boxed{O((dn+1)\log(dn+2))}$$

operations, without first expanding C_d.

Proof. Compute $P = \mathcal{J}_n F$, insert zeros to form $P(z^d)$, and apply the inverse transform with degree bound dn. Indeed,

$$P(z^d) = z^{dn} F(z^d + z^{-d}) = \mathcal{J}_{dn}(F \circ C_d)(z).$$

The output size bounds the cost of all three operations. □

Theorem 718 (Fast composition with an arbitrary cubic)**.** *If $\deg F \le n$ and g has degree at most three, then*

$$\boxed{F(g(x))}$$

can be computed in $O((n+1)\log(n+2))$ operations over K. No scalar square-root computation is required.

Proof. Degrees at most two were handled above. Write $g(x) = ax^3 + bx^2 + cx + d$, with $a \neq 0$, and set

$$h = \frac{b}{3a}, \qquad p = \frac{c}{a} - 3h^2, \qquad q = d - a(h^3 + ph).$$

Then $g(x) = a((x+h)^3 + p(x+h)) + q$. If $p = 0$, affine substitutions and $x \mapsto x^3$ suffice.

For $p \neq 0$, work in the two-dimensional K-algebra

$$E = K[\xi]/(\xi^2 + p/3).$$

The element ξ is a unit, with inverse $-3\xi/p$. Define $\widetilde{F}(t) = F(a\xi^3 t + q)$. Then

$$F(g(x)) = \widetilde{F}\left(C_3\left(\frac{x+h}{\xi}\right)\right),$$

because $\xi^3 C_3(u/\xi) = u^3 - 3\xi^2 u = u^3 + pu$. Compute the affine change defining $\widetilde{F}$, compose with C_3 by Theorem 717, and make the final affine substitution $t = (x+h)/\xi$.

Every element of E is represented by two coefficients in K. Addition and multiplication use a constant number of base-field operations; the only additional inverse is that of ξ. All other divisions in the Taylor shifts and homogeneous substitutions are by nonzero scalars from K. Hence E need not be a field, and the overhead is constant. The result equals $F(g(x)) \in K[x]$, so its ξ-components vanish and its constant components are the required coefficients. □

Algorithm 126 Composition with a cubic polynomial

Input: $\deg F \le n$, $g(x) = ax^3 + bx^2 + cx + d$, $a \neq 0$
Output: $F(g(x))$

$h \leftarrow b/(3a)$, $p \leftarrow c/a - 3h^2$, $q \leftarrow d - a(h^3 + ph)$
if $p = 0$ **then**
  $G(t) \leftarrow F(at + q)$
  return $G((x+h)^3)$, by insertion of zeros and a Taylor shift
Use coefficient pairs for $E = K[\xi]/(\xi^2 + p/3)$
$G(t) \leftarrow F(a\xi^3 t + q)$
$P \leftarrow \textsc{Joukowski}(G, n)$
$Q(z) \leftarrow P(z^3)$
$H \leftarrow \textsc{InverseJoukowski}(Q, 3n)$
$R(x) \leftarrow H((x+h)/\xi)$
return the constant component of every coefficient of R

For a fixed substitution, these maps are linear in the outer coefficients. Their transposes give power projection and coefficient pairing within the same bounds.

17.11 Dirichlet Series

Ordinary power series use addition of indices; Dirichlet series use multiplication. Write $e_n = n^{-s}$ as a formal symbol with $e_a e_b = e_{ab}$. Then

$$f = \sum_{n \ge 1} f_n e_n, \qquad (f * g)_n = \sum_{ab=n} f_a g_b, \qquad \delta = e_1.$$

We work over a characteristic-zero field until stated otherwise. No analytic convergence is involved. The coefficient f_1, not f_0, plays the role of the constant term, and f is a unit exactly when $f_1 \neq 0$.

Unique factorization identifies e_n with $\prod_p x_p^{v_p(n)}$. This gives a power-series algebra in prime-indexed variables, allowing infinitely many monomials even of one fixed total degree. The operations are defined coefficientwise: each monomial of a product receives only finitely many contributions.

Full coefficient arrays are computed by triangular divisor recurrences. Compressed prefix tables support convolution but not arbitrary coefficientwise operations, so they require different logarithm and exponential algorithms.

17.11.1 The Dirichlet Derivative and Triangular Recurrences

For $a_1 = 0$, define

$$\exp_*(a) = \sum_{j\ge 0} \frac{a^{*j}}{j!}, \qquad \log_*(\delta + a) = \sum_{j\ge 1} \frac{(-1)^{j+1}}{j} a^{*j}.$$

These sums are coefficientwise finite: a product of j indices at least two is at least 2^j. More precisely, the coefficient at n can involve at most $\Omega(n)$ nonconstant factors, where $\Omega(n) = \sum_p v_p(n)$.

For every unit f, define its *normalized logarithm* by

$$\operatorname{Log}_* f = \log_*(f/f_1).$$

It has coefficient zero at 1, and does not require a scalar logarithm of f_1. In particular, $\exp_*(\operatorname{Log}_* f) = f/f_1$.

The Euler derivation. The prime-variable interpretation suggests the Euler derivation

$$\partial f = \sum_{n\ge 1} \Omega(n) f_n e_n.$$

Since $\Omega(ab) = \Omega(a) + \Omega(b)$,

$$\boxed{\partial(f * g) = (\partial f) * g + f * (\partial g).} \tag{382}$$

Every $\Omega(n)$ with $n > 1$ is positive. Thus, once the coefficient at 1 is prescribed, integration amounts to dividing the n-th coefficient by $\Omega(n)$.

The usual differentiation identities now hold formally:

$$f * \partial(\operatorname{Log}_* f) = \partial f, \qquad \partial(\exp_* a) = (\exp_* a) * \partial a.$$

Apply (382) termwise to the coefficientwise finite defining sums.

Logarithm and exponential. For $\ell = \operatorname{Log}_* f$, coefficient comparison gives

$$\boxed{f_1 \Omega(n) \ell_n = \Omega(n) f_n - \sum_{\substack{d\mid n \\ 1<d<n}} \Omega(d) \ell_d f_{n/d}, \qquad \ell_1 = 0.} \tag{383}$$

For $h = \exp_* a$, where $a_1 = 0$, it gives

$$\boxed{\Omega(n) h_n = \Omega(n) a_n + \sum_{\substack{d\mid n \\ 1<d<n}} h_d \Omega(n/d) a_{n/d}, \qquad h_1 = 1.} \tag{384}$$

Every unknown on either right-hand side has smaller index.

Evaluate the recurrences directly with divisor-sieve accumulators. For the logarithm, initialize $R_n = \Omega(n) f_n$ for the logarithm. After computing $\ell_d = R_d/(f_1 \Omega(d))$, propagate $-\Omega(d)\ell_d f_b$ to R_{db} for $b \ge 2$. For the exponential, initialize $R_n = \Omega(n) a_n$, compute $h_d = R_d/\Omega(d)$, and propagate $h_d \Omega(b) a_b$ to R_{db}.

Powers, inverses, and unit roots. For a unit f, a scalar $\alpha \in K$, and a prescribed $c \in K$, define

$$h = c \exp_*\big(\alpha \operatorname{Log}_* f\big).$$

The scalar factor is supplied separately; f_1^α need not be defined in the coefficient field. Its differential equation is

$$\boxed{f * \partial h = \alpha h * \partial f, \qquad h_1 = c.}$$

Comparing coefficients and reindexing one sum gives

$$\boxed{f_1\Omega(n)h_n = \alpha c\Omega(n)f_n + \sum_{\substack{d\mid n\\ 1<d<n}} \big(\alpha\Omega(n/d) - \Omega(d)\big)h_d f_{n/d}.} \tag{385}$$

The condition $h_1 = c$ and this recurrence uniquely determine h.

For an inverse, take $\alpha = -1$, $c = f_1^{-1}$. For an integer power f^{*k}, take $\alpha = k$, $c = f_1^k$. For a k-th root of a unit, choose a scalar c with $c^k = f_1$ and take $\alpha = 1/k$; the resulting series is the unique root with that coefficient at 1. The scalar root must be supplied, and k must be invertible in the coefficient field.

For $\alpha = -1$, the summand weight becomes $-\Omega(n)$. Cancelling $\Omega(n)$ recovers the simpler ordinary inverse recurrence. More generally, solving $f * g = h$ directly uses

$$g_1 = h_1/f_1, \qquad g_n = \frac{h_n - \sum_{\substack{d\mid n\\ d>1}} f_d g_{n/d}}{f_1}.$$

Theorem 719 (Full Dirichlet-series operations)**.** *Through index N, convolution, division by a unit, normalized logarithm, exponential, and the normalized powers and unit roots just described can be computed in*

$$\boxed{O(N\log(N+2))}$$

field operations and $O(N)$ space. The cost of obtaining any supplied scalar power or scalar root is separate.

Proof. A sieve computes $\Omega(1),\dots,\Omega(N)$. Each triangular recurrence is processed in increasing index order by propagating contributions along pairs $db \le N$. There are $\sum_{d\le N}\lfloor N/d\rfloor = O(N\log N)$ such pairs. Convolution and division have the same enumeration bound. Each recurrence uniquely determines the coefficients of its solution. □

Algorithm 127 A Dirichlet power with prescribed coefficient at one

Input: $f_1,\dots,f_N$, $f_1 \neq 0$, scalars α, c
Output: $h = c\exp_*(\alpha\operatorname{Log}_* f)$ through N

```
Sieve Ω(1), ..., Ω(N)
h_1 ← c
R_n ← αcΩ(n) f_n for 2 ≤ n ≤ N
for d = 2, ..., N do
    h_d ← R_d/(f_1 Ω(d))
    for b = 2, ..., ⌊N/d⌋ do
        R_db ← R_db + (αΩ(b) − Ω(d)) h_d f_b
return h
```

For an integer exponent k, computing f_1^k adds $O(\log(|k|+2))$ scalar operations. If $f_1 = 0$, the unit recurrences do not apply. Nonnegative integer powers still exist (the zeroth power is δ): if the first nonzero index is $m \ge 2$, then f^{*k} is supported on indices at least m^k. It therefore vanishes through N when $m^k > N$; otherwise binary powering costs $O(N\log(N+2)\log(k+1))$. Nonunit roots may also exist, but are not covered by the unit-root recurrence.

The displayed unit algorithms also work in positive characteristic when $1,\dots,\lfloor\log_2 N\rfloor$ are invertible, together with any separately required scalar denominators. The divisions by $\Omega(n)$ and the defining logarithm and exponential sums then remain valid. Plain convolution and division by a unit do not need this characteristic restriction.

Newton iteration. The usual identities remain valid:

$$g \longmapsto g * (2\delta - f * g), \qquad g \longmapsto g * (\delta + a - \log_* g),$$

where the second iteration maintains $g_1 = 1$. An error supported on indices at least m has square supported on indices at least m^2. Thus squaring the error squares its index threshold. The triangular recurrences give the same asymptotic bound for full arrays.

17.11.2 Compressed Prefix Tables

Fix $N \geq 2$, and let

$$\mathcal{Q}_N = \{0\} \cup \{\lfloor N/k \rfloor : 1 \leq k \leq N\}.$$

Store f by $F(v) = \sum_{n \leq v} f_n$, $v \in \mathcal{Q}_N$.

Put $s = \lfloor \sqrt{N} \rfloor$, $t = \lfloor N/(s+1) \rfloor$, and $K = s + t$. The positive arguments are $1, \ldots, s$, followed by t larger quotients.

For a stored positive argument,

$$\operatorname{idx}(v) = \begin{cases} v, & v \leq s, \\ K - \lfloor N/v \rfloor + 1, & v > s. \end{cases}$$

For an arbitrary positive threshold a, the first stored argument at least a has index

$$\operatorname{first}(a) = \begin{cases} a, & a \leq s, \\ K - \lfloor N/a \rfloor + 1, & a > s. \end{cases}$$

Index $K + 1$ indicates that no stored argument is large enough. Thus suffix updates at arbitrary thresholds take constant time.

The quotient arguments are closed under division:

$$\left\lfloor \frac{\lfloor N/a \rfloor}{b} \right\rfloor = \left\lfloor \frac{N}{ab} \right\rfloor.$$

Also $v \mapsto \lfloor N/v \rfloor$ is an involution on the positive quotient grid. Indeed, if $v = \lfloor N/k \rfloor$ and $w = \lfloor N/v \rfloor$, then $w \geq k$ and $vw \leq N$, so $v \leq \lfloor N/w \rfloor \leq \lfloor N/k \rfloor = v$. The large arguments are the distinct values $\lfloor N/j \rfloor$ for $1 \leq j \leq t$, listed in reverse order of j. For $a > s$, exactly $\lfloor N/a \rfloor$ of them are at least a, proving the formula for $\operatorname{first}(a)$.

Compatibility with convolution.

Lemma 720 (Compression respects convolution)**.** *If two Dirichlet series have the same prefix table on $\mathcal{Q}_N$, they are interchangeable in any sum, scalar multiple, or convolution whose output is another such table. The same holds for normalized logarithms, exponentials, inverses, and powers whenever these are defined through index N.*

Proof. Only convolution needs proof. For $h = f * g$,

$$H(v) = \sum_{b \leq v} g_b F(\lfloor v/b \rfloor).$$

All arguments $\lfloor v/b \rfloor$ belong to $\mathcal{Q}_N$, so replacing f by a series with the same stored prefixes changes no output. By commutativity, the same holds for replacing g. The formal functions in the statement are finite convolution expressions through index N, after normalizing their coefficient at one. That coefficient is itself stored as $F(1)$. □

Compressed prefix tables do not determine ∂. For $N = 10$, $e_6 - e_7$ has zero prefix sum on $\{0, 1, 2, 3, 5, 10\}$, but its derivative $2e_6 - e_7$ has prefix sum one at 10. The full-array logarithmic-derivative algorithm therefore cannot be used directly on compressed tables.

17.11.3 Compressed Multiplication

Write

$$J(z) = H(\lfloor N/z \rfloor) = \sum_{\substack{x,y \geq 1 \\ xyz \leq N}} f_x g_y, \qquad z \in \mathcal{Q}_N \setminus \{0\}.$$

This indexes the output table by the reversed quotient grid.

Partition the triples by their largest coordinate, breaking ties in the order x, y, z:

$$x \geq y,\ x \geq z; \qquad y > x,\ y \geq z; \qquad z > x,\ z > y.$$

Fix the two smaller coordinates and sum over the interval of possible values of the largest. For an input coordinate this uses a prefix sum; for the output coordinate it uses a range addition.

For an ordered pair (a, b) with $ab \max\{a, b\} \leq N$, put $m = \max\{a, b\}$ and $r = \lfloor N/(ab) \rfloor$.

When x is largest, set $y = a, z = b$, and add

$$g_a(F(r) - F(m-1))$$

to $J(b)$.

When y is largest, set $x = a, z = b$. Its interval starts at $\ell = \max\{a+1, b\}$. If $\ell \leq r$, add

$$f_a(G(r) - G(\ell - 1))$$

to $J(b)$.

When z is largest, set $x = a, y = b$. Range-add $f_a g_b$ to stored $J(z)$ with $m + 1 \leq z \leq r$.

The fixed coordinates are at most $\sqrt{N}$, so their coefficients are differences of stored prefixes. The upper endpoint r is a quotient argument, and every lower endpoint minus one is at most s.

Theorem 721 (Compressed Dirichlet multiplication)**.** *Given arbitrary compressed prefix tables F, G, the product table can be computed in*

$$\boxed{O(N^{2/3})}$$

operations and $O(\sqrt{N})$ space.

Proof. The three regions partition all triples. Write $u = \min\{a, b\}$, $v = \max\{a, b\}$. Then $u \leq N^{1/3}$ and $u \leq v \leq \sqrt{N/u}$. The number of fixed pairs is

$$O\left(\sum_{u \leq N^{1/3}} \sqrt{\frac{N}{u}}\right) = O(N^{2/3}).$$

Each pair gives a constant number of point or range updates to a difference array on the quotient grid. Prefix-sum it and undo the quotient reindexing. □

Algorithm 128 Compressed Dirichlet multiplication

Input: Prefix tables F, G on $\mathcal{Q}_N$
Output: Prefix table H of $f * g$

```
Recover f_a, g_a for 1 ≤ a ≤ s by prefix differences
Initialize a difference array for J(z) on the positive quotient grid
for u = 1, ..., ⌊N^{1/3}⌋ do
    for v = u, ..., ⌊√(N/u)⌋ do
        ProcessPair(u, v)
        if u < v then
            ProcessPair(v, u)
Prefix-sum the difference array
H(v) ← J(⌊N/v⌋), H(0) ← 0
return H

function ProcessPair(a, b)
    m ← max{a, b}, r ← ⌊N/(ab)⌋
    Point-add g_a(F(r) − F(m − 1)) to J(b)
    ℓ ← max{a + 1, b}
    if ℓ ≤ r then
        Point-add f_a(G(r) − G(ℓ − 1)) to J(b)
    if m + 1 ≤ r then
        Range-add f_a g_b on stored arguments in [m + 1, r]
```

A range $[\ell, r]$, with r stored, uses difference-array endpoints $\mathrm{first}(\ell)$ and $\mathrm{idx}(r) + 1$.

17.11.4 Compressed Logarithms and Exponentials

The defining logarithm and exponential use $O(\log N)$ convolution powers. Eliminating an initial range of nonconstant coefficients reduces this to a constant number of products.

Removing one elementary factor costs $O(\sqrt{N})$. Choose B so that preprocessing costs no more than an $O(N^{2/3})$ compressed product: the balance $B\sqrt{N} = N^{2/3}$ gives $B = \lfloor N^{1/6} \rfloor$. The remaining nonconstant support starts at $B + 1$, whose sixth power exceeds N, so only five convolution powers contribute.

Elementary factors. Multiplication by $\delta - ce_m$, $m \geq 2$, changes prefixes by

$$H(v) = F(v) - cF(\lfloor v/m \rfloor).$$

Perform this in decreasing order of v.

Division satisfies

$$H(v) = F(v) + cH(\lfloor v/m \rfloor),$$

and is performed in increasing order. Each operation costs $O(\sqrt{N})$.

Eliminating small coefficients. For a general unit, first divide its prefix table by f_1; this does not change its normalized logarithm. With $f_1 = 1$, process $m = 2, \ldots, B$. Let c_m be the current coefficient at m, and multiply by $\delta - c_m e_m$. This kills coefficient m without changing smaller coefficients.

The result is $R = \delta + u$, with $u_n = 0$ for $n \leq B$, and

$$f = R * \prod_{m=2}^{B} (\delta - c_m e_m)^{-1}.$$

Hence

$$\boxed{\operatorname{Log}_* f = \sum_{j=1}^{5} \frac{(-1)^{j+1}}{j} u^{*j} + \sum_{m=2}^{B} \sum_{k:\, m^k \le N} \frac{c_m^k}{k} e_{m^k}.}$$

The last sum has $O(B \log N)$ explicitly listed terms. Generate m^k and c_m^k successively for each m. A term γe_a contributes γ to precisely the stored prefixes with argument at least a, so it is one suffix update at $\operatorname{first}(a)$, without storing the full N-coefficient array.

Exponential. Given $a_1 = 0$, choose $c_2, \dots, c_B$ so that

$$L = \sum_{m=2}^{B} \sum_{k:\, m^k \le N} \frac{c_m^k}{k} e_{m^k}$$

agrees with a through index B. This is triangular: the contribution of c_m at index m is c_m, and its remaining contributions occur at larger indices.

Then $r = a - L$ vanishes through B, and

$$\boxed{\exp_* a = \left(\delta + \sum_{j=1}^{5} \frac{r^{*j}}{j!}\right) * \prod_{m=2}^{B} (\delta - c_m e_m)^{-1}.}$$

Theorem 722 (Compressed formal operations)**.** *Normalized logarithms, exponentials, inverses, quotients, and fixed normalized powers can be computed on $\mathcal{Q}_N$ in*

$$\boxed{O(N^{2/3})}$$

operations and $O(\sqrt{N})$ space.

Proof. The B elementary-factor updates cost $O(B\sqrt{N}) = O(N^{2/3})$. The truncated logarithm or exponential uses a constant number of compressed products, and constructing the correction costs $O(B \log N + \sqrt{N})$.

For a unit, use $f^{-1} = f_1^{-1} \exp_*(-\operatorname{Log}_* f)$. Normalized powers are $(f/f_1)^{*\alpha} = \exp_*(\alpha \operatorname{Log}_* f)$; a prescribed scalar factor can be restored afterward. A quotient needs one additional compressed product. □

Algorithm 129 Compressed Dirichlet logarithm

Input: Prefix table F, with $f_1 \neq 0$
Output: Prefix table of $\operatorname{Log}_* f$

```
B ← ⌊N^{1/6}⌋, R ← F/F(1)
for m = 2, ..., B do
    c_m ← R(m) − R(m − 1)
    for positive v ∈ Q_N in decreasing order do
        R(v) ← R(v) − c_m R(⌊v/m⌋)
Build L by suffix-adding c_m^k/k at threshold m^k, for all m^k ≤ N
u ← R − δ, P ← u, A ← L
for j = 1, ..., 5 do
    A ← A + (−1)^{j+1} P/j
    if j < 5 then
        P ← P ∗ u by compressed multiplication
return A
```

Algorithm 130 Compressed Dirichlet exponential

Input: Prefix table A, with $a_1 = 0$
Output: Prefix table of $\exp_* a$

```
B ← ⌊N^{1/6}⌋
b_m ← A(m) − A(m − 1) for 2 ≤ m ≤ B
Initialize a difference array for L
for m = 2, ..., B do
    c_m ← b_m
    for k ≥ 1 with m^k ≤ N do
        Suffix-add c_m^k/k to L at threshold m^k
        if m^k ≤ B then
            b_{m^k} ← b_{m^k} − c_m^k/k
Prefix-sum the difference array to obtain L
r ← A − L, P ← δ, E ← δ
for j = 1, ..., 5 do
    P ← P ∗ r by compressed multiplication
    E ← E + P/j!
for m = 2, ..., B do
    for positive v ∈ Q_N in increasing order do
        E(v) ← E(v) + c_m E(⌊v/m⌋)
return E
```

The prefix table of δ is one at every positive argument and zero at zero. Individual coefficients are used only in the small range where consecutive prefixes are stored.

17.11.5 Prime Counting

Let $Z = \sum_{n \ge 1} e_n$. Unique prime factorization gives

$$Z = \prod_{p \text{ prime}} (\delta - e_p)^{-1}, \qquad \boxed{\operatorname{Log}_* Z = \sum_{p} \sum_{k \ge 1} \frac{1}{k} e_{p^k}.}$$

The logarithm is supported only on prime powers.

If $J(x)$ is its prefix sum, then

$$J(x) = \sum_{k=1}^{\lfloor \log_2 x \rfloor} \frac{1}{k} \pi(\lfloor x^{1/k} \rfloor).$$

Therefore

$$\boxed{\pi(N) = J(N) - \sum_{k=2}^{\lfloor \log_2 N \rfloor} \frac{1}{k} \pi(\lfloor N^{1/k} \rfloor).}$$

Theorem 723 (Prime counting by Dirichlet logarithms)**.** *The value $\pi(N)$ can be computed in $O(N^{2/3})$ arithmetic operations and $O(\sqrt{N})$ space.*

Proof. The prefix table of Z is $Z(v) = v$. Compute its logarithm table to obtain $J(N)$. Every correction argument is at most $\sqrt{N}$, so a sieve through $\lfloor \sqrt{N} \rfloor$ supplies the remaining prime counts. Compute the roots as exact integer roots. □

Algorithm 131 Prime counting by a compressed Dirichlet logarithm

Input: Integer $N \geq 2$
Output: $\pi(N)$

Construct $\mathcal{Q}_N$, and set $Z(v) \leftarrow v$
Compute the prefix table J of $\mathrm{Log}_* Z$ using Algorithm 129
Sieve prime counts through $\lfloor \sqrt{N} \rfloor$
$a \leftarrow J(N)$
for $k = 2, \ldots, \lfloor \log_2 N \rfloor$ **do**
    $r \leftarrow \max\{j \in \mathbb{Z}_{\geq 0} : j^k \leq N\}$
    $a \leftarrow a - \pi(r)/k$
return a

For Z, the small-factor coefficients c_m are one at primes and zero at composites. Thus the logarithm algorithm is a partial sieve followed by a short formal logarithm.

For a completely multiplicative weight w,

$$\mathrm{Log}_* \left(\sum_{n \geq 1} w(n) e_n \right) = \sum_p \sum_{k \geq 1} \frac{w(p)^k}{k} e_{p^k}.$$

Whenever the weighted prefix table is available, the same method computes weighted prime sums after subtracting higher prime powers. For instance, $w(n) = n$ gives the sum of primes up to N.

17.11.6 Logarithmic Binning and Boundary Correction

Instead of the exact interval grouping used in the $N^{2/3}$ algorithm, approximate the product comparison by adding logarithmic indices. Ordinary convolution computes these approximate sums; exact correction over a short boundary interval gives $N^{1/2+o(1)}$ time.

Large coordinates. Put $s = \lfloor \sqrt{N} \rfloor$. Compute the product coefficients and their prefixes through s by the divisor-sieve algorithm in $O(s \log(s+2))$ operations. These are all the small quotient outputs. The remaining outputs have the form $H(\lfloor N/z \rfloor)$ with $1 \leq z \leq s$.

For each such z, the terms with $x > s$ contribute

$$\sum_{y \leq N/(z(s+1))} g_y \big(F(\lfloor N/(yz) \rfloor) - F(s) \big).$$

The terms with $y > s$ are symmetric, and the two cases are disjoint because $(s+1)^2 > N$. All required coefficients and prefixes are stored. Over $z \leq s$, these sums cost $\sum_{z \leq s} O(N/(z(s+1))) = O(\sqrt{N} \log N)$. It remains to find

$$T_z = \sum_{\substack{x,y \leq s \\ xyz \leq N}} f_x g_y \qquad (1 \leq z \leq s).$$

Logarithmic bins. At logarithmic scale b, the arrays have length $O(b \log N)$ and rounding affects an interval of length $O(N/b)$. Balance the two by choosing $b = \lceil \sqrt{N} \rceil$.

Choose nonnegative integers $\ell(j)$, for $1 \leq j \leq s$ and for $j = N$, satisfying

$$|\ell(j) - b \log_2 j| \leq 1.$$

Define the ordinary polynomials

$$A_{\mathrm{bin}}(u) = \sum_{x=1}^{s} f_x u^{\ell(x)}, \qquad B_{\mathrm{bin}}(u) = \sum_{y=1}^{s} g_y u^{\ell(y)},$$

and compute $C(u) = A_{\mathrm{bin}}(u)B_{\mathrm{bin}}(u)$. The multiplication uses ordinary convolution of the bin indices. Let $\widehat{C}(t) = \sum_{r\le t}[u^r]C$, with value zero for $t < 0$ and the full coefficient sum for $t \ge \deg C$.

Set $L = \ell(N) + 4$. The provisional answer

$$\widehat{T}_z = \widehat{C}(L - \ell(z))$$

is the sum over all $x, y \le s$ for which $\ell(x) + \ell(y) + \ell(z) \le L$. Every triple satisfying the original product bound is included, since

$$xyz \le N \quad\Longrightarrow\quad \ell(x) + \ell(y) + \ell(z) \le b\log_2 N + 3 \le L.$$

Conversely, an included triple satisfies

$$b\log_2(xyz) \le \ell(x) + \ell(y) + \ell(z) + 3 \le b\log_2 N + 8.$$

For $b \ge 8$, the convexity inequality $2^u \le 1 + u$ on $0 \le u \le 1$ therefore gives

$$\boxed{N < xyz \le N2^{8/b} \le N + 8N/b} \tag{386}$$

for every false positive. It remains to subtract these excess terms.

Boundary correction. Put $W = \lceil 8N/b\rceil$. Factor all integers in $(N, N + W]$ by a segmented sieve. Explicitly, initialize a residual entry equal to each integer in the interval, sieve the primes through $\sqrt{N+W}$, and divide each interval multiple repeatedly by the current prime, recording its exponent. Any final residual greater than one is prime. The prime sieve and the divisions take $O(\sqrt{N}\log(N+2))$ elementary operations.

For every factored integer m, enumerate its ordered factorizations $xyz = m$. Retain those with $x, y, z \le s$ and $\ell(x)+\ell(y)+\ell(z) \le L$, and subtract $f_x g_y$ from $\widehat{T}_z$. By (386), these are exactly the spurious contributions. This argument does not require positive coefficients.

If $m = \prod_p p^{a_p}$, the number of ordered factorizations is

$$d_3(m) = \prod_p \binom{a_p + 2}{2} = m^{o(1)}.$$

To prove the estimate, fix $\varepsilon > 0$. For sufficiently large p, $\binom{a+2}{2} \le 3^a \le p^{\varepsilon a}$. For each of the finitely many smaller primes, the polynomial in a is at most a constant times $p^{\varepsilon a}$. Multiplying gives $d_3(m) = O_\varepsilon(m^\varepsilon)$.

To enumerate the factorizations, recursively distribute each prime exponent among the three coordinates, multiplying their partial values by the corresponding powers. The recursion has at most $O(d_3(m)\log(m+2))$ nodes. Consequently, correcting all $O(\sqrt{N})$ boundary integers costs $N^{1/2+o(1)}$.

Theorem 724 (Square-root-scale compressed multiplication)**.** *Compressed Dirichlet multiplication on $\mathcal{Q}_N$ can be performed in*

$$\boxed{N^{1/2+o(1)}}$$

field and index-arithmetic operations, using $O(\sqrt{N}\log(N+2))$ stored coefficients and index data. No coefficient-sign restriction is needed.

Proof. The small outputs and large-coordinate contributions cost $O(\sqrt{N}\log N)$. The bin polynomials have degree $O(\sqrt{N}\log N)$, so their ordinary product costs $O(\sqrt{N}\log^2 N)$. The segmented sieve and the exact boundary correction have the cost proved above. All large quotient outputs are then obtained by adding back the separately computed large-coordinate contributions.

To compute the logarithmic indices, approximate $\log_2 j$ to error at most $1/(2b)$, multiply by b, and round to a nearest integer. The resulting error is at most one. To obtain the approximation, separate off the integer binary exponent and reduce the argument to $[1, 2]$. On that interval,

$$\log u = 2\sum_{k\ge0}\frac{z^{2k+1}}{2k+1}, \qquad z = \frac{u-1}{u+1}, \quad |z| \le \frac{1}{3}.$$

The tail decreases geometrically. Using $O(\log N)$ terms and polylogarithmic-bit arithmetic, including an approximation to $\log 2$, gives the required accuracy. The bound counts field operations on coefficients and bit operations for the logarithmic indices. □

Algorithm 132 Compressed multiplication by logarithmic binning

Input: Prefix tables F, G on $\mathcal{Q}_N$
Output: Prefix table H of $f * g$

if $N < 64$ **then**
  return the product from Algorithm 128
$s \leftarrow \lfloor \sqrt{N} \rfloor$, $b \leftarrow \lceil \sqrt{N} \rceil$
Recover $f_1, \dots, f_s$ and $g_1, \dots, g_s$
Compute $H(1), \dots, H(s)$ by a divisor sieve
Compute admissible $\ell(j)$ for $j \le s$ and $j = N$
Form $A_{\mathrm{bin}}(u), B_{\mathrm{bin}}(u)$; multiply them as ordinary polynomials
Let $\widehat{C}$ be the prefix sums of the product coefficients
$L \leftarrow \ell(N) + 4$, $W \leftarrow \lceil 8N/b \rceil$
$T_z \leftarrow \widehat{C}(L - \ell(z))$ for $1 \le z \le s$
Factor every integer in $(N, N + W]$ by the segmented sieve described above
for each such integer m **do**
  for each allocation of its prime exponents among x, y, z **do**
    if $x, y, z \le s$ and $\ell(x) + \ell(y) + \ell(z) \le L$ **then**
      $T_z \leftarrow T_z - f_x g_y$
for $z = 1, \dots, s$ **do**
  $R \leftarrow T_z$
  for $a = 1, \dots, \lfloor N/(z(s+1)) \rfloor$ **do**
    $R \leftarrow R + g_a(F(\lfloor N/(az) \rfloor) - F(s))$
    $R \leftarrow R + f_a(G(\lfloor N/(az) \rfloor) - G(s))$
  $H(\lfloor N/z \rfloor) \leftarrow R$
$H(0) \leftarrow 0$
return H

Using this multiplication routine, the defining logarithm and exponential series need only $O(\log N)$ products, absorbed into $N^{o(1)}$. Normalized logarithms, exponentials, inverses, powers, and prime counting therefore also admit $N^{1/2+o(1)}$-time algorithms.

18 Set Power Series

18.1 Subset Convolution

Given two functions $f, g : 2^V \to \mathbb{F}$ over a set V of size n, their subset convolution $h = f * g$ is defined as:

$$h(S) = \sum_{A \subseteq S} f(A) g(S \setminus A)$$

Computing this naively requires $O(3^n)$ time. We can optimize this to $O(n^2 2^n)$ using the Zeta and Möbius transforms.

Define the ranked representations $\hat{f}$ and $\hat{g}$:

$$\hat{f}_k(S) = \begin{cases} f(S) & \text{if } |S| = k \\ 0 & \text{otherwise} \end{cases}$$

The Zeta transform computes: $\mathcal{Z}(f)(S) = \sum_{A \subseteq S} f(A)$. We have that point-wise multiplication after the Zeta transform computes:

$$\mathcal{Z}^{-1}\big(\mathcal{Z}(u) \cdot \mathcal{Z}(v)\big)(S) = \sum_{A \cup B = S} u(A)v(B)$$

By performing a standard linear convolution over the rank dimension and a point-wise multiplication in the Zeta domain, we compute a new ranked function $\hat{h}$:

$$\hat{h}_i(S) = \mathcal{Z}^{-1}\left(\sum_{j=0}^{i} \mathcal{Z}(\hat{f}_j) \cdot \mathcal{Z}(\hat{g}_{i-j})\right)(S) = \sum_{A \cup B = S} \hat{f}_j(A)\hat{g}_{i-j}(B)$$

To extract the final answer for a set S, we evaluate the rank slice corresponding to its cardinality: $h(S) = \hat{h}_{|S|}(S)$.
By substituting $i = |S|$, the non-zero terms mandate that $|A| = j$ and $|B| = i - j$, which means $|A| + |B| = i = |S|$ so $A \cap B = \emptyset$.

Complexity

- **Time:** The Forward and Inverse Zeta Transforms process $n + 1$ arrays of size 2^n, taking $O(n \cdot n2^n)$ time. The ranked point-wise multiplication takes $O(n^2)$ for each of the 2^n sets. Total Time: $O(n^2 2^n)$.
- **Space:** Allocating the ranked arrays requires $(n + 1) \times 2^n$ memory. Total Space: $O(n2^n)$.

Algorithm 133 Subset Convolution

```
function ZetaTransform(A, n)
    for i ← 0 to n − 1 do
        for S ← 0 to 2^n − 1 do
            if (S & (1 ≪ i)) ≠ 0 then
                A[S] ← A[S] + A[S ⊕ (1 ≪ i)]
function MobiusTransform(A, n)
    for i ← 0 to n − 1 do
        for S ← 0 to 2^n − 1 do
            if (S & (1 ≪ i)) ≠ 0 then
                A[S] ← A[S] − A[S ⊕ (1 ≪ i)]
function SubsetConvolution(f, g, n)
    Initialize f̂, ĝ, ĥ as arrays of size (n + 1) × 2^n filled with 0
    for S ← 0 to 2^n − 1 do
        f̂[|S|][S] ← f[S]
        ĝ[|S|][S] ← g[S]
    for k ← 0 to n do
        ZetaTransform(f̂[k], n)
        ZetaTransform(ĝ[k], n)
    for i ← 0 to n do
        for j ← 0 to i do
            for S ← 0 to 2^n − 1 do
                ĥ[i][S] ← ĥ[i][S] + f̂[j][S] · ĝ[i − j][S]
    for k ← 0 to n do
        MobiusTransform(ĥ[k], n)
    Initialize h as an array of size 2^n
    for S ← 0 to 2^n − 1 do
        h[S] ← ĥ[|S|][S]
    return h
```

18.2 Transposed Subset Convolution

Given two functions $f, g : 2^V \to \mathbb{F}$ over a set V of size n, the Transposed Subset Convolution is equivalent to Superset Convolution, and computes h defined as:

$$h(S) = \sum_{Y \supseteq S} f(Y)g(Y \setminus S)$$

We reverse f to get $\tilde{f}$, then compute the subset convolution with g, then reverse the output.

To see why this yields the superset convolution, we expand the definition of $h(S)$ from the three steps above by substituting $K = U \setminus S$:

$$h(S) = \tilde{h}(U \setminus S) = \sum_{X \subseteq U \setminus S} f(U \setminus X)g((U \setminus S) \setminus X)$$

Letting $Y = U \setminus X$, the summation condition $X \subseteq U \setminus S$ becomes $S \subseteq Y$. The sum becomes:

$$h(S) = \sum_{S \subseteq Y} f(Y)g(Y \setminus S)$$

18.3 Inversion and Quotient

Given a subset power series $g : 2^V \to \mathbb{F}$ where $g(\emptyset) \neq 0$, its inverse $h = g^{-1}$ is the unique series satisfying $h * g = \varepsilon$, where $\varepsilon(\emptyset) = 1$ and $\varepsilon(S) = 0$ for all $S \neq \emptyset$. More generally, the quotient $h = f/g$ is the series

satisfying $h * g = f$.

We can compute the quotient layer-by-layer (by rank) in $O(n^2 2^n)$ time by a DP directly in the Ranked Zeta Transform domain.

Expanding the definition of the subset convolution evaluated at a specific set X, we isolate the contribution of $h(X)$:

$$f(X) = \sum_{A \subseteq X} h(A)g(X \setminus A) = h(X)g(\emptyset) + \sum_{A \subsetneq X} h(A)g(X \setminus A)$$

Let $E(X) = \sum_{A \subsetneq X} h(A)g(X \setminus A)$. If we process the sets in increasing order of cardinality $l = |X|$, all required values of $h(A)$ for $|A| < l$ will have already been computed.

This allows us to inductively construct the ranked components $\hat{h}_l$ of the quotient. For a fixed rank l, the transformed accumulation of the smaller ranks is:

$$\mathcal{Z}(\hat{E}_l) = \sum_{i=1}^{l} \mathcal{Z}(\hat{h}_{l-i}) \cdot \mathcal{Z}(\hat{g}_i)$$

We recover E in the subset domain by applying the inverse Zeta (Möbius) transform. Then, for every set X with $|X| = l$, we extract the value of $h(X)$:

$$h(X) = \big(f(X) - E(X)\big) \cdot g(\emptyset)^{-1}$$

Finally, we apply the forward Zeta transform to $\hat{h}_l$ so that its transformed values $\mathcal{Z}(\hat{h}_l)$ are available for computing the subsequent ranks $l + 1, \ldots, n$.

Algorithm The computation proceeds iteratively over the rank dimension:

1. **Initialization:** Compute the inverse scalar $g(\emptyset)^{-1}$. Initialize $(n+1) \times 2^n$ arrays $\hat{g}$ and $\hat{h}$. Fill $\hat{g}$ by placing $g(S)$ into $\hat{g}_{|S|}[S]$ and apply the Fast Zeta Transform to every rank slice of $\hat{g}$.
2. **Layered DP:** For each rank l from 0 to n:
   (a) **Accumulate:** For each element S, compute $\hat{E}_l[S] = \sum_{i=1}^{l} \hat{h}_{l-i}[S] \cdot \hat{g}_i[S]$.
   (b) **Möbius Inversion:** Apply the Fast Möbius Transform to $\hat{E}_l$.
   (c) **Extract:** For every set X where $|X| = l$, compute $\hat{h}_l[X] = \big(f(X) - \hat{E}_l[X]\big) \cdot g(\emptyset)^{-1}$.
   (d) **Zeta Transform:** Apply the Fast Zeta Transform to $\hat{h}_l$.

For SPS inversion, this identical algorithm is used by setting $f(\emptyset) = 1$ and $f(X) = 0$ for all $|X| > 0$.

Algorithm 134 SPS Quotient

```
function SPSQuotient(f, g, n)
    g_inv ← g[0]^{-1}
    Initialize ĝ, ĥ as arrays of size (n + 1) × 2^n filled with 0
    for S ← 0 to 2^n − 1 do
        ĝ[|S|][S] ← g[S]
    for k ← 0 to n do
        FastZetaTransform(ĝ[k], n)
    for l ← 0 to n do
        for i ← 1 to l do
            for S ← 0 to 2^n − 1 do
                ĥ[l][S] ← ĥ[l][S] + ĥ[l − i][S] · ĝ[i][S]
        FastMobiusTransform(ĥ[l], n)
        for X ← 0 to 2^n − 1 do
            if |X| == l then
                ĥ[l][X] ← (f[X] − ĥ[l][X]) · g_inv
            else
                ĥ[l][X] ← 0                          ▷ Clear side-effects from the Möbius transform
        FastZetaTransform(ĥ[l], n)
    Initialize h as an array of size 2^n
    for S ← 0 to 2^n − 1 do
        h[S] ← ĥ[|S|][S]
    return h
```

18.4 Exponential and Logarithm

Given a subset power series $f : 2^V \to \mathbb{F}$, we can compute its exponential $h = \exp(f)$ and logarithm $h = \log(f)$ in $O(n^2 2^n)$ time. For the exponential to exist, we require $f(\emptyset) = 0$. For the logarithm to exist, we require $f(\emptyset) = 1$.

The coefficient of the partial derivative $\frac{\partial}{\partial x_i}$ on a subset S (where $i \notin S$) yields the coefficient of $S \cup \{i\}$ in the original series.

By the chain rule, the exponential $h = \exp(f)$ satisfies the differential equation:

$$\frac{\partial h}{\partial x_i} = h * \frac{\partial f}{\partial x_i}$$

We build the series h iteratively, adding elements from 0 to $n-1$. Let $h_{<i}$ denote the series restricted to subsets drawn from $\{0, \ldots, i-1\}$. When we introduce the i-th element, we want to compute the slice of h for subsets where i is the maximum element. We denote this slice as $h_{\text{slice}(i)}$.

By restricting the equation above to the domain of the first i elements, we have:

$$h_{\text{slice}(i)} = h_{<i} * f_{\text{slice}(i)}$$

This is a subset convolution of size 2^i. Since $f(\emptyset) = 0$, the i-th half of the output is determined by the i-th half of f and the already-computed lower half of h.

Similarly, for the logarithm $h = \log(f)$, the chain rule dictates:

$$\frac{\partial h}{\partial x_i} = \frac{\partial f}{\partial x_i} * f^{-1} \implies \frac{\partial f}{\partial x_i} = \frac{\partial h}{\partial x_i} * f$$

Restricting this to the new element i, we obtain:

$$f_{\text{slice}(i)} = h_{\text{slice}(i)} * f_{<i}$$

This means the new slice of the logarithm can be directly computed using the SPS quotient:

$$h_{\text{slice}(i)} = f_{\text{slice}(i)} / f_{<i}$$

Algorithm The algorithm iteratively computes the exact slices of size 2^i for $i = 0$ to $n - 1$.

- **Exp:** We initialize $h[\emptyset] = 1$. At step i, we extract the previously computed half $h[0 \dots 2^i - 1]$ and multiply it by $f[2^i \dots 2^{i+1} - 1]$ using a Subset Convolution of size 2^i. We write the output into $h[2^i \dots 2^{i+1} - 1]$.
- **Log:** We initialize $h[\emptyset] = 0$. At step i, we compute the Subset Quotient of $f[2^i \dots 2^{i+1} - 1]$ divided by $f[0 \dots 2^i - 1]$ and write the output into $h[2^i \dots 2^{i+1} - 1]$.

Algorithm 135 SPS Exponential and Logarithm

```
function SPSExp(f, n)
    if f[0] ≠ 0 then
        return Error
    Initialize h as an array of size 2^n filled with 0
    h[0] ← 1
    for i ← 0 to n − 1 do
        h[2^i ... 2^{i+1} − 1] ← h[0 ... 2^i − 1] ∗ f[2^i ... 2^{i+1} − 1]
    return h
function SPSLog(f, n)
    if f[0] == 0 then
        return Error
    Initialize h as an array of size 2^n filled with 0
    for i ← 0 to n − 1 do
        h[2^i ... 2^{i+1} − 1] ← f[2^i ... 2^{i+1} − 1] ∗ f[0 ... 2^i − 1]^{−1}
    return h
```

Complexity For both operations, at the i-th iteration, we perform a subset convolution (or quotient) on arrays of length 2^i, which requires exactly $O(i^2 2^i)$ time. The total complexity is the sum:

$$T(n) = \sum_{i=0}^{n-1} O(i^2 2^i) = O(n^2 2^n)$$

18.5 Composition

Given a formal power series $F(x) = \sum_{k=0}^{m-1} f_k x^k$ and a subset power series $G : 2^V \to \mathbb{F}$ over n variables, their composition $H(S) = F(G(S))$ can be computed in $O(n^2 2^n)$ time.

1. Handling $G(\emptyset) \neq 0$ Subset composition requires $G(\emptyset) = 0$ to converge. We first apply the Inverse Borel transform to F, which maps $f_i \to i! f_i = F^{(i)}(0)$.

If the constant term is non-zero (let $a = G(\emptyset)$), we must shift the power series. We have $F^{(i)}(a) = \sum_{j \geq i} F^{(j)}(0) \frac{a^{j-i}}{(j-i)!}$. This shift is computed as a semi-correlation of the transformed coefficients with $\exp(ax)$. Once shifted, we set $G(\emptyset) = 0$.

2. Taylor Expansion We evaluate $F(G)$ by introducing the elements of the set one by one. Let $G_{<k}$ denote the subset power series restricted to the first k elements, and let $G_{\text{slice}(k)}$ be the slice of G where the k-th element is the strict maximum (with x_k factored out). We can iteratively decompose G:

$$G_{\leq k} = G_{<k} + x_k G_{\text{slice}(k)}$$

We use the Taylor expansion of F evaluated at $A + B$, where $A = G_{<k}$ and $B = x_k G_{\text{slice}(k)}$:

$$F(A + B) = F(A) + B * F'(A) + \frac{B * B}{2!} * F''(A) + \dots$$

In subset convolution, elements must be disjoint to yield a non-zero product and every term in B contains the element x_k, thus we only need to consider the first derivative:

$$F(G_{\leq k}) = F(G_{<k}) + x_k G_{\text{slice}(k)} * F'(G_{<k})$$

Notice that computing $F(G_{\leq k})$ requires $F'(G_{<k})$. Consequently, computing $F'(G_{<k})$ will require $F''(G_{<k-1})$, etc. The transition for evaluating the i-th derivative is:

$$F^{(i)}(G_{\leq k}) = F^{(i)}(G_{<k}) + x_k G_{\text{slice}(k)} * F^{(i+1)}(G_{<k})$$

3. DP Optimization We maintain a single working array C of length 2^n. We iterate the derivative order i downwards from n to 0. For a fixed i, our goal is to transform the state of C from representing $F^{(i+1)}(G)$ into $F^{(i)}(G)$.

We iterate the introduced variable index j downwards. At the start of step j, the lower prefix $C[0 \dots 2^j - 1]$ holds the evaluation of $F^{(i+1)}(G_{<j})$. We compute its subset convolution with $G_{\text{slice}(j)}$ and write the result into the upper half $C[2^j \dots 2^{j+1} - 1]$.

Because we iterate j downwards, we compute the upper halves first, preventing the overwriting of the smaller prefixes. We then set the base case (the empty set evaluated at the i-th derivative) by assigning $C[\emptyset] = f_i$. This is correct as our coefficients were mapped to $F^{(i)}(a)$.

Algorithm 136 SPS Composition

```
function SPSCompose(F, G, n, m)
    F ← InverseBorel(F)
    a ← G[0]
    if a ≠ 0 then
        F ← SemiCorr(F, exp(ax))
        G[0] ← 0
    Initialize C as an array of size 2^n filled with 0
    for i ← n to 0 step −1 do
        for j ← n − i − 1 to 0 step −1 do
            X ← C[0 ... 2^j − 1]
            G_slice(j) ← G[2^j ... 2^(j+1) − 1]
            C[2^j ... 2^(j+1) − 1] ← X * G_slice(j)
        if i < m then
            C[0] ← F[i]
        else
            C[0] ← 0
    return C
```

Complexity The Inverse Borel Transform takes $O(m)$ time, and the Taylor shift takes $O(m \log m)$ time. The outer loop executes $O(n)$ times, and the inner loop performs a subset convolution on slices of size 2^j, requiring $O(j^2 2^j)$ operations. Summing this over i and j gives $O(n^2 2^n)$. Thus, the complexity is $O(n^2 2^n + m \log m)$.

18.6 Power Projection

Given a subset power series $G : 2^V \to \mathbb{F}$ over n variables and a weight function $W : 2^V \to \mathbb{F}$, the Power Projection problem asks us to compute the sequence of scalars c_k for $k = 0 \dots m-1$, defined as the inner product of W with the convolution powers of G:

$$c_k = \langle W, G^k \rangle = \sum_{S \subseteq V} W(S)[x^S]G^k$$

This is the transpose of the subset composition algorithm. Let $L_G(F) = F(G)$ be the operator defined by subset composition. Its adjoint L_G^T maps the vector W to the polynomial $C(x) = \sum c_k x^k$ such that $\langle W, L_G(F) \rangle = \langle L_G^T(W), F \rangle$.

Assume $G(\emptyset) = 0$. In the forward composition DP, the state vector $C \in \mathbb{F}^{2^n}$ is updated iteratively by nesting loops: i downwards from n to 0, and j downwards from $n-i-1$ to 0, we reverse this in the transposed algorithm.

Let X denote the lower half of the state $(0 \dots 2^j - 1)$ and Y denote the upper half $(2^j \dots 2^{j+1} - 1)$. The forward transition at step j is the map:

$$Y \leftarrow G_{\text{slice}(j)} * X$$

Let X^* and Y^* be the corresponding halves in the transposed algorithm. The adjoint of reading from X to overwrite Y is to project the dual values from Y^* through the transposed operator and accumulate them into X^*, then to clear Y^*.

$$X^* \leftarrow X^* + G_{\text{slice}(j)} *^T Y^*$$
$$Y^* \leftarrow 0$$

Finally, the forward DP puts the input coefficient f_i into $C[\emptyset]$ at the conclusion of the j iterations. The transposed operation extracts this value into the output $c_i = W[\emptyset]$ and clears $W[\emptyset] = 0$ at the start of the transposed iterations.

If the original series had a non-zero constant term $a = G(\emptyset)$, the forward composition shifted $F^{(i)}(0)$ to $F^{(i)}(a)$. The algorithm computes the semicorrelation with $\exp(ax)$, the transpose of which is multiplication with $\exp(ax)$.

In the forward algorithm, we applied the Inverse Borel transform, since this is diagonal, it's self-adjoint.

Algorithm 137 SPS Power Projection

```
function SPSPowerProj(G, W, n, m)
    a ← G[0]
    G[0] ← 0
    Initialize C as an array of size n + 1 filled with 0
    Initialize S as an intermediate array of size 2^n
    for i ← 0 to n do
        C[i] ← W[0]
        W[0] ← 0
        for j ← 0 to n − i − 1 do
            X ← W[0 ... 2^j − 1]
            Y ← W[2^j ... 2^{j+1} − 1]
            G_slice(j) ← G[2^j ... 2^{j+1} − 1]
            S ← Y *^T G_slice(j)
            X ← X + S
            Y ← 0
    if a ≠ 0 then
        C ← C · exp(ax) (mod x^m)
    C ← InverseBorel(C)
    return C
```

18.7 Power

Given a subset power series $f : 2^V \to \mathbb{F}$ over n variables, computing the k-th power $h = f^k$ can be done in $O(n^2 2^n)$ time.

Case 1: $f(\emptyset) \neq 0$ If the constant term $a_0 = f(\emptyset)$ is non-zero, we can compute the power using the exponential and logarithm.

$$f^k = a_0^k \exp\left(k \log\left(\frac{f}{a_0}\right)\right)$$

Case 2: $f(\emptyset) = 0$ If the constant term is zero, every non-zero term in the series corresponds to a subset of size ≥ 1. Consequently, convolving the series with itself k times means every term in the result must have a subset size of at least k. Thus, if $k > n$, f^k is identically zero.

If $k \leq n$, we compute the power as a composition: $h(S) = P(f(S))$, where $P(x) = x^k$.

We can specialize our SPS Composition algorithm. We have:

$$P^{(i)}(0) = \begin{cases} k! & \text{if } i = k \\ 0 & \text{if } i < k \end{cases}$$

We C with $C[\emptyset] = k!$, which represents $P^{(k)}(f)$. We then run the DP downwards from $i = k - 1$ to 0. At the end of each derivative step, we set the base case $C[\emptyset] = P^{(i)}(0) = 0$.

Algorithm 138 SPS Power

```
function SPSPower(f, k, n)
    a_0 ← f[0]
    if a_0 ≠ 0 then
        a_0^inv ← (a_0)^{-1}
        f_norm ← f · a_0^inv
        L ← SPSLog(f_norm, n)
        L ← L · k
        E ← SPSExp(L, n)
        return E · (a_0^k)
    else
        if k > n then
            return Array of 0s
        Initialize C as an array of size 2^n filled with 0
        C[0] ← k!
        for i ← k − 1 to 0 step −1 do
            for j ← n − i − 1 to 0 step −1 do
                X ← C[0 ... 2^j − 1]
                f_slice(j) ← f[2^j ... 2^{j+1} − 1]
                C[2^j ... 2^{j+1} − 1] ← SubsetConvolution(X, f_slice(j))
            C[0] ← 0
        return C
```

Complexity If $f(\emptyset) \neq 0$, we perform one subset logarithm and one subset exponential, taking $O(n^2 2^n)$ time. If $f(\emptyset) = 0$, the DP matches the complexity of composition, and thus is also $O(n^2 2^n)$ time.

References